



Red Hat build of Apache Camel 4.10

Red Hat build of Apache Camel for Spring Boot Reference

Red Hat build of Apache Camel for Spring Boot Reference

Red Hat build of Apache Camel 4.10 Red Hat build of Apache Camel for Spring Boot Reference

Red Hat build of Apache Camel for Spring Boot Reference

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

This guide describes the settings for Red Hat build of Apache Camel components.

Table of Contents

PREFACE	55
MAKING OPEN SOURCE MORE INCLUSIVE	55
CHAPTER 1. COMPONENT STARTERS	56
CHAPTER 2. AMQP	72
2.1. DEPENDENCIES	72
2.2. URI FORMAT	72
2.3. CONFIGURING OPTIONS	72
2.3.1. Configuring Component Options	72
2.3.2. Configuring Endpoint Options	72
2.4. COMPONENT OPTIONS	73
2.5. ENDPOINT OPTIONS	92
2.5.1. Path Parameters (2 parameters)	93
2.5.2. Query Parameters (96 parameters)	93
2.6. USAGE	112
2.7. CONFIGURING AMQP COMPONENT	113
2.8. USING TOPICS	114
2.9. SPRING BOOT AUTO-CONFIGURATION	114
CHAPTER 3. AVRO	133
3.1. DEPENDENCIES	133
3.2. AVRO DATAFORMAT OPTIONS	133
3.3. AVRO DATA FORMAT USAGE	133
3.4. SPRING BOOT AUTO-CONFIGURATION	134
CHAPTER 4. AVRO JACKSON	135
4.1. DEPENDENCIES	135
4.2. CONFIGURING THE SCHEMARESOLVER	135
4.3. AVRO JACKSON OPTIONS	135
4.4. USING CUSTOM AVROMAPPER	137
4.5. SPRING BOOT AUTO-CONFIGURATION	137
CHAPTER 5. AWS CLOUDWATCH	141
5.1. DEPENDENCIES	141
5.2. URI FORMAT	141
5.3. CONFIGURING OPTIONS	141
5.3.1. Configuring Component Options	141
5.3.2. Configuring Endpoint Options	142
5.4. COMPONENT OPTIONS	142
5.5. ENDPOINT OPTIONS	144
5.5.1. Path Parameters (1 parameters)	144
5.5.2. Query Parameters (16 parameters)	144
5.6. USAGE	146
5.6.1. Static credentials vs Default Credential Provider	146
5.6.2. Message headers evaluated by the CW producer	146
5.6.3. Advanced CloudWatchClient configuration	147
5.7. EXAMPLES	147
5.7.1. Producer Example	147
5.8. SPRING BOOT AUTO-CONFIGURATION	147
CHAPTER 6. AWS DYNAMODB	150
6.1. DEPENDENCIES	150

6.2. URI FORMAT	150
6.3. CONFIGURING OPTIONS	150
6.3.1. Configuring Component Options	150
6.3.2. Configuring Endpoint Options	150
6.4. COMPONENT OPTIONS	151
6.5. ENDPOINT OPTIONS	153
6.5.1. Path Parameters (1 parameters)	154
6.5.2. Query Parameters (20 parameters)	154
6.6. USAGE	156
6.6.1. Static credentials vs Default Credential Provider	156
6.6.2. Message headers evaluated by the DDB producer	156
6.6.3. Message headers set during BatchGetItems operation	158
6.6.4. Message headers set during DeleteItem operation	158
6.6.5. Message headers set during DeleteTable operation	158
6.6.6. Message headers set during DescribeTable operation	159
6.6.7. Message headers set during GetItem operation	160
6.6.8. Message headers set during PutItem operation	160
6.6.9. Message headers set during Query operation	160
6.6.10. Message headers set during Scan operation	160
6.6.11. Message headers set during UpdateItem operation	161
6.6.12. Advanced AmazonDynamoDB configuration	161
6.7. SUPPORTED PRODUCER OPERATIONS	161
6.8. EXAMPLES	161
6.8.1. Producer Examples	161
6.9. SPRING BOOT AUTO-CONFIGURATION	162
CHAPTER 7. AWS KINESIS	167
7.1. DEPENDENCIES	167
7.2. URI FORMAT	167
7.3. CONFIGURING OPTIONS	167
7.3.1. Configuring Component Options	167
7.3.2. Configuring Endpoint Options	168
7.4. COMPONENT OPTIONS	168
7.5. ENDPOINT OPTIONS	173
7.5.1. Path Parameters (1 parameters)	174
7.5.2. Query Parameters (50 parameters)	174
7.6. MESSAGE HEADERS	181
7.7. USAGE	182
7.7.1. Batch Consumer	182
7.7.2. Batch Producer	182
7.7.3. Static credentials, Default Credential Provider and Profile Credentials Provider	182
7.7.4. AmazonKinesis configuration	183
7.7.5. Providing AWS Credentials	183
7.7.6. AWS Kinesis KCL Consumer	183
7.8. SPRING BOOT AUTO-CONFIGURATION	183
CHAPTER 8. AWS 2 LAMBDA	192
8.1. DEPENDENCIES	192
8.2. URI FORMAT	192
8.3. CONFIGURING OPTIONS	192
8.3.1. Configuring Component Options	192
8.3.2. Configuring Endpoint Options	193
8.4. COMPONENT OPTIONS	193

8.5. ENDPOINT OPTIONS	195
8.5.1. Path Parameters (1 parameters)	196
8.5.2. Query Parameters (14 parameters)	196
8.6. USAGE	198
8.6.1. Static credentials vs Default Credential Provider	198
8.6.2. Message headers evaluated by the Lambda producer	199
8.7. LIST OF AVAILABLE OPERATIONS	202
8.8. EXAMPLES	203
8.8.1. Producer Example	203
8.8.2. Producer Examples	203
8.9. USING A POJO AS BODY	203
8.10. DEPENDENCIES	204
8.11. SPRING BOOT AUTO-CONFIGURATION	204
CHAPTER 9. AWS S3 STORAGE SERVICE	207
9.1. DEPENDENCIES	207
9.2. URI FORMAT	207
9.3. CONFIGURING OPTIONS	207
9.3.1. Configuring Component Options	207
9.3.2. Configuring Endpoint Options	208
9.4. COMPONENT OPTIONS	208
9.5. ENDPOINT OPTIONS	214
9.5.1. Path Parameters (1 parameters)	214
9.5.2. Query Parameters (68 parameters)	214
9.6. BATCH CONSUMER	222
9.7. USAGE	222
9.7.1. Message headers evaluated by the S3 producer	222
9.7.2. Message headers set by the S3 producer	224
9.7.3. Message headers set by the S3 consumer	224
9.7.4. S3 Producer operations	225
9.7.5. Advanced AmazonS3 configuration	226
9.7.6. Use KMS with the S3 component	226
9.7.7. Static credentials vs Default Credential Provider	226
9.7.8. S3 Producer Operation examples	226
9.8. STREAMING UPLOAD MODE	229
9.9. BUCKET AUTOCREATION	231
9.10. MOVING STUFF BETWEEN A BUCKET AND ANOTHER BUCKET	231
9.11. MOVEAFTERREAD CONSUMER OPTION	231
9.12. USING CUSTOMER KEY AS ENCRYPTION	232
9.13. USING A POJO AS BODY	232
9.14. CREATE S3 CLIENT AND ADD COMPONENT TO REGISTRY	232
9.15. DEPENDENCIES	233
9.16. SPRING BOOT AUTO-CONFIGURATION	233
CHAPTER 10. AWS SIMPLE NOTIFICATION SYSTEM (SNS)	240
10.1. DEPENDENCIES	240
10.2. URI FORMAT	240
10.3. CONFIGURING OPTIONS	240
10.3.1. Configuring Component Options	240
10.3.2. Configuring Endpoint Options	241
10.4. COMPONENT OPTIONS	241
10.5. ENDPOINT OPTIONS	244
10.5.1. Path Parameters (1 parameters)	244

10.5.2. Query Parameters (23 parameters)	244
10.6. USAGE	247
10.6.1. Static credentials vs Default Credential Provider	247
10.6.2. Message headers evaluated by the SNS producer	247
10.6.3. Message headers set by the SNS producer	247
10.6.4. Advanced AmazonSNS configuration	248
10.6.5. Create a subscription between an AWS SNS Topic and an AWS SQS Queue	248
10.7. TOPIC AUTOCREATION	248
10.8. SNS FIFO	248
10.8.1. SNS Fifo Topic Message group Id Strategy and message Deduplication Id Strategy	249
10.9. EXAMPLES	249
10.9.1. Producer Examples	249
10.10. DEPENDENCIES	249
10.11. SPRING BOOT AUTO-CONFIGURATION	250
CHAPTER 11. AWS SIMPLE QUEUE SERVICE (SQS)	253
11.1. DEPENDENCIES	253
11.2. URI FORMAT	253
11.3. CONFIGURING OPTIONS	253
11.3.1. Configuring Component Options	253
11.3.2. Configuring Endpoint Options	254
11.4. COMPONENT OPTIONS	254
11.5. ENDPOINT OPTIONS	259
11.5.1. Path Parameters (1 parameters)	259
11.5.2. Query Parameters (61 parameters)	259
11.6. BATCH CONSUMER	266
11.7. USAGE	267
11.7.1. Static credentials vs Default Credential Provider	267
11.7.2. Message headers set by the SQS producer	267
11.7.3. Message headers set by the SQS consumer	267
11.7.4. Advanced AmazonSQS configuration	268
11.7.5. Creating or updating an SQS Queue	268
11.7.6. DelayQueue VS Delay for Single message	268
11.7.7. Server Side Encryption	268
11.8. JMS-STYLE SELECTORS	268
11.9. AVAILABLE PRODUCER OPERATIONS	269
11.10. SEND MESSAGE	269
11.11. SEND BATCH MESSAGE	269
11.12. DELETE SINGLE MESSAGE	270
11.13. LIST QUEUES	270
11.14. PURGE QUEUE	270
11.15. QUEUE AUTOCREATION	270
11.16. SEND BATCH MESSAGE AND MESSAGE DEDUPLICATION STRATEGY	270
11.17. DEPENDENCIES	271
11.18. SPRING BOOT AUTO-CONFIGURATION	271
CHAPTER 12. AWS SECRETS MANAGER	277
12.1. DEPENDENCIES	277
12.2. URI FORMAT	277
12.3. CONFIGURING OPTIONS	277
12.3.1. Configuring Component Options	277
12.3.2. Configuring Endpoint Options	278
12.4. COMPONENT OPTIONS	278

12.5. ENDPOINT OPTIONS	282
12.5.1. Path Parameters (1 parameters)	282
12.5.2. Query Parameters (19 parameters)	282
12.5.3. Static credentials, Default Credential Provider and Profile Credentials Provider	287
12.5.4. Using AWS Secrets Manager Property Function	288
12.5.5. Automatic Camel context reloading on Secret Refresh	290
12.5.6. Automatic Camel context reloading on Secret Refresh with Eventbridge and AWS SQS Services	291
12.6. MESSAGE HEADERS	292
12.6.1. Secrets Manager Producer operations	293
12.7. SPRING BOOT AUTO-CONFIGURATION	294
CHAPTER 13. AZURE KEY VAULT	298
13.1. DEPENDENCIES	298
13.2. URI FORMAT	298
13.3. CONFIGURING OPTIONS	298
13.3.1. Configuring Component Options	298
13.3.2. Configuring Endpoint Options	298
13.4. COMPONENT OPTIONS	299
13.5. ENDPOINT OPTIONS	299
13.5.1. Path Parameters (1 parameters)	300
13.5.2. Query Parameters (7 parameters)	300
13.6. USAGE	301
13.6.1. Using Azure Key Vault Property Function	301
13.6.2. Automatic Camel context reloading on Secret Refresh	303
13.6.3. Automatic Camel context reloading on Secret Refresh - Required Infrastructure's creation	304
13.7. MESSAGE HEADERS	306
13.7.1. Azure Key Vault Producer operations	306
13.8. EXAMPLES	306
13.8.1. Producer Examples	306
13.9. SPRING BOOT AUTO-CONFIGURATION	307
CHAPTER 14. AZURE SERVICEBUS	308
14.1. DEPENDENCIES	308
14.2. CONFIGURING OPTIONS	308
14.2.1. Configuring Component Options	308
14.2.2. Configuring Endpoint Options	308
14.3. COMPONENT OPTIONS	309
14.4. ENDPOINT OPTIONS	313
14.4.1. Path Parameters (1 parameters)	313
14.4.2. Query Parameters (25 parameters)	313
14.5. ASYNC CONSUMER AND PRODUCER	317
14.6. MESSAGE HEADERS	317
14.6.1. Message Body	321
14.6.2. Azure ServiceBus Producer operations	321
14.6.3. Azure ServiceBus Consumer operations	322
14.6.3.1. Examples	322
14.7. SPRING BOOT AUTO-CONFIGURATION	323
CHAPTER 15. AZURE STORAGE BLOB SERVICE	327
15.1. DEPENDENCIES	327
15.2. URI FORMAT	327
15.3. CONFIGURING OPTIONS	327
15.3.1. Configuring Component Options	327
15.3.2. Configuring Endpoint Options	328

15.4. COMPONENT OPTIONS	328
15.5. ENDPOINT OPTIONS	333
15.5.1. Path Parameters (2 parameters)	333
15.5.2. Query Parameters (50 parameters)	334
15.6. USAGE	342
15.6.1. Message headers	342
15.6.2. Message headers set by either component producer or consumer	348
15.6.3. Advanced Azure Storage Blob configuration	351
15.6.4. Automatic detection of BlobServiceClient client in registry	352
15.6.5. Azure Storage Blob Producer operations	352
15.6.6. Consumer Examples	354
15.6.7. Producer Operations Examples	355
15.6.8. SAS Token generation example	360
15.7. SPRING BOOT AUTO-CONFIGURATION	361
CHAPTER 16. AZURE STORAGE QUEUE SERVICE	366
16.1. DEPENDENCIES	366
16.2. URI FORMAT	366
16.3. CONFIGURING OPTIONS	366
16.3.1. Configuring Component Options	366
16.3.2. Configuring Endpoint Options	367
16.4. COMPONENT OPTIONS	367
16.5. ENDPOINT OPTIONS	370
16.5.1. Path Parameters (2 parameters)	370
16.5.2. Query Parameters (31 parameters)	370
16.6. USAGE	375
16.6.1. Message headers evaluated by the component producer	375
16.6.2. Message headers set by either component producer or consumer	377
16.6.3. Advanced Azure Storage Queue configuration	378
16.6.4. Automatic detection of QueueServiceClient client in registry	378
16.6.5. Azure Storage Queue Producer operations	378
16.6.6. Consumer Examples	379
16.6.7. Producer Operations Examples	380
16.6.8. Development Notes (Important)	382
16.7. SPRING BOOT AUTO-CONFIGURATION	382
CHAPTER 17. BEAN	385
17.1. DEPENDENCIES	385
17.2. URI FORMAT	385
17.3. CONFIGURING OPTIONS	385
17.3.1. Configuring Component Options	385
17.3.2. Configuring Endpoint Options	385
17.4. COMPONENT OPTIONS	386
17.5. ENDPOINT OPTIONS	387
17.5.1. Path Parameters (1 parameters)	387
17.5.2. Query Parameters (5 parameters)	388
17.6. EXAMPLES	389
17.7. BEAN AS ENDPOINT	389
17.8. JAVA DSL BEAN SYNTAX	390
17.9. BEAN BINDING	390
17.10. SPRING BOOT AUTO-CONFIGURATION	390
CHAPTER 18. BEANIO	394
18.1. DEPENDENCIES	394

18.2. OPTIONS	394
18.3. USAGE	402
18.4. SPRING BOOT AUTO-CONFIGURATION	404
CHAPTER 19. BEAN VALIDATOR	406
19.1. DEPENDENCIES	406
19.2. URI FORMAT	406
19.3. CONFIGURING OPTIONS	406
19.3.1. Configuring Component Options	406
19.3.2. Configuring Endpoint Options	406
19.4. COMPONENT OPTIONS	407
19.5. ENDPOINT OPTIONS	408
19.5.1. Path Parameters (1 parameters)	408
19.5.2. Query Parameters (8 parameters)	408
19.6. EXAMPLE	409
19.7. SPRING BOOT AUTO-CONFIGURATION	412
CHAPTER 20. BINDY	414
20.1. DEPENDENCIES	415
20.2. OPTIONS	415
20.3. ANNOTATIONS	415
20.3.1. 1. CsvRecord	416
20.3.2. 2. Link	420
20.3.3. 3. DataField	421
20.3.4. 4. FixedLengthRecord	427
20.3.5. 5. Message	435
20.3.6. 6. KeyValuePairField	436
20.3.7. 7. Section	438
20.3.8. 8. OneToMany	439
20.3.9. 9. BindyConverter	442
20.3.10. 10. FormatFactories	442
20.4. SUPPORTED DATATYPES	443
20.5. USING THE JAVA DSL	444
20.5.1. Setting locale	444
20.5.2. Unmarshaling	444
20.5.3. Marshaling	445
20.6. USING SPRING XML	445
20.7. DEPENDENCIES	446
20.8. SPRING BOOT AUTO-CONFIGURATION	447
CHAPTER 21. BROWSE	449
21.1. DEPENDENCIES	449
21.2. URI FORMAT	449
21.3. CONFIGURING OPTIONS	449
21.3.1. Configuring Component Options	449
21.3.2. Configuring Endpoint Options	449
21.4. COMPONENT OPTIONS	450
21.5. ENDPOINT OPTIONS	451
21.5.1. Path Parameters (1 parameters)	451
21.5.2. Query Parameters (4 parameters)	451
21.6. SAMPLE	452
21.7. SPRING BOOT AUTO-CONFIGURATION	452
CHAPTER 22. CASSANDRA CQL	454

22.1. DEPENDENCIES	454
22.2. CONFIGURING OPTIONS	454
22.2.1. Configuring Component Options	454
22.2.2. Configuring Endpoint Options	454
22.3. COMPONENT OPTIONS	455
22.4. ENDPOINT OPTIONS	456
22.4.1. Path Parameters (4 parameters)	456
22.4.2. Query Parameters (30 parameters)	456
22.5. ENDPOINT CONNECTION SYNTAX	461
22.6. MESSAGES	461
22.6.1. Incoming Message	461
22.6.2. Outgoing Message	461
22.7. REPOSITORIES	462
22.8. IDEMPOTENT REPOSITORY	462
22.9. AGGREGATION REPOSITORY	463
22.10. EXAMPLES	463
22.11. SPRING BOOT AUTO-CONFIGURATION	464
CHAPTER 23. CICS	465
23.1. DEPENDENCIES	465
23.2. URI FORMAT	465
23.3. CONFIGURING OPTIONS	466
23.3.1. Configuring Component Options	466
23.3.2. Configuring Endpoint Options	466
23.4. COMPONENT OPTIONS	466
23.5. ENDPOINT OPTIONS	468
23.5.1. Path Parameters (2 parameters)	468
23.5.2. Query Parameters (15 parameters)	468
23.6. MESSAGE HEADERS	470
23.7. SAMPLES	474
23.7.1. Using Commarea	474
23.7.2. Using Channel with a single input container	474
23.7.3. Using Channel with multiple input container	475
23.8. SPRING BOOT AUTO-CONFIGURATION	475
CHAPTER 24. CONSTANT	478
24.1. DEPENDENCIES	478
24.2. CONSTANT OPTIONS	478
24.3. EXAMPLE	478
24.3.1. Specifying type of value	479
24.4. LOADING CONSTANT FROM EXTERNAL RESOURCE	479
24.5. DEPENDENCIES	479
24.6. SPRING BOOT AUTO-CONFIGURATION	479
CHAPTER 25. CONTROL BUS	497
25.1. COMMANDS	497
25.2. DEPENDENCIES	497
25.3. CONFIGURING OPTIONS	497
25.3.1. Configuring Component Options	498
25.3.2. Configuring Endpoint Options	498
25.4. COMPONENT OPTIONS	498
25.5. ENDPOINT OPTIONS	499
25.5.1. Path Parameters (2 parameters)	499
25.5.1.1. Query Parameters (6 parameters)	500

25.6. USING ROUTE COMMAND	502
25.7. GETTING PERFORMANCE STATISTICS	502
25.8. USING SIMPLE LANGUAGE	503
25.9. SPRING BOOT AUTO-CONFIGURATION	503
CHAPTER 26. CRON	505
26.1. DEPENDENCIES	505
26.2. CONFIGURING OPTIONS	505
26.2.1. Configuring Component Options	505
26.2.2. Configuring Endpoint Options	506
26.3. COMPONENT OPTIONS	506
26.4. ENDPOINT OPTIONS	507
26.4.1. Path Parameters (1 parameters)	507
26.4.2. Query Parameters (4 parameters)	507
26.5. USAGE	508
26.6. SPRING BOOT AUTO-CONFIGURATION	508
CHAPTER 27. CRYPTO (JCE)	510
27.1. DEPENDENCIES	510
27.2. INTRODUCTION	510
27.3. URI FORMAT	510
27.4. CONFIGURING OPTIONS	511
27.4.1. Configuring Component Options	511
27.4.2. Configuring Endpoint Options	511
27.5. COMPONENT OPTIONS	511
27.6. ENDPOINT OPTIONS	515
27.6.1. Path Parameters (2 parameters)	515
27.6.2. Query Parameters (19 parameters)	516
27.7. MESSAGE HEADERS	519
27.8. USING	520
27.8.1. Raw keys	520
27.8.2. KeyStores and Aliases.	520
27.8.3. Changing JCE Provider and Algorithm	521
27.8.4. Changing the Signature Message Header	521
27.8.5. Supplying Keys dynamically.	522
27.9. SPRING BOOT AUTO-CONFIGURATION	522
CHAPTER 28. CSIMPLE	533
28.1. DEPENDENCIES	533
28.2. DIFFERENT BETWEEN CSIMPLE AND SIMPLE	533
28.2.1. Additional CSimple functions	533
28.3. COMPILATION	534
28.3.1. Using camel-csimple-maven-plugin	534
28.3.2. Using camel-csimple-joor	536
28.4. CSIMPLE LANGUAGE OPTIONS	536
28.5. LIMITATIONS	536
28.6. AUTO IMPORTS	537
28.7. CONFIGURATION FILE	537
28.8. SEE ALSO	537
28.9. SPRING BOOT AUTO-CONFIGURATION	537
CHAPTER 29. CXF	555
29.1. DEPENDENCIES	555
29.2. URI FORMAT	555

29.3. CONFIGURING OPTIONS	555
29.3.1. Configuring Component Options	555
29.3.2. Configuring Endpoint Options	556
29.4. COMPONENT OPTIONS	556
29.5. ENDPOINT OPTIONS	557
29.5.1. Path Parameters (2 parameters)	557
29.5.2. Query Parameters (35 parameters)	558
29.5.3. Descriptions of the dataformats	561
29.5.4. How to enable CXF's LoggingOutInterceptor in RAW mode	562
29.5.5. Description of relayHeaders option	563
29.5.6. Available only in POJO mode	563
29.6. CONFIGURE THE CXF ENDPOINTS WITH SPRING	565
29.7. HOW TO MAKE THE CAMEL-CXF COMPONENT USE LOG4J INSTEAD OF JAVA.UTIL.LOGGING	568
29.8. HOW TO LET CAMEL-CXF RESPONSE START WITH XML PROCESSING INSTRUCTION	568
29.9. HOW TO OVERRIDE THE CXF PRODUCER ADDRESS FROM MESSAGE HEADER	569
29.10. HOW TO CONSUME A MESSAGE FROM A CAMEL-CXF ENDPOINT IN POJO DATA FORMAT	569
29.11. HOW TO PREPARE THE MESSAGE FOR THE CAMEL-CXF ENDPOINT IN POJO DATA FORMAT	570
29.12. HOW TO DEAL WITH THE MESSAGE FOR A CAMEL-CXF ENDPOINT IN PAYLOAD DATA FORMAT	571
29.13. HOW TO GET AND SET SOAP HEADERS IN POJO MODE	572
29.14. HOW TO GET AND SET SOAP HEADERS IN PAYLOAD MODE	573
29.15. SOAP HEADERS ARE NOT AVAILABLE IN RAW MODE	574
29.16. HOW TO THROW A SOAP FAULT FROM CAMEL	574
29.17. HOW TO PROPAGATE A CAMEL-CXF ENDPOINT'S REQUEST AND RESPONSE CONTEXT	575
29.18. ATTACHMENT SUPPORT	575
29.19. STREAMING SUPPORT IN PAYLOAD MODE	578
29.20. USING THE GENERIC CXF DISPATCH MODE	579
29.21. SPRING BOOT AUTO-CONFIGURATION	579
CHAPTER 30. CXF-RS	582
30.1. DEPENDENCIES	582
30.2. URI FORMAT	582
30.3. CONFIGURING OPTIONS	582
30.3.1. Configuring Component Options	582
30.3.2. Configuring Endpoint Options	583
30.4. COMPONENT OPTIONS	583
30.5. ENDPOINT OPTIONS	584
30.5.1. Path Parameters (2 parameters)	584
30.5.2. Query Parameters (31 parameters)	585
30.6. MESSAGE HEADERS	589
30.7. HOW TO CONFIGURE THE REST ENDPOINT IN CAMEL	592
30.8. HOW TO OVERRIDE THE CXF PRODUCER ADDRESS FROM MESSAGE HEADER	592
30.9. CONSUMING A REST REQUEST - SIMPLE BINDING STYLE	592
30.9.1. Enabling the Simple Binding Style	593
30.9.2. Examples of request binding with different method signatures	593
30.9.3. Examples of the Simple Binding Style	594
30.10. CONSUMING A REST REQUEST - DEFAULT BINDING STYLE	594
30.11. HOW TO INVOKE THE REST SERVICE THROUGH CAMEL-CXF-RS PRODUCER	596
30.12. SPRING BOOT AUTO-CONFIGURATION	597
CHAPTER 31. DATA FORMAT	599
31.1. DEPENDENCIES	599
31.2. URI FORMAT	599
31.3. CONFIGURING OPTIONS	599

31.3.1. Configuring Component Options	599
31.3.2. Configuring Endpoint Options	599
31.4. COMPONENT OPTIONS	600
31.5. ENDPOINT OPTIONS	600
31.5.1. Path Parameters (2 parameters)	601
31.5.2. Query Parameters (1 parameters)	601
31.6. SAMPLES	601
31.7. SPRING BOOT AUTO-CONFIGURATION	602
CHAPTER 32. DATASET	603
32.1. DEPENDENCIES	603
32.2. URI FORMAT	603
32.3. CONFIGURING OPTIONS	603
32.3.1. Configuring Component Options	603
32.3.2. Configuring Endpoint Options	604
32.4. COMPONENT OPTIONS	604
32.5. ENDPOINT OPTIONS	605
32.5.1. Path Parameters (1 parameters)	605
32.5.2. Query Parameters (21 parameters)	606
32.6. CONFIGURING DATASET	609
32.7. EXAMPLE	609
32.8. DATASETSUPPORT (ABSTRACT CLASS)	610
32.8.1. Properties on DataSetSupport	610
32.9. SIMPLEDATASET	610
32.9.1. Additional Properties on SimpleDataSet	610
32.10. LISTDATASET	611
32.10.1. Additional Properties on ListDataSet	611
32.11. FILEDATASET	611
32.11.1. Additional Properties on FileDataSet	611
32.12. SPRING BOOT AUTO-CONFIGURATION	612
CHAPTER 33. DIRECT	615
33.1. DEPENDENCIES	615
33.2. URI FORMAT	615
33.3. CONFIGURING OPTIONS	615
33.3.1. Configuring Component Options	615
33.3.2. Configuring Endpoint Options	616
33.4. COMPONENT OPTIONS	616
33.5. ENDPOINT OPTIONS	617
33.5.1. Path Parameters (1 parameters)	617
33.5.2. Query Parameters (8 parameters)	617
33.6. SAMPLES	619
33.7. SPRING BOOT AUTO-CONFIGURATION	619
CHAPTER 34. ELASTICSEARCH	621
34.1. DEPENDENCIES	621
34.2. URI FORMAT	621
34.3. CONFIGURING OPTIONS	621
34.3.1. Configuring Component Options	621
34.3.2. Configuring Endpoint Options	621
34.4. COMPONENT OPTIONS	622
34.5. ENDPOINT OPTIONS	623
34.5.1. Path Parameters (1 parameters)	624
34.5.2. Query Parameters (19 parameters)	624

34.6. MESSAGE HEADERS	626
34.7. MESSAGE OPERATIONS	628
34.8. CONFIGURE THE COMPONENT AND ENABLE BASIC AUTHENTICATION	632
34.9. INDEX EXAMPLE	632
34.10. SEARCH EXAMPLE	632
34.11. MULTISEARCH EXAMPLE	633
34.12. DOCUMENT TYPE	634
34.13. USING CAMEL ELASTICSEARCH WITH SPRING BOOT	634
34.13.1. Use RestClient provided by Spring Boot	634
34.13.2. Disable Sniffer when using Spring Boot	634
34.14. SPRING BOOT AUTO-CONFIGURATION	635
CHAPTER 35. EXCHANGEPROPERTY	637
35.1. DEPENDENCIES	637
35.2. EXCHANGE PROPERTY OPTIONS	637
35.3. EXAMPLE	637
35.4. SPRING BOOT AUTO-CONFIGURATION	637
CHAPTER 36. FHIR	655
36.1. DEPENDENCIES	655
36.2. URI FORMAT	655
36.3. CONFIGURING OPTIONS	655
36.3.1. Configuring Component Options	656
36.3.2. Configuring Endpoint Options	656
36.4. COMPONENT OPTIONS	656
36.5. ENDPOINT OPTIONS	660
36.5.1. Path Parameters (2 parameters)	660
36.5.2. Query Parameters (44 parameters)	660
36.6. API PARAMETERS (13 APIS)	665
36.6.1. API: capabilities	666
36.6.1.1. Method ofType	667
36.6.2. API: create	667
36.6.2.1. Method resource	668
36.6.3. API: delete	668
36.6.3.1. Method resource	669
36.6.3.2. Method resourceById	669
36.6.3.3. Method resourceConditionalByUrl	670
36.6.4. API: history	670
36.6.4.1. Method onInstance	671
36.6.4.2. Method onServer	671
36.6.4.3. Method onType	672
36.6.5. API: load-page	673
36.6.5.1. Method byUrl	673
36.6.5.2. Method next	674
36.6.5.3. Method previous	674
36.6.6. API: meta	675
36.6.6.1. Method add	675
36.6.6.2. Method delete	675
36.6.6.3. Method getFromResource	676
36.6.6.4. Method getFromServer	676
36.6.6.5. Method getFromType	677
36.6.7. API: operation	677
36.6.7.1. Method onInstance	678

36.6.7.2. Method onInstanceVersion	679
36.6.7.3. Method onServer	680
36.6.7.4. Method onType	680
36.6.7.5. Method processMessage	681
36.6.8. API: patch	682
36.6.8.1. Method patchById	682
36.6.8.2. Method patchByUrl	683
36.6.9. API: read	684
36.6.9.1. Method resourceById	684
36.6.9.2. Method resourceByUrl	685
36.6.10. API: search	686
36.6.10.1. Method searchByUrl	687
36.6.11. API: transaction	687
36.6.11.1. Method withBundle	688
36.6.11.2. Method withResources	688
36.6.12. API: update	688
36.6.12.1. Method resource	689
36.6.12.2. Method resourceBySearchUrl	690
36.6.13. API: validate	690
36.6.13.1. Method resource	691
36.7. SPRING BOOT AUTO-CONFIGURATION	691
CHAPTER 37. FILE	699
37.1. DEPENDENCIES	699
37.2. URI FORMAT	699
37.3. CONFIGURING OPTIONS	699
37.3.1. Configuring Component Options	699
37.3.2. Configuring Endpoint Options	700
37.4. COMPONENT OPTIONS	700
37.5. ENDPOINT OPTIONS	701
37.5.1. Path Parameters (1 parameters)	701
37.5.2. Query Parameters (94 parameters)	701
37.6. MOVE AND DELETE OPERATIONS	717
37.7. FINE GRAINED CONTROL OVER MOVE AND PREMOVE OPTION	718
37.8. ABOUT MOVEFAILED	718
37.9. MESSAGE HEADERS	719
37.9.1. File producer only	719
37.9.2. File consumer only	719
37.10. BATCH CONSUMER	720
37.11. EXCHANGE PROPERTIES, FILE CONSUMER ONLY	720
37.12. USING CHARSET	720
37.13. COMMON GOTCHAS WITH FOLDER AND FILENAMES	721
37.14. FILENAME EXPRESSION	722
37.15. CONSUMING FILES FROM FOLDERS WHERE OTHERS DROP FILES DIRECTLY	722
37.16. USING DONE FILES	722
37.17. WRITING DONE FILES	723
37.18. SAMPLES	723
37.18.1. Read from a directory and write to another directory	723
37.18.2. Read from a directory and write to another directory using a overrule dynamic name	723
37.18.3. Reading recursively from a directory and writing to another	724
37.19. USING FLATTEN	724
37.20. READING FROM A DIRECTORY AND THE DEFAULT MOVE OPERATION	724
37.21. READ FROM A DIRECTORY AND PROCESS THE MESSAGE IN JAVA	725

37.22. WRITING TO FILES	725
37.22.1. Write to subdirectory using Exchange.FILE_NAME	725
37.22.2. Writing file through the temporary directory relative to the final destination	725
37.23. USING EXPRESSION FOR FILENAMES	725
37.24. AVOIDING READING THE SAME FILE MORE THAN ONCE (IDEMPOTENT CONSUMER)	726
37.25. USING A FILE BASED IDEMPOTENT REPOSITORY	726
37.26. USING A JPA BASED IDEMPOTENT REPOSITORY	727
37.27. FILTER USING ORG.APACHE.CAMEL.COMPONENT.FILE.GENERICFILEFILTER	727
37.28. FILTERING USING ANT PATH MATCHER	728
37.28.1. Sorting using Comparator	728
37.28.2. Sorting using sortBy	729
37.29. USING GENERICFILEPROCESSSTRATEGY	730
37.30. USING FILTER	730
37.31. USING BRIDGEERRORHANDLER	730
37.32. DEBUG LOGGING	731
37.33. SPRING BOOT AUTO-CONFIGURATION	731
CHAPTER 38. FILE LANGUAGE	733
38.1. DEPENDENCIES	733
38.2. FILE LANGUAGE OPTIONS	733
38.3. SYNTAX	733
38.4. FILE TOKEN EXAMPLE	736
38.4.1. Relative paths	736
38.4.2. Absolute paths	737
38.5. SAMPLES	737
38.6. SPRING BOOT AUTO-CONFIGURATION	738
CHAPTER 39. FLINK	756
39.1. DEPENDENCIES	756
39.2. URI FORMAT	756
39.3. CONFIGURING OPTIONS	756
39.3.1. Configuring Component Options	756
39.3.2. Configuring Endpoint Options	757
39.4. COMPONENT OPTIONS	757
39.5. ENDPOINT OPTIONS	758
39.5.1. Path Parameters (1 parameters)	758
39.5.2. Query Parameters (6 parameters)	758
39.6. MESSAGE HEADERS	759
39.7. FLINK DATASET CALLBACK	760
39.8. FLINK DATASTREAM CALLBACK	760
39.9. CAMEL-FLINK PRODUCER CALL	760
39.10. SPRING BOOT AUTO-CONFIGURATION	761
CHAPTER 40. FTP	762
40.1. DEPENDENCIES	762
40.2. URI FORMAT	762
40.3. CONFIGURING OPTIONS	762
40.3.1. Configuring Component Options	763
40.3.2. Configuring Endpoint Options	763
40.4. COMPONENT OPTIONS	763
40.5. ENDPOINT OPTIONS	764
40.5.1. Path Parameters (3 parameters)	764
40.5.2. Query Parameters (111 parameters)	765
40.6. FTPS COMPONENT DEFAULT TRUST STORE	783

40.7. EXAMPLES	784
40.8. CONCURRENCY	784
40.9. MORE INFORMATION	784
40.10. DEFAULT WHEN CONSUMING FILES	784
40.10.1. limitations	784
40.11. MESSAGE HEADERS	785
40.11.1. Exchange Properties	785
40.12. ABOUT TIMEOUTS	786
40.13. USING LOCAL WORK DIRECTORY	786
40.14. STEPWISE CHANGING DIRECTORIES	786
40.15. USING STEPWISE=TRUE (DEFAULT MODE)	787
40.16. USING STEPWISE=FALSE	788
40.17. SAMPLES	789
40.17.1. Consuming a remote FTPS server (implicit SSL) and client authentication	789
40.17.2. Consuming a remote FTPS server (explicit TLS) and a custom trust store configuration	790
40.18. CUSTOM FILTERING	790
40.19. FILTERING USING ANT PATH MATCHER	790
40.20. USING A PROXY WITH SFTP	790
40.21. SETTING PREFERRED SFTP AUTHENTICATION METHOD	791
40.22. CONSUMING A SINGLE FILE USING A FIXED NAME	791
40.23. DEBUG LOGGING	791
40.24. SPRING BOOT AUTO-CONFIGURATION	792
CHAPTER 41. GOOGLE BIGQUERY	795
41.1. DEPENDENCIES	795
41.2. AUTHENTICATION CONFIGURATION	795
41.3. URI FORMAT	795
41.4. CONFIGURING OPTIONS	796
41.4.1. Configuring Component Options	796
41.4.2. Configuring Endpoint Options	796
41.5. COMPONENT OPTIONS	796
41.6. ENDPOINT OPTIONS	797
41.6.1. Path Parameters (3 parameters)	797
41.6.2. Query Parameters (4 parameters)	798
41.7. MESSAGE HEADERS	798
41.8. PRODUCER ENDPOINTS	799
41.9. TEMPLATE TABLES	799
41.10. PARTITIONING	800
41.11. ENSURING DATA CONSISTENCY	800
41.12. SPRING BOOT AUTO-CONFIGURATION	800
CHAPTER 42. GOOGLE PUBSUB	803
42.1. DEPENDENCIES	803
42.2. URI FORMAT	803
42.3. CONFIGURING OPTIONS	803
42.3.1. Configuring Component Options	803
42.3.2. Configuring Endpoint Options	803
42.4. COMPONENT OPTIONS	804
42.5. ENDPOINT OPTIONS	805
42.5.1. Path Parameters (2 parameters)	805
42.5.2. Query Parameters (15 parameters)	806
42.6. MESSAGE HEADERS	808
42.7. PRODUCER ENDPOINTS	808

42.8. CONSUMER ENDPOINTS	809
42.9. MESSAGE BODY	809
42.10. AUTHENTICATION CONFIGURATION	809
42.11. ROLLBACK AND REDELIVERY	809
42.12. SPRING BOOT AUTO-CONFIGURATION	810
CHAPTER 43. GOOGLE SECRET MANAGER	812
43.1. DEPENDENCIES	812
43.2. AUTHENTICATION CONFIGURATION	812
43.3. URI FORMAT	812
43.4. CONFIGURING OPTIONS	812
43.4.1. Configuring Component Options	813
43.4.2. Configuring Endpoint Options	813
43.5. COMPONENT OPTIONS	813
43.6. ENDPOINT OPTIONS	814
43.6.1. Path Parameters (1 parameters)	814
43.6.2. Query Parameters (5 parameters)	814
43.7. MESSAGE HEADERS	815
43.7.1. Using GCP Secret Manager Properties Source	816
43.7.2. Automatic CamelContext reloading on Secret Refresh	818
43.7.3. Automatic CamelContext reloading on Secret Refresh - Required infrastructure's creation	819
43.7.4. Google Secret Manager Producer operations	821
43.7.5. Google Secret Manager Producer Operation examples	821
43.8. SPRING BOOT AUTO-CONFIGURATION	822
CHAPTER 44. GRAPHQL	823
44.1. DEPENDENCIES	823
44.2. CONFIGURING OPTIONS	823
44.2.1. Configuring Component Options	823
44.2.2. Configuring Endpoint Options	823
44.3. COMPONENT OPTIONS	824
44.4. ENDPOINT OPTIONS	824
44.4.1. Path Parameters (1 parameters)	824
44.4.2. Query Parameters (12 parameters)	825
44.5. MESSAGE BODY	825
44.6. EXAMPLES	826
44.6.1. Queries	826
44.6.2. Mutations	827
44.7. SPRING BOOT AUTO-CONFIGURATION	828
CHAPTER 45. GROOVY	829
45.1. DEPENDENCIES	829
45.2. URI FORMAT	829
45.3. GROOVY OPTIONS	829
45.4. EXAMPLES	829
45.5. GROOVY CONTEXT	830
45.6. HOW TO GET THE RESULT FROM MULTIPLE STATEMENTS SCRIPT	830
45.7. CUSTOMIZING GROOVY SHELL	830
45.8. LOADING SCRIPT FROM EXTERNAL RESOURCE	831
45.9. SPRING BOOT AUTO-CONFIGURATION	831
CHAPTER 46. GRPC	832
46.1. DEPENDENCIES	832
46.2. URI FORMAT	832

46.3. CONFIGURING OPTIONS	832
46.3.1. Configuring component options	832
46.3.2. Configuring endpoint options	832
46.4. COMPONENT OPTIONS	833
46.5. ENDPOINT OPTIONS	833
46.5.1. Path Parameters (3 parameters)	834
46.5.2. Query Parameters (29 parameters)	834
46.6. MESSAGE HEADERS	837
46.7. TRANSPORT SECURITY AND AUTHENTICATION SUPPORT	838
46.8. GRPC PRODUCER RESOURCE TYPE MAPPING	839
46.9. EXAMPLES	840
46.10. CONFIGURATION	841
46.11. ADDITIONAL RESOURCES	842
46.12. SPRING BOOT AUTO-CONFIGURATION	842
CHAPTER 47. HEADER	844
47.1. DEPENDENCIES	844
47.2. HEADER OPTIONS	844
47.3. EXAMPLE USAGE	844
47.4. SPRING BOOT AUTO-CONFIGURATION	844
CHAPTER 48. HL7	862
48.1. DEPENDENCIES	862
48.2. HL7 MLLP PROTOCOL	862
48.2.1. Exposing an HL7 listener using Mina	863
48.2.2. Exposing an HL7 listener using Netty (available from Camel 2.15 onwards)	863
48.3. HL7 MODEL USING JAVA.LANG.STRING OR BYTE[]	864
48.4. HL7V2 MODEL USING HAPI	864
48.5. HL7 DATAFORMAT	865
48.5.1. Segment separators	865
48.5.2. Charset	865
48.6. MESSAGE HEADERS	866
48.7. DEPENDENCIES	867
48.8. SPRING BOOT AUTO-CONFIGURATION	868
CHAPTER 49. HTTP	869
49.1. DEPENDENCIES	869
49.2. URI FORMAT	869
49.3. CONFIGURING OPTIONS	869
49.3.1. Configuring Component Options	869
49.3.2. Configuring Endpoint Options	869
49.4. COMPONENT OPTIONS	870
49.5. ENDPOINT OPTIONS	874
49.5.1. Path Parameters (1 parameters)	874
49.5.2. Query Parameters (54 parameters)	874
49.6. MESSAGE HEADERS	881
49.7. MESSAGE BODY	883
49.8. USING SYSTEM PROPERTIES	883
49.9. RESPONSE CODE	884
49.10. EXCEPTIONS	884
49.11. WHICH HTTP METHOD WILL BE USED	885
49.12. HOW TO GET ACCESS TO HTTPSERVLETREQUEST AND HTTPSERVLETRESPONSE	885
49.13. CONFIGURING URI TO CALL	885
49.14. CONFIGURING URI PARAMETERS	886

49.15. HOW TO SET THE HTTP METHOD (GET/PATCH/POST/PUT/DELETE/HEAD/OPTIONS/TRACE) TO THE HTTP PRODUCER	886
49.16. USING CLIENT TIMEOUT - SO_TIMEOUT	886
49.17. CONFIGURING A PROXY	887
49.17.1. Using proxy settings outside of URI	887
49.18. CONFIGURING CHARSET	887
49.18.1. Sample with scheduled poll	887
49.18.2. URI Parameters from the endpoint URI	888
49.18.3. URI Parameters from the Message	888
49.18.4. Getting the Response Code	888
49.19. DISABLING COOKIES	888
49.20. BASIC AUTH WITH THE STREAMING MESSAGE BODY	888
49.21. ADVANCED USAGE	888
49.21.1. Setting up SSL for HTTP Client	888
49.22. SPRING BOOT AUTO-CONFIGURATION	891
CHAPTER 50. HASHICORP VAULT	897
50.1. DEPENDENCIES	897
50.2. CONFIGURING OPTIONS	897
50.2.1. Component Level Options	897
50.2.2. Endpoint Level Options	897
50.3. COMPONENT OPTIONS	898
50.4. ENDPOINT OPTIONS	898
50.4.1. Path Parameters (1 parameters)	898
50.4.2. Query Parameters (8 parameters)	899
50.5. MESSAGE HEADERS	900
50.6. EXAMPLES	900
50.6.1. Using Hashicorp Vault Property Function	901
50.7. SPRING BOOT AUTO-CONFIGURATION	903
CHAPTER 51. INFINISPAN	904
51.1. DEPENDENCIES	904
51.2. URI FORMAT	904
51.3. CONFIGURING OPTIONS	904
51.3.1. Configuring Component Options	904
51.3.2. Configuring Endpoint Options	904
51.4. COMPONENT OPTIONS	905
51.5. ENDPOINT OPTIONS	909
51.5.1. Path Parameters (1 parameters)	909
51.5.2. Query Parameters (26 parameters)	909
51.6. CAMEL OPERATIONS	912
51.7. MESSAGE HEADERS	916
51.8. EXAMPLES	917
51.9. USING THE INFINISPAN BASED IDEMPOTENT REPOSITORY	919
51.10. USING THE INFINISPAN BASED AGGREGATION REPOSITORY	920
51.11. SPRING BOOT AUTO-CONFIGURATION	921
CHAPTER 52. INFINISPAN EMBEDDED	925
52.1. DEPENDENCIES	925
52.2. URI FORMAT	925
52.3. CONFIGURING OPTIONS	925
52.3.1. Configuring Component Options	925
52.3.2. Configuring Endpoint Options	926
52.4. COMPONENT OPTIONS	926

52.5. ENDPOINT OPTIONS	931
52.5.1. Path Parameters (1 parameters)	931
52.5.2. Query Parameters (20 parameters)	931
52.6. MESSAGE HEADERS	936
52.7. CAMEL OPERATIONS	940
52.8. EXAMPLES	944
52.9. USING THE INFINISPAN BASED IDEMPOTENT REPOSITORY	945
52.10. USING THE INFINISPAN BASED AGGREGATION REPOSITORY	946
52.11. SPRING BOOT AUTO-CONFIGURATION	947
CHAPTER 53. JACKSONXML	953
53.1. DEPENDENCIES	953
53.2. JACKSONXML OPTIONS	953
53.2.1. Using Jackson XML in Spring DSL	955
53.2.2. Excluding POJO fields from marshalling	955
53.3. INCLUDE/EXCLUDE FIELDS USING THE JSONVIEW ATTRIBUTE WITH `JACKSONXML` DATAFORMAT	955
53.4. SETTING SERIALIZATION INCLUDE OPTION	956
53.5. UNMARSHALLING FROM XML TO POJO WITH DYNAMIC CLASS NAME	956
53.6. UNMARSHALLING FROM XML TO LIST<MAP> OR LIST<POJO>	956
53.7. USING CUSTOM JACKSON MODULES	957
53.8. ENABLING OR DISABLE FEATURES USING JACKSON	957
53.9. CONVERTING MAPS TO POJO USING JACKSON	958
53.10. FORMATTED XML MARSHALLING (PRETTY-PRINTING)	958
53.11. SPRING BOOT AUTO-CONFIGURATION	959
CHAPTER 54. JAXB	961
54.1. DEPENDENCIES	961
54.2. OPTIONS	961
54.3. USING THE JAVA DSL	963
54.4. USING SPRING XML	963
54.5. MULTIPLE CONTEXT PATHS	964
54.6. PARTIAL MARSHALLING / UNMARSHALLING	964
54.7. FRAGMENT	964
54.8. IGNORING NON-XML CHARACTERS	965
54.9. WORKING WITH THE OBJECTFACTORY	965
54.10. SETTING THE ENCODING	965
54.11. CONTROLLING NAMESPACE PREFIX MAPPING	966
54.12. SCHEMA VALIDATION	966
54.13. SCHEMA LOCATION	967
54.14. MARSHAL DATA THAT IS ALREADY XML	967
54.15. SPRING BOOT AUTO-CONFIGURATION	967
CHAPTER 55. JASYPT	970
55.1. DEPENDENCIES	970
55.2. TOOLING	970
55.3. PROTECTING THE MASTER PASSWORD	971
55.4. EXAMPLE WITH JAVA DSL	971
55.5. EXAMPLE WITH SPRING XML	972
55.6. SPRING BOOT AUTO-CONFIGURATION	973
CHAPTER 56. JDBC	975
56.1. DEPENDENCIES	975
56.2. URI FORMAT	975

56.3. CONFIGURING OPTIONS	975
56.3.1. Configuring Component Options	975
56.3.2. Configuring Endpoint Options	975
56.4. COMPONENT OPTIONS	976
56.5. ENDPOINT OPTIONS	976
56.5.1. Path Parameter (1 parameter)	977
56.5.2. Query Parameters (14 parameters)	977
56.6. MESSAGE HEADERS	979
56.7. RESULT	980
56.8. GENERATED KEYS	980
56.9. USING NAMED PARAMETERS	981
56.10. SAMPLES	981
56.11. SAMPLE - POLLING THE DATABASE EVERY MINUTE	982
56.12. SAMPLE - MOVE DATA BETWEEN DATA SOURCES	982
56.13. SPRING BOOT AUTO-CONFIGURATION	982
CHAPTER 57. JIRA	984
57.1. DEPENDENCIES	984
57.2. URI FORMAT	984
57.3. CONFIGURING OPTIONS	985
57.3.1. Configuring Component Options	985
57.3.2. Configuring Endpoint Options	985
57.4. COMPONENT OPTIONS	986
57.5. ENDPOINT OPTIONS	987
57.5.1. Path Parameters (1 parameters)	987
57.5.2. Query Parameters (16 parameters)	988
57.6. CLIENT FACTORY	990
57.7. AUTHENTICATION	990
57.7.1. Basic authentication requirements:	991
57.7.2. OAuth authentication requirements:	991
57.8. JQL	991
57.9. OPERATIONS	991
57.10. ADDISSUE	992
57.11. ADDCOMMENT	992
57.12. ATTACH	992
57.13. DELETEISSUE	992
57.14. TRANSITIONISSUE	992
57.15. UPDATEISSUE	993
57.16. WATCHER	993
57.17. WATCHUPDATES (CONSUMER)	993
57.18. SPRING BOOT AUTO-CONFIGURATION	993
CHAPTER 58. JMS	996
58.1. DEPENDENCIES	996
58.2. URI FORMAT	996
58.2.1. Using ActiveMQ	997
58.2.2. Transactions and Cache Levels	997
58.2.3. Durable Subscriptions with JMS 1.1	997
58.2.3.1. Durable Subscriptions with JMS 2.0	998
58.2.4. Message Header Mapping	998
58.3. CONFIGURING OPTIONS	998
58.3.1. Configuring Component Options	998
58.3.2. Configuring Endpoint Options	999

58.4. COMPONENT OPTIONS	999
58.5. ENDPOINT OPTIONS	1018
58.5.1. Path Parameters (2 parameters)	1019
58.5.2. Query Parameters (95 parameters)	1019
58.6. SAMPLES	1037
58.6.1. Receiving from JMS	1037
58.6.2. Sending to JMS	1038
58.6.3. Using Annotations	1038
58.6.4. Spring DSL sample	1038
58.6.5. Other samples	1038
58.6.6. Using JMS as a Dead Letter Queue storing Exchange	1038
58.6.7. Using JMS as a Dead Letter Channel storing error only	1039
58.7. MESSAGE MAPPING BETWEEN JMS AND CAMEL	1039
58.7.1. Disabling auto-mapping of JMS messages	1040
58.7.2. Using a custom MessageConverter	1040
58.7.3. Controlling the mapping strategy selected	1040
58.8. MESSAGE FORMAT WHEN SENDING	1041
58.9. MESSAGE FORMAT WHEN RECEIVING	1041
58.10. ABOUT USING CAMEL TO SEND AND RECEIVE MESSAGES AND JMSREPLYTO	1042
58.10.1. JmsProducer	1043
58.10.2. JmsConsumer	1043
58.11. REUSE ENDPOINT AND SEND TO DIFFERENT DESTINATIONS COMPUTED AT RUNTIME	1044
58.12. CONFIGURING DIFFERENT JMS PROVIDERS	1045
58.12.1. Using JNDI to find the ConnectionFactory	1045
58.13. CONCURRENT CONSUMING	1045
58.13.1. Concurrent Consuming with async consumer	1045
58.14. REQUEST/REPLY OVER JMS	1046
58.14.1. Request/reply over JMS and using a shared fixed reply queue	1048
58.14.2. Request/reply over JMS and using an exclusive fixed reply queue	1048
58.15. SYNCHRONIZING CLOCKS BETWEEN SENDERS AND RECEIVERS	1049
58.16. ABOUT TIME TO LIVE	1049
58.17. ENABLING TRANSACTED CONSUMPTION	1050
58.18. USING JMSREPLYTO FOR LATE REPLIES	1050
58.19. USING A REQUEST TIMEOUT	1051
58.20. SENDING AN INONLY MESSAGE AND KEEPING THE JMSREPLYTO HEADER	1051
58.21. SETTING JMS PROVIDER OPTIONS ON THE DESTINATION	1051
58.22. SPRING BOOT AUTO-CONFIGURATION	1052
CHAPTER 59. JOLOKIA	1070
59.1. DEPENDENCIES	1070
59.2. USING JOLOKIA AGENT	1070
59.3. USING JOLOKIA RESTRICTOR	1070
59.4. IMPLEMENTING SPRING JOLOKIA CONFIGURATION	1071
59.5. USING CAMEL LOG HANDLER HOLDER CONFIGURATION	1071
59.6. USING JOLOKIA CONFIGURATION IN KUBERNETES	1071
CHAPTER 60. JPA	1073
60.1. DEPENDENCIES	1073
60.2. SENDING TO THE ENDPOINT	1073
60.3. CONSUMING FROM THE ENDPOINT	1073
60.4. URI FORMAT	1074
60.5. CONFIGURING OPTIONS	1074
60.5.1. Configuring Component Options	1074

60.5.2. Configuring Endpoint Options	1074
60.6. COMPONENT OPTIONS	1075
60.6.1. Endpoint Options	1076
60.6.1.1. Path Parameters (1 parameters)	1076
60.6.1.2. Query Parameters (44 parameters)	1076
60.7. MESSAGE HEADERS	1082
60.8. CONFIGURING ENTITYMANAGERFACTORY	1083
60.9. CONFIGURING TRANSACTIONMANAGER	1083
60.10. USING A CONSUMER WITH A NAMED QUERY	1083
60.11. USING A CONSUMER WITH A QUERY	1084
60.12. USING A CONSUMER WITH A NATIVE QUERY	1084
60.13. USING A PRODUCER WITH A NAMED QUERY	1084
60.14. USING A PRODUCER WITH A QUERY	1084
60.15. USING A PRODUCER WITH A NATIVE QUERY	1084
60.16. USING THE JPA-BASED IDEMPOTENT REPOSITORY	1085
60.17. SPRING BOOT AUTO-CONFIGURATION	1086
CHAPTER 61. JQ	1088
61.1. DEPENDENCIES	1088
61.2. JQ OPTIONS	1088
61.3. EXAMPLES	1088
61.4. MESSAGE BODY TYPES	1088
61.5. USING HEADER AS INPUT	1089
61.6. CAMEL SUPPLIED JQ FUNCTIONS	1089
61.7. SPRING BOOT AUTO-CONFIGURATION	1089
CHAPTER 62. JSLT	1090
62.1. DEPENDENCIES	1090
62.2. URI FORMAT	1090
62.3. CONFIGURING OPTIONS	1090
62.3.1. Configuring Component Options	1090
62.3.2. Configuring Endpoint Options	1090
62.4. COMPONENT OPTIONS	1091
62.4.1. Endpoint Options	1092
62.4.1.1. Path Parameters (1 parameters)	1092
62.4.1.2. Query Parameters (7 parameters)	1092
62.5. MESSAGE HEADERS	1093
62.6. PASSING VALUES TO JSLT	1094
62.7. SAMPLES	1094
62.8. SPRING BOOT AUTO-CONFIGURATION	1095
CHAPTER 63. JSON GSON	1097
63.1. DEPENDENCIES	1097
63.2. GSON OPTIONS	1097
63.3. SPRING BOOT AUTO-CONFIGURATION	1097
CHAPTER 64. JSON JACKSON	1099
64.1. DEPENDENCIES	1099
64.2. JACKSON OPTIONS	1099
64.3. USING CUSTOM OBJECTMAPPER	1104
64.4. USING JACKSON FOR AUTOMATIC TYPE CONVERSION	1104
64.5. SPRING BOOT AUTO-CONFIGURATION	1104
CHAPTER 65. JSONPATH	1108

65.1. DEPENDENCIES	1108
65.2. JSONPATH OPTIONS	1108
65.3. EXAMPLES	1109
65.4. JSONPATH SYNTAX	1110
65.4.1. Easy JSONPath Syntax	1110
65.5. SUPPORTED MESSAGE BODY TYPES	1110
65.6. SUPPRESSING EXCEPTIONS	1111
65.7. INLINE SIMPLE EXPRESSIONS	1111
65.8. JSONPATH INJECTION	1112
65.9. ENCODING DETECTION	1112
65.10. SPLIT JSON DATA INTO SUB ROWS AS JSON	1113
65.11. USING HEADER AS INPUT	1113
65.12. SPRING BOOT AUTO-CONFIGURATION	1113
CHAPTER 66. KAFKA	1115
66.1. DEPENDENCIES	1115
66.2. URI FORMAT	1115
66.3. CONFIGURING OPTIONS	1115
66.3.1. Configuring Component Options	1115
66.3.2. Configuring Endpoint Options	1115
66.4. COMPONENT OPTIONS	1116
66.5. ENDPOINT OPTIONS	1135
66.5.1. Path Parameters (1 parameters)	1135
66.5.2. Query Parameters (108 parameters)	1135
66.6. MESSAGE HEADERS	1153
66.7. USAGE	1155
66.7.1. Consumer error handling	1155
66.7.2. Consumer error handling (advanced)	1156
66.8. USING THE KAFKA IDEMPOTENT REPOSITORY	1156
66.9. USING MANUAL COMMIT WITH KAFKA CONSUMER	1159
66.10. PAUSABLE CONSUMERS	1160
66.11. KAFKA HEADERS PROPAGATION	1160
66.12. KAFKA TRANSACTION	1161
66.13. SETTING KERBEROS CONFIG FILE	1161
66.14. BATCHING CONSUMER	1162
66.14.1. Automatic Commits	1162
66.14.1.1. Handling Errors with Automatic Commits	1162
66.14.2. Manual Commits	1163
66.14.3. Dealing with long polling timeouts	1164
66.15. CUSTOM SUBSCRIPTION ADAPTERS	1164
66.16. INTEROPERABILITY	1165
66.16.1. JMS	1165
66.17. EXAMPLES	1165
66.17.1. Consuming messages from Kafka	1165
66.17.2. Producing messages to Kafka	1166
66.17.3. SSL configuration	1167
66.18. SPRING BOOT AUTO-CONFIGURATION	1168
CHAPTER 67. KAMELET	1186
67.1. DEPENDENCIES	1186
67.2. URI FORMAT	1186
67.3. CONFIGURING OPTIONS	1186
67.3.1. Configuring Component Options	1186

67.3.2. Configuring Endpoint Options	1186
67.4. COMPONENT OPTIONS	1187
67.5. ENDPOINT OPTIONS	1188
67.5.1. Path Parameters (2 parameters)	1188
67.5.2. Query Parameters (8 parameters)	1189
67.6. DISCOVERY	1190
67.7. SAMPLES	1190
67.8. SPRING BOOT AUTO-CONFIGURATION	1191
CHAPTER 68. KAMELET MAIN	1193
68.1. INITIAL CONFIGURATION	1193
68.2. AUTOMATIC DEPENDENCIES DOWNLOADING	1193
CHAPTER 69. KUBERNETES	1194
69.1. KUBERNETES COMPONENTS	1194
69.2. DEPENDENCIES	1195
69.3. USAGE	1195
69.3.1. Producer examples	1195
69.4. USING KUBERNETES CONFIGMAPS AND SECRETS	1195
69.5. SPRING BOOT AUTO-CONFIGURATION	1195
CHAPTER 70. KUBERNETES CONFIGMAP	1213
70.1. DEPENDENCIES	1213
70.2. CONFIGURING OPTIONS	1213
70.2.1. Configuring Component Options	1213
70.2.2. Configuring Endpoint Options	1213
70.3. COMPONENT OPTIONS	1214
70.4. ENDPOINT OPTIONS	1214
70.4.1. Path Parameters (1 parameters)	1214
70.4.2. Query Parameters (33 parameters)	1215
70.5. MESSAGE HEADERS	1217
70.6. SUPPORTED PRODUCER OPERATION	1219
70.7. KUBERNETES CONFIGMAPS PRODUCER EXAMPLES	1219
70.8. KUBERNETES CONFIGMAPS CONSUMER EXAMPLE	1220
70.9. USING CONFIGMAP PROPERTIES FUNCTION WITH KUBERNETES	1220
70.9.1. Using configmap with Kubernetes	1221
70.10. AUTOMATIC CAMEL CONTEXT RELOADING ON CONFIGMAPS REFRESH	1221
70.11. SPRING BOOT AUTO-CONFIGURATION	1221
CHAPTER 71. KUBERNETES CUSTOM RESOURCES	1239
71.1. DEPENDENCIES	1239
71.2. CONFIGURING OPTIONS	1239
71.2.1. Configuring Component Options	1239
71.2.2. Configuring Endpoint Options	1239
71.3. COMPONENT OPTIONS	1240
71.4. ENDPOINT OPTIONS	1240
71.4.1. Path Parameters (1 parameters)	1240
71.4.2. Query Parameters (33 parameters)	1241
71.5. MESSAGE HEADERS	1243
71.6. SUPPORTED PRODUCER OPERATION	1246
71.7. SPRING BOOT AUTO-CONFIGURATION	1246
CHAPTER 72. KUBERNETES DEPLOYMENTS	1263
72.1. DEPENDENCIES	1263

72.2. CONFIGURING OPTIONS	1263
72.2.1. Configuring Component Options	1263
72.2.2. Configuring Endpoint Options	1263
72.3. COMPONENT OPTIONS	1264
72.4. ENDPOINT OPTIONS	1264
72.4.1. Path Parameters (1 parameters)	1264
72.4.2. Query Parameters (33 parameters)	1265
72.5. MESSAGE HEADERS	1267
72.6. SUPPORTED PRODUCER OPERATION	1269
72.7. KUBERNETES DEPLOYMENTS PRODUCER EXAMPLES	1269
72.7.1. Kubernetes Deployments Consumer Example	1270
72.8. SPRING BOOT AUTO-CONFIGURATION	1270
CHAPTER 73. KUBERNETES EVENT	1288
73.1. DEPENDENCIES	1288
73.2. CONFIGURING OPTIONS	1288
73.2.1. Configuring Component Options	1288
73.2.2. Configuring Endpoint Options	1288
73.3. COMPONENT OPTIONS	1289
73.4. ENDPOINT OPTIONS	1289
73.4.1. Path Parameters (1 parameters)	1289
73.4.2. Query Parameters (33 parameters)	1290
73.5. MESSAGE HEADERS	1292
73.6. SUPPORTED PRODUCER OPERATION	1295
73.7. KUBERNETES EVENTS PRODUCER EXAMPLES	1295
73.8. KUBERNETES EVENTS CONSUMER EXAMPLE	1298
73.9. SPRING BOOT AUTO-CONFIGURATION	1298
CHAPTER 74. KUBERNETES HPA	1316
74.1. DEPENDENCIES	1316
74.2. CONFIGURING OPTIONS	1316
74.2.1. Configuring Component Options	1316
74.2.2. Configuring Endpoint Options	1316
74.3. COMPONENT OPTIONS	1317
74.4. ENDPOINT OPTIONS	1317
74.4.1. Path Parameters (1 parameters)	1317
74.4.2. Query Parameters (33 parameters)	1318
74.5. MESSAGE HEADERS	1320
74.6. SUPPORTED PRODUCER OPERATION	1322
74.7. KUBERNETES HPA PRODUCER EXAMPLES	1322
74.8. KUBERNETES HPA CONSUMER EXAMPLE	1323
74.9. SPRING BOOT AUTO-CONFIGURATION	1323
CHAPTER 75. KUBERNETES JOB	1341
75.1. DEPENDENCIES	1341
75.2. CONFIGURING OPTIONS	1341
75.2.1. Configuring Component Options	1341
75.2.2. Configuring Endpoint Options	1341
75.3. COMPONENT OPTIONS	1342
75.4. ENDPOINT OPTIONS	1342
75.4.1. Path Parameters (1 parameters)	1342
75.4.2. Query Parameters (33 parameters)	1343
75.5. MESSAGE HEADERS	1345
75.6. SUPPORTED PRODUCER OPERATION	1346

75.7. KUBERNETES JOB PRODUCER EXAMPLES	1347
75.8. SPRING BOOT AUTO-CONFIGURATION	1349
CHAPTER 76. KUBERNETES NAMESPACES	1367
76.1. DEPENDENCIES	1367
76.2. CONFIGURING OPTIONS	1367
76.2.1. Configuring Component Options	1367
76.2.2. Configuring Endpoint Options	1367
76.3. COMPONENT OPTIONS	1368
76.4. ENDPOINT OPTIONS	1368
76.4.1. Path Parameters (1 parameters)	1368
76.4.2. Query Parameters (33 parameters)	1369
76.5. MESSAGE HEADERS	1371
76.6. SUPPORTED PRODUCER OPERATION	1372
76.7. KUBERNETES NAMESPACES PRODUCER EXAMPLES	1373
76.8. KUBERNETES NAMESPACES CONSUMER EXAMPLE	1373
76.9. SPRING BOOT AUTO-CONFIGURATION	1374
CHAPTER 77. KUBERNETES NODES	1392
77.1. DEPENDENCIES	1392
77.2. CONFIGURING OPTIONS	1392
77.2.1. Configuring Component Options	1392
77.2.2. Configuring Endpoint Options	1392
77.3. COMPONENT OPTIONS	1393
77.4. ENDPOINT OPTIONS	1393
77.4.1. Path Parameters (1 parameters)	1393
77.4.2. Query Parameters (33 parameters)	1394
77.5. MESSAGE HEADERS	1396
77.6. SUPPORTED PRODUCER OPERATION	1398
77.7. KUBERNETES NODES PRODUCER EXAMPLES	1398
77.8. KUBERNETES NODES CONSUMER EXAMPLE	1399
77.9. SPRING BOOT AUTO-CONFIGURATION	1399
CHAPTER 78. KUBERNETES PERSISTENT VOLUME	1417
78.1. DEPENDENCIES	1417
78.2. CONFIGURING OPTIONS	1417
78.2.1. Configuring Component Options	1417
78.2.2. Configuring Endpoint Options	1417
78.3. COMPONENT OPTIONS	1418
78.4. ENDPOINT OPTIONS	1418
78.4.1. Path Parameters (1 parameters)	1418
78.4.2. Query Parameters (21 parameters)	1418
78.5. MESSAGE HEADERS	1420
78.6. SUPPORTED PRODUCER OPERATION	1421
78.7. KUBERNETES PERSISTENT VOLUMES PRODUCER EXAMPLES	1421
78.8. SPRING BOOT AUTO-CONFIGURATION	1422
CHAPTER 79. KUBERNETES PERSISTENT VOLUME CLAIM	1439
79.1. DEPENDENCIES	1439
79.2. CONFIGURING OPTIONS	1439
79.2.1. Configuring Component Options	1439
79.2.2. Configuring Endpoint Options	1439
79.3. COMPONENT OPTIONS	1440
79.4. ENDPOINT OPTIONS	1440

79.4.1. Path Parameters (1 parameters)	1440
79.4.2. Query Parameters (21 parameters)	1440
79.5. MESSAGE HEADERS	1442
79.6. SUPPORTED PRODUCER OPERATION	1443
79.7. KUBERNETES PERSISTENT VOLUME CLAIMS PRODUCER EXAMPLES	1443
79.8. SPRING BOOT AUTO-CONFIGURATION	1444
CHAPTER 80. KUBERNETES PODS	1461
80.1. DEPENDENCIES	1461
80.2. CONFIGURING OPTIONS	1461
80.2.1. Configuring Component Options	1461
80.2.2. Configuring Endpoint Options	1461
80.3. COMPONENT OPTIONS	1462
80.4. ENDPOINT OPTIONS	1462
80.4.1. Path Parameters (1 parameters)	1462
80.4.2. Query Parameters (33 parameters)	1463
80.5. MESSAGE HEADERS	1465
80.6. SUPPORTED PRODUCER OPERATION	1467
80.7. KUBERNETES PODS PRODUCER EXAMPLES	1467
80.8. KUBERNETES PODS CONSUMER EXAMPLE	1468
80.9. SPRING BOOT AUTO-CONFIGURATION	1468
CHAPTER 81. KUBERNETES REPLICATION CONTROLLER	1486
81.1. DEPENDENCIES	1486
81.2. CONFIGURING OPTIONS	1486
81.2.1. Configuring Component Options	1486
81.2.2. Configuring Endpoint Options	1486
81.3. COMPONENT OPTIONS	1487
81.4. ENDPOINT OPTIONS	1487
81.4.1. Path Parameters (1 parameters)	1487
81.4.2. Query Parameters (33 parameters)	1488
81.5. MESSAGE HEADERS	1490
81.6. SUPPORTED PRODUCER OPERATION	1492
81.7. KUBERNETES REPLICATION CONTROLLERS PRODUCER EXAMPLES	1493
81.8. KUBERNETES REPLICATION CONTROLLERS CONSUMER EXAMPLE	1493
81.9. SPRING BOOT AUTO-CONFIGURATION	1493
CHAPTER 82. KUBERNETES RESOURCES QUOTA	1511
82.1. DEPENDENCIES	1511
82.2. CONFIGURING OPTIONS	1511
82.2.1. Configuring Component Options	1511
82.2.2. Configuring Endpoint Options	1511
82.3. COMPONENT OPTIONS	1512
82.4. ENDPOINT OPTIONS	1512
82.4.1. Path Parameters (1 parameters)	1512
82.4.2. Query Parameters (21 parameters)	1512
82.5. MESSAGE HEADERS	1514
82.6. SUPPORTED PRODUCER OPERATION	1515
82.7. KUBERNETES RESOURCE QUOTA PRODUCER EXAMPLES	1515
82.8. SPRING BOOT AUTO-CONFIGURATION	1516
CHAPTER 83. KUBERNETES SECRETS	1534
83.1. DEPENDENCIES	1534
83.2. CONFIGURING OPTIONS	1534

83.2.1. Configuring Component Options	1534
83.2.2. Configuring Endpoint Options	1534
83.3. COMPONENT OPTIONS	1535
83.4. ENDPOINT OPTIONS	1535
83.4.1. Path Parameters (1 parameters)	1535
83.4.2. Query Parameters (21 parameters)	1535
83.5. MESSAGE HEADERS	1537
83.6. SUPPORTED PRODUCER OPERATION	1538
83.7. KUBERNETES SECRETS PRODUCER EXAMPLES	1538
83.8. USING SECRETS PROPERTIES FUNCTION WITH KUBERNETES	1539
83.9. AUTOMATIC CAMEL CONTEXT RELOADING ON SECRET REFRESH	1540
83.10. SPRING BOOT AUTO-CONFIGURATION	1540
CHAPTER 84. KUBERNETES SERVICE ACCOUNT	1558
84.1. DEPENDENCIES	1558
84.2. CONFIGURING OPTIONS	1558
84.2.1. Configuring Component Options	1558
84.2.2. Configuring Endpoint Options	1558
84.3. COMPONENT OPTIONS	1559
84.4. ENDPOINT OPTIONS	1559
84.4.1. Path Parameters (1 parameters)	1559
84.4.2. Query Parameters (21 parameters)	1559
84.5. MESSAGE HEADERS	1561
84.6. SUPPORTED PRODUCER OPERATION	1562
84.7. KUBERNETES SERVICEACCOUNTS PRODUCE EXAMPLES	1562
84.8. SPRING BOOT AUTO-CONFIGURATION	1563
CHAPTER 85. KUBERNETES SERVICES	1581
85.1. DEPENDENCIES	1581
85.2. CONFIGURING OPTIONS	1581
85.2.1. Configuring Component Options	1581
85.2.2. Configuring Endpoint Options	1581
85.3. COMPONENT OPTIONS	1582
85.4. ENDPOINT OPTIONS	1582
85.4.1. Path Parameters (1 parameters)	1582
85.4.2. Query Parameters (33 parameters)	1583
85.5. MESSAGE HEADERS	1585
85.6. SUPPORTED PRODUCER OPERATION	1587
85.7. KUBERNETES SERVICES PRODUCER EXAMPLES	1587
85.8. KUBERNETES SERVICES CONSUMER EXAMPLE	1588
85.8.1. Spring Boot Auto-Configuration	1588
CHAPTER 86. OPENSIFT BUILDS	1606
86.1. DEPENDENCIES	1606
86.2. CONFIGURING OPTIONS	1606
86.2.1. Configuring Component Options	1606
86.2.2. Configuring Endpoint Options	1606
86.3. COMPONENT OPTIONS	1607
86.4. ENDPOINT OPTIONS	1607
86.4.1. Path Parameters (1 parameters)	1607
86.4.2. Query Parameters (21 parameters)	1607
86.5. MESSAGE HEADERS	1609
86.6. SUPPORTED PRODUCER OPERATION	1610
86.7. OPENSIFT BUILDS PRODUCER EXAMPLES	1610

86.8. SPRING BOOT AUTO-CONFIGURATION	1611
CHAPTER 87. OPENSIFT BUILD CONFIG	1629
87.1. DEPENDENCIES	1629
87.2. CONFIGURING OPTIONS	1629
87.2.1. Configuring Component Options	1629
87.2.2. Configuring Endpoint Options	1629
87.3. COMPONENT OPTIONS	1630
87.4. ENDPOINT OPTIONS	1630
87.4.1. Path Parameters (1 parameters)	1630
87.4.2. Query Parameters (21 parameters)	1630
87.5. MESSAGE HEADERS	1632
87.6. SUPPORTED PRODUCER OPERATION	1633
87.7. OPENSIFT BUILD CONFIGS PRODUCER EXAMPLES	1633
87.8. SPRING BOOT AUTO-CONFIGURATION	1634
CHAPTER 88. OPENSIFT DEPLOYMENT CONFIGS	1651
88.1. DEPENDENCIES	1651
88.2. CONFIGURING OPTIONS	1651
88.2.1. Configuring Component Options	1651
88.2.2. Configuring Endpoint Options	1651
88.3. COMPONENT OPTIONS	1652
88.4. ENDPOINT OPTIONS	1652
88.4.1. Path Parameters (1 parameters)	1652
88.4.2. Query Parameters (33 parameters)	1653
88.5. MESSAGE HEADERS	1655
88.6. SUPPORTED PRODUCER OPERATION	1657
88.7. OPENSIFT DEPLOYMENT CONFIGS PRODUCER EXAMPLES	1657
88.8. OPENSIFT DEPLOYMENT CONFIGS CONSUMER EXAMPLE	1658
88.9. SPRING BOOT AUTO-CONFIGURATION	1658
CHAPTER 89. KUDU	1676
89.1. DEPENDENCIES	1676
89.2. PREREQUISITES	1676
89.3. CONFIGURING OPTIONS	1676
89.3.1. Configuring Component Options	1676
89.3.2. Configuring Endpoint Options	1676
89.4. COMPONENT OPTIONS	1677
89.5. ENDPOINT OPTIONS	1677
89.5.1. Path Parameters (3 parameters)	1677
89.5.2. Query Parameters (2 parameters)	1678
89.6. MESSAGE HEADERS	1678
89.7. INPUT BODY FORMATS	1679
89.7.1. Insert, delete, update, and upsert	1679
89.8. OUTPUT BODY FORMATS	1679
89.8.1. Scan	1680
89.9. SPRING BOOT AUTO-CONFIGURATION	1680
CHAPTER 90. LANGUAGE	1681
90.1. DEPENDENCIES	1681
90.2. URI FORMAT	1681
90.3. CONFIGURING OPTIONS	1681
90.3.1. Configuring Component Options	1681
90.3.2. Configuring Endpoint Options	1682

90.4. COMPONENT OPTIONS	1682
90.5. ENDPOINT OPTIONS	1683
90.5.1. Path Parameters (2 parameters)	1683
90.5.2. Query Parameters (7 parameters)	1684
90.6. MESSAGE HEADERS	1684
90.7. EXAMPLES	1685
90.8. LOADING SCRIPTS FROM RESOURCES	1685
90.9. SPRING BOOT AUTO-CONFIGURATION	1685
CHAPTER 91. LDAP	1687
91.1. DEPENDENCIES	1687
91.2. URI FORMAT	1687
91.3. CONFIGURING OPTIONS	1687
91.3.1. Configuring Component Options	1687
91.3.2. Configuring Endpoint Options	1687
91.4. COMPONENT OPTIONS	1688
91.5. ENDPOINT OPTIONS	1688
91.5.1. Path Parameters (1 parameters)	1688
91.5.2. Query Parameters (5 parameters)	1689
91.6. RESULT	1690
91.7. DIRCONTEXT	1690
91.8. SECURITY CONCERNS RELATED TO LDAP INJECTION	1690
91.9. SAMPLES	1691
91.9.1. Binding using credentials	1691
91.10. CONFIGURING SSL	1692
91.11. SPRING BOOT AUTO-CONFIGURATION	1694
CHAPTER 92. LOG	1696
92.1. DEPENDENCIES	1696
92.2. URI FORMAT	1696
92.3. CONFIGURING OPTIONS	1697
92.3.1. Configuring Component Options	1697
92.3.2. Configuring Endpoint Options	1697
92.4. COMPONENT OPTIONS	1697
92.5. ENDPOINT OPTIONS	1698
92.5.1. Path Parameters (1 parameters)	1698
92.5.2. Query Parameters (27 parameters)	1698
92.6. REGULAR LOGGER SAMPLE	1701
92.7. REGULAR LOGGER WITH FORMATTER SAMPLE	1701
92.8. THROUGHPUT LOGGER WITH GROUPSIZE SAMPLE	1702
92.9. THROUGHPUT LOGGER WITH GROUPINTERVAL SAMPLE	1702
92.10. MASKING SENSITIVE INFORMATION LIKE PASSWORD	1702
92.11. FULL CUSTOMIZATION OF THE LOGGING OUTPUT	1703
92.11.1. Convention over configuration	1703
92.12. SPRING BOOT AUTO-CONFIGURATION	1704
CHAPTER 93. LRA	1705
93.1. DEPENDENCIES	1705
93.2. SPRING BOOT AUTO-CONFIGURATION	1705
CHAPTER 94. MAIL	1706
94.1. DEPENDENCIES	1706
94.2. URI FORMAT	1706
94.3. CONFIGURING OPTIONS	1706

94.3.1. Configuring Component Options	1707
94.3.2. Configuring Endpoint Options	1707
94.4. COMPONENT OPTIONS	1707
94.5. ENDPOINT OPTIONS	1712
94.5.1. Path Parameters (2 parameters)	1713
94.5.2. Query Parameters (66 parameters)	1713
94.5.3. Sample endpoints	1721
94.5.4. Component alias names	1721
94.5.5. Default ports	1721
94.6. SSL SUPPORT	1722
94.6.1. Using the JSSE Configuration Utility	1722
94.6.2. Configuring JavaMail Directly	1723
94.7. MAIL MESSAGE CONTENT	1723
94.8. HEADERS TAKE PRECEDENCE OVER PRE-CONFIGURED RECIPIENTS	1723
94.9. MULTIPLE RECIPIENTS FOR EASIER CONFIGURATION	1723
94.10. SETTING SENDER NAME AND EMAIL	1724
94.11. JAVAMAIL API (EX SUN JAVAMAIL)	1724
94.12. SAMPLES	1724
94.13. SENDING MAIL WITH ATTACHMENT SAMPLE	1724
94.14. SSL SAMPLE	1725
94.15. CONSUMING MAILS WITH ATTACHMENT SAMPLE	1725
94.16. HOW TO SPLIT A MAIL MESSAGE WITH ATTACHMENTS	1726
94.17. USING CUSTOM SEARCHTERM	1726
94.18. POLLING OPTIMIZATION	1728
94.19. USING HEADERS WITH ADDITIONAL JAVA MAIL SENDER PROPERTIES	1728
94.20. SPRING BOOT AUTO-CONFIGURATION	1728
CHAPTER 95. MAIL MICROSOFT OAUTH	1735
95.1. DEPENDENCIES	1735
95.2. MICROSOFT EXCHANGE ONLINE OAUTH2 MAIL AUTHENTICATOR IMAP SAMPLE	1735
CHAPTER 96. MAPSTRUCT	1736
96.1. DEPENDENCIES	1736
96.2. URI FORMAT	1736
96.3. CONFIGURING OPTIONS	1736
96.3.1. Configuring Component Options	1736
96.3.2. Configuring Endpoint Options	1736
96.4. COMPONENT OPTIONS	1737
96.5. ENDPOINT OPTIONS	1738
96.5.1. Path Parameters (1 parameters)	1738
96.5.2. Query Parameters (2 parameters)	1738
96.6. SETTING UP MAPSTRUCT	1738
96.7. SPRING BOOT AUTO-CONFIGURATION	1739
CHAPTER 97. MASTER	1740
97.1. DEPENDENCIES	1740
97.2. USING THE MASTER ENDPOINT	1740
97.3. URI FORMAT	1740
97.4. CONFIGURING OPTIONS	1740
97.4.1. Configuring Component Options	1741
97.4.2. Configuring Endpoint Options	1741
97.5. COMPONENT OPTIONS	1741
97.6. ENDPOINT OPTIONS	1742
97.6.1. Path Parameters (2 parameters)	1742

97.6.2. Query Parameters (3 parameters)	1742
97.7. EXAMPLE	1743
97.8. IMPLEMENTATIONS	1744
97.9. SPRING BOOT AUTO-CONFIGURATION	1744
CHAPTER 98. MICROMETER	1746
98.1. DEPENDENCIES	1746
98.2. URI FORMAT	1746
98.3. CONFIGURING OPTIONS	1746
98.3.1. Configuring component options	1746
98.3.2. Configuring endpoint options	1747
98.4. COMPONENT OPTIONS	1747
98.5. ENDPOINT OPTIONS	1748
98.5.1. Path Parameters (3 parameters)	1748
98.5.2. Query Parameters (6 parameters)	1748
98.6. MESSAGE HEADERS	1749
98.7. METER REGISTRY	1750
98.8. DEFAULT CAMEL METRICS	1751
98.9. USAGE OF PRODUCERS	1752
98.9.1. Headers	1752
98.10. COUNTER	1753
98.10.1. Options	1753
98.10.2. Headers	1753
98.11. DISTRIBUTION SUMMARY	1754
98.11.1. Options	1754
98.11.2. Headers	1754
98.12. TIMER	1755
98.12.1. Options	1755
98.12.2. Headers	1755
98.13. USING MICROMETER ROUTE POLICY FACTORY	1756
98.14. USING MICROMETER MESSAGE HISTORY FACTORY	1757
98.15. MICROMETER EVENT NOTIFICATION	1758
98.16. INSTRUMENTING CAMEL THREAD POOLS	1758
98.17. EXPOSING MICROMETER STATISTICS IN JMX	1759
98.18. USING CAMEL MICROMETER WITH SPRING BOOT	1759
98.19. SPRING BOOT AUTO CONFIGURATION	1759
CHAPTER 99. MINIO	1762
99.1. PREREQUISITES	1762
99.2. DEPENDENCIES	1762
99.3. URI FORMAT	1762
99.4. CONFIGURING OPTIONS	1762
99.4.1. Configuring Component Options	1762
99.4.2. Configuring Endpoint Options	1763
99.5. COMPONENT OPTIONS	1763
99.6. ENDPOINT OPTIONS	1768
99.6.1. Path Parameters (1 parameters)	1768
99.6.2. Query Parameters (63 parameters)	1768
99.7. BATCH CONSUMER	1775
99.8. MESSAGE HEADERS	1775
99.8.1. Minio Producer operations	1779
99.8.2. Advanced Minio configuration	1779
99.8.3. Minio Producer Operation examples	1779

99.9. BUCKET AUTOCREATION	1781
99.10. AUTOMATIC DETECTION OF MINIO CLIENT IN REGISTRY	1781
99.11. MOVING STUFF BETWEEN A BUCKET AND ANOTHER BUCKET	1781
99.12. MOVEAFTERREAD CONSUMER OPTION	1782
99.13. USING A POJO AS BODY	1782
99.14. SPRING BOOT AUTO-CONFIGURATION	1782
CHAPTER 100. MLLP	1788
100.1. DEPENDENCIES	1788
100.2. CONFIGURING OPTIONS	1788
100.2.1. Configuring Component Options	1788
100.2.2. Configuring Endpoint Options	1789
100.3. COMPONENT OPTIONS	1789
100.4. ENDPOINT OPTIONS	1793
100.4.1. Path Parameters (2 parameters)	1793
100.4.2. Query Parameters (26 parameters)	1793
100.5. MLLP CONSUMER	1796
100.5.1. Message Headers	1797
100.5.2. Exchange Properties	1797
100.6. MLLP PRODUCER	1798
100.6.1. Message Headers	1798
100.6.2. Exchange Properties	1799
100.7. SPRING BOOT AUTO-CONFIGURATION	1799
CHAPTER 101. MOCK	1804
101.1. DEPENDENCIES	1804
101.2. URI FORMAT	1805
101.3. CONFIGURING OPTIONS	1805
101.3.1. Configuring Component Options	1805
101.3.2. Configuring Endpoint Options	1805
101.4. COMPONENT OPTIONS	1805
101.5. ENDPOINT OPTIONS	1806
101.5.1. Path Parameters (1 parameters)	1806
101.5.2. Query Parameters (12 parameters)	1807
101.6. SIMPLE EXAMPLE	1809
101.7. USING ASSERTPERIOD	1809
101.8. SETTING EXPECTATIONS	1810
101.9. ADDING EXPECTATIONS TO SPECIFIC MESSAGES	1811
101.10. MOCKING EXISTING ENDPOINTS	1811
101.11. MOCKING EXISTING ENDPOINTS USING THE CAMEL-TEST COMPONENT	1813
101.12. MOCKING EXISTING ENDPOINTS WITH XML DSL	1814
101.13. MOCKING ENDPOINTS AND SKIP SENDING TO ORIGINAL ENDPOINT	1815
101.14. LIMITING THE NUMBER OF MESSAGES TO KEEP	1816
101.15. TESTING WITH ARRIVAL TIMES	1817
101.16. SPRING BOOT AUTO-CONFIGURATION	1818
CHAPTER 102. MONGODB	1819
102.1. DEPENDENCIES	1819
102.2. URI FORMAT	1819
102.3. CONFIGURING OPTIONS	1819
102.3.1. Configuring Component Options	1819
102.3.2. Configuring Endpoint Options	1820
102.4. COMPONENT OPTIONS	1820
102.5. ENDPOINT OPTIONS	1821

102.5.1. Path Parameters (1 parameters)	1821
102.5.2. Query Parameters (27 parameters)	1822
102.6. CONFIGURATION OF DATABASE IN SPRING XML	1827
102.7. SAMPLE ROUTE	1828
102.8. MONGODB OPERATIONS - PRODUCER ENDPOINTS	1828
102.8.1. Query operations	1828
102.8.1.1. findById	1828
102.8.1.2. findOneByQuery	1828
102.8.1.3. Example without a query selector (returns the first document in a collection)	1829
102.8.1.4. Example with a query selector (returns the first matching document in a collection):	1829
102.8.1.5. findAll	1829
102.8.1.5.1. Example without a query selector (returns all documents in a collection)	1829
102.8.1.5.2. Example with a query selector (returns all matching documents in a collection)	1829
102.8.1.5.3. Example with option outputType=Mongolterable and batch size	1830
102.8.1.6. count	1831
102.8.1.7. Specifying a fields filter (projection)	1831
102.8.1.8. Specifying a sort clause	1832
102.8.2. Create/update operations	1832
102.8.2.1. insert	1833
102.8.2.2. save	1833
102.8.2.3. update	1834
102.8.3. Delete operations	1835
102.8.3.1. remove	1835
102.8.4. Bulk Write Operations	1836
102.8.4.1. bulkWrite	1836
102.8.5. Other operations	1836
102.8.5.1. aggregate	1836
102.8.5.2. getDbStats	1837
102.8.5.3. getColStats	1838
102.8.5.4. command	1839
102.8.6. Dynamic operations	1839
102.9. CONSUMERS	1839
102.9.1. Tailable Cursor Consumer	1839
102.10. HOW THE TAILABLE CURSOR CONSUMER WORKS	1840
102.11. PERSISTENT TAIL TRACKING	1840
102.12. ENABLING PERSISTENT TAIL TRACKING	1841
102.12.1. Change Streams Consumer	1841
102.13. TYPE CONVERSIONS	1842
102.14. SPRING BOOT AUTO-CONFIGURATION	1843
CHAPTER 103. MYBATIS	1845
103.1. DEPENDENCIES	1845
103.2. URI FORMAT	1845
103.3. CONFIGURING OPTIONS	1845
103.3.1. Configuring Component Options	1845
103.3.2. Configuring Endpoint Options	1846
103.4. COMPONENT OPTIONS	1846
103.5. ENDPOINT OPTIONS	1847
103.5.1. Path Parameters (1 parameters)	1848
103.5.2. Query Parameters (30 parameters)	1848
103.6. MESSAGE HEADERS	1855
103.7. MESSAGE BODY	1855
103.8. SAMPLES	1855

103.9. USING STATEMENTTYPE FOR BETTER CONTROL OF MYBATIS	1856
103.9.1. Using InsertList StatementType	1856
103.9.2. Using UpdateList StatementType	1856
103.9.3. Using DeleteList StatementType	1857
103.9.4. Notice on InsertList, UpdateList and DeleteList StatementTypes	1857
103.9.5. scheduled polling example	1857
103.9.6. Using onConsume	1858
103.9.7. Participating in transactions	1858
103.10. MYBATIS SPRING BOOT STARTER INTEGRATION	1859
103.11. SPRING BOOT AUTO-CONFIGURATION	1859
CHAPTER 104. NETTY	1864
104.1. DEPENDENCIES	1864
104.2. URI FORMAT	1864
104.3. CONFIGURING OPTIONS	1864
104.3.1. Configuring Component Options	1864
104.3.2. Configuring Endpoint Options	1865
104.4. COMPONENT OPTIONS	1865
104.5. ENDPOINT OPTIONS	1874
104.5.1. Path Parameters (3 parameters)	1874
104.5.2. Query Parameters (71 parameters)	1875
104.6. REGISTRY BASED OPTIONS	1884
104.6.1. Using non shareable encoders or decoders	1885
104.7. SENDING MESSAGES TO/FROM A NETTY ENDPOINT	1885
104.7.1. Netty Producer	1885
104.7.2. Netty Consumer	1885
104.8. EXAMPLES	1885
104.8.1. A UDP Netty endpoint using Request-Reply and serialized object payload	1885
104.8.2. A TCP based Netty consumer endpoint using One-way communication	1886
104.8.3. An SSL/TCP based Netty consumer endpoint using Request-Reply communication	1886
104.8.4. Using Multiple Codecs	1888
104.9. CLOSING CHANNEL WHEN COMPLETE	1890
104.10. CUSTOM PIPELINE	1890
104.10.1. Using custom pipeline factory	1890
104.11. REUSING NETTY BOSS AND WORKER THREAD POOLS	1891
104.12. MULTIPLEXING CONCURRENT MESSAGES OVER A SINGLE CONNECTION WITH REQUEST/REPLY	1892
104.13. SPRING BOOT AUTO-CONFIGURATION	1893
CHAPTER 105. NETTY HTTP	1903
105.1. DEPENDENCIES	1903
105.2. URI FORMAT	1903
105.3. CONFIGURING OPTIONS	1904
105.3.1. Configuring component options	1904
105.3.2. Configuring endpoint options	1904
105.4. COMPONENT OPTIONS	1904
105.5. ENDPOINT OPTIONS	1914
105.5.1. Path Parameters (4 parameters)	1914
105.5.2. Query Parameters (85 parameters)	1915
105.6. MESSAGE HEADERS	1926
105.7. ACCESS TO NETTY TYPES	1930
105.8. EXAMPLES	1930
105.8.1. How do I let Netty match wildcards	1930

105.8.2. Using multiple routes with same port	1931
105.8.3. Reusing same server bootstrap configuration with multiple routes	1932
105.8.4. Reusing same server bootstrap configuration with multiple routes across multiple bundles in OSGi container	1932
105.8.5. Implementing a reverse proxy	1932
105.9. USING HTTP BASIC AUTHENTICATION	1933
105.9.1. Specifying ACL on web resources	1933
105.10. SPRING BOOT AUTO-CONFIGURATION	1934
CHAPTER 106. OBSERVABILITY SERVICES	1944
106.1. DEPENDENCIES	1944
106.2. USAGE	1944
106.2.1. Auto-detection from classpath	1944
106.3. COMPONENTS AVAILABLE	1944
106.3.1. List of known endpoints	1944
106.4. OPENTELEMETRY CONFIGURATION	1945
106.4.1. Testing OpenTelemetry collector	1945
106.4.2. OpenTelemetry Agent configuration	1945
106.5. JMX CONFIGURATION	1945
106.5.1. Testing JMX	1945
106.5.2. JMX Agent configuration	1946
CHAPTER 107. OLINGO4	1947
107.1. DEPENDENCIES	1947
107.2. URI FORMAT	1947
107.3. CONFIGURING OPTIONS	1947
107.3.1. Configuring Component Options	1947
107.3.2. Configuring Endpoint Options	1948
107.4. COMPONENT OPTIONS	1948
107.5. ENDPOINT OPTIONS	1950
107.5.1. Path Parameters (2 parameters)	1950
107.5.2. Query Parameters (32 parameters)	1950
107.6. API PARAMETERS (1 APIS)	1954
107.6.1. API: DEFAULT	1954
107.6.1.1. Method action	1955
107.6.1.2. Method batch	1956
107.6.1.3. Method create	1956
107.6.1.4. Method delete	1957
107.6.1.5. Method merge	1957
107.6.1.6. Method patch	1958
107.6.1.7. Method read	1958
107.6.1.8. Method update	1959
107.6.1.9. Method uread	1959
107.7. MESSAGE HEADERS	1960
107.8. ENDPOINT HTTP HEADERS	1960
107.9. ODATA RESOURCE TYPE MAPPING	1961
107.10. SAMPLES	1961
107.11. SPRING BOOT AUTO-CONFIGURATION	1962
CHAPTER 108. OPENAPI JAVA	1965
108.1. DEPENDENCIES	1965
108.2. USING OPENAPI IN REST-DSL	1965
108.3. OPTIONS	1966
108.4. ADDING SECURITY DEFINITIONS IN API DOC	1967

108.5. JSON OR YAML	1967
108.6. USEXFORWARDHEADERS AND API URL RESOLUTION	1967
108.7. EXAMPLES	1968
108.8. SPRING BOOT AUTO-CONFIGURATION	1968
CHAPTER 109. OPENSEARCH	1969
109.1. DEPENDENCIES	1969
109.2. URI FORMAT	1969
109.3. CONFIGURING OPTIONS	1969
109.3.1. Component Level Options	1969
109.3.2. Endpoint Level Options	1969
109.4. COMPONENT OPTIONS	1970
109.5. ENDPOINT OPTIONS	1971
109.5.1. Path Parameters (1 parameters)	1972
109.5.2. Query Parameters (19 parameters)	1972
109.6. MESSAGE HEADERS	1974
109.7. USAGE	1976
109.7.1. Message Operations	1976
109.7.2. Configure the component and enable basic authentication	1978
109.7.3. Document type	1979
109.8. EXAMPLES	1979
109.8.1. Index Example	1979
109.8.2. Search Example	1979
109.8.3. MultiSearch Example	1980
CHAPTER 110. OPENTELEMETRY	1982
110.1. DEPENDENCIES	1982
110.2. CONFIGURATION PROPERTIES	1982
110.2.1. Using Camel OpenTelemetry	1982
110.3. SPRING BOOT	1983
110.4. USING TRACING STRATEGY	1983
110.5. JAVA AGENT	1983
110.6. ADDING THE ROUTEID ATTRIBUTE TO THE SPAN	1984
110.7. SPRING BOOT AUTO-CONFIGURATION	1984
110.8. MDC LOGGING	1985
CHAPTER 111. PAHO	1986
111.1. DEPENDENCIES	1986
111.2. URI FORMAT	1986
111.3. CONFIGURING OPTIONS	1986
111.3.1. Configuring Component Options	1986
111.3.2. Configuring Endpoint Options	1986
111.4. COMPONENT OPTIONS	1987
111.5. ENDPOINT OPTIONS	1993
111.5.1. Path Parameters (1 parameters)	1993
111.5.2. Query Parameters (31 parameters)	1994
111.6. HEADERS	1999
111.7. DEFAULT PAYLOAD TYPE	2000
111.8. SAMPLES	2000
111.9. SPRING BOOT AUTO-CONFIGURATION	2001
CHAPTER 112. PAHO MQTT 5	2008
112.1. DEPENDENCIES	2008
112.2. URI FORMAT	2008

112.3. CONFIGURING OPTIONS	2008
112.3.1. Configuring Component Options	2008
112.3.2. Configuring Endpoint Options	2008
112.4. COMPONENT OPTIONS	2009
112.5. ENDPOINT OPTIONS	2015
112.5.1. Path Parameters (1 parameters)	2016
112.5.2. Query Parameters (32 parameters)	2016
112.6. HEADERS	2022
112.7. DEFAULT PAYLOAD TYPE	2022
112.8. SAMPLES	2023
112.9. SPRING BOOT AUTO-CONFIGURATION	2023
CHAPTER 113. PLATFORM HTTP	2031
113.1. DEPENDENCIES	2031
113.2. PLATFORM HTTP PROVIDER	2031
113.3. CONFIGURING OPTIONS	2031
113.3.1. Configuring Component Options	2031
113.3.2. Configuring Endpoint Options	2032
113.4. COMPONENT OPTIONS	2032
113.4.1. Endpoint Options	2033
113.4.1.1. Path Parameters (1 parameters)	2033
113.4.1.2. Query Parameters (21 parameters)	2033
113.5. IMPLEMENTING A REVERSE PROXY	2036
113.6. SPRING BOOT AUTO-CONFIGURATION	2036
CHAPTER 114. PROTOBUF JACKSON	2038
114.1. DEPENDENCIES	2038
114.2. CONFIGURING THE SCHEMARESOLVER	2038
114.3. PROTOBUF JACKSON OPTIONS	2038
114.4. USING CUSTOM PROTOBUFMAPPER	2040
114.5. SPRING BOOT AUTO-CONFIGURATION	2040
CHAPTER 115. QUARTZ	2044
115.1. DEPENDENCIES	2044
115.2. URI FORMAT	2044
115.3. CONFIGURING OPTIONS	2044
115.3.1. Configuring Component Options	2044
115.3.2. Configuring Endpoint Options	2045
115.4. COMPONENT OPTIONS	2045
115.5. ENDPOINT OPTIONS	2046
115.5.1. Path Parameters (2 parameters)	2047
115.5.2. Query Parameters (17 parameters)	2047
115.5.3. Configuring quartz.properties file	2049
115.6. ENABLING QUARTZ SCHEDULER IN JMX	2049
115.7. STARTING THE QUARTZ SCHEDULER	2049
115.8. CLUSTERING	2050
115.9. MESSAGE HEADERS	2050
115.10. USING CRON TRIGGERS	2050
115.11. SPECIFYING TIME ZONE	2050
115.12. CONFIGURING MISFIRE INSTRUCTIONS	2051
115.12.1. SimpleTrigger.MISFIRE_INSTRUCTION_FIRE_NOW = 1 (default)	2051
115.12.2. SimpleTrigger.MISFIRE_INSTRUCTION_RESCHEDULE_NOW_WITH_EXISTING_REPEAT_COUNT = 2	2051
115.12.3. SimpleTrigger.MISFIRE_INSTRUCTION_RESCHEDULE_NOW_WITH_REMAINING_REPEAT_COUNT =	

3	2051
115.12.4. SimpleTrigger.MISFIRE_INSTRUCTION_RESCHEDULE_NEXT_WITH_REMAINING_COUNT = 4	2052
115.12.5. SimpleTrigger.MISFIRE_INSTRUCTION_RESCHEDULE_NEXT_WITH_EXISTING_COUNT = 5	2052
115.12.6. CronTrigger.MISFIRE_INSTRUCTION_FIRE_ONCE_NOW = 1 (default)	2052
115.12.7. CronTrigger.MISFIRE_INSTRUCTION_DO_NOTHING = 2	2052
115.13. USING QUARTZSCHEDULEDPOLLCONSUMERSCHEDULER	2052
115.14. CRON COMPONENT SUPPORT	2053
115.15. SPRING BOOT AUTO-CONFIGURATION	2054
CHAPTER 116. REF	2056
116.1. DEPENDENCIES	2056
116.2. URI FORMAT	2056
116.3. CONFIGURING OPTIONS	2056
116.3.1. Configuring Component Options	2056
116.3.2. Configuring Endpoint Options	2056
116.4. COMPONENT OPTIONS	2057
116.5. ENDPOINT OPTIONS	2058
116.5.1. Path Parameters (1 parameters)	2058
116.5.2. Query Parameters (4 parameters)	2058
116.6. RUNTIME LOOKUP	2059
116.7. SAMPLE	2059
116.8. SPRING BOOT AUTO-CONFIGURATION	2060
CHAPTER 117. REF	2061
117.1. DEPENDENCIES	2061
117.2. REF LANGUAGE OPTIONS	2061
117.3. EXAMPLE USAGE	2061
117.4. SPRING BOOT AUTO-CONFIGURATION	2061
CHAPTER 118. REST	2079
118.1. DEPENDENCIES	2079
118.2. URI FORMAT	2079
118.3. CONFIGURING OPTIONS	2079
118.3.1. Configuring Component Options	2079
118.3.2. Configuring Endpoint Options	2079
118.4. COMPONENT OPTIONS	2080
118.5. ENDPOINT OPTIONS	2081
118.5.1. Path Parameters (3 parameters)	2082
118.5.2. Query Parameters (16 parameters)	2082
118.6. SUPPORTED REST COMPONENTS	2084
118.7. PATH AND URITEMPLATE SYNTAX	2085
118.8. REST PRODUCER EXAMPLES	2085
118.9. REST PRODUCER BINDING	2086
118.10. MORE EXAMPLES	2087
118.11. SPRING BOOT AUTO-CONFIGURATION	2087
CHAPTER 119. SAGA	2090
119.1. DEPENDENCIES	2090
119.2. URI FORMAT	2090
119.3. CONFIGURING OPTIONS	2090
119.3.1. Configuring Component Options	2090
119.3.2. Configuring Endpoint Options	2090
119.4. COMPONENT OPTIONS	2091
119.5. ENDPOINT OPTIONS	2091

119.5.1. Path Parameters (1 parameters)	2092
119.5.2. Query Parameters (1 parameters)	2092
119.6. USING CAMEL-SAGA WITH SPRING BOOT AND LRA COORDINATOR	2092
119.7. SPRING BOOT AUTO-CONFIGURATION	2092
CHAPTER 120. SALESFORCE	2094
120.1. DEPENDENCIES	2094
120.2. CONFIGURING OPTIONS	2095
120.2.1. Configuring Component Options	2095
120.2.2. Configuring Endpoint Options	2095
120.3. COMPONENT OPTIONS	2095
120.4. ENDPOINT OPTIONS	2105
120.4.1. Path Parameters (2 parameters)	2105
120.4.2. Query Parameters (57 parameters)	2107
120.5. AUTHENTICATING TO SALESFORCE	2113
120.6. URI FORMAT	2114
120.7. PASSING IN SALESFORCE HEADERS AND FETCHING SALESFORCE RESPONSE HEADERS	2114
120.8. SUPPORTED SALESFORCE APIS	2115
120.8.1. Rest API	2115
120.8.2. Bulk 2.0 API	2116
120.8.3. Rest Bulk (original) API	2117
120.8.4. Rest Streaming API	2119
120.8.5. Platform events	2119
120.8.6. Change data capture events	2120
120.9. EXAMPLES	2120
120.9.1. Uploading a document to a ContentWorkspace	2120
120.10. USING SALESFORCE LIMITS API	2121
120.11. WORKING WITH APPROVALS	2121
120.12. USING SALESFORCE RECENT ITEMS API	2122
120.13. USING SALESFORCE COMPOSITE API TO SUBMIT SUBJECT TREE	2122
120.14. USING SALESFORCE COMPOSITE API TO SUBMIT MULTIPLE REQUESTS IN A BATCH	2123
120.15. USING SALESFORCE COMPOSITE API TO SUBMIT MULTIPLE CHAINED REQUESTS	2124
120.16. USING "RAW" SALESFORCE COMPOSITE	2125
120.17. USING RAW OPERATION	2126
120.17.1. Query example	2127
120.17.2. SObject example	2127
120.18. USING COMPOSITE SUBJECT COLLECTIONS	2127
120.18.1. compositeRetrieveObjectCollections	2127
120.18.2. compositeCreateObjectCollections	2128
120.18.3. compositeUpdateObjectCollections	2129
120.18.4. compositeUpsertObjectCollections	2129
120.18.5. compositeDeleteObjectCollections	2129
120.19. SENDING NULL VALUES TO SALESFORCE	2130
120.20. GENERATING SOQL QUERY STRINGS	2130
120.21. CAMEL SALESFORCE MAVEN PLUGIN	2130
120.22. SPRING BOOT AUTO-CONFIGURATION	2131
CHAPTER 121. SAP COMPONENT	2141
121.1. DEPENDENCIES	2141
121.1.1. Additional platform restrictions for the SAP component	2141
121.1.2. SAP JCo and SAP IDoc libraries	2141
121.1.2.1. Installing SAP JCo and SAP IDoc libraries	2141
121.2. URI FORMAT	2144

121.2.1. Options for RFC destination endpoints	2145
121.2.2. Options for RFC server endpoints	2146
121.2.3. Options for the IDoc List Server endpoint	2146
121.2.4. Summary of the RFC and IDoc endpoints	2146
121.2.5. SAP RFC destination endpoint	2148
121.2.6. SAP RFC server endpoint	2148
121.2.7. SAP IDoc and IDoc list destination endpoints	2149
121.2.8. SAP IDoc list server endpoint	2149
121.2.9. Metadata repositories	2149
121.3. CONFIGURATION	2149
121.3.1. Configuration Overview	2150
121.3.2. Destination Configuration	2151
121.3.2.1. Interceptor for tRFC and qRFC destinations	2152
121.3.2.2. Log on and authentication options	2152
121.3.2.3. Connection options	2153
121.3.2.4. Connection pool options	2155
121.3.2.5. Secure network connection options	2156
121.3.2.6. Repository options	2157
121.3.2.7. Trace configuration options	2158
121.3.3. Server Configuration	2158
121.3.3.1. Required options	2160
121.3.3.2. Secure network connection options	2160
121.3.3.3. Other options	2161
121.3.4. Repository Configuration	2162
121.3.4.1. Function template properties	2162
121.3.4.2. List field metadata properties	2164
121.3.4.3. Record metadata properties	2166
121.3.4.4. Record field metadata properties	2167
121.4. MESSAGE HEADERS	2169
121.5. EXCHANGE PROPERTIES	2170
121.6. MESSAGE BODY FOR RFC	2170
121.6.1. Request and response objects	2170
121.6.2. Structure objects	2171
121.6.3. Field types	2171
121.6.3.1. Elementary field types	2171
121.6.3.2. Character field types	2173
121.6.3.3. Numeric field types	2173
121.6.3.4. Hexadecimal field types	2174
121.6.3.5. String field types	2175
121.6.3.6. Complex field types	2175
121.6.3.7. Structure field types	2175
121.6.3.8. Table field types	2175
121.6.3.9. Table objects	2175
121.7. MESSAGE BODY FOR IDOC	2176
121.7.1. IDoc message type	2176
121.7.2. The IDoc document model	2176
121.7.3. How an IDoc is related to a Document object	2179
121.8. DOCUMENT ATTRIBUTES	2180
121.8.1. Setting document attributes in Java	2182
121.8.2. Setting document attributes in XML	2182
121.9. TRANSACTION SUPPORT	2182
121.9.1. BAPI transaction model	2183
121.9.2. RFC transaction model	2183

121.9.3. Which transaction model to use?	2183
121.9.4. Transactional RFC destination endpoints	2183
121.9.5. Transactional RFC server endpoints	2184
121.10. XML SERIALIZATION FOR RFC	2184
121.10.1. XML namespace	2184
121.10.2. Request and response XML documents	2184
121.10.3. Structure fields	2184
121.10.4. Table fields	2185
121.10.5. Elementary fields	2186
121.10.6. Date and time formats	2186
121.11. XML SERIALIZATION FOR IDOC	2186
121.11.1. XML namespace	2187
121.11.2. Built-in type converter	2187
121.11.3. Sample IDoc message body in XML format	2187
121.12. EXAMPLE 1: READING DATA FROM SAP	2188
121.12.1. Java DSL for route	2188
121.12.2. XML DSL for route	2188
121.12.3. createFlightCustomerGetListRequest bean	2188
121.12.4. returnFlightCustomerInfo bean	2189
121.13. EXAMPLE 2: WRITING DATA TO SAP	2190
121.13.1. Java DSL for route	2190
121.13.2. XML DSL for route	2190
121.13.3. Transaction support	2191
121.13.4. Populating request parameters	2191
121.14. EXAMPLE 3: HANDLING REQUESTS FROM SAP	2191
121.14.1. Java DSL for route	2191
121.14.2. XML DSL for route	2191
121.14.3. BookFlightRequest bean	2192
121.14.4. BookFlightResponse bean	2193
121.14.5. FlightInfo bean	2193
121.14.6. ConnectionInfoTable bean	2194
121.14.7. ConnectionInfo bean	2194
CHAPTER 122. XQUERY	2196
122.1. DEPENDENCIES	2196
122.2. XQUERY LANGUAGE OPTIONS	2196
122.3. VARIABLES	2196
122.4. EXAMPLE	2197
122.4.1. Using namespaces	2197
122.5. USING XQUERY AS TRANSFORMATION	2198
122.6. LOADING SCRIPT FROM EXTERNAL RESOURCE	2199
122.7. LEARNING XQUERY	2199
122.8. SPRING BOOT AUTO-CONFIGURATION	2199
CHAPTER 123. SCHEDULER	2202
123.1. DEPENDENCIES	2202
123.2. URI FORMAT	2202
123.3. CONFIGURING OPTIONS	2202
123.3.1. Configuring Component Options	2202
123.3.2. Configuring Endpoint Options	2203
123.4. COMPONENT OPTIONS	2203
123.5. ENDPOINT OPTIONS	2204
123.5.1. Path Parameters (1 parameters)	2204

123.5.2. Query Parameters (21 parameters)	2204
123.6. MORE INFORMATION	2207
123.7. EXCHANGE PROPERTIES	2207
123.8. SAMPLE	2207
123.9. FORCING THE SCHEDULER TO TRIGGER IMMEDIATELY WHEN COMPLETED	2208
123.10. FORCING THE SCHEDULER TO BE IDLE	2208
123.11. SPRING BOOT AUTO-CONFIGURATION	2208
CHAPTER 124. SEDA	2210
124.1. DEPENDENCIES	2210
124.2. URI FORMAT	2210
124.3. CONFIGURING OPTIONS	2210
124.3.1. Configuring Component Options	2210
124.3.2. Configuring Endpoint Options	2211
124.4. COMPONENT OPTIONS	2211
124.5. ENDPOINT OPTIONS	2213
124.5.1. Path Parameters (1 parameters)	2213
124.5.2. Query Parameters (18 parameters)	2213
124.6. CHOOSING BLOCKINGQUEUE IMPLEMENTATION	2216
124.7. USE OF REQUEST REPLY	2216
124.8. CONCURRENT CONSUMERS	2217
124.9. THREAD POOLS	2217
124.10. SAMPLE	2217
124.11. USING MULTIPLECONSUMERS	2218
124.12. EXTRACTING QUEUE INFORMATION	2219
124.13. SPRING BOOT AUTO-CONFIGURATION	2219
CHAPTER 125. SERVLET	2221
125.1. DEPENDENCIES	2221
125.2. URI FORMAT	2221
125.3. CONFIGURING OPTIONS	2221
125.3.1. Configuring Component Options	2221
125.3.2. Configuring Endpoint Options	2222
125.4. COMPONENT OPTIONS	2222
125.5. ENDPOINT OPTIONS	2223
125.5.1. Path Parameters (1 parameters)	2224
125.5.2. Query Parameters (22 parameters)	2224
125.6. MESSAGE HEADERS	2227
125.7. USAGE	2227
125.8. SPRING BOOT AUTO-CONFIGURATION	2227
CHAPTER 126. SIMPLE	2230
126.1. DEPENDENCIES	2230
126.2. SIMPLE LANGUAGE OPTIONS	2230
126.3. VARIABLES	2231
126.4. OGNL EXPRESSION SUPPORT	2234
126.5. OPERATOR SUPPORT	2236
126.5.1. Comparing with different types	2239
126.5.2. Using and / or	2240
126.6. EXAMPLES	2240
126.7. SETTING RESULT TYPE	2242
126.8. USING NEW LINES OR TABS IN XML DSLS	2242
126.9. LEADING AND TRAILING WHITESPACE HANDLING	2243
126.10. LOADING SCRIPT FROM EXTERNAL RESOURCE	2243

126.11. SPRING BOOT AUTO-CONFIGURATION	2243
CHAPTER 127. SLACK	2261
127.1. DEPENDENCIES	2261
127.2. URI FORMAT	2261
127.3. CONFIGURING OPTIONS	2261
127.3.1. Configuring Component Options	2261
127.3.2. Configuring Endpoint Options	2262
127.4. COMPONENT OPTIONS	2262
127.5. ENDPOINT OPTIONS	2263
127.5.1. Path Parameters (1 parameters)	2263
127.5.2. Query Parameters (29 parameters)	2263
127.6. CONFIGURING IN SPRINT XML	2267
127.7. EXAMPLE	2267
127.8. PRODUCER	2268
127.9. CONSUMER	2268
127.10. SPRING BOOT AUTO-CONFIGURATION	2269
CHAPTER 128. SMB	2271
128.1. DEPENDENCIES	2271
128.2. URI FORMAT	2271
128.3. CONFIGURING OPTIONS	2271
128.3.1. Configuring Component Options	2271
128.3.2. Configuring Endpoint Options	2271
128.4. COMPONENT OPTIONS	2272
128.5. ENDPOINT OPTIONS	2272
128.5.1. Path Parameters (3 parameters)	2273
128.6. QUERY PARAMETERS (26 PARAMETERS)	2273
128.7. EXAMPLES	2276
CHAPTER 129. SMOOKS COMPONENT	2278
129.1. DEPENDENCIES	2278
129.2. URI FORMAT	2278
129.3. CONFIGURING OPTIONS	2278
129.3.1. Configuring component options	2278
129.3.2. Configuring endpoint options	2279
129.4. COMPONENT OPTIONS	2279
129.5. ENDPOINT OPTIONS	2280
129.5.1. Path Parameters (1 parameters)	2280
129.5.2. Query Parameters (3 parameters)	2280
129.6. MESSAGE HEADERS	2281
129.7. USAGE	2281
129.7.1. Bean routing	2282
CHAPTER 130. SMOOKS DATA FORMAT	2284
130.1. DEPENDENCIES	2284
130.2. USAGE	2284
130.3. SMOOKS DATA FORMAT OPTIONS	2285
130.4. SPRING BOOT AUTO-CONFIGURATION	2285
CHAPTER 131. SNMP	2287
131.1. DEPENDENCIES	2287
131.2. URI FORMAT	2287
131.3. SNMP PRODUCER	2287

131.4. CONFIGURING OPTIONS	2287
131.4.1. Configuring component options	2287
131.4.2. Configuring endpoint options	2287
131.5. COMPONENT OPTIONS	2288
131.6. ENDPOINT OPTIONS	2289
131.6.1. Path Parameters (2 parameters)	2289
131.6.2. Query Parameters (36 parameters)	2289
131.7. THE RESULT OF A POLL	2295
131.8. EXAMPLES	2296
131.9. SPRING BOOT AUTO-CONFIGURATION	2296
CHAPTER 132. SOAP	2298
132.1. DEPENDENCIES	2298
132.2. SOAP OPTIONS	2298
132.3. ELEMENTNAMESTRATEGY	2299
132.4. USING THE JAVA DSL	2299
132.4.1. Using SOAP 1.2	2300
132.5. MULTI-PART MESSAGES	2300
132.5.1. Holder Object mapping	2301
132.6. EXAMPLES	2301
132.6.1. Webservice client	2301
132.6.2. Webservice Server	2301
132.7. SPRING BOOT AUTO-CONFIGURATION	2302
CHAPTER 133. SPLUNK	2304
133.1. DEPENDENCIES	2304
133.2. URI FORMAT	2304
133.3. CONFIGURING OPTIONS	2304
133.3.1. Configuring Component Options	2304
133.3.2. Configuring Endpoint Options	2304
133.4. COMPONENT OPTIONS	2305
133.5. ENDPOINT OPTIONS	2306
133.5.1. Path Parameters (1 parameters)	2306
133.5.2. Query Parameters (45 parameters)	2306
133.6. PRODUCER ENDPOINTS	2311
133.7. CONSUMER ENDPOINTS:	2311
133.8. MESSAGE BODY	2312
133.9. USE CASES	2312
133.10. OTHER COMMENTS	2313
133.11. SPRING BOOT AUTO-CONFIGURATION	2313
CHAPTER 134. SPRING BATCH	2315
134.1. DEPENDENCIES	2315
134.2. URI FORMAT	2315
134.3. CONFIGURING OPTIONS	2315
134.3.1. Configuring Component Options	2315
134.3.2. Configuring Endpoint Options	2316
134.4. COMPONENT OPTIONS	2316
134.5. ENDPOINT OPTIONS	2317
134.5.1. Path Parameters (1 parameters)	2318
134.5.2. Query Parameters (4 parameters)	2318
134.6. USAGE	2319
134.7. EXAMPLES	2319
134.8. SUPPORT CLASSES	2319

134.8.1. CamelItemReader	2319
134.8.2. CamelItemWriter	2320
134.8.3. CamelItemProcessor	2320
134.8.4. CamelJobExecutionListener	2321
134.9. SPRING BOOT AUTO-CONFIGURATION	2321
CHAPTER 135. SPRING JDBC	2324
135.1. DEPENDENCIES	2324
135.2. CONFIGURING OPTIONS	2324
135.2.1. Configuring Component Options	2324
135.2.2. Configuring Endpoint Options	2325
135.3. COMPONENT OPTIONS	2325
135.4. ENDPOINT OPTIONS	2327
135.4.1. Path Parameters (1 parameters)	2327
135.4.2. Query Parameters (14 parameters)	2327
135.5. SPRING BOOT AUTO-CONFIGURATION	2331
CHAPTER 136. SPRING LDAP	2334
136.1. DEPENDENCIES	2334
136.2. URI FORMAT	2334
136.3. CONFIGURING OPTIONS	2334
136.3.1. Configuring Component Options	2334
136.3.2. Configuring Endpoint Options	2334
136.4. COMPONENT OPTIONS	2335
136.5. ENDPOINT OPTIONS	2336
136.5.1. Path Parameters (1 parameters)	2336
136.5.2. Query Parameters (3 parameters)	2336
136.6. USAGE	2338
136.6.1. Search	2338
136.6.2. Bind	2338
136.6.3. Unbind	2339
136.6.4. Authenticate	2339
136.6.5. Modify Attributes	2339
136.6.6. Function-Driven	2339
136.7. SPRING BOOT AUTO-CONFIGURATION	2340
CHAPTER 137. SPRING RABBITMQ	2342
137.1. DEPENDENCIES	2342
137.2. URI FORMAT	2342
137.3. CONFIGURING OPTIONS	2342
137.3.1. Configuring Component Options	2342
137.3.2. Configuring Endpoint Options	2343
137.4. COMPONENT OPTIONS	2343
137.5. ENDPOINT OPTIONS	2349
137.5.1. Path Parameters (1 parameters)	2350
137.5.2. Query Parameters (34 parameters)	2350
137.6. MESSAGE HEADERS	2358
137.7. USING A CONNECTION FACTORY	2359
137.8. DEFAULT EXCHANGE NAME	2359
137.9. AUTO DECLARE EXCHANGES, QUEUES AND BINDINGS	2359
137.10. MAPPING FROM CAMEL TO RABBITMQ	2361
137.11. REQUEST / REPLY	2361
137.12. REUSE ENDPOINT AND SEND TO DIFFERENT DESTINATIONS COMPUTED AT RUNTIME	2362
137.13. USING TOD	2362

137.14. SPRING BOOT AUTO-CONFIGURATION	2363
CHAPTER 138. SPRING REDIS	2370
138.1. DEPENDENCIES	2370
138.2. URI FORMAT	2370
138.3. CONFIGURING OPTIONS	2370
138.3.1. Configuring Component Options	2370
138.3.2. Configuring Endpoint Options	2371
138.4. COMPONENT OPTIONS	2371
138.5. ENDPOINT OPTIONS	2372
138.5.1. Path Parameters (2 parameters)	2372
138.5.2. Query Parameters (10 parameters)	2372
138.6. MESSAGE HEADERS	2378
138.7. USAGE	2381
138.7.1. Message headers evaluated by the Redis producer	2382
138.8. SPRING BOOT AUTO-CONFIGURATION	2394
CHAPTER 139. SPRING SECURITY	2396
139.1. DEPENDENCIES	2396
139.2. CREATING AUTHORIZATION POLICIES	2396
139.3. CONTROLLING ACCESS TO CAMEL ROUTES	2397
139.4. AUTHENTICATION	2399
139.5. HANDLING AUTHENTICATION AND AUTHORIZATION ERRORS	2400
139.6. SPRING BOOT AUTO-CONFIGURATION	2400
CHAPTER 140. SPRING WEBSERVICE	2401
140.1. DEPENDENCIES	2401
140.2. URI FORMAT	2401
140.3. CONFIGURING OPTIONS	2402
140.3.1. Configuring Component Options	2402
140.3.2. Configuring Endpoint Options	2402
140.4. COMPONENT OPTIONS	2403
140.5. ENDPOINT OPTIONS	2404
140.5.1. Path Parameters (4 parameters)	2405
140.5.2. Query Parameters (21 parameters)	2406
140.6. MESSAGE HEADERS	2412
140.7. ACCESSING WEB SERVICES	2414
140.8. SENDING SOAP AND WS-ADDRESSING ACTION HEADERS	2414
140.9. USING SOAP HEADERS	2414
140.10. THE HEADER AND ATTACHMENT PROPAGATION	2415
140.11. HOW TO TRANSFORM THE SOAP HEADER USING A STYLESHEET	2415
140.12. HOW TO USE MTOM ATTACHMENTS	2416
140.13. THE CUSTOM HEADER AND ATTACHMENT FILTERING	2416
140.14. USING A CUSTOM MESSAGESENDER AND MESSAGEFACTORY	2417
140.15. EXPOSING WEB SERVICES	2418
140.16. ENDPOINT MAPPING IN ROUTES	2418
140.16.1. Alternative configuration, using existing endpoint mappings	2419
140.17. POJO (UN)MARSHALLING	2420
140.18. SPRING BOOT AUTO-CONFIGURATION	2420
CHAPTER 141. SQL	2423
141.1. DEPENDENCIES	2423
141.2. URI FORMAT	2423
141.3. CONFIGURING OPTIONS	2424

141.3.1. Configuring Component Options	2424
141.3.2. Configuring Endpoint Options	2424
141.4. COMPONENT OPTIONS	2425
141.5. ENDPOINT OPTIONS	2426
141.5.1. Path Parameters (1 parameters)	2426
141.5.2. Query Parameters (45 parameters)	2426
141.6. TREATMENT OF THE MESSAGE BODY	2432
141.7. RESULT OF THE QUERY	2432
141.8. USING STREAMLIST	2433
141.9. HEADER VALUES	2433
141.10. GENERATED KEYS	2434
141.11. DATASOURCE	2434
141.12. USING NAMED PARAMETERS	2434
141.13. USING EXPRESSION PARAMETERS IN PRODUCERS	2434
141.13.1. Using expression parameters in consumers	2435
141.14. USING IN QUERIES WITH DYNAMIC VALUES	2435
141.15. USING THE JDBC BASED IDEMPOTENT REPOSITORY	2436
141.15.1. Customize the JDBC idempotency repository	2437
141.15.2. Orphan Lock aware Jdbc IdempotentRepository	2438
141.15.3. Caching Jdbc IdempotentRepository	2438
141.16. USING THE JDBC BASED AGGREGATION REPOSITORY	2439
141.16.1. Database	2439
141.17. STORING BODY AND HEADERS AS TEXT	2440
141.17.1. Codec (Serialization)	2440
141.17.2. Transaction	2441
141.17.2.1. Service (Start/Stop)	2441
141.17.3. Aggregator configuration	2441
141.17.4. Optimistic locking	2441
141.17.5. Propagation behavior	2442
141.17.6. PostgreSQL case	2442
141.18. CAMEL SQL STARTER	2443
141.19. SPRING BOOT AUTO-CONFIGURATION	2443
CHAPTER 142. SQL STORED PROCEDURE	2446
142.1. DEPENDENCIES	2446
142.2. URI FORMAT	2446
142.3. CONFIGURING OPTIONS	2446
142.3.1. Configuring Component Options	2446
142.3.2. Configuring Endpoint Options	2447
142.4. COMPONENT OPTIONS	2447
142.5. ENDPOINT OPTIONS	2448
142.5.1. Path Parameters (1 parameters)	2448
142.5.2. Query Parameters (8 parameters)	2448
142.6. MESSAGE HEADERS	2449
142.7. DECLARING THE STORED PROCEDURE TEMPLATE	2450
142.7.1. IN Parameters	2450
142.7.2. OUT Parameters	2451
142.7.3. INOUT Parameters	2451
142.7.4. Query Timeout	2452
142.8. CAMEL SQL STARTER	2452
142.9. SPRING BOOT AUTO-CONFIGURATION	2452
CHAPTER 143. SSH	2455

143.1. DEPENDENCIES	2455
143.2. URI FORMAT	2455
143.3. CONFIGURING OPTIONS	2455
143.3.1. Configuring Component Options	2455
143.3.2. Configuring Endpoint Options	2455
143.4. COMPONENT OPTIONS	2456
143.5. ENDPOINT OPTIONS	2459
143.5.1. Path Parameters (2 parameters)	2459
143.5.2. Query Parameters (39 parameters)	2459
143.6. MESSAGE HEADERS	2464
143.7. USAGE AS A PRODUCER ENDPOINT	2465
143.8. AUTHENTICATION	2465
143.9. CERTIFICATE DEPENDENCIES	2466
143.10. SPRING BOOT AUTO-CONFIGURATION	2466
CHAPTER 144. STUB	2470
144.1. DEPENDENCIES	2470
144.2. URI FORMAT	2470
144.3. CONFIGURING OPTIONS	2470
144.3.1. Configuring Component Options	2470
144.3.2. Configuring Endpoint Options	2471
144.4. COMPONENT OPTIONS	2471
144.5. ENDPOINT OPTIONS	2473
144.5.1. Path Parameters (1 parameters)	2473
144.5.2. Query Parameters (18 parameters)	2473
144.6. EXAMPLES	2476
144.7. SPRING BOOT AUTO-CONFIGURATION	2476
CHAPTER 145. TELEGRAM	2478
145.1. DEPENDENCIES	2478
145.2. URI FORMAT	2478
145.3. CONFIGURING OPTIONS	2478
145.3.1. Configuring Component Options	2478
145.3.2. Configuring Endpoint Options	2479
145.4. COMPONENT OPTIONS	2479
145.5. ENDPOINT OPTIONS	2480
145.5.1. Path Parameters (1 parameters)	2480
145.5.2. Query Parameters (30 parameters)	2481
145.5.3. Message Headers	2485
145.6. USAGE	2485
145.7. PRODUCER EXAMPLE	2485
145.8. CONSUMER EXAMPLE	2487
145.9. REACTIVE CHAT-BOT EXAMPLE	2487
145.10. GETTING THE CHAT ID	2488
145.11. CUSTOMIZING KEYBOARD	2489
145.12. WEBHOOK MODE	2490
145.13. SPRING BOOT AUTO-CONFIGURATION	2490
CHAPTER 146. TIMER	2492
146.1. DEPENDENCIES	2492
146.2. URI FORMAT	2492
146.3. CONFIGURING OPTIONS	2492
146.3.1. Configuring Component Options	2492
146.3.2. Configuring Endpoint Options	2493

146.4. COMPONENT OPTIONS	2493
146.5. ENDPOINT OPTIONS	2494
146.5.1. Path Parameters (1 parameters)	2494
146.5.2. Query Parameters (13 parameters)	2494
146.6. EXCHANGE PROPERTIES	2495
146.7. SAMPLE	2496
146.8. FIRING AS SOON AS POSSIBLE	2496
146.9. FIRING ONLY ONCE	2496
146.10. SPRING BOOT AUTO-CONFIGURATION	2497
CHAPTER 147. TOKENIZE	2498
147.1. DEPENDENCIES	2498
147.2. TOKENIZE OPTIONS	2498
147.3. EXAMPLE	2499
147.4. SEE ALSO	2499
147.5. SPRING BOOT AUTO-CONFIGURATION	2499
CHAPTER 148. VALIDATOR	2517
148.1. DEPENDENCIES	2517
148.2. URI FORMAT	2517
148.3. CONFIGURING OPTIONS	2517
148.3.1. Configuring Component Options	2517
148.3.2. Configuring Endpoint Options	2518
148.4. COMPONENT OPTIONS	2518
148.5. ENDPOINT OPTIONS	2519
148.5.1. Path Parameters (1 parameters)	2519
148.5.2. Query Parameters (10 parameters)	2519
148.6. EXAMPLE	2521
148.7. ADVANCED: JMX METHOD CLEARCACHEDSCHEMA	2521
148.8. SPRING BOOT AUTO-CONFIGURATION	2521
CHAPTER 149. VELOCITY	2522
149.1. DEPENDENCIES	2522
149.2. URI FORMAT	2522
149.3. CONFIGURING OPTIONS	2522
149.3.1. Configuring Component Options	2522
149.3.2. Configuring Endpoint Options	2522
149.4. COMPONENT OPTIONS	2523
149.5. ENDPOINT OPTIONS	2524
149.5.1. Path Parameters (1 parameters)	2524
149.5.2. Query Parameters (7 parameters)	2524
149.6. MESSAGE HEADERS	2525
149.7. VELOCITY CONTEXT	2526
149.8. HOT RELOADING	2527
149.9. DYNAMIC TEMPLATES	2527
149.10. SAMPLES	2527
149.11. THE EMAIL SAMPLE	2528
149.12. SPRING BOOT AUTO-CONFIGURATION	2529
CHAPTER 150. VERT.X HTTP CLIENT	2531
150.1. DEPENDENCIES	2531
150.2. URI FORMAT	2531
150.3. CONFIGURING OPTIONS	2531
150.3.1. Configuring component options	2531

150.3.2. Configuring endpoint options	2531
150.4. COMPONENT OPTIONS	2532
150.5. ENDPOINT OPTIONS	2534
150.5.1. Path Parameters (1 parameters)	2534
150.5.2. Query Parameters (23 parameters)	2534
150.6. MESSAGE HEADERS	2536
150.7. USAGE	2537
150.8. URI PARAMETERS	2538
150.9. RESPONSE CODE	2538
150.10. THROWEXCEPTIONONFAILURE	2538
150.11. EXCEPTIONS	2538
150.12. HTTP METHOD	2538
150.13. HTTP FORM PARAMETERS	2539
150.14. MULTIPART FORM DATA	2539
150.15. CUSTOMIZING VERT.X WEB CLIENT OPTIONS	2539
150.15.1. SSL	2539
150.16. SESSION MANAGEMENT	2539
150.17. SPRING BOOT AUTO-CONFIGURATION	2539
CHAPTER 151. VERT.X WEBSOCKET	2543
151.1. DEPENDENCIES	2543
151.2. URI FORMAT	2543
151.3. CONFIGURING OPTIONS	2543
151.3.1. Configuring Component Options	2543
151.3.2. Configuring Endpoint Options	2543
151.4. COMPONENT OPTIONS	2544
151.5. ENDPOINT OPTIONS	2545
151.5.1. Path Parameters (3 parameters)	2545
151.5.2. Query Parameters (18 parameters)	2545
151.6. MESSAGE HEADERS	2548
151.7. USAGE	2548
151.8. PATH AND QUERY PARAMETERS	2549
151.9. SENDING MESSAGES TO PEERS CONNECTED TO THE VERTX-WEBSOCKET SERVER CONSUMER	2549
151.10. SSL	2550
151.11. SPRING BOOT AUTO-CONFIGURATION	2550
CHAPTER 152. WEBHOOK	2552
152.1. DEPENDENCIES	2552
152.2. URI FORMAT	2552
152.3. CONFIGURING OPTIONS	2552
152.3.1. Configuring Component Options	2552
152.3.2. Configuring Endpoint Options	2553
152.4. COMPONENT OPTIONS	2553
152.5. ENDPOINT OPTIONS	2554
152.5.1. Path Parameters (1 parameters)	2554
152.5.2. Query Parameters (8 parameters)	2554
152.6. EXAMPLES	2555
152.7. SPRING BOOT AUTO-CONFIGURATION	2555
CHAPTER 153. XJ	2557
153.1. DEPENDENCIES	2557
153.2. URI FORMAT	2557
153.3. CONFIGURING OPTIONS	2557

153.3.1. Configuring component options	2557
153.3.2. Configuring endpoint options	2558
153.4. COMPONENT OPTIONS	2558
153.5. ENDPOINT OPTIONS	2559
153.5.1. Path Parameters (1 parameters)	2559
153.5.2. Query Parameters (19 parameters)	2560
153.6. MESSAGE HEADERS	2562
153.7. USING XJ ENDPOINTS	2563
153.7.1. Converting JSON to XML	2563
153.7.2. Converting XML to JSON	2565
153.7.2.1. Available type hints	2569
153.8. SPRING BOOT AUTO-CONFIGURATION	2569
CHAPTER 154. XML TOKENIZE	2572
154.1. DEPENDENCIES	2572
154.2. XML TOKENIZER OPTIONS	2572
154.3. EXAMPLE	2573
154.4. SPRING BOOT AUTO-CONFIGURATION	2573
CHAPTER 155. XPATH	2574
155.1. DEPENDENCIES	2574
155.2. XPATH LANGUAGE OPTIONS	2574
155.3. NAMESPACES	2575
155.4. VARIABLES	2575
155.4.1. Namespace given	2576
155.4.2. No namespace given	2576
155.5. FUNCTIONS	2576
155.5.1. Functions example	2577
155.6. STREAM BASED MESSAGE BODIES	2577
155.7. SETTING RESULT TYPE	2578
155.8. USING XPATH ON HEADERS	2578
155.9. EXAMPLE	2578
155.10. USING NAMESPACES	2579
155.11. USING @XPATH ANNOTATION FOR BEAN INTEGRATION	2580
155.12. USING XPATHBUILDER WITHOUT AN EXCHANGE	2580
155.13. USING SAXON WITH XPATHBUILDER	2581
155.13.1. Setting a custom XPathFactory using System Property	2581
155.13.2. Enabling Saxon from XML DSL	2581
155.14. NAMESPACE AUDITING TO AID DEBUGGING	2582
155.14.1. Logging the Namespace Context of your XPath expression/predicate	2582
155.14.2. Auditing namespaces	2582
155.15. LOADING SCRIPT FROM EXTERNAL RESOURCE	2583
155.16. SPRING BOOT AUTO-CONFIGURATION	2583
CHAPTER 156. XSLT	2585
156.1. DEPENDENCIES	2585
156.2. URI FORMAT	2585
156.3. CONFIGURING OPTIONS	2585
156.3.1. Configuring Component Options	2585
156.3.2. Configuring Endpoint Options	2586
156.4. COMPONENT OPTIONS	2586
156.5. ENDPOINT OPTIONS	2587
156.5.1. Path Parameters (1 parameters)	2587
156.5.2. Query Parameters (13 parameters)	2588

156.6. USING XSLT ENDPOINTS	2590
156.7. GETTING USEABLE PARAMETERS INTO THE XSLT	2590
156.8. SPRING XML VERSIONS	2590
156.9. USING XSL:INCLUDE	2591
156.10. USING XSL:INCLUDE AND DEFAULT PREFIX	2591
156.11. DYNAMIC STYLESHEETS	2591
156.12. ACCESSING WARNINGS, ERRORS AND FATALERRORS FROM XSLT ERRORLISTENER	2591
156.13. SPRING BOOT AUTO-CONFIGURATION	2592
CHAPTER 157. XSLT SAXON	2594
157.1. DEPENDENCIES	2594
157.2. URI FORMAT	2594
157.3. CONFIGURING OPTIONS	2594
157.3.1. Configuring Component Options	2595
157.3.2. Configuring Endpoint Options	2595
157.4. COMPONENT OPTIONS	2595
157.5. ENDPOINT OPTIONS	2596
157.5.1. Path Parameters (1 parameters)	2597
157.5.2. Query Parameters (18 parameters)	2597
157.6. USING XSLT ENDPOINTS	2599
157.7. GETTING USEABLE PARAMETERS INTO THE XSLT	2600
157.8. SPRING XML VERSIONS	2600
157.9. USING XSL:INCLUDE	2600
157.10. USING XSL:INCLUDE AND DEFAULT PREFIX	2601
157.11. USING SAXON EXTENSION FUNCTIONS	2601
157.12. DYNAMIC STYLESHEETS	2601
157.13. ACCESSING WARNINGS, ERRORS AND FATALERRORS FROM XSLT ERRORLISTENER	2601
157.14. SPRING BOOT AUTO-CONFIGURATION	2602
CHAPTER 158. YAML DSL	2604
158.1. DEFINING A ROUTE	2604
158.2. DEFINING ENDPOINTS	2605
158.3. DEFINING BEANS	2606
158.4. CONFIGURING OPTIONS	2607
158.4.1. Configuring Component Options	2607
158.4.2. Configuring Endpoint Options	2607
158.5. CONFIGURING OPTIONS ON LANGUAGES	2607
158.6. EXTERNAL EXAMPLES	2608
CHAPTER 159. ZIP FILE	2609
159.1. DEPENDENCIES	2609
159.2. ZIPFILE OPTIONS	2609
159.3. MARSHAL	2609
159.4. UNMARSHAL	2610
159.4.1. Aggregate	2611
159.5. SPRING BOOT AUTO-CONFIGURATION	2611

PREFACE

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see [our CTO Chris Wright's message](#).

CHAPTER 1. COMPONENT STARTERS

Camel Spring Boot supports the following Camel artifacts as Spring Boot Starters:

- [Table 1.1, “Camel Components”](#)
- [Table 1.2, “Camel Data Formats”](#)
- [Table 1.3, “Camel Languages”](#)
- [Table 1.4, “Miscellaneous Extensions”](#)



NOTE

The BOM for Red Hat build of Apache Camel for Camel Spring Boot lists both supported and unsupported components. See [Component Starters](#) for the latest list of supported components.

Table 1.1. Camel Components

Component	Artifact	Description	Support on IBM Power and IBM Z
AMQP	camel-amqp-starter	Messaging with AMQP protocol using Apache QPid Client.	Yes
AWS Cloudwatch	camel-aws2-cw-starter	Sending metrics to AWS CloudWatch using AWS SDK version 2.x.	Yes
AWS DynamoDB	camel-aws2-ddb-starter	Store and retrieve data from AWS DynamoDB service using AWS SDK version 2.x.	Yes
AWS Kinesis	camel-aws2-kinesis-starter	Consume and produce records from and to AWS Kinesis Streams using AWS SDK version 2.x.	Yes
AWS Lambda	camel-aws2-lambda-starter	Manage and invoke AWS Lambda functions using AWS SDK version 2.x.	Yes
AWS S3 Storage Service	camel-aws2-s3-starter	Store and retrieve objects from AWS S3 Storage Service using AWS SDK version 2.x.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
AWS Simple Notification System (SNS)	camel-aws2-sns-starter	Send messages to an AWS Simple Notification Topic using AWS SDK version 2.x.	Yes
AWS Simple Queue Service (SQS)	camel-aws2-sqs-starter	Send and receive messages to/from AWS SQS service using AWS SDK version 2.x.	Yes
AWS Secrets Manager	camel-aws-secrets-manager-starter	Manage secrets using AWS Secrets Manager.	Yes
Azure Key Vault	camel-azure-key-vault-starter	Manage secrets and keys in Azure Key Vault Service	Yes
Azure ServiceBus	camel-azure-servicebus-starter	Send and receive messages to/from Azure Event Bus.	Yes
Azure Storage Blob Service	camel-azure-storage-blob-starter	Store and retrieve blobs from Azure Storage Blob Service using SDK v12.	Yes
Azure Storage Queue Service	camel-azure-storage-queue-starter	The azure-storage-queue component is used for storing and retrieving the messages to/from Azure Storage Queue using Azure SDK v12.	Yes
Bean	camel-bean-starter	Invoke methods of Java beans stored in Camel registry.	Yes
Bean Validator	camel-bean-validator-starter	Validate the message body using the Java Bean Validation API.	Yes
Browse	camel-browse-starter	Inspect the messages received on endpoints supporting BrowseableEndpoint.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Cassandra CQL	camel-cassandraql-starter	Integrate with Cassandra 2.0 using the CQL3 API (not the Thrift API). Based on Cassandra Java Driver provided by DataStax.	Yes
CICS	camel-cics-starter	Interact with CICS® general-purpose transaction processing subsystem.	No
Control Bus	camel-controlbus-starter	Manage and monitor Camel routes.	Yes
Cron	camel-cron-starter	A generic interface for triggering events at times specified through the Unix cron syntax.	Yes
Crypto (JCE)	camel-crypto-starter	Sign and verify exchanges using the Signature Service of the Java Cryptographic Extension (JCE).	Yes
CXF	camel-cxf-soap-starter	Expose SOAP WebServices using Apache CXF or connect to external WebServices using CXF WS client.	Yes
CXF-RS	camel-cxf-rest-starter	Expose JAX-RS REST services using Apache CXF or connect to external REST services using CXF REST client.	Yes
Data Format	camel-dataformat-starter	Use a Camel Data Format as a regular Camel Component.	Yes
Dataset	camel-dataset-starter	Provide data for load and soak testing of your Camel application.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Direct	camel-direct-starter	Call another endpoint from the same Camel Context synchronously.	Yes
Elastic Search	camel-elasticsearch-starter	Send requests to ElasticSearch via Java Client API.	Yes
FHIR	camel-fhir-starter	Exchange information in the healthcare domain using the FHIR (Fast Healthcare Interoperability Resources) standard.	No
File	camel-file-starter	Read and write files.	Yes
Flink	camel-flink-starter	Send DataSet jobs to an Apache Flink cluster.	Yes
FTP	camel-ftp-starter	Upload and download files to/from FTP servers.	Yes
Google BigQuery	camel-google-bigquery-starter	Google BigQuery data warehouse for analytics.	Yes
Google Pubsub	camel-google-pubsub-starter	Send and receive messages to/from Google Cloud Platform PubSub Service.	Yes
Google Secret Manager	camel-google-secret-manager-starter	Manage Google Secret Manager Secrets	Yes
GraphQL	camel-graphql-starter	Send GraphQL queries and mutations to external systems.	Yes
gRPC	camel-grpc-starter	Expose gRPC endpoints and access external gRPC endpoints.	Yes
Hashicorp Vault	camel-hashicorp-starter	Manage secrets in Hashicorp Vault Service.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
HTTP	camel-http-starter	Send requests to external HTTP servers using Apache HTTP Client 4.x.	Yes
Infinispan	camel-infinispan-starter	Read and write from/to Infinispan distributed key/value store and data grid.	No
Infinispan Embedded	camel-infinispan-embedded-starter	Read and write from/to Infinispan distributed key/value store and data grid.	Yes
JDBC	camel-jdbc-starter	Access databases through SQL and JDBC.	Yes
Jira	camel-jira-starter	Interact with JIRA issue tracker.	Yes
JMS	camel-jms-starter	Sent and receive messages to/from a JMS Queue or Topic.	Yes
Jolokia	camel-jolokia-starter	integrates the Jolokia agent configuration in Spring Boot.	Yes
JPA	camel-jpa-starter	Store and retrieve Java objects from databases using Java Persistence API (JPA).	Yes
JSLT	camel-jslt-starter	Query or transform JSON payloads using an JSLT.	Yes
Kafka	camel-kafka-starter	Sent and receive messages to/from an Apache Kafka broker.	Yes
Kamelet	camel-kamelet-starter	To call Kamelets	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Kubernetes ConfigMap	camel-kubernetes-starter	Perform operations on Kubernetes ConfigMaps and get notified on ConfigMaps changes.	Yes
Kubernetes Custom Resources	camel-kubernetes-starter	Perform operations on Kubernetes Custom Resources and get notified on Deployment changes.	Yes
Kubernetes Deployments	camel-kubernetes-starter	Perform operations on Kubernetes Deployments and get notified on Deployment changes.	Yes
Kubernetes Event	camel-kubernetes-starter	Perform operations on Kubernetes Events and get notified on Events changes.	Yes
Kubernetes HPA	camel-kubernetes-starter	Perform operations on Kubernetes Horizontal Pod Autoscalers (HPA) and get notified on HPA changes.	Yes
Kubernetes Job	camel-kubernetes-starter	Perform operations on Kubernetes Jobs.	Yes
Kubernetes Namespaces	camel-kubernetes-starter	Perform operations on Kubernetes Namespaces and get notified on Namespace changes.	Yes
Kubernetes Nodes	camel-kubernetes-starter	Perform operations on Kubernetes Nodes and get notified on Node changes.	Yes
Kubernetes Persistent Volume	camel-kubernetes-starter	Perform operations on Kubernetes Persistent Volumes and get notified on Persistent Volume changes.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Kubernetes Persistent Volume Claim	camel-kubernetes-starter	Perform operations on Kubernetes Persistent Volumes Claims and get notified on Persistent Volumes Claim changes.	Yes
Kubernetes Pods	camel-kubernetes-starter	Perform operations on Kubernetes Pods and get notified on Pod changes.	Yes
Kubernetes Replication Controller	camel-kubernetes-starter	Yes Perform operations on Kubernetes Replication Controllers and get notified on Replication Controllers changes.	Yes
Kubernetes Resources Quota	camel-kubernetes-starter	Perform operations on Kubernetes Resources Quotas.	Yes
Kubernetes Secrets	camel-kubernetes-starter	Perform operations on Kubernetes Secrets.	Yes
Kubernetes Service Account	camel-kubernetes-starter	Perform operations on Kubernetes Service Accounts.	Yes
Kubernetes Services	camel-kubernetes-starter	Perform operations on Kubernetes Services and get notified on Service changes.	Yes
Kudu	camel-kudu-starter	Interact with Apache Kudu, a free and open source column-oriented data store of the Apache Hadoop ecosystem.	No
Language	camel-language-starter	Execute scripts in any of the languages supported by Camel.	Yes
LDAP	camel-ldap-starter	Perform searches on LDAP servers.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Log	camel-log-starter	Log messages to the underlying logging mechanism.	Yes
LRA	camel-lra-starter	Camel saga binding for Long-Running-Action framework.	Yes
Mail	camel-mail-starter	Send and receive emails using imap, pop3 and smtp protocols.	Yes
Mail Microsoft OAuth	camel-mail-microsoft-oauth-starter	Camel Mail OAuth2 Authenticator for Microsoft Exchange Online.	Yes
MapStruct	camel-mapstruct-starter	Type Conversion using Mapstruct.	Yes
Master	camel-master-starter	Have only a single consumer in a cluster consuming from a given endpoint; with automatic failover if the JVM dies.	Yes
Micrometer	camel-micrometer-starter	Collect various metrics directly from Camel routes using the Micrometer library.	Yes
Minio	camel-minio-starter	Store and retrieve objects from Minio Storage Service using Minio SDK.	Yes
MLLP	camel-mlp-starter	Communicate with external systems using the MLLP protocol.	Yes
Mock	camel-mock-starter	Test routes and mediation rules using mocks.	Yes
MongoDB	camel-mongodb-starter	Perform operations on MongoDB documents and collections.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
MyBatis	camel-mybatis-starter	Performs a query, poll, insert, update or delete in a relational database using MyBatis.	Yes
Netty	camel-netty-starter	Socket level networking using TCP or UDP with Netty 4.x.	Yes
Observability Services	camel-observability-services	Camel Observability Services	Yes
Olingo4	camel-olingo4-starter	Communicate with OData 4.0 services using Apache Olingo OData API.	Yes
OpenSearch	camel-opensearch-starter	Send requests to OpenSearch via Java Client API.	Yes
Openshift Builds	camel-kubernetes-starter	Perform operations on OpenShift Builds.	Yes
Openshift Deployment Configs	camel-kubernetes-starter	Perform operations on Openshift Deployment Configs and get notified on Deployment Config changes.	Yes
Netty HTTP	camel-netty-http-starter	Netty HTTP server and client using the Netty 4.x.	Yes
Paho	camel-paho-starter	Communicate with MQTT message brokers using Eclipse Paho MQTT Client.	Yes
Paho MQTT 5	camel-paho-mqtt5-starter	Communicate with MQTT message brokers using Eclipse Paho MQTT v5 Client.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Platform HTTP	camel-platform-http-starter	Expose HTTP endpoints using the HTTP server available in the current platform.	Yes
Quartz	camel-quartz-starter	Schedule sending of messages using the Quartz 2.x scheduler.	Yes
Ref	camel-ref-starter	Route messages to an endpoint looked up dynamically by name in the Camel Registry.	Yes
REST	camel-rest-starter	Expose REST services or call external REST services.	Yes
Saga	camel-saga-starter	Execute custom actions within a route using the Saga EIP.	Yes
Salesforce	camel-salesforce-starter	Communicate with Salesforce using Java DTOs.	Yes
SAP	camel-sap-starter	Uses the SAP Java Connector (SAP JCo) library to facilitate bidirectional communication with SAP and the SAP IDoc library to facilitate the transmission of documents in the Intermediate Document (IDoc) format.	Yes
Scheduler	camel-scheduler-starter	Generate messages in specified intervals using <code>java.util.concurrent.ScheduledExecutorService</code> .	Yes
SEDA	camel-seda-starter	Asynchronously call another endpoint from any Camel Context in the same JVM.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Servlet	camel-servlet-starter	Serve HTTP requests by a Servlet.	Yes
Slack	camel-slack-starter	Send and receive messages to/from Slack.	Yes
SMB	camel-smb-starter	Receive files from SMB (Server Message Block) shares.	Yes
Smooks	camel-smooks-starter	Transform and bind XML as well as non-XML data, including EDI, CSV, JSON, and YAML using Smooks.	Yes
SNMP	camel-snmp-starter	Receive traps and poll SNMP (Simple Network Management Protocol) capable devices.	Yes
Splunk	camel-splunk-starter	Publish or search for events in Splunk.	No
Spring Batch	camel-spring-batch-starter	Send messages to Spring Batch for further processing.	Yes
Spring JDBC	camel-spring-jdbc-starter	Access databases through SQL and JDBC with Spring Transaction support.	Yes
Spring LDAP	camel-spring-ldap-starter	Perform searches in LDAP servers using filters as the message payload.	Yes
Spring RabbitMQ	camel-spring-rabbitmq-starter	Send and receive messages from RabbitMQ using Spring RabbitMQ client.	Yes
Spring Redis	camel-spring-redis-starter	Send and receive messages from Redis.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Spring Webservice	camel-spring-ws-starter	You can use this component to integrate with Spring Web Services. It offers client-side support for accessing web services and server-side support for creating your contract-first web services.	Yes
SQL	camel-sql-starter	Perform SQL queries using Spring JDBC.	Yes
SQL Stored Procedure	camel-sql-starter	Perform SQL queries as a JDBC Stored Procedures using Spring JDBC.	Yes
SSH	camel-ssh-starter	Execute commands on remote hosts using SSH.	Yes
Stub	camel-stub-starter	Stub out any physical endpoints while in development or testing.	Yes
Telegram	camel-telegram-starter	Send and receive messages acting as a Telegram Bot Telegram Bot API.	Yes
Timer	camel-timer-starter	Generate messages in specified intervals using java.util.Timer.	Yes
Validator	camel-validator-starter	Validate the payload using XML Schema and JAXP Validation.	Yes
Velocity	camel-velocity-starter	Transform messages using a Velocity template.	Yes
Vert.x HTTP Client	camel-vertx-http-starter	Send requests to external HTTP servers using Vert.x.	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
Vert.x WebSocket	camel-vertx-websocket-starter	Expose WebSocket endpoints and connect to remote WebSocket servers using Vert.x.	Yes
Webhook	camel-webhook-starter	Expose webhook endpoints to receive push notifications for other Camel components.	Yes
XJ	camel-xj-starter	Transform JSON and XML message using a XSLT.	Yes
XSLT	camel-xslt-starter	Transforms XML payload using an XSLT template.	Yes
XSLT Saxon	camel-xslt-saxon-starter	Transform XML payloads using an XSLT template using Saxon.	Yes

Table 1.2. Camel Data Formats

Component	Artifact	Description	Support on IBM Power and IBM Z
Avro	camel-avro-starter	Serialize and deserialize messages using Apache Avro binary data format.	Yes
Avro Jackson	camel-jackson-avro-starter	Marshal POJOs to Avro and back using Jackson.	Yes
Bindy	camel-bindy-starter	Marshal and unmarshal between POJOs and key-value pair (KVP) format using Camel Bindy.	Yes
BeanIO	camel-beanio-starter	Marshal and unmarshal Java beans to and from flat files (such as CSV, delimited, or fixed length formats).	Yes

Component	Artifact	Description	Support on IBM Power and IBM Z
HL7	camel-hl7-starter	Marshal and unmarshal HL7 (Health Care) model objects using the HL7 MLLP codec.	Yes
JacksonXML	camel-jacksonxml-starter	Unmarshal a XML payloads to POJOs and back using XMLMapper extension of Jackson.	Yes
JAXB	camel-jaxb-starter	Unmarshal XML payloads to POJOs and back using JAXB2 XML marshalling standard.	Yes
JSON Gson	camel-gson-starter	Marshal POJOs to JSON and back using Gson	Yes
JSON Jackson	camel-jackson-starter	Marshal POJOs to JSON and back using Jackson	Yes
Protobuf Jackson	camel-jackson-protobuf-starter	Marshal POJOs to Protobuf and back using Jackson.	Yes
SOAP	camel-soap-starter	Marshal Java objects to SOAP messages and back.	Yes
Zip File	camel-zipfile-starter	Compression and decompress streams using java.util.zip.ZipStream.	Yes

Table 1.3. Camel Languages

Language	Artifact	Description	Support on IBM Power and IBM Z
Constant	camel-core-starter	A fixed value set only once during the route startup.	Yes

Language	Artifact	Description	Support on IBM Power and IBM Z
CSimple	camel-core-starter	Evaluate a compiled simple expression.	Yes
ExchangeProperty	camel-core-starter	Gets a property from the Exchange.	Yes
File	camel-core-starter	File related capabilities for the Simple language.	Yes
Groovy	camel-groovy-starter	Evaluates a Groovy script.	Yes
Header	camel-core-starter	Gets a header from the Exchange.	Yes
JQ	camel-jq-starter	Evaluates a JQ expression against a JSON message body.	Yes
JSONPath	camel-jsonpath-starter	Evaluates a JSONPath expression against a JSON message body.	Yes
Ref	camel-core-starter	Uses an existing expression from the registry.	Yes
Simple	camel-core-starter	Evaluates a Camel simple expression.	Yes
Tokenize	camel-core-starter	Tokenize text payloads using delimiter patterns.	Yes
XML Tokenize	camel-xml-jaxp-starter	Tokenize XML payloads.	Yes
XPath	camel-xpath-starter	Evaluates an XPath expression against an XML payload.	Yes
XQuery	camel-saxon-starter	Query and/or transform XML payloads using XQuery and Saxon.	Yes

Table 1.4. Miscellaneous Extensions

Extensions	Artifact	Description	Support on IBM Power and IBM Z
Jasypt	camel-jasypt-starter	Security using Jasypt	Yes
Kamelet Main	camel-kamelet-main-starter	Main to run Kamelet standalone	Yes
Openapi Java	camel-openapi-java-starter	Rest-dsl support for using openapi doc	Yes
OpenTelemetry	camel-opentelemetry-starter	Distributed tracing using OpenTelemetry	Yes
Spring Security	camel-spring-security-starter	Security using Spring Security	Yes
YAML DSL	camel-yaml-dsl-starter	Camel DSL with YAML	Yes

CHAPTER 2. AMQP

Since Camel 1.2

Both producer and consumer are supported

The AMQP component supports the [AMQP 1.0 protocol](#) using the JMS Client API of the [Qpid](#) project.

2.1. DEPENDENCIES

When using **camel-amqp** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-amqp-starter</artifactId>
</dependency>
```

2.2. URI FORMAT

```
amqp:[queue:]topic:]destinationName[?options]
```

2.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

2.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

2.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

2.4. COMPONENT OPTIONS

The AMQP component supports 100 options, which are listed below.

Name	Description	Default	Type
clientId (common)	Sets the JMS client ID to use. Note that this value, if specified, must be unique and can only be used by a single JMS connection instance. The clientId option is compulsory with JMS 1.1 durable topic subscriptions, because the client ID is used to control which client messages have to be stored for. With JMS 2.0 clients, clientId may be omitted, which creates a 'global' subscription.		String
connectionFactory (common)	The connection factory to be use. A connection factory must be configured either on the component or endpoint.		ConnectionFactory
disableReplyTo (common)	Specifies whether Camel ignores the JMSReplyTo header in messages. If true, Camel does not send a reply back to the destination specified in the JMSReplyTo header. You can use this option if you want Camel to consume from a route and you do not want Camel to automatically send back a reply message because another component in your code handles the reply message. You can also use this option if you want to use Camel as a proxy between different message brokers and you want to route message from one system to another.	false	boolean
durableSubscriptionName (common)	The durable subscriber name for specifying durable topic subscriptions. The clientId option must be configured for a JMS 1.1 durable subscription, and may be configured for JMS 2.0, to create a private durable subscription.		String

Name	Description	Default	Type
includeAmqpAnnotations (common)	Whether to include AMQP annotations when mapping from AMQP to Camel Message. Setting this to true maps AMQP message annotations that contain a JMS_AMQP_MA_ prefix to message headers. Due to limitations in Apache Qpid JMS API, currently delivery annotations are ignored.	false	boolean
jmsMessageType (common)	Allows you to force the use of a specific javax.jms.Message implementation for sending JMS messages. Possible values are: Bytes, Map, Object, Stream, Text. By default, Camel would determine which JMS message type to use from the In body type. This option allows you to specify it. Enum values: • Bytes • Map • Object • Stream • Text		JmsMessageType
replyTo (common)	Provides an explicit ReplyTo destination (overrides any incoming value of Message.getJMSReplyTo() in consumer).		String
testConnectionOnStartup (common)	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	boolean

Name	Description	Default	Type
acknowledgmentModeName (consumer)	The JMS acknowledgement name, which is one of: SESSION_TRANSACTED, CLIENT_ACKNOWLEDGE, AUTO_ACKNOWLEDGE, DUPS_OK_ACKNOWLEDGE. Enum values: • SESSION_TRANSACTED • CLIENT_ACKNOWLEDGE • AUTO_ACKNOWLEDGE • DUPS_OK_ACKNOWLEDGE	AUTO_ACKNOWLEDGE	String
artemisConsumerPriority (consumer)	Consumer priorities allow you to ensure that high priority consumers receive messages while they are active. Normally, active consumers connected to a queue receive messages from it in a round-robin fashion. When consumer priorities are in use, messages are delivered round-robin if multiple active consumers exist with the same high priority. Messages will only go to lower priority consumers when the high priority consumers do not have credit available to consume the message, or those high priority consumers have declined to accept the message (for instance because it does not meet the criteria of any selectors associated with the consumer).		int
asyncConsumer (consumer)	Whether the JmsConsumer processes the Exchange asynchronously. If enabled then the JmsConsumer may pickup the next message from the JMS queue, while the previous message is being processed asynchronously (by the Asynchronous Routing Engine). This means that messages may be processed not 100% strictly in order. If disabled (as default) then the Exchange is fully processed before the JmsConsumer picks up the next message from the JMS queue. Note if transacted has been enabled, then asyncConsumer=true does not run asynchronously, as transaction must be executed synchronously (Camel 3.0 may support async transactions).	false	boolean
autoStartup (consumer)	Specifies whether the consumer container should auto-startup.	true	boolean

Name	Description	Default	Type
cacheLevel (consumer)	Sets the cache level by ID for the underlying JMS resources. See <code>cacheLevelName</code> option for more details.		int
cacheLevelName (consumer)	Sets the cache level by name for the underlying JMS resources. Possible values are: <code>CACHE_AUTO</code>, <code>CACHE_CONNECTION</code>, <code>CACHE_CONSUMER</code>, <code>CACHE_NONE</code>, and <code>CACHE_SESSION</code>. The default setting is <code>CACHE_AUTO</code>. See the Spring documentation and Transactions Cache Levels for more information. Enum values: • <code>CACHE_AUTO</code> • <code>CACHE_CONNECTION</code> • <code>CACHE_CONSUMER</code> • <code>CACHE_NONE</code> • <code>CACHE_SESSION</code>	<code>CACHE_AUTO</code>	String
concurrentConsumers (consumer)	Specifies the default number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.	1	int
maxConcurrentConsumers (consumer)	Specifies the maximum number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToMaxConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.		int
replyToDeliveryPersistent (consumer)	Specifies whether to use persistent delivery by default for replies.	true	boolean

Name	Description	Default	Type
selector (consumer)	Sets the JMS selector to use.		String
subscriptionDurable (consumer)	Set whether to make the subscription durable. The durable subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a durable subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well.	false	boolean
subscriptionName (consumer)	Set the name of a subscription to create. To be applied in case of a topic (pub-sub domain) with a shared or durable subscription. The subscription name needs to be unique within this client's JMS client id, if the client ID is configured. Default is the class name of the specified message listener. Note: Only 1 concurrent consumer (which is the default of this message listener container) is allowed for each subscription, except for a shared subscription (which requires JMS 2.0).		String
subscriptionShared (consumer)	Set whether to make the subscription shared. The shared subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a shared subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Note that shared subscriptions may also be durable, so this flag can (and often will) be combined with <code>subscriptionDurable</code> as well. Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well. Requires a JMS 2.0 compatible message broker.	false	boolean

Name	Description	Default	Type
acceptMessagesWhileStopping (consumer (advanced))	Specifies whether the consumer accept messages while it is stopping. You may consider enabling this option, if you start and stop JMS routes at runtime, while there are still messages enqueued on the queue. If this option is false, and you stop the JMS route, then messages may be rejected, and the JMS broker would have to attempt redeliveries, which yet again may be rejected, and eventually the message may be moved at a dead letter queue on the JMS broker. To avoid this its recommended to enable this option.	false	boolean
allowReplyManagerQuickStop (consumer (advanced))	Whether the DefaultMessageListenerContainer used in the reply managers for request-reply messaging allow the DefaultMessageListenerContainer.runningAllowed flag to quick stop in case JmsConfiguration#isAcceptMessagesWhileStopping is enabled, and org.apache.camel.CamelContext is currently being stopped. This quick stop ability is enabled by default in the regular JMS consumers but to enable for reply managers you must enable this flag.	false	boolean
consumerType (consumer (advanced))	The consumer type to use, which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use <code>org.springframework.jms.listener.DefaultMessageListenerContainer</code>, Simple will use <code>org.springframework.jms.listener.SimpleMessageListenerContainer</code>. When Custom is specified, the <code>MessageListenerContainerFactory</code> defined by the <code>messageListenerContainerFactory</code> option will determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use. Enum values: • Simple • Default • Custom	Default	ConsumerType

Name	Description	Default	Type
defaultTaskExecutorType (consumer (advanced))	Specifies what default TaskExecutor type to use in the DefaultMessageListenerContainer, for both consumer endpoints and the ReplyTo consumer of producer endpoints. Possible values: SimpleAsync (uses Spring's SimpleAsyncTaskExecutor) or ThreadPool (uses Spring's ThreadPoolTaskExecutor with optimal values - cached threadpool-like). If not set, it defaults to the previous behaviour, which uses a cached thread pool for consumer endpoints and SimpleAsync for reply consumers. The use of ThreadPool is recommended to reduce thread trash in elastic configurations with dynamically increasing and decreasing concurrent consumers. Enum values: • ThreadPool • SimpleAsync		DefaultTaskExecutorType
eagerLoadingOfProperties (consumer (advanced))	Enables eager loading of JMS properties and payload as soon as a message is loaded which generally is inefficient as the JMS properties may not be required but sometimes can catch early any issues with the underlying JMS provider and the use of JMS properties. See also the option eagerPoisonBody.	false	boolean
eagerPoisonBody (consumer (advanced))	If eagerLoadingOfProperties is enabled and the JMS message payload (JMS body or JMS properties) is poison (cannot be read/mapped), then set this text as the message body instead so the message can be processed (the cause of the poison are already stored as exception on the Exchange). This can be turned off by setting eagerPoisonBody=false. See also the option eagerLoadingOfProperties.	Poison JMS message due to \${exception.message}	String
exposeListenerSession (consumer (advanced))	Specifies whether the listener session should be exposed when consuming messages.	false	boolean

Name	Description	Default	Type
replyToConsumerType (consumer (advanced))	The consumer type of the reply consumer (when doing request/reply), which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use <code>org.springframework.jms.listener.DefaultMessageListenerContainer</code>, Simple will use <code>org.springframework.jms.listener.SimpleMessageListenerContainer</code>. When Custom is specified, the <code>MessageListenerContainerFactory</code> defined by the <code>messageListenerContainerFactory</code> option will determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use. Enum values: • Simple • Default • Custom	Default	ConsumerType
replyToSameDestinationAllowed (consumer (advanced))	Whether a JMS consumer is allowed to send a reply message to the same destination that the consumer is using to consume from. This prevents an endless loop by consuming and sending back the same message to itself.	false	boolean
taskExecutor (consumer (advanced))	Allows you to specify a custom task executor for consuming messages.		TaskExecutor
deliveryDelay (producer)	Sets delivery delay to use for send calls for JMS. This option requires JMS 2.0 compliant broker.	-1	long
deliveryMode (producer)	Specifies the delivery mode to be used. Possible values are those defined by <code>javax.jms.DeliveryMode</code>. <code>NON_PERSISTENT</code> = 1 and <code>PERSISTENT</code> = 2. Enum values: • 1 • 2		Integer
deliveryPersistent (producer)	Specifies whether persistent delivery is used by default.	true	boolean

Name	Description	Default	Type
explicitQosEnabled (producer)	Set if the deliveryMode, priority or timeToLive qualities of service should be used when sending messages. This option is based on Spring's JmsTemplate. The deliveryMode, priority and timeToLive options are applied to the current endpoint. This contrasts with the preserveMessageQos option, which operates at message granularity, reading QoS properties exclusively from the Camel In message headers.	false	Boolean
formatDateHeadersToIso8601 (producer)	Sets whether JMS date properties should be formatted according to the ISO 8601 standard.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
preserveMessageQos (producer)	Set to true, if you want to send message using the QoS settings specified on the message, instead of the QoS settings on the JMS endpoint. The following three headers are considered JMSPriority, JMSDeliveryMode, and JMSExpiration. You can provide all or only some of them. If not provided, Camel will fall back to use the values from the endpoint instead. So, when using this option, the headers override the values from the endpoint. The explicitQosEnabled option, by contrast, will only use options set on the endpoint, and not values from the message header.	false	boolean

Name	Description	Default	Type
priority (producer)	Values greater than 1 specify the message priority when sending (where 1 is the lowest priority and 9 is the highest). The <code>explicitQosEnabled</code> option must also be enabled in order for this option to have any effect. Enum values: • 1 • 2 • 3 • 4 • 5 • 6 • 7 • 8 • 9	4	int
replyToConcurrentConsumers (producer)	Specifies the default number of concurrent consumers when doing request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.	1	int
replyToMaxConcurrentConsumers (producer)	Specifies the maximum number of concurrent consumers when using request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.		int
replyToOnTimeoutMaxConcurrentConsumers (producer)	Specifies the maximum number of concurrent consumers for continue routing when timeout occurred when using request/reply over JMS.	1	int
replyToOverride (producer)	Provides an explicit <code>ReplyTo</code> destination in the JMS message, which overrides the setting of <code>replyTo</code> . It is useful if you want to forward the message to a remote Queue and receive the reply message from the <code>ReplyTo</code> destination.		String

Name	Description	Default	Type
replyToType (producer)	Allows for explicitly specifying which kind of strategy to use for replyTo queues when doing request/reply over JMS. Possible values are: Temporary, Shared, or Exclusive. By default Camel will use temporary queues. However if replyTo has been configured, then Shared is used by default. This option allows you to use exclusive queues instead of shared ones. See Camel JMS documentation for more details, and especially the notes about the implications if running in a clustered environment, and the fact that Shared reply queues has lower performance than its alternatives Temporary and Exclusive. Enum values: • Temporary • Shared • Exclusive		ReplyToType
requestTimeout (producer)	The timeout for waiting for a reply when using the InOut Exchange Pattern (in milliseconds). The default is 20 seconds. You can include the header CamelJmsRequestTimeout to override this endpoint configured timeout value, and thus have per message individual timeout values. See also the requestTimeoutCheckerInterval option.	20000	long
timeToLive (producer)	When sending messages, specifies the time-to-live of the message (in milliseconds).	-1	long
allowAdditionalHeaders (producer (advanced))	This option is used to allow additional headers which may have values that are invalid according to JMS specification. For example some message systems such as WMQ do this with header names using prefix JMS_IBM_MQMD_ containing values with byte array or other invalid types. You can specify multiple header names separated by comma, and use as suffix for wildcard matching.		String
allowNullBody (producer (advanced))	Whether to allow sending messages with no body. If this option is false and the message body is null, then an JMSEException is thrown.	true	boolean

Name	Description	Default	Type
alwaysCopyMessage (producer (advanced))	If true, Camel will always make a JMS message copy of the message when it is passed to the producer for sending. Copying the message is needed in some situations, such as when a <code>replyToDestinationSelectorName</code> is set (incidentally, Camel will set the <code>alwaysCopyMessage</code> option to true, if a <code>replyToDestinationSelectorName</code> is set).	false	boolean
correlationProperty (producer (advanced))	When using <code>InOut</code> exchange pattern use this JMS property instead of <code>JMSCorrelationID</code> JMS property to correlate messages. If set messages will be correlated solely on the value of this property <code>JMSCorrelationID</code> property will be ignored and not set by Camel.		String
disableTimeToLive (producer (advanced))	Use this option to force disabling time to live. For example when you do request/reply over JMS, then Camel will by default use the <code>requestTimeout</code> value as time to live on the message being sent. The problem is that the sender and receiver systems have to have their clocks synchronized, so they are in sync. This is not always so easy to archive. So you can use <code>disableTimeToLive=true</code> to not set a time to live value on the sent message. Then the message will not expire on the receiver system. See below in section About time to live for more details.	false	boolean
forceSendOriginalMessage (producer (advanced))	When using <code>mapJmsMessage=false</code> Camel will create a new JMS message to send to a new JMS destination if you touch the headers (get or set) during the route. Set this option to true to force Camel to send the original JMS message that was received.	false	boolean
includeSentJMSMessageID (producer (advanced))	Only applicable when sending to JMS destination using <code>InOnly</code> (eg fire and forget). Enabling this option will enrich the Camel Exchange with the actual <code>JMSMessageID</code> that was used by the JMS client when the message was sent to the JMS destination.	false	boolean

Name	Description	Default	Type
replyToCacheLevelName (producer (advanced))	Sets the cache level by name for the reply consumer when doing request/reply over JMS. This option only applies when using fixed reply queues (not temporary). Camel will by default use: <code>CACHE_CONSUMER</code> for exclusive or shared w/ <code>replyToSelectorName</code>. And <code>CACHE_SESSION</code> for shared without <code>replyToSelectorName</code>. Some JMS brokers such as IBM WebSphere may require to set the <code>replyToCacheLevelName=CACHE_NONE</code> to work. Note: If using temporary queues then <code>CACHE_NONE</code> is not allowed, and you must use a higher value such as <code>CACHE_CONSUMER</code> or <code>CACHE_SESSION</code>. Enum values: • <code>CACHE_AUTO</code> • <code>CACHE_CONNECTION</code> • <code>CACHE_CONSUMER</code> • <code>CACHE_NONE</code> • <code>CACHE_SESSION</code>		String
replyToSelectorName (producer (advanced))	Sets the JMS Selector using the fixed name to be used so you can filter out your own replies from the others when using a shared queue (that is, if you are not using a temporary reply queue).		String
streamMessageTypeEnabled (producer (advanced))	Sets whether <code>StreamMessage</code> type is enabled or not. Message payloads of streaming kind such as files, <code>InputStream</code> , etc will either be sent as <code>BytesMessage</code> or <code>StreamMessage</code> . This option controls which kind will be used. By default <code>BytesMessage</code> is used which enforces the entire message payload to be read into memory. By enabling this option the message payload is read into memory in chunks and each chunk is then written to the <code>StreamMessage</code> until no more data.	false	boolean
allowAutoWiredConnectionFactory (advanced)	Whether to auto-discover <code>ConnectionFactory</code> from the registry, if no connection factory has been configured. If only one instance of <code>ConnectionFactory</code> is found then it will be used. This is enabled by default.	true	boolean

Name	Description	Default	Type
allowAutoWiredDestinationResolver (advanced)	Whether to auto-discover DestinationResolver from the registry, if no destination resolver has been configured. If only one instance of DestinationResolver is found then it will be used. This is enabled by default.	true	boolean
allowSerializedHeaders (advanced)	Controls whether or not to include serialized headers. Applies only when transferExchange is true. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
artemisStreamingEnabled (advanced)	Whether optimizing for Apache Artemis streaming mode. This can reduce memory overhead when using Artemis with JMS StreamMessage types. This option must only be enabled if Apache Artemis is being used.	false	boolean
asyncStartListener (advanced)	Whether to startup the JmsConsumer message listener asynchronously, when starting a route. For example if a JmsConsumer cannot get a connection to a remote JMS broker, then it may block while retrying and/or failover. This will cause Camel to block while starting routes. By setting this option to true, you will let routes startup, while the JmsConsumer connects to the JMS broker using a dedicated thread in asynchronous mode. If this option is used, then beware that if the connection could not be established, then an exception is logged at WARN level, and the consumer will not be able to receive messages; You can then restart the route to retry.	false	boolean
asyncStopListener (advanced)	Whether to stop the JmsConsumer message listener asynchronously, when stopping a route.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
configuration (advanced)	To use a shared JMS configuration.		JmsConfiguration

Name	Description	Default	Type
destinationResolver (advanced)	A pluggable <code>org.springframework.jms.support.destination.DestinationResolver</code> that allows you to use your own resolver (for example, to lookup the real destination in a JNDI registry).		<code>DestinationResolver</code>
errorHandler (advanced)	Specifies a <code>org.springframework.util.ErrorHandler</code> to be invoked in case of any uncaught exceptions thrown while processing a <code>Message</code> . By default these exceptions will be logged at the <code>WARN</code> level, if no <code>errorHandler</code> has been configured. You can configure logging level and whether stack traces should be logged using <code>errorHandlerLoggingLevel</code> and <code>errorHandlerLogStackTrace</code> options. This makes it much easier to configure, than having to code a custom <code>ErrorHandler</code> .		<code>ErrorHandler</code>
exceptionListener (advanced)	Specifies the JMS Exception Listener that is to be notified of any underlying JMS exceptions.		<code>ExceptionListener</code>
idleConsumerLimit (advanced)	Specify the limit for the number of consumers that are allowed to be idle at any given time.	1	int
idleTaskExecutionLimit (advanced)	Specifies the limit for idle executions of a receive task, not having received any message within its execution. If this limit is reached, the task will shut down and leave receiving to other executing tasks (in the case of dynamic scheduling; see the <code>maxConcurrentConsumers</code> setting). There is additional doc available from Spring.	1	int
includeAllJMSXProperties (advanced)	Whether to include all <code>JMSXxxx</code> properties when mapping from JMS to Camel <code>Message</code> . Setting this to <code>true</code> will include properties such as <code>JMSXAppID</code> , and <code>JMSXUserID</code> etc. Note: If you are using a custom <code>headerFilterStrategy</code> then this option does not apply.	false	boolean

Name	Description	Default	Type
jmsKeyFormatStrategy (advanced)	Pluggable strategy for encoding and decoding JMS keys so they can be compliant with the JMS specification. Camel provides two implementations out of the box: default and passthrough. The default strategy will safely marshal dots and hyphens (. and -). The passthrough strategy leaves the key as is. Can be used for JMS brokers which do not care whether JMS header keys contain illegal characters. You can provide your own implementation of the <code>org.apache.camel.component.jms.JmsKeyFormatStrategy</code> and refer to it using the <code>#</code> notation. Enum values: • default • passthrough		JmsKeyFormatStrategy
mapJmsMessage (advanced)	Specifies whether Camel should auto map the received JMS message to a suited payload type, such as <code>javax.jms.TextMessage</code> to a <code>String</code> etc.	true	boolean
maxMessagesPerTask (advanced)	The number of messages per task. -1 is unlimited. If you use a range for concurrent consumers (eg min max), then this option can be used to set a value to eg 100 to control how fast the consumers will shrink when less work is required.	-1	int
messageConverter (advanced)	To use a custom Spring <code>org.springframework.jms.support.converter.MessageConverter</code> so you can be in control how to map to/from a <code>javax.jms.Message</code> .		MessageConverter
messageCreatedStrategy (advanced)	To use the given <code>MessageCreatedStrategy</code> which are invoked when Camel creates new instances of <code>javax.jms.Message</code> objects when Camel is sending a JMS message.		MessageCreatedStrategy
messageIdEnabled (advanced)	When sending, specifies whether message IDs should be added. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the message ID set to null; if the provider ignores the hint, the message ID must be set to its normal unique value.	true	boolean

Name	Description	Default	Type
messageListenerContainerFactory (advanced)	Registry ID of the MessageListenerContainerFactory used to determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use to consume messages. Setting this will automatically set <code>consumerType</code> to Custom.		MessageListenerContainerFactory
messageTimestampEnabled (advanced)	Specifies whether timestamps should be enabled by default on sending messages. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the timestamp set to zero; if the provider ignores the hint the timestamp must be set to its normal value.	true	boolean
pubSubNoLocal (advanced)	Specifies whether to inhibit the delivery of messages published by its own connection.	false	boolean
queueBrowseStrategy (advanced)	To use a custom QueueBrowseStrategy when browsing queues.		QueueBrowseStrategy
receiveTimeout (advanced)	The timeout for receiving messages (in milliseconds).	1000	long
recoveryInterval (advanced)	Specifies the interval between recovery attempts, i.e. when a connection is being refreshed, in milliseconds. The default is 5000 ms, that is, 5 seconds.	5000	long
requestTimeoutCheckerInterval (advanced)	Configures how often Camel should check for timed out Exchanges when doing request/reply over JMS. By default Camel checks once per second. But if you must react faster when a timeout occurs, then you can lower this interval, to check more frequently. The timeout is determined by the option <code>requestTimeout</code> .	1000	long
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean

Name	Description	Default	Type
transferException (advanced)	If enabled and you are using Request Reply messaging (InOut) and an Exchange failed on the consumer side, then the caused Exception will be send back in response as a javax.jms.ObjectMessage. If the client is Camel, the returned Exception is rethrown. This allows you to use Camel JMS as a bridge in your routing - for example, using persistent queues to enable robust routing. Notice that if you also have transferExchange enabled, this option takes precedence. The caught exception is required to be serializable. The original Exception on the consumer side can be wrapped in an outer exception such as org.apache.camel.RuntimeCamelException when returned to the producer. Use this with caution as the data is using Java Object serialization and requires the received to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumer!.	false	boolean
transferExchange (advanced)	You can transfer the exchange over the wire instead of just the body and headers. The following fields are transferred: In body, Out body, Fault body, In headers, Out headers, Fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level. You must enable this option on both the producer and consumer side, so Camel knows the payloads is an Exchange and not a regular payload. Use this with caution as the data is using Java Object serialization and requires the receiver to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumers having to use compatible Camel versions!.	false	boolean
useMessageIDAsCorrelationID (advanced)	Specifies whether JMSMessageID should always be used as JMSCorrelationID for InOut messages.	false	boolean
waitForProvisionCorrelationToBeUpdatedCounter (advanced)	Number of times to wait for provisional correlation id to be updated to the actual correlation id when doing request/reply over JMS and when the option useMessageIDAsCorrelationID is enabled.	50	int
waitForProvisionCorrelationToBeUpdatedThreadSleepingTime (advanced)	Interval in millis to sleep each time while waiting for provisional correlation id to be updated.	100	long

Name	Description	Default	Type
headerFilterStrategy (filter)	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message.		HeaderFilterStrategy
errorHandlerLoggingLevel (logging)	Allows to configure the default errorHandler logging level for logging uncaught exceptions. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	WARN	LoggingLevel
errorHandlerLogStackTrace (logging)	Allows to control whether stacktraces should be logged or not, by the default errorHandler.	true	boolean
password (security)	Password to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
username (security)	Username to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
transacted (transaction)	Specifies whether to use transacted mode.	false	boolean

Name	Description	Default	Type
transactedInOut (transaction)	Specifies whether InOut operations (request reply) default to using transacted mode. If this flag is set to true, then Spring JmsTemplate will have sessionTransacted set to true, and the acknowledgeMode as transacted on the JmsTemplate used for InOut operations. Note from Spring JMS: that within a JTA transaction, the parameters passed to createQueue, createTopic methods are not taken into account. Depending on the Java EE transaction context, the container makes its own decisions on these values. Analogously, these parameters are not taken into account within a locally managed transaction either, since Spring JMS operates on an existing JMS Session in this case. Setting this flag to true will use a short local JMS transaction when running outside of a managed transaction, and a synchronized local JMS transaction in case of a managed transaction (other than an XA transaction) being present. This has the effect of a local JMS transaction being managed alongside the main transaction (which might be a native JDBC transaction), with the JMS transaction committing right after the main transaction.	false	boolean
lazyCreateTransactionManager (transaction advanced))	If true, Camel will create a JmsTransactionManager, if there is no transactionManager injected when option transacted=true.	true	boolean
transactionManager (transaction advanced))	The Spring transaction manager to use.		PlatformTransactionManager
transactionName (transaction advanced))	The name of the transaction to use.		String
transactionTimeout (transaction advanced))	The timeout value of the transaction (in seconds), if using transacted mode.	-1	int

2.5. ENDPOINT OPTIONS

The AMQP endpoint is configured using URI syntax:

```
amqp:destinationType:destinationName
```

with the following path and query parameters:

2.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
destinationType (common)	The kind of destination to use. Enum values: • queue • topic • temp-queue • temp-topic	queue	String
destinationName (common)	Required Name of the queue or topic to use as destination.		String

2.5.2. Query Parameters (96 parameters)

Name	Description	Default	Type
clientId (common)	Sets the JMS client ID to use. Note that this value, if specified, must be unique and can only be used by a single JMS connection instance. It is typically only required for durable topic subscriptions with JMS 1.1. The clientId option is compulsory with JMS 1.1 durable topic subscriptions, because the client ID is used to control which client messages have to be stored for. With JMS 2.0 clients, clientId may be omitted, which creates a 'global' subscription.		String
connectionFactory (common)	The connection factory to be use. A connection factory must be configured either on the component or endpoint.		ConnectionFactory
disableReplyTo (common)	Specifies whether Camel ignores the JMSReplyTo header in messages. If true, Camel does not send a reply back to the destination specified in the JMSReplyTo header. You can use this option if you want Camel to consume from a route and you do not want Camel to automatically send back a reply message because another component in your code handles the reply message. You can also use this option if you want to use Camel as a proxy between different message brokers and you want to route message from one system to another.	false	boolean

Name	Description	Default	Type
durableSubscriptionName (common)	The durable subscriber name for specifying durable topic subscriptions. The clientId option must be configured for a JMS 1.1 durable subscription, and may be configured for JMS 2.0, to create a private durable subscription.		String
jmsMessageType (common)	Allows you to force the use of a specific javax.jms.Message implementation for sending JMS messages. Possible values are: Bytes, Map, Object, Stream, Text. By default, Camel would determine which JMS message type to use from the In body type. This option allows you to specify it. Enum values: • Bytes • Map • Object • Stream • Text		JmsMessageType
replyTo (common)	Provides an explicit ReplyTo destination (overrides any incoming value of Message.getJMSReplyTo() in consumer).		String
testConnectionOnStartup (common)	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	boolean

Name	Description	Default	Type
acknowledgmentModeName (consumer)	The JMS acknowledgement name, which is one of: SESSION_TRANSACTED, CLIENT_ACKNOWLEDGE, AUTO_ACKNOWLEDGE, DUPS_OK_ACKNOWLEDGE. Enum values: • SESSION_TRANSACTED • CLIENT_ACKNOWLEDGE • AUTO_ACKNOWLEDGE • DUPS_OK_ACKNOWLEDGE	AUTO_ACKNOWLEDGE	String
artemisConsumerPriority (consumer)	Consumer priorities allow you to ensure that high priority consumers receive messages while they are active. Normally, active consumers connected to a queue receive messages from it in a round-robin fashion. When consumer priorities are in use, messages are delivered round-robin if multiple active consumers exist with the same high priority. Messages will only go to lower priority consumers when the high priority consumers do not have credit available to consume the message, or those high priority consumers have declined to accept the message (for instance because it does not meet the criteria of any selectors associated with the consumer).		int
asyncConsumer (consumer)	Whether the JmsConsumer processes the Exchange asynchronously. If enabled then the JmsConsumer may pickup the next message from the JMS queue, while the previous message is being processed asynchronously (by the Asynchronous Routing Engine). This means that messages may be processed not 100% strictly in order. If disabled (as default) then the Exchange is fully processed before the JmsConsumer picks up the next message from the JMS queue. Note if transacted has been enabled, then asyncConsumer=true does not run asynchronously, as transaction must be executed synchronously (Camel 3.0 may support async transactions).	false	boolean
autoStartup (consumer)	Specifies whether the consumer container should auto-startup.	true	boolean

Name	Description	Default	Type
cacheLevel (consumer)	Sets the cache level by ID for the underlying JMS resources. See <code>cacheLevelName</code> option for more details.		int
cacheLevelName (consumer)	Sets the cache level by name for the underlying JMS resources. Possible values are: <code>CACHE_AUTO</code>, <code>CACHE_CONNECTION</code>, <code>CACHE_CONSUMER</code>, <code>CACHE_NONE</code>, and <code>CACHE_SESSION</code>. The default setting is <code>CACHE_AUTO</code>. See the Spring documentation and Transactions Cache Levels for more information. Enum values: ● <code>CACHE_AUTO</code> ● <code>CACHE_CONNECTION</code> ● <code>CACHE_CONSUMER</code> ● <code>CACHE_NONE</code> ● <code>CACHE_SESSION</code>	<code>CACHE_AUTO</code>	String
concurrentConsumers (consumer)	Specifies the default number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.	1	int
maxConcurrentConsumers (consumer)	Specifies the maximum number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToMaxConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.		int
replyToDeliveryPersistent (consumer)	Specifies whether to use persistent delivery by default for replies.	true	boolean
selector (consumer)	Sets the JMS selector to use.		String

Name	Description	Default	Type
subscriptionDurable (consumer)	Set whether to make the subscription durable. The durable subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a durable subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well.	false	boolean
subscriptionName (consumer)	Set the name of a subscription to create. To be applied in case of a topic (pub-sub domain) with a shared or durable subscription. The subscription name needs to be unique within this client's JMS client id, if the client ID is configured. Default is the class name of the specified message listener. Note: Only 1 concurrent consumer (which is the default of this message listener container) is allowed for each subscription, except for a shared subscription (which requires JMS 2.0).		String
subscriptionShared (consumer)	Set whether to make the subscription shared. The shared subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a shared subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Note that shared subscriptions may also be durable, so this flag can (and often will) be combined with <code>subscriptionDurable</code> as well. Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well. Requires a JMS 2.0 compatible message broker.	false	boolean
acceptMessagesWhileStopping (consumer (advanced))	Specifies whether the consumer accept messages while it is stopping. You may consider enabling this option, if you start and stop JMS routes at runtime, while there are still messages enqueued on the queue. If this option is false, and you stop the JMS route, then messages may be rejected, and the JMS broker would have to attempt redeliveries, which yet again may be rejected, and eventually the message may be moved at a dead letter queue on the JMS broker. To avoid this its recommended to enable this option.	false	boolean

Name	Description	Default	Type
allowReplyManagerQuickStop (consumer (advanced))	Whether the DefaultMessageListenerContainer used in the reply managers for request-reply messaging allow the DefaultMessageListenerContainer.runningAllowed flag to quick stop in case JmsConfiguration#isAcceptMessagesWhileStopping is enabled, and org.apache.camel.CamelContext is currently being stopped. This quick stop ability is enabled by default in the regular JMS consumers but to enable for reply managers you must enable this flag.	false	boolean
consumerType (consumer (advanced))	The consumer type to use, which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use org.springframework.jms.listener.DefaultMessageListenerContainer, Simple will use org.springframework.jms.listener.SimpleMessageListenerContainer. When Custom is specified, the MessageListenerContainerFactory defined by the messageListenerContainerFactory option will determine what org.springframework.jms.listener.AbstractMessageListenerContainer to use. Enum values: • Simple • Default • Custom	Default	ConsumerType

Name	Description	Default	Type
defaultTaskExecutorType (consumer (advanced))	Specifies what default TaskExecutor type to use in the DefaultMessageListenerContainer, for both consumer endpoints and the ReplyTo consumer of producer endpoints. Possible values: SimpleAsync (uses Spring's SimpleAsyncTaskExecutor) or ThreadPool (uses Spring's ThreadPoolTaskExecutor with optimal values - cached threadpool-like). If not set, it defaults to the previous behaviour, which uses a cached thread pool for consumer endpoints and SimpleAsync for reply consumers. The use of ThreadPool is recommended to reduce thread trash in elastic configurations with dynamically increasing and decreasing concurrent consumers. Enum values: • ThreadPool • SimpleAsync		DefaultTaskExecutorType
eagerLoadingOfProperties (consumer (advanced))	Enables eager loading of JMS properties and payload as soon as a message is loaded which generally is inefficient as the JMS properties may not be required but sometimes can catch early any issues with the underlying JMS provider and the use of JMS properties. See also the option eagerPoisonBody.	false	boolean
eagerPoisonBody (consumer (advanced))	If eagerLoadingOfProperties is enabled and the JMS message payload (JMS body or JMS properties) is poison (cannot be read/mapped), then set this text as the message body instead so the message can be processed (the cause of the poison are already stored as exception on the Exchange). This can be turned off by setting eagerPoisonBody=false. See also the option eagerLoadingOfProperties.	Poison JMS message due to \${exception.message}	String
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
exposeListenerSession (consumer (advanced))	Specifies whether the listener session should be exposed when consuming messages.	false	boolean
replyToConsumerType (consumer (advanced))	The consumer type of the reply consumer (when doing request/reply), which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use <code>org.springframework.jms.listener.DefaultMessageListenerContainer</code>, Simple will use <code>org.springframework.jms.listener.SimpleMessageListenerContainer</code>. When Custom is specified, the <code>MessageListenerContainerFactory</code> defined by the <code>messageListenerContainerFactory</code> option will determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use. Enum values: ● Simple ● Default ● Custom	Default	ConsumerType
replyToSameDestinationAllowed (consumer (advanced))	Whether a JMS consumer is allowed to send a reply message to the same destination that the consumer is using to consume from. This prevents an endless loop by consuming and sending back the same message to itself.	false	boolean
taskExecutor (consumer (advanced))	Allows you to specify a custom task executor for consuming messages.		TaskExecutor

Name	Description	Default	Type
deliveryDelay (producer)	Sets delivery delay to use for send calls for JMS. This option requires JMS 2.0 compliant broker.	-1	long
deliveryMode (producer)	Specifies the delivery mode to be used. Possible values are those defined by <code>javax.jms.DeliveryMode</code> . <code>NON_PERSISTENT</code> = 1 and <code>PERSISTENT</code> = 2. Enum values: • 1 • 2		Integer
deliveryPersistent (producer)	Specifies whether persistent delivery is used by default.	true	boolean
explicitQosEnabled (producer)	Set if the <code>deliveryMode</code> , <code>priority</code> or <code>timeToLive</code> qualities of service should be used when sending messages. This option is based on Spring's <code>JmsTemplate</code> . The <code>deliveryMode</code> , <code>priority</code> and <code>timeToLive</code> options are applied to the current endpoint. This contrasts with the <code>preserveMessageQos</code> option, which operates at message granularity, reading QoS properties exclusively from the Camel In message headers.	false	Boolean
formatDateHeadersToIso8601 (producer)	Sets whether JMS date properties should be formatted according to the ISO 8601 standard.	false	boolean
preserveMessageQos (producer)	Set to true, if you want to send message using the QoS settings specified on the message, instead of the QoS settings on the JMS endpoint. The following three headers are considered <code>JMSPriority</code> , <code>JMSDeliveryMode</code> , and <code>JMSExpiration</code> . You can provide all or only some of them. If not provided, Camel will fall back to use the values from the endpoint instead. So, when using this option, the headers override the values from the endpoint. The <code>explicitQosEnabled</code> option, by contrast, will only use options set on the endpoint, and not values from the message header.	false	boolean

Name	Description	Default	Type
priority (producer)	Values greater than 1 specify the message priority when sending (where 1 is the lowest priority and 9 is the highest). The <code>explicitQosEnabled</code> option must also be enabled in order for this option to have any effect. Enum values: • 1 • 2 • 3 • 4 • 5 • 6 • 7 • 8 • 9	4	int
replyToConcurrentConsumers (producer)	Specifies the default number of concurrent consumers when doing request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.	1	int
replyToMaxConcurrentConsumers (producer)	Specifies the maximum number of concurrent consumers when using request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.		int
replyToOnTimeoutMaxConcurrentConsumers (producer)	Specifies the maximum number of concurrent consumers for continue routing when timeout occurred when using request/reply over JMS.	1	int
replyToOverride (producer)	Provides an explicit <code>ReplyTo</code> destination in the JMS message, which overrides the setting of <code>replyTo</code> . It is useful if you want to forward the message to a remote Queue and receive the reply message from the <code>ReplyTo</code> destination.		String

Name	Description	Default	Type
replyToType (producer)	Allows for explicitly specifying which kind of strategy to use for replyTo queues when doing request/reply over JMS. Possible values are: Temporary, Shared, or Exclusive. By default Camel will use temporary queues. However if replyTo has been configured, then Shared is used by default. This option allows you to use exclusive queues instead of shared ones. See Camel JMS documentation for more details, and especially the notes about the implications if running in a clustered environment, and the fact that Shared reply queues has lower performance than its alternatives Temporary and Exclusive. Enum values: • Temporary • Shared • Exclusive		ReplyToType
requestTimeout (producer)	The timeout for waiting for a reply when using the InOut Exchange Pattern (in milliseconds). The default is 20 seconds. You can include the header CamelJmsRequestTimeout to override this endpoint configured timeout value, and thus have per message individual timeout values. See also the requestTimeoutCheckerInterval option.	20000	long
timeToLive (producer)	When sending messages, specifies the time-to-live of the message (in milliseconds).	-1	long
allowAdditionalHeaders (producer (advanced))	This option is used to allow additional headers which may have values that are invalid according to JMS specification. For example some message systems such as WMQ do this with header names using prefix JMS_IBM_MQMD_ containing values with byte array or other invalid types. You can specify multiple header names separated by comma, and use as suffix for wildcard matching.		String
allowNullBody (producer (advanced))	Whether to allow sending messages with no body. If this option is false and the message body is null, then an JMSEException is thrown.	true	boolean

Name	Description	Default	Type
alwaysCopyMessage (producer (advanced))	If true, Camel will always make a JMS message copy of the message when it is passed to the producer for sending. Copying the message is needed in some situations, such as when a <code>replyToDestinationSelectorName</code> is set (incidentally, Camel will set the <code>alwaysCopyMessage</code> option to true, if a <code>replyToDestinationSelectorName</code> is set).	false	boolean
correlationProperty (producer (advanced))	When using <code>InOut</code> exchange pattern use this JMS property instead of <code>JMSCorrelationID</code> JMS property to correlate messages. If set messages will be correlated solely on the value of this property <code>JMSCorrelationID</code> property will be ignored and not set by Camel.		String
disableTimeToLive (producer (advanced))	Use this option to force disabling time to live. For example when you do request/reply over JMS, then Camel will by default use the <code>requestTimeout</code> value as time to live on the message being sent. The problem is that the sender and receiver systems have to have their clocks synchronized, so they are in sync. This is not always so easy to archive. So you can use <code>disableTimeToLive=true</code> to not set a time to live value on the sent message. Then the message will not expire on the receiver system. See below in section About time to live for more details.	false	boolean
forceSendOriginalMessage (producer (advanced))	When using <code>mapJmsMessage=false</code> Camel will create a new JMS message to send to a new JMS destination if you touch the headers (get or set) during the route. Set this option to true to force Camel to send the original JMS message that was received.	false	boolean
includeSentJMSMessageID (producer (advanced))	Only applicable when sending to JMS destination using <code>InOnly</code> (eg fire and forget). Enabling this option will enrich the Camel Exchange with the actual <code>JMSMessageID</code> that was used by the JMS client when the message was sent to the JMS destination.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
replyToCacheLevelName (producer (advanced))	Sets the cache level by name for the reply consumer when doing request/reply over JMS. This option only applies when using fixed reply queues (not temporary). Camel will by default use: <code>CACHE_CONSUMER</code> for exclusive or shared w/ <code>replyToSelectorName</code>. And <code>CACHE_SESSION</code> for shared without <code>replyToSelectorName</code>. Some JMS brokers such as IBM WebSphere may require to set the <code>replyToCacheLevelName=CACHE_NONE</code> to work. Note: If using temporary queues then <code>CACHE_NONE</code> is not allowed, and you must use a higher value such as <code>CACHE_CONSUMER</code> or <code>CACHE_SESSION</code>. Enum values: ● <code>CACHE_AUTO</code> ● <code>CACHE_CONNECTION</code> ● <code>CACHE_CONSUMER</code> ● <code>CACHE_NONE</code> ● <code>CACHE_SESSION</code>		String
replyToDestinationSelectorName (producer (advanced))	Sets the JMS Selector using the fixed name to be used so you can filter out your own replies from the others when using a shared queue (that is, if you are not using a temporary reply queue).		String

Name	Description	Default	Type
streamMessageTypeEnabled (producer (advanced))	Sets whether StreamMessage type is enabled or not. Message payloads of streaming kind such as files, InputStream, etc will either be sent as BytesMessage or StreamMessage. This option controls which kind will be used. By default BytesMessage is used which enforces the entire message payload to be read into memory. By enabling this option the message payload is read into memory in chunks and each chunk is then written to the StreamMessage until no more data.	false	boolean
allowSerializedHeaders (advanced)	Controls whether or not to include serialized headers. Applies only when transferExchange is true. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
artemisStreamingEnabled (advanced)	Whether optimizing for Apache Artemis streaming mode. This can reduce memory overhead when using Artemis with JMS StreamMessage types. This option must only be enabled if Apache Artemis is being used.	false	boolean
asyncStartListener (advanced)	Whether to startup the JmsConsumer message listener asynchronously, when starting a route. For example if a JmsConsumer cannot get a connection to a remote JMS broker, then it may block while retrying and/or failover. This will cause Camel to block while starting routes. By setting this option to true, you will let routes startup, while the JmsConsumer connects to the JMS broker using a dedicated thread in asynchronous mode. If this option is used, then beware that if the connection could not be established, then an exception is logged at WARN level, and the consumer will not be able to receive messages; You can then restart the route to retry.	false	boolean
asyncStopListener (advanced)	Whether to stop the JmsConsumer message listener asynchronously, when stopping a route.	false	boolean
destinationResolver (advanced)	A pluggable org.springframework.jms.support.destination.DestinationResolver that allows you to use your own resolver (for example, to lookup the real destination in a JNDI registry).		DestinationResolver

Name	Description	Default	Type
errorHandler (advanced)	Specifies a <code>org.springframework.util.ErrorHandler</code> to be invoked in case of any uncaught exceptions thrown while processing a Message. By default these exceptions will be logged at the WARN level, if no errorHandler has been configured. You can configure logging level and whether stack traces should be logged using <code>errorHandlerLoggingLevel</code> and <code>errorHandlerLogStackTrace</code> options. This makes it much easier to configure, than having to code a custom errorHandler.		ErrorHandler
exceptionListener (advanced)	Specifies the JMS Exception Listener that is to be notified of any underlying JMS exceptions.		ExceptionListener
headerFilterStrategy (advanced)	To use a custom HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
idleConsumerLimit (advanced)	Specify the limit for the number of consumers that are allowed to be idle at any given time.	1	int
idleTaskExecutionLimit (advanced)	Specifies the limit for idle executions of a receive task, not having received any message within its execution. If this limit is reached, the task will shut down and leave receiving to other executing tasks (in the case of dynamic scheduling; see the <code>maxConcurrentConsumers</code> setting). There is additional doc available from Spring.	1	int
includeAllJMSXProperties (advanced)	Whether to include all JMSxxxx properties when mapping from JMS to Camel Message. Setting this to true will include properties such as <code>JMSXAppID</code> , and <code>JMSXUserID</code> etc. Note: If you are using a custom headerFilterStrategy then this option does not apply.	false	boolean

Name	Description	Default	Type
jmsKeyFormatStrategy (advanced)	Pluggable strategy for encoding and decoding JMS keys so they can be compliant with the JMS specification. Camel provides two implementations out of the box: default and passthrough. The default strategy will safely marshal dots and hyphens (. and -). The passthrough strategy leaves the key as is. Can be used for JMS brokers which do not care whether JMS header keys contain illegal characters. You can provide your own implementation of the <code>org.apache.camel.component.jms.JmsKeyFormatStrategy</code> and refer to it using the <code>#</code> notation. Enum values: • default • passthrough		JmsKeyFormatStrategy
mapJmsMessage (advanced)	Specifies whether Camel should auto map the received JMS message to a suited payload type, such as <code>javax.jms.TextMessage</code> to a <code>String</code> etc.	true	boolean
maxMessagesPerTask (advanced)	The number of messages per task. -1 is unlimited. If you use a range for concurrent consumers (eg min max), then this option can be used to set a value to eg 100 to control how fast the consumers will shrink when less work is required.	-1	int
messageConverter (advanced)	To use a custom Spring <code>org.springframework.jms.support.converter.MessageConverter</code> so you can be in control how to map to/from a <code>javax.jms.Message</code> .		MessageConverter
messageCreatedStrategy (advanced)	To use the given <code>MessageCreatedStrategy</code> which are invoked when Camel creates new instances of <code>javax.jms.Message</code> objects when Camel is sending a JMS message.		MessageCreatedStrategy
messageIdEnabled (advanced)	When sending, specifies whether message IDs should be added. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the message ID set to null; if the provider ignores the hint, the message ID must be set to its normal unique value.	true	boolean

Name	Description	Default	Type
messageListenerContainerFactory (advanced)	Registry ID of the MessageListenerContainerFactory used to determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use to consume messages. Setting this will automatically set <code>consumerType</code> to Custom.		MessageListenerContainerFactory
messageTimestampEnabled (advanced)	Specifies whether timestamps should be enabled by default on sending messages. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the timestamp set to zero; if the provider ignores the hint the timestamp must be set to its normal value.	true	boolean
pubSubNoLocal (advanced)	Specifies whether to inhibit the delivery of messages published by its own connection.	false	boolean
receiveTimeout (advanced)	The timeout for receiving messages (in milliseconds).	1000	long
recoveryInterval (advanced)	Specifies the interval between recovery attempts, i.e. when a connection is being refreshed, in milliseconds. The default is 5000 ms, that is, 5 seconds.	5000	long
requestTimeoutCheckerInterval (advanced)	Configures how often Camel should check for timed out Exchanges when doing request/reply over JMS. By default Camel checks once per second. But if you must react faster when a timeout occurs, then you can lower this interval, to check more frequently. The timeout is determined by the option <code>requestTimeout</code> .	1000	long
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean

Name	Description	Default	Type
transferException (advanced)	If enabled and you are using Request Reply messaging (InOut) and an Exchange failed on the consumer side, then the caused Exception will be send back in response as a javax.jms.ObjectMessage. If the client is Camel, the returned Exception is rethrown. This allows you to use Camel JMS as a bridge in your routing - for example, using persistent queues to enable robust routing. Notice that if you also have transferExchange enabled, this option takes precedence. The caught exception is required to be serializable. The original Exception on the consumer side can be wrapped in an outer exception such as org.apache.camel.RuntimeCamelException when returned to the producer. Use this with caution as the data is using Java Object serialization and requires the received to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumer!.	false	boolean
transferExchange (advanced)	You can transfer the exchange over the wire instead of just the body and headers. The following fields are transferred: In body, Out body, Fault body, In headers, Out headers, Fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level. You must enable this option on both the producer and consumer side, so Camel knows the payloads is an Exchange and not a regular payload. Use this with caution as the data is using Java Object serialization and requires the receiver to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumers having to use compatible Camel versions!.	false	boolean
useMessageIDAsCorrelationID (advanced)	Specifies whether JMSMessageID should always be used as JMSCorrelationID for InOut messages.	false	boolean
waitForProvisionCorrelationToBeUpdatedCounter (advanced)	Number of times to wait for provisional correlation id to be updated to the actual correlation id when doing request/reply over JMS and when the option useMessageIDAsCorrelationID is enabled.	50	int
waitForProvisionCorrelationToBeUpdatedThreadSleepingTime (advanced)	Interval in millis to sleep each time while waiting for provisional correlation id to be updated.	100	long

Name	Description	Default	Type
errorHandlerLoggingLevel (logging)	Allows to configure the default errorHandler logging level for logging uncaught exceptions. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	WARN	LogLevel
errorHandlerLogStackTrace (logging)	Allows to control whether stacktraces should be logged or not, by the default errorHandler.	true	boolean
password (security)	Password to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
username (security)	Username to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
transacted (transaction)	Specifies whether to use transacted mode.	false	boolean

Name	Description	Default	Type
transactedInOut (transaction)	Specifies whether InOut operations (request reply) default to using transacted mode. If this flag is set to true, then Spring JmsTemplate will have sessionTransacted set to true, and the acknowledgeMode as transacted on the JmsTemplate used for InOut operations. Note from Spring JMS: that within a JTA transaction, the parameters passed to createQueue, createTopic methods are not taken into account. Depending on the Java EE transaction context, the container makes its own decisions on these values. Analogously, these parameters are not taken into account within a locally managed transaction either, since Spring JMS operates on an existing JMS Session in this case. Setting this flag to true will use a short local JMS transaction when running outside of a managed transaction, and a synchronized local JMS transaction in case of a managed transaction (other than an XA transaction) being present. This has the effect of a local JMS transaction being managed alongside the main transaction (which might be a native JDBC transaction), with the JMS transaction committing right after the main transaction.	false	boolean
lazyCreateTransactionManager (transaction (advanced))	If true, Camel will create a JmsTransactionManager, if there is no transactionManager injected when option transacted=true.	true	boolean
transactionManager (transaction (advanced))	The Spring transaction manager to use.		PlatformTransactionManager
transactionName (transaction (advanced))	The name of the transaction to use.		String
transactionTimeout (transaction (advanced))	The timeout value of the transaction (in seconds), if using transacted mode.	-1	int

2.6. USAGE

As AMQP component is inherited from JMS component, the usage of the former is almost identical to the latter:

Using AMQP component


```
// Consuming from AMQP queue
from("amqp:queue:incoming").
  to(...);

// Sending message to the AMQP topic
from(...).
  to("amqp:topic:notify");
```

2.7. CONFIGURING AMQP COMPONENT

Creating AMQP 1.0 component

```
AMQPComponent amqp = AMQPComponent.amqpComponent("amqp://localhost:5672");

AMQPComponent authorizedAmqp = AMQPComponent.amqpComponent("amqp://localhost:5672",
  "user", "password");
```

You can also add an instance of **org.apache.camel.component.amqp.AMQPConnectionDetails** to the registry in order to automatically configure the AMQP component. For example for Spring Boot you just have to define bean:

AMQP connection details auto-configuration

```
@Bean
AMQPConnectionDetails amqpConnection() {
    return new AMQPConnectionDetails("amqp://localhost:5672");
}

@Bean
AMQPConnectionDetails securedAmqpConnection() {
    return new AMQPConnectionDetails("amqp://localhost:5672", "username", "password");
}
```

Likewise, you can also use CDI producer methods when using Camel-CDI

AMQP connection details auto-configuration for CDI

```
@Produces
AMQPConnectionDetails amqpConnection() {
    return new AMQPConnectionDetails("amqp://localhost:5672");
}
```

You can also rely on the to read the AMQP connection details. Factory method **AMQPConnectionDetails.discoverAMQP()** attempts to read Camel properties in a Kubernetes-like convention, just as demonstrated on the snippet below:

AMQP connection details auto-configuration

```
export AMQP_SERVICE_HOST = "mybroker.com"
export AMQP_SERVICE_PORT = "6666"
export AMQP_SERVICE_USERNAME = "username"
export AMQP_SERVICE_PASSWORD = "password"
```

```
...
@Bean
AMQPConnectionDetails amqpConnection() {
    return AMQPConnectionDetails.discoverAMQP();
}
```

Enabling AMQP specific options

If you, for example, need to enable **amqp.traceFrames** you can do that by appending the option to your URI, like the following example:

```
AMQPComponent amqp = AMQPComponent.amqpComponent("amqp://localhost:5672?
amqp.traceFrames=true");
```

For reference refer [QPID JMS client configuration](#).

2.8. USING TOPICS

To have using topics working with **camel-amqp** you need to configure the component to use **topic://** as topic prefix, as shown below:

```
<bean id="amqp" class="org.apache.camel.component.amqp.AmqpComponent">
  <property name="connectionFactory">
    <bean class="org.apache.qpid.jms.JmsConnectionFactory" factory-method="createFromURL">
      <property name="remoteURI" value="amqp://localhost:5672" />
      <property name="topicPrefix" value="topic://" /> <!-- only necessary when connecting to
ActiveMQ over AMQP 1.0 -->
    </bean>
  </property>
</bean>
```

Keep in mind that both **AMQPComponent#amqpComponent()** methods and **AMQPConnectionDetails** pre-configure the component with the topic prefix, so you don't have to configure it explicitly.

2.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 101 options, which are listed below.

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
camel.component.amqp.accept-messages-while-stopping	Specifies whether the consumer accept messages while it is stopping. You may consider enabling this option, if you start and stop JMS routes at runtime, while there are still messages enqueued on the queue. If this option is false, and you stop the JMS route, then messages may be rejected, and the JMS broker would have to attempt redeliveries, which yet again may be rejected, and eventually the message may be moved at a dead letter queue on the JMS broker. To avoid this its recommended to enable this option.	false	Boolean
camel.component.amqp.acknowledgement-mode-name	The JMS acknowledgement name, which is one of: SESSION_TRANSACTED, CLIENT_ACKNOWLEDGE, AUTO_ACKNOWLEDGE, DUPS_OK_ACKNOWLEDGE.	AUTO_ACKNOWLEDGE	String
camel.component.amqp.allow-additional-headers	This option is used to allow additional headers which may have values that are invalid according to JMS specification. For example some message systems such as WMQ do this with header names using prefix JMS_IBM_MQMD_ containing values with byte array or other invalid types. You can specify multiple header names separated by comma, and use as suffix for wildcard matching.		String
camel.component.amqp.allow-auto-wired-connection-factory	Whether to auto-discover ConnectionFactory from the registry, if no connection factory has been configured. If only one instance of ConnectionFactory is found then it will be used. This is enabled by default.	true	Boolean
camel.component.amqp.allow-auto-wired-destination-resolver	Whether to auto-discover DestinationResolver from the registry, if no destination resolver has been configured. If only one instance of DestinationResolver is found then it will be used. This is enabled by default.	true	Boolean
camel.component.amqp.allow-null-body	Whether to allow sending messages with no body. If this option is false and the message body is null, then an JMSEException is thrown.	true	Boolean

Name	Description	Default	Type
camel.component.amqp.allow-reply-manager-quick-stop	Whether the DefaultMessageListenerContainer used in the reply managers for request-reply messaging allow the DefaultMessageListenerContainer.runningAllowed flag to quick stop in case JmsConfiguration#isAcceptMessagesWhileStopping is enabled, and org.apache.camel.CamelContext is currently being stopped. This quick stop ability is enabled by default in the regular JMS consumers but to enable for reply managers you must enable this flag.	false	Boolean
camel.component.amqp.allow-serialized-headers	Controls whether or not to include serialized headers. Applies only when transferExchange is true. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	Boolean
camel.component.amqp.always-copy-message	If true, Camel will always make a JMS message copy of the message when it is passed to the producer for sending. Copying the message is needed in some situations, such as when a replyToDestinationSelectorName is set (incidentally, Camel will set the alwaysCopyMessage option to true, if a replyToDestinationSelectorName is set).	false	Boolean
camel.component.amqp.artemis-consumer-priority	Consumer priorities allow you to ensure that high priority consumers receive messages while they are active. Normally, active consumers connected to a queue receive messages from it in a round-robin fashion. When consumer priorities are in use, messages are delivered round-robin if multiple active consumers exist with the same high priority. Messages will only going to lower priority consumers when the high priority consumers do not have credit available to consume the message, or those high priority consumers have declined to accept the message (for instance because it does not meet the criteria of any selectors associated with the consumer).		Integer
camel.component.amqp.artemis-streaming-enabled	Whether optimizing for Apache Artemis streaming mode. This can reduce memory overhead when using Artemis with JMS StreamMessage types. This option must only be enabled if Apache Artemis is being used.	false	Boolean

Name	Description	Default	Type
camel.component.amqp.async-consumer	Whether the JmsConsumer processes the Exchange asynchronously. If enabled then the JmsConsumer may pickup the next message from the JMS queue, while the previous message is being processed asynchronously (by the Asynchronous Routing Engine). This means that messages may be processed not 100% strictly in order. If disabled (as default) then the Exchange is fully processed before the JmsConsumer picks up the next message from the JMS queue. Note if transacted has been enabled, then <code>asyncConsumer=true</code> does not run asynchronously, as transaction must be executed synchronously (Camel 3.0 may support async transactions).	false	Boolean
camel.component.amqp.async-start-listener	Whether to startup the JmsConsumer message listener asynchronously, when starting a route. For example if a JmsConsumer cannot get a connection to a remote JMS broker, then it may block while retrying and/or failover. This will cause Camel to block while starting routes. By setting this option to true, you will let routes startup, while the JmsConsumer connects to the JMS broker using a dedicated thread in asynchronous mode. If this option is used, then beware that if the connection could not be established, then an exception is logged at WARN level, and the consumer will not be able to receive messages; You can then restart the route to retry.	false	Boolean
camel.component.amqp.async-stop-listener	Whether to stop the JmsConsumer message listener asynchronously, when stopping a route.	false	Boolean
camel.component.amqp.auto-startup	Specifies whether the consumer container should auto-startup.	true	Boolean
camel.component.amqp.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.amqp.cache-level	Sets the cache level by ID for the underlying JMS resources. See <code>cacheLevelName</code> option for more details.		Integer
camel.component.amqp.cache-level-name	Sets the cache level by name for the underlying JMS resources. Possible values are: <code>CACHE_AUTO</code> , <code>CACHE_CONNECTION</code> , <code>CACHE_CONSUMER</code> , <code>CACHE_NONE</code> , and <code>CACHE_SESSION</code> . The default setting is <code>CACHE_AUTO</code> . See the Spring documentation and Transactions Cache Levels for more information.	<code>CACHE_AUTO</code>	String
camel.component.amqp.client-id	Sets the JMS client ID to use. Note that this value, if specified, must be unique and can only be used by a single JMS connection instance. It is typically only required for durable topic subscriptions with JMS 1.1.		String
camel.component.amqp.concurrent-consumers	Specifies the default number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.	1	Integer
camel.component.amqp.configuration	To use a shared JMS configuration. The option is a <code>org.apache.camel.component.jms.JmsConfiguration</code> type.		<code>JmsConfiguration</code>
camel.component.amqp.connection-factory	The connection factory to be use. A connection factory must be configured either on the component or endpoint. The option is a <code>javax.jms.ConnectionFactory</code> type.		<code>ConnectionFactory</code>
camel.component.amqp.consumer-type	The consumer type to use, which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use <code>org.springframework.jms.listener.DefaultMessageListenerContainer</code> , Simple will use <code>org.springframework.jms.listener.SimpleMessageListenerContainer</code> . When Custom is specified, the <code>MessageListenerContainerFactory</code> defined by the <code>messageListenerContainerFactory</code> option will determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use.		<code>ConsumerType</code>

Name	Description	Default	Type
camel.component.amqp.correlation-property	When using InOut exchange pattern use this JMS property instead of JMSCorrelationID JMS property to correlate messages. If set messages will be correlated solely on the value of this property JMSCorrelationID property will be ignored and not set by Camel.		String
camel.component.amqp.default-task-executor-type	Specifies what default TaskExecutor type to use in the DefaultMessageListenerContainer, for both consumer endpoints and the ReplyTo consumer of producer endpoints. Possible values: SimpleAsync (uses Spring's SimpleAsyncTaskExecutor) or ThreadPool (uses Spring's ThreadPoolTaskExecutor with optimal values - cached threadpool-like). If not set, it defaults to the previous behaviour, which uses a cached thread pool for consumer endpoints and SimpleAsync for reply consumers. The use of ThreadPool is recommended to reduce thread trash in elastic configurations with dynamically increasing and decreasing concurrent consumers.		DefaultTaskExecutorType
camel.component.amqp.delivery-delay	Sets delivery delay to use for send calls for JMS. This option requires JMS 2.0 compliant broker.	-1	Long
camel.component.amqp.delivery-mode	Specifies the delivery mode to be used. Possible values are those defined by javax.jms.DeliveryMode. NON_PERSISTENT = 1 and PERSISTENT = 2.		Integer
camel.component.amqp.delivery-persistent	Specifies whether persistent delivery is used by default.	true	Boolean
camel.component.amqp.destination-resolver	A pluggable org.springframework.jms.support.destination.DestinationResolver that allows you to use your own resolver (for example, to lookup the real destination in a JNDI registry). The option is a org.springframework.jms.support.destination.DestinationResolver type.		DestinationResolver

Name	Description	Default	Type
camel.component.amqp.disable-reply-to	Specifies whether Camel ignores the JMSReplyTo header in messages. If true, Camel does not send a reply back to the destination specified in the JMSReplyTo header. You can use this option if you want Camel to consume from a route and you do not want Camel to automatically send back a reply message because another component in your code handles the reply message. You can also use this option if you want to use Camel as a proxy between different message brokers and you want to route message from one system to another.	false	Boolean
camel.component.amqp.disable-time-to-live	Use this option to force disabling time to live. For example when you do request/reply over JMS, then Camel will by default use the requestTimeout value as time to live on the message being sent. The problem is that the sender and receiver systems have to have their clocks synchronized, so they are in sync. This is not always so easy to archive. So you can use disableTimeToLive=true to not set a time to live value on the sent message. Then the message will not expire on the receiver system. See below in section About time to live for more details.	false	Boolean
camel.component.amqp.durable-subscription-name	The durable subscriber name for specifying durable topic subscriptions. The clientId option must be configured as well.		String
camel.component.amqp.eager-loading-of-properties	Enables eager loading of JMS properties and payload as soon as a message is loaded which generally is inefficient as the JMS properties may not be required but sometimes can catch early any issues with the underlying JMS provider and the use of JMS properties. See also the option eagerPoisonBody.	false	Boolean
camel.component.amqp.eager-poison-body	If eagerLoadingOfProperties is enabled and the JMS message payload (JMS body or JMS properties) is poison (cannot be read/mapped), then set this text as the message body instead so the message can be processed (the cause of the poison are already stored as exception on the Exchange). This can be turned off by setting eagerPoisonBody=false. See also the option eagerLoadingOfProperties.	Poison JMS message due to \${exception.message}	String
camel.component.amqp.enabled	Whether to enable auto configuration of the amqp component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.amqp.error-handler	Specifies a <code>org.springframework.util.ErrorHandler</code> to be invoked in case of any uncaught exceptions thrown while processing a <code>Message</code> . By default these exceptions will be logged at the <code>WARN</code> level, if no <code>ErrorHandler</code> has been configured. You can configure logging level and whether stack traces should be logged using <code>errorHandlerLoggingLevel</code> and <code>errorHandlerLogStackTrace</code> options. This makes it much easier to configure, than having to code a custom <code>ErrorHandler</code> . The option is a <code>org.springframework.util.ErrorHandler</code> type.		<code>ErrorHandler</code>
camel.component.amqp.error-handler-log-stack-trace	Allows to control whether stacktraces should be logged or not, by the default <code>ErrorHandler</code> .	<code>true</code>	<code>Boolean</code>
camel.component.amqp.error-handler-logging-level	Allows to configure the default <code>ErrorHandler</code> logging level for logging uncaught exceptions.		<code>LoggingLevel</code>
camel.component.amqp.exception-listener	Specifies the <code>JMS Exception Listener</code> that is to be notified of any underlying <code>JMS</code> exceptions. The option is a <code>javax.jms.ExceptionListener</code> type.		<code>ExceptionListener</code>
camel.component.amqp.explicit-qos-enabled	Set if the <code>deliveryMode</code> , <code>priority</code> or <code>timeToLive</code> qualities of service should be used when sending messages. This option is based on Spring's <code>JmsTemplate</code> . The <code>deliveryMode</code> , <code>priority</code> and <code>timeToLive</code> options are applied to the current endpoint. This contrasts with the <code>preserveMessageQos</code> option, which operates at message granularity, reading <code>QoS</code> properties exclusively from the Camel In message headers.	<code>false</code>	<code>Boolean</code>
camel.component.amqp.expose-listener-session	Specifies whether the listener session should be exposed when consuming messages.	<code>false</code>	<code>Boolean</code>
camel.component.amqp.force-send-original-message	When using <code>mapJmsMessage=false</code> Camel will create a new <code>JMS</code> message to send to a new <code>JMS</code> destination if you touch the headers (get or set) during the route. Set this option to <code>true</code> to force Camel to send the original <code>JMS</code> message that was received.	<code>false</code>	<code>Boolean</code>

Name	Description	Default	Type
camel.component.amqp.format-date-headers-to-iso8601	Sets whether JMS date properties should be formatted according to the ISO 8601 standard.	false	Boolean
camel.component.amqp.header-filter-strategy	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		HeaderFilterStrategy
camel.component.amqp.idle-consumer-limit	Specify the limit for the number of consumers that are allowed to be idle at any given time.	1	Integer
camel.component.amqp.idle-task-execution-limit	Specifies the limit for idle executions of a receive task, not having received any message within its execution. If this limit is reached, the task will shut down and leave receiving to other executing tasks (in the case of dynamic scheduling; see the <code>maxConcurrentConsumers</code> setting). There is additional doc available from Spring.	1	Integer
camel.component.amqp.include-all-jmsx-properties	Whether to include all JMSXxxx properties when mapping from JMS to Camel Message. Setting this to true will include properties such as JMSXAppID, and JMSXUserID etc. Note: If you are using a custom <code>headerFilterStrategy</code> then this option does not apply.	false	Boolean
camel.component.amqp.include-amqp-annotations	Whether to include AMQP annotations when mapping from AMQP to Camel Message. Setting this to true maps AMQP message annotations that contain a <code>JMS_AMQP_MA_</code> prefix to message headers. Due to limitations in Apache Qpid JMS API, currently delivery annotations are ignored.	false	Boolean
camel.component.amqp.include-sent-jms-message-id	Only applicable when sending to JMS destination using <code>InOnly</code> (eg fire and forget). Enabling this option will enrich the Camel Exchange with the actual <code>JMSMessageID</code> that was used by the JMS client when the message was sent to the JMS destination.	false	Boolean

Name	Description	Default	Type
camel.component.amqp.jms-key-format-strategy	Pluggable strategy for encoding and decoding JMS keys so they can be compliant with the JMS specification. Camel provides two implementations out of the box: default and passthrough. The default strategy will safely marshal dots and hyphens (. and -). The passthrough strategy leaves the key as is. Can be used for JMS brokers which do not care whether JMS header keys contain illegal characters. You can provide your own implementation of the <code>org.apache.camel.component.jms.JmsKeyFormatStrategy</code> and refer to it using the <code>#</code> notation.		JmsKeyFormatStrategy
camel.component.amqp.jms-message-type	Allows you to force the use of a specific <code>javax.jms.Message</code> implementation for sending JMS messages. Possible values are: Bytes, Map, Object, Stream, Text. By default, Camel would determine which JMS message type to use from the In body type. This option allows you to specify it.		JmsMessageType
camel.component.amqp.lazy-create-transaction-manager	If true, Camel will create a <code>JmsTransactionManager</code> , if there is no <code>transactionManager</code> injected when option <code>transacted=true</code> .	true	Boolean
camel.component.amqp.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.amqp.map-jms-message	Specifies whether Camel should auto map the received JMS message to a suited payload type, such as <code>javax.jms.TextMessage</code> to a String etc.	true	Boolean

Name	Description	Default	Type
camel.component.amqp.max-concurrent-consumers	Specifies the maximum number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToMaxConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.		Integer
camel.component.amqp.max-messages-per-task	The number of messages per task. -1 is unlimited. If you use a range for concurrent consumers (eg min max), then this option can be used to set a value to eg 100 to control how fast the consumers will shrink when less work is required.	-1	Integer
camel.component.amqp.message-converter	To use a custom Spring <code>org.springframework.jms.support.converter.MessageConverter</code> so you can be in control how to map to/from a <code>javax.jms.Message</code> . The option is a <code>org.springframework.jms.support.converter.MessageConverter</code> type.		<code>MessageConverter</code>
camel.component.amqp.message-created-strategy	To use the given <code>MessageCreatedStrategy</code> which are invoked when Camel creates new instances of <code>javax.jms.Message</code> objects when Camel is sending a JMS message. The option is a <code>org.apache.camel.component.jms.MessageCreatedStrategy</code> type.		<code>MessageCreatedStrategy</code>
camel.component.amqp.message-id-enabled	When sending, specifies whether message IDs should be added. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the message ID set to null; if the provider ignores the hint, the message ID must be set to its normal unique value.	true	Boolean
camel.component.amqp.message-listener-container-factory	Registry ID of the <code>MessageListenerContainerFactory</code> used to determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use to consume messages. Setting this will automatically set <code>consumerType</code> to Custom. The option is a <code>org.apache.camel.component.jms.MessageListenerContainerFactory</code> type.		<code>MessageListenerContainerFactory</code>

Name	Description	Default	Type
camel.component.amqp.message-timestamp-enabled	Specifies whether timestamps should be enabled by default on sending messages. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the timestamp set to zero; if the provider ignores the hint the timestamp must be set to its normal value.	true	Boolean
camel.component.amqp.password	Password to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
camel.component.amqp.preserve-message-qos	Set to true, if you want to send message using the QoS settings specified on the message, instead of the QoS settings on the JMS endpoint. The following three headers are considered JMSPriority, JMSDeliveryMode, and JMSExpiration. You can provide all or only some of them. If not provided, Camel will fall back to use the values from the endpoint instead. So, when using this option, the headers override the values from the endpoint. The explicitQosEnabled option, by contrast, will only use options set on the endpoint, and not values from the message header.	false	Boolean
camel.component.amqp.priority	Values greater than 1 specify the message priority when sending (where 1 is the lowest priority and 9 is the highest). The explicitQosEnabled option must also be enabled in order for this option to have any effect.	4	Integer
camel.component.amqp.pub-sub-no-local	Specifies whether to inhibit the delivery of messages published by its own connection.	false	Boolean
camel.component.amqp.queue-browse-strategy	To use a custom QueueBrowseStrategy when browsing queues. The option is a org.apache.camel.component.jms.QueueBrowseStrategy type.		QueueBrowseStrategy
camel.component.amqp.receive-timeout	The timeout for receiving messages (in milliseconds). The option is a long type.	1000	Long
camel.component.amqp.recovery-interval	Specifies the interval between recovery attempts, i.e. when a connection is being refreshed, in milliseconds. The default is 5000 ms, that is, 5 seconds. The option is a long type.	5000	Long

Name	Description	Default	Type
camel.component.amqp.reply-to	Provides an explicit ReplyTo destination (overrides any incoming value of <code>Message.getJMSReplyTo()</code> in consumer).		String
camel.component.amqp.reply-to-cache-level-name	Sets the cache level by name for the reply consumer when doing request/reply over JMS. This option only applies when using fixed reply queues (not temporary). Camel will by default use: <code>CACHE_CONSUMER</code> for exclusive or shared w/ <code>replyToSelectorName</code> . And <code>CACHE_SESSION</code> for shared without <code>replyToSelectorName</code> . Some JMS brokers such as IBM WebSphere may require to set the <code>replyToCacheLevelName=CACHE_NONE</code> to work. Note: If using temporary queues then <code>CACHE_NONE</code> is not allowed, and you must use a higher value such as <code>CACHE_CONSUMER</code> or <code>CACHE_SESSION</code> .		String
camel.component.amqp.reply-to-concurrent-consumers	Specifies the default number of concurrent consumers when doing request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.	1	Integer
camel.component.amqp.reply-to-consumer-type	The consumer type of the reply consumer (when doing request/reply), which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use <code>org.springframework.jms.listener.DefaultMessageListenerContainer</code> , Simple will use <code>org.springframework.jms.listener.SimpleMessageListenerContainer</code> . When Custom is specified, the <code>MessageListenerContainerFactory</code> defined by the <code>messageListenerContainerFactory</code> option will determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use.		ConsumerType
camel.component.amqp.reply-to-delivery-persistent	Specifies whether to use persistent delivery by default for replies.	true	Boolean
camel.component.amqp.reply-to-destination-selector-name	Sets the JMS Selector using the fixed name to be used so you can filter out your own replies from the others when using a shared queue (that is, if you are not using a temporary reply queue).		String

Name	Description	Default	Type
camel.component.amqp.reply-to-max-concurrent-consumers	Specifies the maximum number of concurrent consumers when using request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.		Integer
camel.component.amqp.reply-to-on-timeout-max-concurrent-consumers	Specifies the maximum number of concurrent consumers for continue routing when timeout occurred when using request/reply over JMS.	1	Integer
camel.component.amqp.reply-to-override	Provides an explicit <code>ReplyTo</code> destination in the JMS message, which overrides the setting of <code>replyTo</code> . It is useful if you want to forward the message to a remote Queue and receive the reply message from the <code>ReplyTo</code> destination.		String
camel.component.amqp.reply-to-same-destination-allowed	Whether a JMS consumer is allowed to send a reply message to the same destination that the consumer is using to consume from. This prevents an endless loop by consuming and sending back the same message to itself.	false	Boolean
camel.component.amqp.reply-to-type	Allows for explicitly specifying which kind of strategy to use for <code>replyTo</code> queues when doing request/reply over JMS. Possible values are: <code>Temporary</code> , <code>Shared</code> , or <code>Exclusive</code> . By default Camel will use temporary queues. However if <code>replyTo</code> has been configured, then <code>Shared</code> is used by default. This option allows you to use exclusive queues instead of shared ones. See Camel JMS documentation for more details, and especially the notes about the implications if running in a clustered environment, and the fact that <code>Shared</code> reply queues has lower performance than its alternatives <code>Temporary</code> and <code>Exclusive</code> .		<code>ReplyToType</code>
camel.component.amqp.request-timeout	The timeout for waiting for a reply when using the InOut Exchange Pattern (in milliseconds). The default is 20 seconds. You can include the header <code>CamelJmsRequestTimeout</code> to override this endpoint configured timeout value, and thus have per message individual timeout values. See also the <code>requestTimeoutCheckerInterval</code> option. The option is a long type.	20000	Long

Name	Description	Default	Type
camel.component.amqp.request-timeout-checker-interval	Configures how often Camel should check for timed out Exchanges when doing request/reply over JMS. By default Camel checks once per second. But if you must react faster when a timeout occurs, then you can lower this interval, to check more frequently. The timeout is determined by the option requestTimeout. The option is a long type.	1000	Long
camel.component.amqp.selector	Sets the JMS selector to use.		String
camel.component.amqp.stream-message-type-enabled	Sets whether StreamMessage type is enabled or not. Message payloads of streaming kind such as files, InputStream, etc will either be sent as BytesMessage or StreamMessage. This option controls which kind will be used. By default BytesMessage is used which enforces the entire message payload to be read into memory. By enabling this option the message payload is read into memory in chunks and each chunk is then written to the StreamMessage until no more data.	false	Boolean
camel.component.amqp.subscription-durable	Set whether to make the subscription durable. The durable subscription name to be used can be specified through the subscriptionName property. Default is false. Set this to true to register a durable subscription, typically in combination with a subscriptionName value (unless your message listener class name is good enough as subscription name). Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the pubSubDomain flag as well.	false	Boolean
camel.component.amqp.subscription-name	Set the name of a subscription to create. To be applied in case of a topic (pub-sub domain) with a shared or durable subscription. The subscription name needs to be unique within this client's JMS client id. Default is the class name of the specified message listener. Note: Only 1 concurrent consumer (which is the default of this message listener container) is allowed for each subscription, except for a shared subscription (which requires JMS 2.0).		String

Name	Description	Default	Type
camel.component.amqp.subscription-shared	Set whether to make the subscription shared. The shared subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a shared subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Note that shared subscriptions may also be durable, so this flag can (and often will) be combined with <code>subscriptionDurable</code> as well. Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well. Requires a JMS 2.0 compatible message broker.	false	Boolean
camel.component.amqp.synchronous	Sets whether synchronous processing should be strictly used.	false	Boolean
camel.component.amqp.task-executor	Allows you to specify a custom task executor for consuming messages. The option is a <code>org.springframework.core.task.TaskExecutor</code> type.		TaskExecutor
camel.component.amqp.test-connection-on-startup	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	Boolean
camel.component.amqp.time-to-live	When sending messages, specifies the time-to-live of the message (in milliseconds).	-1	Long
camel.component.amqp.transacted	Specifies whether to use transacted mode.	false	Boolean

Name	Description	Default	Type
camel.component.amqp.transacted-in-out	Specifies whether InOut operations (request reply) default to using transacted mode. If this flag is set to true, then Spring JmsTemplate will have sessionTransacted set to true, and the acknowledgeMode as transacted on the JmsTemplate used for InOut operations. Note from Spring JMS: that within a JTA transaction, the parameters passed to createQueue, createTopic methods are not taken into account. Depending on the Java EE transaction context, the container makes its own decisions on these values. Analogously, these parameters are not taken into account within a locally managed transaction either, since Spring JMS operates on an existing JMS Session in this case. Setting this flag to true will use a short local JMS transaction when running outside of a managed transaction, and a synchronized local JMS transaction in case of a managed transaction (other than an XA transaction) being present. This has the effect of a local JMS transaction being managed alongside the main transaction (which might be a native JDBC transaction), with the JMS transaction committing right after the main transaction.	false	Boolean
camel.component.amqp.transaction-manager	The Spring transaction manager to use. The option is a <code>org.springframework.transaction.PlatformTransactionManager</code> type.		PlatformTransactionManager
camel.component.amqp.transaction-name	The name of the transaction to use.		String
camel.component.amqp.transaction-timeout	The timeout value of the transaction (in seconds), if using transacted mode.	-1	Integer

Name	Description	Default	Type
camel.component.amqp.transfer-exception	If enabled and you are using Request Reply messaging (InOut) and an Exchange failed on the consumer side, then the caused Exception will be send back in response as a javax.jms.ObjectMessage. If the client is Camel, the returned Exception is rethrown. This allows you to use Camel JMS as a bridge in your routing - for example, using persistent queues to enable robust routing. Notice that if you also have transferExchange enabled, this option takes precedence. The caught exception is required to be serializable. The original Exception on the consumer side can be wrapped in an outer exception such as org.apache.camel.RuntimeCamelException when returned to the producer. Use this with caution as the data is using Java Object serialization and requires the received to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumer!.	false	Boolean
camel.component.amqp.transfer-exchange	You can transfer the exchange over the wire instead of just the body and headers. The following fields are transferred: In body, Out body, Fault body, In headers, Out headers, Fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level. You must enable this option on both the producer and consumer side, so Camel knows the payloads is an Exchange and not a regular payload. Use this with caution as the data is using Java Object serialization and requires the receiver to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumers having to use compatible Camel versions!.	false	Boolean
camel.component.amqp.use-message-id-as-correlation-id	Specifies whether JMSMessageID should always be used as JMSCorrelationID for InOut messages.	false	Boolean
camel.component.amqp.username	Username to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
camel.component.amqp.wait-for-provision-correlation-to-be-updated-counter	Number of times to wait for provisional correlation id to be updated to the actual correlation id when doing request/reply over JMS and when the option useMessageIDAsCorrelationID is enabled.	50	Integer

Name	Description	Default	Type
<code>camel.component.amqp.wait-for-provision-correlation-to-be-updated-thread-sleeping-time</code>	Interval in millis to sleep each time while waiting for provisional correlation id to be updated. The option is a long type.	100	Long

CHAPTER 3. AVRO

This component provides a dataformat for avro, which allows serialization and deserialization of messages using Apache Avro's binary dataformat. Since Camel 3.2 rpc functionality was moved into separate **camel-avro-rpc** component.

You can easily generate classes from a schema, using maven, ant etc. More details can be found at the [Apache Avro documentation](#).

3.1. DEPENDENCIES

When using **camel-avro** with Red Hat build of Camel Spring Boot, add the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-avro-starter</artifactId>
</dependency>
```

3.2. AVRO DATAFORMAT OPTIONS

The Avro dataformat supports 1 options, which are listed below.

Name	Default	Java Type	Description
instanceClassName		String	Class name to use for marshal and unmarshalling.

3.3. AVRO DATA FORMAT USAGE

Using the avro data format is as easy as specifying that the class that you want to marshal or unmarshal in your route.

```
AvroDataFormat format = new AvroDataFormat(Value.SCHEMA$);

from("direct:in").marshal(format).to("direct:marshal");
from("direct:back").unmarshal(format).to("direct:unmarshal");
```

Where Value is an Avro Maven Plugin Generated class.

or in XML

```
<camelContext id="camel" xmlns="http://camel.apache.org/schema/spring">
  <route>
    <from uri="direct:in"/>
    <marshal>
      <avro instanceClass="org.apache.camel.dataformat.avro.Message"/>
    </marshal>
```

```

        <to uri="log:out"/>
    </route>
</camelContext>

```

An alternative can be to specify the dataformat inside the context and reference it from your route.

```

<camelContext id="camel" xmlns="http://camel.apache.org/schema/spring">
    <dataFormats>
        <avro id="avro" instanceClass="org.apache.camel.dataformat.avro.Message"/>
    </dataFormats>
    <route>
        <from uri="direct:in"/>
        <marshal><custom ref="avro"/></marshal>
        <to uri="log:out"/>
    </route>
</camelContext>

```

In the same manner you can umarshal using the avro data format.

3.4. SPRING BOOT AUTO-CONFIGURATION

When using avro with Spring Boot make sure to add the Maven dependency to have support for auto configuration. The component supports 2 options, which are listed below.

Name	Description	Default	Type
camel.dataformat.avro.enabled	Whether to enable auto configuration of the avro data format. This is enabled by default.		Boolean
camel.dataformat.avro.instance-class-name	Class name to use for marshal and unmarshalling.		String

CHAPTER 4. AVRO JACKSON

Jackson Avro is a Data Format which uses the [Jackson library](#) with the [Avro extension](#) to unmarshal an Avro payload into Java objects or to marshal Java objects into an Avro payload.



NOTE

If you are familiar with Jackson, this Avro data format behaves in the same way as its JSON counterpart, and thus can be used with classes annotated for JSON serialization/deserialization.

```
from("kafka:topic").
  unmarshal().avro(AvroLibrary.Jackson, JsonNode.class).
  to("log:info");
```

4.1. DEPENDENCIES

When using **avro-jackson** with Red Hat build of Camel Spring Boot make sure to add the Maven dependency to have support for auto configuration.

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jackson-avro-starter</artifactId>
</dependency>
```

4.2. CONFIGURING THE SCHEMARESOLVER

Since Avro serialization is schema-based, this data format requires that you provide a SchemaResolver object that is able to lookup the schema for each exchange that is going to be marshalled/unmarshalled.

You can add a single SchemaResolver to the registry and it will be looked up automatically. Or you can explicitly specify the reference to a custom SchemaResolver.

4.3. AVRO JACKSON OPTIONS

The Avro Jackson dataformat supports 18 options, which are listed below.

Name	Default	Java Type	Description
objectMapper		String	Lookup and use the existing ObjectMapper with the given id when using Jackson.
useDefaultObjectMapper		Boolean	Whether to lookup and use default Jackson ObjectMapper from the registry.
unmarshalType		String	Class name of the java type to use when unmarshalling.

Name	Default	Java Type	Description
jsonView		String	When marshalling a POJO to JSON you might want to exclude certain fields from the JSON output. With Jackson you can use JSON views to accomplish this. This option is to refer to the class which has JsonView annotations.
include		String	If you want to marshal a pojo to JSON, and the pojo has some fields with null values. And you want to skip these null values, you can set this option to NON_NULL.
allowJmsType		Boolean	Used for JMS users to allow the JMSType header from the JMS spec to specify a FQN classname to use to unmarshal to.
collectionType		String	Refers to a custom collection type to lookup in the registry to use. This option should rarely be used, but allows to use different collection types than java.util.Collection based as default.
useList		Boolean	To unmarshal to a List of Map or a List of Pojo.
moduleClassNames		String	To use custom Jackson modules com.fasterxml.jackson.databind.Module specified as a String with FQN class names. Multiple classes can be separated by comma.
moduleRefs		String	To use custom Jackson modules referred from the Camel registry. Multiple modules can be separated by comma.
enableFeatures		String	Set of features to enable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature Multiple features can be separated by comma.
disableFeatures		String	Set of features to disable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature Multiple features can be separated by comma.

Name	Default	Java Type	Description
allowUnmarshalType		Boolean	If enabled then Jackson is allowed to attempt to use the CamelJacksonUnmarshalType header during the unmarshalling. This should only be enabled when desired to be used.
timezone		String	If set then Jackson will use the Timezone when marshalling/unmarshalling.
autoDiscoverObjectMapper		Boolean	If set to true then Jackson will lookup for an objectMapper into the registry.
contentTypeHeader		Boolean	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.
schemaResolver		String	Optional schema resolver used to lookup schemas for the data in transit.
autoDiscoverSchemaResolver		Boolean	When not disabled, the SchemaResolver will be looked up into the registry.

4.4. USING CUSTOM AVROMAPPER

You can configure **JacksonAvroDataFormat** to use a custom **AvroMapper** in case you need more control of the mapping configuration.

If you setup a single **AvroMapper** in the registry, then Camel will automatic lookup and use this **AvroMapper**.

4.5. SPRING BOOT AUTO-CONFIGURATION

The component supports 19 options, which are listed below.

Name	Description	Default	Type
camel.dataformat.avro-jackson.allow-jms-type	Used for JMS users to allow the JMSType header from the JMS spec to specify a FQN classname to use to unmarshal to.	false	Boolean

Name	Description	Default	Type
<code>camel.dataformat.avro-jackson.allow-unmarshall-type</code>	If enabled then Jackson is allowed to attempt to use the <code>CamelJacksonUnmarshalType</code> header during the unmarshalling. This should only be enabled when desired to be used.	false	Boolean
<code>camel.dataformat.avro-jackson.auto-discover-object-mapper</code>	If set to true then Jackson will lookup for an <code>objectMapper</code> into the registry.	false	Boolean
<code>camel.dataformat.avro-jackson.auto-discover-schema-resolver</code>	When not disabled, the <code>SchemaResolver</code> will be looked up into the registry.	true	Boolean
<code>camel.dataformat.avro-jackson.collection-type</code>	Refers to a custom collection type to lookup in the registry to use. This option should rarely be used, but allows to use different collection types than <code>java.util.Collection</code> based as default.		String
<code>camel.dataformat.avro-jackson.content-type-header</code>	Whether the data format should set the <code>Content-Type</code> header with the type from the data format. For example <code>application/xml</code> for data formats marshalling to XML, or <code>application/json</code> for data formats marshalling to JSON.	true	Boolean
<code>camel.dataformat.avro-jackson.disable-features</code>	Set of features to disable on the Jackson <code>com.fasterxml.jackson.databind.ObjectMapper</code> . The features should be a name that matches a enum from <code>com.fasterxml.jackson.databind.SerializationFeature</code> , <code>com.fasterxml.jackson.databind.DeserializationFeature</code> , or <code>com.fasterxml.jackson.databind.MapperFeature</code> . Multiple features can be separated by comma.		String
<code>camel.dataformat.avro-jackson.enable-features</code>	Set of features to enable on the Jackson <code>com.fasterxml.jackson.databind.ObjectMapper</code> . The features should be a name that matches a enum from <code>com.fasterxml.jackson.databind.SerializationFeature</code> , <code>com.fasterxml.jackson.databind.DeserializationFeature</code> , or <code>com.fasterxml.jackson.databind.MapperFeature</code> . Multiple features can be separated by comma.		String
<code>camel.dataformat.avro-jackson.enabled</code>	Whether to enable auto configuration of the avro-jackson data format. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.dataformat.avro-jackson.include</code>	If you want to marshal a pojo to JSON, and the pojo has some fields with null values. And you want to skip these null values, you can set this option to <code>NON_NULL</code> .		String
<code>camel.dataformat.avro-jackson.json-view</code>	When marshalling a POJO to JSON you might want to exclude certain fields from the JSON output. With Jackson you can use JSON views to accomplish this. This option is to refer to the class which has <code>JsonView</code> annotations.		String
<code>camel.dataformat.avro-jackson.module-class-names</code>	To use custom Jackson modules <code>com.fasterxml.jackson.databind.Module</code> specified as a String with FQN class names. Multiple classes can be separated by comma.		String
<code>camel.dataformat.avro-jackson.module-refs</code>	To use custom Jackson modules referred from the Camel registry. Multiple modules can be separated by comma.		String
<code>camel.dataformat.avro-jackson.object-mapper</code>	Lookup and use the existing ObjectMapper with the given id when using Jackson.		String
<code>camel.dataformat.avro-jackson.schema-resolver</code>	Optional schema resolver used to lookup schemas for the data in transit.		String
<code>camel.dataformat.avro-jackson.timezone</code>	If set then Jackson will use the Timezone when marshalling/unmarshalling.		String
<code>camel.dataformat.avro-jackson.unmarshal-type</code>	Class name of the java type to use when unmarshalling.		String
<code>camel.dataformat.avro-jackson.use-default-object-mapper</code>	Whether to lookup and use default Jackson ObjectMapper from the registry.	true	Boolean

Name	Description	Default	Type
<code>camel.dataformat.avro-jackson.use-list</code>	To unmarshal to a List of Map or a List of Pojo.	false	Boolean

CHAPTER 5. AWS CLOUDWATCH

Only producer is supported

The AWS2 Cloudwatch component allows messages to be sent to an [Amazon CloudWatch](#) metrics. The implementation of the Amazon API is provided by the [AWS SDK](#).

Prerequisites

You must have a valid Amazon Web Services developer account, and be signed up to use Amazon CloudWatch. More information is available at [Amazon CloudWatch](#).

5.1. DEPENDENCIES

When using **aws2-cw** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-aws2-cw-starter</artifactId>
</dependency>
```

5.2. URI FORMAT

```
aws2-cw://namespace[?options]
```

The metrics will be created if they don't already exists. You can append query options to the URI in the following format, **?options=value&option2=value&...**

5.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

5.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

5.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

5.4. COMPONENT OPTIONS

The AWS CloudWatch component supports 18 options, which are listed below.

Name	Description	Default	Type
amazonCwClient (producer)	Autowired To use the AmazonCloudWatch as the client.		CloudWatchClient
configuration (producer)	The component configuration.		Cw2Configuration
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
name (producer)	The metric name.		String

Name	Description	Default	Type
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
proxyHost (producer)	To define a proxy host when instantiating the CW client.		String
proxyPort (producer)	To define a proxy port when instantiating the CW client.		Integer
proxyProtocol (producer)	To define a proxy protocol when instantiating the CW client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
region (producer)	The region in which CW client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String
timestamp (producer)	The metric timestamp.		Instant
trustAllCertificates (producer)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
unit (producer)	The metric unit.		String
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String
useDefaultCredentialsProvider (producer)	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
value (producer)	The metric value.		Double

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

5.5. ENDPOINT OPTIONS

The AWS CloudWatch endpoint is configured using URI syntax:

```
aws2-cw:namespace
```

with the following path and query parameters:

5.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
namespace (producer)	Required The metric namespace.		String

5.5.2. Query Parameters (16 parameters)

Name	Description	Default	Type
amazonCwClient (producer)	Autowired To use the AmazonCloudWatch as the client.		CloudWatchClient

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
name (producer)	The metric name.		String
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
proxyHost (producer)	To define a proxy host when instantiating the CW client.		String
proxyPort (producer)	To define a proxy port when instantiating the CW client.		Integer
proxyProtocol (producer)	To define a proxy protocol when instantiating the CW client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
region (producer)	The region in which CW client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String
timestamp (producer)	The metric timestamp.		Instant
trustAllCertificates (producer)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
unit (producer)	The metric unit.		String

Name	Description	Default	Type
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
useDefaultCredentialsProvider (producer)	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
value (producer)	The metric value.		Double
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

Required CW component options

You have to provide the `amazonCwClient` in the Registry or your `accessKey` and `secretKey` to access the [Amazon's CloudWatch](#).

5.6. USAGE

5.6.1. Static credentials vs Default Credential Provider

You have the possibility of avoiding the usage of explicit static credentials, by specifying the `useDefaultCredentialsProvider` option and set it to true.

- Java system properties - `aws.accessKeyId` and `aws.secretKey`
- Environment variables - `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`.
- Web Identity Token from AWS STS.
- The shared credentials and config files.
- Amazon ECS container credentials - loaded from the Amazon ECS if the environment variable `AWS_CONTAINER_CREDENTIALS_RELATIVE_URI` is set.
- Amazon EC2 Instance profile credentials.

For more information about this you can look at [AWS credentials documentation](#)

5.6.2. Message headers evaluated by the CW producer

Header	Type	Description
CamelAwsCwMetricName	String	The Amazon CW metric name.

Header	Type	Description
CamelAwsCwMetricValue	Double	The Amazon CW metric value.
CamelAwsCwMetricUnit	String	The Amazon CW metric unit.
CamelAwsCwMetricName space	String	The Amazon CW metric namespace.
CamelAwsCwMetricTimes tamp	Date	The Amazon CW metric timestamp.
CamelAwsCwMetricDime nsionName	String	The Amazon CW metric dimension name.
CamelAwsCwMetricDime nsionValue	String	The Amazon CW metric dimension value.
CamelAwsCwMetricDime nsions	Map<String, String>	A map of dimension names and dimension values.

5.6.3. Advanced CloudWatchClient configuration

If you need more control over the **CloudWatchClient** instance configuration you can create your own instance and refer to it from the URI:

```
from("direct:start")
.to("aws2-cw://namespace?amazonCwClient=#client");
```

The **#client** refers to a **CloudWatchClient** in the Registry.

5.7. EXAMPLES

5.7.1. Producer Example

```
from("direct:start")
.to("aws2-cw://http://camel.apache.org/aws-cw");
```

and sends something like

```
exchange.getIn().setHeader(Cw2Constants.METRIC_NAME, "ExchangesCompleted");
exchange.getIn().setHeader(Cw2Constants.METRIC_VALUE, "2.0");
exchange.getIn().setHeader(Cw2Constants.METRIC_UNIT, "Count");
```

5.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 19 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.aws2-cw.access-key</code>	Amazon AWS Access Key.		String
<code>camel.component.aws2-cw.amazon-cw-client</code>	To use the AmazonCloudWatch as the client. The option is a <code>software.amazon.awssdk.services.cloudwatch.CloudWatchClient</code> type.		CloudWatchClient
<code>camel.component.aws2-cw.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.aws2-cw.configuration</code>	The component configuration. The option is a <code>org.apache.camel.component.aws2.cw.Cw2Configuration</code> type.		Cw2Configuration
<code>camel.component.aws2-cw.enabled</code>	Whether to enable auto configuration of the <code>aws2-cw</code> component. This is enabled by default.		Boolean
<code>camel.component.aws2-cw.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.aws2-cw.name</code>	The metric name.		String
<code>camel.component.aws2-cw.override-endpoint</code>	Set the need for overriding the endpoint. This option needs to be used in combination with <code>uriEndpointOverride</code> option.	false	Boolean
<code>camel.component.aws2-cw.proxy-host</code>	To define a proxy host when instantiating the CW client.		String

Name	Description	Default	Type
<code>camel.component.aws2-cw.proxy-port</code>	To define a proxy port when instantiating the CW client.		Integer
<code>camel.component.aws2-cw.proxy-protocol</code>	To define a proxy protocol when instantiating the CW client.		Protocol
<code>camel.component.aws2-cw.region</code>	The region in which CW client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
<code>camel.component.aws2-cw.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-cw.timestamp</code>	The metric timestamp. The option is a <code>java.time.Instant</code> type.		Instant
<code>camel.component.aws2-cw.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean
<code>camel.component.aws2-cw.unit</code>	The metric unit.		String
<code>camel.component.aws2-cw.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
<code>camel.component.aws2-cw.use-default-credentials-provider</code>	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	Boolean
<code>camel.component.aws2-cw.value</code>	The metric value.		Double

CHAPTER 6. AWS DYNAMODB

Only producer is supported

The AWS2 DynamoDB component supports storing and retrieving data from/to service.

Prerequisites

You must have a valid Amazon Web Services developer account, and be signed up to use Amazon DynamoDB. More information is available at [Amazon DynamoDB](#).

6.1. DEPENDENCIES

When using **aws2-ddb** Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-aws2-ddb-starter</artifactId>
</dependency>
```

6.2. URI FORMAT

```
aws2-ddb://domainName[?options]
```

You can append query options to the URI in the following format, **?options=value&option2=value&...**

6.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

6.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

6.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

6.4. COMPONENT OPTIONS

The AWS DynamoDB component supports 22 options, which are listed below.

Name	Description	Default	Type
amazonDDBClient (producer)	Autowired To use the AmazonDynamoDB as the client.		DynamoDbClient
configuration (producer)	The component configuration.		Ddb2Configuration
consistentRead (producer)	Determines whether or not strong consistency should be enforced when data is read.	false	boolean
enabledInitialDescribeTable (producer)	Set whether the initial Describe table operation in the DDB Endpoint must be done, or not.	true	boolean
keyAttributeName (producer)	Attribute name when creating table.		String
keyAttributeType (producer)	Attribute type when creating table.		String
keyScalarType (producer)	The key scalar type, it can be S (String), N (Number) and B (Bytes).		String

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
operation (producer)	What operation to perform. Enum values: ● BatchGetItems ● DeleteItem ● DeleteTable ● DescribeTable ● GetItem ● PutItem ● Query ● Scan ● UpdateItem ● UpdateTable	PutItem	Ddb2Operations
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
proxyHost (producer)	To define a proxy host when instantiating the DDB client.		String
proxyPort (producer)	The region in which DynamoDB client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		Integer

Name	Description	Default	Type
proxyProtocol (producer)	To define a proxy protocol when instantiating the DDB client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
readCapacity (producer)	The provisioned throughput to reserve for reading resources from your table.		Long
region (producer)	The region in which DDB client needs to work.		String
trustAllCertificates (producer)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
useDefaultCredentialsProvider (producer)	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
writeCapacity (producer)	The provisioned throughput to reserved for writing resources to your table.		Long
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

6.5. ENDPOINT OPTIONS

The AWS DynamoDB endpoint is configured using URI syntax:

```
aws2-ddb:tableName
```

with the following path and query parameters:

6.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
tableName (producer)	Required The name of the table currently worked with.		String

6.5.2. Query Parameters (20 parameters)

Name	Description	Default	Type
amazonDDBClient (producer)	Autowired To use the AmazonDynamoDB as the client.		DynamoDbClient
consistentRead (producer)	Determines whether or not strong consistency should be enforced when data is read.	false	boolean
enabledInitialDescribeTable (producer)	Set whether the initial Describe table operation in the DDB Endpoint must be done, or not.	true	boolean
keyAttributeName (producer)	Attribute name when creating table.		String
keyAttributeType (producer)	Attribute type when creating table.		String
keyScalarType (producer)	The key scalar type, it can be S (String), N (Number) and B (Bytes).		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
operation (producer)	What operation to perform. Enum values: • BatchGetItems • DeleteItem • DeleteTable • DescribeTable • GetItem • PutItem • Query • Scan • UpdateItem • UpdateTable	PutItem	Ddb2Operations
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
proxyHost (producer)	To define a proxy host when instantiating the DDB client.		String
proxyPort (producer)	The region in which DynamoDB client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		Integer
proxyProtocol (producer)	To define a proxy protocol when instantiating the DDB client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
readCapacity (producer)	The provisioned throughput to reserve for reading resources from your table.		Long
region (producer)	The region in which DDB client needs to work.		String

Name	Description	Default	Type
trustAllCertificates (producer)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
useDefaultCredentialsProvider (producer)	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
writeCapacity (producer)	The provisioned throughput to reserved for writing resources to your table.		Long
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

Required DDB component options

You have to provide the `amazonDDBClient` in the Registry or your `accessKey` and `secretKey` to access the [Amazon's DynamoDB](#).

6.6. USAGE

6.6.1. Static credentials vs Default Credential Provider

You have the possibility of avoiding the usage of explicit static credentials, by specifying the `useDefaultCredentialsProvider` option and set it to true.

- Java system properties - `aws.accessKeyId` and `aws.secretKey`
- Environment variables - `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`.
- Web Identity Token from AWS STS.
- The shared credentials and config files.
- Amazon ECS container credentials - loaded from the Amazon ECS if the environment variable `AWS_CONTAINER_CREDENTIALS_RELATIVE_URI` is set.
- Amazon EC2 Instance profile credentials.

For more information about this you can look at [AWS credentials documentation](#)

6.6.2. Message headers evaluated by the DDB producer

Header	Type	Description
CamelAwsDdbBatchItems	Map<String, KeysAndAttributes>	A map of the table name and corresponding items to get by primary key.
CamelAwsDdbTableName	String	Table Name for this operation.
CamelAwsDdbKey	Key	The primary key that uniquely identifies each item in a table.
CamelAwsDdbReturnValues	String	Use this parameter if you want to get the attribute name-value pairs before or after they are modified(NONE, ALL_OLD, UPDATED_OLD, ALL_NEW, UPDATED_NEW).
CamelAwsDdbUpdateCondition	Map<String, ExpectedAttributeValue>	Designates an attribute for a conditional modification.
CamelAwsDdbAttributeNames	Collection<String>	If attribute names are not specified then all attributes will be returned.
CamelAwsDdbConsistentRead	Boolean	If set to true, then a consistent read is issued, otherwise eventually consistent is used.
CamelAwsDdbIndexName	String	If set will be used as Secondary Index for Query operation.
CamelAwsDdbItem	Map<String, AttributeValue>	A map of the attributes for the item, and must include the primary key values that define the item.
CamelAwsDdbExactCount	Boolean	If set to true, Amazon DynamoDB returns a total number of items that match the query parameters, instead of a list of the matching items and their attributes.
CamelAwsDdbKeyConditions	Map<String, Condition>	This header specify the selection criteria for the query, and merge together the two old headers CamelAwsDdbHashKeyValue and CamelAwsDdbScanRangeKeyCondition
CamelAwsDdbStartKey	Key	Primary key of the item from which to continue an earlier query.
CamelAwsDdbHashKeyValue	AttributeValue	Value of the hash component of the composite primary key.
CamelAwsDdbLimit	Integer	The maximum number of items to return.

Header	Type	Description
CamelAwsDdbScanRangeKeyCondition	Condition	A container for the attribute values and comparison operators to use for the query.
CamelAwsDdbScanIndexForward	Boolean	Specifies forward or backward traversal of the index.
CamelAwsDdbScanFilter	Map<String, Condition>	Evaluates the scan results and returns only the desired values.
CamelAwsDdbUpdateValues	Map<String, AttributeValueUpdate>	Map of attribute name to the new value and action for the update.

6.6.3. Message headers set during BatchGetItems operation

Header	Type	Description
CamelAwsDdbBatchResponse	Map<String, BatchResponse>	Table names and the respective item attributes from the tables.
CamelAwsDdbUnprocessedKeys	Map<String, KeysAndAttributes>	Contains a map of tables and their respective keys that were not processed with the current response.

6.6.4. Message headers set during DeleteItem operation

Header	Type	Description
CamelAwsDdbAttributes	Map<String, AttributeValue>	The list of attributes returned by the operation.

6.6.5. Message headers set during DeleteTable operation

Header	Type	Description
CamelAwsDdbProvisionedThroughput		
ProvisionedThroughputDescription		The value of the ProvisionedThroughput property for this table

Header	Type	Description
CamelAwsDdbCreationDate	Date	Creation DateTime of this table.
CamelAwsDdbTableItemCount	Long	Item count for this table.
CamelAwsDdbKeySchema	KeySchema	The KeySchema that identifies the primary key for this table. From Camel 2.16.0 the type of this header is List<KeySchemaElement> and not KeySchema
CamelAwsDdbTableName	String	The table name.
CamelAwsDdbTableSize	Long	The table size in bytes.
CamelAwsDdbTableStatus	String	The status of the table: CREATING, UPDATING, DELETING, ACTIVE

6.6.6. Message headers set during DescribeTable operation

Header	Type	Description
CamelAwsDdbProvisionedThroughput	<code>\ {{ProvisionedThroughputDescription}} }</code>	The value of the ProvisionedThroughput property for this table
CamelAwsDdbCreationDate	Date	Creation DateTime of this table.
CamelAwsDdbTableItemCount	Long	Item count for this table.
CamelAwsDdbKeySchema	<code>{{KeySchema}}</code>	The KeySchema that identifies the primary key for this table.
CamelAwsDdbTableName	String	The table name.
CamelAwsDdbTableSize	Long	The table size in bytes.
CamelAwsDdbTableStatus	String	The status of the table: CREATING, UPDATING, DELETING, ACTIVE
CamelAwsDdbReadCapacity	Long	ReadCapacityUnits property of this table.
CamelAwsDdbWriteCapacity	Long	WriteCapacityUnits property of this table.

6.6.7. Message headers set during GetItem operation

Header	Type	Description
CamelAwsDdbAttributes	Map<String, AttributeValue>	The list of attributes returned by the operation.

6.6.8. Message headers set during PutItem operation

Header	Type	Description
CamelAwsDdbAttributes	Map<String, AttributeValue>	The list of attributes returned by the operation.

6.6.9. Message headers set during Query operation

Header	Type	Description
CamelAwsDdbItems	List<java.util.Map<String, AttributeValue>>	The list of attributes returned by the operation.
CamelAwsDdbLastEvaluatedKey	Key	Primary key of the item where the query operation stopped, inclusive of the previous result set.
CamelAwsDdbConsumedCapacity	Double	The number of Capacity Units of the provisioned throughput of the table consumed during the operation.
CamelAwsDdbCount	Integer	Number of items in the response.

6.6.10. Message headers set during Scan operation

Header	Type	Description
CamelAwsDdbItems	List<java.util.Map<String, AttributeValue>>	The list of attributes returned by the operation.
CamelAwsDdbLastEvaluatedKey	Key	Primary key of the item where the query operation stopped, inclusive of the previous result set.
CamelAwsDdbConsumedCapacity	Double	The number of Capacity Units of the provisioned throughput of the table consumed during the operation.

Header	Type	Description
CamelAwsDdbCount	Integer	Number of items in the response.
CamelAwsDdbScannedCount	Integer	Number of items in the complete scan before any filters are applied.

6.6.11. Message headers set during UpdateItem operation

Header	Type	Description
CamelAwsDdbAttributes	Map<String, AttributeValue>	The list of attributes returned by the operation.

6.6.12. Advanced AmazonDynamoDB configuration

If you need more control over the **AmazonDynamoDB** instance configuration you can create your own instance and refer to it from the URI:

```
from("direct:start")
.to("aws2-ddb://domainName?amazonDDBClient=#client");
```

The **#client** refers to a **DynamoDbClient** in the Registry.

6.7. SUPPORTED PRODUCER OPERATIONS

- BatchGetItems
- DeleteItem
- DeleteTable
- DescribeTable
- GetItem
- PutItem
- Query
- Scan
- UpdateItem
- UpdateTable

6.8. EXAMPLES

6.8.1. Producer Examples

- PutItem: this operation will create an entry into DynamoDB

```
from("direct:start")
  .setHeader(Ddb2Constants.OPERATION, Ddb2Operations.PutItem)
  .setHeader(Ddb2Constants.CONSISTENT_READ, "true")
  .setHeader(Ddb2Constants.RETURN_VALUES, "ALL_OLD")
  .setHeader(Ddb2Constants.ITEM, attributeMap)
  .setHeader(Ddb2Constants.ATTRIBUTE_NAMES, attributeMap.keySet());
  .to("aws2-ddb://" + tableName + "?keyAttributeName=" + attributeName + "&keyAttributeType=" +
    KeyType.HASH
    + "&keyScalarType=" + ScalarAttributeType.S
    + "&readCapacity=1&writeCapacity=1");
```

Maven users will need to add the following dependency to their pom.xml.

pom.xml

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-aws2-ddb</artifactId>
  <version>${camel-version}</version>
</dependency>
```

where **{camel-version}** must be replaced by the actual version of Camel.

6.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 40 options, which are listed below.

Name	Description	Default	Type
camel.component.aws2-ddb.access-key	Amazon AWS Access Key.		String
camel.component.aws2-ddb.amazon-d-d-b-client	To use the AmazonDynamoDB as the client. The option is a software.amazon.awssdk.services.dynamodb.DynamoDbClient type.		DynamoDbClient
camel.component.aws2-ddb.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.aws2-ddb.configuration	The component configuration. The option is a org.apache.camel.component.aws2.ddb.Ddb2Configuration type.		Ddb2Configuration

Name	Description	Default	Type
<code>camel.component.aws2-ddb.consistent-read</code>	Determines whether or not strong consistency should be enforced when data is read.	false	Boolean
<code>camel.component.aws2-ddb.enabled</code>	Whether to enable auto configuration of the aws2-ddb component. This is enabled by default.		Boolean
<code>camel.component.aws2-ddb.enabled-initial-describe-table</code>	Set whether the initial Describe table operation in the DDB Endpoint must be done, or not.	true	Boolean
<code>camel.component.aws2-ddb.key-attribute-name</code>	Attribute name when creating table.		String
<code>camel.component.aws2-ddb.key-attribute-type</code>	Attribute type when creating table.		String
<code>camel.component.aws2-ddb.key-scalar-type</code>	The key scalar type, it can be S (String), N (Number) and B (Bytes).		String
<code>camel.component.aws2-ddb.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.aws2-ddb.operation</code>	What operation to perform.		Ddb2Operations
<code>camel.component.aws2-ddb.override-endpoint</code>	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	Boolean

Name	Description	Default	Type
<code>camel.component.aws2-ddb.proxy-host</code>	To define a proxy host when instantiating the DDB client.		String
<code>camel.component.aws2-ddb.proxy-port</code>	The region in which DynamoDB client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		Integer
<code>camel.component.aws2-ddb.proxy-protocol</code>	To define a proxy protocol when instantiating the DDB client.		Protocol
<code>camel.component.aws2-ddb.read-capacity</code>	The provisioned throughput to reserve for reading resources from your table.		Long
<code>camel.component.aws2-ddb.region</code>	The region in which DDB client needs to work.		String
<code>camel.component.aws2-ddb.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-ddb.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean
<code>camel.component.aws2-ddb.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
<code>camel.component.aws2-ddb.use-default-credentials-provider</code>	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	Boolean
<code>camel.component.aws2-ddb.write-capacity</code>	The provisioned throughput to reserved for writing resources to your table.		Long

Name	Description	Default	Type
<code>camel.component.aws2-ddbstream.access-key</code>	Amazon AWS Access Key.		String
<code>camel.component.aws2-ddbstream.amazon-dynamo-db-streams-client</code>	Amazon DynamoDB client to use for all requests for this endpoint. The option is a <code>software.amazon.awssdk.services.dynamodb.streams.DynamoDbStreamsClient</code> type.		<code>DynamoDbStreamsClient</code>
<code>camel.component.aws2-ddbstream.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.aws2-ddbstream.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.aws2-ddbstream.configuration</code>	The component configuration. The option is a <code>org.apache.camel.component.aws2.ddbstream.Ddb2StreamConfiguration</code> type.		<code>Ddb2StreamConfiguration</code>
<code>camel.component.aws2-ddbstream.enabled</code>	Whether to enable auto configuration of the <code>aws2-ddbstream</code> component. This is enabled by default.		Boolean
<code>camel.component.aws2-ddbstream.max-results-per-request</code>	Maximum number of records that will be fetched in each poll.		Integer
<code>camel.component.aws2-ddbstream.override-endpoint</code>	Set the need for overriding the endpoint. This option needs to be used in combination with <code>uriEndpointOverride</code> option.	false	Boolean

Name	Description	Default	Type
<code>camel.component.aws2-ddbstream.proxy-host</code>	To define a proxy host when instantiating the DDBStreams client.		String
<code>camel.component.aws2-ddbstream.proxy-port</code>	To define a proxy port when instantiating the DDBStreams client.		Integer
<code>camel.component.aws2-ddbstream.proxy-protocol</code>	To define a proxy protocol when instantiating the DDBStreams client.		Protocol
<code>camel.component.aws2-ddbstream.region</code>	The region in which DDBStreams client needs to work.		String
<code>camel.component.aws2-ddbstream.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-ddbstream.stream-iterator-type</code>	Defines where in the DynamoDB stream to start getting records. Note that using FROM_START can cause a significant delay before the stream has caught up to real-time.		Ddb2StreamConfiguration\$StreamIteratorType
<code>camel.component.aws2-ddbstream.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean
<code>camel.component.aws2-ddbstream.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String
<code>camel.component.aws2-ddbstream.use-default-credentials-provider</code>	Set whether the DynamoDB Streams client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	Boolean

CHAPTER 7. AWS KINESIS

Both producer and consumer are supported

The AWS2 Kinesis component supports receiving messages from and sending messages to Amazon Kinesis (no Batch supported) service.

The AWS2 Kinesis component also supports Synchronous and Asynchronous Client. So if you need the connection (client) to be async, configure the 'asyncClient' option (can be found in DSL also) as **true**.

Prerequisites

You must have a valid Amazon Web Services developer account, and be signed up to use Amazon Kinesis. More information are available at [AWS Kinesis](#).

7.1. DEPENDENCIES

When using **aws2-kinesis** Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-aws2-kinesis-starter</artifactId>
</dependency>
```

7.2. URI FORMAT

```
aws2-kinesis://stream-name[?options]
```

The stream needs to be created prior to it being used. You can append query options to the URI in the following format, **?options=value&option2=value&...**

7.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

7.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).

- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

7.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

7.4. COMPONENT OPTIONS

The AWS Kinesis component supports 36 options, which are listed below.

Name	Description	Default	Type
cborEnabled (common)	This option will set the CBOR_ENABLED property during the execution.	true	boolean
configuration (common)	Component configuration.		Kinesis2Configuration
overrideEndpoint (common)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
region (common)	The region in which Kinesis Firehose client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id(). Enum values: * ap-south-2 * ap-south-1		String

Name	Description	Default	Type
	* eu-south-1		
	* eu-south-2		
	* us-gov-east-1		
	* me-central-1		
	* il-central-1		
	* ca-central-1		
	* eu-central-1		
	* us-iso-west-1		
	* eu-central-2		
	* eu-isoe-west-1		
	* us-west-1		
	* us-west-2		
	* af-south-1		
	* eu-north-1		
	* eu-west-3		
	* eu-west-2		
	* eu-west-1		
	* ap-northeast-3		
	* ap-northeast-2		
	* ap-northeast-1		
	* me-south-1		
	* sa-east-1		
	* ap-east-1		
	* cn-north-1		
	* ca-west-1		
	* us-gov-west-1		
	* ap-southeast-1		
	* ap-southeast-2		
	* us-iso-east-1		
	* ap-southeast-3		
	* ap-southeast-4		

Name	Description * us-east-1 * us-east-2	Default	Type
	* cn-northwest-1 * us-isob-east-1 * aws-global * aws-cn-global * aws-us-gov-global * aws-iso-global * aws-iso-b-global		
uriEndpointOverride (common)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore <code>bridgeErrorHandler</code> is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
iteratorType (consumer)	Defines where in the Kinesis stream to start getting records. Enum values: * AT_SEQUENCE_NUMBER * AFTER_SEQUENCE_NUMBER * TRIM_HORIZON * LATEST * AT_TIMESTAMP * null	TRIM_HORIZON	ShardIteratorType

Name	Description	Default	Type
maxResultsPerRequest (consumer)	Maximum number of records that will be fetched in each poll.	1	int
messageTimestamp (consumer)	The message timestamp to start polling from. Required if iteratorType is set to AT_TIMESTAMP.		String
sequenceNumber (consumer)	The sequence number to start polling from. Required if iteratorType is set to AFTER_SEQUENCE_NUMBER or AT_SEQUENCE_NUMBER.		String
shardClosed (consumer)	Define what will be the behavior in case of shard closed. Possible value are ignore, silent and fail. In case of ignore a WARN message will be logged once and the consumer will not process new messages until restarted, in case of silent there will be no logging and the consumer will not process new messages until restarted, in case of fail a ReachedClosedStateException will be thrown. Enum values: * ignore * fail * silent	ignore	Kinesis2ShardClosedStrategyEnum
shardId (consumer)	Defines which shardId in the Kinesis stream to get records from.		String
shardMonitorInterval (consumer (advanced))	The interval in milliseconds to wait between shard polling.	10000	long
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
amazonKinesisAsyncClient (advanced)	Autowired Supply a pre-constructed Amazon Kinesis async client to use for the KCL Consumer.		KinesisAsyncClient
amazonKinesisClient (advanced)	Autowired Amazon Kinesis client to use for all requests for this endpoint.		KinesisClient
applicationName (advanced)	Name of the KCL application. This defaults to the stream name.		String
asyncClient (advanced)	If we want to a KinesisAsyncClient instance set it to true.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
cloudWatchAsyncClient (advanced)	If we want to a KCL Consumer, we can pass an instance of CloudWatchAsyncClient.		CloudWatchAsyncClient
dynamoDbAsyncClient (advanced)	If we want to a KCL Consumer, we can pass an instance of DynamoDbAsyncClient.		DynamoDbAsyncClient
kclDisableCloudwatchMetricsExport (advanced)	If we want to use a KCL Consumer and disable the CloudWatch Metrics Export.	false	boolean
useKclConsumers (advanced)	If we want to a KCL Consumer set it to true.	false	boolean
healthCheckConsumerEnabled (health)	Used for enabling or disabling all consumer based health checks from this component.	true	boolean
healthCheckProducerEnabled (health)	Used for enabling or disabling all producer based health checks from this component. NOTE: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting camel.health.producersEnabled=true .	true	boolean
proxyHost (proxy)	To define a proxy host when instantiating the Kinesis client.		String

Name	Description	Default	Type
proxyPort (proxy)	To define a proxy port when instantiating the Kinesis client.		Integer
proxyProtocol (proxy)	To define a proxy protocol when instantiating the Kinesis client. Enum values: * HTTP * HTTPS	HTTPS	Protocol
accessKey (security)	Amazon AWS Access Key.		String
profileCredentialsName (security)	If using a profile credentials provider this parameter will set the profile name.		String
secretKey (security)	Amazon AWS Secret Key.		String
sessionToken (security)	Amazon AWS Session Token used when the user needs to assume a IAM role.		String
trustAllCertificates (security)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
useDefaultCredentialsProvider (security)	Set whether the Kinesis client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
useProfileCredentialsProvider (security)	Set whether the Kinesis client should expect to load credentials through a profile credentials provider.	false	boolean
useSessionCredentials (security)	Set whether the Kinesis client should expect to use Session Credentials. This is useful in situation in which the user needs to assume a IAM role for doing operations in Kinesis.	false	boolean

7.5. ENDPOINT OPTIONS

The AWS Kinesis endpoint is configured using URI syntax:

```
aws2-kinesis:streamName
```

with the following path and query parameters:

7.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
streamName (common)	Required Name of the stream.		String

7.5.2. Query Parameters (50 parameters)

Name	Description	Default	Type
amazonKinesisClient (common)	Autowired Amazon Kinesis client to use for all requests for this endpoint.		KinesisClient
cborEnabled (common)	This option will set the CBOR_ENABLED property during the execution.	true	boolean
overrideEndpoint (common)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
proxyHost (common)	To define a proxy host when instantiating the Kinesis client.		String
proxyPort (common)	To define a proxy port when instantiating the Kinesis client.		Integer
proxyProtocol (common)	To define a proxy protocol when instantiating the Kinesis client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
region (common)	The region in which Kinesis Firehose client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id(). Enum values: • ap-south-2 • ap-south-1 • eu-south-1		String

Name	• eu-south-2 • us-gov-east-1 Description	Default	Type
	• me-central-1 • il-central-1 • ca-central-1 • eu-central-1 • us-iso-west-1 • eu-central-2 • eu-isoe-west-1 • us-west-1 • us-west-2 • af-south-1 • eu-north-1 • eu-west-3 • eu-west-2 • eu-west-1 • ap-northeast-3 • ap-northeast-2 • ap-northeast-1 • me-south-1 • sa-east-1 • ap-east-1 • cn-north-1 • ca-west-1 • us-gov-west-1 • ap-southeast-1 • ap-southeast-2 • us-iso-east-1 • ap-southeast-3 • ap-southeast-4 • us-east-1 • us-east-2 • cn-northwest-1		

Name	Description - us-isob-east-1 - aws-global	Default	Type
	- aws-cn-global - aws-us-gov-global - aws-iso-global - aws-iso-b-global		
trustAllCertificates (common)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (common)	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String
useDefaultCredentialsProvider (common)	Set whether the Kinesis client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
iteratorType (consumer)	Defines where in the Kinesis stream to start getting records. Enum values: - AT_SEQUENCE_NUMBER - AFTER_SEQUENCE_NUMBER - TRIM_HORIZON - LATEST - AT_TIMESTAMP - null	TRIM_HORIZON	ShardIteratorType
maxResultsPerRequest (consumer)	Maximum number of records that will be fetched in each poll.	1	int

Name	Description	Default	Type
resumeStrategy (consumer)	Defines a resume strategy for AWS Kinesis. The default strategy reads the sequenceNumber if provided.	Kinesis UserC onfigur ationR esume Strate gy	KinesisResumeStr ategy
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
sequenceNumber (consumer)	The sequence number to start polling from. Required if iteratorType is set to AFTER_SEQUENCE_NUMBER or AT_SEQUENCE_NUMBER.		String
shardClosed (consumer)	Define what will be the behavior in case of shard closed. Possible value are ignore, silent and fail. In case of ignore a message will be logged and the consumer will restart from the beginning, in case of silent there will be no logging and the consumer will start from the beginning, in case of fail a ReachedClosedStateException will be raised. Enum values: ● ignore ● fail ● silent	ignore	Kinesis2ShardClos edStrategyEnum
shardId (consumer)	Defines which shardId in the Kinesis stream to get records from.		String
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
shardMonitorInterval (consumer (advanced))	The interval in milliseconds to wait between shard polling.	10000	long
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
amazonKinesisAsyncClient (advanced)	Autowired Supply a pre-constructed Amazon Kinesis async client to use for the KCL Consumer.		KinesisAsyncClient
amazonKinesisClient (advanced)	Autowired Amazon Kinesis client to use for all requests for this endpoint.		KinesisClient
applicationName (advanced)	Name of the KCL application. This defaults to the stream name.		String
asyncClient (advanced)	If we want to a KinesisAsyncClient instance set it to true.	false	boolean
cloudWatchAsyncClient (advanced)	If we want to a KCL Consumer, we can pass an instance of CloudWatchAsyncClient.		CloudWatchAsyncClient

Name	Description	Default	Type
dynamoDbAsyncClient (advanced)	If we want to a KCL Consumer, we can pass an instance of DynamoDbAsyncClient.		DynamoDbAsyncClient
kclDisableCloudwatchMetricsExport (advanced)	If we want to use a KCL Consumer and disable the CloudWatch Metrics Export.	false	boolean
useKclConsumers (advanced)	If we want to a KCL Consumer set it to true.	false	boolean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long

Name	Description	Default	Type
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit

Name	Description	Default	Type
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See <code>ScheduledExecutorService</code> in JDK for details.	true	boolean
accessKey (security)	Amazon AWS Access Key.		String
profileCredentialsName (security)	If using a profile credentials provider this parameter will set the profile name.		String
secretKey (security)	Amazon AWS Secret Key.		String
sessionToken (security)	Amazon AWS Session Token used when the user needs to assume a IAM role.		String
trustAllCertificates (security)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
useDefaultCredentialsProvider (security)	Set whether the Kinesis client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
useProfileCredentialsProvider (security)	Set whether the Kinesis client should expect to load credentials through a profile credentials provider.	false	boolean
useSessionCredentials (security)	Set whether the Kinesis client should expect to use Session Credentials. This is useful in situation in which the user needs to assume a IAM role for doing operations in Kinesis.	false	boolean

Required Kinesis component options

You have to provide the **KinesisClient** in the Registry with proxies and relevant credentials configured.

7.6. MESSAGE HEADERS

Name	Description
CamelAwsKinesisSequenceNumber (common) Constant: SEQUENCE_NUMBER	The sequence number of the record as defined in PutRecord syntax .

Name	Description
CamelAwsKinesisApproximateArrivalTimestamp (common) Constant: APPROX_ARRIVAL_TIME	The time AWS assigned as the arrival time of the record.
CamelAwsKinesisPartitionKey (common) Constant: PARTITION_KEY	Identifies which shard in the stream the data record is assigned to.
CamelMessageTimestamp (common) Constant: MESSAGE_TIMESTAMP	The timestamp of the message.
CamelAwsKinesisShardId (common) Constant: SHARD_ID	The shard ID of the shard where the data record was place

7.7. USAGE

7.7.1. Batch Consumer

This component implements the Batch Consumer. This allows you for instance to know how many messages exists in this batch and for instance let the Aggregator aggregate this number of messages.

The consumer is able to consume either from a single specific shard or all available shards (multiple shards consumption) of Amazon Kinesis. Therefore, if you leave the 'shardId' property in the DSL configuration empty, then it'll consume all available shards otherwise only the specified shard corresponding to the shardId will be consumed.

7.7.2. Batch Producer

This component implements the Batch Producer. This allows you to send multiple messages in a single request to Amazon Kinesis. Messages with batch size more than 500 is allowed. Producer will split them into multiple requests.

The batch type needs to implement the **Iterable** interface. For example, it can be a **List**, **Set** or any other collection type. The message type can be one or more of types **byte[]**, **ByteBuffer**, UTF-8 **String**, or **InputStream**. Other types are not supported.

7.7.3. Static credentials, Default Credential Provider and Profile Credentials Provider

You have the possibility of avoiding the usage of explicit static credentials, by specifying the `useDefaultCredentialsProvider` option and set it to true.

The order of evaluation for Default Credentials Provider is the following:

- Java system properties - **aws.accessKeyId** and **aws.secretAccessKey**
- Environment variables - **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY**.

- Web Identity Token from AWS STS.
- The shared credentials and config files.
- Amazon ECS container credentials - loaded from the Amazon ECS if the environment variable **AWS_CONTAINER_CREDENTIALS_RELATIVE_URI** is set.
- Amazon EC2 Instance profile credentials.

You have also the possibility of using Profile Credentials Provider, by specifying the **useProfileCredentialsProvider** option to true and **profileCredentialsName** to the profile name.

Only one of static, default and profile credentials could be used at the same time.

For more information see [AWS credentials documentation](#).

7.7.4. AmazonKinesis configuration

You then have to reference the **KinesisClient** in the **amazonKinesisClient** URI option.

```
from("aws2-kinesis://mykinesisstream?amazonKinesisClient=#kinesisClient")
.to("log:out?showAll=true");
```

7.7.5. Providing AWS Credentials

It is recommended that the credentials are obtained by using the [DefaultAWSCredentialsProviderChain](#) that is the default when creating a new ClientConfiguration instance, however, a different [AWSCredentialsProvider](#) can be specified when calling `createClient(...)`.

7.7.6. AWS Kinesis KCL Consumer

The component supports also the KCL (Kinesis Client Library) for consuming from a Kinesis Data Stream. To enable this feature, set the three different parameters in your endpoint and set the region:

```
from("aws2-kinesis://mykinesisstream?
asyncClient=true&useDefaultCredentialsProvider=true&useKclConsumers=true&region=myregion")
.to("log:out?showAll=true");
```

This feature makes it possible to automatically checkpoint the Shard Iterations by combining the usage of KCL, DynamoDB Table and CloudWatch alarms. This works out of the box, by simply using your AWS Credentials.



NOTE

AWS Kinesis consumer with KCL needs approximately 60-70 seconds to startup to prepare everything, for example, listing the shards, creating/querying the Lease table on Dynamo DB.

7.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 58 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.aws2-kinesis-firehose.access-key</code>	Amazon AWS Access Key.		String
<code>camel.component.aws2-kinesis-firehose.amazon-kinesis-firehose-client</code>	Amazon Kinesis Firehose client to use for all requests for this endpoint. The option is a <code>software.amazon.awssdk.services.firehose.FirehoseClient</code> type.		FirehoseClient
<code>camel.component.aws2-kinesis-firehose.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.aws2-kinesis-firehose.cbor-enabled</code>	This option will set the <code>CBOR_ENABLED</code> property during the execution.	true	Boolean
<code>camel.component.aws2-kinesis-firehose.configuration</code>	Component configuration. The option is a <code>org.apache.camel.component.aws2.firehose.KinesisFirehose2Configuration</code> type.		KinesisFirehose2Configuration
<code>camel.component.aws2-kinesis-firehose.enabled</code>	Whether to enable auto configuration of the <code>aws2-kinesis-firehose</code> component. This is enabled by default.		Boolean
<code>camel.component.aws2-kinesis-firehose.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.aws2-kinesis-firehose.operation</code>	The operation to do in case the user don't want to send only a record.		KinesisFirehose2Operations

Name	Description	Default	Type
<code>camel.component.aws2-kinesis-firehose.override-endpoint</code>	Set the need for overriding the endpoint. This option needs to be used in combination with <code>uriEndpointOverride</code> option.	false	Boolean
<code>camel.component.aws2-kinesis-firehose.profile-credentials-name</code>	If using a profile credentials provider this parameter will set the profile name.		String
<code>camel.component.aws2-kinesis-firehose.proxy-host</code>	To define a proxy host when instantiating the Kinesis Firehose client.		String
<code>camel.component.aws2-kinesis-firehose.proxy-port</code>	To define a proxy port when instantiating the Kinesis Firehose client.		Integer
<code>camel.component.aws2-kinesis-firehose.proxy-protocol</code>	To define a proxy protocol when instantiating the Kinesis Firehose client.		Protocol
<code>camel.component.aws2-kinesis-firehose.region</code>	The region in which Kinesis Firehose client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example <code>ap-east-1</code>) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
<code>camel.component.aws2-kinesis-firehose.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-kinesis-firehose.session-token</code>	Amazon AWS Session Token used when the user needs to assume a IAM role.		String
<code>camel.component.aws2-kinesis-firehose.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean

Name	Description	Default	Type
<code>camel.component.aws2-kinesis-firehose.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
<code>camel.component.aws2-kinesis-firehose.use-default-credentials-provider</code>	Set whether the Kinesis Firehose client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	Boolean
<code>camel.component.aws2-kinesis-firehose.use-profile-credentials-provider</code>	Set whether the Kinesis Firehose client should expect to load credentials through a profile credentials provider.	false	Boolean
<code>camel.component.aws2-kinesis-firehose.use-session-credentials</code>	Set whether the Kinesis Firehose client should expect to use Session Credentials. This is useful in situation in which the user needs to assume a IAM role for doing operations in Kinesis Firehose.	false	Boolean
<code>camel.component.aws2-kinesis.access-key</code>	Amazon AWS Access Key.		String
<code>*camel.component.aws2-kinesis.amazon-kinesis-async-client</code>	Supply a pre-constructed Amazon Kinesis async client to use for the KCL Consumer. The option is a <code>software.amazon.awssdk.services.kinesis.KinesisAsyncClient</code> type.		KinesisAsyncClient
<code>camel.component.aws2-kinesis.amazon-kinesis-client</code>	Amazon Kinesis client to use for all requests for this endpoint. The option is a <code>software.amazon.awssdk.services.kinesis.KinesisClient</code> type.		KinesisClient
<code>camel.component.aws2-kinesis.application-name</code>	Name of the KCL application. This defaults to the stream name.		String

Name	Description	Default	Type
<code>camel.component.aws2-kinesis.async-client</code>	If we want to a <code>KinesisAsyncClient</code> instance set it to true.	false	Boolean
<code>camel.component.aws2-kinesis.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.aws2-kinesis.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.aws2-kinesis.cbor-enabled</code>	This option will set the <code>CBOR_ENABLED</code> property during the execution.	true	Boolean
<code>camel.component.aws2-kinesis.cloud-watch-async-client</code>	If we want to a KCL Consumer, we can pass an instance of <code>CloudWatchAsyncClient</code> . The option is a <code>software.amazon.awssdk.services.cloudwatch.CloudWatchAsyncClient</code> type.		<code>CloudWatchAsyncClient</code>
<code>camel.component.aws2-kinesis.configuration</code>	Component configuration. The option is a <code>org.apache.camel.component.aws2.kinesis.Kinesis2Configuration</code> type.		<code>Kinesis2Configuration</code>
<code>camel.component.aws2-kinesis.dynamodb-async-client</code>	If we want to a KCL Consumer, we can pass an instance of <code>DynamoDbAsyncClient</code> . The option is a <code>software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient</code> type.		<code>DynamoDbAsyncClient</code>
<code>camel.component.aws2-kinesis.enabled</code>	Whether to enable auto configuration of the <code>aws2-kinesis</code> component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.aws2-kinesis.health-check-consumer-enabled</code>	Used for enabling or disabling all consumer based health checks from this component.	true	Boolean
<code>camel.component.aws2-kinesis.health-check-producer-enabled</code>	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting camel.health.producersEnabled=true .	true	Boolean
<code>camel.component.aws2-kinesis.iterator-type</code>	Defines where in the Kinesis stream to start getting records.		ShardIteratorType
<code>camel.component.aws2-kinesis.kcl-disable-cloudwatch-metrics-export</code>	If we want to use a KCL Consumer and disable the CloudWatch Metrics Export.	false	Boolean
<code>camel.component.aws2-kinesis.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.aws2-kinesis.max-results-per-request</code>	Maximum number of records that will be fetched in each poll.	1	Integer

Name	Description	Default	Type
<code>camel.component.aws2-kinesis.message-timestamp</code>	The message timestamp to start polling from. Required if <code>iteratorType</code> is set to <code>AT_TIMESTAMP</code> .		String
<code>camel.component.aws2-kinesis.override-endpoint</code>	Set the need for overriding the endpoint. This option needs to be used in combination with <code>uriEndpointOverride</code> option.	false	Boolean
<code>camel.component.aws2-kinesis.profile-credentials-name</code>	If using a profile credentials provider this parameter will set the profile name.		String
<code>camel.component.aws2-kinesis.proxy-host</code>	To define a proxy host when instantiating the Kinesis client.		String
<code>camel.component.aws2-kinesis.proxy-port</code>	To define a proxy port when instantiating the Kinesis client.		Integer
<code>camel.component.aws2-kinesis.proxy-protocol</code>	To define a proxy protocol when instantiating the Kinesis client.		Protocol
<code>camel.component.aws2-kinesis.region</code>	The region in which Kinesis Firehose client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example <code>ap-east-1</code>) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
<code>camel.component.aws2-kinesis.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-kinesis.sequence-number</code>	The sequence number to start polling from. Required if <code>iteratorType</code> is set to <code>AFTER_SEQUENCE_NUMBER</code> or <code>AT_SEQUENCE_NUMBER</code> .		String

Name	Description	Default	Type
<code>camel.component.aws2-kinesis.session-token</code>	Amazon AWS Session Token used when the user needs to assume a IAM role.		String
<code>camel.component.aws2-kinesis.shard-closed</code>	Define what will be the behavior in case of shard closed. Possible value are ignore, silent and fail. In case of ignore a message will be logged and the consumer will restart from the beginning, in case of silent there will be no logging and the consumer will start from the beginning, in case of fail a <code>ReachedClosedStateException</code> will be raised.		<code>Kinesis2ShardClosedStrategyEnum</code>
<code>camel.component.aws2-kinesis.shard-id</code>	Defines which shardId in the Kinesis stream to get records from.		String
<code>*camel.component.aws2-kinesis.shard-monitor-interval</code>	The interval in milliseconds to wait between shard polling.	10000	Long
<code>camel.component.aws2-kinesis.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean
<code>camel.component.aws2-kinesis.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
<code>camel.component.aws2-kinesis.use-default-credentials-provider</code>	Set whether the Kinesis client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	Boolean
<code>camel.component.aws2-kinesis.use-profile-credentials-provider</code>	Set whether the Kinesis client should expect to load credentials through a profile credentials provider.	false	Boolean
<code>camel.component.aws2-kinesis.use-session-credentials</code>	Set whether the Kinesis client should expect to use Session Credentials. This is useful in situation in which the user needs to assume a IAM role for doing operations in Kinesis.	false	Boolean

Name	Description	Default	Type
------	-------------	---------	------

CHAPTER 8. AWS 2 LAMBDA

Only producer is supported

The AWS2 Lambda component supports create, get, list, delete and invoke [AWS Lambda](#) functions.

Prerequisites

You must have a valid Amazon Web Services developer account, and be signed up to use Amazon Lambda. More information is available at [AWS Lambda](#).

When creating a Lambda function, you need to specify a IAM role which has at least the AWSLambdaBasicExecuteRole policy attached.

8.1. DEPENDENCIES

When using **aws2-lambda** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-aws2-lambda-starter</artifactId>
</dependency>
```

8.2. URI FORMAT

```
aws2-lambda://functionName[?options]
```

You can append query options to the URI in the following format, **options=value&option2=value&...**

8.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

8.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).

- directly in the Java code.

8.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

8.4. COMPONENT OPTIONS

The AWS Lambda component supports 16 options, which are listed below.

Name	Description	Default	Type
configuration (producer)	Component configuration.		Lambda2Configuration
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
operation (producer)	The operation to perform. It can be listFunctions, getFunction, createFunction, deleteFunction or invokeFunction. Enum values: • listFunctions • getFunction • createAlias • deleteAlias • getAlias • listAliases • createFunction • deleteFunction • invokeFunction • updateFunction • createEventSourceMapping • deleteEventSourceMapping • listEventSourceMapping • listTags • tagResource • untagResource • publishVersion • listVersions	invokeFunction	Lambda2Operations
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
pojoRequest (producer)	If we want to use a POJO request as body or not.	false	boolean
region (producer)	The region in which Lambda client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String

Name	Description	Default	Type
trustAllCertificates (producer)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
useDefaultCredentialsProvider (producer)	Set whether the Lambda client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
awsLambdaClient (advanced)	Autowired To use a existing configured <code>AwsLambdaClient</code> as client.		<code>LambdaClient</code>
proxyHost (proxy)	To define a proxy host when instantiating the Lambda client.		String
proxyPort (proxy)	To define a proxy port when instantiating the Lambda client.		Integer
proxyProtocol (proxy)	To define a proxy protocol when instantiating the Lambda client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

8.5. ENDPOINT OPTIONS

The AWS Lambda endpoint is configured using URI syntax:

`aws2-lambda:function`

with the following path and query parameters:

8.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
function (producer)	Required Name of the Lambda function.		String

8.5.2. Query Parameters (14 parameters)

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
operation (producer)	The operation to perform. It can be listFunctions, getFunction, createFunction, deleteFunction or invokeFunction. Enum values: • listFunctions • getFunction • createAlias • deleteAlias • getAlias • listAliases • createFunction • deleteFunction • invokeFunction • updateFunction • createEventSourceMapping • deleteEventSourceMapping • listEventSourceMapping • listTags • tagResource • untagResource • publishVersion • listVersions	invokeFunction	Lambda2Operations
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
pojoRequest (producer)	If we want to use a POJO request as body or not.	false	boolean
region (producer)	The region in which Lambda client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String

Name	Description	Default	Type
trustAllCertificates (producer)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
useDefaultCredentialsProvider (producer)	Set whether the Lambda client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
awsLambdaClient (advanced)	Autowired To use an existing configured <code>AwsLambdaClient</code> as client.		<code>LambdaClient</code>
proxyHost (proxy)	To define a proxy host when instantiating the Lambda client.		String
proxyPort (proxy)	To define a proxy port when instantiating the Lambda client.		Integer
proxyProtocol (proxy)	To define a proxy protocol when instantiating the Lambda client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

Required Lambda component options

You have to provide the `awsLambdaClient` in the Registry or your `accessKey` and `secretKey` to access the [Amazon Lambda](#) service..

8.6. USAGE

8.6.1. Static credentials vs Default Credential Provider

You have the possibility of avoiding the usage of explicit static credentials, by specifying the `useDefaultCredentialsProvider` option and set it to true.

- Java system properties - `aws.accessKeyId` and `aws.secretKey`

- Environment variables - AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY.
- Web Identity Token from AWS STS.
- The shared credentials and config files.
- Amazon ECS container credentials - loaded from the Amazon ECS if the environment variable AWS_CONTAINER_CREDENTIALS_RELATIVE_URI is set.
- Amazon EC2 Instance profile credentials.

For more information about this you can look at [AWS credentials documentation](#)

8.6.2. Message headers evaluated by the Lambda producer

Operation	Header	Type	Description	Required
All	CamelAwsLambdaOperation	String	The operation we want to perform. Override operation passed as query parameter	Yes
createFunction	CamelAwsLambdaS3Bucket	String	Amazon S3 bucket name where the .zip file containing your deployment package is stored. This bucket must reside in the same AWS region where you are creating the Lambda function.	No
createFunction	CamelAwsLambdaS3Key	String	The Amazon S3 object (the deployment package) key name you want to upload.	No
createFunction	CamelAwsLambdaS3ObjectVersion	String	The Amazon S3 object (the deployment package) version you want to upload.	No
createFunction	CamelAwsLambdaZipFile	String	The local path of the zip file (the deployment package). Content of zip file can also be put in Message body.	No

Operation	Header	Type	Description	Required
createFunction	CamelAwsLambdaRole	String	The Amazon Resource Name (ARN) of the IAM role that Lambda assumes when it executes your function to access any other Amazon Web Services (AWS) resources.	Yes
createFunction	CamelAwsLambdaRuntime	String	The runtime environment for the Lambda function you are uploading. (nodejs, nodejs4.3, nodejs6.10, java8, python2.7, python3.6, dotnetcore1.0, nodejs4.3-edge)	Yes
createFunction	CamelAwsLambdaHandler	String	The function within your code that Lambda calls to begin execution. For Node.js, it is the module-name.export value in your function. For Java, it can be package.class-name::handler or package.class-name.	Yes
createFunction	CamelAwsLambdaDescription	String	The user-provided description.	No
createFunction	CamelAwsLambdaTargetArn	String	The parent object that contains the target ARN (Amazon Resource Name) of an Amazon SQS queue or Amazon SNS topic.	No
createFunction	CamelAwsLambdaMemorySize	Integer	The memory size, in MB, you configured for the function. Must be a multiple of 64 MB.	No

Operation	Header	Type	Description	Required
createFunction	CamelAwsLambdaKMSKeyArn	String	The Amazon Resource Name (ARN) of the KMS key used to encrypt your function's environment variables. If not provided, AWS Lambda will use a default service key.	No
createFunction	CamelAwsLambdaPublish	Boolean	This boolean parameter can be used to request AWS Lambda to create the Lambda function and publish a version as an atomic operation.	No
createFunction	CamelAwsLambdaTimeout	Integer	The function execution time at which Lambda should terminate the function. The default is 3 seconds.	No
createFunction	CamelAwsLambdaTracingConfig	String	Your function's tracing settings (Active or PassThrough).	No
createFunction	CamelAwsLambdaEnvironmentVariables	Map<String, String>	The key-value pairs that represent your environment's configuration settings.	No
createFunction	CamelAwsLambdaEnvironmentTags	Map<String, String>	The list of tags (key-value pairs) assigned to the new function.	No
createFunction	CamelAwsLambdaSecurityGroupIds	List<String>	If your Lambda function accesses resources in a VPC, a list of one or more security groups IDs in your VPC.	No
createFunction	CamelAwsLambdaSubnetIds	List<String>	If your Lambda function accesses resources in a VPC, a list of one or more subnet IDs in your VPC.	No
createAlias	CamelAwsLambdaFunctionVersion	String	The function version to set in the alias	Yes

Operation	Header	Type	Description	Required
-----------	--------	------	-------------	----------

createAlias	CamelAwsLambdaAliasFunctionName	String	The function name to set in the alias	Yes
createAlias	CamelAwsLambdaAliasFunctionDescription	String	The function description to set in the alias	No
deleteAlias	CamelAwsLambdaAliasFunctionName	String	The function name of the alias	Yes
getAlias	CamelAwsLambdaAliasFunctionName	String	The function name of the alias	Yes
listAliases	CamelAwsLambdaFunctionVersion	String	The function version to set in the alias	No

8.7. LIST OF AVAILABLE OPERATIONS

- listFunctions
- getFunction
- createFunction
- deleteFunction
- invokeFunction
- updateFunction
- createEventSourceMapping
- deleteEventSourceMapping
- listEventSourceMapping
- listTags
- tagResource
- untagResource
- publishVersion
- listVersions

- createAlias
- deleteAlias
- getAlias
- listAliases

8.8. EXAMPLES

8.8.1. Producer Example

To have a full understanding of how the component works, you may have a look at these [integration tests](#).

8.8.2. Producer Examples

- CreateFunction: this operation will create a function for you in AWS Lambda

```
from("direct:createFunction").to("aws2-lambda://GetHelloWithName?
operation=createFunction").to("mock:result");
```

and by sending

```
template.send("direct:createFunction", ExchangePattern.InOut, new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(Lambda2Constants.RUNTIME, "nodejs6.10");
        exchange.getIn().setHeader(Lambda2Constants.HANDLER, "GetHelloWithName.handler");
        exchange.getIn().setHeader(Lambda2Constants.DESRIPTION, "Hello with node.js on
Lambda");
        exchange.getIn().setHeader(Lambda2Constants.ROLE,
            "arn:aws:iam::643534317684:role/lambda-execution-role");
        ClassLoader classLoader = getClass().getClassLoader();
        File file = new File(
            classLoader

.getResource("org/apache/camel/component/aws2/lambda/function/node/GetHelloWithName.zip")
            .getFile());
        FileInputStream inputStream = new FileInputStream(file);
        exchange.getIn().setBody(inputStream);
    }
});
```

8.9. USING A POJO AS BODY

Sometimes build an AWS Request can be complex, because of multiple options. We introduce the possibility to use a POJO as body. In AWS Lambda there are multiple operations you can submit, as an example for Get Function request, you can do something like:

```
from("direct:getFunction")
    .setBody(GetFunctionRequest.builder().functionName("test").build())
    .to("aws2-lambda://GetHelloWithName?
```

```
awsLambdaClient=#awsLambdaClient&operation=getFunction&pojoRequest=true")
```

In this way you'll pass the request directly without the need of passing headers and options specifically related to this operation.

8.10. DEPENDENCIES

Maven users will need to add the following dependency to their pom.xml.

pom.xml

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-aws2-lambda</artifactId>
  <version>${camel-version}</version>
</dependency>
```

where **{camel-version}** must be replaced by the actual version of Camel.

8.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 17 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.aws2-lambda.access-key</code>	Amazon AWS Access Key.		String
<code>camel.component.aws2-lambda.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.aws2-lambda.aws-lambda-client</code>	To use a existing configured <code>AwsLambdaClient</code> as client. The option is a <code>software.amazon.awssdk.services.lambda.LambdaClient</code> type.		<code>LambdaClient</code>
<code>camel.component.aws2-lambda.configuration</code>	Component configuration. The option is a <code>org.apache.camel.component.aws2.lambda.Lambda2Configuration</code> type.		<code>Lambda2Configuration</code>

Name	Description	Default	Type
<code>camel.component.aws2-lambda.enabled</code>	Whether to enable auto configuration of the aws2-lambda component. This is enabled by default.		Boolean
<code>camel.component.aws2-lambda.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.aws2-lambda.operation</code>	The operation to perform. It can be listFunctions, getFunction, createFunction, deleteFunction or invokeFunction.		Lambda2Operations
<code>camel.component.aws2-lambda.override-endpoint</code>	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	Boolean
<code>camel.component.aws2-lambda.pojo-request</code>	If we want to use a POJO request as body or not.	false	Boolean
<code>camel.component.aws2-lambda.proxy-host</code>	To define a proxy host when instantiating the Lambda client.		String
<code>camel.component.aws2-lambda.proxy-port</code>	To define a proxy port when instantiating the Lambda client.		Integer
<code>camel.component.aws2-lambda.proxy-protocol</code>	To define a proxy protocol when instantiating the Lambda client.		Protocol

Name	Description	Default	Type
<code>camel.component.aws2-lambda.region</code>	The region in which Lambda client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
<code>camel.component.aws2-lambda.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-lambda.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean
<code>camel.component.aws2-lambda.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
<code>camel.component.aws2-lambda.use-default-credentials-provider</code>	Set whether the Lambda client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	Boolean

CHAPTER 9. AWS S3 STORAGE SERVICE

Both producer and consumer are supported

The AWS2 S3 component supports storing and retrieving objects from/to [Amazon's S3](#) service.

Prerequisites

You must have a valid Amazon Web Services developer account, and be signed up to use Amazon S3. More information is available at link:<https://aws.amazon.com/s3> [Amazon S3].

9.1. DEPENDENCIES

When using **aws2-s3** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-aws2-s3-starter</artifactId>
</dependency>
```

9.2. URI FORMAT

```
aws2-s3://bucketNameOrArn[?options]
```

The bucket will be created if it don't already exists. You can append query options to the URI in the following format,

options=value&option2=value&...

9.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

9.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).

- directly in the Java code.

9.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

9.4. COMPONENT OPTIONS

The AWS S3 Storage Service component supports 50 options, which are listed below.

Name	Description	Default	Type
amazonS3Client (common)	Autowired Reference to a <code>com.amazonaws.services.s3.AmazonS3</code> in the registry.		S3Client
amazonS3Presigner (common)	Autowired An S3 Presigner for Request, used mainly in <code>createDownloadLink</code> operation.		S3Presigner
autoCreateBucket (common)	Setting the autocreation of the S3 bucket <code>bucketName</code> . This will apply also in case of <code>moveAfterRead</code> option enabled and it will create the <code>destinationBucket</code> if it doesn't exist already.	false	boolean
configuration (common)	The component configuration.		AWS2S3Configuration
overrideEndpoint (common)	Set the need for overriding the endpoint. This option needs to be used in combination with <code>uriEndpointOverride</code> option.	false	boolean
pojoRequest (common)	If we want to use a POJO request as body or not.	false	boolean

Name	Description	Default	Type
policy (common)	The policy for this queue to set in the <code>com.amazonaws.services.s3.AmazonS3#setBucketPolicy()</code> method.		String
proxyHost (common)	To define a proxy host when instantiating the SQS client.		String
proxyPort (common)	Specify a proxy port to be used inside the client definition.		Integer
proxyProtocol (common)	To define a proxy protocol when instantiating the S3 client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
region (common)	The region in which S3 client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example <code>ap-east-1</code>) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
trustAllCertificates (common)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (common)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
useDefaultCredentialsProvider (common)	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
customerAlgorithm (common (advanced))	Define the customer algorithm to use in case <code>CustomerKey</code> is enabled.		String
customerKeyId (common (advanced))	Define the id of Customer key to use in case <code>CustomerKey</code> is enabled.		String
customerKeyMD5 (common (advanced))	Define the MD5 of Customer key to use in case <code>CustomerKey</code> is enabled.		String

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
deleteAfterRead (consumer)	Delete objects from S3 after they have been retrieved. The delete is only performed if the Exchange is committed. If a rollback occurs, the object is not deleted. If this option is false, then the same objects will be retrieve over and over again on the polls. Therefore you need to use the Idempotent Consumer EIP in the route to filter out duplicates. You can filter using the <code>AWS2S3Constants#BUCKET_NAME</code> and <code>AWS2S3Constants#KEY</code> headers, or only the <code>AWS2S3Constants#KEY</code> header.	true	boolean
delimiter (consumer)	The delimiter which is used in the <code>com.amazonaws.services.s3.model.ListObjectsRequest</code> to only consume objects we are interested in.		String
destinationBucket (consumer)	Define the destination bucket where an object must be moved when <code>moveAfterRead</code> is set to true.		String
destinationBucketPrefix (consumer)	Define the destination bucket prefix to use when an object must be moved and <code>moveAfterRead</code> is set to true.		String
destinationBucketSuffix (consumer)	Define the destination bucket suffix to use when an object must be moved and <code>moveAfterRead</code> is set to true.		String
doneFileName (consumer)	If provided, Camel will only consume files if a done file exists.		String
fileName (consumer)	To get the object from the bucket with the given file name.		String
ignoreBody (consumer)	If it is true, the S3 Object Body will be ignored completely, if it is set to false the S3 Object will be put in the body. Setting this to true, will override any behavior defined by <code>includeBody</code> option.	false	boolean

Name	Description	Default	Type
includeBody (consumer)	If it is true, the S3Object exchange will be consumed and put into the body and closed. If false the S3Object stream will be put raw into the body and the headers will be set with the S3 object metadata. This option is strongly related to autocloseBody option. In case of setting includeBody to true because the S3Object stream will be consumed then it will also be closed, while in case of includeBody false then it will be up to the caller to close the S3Object stream. However setting autocloseBody to true when includeBody is false it will schedule to close the S3Object stream automatically on exchange completion.	true	boolean
includeFolders (consumer)	If it is true, the folders/directories will be consumed. If it is false, they will be ignored, and Exchanges will not be created for those.	true	boolean
moveAfterRead (consumer)	Move objects from S3 bucket to a different bucket after they have been retrieved. To accomplish the operation the destinationBucket option must be set. The copy bucket operation is only performed if the Exchange is committed. If a rollback occurs, the object is not moved.	false	boolean
prefix (consumer)	The prefix which is used in the com.amazonaws.services.s3.model.ListObjectsRequest to only consume objects we are interested in.		String
autocloseBody (consumer (advanced))	If this option is true and includeBody is false, then the S3Object.close() method will be called on exchange completion. This option is strongly related to includeBody option. In case of setting includeBody to false and autocloseBody to false, it will be up to the caller to close the S3Object stream. Setting autocloseBody to true, will close the S3Object stream automatically.	true	boolean
batchMessageNumber (producer)	The number of messages composing a batch in streaming upload mode.	10	int
batchSize (producer)	The batch size (in bytes) in streaming upload mode.	100000	int
deleteAfterWrite (producer)	Delete file object after the S3 file has been uploaded.	false	boolean

Name	Description	Default	Type
keyName (producer)	Setting the key name for an element in the bucket through endpoint parameter.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
multiPartUpload (producer)	If it is true, camel will upload the file with multi part format, the part size is decided by the option of partSize.	false	boolean
namingStrategy (producer)	The naming strategy to use in streaming upload mode. Enum values: ● progressive ● random	progressive	AWSS3NamingStrategyEnum
operation (producer)	The operation to do in case the user don't want to do only an upload. Enum values: ● copyObject ● listObjects ● deleteObject ● deleteBucket ● listBuckets ● getObject ● getObjectRange ● createDownloadLink		AWS2S3Operations

Name	Description	Default	Type
partSize (producer)	Setup the partSize which is used in multi part upload, the default size is 25M.	26214400	long
restartingPolicy (producer)	The restarting policy to use in streaming upload mode. Enum values: ● override ● lastPart	override	AWSS3RestartingPolicyEnum
storageClass (producer)	The storage class to set in the <code>com.amazonaws.services.s3.model.PutObjectRequest</code> request.		String
streamingUploadMode (producer)	When stream mode is true the upload to bucket will be done in streaming.	false	boolean
streamingUploadTimeout (producer)	While streaming upload mode is true, this option set the timeout to complete upload.		long
awsKMSKeyId (producer (advanced))	Define the id of KMS key to use in case KMS is enabled.		String
useAwsKMS (producer (advanced))	Define if KMS must be used or not.	false	boolean
useCustomerKey (producer (advanced))	Define if Customer Key must be used or not.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

9.5. ENDPOINT OPTIONS

The AWS S3 Storage Service endpoint is configured using URI syntax:

```
aws2-s3://bucketNameOrArn
```

with the following path and query parameters:

9.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
bucketNameOrArn (common)	Required Bucket name or ARN.		String

9.5.2. Query Parameters (68 parameters)

Name	Description	Default	Type
amazonS3Client (common)	Autowired Reference to a <code>com.amazonaws.services.s3.AmazonS3</code> in the registry.		S3Client
amazonS3Presigner (common)	Autowired An S3 Presigner for Request, used mainly in <code>createDownloadLink</code> operation.		S3Presigner
autoCreateBucket (common)	Setting the autocreation of the S3 bucket <code>bucketName</code> . This will apply also in case of <code>moveAfterRead</code> option enabled and it will create the <code>destinationBucket</code> if it doesn't exist already.	false	boolean
overrideEndpoint (common)	Set the need for overriding the endpoint. This option needs to be used in combination with <code>uriEndpointOverride</code> option.	false	boolean
pojoRequest (common)	If we want to use a POJO request as body or not.	false	boolean
policy (common)	The policy for this queue to set in the <code>com.amazonaws.services.s3.AmazonS3#setBucketPolicy()</code> method.		String
proxyHost (common)	To define a proxy host when instantiating the SQS client.		String
proxyPort (common)	Specify a proxy port to be used inside the client definition.		Integer

Name	Description	Default	Type
proxyProtocol (common)	To define a proxy protocol when instantiating the S3 client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
region (common)	The region in which S3 client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String
trustAllCertificates (common)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (common)	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String
useDefaultCredentialsProvider (common)	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
customerAlgorithm (common (advanced))	Define the customer algorithm to use in case CustomerKey is enabled.		String
customerKeyId (common (advanced))	Define the id of Customer key to use in case CustomerKey is enabled.		String
customerKeyMD5 (common (advanced))	Define the MD5 of Customer key to use in case CustomerKey is enabled.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
deleteAfterRead (consumer)	Delete objects from S3 after they have been retrieved. The delete is only performed if the Exchange is committed. If a rollback occurs, the object is not deleted. If this option is false, then the same objects will be retrieve over and over again on the polls. Therefore you need to use the Idempotent Consumer EIP in the route to filter out duplicates. You can filter using the <code>AWS2S3Constants#BUCKET_NAME</code> and <code>AWS2S3Constants#KEY</code> headers, or only the <code>AWS2S3Constants#KEY</code> header.	true	boolean
delimiter (consumer)	The delimiter which is used in the <code>com.amazonaws.services.s3.model.ListObjectsRequest</code> to only consume objects we are interested in.		String
destinationBucket (consumer)	Define the destination bucket where an object must be moved when <code>moveAfterRead</code> is set to true.		String
destinationBucketPrefix (consumer)	Define the destination bucket prefix to use when an object must be moved and <code>moveAfterRead</code> is set to true.		String
destinationBucketSuffix (consumer)	Define the destination bucket suffix to use when an object must be moved and <code>moveAfterRead</code> is set to true.		String
doneFileName (consumer)	If provided, Camel will only consume files if a done file exists.		String
fileName (consumer)	To get the object from the bucket with the given file name.		String
ignoreBody (consumer)	If it is true, the S3 Object Body will be ignored completely, if it is set to false the S3 Object will be put in the body. Setting this to true, will override any behavior defined by <code>includeBody</code> option.	false	boolean

Name	Description	Default	Type
includeBody (consumer)	If it is true, the S3Object exchange will be consumed and put into the body and closed. If false the S3Object stream will be put raw into the body and the headers will be set with the S3 object metadata. This option is strongly related to autocloseBody option. In case of setting includeBody to true because the S3Object stream will be consumed then it will also be closed, while in case of includeBody false then it will be up to the caller to close the S3Object stream. However setting autocloseBody to true when includeBody is false it will schedule to close the S3Object stream automatically on exchange completion.	true	boolean
includeFolders (consumer)	If it is true, the folders/directories will be consumed. If it is false, they will be ignored, and Exchanges will not be created for those.	true	boolean
maxConnections (consumer)	Set the maxConnections parameter in the S3 client configuration.	60	int
maxMessagesPer Poll (consumer)	Gets the maximum number of messages as a limit to poll at each polling. Gets the maximum number of messages as a limit to poll at each polling. The default value is 10. Use 0 or a negative number to set it as unlimited.	10	int
moveAfterRead (consumer)	Move objects from S3 bucket to a different bucket after they have been retrieved. To accomplish the operation the destinationBucket option must be set. The copy bucket operation is only performed if the Exchange is committed. If a rollback occurs, the object is not moved.	false	boolean
prefix (consumer)	The prefix which is used in the com.amazonaws.services.s3.model.ListObjectsRequest to only consume objects we are interested in.		String
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean

Name	Description	Default	Type
autocloseBody (consumer (advanced))	If this option is true and includeBody is false, then the <code>S3Object.close()</code> method will be called on exchange completion. This option is strongly related to includeBody option. In case of setting includeBody to false and autocloseBody to false, it will be up to the caller to close the <code>S3Object</code> stream. Setting autocloseBody to true, will close the <code>S3Object</code> stream automatically.	true	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an <code>Exchange</code> have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
batchMessageNumber (producer)	The number of messages composing a batch in streaming upload mode.	10	int
batchSize (producer)	The batch size (in bytes) in streaming upload mode.	100000	int
deleteAfterWrite (producer)	Delete file object after the S3 file has been uploaded.	false	boolean
keyName (producer)	Setting the key name for an element in the bucket through endpoint parameter.		String

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
multiPartUpload (producer)	If it is true, camel will upload the file with multi part format, the part size is decided by the option of partSize.	false	boolean
namingStrategy (producer)	The naming strategy to use in streaming upload mode. Enum values: ● progressive ● random	progressive	AWSS3NamingStrategyEnum
operation (producer)	The operation to do in case the user don't want to do only an upload. Enum values: ● copyObject ● listObjects ● deleteObject ● deleteBucket ● listBuckets ● getObject ● getObjectRange ● createDownloadLink		AWS2S3Operations
partSize (producer)	Setup the partSize which is used in multi part upload, the default size is 25M.	26214400	long

Name	Description	Default	Type
restartingPolicy (producer)	The restarting policy to use in streaming upload mode. Enum values: ● <code>override</code> ● <code>lastPart</code>	<code>override</code>	<code>AWSS3RestartingPolicyEnum</code>
storageClass (producer)	The storage class to set in the <code>com.amazonaws.services.s3.model.PutObjectRequest</code> request.		<code>String</code>
streamingUploadMode (producer)	When stream mode is true the upload to bucket will be done in streaming.	<code>false</code>	<code>boolean</code>
streamingUploadTimeout (producer)	While streaming upload mode is true, this option set the timeout to complete upload.		<code>long</code>
awsKMSKeyId (producer (advanced))	Define the id of KMS key to use in case KMS is enabled.		<code>String</code>
useAwsKMS (producer (advanced))	Define if KMS must be used or not.	<code>false</code>	<code>boolean</code>
useCustomerKey (producer (advanced))	Define if Customer Key must be used or not.	<code>false</code>	<code>boolean</code>
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the <code>backoffMultiplier</code> should kick-in.		<code>int</code>
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the <code>backoffMultiplier</code> should kick-in.		<code>int</code>
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then <code>backoffIdleThreshold</code> and/or <code>backoffErrorThreshold</code> must also be configured.		<code>int</code>
delay (scheduler)	Milliseconds before the next poll.	<code>500</code>	<code>long</code>

Name	Description	Default	Type
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean

Name	Description	Default	Type
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

Required S3 component options

You have to provide the `amazonS3Client` in the Registry or your `accessKey` and `secretKey` to access the [Amazon's S3](#).

9.6. BATCH CONSUMER

This component implements the Batch Consumer.

This allows you for instance to know how many messages exists in this batch and for instance let the Aggregator aggregate this number of messages.

9.7. USAGE

For example in order to read file **hello.txt** from bucket **helloBucket**, use the following snippet:

```
from("aws2-s3://helloBucket?
accessKey=yourAccessKey&secretKey=yourSecretKey&prefix=hello.txt")
.to("file:/var/downloaded");
```

9.7.1. Message headers evaluated by the S3 producer

Header	Type	Description
CamelAwsS3BucketName	String	The bucket Name which this object will be stored or which will be used for the current operation
CamelAwsS3BucketDestinationName	String	The bucket Destination Name which will be used for the current operation
CamelAwsS3ContentLength	Long	The content length of this object.
CamelAwsS3ContentType	String	The content type of this object.
CamelAwsS3ContentControl	String	The content control of this object.
CamelAwsS3ContentDisposition	String	The content disposition of this object.
CamelAwsS3ContentEncoding	String	The content encoding of this object.
CamelAwsS3ContentMD5	String	The md5 checksum of this object.
CamelAwsS3DestinationKey	String	The Destination key which will be used for the current operation
CamelAwsS3Key	String	The key under which this object will be stored or which will be used for the current operation
CamelAwsS3LastModified	java.util.Date	The last modified timestamp of this object.
CamelAwsS3Operation	String	The operation to perform. Permitted values are copyObject, deleteObject, listBuckets, deleteBucket, listObjects
CamelAwsS3StorageClass	String	The storage class of this object.
CamelAwsS3CannedAcl	String	The canned acl that will be applied to the object. see software.amazon.awssdk.services.s3.model.ObjectCannedACL for allowed values.
CamelAwsS3Acl	software.amazon.awssdk.services.s3.model.BucketCannedACL	A well constructed Amazon S3 Access Control List object. see software.amazon.awssdk.services.s3.model.BucketCannedACL for more details

Header	Type	Description
CamelAwsS3ServerSideEncryption	String	Sets the server-side encryption algorithm when encrypting the object using AWS-managed keys. For example use AES256.
CamelAwsS3VersionId	String	The version Id of the object to be stored or returned from the current operation
CamelAwsS3Metadata	Map<String, String>	A map of metadata to be stored with the object in S3. More details about metadata .

9.7.2. Message headers set by the S3 producer

Header	Type	Description
CamelAwsS3ETag	String	The ETag value for the newly uploaded object.
CamelAwsS3VersionId	String	The optional version ID of the newly uploaded object.

9.7.3. Message headers set by the S3 consumer

Header	Type	Description
CamelAwsS3Key	String	The key under which this object is stored.
CamelAwsS3BucketName	String	The name of the bucket in which this object is contained.
CamelAwsS3ETag	String	The hex encoded 128-bit MD5 digest of the associated object according to RFC 1864. This data is used as an integrity check to verify that the data received by the caller is the same data that was sent by Amazon S3.
CamelAwsS3LastModified	Date	The value of the Last-Modified header, indicating the date and time at which Amazon S3 last recorded a modification to the associated object.
CamelAwsS3VersionId	String	The version ID of the associated Amazon S3 object if available. Version IDs are only assigned to objects when an object is uploaded to an Amazon S3 bucket that has object versioning enabled.

Header	Type	Description
CamelAwsS3ContentType	String	The Content-Type HTTP header, which indicates the type of content stored in the associated object. The value of this header is a standard MIME type.
CamelAwsS3ContentMD5	String	The base64 encoded 128-bit MD5 digest of the associated object (content - not including headers) according to RFC 1864. This data is used as a message integrity check to verify that the data received by Amazon S3 is the same data that the caller sent.
CamelAwsS3ContentLength	Long	The Content-Length HTTP header indicating the size of the associated object in bytes.
CamelAwsS3ContentEncoding	String	The optional Content-Encoding HTTP header specifying what content encodings have been applied to the object and what decoding mechanisms must be applied in order to obtain the media-type referenced by the Content-Type field.
CamelAwsS3ContentDisposition	String	The optional Content-Disposition HTTP header, which specifies presentational information such as the recommended filename for the object to be saved as.
CamelAwsS3ContentControl	String	The optional Cache-Control HTTP header which allows the user to specify caching behavior along the HTTP request/reply chain.
CamelAwsS3ServerSideEncryption	String	The server-side encryption algorithm when encrypting the object using AWS-managed keys.
CamelAwsS3Metadata	Map<String, String>	A map of metadata stored with the object in S3. More details about metadata .

9.7.4. S3 Producer operations

Camel-AWS2-S3 component provides the following operation on the producer side:

- copyObject
- deleteObject
- listBuckets
- deleteBucket
- listObjects

- getObject (this will return an S3Object instance)
- getObjectRange (this will return an S3Object instance)
- createDownloadLink

If you don't specify an operation explicitly the producer will do: - a single file upload - a multipart upload if multiPartUpload option is enabled.

9.7.5. Advanced AmazonS3 configuration

If your Camel Application is running behind a firewall or if you need to have more control over the **S3Client** instance configuration, you can create your own instance and refer to it in your Camel aws2-s3 component configuration:

```
from("aws2-s3://MyBucket?amazonS3Client=#client&delay=5000&maxMessagesPerPoll=5")
.to("mock:result");
```

9.7.6. Use KMS with the S3 component

To use AWS KMS to encrypt/decrypt data by using AWS infrastructure you can use the options introduced in 2.21.x like in the following example

```
from("file:tmp/test?fileName=test.txt")
.setHeader(S3Constants.KEY, constant("testFile"))
.to("aws2-s3://mybucket?amazonS3Client=#client&useAwsKMS=true&awsKMSKeyId=3f0637ad-296a-3dfe-a796-e60654fb128c");
```

In this way you'll ask to S3, to use the KMS key 3f0637ad-296a-3dfe-a796-e60654fb128c, to encrypt the file test.txt. When you'll ask to download this file, the decryption will be done directly before the download.

9.7.7. Static credentials vs Default Credential Provider

You have the possibility of avoiding the usage of explicit static credentials, by specifying the useDefaultCredentialsProvider option and set it to true.

- Java system properties - aws.accessKeyId and aws.secretKey
- Environment variables - AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY.
- Web Identity Token from AWS STS.
- The shared credentials and config files.
- Amazon ECS container credentials - loaded from the Amazon ECS if the environment variable AWS_CONTAINER_CREDENTIALS_RELATIVE_URI is set.
- Amazon EC2 Instance profile credentials.

For more information about this you can look at [AWS credentials documentation](#)

9.7.8. S3 Producer Operation examples

- Single Upload: This operation will upload a file to S3 based on the body content

```
from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(S3Constants.KEY, "camel.txt");
        exchange.getIn().setBody("Camel rocks!");
    }
})
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client")
.to("mock:result");
```

This operation will upload the file camel.txt with the content "Camel rocks!" in the mycamelbucket bucket

- Multipart Upload: This operation will perform a multipart upload of a file to S3 based on the body content

```
from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(AWS2S3Constants.KEY, "empty.txt");
        exchange.getIn().setBody(new File("src/empty.txt"));
    }
})
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client&multiPartUpload=true&autoCreateBucket=true&partSize=1048576'")
.to("mock:result");
```

This operation will perform a multipart upload of the file empty.txt with based on the content the file src/empty.txt in the mycamelbucket bucket

- CopyObject: this operation copy an object from one bucket to a different one

```
from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(S3Constants.BUCKET_DESTINATION_NAME,
"camelDestinationBucket");
        exchange.getIn().setHeader(S3Constants.KEY, "camelKey");
        exchange.getIn().setHeader(S3Constants.DESTINATION_KEY, "camelDestinationKey");
    }
})
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client&operation=copyObject")
.to("mock:result");
```

This operation will copy the object with the name expressed in the header camelDestinationKey to the camelDestinationBucket bucket, from the bucket mycamelbucket.

- DeleteObject: this operation deletes an object from a bucket

```

from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(S3Constants.KEY, "camelKey");
    }
})
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client&operation=deleteObject")
.to("mock:result");

```

This operation will delete the object camelKey from the bucket mycamelbucket.

- ListBuckets: this operation list the buckets for this account in this region

```

from("direct:start")
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client&operation=listBuckets")
.to("mock:result");

```

This operation will list the buckets for this account

- DeleteBucket: this operation delete the bucket specified as URI parameter or header

```

from("direct:start")
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client&operation=deleteBucket")
.to("mock:result");

```

This operation will delete the bucket mycamelbucket

- ListObjects: this operation list object in a specific bucket

```

from("direct:start")
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client&operation=listObjects")
.to("mock:result");

```

This operation will list the objects in the mycamelbucket bucket

- GetObject: this operation get a single object in a specific bucket

```

from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(S3Constants.KEY, "camelKey");
    }
})
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client&operation=getObject")
.to("mock:result");

```

This operation will return an S3Object instance related to the camelKey object in mycamelbucket bucket.

- GetObjectRange: this operation get a single object range in a specific bucket

```

from("direct:start").process(new Processor() {

```

```

@Override
public void process(Exchange exchange) throws Exception {
    exchange.getIn().setHeader(S3Constants.KEY, "camelKey");
    exchange.getIn().setHeader(S3Constants.RANGE_START, "0");
    exchange.getIn().setHeader(S3Constants.RANGE_END, "9");
}
})
.to("aws2-s3://mycamelbucket?amazonS3Client=#amazonS3Client&operation=getObjectRange")
.to("mock:result");

```

This operation will return an S3Object instance related to the camelKey object in mycamelbucket bucket, containing a the bytes from 0 to 9.

- CreateDownloadLink: this operation will return a download link through S3 Presigner

```

from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(S3Constants.KEY, "camelKey");
    }
})
.to("aws2-s3://mycamelbucket?
accessKey=xxx&secretKey=yyy&region=region&operation=createDownloadLink")
.to("mock:result");

```

This operation will return a download link url for the file camel-key in the bucket mycamelbucket and region region

9.8. STREAMING UPLOAD MODE

With the stream mode enabled users will be able to upload data to S3 without knowing ahead of time the dimension of the data, by leveraging multipart upload. The upload will be completed when: the batchSize has been completed or the batchSize has been reached. There are two possible naming strategy:

- progressive
With the progressive strategy each file will have the name composed by keyName option and a progressive counter, and eventually the file extension (if any)
- random.
With the random strategy a UUID will be added after keyName and eventually the file extension will appended.

As an example:

```

from(kafka("topic1").brokers("localhost:9092"))
    .log("Kafka Message is: ${body}")
    .to(aws2S3("camel-
bucket").streamingUploadMode(true).batchMessageNumber(25).namingStrategy(AWS2S3EndpointBu
ilderFactory.AWSS3NamingStrategyEnum.progressive).keyName("
{{kafkaTopic1}}/{{kafkaTopic1}}.txt"));

from(kafka("topic2").brokers("localhost:9092"))

```

```

        .log("Kafka Message is: ${body}")
        .to(aws2S3("camel-
bucket").streamingUploadMode(true).batchMessageNumber(25).namingStrategy(AWS2S3EndpointBu
ilderFactory.AWSS3NamingStrategyEnum.progressive).keyName("
{{kafkaTopic2}}/{{kafkaTopic2}}.txt"));

```

The default size for a batch is 1 Mb, but you can adjust it according to your requirements.

When you'll stop your producer route, the producer will take care of flushing the remaining buffered messaged and complete the upload.

In Streaming upload you'll be able restart the producer from the point where it left. It's important to note that this feature is critical only when using the progressive naming strategy.

By setting the restartingPolicy to lastPart, you will restart uploading files and contents from the last part number the producer left.

Example

1. Start the route with progressive naming strategy and keyname equals to camel.txt, with batchMessageNumber equals to 20, and restartingPolicy equals to lastPart - Send 70 messages.
2. Stop the route
3. On your S3 bucket you should now see 4 files: * camel.txt
 - camel-1.txt
 - camel-2.txt
 - camel-3.txt

The first three will have 20 messages, while the last one only 10.
4. Restart the route.
5. Send 25 messages.
6. Stop the route.
7. You'll now have 2 other files in your bucket: camel-5.txt and camel-6.txt, the first with 20 messages and second with 5 messages.
8. Go ahead

This won't be needed when using the random naming strategy.

On the opposite you can specify the override restartingPolicy. In that case you'll be able to override whatever you written before (for that particular keyName) on your bucket.



NOTE

In Streaming upload mode the only keyName option that will be taken into account is the endpoint option. Using the header will throw an NPE and this is done by design. Setting the header means potentially change the file name on each exchange and this is against the aim of the streaming upload producer. The keyName needs to be fixed and static. The selected naming strategy will do the rest of the work.

Another possibility is specifying a `streamingUploadTimeout` with `batchMessageNumber` and `batchSize` options. With this option the user will be able to complete the upload of a file after a certain time passed. In this way the upload completion will be passed on three tiers: the timeout, the number of messages and the batch size.

As an example:

```
from(kafka("topic1").brokers("localhost:9092"))
  .log("Kafka Message is: ${body}")
  .to(aws2S3("camel-
bucket").streamingUploadMode(true).batchMessageNumber(25).streamingUploadTimeout(10000).na
mingStrategy(AWS2S3EndpointBuilderFactory.AWSS3NamingStrategyEnum.progressive).keyName(
"{{kafkaTopic1}}/{{kafkaTopic1}}.txt"));
```

In this case the upload will be completed after 10 seconds.

9.9. BUCKET AUTOCREATION

With the option **autoCreateBucket** users are able to avoid the autocreation of an S3 Bucket in case it doesn't exist. The default for this option is **true**. If set to false any operation on a not-existent bucket in AWS won't be successful and an error will be returned.

9.10. MOVING STUFF BETWEEN A BUCKET AND ANOTHER BUCKET

Some users like to consume stuff from a bucket and move the content in a different one without using the `copyObject` feature of this component. If this is case for you, don't forget to remove the `bucketName` header from the incoming exchange of the consumer, otherwise the file will be always overwritten on the same original bucket.

9.11. MOVEAFTERREAD CONSUMER OPTION

In addition to `deleteAfterRead` it has been added another option, `moveAfterRead`. With this option enabled the consumed object will be moved to a target `destinationBucket` instead of being only deleted. This will require specifying the `destinationBucket` option. As example:

```
from("aws2-s3://mycamelbucket?
amazonS3Client=#amazonS3Client&moveAfterRead=true&destinationBucket=myothercamelbucket")
.to("mock:result");
```

In this case the objects consumed will be moved to `myothercamelbucket` bucket and deleted from the original one (because of `deleteAfterRead` set to true as default).

You have also the possibility of using a key prefix/suffix while moving the file to a different bucket. The options are `destinationBucketPrefix` and `destinationBucketSuffix`.

Taking the above example, you could do something like:

```
from("aws2-s3://mycamelbucket?
amazonS3Client=#amazonS3Client&moveAfterRead=true&destinationBucket=myothercamelbucket&de
stinationBucketPrefix=RAW(pre-)&destinationBucketSuffix=RAW(-suff)")
.to("mock:result");
```

In this case the objects consumed will be moved to myothercamelbucket bucket and deleted from the original one (because of deleteAfterRead set to true as default).

So if the file name is test, in the myothercamelbucket you should see a file called pre-test-suff.

9.12. USING CUSTOMER KEY AS ENCRYPTION

We introduced also the customer key support (an alternative of using KMS). The following code shows an example.

```
String key = UUID.randomUUID().toString();
byte[] secretKey = generateSecretKey();
String b64Key = Base64.getEncoder().encodeToString(secretKey);
String b64KeyMd5 = Md5Utils.md5AsBase64(secretKey);

String awsEndpoint = "aws2-s3://mycamel?
autoCreateBucket=false&useCustomerKey=true&customerKeyId=RAW(" + b64Key +
"&customerKeyMD5=RAW(" + b64KeyMd5 + "&customerAlgorithm=" + AES256.name());

from("direct:putObject")
    .setHeader(AWS2S3Constants.KEY, constant("test.txt"))
    .setBody(constant("Test"))
    .to(awsEndpoint);
```

9.13. USING A POJO AS BODY

Sometimes build an AWS Request can be complex, because of multiple options. We introduce the possibility to use a POJO as body. In AWS S3 there are multiple operations you can submit, as an example for List brokers request, you can do something like:

```
from("direct:aws2-s3")
    .setBody(ListObjectsRequest.builder().bucket(bucketName).build())
    .to("aws2-s3://test?
amazonS3Client=#amazonS3Client&operation=listObjects&pojoRequest=true")
```

In this way you'll pass the request directly without the need of passing headers and options specifically related to this operation.

9.14. CREATE S3 CLIENT AND ADD COMPONENT TO REGISTRY

Sometimes you would want to perform some advanced configuration using AWS2S3Configuration which also allows to set the S3 client. You can create and set the S3 client in the component configuration as shown in the following example

```
String awsBucketAccessKey = "your_access_key";
String awsBucketSecretKey = "your_secret_key";

S3Client s3Client =
S3Client.builder().credentialsProvider(StaticCredentialsProvider.create(AwsBasicCredentials.create(aws
BucketAccessKey, awsBucketSecretKey)))
    .region(Region.US_EAST_1).build();

AWS2S3Configuration configuration = new AWS2S3Configuration();
```



```
configuration.setAmazonS3Client(s3Client);
configuration.setAutoDiscoverClient(true);
configuration.setBucketName("s3bucket2020");
configuration.setRegion("us-east-1");
```

Now you can configure the S3 component (using the configuration object created above) and add it to the registry in the configure method before initialization of routes.

```
AWS2S3Component s3Component = new AWS2S3Component(getContext());
s3Component.setConfiguration(configuration);
s3Component.setLazyStartProducer(true);
camelContext.addComponent("aws2-s3", s3Component);
```

Now your component will be used for all the operations implemented in camel routes.

9.15. DEPENDENCIES

Maven users will need to add the following dependency to their **pom.xml**.

pom.xml

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-aws2-s3</artifactId>
  <version>${camel-version}</version>
</dependency>
```

where **{camel-version}** must be replaced by the actual version of Camel.

9.16. SPRING BOOT AUTO-CONFIGURATION

The component supports 51 options, which are listed below.

Name	Description	Default	Type
camel.component.aws2-s3.access-key	Amazon AWS Access Key.		String
camel.component.aws2-s3.amazon-s3-client	Reference to a <code>com.amazonaws.services.s3.AmazonS3</code> in the registry. The option is a <code>software.amazon.awssdk.services.s3.S3Client</code> type.		S3Client
camel.component.aws2-s3.amazon-s3-presigner	An S3 Presigner for Request, used mainly in <code>createDownloadLink</code> operation. The option is a <code>software.amazon.awssdk.services.s3.presigner.S3Presigner</code> type.		S3Presigner

Name	Description	Default	Type
camel.component.aws2-s3.auto-create-bucket	Setting the autocreation of the S3 bucket bucketName. This will apply also in case of moveAfterRead option enabled and it will create the destinationBucket if it doesn't exist already.	false	Boolean
camel.component.aws2-s3.autoclose-body	If this option is true and includeBody is false, then the S3Object.close() method will be called on exchange completion. This option is strongly related to includeBody option. In case of setting includeBody to false and autocloseBody to false, it will be up to the caller to close the S3Object stream. Setting autocloseBody to true, will close the S3Object stream automatically.	true	Boolean
camel.component.aws2-s3.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.aws2-s3.aws-kms-key-id	Define the id of KMS key to use in case KMS is enabled.		String
camel.component.aws2-s3.batch-message-number	The number of messages composing a batch in streaming upload mode.	10	Integer
camel.component.aws2-s3.batch-size	The batch size (in bytes) in streaming upload mode.	1000000	Integer
camel.component.aws2-s3.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.aws2-s3.configuration</code>	The component configuration. The option is a <code>org.apache.camel.component.aws2.s3.AWS2S3Configuration</code> type.		<code>AWS2S3Configuration</code>
<code>camel.component.aws2-s3.customer-algorithm</code>	Define the customer algorithm to use in case <code>CustomerKey</code> is enabled.		String
<code>camel.component.aws2-s3.customer-key-id</code>	Define the id of Customer key to use in case <code>CustomerKey</code> is enabled.		String
<code>camel.component.aws2-s3.customer-key-md5</code>	Define the MD5 of Customer key to use in case <code>CustomerKey</code> is enabled.		String
<code>camel.component.aws2-s3.delete-after-read</code>	Delete objects from S3 after they have been retrieved. The delete is only performed if the Exchange is committed. If a rollback occurs, the object is not deleted. If this option is false, then the same objects will be retrieve over and over again on the polls. Therefore you need to use the Idempotent Consumer EIP in the route to filter out duplicates. You can filter using the <code>AWS2S3Constants#BUCKET_NAME</code> and <code>AWS2S3Constants#KEY</code> headers, or only the <code>AWS2S3Constants#KEY</code> header.	true	Boolean
<code>camel.component.aws2-s3.delete-after-write</code>	Delete file object after the S3 file has been uploaded.	false	Boolean
<code>camel.component.aws2-s3.delimiter</code>	The delimiter which is used in the <code>com.amazonaws.services.s3.model.ListObjectsRequest</code> to only consume objects we are interested in.		String
<code>camel.component.aws2-s3.destination-bucket</code>	Define the destination bucket where an object must be moved when <code>moveAfterRead</code> is set to true.		String
<code>camel.component.aws2-s3.destination-bucket-prefix</code>	Define the destination bucket prefix to use when an object must be moved and <code>moveAfterRead</code> is set to true.		String

Name	Description	Default	Type
camel.component.aws2-s3.destination-bucket-suffix	Define the destination bucket suffix to use when an object must be moved and moveAfterRead is set to true.		String
camel.component.aws2-s3.done-file-name	If provided, Camel will only consume files if a done file exists.		String
camel.component.aws2-s3.enabled	Whether to enable auto configuration of the aws2-s3 component. This is enabled by default.		Boolean
camel.component.aws2-s3.file-name	To get the object from the bucket with the given file name.		String
camel.component.aws2-s3.ignore-body	If it is true, the S3 Object Body will be ignored completely, if it is set to false the S3 Object will be put in the body. Setting this to true, will override any behavior defined by includeBody option.	false	Boolean
camel.component.aws2-s3.include-body	If it is true, the S3Object exchange will be consumed and put into the body and closed. If false the S3Object stream will be put raw into the body and the headers will be set with the S3 object metadata. This option is strongly related to autocloseBody option. In case of setting includeBody to true because the S3Object stream will be consumed then it will also be closed, while in case of includeBody false then it will be up to the caller to close the S3Object stream. However setting autocloseBody to true when includeBody is false it will schedule to close the S3Object stream automatically on exchange completion.	true	Boolean
camel.component.aws2-s3.include-folders	If it is true, the folders/directories will be consumed. If it is false, they will be ignored, and Exchanges will not be created for those.	true	Boolean
camel.component.aws2-s3.key-name	Setting the key name for an element in the bucket through endpoint parameter.		String

Name	Description	Default	Type
camel.component.aws2-s3.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.aws2-s3.move-after-read	Move objects from S3 bucket to a different bucket after they have been retrieved. To accomplish the operation the destinationBucket option must be set. The copy bucket operation is only performed if the Exchange is committed. If a rollback occurs, the object is not moved.	false	Boolean
camel.component.aws2-s3.multi-part-upload	If it is true, camel will upload the file with multi part format, the part size is decided by the option of partSize.	false	Boolean
camel.component.aws2-s3.naming-strategy	The naming strategy to use in streaming upload mode.		AWSS3NamingStrategyEnum
camel.component.aws2-s3.operation	The operation to do in case the user don't want to do only an upload.		AWS2S3Operations
camel.component.aws2-s3.override-endpoint	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	Boolean
camel.component.aws2-s3.part-size	Setup the partSize which is used in multi part upload, the default size is 25M.	26214400	Long
camel.component.aws2-s3.pojo-request	If we want to use a POJO request as body or not.	false	Boolean
camel.component.aws2-s3.policy	The policy for this queue to set in the com.amazonaws.services.s3.AmazonS3#setBucketPolicy() method.		String

Name	Description	Default	Type
<code>camel.component.aws2-s3.prefix</code>	The prefix which is used in the <code>com.amazonaws.services.s3.model.ListObjectsRequest</code> to only consume objects we are interested in.		String
<code>camel.component.aws2-s3.proxy-host</code>	To define a proxy host when instantiating the SQS client.		String
<code>camel.component.aws2-s3.proxy-port</code>	Specify a proxy port to be used inside the client definition.		Integer
<code>camel.component.aws2-s3.proxy-protocol</code>	To define a proxy protocol when instantiating the S3 client.		Protocol
<code>camel.component.aws2-s3.region</code>	The region in which S3 client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example <code>ap-east-1</code>) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
<code>camel.component.aws2-s3.restarting-policy</code>	The restarting policy to use in streaming upload mode.		<code>AWSS3RestartingPolicyEnum</code>
<code>camel.component.aws2-s3.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-s3.storage-class</code>	The storage class to set in the <code>com.amazonaws.services.s3.model.PutObjectRequest</code> request.		String
<code>camel.component.aws2-s3.streaming-upload-mode</code>	When stream mode is true the upload to bucket will be done in streaming.	false	Boolean
<code>camel.component.aws2-s3.streaming-upload-timeout</code>	While streaming upload mode is true, this option set the timeout to complete upload.		Long
<code>camel.component.aws2-s3.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean

Name	Description	Default	Type
<code>camel.component.aws2-s3.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
<code>camel.component.aws2-s3.use-aws-k-m-s</code>	Define if KMS must be used or not.	false	Boolean
<code>camel.component.aws2-s3.use-customer-key</code>	Define if Customer Key must be used or not.	false	Boolean
<code>camel.component.aws2-s3.use-default-credentials-provider</code>	Set whether the S3 client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	Boolean

CHAPTER 10. AWS SIMPLE NOTIFICATION SYSTEM (SNS)

Only producer is supported

The AWS2 SNS component allows messages to be sent to an [Amazon Simple Notification](#) Topic. The implementation of the Amazon API is provided by the [AWS SDK](#).

Prerequisites

You must have a valid Amazon Web Services developer account, and be signed up to use Amazon SNS. More information is available at [Amazon SNS](#).

10.1. DEPENDENCIES

When using **aws2-sns** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-aws2-sns-starter</artifactId>
</dependency>
```

10.2. URI FORMAT

```
aws2-sns://topicNameOrArn[?options]
```

The topic will be created if they don't already exists. You can append query options to the URI in the following format,

```
?options=value&option2=value&...
```

10.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

10.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).

- directly in the Java code.

10.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

10.4. COMPONENT OPTIONS

The AWS Simple Notification System (SNS) component supports 24 options, which are listed below.

Name	Description	Default	Type
amazonSNSClient (producer)	Autowired To use the AmazonSNS as the client.		SnsClient
autoCreateTopic (producer)	Setting the autocreation of the topic.	false	boolean
configuration (producer)	Component configuration.		Sns2Configuration
kmsMasterKeyId (producer)	The ID of an AWS-managed customer master key (CMK) for Amazon SNS or a custom CMK.		String

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
messageDeduplicationIdStrategy (producer)	Only for FIFO Topic. Strategy for setting the messageDeduplicationId on the message. Can be one of the following options: useExchangeId, useContentBasedDeduplication. For the useContentBasedDeduplication option, no messageDeduplicationId will be set on the message. Enum values: ● useExchangeId ● useContentBasedDeduplication	useExchangeId	String
messageGroupIdStrategy (producer)	Only for FIFO Topic. Strategy for setting the messageGroupId on the message. Can be one of the following options: useConstant, useExchangeId, usePropertyValue. For the usePropertyValue option, the value of property CamelAwsMessageGroupId will be used. Enum values: ● useConstant ● useExchangeId ● usePropertyValue		String
messageStructure (producer)	The message structure to use such as json.		String
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
policy (producer)	The policy for this topic. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String

Name	Description	Default	Type
proxyHost (producer)	To define a proxy host when instantiating the SNS client.		String
proxyPort (producer)	To define a proxy port when instantiating the SNS client.		Integer
proxyProtocol (producer)	To define a proxy protocol when instantiating the SNS client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
queueUrl (producer)	The queueUrl to subscribe to.		String
region (producer)	The region in which SNS client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String
serverSideEncryptionEnabled (producer)	Define if Server Side Encryption is enabled or not on the topic.	false	boolean
subject (producer)	The subject which is used if the message header 'CamelAwsSnsSubject' is not present.		String
subscribeSNSstoSQS (producer)	Define if the subscription between SNS Topic and SQS must be done or not.	false	boolean
trustAllCertificates (producer)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String
useDefaultCredentialsProvider (producer)	Set whether the SNS client should expect to load credentials on an AWS infra instance or to expect static credentials to be passed in.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

10.5. ENDPOINT OPTIONS

The AWS Simple Notification System (SNS) endpoint is configured using URI syntax:

```
aws2-sns:topicNameOrArn
```

with the following path and query parameters:

10.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
topicNameOrArn (producer)	Required Topic name or ARN.		String

10.5.2. Query Parameters (23 parameters)

Name	Description	Default	Type
amazonSNSClient (producer)	Autowired To use the AmazonSNS as the client.		SnsClient
autoCreateTopic (producer)	Setting the autocreation of the topic.	false	boolean
headerFilterStrategy (producer)	To use a custom HeaderFilterStrategy to map headers to/from Camel.		HeaderFilterStrategy

Name	Description	Default	Type
kmsMasterKeyId (producer)	The ID of an AWS-managed customer master key (CMK) for Amazon SNS or a custom CMK.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
messageDeduplicationIdStrategy (producer)	Only for FIFO Topic. Strategy for setting the messageDeduplicationId on the message. Can be one of the following options: useExchangedId, useContentBasedDeduplication. For the useContentBasedDeduplication option, no messageDeduplicationId will be set on the message. Enum values: ● useExchangedId ● useContentBasedDeduplication	useExchangedId	String
messageGroupIdStrategy (producer)	Only for FIFO Topic. Strategy for setting the messageGroupId on the message. Can be one of the following options: useConstant, useExchangedId, usePropertyValue. For the usePropertyValue option, the value of property CamelAwsMessageGroupId will be used. Enum values: ● useConstant ● useExchangedId ● usePropertyValue		String
messageStructure (producer)	The message structure to use such as json.		String
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean

Name	Description	Default	Type
policy (producer)	The policy for this topic. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
proxyHost (producer)	To define a proxy host when instantiating the SNS client.		String
proxyPort (producer)	To define a proxy port when instantiating the SNS client.		Integer
proxyProtocol (producer)	To define a proxy protocol when instantiating the SNS client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
queueUrl (producer)	The queueUrl to subscribe to.		String
region (producer)	The region in which SNS client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String
serverSideEncryptionEnabled (producer)	Define if Server Side Encryption is enabled or not on the topic.	false	boolean
subject (producer)	The subject which is used if the message header 'CamelAwsSnsSubject' is not present.		String
subscribeSNStoSQS (producer)	Define if the subscription between SNS Topic and SQS must be done or not.	false	boolean
trustAllCertificates (producer)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String
useDefaultCredentialsProvider (producer)	Set whether the SNS client should expect to load credentials on an AWS infra instance or to expect static credentials to be passed in.	false	boolean

Name	Description	Default	Type
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

Required SNS component options

You have to provide the `amazonSNSClient` in the Registry or your `accessKey` and `secretKey` to access the [Amazon's SNS](#).

10.6. USAGE

10.6.1. Static credentials vs Default Credential Provider

You have the possibility of avoiding the usage of explicit static credentials, by specifying the `useDefaultCredentialsProvider` option and set it to `true`.

- Java system properties - `aws.accessKeyId` and `aws.secretKey`
- Environment variables - `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`.
- Web Identity Token from AWS STS.
- The shared credentials and config files.
- Amazon ECS container credentials - loaded from the Amazon ECS if the environment variable `AWS_CONTAINER_CREDENTIALS_RELATIVE_URI` is set.
- Amazon EC2 Instance profile credentials.

For more information about this you can look at [AWS credentials documentation](#).

10.6.2. Message headers evaluated by the SNS producer

Header	Type	Description
CamelAwsSnsSubject	String	The Amazon SNS message subject. If not set, the subject from the SnsConfiguration is used.

10.6.3. Message headers set by the SNS producer

Header	Type	Description
CamelAwsSnsMessageId	String	The Amazon SNS message ID.

10.6.4. Advanced AmazonSNS configuration

If you need more control over the **SnsClient** instance configuration you can create your own instance and refer to it from the URI:

```
from("direct:start")
.to("aws2-sns://MyTopic?amazonSNSClient=#client");
```

The **#client** refers to a **AmazonSNS** in the Registry.

10.6.5. Create a subscription between an AWS SNS Topic and an AWS SQS Queue

You can create a subscription of an SQS Queue to an SNS Topic in this way:

```
from("direct:start")
.to("aws2-sns://test-camel-sns1?
amazonSNSClient=#amazonSNSClient&subscribeSNStoSQS=true&queueUrl=https://sqs.eu-central-
1.amazonaws.com/780410022472/test-camel");
```

The **#amazonSNSClient** refers to a **SnsClient** in the Registry. By specifying **subscribeSNStoSQS** to true and a **queueUrl** of an existing SQS Queue, you'll be able to subscribe your SQS Queue to your SNS Topic.

At this point you can consume messages coming from SNS Topic through your SQS Queue

```
from("aws2-sqs://test-camel?
amazonSQSClient=#amazonSQSClient&delay=50&maxMessagesPerPoll=5")
.to(...);
```

10.7. TOPIC AUTOCREATION

With the option **autoCreateTopic** users are able to avoid the autocreation of an SNS Topic in case it doesn't exist. The default for this option is **true**. If set to false any operation on a not-existent topic in AWS won't be successful and an error will be returned.

10.8. SNS FIFO

SNS FIFO are supported. While creating the SQS queue you will subscribe to the SNS topic there is an important point to remember, you'll need to make possible for the SNS Topic to send message to the SQS Queue.

Example

Suppose you created an SNS FIFO Topic called **Order.fifo** and an SQS Queue called **QueueSub.fifo**.

In the access Policy of the **QueueSub.fifo** you should submit something like this:

```
{
  "Version": "2008-10-17",
  "Id": "__default_policy_ID",
  "Statement": [
    {
      "Sid": "__owner_statement",
```



```

    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::780560123482:root"
    },
    "Action": "SQS:*",
    "Resource": "arn:aws:sqs:eu-west-1:780560123482:QueueSub.fifo"
  },
  {
    "Effect": "Allow",
    "Principal": {
      "Service": "sns.amazonaws.com"
    },
    "Action": "SQS:SendMessage",
    "Resource": "arn:aws:sqs:eu-west-1:780560123482:QueueSub.fifo",
    "Condition": {
      "ArnLike": {
        "aws:SourceArn": "arn:aws:sns:eu-west-1:780410022472:Order.fifo"
      }
    }
  }
]
}

```

This is a critical step to make the subscription work correctly.

10.8.1. SNS Fifo Topic Message group Id Strategy and message Deduplication Id Strategy

When sending something to the FIFO topic you'll need to always set up a message group Id strategy.

If the content-based message deduplication has been enabled on the SNS Fifo topic, there won't be the need of setting a message deduplication id strategy, otherwise you'll have to set it.

10.9. EXAMPLES

10.9.1. Producer Examples

Sending to a topic

```

from("direct:start")
.to("aws2-sns://camel-topic?subject=The+subject+message&autoCreateTopic=true");

```

10.10. DEPENDENCIES

Maven users will need to add the following dependency to their pom.xml.

pom.xml

```

<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-aws2-sns</artifactId>
  <version>${camel-version}</version>
</dependency>

```

where **{camel-version}** must be replaced by the actual version of Camel.

10.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 25 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.aws2-sns.access-key</code>	Amazon AWS Access Key.		String
<code>camel.component.aws2-sns.amazon-sns-client</code>	To use the AmazonSNS as the client. The option is a <code>software.amazon.awssdk.services.sns.SnsClient</code> type.		SnsClient
<code>camel.component.aws2-sns.auto-create-topic</code>	Setting the autocreation of the topic.	false	Boolean
<code>camel.component.aws2-sns.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.aws2-sns.configuration</code>	Component configuration. The option is a <code>org.apache.camel.component.aws2.sns.Sns2Configuration</code> type.		Sns2Configuration
<code>camel.component.aws2-sns.enabled</code>	Whether to enable auto configuration of the aws2-sns component. This is enabled by default.		Boolean
<code>camel.component.aws2-sns.kms-master-key-id</code>	The ID of an AWS-managed customer master key (CMK) for Amazon SNS or a custom CMK.		String

Name	Description	Default	Type
camel.component .aws2-sns.lazy- start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component .aws2- sns.message- deduplication-id- strategy	Only for FIFO Topic. Strategy for setting the messageDeduplicationId on the message. Can be one of the following options: useExchangeId, useContentBasedDeduplication. For the useContentBasedDeduplication option, no messageDeduplicationId will be set on the message.	useExchangeId	String
camel.component .aws2- sns.message- group-id- strategy	Only for FIFO Topic. Strategy for setting the messageGroupId on the message. Can be one of the following options: useConstant, useExchangeId, usePropertyValue. For the usePropertyValue option, the value of property CamelAwsMessageGroupId will be used.		String
camel.component .aws2- sns.message- structure	The message structure to use such as json.		String
camel.component .aws2- sns.override- endpoint	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	Boolean
camel.component .aws2-sns.policy	The policy for this topic. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
camel.component .aws2-sns.proxy- host	To define a proxy host when instantiating the SNS client.		String
camel.component .aws2-sns.proxy- port	To define a proxy port when instantiating the SNS client.		Integer

Name	Description	Default	Type
<code>camel.component.aws2-sns.proxy-protocol</code>	To define a proxy protocol when instantiating the SNS client.		Protocol
<code>camel.component.aws2-sns.queue-url</code>	The queueUrl to subscribe to.		String
<code>camel.component.aws2-sns.region</code>	The region in which SNS client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
<code>camel.component.aws2-sns.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-sns.server-side-encryption-enabled</code>	Define if Server Side Encryption is enabled or not on the topic.	false	Boolean
<code>camel.component.aws2-sns.subject</code>	The subject which is used if the message header 'CamelAwsSnsSubject' is not present.		String
<code>camel.component.aws2-sns.subscribe-s-n-sto-s-q-s</code>	Define if the subscription between SNS Topic and SQS must be done or not.	false	Boolean
<code>camel.component.aws2-sns.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean
<code>camel.component.aws2-sns.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
<code>camel.component.aws2-sns.use-default-credentials-provider</code>	Set whether the SNS client should expect to load credentials on an AWS infra instance or to expect static credentials to be passed in.	false	Boolean

CHAPTER 11. AWS SIMPLE QUEUE SERVICE (SQS)

Both producer and consumer are supported

The AWS2 SQS component supports sending and receiving messages to [Amazon's SQS service](#).

Prerequisites

You must have a valid Amazon Web Services developer account, and be signed up to use Amazon SQS. More information is available at [Amazon SQS](#).

11.1. DEPENDENCIES

When using aws2-sqs with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-aws2-sqs-starter</artifactId>
</dependency>
```

11.2. URI FORMAT

```
aws2-sqs://queueNameOrArn[?options]
```

The queue will be created if they don't already exists. You can append query options to the URI in the following format,

?options=value&option2=value&...

11.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

11.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).

- directly in the Java code.

11.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

11.4. COMPONENT OPTIONS

The AWS Simple Queue Service (SQS) component supports 43 options, which are listed below.

Name	Description	Default	Type
amazonAWSHost (common)	The hostname of the Amazon AWS cloud.	amazonsqs.com	String
amazonSQSClient (common)	Autowired To use the AmazonSQS as client.		SqsClient
autoCreateQueue (common)	Setting the autocreation of the queue.	false	boolean
configuration (common)	The AWS SQS default configuration.		Sqs2Configuration
overrideEndpoint (common)	Set the need for overriding the endpoint. This option needs to be used in combination with <code>uriEndpointOverride</code> option.	false	boolean
protocol (common)	The underlying protocol used to communicate with SQS.	https	String

Name	Description	Default	Type
proxyProtocol (common)	To define a proxy protocol when instantiating the SQS client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
queueOwnerAWSAccountId (common)	Specify the queue owner aws account id when you need to connect the queue with different account owner.		String
region (common)	The region in which SQS client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String
trustAllCertificates (common)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (common)	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String
useDefaultCredentialsProvider (common)	Set whether the SQS client should expect to load credentials on an AWS infra instance or to expect static credentials to be passed in.	false	boolean
attributeNames (consumer)	A list of attribute names to receive when consuming. Multiple names can be separated by comma.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
concurrentConsumers (consumer)	Allows you to use multiple threads to poll the sqs queue to increase throughput.	1	int
defaultVisibilityTimeout (consumer)	The default visibility timeout (in seconds).		Integer

Name	Description	Default	Type
deleteAfterRead (consumer)	Delete message from SQS after it has been read.	true	boolean
deleteIfFiltered (consumer)	Whether or not to send the DeleteMessage to the SQS queue if the exchange has property with key <code>Sqs2Constants#SQS_DELETE_FILTERED</code> (CamelAwsSqsDeleteFiltered) set to true.	true	boolean
extendMessageVisibility (consumer)	If enabled then a scheduled background task will keep extending the message visibility on SQS. This is needed if it takes a long time to process the message. If set to true <code>defaultVisibilityTimeout</code> must be set.	false	boolean
kmsDataKeyReusePeriodSeconds (consumer)	The length of time, in seconds, for which Amazon SQS can reuse a data key to encrypt or decrypt messages before calling AWS KMS again. An integer representing seconds, between 60 seconds (1 minute) and 86,400 seconds (24 hours). Default: 300 (5 minutes).		Integer
kmsMasterKeyId (consumer)	The ID of an AWS-managed customer master key (CMK) for Amazon SQS or a custom CMK.		String
messageAttributeNames (consumer)	A list of message attribute names to receive when consuming. Multiple names can be separated by comma.		String
serverSideEncryptionEnabled (consumer)	Define if Server Side Encryption is enabled or not on the queue.	false	boolean
visibilityTimeout (consumer)	The duration (in seconds) that the received messages are hidden from subsequent retrieve requests after being retrieved by a <code>ReceiveMessage</code> request to set in the <code>com.amazonaws.services.sqs.model.SetQueueAttributesRequest</code> . This only make sense if its different from <code>defaultVisibilityTimeout</code> . It changes the queue visibility timeout attribute permanently.		Integer
waitTimeSeconds (consumer)	Duration in seconds (0 to 20) that the <code>ReceiveMessage</code> action call will wait until a message is in the queue to include in the response.		Integer
batchSeparator (producer)	Set the separator when passing a String to send batch message operation.	,	String

Name	Description	Default	Type
delaySeconds (producer)	Delay sending messages for a number of seconds.		Integer
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
messageDeduplicationIdStrategy (producer)	Only for FIFO queues. Strategy for setting the messageDeduplicationId on the message. Can be one of the following options: useExchangedId, useContentBasedDeduplication. For the useContentBasedDeduplication option, no messageDeduplicationId will be set on the message. Enum values: ● useExchangedId ● useContentBasedDeduplication	useExchangedId	String
messageGroupIdStrategy (producer)	Only for FIFO queues. Strategy for setting the messageGroupId on the message. Can be one of the following options: useConstant, useExchangedId, usePropertyValue. For the usePropertyValue option, the value of property CamelAwsMessageGroupId will be used. Enum values: ● useConstant ● useExchangedId ● usePropertyValue		String

Name	Description	Default	Type
operation (producer)	The operation to do in case the user don't want to send only a message. Enum values: ● <code>sendBatchMessage</code> ● <code>deleteMessage</code> ● <code>listQueues</code> ● <code>purgeQueue</code> ● <code>deleteQueue</code>		<code>Sqs2Operations</code>
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	<code>true</code>	<code>boolean</code>
delayQueue (advanced)	Define if you want to apply <code>delaySeconds</code> option to the queue or on single messages.	<code>false</code>	<code>boolean</code>
queueUrl (advanced)	To define the <code>queueUrl</code> explicitly. All other parameters, which would influence the <code>queueUrl</code> , are ignored. This parameter is intended to be used, to connect to a mock implementation of SQS, for testing purposes.		<code>String</code>
proxyHost (proxy)	To define a proxy host when instantiating the SQS client.		<code>String</code>
proxyPort (proxy)	To define a proxy port when instantiating the SQS client.		<code>Integer</code>
maximumMessageSize (queue)	The <code>maximumMessageSize</code> (in bytes) an SQS message can contain for this queue.		<code>Integer</code>
messageRetentionPeriod (queue)	The <code>messageRetentionPeriod</code> (in seconds) a message will be retained by SQS for this queue.		<code>Integer</code>
policy (queue)	The policy for this queue. It can be loaded by default from classpath, but you can prefix with <code>classpath:</code> , <code>file:</code> , or <code>http:</code> to load the resource from different systems.		<code>String</code>

Name	Description	Default	Type
receiveMessageWaitTimeSeconds (queue)	If you do not specify WaitTimeSeconds in the request, the queue attribute ReceiveMessageWaitTimeSeconds is used to determine how long to wait.		Integer
redrivePolicy (queue)	Specify the policy that send message to DeadLetter queue. See detail at Amazon docs.		String
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

11.5. ENDPOINT OPTIONS

The AWS Simple Queue Service (SQS) endpoint is configured using URI syntax:

```
aws2-sqs:queueNameOrArn
```

with the following path and query parameters:

11.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
queueNameOrArn (common)	Required Queue name or ARN.		String

11.5.2. Query Parameters (61 parameters)

Name	Description	Default	Type
amazonAWSHost (common)	The hostname of the Amazon AWS cloud.	amazonaws.com	String
amazonSQSClient (common)	Autowired To use the AmazonSQS as client.		SqsClient
autoCreateQueue (common)	Setting the autocreation of the queue.	false	boolean

Name	Description	Default	Type
headerFilterStrategy (common)	To use a custom HeaderFilterStrategy to map headers to/from Camel.		HeaderFilterStrategy
overrideEndpoint (common)	Set the need for overriding the endpoint. This option needs to be used in combination with uriEndpointOverride option.	false	boolean
protocol (common)	The underlying protocol used to communicate with SQS.	https	String
proxyProtocol (common)	To define a proxy protocol when instantiating the SQS client. Enum values: • HTTP • HTTPS	HTTPS	Protocol
queueOwnerAWSAccountId (common)	Specify the queue owner aws account id when you need to connect the queue with different account owner.		String
region (common)	The region in which SQS client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String
trustAllCertificates (common)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
uriEndpointOverride (common)	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String
useDefaultCredentialsProvider (common)	Set whether the SQS client should expect to load credentials on an AWS infra instance or to expect static credentials to be passed in.	false	boolean
attributeNames (consumer)	A list of attribute names to receive when consuming. Multiple names can be separated by comma.		String

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
concurrentConsumers (consumer)	Allows you to use multiple threads to poll the sqs queue to increase throughput.	1	int
defaultVisibilityTimeout (consumer)	The default visibility timeout (in seconds).		Integer
deleteAfterRead (consumer)	Delete message from SQS after it has been read.	true	boolean
deleteIfFiltered (consumer)	Whether or not to send the DeleteMessage to the SQS queue if the exchange has property with key <code>Sqs2Constants#SQS_DELETE_FILTERED</code> (CamelAwsSqsDeleteFiltered) set to true.	true	boolean
extendMessageVisibility (consumer)	If enabled then a scheduled background task will keep extending the message visibility on SQS. This is needed if it takes a long time to process the message. If set to true defaultVisibilityTimeout must be set. See details at Amazon docs.	false	boolean
kmsDataKeyReusePeriodSeconds (consumer)	The length of time, in seconds, for which Amazon SQS can reuse a data key to encrypt or decrypt messages before calling AWS KMS again. An integer representing seconds, between 60 seconds (1 minute) and 86,400 seconds (24 hours). Default: 300 (5 minutes).		Integer
kmsMasterKeyId (consumer)	The ID of an AWS-managed customer master key (CMK) for Amazon SQS or a custom CMK.		String
maxMessagesPerPoll (consumer)	Gets the maximum number of messages as a limit to poll at each polling. Is default unlimited, but use 0 or negative number to disable it as unlimited.		int
messageAttributeNames (consumer)	A list of message attribute names to receive when consuming. Multiple names can be separated by comma.		String

Name	Description	Default	Type
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
serverSideEncryptionEnabled (consumer)	Define if Server Side Encryption is enabled or not on the queue.	false	boolean
visibilityTimeout (consumer)	The duration (in seconds) that the received messages are hidden from subsequent retrieve requests after being retrieved by a <code>ReceiveMessage</code> request to set in the <code>com.amazonaws.services.sqs.model.SetQueueAttributesRequest</code> . This only make sense if its different from <code>defaultVisibilityTimeout</code> . It changes the queue visibility timeout attribute permanently.		Integer
waitTimeSeconds (consumer)	Duration in seconds (0 to 20) that the <code>ReceiveMessage</code> action call will wait until a message is in the queue to include in the response.		Integer
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an <code>Exchange</code> have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
batchSeparator (producer)	Set the separator when passing a <code>String</code> to send batch message operation.	,	<code>String</code>

Name	Description	Default	Type
delaySeconds (producer)	Delay sending messages for a number of seconds.		Integer
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
messageDeduplicationIdStrategy (producer)	Only for FIFO queues. Strategy for setting the messageDeduplicationId on the message. Can be one of the following options: useExchangeId, useContentBasedDeduplication. For the useContentBasedDeduplication option, no messageDeduplicationId will be set on the message. Enum values: ● useExchangeId ● useContentBasedDeduplication	useExchangeId	String
messageGroupIdStrategy (producer)	Only for FIFO queues. Strategy for setting the messageGroupId on the message. Can be one of the following options: useConstant, useExchangeId, usePropertyValue. For the usePropertyValue option, the value of property CamelAwsMessageGroupId will be used. Enum values: ● useConstant ● useExchangeId ● usePropertyValue		String

Name	Description	Default	Type
operation (producer)	The operation to do in case the user don't want to send only a message. Enum values: ● <code>sendBatchMessage</code> ● <code>deleteMessage</code> ● <code>listQueues</code> ● <code>purgeQueue</code> ● <code>deleteQueue</code>		<code>Sqs2Operations</code>
delayQueue (advanced)	Define if you want to apply <code>delaySeconds</code> option to the queue or on single messages.	<code>false</code>	<code>boolean</code>
queueUrl (advanced)	To define the <code>queueUrl</code> explicitly. All other parameters, which would influence the <code>queueUrl</code> , are ignored. This parameter is intended to be used, to connect to a mock implementation of SQS, for testing purposes.		<code>String</code>
proxyHost (proxy)	To define a proxy host when instantiating the SQS client.		<code>String</code>
proxyPort (proxy)	To define a proxy port when instantiating the SQS client.		<code>Integer</code>
maximumMessageSize (queue)	The <code>maximumMessageSize</code> (in bytes) an SQS message can contain for this queue.		<code>Integer</code>
messageRetentionPeriod (queue)	The <code>messageRetentionPeriod</code> (in seconds) a message will be retained by SQS for this queue.		<code>Integer</code>
policy (queue)	The policy for this queue. It can be loaded by default from classpath, but you can prefix with <code>classpath:</code> , <code>file:</code> , or <code>http:</code> to load the resource from different systems.		<code>String</code>
receiveMessageWaitTimeSeconds (queue)	If you do not specify <code>WaitTimeSeconds</code> in the request, the queue attribute <code>ReceiveMessageWaitTimeSeconds</code> is used to determine how long to wait.		<code>Integer</code>
redrivePolicy (queue)	Specify the policy that send message to DeadLetter queue. See detail at Amazon docs.		<code>String</code>

Name	Description	Default	Type
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService

Name	Description	Default	Type
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

Required SQS component options

You have to provide the `amazonSQSClient` in the Registry or your `accessKey` and `secretKey` to access the [Amazon's SQS](#).

11.6. BATCH CONSUMER

This component implements the Batch Consumer.

This allows you for instance to know how many messages exists in this batch and for instance let the Aggregator aggregate this number of messages.

11.7. USAGE

11.7.1. Static credentials vs Default Credential Provider

You have the possibility of avoiding the usage of explicit static credentials, by specifying the **useDefaultCredentialsProvider** option and set it to true.

- Java system properties - **aws.accessKeyId** and **aws.secretKey**
- Environment variables - **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY**.
- Web Identity Token from AWS STS.
- The shared credentials and config files.
- Amazon ECS container credentials - loaded from the Amazon ECS if the environment variable **AWS_CONTAINER_CREDENTIALS_RELATIVE_URI** is set.
- Amazon EC2 Instance profile credentials.

For more information about this you can look at [AWS credentials documentation](#)

11.7.2. Message headers set by the SQS producer

Header	Type	Description
CamelAwsSqsMD5OfBody	String	The MD5 checksum of the Amazon SQS message.
CamelAwsSqsMessageId	String	The Amazon SQS message ID.
CamelAwsSqsDelaySeconds	Integer	The delay seconds that the Amazon SQS message can be seen by others.

11.7.3. Message headers set by the SQS consumer

Header	Type	Description
CamelAwsSqsMD5OfBody	String	The MD5 checksum of the Amazon SQS message.
CamelAwsSqsMessageId	String	The Amazon SQS message ID.
CamelAwsSqsReceiptHandle	String	The Amazon SQS message receipt handle.
CamelAwsSqsMessageAttributes	Map<String, String>	The Amazon SQS message attributes.

11.7.4. Advanced AmazonSQS configuration

If your Camel Application is running behind a firewall or if you need to have more control over the **SqsClient** instance configuration, you can create your own instance:

```
from("aws2-sqs://MyQueue?amazonSQSClient=#client&delay=5000&maxMessagesPerPoll=5")
.to("mock:result");
```

11.7.5. Creating or updating an SQS Queue

In the SQS Component, when an endpoint is started, a check is executed to obtain information about the existence of the queue or not. You're able to customize the creation through the **QueueAttributeName** mapping with the **SQSConfiguration** option.

```
from("aws2-sqs://MyQueue?amazonSQSClient=#client&delay=5000&maxMessagesPerPoll=5")
.to("mock:result");
```

In this example if the **MyQueue** queue is not already created on AWS (and the **autoCreateQueue** option is set to true), it will be created with default parameters from the SQS configuration. If it's already up on AWS, the SQS configuration options will be used to override the existent AWS configuration.

11.7.6. DelayQueue VS Delay for Single message

When the option **delayQueue** is set to true, the SQS Queue will be a **DelayQueue** with the **DelaySeconds** option as delay. For more information about **DelayQueue** you can read the [AWS SQS documentation](#). One important information to take into account is the following:

- For standard queues, the per-queue delay setting is not retroactive—changing the setting doesn't affect the delay of messages already in the queue.
- For FIFO queues, the per-queue delay setting is retroactive—changing the setting affects the delay of messages already in the queue.

as stated in the official documentation. If you want to specify a delay on single messages, you can ignore the **delayQueue** option, while you can set this option to true, if you need to add a fixed delay to all messages enqueued.

11.7.7. Server Side Encryption

There is a set of Server Side Encryption attributes for a queue. The related options are **serverSideEncryptionEnabled**, **keyMasterKeyId** and **kmsDataKeyReusePeriod**. The SSE is disabled by default. You need to explicitly set the option to true and set the related parameters as queue attributes.

11.8. JMS-STYLE SELECTORS

SQS does not allow selectors, but you can effectively achieve this by using the Camel Filter EIP and setting an appropriate **visibilityTimeout**. When SQS dispatches a message, it will wait up to the visibility timeout before it will try to dispatch the message to a different consumer unless a **DeleteMessage** is received. By default, Camel will always send the **DeleteMessage** at the end of the route, unless the route ended in failure. To achieve appropriate filtering and not send the **DeleteMessage** even on successful completion of the route, use a Filter:

■

```

from("aws2-sqs://MyQueue?
amazonSQSClient=#client&defaultVisibilityTimeout=5000&deleteIfFiltered=false&deleteAfterRead=false
")
.filter("${header.login} == true")
.setProperty(Sqs2Constants.SQS_DELETE_FILTERED, constant(true))
.to("mock:filter");

```

In the above code, if an exchange doesn't have an appropriate header, it will not make it through the filter AND also not be deleted from the SQS queue. After 5000 milliseconds, the message will become visible to other consumers.

Note we must set the property **Sqs2Constants.SQS_DELETE_FILTERED** to **true** to instruct Camel to send the **DeleteMessage**, if being filtered.

11.9. AVAILABLE PRODUCER OPERATIONS

- single message (default)
- `sendBatchMessage`
- `deleteMessage`
- `listQueues`

11.10. SEND MESSAGE

You can set a **SendMessageBatchRequest** or an **Iterable**

```

from("direct:start")
.setBody(constant("Camel rocks!"))
.to("aws2-sqs://camel-1?accessKey=RAW(XXX)&secretKey=RAW(XXX)&region=eu-west-1");

```

11.11. SEND BATCH MESSAGE

You can set a **SendMessageBatchRequest** or an **Iterable**

```

from("direct:start")
.setHeader(SqsConstants.SQS_OPERATION, constant("sendBatchMessage"))
.process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Collection c = new ArrayList();
        c.add("team1");
        c.add("team2");
        c.add("team3");
        c.add("team4");
        exchange.getIn().setBody(c);
    }
})
.to("aws2-sqs://camel-1?accessKey=RAW(XXX)&secretKey=RAW(XXX)&region=eu-west-1");

```

As result you'll get an exchange containing a **SendMessageBatchResponse** instance, that you can examine to check what messages were successful and what not. The id set on each message of the batch will be a Random UUID.

11.12. DELETE SINGLE MESSAGE

Use **deleteMessage** operation to delete a single message. You'll need to set a receipt handle header for the message you want to delete.

```
from("direct:start")
  .setHeader(SqsConstants.SQS_OPERATION, constant("deleteMessage"))
  .setHeader(SqsConstants.RECEIPT_HANDLE, constant("123456"))
  .to("aws2-sqs://camel-1?accessKey=RAW(XXX)&secretKey=RAW(XXX)&region=eu-west-1");
```

As result you'll get an exchange containing a **DeleteMessageResponse** instance, that you can use to check if the message was deleted or not.

11.13. LIST QUEUES

Use **listQueues** operation to list queues.

```
from("direct:start")
  .setHeader(SqsConstants.SQS_OPERATION, constant("listQueues"))
  .to("aws2-sqs://camel-1?accessKey=RAW(XXX)&secretKey=RAW(XXX)&region=eu-west-1");
```

As result you'll get an exchange containing a **ListQueuesResponse** instance, that you can examine to check the actual queues.

11.14. PURGE QUEUE

Use **purgeQueue** operation to purge queue.

```
from("direct:start")
  .setHeader(SqsConstants.SQS_OPERATION, constant("purgeQueue"))
  .to("aws2-sqs://camel-1?accessKey=RAW(XXX)&secretKey=RAW(XXX)&region=eu-west-1");
```

As result you'll get an exchange containing a **PurgeQueueResponse** instance.

11.15. QUEUE AUTOCREATION

With the option **autoCreateQueue** users are able to avoid the autocreation of an SQS Queue in case it doesn't exist. The default for this option is **true**. If set to false any operation on a not-existent queue in AWS won't be successful and an error will be returned.

11.16. SEND BATCH MESSAGE AND MESSAGE DEDUPLICATION STRATEGY

In case you're using a **SendBatchMessage** Operation, you can set two different kind of Message Deduplication Strategy: - useExchangeId - useContentBasedDeduplication

The first one will use a **ExchangeIdMessageDeduplicationIdStrategy**, that will use the Exchange ID as parameter. The other one will use a **NullMessageDeduplicationIdStrategy**, that will use the body as deduplication element.

In case of send batch message operation, you'll need to use the **useContentBasedDeduplication** and on the Queue you're pointing you'll need to enable the **content based deduplication** option.

11.17. DEPENDENCIES

Maven users will need to add the following dependency to their pom.xml.

pom.xml

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-aws2-sqs</artifactId>
  <version>${camel-version}</version>
</dependency>
```

where **{camel-version}** must be replaced by the actual version of Camel.

11.18. SPRING BOOT AUTO-CONFIGURATION

The component supports 44 options, which are listed below.

Name	Description	Default	Type
camel.component.aws2-sqs.access-key	Amazon AWS Access Key.		String
camel.component.aws2-sqs.amazon-aws-host	The hostname of the Amazon AWS cloud.	amazonaws.com	String
camel.component.aws2-sqs.amazon-sqs-client	To use the AmazonSQS as client. The option is a software.amazon.awssdk.services.sqs.SqsClient type.		SqsClient
camel.component.aws2-sqs.attribute-names	A list of attribute names to receive when consuming. Multiple names can be separated by comma.		String
camel.component.aws2-sqs.auto-create-queue	Setting the autocreation of the queue.	false	Boolean

Name	Description	Default	Type
<code>camel.component.aws2-sqs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.aws2-sqs.batch-separator</code>	Set the separator when passing a String to send batch message operation.	,	String
<code>camel.component.aws2-sqs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.aws2-sqs.concurrent-consumers</code>	Allows you to use multiple threads to poll the sqs queue to increase throughput.	1	Integer
<code>camel.component.aws2-sqs.configuration</code>	The AWS SQS default configuration. The option is a <code>org.apache.camel.component.aws2.sqs.Sqs2Configuration</code> type.		Sqs2Configuration
<code>camel.component.aws2-sqs.default-visibility-timeout</code>	The default visibility timeout (in seconds).		Integer
<code>camel.component.aws2-sqs.delay-queue</code>	Define if you want to apply <code>delaySeconds</code> option to the queue or on single messages.	false	Boolean
<code>camel.component.aws2-sqs.delay-seconds</code>	Delay sending messages for a number of seconds.		Integer
<code>camel.component.aws2-sqs.delete-after-read</code>	Delete message from SQS after it has been read.	true	Boolean

Name	Description	Default	Type
<code>camel.component.aws2-sqs.delete-if-filtered</code>	Whether or not to send the DeleteMessage to the SQS queue if the exchange has property with key <code>Sqs2Constants#SQS_DELETE_FILTERED</code> (CamelAwsSqsDeleteFiltered) set to true.	true	Boolean
<code>camel.component.aws2-sqs.enabled</code>	Whether to enable auto configuration of the aws2-sqs component. This is enabled by default.		Boolean
<code>camel.component.aws2-sqs.extend-message-visibility</code>	If enabled then a scheduled background task will keep extending the message visibility on SQS. This is needed if it takes a long time to process the message. If set to true <code>defaultVisibilityTimeout</code> must be set. See details at Amazon docs.	false	Boolean
<code>camel.component.aws2-sqs.kms-data-key-reuse-period-seconds</code>	The length of time, in seconds, for which Amazon SQS can reuse a data key to encrypt or decrypt messages before calling AWS KMS again. An integer representing seconds, between 60 seconds (1 minute) and 86,400 seconds (24 hours). Default: 300 (5 minutes).		Integer
<code>camel.component.aws2-sqs.kms-master-key-id</code>	The ID of an AWS-managed customer master key (CMK) for Amazon SQS or a custom CMK.		String
<code>camel.component.aws2-sqs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.aws2-sqs.maximum-message-size</code>	The <code>maximumMessageSize</code> (in bytes) an SQS message can contain for this queue.		Integer
<code>camel.component.aws2-sqs.message-attribute-names</code>	A list of message attribute names to receive when consuming. Multiple names can be separated by comma.		String

Name	Description	Default	Type
<code>camel.component.aws2-sqs.message-deduplication-id-strategy</code>	Only for FIFO queues. Strategy for setting the <code>messageDeduplicationId</code> on the message. Can be one of the following options: <code>useExchangeId</code> , <code>useContentBasedDeduplication</code> . For the <code>useContentBasedDeduplication</code> option, no <code>messageDeduplicationId</code> will be set on the message.	<code>useExchangeId</code>	String
<code>camel.component.aws2-sqs.message-group-id-strategy</code>	Only for FIFO queues. Strategy for setting the <code>messageGroupId</code> on the message. Can be one of the following options: <code>useConstant</code> , <code>useExchangeId</code> , <code>usePropertyValue</code> . For the <code>usePropertyValue</code> option, the value of property <code>CamelAwsMessageGroupId</code> will be used.		String
<code>camel.component.aws2-sqs.message-retention-period</code>	The <code>messageRetentionPeriod</code> (in seconds) a message will be retained by SQS for this queue.		Integer
<code>camel.component.aws2-sqs.operation</code>	The operation to do in case the user don't want to send only a message.		<code>Sqs2Operations</code>
<code>camel.component.aws2-sqs.override-endpoint</code>	Set the need for overriding the endpoint. This option needs to be used in combination with <code>uriEndpointOverride</code> option.	<code>false</code>	Boolean
<code>camel.component.aws2-sqs.policy</code>	The policy for this queue. It can be loaded by default from classpath, but you can prefix with <code>classpath:</code> , <code>file:</code> , or <code>http:</code> to load the resource from different systems.		String
<code>camel.component.aws2-sqs.protocol</code>	The underlying protocol used to communicate with SQS.	<code>https</code>	String
<code>camel.component.aws2-sqs.proxy-host</code>	To define a proxy host when instantiating the SQS client.		String
<code>camel.component.aws2-sqs.proxy-port</code>	To define a proxy port when instantiating the SQS client.		Integer

Name	Description	Default	Type
<code>camel.component.aws2-sqs.proxy-protocol</code>	To define a proxy protocol when instantiating the SQS client.		Protocol
<code>camel.component.aws2-sqs.queue-owner-a-w-s-account-id</code>	Specify the queue owner aws account id when you need to connect the queue with different account owner.		String
<code>camel.component.aws2-sqs.queue-url</code>	To define the queueUrl explicitly. All other parameters, which would influence the queueUrl, are ignored. This parameter is intended to be used, to connect to a mock implementation of SQS, for testing purposes.		String
<code>camel.component.aws2-sqs.receive-message-wait-time-seconds</code>	If you do not specify WaitTimeSeconds in the request, the queue attribute ReceiveMessageWaitTimeSeconds is used to determine how long to wait.		Integer
<code>camel.component.aws2-sqs.redrive-policy</code>	Specify the policy that send message to DeadLetter queue. See detail at Amazon docs.		String
<code>camel.component.aws2-sqs.region</code>	The region in which SQS client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1) You'll need to use the name Region.EU_WEST_1.id().		String
<code>camel.component.aws2-sqs.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws2-sqs.server-side-encryption-enabled</code>	Define if Server Side Encryption is enabled or not on the queue.	false	Boolean
<code>camel.component.aws2-sqs.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean
<code>camel.component.aws2-sqs.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with overrideEndpoint option.		String

Name	Description	Default	Type
<code>camel.component.aws2-sqs.use-default-credentials-provider</code>	Set whether the SQS client should expect to load credentials on an AWS infra instance or to expect static credentials to be passed in.	false	Boolean
<code>camel.component.aws2-sqs.visibility-timeout</code>	The duration (in seconds) that the received messages are hidden from subsequent retrieve requests after being retrieved by a <code>ReceiveMessage</code> request to set in the <code>com.amazonaws.services.sqs.model.SetQueueAttributesRequest</code> . This only make sense if its different from <code>defaultVisibilityTimeout</code> . It changes the queue visibility timeout attribute permanently.		Integer
<code>camel.component.aws2-sqs.wait-time-seconds</code>	Duration in seconds (0 to 20) that the <code>ReceiveMessage</code> action call will wait until a message is in the queue to include in the response.		Integer

CHAPTER 12. AWS SECRETS MANAGER

Only producer is supported

The AWS Secrets Manager component supports [AWS Secrets Manager](#) service.

Prerequisites

You must have a valid Amazon Web Services developer account, and be signed up to use Amazon Secrets Manager. More information is available at [AWS Secrets Manager](#).

12.1. DEPENDENCIES

When using **camel-aws-secrets-manager** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-aws-secrets-manager-starter</artifactId>
</dependency>
```

12.2. URI FORMAT

```
aws-secrets-manager://label[?options]
```

You can append query options to the URI in the following format:

```
`?options=value&option2=value&...`
```

12.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

12.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

12.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

12.4. COMPONENT OPTIONS

The AWS Secrets Manager component supports 23 options, which are listed below.

Name	Description	Default	Type
binaryPayload (producer)	Set if the secret is binary or not.	false	boolean
configuration (producer)	Component configuration.		SecretsManagerConfiguration
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
operation (producer)	Required The operation to perform. Enum values: • listSecrets • createSecret • getSecret • describeSecret • deleteSecret • rotateSecret • updateSecret • restoreSecret • replicateSecretToRegions		SecretsManagerOperations
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with the uriEndpointOverride option.	false	boolean
pojoRequest (producer)	If we want to use a POJO request as body or not.	false	boolean
profileCredentialsName (producer)	If using a profile credentials provider, this parameter will set the profile name.		String
region (producer)	The region in which a Secrets Manager client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example, ap-east-1) You'll need to use the name Region.EU_WEST_1.id(). Enum values: • ap-south-2 • ap-south-1 • eu-south-1 • eu-south-2 • us-gov-east-1 • me-central-1 • il-central-1 • ca-central-1		String

Name	● eu-central-1 ● us-iso-west-1 Description	Default	Type
	● eu-central-2 ● eu-isoe-west-1 ● us-west-1 ● us-west-2 ● af-south-1 ● eu-north-1 ● eu-west-3 ● eu-west-2 ● eu-west-1 ● ap-northeast-3 ● ap-northeast-2 ● ap-northeast-1 ● me-south-1 ● sa-east-1 ● ap-east-1 ● cn-north-1 ● ca-west-1 ● us-gov-west-1 ● ap-southeast-1 ● ap-southeast-2 ● us-iso-east-1 ● ap-southeast-3 ● ap-southeast-4 ● us-east-1 ● us-east-2 ● cn-northwest-1 ● us-isob-east-1 ● aws-global ● aws-cn-global ● aws-us-gov-global ● aws-iso-global		

Name	aws-iso-b-global Description	Default	Type
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.	<code>true</code>	String
useProfileCredentialsProvider (producer)	Set whether the Secrets Manager client should expect to load credentials through a profile credentials provider.	<code>false</code>	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	<code>true</code>	boolean
secretsManagerClient (advanced)	Autowired To use an existing configured AWS Secrets Manager client.		SecretsManagerClient
healthCheckConsumerEnabled (health)	Used for enabling or disabling all consumer based health checks from this component.	<code>true</code>	boolean
healthCheckProducerEnabled (health)	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting <code>camel.health.producersEnabled=true</code> .	<code>true</code>	boolean
proxyHost (proxy)	To define a proxy host when instantiating the Secrets Manager client.		String
proxyPort (proxy)	To define a proxy port when instantiating the Secrets Manager client.		Integer
proxyProtocol (proxy)	To define a proxy protocol when instantiating the Secrets Manager client. Enum values: - HTTP - HTTPS	HTTPS	Protocol
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String

Name	Description	Default	Type
sessionToken (security)	Amazon AWS Session Token used when the user needs to assume an IAM role.		String
trustAllCertificates (security)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
useDefaultCredentialsProvider (security)	Set whether the Translate client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
useSessionCredentials (security)	Set whether the Secrets Manager client should expect to use Session Credentials. This is useful in a situation in which the user needs to assume an IAM role for doing operations in Secrets Manager.	false	boolean

12.5. ENDPOINT OPTIONS

The AWS Secrets Manager endpoint is configured using URI syntax:

```
aws-secrets-manager:label
```

With the following *path* and *query* parameters:

12.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
label (producer)	Required Logical name.		String

12.5.2. Query Parameters (19 parameters)

Name	Description	Default	Type
binaryPayload (producer)	Set if the secret is binary or not.	false	boolean

Name	Description	Default	Type
operation (producer)	Required The operation to perform. Enum values: • listSecrets • createSecret • getSecret • describeSecret • deleteSecret • rotateSecret • updateSecret • restoreSecret • replicateSecretToRegions		SecretsManagerOperations
overrideEndpoint (producer)	Set the need for overriding the endpoint. This option needs to be used in combination with the uriEndpointOverride option.	false	boolean
pojoRequest (producer)	If we want to use a POJO request as body or not.	false	boolean
profileCredentialsName (producer)	If using a profile credentials provider, this parameter will set the profile name.		String
region (producer)	The region in which a Secrets Manager client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example, ap-east-1) You'll need to use the name Region.EU_WEST_1.id(). Enum values:		String

Name	Description	Default	Type
	● ap-south-2 ● ap-south-1 ● eu-south-1 ● eu-south-2 ● us-gov-east-1 ● me-central-1 ● il-central-1 ● ca-central-1 ● eu-central-1 ● us-iso-west-1 ● eu-central-2 ● eu-isoe-west-1 ● us-west-1 ● us-west-2 ● af-south-1 ● eu-north-1 ● eu-west-3 ● eu-west-2 ● eu-west-1 ● ap-northeast-3 ● ap-northeast-2 ● ap-northeast-1 ● me-south-1 ● sa-east-1 ● ap-east-1 ● cn-north-1 ● ca-west-1 ● us-gov-west-1 ● ap-southeast-1 ● ap-southeast-2 ● us-iso-east-1 ● ap-southeast-3		

Name	• ap-southeast-1 Description	Default	Type
	• us-east-1 • us-east-2 • cn-northwest-1 • us-isob-east-1 • aws-global • aws-cn-global • aws-us-gov-global • aws-iso-global • aws-iso-b-global		
uriEndpointOverride (producer)	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
useProfileCredentialsProvider (producer)	Set whether the Secrets Manager client should expect to load credentials through a profile credentials provider.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
secretsManagerClient (advanced)	Autowired To use an existing configured AWS Secrets Manager client.		SecretsManagerClient
proxyHost (proxy)	To define a proxy host when instantiating the Secrets Manager client.		String
proxyPort (proxy)	To define a proxy port when instantiating the Secrets Manager client.		Integer
proxyProtocol (proxy)	To define a proxy protocol when instantiating the Secrets Manager client. Enum values: ● HTTP ● HTTPS	HTTPS	Protocol

Name	Description	Default	Type
accessKey (security)	Amazon AWS Access Key.		String
secretKey (security)	Amazon AWS Secret Key.		String
sessionToken (security)	Amazon AWS Session Token used when the user needs to assume an IAM role.		String
trustAllCertificates (security)	If we want to trust all certificates in case of overriding the endpoint.	false	boolean
useDefaultCredentialsProvider (security)	Set whether the Translate client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	boolean
useSessionCredentials (security)	Set whether the Secrets Manager client should expect to use Session Credentials. This is useful in a situation in which the user needs to assume an IAM role for doing operations in Secrets Manager.	false	boolean

12.5.3. Static credentials, Default Credential Provider and Profile Credentials Provider

You can avoid the usage of explicit static credentials by specifying the **useDefaultCredentialsProvider** option and setting it to **true**.

The order of evaluation for Default Credentials Provider is as per the following:

- Java system properties - **aws.accessKeyId** and **aws.secretKey**.
- Environment variables - **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY**.
- Web Identity Token from AWS STS.
- The shared credentials and config files.

- Amazon ECS container credentials - loaded from the Amazon ECS if the environment variable **AWS_CONTAINER_CREDENTIALS_RELATIVE_URI** is set.
- Amazon EC2 Instance profile credentials.

You can use the Profile Credentials Provider, by specifying the **useProfileCredentialsProvider** option to true and **profileCredentialsName** to the profile name.

Only one of static, default and profile credentials could be used at the same time. For more information about this, see [AWS credentials documentation](#).

12.5.4. Using AWS Secrets Manager Property Function

To use this function, you need to provide credentials to AWS Secrets Manager Service as environment variables:

```
export CAMEL_VAULT_AWS_ACCESS_KEY=accessKey
export CAMEL_VAULT_AWS_SECRET_KEY=secretKey
export CAMEL_VAULT_AWS_REGION=region
```

You can also configure the credentials in the **application.properties** file such as:

```
camel.vault.aws.accessKey = accessKey
camel.vault.aws.secretKey = secretKey
camel.vault.aws.region = region
```

If you want to use the [AWS default credentials provider](#) instead, you need to provide the following env variables:

```
export CAMEL_VAULT_AWS_USE_DEFAULT_CREDENTIALS_PROVIDER=true
export CAMEL_VAULT_AWS_REGION=region
```

You can also configure the credentials in the **application.properties** file such as:

```
camel.vault.aws.defaultCredentialsProvider = true
camel.vault.aws.region = region
```

It is also possible to specify a particular profile name for accessing AWS Secrets Manager as follows:

```
export CAMEL_VAULT_AWS_USE_PROFILE_CREDENTIALS_PROVIDER=true
export CAMEL_VAULT_AWS_PROFILE_NAME=test-account
export CAMEL_VAULT_AWS_REGION=region
```

You can also configure the credentials in the **application.properties** file such as:

```
camel.vault.aws.profileCredentialsProvider = true
camel.vault.aws.profileName = test-account
camel.vault.aws.region = region
```




NOTE

camel.vault.aws configuration only applies to the AWS Secrets Manager properties function (for example, when resolving properties). When using the **operation** option to create, get, list secrets etc., provide the usual options for connecting to AWS Services.

At this point, you can reference a property in the following way:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{aws:route}}"/>
  </route>
</camelContext>
```

Where the route will be the name of the secret stored in the AWS Secrets Manager Service.

You could specify a default value in case the secret is not present on AWS Secret Manager:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{aws:route:default}}"/>
  </route>
</camelContext>
```

In this case, if the secret doesn't exist, the property will fall back to "default" as value.

Also, you are able to get a particular field of the secret, if you have, for example, a secret named database of this form:

```
{
  "username": "admin",
  "password": "password123",
  "engine": "postgres",
  "host": "127.0.0.1",
  "port": "3128",
  "dbname": "db"
}
```

You can get a single secret value in your route as follows:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{aws:database/username}}"/>
  </route>
</camelContext>
```

Or re-use the property as part of an endpoint.

You can specify a default value in case the particular field of secret is not present on AWS Secret Manager:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{aws:database/username:admin}}"/>
  </route>
</camelContext>
```

In this case, if the secret does not exist or the secret exists, but the username field is not part of the secret, the property will fall back to "admin" as its value.

There is also the syntax to get a particular version of the secret for both the approach, with field/default value specified or only with secret:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{aws:route@bf9b4f4b-8e63-43fd-a73c-3e2d3748b451}}"/>
  </route>
</camelContext>
```

This approach will return the RAW route secret with the version 'bf9b4f4b-8e63-43fd-a73c-3e2d3748b451'.

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{aws:route:default@bf9b4f4b-8e63-43fd-a73c-3e2d3748b451}}"/>
  </route>
</camelContext>
```

This approach will return the route secret value with version 'bf9b4f4b-8e63-43fd-a73c-3e2d3748b451' or default value in case the secret doesn't exist or the version doesn't exist.

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{aws:database/username:admin@bf9b4f4b-8e63-43fd-a73c-3e2d3748b451}}"/>
  </route>
</camelContext>
```

This approach will return the username field of the database secret with version 'bf9b4f4b-8e63-43fd-a73c-3e2d3748b451' or admin in case the secret doesn't exist or the version doesn't exist.

The only requirement is adding the camel-aws-secrets-manager jar to your Camel application.

12.5.5. Automatic Camel context reloading on Secret Refresh

Being able to reload Camel context on a Secret Refresh could be done by specifying the usual credentials (the same used for AWS Secret Manager Property Function).

With Environment variables:

```
export CAMEL_VAULT_AWS_USE_DEFAULT_CREDENTIALS_PROVIDER=accessKey
export CAMEL_VAULT_AWS_REGION=region
```

or as plain Camel main properties:

```
camel.vault.aws.useDefaultCredentialProvider = true
camel.vault.aws.region = region
```

Or by specifying `accessKey`/`SecretKey` and `region`, instead of using the default credentials provider chain.

To enable the automatic refresh, set the additional properties as follows:

```
camel.vault.aws.refreshEnabled=true
camel.vault.aws.refreshPeriod=60000
camel.vault.aws.secrets=Secret
camel.main.context-reload-enabled = true
```

where **`camel.vault.aws.refreshEnabled`** will enable the automatic context reload, **`camel.vault.aws.refreshPeriod`** is the interval of time between two different checks for update events and **`camel.vault.aws.secrets`** is a regex representing the secrets we want to track for updates.

Note that **`camel.vault.aws.secrets`** is not mandatory. If it is not specified, the task responsible for checking updates events will take it into accounts or the properties with an **`aws:`** prefix.

12.5.6. Automatic Camel context reloading on Secret Refresh with Eventbridge and AWS SQS Services

Another option is to use AWS EventBridge in conjunction with the AWS SQS service.

On the AWS side, the following resources need to be created:

- an AWS CloudTrail trail
- an AWS SQS Queue
- an Eventbridge rule of the following kind

```
{
  "source": ["aws.secretsmanager"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventSource": ["secretsmanager.amazonaws.com"]
  }
}
```

This rule will make the event related to AWS Secrets Manager filtered

- You need to set the a Rule target to the AWS SQS Queue for Eventbridge rule
- You need to give permission to the Eventbridge rule, to write on the above SQS Queue. For doing this you'll need to define a json file like this:

```
{
```

```
"Policy": "{ \"Version\": \"2012-10-17\", \"Id\": \"<queue_arn>/SQSDefaultPolicy\", \"Statement\":
[ { \"Sid\": \"EventsToMyQueue\", \"Effect\": \"Allow\", \"Principal\": { \"Service\":
\"events.amazonaws.com\" }, \"Action\": \"sqs:SendMessage\", \"Resource\": \"<queue_arn>\",
\"Condition\": { \"ArnEquals\": { \"aws:SourceArn\": \"<eventbridge_rule_arn>\" } } } ] }"
```

Change the values for **queue_arn** and **eventbridge_rule_arn** and save the file with **policy.json** name and run the following command with AWS CLI:

```
aws sqs set-queue-attributes --queue-url <queue_url> --attributes file://policy.json
```

where **queue_url** is the AWS SQS Queue URL of the just created Queue.

Now you should be able to set up the configuration on the Camel side. To enable the SQS notification add the following properties:

```
camel.vault.aws.refreshEnabled=true
camel.vault.aws.refreshPeriod=60000
camel.vault.aws.secrets=Secret
camel.main.context-reload-enabled = true
camel.vault.aws.useSqsNotification=true
camel.vault.aws.sqsQueueUrl=<queue_url>
```

where **queue_url** is the AWS SQS Queue URL of the just created Queue.

Whenever an event of **PutSecretValue** for the Secret named **Secret** happens, a message will be enqueued in the AWS SQS Queue and consumed on the Camel side and a context reload will be triggered.

12.6. MESSAGE HEADERS

The AWS Secrets Manager component supports 8 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelAwsSecretsManagerOperation (producer) Constant: OPERATION	The operation we want to perform.		String
CamelAwsSecretsManagerMaxResults (producer) Constant: MAX_RESULTS	The number of results to include in the response.		Integer
CamelAwsSecretsManagerSecretName (producer) Constant: SECRET_NAME	The name of the secret.		String

Name	Description	Default	Type
CamelAwsSecretsManagerSecretDescription (producer) Constant: SECRET_DESCRIPTION	The description of the secret.		String
CamelAwsSecretsManagerSecretId (producer) Constant: SECRET_ID	The ARN or name of the secret.		String
CamelAwsSecretsManagerLambdaRotationFunctionArn (producer) Constant: LAMBDA_ROTATION_FUNCTION_ARN	The ARN of the Lambda rotation function that can rotate the secret.		String
CamelAwsSecretsManagerSecretVersionId (producer) Constant: SECRET_VERSION_ID	The unique identifier of the version of the secret.		String
CamelAwsSecretsManagerSecretReplicationRegions (producer) Constant: SECRET_REPLICATION_REGIONS	A comma separated list of Regions in which to replicate the secret.		String

12.6.1. Secrets Manager Producer operations

Camel-AWS-Secrets-manager component provides the following operation on the producer side:

- listSecrets
- createSecret
- deleteSecret
- describeSecret
- rotateSecret
- getSecret
- updateSecret

- `replicateSecretToRegions`

12.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 24 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.aws-secrets-manager.access-key</code>	Amazon AWS Access Key.		String
<code>camel.component.aws-secrets-manager.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.aws-secrets-manager.binary-payload</code>	Set if the secret is binary or not.	false	Boolean
<code>camel.component.aws-secrets-manager.configuration</code>	Component configuration. The option is a <code>org.apache.camel.component.aws.secretsmanager.SecretsManagerConfiguration</code> type.		SecretsManagerConfiguration
<code>camel.component.aws-secrets-manager.enabled</code>	Whether to enable auto configuration of the <code>aws-secrets-manager</code> component. This is enabled by default.		Boolean
<code>camel.component.aws-secrets-manager.health-check-consumer-enabled</code>	Used for enabling or disabling all consumer based health checks from this component.	true	Boolean
<code>camel.component.aws-secrets-manager.health-check-producer-enabled</code>	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting <code>camel.health.producersEnabled=true</code> .	true	Boolean

Name	Description	Default	Type
<code>camel.component.aws-secrets-manager.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.aws-secrets-manager.operation</code>	The operation to perform.		SecretsManagerOperations
<code>camel.component.aws-secrets-manager.override-endpoint</code>	Set the need for overriding the endpoint. This option needs to be used in combination with the <code>uriEndpointOverride</code> option.	false	Boolean
<code>camel.component.aws-secrets-manager.pojo-request</code>	If we want to use a POJO request as body or not.	false	Boolean
<code>camel.component.aws-secrets-manager.profile-credentials-name</code>	If using a profile credentials provider, this parameter will set the profile name.		String
<code>camel.component.aws-secrets-manager.proxy-host</code>	To define a proxy host when instantiating the Secrets Manager client.		String
<code>camel.component.aws-secrets-manager.proxy-port</code>	To define a proxy port when instantiating the Secrets Manager client.		Integer
<code>camel.component.aws-secrets-manager.proxy-protocol</code>	To define a proxy protocol when instantiating the Secrets Manager client.		Protocol

Name	Description	Default	Type
<code>camel.component.aws-secrets-manager.region</code>	The region in which a Secrets Manager client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example, ap-east-1) You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
<code>camel.component.aws-secrets-manager.secret-key</code>	Amazon AWS Secret Key.		String
<code>camel.component.aws-secrets-manager.secrets-manager-client</code>	To use an existing configured AWS Secrets Manager client. The option is a <code>software.amazon.awssdk.services.secretsmanager.SecretsManagerClient</code> type.		SecretsManagerClient
<code>camel.component.aws-secrets-manager.session-token</code>	Amazon AWS Session Token used when the user needs to assume an IAM role.		String
<code>camel.component.aws-secrets-manager.trust-all-certificates</code>	If we want to trust all certificates in case of overriding the endpoint.	false	Boolean
<code>camel.component.aws-secrets-manager.uri-endpoint-override</code>	Set the overriding uri endpoint. This option needs to be used in combination with <code>overrideEndpoint</code> option.		String
<code>camel.component.aws-secrets-manager.use-default-credentials-provider</code>	Set whether the Translate client should expect to load credentials through a default credentials provider or to expect static credentials to be passed in.	false	Boolean
<code>camel.component.aws-secrets-manager.use-profile-credentials-provider</code>	Set whether the Secrets Manager client should expect to load credentials through a profile credentials provider.	false	Boolean

Name	Description	Default	Type
<code>camel.component.aws-secrets-manager.use-session-credentials</code>	Set whether the Secrets Manager client should expect to use Session Credentials. This is useful in a situation in which the user needs to assume an IAM role for doing operations in Secrets Manager.	false	Boolean

CHAPTER 13. AZURE KEY VAULT

Only producer is supported

The azure-key-vault component that integrates [Azure Key Vault](#).

Prerequisites

You must have a valid Windows Azure Key Vault account. More information is available at [Azure Documentation Portal](#).

13.1. DEPENDENCIES

When using **camel-azure-key-vault** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-azure-key-vault-starter</artifactId>
</dependency>
```

13.2. URI FORMAT

```
azure-key-vault://vaultName[?options]
```

13.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

13.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

13.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

13.4. COMPONENT OPTIONS

The Azure Key Vault component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

13.5. ENDPOINT OPTIONS

The Azure Key Vault endpoint is configured using URI syntax:

```
azure-key-vault:vaultName
```

With the following *path* and *query* parameters:

13.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
vaultName (producer)	Vault Name to be used.		String

13.5.2. Query Parameters (7 parameters)

Name	Description	Default	Type
credentialType (common)	Determines the credential strategy to adopt. Enum values: • CLIENT_SECRET • AZURE_IDENTITY	CLIENT_SECRET	CredentialType
operation (producer)	Operation to be performed. Enum values: • createSecret • getSecret • deleteSecret • purgeDeletedSecret		KeyVaultOperation
secretClient (producer)	Autowired Instance of Secret client.		SecretClient
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
clientId (security)	Client Id to be used.		String

Name	Description	Default	Type
clientSecret (security)	Client Secret to be used.		String
tenantId (security)	Tenant Id to be used.		String

13.6. USAGE

13.6.1. Using Azure Key Vault Property Function

To use this function, you'll need to provide credentials to Azure Key Vault Service as environment variables:

```
export $CAMEL_VAULT_AZURE_TENANT_ID=tenantId
export $CAMEL_VAULT_AZURE_CLIENT_ID=clientId
export $CAMEL_VAULT_AZURE_CLIENT_SECRET=clientSecret
export $CAMEL_VAULT_AZURE_VAULT_NAME=vaultName
```

You can also configure the credentials in the **application.properties** file such as:

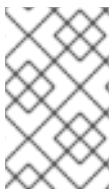
```
camel.vault.azure.tenantId = accessKey
camel.vault.azure.clientId = clientId
camel.vault.azure.clientSecret = clientSecret
camel.vault.azure.vaultName = vaultName
```

Or you can enable the usage of Azure Identity in the following way:

```
export $CAMEL_VAULT_AZURE_IDENTITY_ENABLED=true
export $CAMEL_VAULT_AZURE_VAULT_NAME=vaultName
```

You can also enable the usage of Azure Identity in the **application.properties** file such as:

```
camel.vault.azure.azureIdentityEnabled = true
camel.vault.azure.vaultName = vaultName
```



NOTE

camel.vault.azure configuration only applies to the Azure Key Vault properties function (E.g when resolving properties). When using the **operation** option to create, get, list secrets etc., you should provide the usual options for connecting to Azure Services.

At this point, you'll be able to reference a property in the following way:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
```

```
<to uri="{{azure:route}}"/>
</route>
</camelContext>
```

Where route will be the name of the secret stored in the Azure Key Vault Service.

You could specify a default value in case the secret is not present on Azure Key Vault Service:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{azure:route:default}}"/>
  </route>
</camelContext>
```

In this case, if the secret doesn't exist, the property will fall back to "default" as value.

Also, you are able to get a particular field of the secret, if you have, for example, a secret named database of this form:

```
{
  "username": "admin",
  "password": "password123",
  "engine": "postgres",
  "host": "127.0.0.1",
  "port": "3128",
  "dbname": "db"
}
```

You're able to do get single secret value in your route, like for example:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{azure:database/username}}"/>
  </route>
</camelContext>
```

Or re-use the property as part of an endpoint.

You could specify a default value in case the particular field of secret is not present on Azure Key Vault:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{azure:database/username:admin}}"/>
  </route>
</camelContext>
```

In this case, if the secret doesn't exist or the secret exists, but the username field is not part of the secret, the property will fall back to "admin" as value.

There is also the syntax to get a particular version of the secret for both the approach, with field/default value specified or only with secret:

■

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{azure:route@bf9b4f4b-8e63-43fd-a73c-3e2d3748b451}}"/>
  </route>
</camelContext>
```

This approach will return the RAW route secret with the version 'bf9b4f4b-8e63-43fd-a73c-3e2d3748b451'.

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{azure:route:default@bf9b4f4b-8e63-43fd-a73c-3e2d3748b451}}"/>
  </route>
</camelContext>
```

This approach will return the route secret value with version 'bf9b4f4b-8e63-43fd-a73c-3e2d3748b451' or default value in case the secret doesn't exist or the version doesn't exist.

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{azure:database/username:admin@bf9b4f4b-8e63-43fd-a73c-3e2d3748b451}}"/>
  </route>
</camelContext>
```

This approach will return the username field of the database secret with version 'bf9b4f4b-8e63-43fd-a73c-3e2d3748b451' or admin in case the secret doesn't exist or the version doesn't exist.

For the moment we are not considering the rotation function if any are applied, but it is in the work to be done.

The only requirement is adding the camel-azure-key-vault jar to your Camel application.

13.6.2. Automatic Camel context reloading on Secret Refresh

Being able to reload Camel context on a Secret Refresh could be done by specifying the usual credentials (the same used for Azure Key Vault Property Function).

With Environment variables:

```
export $CAMEL_VAULT_AZURE_TENANT_ID=tenantId
export $CAMEL_VAULT_AZURE_CLIENT_ID=clientId
export $CAMEL_VAULT_AZURE_CLIENT_SECRET=clientSecret
export $CAMEL_VAULT_AZURE_VAULT_NAME=vaultName
```

or as plain Camel main properties:

```
camel.vault.azure.tenantId = accessKey
camel.vault.azure.clientId = clientId
camel.vault.azure.clientSecret = clientSecret
camel.vault.azure.vaultName = vaultName
```

If you want to use Azure Identity with environment variables, you can do in the following way:

```
export $CAMEL_VAULT_AZURE_IDENTITY_ENABLED=true
export $CAMEL_VAULT_AZURE_VAULT_NAME=vaultName
```

You can also enable the usage of Azure Identity in the **application.properties** file such as:

```
camel.vault.azure.azureIdentityEnabled = true
camel.vault.azure.vaultName = vaultName
```

To enable the automatic refresh, you'll need additional properties to set:

```
camel.vault.azure.refreshEnabled=true
camel.vault.azure.refreshPeriod=60000
camel.vault.azure.secrets=Secret
camel.vault.azure.eventhubConnectionString=eventhub_conn_string
camel.vault.azure.blobAccountName=blob_account_name
camel.vault.azure.blobContainerName=blob_container_name
camel.vault.azure.blobAccessKey=blob_access_key
camel.main.context-reload-enabled = true
```

where **camel.vault.azure.refreshEnabled** will enable the automatic context reload, **camel.vault.azure.refreshPeriod** is the interval of time between two different checks for update events and **camel.vault.azure.secrets** is a regex representing the secrets we want to track for updates.

where **camel.vault.azure.eventhubConnectionString** is the eventhub connection string to get notification from, **camel.vault.azure.blobAccountName**, **camel.vault.azure.blobContainerName** and **camel.vault.azure.blobAccessKey** are the Azure Storage Blob parameters for the checkpoint store needed by Azure Eventhub.

Note that **camel.vault.azure.secrets** is not mandatory: if not specified the task responsible for checking updates events will take into accounts or the properties with an **azure:** prefix.

The only requirement is adding the camel-azure-key-vault jar to your Camel application.

13.6.3. Automatic Camel context reloading on Secret Refresh - Required Infrastructure's creation

First of all we need to create an application

```
az ad app create --display-name test-app-key-vault
```

Then we need to obtain credentials

```
az ad app credential reset --id <appld> --append --display-name 'Description: Key Vault app client' --end-date '2024-12-31'
```

This will return a result like this

```
{
  "appId": "appld",
  "password": "pwd",
```



```
"tenant": "tenantId"
}
```

You should take note of the password and use it as clientSecret parameter, together with the clientId and tenantId.

Now create the key vault

```
az keyvault create --name <vaultName> --resource-group <resourceGroup>
```

Create a service principal associated with the application Id

```
az ad sp create --id <appld>
```

At this point we need to add a role to the application with role assignment

```
az role assignment create --assignee <appld> --role "Key Vault Administrator" --scope
/subscriptions/<subscriptionId>/resourceGroups/<resourceGroup>/providers/Microsoft.KeyVault/vaults/<
vaultName>
```

Last step is to create policy on what can be or cannot be done with the application. In this case we just want to read the secret value. So This should be enough.

```
az keyvault set-policy --name <vaultName> --spn <appld> --secret-permissions get
```

You can create a secret through Azure CLI with the following command:

```
az keyvault secret set --name <secret_name> --vault-name <vaultName> -f <json-secret>
```

Now we need to setup the Eventhub/EventGrid notification for being informed about secrets updates.

First of all we'll need a Blob account and Blob container, to track Eventhub consuming activities.

```
az storage account create --name <blobAccountName> --resource-group <resourceGroup>
```

Then create a container

```
az storage container create --account-name <blobAccountName> --name <blobContainerName>
```

Then recover the access key for this purpose

```
az storage account keys list -g <resourceGroup> -n <blobAccountName>
```

Take note of the blob Account name, blob Container name and Blob Access Key to be used for setting up the vault.

Let's now create the Eventhub side

Create the namespace first

```
az eventhubs namespace create --resource-group <resourceGroup> --name <eventhub-namespace>
--location westus --sku Standard --enable-auto-inflate --maximum-throughput-units 20
```

Now create the resource

```
az eventhubs eventhub create --resource-group <resourceGroup> --namespace-name <eventhub-namespace> --name <eventhub-name> --cleanup-policy Delete --partition-count 15
```

In the Azure portal create a shared policy for the just created eventhub resource with "MANAGE" permissions and copy the connection string.

You now have all the required parameters to set up the vault.

13.7. MESSAGE HEADERS

The Azure Key Vault component supports 2 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelAzureKeyVaultProducerOperation (producer) Constant: OPERATION	Overrides the desired operation to be used in the producer.		KeyVaultOperationDefinition
CamelAzureKeyVaultSecretName (producer) Constant: SECRET_NAME	The secret name to be used in Key Vault.		String

13.7.1. Azure Key Vault Producer operations

Azure Key Vault component provides the following operation on the producer side:

- createSecret
- getSecret
- deleteSecret
- purgeDeletedSecret

13.8. EXAMPLES

13.8.1. Producer Examples

- createSecret: this operation will create a secret in Azure Key Vault

```
from("direct:createSecret")
  .setHeader(KeyVaultConstants.SECRET_NAME, "Test")
  .setBody(constant("Test"))
  .to("azure-key-vault://test123?
clientId=RAW({{clientId}})&clientSecret=RAW({{clientSecret}})&tenantId=RAW({{tenantId}})")
```

- getSecret: this operation will get a secret from Azure Key Vault

```

from("direct:getSecret")
  .setHeader(KeyVaultConstants.SECRET_NAME, "Test")
  .to("azure-key-vault://test123?
clientId=RAW({{clientId}})&clientSecret=RAW({{clientSecret}})&tenantId=RAW({{tenantId}})&operation=
getSecret")

```

- `deleteSecret`: this operation will delete a Secret from Azure Key Vault

```

from("direct:deleteSecret")
  .setHeader(KeyVaultConstants.SECRET_NAME, "Test")
  .to("azure-key-vault://test123?
clientId=RAW({{clientId}})&clientSecret=RAW({{clientSecret}})&tenantId=RAW({{tenantId}})&operation=
deleteSecret")

```

- `purgeDeletedSecret`: this operation will purge a deleted Secret from Azure Key Vault

```

from("direct:purgeDeletedSecret")
  .setHeader(KeyVaultConstants.SECRET_NAME, "Test")
  .to("azure-key-vault://test123?
clientId=RAW({{clientId}})&clientSecret=RAW({{clientSecret}})&tenantId=RAW({{tenantId}})&operation=
purgeDeletedSecret")

```

13.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.azure-key-vault.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.azure-key-vault.enabled</code>	Whether to enable auto configuration of the azure-key-vault component. This is enabled by default.		Boolean
<code>camel.component.azure-key-vault.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 14. AZURE SERVICEBUS

Since Camel 3.12

Both producer and consumer are supported

The `azure-servicebus` component that integrates [Azure ServiceBus](#). Azure ServiceBus is a fully managed enterprise integration message broker. Service Bus can decouple applications and services. Service Bus offers a reliable and secure platform for asynchronous transfer of data and state. Data is transferred between different applications and services using messages.

Prerequisites

You must have a valid Windows Azure Storage account. More information is available at [Azure Documentation Portal](#).

14.1. DEPENDENCIES

When using **azure-servicebus** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-azure-servicebus-starter</artifactId>
</dependency>
```

14.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

14.2.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

14.2.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

14.3. COMPONENT OPTIONS

The Azure ServiceBus component supports 25 options, which are listed below.

Name	Description	Default	Type
amqpRetryOptions (common)	Sets the retry options for Service Bus clients. If not specified, the default retry options are used.		AmqpRetryOptions
amqpTransportType (common)	Sets the transport type by which all the communication with Azure Service Bus occurs. Default value is <code>AmqpTransportType#AMQP</code> . Enum values: • <code>Amqp</code> • <code>AmqpWebSockets</code>	AMQP	AmqpTransportType
clientOptions (common)	Sets the ClientOptions to be sent from the client built from this builder, enabling customization of certain properties, as well as support the addition of custom header information. Refer to the ClientOptions documentation for more information.		ClientOptions
configuration (common)	The component configurations.		ServiceBusConfiguration

Name	Description	Default	Type
proxyOptions (common)	Sets the proxy configuration to use for ServiceBusSenderAsyncClient. When a proxy is configured, AmqpTransportType#AMQP_WEB_SOCKETS must be used for the transport type.		ProxyOptions
serviceBusType (common)	Required The service bus type of connection to execute. Queue is for typical queue option and topic for subscription based model. Enum values: • queue • topic	queue	ServiceBusType
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
consumerOperation (consumer)	Sets the desired operation to be used in the consumer. Enum values: • receiveMessages • peekMessages	receiveMessages	ServiceBusConsumerOperationDefinition
disableAutoComplete (consumer)	Disables auto-complete and auto-abandon of received messages. By default, a successfully processed message is <code>\\link ServiceBusReceiverAsyncClient#complete(ServiceBusReceivedMessage) completed}</code> . If an error happens when the message is processed, it is <code>\\link ServiceBusReceiverAsyncClient#abandon(ServiceBusReceivedMessage) abandoned}</code> .	false	boolean

Name	Description	Default	Type
maxAutoLockRenewDuration (consumer)	Sets the amount of time to continue auto-renewing the lock. Setting Duration#ZERO or null disables auto-renewal. For <code>ServiceBusReceiveMode#RECEIVE_AND_DELETE</code> mode, auto-renewal is disabled.	5m	Duration
peekNumMaxMessages (consumer)	Set the max number of messages to be peeked during the peek operation.		Integer
prefetchCount (consumer)	Sets the prefetch count of the receiver. For both <code>ServiceBusReceiveMode#PEEK_LOCK</code> and <code>ServiceBusReceiveMode#RECEIVE_AND_DELETE</code> modes the default value is 1. Prefetch speeds up the message flow by aiming to have a message readily available for local retrieval when and before the application asks for one using <code>ServiceBusReceiverAsyncClient#receiveMessages()</code> . Setting a non-zero value will prefetch that number of messages. Setting the value to zero turns prefetch off.		int
receiverAsyncClient (consumer)	Autowired Sets the receiverAsyncClient in order to consume messages by the consumer.		ServiceBusReceiverAsyncClient
serviceBusReceiveMode (consumer)	Sets the receive mode for the receiver. Enum values: • PEEK_LOCK • RECEIVE_AND_DELETE	PEEK_LOCK	ServiceBusReceiveMode
subQueue (consumer)	Sets the type of the SubQueue to connect to. Enum values: • NONE • DEAD_LETTER_QUEUE • TRANSFER_DEAD_LETTER_QUEUE		SubQueue
subscriptionName (consumer)	Sets the name of the subscription in the topic to listen to. <code>topicOrQueueName</code> and <code>serviceBusType=topic</code> must also be set. This property is required if <code>serviceBusType=topic</code> and the consumer is in use.		String

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
producerOperation (producer)	Sets the desired operation to be used in the producer. Enum values: ● sendMessages ● scheduleMessages	sendMessage	ServiceBusProducerOperationDefinition
scheduledEnqueueTime (producer)	Sets OffsetDateTime at which the message should appear in the Service Bus queue or topic.		OffsetDateTime
senderAsyncClient (producer)	Autowired Sets SenderAsyncClient to be used in the producer.		ServiceBusSenderAsyncClient
serviceBusTransactionContext (producer)	Represents transaction in service. This object just contains transaction id.		ServiceBusTransactionContext
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
connectionString (security)	Sets the connection string for a Service Bus namespace or a specific Service Bus resource.		String
fullyQualifiedNamespace (security)	Fully Qualified Namespace of the service bus.		String
tokenCredential (security)	A TokenCredential for Azure AD authentication, implemented in com.azure.identity.		TokenCredential

14.4. ENDPOINT OPTIONS

The Azure ServiceBus endpoint is configured using URI syntax:

```
azure-servicebus:topicOrQueueName
```

with the following path and query parameters:

14.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
topicOrQueueName (common)	Selected topic name or the queue name, that is depending on serviceBusType config. For example if serviceBusType=queue, then this will be the queue name and if serviceBusType=topic, this will be the topic name.		String

14.4.2. Query Parameters (25 parameters)

Name	Description	Default	Type
amqpRetryOptions (common)	Sets the retry options for Service Bus clients. If not specified, the default retry options are used.		AmqpRetryOptions
amqpTransportType (common)	Sets the transport type by which all the communication with Azure Service Bus occurs. Default value is AmqpTransportType#AMQP. Enum values: • Amqp • AmqpWebSockets	AMQP	AmqpTransportType
clientOptions (common)	Sets the ClientOptions to be sent from the client built from this builder, enabling customization of certain properties, as well as support the addition of custom header information. Refer to the ClientOptions documentation for more information.		ClientOptions
proxyOptions (common)	Sets the proxy configuration to use for ServiceBusSenderAsyncClient. When a proxy is configured, AmqpTransportType#AMQP_WEB_SOCKETS must be used for the transport type.		ProxyOptions

Name	Description	Default	Type
serviceBusType (common)	Required The service bus type of connection to execute. Queue is for typical queue option and topic for subscription based model. Enum values: • queue • topic	queue	ServiceBusType
consumerOperation (consumer)	Sets the desired operation to be used in the consumer. Enum values: • receiveMessages • peekMessages	receiveMessages	ServiceBusConsumerOperationDefinition
disableAutoComplete (consumer)	Disables auto-complete and auto-abandon of received messages. By default, a successfully processed message is <code>ServiceBusReceiverAsyncClient#complete(ServiceBusReceivedMessage) completed</code>. If an error happens when the message is processed, it is <code>ServiceBusReceiverAsyncClient#abandon(ServiceBusReceivedMessage) abandoned</code>.	false	boolean
maxAutoLockRenewDuration (consumer)	Sets the amount of time to continue auto-renewing the lock. Setting <code>Duration#ZERO</code> or null disables auto-renewal. For <code>ServiceBusReceiveMode#RECEIVE_AND_DELETE</code> mode, auto-renewal is disabled.	5m	Duration
peekNumMaxMessages (consumer)	Set the max number of messages to be peeked during the peek operation.		Integer

Name	Description	Default	Type
prefetchCount (consumer)	Sets the prefetch count of the receiver. For both <code>{link ServiceBusReceiveMode#PEEK_LOCK PEEK_LOCK}</code> and <code>{link ServiceBusReceiveMode#RECEIVE_AND_DELETE RECEIVE_AND_DELETE}</code> modes the default value is 1. Prefetch speeds up the message flow by aiming to have a message readily available for local retrieval when and before the application asks for one using <code>ServiceBusReceiverAsyncClient#receiveMessages()</code> . Setting a non-zero value will prefetch that number of messages. Setting the value to zero turns prefetch off.		int
receiverAsyncClient (consumer)	Autowired Sets the receiverAsyncClient in order to consume messages by the consumer.		ServiceBusReceiverAsyncClient
serviceBusReceiveMode (consumer)	Sets the receive mode for the receiver. Enum values: ● PEEK_LOCK ● RECEIVE_AND_DELETE	PEEK_LOCK	ServiceBusReceiveMode
subQueue (consumer)	Sets the type of the SubQueue to connect to. Enum values: ● NONE ● DEAD_LETTER_QUEUE ● TRANSFER_DEAD_LETTER_QUEUE		SubQueue
subscriptionName (consumer)	Sets the name of the subscription in the topic to listen to. <code>topicOrQueueName</code> and <code>serviceBusType=topic</code> must also be set. This property is required if <code>serviceBusType=topic</code> and the consumer is in use.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
producerOperation (producer)	Sets the desired operation to be used in the producer. Enum values: • <code>sendMessages</code> • <code>scheduleMessages</code>	<code>sendMessage</code> s	<code>ServiceBusProducerOperationDefinition</code>
scheduledEnqueueTime (producer)	Sets <code>OffsetDateTime</code> at which the message should appear in the Service Bus queue or topic.		<code>OffsetDateTime</code>
senderAsyncClient (producer)	Autowired Sets <code>SenderAsyncClient</code> to be used in the producer.		<code>ServiceBusSenderAsyncClient</code>
serviceBusTransactionContext (producer)	Represents transaction in service. This object just contains transaction id.		<code>ServiceBusTransactionContext</code>
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>boolean</code>
connectionString (security)	Sets the connection string for a Service Bus namespace or a specific Service Bus resource.		<code>String</code>

Name	Description	Default	Type
fullyQualifiedNamespace (security)	Fully Qualified Namespace of the service bus.		String
tokenCredential (security)	A TokenCredential for Azure AD authentication, implemented in com.azure.identity.		TokenCredential

14.5. ASYNC CONSUMER AND PRODUCER

This component implements the async Consumer and producer. This allows camel route to consume and produce events asynchronously without blocking any threads.

14.6. MESSAGE HEADERS

The Azure ServiceBus component supports 25 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelAzureServiceBusApplicationProperties (common) Constant: APPLICATION_PROPERTIES	The application properties (also known as custom properties) on messages sent and received by the producer and consumer, respectively.		Map
CamelAzureServiceBusContentType (consumer) Constant: CONTENT_TYPE	Gets the content type of the message.		String
CamelAzureServiceBusCorrelationId (consumer) Constant: CORRELATION_ID	Gets a correlation identifier.		String

Name	Description	Default	Type
CamelAzureServiceBusDeadLetterErrorDescription (consumer) Constant: DEAD_LETTER_ERROR_DESCRIPTION	Gets the description for a message that has been dead-lettered.		String
CamelAzureServiceBusDeadLetterReason (consumer) Constant: DEAD_LETTER_REASON	Gets the reason a message was dead-lettered.		String
CamelAzureServiceBusDeadLetterSource (consumer) Constant: DEAD_LETTER_SOURCE	Gets the name of the queue or subscription that this message was enqueued on, before it was dead-lettered.		String
CamelAzureServiceBusDeliveryCount (consumer) Constant: DELIVERY_COUNT	Gets the number of the times this message was delivered to clients.		long
CamelAzureServiceBusEnqueuedSequenceNumber (consumer) Constant: ENQUEUED_SEQUENCE_NUMBER	Gets the enqueued sequence number assigned to a message by Service Bus.		long

Name	Description	Default	Type
CamelAzureServiceBusEnqueuedTime (consumer) Constant: ENQUEUED_TIME	Gets the datetime at which this message was enqueued in Azure Service Bus.		OffsetDateTime
CamelAzureServiceBusExpiresAt (consumer) Constant: EXPIRES_AT	Gets the datetime at which this message will expire.		OffsetDateTime
CamelAzureServiceBusLockToken (consumer) Constant: LOCK_TOKEN	Gets the lock token for the current message.		String
CamelAzureServiceBusLockedUntil (consumer) Constant: LOCKED_UNTIL	Gets the datetime at which the lock of this message expires.		OffsetDateTime
CamelAzureServiceBusMessageId (consumer) Constant: MESSAGE_ID	Gets the identifier for the message.		String
CamelAzureServiceBusPartitionKey (consumer) Constant: PARTITION_KEY	Gets the partition key for sending a message to a partitioned entity.		String
CamelAzureServiceBusRawAmqpMessage (consumer) Constant: RAW_AMQP_MESSAGE	The representation of message as defined by AMQP protocol.		AmqpAnnotatedMessage

Name	Description	Default	Type
CamelAzureServiceBusReplyTo (consumer) Constant: REPLY_TO	Gets the address of an entity to send replies to.		String
CamelAzureServiceBusReplyToSessionId (consumer) Constant: REPLY_TO_SESSION_ID	Gets or sets a session identifier augmenting the ReplyTo address.		String
CamelAzureServiceBusSequenceNumber (consumer) Constant: SEQUENCE_NUMBER	Gets the unique number assigned to a message by Service Bus.		long
CamelAzureServiceBusSessionId (consumer) Constant: SESSION_ID	Gets the session id of the message.		String
CamelAzureServiceBusSubject (consumer) Constant: SUBJECT	Gets the subject for the message.		String
CamelAzureServiceBusTimeToLive (consumer) Constant: TIME_TO_LIVE	Gets the duration before this message expires.		Duration
CamelAzureServiceBusTo (consumer) Constant: TO	Gets the to address.		String

Name	Description	Default	Type
CamelAzureServiceBusScheduledEnqueueTime (common) Constant: SCHEDULED_ENQUEUE_TIME	(producer) Overrides the OffsetDateTime at which the message should appear in the Service Bus queue or topic. (consumer) Gets the scheduled enqueue time of this message.		OffsetDateTime
CamelAzureServiceBusServiceBusTransactionContext (producer) Constant: SERVICE_BUS_TRANSACTION_CONTEXT	Overrides the transaction in service. This object just contains transaction id.		ServiceBusTransactionContext
CamelAzureServiceBusProducerOperation (producer) Constant: PRODUCER_OPERATION	Overrides the desired operation to be used in the producer. Enum values: • <code>sendMessages</code> • <code>scheduleMessages</code>		ServiceBusProducerOperationDefinition

14.6.1. Message Body

In the producer, this component accepts message body of **String** type or **List<String>** to send batch messages.

In the consumer, the returned message body will be of type `String`.

14.6.2. Azure ServiceBus Producer operations

Operation	Description
sendMessages	Sends a set of messages to a Service Bus queue or topic using a batched approach.
scheduleMessages	Sends a scheduled message to the Azure Service Bus entity this sender is connected to. A scheduled message is enqueued and made available to receivers only at the scheduled enqueue time.

14.6.3. Azure ServiceBus Consumer operations

Operation	Description
receiveMessages	Receives an infinite stream of messages from the Service Bus entity.
peekMessages	Reads the next batch of active messages without changing the state of the receiver or the message source.

14.6.3.1. Examples

- **sendMessages**

```
from("direct:start")
  .process(exchange -> {
    final List<Object> inputBatch = new LinkedList<>();
    inputBatch.add("test batch 1");
    inputBatch.add("test batch 2");
    inputBatch.add("test batch 3");
    inputBatch.add(123456);

    exchange.getIn().setBody(inputBatch);
  })
  .to("azure-servicebus:test/?connectionString=test")
  .to("mock:result");
```

- **scheduleMessages**

```
from("direct:start")
  .process(exchange -> {
    final List<Object> inputBatch = new LinkedList<>();
    inputBatch.add("test batch 1");
    inputBatch.add("test batch 2");
    inputBatch.add("test batch 3");
    inputBatch.add(123456);

    exchange.getIn().setHeader(ServiceBusConstants.SCHEDULED_ENQUEUE_TIME,
      OffsetDateTime.now());
    exchange.getIn().setBody(inputBatch);
  })
  .to("azure-servicebus:test/?connectionString=test&producerOperation=scheduleMessages")
  .to("mock:result");
```

- **receiveMessages**

```
from("azure-servicebus:test/?connectionString=test")
  .log("${body}")
  .to("mock:result");
```

- `peekMessages`

```
from("azure-servicebus:test//?
connectionString=test&consumerOperation=peekMessages&peekNumMaxMessages=3")
.log("${body}")
.to("mock:result");
```

14.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 26 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.azure-servicebus.amqp-retry-options</code>	Sets the retry options for Service Bus clients. If not specified, the default retry options are used. The option is a <code>com.azure.core.amqp.AmqpRetryOptions</code> type.		<code>AmqpRetryOptions</code>
<code>camel.component.azure-servicebus.amqp-transport-type</code>	Sets the transport type by which all the communication with Azure Service Bus occurs. Default value is <code>AmqpTransportType#AMQP</code> .		<code>AmqpTransportType</code>
<code>camel.component.azure-servicebus.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	<code>true</code>	<code>Boolean</code>
<code>camel.component.azure-servicebus.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	<code>false</code>	<code>Boolean</code>
<code>camel.component.azure-servicebus.client-options</code>	Sets the <code>ClientOptions</code> to be sent from the client built from this builder, enabling customization of certain properties, as well as support the addition of custom header information. Refer to the <code>ClientOptions</code> documentation for more information. The option is a <code>com.azure.core.util.ClientOptions</code> type.		<code>ClientOptions</code>

Name	Description	Default	Type
<code>camel.component.azure-servicebus.configuration</code>	The component configurations. The option is a <code>org.apache.camel.component.azure.servicebus.ServiceBusConfiguration</code> type.		<code>ServiceBusConfiguration</code>
<code>camel.component.azure-servicebus.connection-string</code>	Sets the connection string for a Service Bus namespace or a specific Service Bus resource.		<code>String</code>
<code>camel.component.azure-servicebus.consumer-operation</code>	Sets the desired operation to be used in the consumer.		<code>ServiceBusConsumerOperationDefinition</code>
<code>camel.component.azure-servicebus.disable-auto-complete</code>	Disables auto-complete and auto-abandon of received messages. By default, a successfully processed message is <code>\\link ServiceBusReceiverAsyncClient#complete(ServiceBusReceivedMessage) completed</code> . If an error happens when the message is processed, it is <code>\\link ServiceBusReceiverAsyncClient#abandon(ServiceBusReceivedMessage) abandoned</code> .	<code>false</code>	<code>Boolean</code>
<code>camel.component.azure-servicebus.enabled</code>	Whether to enable auto configuration of the azure-servicebus component. This is enabled by default.		<code>Boolean</code>
<code>camel.component.azure-servicebus.fully-qualified-namespace</code>	Fully Qualified Namespace of the service bus.		<code>String</code>
<code>camel.component.azure-servicebus.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>Boolean</code>

Name	Description	Default	Type
camel.component.azure-servicebus.max-auto-lock-renew-duration	Sets the amount of time to continue auto-renewing the lock. Setting Duration#ZERO or null disables auto-renewal. For <code>ServiceBusReceiveMode#RECEIVE_AND_DELETE</code> mode, auto-renewal is disabled. The option is a <code>java.time.Duration</code> type.		Duration
camel.component.azure-servicebus.peek-num-max-messages	Set the max number of messages to be peeked during the peek operation.		Integer
camel.component.azure-servicebus.prefetch-count	Sets the prefetch count of the receiver. For both <code>ServiceBusReceiveMode#PEEK_LOCK</code> and <code>ServiceBusReceiveMode#RECEIVE_AND_DELETE</code> modes the default value is 1. Prefetch speeds up the message flow by aiming to have a message readily available for local retrieval when and before the application asks for one using <code>ServiceBusReceiverAsyncClient#receiveMessages()</code> . Setting a non-zero value will prefetch that number of messages. Setting the value to zero turns prefetch off.		Integer
camel.component.azure-servicebus.producer-operation	Sets the desired operation to be used in the producer.		ServiceBusProducerOperationDefinition
camel.component.azure-servicebus.proxy-options	Sets the proxy configuration to use for <code>ServiceBusSenderAsyncClient</code> . When a proxy is configured, <code>AmqpTransportType#AMQP_WEB_SOCKETS</code> must be used for the transport type. The option is a <code>com.azure.core.amqp.ProxyOptions</code> type.		ProxyOptions
camel.component.azure-servicebus.receiver-async-client	Sets the <code>receiverAsyncClient</code> in order to consume messages by the consumer. The option is a <code>com.azure.messaging.servicebus.ServiceBusReceiverAsyncClient</code> type.		ServiceBusReceiverAsyncClient
camel.component.azure-servicebus.scheduled-enqueue-time	Sets <code>OffsetDateTime</code> at which the message should appear in the Service Bus queue or topic. The option is a <code>java.time.OffsetDateTime</code> type.		OffsetDateTime

Name	Description	Default	Type
camel.component.azure-servicebus.sender-async-client	Sets SenderAsyncClient to be used in the producer. The option is a com.azure.messaging.servicebus.ServiceBusSenderAsyncClient type.		ServiceBusSenderAsyncClient
camel.component.azure-servicebus.service-bus-receive-mode	Sets the receive mode for the receiver.		ServiceBusReceiveMode
camel.component.azure-servicebus.service-bus-transaction-context	Represents transaction in service. This object just contains transaction id. The option is a com.azure.messaging.servicebus.ServiceBusTransactionContext type.		ServiceBusTransactionContext
camel.component.azure-servicebus.service-bus-type	The service bus type of connection to execute. Queue is for typical queue option and topic for subscription based model.		ServiceBusType
camel.component.azure-servicebus.sub-queue	Sets the type of the SubQueue to connect to.		SubQueue
camel.component.azure-servicebus.subscription-name	Sets the name of the subscription in the topic to listen to. topicOrQueueName and serviceBusType=topic must also be set. This property is required if serviceBusType=topic and the consumer is in use.		String
camel.component.azure-servicebus.token-credential	A TokenCredential for Azure AD authentication, implemented in com.azure.identity. The option is a com.azure.core.credential.TokenCredential type.		TokenCredential

CHAPTER 15. AZURE STORAGE BLOB SERVICE

Both producer and consumer are supported

The Azure Storage Blob component is used for storing and retrieving blobs from [Azure Storage Blob Service](#) using **Azure APIs v12**. However in case of versions above v12, we will see if this component can adopt these changes depending on how much breaking changes can result.

Prerequisites

You must have a valid Windows Azure Storage account. More information is available at [Azure Documentation Portal](#).

15.1. DEPENDENCIES

When using **azure-storage-blob** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-azure-storage-blob-starter</artifactId>
</dependency>
```

15.2. URI FORMAT

```
azure-storage-blob://accountName[/containerName][?options]
```

In case of consumer, **accountName**, **containerName** are required. In case of producer, it depends on the operation that is being requested, for example if operation is on a container level, for example, **createContainer**, **accountName** and **containerName** are only required, but in case of operation being requested in blob level, for example, **getBlob**, **accountName**, **containerName** and **blobName** are required.

The blob will be created if it does not already exist. You can append query options to the URI in the following format,

?options=value&option2=value&...

15.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

15.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a

few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

15.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

15.4. COMPONENT OPTIONS

The Azure Storage Blob Service component supports 35 options, which are listed below.

Name	Description	Default	Type
blobName (common)	The blob name, to consume specific blob from a container. However on producer, is only required for the operations on the blob level.		String
blobOffset (common)	Set the blob offset for the upload or download operations, default is 0.	0	long

Name	Description	Default	Type
blobType (common)	The blob type in order to initiate the appropriate settings for each blob type. Enum values: • blockblob • appendblob • pageblob	blockblob	BlobType
closeStreamAfterRead (common)	Close the stream after read or keep it open, default is true.	true	boolean
configuration (common)	The component configurations.		BlobConfiguration
credentials (common)	StorageSharedKeyCredential can be injected to create the azure client, this holds the important authentication information.		StorageSharedKeyCredential
credentialType (common)	Determines the credential strategy to adopt. Enum values: • SHARED_ACCOUNT_KEY • SHARED_KEY_CREDENTIAL • AZURE_IDENTITY • AZURE_SAS	AZURE_IDENTITY	CredentialType
dataCount (common)	How many bytes to include in the range. Must be greater than or equal to 0 if specified.		Long
fileDir (common)	The file directory where the downloaded blobs will be saved to, this can be used in both, producer and consumer.		String
maxResultsPerPage (common)	Specifies the maximum number of blobs to return, including all BlobPrefix elements. If the request does not specify maxResultsPerPage or specifies a value greater than 5,000, the server will return up to 5,000 items.		Integer

Name	Description	Default	Type
maxRetryRequests (common)	Specifies the maximum number of additional HTTP Get requests that will be made while reading the data from a response body.	0	int
prefix (common)	Filters the results to return only blobs whose names begin with the specified prefix. May be null to return all blobs.		String
regex (common)	Filters the results to return only blobs whose names match the specified regular expression. May be null to return all if both prefix and regex are set, regex takes the priority and prefix is ignored.		String
sasToken (common)	Set a SAS Token in case of usage of Shared Access Signature		String
serviceClient (common)	Autowired Client to a storage account. This client does not hold any state about a particular storage account but is instead a convenient way of sending off appropriate requests to the resource on the service. It may also be used to construct URLs to blobs and containers. This client contains operations on a service account. Operations on a container are available on BlobContainerClient through <code>BlobServiceClient#getBlobContainerClient(String)</code> , and operations on a blob are available on BlobClient through <code>BlobContainerClient#getBlobClient(String)</code> .		BlobServiceClient
timeout (common)	An optional timeout value beyond which a RuntimeException will be raised.		Duration
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
blobSequenceNumber (producer)	A user-controlled value that you can use to track requests. The value of the sequence number must be between 0 and 263 - 1. The default value is 0.	0	Long

Name	Description	Default	Type
blockListType (producer)	Specifies which type of blocks to return. Enum values: ● committed ● uncommitted ● all	COMMITTED	BlockListType
changeFeedContext (producer)	When using getChangeFeed producer operation, this gives additional context that is passed through the Http pipeline during the service call.		Context
changeFeedEndTime (producer)	When using getChangeFeed producer operation, this filters the results to return events approximately before the end time. Note: A few events belonging to the next hour can also be returned. A few events belonging to this hour can be missing; to ensure all events from the hour are returned, round the end time up by an hour.		OffsetDateTime
changeFeedStartTime (producer)	When using getChangeFeed producer operation, this filters the results to return events approximately after the start time. Note: A few events belonging to the previous hour can also be returned. A few events belonging to this hour can be missing; to ensure all events from the hour are returned, round the start time down by an hour.		OffsetDateTime
closeStreamAfterWrite (producer)	Close the stream after write or keep it open, default is true.	true	boolean
commitBlockListLater (producer)	When is set to true, the staged blocks will not be committed directly.	true	boolean
createAppendBlob (producer)	When is set to true, the append blocks will be created when committing append blocks.	true	boolean
createPageBlob (producer)	When is set to true, the page blob will be created when uploading page blob.	true	boolean
downloadLinkExpiration (producer)	Override the default expiration (millis) of URL download link.		Long

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
operation (producer)	The blob operation that can be used with this component on the producer. Enum values: ● listBlobContainers ● createBlobContainer ● deleteBlobContainer ● listBlobs ● getBlob ● deleteBlob ● downloadBlobToFile ● downloadLink ● uploadBlockBlob ● stageBlockBlobList ● commitBlobBlockList ● getBlobBlockList ● createAppendBlob ● commitAppendBlob ● createPageBlob ● uploadPageBlob ● resizePageBlob ● clearPageBlob ● getPageBlobRanges	listBlobContainers	BlobOperationsDefinition

Name	Description	Default	Type
pageBlobSize (producer)	Specifies the maximum size for the page blob, up to 8 TB. The page blob size must be aligned to a 512-byte boundary.	512	Long
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
healthCheckConsumerEnabled (health)	Used for enabling or disabling all consumer based health checks from this component.	true	boolean
healthCheckProducerEnabled (health)	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting camel.health.producersEnabled=true.	true	boolean
accessKey (security)	Access key for the associated azure account name to be used for authentication with azure blob services.		String
sourceBlobAccessKey (security)	Source Blob Access Key: for copyblob operation, sadly, we need to have an accessKey for the source blob we want to copy Passing an accessKey as header, it's unsafe so we could set as key.		String

15.5. ENDPOINT OPTIONS

The Azure Storage Blob Service endpoint is configured using URI syntax:

```
azure-storage-blob:accountName/containerName
```

with the following path and query parameters:

15.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
accountName (common)	Azure account name to be used for authentication with azure blob services.		String

Name	Description	Default	Type
containerName (common)	The blob container name.		String

15.5.2. Query Parameters (50 parameters)

Name	Description	Default	Type
blobName (common)	The blob name, to consume specific blob from a container. However on producer, is only required for the operations on the blob level.		String
blobOffset (common)	Set the blob offset for the upload or download operations, default is 0.	0	long
blobServiceClient (common)	Client to a storage account. This client does not hold any state about a particular storage account but is instead a convenient way of sending off appropriate requests to the resource on the service. It may also be used to construct URLs to blobs and containers. This client contains operations on a service account. Operations on a container are available on BlobContainerClient through <code>getBlobContainerClient(String)</code> , and operations on a blob are available on BlobClient through <code>getBlobContainerClient(String).getBlobClient(String)</code> .		BlobServiceClient
blobType (common)	The blob type in order to initiate the appropriate settings for each blob type. Enum values: • blockblob • appendblob • pageblob	blockblob	BlobType
closeStreamAfterRead (common)	Close the stream after read or keep it open, default is true.	true	boolean
credentials (common)	StorageSharedKeyCredential can be injected to create the azure client, this holds the important authentication information.		StorageSharedKeyCredential

Name	Description	Default	Type
credentialType (common)	Determines the credential strategy to adopt. Enum values: • SHARED_ACCOUNT_KEY • SHARED_KEY_CREDENTIAL • AZURE_IDENTITY • AZURE_SAS	AZURE_IDENTITY	CredentialType
dataCount (common)	How many bytes to include in the range. Must be greater than or equal to 0 if specified.		Long
fileDir (common)	The file directory where the downloaded blobs will be saved to, this can be used in both, producer and consumer.		String
maxResultsPerPage (common)	Specifies the maximum number of blobs to return, including all BlobPrefix elements. If the request does not specify maxResultsPerPage or specifies a value greater than 5,000, the server will return up to 5,000 items.		Integer
maxRetryRequests (common)	Specifies the maximum number of additional HTTP Get requests that will be made while reading the data from a response body.	0	int
prefix (common)	Filters the results to return only blobs whose names begin with the specified prefix. May be null to return all blobs.		String
regex (common)	Filters the results to return only blobs whose names match the specified regular expression. May be null to return all if both prefix and regex are set, regex takes the priority and prefix is ignored.		String
sasToken (common)	Set a SAS Token in case of usage of Shared Access Signature.		String

Name	Description	Default	Type
serviceClient (common)	Autowired Client to a storage account. This client does not hold any state about a particular storage account but is instead a convenient way of sending off appropriate requests to the resource on the service. It may also be used to construct URLs to blobs and containers. This client contains operations on a service account. Operations on a container are available on BlobContainerClient through BlobServiceClient#getBlobContainerClient(String), and operations on a blob are available on BlobClient through BlobContainerClient#getBlobClient(String).		BlobServiceClient
timeout (common)	An optional timeout value beyond which a RuntimeException will be raised.		Duration
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern

Name	Description	Default	Type
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
blobSequenceNumber (producer)	A user-controlled value that you can use to track requests. The value of the sequence number must be between 0 and 263 - 1. The default value is 0.	0	Long
blockListType (producer)	Specifies which type of blocks to return. Enum values: ● committed ● uncommitted ● all	COMMITTED	BlockListType
changeFeedContext (producer)	When using <code>getChangeFeed</code> producer operation, this gives additional context that is passed through the Http pipeline during the service call.		Context
changeFeedEndTime (producer)	When using <code>getChangeFeed</code> producer operation, this filters the results to return events approximately before the end time. Note: A few events belonging to the next hour can also be returned. A few events belonging to this hour can be missing; to ensure all events from the hour are returned, round the end time up by an hour.		OffsetDateTime
changeFeedStartTime (producer)	When using <code>getChangeFeed</code> producer operation, this filters the results to return events approximately after the start time. Note: A few events belonging to the previous hour can also be returned. A few events belonging to this hour can be missing; to ensure all events from the hour are returned, round the start time down by an hour.		OffsetDateTime
closeStreamAfterWrite (producer)	Close the stream after write or keep it open, default is true.	true	boolean
commitBlockListLater (producer)	When is set to true, the staged blocks will not be committed directly.	true	boolean

Name	Description	Default	Type
createAppendBlob (producer)	When is set to true, the append blocks will be created when committing append blocks.	true	boolean
createPageBlob (producer)	When is set to true, the page blob will be created when uploading page blob.	true	boolean
downloadLinkExpiration (producer)	Override the default expiration (millis) of URL download link.		Long
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
operation (producer)	The blob operation that can be used with this component on the producer. Enum values: • listBlobContainers • createBlobContainer • deleteBlobContainer • listBlobs • getBlob • deleteBlob • downloadBlobToFile • downloadLink • uploadBlockBlob • stageBlockBlobList • commitBlobBlockList • getBlobBlockList • createAppendBlob • commitAppendBlob • createPageBlob • uploadPageBlob • resizePageBlob • clearPageBlob • getPageBlobRanges	listBlobContainers	BlobOperationsDefinition
pageBlobSize (producer)	Specifies the maximum size for the page blob, up to 8 TB. The page blob size must be aligned to a 512-byte boundary.	512	Long
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int

Name	Description	Default	Type
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then <code>backoffIdleThreshold</code> and/or <code>backoffErrorThreshold</code> must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the <code>ScheduledPollConsumer</code> will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object

Name	Description	Default	Type
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
accessKey (security)	Access key for the associated azure account name to be used for authentication with azure blob services.		String
sourceBlobAccessKey (security)	Source Blob Access Key: for copyblob operation, sadly, we need to have an accessKey for the source blob we want to copy Passing an accessKey as header, it's unsafe so we could set as key.		String

Required information options

To use this component, you have 3 options in order to provide the required Azure authentication information:

- Provide a link: [BlobServiceClient](#) instance which can be injected into blobServiceClient. Note: No need to create a specific client, for Examples: **BlockBlobClient**, the **BlobServiceClient** represents the upper level which can be used to retrieve lower level clients.
- Provide an Azure Identity, when specifying **credentialType=AZURE_IDENTITY** and providing required [environment variables](#). This enables service principal (for example, app registration) authentication with secret/certificate as well as username password. Note that this is the default authentication strategy.
- Provide a shared storage account key, when specifying

credentialType=SHARED_ACCOUNT_KEY and providing **accountName** and **accessKey** for your Azure account, this is the simplest way to get started. The **accessKey** can be generated through your Azure portal.

- Provide a shared storage account key, when specifying **credentialType=SHARED_KEY_CREDENTIAL** and providing a [StorageSharedKeyCredential](#) instance which can be injected into credentials option.
- Via Azure SAS, when specifying **credentialType=AZURE_SAS** and providing a SAS Token parameter through the `sasToken` parameter.

15.6. USAGE

For example, in order to download a blob content from the blob **hello.txt** located on the **container1** in the **camelazure** storage account, use the following snippet:

```
from("azure-storage-blob://camelazure/container1?
blobName=hello.txt&credentialType=SHARED_ACCOUNT_KEY&accessKey=RAW(yourAccessKey)"
).to("file://blobdirectory");
```

15.6.1. Message headers

The Azure Storage Blob Service component supports 63 message header(s), which is/are listed below:

Header	Variable Name	Type	Operations	Description
CamelAzureStorageBlobTimeout	BlobConstants.TIMEOUT	Duration	All	An optional timeout value beyond which a {@link RuntimeException} will be raised.
CamelAzureStorageBlobMetadata	BlobConstants.METADATA	Map<String,String>	Operations related to container and blob	Metadata to associate with the container or blob.
CamelAzureStorageBlobPublicAccessType	BlobConstants.PUBLIC_ACCESS_TYPE	PublicAccessType	createContainer	Specifies how the data in this container is available to the public. Pass null for no public access.

Header	Variable Name	Type	Operations	Description
CamelAzureStorageBlobRequestCondition	BlobConstants.BLOB_REQUEST_CONDITION	BlobRequestConditions	Operations related to container and blob	This contains values which will restrict the successful operation of a variety of requests to the conditions present. These conditions are entirely optional.
CamelAzureStorageBlobListDetails	BlobConstants.BLOB_LIST_DETAILS	BlobListDetails	listBlobs	The details for listing specific blobs
CamelAzureStorageBlobPrefix	BlobConstants.PREFIX	String	listBlobs, getBlob	Filters the results to return only blobs whose names begin with the specified prefix. May be null to return all blobs.
CamelAzureStorageBlobMaxResultsPerPage	BlobConstants.MAX_RESULTS_PER_PAGE	Integer	listBlobs	Specifies the maximum number of blobs to return, including all BlobPrefix elements. If the request does not specify <code>maxResultsPerPage</code> or specifies a value greater than 5,000, the server will return up to 5,000 items.
CamelAzureStorageBlobListBlobOptions	BlobConstants.LIST_BLOB_OPTIONS	ListBlobsOptions	listBlobs	Defines options available to configure the behavior of a call to <code>listBlobsFlatSegment</code> on a <code>{@link BlobContainerClient}</code> object.

Header	Variable Name	Type	Operations	Description
CamelAzureStorageBlobHttpHeaders	BlobConstants.BLOB_HTTP_HEADERS	BlobHttpHeaders	uploadBlockBlob, commitBlockBlobList, createAppendBlob, createPageBlob	Additional parameters for a set of operations.
CamelAzureStorageBlobAccessTier	BlobConstants.ACCESS_TIER	AccessTier	uploadBlockBlob, commitBlockBlobList	Defines values for AccessTier.
CamelAzureStorageBlobContentMD5	BlobConstants.CONTENT_MD5	byte[]	Most operations related to upload blob	An MD5 hash of the block content. This hash is used to verify the integrity of the block during transport. When this header is specified, the storage service compares the hash of the content that has arrived with this header value. Note that this MD5 hash is not stored with the blob. If the two hashes do not match, the operation will fail.
CamelAzureStorageBlobPageBlobRange	BlobConstants.PAGE_BLOB_RANGE	PageRange	Operations related to page blob	A {@link PageRange} object. Given that pages must be aligned with 512-byte boundaries, the start offset must be a modulus of 512 and the end offset must be a modulus of 512 - 1. Examples of valid byte ranges are 0-511, 512-1023, etc.

Header	Variable Name	Type	Operations	Description
CamelAzureStorageBlobCommitBlobBlockListLater	BlobConstants.COMMIT_BLOCK_LIST_LATER	boolean	stageBlockBlobList	When is set to true , the staged blocks will not be committed directly.
CamelAzureStorageBlobCreateAppendBlob	BlobConstants.CREATE_APPEND_BLOB	boolean	commitAppendBlob	When is set to true , the append blocks will be created when committing append blocks.
CamelAzureStorageBlobCreatePageBlob	BlobConstants.CREATE_PAGE_BLOB	boolean	uploadPageBlob	When is set to true , the page blob will be created when uploading page blob.
CamelAzureStorageBlobBlockListType	BlobConstants.BLOCK_LIST_TYPE	BlockListType	getBlobBlockList	Specifies which type of blocks to return.
CamelAzureStorageBlobPageBlobSize	BlobConstants.PAGE_BLOB_SIZE	Long	createPageBlob , resizePageBlob	Specifies the maximum size for the page blob, up to 8 TB. The page blob size must be aligned to a 512-byte boundary.
CamelAzureStorageBlobSequenceNumber	BlobConstants.BLOB_SEQUENCE_NUMBER	Long	createPageBlob	A user-controlled value that you can use to track requests. The value of the sequence number must be between 0 and $2^{63} - 1$. The default value is 0.

Header	Variable Name	Type	Operations	Description
CamelAzureStorageBlobDeleteSnapshotsOptionType	BlobConstants.DELETE_SNAPSHOT_OPTION_TYPE	DeleteSnapshotOptionType	deleteBlob	Specifies the behavior for deleting the snapshots on this blob. <code>\{Include}</code> will delete the base blob and all snapshots. <code>\{Only}</code> will delete only the snapshots. If a snapshot is being deleted, you must pass null.
CamelAzureStorageBlobListBlobContainersOptions	BlobConstants.LIST_BLOB_CONTAINERS_OPTIONS	ListBlobContainersOptions	listBlobContainers	A <code>\{@link ListBlobContainersOptions}</code> which specifies what data should be returned by the service.
CamelAzureStorageBlobParallelTransferOptions	BlobConstants.PARALLEL_TRANSFER_OPTIONS	ParallelTransferOptions	downloadBlobToFile	<code>\{@link ParallelTransferOptions}</code> to use to download to file. Number of parallel transfers parameter is ignored.
CamelAzureStorageBlobFileDir	BlobConstants.FILE_DIR	String	downloadBlobToFile	The file directory where the downloaded blobs will be saved to.
CamelAzureStorageBlobDownloadLinkExpiration	BlobConstants.DOWNLOAD_LINK_EXPIRATION	Long	downloadLink	Override the default expiration (millis) of URL download link.
CamelAzureStorageBlobBlobName	BlobConstants.BLOB_NAME	String	Operations related to blob	Override/set the blob name on the exchange headers.
CamelAzureStorageBlobContainerName	BlobConstants.BLOB_CONTAINER_NAME	String	Operations related to container and blob	Override/set the container name on the exchange headers.

Header	Variable Name	Type	Operations	Description
CamelAzureStorageBlobOperation	BlobConstants.BLOB_OPERATION	BlobOperations Definition	All	Specify the producer operation to execute, please see the doc on this page related to producer operation.
CamelAzureStorageBlobRegex	BlobConstants.REGEX	String	listBlobs, getBlob	Filters the results to return only blobs whose names match the specified regular expression. May be null to return all. If both prefix and regex are set, regex takes the priority and prefix is ignored.
CamelAzureStorageBlobChangeFeedStartTime	BlobConstants.CHANGE_FEED_START_TIME	OffsetDateTime	getChangeFeed	It filters the results to return events approximately after the start time. Note: A few events belonging to the previous hour can also be returned. A few events belonging to this hour can be missing; to ensure all events from the hour are returned, round the start time down by an hour.

Header	Variable Name	Type	Operations	Description
CamelAzureStorageBlobChangeFeedEndTime	BlobConstants.CHANGE_FEED_END_TIME	OffsetDateTime	getChangeFeed	It filters the results to return events approximately before the end time. Note: A few events belonging to the next hour can also be returned. A few events belonging to this hour can be missing; to ensure all events from the hour are returned, round the end time up by an hour.
CamelAzureStorageBlobChangeFeedContext	BlobConstants.CHANGE_FEED_CONTEXT	Context	getChangeFeed	This gives additional context that is passed through the Http pipeline during the service call.
CamelAzureStorageBlobSourceBlobAccountName	BlobConstants.SOURCE_BLOB_ACCOUNT_NAME	String	copyBlob	The source blob account name to be used as source account name in a copy blob operation
CamelAzureStorageBlobSourceBlobContainerName	BlobConstants.SOURCE_BLOB_CONTAINER_NAME	String	copyBlob	The source blob container name to be used as source container name in a copy blob operation

15.6.2. Message headers set by either component producer or consumer

Header	Variable Name	Type	Description
CamelAzureStorageBlobAccessTier	BlobConstants.ACCESS_TIER	AccessTier	Access tier of the blob.

Header	Variable Name	Type	Description
CamelAzureStorageBlobAccessTierChangeTime	BlobConstants.ACCESS_TIER_CHANGE_TIME	OffsetDateTime	Datetime when the access tier of the blob last changed.
CamelAzureStorageBlobArchiveStatus	BlobConstants.ARCHIVE_STATUS	ArchiveStatus	Archive status of the blob.
CamelAzureStorageBlobCreationTime	BlobConstants.CREATION_TIME	OffsetDateTime	Creation time of the blob.
CamelAzureStorageBlobSequenceNumber	BlobConstants.BLOB_SEQUENCE_NUMBER	Long	The current sequence number for a page blob.
CamelAzureStorageBlobBlobSize	BlobConstants.BLOB_SIZE	long	The size of the blob.
CamelAzureStorageBlobBlobType	BlobConstants.BLOB_TYPE	BlobType	The type of the blob.
CamelAzureStorageBlobCacheControl	BlobConstants.CACHE_CONTROL	String	Cache control specified for the blob.
CamelAzureStorageBlobCommittedBlockCount	BlobConstants.COMMITTED_BLOCK_COUNT	Integer	Number of blocks committed to an append blob
CamelAzureStorageBlobContentDisposition	BlobConstants.CONTENT_DISPOSITION	String	Content disposition specified for the blob.
CamelAzureStorageBlobContentEncoding	BlobConstants.CONTENT_ENCODING	String	Content encoding specified for the blob.
CamelAzureStorageBlobContentLanguage	BlobConstants.CONTENT_LANGUAGE	String	Content language specified for the blob.

Header	Variable Name	Type	Description
CamelAzureStorageBlobContentMd5	BlobConstants.CONTENT_MD5	byte[]	Content MD5 specified for the blob.
CamelAzureStorageBlobContentType	BlobConstants.CONTENT_TYPE	String	Content type specified for the blob.
CamelAzureStorageBlobCopyCompletionTime	BlobConstants.COPY_COMPILATION_TIME	OffsetDateTime	Datetime when the last copy operation on the blob completed.
CamelAzureStorageBlobCopyDestinationSnapshot	BlobConstants.COPY_DESTINATION_SNAPSHOT	String	Snapshot identifier of the last incremental copy snapshot for the blob.
CamelAzureStorageBlobCopyId	BlobConstants.COPY_ID	String	Identifier of the last copy operation performed on the blob.
CamelAzureStorageBlobCopyProgress	BlobConstants.COPY_PROGRESS	String	Progress of the last copy operation performed on the blob.
CamelAzureStorageBlobCopySource	BlobConstants.COPY_SOURCE	String	Source of the last copy operation performed on the blob.
CamelAzureStorageBlobCopyStatus	BlobConstants.COPY_STATUS	CopyStatusType	Status of the last copy operation performed on the blob.
CamelAzureStorageBlobCopyStatusDescription	BlobConstants.COPY_STATUS_DESCRIPTION	String	Description of the last copy operation on the blob.
CamelAzureStorageBlobETag	BlobConstants.E_TAG	String	The E Tag of the blob
CamelAzureStorageBlobsAccessTierInferred	BlobConstants.IS_ACCESS_TIER_INFERRED	boolean	Flag indicating if the access tier of the blob was inferred from properties of the blob.
CamelAzureStorageBlobsIncrementalCopy	BlobConstants.IS_INCREMENTAL_COPY	boolean	Flag indicating if the blob was incrementally copied.

Header	Variable Name	Type	Description
CamelAzureStorageBlobsServerEncrypted	BlobConstants.IS_SERVER_ENCRYPTED	boolean	Flag indicating if the blob's content is encrypted on the server.
CamelAzureStorageBlobLastModified	BlobConstants.LAST_MODIFIED	OffsetDateTime	Datetime when the blob was last modified.
CamelAzureStorageBlobLeaseDuration	BlobConstants.LEASE_DURATION	LeaseDurationType	Type of lease on the blob.
CamelAzureStorageBlobLeaseState	BlobConstants.LEASE_STATE	LeaseStateType	State of the lease on the blob.
CamelAzureStorageBlobLeaseStatus	BlobConstants.LEASE_STATUS	LeaseStatusType	Status of the lease on the blob.
CamelAzureStorageBlobMetadata	BlobConstants.METADATA	Map<String, String>	Additional metadata associated with the blob.
CamelAzureStorageBlobAppendOffset	BlobConstants.APPEND_OFFSET	String	The offset at which the block was committed to the block blob.
CamelAzureStorageBlobFileName	BlobConstants.FILE_NAME	String	The downloaded filename from the operation downloadBlobToFile .
CamelAzureStorageBlobDownloadLink	BlobConstants.DOWNLOAD_LINK	String	The download link generated by downloadLink operation.
CamelAzureStorageBlobRawHttpHeaders	BlobConstants.RAW_HTTP_HEADERS	HttpHeaders	Returns non-parsed httpHeaders that can be used by the user.

15.6.3. Advanced Azure Storage Blob configuration

If your Camel Application is running behind a firewall or if you need to have more control over the **BlobServiceClient** instance configuration, you can create your own instance:

```
StorageSharedKeyCredential credential = new StorageSharedKeyCredential("yourAccountName",
```

```

"yourAccessKey");
String uri = String.format("https://%s.blob.core.windows.net", "yourAccountName");

BlobServiceClient client = new BlobServiceClientBuilder()
    .endpoint(uri)
    .credential(credential)
    .buildClient();
// This is camel context
context.getRegistry().bind("client", client);

```

Then refer to this instance in your Camel **azure-storage-blob** component configuration:

```

from("azure-storage-blob://cameldev/container1?blobName=myblob&serviceClient=#client")
.to("mock:result");

```

15.6.4. Automatic detection of BlobServiceClient client in registry

The component is capable of detecting the presence of an BlobServiceClient bean into the registry. If it's the only instance of that type it will be used as client and you won't have to define it as uri parameter, like the example above. This may be really useful for smarter configuration of the endpoint.

15.6.5. Azure Storage Blob Producer operations

Camel Azure Storage Blob component provides wide range of operations on the producer side:

Operations on the service level

For these operations, **accountName** is **required**.

Operation	Description
listBlobContainers	Get the content of the blob. You can restrict the output of this operation to a blob range.
getChangeFeed	Returns transaction logs of all the changes that occur to the blobs and the blob metadata in your storage account. The change feed provides ordered, guaranteed, durable, immutable, read-only log of these changes.

Operations on the container level

For these operations, **accountName** and **containerName** are **required**.

Operation	Description
createBlobContainer	Creates a new container within a storage account. If a container with the same name already exists, the producer will ignore it.
deleteBlobContainer	Deletes the specified container in the storage account. If the container doesn't exist the operation fails.
listBlobs	Returns a list of blobs in this container, with folder structures flattened.

Operations on the blob level

For these operations, **accountName**, **containerName** and **blobName** are required.

Operation	Blob Type	Description
getBlob	Common	Get the content of the blob. You can restrict the output of this operation to a blob range.
deleteBlob	Common	Delete a blob.
downloadBlobToFile	Common	Downloads the entire blob into a file specified by the path. The file will be created and must not exist, if the file already exists a <code>{@link FileAlreadyExistsException}</code> will be thrown.
downloadLink	Common	Generates the download link for the specified blob using shared access signatures (SAS). This by default only limit to 1hour of allowed access. However, you can override the default expiration duration through the headers.
uploadBlockBlob	BlockBlob	Creates a new block blob, or updates the content of an existing block blob. Updating an existing block blob overwrites any existing metadata on the blob. Partial updates are not supported with <code>PutBlob</code> ; the content of the existing blob is overwritten with the new content.
stageBlockBlobList	BlockBlob	Uploads the specified block to the block blob's "staging area" to be later committed by a call to <code>commitBlobBlockList</code> . However in case header <code>CamelAzureStorageBlobCommitBlobBlockListLater</code> or config <code>commitBlockListLater</code> is set to false, this will commit the blocks immediately after staging the blocks.
commitBlobBlockList	BlockBlob	Writes a blob by specifying the list of block IDs that are to make up the blob. In order to be written as part of a blob, a block must have been successfully written to the server in a prior <code>stageBlockBlobList</code> operation. You can call <code>commitBlobBlockList</code> to update a blob by uploading only those blocks that have changed, then committing the new and existing blocks together. Any blocks not specified in the block list and permanently deleted.
getBlobBlockList	BlockBlob	Returns the list of blocks that have been uploaded as part of a block blob using the specified block list filter.

Operation	Blob Type	Description
createAppendBlob	AppendBlob	Creates a 0-length append blob. Call <code>commitAppendBlob</code> operation to append data to an append blob.
commitAppendBlob	AppendBlob	Commits a new block of data to the end of the existing append blob. In case of header CamelAzureStorageBlobCreateAppendBlob or config createAppendBlob is set to true, it will attempt to create the appendBlob through internal call to createAppendBlob operation first before committing.
createPageBlob	PageBlob	Creates a page blob of the specified length. Call uploadPageBlob operation to upload data data to a page blob.
uploadPageBlob	PageBlob	Writes one or more pages to the page blob. The write size must be a multiple of 512. In case of header CamelAzureStorageBlobCreatePageBlob or config createPageBlob is set to true, it will attempt to create the appendBlob through internal call to createPageBlob operation first before uploading.
resizePageBlob	PageBlob	Resizes the page blob to the specified size (which must be a multiple of 512).
clearPageBlob	PageBlob	Frees the specified pages from the page blob. The size of the range must be a multiple of 512.
getPageBlobRanges	PageBlob	Returns the list of valid page ranges for a page blob or snapshot of a page blob.
copyBlob	Common	Copy a blob from one container to another one, even from different accounts.

Refer to the example section in this page to learn how to use these operations into your camel application.

15.6.6. Consumer Examples

To consume a blob into a file using file component, this can be done like this:

```
from("azure-storage-blob://camelazure/container1?
blobName=hello.txt&accountName=yourAccountName&accessKey=yourAccessKey")
to("file://blobdirectory");
```

However, you can also write to file directly without using the file component, you will need to specify **fileDir** folder path in order to save your blob in your machine.

```
from("azure-storage-blob://camelazure/container1?
blobName=hello.txt&accountName=yourAccountName&accessKey=yourAccessKey&fileDir=/var/to/awesome/dir").
to("mock:results");
```

Also, the component supports batch consumer, hence you can consume multiple blobs with only specifying the container name, the consumer will return multiple exchanges depending on the number of the blobs in the container.

Example

```
from("azure-storage-blob://camelazure/container1?
accountName=yourAccountName&accessKey=yourAccessKey&fileDir=/var/to/awesome/dir").
to("mock:results");
```

15.6.7. Producer Operations Examples

- **listBlobContainers**

```
from("direct:start")
.process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setHeader(BlobConstants.LIST_BLOB_CONTAINERS_OPTIONS, new
ListBlobContainersOptions().setMaxResultsPerPage(10));
})
.to("azure-storage-blob://camelazure?operation=listBlobContainers&client&serviceClient=#client")
.to("mock:result");
```

- **createBlobContainer**

```
from("direct:start")
.process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setHeader(BlobConstants.BLOB_CONTAINER_NAME, "newContainerName");
})
.to("azure-storage-blob://camelazure/container1?
operation=createBlobContainer&serviceClient=#client")
.to("mock:result");
```

- **deleteBlobContainer:**

```
from("direct:start")
.process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setHeader(BlobConstants.BLOB_CONTAINER_NAME, "overridenName");
```

```

    })
    .to("azure-storage-blob://camelazure/container1?
operation=deleteBlobContainer&serviceClient=#client")
    .to("mock:result");

```

- **listBlobs:**

```

from("direct:start")
    .process(exchange -> {
        // set the header you want the producer to evaluate, refer to the previous
        // section to learn about the headers that can be set
        // e.g:
        exchange.getIn().setHeader(BlobConstants.BLOB_CONTAINER_NAME, "overridenName");
    })
    .to("azure-storage-blob://camelazure/container1?operation=listBlobs&serviceClient=#client")
    .to("mock:result");

```

- **getBlob:**

We can either set an **outputStream** in the exchange body and write the data to it. E.g:

```

from("direct:start")
    .process(exchange -> {
        // set the header you want the producer to evaluate, refer to the previous
        // section to learn about the headers that can be set
        // e.g:
        exchange.getIn().setHeader(BlobConstants.BLOB_CONTAINER_NAME, "overridenName");

        // set our body
        exchange.getIn().setBody(outputStream);
    })
    .to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=getBlob&serviceClient=#client")
    .to("mock:result");

```

If we don't set a body, then this operation will give us an **InputStream** instance which can proceed further downstream:

```

from("direct:start")
    .to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=getBlob&serviceClient=#client")
    .process(exchange -> {
        InputStream inputStream = exchange.getMessage().getBody(InputStream.class);
        // We use Apache common IO for simplicity, but you are free to do whatever dealing
        // with inputStream
        System.out.println(IUtils.toString(inputStream, StandardCharsets.UTF_8.name()));
    })
    .to("mock:result");

```

- **deleteBlob:**

```

from("direct:start")
    .process(exchange -> {
        // set the header you want the producer to evaluate, refer to the previous

```

```
// section to learn about the headers that can be set
// e.g:
exchange.getIn().setHeader(BlobConstants.BLOB_NAME, "overridenName");
})
.to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=deleteBlob&serviceClient=#client")
.to("mock:result");
```

- **downloadBlobToFile:**

```
from("direct:start")
.process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setHeader(BlobConstants.BLOB_NAME, "overridenName");
})
.to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=downloadBlobToFile&fileDir=/var/mydir&serviceClient=#client")
.to("mock:result");
```

- **downloadLink**

```
from("direct:start")
.to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=downloadLink&serviceClient=#client")
.process(exchange -> {
    String link = exchange.getMessage().getHeader(BlobConstants.DOWNLOAD_LINK,
String.class);
    System.out.println("My link " + link);
})
.to("mock:result");
```

- **uploadBlockBlob**

```
from("direct:start")
.process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setHeader(BlobConstants.BLOB_NAME, "overridenName");
    exchange.getIn().setBody("Block Blob");
})
.to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=uploadBlockBlob&serviceClient=#client")
.to("mock:result");
```

- **stageBlockBlobList**

```
from("direct:start")
.process(exchange -> {
    final List<BlobBlock> blocks = new LinkedList<>();
    blocks.add(BlobBlock.createBlobBlock(new ByteArrayInputStream("Hello".getBytes())));
    blocks.add(BlobBlock.createBlobBlock(new ByteArrayInputStream("From".getBytes())));
```

```

        blocks.add(BlobBlock.createBlobBlock(new ByteArrayInputStream("Camel".getBytes())));

        exchange.getIn().setBody(blocks);
    })
    .to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=stageBlockBlobList&serviceClient=#client")
    .to("mock:result");

```

- **commitBlockBlobList**

```

from("direct:start")
    .process(exchange -> {
        // We assume here you have the knowledge of these blocks you want to commit
        final List<Block> blocksIds = new LinkedList<>();
        blocksIds.add(new Block().setName("id-1"));
        blocksIds.add(new Block().setName("id-2"));
        blocksIds.add(new Block().setName("id-3"));

        exchange.getIn().setBody(blocksIds);
    })
    .to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=commitBlockBlobList&serviceClient=#client")
    .to("mock:result");

```

- **getBlobBlockList**

```

from("direct:start")
    .to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=getBlobBlockList&serviceClient=#client")
    .log("${body}")
    .to("mock:result");

```

- **createAppendBlob**

```

from("direct:start")
    .to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=createAppendBlob&serviceClient=#client")
    .to("mock:result");

```

- **commitAppendBlob**

```

from("direct:start")
    .process(exchange -> {
        final String data = "Hello world from my awesome tests!";
        final InputStream dataStream = new
ByteArrayInputStream(data.getBytes(StandardCharsets.UTF_8));

        exchange.getIn().setBody(dataStream);

        // of course you can set whatever headers you like, refer to the headers section to learn more
    })
    .to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=commitAppendBlob&serviceClient=#client")
    .to("mock:result");

```

- **createPageBlob**

```
from("direct:start")
.to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=createPageBlob&serviceClient=#client")
.to("mock:result");
```

- **uploadPageBlob**

```
from("direct:start")
.process(exchange -> {
    byte[] dataBytes = new byte[512]; // we set range for the page from 0-511
    new Random().nextBytes(dataBytes);
    final InputStream dataStream = new ByteArrayInputStream(dataBytes);
    final PageRange pageRange = new PageRange().setStart(0).setEnd(511);

    exchange.getIn().setHeader(BlobConstants.PAGE_BLOB_RANGE, pageRange);
    exchange.getIn().setBody(dataStream);
})
.to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=uploadPageBlob&serviceClient=#client")
.to("mock:result");
```

- **resizePageBlob**

```
from("direct:start")
.process(exchange -> {
    final PageRange pageRange = new PageRange().setStart(0).setEnd(511);

    exchange.getIn().setHeader(BlobConstants.PAGE_BLOB_RANGE, pageRange);
})
.to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=resizePageBlob&serviceClient=#client")
.to("mock:result");
```

- **clearPageBlob**

```
from("direct:start")
.process(exchange -> {
    final PageRange pageRange = new PageRange().setStart(0).setEnd(511);

    exchange.getIn().setHeader(BlobConstants.PAGE_BLOB_RANGE, pageRange);
})
.to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=clearPageBlob&serviceClient=#client")
.to("mock:result");
```

- **getPageBlobRanges**

```
from("direct:start")
.process(exchange -> {
    final PageRange pageRange = new PageRange().setStart(0).setEnd(511);

    exchange.getIn().setHeader(BlobConstants.PAGE_BLOB_RANGE, pageRange);
```

```

    })
    .to("azure-storage-blob://camelazure/container1?
blobName=blob&operation=getPageBlobRanges&serviceClient=#client")
    .log("${body}")
    .to("mock:result");

```

- **copyBlob**

```

from("direct:copyBlob")
    .process(exchange -> {
        exchange.getIn().setHeader(BlobConstants.BLOB_NAME, "file.txt");
        exchange.getMessage().setHeader(BlobConstants.SOURCE_BLOB_CONTAINER_NAME,
"containerblob1");
        exchange.getMessage().setHeader(BlobConstants.SOURCE_BLOB_ACCOUNT_NAME,
"account");
    })
    .to("azure-storage-blob://account/containerblob2?
operation=copyBlob&sourceBlobAccessKey=RAW(accessKey)")
    .to("mock:result");

```

In this way the file.txt in the container containerblob1 of the account 'account', will be copied to the container containerblob2 of the same account.

15.6.8. SAS Token generation example

SAS Blob Container tokens can be generated programmatically or via Azure UI. To generate the token with java code, the following can be done:

```

BlobContainerClient blobClient = new BlobContainerClientBuilder()
    .endpoint(String.format("https://%s.blob.core.windows.net/%s", accountName, accessKey))
    .containerName(containerName)
    .credential(new StorageSharedKeyCredential(accountName, accessKey))
    .buildClient();

// Create a SAS token that's valid for 1 day, as an example
OffsetDateTime expiryTime = OffsetDateTime.now().plusDays(1);

// Assign permissions to the SAS token
BlobContainerSasPermission blobContainerSasPermission = new
BlobContainerSasPermission()
    .setWritePermission(true)
    .setListPermission(true)
    .setCreatePermission(true)
    .setDeletePermission(true)
    .setAddPermission(true)
    .setReadPermission(true);

BlobServiceSasSignatureValues sasSignatureValues = new
BlobServiceSasSignatureValues(expiryTime, blobContainerSasPermission);

return blobClient.generateSas(sasSignatureValues);

```

The generated SAS token can be then stored to an **application.properties** file so that it can be loaded by the camel route, for example:


```
camel.component.azure-storage-blob.sas-token=MY_TOKEN_HERE

from("direct:copyBlob")
.to("azure-storage-blob://account/containerblob2?
operation=uploadBlockBlob&credentialType=AZURE_SAS")
```

15.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 36 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.azure-storage-blob.access-key</code>	Access key for the associated azure account name to be used for authentication with azure blob services.		String
<code>camel.component.azure-storage-blob.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.azure-storage-blob.blob-name</code>	The blob name, to consume specific blob from a container. However on producer, is only required for the operations on the blob level.		String
<code>camel.component.azure-storage-blob.blob-offset</code>	Set the blob offset for the upload or download operations, default is 0.	0	Long
<code>camel.component.azure-storage-blob.blob-sequence-number</code>	A user-controlled value that you can use to track requests. The value of the sequence number must be between 0 and 263 - 1. The default value is 0.	0	Long
<code>camel.component.azure-storage-blob.blob-type</code>	The blob type in order to initiate the appropriate settings for each blob type.		BlobType
<code>camel.component.azure-storage-blob.block-list-type</code>	Specifies which type of blocks to return.		BlockListType

Name	Description	Default	Type
camel.component.azure-storage-blob.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.azure-storage-blob.change-feed-context	When using <code>getChangeFeed</code> producer operation, this gives additional context that is passed through the Http pipeline during the service call. The option is a <code>com.azure.core.util.Context</code> type.		Context
camel.component.azure-storage-blob.change-feed-end-time	When using <code>getChangeFeed</code> producer operation, this filters the results to return events approximately before the end time. Note: A few events belonging to the next hour can also be returned. A few events belonging to this hour can be missing; to ensure all events from the hour are returned, round the end time up by an hour. The option is a <code>java.time.OffsetDateTime</code> type.		OffsetDateTime
camel.component.azure-storage-blob.change-feed-start-time	When using <code>getChangeFeed</code> producer operation, this filters the results to return events approximately after the start time. Note: A few events belonging to the previous hour can also be returned. A few events belonging to this hour can be missing; to ensure all events from the hour are returned, round the start time down by an hour. The option is a <code>java.time.OffsetDateTime</code> type.		OffsetDateTime
camel.component.azure-storage-blob.close-stream-after-read	Close the stream after read or keep it open, default is true.	true	Boolean
camel.component.azure-storage-blob.close-stream-after-write	Close the stream after write or keep it open, default is true.	true	Boolean

Name	Description	Default	Type
<code>camel.component.azure-storage-blob.commit-block-list-later</code>	When is set to true, the staged blocks will not be committed directly.	true	Boolean
<code>camel.component.azure-storage-blob.configuration</code>	The component configurations. The option is a <code>org.apache.camel.component.azure.storage.blob.BlobConfiguration</code> type.		BlobConfiguration
<code>camel.component.azure-storage-blob.create-append-blob</code>	When is set to true, the append blocks will be created when committing append blocks.	true	Boolean
<code>camel.component.azure-storage-blob.create-page-blob</code>	When is set to true, the page blob will be created when uploading page blob.	true	Boolean
<code>camel.component.azure-storage-blob.credential-type</code>	Determines the credential strategy to adopt.		CredentialType
<code>camel.component.azure-storage-blob.credentials</code>	<code>StorageSharedKeyCredential</code> can be injected to create the azure client, this holds the important authentication information. The option is a <code>com.azure.storage.common.StorageSharedKeyCredential</code> type.		StorageSharedKeyCredential
<code>camel.component.azure-storage-blob.data-count</code>	How many bytes to include in the range. Must be greater than or equal to 0 if specified.		Long
<code>camel.component.azure-storage-blob.download-link-expiration</code>	Override the default expiration (millis) of URL download link.		Long
<code>camel.component.azure-storage-blob.enabled</code>	Whether to enable auto configuration of the <code>azure-storage-blob</code> component. This is enabled by default.		Boolean
<code>camel.component.azure-storage-blob.file-dir</code>	The file directory where the downloaded blobs will be saved to, this can be used in both, producer and consumer.		String

Name	Description	Default	Type
<code>camel.component.azure-storage-blob.health-check-consumer-enabled</code>	Used for enabling or disabling all consumer based health checks from this component.	true	Boolean
<code>camel.component.azure-storage-blob.health-check-producer-enabled</code>	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting <code>camel.health.producersEnabled=true</code> .	true	Boolean
<code>camel.component.azure-storage-blob.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.azure-storage-blob.max-results-per-page</code>	Specifies the maximum number of blobs to return, including all BlobPrefix elements. If the request does not specify <code>maxResultsPerPage</code> or specifies a value greater than 5,000, the server will return up to 5,000 items.		Integer
<code>camel.component.azure-storage-blob.max-retry-requests</code>	Specifies the maximum number of additional HTTP Get requests that will be made while reading the data from a response body.	0	Integer
<code>camel.component.azure-storage-blob.operation</code>	The blob operation that can be used with this component on the producer.		BlobOperationsDefinition
<code>camel.component.azure-storage-blob.page-blob-size</code>	Specifies the maximum size for the page blob, up to 8 TB. The page blob size must be aligned to a 512-byte boundary.	512	Long
<code>camel.component.azure-storage-blob.prefix</code>	Filters the results to return only blobs whose names begin with the specified prefix. May be null to return all blobs.		String

Name	Description	Default	Type
camel.component.azure-storage-blob.regex	Filters the results to return only blobs whose names match the specified regular expression. May be null to return all if both prefix and regex are set, regex takes the priority and prefix is ignored.		String
camel.component.azure-storage-blob.sas-token	Set a SAS Token in case of usage of Shared Access Signature		String
camel.component.azure-storage-blob.service-client	Client to a storage account. This client does not hold any state about a particular storage account but is instead a convenient way of sending off appropriate requests to the resource on the service. It may also be used to construct URLs to blobs and containers. This client contains operations on a service account. Operations on a container are available on BlobContainerClient through BlobServiceClient#getBlobContainerClient(String), and operations on a blob are available on BlobClient through BlobContainerClient#getBlobClient(String). The option is a com.azure.storage.blob.BlobServiceClient type.		BlobServiceClient
camel.component.azure-storage-blob.source-blob-access-key	Source Blob Access Key: for copyblob operation, sadly, we need to have an accessKey for the source blob we want to copy Passing an accessKey as header, it's unsafe so we could set as key.		String
camel.component.azure-storage-blob.timeout	An optional timeout value beyond which a RuntimeException will be raised. The option is a java.time.Duration type.		Duration

CHAPTER 16. AZURE STORAGE QUEUE SERVICE

Both producer and consumer are supported

The Azure Storage Queue component supports storing and retrieving the messages to/from [Azure Storage Queue](#) service using **Azure APIs v12**. However in case of versions above v12, we will see if this component can adopt these changes depending on how much breaking changes can result.

Prerequisites

You must have a valid Windows Azure Storage account. More information is available at [Azure Documentation Portal](#).

16.1. DEPENDENCIES

When using **azure-storage-queue** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-azure-storage-queue-starter</artifactId>
</dependency>
```

16.2. URI FORMAT

```
azure-storage-queue://accountName[/queueName][?options]
```

In case of consumer, `accountName` and `queueName` are required. In case of producer, it depends on the operation that being requested, for example if operation is on a service level, e.b: `listQueues`, only `accountName` is required, but in case of operation being requested on the queue level, for example, `createQueue`, `sendMessage`.. etc, both `accountName` and `queueName` are required.

The queue will be created if it does not already exist. You can append query options to the URI in the following format,

?options=value&option2=value&...

16.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

16.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

16.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

16.4. COMPONENT OPTIONS

The Azure Storage Queue Service component supports 15 options, which are listed below.

Name	Description	Default	Type
configuration (common)	The component configurations.		QueueConfigurati on
serviceClient (common)	Autowired Service client to a storage account to interact with the queue service. This client does not hold any state about a particular storage account but is instead a convenient way of sending off appropriate requests to the resource on the service. This client contains all the operations for interacting with a queue account in Azure Storage. Operations allowed by the client are creating, listing, and deleting queues, retrieving and updating properties of the account, and retrieving statistics of the account.		QueueServiceClie nt

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
createQueue (producer)	When is set to true, the queue will be automatically created when sending messages to the queue.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
operation (producer)	Queue service operation hint to the producer. Enum values: • <code>listQueues</code> • <code>createQueue</code> • <code>deleteQueue</code> • <code>clearQueue</code> • <code>sendMessage</code> • <code>deleteMessage</code> • <code>receiveMessages</code> • <code>peekMessages</code> • <code>updateMessage</code>		QueueOperationDefinition

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
maxMessages (queue)	Maximum number of messages to get, if there are less messages exist in the queue than requested all the messages will be returned. If left empty only 1 message will be retrieved, the allowed range is 1 to 32 messages.	1	Integer
messageId (queue)	The ID of the message to be deleted or updated.		String
popReceipt (queue)	Unique identifier that must match for the message to be deleted or updated.		String
timeout (queue)	An optional timeout applied to the operation. If a response is not returned before the timeout concludes a RuntimeException will be thrown.		Duration
timeToLive (queue)	How long the message will stay alive in the queue. If unset the value will default to 7 days, if -1 is passed the message will not expire. The time to live must be -1 or any positive number. The format should be in this form: PnDTnHnMn.nS., e.g: PT20.345S – parses as 20.345 seconds, P2D – parses as 2 days However, in case you are using EndpointDsl/ComponentDsl, you can do something like Duration.ofSeconds() since these Java APIs are typesafe.		Duration
visibilityTimeout (queue)	The timeout period for how long the message is invisible in the queue. The timeout must be between 1 seconds and 7 days. The format should be in this form: PnDTnHnMn.nS., e.g: PT20.345S – parses as 20.345 seconds, P2D – parses as 2 days However, in case you are using EndpointDsl/ComponentDsl, you can do something like Duration.ofSeconds() since these Java APIs are typesafe.		Duration
accessKey (security)	Access key for the associated azure account name to be used for authentication with azure queue services.		String
credentials (security)	StorageSharedKeyCredential can be injected to create the azure client, this holds the important authentication information.		StorageSharedKey Credential

16.5. ENDPOINT OPTIONS

The Azure Storage Queue Service endpoint is configured using URI syntax:

```
azure-storage-queue:accountName/queueName
```

with the following path and query parameters:

16.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
accountName (common)	Azure account name to be used for authentication with azure queue services.		String
queueName (common)	The queue resource name.		String

16.5.2. Query Parameters (31 parameters)

Name	Description	Default	Type
serviceClient (common)	Autowired Service client to a storage account to interact with the queue service. This client does not hold any state about a particular storage account but is instead a convenient way of sending off appropriate requests to the resource on the service. This client contains all the operations for interacting with a queue account in Azure Storage. Operations allowed by the client are creating, listing, and deleting queues, retrieving and updating properties of the account, and retrieving statistics of the account.		QueueServiceClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an <code>Exchange</code> have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
createQueue (producer)	When is set to true, the queue will be automatically created when sending messages to the queue.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
operation (producer)	Queue service operation hint to the producer. Enum values: • listQueues • createQueue • deleteQueue • clearQueue • sendMessage • deleteMessage • receiveMessages • peekMessages • updateMessage		QueueOperationDefinition
maxMessages (queue)	Maximum number of messages to get, if there are less messages exist in the queue than requested all the messages will be returned. If left empty only 1 message will be retrieved, the allowed range is 1 to 32 messages.	1	Integer
messageId (queue)	The ID of the message to be deleted or updated.		String
popReceipt (queue)	Unique identifier that must match for the message to be deleted or updated.		String
timeout (queue)	An optional timeout applied to the operation. If a response is not returned before the timeout concludes a RuntimeException will be thrown.		Duration
timeToLive (queue)	How long the message will stay alive in the queue. If unset the value will default to 7 days, if -1 is passed the message will not expire. The time to live must be -1 or any positive number. The format should be in this form: PnDTnHnMn.nS., e.g: PT20.345S – parses as 20.345 seconds, P2D – parses as 2 days However, in case you are using EndpointDsl/ComponentDsl, you can do something like Duration.ofSeconds() since these Java APIs are typesafe.		Duration

Name	Description	Default	Type
visibilityTimeout (queue)	The timeout period for how long the message is invisible in the queue. The timeout must be between 1 seconds and 7 days. The format should be in this form: PnDTnHnMn.nS, e.g: PT20.345S – parses as 20.345 seconds, P2D – parses as 2 days However, in case you are using EndpointDsl/ComponentDsl, you can do something like Duration.ofSeconds() since these Java APIs are typesafe.		Duration
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long

Name	Description	Default	Type
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit

Name	Description	Default	Type
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See <code>ScheduledExecutorService</code> in JDK for details.	true	boolean
accessKey (security)	Access key for the associated azure account name to be used for authentication with azure queue services.		String
credentials (security)	<code>StorageSharedKeyCredential</code> can be injected to create the azure client, this holds the important authentication information.		<code>StorageSharedKeyCredential</code>

Required information options

To use this component, you have 3 options in order to provide the required Azure authentication information:

- Provide **accountName** and **accessKey** for your Azure account, this is the simplest way to get started. The accessKey can be generated through your Azure portal.
- Provide a [StorageSharedKeyCredential](#) instance which can be provided into **credentials** option.
- Provide a [QueueServiceClient](#) instance which can be provided into **serviceClient**. Note: You don't need to create a specific client, e.g: `QueueClient`, the `QueueServiceClient` represents the upper level which can be used to retrieve lower level clients.

16.6. USAGE

For example in order to get a message content from the queue **messageQueue** in the **storageAccount** storage account and, use the following snippet:

```
from("azure-storage-queue://storageAccount/messageQueue?accessKey=yourAccessKey").
to("file://queuedirectory");
```

16.6.1. Message headers evaluated by the component producer

Header	Variable Name	Type	Operations	Description
CamelAzureStorageQueueSegmentOptions	QueueConstants.QUEUES_SEGMENT_OPTIONS	QueuesSegmentOptions	listQueues	Options for listing queues
CamelAzureStorageQueueTimeout	QueueConstants.TIMEOUT	Duration	All	An optional timeout value beyond which a <code>\{@link RuntimeException}</code> will be raised.

Header	Variable Name	Type	Operations	Description
CamelAzureStorageQueueMetadata	QueueConstants.METADATA	Map<String,String>	createQueue	Metadata to associate with the queue
CamelAzureStorageQueueTimeToLive	QueueConstants.TIME_TO_LIVE	Duration	sendMessage	How long the message will stay alive in the queue. If unset the value will default to 7 days, if -1 is passed the message will not expire. The time to live must be -1 or any positive number.
CamelAzureStorageQueueVisibilityTimeout	QueueConstants.VISIBILITY_TIMEOUT	Duration	sendMessage, receiveMessages, updateMessage	The timeout period for how long the message is invisible in the queue. If unset the value will default to 0 and the message will be instantly visible. The timeout must be between 0 seconds and 7 days.
CamelAzureStorageQueueCreateQueue	QueueConstants.CREATE_QUEUE	boolean	sendMessage	When is set to true , the queue will be automatically created when sending messages to the queue.
CamelAzureStorageQueuePopReceipt	QueueConstants.POP_RECEIPT	String	deleteMessage, updateMessage	Unique identifier that must match for the message to be deleted or updated.
CamelAzureStorageQueueMessageId	QueueConstants.MESSAGE_ID	String	deleteMessage, updateMessage	The ID of the message to be deleted or updated.

Header	Variable Name	Type	Operations	Description
CamelAzureStorageQueueMaxMessages	QueueConstants.MAX_MESSAGES	Integer	receiveMessages, peekMessages	Maximum number of messages to get, if there are less messages exist in the queue than requested all the messages will be returned. If left empty only 1 message will be retrieved, the allowed range is 1 to 32 messages.
CamelAzureStorageQueueOperation	QueueConstants.QUEUE_OPERATION	QueueOperationDefinition	All	Specify the producer operation to execute, please see the doc on this page related to producer operation.
CamelAzureStorageQueueName	QueueConstants.QUEUE_NAME	String	All	Override the queue name.

16.6.2. Message headers set by either component producer or consumer

Header	Variable Name	Type	Description
CamelAzureStorageQueueMessageId	QueueConstants.MESSAGE_ID	String	The ID of message that being sent to the queue.
CamelAzureStorageQueueInsertionTime	QueueConstants.INSERTION_TIME	OffsetDateTime	The time the Message was inserted into the Queue.
CamelAzureStorageQueueExpirationTime	QueueConstants.EXPIRATION_TIME	OffsetDateTime	The time that the Message will expire and be automatically deleted.
CamelAzureStorageQueuePopReceipt	QueueConstants.POP_RECEIPT	String	This value is required to delete/update the Message. If deletion fails using this popreceipt then the message has been dequeued by another client.

Header	Variable Name	Type	Description
CamelAzureStorageQueueTimeNextVisible	QueueConstants.TIME_NEXT_VISIBLE	OffsetDateTime	The time that the message will again become visible in the Queue.
CamelAzureStorageQueueDequeueCount	QueueConstants.DEQUEUE_COUNT	long	The number of times the message has been dequeued.
CamelAzureStorageQueueRawHttpHeaders	QueueConstants.RAW_HTTP_HEADERS	HttpHeaders	Returns non-parsed httpHeaders that can be used by the user.

16.6.3. Advanced Azure Storage Queue configuration

If your Camel Application is running behind a firewall or if you need to have more control over the **QueueServiceClient** instance configuration, you can create your own instance:

```
StorageSharedKeyCredential credential = new StorageSharedKeyCredential("yourAccountName",
"yourAccessKey");
String uri = String.format("https://%s.queue.core.windows.net", "yourAccountName");

QueueServiceClient client = new QueueServiceClientBuilder()
    .endpoint(uri)
    .credential(credential)
    .buildClient();
// This is camel context
context.getRegistry().bind("client", client);
```

Then refer to this instance in your Camel **azure-storage-queue** component configuration:

```
from("azure-storage-queue://cameldev/queue1?serviceClient=#client")
.to("file://outputFolder?fileName=output.txt&fileExist=Append");
```

16.6.4. Automatic detection of QueueServiceClient client in registry

The component is capable of detecting the presence of an QueueServiceClient bean into the registry. If it's the only instance of that type it will be used as client and you won't have to define it as uri parameter, like the example above. This may be really useful for smarter configuration of the endpoint.

16.6.5. Azure Storage Queue Producer operations

Camel Azure Storage Queue component provides wide range of operations on the producer side:

Operations on the service level

For these operations, **accountName** is required.

Operation	Description
listQueues	Lists the queues in the storage account that pass the filter starting at the specified marker.

Operations on the queue level

For these operations, **accountName** and **queueName** are required.

Operation	Description
createQueue	Creates a new queue.
deleteQueue	Permanently deletes the queue.
clearQueue	Deletes all messages in the queue..
sendMessage	Default Producer Operation Sends a message with a given time-to-live and a timeout period where the message is invisible in the queue. The message text is evaluated from the exchange message body. By default, if the queue doesn't exist, it will create an empty queue first. If you want to disable this, set the config createQueue or header CamelAzureStorageQueueCreateQueue to false .
deleteMessage	Deletes the specified message in the queue.
receiveMessages	Retrieves up to the maximum number of messages from the queue and hides them from other operations for the timeout period. However it will not dequeue the message from the queue due to reliability reasons.
peekMessages	Peek messages from the front of the queue up to the maximum number of messages.
updateMessage	Updates the specific message in the queue with a new message and resets the visibility timeout. The message text is evaluated from the exchange message body.

Refer to the example section in this page to learn how to use these operations into your camel application.

16.6.6. Consumer Examples

To consume a queue into a file component with maximum 5 messages in one batch, this can be done like this:

```
from("azure-storage-queue://cameldev/queue1?serviceClient=#client&maxMessages=5")
.to("file://outputFolder?fileName=output.txt&fileExist=Append");
```

16.6.7. Producer Operations Examples

- **listQueues:**

```
from("direct:start")
  .process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g, to only returns list of queues with 'awesome' prefix:
    exchange.getIn().setHeader(QueueConstants.QUEUES_SEGMENT_OPTIONS, new
QueuesSegmentOptions().setPrefix("awesome"));
  })
  .to("azure-storage-queue://cameldev?serviceClient=#client&operation=listQueues")
  .log("${body}")
  .to("mock:result");
```

- **createQueue:**

```
from("direct:start")
  .process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setHeader(QueueConstants.QUEUE_NAME, "overrideName");
  })
  .to("azure-storage-queue://cameldev/test?serviceClient=#client&operation=createQueue");
```

- **deleteQueue:**

```
from("direct:start")
  .process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setHeader(QueueConstants.QUEUE_NAME, "overrideName");
  })
  .to("azure-storage-queue://cameldev/test?serviceClient=#client&operation=deleteQueue");
```

- **clearQueue:**

```
from("direct:start")
  .process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setHeader(QueueConstants.QUEUE_NAME, "overrideName");
  })
  .to("azure-storage-queue://cameldev/test?serviceClient=#client&operation=clearQueue");
```

- **sendMessage:**

```
from("direct:start")
  .process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
```

```

// section to learn about the headers that can be set
// e.g:
exchange.getIn().setBody("message to send");
// we set a visibility of 1 min
exchange.getIn().setHeader(QueueConstants.VISIBILITY_TIMEOUT, Duration.ofMinutes(1));
})
.to("azure-storage-queue://cameldev/test?serviceClient=#client");

```

- **deleteMessage:**

```

from("direct:start")
.process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    // Mandatory header:
    exchange.getIn().setHeader(QueueConstants.MESSAGE_ID, "1");
    // Mandatory header:
    exchange.getIn().setHeader(QueueConstants.POP_RECEIPT, "PAAAAHEEERXXX-1");
})
.to("azure-storage-queue://cameldev/test?serviceClient=#client&operation=deleteMessage");

```

- **receiveMessages:**

```

from("direct:start")
.to("azure-storage-queue://cameldev/test?serviceClient=#client&operation=receiveMessages")
.process(exchange -> {
    final List<QueueMessageItem> messageItems = exchange.getMessage().getBody(List.class);
    messageItems.forEach(messageItem -> System.out.println(messageItem.getMessageText()));
})
.to("mock:result");

```

- **peekMessages:**

```

from("direct:start")
.to("azure-storage-queue://cameldev/test?serviceClient=#client&operation=peekMessages")
.process(exchange -> {
    final List<PeekedMessageItem> messageItems = exchange.getMessage().getBody(List.class);
    messageItems.forEach(messageItem -> System.out.println(messageItem.getMessageText()));
})
.to("mock:result");

```

- **updateMessage:**

```

from("direct:start")
.process(exchange -> {
    // set the header you want the producer to evaluate, refer to the previous
    // section to learn about the headers that can be set
    // e.g:
    exchange.getIn().setBody("new message text");
    // Mandatory header:
    exchange.getIn().setHeader(QueueConstants.MESSAGE_ID, "1");
    // Mandatory header:
    exchange.getIn().setHeader(QueueConstants.POP_RECEIPT, "PAAAAHEEERXXX-1");
})

```

```
// Mandatory header:
exchange.getIn().setHeader(QueueConstants.VISIBILITY_TIMEOUT, Duration.ofMinutes(1));
})
.to("azure-storage-queue://cameldev/test?serviceClient=#client&operation=updateMessage");
```

16.6.8. Development Notes (Important)

When developing on this component, you will need to obtain your Azure accessKey in order to run the integration tests. In addition to the mocked unit tests you **will need to run the integration tests with every change you make or even client upgrade as the Azure client can break things even on minor versions upgrade**. To run the integration tests, on this component directory, run the following maven command:

```
mvn verify -PfullTests -DaccountName=myacc -DaccessKey=mykey
```

Whereby **accountName** is your Azure account name and **accessKey** is the access key being generated from Azure portal.

16.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 16 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.azure-storage-queue.access-key</code>	Access key for the associated azure account name to be used for authentication with azure queue services.		String
<code>camel.component.azure-storage-queue.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.azure-storage-queue.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.azure-storage-queue.configuration</code>	The component configurations. The option is a <code>org.apache.camel.component.azure.storage.queue.QueueConfiguration</code> type.		QueueConfiguration

Name	Description	Default	Type
<code>camel.component.azure-storage-queue.create-queue</code>	When is set to true, the queue will be automatically created when sending messages to the queue.	false	Boolean
<code>camel.component.azure-storage-queue.credentials</code>	StorageSharedKeyCredential can be injected to create the azure client, this holds the important authentication information. The option is a <code>com.azure.storage.common.StorageSharedKeyCredential</code> type.		StorageSharedKeyCredential
<code>camel.component.azure-storage-queue.enabled</code>	Whether to enable auto configuration of the azure-storage-queue component. This is enabled by default.		Boolean
<code>camel.component.azure-storage-queue.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.azure-storage-queue.max-messages</code>	Maximum number of messages to get, if there are less messages exist in the queue than requested all the messages will be returned. If left empty only 1 message will be retrieved, the allowed range is 1 to 32 messages.	1	Integer
<code>camel.component.azure-storage-queue.message-id</code>	The ID of the message to be deleted or updated.		String
<code>camel.component.azure-storage-queue.operation</code>	Queue service operation hint to the producer.		QueueOperationDefinition
<code>camel.component.azure-storage-queue.pop-receipt</code>	Unique identifier that must match for the message to be deleted or updated.		String

Name	Description	Default	Type
camel.component.azure-storage-queue.service-client	Service client to a storage account to interact with the queue service. This client does not hold any state about a particular storage account but is instead a convenient way of sending off appropriate requests to the resource on the service. This client contains all the operations for interacting with a queue account in Azure Storage. Operations allowed by the client are creating, listing, and deleting queues, retrieving and updating properties of the account, and retrieving statistics of the account. The option is a <code>com.azure.storage.queue.QueueServiceClient</code> type.		QueueServiceClient
camel.component.azure-storage-queue.time-to-live	How long the message will stay alive in the queue. If unset the value will default to 7 days, if -1 is passed the message will not expire. The time to live must be -1 or any positive number. The format should be in this form: <code>PnDTnHnMn.nS</code> , e.g: <code>PT20.345S</code> – parses as 20.345 seconds, <code>P2D</code> – parses as 2 days However, in case you are using <code>EndpointDsl/ComponentDsl</code> , you can do something like <code>Duration.ofSeconds()</code> since these Java APIs are typesafe. The option is a <code>java.time.Duration</code> type.		Duration
camel.component.azure-storage-queue.timeout	An optional timeout applied to the operation. If a response is not returned before the timeout concludes a <code>RuntimeException</code> will be thrown. The option is a <code>java.time.Duration</code> type.		Duration
camel.component.azure-storage-queue.visibility-timeout	The timeout period for how long the message is invisible in the queue. The timeout must be between 1 seconds and 7 days. The format should be in this form: <code>PnDTnHnMn.nS</code> , e.g: <code>PT20.345S</code> – parses as 20.345 seconds, <code>P2D</code> – parses as 2 days However, in case you are using <code>EndpointDsl/ComponentDsl</code> , you can do something like <code>Duration.ofSeconds()</code> since these Java APIs are typesafe. The option is a <code>java.time.Duration</code> type.		Duration

CHAPTER 17. BEAN

Only producer is supported

The Bean component binds beans to Camel message exchanges.

17.1. DEPENDENCIES

When using **bean** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-bean-starter</artifactId>
</dependency>
```

17.2. URI FORMAT

```
bean:beanName[?options]
```

Where **beanID** can be any string which is used to look up the bean in the Registry

17.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

17.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

17.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

17.4. COMPONENT OPTIONS

The Bean component supports 4 options, which are listed below.

Name	Description	Default	Type
cache (producer)	Deprecated Use singleton option instead.	true	Boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
scope (producer)	Scope of bean. When using singleton scope (default) the bean is created or looked up only once and reused for the lifetime of the endpoint. The bean should be thread-safe in case concurrent threads is calling the bean at the same time. When using request scope the bean is created or looked up once per request (exchange). This can be used if you want to store state on a bean while processing a request and you want to call the same bean instance multiple times while processing the request. The bean does not have to be thread-safe as the instance is only called from the same request. When using delegate scope, then the bean will be looked up or created per call. However in case of lookup then this is delegated to the bean registry such as Spring or CDI (if in use), which depends on their configuration can act as either singleton or prototype scope. so when using prototype then this depends on the delegated registry. Enum values: • Singleton • Request • Prototype	Singleton	BeanScope
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

17.5. ENDPOINT OPTIONS

The Bean endpoint is configured using URI syntax:

```
bean:beanName
```

with the following path and query parameters:

17.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
beanName (common)	Required Sets the name of the bean to invoke.		String

17.5.2. Query Parameters (5 parameters)

Name	Description	Default	Type
cache (common)	Deprecated Use scope option instead.		Boolean
method (common)	Sets the name of the method to invoke on the bean.		String
scope (common)	Scope of bean. When using singleton scope (default) the bean is created or looked up only once and reused for the lifetime of the endpoint. The bean should be thread-safe in case concurrent threads is calling the bean at the same time. When using request scope the bean is created or looked up once per request (exchange). This can be used if you want to store state on a bean while processing a request and you want to call the same bean instance multiple times while processing the request. The bean does not have to be thread-safe as the instance is only called from the same request. When using prototype scope, then the bean will be looked up or created per call. However in case of lookup then this is delegated to the bean registry such as Spring or CDI (if in use), which depends on their configuration can act as either singleton or prototype scope. so when using prototype then this depends on the delegated registry. Enum values: • Singleton • Request • Prototype	Singleton	BeanScope

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
parameters (advanced)	Used for configuring additional properties on the bean.		Map

17.6. EXAMPLES

The object instance that is used to consume messages must be explicitly registered with the Registry. For example, if you are using Spring you must define the bean in the Spring configuration XML file.

You can also register beans manually via Camel's **Registry** with the **bind** method.

Once an endpoint has been registered, you can build Camel routes that use it to process exchanges.

A **bean:** endpoint cannot be defined as the input to the route; i.e. you cannot consume from it, you can only route from some inbound message Endpoint to the bean endpoint as output. So consider using a **direct:** or **queue:** endpoint as the input.

You can use the **createProxy()** methods on [ProxyHelper](#) to create a proxy that will generate exchanges and send them to any endpoint:

And the same route using XML DSL:

```
<route>
  <from uri="direct:hello"/>
  <to uri="bean:bye"/>
</route>
```

17.7. BEAN AS ENDPOINT

Camel also supports invoking [Bean](#) as an Endpoint. What happens is that when the exchange is routed to the **myBean** Camel will use the Bean Binding to invoke the bean. The source for the bean is just a plain POJO.

Camel will use Bean Binding to invoke the **sayHello** method, by converting the Exchange's In body to the **String** type and storing the output of the method on the Exchange Out body.

17.8. JAVA DSL BEAN SYNTAX

Java DSL comes with syntactic sugar for the component. Instead of specifying the bean explicitly as the endpoint (i.e. `to("bean:beanName")`) you can use the following syntax:

```
// Send message to the bean endpoint
// and invoke method resolved using Bean Binding.
from("direct:start").bean("beanName");

// Send message to the bean endpoint
// and invoke given method.
from("direct:start").bean("beanName", "methodName");
```

Instead of passing name of the reference to the bean (so that Camel will lookup for it in the registry), you can specify the bean itself:

```
// Send message to the given bean instance.
from("direct:start").bean(new ExampleBean());

// Explicit selection of bean method to be invoked.
from("direct:start").bean(new ExampleBean(), "methodName");

// Camel will create the instance of bean and cache it for you.
from("direct:start").bean(ExampleBean.class);
```

17.9. BEAN BINDING

How bean methods to be invoked are chosen (if they are not specified explicitly through the **method** parameter) and how parameter values are constructed from the Message are all defined by the Bean Binding mechanism which is used throughout all of the various Bean Integration mechanisms in Camel.

17.10. SPRING BOOT AUTO-CONFIGURATION

The component supports 13 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.bean.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.bean.enabled</code>	Whether to enable auto configuration of the bean component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component. .bean.lazy-start- producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component. .bean.scope	Scope of bean. When using singleton scope (default) the bean is created or looked up only once and reused for the lifetime of the endpoint. The bean should be thread-safe in case concurrent threads is calling the bean at the same time. When using request scope the bean is created or looked up once per request (exchange). This can be used if you want to store state on a bean while processing a request and you want to call the same bean instance multiple times while processing the request. The bean does not have to be thread-safe as the instance is only called from the same request. When using delegate scope, then the bean will be looked up or created per call. However in case of lookup then this is delegated to the bean registry such as Spring or CDI (if in use), which depends on their configuration can act as either singleton or prototype scope. so when using prototype then this depends on the delegated registry.		BeanScope
camel.component. .class.autowired- enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component. .class.enabled	Whether to enable auto configuration of the class component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.scope	Scope of bean. When using singleton scope (default) the bean is created or looked up only once and reused for the lifetime of the endpoint. The bean should be thread-safe in case concurrent threads is calling the bean at the same time. When using request scope the bean is created or looked up once per request (exchange). This can be used if you want to store state on a bean while processing a request and you want to call the same bean instance multiple times while processing the request. The bean does not have to be thread-safe as the instance is only called from the same request. When using delegate scope, then the bean will be looked up or created per call. However in case of lookup then this is delegated to the bean registry such as Spring or CDI (if in use), which depends on their configuration can act as either singleton or prototype scope. so when using prototype then this depends on the delegated registry.		BeanScope
camel.language.bean.enabled	Whether to enable auto configuration of the bean language. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.language.bean.scope</code>	Scope of bean. When using singleton scope (default) the bean is created or looked up only once and reused for the lifetime of the endpoint. The bean should be thread-safe in case concurrent threads is calling the bean at the same time. When using request scope the bean is created or looked up once per request (exchange). This can be used if you want to store state on a bean while processing a request and you want to call the same bean instance multiple times while processing the request. The bean does not have to be thread-safe as the instance is only called from the same request. When using prototype scope, then the bean will be looked up or created per call. However in case of lookup then this is delegated to the bean registry such as Spring or CDI (if in use), which depends on their configuration can act as either singleton or prototype scope. So when using prototype scope then this depends on the bean registry implementation.	Singleton	String
<code>camel.language.bean.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.component.bean.cache</code>	Deprecated Use singleton option instead.	true	Boolean
<code>camel.component.class.cache</code>	Deprecated Use singleton option instead.	true	Boolean

CHAPTER 18. BEANIO

Since Camel 2.10

The BeanIO Data Format uses [BeanIO](#) to handle flat payloads (such as XML, CSV, delimited, or fixed length formats).

BeanIO is configured using a mapping XML file where you define the mapping from the flat format to Objects (POJOs). This mapping file is mandatory to use.

18.1. DEPENDENCIES

When using beanio with Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-beanio-starter</artifactId>
</dependency>
```

18.2. OPTIONS

The BeanIO dataformat supports 8 options, which are listed below.

Name	Default	Java Type	Description
mapping (common)		String	Required The BeanIO mapping file. Is by default loaded from

Name	Default	Java Type	Description
			path. You can prefix with file; http; or class path: to denote from where to load the mapping file.

Name	Default	Java Type	Description
streamName (common)		String	Required The name of the stream to use.
ignoreUnidentifiedRecords (common)	false	Boolean	Whether to ignore unidentified records.

Name	Default	Java Type	Description
<code>ignoreUnexpectedRecords</code> (common)	false	Boolean	Whether to ignore unexpected records.
<code>ignoreInvalidRecords</code> (common)	false	Boolean	Whether to ignore invalid records.

Name	Default	Java Type	Description
encoding (advanced)		String	The charset to use. Is by default the JVM platform default charset.
beanReaderErrorHandlerType (advanced)		String	To use a custom or g.a.p.a.ch.e.c.a.m.e.l.d.a.t.a.f.o.r.m.a.t.b.e.a.n.i.o.

Name	Default	Java Type	BeanIO Description
			Handler as error handler while parsing. Configure the fully qualified class name of the error handler. Notice the options ignore

Name	Default	Java Type	Description
			Records, ignore Unexpected Records, and ignore invalid Records may not be in use when you use a custom error handler.

Name marshalSingleObject (advanced)	Default	Java Type	Description
			ns co ntr ols wh et he r to un ma rs hal as a list of ob jec ts or as a sin gl e ob jec t on ly. Th e for m er is th e de fa ult m od e, an d th e lat ter is

Name	Default	Java Type	Description
			defined in special use-cases wherever be an omaps the Camel messages to a single POJO bean.

18.3. USAGE

An example of a [mapping file is here](#).

To use the **BeanIODataFormat** you need to configure the data format with the mapping file, as well the name of the stream.

This can be done as shown below. The streamName is **employeeFile**.

- Java
- XML

```
DataFormat format = new BeanIODataFormat(
    "org/apache/camel/dataformat/beanio/mappings.xml",
    "employeeFile");

// a route which uses the bean io data format to format the CSV data
// to java objects
from("direct:unmarshal")
    .unmarshal(format)
    // and then split the message body, so we get a message for each row
    .split(body())
    .to("mock:beanio-unmarshal");

// convert a list of java objects back to flat format
from("direct:marshal")
    .marshal(format)
    .to("mock:beanio-marshal");
```

```
<route>
  <from uri="direct:unmarshal"/>
  <unmarshal>
    <beanio mapping="org/apache/camel/dataformat/beanio/mappings.xml"
streamName="employeeFile"/>
  </unmarshal>
  <split>
    <simple>${body}</simple>
    <to uri="mock:beanio-unmarshal"/>
  </split>
</route>

<route>
  <from uri="direct:marshal"/>
  <marshal>
    <beanio mapping="org/apache/camel/dataformat/beanio/mappings.xml"
streamName="employeeFile"/>
  </marshal>
  <to uri="mock:beanio-marshal"/>
</route>
```

To use the BeanIO data format in XML, you need to configure it using the **<beanio>** XML tag as shown below. The routes are similar to the example above.

The first route is for transforming CSV data into a **List<Employee>** Java objects. Which we then split, so the mock endpoint receives a message for each row.

The second route is for the reverse operation, to transform a **List<Employee>** into a stream of CSV data.

The CSV data could, for example, be as below:

Joe,Smith,Developer,75000,10012009
 Jane,Doe,Architect,80000,01152008
 Jon,Anderson,Manager,85000,03182007

18.4. SPRING BOOT AUTO-CONFIGURATION

The component supports 9 options, which are listed below.

Name	Description	Default	Type
camel.dataformat.beanio.bean-reader-error-handler-type	To use a custom <code>org.apache.camel.dataformat.beanio.BeanIOErrorHandler</code> as error handler while parsing. Configure the fully qualified class name of the error handler. Notice the options <code>ignoreUnidentifiedRecords</code> , <code>ignoreUnexpectedRecords</code> , and <code>ignoreInvalidRecords</code> may not be in use when you use a custom error handler.		String
camel.dataformat.beanio.enabled	Whether to enable auto configuration of the beanio data format. This is enabled by default.		Boolean
camel.dataformat.beanio.encoding	The charset to use. Is by default the JVM platform default charset.		String
camel.dataformat.beanio.ignore-invalid-records	Whether to ignore invalid records.	false	Boolean
camel.dataformat.beanio.ignore-unexpected-records	Whether to ignore unexpected records.	false	Boolean
camel.dataformat.beanio.ignore-unidentified-records	Whether to ignore unidentified records.	false	Boolean
camel.dataformat.beanio.mapping	The BeanIO mapping file. Is by default loaded from the classpath. You can prefix with <code>file:</code> , <code>http:</code> , or <code>classpath:</code> to denote from where to load the mapping file.		String
camel.dataformat.beanio.stream-name	The name of the stream to use.		String

Name	Description	Default	Type
camel.dataformat.beanio.unmarshal-single-object	This option controls whether to unmarshal as a list of objects or as a single object only. The former is the default mode, and the latter is only intended in special use-cases where beanio maps the Camel message to a single POJO bean.	false	Boolean

CHAPTER 19. BEAN VALIDATOR

Only producer is supported

The Validator component performs bean validation of the message body using the Java Bean Validation API (). Camel uses the reference implementation, which is [Hibernate Validator](#).

19.1. DEPENDENCIES

When using **bean-validator** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-bean-validator-starter</artifactId>
</dependency>
```

19.2. URI FORMAT

```
bean-validator:label[?options]
```

Where **label** is an arbitrary text value describing the endpoint. You can append query options to the URI in the following format,

?option=value&option=value&...

19.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

19.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

19.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

19.4. COMPONENT OPTIONS

The Bean Validator component supports 8 options, which are listed below.

Name	Description	Default	Type
ignoreXmlConfiguration (producer)	Whether to ignore data from the META-INF/validation.xml file.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
constraintValidatorFactory (advanced)	To use a custom ConstraintValidatorFactory.		ConstraintValidatorFactory

Name	Description	Default	Type
messageInterpolator (advanced)	To use a custom MessageInterpolator.		MessageInterpolator
traversableResolver (advanced)	To use a custom TraversableResolver.		TraversableResolver
validationProviderResolver (advanced)	To use a a custom ValidationProviderResolver.		ValidationProviderResolver
validatorFactory (advanced)	Autowired To use a custom ValidatorFactory.		ValidatorFactory

19.5. ENDPOINT OPTIONS

The Bean Validator endpoint is configured using URI syntax:

```
bean-validator:label
```

with the following path and query parameters:

19.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
label (producer)	Required Where label is an arbitrary text value describing the endpoint.		String

19.5.2. Query Parameters (8 parameters)

Name	Description	Default	Type
group (producer)	To use a custom validation group.	javax.validation.groups.Default	String
ignoreXmlConfiguration (producer)	Whether to ignore data from the META-INF/validation.xml file.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
constraintValidatorFactory (advanced)	To use a custom ConstraintValidatorFactory.		ConstraintValidatorFactory
messageInterpolator (advanced)	To use a custom MessageInterpolator.		MessageInterpolator
traversableResolver (advanced)	To use a custom TraversableResolver.		TraversableResolver
validationProviderResolver (advanced)	To use a a custom ValidationProviderResolver.		ValidationProviderResolver
validatorFactory (advanced)	To use a custom ValidatorFactory.		ValidatorFactory

19.6. EXAMPLE

Assumed we have a java bean with the following annotations

Car.java

```
public class Car {

    @NotNull
    private String manufacturer;

    @NotNull
    @Size(min = 5, max = 14, groups = OptionalChecks.class)
    private String licensePlate;

    // getter and setter
}
```

and an interface definition for our custom validation group

OptionalChecks.java

```
public interface OptionalChecks {  
}
```

with the following Camel route, only the **@NotNull** constraints on the attributes `manufacturer` and `licensePlate` will be validated (Camel uses the default group **javax.validation.groups.Default**).

```
from("direct:start")  
  .to("bean-validator://x")  
  .to("mock:end")
```

If you want to check the constraints from the group **OptionalChecks**, you have to define the route like this

```
from("direct:start")  
  .to("bean-validator://x?group=OptionalChecks")  
  .to("mock:end")
```

If you want to check the constraints from both groups, you have to define a new interface first

AllChecks.java

```
@GroupSequence({Default.class, OptionalChecks.class})  
public interface AllChecks {  
}
```

and then your route definition should look like this

```
from("direct:start")  
  .to("bean-validator://x?group=AllChecks")  
  .to("mock:end")
```

And if you have to provide your own message interpolator, traversable resolver and constraint validator factory, you have to write a route like this

```
<bean id="myMessageInterpolator" class="my.ConstraintValidatorFactory" />  
<bean id="myTraversableResolver" class="my.TraversableResolver" />  
<bean id="myConstraintValidatorFactory" class="my.ConstraintValidatorFactory" />
```

```
from("direct:start")  
  .to("bean-validator://x?group=AllChecks&messageInterpolator=#myMessageInterpolator  
&traversableResolver=#myTraversableResolver&constraintValidatorFactory=#myConstraintValidatorFactory")  
  .to("mock:end")
```

It's also possible to describe your constraints as XML and not as Java annotations. In this case, you have to provide the file **META-INF/validation.xml** which could look like this

validation.xml

-

```

<validation-config
  xmlns="http://jboss.org/xml/ns/javax/validation/configuration"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jboss.org/xml/ns/javax/validation/configuration">

  <default-provider>org.hibernate.validator.HibernateValidator</default-provider>
  <message-
interpolator>org.hibernate.validator.engine.ResourceBundleMessageInterpolator</message-
interpolator>
  <traversable-
resolver>org.hibernate.validator.engine.resolver.DefaultTraversableResolver</traversable-resolver>
  <constraint-validator-
factory>org.hibernate.validator.engine.ConstraintValidatorFactoryImpl</constraint-validator-factory>
  <constraint-mapping>/constraints-car.xml</constraint-mapping>

</validation-config>

```

and the **constraints-car.xml** file

constraints-car.xml

```

<constraint-mappings xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jboss.org/xml/ns/javax/validation/mapping validation-mapping-1.0.xsd"
  xmlns="http://jboss.org/xml/ns/javax/validation/mapping">

  <default-package>org.apache.camel.component.bean.validator</default-package>

  <bean class="CarWithoutAnnotations" ignore-annotations="true">
    <field name="manufacturer">
      <constraint annotation="javax.validation.constraints.NotNull" />
    </field>

    <field name="licensePlate">
      <constraint annotation="javax.validation.constraints.NotNull" />

      <constraint annotation="javax.validation.constraints.Size">
        <groups>
          <value>org.apache.camel.component.bean.validator.OptionalChecks</value>
        </groups>
        <element name="min">5</element>
        <element name="max">14</element>
      </constraint>
    </field>
  </bean>
</constraint-mappings>

```

Here is the XML syntax for the example route definition for [OrderedChecks](#).

Note that the body should include an instance of a class to validate.

```

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd

```

```

http://camel.apache.org/schema/spring http://camel.apache.org/schema/spring/camel-spring.xsd">

<camelContext id="camel" xmlns="http://camel.apache.org/schema/spring">
  <route>
    <from uri="direct:start"/>
    <to uri="bean-validator://x?
group=org.apache.camel.component.bean.validator.OrderedChecks"/>
  </route>
</camelContext>
</beans>

```

19.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 9 options, which are listed below.

Name	Description	Default	Type
camel.component.bean-validator.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.bean-validator.constraint-validator-factory	To use a custom ConstraintValidatorFactory. The option is a <code>javax.validation.ConstraintValidatorFactory</code> type.		ConstraintValidatorFactory
camel.component.bean-validator.enabled	Whether to enable auto configuration of the bean-validator component. This is enabled by default.		Boolean
camel.component.bean-validator.ignore-xml-configuration	Whether to ignore data from the META-INF/validation.xml file.	false	Boolean
camel.component.bean-validator.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component .bean- validator.message- interpolator	To use a custom MessageInterpolator. The option is a javax.validation.MessageInterpolator type.		MessageInterpolator
camel.component .bean- validator.traversa- ble-resolver	To use a custom TraversableResolver. The option is a javax.validation.TraversableResolver type.		TraversableResolv er
camel.component .bean- validator.validatio- n-provider- resolver	To use a a custom ValidationProviderResolver. The option is a javax.validation.ValidationProviderResolver type.		ValidationProvider Resolver
camel.component .bean- validator.validato- r-factory	To use a custom ValidatorFactory. The option is a javax.validation.ValidatorFactory type.		ValidatorFactory

CHAPTER 20. BINDY

The goal of this component is to allow the parsing/binding of non-structured data (or to be more precise non-XML data) to/from Java Beans that have binding mappings defined with annotations. Using Bindy, you can bind data from sources such as :

- CSV records,
- Fixed-length records,
- FIX messages,
- or almost any other non-structured data

to one or many Plain Old Java Object (POJO). Bindy converts the data according to the type of the java property. POJOs can be linked together with one-to-many relationships available in some cases. Moreover, for data type like Date, Double, Float, Integer, Short, Long and BigDecimal, you can provide the pattern to apply during the formatting of the property.

For the BigDecimal numbers, you can also define the precision and the decimal or grouping separators.

Type	Format Type	Pattern example	Link
Date	DateFormat	dd-MM-yyyy	https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/text/SimpleDateFormat.html
Decimal*	DecimalFormat	##.###.###	https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/text/DecimalFormat.html

Where Decimal = Double, Integer, Float, Short, Long

Format supported

This first release only support comma separated values fields and key value pair fields (e.g. : FIX messages).

To work with camel-bindy, you must first define your model in a package (e.g. com.acme.model) and for each model class (e.g. Order, Client, Instrument, ...) add the required annotations (described hereafter) to the Class or field.

Multiple models

As you configure bindy using class names instead of package names you can put multiple models in the same package.

20.1. DEPENDENCIES

When using **bindy-csv** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-bindy-starter</artifactId>
</dependency>
```

20.2. OPTIONS

The Bindy dataformat supports 5 options, which are listed below.

Name	Default	Java Type	Description
type		Enum	Required Whether to use Csv, Fixed, or KeyValue. Enum values: • Csv • Fixed • KeyValue
classType		String	Name of model class to use.
locale		String	To configure a default locale to use, such as us for united states. To use the JVM platform default locale then use the name default.
unwrapSingleInstance		Boolean	When unmarshalling should a single instance be unwrapped and returned instead of wrapped in a java.util.List.
allowEmptyStream		Boolean	Whether to allow empty streams in the unmarshal process. If true, no exception will be thrown when a body without records is provided.

20.3. ANNOTATIONS

The annotations created allow to map different concept of your model to the POJO like:

- Type of record (CSV, key value pair (e.g. FIX message), fixed length ...),
- Link (to link object in another object),
- DataField and their properties (int, type, ...),
- KeyValuePairField (for key = value format like we have in FIX financial messages),

- Section (to identify header, body and footer section),
- OneToMany,
- BindyConverter,
- FormatFactories

This section will describe them.

20.3.1.1. CsvRecord

The CsvRecord annotation is used to identified the root class of the model. It represents a record = "a line of a CSV file" and can be linked to several children model classes.

Annotation name	Record type	Level
CsvRecord	CSV	Class

Parameter name	Type	Required	Default value	Info
separator	String	✓		Separator used to split a record in tokens (mandatory) - can be ',' or ';' or 'anything'. The only whitespace character supported is tab (\t). No other whitespace characters (spaces) are not supported. This value is interpreted as a regular expression. If you want to use a sign which has a special meaning in regular expressions, e.g. the ' ' sign, then you have to mask it, like ' '.
allowEmptyStream	boolean		false	The allowEmptyStream parameter will allow to process the unavailable stream for CSV file.
autospanLine	boolean		false	Last record spans rest of line (optional) - if enabled then the last column is auto spanned to end of line, for example if its a comment, etc this allows the line to contain all characters, also the delimiter char.
crLf	String		WINDOWS	Character to be used to add a carriage return after each record (optional) - allow to define the carriage return character to use. If you specify a value other than the three listed before, the value you enter (custom) will be used as the CRLF character(s). Three values can be used : WINDOWS, UNIX, MAC, or custom.
endWithLineBreak	boolean		true	The endWithLineBreak parameter flags if the CSV file should end with a line break or not (optional)

Parameter name	Type	Required	Default value	Info
generateHeaderColumns	boolean		false	The generateHeaderColumns parameter allow to add in the CSV generated the header containing names of the columns
isOrdered	boolean		false	Indicates if the message must be ordered in output
name	String			Name describing the record (optional)
quote	String		"	Whether to marshal columns with the given quote character (optional) – allow to specify a quote character of the fields when CSV is generated. This annotation is associated to the root class of the model and must be declared one time.
quoting	boolean		false	Indicate if the values (and headers) must be quoted when marshaling (optional)
quotingEscaped	boolean		false	Indicate if the values must be escaped when quoting (optional)
removeQuotes	boolean		true	The remove quotes parameter flags if unmarshalling should try to remove quotes for each field
skipField	boolean		false	The skipField parameter will allow to skip fields of a CSV file. If some fields are not necessary, they can be skipped.
skipFirstLine	boolean		false	The skipFirstLine parameter will allow to skip or not the first line of a CSV file. This line often contains columns definition
trimLine	boolean		true	Whether to trim each line (stand and end) before parsing the line into data fields.

case 1: separator = ','

The separator used to segregate the fields in the CSV record is , :

```
10, J, Pauline, M, XD12345678, Fortis Dynamic 15/15, 2500, USD, 08-01-2009
```

```
@CsvRecord( separator = "," )
public Class Order {

}
```

case 2 : separator = ';' ;

Compare to the previous case, the separator here is ; instead of , :

```
10; J; Pauline; M; XD12345678; Fortis Dynamic 15/15; 2500; USD; 08-01-2009
```

```
@CsvRecord( separator = ";" )
public Class Order {

}
```

case 3 : separator = '|' ;

Compare to the previous case, the separator here is | instead of ; :

```
10| J| Pauline| M| XD12345678| Fortis Dynamic 15/15| 2500| USD| 08-01-2009
```

```
@CsvRecord( separator = "|" )
public Class Order {

}
```

case 4 : separator = '\"','\"'**Applies for Camel 2.8.2 or older**

When the field to be parsed of the CSV record contains , or ; which is also used as separator, we should find another strategy to tell camel bindy how to handle this case. To define the field containing the data with a comma, you will use single or double quotes as delimiter (e.g : '10', 'Street 10, NY', 'USA' or "10", "Street 10, NY", "USA").

**NOTE**

In this case, the first and last character of the line which are a single or double quotes will be removed by bindy.

```
"10","J","Pauline"," M","XD12345678","Fortis Dynamic 15,15","2500","USD","08-01-2009"
```

```
@CsvRecord( separator = "\"','\"" )
public Class Order {

}
```

Bindy automatically detects if the record is enclosed with either single or double quotes and automatic remove those quotes when unmarshalling from CSV to Object. Therefore do **not** include the quotes in the separator, but simply do as below:

```
"10","J","Pauline"," M","XD12345678","Fortis Dynamic 15,15","2500","USD","08-01-2009"
```

```
@CsvRecord( separator = "," )
public Class Order {
```

```
}
```

Note that if you want to marshal from Object to CSV and use quotes, then you need to specify which quote character to use, using the **quote** attribute on the **@CsvRecord** as shown below:

```
@CsvRecord( separator = ",", quote = "\"" )
public Class Order {

}
```

case 5 : separator & skipFirstLine

The feature is interesting when the client wants to have in the first line of the file, the name of the data fields :

```
order id, client id, first name, last name, isin code, instrument name, quantity, currency, date
```

To inform bindy that this first line must be skipped during the parsing process, then we use the attribute :

```
@CsvRecord(separator = ",", skipFirstLine = true)
public Class Order {

}
```

case 6 : generateHeaderColumns

To add at the first line of the CSV generated, the attribute generateHeaderColumns must be set to true in the annotation like this :

```
@CsvRecord( generateHeaderColumns = true )
public Class Order {

}
```

As a result, Bindy during the unmarshaling process will generate CSV like this :

```
order id, client id, first name, last name, isin code, instrument name, quantity, currency, date
10, J, Pauline, M, XD12345678, Fortis Dynamic 15/15, 2500, USD, 08-01-2009
```

case 7 : carriage return

If the platform where camel-bindy will run is not Windows but Macintosh or Unix, then you can change the crlf property like this. Three values are available : WINDOWS, UNIX or MAC

```
@CsvRecord(separator = ",", crlf="MAC")
public Class Order {

}
```

Additionally, if for some reason you need to add a different line ending character, you can opt to specify it using the crlf parameter. In the following example, we can end the line with a comma followed by the newline character:

```
■
```

```
@CsvRecord(separator = ",", crlf=",\n")
public Class Order {

}
```

case 8 : isOrdered

Sometimes, the order to follow during the creation of the CSV record from the model is different from the order used during the parsing. Then, in this case, we can use the attribute **isOrdered = true** to indicate this in combination with attribute **position** of the DataField annotation.

```
@CsvRecord(isOrdered = true)
public Class Order {

    @DataField(pos = 1, position = 11)
    private int orderNr;

    @DataField(pos = 2, position = 10)
    private String clientNr;

}
```



NOTE

pos is used to parse the file stream, while **position** is used to generate the CSV.

20.3.2. 2. Link

The link annotation will allow to link objects together.

Annotation name	Record type	Level
Link	all	Class & Property

Parameter name	Type	Required	Default value	Info
linkType	LinkType		OneToOne	Type of link identifying the relation between the classes

Only one-to-one relation is allowed as of the current version.

E.g : If the model class Client is linked to the Order class, then use annotation Link in the Order class like this :

Property Link

```
@CsvRecord(separator = ",")
public class Order {
```

```

@DataField(pos = 1)
private int orderNr;

@Link
private Client client;
}

```

And for the class Client :

Class Link

```

@Link
public class Client {

}

```

20.3.3. 3. DataField

The DataField annotation defines the property of the field. Each datafield is identified by its position in the record, a type (string, int, date, ...) and optionally of a pattern.

Annotation name	Record type	Level
DataField	all	Property

Parameter name	Type	Required	Default value	Info
pos	int	✓		Position of the data in the input record, must start from 1 (mandatory). See the position parameter.
align	String		R	Align the text to the right or left. Use values <code><tt>R</tt></code> or <code><tt>L</tt></code> .
clip	boolean		false	Indicates to clip data in the field if it exceeds the allowed length when using fixed length.
columnName	String			Name of the header column (optional). Uses the name of the property as default. Only applicable when CsvRecord has generateHeaderColumns = true
decimalSeparator	String			Decimal Separator to be used with BigDecimal number
defaultValue	String			Field's default value in case no value is set

Parameter name	Type	Required	Default value	Info
delimiter	String			Optional delimiter to be used if the field has a variable length
groupingSeparator	String			Grouping Separator to be used with BigDecimal number when we would like to format/parse to number with grouping e.g. 123,456.789
impliedDecimalSeparator	boolean		false	Indicates if there is a decimal point implied at a specified location
length	int		0	Length of the data block (number of characters) if the record is set to a fixed length
lengthPos	int		0	Identifies a data field in the record that defines the expected fixed length for this field
method	String			Method name to call to apply such customization on DataField. This must be the method on the datafield itself or you must provide static fully qualified name of the class's method e.g: see unit test <code>org.apache.camel.dataformat.bindy.csv.BindySimpleCsvFunctionWithExternalMethodTest.replaceToBar</code>
name	String			Name of the field (optional)
paddingChar	char			The char to pad with if the record is set to a fixed length
pattern	String			Pattern that the Java formatter (SimpleDateFormat by example) will use to transform the data (optional). If using pattern, then setting locale on bindy data format is recommended. Either set to a known locale such as "us" or use "default" to use platform default locale.
position	int		0	Position of the field in the output message generated (should start from 1). Must be used when the position of the field in the CSV generated (output message) must be different compare to input position (pos). See the pos parameter.
precision	int		0	precision of the <code>{@link java.math.BigDecimal}</code> number to be created

Parameter name	Type	Required	Default value	Info
required	boolean		false	Indicates if the field is mandatory
rounding	String		CEILING	Round mode to be used to round/scale a BigDecimal Values : UP, DOWN, CEILING, FLOOR, HALF_UP, HALF_DOWN, HALF_EVEN, UNNECESSARY e.g : Number = 123456.789, Precision = 2, Rounding = CEILING Result : 123456.79
timezone	String			Timezone to be used.
trim	boolean		false	Indicates if the value should be trimmed

case 1: pos

This parameter/attribute represents the position of the field in the CSV record.

Position

```
@CsvRecord(separator = ";")
public class Order {

    @DataField(pos = 1)
    private int orderNr;

    @DataField(pos = 5)
    private String isinCode;

}
```

As you can see in this example the position starts at **1** but continues at **5** in the class Order. The numbers from **2** to **4** are defined in the class Client (see here after).

Position continues in another model class

```
public class Client {

    @DataField(pos = 2)
    private String clientNr;

    @DataField(pos = 3)
    private String firstName;

    @DataField(pos = 4)
    private String lastName;

}
```

case 2 : pattern

The pattern allows to enrich or validates the format of your data

Pattern

```

@CsvRecord(separator = ",")
public class Order {

    @DataField(pos = 1)
    private int orderNr;

    @DataField(pos = 5)
    private String isinCode;

    @DataField(name = "Name", pos = 6)
    private String instrumentName;

    @DataField(pos = 7, precision = 2)
    private BigDecimal amount;

    @DataField(pos = 8)
    private String currency;

    // pattern used during parsing or when the date is created
    @DataField(pos = 9, pattern = "dd-MM-yyyy")
    private Date orderDate;
}

```

case 3 : precision

The precision is helpful when you want to define the decimal part of your number.

Precision

```

@CsvRecord(separator = ",")
public class Order {

    @DataField(pos = 1)
    private int orderNr;

    @Link
    private Client client;

    @DataField(pos = 5)
    private String isinCode;

    @DataField(name = "Name", pos = 6)
    private String instrumentName;

    @DataField(pos = 7, precision = 2)
    private BigDecimal amount;

    @DataField(pos = 8)
    private String currency;
}

```



```

    @DataField(pos = 9, pattern = "dd-MM-yyyy")
    private Date orderDate;
}

```

case 4 : Position is different in output

The position attribute will inform bindy how to place the field in the CSV record generated. By default, the position used corresponds to the position defined with the attribute **pos**. If the position is different (that means that we have an asymmetric process comparing marshaling from unmarshaling) then we can use **position** to indicate this.

Here is an example:

Position is different in output

```

@DataRecord(separator = ",", isOrdered = true)
public class Order {

    // Positions of the fields start from 1 and not from 0

    @DataField(pos = 1, position = 11)
    private int orderNr;

    @DataField(pos = 2, position = 10)
    private String clientNr;

    @DataField(pos = 3, position = 9)
    private String firstName;

    @DataField(pos = 4, position = 8)
    private String lastName;

    @DataField(pos = 5, position = 7)
    private String instrumentCode;

    @DataField(pos = 6, position = 6)
    private String instrumentNumber;
}

```

This attribute of the annotation **@DataField** must be used in combination with attribute **isOrdered = true** of the annotation **@CsvRecord**.

case 5 : required

If a field is mandatory, simply use the attribute **required** set to true.

Required

```

@DataRecord(separator = ",")
public class Order {

    @DataField(pos = 1)
    private int orderNr;

    @DataField(pos = 2, required = true)
    private String clientNr;
}

```

```
@DataField(pos = 3, required = true)
private String firstName;

@DataField(pos = 4, required = true)
private String lastName;
}
```

If this field is not present in the record, then an error will be raised by the parser with the following information :

Some fields are missing (optional or mandatory), line :

case 6 : trim

If a field has leading and/or trailing spaces which should be removed before they are processed, simply use the attribute **trim** set to true.

Trim

```
@CsvRecord(separator = ",")
public class Order {

    @DataField(pos = 1, trim = true)
    private int orderNr;

    @DataField(pos = 2, trim = true)
    private Integer clientNr;

    @DataField(pos = 3, required = true)
    private String firstName;

    @DataField(pos = 4)
    private String lastName;
}
```

case 7 : defaultValue

If a field is not defined then uses the value indicated by the **defaultValue** attribute.

Default value

```
@CsvRecord(separator = ",")
public class Order {

    @DataField(pos = 1)
    private int orderNr;

    @DataField(pos = 2)
    private Integer clientNr;

    @DataField(pos = 3, required = true)
    private String firstName;
}
```

```

    @DataField(pos = 4, defaultValue = "Barin")
    private String lastName;
}

```

case 8 : columnName

Specifies the column name for the property only if **@CsvRecord** has annotation **generateHeaderColumns = true**.

Column Name

```

@CsvRecord(separator = ";", generateHeaderColumns = true)
public class Order {

    @DataField(pos = 1)
    private int orderNr;

    @DataField(pos = 5, columnName = "ISIN")
    private String isinCode;

    @DataField(name = "Name", pos = 6)
    private String instrumentName;
}

```

This attribute is only applicable to optional fields.

20.3.4. 4. FixedLengthRecord

The FixedLengthRecord annotation is used to identified the root class of the model. It represents a record = "a line of a file/message containing data fixed length (number of characters) formatted" and can be linked to several children model classes. This format is a bit particular because data of a field can be aligned to the right or to the left.

When the size of the data does not fill completely the length of the field, we can then add 'pad' characters.

Annotation name	Record type	Level
FixedLengthRecord	fixed	Class

Parameter name	Type	Required	Default value	Info
countGrapheme	boolean		false	Indicates how chars are counted

Parameter name	Type	Required	Default value	Info
crlf	String		WINDOWS	Character to be used to add a carriage return after each record (optional). Possible values: WINDOWS, UNIX, MAC, or custom. This option is used only during marshalling, whereas unmarshalling uses system default JDK provided line delimiter unless eol is customized.
eol	String			Character to be used to process considering end of line after each record while unmarshalling (optional – default: "", which help default JDK provided line delimiter to be used unless any other line delimiter provided) This option is used only during unmarshalling, where marshalling uses system default provided line delimiter as "WINDOWS" unless any other value is provided.
footer	Class		void	Indicates that the record(s) of this type may be followed by a single footer record at the end of the file
header	Class		void	Indicates that the record(s) of this type may be preceded by a single header record at the beginning of in the file
ignore Missing Chars	boolean		false	Indicates whether too short lines will be ignored
ignore Trailing Chars	boolean		false	Indicates that characters beyond the last mapped field can be ignored when unmarshalling / parsing. This annotation is associated to the root class of the model and must be declared one time.
length	int		0	The fixed length of the record (number of characters). It means that the record will always be that long padded with <code>\{#paddingChar()}'s</code>
name	String			Name describing the record (optional)
paddingChar	char			The char to pad with.
skipFooter	boolean		false	Configures the data format to skip marshalling / unmarshalling of the footer record. Configure this parameter on the primary record (e.g., not the header or footer).
skipHeader	boolean		false	Configures the data format to skip marshalling / unmarshalling of the header record. Configure this parameter on the primary record (e.g., not the header or footer).

A record may not be both a header/footer and a primary fixed-length record.

case 1 : Simple fixed length record

This simple example shows how to design the model to parse/format a fixed message

```
10A9PaulineMISINXD12345678BUYShare2500.45USD01-08-2009
```

Fixed-simple

```
@FixedLengthRecord(length=54, paddingChar=' ')
public static class Order {

    @DataField(pos = 1, length=2)
    private int orderNr;

    @DataField(pos = 3, length=2)
    private String clientNr;

    @DataField(pos = 5, length=7)
    private String firstName;

    @DataField(pos = 12, length=1, align="L")
    private String lastName;

    @DataField(pos = 13, length=4)
    private String instrumentCode;

    @DataField(pos = 17, length=10)
    private String instrumentNumber;

    @DataField(pos = 27, length=3)
    private String orderType;

    @DataField(pos = 30, length=5)
    private String instrumentType;

    @DataField(pos = 35, precision = 2, length=7)
    private BigDecimal amount;

    @DataField(pos = 42, length=3)
    private String currency;

    @DataField(pos = 45, length=10, pattern = "dd-MM-yyyy")
    private Date orderDate;
}
```

case 2 : Fixed length record with alignment and padding

This more elaborated example show how to define the alignment for a field and how to assign a padding character which is ' ' here:

```
10A9 PaulineM ISINXD12345678BUYShare2500.45USD01-08-2009
```

Fixed-padding-align

```
@FixedLengthRecord(length=60, paddingChar=' ')
```

```

public static class Order {

    @DataField(pos = 1, length=2)
    private int orderNr;

    @DataField(pos = 3, length=2)
    private String clientNr;

    @DataField(pos = 5, length=9)
    private String firstName;

    @DataField(pos = 14, length=5, align="L") // align text to the LEFT zone of the block
    private String lastName;

    @DataField(pos = 19, length=4)
    private String instrumentCode;

    @DataField(pos = 23, length=10)
    private String instrumentNumber;

    @DataField(pos = 33, length=3)
    private String orderType;

    @DataField(pos = 36, length=5)
    private String instrumentType;

    @DataField(pos = 41, precision = 2, length=7)
    private BigDecimal amount;

    @DataField(pos = 48, length=3)
    private String currency;

    @DataField(pos = 51, length=10, pattern = "dd-MM-yyyy")
    private Date orderDate;
}

```

case 3 : Field padding

Sometimes, the default padding defined for record cannot be applied to the field as we have a number format where we would like to pad with '0' instead of ' '. In this case, you can use in the model the attribute **paddingChar** on **@DataField** to set this value.

```
10A9 PaulineM ISINXD12345678BUYShare000002500.45USD01-08-2009
```

Fixed-padding-field

```

@FixedLengthRecord(length = 65, paddingChar = ' ')
public static class Order {

    @DataField(pos = 1, length = 2)
    private int orderNr;

    @DataField(pos = 3, length = 2)
    private String clientNr;

    @DataField(pos = 5, length = 9)

```

```

private String firstName;

@DataField(pos = 14, length = 5, align = "L")
private String lastName;

@DataField(pos = 19, length = 4)
private String instrumentCode;

@DataField(pos = 23, length = 10)
private String instrumentNumber;

@DataField(pos = 33, length = 3)
private String orderType;

@DataField(pos = 36, length = 5)
private String instrumentType;

@DataField(pos = 41, precision = 2, length = 12, paddingChar = '0')
private BigDecimal amount;

@DataField(pos = 53, length = 3)
private String currency;

@DataField(pos = 56, length = 10, pattern = "dd-MM-yyyy")
private Date orderDate;
}

```

case 4: Fixed length record with delimiter

Fixed-length records sometimes have delimited content within the record. The `firstName` and `lastName` fields are delimited with the `^` character in the following example:

```
10A9Pauline^M^ISINXD12345678BUYShare000002500.45USD01-08-2009
```

Fixed-delimited

```

@FixedLengthRecord
public static class Order {

    @DataField(pos = 1, length = 2)
    private int orderNr;

    @DataField(pos = 2, length = 2)
    private String clientNr;

    @DataField(pos = 3, delimiter = "^")
    private String firstName;

    @DataField(pos = 4, delimiter = "^")
    private String lastName;

    @DataField(pos = 5, length = 4)
    private String instrumentCode;

    @DataField(pos = 6, length = 10)

```

```

private String instrumentNumber;

@DataField(pos = 7, length = 3)
private String orderType;

@DataField(pos = 8, length = 5)
private String instrumentType;

@DataField(pos = 9, precision = 2, length = 12, paddingChar = '0')
private BigDecimal amount;

@DataField(pos = 10, length = 3)
private String currency;

@DataField(pos = 11, length = 10, pattern = "dd-MM-yyyy")
private Date orderDate;
}

```

The **pos** value(s) in a fixed-length record may optionally be defined using ordinal, sequential values instead of precise column numbers.

case 5 : Fixed length record with record-defined field length

Occasionally a fixed-length record may contain a field that define the expected length of another field within the same record. In the following example the length of the **instrumentNumber** field value is defined by the value of **instrumentNumberLen** field in the record.

```
10A9Pauline^M^ISIN10XD12345678BUYShare000002500.45USD01-08-2009
```

Fixed-delimited

```

@FixedLengthRecord
public static class Order {

    @DataField(pos = 1, length = 2)
    private int orderNr;

    @DataField(pos = 2, length = 2)
    private String clientNr;

    @DataField(pos = 3, delimiter = "^")
    private String firstName;

    @DataField(pos = 4, delimiter = "^")
    private String lastName;

    @DataField(pos = 5, length = 4)
    private String instrumentCode;

    @DataField(pos = 6, length = 2, align = "R", paddingChar = '0')
    private int instrumentNumberLen;

    @DataField(pos = 7, lengthPos=6)
    private String instrumentNumber;
}

```



```

@DataField(pos = 8, length = 3)
private String orderType;

@DataField(pos = 9, length = 5)
private String instrumentType;

@DataField(pos = 10, precision = 2, length = 12, paddingChar = '0')
private BigDecimal amount;

@DataField(pos = 11, length = 3)
private String currency;

@DataField(pos = 12, length = 10, pattern = "dd-MM-yyyy")
private Date orderDate;
}

```

case 6 : Fixed length record with header and footer

Bindy will discover fixed-length header and footer records that are configured as part of the model – provided that the annotated classes exist either in the same package as the primary **@FixedLengthRecord** class, or within one of the configured scan packages. The following text illustrates two fixed-length records that are bracketed by a header record and footer record.

```

101-08-2009
10A9 PaulineM ISINXD12345678BUYShare000002500.45USD01-08-2009
10A9 RichN ISINXD12345678BUYShare000002700.45USD01-08-2009
90000000002

```

Fixed-header-and-footer-main-class

```

@FixedLengthRecord(header = OrderHeader.class, footer = OrderFooter.class)
public class Order {

    @DataField(pos = 1, length = 2)
    private int orderNr;

    @DataField(pos = 2, length = 2)
    private String clientNr;

    @DataField(pos = 3, length = 9)
    private String firstName;

    @DataField(pos = 4, length = 5, align = "L")
    private String lastName;

    @DataField(pos = 5, length = 4)
    private String instrumentCode;

    @DataField(pos = 6, length = 10)
    private String instrumentNumber;

    @DataField(pos = 7, length = 3)
    private String orderType;

    @DataField(pos = 8, length = 5)
    private String instrumentType;
}

```

```

@DataField(pos = 9, precision = 2, length = 12, paddingChar = '0')
private BigDecimal amount;

@DataField(pos = 10, length = 3)
private String currency;

@DataField(pos = 11, length = 10, pattern = "dd-MM-yyyy")
private Date orderDate;
}

@FixedLengthRecord
public class OrderHeader {
    @DataField(pos = 1, length = 1)
    private int recordType = 1;

    @DataField(pos = 2, length = 10, pattern = "dd-MM-yyyy")
    private Date recordDate;
}

@FixedLengthRecord
public class OrderFooter {

    @DataField(pos = 1, length = 1)
    private int recordType = 9;

    @DataField(pos = 2, length = 9, align = "R", paddingChar = '0')
    private int numberOfRecordsInTheFile;
}

```

case 7 : Skipping content when parsing a fixed length record

It is common to integrate with systems that provide fixed-length records containing more information than needed for the target use case. It is useful in this situation to skip the declaration and parsing of those fields that we do not need. To accomodate this, Bindy will skip forward to the next mapped field within a record if the **pos** value of the next declared field is beyond the cursor position of the last parsed field. Using absolute **pos** locations for the fields of interest (instead of ordinal values) causes Bindy to skip content between two fields.

Similarly, it is possible that none of the content beyond some field is of interest. In this case, you can tell Bindy to skip parsing of everything beyond the last mapped field by setting the **ignoreTrailingChars** property on the **@FixedLengthRecord** declaration.

```

@FixedLengthRecord(ignoreTrailingChars = true)
public static class Order {

    @DataField(pos = 1, length = 2)
    private int orderNr;

    @DataField(pos = 3, length = 2)
    private String clientNr;

    // any characters that appear beyond the last mapped field will be ignored
}

```

20.3.5. 5. Message

The Message annotation is used to identify the class of your model who will contain key value pairs fields. This kind of format is used mainly in Financial Exchange Protocol Messages (FIX). Nevertheless, this annotation can be used for any other format where data are identified by keys. The key pair values are separated each other by a separator which can be a special character like a tab delimiter (unicode representation : `\u0009`) or a start of heading (unicode representation : `\u0001`)



NOTE

To work with FIX messages, the model must contain a Header and Trailer classes linked to the root message class which could be a Order class. This is not mandatory but will be very helpful when you will use camel-bindy in combination with camel-fix which is a Fix gateway based on quickFix project .

Annotation name	Record type	Level
Message	key value pair	Class

Parameter name	Type	Required	Default value	Info
keyValuePairSeparator	String	✓		Key value pair separator is used to split the values from their keys (mandatory). Can be <code>'\u0001'</code> , <code>'\u0009'</code> , <code>'#'</code> , or <code>'anything'</code> .
pairSeparator	String	✓		Pair separator used to split the key value pairs in tokens (mandatory). Can be <code>'='</code> , <code>','</code> , or <code>'anything'</code> .
crlf	String		WINDOWS	Character to be used to add a carriage return after each record (optional). Possible values = WINDOWS, UNIX, MAC, or custom. If you specify a value other than the three listed before, the value you enter (custom) will be used as the CRLF character(s).
isOrdered	boolean		false	Indicates if the message must be ordered in output. This annotation is associated to the message class of the model and must be declared one time.
name	String			Name describing the message (optional)
type	String		FIX	type is used to define the type of the message (e.g. FIX, EMX, ...) (optional)
version	String		4.1	version defines the version of the message (e.g. 4.1, ...) (optional)

case 1 : separator = `'\u0001'`

The separator used to segregate the key value pair fields in a FIX message is the ASCII **01** character or in unicode format **\u0001**. This character must be escaped a second time to avoid a java runtime error. Here is an example :

```
8=FIX.4.1 9=20 34=1 35=0 49=INVMGR 56=BRKR 1=BE.CHM.001 11=CHM0001-01 22=4 ...
```

and how to use the annotation:

FIX - message

```
@Message(keyValuePairSeparator = "=", pairSeparator = "\u0001", type="FIX", version="4.1")
public class Order {

}
```

Look at test cases

The ASCII character like tab, ... cannot be displayed in WIKI page. So, have a look to the test case of camel-bindy to see exactly how the FIX message looks like (<https://github.com/apache/camel/blob/main/components/camel-bindy/src/test/data/fix/fix.txt>) and the Order, Trailer, Header classes (<https://github.com/apache/camel/blob/main/components/camel-bindy/src/test/java/org/apache/camel/dataformat/bindy/model/fix/simple/Order.java>).

20.3.6. 6. KeyValuePairField

The KeyValuePairField annotation defines the property of a key value pair field. Each KeyValuePairField is identified by a tag (= key) and its value associated, a type (string, int, date, ...), optionally a pattern and if the field is required.

Annotation name	Record type	Level
KeyValuePairField	Key Value Pair - FIX	Property

Parameter name	Type	Required	Default value	Info
tag	int	✓		tag identifying the field in the message (mandatory) - must be unique
impliedDecimalSeparator	boolean		false	Camel 2.11: Indicates if there is a decimal point implied at a specified location
name	String			name of the field (optional)

Parameter name	Type	Required	Default value	Info
pattern	String			pattern that the formatter will use to transform the data (optional)
position	int		0	Position of the field in the message generated - must be used when the position of the key/tag in the FIX message must be different
precision	int		0	precision of the BigDecimal number to be created
required	boolean		false	Indicates if the field is mandatory
timezone	String			Timezone to be used.

case 1: tag

This parameter represents the key of the field in the message:

FIX message - Tag

```

@Message(keyValuePairSeparator = "=", pairSeparator = "\u0001", type="FIX", version="4.1")
public class Order {

    @Link Header header;

    @Link Trailer trailer;

    @KeyValuePairField(tag = 1) // Client reference
    private String Account;

    @KeyValuePairField(tag = 11) // Order reference
    private String ClOrdId;

    @KeyValuePairField(tag = 22) // Fund ID type (Sedol, ISIN, ...)
    private String IDSource;

    @KeyValuePairField(tag = 48) // Fund code
    private String SecurityId;

    @KeyValuePairField(tag = 54) // Movement type ( 1 = Buy, 2 = sell)
    private String Side;

    @KeyValuePairField(tag = 58) // Free text
    private String Text;
}

```

case 2 : Different position in output

If the tags/keys that we will put in the FIX message must be sorted according to a predefined order, then use the attribute **position** of the annotation **@KeyValuePairField**.

FIX message - Tag - sort

```
@Message(keyValuePairSeparator = "=", pairSeparator = "\\u0001", type = "FIX", version = "4.1",
isOrdered = true)
public class Order {

    @Link Header header;

    @Link Trailer trailer;

    @KeyValuePairField(tag = 1, position = 1) // Client reference
    private String account;

    @KeyValuePairField(tag = 11, position = 3) // Order reference
    private String clOrdId;
}
```

20.3.7. 7. Section

In FIX message of fixed length records, it is common to have different sections in the representation of the information : header, body and section. The purpose of the annotation **@Section** is to inform bindy about which class of the model represents the header (= section 1), body (= section 2) and footer (= section 3)

Only one attribute/parameter exists for this annotation.

Annotation name	Record type	Level
Section	FIX	Class

Parameter name	Type	Required	Default value	Info
number	int	✓		Number of the section

case 1 : Section

Definition of the header section:

FIX message - Section - Header

```
@Section(number = 1)
public class Header {
```

```

@KeyValuePairField(tag = 8, position = 1) // Message Header
private String beginString;

@KeyValuePairField(tag = 9, position = 2) // Checksum
private int bodyLength;
}

```

Definition of the body section:

FIX message - Section - Body

```

@section(number = 2)
@Message(keyValuePairSeparator = "=", pairSeparator = "\\u0001", type = "FIX", version = "4.1",
isOrdered = true)
public class Order {

    @Link Header header;

    @Link Trailer trailer;

    @KeyValuePairField(tag = 1, position = 1) // Client reference
    private String account;

    @KeyValuePairField(tag = 11, position = 3) // Order reference
    private String clOrdId;
}

```

Definition of the footer section:

FIX message - Section - Footer

```

@section(number = 3)
public class Trailer {

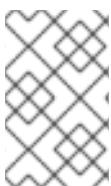
    @KeyValuePairField(tag = 10, position = 1)
    // CheckSum
    private int checkSum;

    public int getCheckSum() {
        return checkSum;
    }
}

```

20.3.8. 8. OneToMany

The purpose of the annotation **@OneToMany** is to allow to work with a **List<?>** field defined a POJO class or from a record containing repetitive groups.



NOTE

Restrictions for OneToMany

Be careful, the one to many of bindy does not allow to handle repetitions defined on several levels of the hierarchy.

The relation OneToMany ONLY WORKS in the following cases :

- Reading a FIX message containing repetitive groups (= group of tags/keys)
- Generating a CSV with repetitive data

Annotation name	Record type	Level
OneToMany	all	Property

Parameter name	Type	Required	Default value	Info
mappedTo	String			Class name associated to the type of the List<Type of the Class>

case 1: Generating CSV with repetitive data

Here is the CSV output that we want :

```
Claus,Ibsen,Camel in Action 1,2010,35
Claus,Ibsen,Camel in Action 2,2012,35
Claus,Ibsen,Camel in Action 3,2013,35
Claus,Ibsen,Camel in Action 4,2014,35
```



NOTE

The repetitive data concern the title of the book and its publication date while first, last name and age are common and the classes used to modeling this. The Author class contains a List of Book.

Generate CSV with repetitive data

```
@CsvRecord(separator=",")
public class Author {

    @DataField(pos = 1)
    private String firstName;

    @DataField(pos = 2)
    private String lastName;

    @OneToMany
    private List<Book> books;

    @DataField(pos = 5)
    private String Age;
}

public class Book {
```



```

    @DataField(pos = 3)
    private String title;

    @DataField(pos = 4)
    private String year;
}

```

case 2 : Reading FIX message containing group of tags/keys

Here is the message that we would like to process in our model :

```

8=FIX 4.19=2034=135=049=INVMGR56=BRKR
1=BE.CHM.00111=CHM0001-0158=this is a camel - bindy test
22=448=BE000124567854=1
22=548=BE000987654354=2
22=648=BE000999999954=3
10=220

```

Tags 22, 48 and 54 are repeated.

And the code:

Reading FIX message containing group of tags/keys

```

public class Order {

    @Link Header header;

    @Link Trailer trailer;

    @KeyValuePairField(tag = 1) // Client reference
    private String account;

    @KeyValuePairField(tag = 11) // Order reference
    private String clOrdId;

    @KeyValuePairField(tag = 58) // Free text
    private String text;

    @OneToMany(mappedTo =
"org.apache.camel.dataformat.bindy.model.fix.complex.onetomany.Security")
    List<Security> securities;
}

public class Security {

    @KeyValuePairField(tag = 22) // Fund ID type (Sedol, ISIN, ...)
    private String idSource;

    @KeyValuePairField(tag = 48) // Fund code
    private String securityCode;

    @KeyValuePairField(tag = 54) // Movement type ( 1 = Buy, 2 = sell)
    private String side;
}

```

20.3.9. 9. BindyConverter

The purpose of the annotation **@BindyConverter** is define a converter to be used on field level. The provided class must implement the `Format` interface.

```
@FixedLengthRecord(length = 10, paddingChar = ' ')
public static class DataModel {
    @DataField(pos = 1, length = 10, trim = true)
    @BindyConverter(CustomConverter.class)
    public String field1;
}

public static class CustomConverter implements Format<String> {
    @Override
    public String format(String object) throws Exception {
        return (new StringBuilder(object)).reverse().toString();
    }

    @Override
    public String parse(String string) throws Exception {
        return (new StringBuilder(string)).reverse().toString();
    }
}
```

20.3.10. 10. FormatFactories

The purpose of the annotation **@FormatFactories** is to define a set of converters at record-level. The provided classes must implement the **FormatFactoryInterface** interface.

```
@CsvRecord(separator = ",")
@FormatFactories({OrderNumberFormatFactory.class})
public static class Order {

    @DataField(pos = 1)
    private OrderNumber orderNr;

    @DataField(pos = 2)
    private String firstName;
}

public static class OrderNumber {
    private int orderNr;

    public static OrderNumber ofString(String orderNumber) {
        OrderNumber result = new OrderNumber();
        result.orderNr = Integer.valueOf(orderNumber);
        return result;
    }
}

public static class OrderNumberFormatFactory extends AbstractFormatFactory {

    {
        supportedClasses.add(OrderNumber.class);
    }
}
```

```

@Override
public Format<?> build(FormattingOptions formattingOptions) {
    return new Format<OrderNumber>() {
        @Override
        public String format(OrderNumber object) throws Exception {
            return String.valueOf(object.orderNr);
        }

        @Override
        public OrderNumber parse(String string) throws Exception {
            return OrderNumber.ofString(string);
        }
    };
}
}

```

20.4. SUPPORTED DATATYPES

The DefaultFormatFactory makes formatting of the following datatype available by returning an instance of the interface FormatFactoryInterface based on the provided FormattingOptions:

- BigDecimal
- BigInteger
- Boolean
- Byte
- Character
- Date
- Double
- Enums
- Float
- Integer
- LocalDate
- LocalDateTime
- LocalTime
- Long
- Short
- String

The DefaultFormatFactory can be overridden by providing an instance of FactoryRegistry in the registry in use (e.g. spring or JNDI).

20.5. USING THE JAVA DSL

The next step instantiates the DataFormat *bindy* class associated with this record type and providing a class as a parameter.

For example the following uses the class **BindyCsvDataFormat** (which corresponds to the class associated with the CSV record type) which is configured with *com.acme.model.MyModel.class* to initialize the model objects configured in this package.

```
DataFormat bindy = new BindyCsvDataFormat(com.acme.model.MyModel.class);
```

20.5.1. Setting locale

Bindy supports configuring the locale on the dataformat, such as

```
BindyCsvDataFormat bindy = new BindyCsvDataFormat(com.acme.model.MyModel.class);

bindy.setLocale("us");
```

Or to use the platform default locale then use "default" as the locale name.

```
BindyCsvDataFormat bindy = new BindyCsvDataFormat(com.acme.model.MyModel.class);

bindy.setLocale("default");
```

20.5.2. Unmarshaling

```
from("file://inbox")
  .unmarshal(bindy)
  .to("direct:handleOrders");
```

Alternatively, you can use a named reference to a data format which can then be defined in your Registry e.g. your Spring XML file:

```
from("file://inbox")
  .unmarshal("myBindyDataFormat")
  .to("direct:handleOrders");
```

The Camel route will pick-up files in the inbox directory, unmarshall CSV records into a collection of model objects and send the collection to the route referenced by **handleOrders**.

The collection returned is a **List of Map** objects. Each Map within the list contains the model objects that were marshalled out of each line of the CSV. The reason behind this is that *each line can correspond to more than one object*. This can be confusing when you simply expect one object to be returned per line.

Each object can be retrieve using its class name.

```
List<Map<String, Object>> unmarshaledModels = (List<Map<String, Object>>)
exchange.getIn().getBody();

int modelCount = 0;
```

```

for (Map<String, Object> model : unmarshaledModels) {
    for (String className : model.keySet()) {
        Object obj = model.get(className);
        LOG.info("Count : " + modelCount + ", " + obj.toString());
    }
    modelCount++;
}

LOG.info("Total CSV records received by the csv bean : " + modelCount);

```

Assuming that you want to extract a single Order object from this map for processing in a route, you could use a combination of a Splitter and a Processor as per the following:

```

from("file://inbox")
    .unmarshal(bindy)
    .split(body())
    .process(new Processor() {
        public void process(Exchange exchange) throws Exception {
            Message in = exchange.getIn();
            Map<String, Object> modelMap = (Map<String, Object>) in.getBody();
            in.setBody(modelMap.get(Order.class.getCanonicalName()));
        }
    })
    .to("direct:handleSingleOrder")
    .end();

```

Take care of the fact that Bindy uses CHARSET_NAME property or the CHARSET_NAME header as define in the Exchange interface to do a charset conversion of the inputstream received for unmarshalling. In some producers (e.g. file-endpoint) you can define a charset. The charset conversion can already been done by this producer. Sometimes you need to remove this property or header from the exchange before sending it to the unmarshal. If you don't remove it the conversion might be done twice which might lead to unwanted results.

```

from("file://inbox?charset=Cp922")
    .removeProperty(Exchange.CHARSET_NAME)
    .unmarshal("myBindyDataFormat")
    .to("direct:handleOrders");

```

20.5.3. Marshaling

To generate CSV records from a collection of model objects, you create the following route :

```

from("direct:handleOrders")
    .marshal(bindy)
    .to("file://outbox")

```

20.6. USING SPRING XML

This is really easy to use Spring as your favorite DSL language to declare the routes to be used for camel-bindy. The following example shows two routes where the first will pick-up records from files, unmarshal the content and bind it to their model. The result is then send to a pojo (doing nothing special) and place them into a queue.

The second route will extract the pojos from the queue and marshal the content to generate a file containing the CSV record.

Spring DSL

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://camel.apache.org/schema/spring
    http://camel.apache.org/schema/spring/camel-spring.xsd">

  <!-- Queuing engine - ActiveMq - work locally in mode virtual memory -->
  <bean id="activemq" class="org.apache.activemq.camel.component.ActiveMQComponent">
    <property name="brokerURL" value="vm://localhost:61616"/>
  </bean>

  <camelContext xmlns="http://camel.apache.org/schema/spring">
    <dataFormats>
      <bindy id="bindyDataformat" type="Csv" classType="org.apache.camel.bindy.model.Order"/>
    </dataFormats>

    <route>
      <from uri="file://src/data/csv/?noop=true" />
      <unmarshal ref="bindyDataformat" />
      <to uri="bean:csv" />
      <to uri="activemq:queue:in" />
    </route>

    <route>
      <from uri="activemq:queue:in" />
      <marshal ref="bindyDataformat" />
      <to uri="file://src/data/csv/out/" />
    </route>
  </camelContext>
</beans>
```



NOTE

Please verify that your model classes implements serializable otherwise the queue manager will raise an error.

20.7. DEPENDENCIES

To use Bindy in your camel routes you need to add the a dependency on **camel-bindy** which implements this data format.

If you use maven you could just add the following to your pom.xml, substituting the version number for the latest & greatest release (see the download page for the latest versions).

```
<dependency>
  <groupId>org.apache.camel</groupId>
```

```

<artifactId>camel-bindy</artifactId>
<version>{CamelSBVersion}</version>
</dependency>

```

20.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 18 options, which are listed below.

Name	Description	Default	Type
<code>camel.dataformat.bindy-csv.allow-empty-stream</code>	Whether to allow empty streams in the unmarshal process. If true, no exception will be thrown when a body without records is provided.	false	Boolean
<code>camel.dataformat.bindy-csv.class-type</code>	Name of model class to use.		String
<code>camel.dataformat.bindy-csv.enabled</code>	Whether to enable auto configuration of the bindy-csv data format. This is enabled by default.		Boolean
<code>camel.dataformat.bindy-csv.locale</code>	To configure a default locale to use, such as us for united states. To use the JVM platform default locale then use the name default.		String
<code>camel.dataformat.bindy-csv.type</code>	Whether to use Csv, Fixed, or KeyValue.		String
<code>camel.dataformat.bindy-csv.unwrap-single-instance</code>	When unmarshalling should a single instance be unwrapped and returned instead of wrapped in a <code>java.util.List</code> .	true	Boolean
<code>camel.dataformat.bindy-fixed.allow-empty-stream</code>	Whether to allow empty streams in the unmarshal process. If true, no exception will be thrown when a body without records is provided.	false	Boolean
<code>camel.dataformat.bindy-fixed.class-type</code>	Name of model class to use.		String
<code>camel.dataformat.bindy-fixed.enabled</code>	Whether to enable auto configuration of the bindy-fixed data format. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.dataformat.bindy-fixed.locale</code>	To configure a default locale to use, such as us for united states. To use the JVM platform default locale then use the name default.		String
<code>camel.dataformat.bindy-fixed.type</code>	Whether to use Csv, Fixed, or KeyValue.		String
<code>camel.dataformat.bindy-fixed.unwrap-single-instance</code>	When unmarshalling should a single instance be unwrapped and returned instead of wrapped in a <code>java.util.List</code> .	true	Boolean
<code>camel.dataformat.bindy-kvp.allow-empty-stream</code>	Whether to allow empty streams in the unmarshal process. If true, no exception will be thrown when a body without records is provided.	false	Boolean
<code>camel.dataformat.bindy-kvp.class-type</code>	Name of model class to use.		String
<code>camel.dataformat.bindy-kvp.enabled</code>	Whether to enable auto configuration of the bindy-kvp data format. This is enabled by default.		Boolean
<code>camel.dataformat.bindy-kvp.locale</code>	To configure a default locale to use, such as us for united states. To use the JVM platform default locale then use the name default.		String
<code>camel.dataformat.bindy-kvp.type</code>	Whether to use Csv, Fixed, or KeyValue.		String
<code>camel.dataformat.bindy-kvp.unwrap-single-instance</code>	When unmarshalling should a single instance be unwrapped and returned instead of wrapped in a <code>java.util.List</code> .	true	Boolean

CHAPTER 21. BROWSE

Both producer and consumer are supported

The Browse component provides a simple `BrowsableEndpoint` which can be useful for testing, visualisation tools or debugging. The exchanges sent to the endpoint are all available to be browsed.

21.1. DEPENDENCIES

When using **browse** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-browse-starter</artifactId>
</dependency>
```

21.2. URI FORMAT

```
browse:someName[?options]
```

Where **someName** can be any string to uniquely identify the endpoint.

21.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

21.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

21.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

21.4. COMPONENT OPTIONS

The Browse component supports 3 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

21.5. ENDPOINT OPTIONS

The Browse endpoint is configured using URI syntax:

```
browse:name
```

with the following path and query parameters:

21.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (common)	Required A name which can be any string to uniquely identify the endpoint.		String

21.5.2. Query Parameters (4 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

21.6. SAMPLE

In the route below, we insert a **browse:** component to be able to browse the Exchanges that are passing through:

```
from("activemq:order.in").to("browse:orderReceived").to("bean:processOrder");
```

We can now inspect the received exchanges from within the Java code:

```
private CamelContext context;

public void inspectReceivedOrders() {
    BrowseableEndpoint browse = context.getEndpoint("browse:orderReceived",
BrowseableEndpoint.class);
    List<Exchange> exchanges = browse.getExchanges();

    // then we can inspect the list of received exchanges from Java
    for (Exchange exchange : exchanges) {
        String payload = exchange.getIn().getBody();
        // do something with payload
    }
}
```

21.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

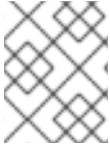
Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
camel.component.browse.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.browse.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.browse.enabled	Whether to enable auto configuration of the browse component. This is enabled by default.		Boolean
camel.component.browse.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 22. CASSANDRA CQL

Both producer and consumer are supported

[Apache Cassandra](#) is an open source NoSQL database designed to handle large amounts on commodity hardware. Like Amazon's DynamoDB, Cassandra has a peer-to-peer and master-less architecture to avoid single point of failure and garanty high availability. Like Google's BigTable, Cassandra data is structured using column families which can be accessed through the Thrift RPC API or a SQL-like API called CQL.



NOTE

This component aims at integrating Cassandra 2.0+ using the CQL3 API (not the Thrift API). It's based on [Cassandra Java Driver](#) provided by DataStax.

22.1. DEPENDENCIES

When using **cql** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-cassandraql-starter</artifactId>
</dependency>
```

22.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

22.2.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

22.2.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

22.3. COMPONENT OPTIONS

The Cassandra CQL component supports 3 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

22.4. ENDPOINT OPTIONS

The Cassandra CQL endpoint is configured using URI syntax:

```
cql:beanRef:hosts:port/keyspace
```

with the following path and query parameters:

22.4.1. Path Parameters (4 parameters)

Name	Description	Default	Type
beanRef (common)	beanRef is defined using bean:id.		String
hosts (common)	Hostname(s) Cassandra server(s). Multiple hosts can be separated by comma.		String
port (common)	Port number of Cassandra server(s).		Integer
keyspace (common)	Keyspace to use.		String

22.4.2. Query Parameters (30 parameters)

Name	Description	Default	Type
clusterName (common)	Cluster name.		String

Name	Description	Default	Type
consistencyLevel (common)	Consistency level to use. Enum values: • ANY • ONE • TWO • THREE • QUORUM • ALL • LOCAL_ONE • LOCAL_QUORUM • EACH_QUORUM • SERIAL • LOCAL_SERIAL		DefaultConsistencyLevel
cql (common)	CQL query to perform. Can be overridden with the message header with key CamelCqlQuery.		String
datacenter (common)	Datacenter to use.	datacenter1	String
loadBalancingPolicyClass (common)	To use a specific LoadBalancingPolicyClass.		String
password (common)	Password for session authentication.		String
prepareStatements (common)	Whether to use PreparedStatements or regular Statements.	true	boolean
resultSetConversionStrategy (common)	To use a custom class that implements logic for converting ResultSet into message body ALL, ONE, LIMIT_10, LIMIT_100...		ResultSetConversionStrategy
session (common)	To use the Session instance (you would normally not use this option).		CqlSession
username (common)	Username for session authentication.		String

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code> ● <code>InOptionalOut</code>		<code>ExchangePattern</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long

Name	Description	Default	Type
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit

Name	Description	Default	Type
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See <code>ScheduledExecutorService</code> in JDK for details.	true	boolean

22.5. ENDPOINT CONNECTION SYNTAX

The endpoint can initiate the Cassandra connection or use an existing one.

URI	Description
cql:localhost/keyspace	Single host, default port, usual for testing
cql:host1,host2/keyspace	Multi host, default port
cql:host1,host2:9042/keyspace	Multi host, custom port
cql:host1,host2	Default port and keyspace
cql:bean:sessionRef	Provided Session reference
cql:bean:clusterRef/keyspace	Provided Cluster reference

To fine tune the Cassandra connection (SSL options, pooling options, load balancing policy, retry policy, reconnection policy...), create your own Cluster instance and give it to the Camel endpoint.

22.6. MESSAGES

22.6.1. Incoming Message

The Camel Cassandra endpoint expects a bunch of simple objects (**Object** or **Object[]** or **Collection<Object>**) which will be bound to the CQL statement as query parameters. If message body is null or empty, then CQL query will be executed without binding parameters.

Headers

- **CamelCqlQuery** (optional, **String** or **RegularStatement**)
CQL query either as a plain String or built using the **QueryBuilder**.

22.6.2. Outgoing Message

The Camel Cassandra endpoint produces one or many a Cassandra Row objects depending on the **resultSetConversionStrategy**:

- **List<Row>** if **resultSetConversionStrategy** is **ALL** or **LIMIT_[0-9]+**
- **Single`Row`** if **resultSetConversionStrategy** is **ONE**

- Anything else, if **resultSetConversionStrategy** is a custom implementation of the **ResultSetConversionStrategy**

22.7. REPOSITORIES

Cassandra can be used to store message keys or messages for the idempotent and aggregation EIP.

Cassandra might not be the best tool for queuing use cases yet, read [Cassandra anti-patterns queues and queue like datasets](#). It's advised to use LeveledCompaction and a small GC grace setting for these tables to allow tombstoned rows to be removed quickly.

22.8. IDEMPOTENT REPOSITORY

The **NamedCassandraIdempotentRepository** stores messages keys in a Cassandra table like this:

CAMEL_IDEMPOTENT.cql

```
CREATE TABLE CAMEL_IDEMPOTENT (
  NAME varchar, -- Repository name
  KEY varchar, -- Message key
  PRIMARY KEY (NAME, KEY)
) WITH compaction = {'class':'LeveledCompactionStrategy'}
AND gc_grace_seconds = 86400;
```

This repository implementation uses lightweight transactions (also known as Compare and Set) and requires Cassandra 2.0.7+.

Alternatively, the **CassandraIdempotentRepository** does not have a **NAME** column and can be extended to use a different data model.

Option	Default	Description
table	CAMEL_IDEMPOTENT	Table name
pkColumns	NAME, `KEY`	Primary key columns
name		Repository name, value used for NAME column
ttl		Key time to live
writeConsistencyLevel		Consistency level used to insert/delete key: ANY, ONE, TWO, QUORUM, LOCAL_QUORUM...
readConsistencyLevel		Consistency level used to read/check key: ONE, TWO, QUORUM, LOCAL_QUORUM...

22.9. AGGREGATION REPOSITORY

The **NamedCassandraAggregationRepository** stores exchanges by correlation key in a Cassandra table like this:

CAMEL_AGGREGATION.cql

```
CREATE TABLE CAMEL_AGGREGATION (
  NAME varchar,      -- Repository name
  KEY varchar,       -- Correlation id
  EXCHANGE_ID varchar, -- Exchange id
  EXCHANGE blob,     -- Serialized exchange
  PRIMARY KEY (NAME, KEY)
) WITH compaction = {'class':'LeveledCompactionStrategy'}
AND gc_grace_seconds = 86400;
```

Alternatively, the **CassandraAggregationRepository** does not have a **NAME** column and can be extended to use a different data model.

Option	Default	Description
table	CAMEL_AGGREGATION	Table name
pkColumns	NAME,KEY	Primary key columns
exchangeIdColumn	EXCHANGE_ID	Exchange Id column
exchangeColumn	EXCHANGE	Exchange content column
name		Repository name, value used for NAME column
ttl		Exchange time to live
writeConsistencyLevel		Consistency level used to insert/delete exchange: ANY, ONE, TWO, QUORUM, LOCAL_QUORUM...
readConsistencyLevel		Consistency level used to read/check exchange: ONE, TWO, QUORUM, LOCAL_QUORUM...

22.10. EXAMPLES

To insert something on a table you can use the following code:

```
String CQL = "insert into camel_user(login, first_name, last_name) values (?, ?, ?);
from(\"direct:input\")
```

```
.to("cql://localhost/camel_ks?cql=" + CQL);
```

At this point you should be able to insert data by using a list as body

```
Arrays.asList("davsclaus", "Claus", "Ibsen")
```

The same approach can be used for updating or querying the table.

22.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.component.cql.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.cql.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.cql.enabled	Whether to enable auto configuration of the cql component. This is enabled by default.		Boolean
camel.component.cql.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 23. CICS

Since Camel 4.4-redhat

Only producer is supported.

This component allows you to interact with the IBM [CICS®](#) general-purpose transaction processing subsystem.



NOTE

Only synchronous mode call are supported.

23.1. DEPENDENCIES

When using **camel-cics** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-cics-starter</artifactId>
</dependency>
```

You must also declare the **ctgclient.jar** dependency when working with **camel-cics** starter.

```
<dependency>
  <artifactId>com.ibm</artifactId>
  <groupId>ctgclient</groupId>
  <scope>system</scope>
  <systemPath>${basedir}/lib/ctgclient.jar</systemPath>
</dependency>
```

This JAR is provided by IBM and is included in the **cics** system.

23.2. URI FORMAT

```
cics://[interfaceType]/[dataExchangeType][?options]
```

Where **interfaceType** is the CICS set of the external API that the **camel-cics** invokes. At the moment, only ECI (External Call Interface) is supported. This component communicates with the CICS server using two kinds of **dataExchangeType**.

- **commarea** is a block of storage, limited to 32763 bytes, allocated by the program.
- **channel** is the new mechanism for exchanging data, analogous to a parameter list.

By default, if **dataExchangeType** is not specified, this component uses **commarea**:

```
cics://eci?host=xxx&port=xxx...
```

To use the **channel** and the container you must specify it explicitly in the URI

```
cics://eci/channel?host=xxx&port=xxx...
```

23.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

23.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

23.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

23.4. COMPONENT OPTIONS

The CICS component supports 17 options, which are listed below.

Name	Description	Default	Type
ctgDebug	Enable debug mode on the underlying IBM CGT client.	false	java.lang.Boolean
eciBinding	The Binding instance to transform a Camel Exchange to EciRequest and vice versa		com.redhat.camel.component.cics.CICSEciBinding
eciTimeout	The ECI timeout value associated with this ECIRequest object. An ECI timeout value of zero indicates that this ECIRequest will not be timed out by CICS Transaction Gateway. An ECI timeout value greater than zero indicates that the ECIRequest may be timed out by CICS Transaction Gateway. ECI timeout can expire before a response is received from CICS. This means that the client does not receive the confirmation from CICS that a unit of work has been backed out or committed.	0	short
encoding	The transfer encoding of the message.	Cp1145	java.lang.String
gatewayFactory	The connection factory to be used		com.redhat.camel.component.cics.pool.CICSGatewayFactory
host	The address of the CICS Transaction Gateway that this instance connects to		java.lang.String
lazyStartProducer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.		boolean
port	The port of the CICS Transaction Gateway that this instance connects to.	2006	int
protocol	the protocol that this component will use to connect to the CICS Transaction Gateway.	tcp	java.lang.String
server	The address of the CICS server that this instance connects to.		java.lang.String

Name	Description	Default	Type
sslKeyring	The full classname of the SSL key ring class or keystore file to be used for the client encrypted connection.		java.lang.String
sslPassword	The password for the encrypted key ring class or keystore		java.lang.String
configuration	To use a shared CICS configuration		com.redhat.camel.component.cics.CICSCSConfiguration
socketConnectionTimeout	The socket connection timeout		int
password	Password to use for authentication		java.lang.String
userId	User ID to use for authentication		java.lang.String

23.5. ENDPOINT OPTIONS

The CICS endpoint is configured using URI syntax:

```
cics://[interfaceType]/[dataExchangeType][?options]
```

With the following *path* and *query* parameters:

23.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
interfaceType	The interface type, can be eci, esi or epi. at the moment only eci is supported.	eci	java.lang.String
dataExchangeType	The kind of data exchange to use Enum value: - commarea - channel	commarea	com.redhat.camel.component.cics.support.CICSDataExchangeType

23.5.2. Query Parameters (15 parameters)

Name	Description	Default	Type
ctgDebug	Enable debug mode on the underlying IBM CGT client.	false	java.lang.Boolean
eciBinding	The Binding instance to transform a Camel Exchange to EciRequest and vice versa		com.redhat.camel.component.cics.CICSEciBinding
eciTimeout	The ECI timeout value associated with this ECIRequest object. An ECI timeout value of zero indicates that this ECIRequest will not be timed out by CICS Transaction Gateway. An ECI timeout value greater than zero indicates that the ECIRequest may be timed out by CICS Transaction Gateway. ECI timeout can expire before a response is received from CICS. This means that the client does not receive the confirmation from CICS that a unit of work has been backed out or committed.	0	short
encoding	Encoding to convert COMMAREA data to before sending.	Cp1145	java.lang.String
gatewayFactory	The connection factory to use		com.redhat.camel.component.cics.pool.CICSGatewayFactory
host	The address of the CICS Transaction Gateway that this instance connects to	localhost	java.lang.String
port	The port of the CICS Transaction Gateway that this instance connects to.	2006	int
protocol	the protocol that this component will use to connect to the CICS Transaction Gateway.	tcp	java.lang.String
server	The address of the CICS server that this instance connects to		java.lang.String

Name	Description	Default	Type
lazyStartProducer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.		boolean
sslKeyring	The full class name of the SSL key ring class or keystore file to be used for the client encrypted connection		java.lang.String
sslPassword	The password for the encrypted key ring class or keystore		java.lang.String
socketConnectionTimeout	The socket connection timeout		int
password	Password to use for authentication		java.lang.String
userId	User ID to use for authentication		java.lang.String

23.6. MESSAGE HEADERS

The CICS component supports 15 message header(s), which is/are listed below:

Name	Description	Default	Type
CICS_RETURN_CODE Constant: com.redhat.camel.component.cics.CICSConstants#CICS_RETURN_CODE_HEADER	Return code from this flow operation.		int

Name	Description	Default	Type
CICS_RETURN_CODE_STRING Constant: com.redhat.camel.component.cics.CICSCSConstants#CICS_RETURN_CODE_STRING_HEADER	The CICS return code as a String. The String is the name of the appropriate Java constant, for example, if this header is ECI_NO_ERROR, then the String returned will be ECI_NO_ERROR. If this header is unknown then the String returned will be ECI_UNKNOWN_CICS_RC.  NOTE For CICS return codes that may have more than one meaning the String returned is a concatenation of the return codes. The only concatenated String is: ECI_ERR_REQUEST_TIMEOUT_OR_ERR_NO_REPLY.		java.lang.String
CICS_EXTEND_MODE Constant: com.redhat.camel.component.cics.CICSCSConstants#CICS_EXTEND_MODE_HEADER	Extend mode of request. The default value is ECI_NO_EXTEND.		int
CICS_LUW_TOKEN Constant: com.redhat.camel.component.cics.CICSCSConstants#CICS_LUW_TOKEN_HEADER	Extended Logical Unit of Work token. The default value is ECI_LUW_NEW.		int
CICS_PROGRAM_NAME Constant: com.redhat.camel.component.cics.CICSCSConstants#CICS_PROGRAM_NAME_HEADER	Program to invoke on CICS server.		java.lang.String

Name	Description	Default	Type
CICS_TRANSACTION_ID Constant: com.redhat.camel. component.cics.CI CSCConstants#CIC S_TRANSACTION _ID_HEADER	Transaction ID to run CICS program under.		java.lang.String
CICS_COMMAREA_SIZE Constant: com.redhat.camel. component.cics.CI CSCConstants#CIC S_COMM_AREA_S IZE_HEADER	Length of COMMAREA. The default value is 0.		int
CICS_CHANNEL_NAME Constant: com.redhat.camel. component.cics.CI CSCConstants#CIC S_CHANNEL_NA ME_HEADER	The name of the channel to create com.redhat.camel.component.cics.CICSCConstants#C ICS_CHANNEL_NAME_HEADER		java.lang.String
CICS_CONTAINER_NAME Constant: com.redhat.camel. component.cics.CI CSCConstants#CIC S_CONTAINER_N AME_HEADER	The name of the container to create.		java.lang.String
CICS_CHANNEL_CCSID Constant: com.redhat.camel. component.cics.CI CSCConstants#CIC S_CHANNEL_CCS ID_HEADER	The CCSID the channel should set as its default.		int

Name	Description	Default	Type
CICS_SERVER Constant: com.redhat.camel. component.cics.CI CSConstants#CIC S_SERVER_HEAD ER	CICS server to direct request to. This header overrides the value configured in the endpoint.		java.lang.String
CICS_USER_ID Constant: com.redhat.camel. component.cics.CI CSConstants#CIC S_USER_ID_HEAD ER	User ID for CICS server. This header overrides the value configured in the endpoint.		java.lang.String
CICS_PASSWORD Constant: com.redhat.camel. component.cics.CI CSConstants#CIC S_PASSWORD_HE ADER	Password or password phrase for CICS server. This header overrides the value configured in the endpoint.		java.lang.String
CICS_ABEND_CO DE Constant: com.redhat.camel. component.cics.CI CSConstants#CIC S_ABEND_CODE_ HEADER	CICS transaction abend code.		java.lang.String
CICS_ECI_REQUE ST_TIMEOUT Constant: com.redhat.camel. component.cics.CI CSConstants#CIC S_ECI_REQUEST_ TIMEOUT_HEADE R	The value, in seconds, of the ECI timeout for the current ECIRRequest. A value of zero indicates that this ECIRRequest will not be timed out by CICS Transaction Gateway	0	short

Name	Description	Default	Type
CICS_ENCODING Constant: com.redhat.camel. component.cics.CI CSConstants#CIC S_ENCODING_HE ADER	Encoding to convert COMMAREA data to before sending.		String

23.7. SAMPLES

23.7.1. Using Commarea

Following sample show how to configure a route that runs a program on a CICS server using COMMAREA. The COMMAREA size has to be defined in **CICS_COMM_AREA_SIZE** header, while the COMMAREA input data is defined in the Camel Exchange body.



NOTE

You must create a COMMAREA that is large enough to contain all the information to be sent to the server and large enough to contain all the information that can be returned from the server.

```
//....
import static com.redhat.camel.component.cics.CICSConstants.CICS_PROGRAM_NAME_HEADER;
import static
com.redhat.camel.component.cics.CICSConstants.CICS_COMM_AREA_SIZE_HEADER;
//....

from("direct:run").
    setHeader(CICS_PROGRAM_NAME_HEADER, "ECIREADY").
    setHeader(CICS_COMM_AREA_SIZE_HEADER, 18).
    setBody(constant("My input data")).
    to("cics:eci/commarea?host=192.168.0.23&port=2006&protocol=tcp&userId=foo&password=bar");
```

The Outcome of the CICS program invocation is mapped to Camel Exchange in this way:

- The numeric value of return code is stored in the **CICS_RETURN_CODE** header
- The COMMAREA output data is stored in the Camel Exchange Body.

23.7.2. Using Channel with a single input container

Following sample shows how to use a channel with a single container to run a CICS program. The channel name and the container name are taken from headers, and the container value from the body:

```
//....
import static com.redhat.camel.component.cics.CICSConstants.CICS_PROGRAM_NAME_HEADER;
import static com.redhat.camel.component.cics.CICSConstants.CICS_CHANNEL_NAME_HEADER;
```

```
import static
com.redhat.camel.component.cics.CICSConstants.CICS_CONTAINER_NAME_HEADER;

//...
from("direct:run").
    setHeader(CICS_PROGRAM_NAME_HEADER, "EC03").
    setHeader(CICS_CHANNEL_NAME_HEADER, "SAMPLECHANNEL").
    setHeader(CICS_CONTAINER_NAME_HEADER, "INPUTDATA").
    setBody(constant("My input data")).
    to("cics:eci/channel?host=192.168.0.23&port=2006&protocol=tcp&userId=foo&password=bar");
```

The container(s) returned is stored in an **java.util.Map<String, Object>**, the key is the container name and the value is the output data of the container.

23.7.3. Using Channel with multiple input container

If you need to run a CICS program that takes multiple container as input, you can create a **java.util.Map<String, Object>** where the keys are the container names and the values are the input data. In this case the **CICS_CONTAINER_NAME** header is ignored.

```
//....
import static com.redhat.camel.component.cics.CICSConstants.CICS_PROGRAM_NAME_HEADER;
import static com.redhat.camel.component.cics.CICSConstants.CICS_CHANNEL_NAME_HEADER;

//...
from("direct:run").
    setHeader(CICS_PROGRAM_NAME_HEADER, "EC03").
    setHeader(CICS_CHANNEL_NAME_HEADER, "SAMPLECHANNEL").
    process(exchange->{
        byte[] thirdContainerData = HexFormat.of().parseHex("e04fd020ea3a6910a2d808002b30309d");
        Map<String, Object> containers = Map.of(
            "firstContainerName", "firstContainerData",
            "secondContainerName", "secondContainerData",
            "thirdContainerName", thirdContainerData
        );
        exchange.getMessage().setBody(containers);
    }).
    to("cics:eci/channel?host=192.168.0.23&port=2006&protocol=tcp&userId=foo&password=bar");
```

23.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 17 options, which are listed below.

Name	Description	Default	Type
camel.component.cics.binding	The Binding instance to transform a Camel Exchange to EciRequest and vice versa.		com.redhat.camel.component.cics.CICSEciBinding

Name	Description	Default	Type
camel.component.cics.configuration	Configuration.		com.redhat.camel.component.cics.CICSConfiguration
camel.component.cics.ctg-debug	Enable debug mode on the underlying IBM CGT client.		java.lang.Boolean
camel.component.cics.eci-timeout	The ECI timeout value associated with this ECIRequest object. An ECI timeout value of zero indicates that this ECIRequest will not be timed out by CICS Transaction Gateway. An ECI timeout value greater than zero indicates that the ECIRequest may be timed out by CICS Transaction Gateway. ECI timeout can expire before a response is received from CICS. This means that the client does not receive the confirmation from CICS that a unit of work has been backed out or committed.		java.lang.Short
camel.component.cics.enabled	Whether to enable auto configuration of the cics component. This is enabled by default.		java.lang.Boolean
camel.component.cics.encoding	The transfer encoding of the message.	Cp1145	java.lang.String
camel.component.cics.gateway-factory	The connection factory to be use. The option is a com.redhat.camel.component.cics.pool.CICSGatewayFactory type.		com.redhat.camel.component.cics.pool.CICSGatewayFactory
camel.component.cics.host	The address of the CICS Transaction Gateway that this instance connects to		java.lang.String
camel.component.cics.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.		java.lang.Boolean
camel.component.cics.password	Password to use for authentication		java.lang.String

Name	Description	Default	Type
camel.component.cics.port	The port of the CICS Transaction Gateway that this instance connects to.	2006	java.lang.Integer
camel.component.cics.protocol	the protocol that this component will use to connect to the CICS Transaction Gateway.	tcp	java.lang.String
camel.component.cics.server	The address of the CICS server that this instance connects to		java.lang.String
camel.component.cics.socket-connection-timeout	The socket connection timeout		java.lang.Integer
camel.component.cics.ssl-keyring	The full classname of the SSL key ring class or keystore file to be used for the client encrypted connection		java.lang.String
camel.component.cics.ssl-password	The password for the encrypted key ring class or keystore		java.lang.String
camel.component.cics.user-id	User ID to use for authentication		java.lang.String

CHAPTER 24. CONSTANT

The Constant Expression Language is really just a way to use a constant value or object.



NOTE

This is a fixed constant value (or object) that is only set once during starting up the route, do not use this if you want dynamic values during routing.

24.1. DEPENDENCIES

When using **constant** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-core-starter</artifactId>
</dependency>
```

24.2. CONSTANT OPTIONS

The Constant language supports 2 options, which are listed below.

Name	Default	Java Type	Description
resultType		String	Sets the class name of the constant type.
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

24.3. EXAMPLE

The **setHeader** EIP can utilize a constant expression like:

```
<route>
  <from uri="seda:a"/>
  <setHeader name="theHeader">
    <constant>the value</constant>
  </setHeader>
  <to uri="mock:b"/>
</route>
```

in this case, the message coming from the seda:a endpoint will have the header with key **theHeader** set its value as **the value** (string type).

And the same example using Java DSL:

```
from("seda:a")
  .setHeader("theHeader", constant("the value"))
  .to("mock:b");
```

24.3.1. Specifying type of value

The option **resultType** can be used to specify the type of the value, when the value is given as a **String** value, which happens when using XML or YAML DSL:

For example to set a header with **int** type you can do:

```
<route>
  <from uri="seda:a"/>
  <setHeader name="zipCode">
    <constant resultType="int">90210</constant>
  </setHeader>
  <to uri="mock:b"/>
</route>
```

24.4. LOADING CONSTANT FROM EXTERNAL RESOURCE

You can externalize the constant and have Camel load it from a resource such as "**classpath:**", "**file:**", or "**http:**".

This is done using the following syntax: "**resource:scheme:location**", eg to refer to a file on the classpath you can do:

```
.setHeader("myHeader").constant("resource:classpath:constant.txt")
```

24.5. DEPENDENCIES

The Constant language is part of **camel-core**.

24.6. SPRING BOOT AUTO-CONFIGURATION

The component supports 147 options, which are listed below.

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.acl-token</code>	Sets the ACL token to be used with Consul.		String
<code>camel.cloud.consul.service-discovery.block-seconds</code>	The seconds to wait for a watch event, default 10 seconds.	10	Integer

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.consul.service-discovery.connect-timeout-millis</code>	Connect timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.datacenter</code>	The data center.		String
<code>camel.cloud.consul.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.consul.service-discovery.password</code>	Sets the password to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.consul.service-discovery.read-timeout-millis</code>	Read timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.url</code>	The Consul agent URL.		String
<code>camel.cloud.consul.service-discovery.username</code>	Sets the username to be used for basic authentication.		String

Name	Description	Default	Type
<code>camel.cloud.console.service-discovery.write-timeout-millis</code>	Write timeout for OkHttpClient.		Long
<code>camel.cloud.dns.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.dns.service-discovery.domain</code>	The domain name;.		String
<code>camel.cloud.dns.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.dns.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.dns.service-discovery.proto</code>	The transport protocol of the desired service.	<code>_tcp</code>	String
<code>camel.cloud.etcd.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.etcd.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.etcd.service-discovery.password</code>	The password to use for basic authentication.		String
<code>camel.cloud.etcd.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map

Name	Description	Default	Type
<code>camel.cloud.etcd.service-discovery.service-path</code>	The path to look for for service discovery.	<code>/services/</code>	String
<code>camel.cloud.etcd.service-discovery.timeout</code>	To set the maximum time an action could take to complete.		Long
<code>camel.cloud.etcd.service-discovery.type</code>	To set the discovery type, valid values are on-demand and watch.	on-demand	String
<code>camel.cloud.etcd.service-discovery.uris</code>	The URIs the client can connect to.		String
<code>camel.cloud.etcd.service-discovery.user-name</code>	The user name to use for basic authentication.		String
<code>camel.cloud.kubernetes.service-discovery.api-version</code>	Sets the API version when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-data</code>	Sets the Certificate Authority data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-file</code>	Sets the Certificate Authority data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-data</code>	Sets the Client Certificate data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-file</code>	Sets the Client Certificate data that are loaded from the file when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.client-key-algo</code>	Sets the Client Keystore algorithm, such as RSA when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-data</code>	Sets the Client Keystore data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-file</code>	Sets the Client Keystore data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-passphrase</code>	Sets the Client Keystore passphrase when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.kubernetes.service-discovery.dns-domain</code>	Sets the DNS domain to use for DNS lookup.		String
<code>camel.cloud.kubernetes.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.kubernetes.service-discovery.lookup</code>	How to perform service lookup. Possible values: client, dns, environment. When using client, then the client queries the kubernetes master to obtain a list of active pods that provides the service, and then random (or round robin) select a pod. When using dns the service name is resolved as name.namespace.svc.dnsDomain. When using dnssrv the service name is resolved with SRV query forsvc... When using environment then environment variables are used to lookup the service. By default environment is used.	environment	String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.master-url</code>	Sets the URL to the master when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.namespace</code>	Sets the namespace to use. Will by default use namespace from the ENV variable KUBERNETES_MASTER.		String
<code>camel.cloud.kubernetes.service-discovery.oauth-token</code>	Sets the OAUTH token for authentication (instead of username/password) when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.password</code>	Sets the password for authentication when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-name</code>	Sets the Port Name to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-protocol</code>	Sets the Port Protocol to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.kubernetes.service-discovery.trust-certs</code>	Sets whether to turn on trust certificate check when using client lookup.	false	Boolean
<code>camel.cloud.kubernetes.service-discovery.username</code>	Sets the username for authentication when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.ribbon.load-balancer.client-name</code>	Sets the Ribbon client name.		String
<code>camel.cloud.ribbon.load-balancer.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.ribbon.load-balancer.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.ribbon.load-balancer.namespace</code>	The namespace.		String
<code>camel.cloud.ribbon.load-balancer.password</code>	The password.		String
<code>camel.cloud.ribbon.load-balancer.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.ribbon.load-balancer.username</code>	The username.		String
<code>camel.hystrix.allow-maximum-size-to-diverge-from-core-size</code>	Allows the configuration for <code>maximumSize</code> to take effect. That value can then be equal to, or higher, than <code>coreSize</code> .	false	Boolean
<code>camel.hystrix.circuit-breaker-enabled</code>	Whether to use a <code>HystrixCircuitBreaker</code> or not. If false no circuit-breaker logic will be used and all requests permitted. This is similar in effect to <code>circuitBreakerForceClosed()</code> except that continues tracking metrics and knowing whether it should be open/closed, this property results in not even instantiating a circuit-breaker.	true	Boolean

Name	Description	Default	Type
camel.hystrix.circuit-breaker-error-threshold-percentage	Error percentage threshold (as whole number such as 50) at which point the circuit breaker will trip open and reject requests. It will stay tripped for the duration defined in <code>circuitBreakerSleepWindowInMilliseconds</code> ; The error percentage this is compared against comes from <code>HystrixCommandMetrics.getHealthCounts()</code> .	50	Integer
camel.hystrix.circuit-breaker-force-closed	If true the <code>HystrixCircuitBreaker#allowRequest()</code> will always return true to allow requests regardless of the error percentage from <code>HystrixCommandMetrics.getHealthCounts()</code> . The <code>circuitBreakerForceOpen()</code> property takes precedence so if it set to true this property does nothing.	false	Boolean
camel.hystrix.circuit-breaker-force-open	If true the <code>HystrixCircuitBreaker.allowRequest()</code> will always return false, causing the circuit to be open (tripped) and reject all requests. This property takes precedence over <code>circuitBreakerForceClosed()</code> ;	false	Boolean
camel.hystrix.circuit-breaker-request-volume-threshold	Minimum number of requests in the <code>metricsRollingStatisticalWindowInMilliseconds()</code> that must exist before the <code>HystrixCircuitBreaker</code> will trip. If below this number the circuit will not trip regardless of error percentage.	20	Integer
camel.hystrix.circuit-breaker-sleep-window-in-milliseconds	The time in milliseconds after a <code>HystrixCircuitBreaker</code> trips open that it should wait before trying requests again.	5000	Integer
camel.hystrix.configurations	Define additional configuration definitions.		Map
camel.hystrix.core-pool-size	Core thread-pool size that gets passed to <code>java.util.concurrent.ThreadPoolExecutor#setCorePoolSize(int)</code> .	10	Integer
camel.hystrix.enabled	Enable the component.	true	Boolean
camel.hystrix.execution-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.run()</code> . Requests beyond the concurrent limit will be rejected. Applicable only when <code>executionIsolationStrategy == SEMAPHORE</code> .	20	Integer

Name	Description	Default	Type
camel.hystrix.execution-isolation-strategy	What isolation strategy <code>HystrixCommand.run()</code> will be executed with. If <code>THREAD</code> then it will be executed on a separate thread and concurrent requests limited by the number of threads in the thread-pool. If <code>SEMAPHORE</code> then it will be executed on the calling thread and concurrent requests limited by the semaphore count.	<code>THREAD</code>	String
camel.hystrix.execution-isolation-thread-interrupt-on-timeout	Whether the execution thread should attempt an interrupt (using <code>Future#cancel</code>) when a thread times out. Applicable only when <code>executionIsolationStrategy() == THREAD</code> .	<code>true</code>	Boolean
camel.hystrix.execution-timeout-enabled	Whether the timeout mechanism is enabled for this command.	<code>true</code>	Boolean
camel.hystrix.execution-timeout-in-milliseconds	Time in milliseconds at which point the command will timeout and halt execution. If <code>executionIsolationThreadInterruptOnTimeout == true</code> and the command is thread-isolated, the executing thread will be interrupted. If the command is semaphore-isolated and a <code>HystrixObservableCommand</code> , that command will get unsubscribed.	<code>1000</code>	Integer
camel.hystrix.fallback-enabled	Whether <code>HystrixCommand.getFallback()</code> should be attempted when failure occurs.	<code>true</code>	Boolean
camel.hystrix.fallback-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.getFallback()</code> . Requests beyond the concurrent limit will fail-fast and not attempt retrieving a fallback.	<code>10</code>	Integer
camel.hystrix.group-key	Sets the group key to use. The default value is <code>CamelHystrix</code> .	<code>CamelHystrix</code>	String
camel.hystrix.keep-alive-time	Keep-alive time in minutes that gets passed to <code>ThreadPoolExecutor#setKeepAliveTime(long, TimeUnit)</code> .	<code>1</code>	Integer

Name	Description	Default	Type
camel.hystrix.max-queue-size	Max queue size that gets passed to BlockingQueue in HystrixConcurrencyStrategy.getBlockingQueue(int) This should only affect the instantiation of a threadpool - it is not eligible to change a queue size on the fly. For that, use queueSizeRejectionThreshold().	-1	Integer
camel.hystrix.max-imum-size	Maximum thread-pool size that gets passed to ThreadPoolExecutor#setMaximumPoolSize(int) . This is the maximum amount of concurrency that can be supported without starting to reject HystrixCommands. Please note that this setting only takes effect if you also set allowMaximumSizeToDivergeFromCoreSize.	10	Integer
camel.hystrix.metrics-health-snapshot-interval-in-milliseconds	Time in milliseconds to wait between allowing health snapshots to be taken that calculate success and error percentages and affect HystrixCircuitBreaker.isOpen() status. On high-volume circuits the continual calculation of error percentage can become CPU intensive thus this controls how often it is calculated.	500	Integer
camel.hystrix.metrics-rolling-percentile-bucket-size	Maximum number of values stored in each bucket of the rolling percentile. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	10	Integer
camel.hystrix.metrics-rolling-percentile-enabled	Whether percentile metrics should be captured using HystrixRollingPercentile inside HystrixCommandMetrics.	true	Boolean
camel.hystrix.metrics-rolling-percentile-window-buckets	Number of buckets the rolling percentile window is broken into. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	6	Integer
camel.hystrix.metrics-rolling-percentile-window-in-milliseconds	Duration of percentile rolling window in milliseconds. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	10000	Integer
camel.hystrix.metrics-rolling-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside HystrixCommandMetrics.	10	Integer

Name	Description	Default	Type
camel.hystrix.metrics-rolling-statistical-window-in-milliseconds	This property sets the duration of the statistical rolling window, in milliseconds. This is how long metrics are kept for the thread pool. The window is divided into buckets and rolls by those increments.	10000	Integer
camel.hystrix.queue-size-rejection-threshold	Queue size rejection threshold is an artificial max size at which rejections will occur even if maxQueueSize has not been reached. This is done because the maxQueueSize of a BlockingQueue can not be dynamically changed and we want to support dynamically changing the queue size that affects rejections. This is used by HystrixCommand when queuing a thread for execution.	5	Integer
camel.hystrix.request-log-enabled	Whether HystrixCommand execution and events should be logged to HystrixRequestLog.	true	Boolean
camel.hystrix.thread-pool-key	Sets the thread pool key to use. Will by default use the same value as groupKey has been configured to use.	Camel Hystrix	String
camel.hystrix.thread-pool-rolling-number-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10	Integer
camel.hystrix.thread-pool-rolling-number-statistical-window-in-milliseconds	Duration of statistical rolling window in milliseconds. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10000	Integer
camel.language.constant.enabled	Whether to enable auto configuration of the constant language. This is enabled by default.		Boolean
camel.language.constant.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.language.csimple.enabled	Whether to enable auto configuration of the csimple language. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.language.csimple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.exchangeproperty.enabled</code>	Whether to enable auto configuration of the exchangeProperty language. This is enabled by default.		Boolean
<code>camel.language.exchangeproperty.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.file.enabled</code>	Whether to enable auto configuration of the file language. This is enabled by default.		Boolean
<code>camel.language.file.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.header.enabled</code>	Whether to enable auto configuration of the header language. This is enabled by default.		Boolean
<code>camel.language.header.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.ref.enabled</code>	Whether to enable auto configuration of the ref language. This is enabled by default.		Boolean
<code>camel.language.ref.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.simple.enabled</code>	Whether to enable auto configuration of the simple language. This is enabled by default.		Boolean
<code>camel.language.simple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.tokenize.enabled</code>	Whether to enable auto configuration of the tokenize language. This is enabled by default.		Boolean
<code>camel.language.tokenize.group-delimiter</code>	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.		String
<code>camel.language.tokenize.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

Name	Description	Default	Type
camel.resilience4j. .automatic- transition-from- open-to-half- open-enabled	Enables automatic transition from OPEN to HALF_OPEN state once the waitDurationInOpenState has passed.	false	Boolean
camel.resilience4j. .circuit-breaker- ref	Refers to an existing io.github.resilience4j.circuitbreaker.CircuitBreaker instance to lookup and use from the registry. When using this, then any other circuit breaker options are not in use.		String
camel.resilience4j. .config-ref	Refers to an existing io.github.resilience4j.circuitbreaker.CircuitBreakerConfig instance to lookup and use from the registry.		String
camel.resilience4j. .configurations	Define additional configuration definitions.		Map
camel.resilience4j. .enabled	Enable the component.	true	Boolean
camel.resilience4j. .failure-rate- threshold	Configures the failure rate threshold in percentage. If the failure rate is equal or greater than the threshold the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 50 percentage.		Float
camel.resilience4j. .minimum- number-of-calls	Configures the minimum number of calls which are required (per sliding window period) before the CircuitBreaker can calculate the error rate. For example, if minimumNumberOfCalls is 10, then at least 10 calls must be recorded, before the failure rate can be calculated. If only 9 calls have been recorded the CircuitBreaker will not transition to open even if all 9 calls have failed. Default minimumNumberOfCalls is 100.	100	Integer
camel.resilience4j. .permitted- number-of-calls- in-half-open- state	Configures the number of permitted calls when the CircuitBreaker is half open. The size must be greater than 0. Default size is 10.	10	Integer

Name	Description	Default	Type
camel.resilience4j.sliding-window-size	Configures the size of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. <code>slidingWindowSize</code> configures the size of the sliding window. Sliding window can either be count-based or time-based. If <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the last <code>slidingWindowSize</code> calls are recorded and aggregated. If <code>slidingWindowType</code> is <code>TIME_BASED</code> , the calls of the last <code>slidingWindowSize</code> seconds are recorded and aggregated. The <code>slidingWindowSize</code> must be greater than 0. The <code>minimumNumberOfCalls</code> must be greater than 0. If the <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the <code>minimumNumberOfCalls</code> cannot be greater than <code>slidingWindowSize</code> . If the <code>slidingWindowType</code> is <code>TIME_BASED</code> , you can pick whatever you want. Default <code>slidingWindowSize</code> is 100.	100	Integer
camel.resilience4j.sliding-window-type	Configures the type of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. Sliding window can either be count-based or time-based. If <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the last <code>slidingWindowSize</code> calls are recorded and aggregated. If <code>slidingWindowType</code> is <code>TIME_BASED</code> , the calls of the last <code>slidingWindowSize</code> seconds are recorded and aggregated. Default <code>slidingWindowType</code> is <code>COUNT_BASED</code> .	<code>COUNT_BASED</code>	String
camel.resilience4j.slow-call-duration-threshold	Configures the duration threshold (seconds) above which calls are considered as slow and increase the slow calls percentage. Default value is 60 seconds.	60	Integer
camel.resilience4j.slow-call-rate-threshold	Configures a threshold in percentage. The CircuitBreaker considers a call as slow when the call duration is greater than <code>slowCallDurationThreshold</code> Duration. When the percentage of slow calls is equal or greater the threshold, the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 100 percentage which means that all recorded calls must be slower than <code>slowCallDurationThreshold</code> .		Float
camel.resilience4j.wait-duration-in-open-state	Configures the wait duration (in seconds) which specifies how long the CircuitBreaker should stay open, before it switches to half open. Default value is 60 seconds.	60	Integer

Name	Description	Default	Type
camel.resilience4j.writable-stack-trace-enabled	Enables writable stack traces. When set to false, <code>Exception.getStackTrace</code> returns a zero length array. This may be used to reduce log spam when the circuit breaker is open as the cause of the exceptions is already known (the circuit breaker is short-circuiting calls).	true	Boolean
camel.rest.api-component	The name of the Camel component to use as the REST API (such as swagger) If no API Component has been explicit configured, then Camel will lookup if there is a Camel component responsible for servicing and generating the REST API documentation, or if a <code>org.apache.camel.spi.RestApiProcessorFactory</code> is registered in the registry. If either one is found, then that is being used.		String
camel.rest.api-context-path	Sets a leading API context-path the REST API services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path.		String
camel.rest.api-context-route-id	Sets the route id to use for the route that services the REST API. The route will by default use an auto assigned route id.		String
camel.rest.api-host	To use an specific hostname for the API documentation (eg swagger) This can be used to override the generated host with this configured hostname.		String
camel.rest.api-property	Allows to configure as many additional properties for the api documentation (swagger). For example set property <code>api.title</code> to my cool stuff.		Map
camel.rest.api-vendor-extension	Whether vendor extension is enabled in the Rest APIs. If enabled then Camel will include additional information as vendor extension (eg keys starting with x-) such as route ids, class names etc. Not all 3rd party API gateways and tools supports vendor-extensions when importing your API docs.	false	Boolean
camel.rest.binding-mode	Sets the binding mode to use. The default value is off.		RestBindingMode

Name	Description	Default	Type
camel.rest.client-request-validation	Whether to enable validation of the client request to check whether the Content-Type and Accept headers from the client is supported by the Rest-DSL configuration of its consumes/produces settings. This can be turned on, to enable this check. In case of validation error, then HTTP Status codes 415 or 406 is returned. The default value is false.	false	Boolean
camel.rest.component	The Camel Rest component to use for the REST transport (consumer), such as netty-http, jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String
camel.rest.component-property	Allows to configure as many additional properties for the rest component in use.		Map
camel.rest.consumer-property	Allows to configure as many additional properties for the rest consumer in use.		Map
camel.rest.context-path	Sets a leading context-path the REST services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path. Or for components such as camel-jetty or camel-netty-http that includes a HTTP server.		String
camel.rest.cors-headers	Allows to configure custom CORS headers.		Map
camel.rest.data-format-property	Allows to configure as many additional properties for the data formats in use. For example set property prettyPrint to true to have json outputted in pretty mode. The properties can be prefixed to denote the option is only for either JSON or XML and for either the IN or the OUT. The prefixes are: json.in. json.out. xml.in. xml.out. For example a key with value xml.out.mustBeJAXBELEMENT is only for the XML data format for the outgoing. A key without a prefix is a common key for all situations.		Map
camel.rest.enable-cors	Whether to enable CORS headers in the HTTP response. The default value is false.	false	Boolean

Name	Description	Default	Type
camel.rest.endpoint-property	Allows to configure as many additional properties for the rest endpoint in use.		Map
camel.rest.host	The hostname to use for exposing the REST service.		String
camel.rest.hostname-resolver	If no hostname has been explicit configured, then this resolver is used to compute the hostname the REST service will be using.		RestHostNameResolver
camel.rest.json-data-format	Name of specific json data format to use. By default json-jackson will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.port	The port number to use for exposing the REST service. Notice if you use servlet component then the port number configured here does not apply, as the port number in use is the actual port number the servlet component is using. eg if using Apache Tomcat its the tomcat http port, if using Apache Karaf its the HTTP service in Karaf that uses port 8181 by default etc. Though in those situations setting the port number here, allows tooling and JMX to know the port number, so its recommended to set the port number to the number that the servlet engine uses.		String
camel.rest.producer-api-doc	Sets the location of the api document (swagger api) the REST producer will use to validate the REST uri and query parameters are valid accordingly to the api document. This requires adding camel-swagger-java to the classpath, and any miss configuration will let Camel fail on startup and report the error(s). The location of the api document is loaded from classpath by default, but you can use file: or http: to refer to resources to load from file or http url.		String
camel.rest.producer-component	Sets the name of the Camel component to use as the REST producer.		String
camel.rest.scheme	The scheme to use for exposing the REST service. Usually http or https is supported. The default value is http.		String
camel.rest.skip-binding-on-error-code	Whether to skip binding on output if there is a custom HTTP error code header. This allows to build custom error messages that do not bind to json / xml etc, as success messages otherwise will do.	false	Boolean

Name	Description	Default	Type
<code>camel.rest.use-x-forward-headers</code>	Whether to use X-Forward headers for Host and related setting. The default value is true.	true	Boolean
<code>camel.rest.xml-data-format</code>	Name of specific XML data format to use. By default JAXB will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
<code>camel.rest.api-context-id-pattern</code>	Deprecated Sets an CamelContext id pattern to only allow Rest APIs from rest services within CamelContext's which name matches the pattern. The pattern name refers to the CamelContext name, to match on the current CamelContext only. For any other value, the pattern uses the rules from <code>PatternHelper#matchPattern(String,String)</code> .		String
<code>camel.rest.api-context-listing</code>	Deprecated Sets whether listing of all available CamelContext's with REST services in the JVM is enabled. If enabled it allows to discover these contexts, if false then only the current CamelContext is in use.	false	Boolean

CHAPTER 25. CONTROL BUS

Only producer is supported

The [Control Bus](#) from the EIP patterns allows for the integration system to be monitored and managed from within the framework.

Use a Control Bus to manage an enterprise integration system. The Control Bus uses the same messaging mechanism used by the application data, but uses separate channels to transmit data that is relevant to the management of components involved in the message flow.

In Camel you can manage and monitor using JMX, or by using a Java API from the **CamelContext**, or from the **org.apache.camel.api.management** package, or use the event notifier which has an example [here](#).

The ControlBus component provides easy management of Camel applications based on the [Control Bus](#) EIP pattern. For example, by sending a message to an Endpoint you can control the lifecycle of routes, or gather performance statistics.

```
controlbus:command[?options]
```

Where **command** can be any string to identify which type of command to use.

25.1. COMMANDS

Command	Description
route	To control routes using the routeId and action parameter.
language	Allows you to specify a to use for evaluating the message body. If there is any result from the evaluation, then the result is put in the message body.

25.2. DEPENDENCIES

When using **controlbus** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-controlbus-starter</artifactId>
</dependency>
```

25.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

25.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

25.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

25.4. COMPONENT OPTIONS

The Control Bus component supports 2 options, which are listed below.

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

25.5. ENDPOINT OPTIONS

The Control Bus endpoint is configured using URI syntax:

```
controlbus:command:language
```

with the following path and query parameters:

25.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
command (producer)	Required Command can be either route or language. Enum values: ● route ● language		String

Name	Description	Default	Type
language (producer)	Allows you to specify the name of a Language to use for evaluating the message body. If there is any result from the evaluation, then the result is put in the message body. Enum values: • bean • constant • el • exchangeProperty • file • groovy • header • jsonpath • mvel • ognl • ref • simple • spel • sql • terser • tokenize • xpath • xquery • xtokenize		Language

25.5.1.1. Query Parameters (6 parameters)

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
action (producer)	To denote an action that can be either: start, stop, or status. To either start or stop a route, or to get the status of the route as output in the message body. You can use suspend and resume from Camel 2.11.1 onwards to either suspend or resume a route. And from Camel 2.11.1 onwards you can use stats to get performance statics returned in XML format; the routeld option can be used to define which route to get the performance stats for, if routeld is not defined, then you get statistics for the entire CamelContext. The restart action will restart the route. Enum values: • start • stop • suspend • resume • restart • status • stats		String
async (producer)	Whether to execute the control bus task asynchronously. Important: If this option is enabled, then any result from the task is not set on the Exchange. This is only possible if executing tasks synchronously.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
loggingLevel (producer)	Logging level used for logging when task is done, or if any exceptions occurred during processing the task. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	INFO	LogLevel
restartDelay (producer)	The delay in millis to use when restarting a route.	1000	int
routeId (producer)	To specify a route by its id. The special keyword <code>current</code> indicates the current route.		String

25.6. USING ROUTE COMMAND

The route command allows you to do common tasks on a given route very easily, for example to start a route, you can send an empty message to this endpoint:

```
template.sendBody("controlbus:route?routeId=foo&action=start", null);
```

To get the status of the route, you can do:

```
String status = template.requestBody("controlbus:route?routeId=foo&action=status", null, String.class);
```

25.7. GETTING PERFORMANCE STATISTICS

This requires JMX to be enabled (is by default) then you can get the performance statistics per route, or for the CamelContext. For example to get the statistics for a route named foo, we can do:

```
String xml = template.requestBody("controlbus:route?routeId=foo&action=stats", null, String.class);
```

The returned statistics is in XML format. Its the same data you can get from JMX with the **dumpRouteStatsAsXml** operation on the **ManagedRouteMBean**.

To get statistics for the entire CamelContext you just omit the routeId parameter as shown below:

```
String xml = template.requestBody("controlbus:route?action=stats", null, String.class);
```

25.8. USING SIMPLE LANGUAGE

You can use the [Simple](#) language with the control bus, for example to stop a specific route, you can send a message to the **"controlbus:language:simple"** endpoint containing the following message:

```
template.sendBody("controlbus:language:simple",
    "${camelContext.getRouteController().stopRoute('myRoute')}");
```

As this is a void operation, no result is returned. However, if you want the route status you can do:

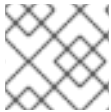
```
String status = template.requestBody("controlbus:language:simple",
    "${camelContext.getRouteStatus('myRoute')}", String.class);
```

It's easier to use the **route** command to control lifecycle of routes. The **language** command allows you to execute a language script that has stronger powers such as [Groovy](#) or to some extend the [Simple](#) language.

For example to shutdown Camel itself you can do:

```
template.sendBody("controlbus:language:simple?async=true", "${camelContext.stop()}");
```

We use **async=true** to stop Camel asynchronously as otherwise we would be trying to stop Camel while it was in-flight processing the message we sent to the control bus component.



NOTE

You can also use other languages such as [Groovy](#), etc.

25.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.component.controlbus.automated-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.controlbus.enabled	Whether to enable auto configuration of the controlbus component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component-controlbus.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 26. CRON

Only consumer is supported

The Cron component is a generic interface component that allows triggering events at specific time interval specified using the Unix cron syntax (e.g. `0/2 * * * * ?` to trigger an event every two seconds).

Being an interface component, the Cron component does not contain a default implementation, instead it requires that the users plug the implementation of their choice.

The following standard Camel components support the Cron endpoints:

- Camel-quartz
- Camel-spring

The Cron component is also supported in **Camel K**, which can use the Kubernetes scheduler to trigger the routes when required by the cron expression. Camel K does not require additional libraries to be plugged when using cron expressions compatible with Kubernetes cron syntax.

26.1. DEPENDENCIES

When using **cron** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>  
  <groupId>org.apache.camel.springboot</groupId>  
  <artifactId>camel-cron-starter</artifactId>  
</dependency>
```

Additional libraries may be needed in order to plug a specific implementation.

26.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

26.2.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).

- directly in the Java code.

26.2.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

26.3. COMPONENT OPTIONS

The Cron component supports 3 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
cronService (advanced)	The id of the CamelCronService to use when multiple implementations are provided.		String

26.4. ENDPOINT OPTIONS

The Cron endpoint is configured using URI syntax:

```
cron:name
```

with the following path and query parameters:

26.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (consumer)	Required The name of the cron trigger.		String

26.4.2. Query Parameters (4 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
schedule (consumer)	Required A cron expression that will be used to generate events.		String
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern

26.5. USAGE

The component can be used to trigger events at specified times, as in the following example:

```
from("cron:tab?schedule=0/1+*+*+*+*+*+*+*+*+*")
  .setBody().constant("event")
  .log("${body}");
```

The schedule expression **0/3+10+++*+?** can be also written as **0/3 10 * * * ?** and triggers an event every three seconds only in the tenth minute of each hour.

Parts in the schedule expression means (in order):

- Seconds (optional)
- Minutes
- Hours
- Day of month
- Month
- Day of week
- Year (optional)

Schedule expressions can be made of 5 to 7 parts. When expressions are composed of 6 parts, the first items is the "seconds" part (and year is considered missing).

Other valid examples of schedule expressions are:

- **0/2 * * * ?** (5 parts, an event every two minutes)
- **0 0/2 * * * MON-FRI 2030** (7 parts, an event every two minutes only in year 2030)

Routes can also be written using the XML DSL.

```
<route>
  <from uri="cron:tab?schedule=0/1+*+*+*+*+*+*+*+*+*"/>
  <setBody>
    <constant>event</constant>
  </setBody>
  <to uri="log:info"/>
</route>
```

26.6. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.component.cron.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.cron.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.cron.cron-service	The id of the CamelCronService to use when multiple implementations are provided.		String
camel.component.cron.enabled	Whether to enable auto configuration of the cron component. This is enabled by default.		Boolean

CHAPTER 27. CRYPTO (JCE)

Since Camel 2.3

Only producer is supported

With Camel cryptographic endpoints and Java Cryptographic extension, it is easy to create Digital Signatures for Exchanges. Camel provides a pair of flexible endpoints which get used in concert to create a signature for an exchange in one part of the exchange workflow and then verify the signature in a later part of the workflow.

27.1. DEPENDENCIES

When using **crypto** with Red Hat build of Camel Spring Boot ensure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-crypto-starter</artifactId>
</dependency>
```

27.2. INTRODUCTION

Digital signatures make use of Asymmetric Cryptographic techniques to sign messages. From a high level, the algorithms use pairs of complimentary keys with the special property that data encrypted with one key can only be decrypted with the other. The private key, is closely guarded and used to 'sign' the message while the other, public key, is shared around to anyone interested in verifying the signed messages. Messages are signed by using the private key to encrypting a digest of the message. This encrypted digest is transmitted along with the message. On the other side the verifier recalculates the message digest and uses the public key to decrypt the digest in the signature. If both digests match, the verifier knows only the holder of the private key could have created the signature.

Camel uses the Signature service from the Java Cryptographic Extension to do all the heavy cryptographic lifting required to create exchange signatures. The following are the resources for explaining the mechanics of Cryptography, Message digests and Digital Signatures and how to leverage them with the JCE.

- Bruce Schneier's Applied Cryptography
- Beginning Cryptography with Java by David Hook
- The ever insightful Wikipedia [Digital_signatures](#)

27.3. URI FORMAT

Camel provides a pair of crypto endpoints to create and verify signatures

```
crypto:sign:name[?options]
crypto:verify:name[?options]
```

- **crypto:sign** creates the signature and stores it in the Header keyed by the constant **org.apache.camel.component.crypto.DigitalSignatureConstants.SIGNATURE**, that is, **"CamelDigitalSignature"**.

- **crypto:verify** reads in the contents of this header and do the verification calculation.

To function, the sign and verify process needs a pair of keys to be shared, signing requiring a **PrivateKey** and verifying a **PublicKey** (or a **Certificate** containing one). Using the JCE, it is very simple to generate these key pairs but it is usually most secure to use a KeyStore to house and share your keys. The DSL is very flexible about how keys are supplied and provides a number of mechanisms.

A **crypto:sign** endpoint is typically defined in one route and the complimentary **crypto:verify** in another, though for simplicity in the examples they appear one after the other. Both signing and verifying should be configured identically.

27.4. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

27.4.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

27.4.2. Configuring Endpoint Options

Endpoints have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a *type safe* way of configuring endpoints and data formats in Java.

Use [Property Placeholders](#) to configure options that allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

27.5. COMPONENT OPTIONS

The Crypto (JCE) component supports 21 options that are listed below.

Name	Description	Default	Type
algorithm (producer)	Sets the JCE name of the Algorithm that should be used for the signer.	SHA256withRSA	String
alias (producer)	Sets the alias used to query the KeyStore for keys and <code>java.security.cert.Certificate Certificates</code> to be used in signing and verifying exchanges. This value can be provided at runtime via the message header <code>org.apache.camel.component.crypto.DigitalSignatureConstants#KEYSTORE_ALIAS</code> .		String
certificateName (producer)	Sets the reference name for a PrivateKey that can be found in the registry.		String
keystore (producer)	Sets the KeyStore that can contain keys and Certificates for use in signing and verifying exchanges. A KeyStore is typically used with an alias, either one supplied in the Route definition or dynamically via the message header <code>CamelSignatureKeyStoreAlias</code> . If no alias is supplied and there is only a single entry in the Keystore, then this single entry will be used.		KeyStore
keystoreName (producer)	Sets the reference name for a Keystore that can be found in the registry.		String

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
privateKey* (producer)	Set the PrivateKey that should be used to sign the exchange.		PrivateKey
privateKeyName (producer)	Sets the reference name for a PrivateKey that can be found in the registry.		String
provider (producer)	Set the id of the security provider that provides the configured Signature algorithm.		String
publicKeyName (producer)	references that should be resolved when the context changes.		String
secureRandomName (producer)	Sets the reference name for a SecureRandom that can be found in the registry.		String

Name	Description	Default	Type
signatureHeaderName (producer)	Set the name of the message header that should be used to store the base64 encoded signature. This defaults to 'CamelDigitalSignature'.		String
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
bufferSize (advanced)	Set the size of the buffer used to read in the Exchange payload data.	2048	Integer
certificate (advanced)	Set the Certificate that should be used to verify the signature in the exchange based on its payload.		Certificate
clearHeaders (advanced)	Determines if the Signature specific headers be cleared after signing and verification. Defaults to true, and should only be made otherwise at your extreme peril as vital private information such as Keys and passwords may escape if unset.	true	boolean

Name	Description	Default	Type
configuration (advanced)	To use the shared DigitalSignatureConfiguration as configuration.		DigitalSignatureConfiguration
keyStoreParameters (advanced)	Sets the KeyStore that can contain keys and Certificates for use in signing and verifying exchanges based on the given KeyStoreParameters. A KeyStore is typically used with an alias, either one supplied in the Route definition or dynamically via the message header CamelSignatureKeyStoreAlias. If no alias is supplied and there is only a single entry in the Keystore, then this single entry will be used.		KeyStoreParameters
publicKey (advanced)	Set the PublicKey that should be used to verify the signature in the exchange.		PublicKey
secureRandom (advanced)	Set the SecureRandom used to initialize the Signature service.		SecureRandom
password (security)	Sets the password used to access an aliased PrivateKey in the KeyStore.		String

27.6. ENDPOINT OPTIONS

The Crypto (JCE) endpoint is configured using URI syntax:

```
crypto:cryptoOperation:name
```

The following are the path and query parameters:

27.6.1. Path Parameters (2 parameters)

Name	Description	Default	Type
cryptoOperation (producer)	Required Set the Crypto operation from that supplied after the crypto scheme in the endpoint uri e.g. crypto:sign sets sign as the operation. Enum values: * sign * verify		CryptoOperation
name (producer)	Required The logical name of this operation.		String

27.6.2. Query Parameters (19 parameters)

Name	Description	Default	Type
algorithm (producer)	Sets the JCE name of the Algorithm that should be used for the signer.	SHA256withRSA	String
alias (producer)	Sets the alias used to query the KeyStore for keys and <code>java.security.cert.Certificate Certificates</code> to be used in signing and verifying exchanges. This value can be provided at runtime via the message header <code>org.apache.camel.component.crypto.DigitalSignatureConstants#KEYSTORE_ALIAS</code>.		String
certificateName (producer)	Sets the reference name for a PrivateKey that can be found in the registry.		String

Name	Description	Default	Type
keystore (producer)	Sets the KeyStore that can contain keys and Certificates for use in signing and verifying exchanges. A KeyStore is typically used with an alias, either one supplied in the Route definition or dynamically via the message header <code>CamelSignatureKeyStoreAlias</code> . If no alias is supplied and there is only a single entry in the Keystore, then this single entry will be used.		KeyStore
keystoreName (producer)	Sets the reference name for a Keystore that can be found in the registry.		String
privateKey (producer)	Set the PrivateKey that should be used to sign the exchange.		PrivateKey
privateKeyName (producer)	Sets the reference name for a PrivateKey that can be found in the registry.		String
provider (producer)	Set the id of the security provider that provides the configured Signature algorithm.		String
publicKeyName (producer)	references that should be resolved when the context changes.		String
secureRandomName (producer)	Sets the reference name for a SecureRandom that can be found in the registry.		String

Name	Description	Default	Type
signatureHeaderName (producer)	Set the name of the message header that should be used to store the base64 encoded signature. This defaults to 'CamelDigitalSignature'.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages using Camel routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time.	false	boolean
bufferSize (advanced)	Set the size of the buffer used to read in the Exchange payload data.	2048	Integer
certificate (advanced)	Set the Certificate that should be used to verify the signature in the exchange based on its payload.		Certificate

Name	Description	Default	Type
clearHeaders (advanced)	Determines if the Signature specific headers be cleared after signing and verification. Defaults to true, and should only be made otherwise at your extreme peril as vital private information such as Keys and passwords may escape if unset.	true	boolean
keyStoreParameters (advanced)	Sets the KeyStore that can contain keys and Certificates for use in signing and verifying exchanges based on the given KeyStoreParameters. A KeyStore is typically used with an alias, either one supplied in the Route definition or dynamically via the message header CamelSignatureKeyStoreAlias. If no alias is supplied and there is only a single entry in the Keystore, then this single entry will be used.		KeyStoreParameters
publicKey (advanced)	Set the PublicKey that should be used to verify the signature in the exchange.		PublicKey
secureRandom (advanced)	Set the SecureRandom used to initialize the Signature service.		SecureRandom
password (security)	Sets the password used to access an aliased PrivateKey in the KeyStore.		String

27.7. MESSAGE HEADERS

The Crypto (JCE) component supports 4 message headers that are listed below.

Name	Description	Default	Type
[CamelSignaturePrivateKey (producer) Constant: SIGNATURE_PRIVATE_KEY	The PrivateKey that should be used to sign the message.		PrivateKey
[CamelSignaturePublicKeyOrCert (producer) Constant: SIGNATURE_PUBLIC_KEY_OR_CERT	The Certificate or PublicKey that should be used to verify the signature.		Certificate or PublicKey
CamelSignatureKeyStoreAlias (producer) Constant: KEYSTORE_ALIAS	The alias used to query the KeyStore for keys and Certificates to be used in signing and verifying exchanges.		String
CamelSignatureKeyStorePassword (producer) Constant: KEYSTORE_PASSWORD	The password used to access an aliased PrivateKey in the KeyStore.		char[]

27.8. USING

27.8.1. Raw keys

The most basic way to sign and verify an exchange is with a KeyPair as follows.

```

KeyPair keyPair = KeyGenerator.getInstance("RSA").generateKeyPair();

from("direct:sign")
  .setHeader(DigitalSignatureConstants.SIGNATURE_PRIVATE_KEY, constant(keys.getPrivate()))
  .to("crypto:sign:message")
  .to("direct:verify");

from("direct:verify")
  .setHeader(DigitalSignatureConstants.SIGNATURE_PUBLIC_KEY_OR_CERT,
    constant(keys.getPublic()))
  .to("crypto:verify:check");

```

The same can be achieved with the [Spring XML Extensions](#) using references to keys.

27.8.2. KeyStores and Aliases.

The JCE provides a very versatile keystore concept for housing pairs of private keys and certificates, keeping them encrypted and password protected. They can be retrieved by applying an alias to the retrieval APIs. There are a number of ways to get keys and Certificates into a keystore, most often this is done with the external 'keytool' application.

The following command will create a keystore containing a key and certificate aliased by **bob**, which can be used in the following examples. The password for the keystore and the key is **letmein**.

```
keytool -genkey -keyalg RSA -keysize 2048 -keystore keystore.jks -storepass letmein -alias bob -
dname "CN=Bob,OU=IT,O=Camel" -noprompt
```

The following route first signs an exchange using Bob's alias from the KeyStore bound into the Camel Registry, and then verifies it using the same alias.

```
from("direct:sign")
  .to("crypto:sign:keystoreSign?alias=bob&keystoreName=myKeystore&password=letmein")
  .log("Signature: ${header.CamelDigitalSignature}")
  .to("crypto:verify:keystoreVerify?alias=bob&keystoreName=myKeystore&password=letmein")
  .log("Verified: ${body}");
```

The following code shows how to load the keystore created using the above **keytool** command and bind it into the registry with the name **myKeystore** for use in the above route. The example makes use of the **@Configuration** and **@BindToRegistry** annotations introduced in Camel 3 to instantiate the KeyStore and register it with the name **myKeyStore**.

```
@Configuration
public class KeystoreConfig {

    @BindToRegistry
    public KeyStore myKeystore() throws Exception {
        KeyStore store = KeyStore.getInstance("JKS");
        try (FileInputStream fis = new FileInputStream("keystore.jks")) {
            store.load(fis, "letmein".toCharArray());
        }
        return store;
    }
}
```

Again in Spring a ref is used to lookup an actual keystore instance.

27.8.3. Changing JCE Provider and Algorithm

Changing the Signature algorithm or the Security provider is a simple matter of specifying their names. You will need to also use Keys that are compatible with the algorithm you choose.

27.8.4. Changing the Signature Message Header

It may be desirable to change the message header used to store the signature. A different header name can be specified in the route definition as follows

```
from("direct:sign")
  .to("crypto:sign:keystoreSign?
alias=bob&keystoreName=myKeystore&password=letmein&signatureHeaderName=mySignature")
  .log("Signature: ${header.mySignature}")
```

```
.to("crypto:verify:keystoreVerify?
alias=bob&keystoreName=myKeystore&password=letmein&signatureHeaderName=mySignature");
```

===Changing the bufferSize

In case you need to update the size of the buffer.

27.8.5. Supplying Keys dynamically.

When using a Recipient list or similar EIP, the recipient of an exchange can vary dynamically. Using the same key across all recipients may be neither feasible nor desirable. It would be useful to be able to specify signature keys dynamically on a per-exchange basis. The exchange could then be dynamically enriched with the key of its target recipient prior to signing. To facilitate this the signature mechanisms allow for keys to be supplied dynamically via the message headers below.

- **DigitalSignatureConstants.SIGNATURE_PRIVATE_KEY, "CamelSignaturePrivateKey"**
- **DigitalSignatureConstants.SIGNATURE_PUBLIC_KEY_OR_CERT, "CamelSignaturePublicKeyOrCert"**

Even better would be to dynamically supply a keystore alias. Again the alias can be supplied in a message header

- **DigitalSignatureConstants.KEYSTORE_ALIAS, "CamelSignatureKeyStoreAlias"**

The header would be set as follows:

```
Exchange unsigned = getMandatoryEndpoint("direct:alias-sign").createExchange();
unsigned.getIn().setBody(payload);
unsigned.getIn().setHeader(DigitalSignatureConstants.KEYSTORE_ALIAS, "bob");
unsigned.getIn().setHeader(DigitalSignatureConstants.KEYSTORE_PASSWORD,
"letmein".toCharArray());
template.send("direct:alias-sign", unsigned);
Exchange signed = getMandatoryEndpoint("direct:alias-sign").createExchange();
signed.getIn().copyFrom(unsigned.getMessage());
signed.getIn().setHeader(DigitalSignatureConstants.KEYSTORE_ALIAS, "bob");
template.send("direct:alias-verify", signed);
```

27.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 47 options that are listed below.

Name	Description	Default	Type
camel.component.crypt.to.algorithm	Sets the JCE name of the Algorithm that should be used for the signer.	SHA256withRSA	String

Name	Description	Default	Type
camel.component.crypto.alias	Sets the alias used to query the KeyStore for keys and <code>{link java.security.cert.Certificate Certificates}</code> to be used in signing and verifying exchanges. This value can be provided at runtime via the message header <code>org.apache.camel.component.crypto.DigitalSignatureConstants#KEYSTORE_ALIAS</code> .		String
camel.component.crypto.autowired-enabled	Whether auto-wiring is enabled. This is used for automatic auto-wiring options (the option must be marked as auto-wired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.crypto.buffer-size	Set the size of the buffer used to read in the Exchange payload data.	2048	Integer
camel.component.crypto.certificate	Set the Certificate that should be used to verify the signature in the exchange based on its payload. The option is a <code>java.security.cert.Certificate</code> type.		Certificate
camel.component.crypto.certificate-name	Sets the reference name for a <code>PrivateKey</code> that can be found in the registry.		String

Name	Description	Default	Type
camel.component.crypto.clear-headers	Determines if the Signature specific headers be cleared after signing and verification. Defaults to true, and should only be made otherwise at your extreme peril as vital private information such as Keys and passwords may escape if unset.	true	Boolean
camel.component.crypto.configuration	To use the shared DigitalSignatureConfiguration as configuration. The option is a org.apache.camel.component.crypto.DigitalSignatureConfiguration type.		DigitalSignatureConfiguration
camel.component.crypto.enabled	Whether to enable auto configuration of the crypto component. This is enabled by default.		Boolean
camel.component.crypto.key-store-parameters	Sets the KeyStore that can contain keys and Certificates for use in signing and verifying exchanges based on the given KeyStoreParameters. A KeyStore is typically used with an alias, either one supplied in the Route definition or dynamically via the message header CamelSignatureKeyStoreAlias. If no alias is supplied and there is only a single entry in the Keystore, then this single entry will be used. The option is a org.apache.camel.support.jsse.KeyStoreParameters type.		KeyStoreParameters

Name	Description	Default	Type
camel.component.crypto.keystore	Sets the KeyStore that can contain keys and Certificates for use in signing and verifying exchanges. A KeyStore is typically used with an alias, either one supplied in the Route definition or dynamically via the message header <code>CamelSignatureKeyStoreAlias</code> . If no alias is supplied and there is only a single entry in the Keystore, then this single entry will be used. The option is a <code>java.security.KeyStore</code> type.		KeyStore
camel.component.crypto.keystore-name	Sets the reference name for a Keystore that can be found in the registry.		String
camel.component.crypto.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.crypto.password	Sets the password used to access an aliased PrivateKey in the KeyStore.		String
camel.component.crypto.private-key	Set the PrivateKey that should be used to sign the exchange. The option is a java.security.PrivateKey type.		PrivateKey
camel.component.crypto.private-key-name	Sets the reference name for a PrivateKey that can be found in the registry.		String
camel.component.crypto.provider	Set the id of the security provider that provides the configured Signature algorithm.		String
camel.component.crypto.public-key	Set the PublicKey that should be used to verify the signature in the exchange. The option is a java.security.PublicKey type.		PublicKey
camel.component.crypto.public-key-name	references that should be resolved when the context changes.		String
camel.component.crypto.secure-random	Set the SecureRandom used to initialize the Signature service. The option is a java.security.SecureRandom type.		SecureRandom
camel.component.crypto.secure-random-name	Sets the reference name for a SecureRandom that can be found in the registry.		String

Name	Description	Default	Type
camel.component.crypto.signature-header-name	Set the name of the message header that should be used to store the base64 encoded signature. This defaults to 'CamelDigitalSignature'.		String
camel.dataformat.crypto.algorithm	The JCE algorithm name indicating the cryptographic algorithm that will be used.		String
camel.dataformat.crypto.algorithm-parameter-ref	A JCE AlgorithmParameterSpec used to initialize the Cipher. Will lookup the type using the given name as a java.security.spec.AlgorithmParameterSpec type.		String
camel.dataformat.crypto.buffer-size	The size of the buffer used in the signature process.	4096	Integer
camel.dataformat.crypto.crypto-provider	The name of the JCE Security Provider that should be used.		String
camel.dataformat.crypto.enabled	Whether to enable auto configuration of the crypto data format. This is enabled by default.		Boolean
camel.dataformat.crypto.init-vector-ref	Refers to a byte array containing the Initialization Vector that will be used to initialize the Cipher.		String
camel.dataformat.crypto.inline	Flag indicating that the configured IV should be inlined into the encrypted data stream. Is by default false.	false	Boolean
camel.dataformat.crypto.key-ref	Refers to the secret key to lookup from the register to use.		String

Name	Description	Default	Type
camel.dataformat.cryptomac-algorithm	The JCE algorithm name indicating the Message Authentication algorithm.	HmacSHA1	String
camel.dataformat.cryptom.should-append-h-m-a-c	Flag indicating that a Message Authentication Code should be calculated and appended to the encrypted data.	true	Boolean
camel.dataformat.pgpp.algorithm	Symmetric key encryption algorithm; possible values are defined in org.bouncycastle.bcpkg.SymmetricKeyAlgorithmTags; for example 2 (= TRIPLE DES), 3 (= CAST5), 4 (= BLOWFISH), 6 (= DES), 7 (= AES_128). Only relevant for encrypting.		Integer
camel.dataformat.pgpp. armored	This option will cause PGP to base64 encode the encrypted text, making it available for copy/paste, etc.	false	Boolean
camel.dataformat.pgpp.compression-algorithm	Compression algorithm; possible values are defined in org.bouncycastle.bcpkg.CompressionAlgorithmTags; for example 0 (= UNCOMPRESSED), 1 (= ZIP), 2 (= ZLIB), 3 (= BZIP2). Only relevant for encrypting.		Integer
camel.dataformat.pgpp.enabled	Whether to enable auto configuration of the pgp data format. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.dataformat.pgp.hash-algorithm	Signature hash algorithm; possible values are defined in org.bouncycastle.bcpg.HashAlgorithmTags; for example 2 (= SHA1), 8 (= SHA256), 9 (= SHA384), 10 (= SHA512), 11 (=SHA224). Only relevant for signing.		Integer
camel.dataformat.pgp.integrity	Adds an integrity check/sign into the encryption file. The default value is true.	true	Boolean
camel.dataformat.pgp.key-file-name	Filename of the keyring; must be accessible as a classpath resource (but you can specify a location in the file system by using the file: prefix).		String
camel.dataformat.pgp.key-userid	The user ID of the key in the PGP keyring used during encryption. Can also be only a part of a user ID. For example, if the user ID is Test User then you can use the part Test User or to address the user ID.		String
camel.dataformat.pgp.password	Password used when opening the private key (not used for encryption).		String

Name	Description	Default	Type
camel.dataformat.pgp.provider	Java Cryptography Extension (JCE) provider, default is Bouncy Castle (BC). Alternatively you can use, for example, the IAIK JCE provider; in this case the provider must be registered beforehand and the Bouncy Castle provider must not be registered beforehand. The Sun JCE provider does not work.		String
camel.dataformat.pgp.signature-key-file-name	Filename of the keyring to use for signing (during encryption) or for signature verification (during decryption); must be accessible as a classpath resource (but you can specify a location in the file system by using the file: prefix).		String
camel.dataformat.pgp.signature-key-ring	Keyring used for signing/verifying as byte array. You can not set the signatureKeyFileName and signatureKeyRing at the same time.		String

Name	Description	Default	Type
camel.dataformat.pgp. signature-key-userid	User ID of the key in the PGP keyring used for signing (during encryption) or signature verification (during decryption). During the signature verification process the specified User ID restricts the public keys from the public keyring which can be used for the verification. If no User ID is specified for the signature verification then any public key in the public keyring can be used for the verification. Can also be only a part of a user ID. For example, if the user ID is Test User then you can use the part Test User or to address the User ID.		String
camel.dataformat.pgp. signature-password	Password used when opening the private key used for signing (during encryption).		String

Name	Description	Default	Type
camel.dataformat.pgp.signature-verification-option	Controls the behavior for verifying the signature during un-marshaling. There are 4 values possible: optional: The PGP message may or may not contain signatures; if it does contain signatures, then a signature verification is executed. required: The PGP message must contain at least one signature; if this is not the case an exception (PGPException) is thrown. A signature verification is executed. ignore: Contained signatures in the PGP message are ignored; no signature verification is executed. no_signature_allowed: The PGP message must not contain a signature; otherwise an exception (PGPException) is thrown.		String

CHAPTER 28. CSIMPLE

The CSimple language is **compiled** Simple language.

28.1. DEPENDENCIES

When using **csimple** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-core-starter</artifactId>
</dependency>
```

28.2. DIFFERENT BETWEEN CSIMPLE AND SIMPLE

The simple language is a dynamic expression language which is runtime parsed into a set of Camel Expressions or Predicates.

The csimple language is parsed into regular Java source code and compiled together with all the other source code, or compiled once during bootstrap via the **camel-csimple-joor** module.

The simple language is generally very lightweight and fast, however for some use-cases with dynamic method calls via OGNL paths, then the simple language does runtime introspection and reflection calls. This has an overhead on performance, and was one of the reasons why csimple was created.

The csimple language requires to be typesafe and method calls via OGNL paths requires to know the type during parsing. This means for csimple languages expressions you would need to provide the class type in the script, whereas simple introspects this at runtime.

In other words the simple language is using *duck typing* (if it looks like a duck, and quacks like a duck, then it is a duck) and csimple is using Java type (typesafety). If there is a type error then simple will report this at runtime, and with csimple there will be a Java compilation error.

28.2.1. Additional CSimple functions

The csimple language includes some additional functions to support common use-cases working with **Collection**, **Map** or array types. The following functions *bodyAsIndex*, *headerAsIndex*, and *exchangePropertyAsIndex* is used for these use-cases as they are typed.

Function	Type	Description
<code>bodyAsIndex(type, index)</code>	Type	To be used for collecting the body from an existing Collection , Map or array (lookup by the index) and then converting the body to the given type determined by its classname. The converted body can be null.
<code>mandatoryBodyAsIndex(type, index)</code>	Type	To be used for collecting the body from an existing Collection , Map or array (lookup by the index) and then converting the body to the given type determined by its classname. Expects the body to be not null.

Function	Type	Description
<code>headerAsIndex(key, type, index)</code>	Type	To be used for collecting a header from an existing Collection , Map or array (lookup by the index) and then converting the header value to the given type determined by its classname. The converted header can be null.
<code>mandatoryHeaderAsIndex(key, type, index)</code>	Type	To be used for collecting a header from an existing Collection , Map or array (lookup by the index) and then converting the header value to the given type determined by its classname. Expects the header to be not null.
<code>exchangePropertyAsIndex(key, type, index)</code>	Type	To be used for collecting an exchange property from an existing Collection , Map or array (lookup by the index) and then converting the exchange property to the given type determined by its classname. The converted exchange property can be null.
<code>mandatoryExchangePropertyAsIndex(key, type, index)</code>	Type	To be used for collecting an exchange property from an existing Collection , Map or array (lookup by the index) and then converting the exchange property to the given type determined by its classname. Expects the exchange property to be not null.

For example given the following simple expression:

```
Hello ${body[0].name}
```

This script has no type information, and the simple language will resolve this at runtime, by introspecting the message body and if it's a collection based then lookup the first element, and then invoke a method named **getName** via reflection.

In csimple (compiled) we want to pre compile this and therefore the end user must provide type information with the *bodyAsIndex* function:

```
Hello ${bodyAsIndex(com.foo.MyUser, 0).name}
```

28.3. COMPILATION

The csimple language is parsed into regular Java source code and compiled together with all the other source code, or it can be compiled once during bootstrap via the **camel-csimple-joor** module.

There are two ways to compile csimple

- using the **camel-csimple-maven-plugin** generating source code at built time.
- using **camel-csimple-joor** which does runtime in-memory compilation during bootstrap of Camel.

28.3.1. Using camel-csimple-maven-plugin

The **camel-csimple-maven-plugin** Maven plugin is used for discovering all the csimple scripts from the source code, and then automatic generate source code in the **src/generated/java** folder, which then gets compiled together with all the other sources.

The maven plugin will do source code scanning of **.java** and **.xml** files (Java and XML DSL). The scanner limits to detect certain code patterns, and it may miss discovering some csimple scripts if they are being used in unusual/rare ways.

The runtime compilation using **camel-csimple-joor** does not have this limitation.

The benefit is all the csimple scripts will be compiled using the regular Java compiler and therefore everything is included out of the box as **.class** files in the application JAR file, and no additional dependencies is required at runtime.

To use **camel-csimple-maven-plugin** you need to add it to your **pom.xml** file as shown:

```
<plugins>
  <!-- generate source code for csimple languages -->
  <plugin>
    <groupId>org.apache.camel</groupId>
    <artifactId>camel-csimple-maven-plugin</artifactId>
    <version>${camel.version}</version>
    <executions>
      <execution>
        <id>generate</id>
        <goals>
          <goal>generate</goal>
        </goals>
      </execution>
    </executions>
  </plugin>
  <!-- include source code generated to maven sources paths -->
  <plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>build-helper-maven-plugin</artifactId>
    <version>3.1.0</version>
    <executions>
      <execution>
        <phase>generate-sources</phase>
        <goals>
          <goal>add-source</goal>
          <goal>add-resource</goal>
        </goals>
        <configuration>
          <sources>
            <source>src/generated/java</source>
          </sources>
          <resources>
            <resource>
              <directory>src/generated/resources</directory>
            </resource>
          </resources>
        </configuration>
      </execution>
    </executions>
  </plugin>
</plugins>
```

```

    </executions>
  </plugin>
</plugins>

```

And then you must also add the **build-helper-maven-plugin** Maven plugin to include **src/generated** to the list of source folders for the Java compiler, to ensure the generated source code is compiled and included in the application JAR file.

See the **camel-example-csimple** example at [Camel Examples](#) which uses the maven plugin.

28.3.2. Using camel-csimple-joor

The jOOR library integrates with the Java compiler and performs runtime compilation of Java code.

The supported runtime when using **camel-simple-joor** is intended for Java standalone, Spring Boot, Camel Quarkus and other microservices runtimes. It is not supported in OSGi, Camel Karaf or any kind of Java Application Server runtime.

jOOR does not support runtime compilation with Spring Boot using *fat jar* packaging (<https://github.com/jOOQ/jOOR/issues/69>), it works with exploded classpath.

To use **camel-simple-joor** you simply just add it as dependency to the classpath:

```

<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-csimple-joor</artifactId>
  <version>{CamelSBProjectVersion}</version>
</dependency>

```

There is no need for adding Maven plugins to the **pom.xml** file.

See the **camel-example-csimple-joor** example at [Camel Examples](#) which uses the jOOR compiler.

28.4. CSIMPLE LANGUAGE OPTIONS

The CSimple language supports 2 options, which are listed below.

Name	Default	Java Type	Description
resultType		String	Sets the class name of the result type (type from output).
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

28.5. LIMITATIONS

Currently, the csimple language does **not** support:

- nested functions (aka functions inside functions)

- the *null safe* operator (`?`).

For example the following scripts cannot compile:

```
Hello ${bean:greeter(${body}, ${header.counter})}
```

```
${bodyAs(MyUser)?.address?.zip} > 10000
```

28.6. AUTO IMPORTS

The csimple language will automatically import from:

```
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;
import org.apache.camel.*;
import org.apache.camel.util.*;
```

28.7. CONFIGURATION FILE

You can configure the csimple language in the **camel-csimple.properties** file which is loaded from the root classpath.

For example you can add additional imports in the **camel-csimple.properties** file by adding:

```
import com.foo.MyUser;
import com.bar.*;
import static com.foo.MyHelper.*;
```

You can also add aliases (key=value) where an alias will be used as a shorthand replacement in the code.

```
echo()=${bodyAs(String)} ${bodyAs(String)}
```

Which allows to use `echo()` in the csimple language script such as:

```
from("direct:hello")
  .transform(csimple("Hello echo()"))
  .log("You said ${body}");
```

The `echo()` alias will be replaced with its value resulting in a script as:

```
.transform(csimple("Hello ${bodyAs(String)} ${bodyAs(String)}"))
```

28.8. SEE ALSO

See the [Simple](#) language as **csimple** has the same set of functions as simple language.

28.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 147 options, which are listed below.

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.acl-token</code>	Sets the ACL token to be used with Consul.		String
<code>camel.cloud.consul.service-discovery.block-seconds</code>	The seconds to wait for a watch event, default 10 seconds.	10	Integer
<code>camel.cloud.consul.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.consul.service-discovery.connect-timeout-millis</code>	Connect timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.datacenter</code>	The data center.		String
<code>camel.cloud.consul.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.consul.service-discovery.password</code>	Sets the password to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.consul.service-discovery.read-timeout-millis</code>	Read timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.url</code>	The Consul agent URL.		String

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.username</code>	Sets the username to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.write-timeout-millis</code>	Write timeout for OkHttpClient.		Long
<code>camel.cloud.dns.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.dns.service-discovery.domain</code>	The domain name;.		String
<code>camel.cloud.dns.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.dns.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.dns.service-discovery.proto</code>	The transport protocol of the desired service.	<code>_tcp</code>	String
<code>camel.cloud.etcd.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.etcd.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.etcd.service-discovery.password</code>	The password to use for basic authentication.		String

Name	Description	Default	Type
<code>camel.cloud.etcd.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.etcd.service-discovery.service-path</code>	The path to look for for service discovery.	<code>/services/</code>	String
<code>camel.cloud.etcd.service-discovery.timeout</code>	To set the maximum time an action could take to complete.		Long
<code>camel.cloud.etcd.service-discovery.type</code>	To set the discovery type, valid values are on-demand and watch.	on-demand	String
<code>camel.cloud.etcd.service-discovery.uris</code>	The URIs the client can connect to.		String
<code>camel.cloud.etcd.service-discovery.user-name</code>	The user name to use for basic authentication.		String
<code>camel.cloud.kubernetes.service-discovery.api-version</code>	Sets the API version when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-data</code>	Sets the Certificate Authority data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-file</code>	Sets the Certificate Authority data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-data</code>	Sets the Client Certificate data when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.client-cert-file</code>	Sets the Client Certificate data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-algo</code>	Sets the Client Keystore algorithm, such as RSA when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-data</code>	Sets the Client Keystore data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-file</code>	Sets the Client Keystore data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-passphrase</code>	Sets the Client Keystore passphrase when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.kubernetes.service-discovery.dns-domain</code>	Sets the DNS domain to use for DNS lookup.		String
<code>camel.cloud.kubernetes.service-discovery.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
camel.cloud.kubernetes.service-discovery.lookup	How to perform service lookup. Possible values: client, dns, environment. When using client, then the client queries the kubernetes master to obtain a list of active pods that provides the service, and then random (or round robin) select a pod. When using dns the service name is resolved as name.namespace.svc.dnsDomain. When using dnssrv the service name is resolved with SRV query forsvc... When using environment then environment variables are used to lookup the service. By default environment is used.	environment	String
camel.cloud.kubernetes.service-discovery.master-url	Sets the URL to the master when using client lookup.		String
camel.cloud.kubernetes.service-discovery.namespace	Sets the namespace to use. Will by default use namespace from the ENV variable KUBERNETES_MASTER.		String
camel.cloud.kubernetes.service-discovery.oauth-token	Sets the OAUTH token for authentication (instead of username/password) when using client lookup.		String
camel.cloud.kubernetes.service-discovery.password	Sets the password for authentication when using client lookup.		String
camel.cloud.kubernetes.service-discovery.port-name	Sets the Port Name to use for DNS/DNSSRV lookup.		String
camel.cloud.kubernetes.service-discovery.port-protocol	Sets the Port Protocol to use for DNS/DNSSRV lookup.		String
camel.cloud.kubernetes.service-discovery.properties	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in com.netflix.client.config.CommonClientConfigKey.		Map

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.trust-certs</code>	Sets whether to turn on trust certificate check when using client lookup.	false	Boolean
<code>camel.cloud.kubernetes.service-discovery.username</code>	Sets the username for authentication when using client lookup.		String
<code>camel.cloud.ribbon.load-balancer.client-name</code>	Sets the Ribbon client name.		String
<code>camel.cloud.ribbon.load-balancer.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.ribbon.load-balancer.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.ribbon.load-balancer.namespace</code>	The namespace.		String
<code>camel.cloud.ribbon.load-balancer.password</code>	The password.		String
<code>camel.cloud.ribbon.load-balancer.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.ribbon.load-balancer.username</code>	The username.		String

Name	Description	Default	Type
camel.hystrix.allow-maximum-size-to-diverge-from-core-size	Allows the configuration for <code>maximumSize</code> to take effect. That value can then be equal to, or higher, than <code>coreSize</code> .	false	Boolean
camel.hystrix.circuit-breaker-enabled	Whether to use a <code>HystrixCircuitBreaker</code> or not. If false no circuit-breaker logic will be used and all requests permitted. This is similar in effect to <code>circuitBreakerForceClosed()</code> except that continues tracking metrics and knowing whether it should be open/closed, this property results in not even instantiating a circuit-breaker.	true	Boolean
camel.hystrix.circuit-breaker-error-threshold-percentage	Error percentage threshold (as whole number such as 50) at which point the circuit breaker will trip open and reject requests. It will stay tripped for the duration defined in <code>circuitBreakerSleepWindowInMilliseconds</code> ; The error percentage this is compared against comes from <code>HystrixCommandMetrics.getHealthCounts()</code> .	50	Integer
camel.hystrix.circuit-breaker-force-closed	If true the <code>HystrixCircuitBreaker#allowRequest()</code> will always return true to allow requests regardless of the error percentage from <code>HystrixCommandMetrics.getHealthCounts()</code> . The <code>circuitBreakerForceOpen()</code> property takes precedence so if it set to true this property does nothing.	false	Boolean
camel.hystrix.circuit-breaker-force-open	If true the <code>HystrixCircuitBreaker.allowRequest()</code> will always return false, causing the circuit to be open (tripped) and reject all requests. This property takes precedence over <code>circuitBreakerForceClosed()</code> .	false	Boolean
camel.hystrix.circuit-breaker-request-volume-threshold	Minimum number of requests in the <code>metricsRollingStatisticalWindowInMilliseconds()</code> that must exist before the <code>HystrixCircuitBreaker</code> will trip. If below this number the circuit will not trip regardless of error percentage.	20	Integer
camel.hystrix.circuit-breaker-sleep-window-in-milliseconds	The time in milliseconds after a <code>HystrixCircuitBreaker</code> trips open that it should wait before trying requests again.	5000	Integer
camel.hystrix.configurations	Define additional configuration definitions.		Map

Name	Description	Default	Type
camel.hystrix.core-pool-size	Core thread-pool size that gets passed to <code>java.util.concurrent.ThreadPoolExecutor#setCorePoolSize(int)</code> .	10	Integer
camel.hystrix.enabled	Enable the component.	true	Boolean
camel.hystrix.execution-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.run()</code> . Requests beyond the concurrent limit will be rejected. Applicable only when <code>executionIsolationStrategy == SEMAPHORE</code> .	20	Integer
camel.hystrix.execution-isolation-strategy	What isolation strategy <code>HystrixCommand.run()</code> will be executed with. If <code>THREAD</code> then it will be executed on a separate thread and concurrent requests limited by the number of threads in the thread-pool. If <code>SEMAPHORE</code> then it will be executed on the calling thread and concurrent requests limited by the semaphore count.	THREAD	String
camel.hystrix.execution-isolation-thread-interrupt-on-timeout	Whether the execution thread should attempt an interrupt (using <code>Future#cancel()</code>) when a thread times out. Applicable only when <code>executionIsolationStrategy() == THREAD</code> .	true	Boolean
camel.hystrix.execution-timeout-enabled	Whether the timeout mechanism is enabled for this command.	true	Boolean
camel.hystrix.execution-timeout-in-milliseconds	Time in milliseconds at which point the command will timeout and halt execution. If <code>executionIsolationThreadInterruptOnTimeout == true</code> and the command is thread-isolated, the executing thread will be interrupted. If the command is semaphore-isolated and a <code>HystrixObservableCommand</code> , that command will get unsubscribed.	1000	Integer
camel.hystrix.fallback-ack-enabled	Whether <code>HystrixCommand.getFallback()</code> should be attempted when failure occurs.	true	Boolean
camel.hystrix.fallback-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.getFallback()</code> . Requests beyond the concurrent limit will fail-fast and not attempt retrieving a fallback.	10	Integer

Name	Description	Default	Type
camel.hystrix.group-key	Sets the group key to use. The default value is CamelHystrix.	Camel Hystrix	String
camel.hystrix.keep-alive-time	Keep-alive time in minutes that gets passed to <code>ThreadPoolExecutor#setKeepAliveTime(long, TimeUnit)</code> .	1	Integer
camel.hystrix.max-queue-size	Max queue size that gets passed to <code>BlockingQueue</code> in <code>HystrixConcurrencyStrategy.getBlockingQueue(int)</code> . This should only affect the instantiation of a threadpool - it is not eligible to change a queue size on the fly. For that, use <code>queueSizeRejectionThreshold()</code> .	-1	Integer
camel.hystrix.maximum-size	Maximum thread-pool size that gets passed to <code>ThreadPoolExecutor#setMaximumPoolSize(int)</code> . This is the maximum amount of concurrency that can be supported without starting to reject <code>HystrixCommands</code> . Please note that this setting only takes effect if you also set <code>allowMaximumSizeToDivergeFromCoreSize</code> .	10	Integer
camel.hystrix.metrics-health-snapshot-interval-in-milliseconds	Time in milliseconds to wait between allowing health snapshots to be taken that calculate success and error percentages and affect <code>HystrixCircuitBreaker.isOpen()</code> status. On high-volume circuits the continual calculation of error percentage can become CPU intensive thus this controls how often it is calculated.	500	Integer
camel.hystrix.metrics-rolling-percentile-bucket-size	Maximum number of values stored in each bucket of the rolling percentile. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-percentile-enabled	Whether percentile metrics should be captured using <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	true	Boolean
camel.hystrix.metrics-rolling-percentile-window-buckets	Number of buckets the rolling percentile window is broken into. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	6	Integer

Name	Description	Default	Type
camel.hystrix.metrics-rolling-percentile-window-in-milliseconds	Duration of percentile rolling window in milliseconds. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	10000	Integer
camel.hystrix.metrics-rolling-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside HystrixCommandMetrics.	10	Integer
camel.hystrix.metrics-rolling-statistical-window-in-milliseconds	This property sets the duration of the statistical rolling window, in milliseconds. This is how long metrics are kept for the thread pool. The window is divided into buckets and rolls by those increments.	10000	Integer
camel.hystrix.queue-size-rejection-threshold	Queue size rejection threshold is an artificial max size at which rejections will occur even if maxQueueSize has not been reached. This is done because the maxQueueSize of a BlockingQueue can not be dynamically changed and we want to support dynamically changing the queue size that affects rejections. This is used by HystrixCommand when queuing a thread for execution.	5	Integer
camel.hystrix.request-log-enabled	Whether HystrixCommand execution and events should be logged to HystrixRequestLog.	true	Boolean
camel.hystrix.thread-pool-key	Sets the thread pool key to use. Will by default use the same value as groupKey has been configured to use.	Camel Hystrix	String
camel.hystrix.thread-pool-rolling-number-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10	Integer
camel.hystrix.thread-pool-rolling-number-statistical-window-in-milliseconds	Duration of statistical rolling window in milliseconds. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10000	Integer
camel.language.constant.enabled	Whether to enable auto configuration of the constant language. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.language.constant.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.csimple.enabled</code>	Whether to enable auto configuration of the csimple language. This is enabled by default.		Boolean
<code>camel.language.csimple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.exchangeproperty.enabled</code>	Whether to enable auto configuration of the exchangeProperty language. This is enabled by default.		Boolean
<code>camel.language.exchangeproperty.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.file.enabled</code>	Whether to enable auto configuration of the file language. This is enabled by default.		Boolean
<code>camel.language.file.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.header.enabled</code>	Whether to enable auto configuration of the header language. This is enabled by default.		Boolean
<code>camel.language.header.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.ref.enabled</code>	Whether to enable auto configuration of the ref language. This is enabled by default.		Boolean
<code>camel.language.ref.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.simple.enabled</code>	Whether to enable auto configuration of the simple language. This is enabled by default.		Boolean
<code>camel.language.simple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

Name	Description	Default	Type
<code>camel.language.tokenize.enabled</code>	Whether to enable auto configuration of the tokenize language. This is enabled by default.		Boolean
<code>camel.language.tokenize.group-delimiter</code>	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.		String
<code>camel.language.tokenize.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.resilience4j.automatic-transition-from-open-to-half-open-enabled</code>	Enables automatic transition from OPEN to HALF_OPEN state once the <code>waitDurationInOpenState</code> has passed.	false	Boolean
<code>camel.resilience4j.circuit-breaker-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreaker</code> instance to lookup and use from the registry. When using this, then any other circuit breaker options are not in use.		String
<code>camel.resilience4j.config-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreakerConfig</code> instance to lookup and use from the registry.		String
<code>camel.resilience4j.configurations</code>	Define additional configuration definitions.		Map
<code>camel.resilience4j.enabled</code>	Enable the component.	true	Boolean
<code>camel.resilience4j.failure-rate-threshold</code>	Configures the failure rate threshold in percentage. If the failure rate is equal or greater than the threshold the <code>CircuitBreaker</code> transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 50 percentage.		Float

Name	Description	Default	Type
camel.resilience4j.minimum-number-of-calls	Configures the minimum number of calls which are required (per sliding window period) before the CircuitBreaker can calculate the error rate. For example, if <code>minimumNumberOfCalls</code> is 10, then at least 10 calls must be recorded, before the failure rate can be calculated. If only 9 calls have been recorded the CircuitBreaker will not transition to open even if all 9 calls have failed. Default <code>minimumNumberOfCalls</code> is 100.	100	Integer
camel.resilience4j.permitted-number-of-calls-in-half-open-state	Configures the number of permitted calls when the CircuitBreaker is half open. The size must be greater than 0. Default size is 10.	10	Integer
camel.resilience4j.sliding-window-size	Configures the size of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. <code>slidingWindowSize</code> configures the size of the sliding window. Sliding window can either be count-based or time-based. If <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the last <code>slidingWindowSize</code> calls are recorded and aggregated. If <code>slidingWindowType</code> is <code>TIME_BASED</code> , the calls of the last <code>slidingWindowSize</code> seconds are recorded and aggregated. The <code>slidingWindowSize</code> must be greater than 0. The <code>minimumNumberOfCalls</code> must be greater than 0. If the <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the <code>minimumNumberOfCalls</code> cannot be greater than <code>slidingWindowSize</code> . If the <code>slidingWindowType</code> is <code>TIME_BASED</code> , you can pick whatever you want. Default <code>slidingWindowSize</code> is 100.	100	Integer
camel.resilience4j.sliding-window-type	Configures the type of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. Sliding window can either be count-based or time-based. If <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the last <code>slidingWindowSize</code> calls are recorded and aggregated. If <code>slidingWindowType</code> is <code>TIME_BASED</code> , the calls of the last <code>slidingWindowSize</code> seconds are recorded and aggregated. Default <code>slidingWindowType</code> is <code>COUNT_BASED</code> .	<code>COUNT_BASED</code>	String
camel.resilience4j.slow-call-duration-threshold	Configures the duration threshold (seconds) above which calls are considered as slow and increase the slow calls percentage. Default value is 60 seconds.	60	Integer

Name	Description	Default	Type
camel.resilience4j.slow-call-rate-threshold	Configures a threshold in percentage. The CircuitBreaker considers a call as slow when the call duration is greater than <code>slowCallDurationThreshold</code> . When the percentage of slow calls is equal or greater the threshold, the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 100 percentage which means that all recorded calls must be slower than <code>slowCallDurationThreshold</code> .		Float
camel.resilience4j.wait-duration-in-open-state	Configures the wait duration (in seconds) which specifies how long the CircuitBreaker should stay open, before it switches to half open. Default value is 60 seconds.	60	Integer
camel.resilience4j.writable-stack-trace-enabled	Enables writable stack traces. When set to false, <code>Exception.getStackTrace</code> returns a zero length array. This may be used to reduce log spam when the circuit breaker is open as the cause of the exceptions is already known (the circuit breaker is short-circuiting calls).	true	Boolean
camel.rest.api-component	The name of the Camel component to use as the REST API (such as swagger) If no API Component has been explicit configured, then Camel will lookup if there is a Camel component responsible for servicing and generating the REST API documentation, or if a <code>org.apache.camel.spi.RestApiProcessorFactory</code> is registered in the registry. If either one is found, then that is being used.		String
camel.rest.api-context-path	Sets a leading API context-path the REST API services will be using. This can be used when using components such as <code>camel-servlet</code> where the deployed web application is deployed using a context-path.		String
camel.rest.api-context-route-id	Sets the route id to use for the route that services the REST API. The route will by default use an auto assigned route id.		String
camel.rest.api-host	To use an specific hostname for the API documentation (eg swagger) This can be used to override the generated host with this configured hostname.		String

Name	Description	Default	Type
camel.rest.api-property	Allows to configure as many additional properties for the api documentation (swagger). For example set property api.title to my cool stuff.		Map
camel.rest.api-vendor-extension	Whether vendor extension is enabled in the Rest APIs. If enabled then Camel will include additional information as vendor extension (eg keys starting with x-) such as route ids, class names etc. Not all 3rd party API gateways and tools supports vendor-extensions when importing your API docs.	false	Boolean
camel.rest.binding-mode	Sets the binding mode to use. The default value is off.		RestBindingMode
camel.rest.client-request-validation	Whether to enable validation of the client request to check whether the Content-Type and Accept headers from the client is supported by the Rest-DSL configuration of its consumes/produces settings. This can be turned on, to enable this check. In case of validation error, then HTTP Status codes 415 or 406 is returned. The default value is false.	false	Boolean
camel.rest.component	The Camel Rest component to use for the REST transport (consumer), such as netty-http, jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String
camel.rest.component-property	Allows to configure as many additional properties for the rest component in use.		Map
camel.rest.consumer-property	Allows to configure as many additional properties for the rest consumer in use.		Map
camel.rest.context-path	Sets a leading context-path the REST services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path. Or for components such as camel-jetty or camel-netty-http that includes a HTTP server.		String
camel.rest.cors-headers	Allows to configure custom CORS headers.		Map

Name	Description	Default	Type
camel.rest.data-format-property	Allows to configure as many additional properties for the data formats in use. For example set property <code>prettyPrint</code> to <code>true</code> to have json outputted in pretty mode. The properties can be prefixed to denote the option is only for either JSON or XML and for either the IN or the OUT. The prefixes are: <code>json.in</code> , <code>json.out</code> , <code>xml.in</code> , <code>xml.out</code> . For example a key with value <code>xml.out.mustBeJAXBElement</code> is only for the XML data format for the outgoing. A key without a prefix is a common key for all situations.		Map
camel.rest.enable-cors	Whether to enable CORS headers in the HTTP response. The default value is false.	false	Boolean
camel.rest.endpoint-property	Allows to configure as many additional properties for the rest endpoint in use.		Map
camel.rest.host	The hostname to use for exposing the REST service.		String
camel.rest.host-name-resolver	If no hostname has been explicit configured, then this resolver is used to compute the hostname the REST service will be using.		RestHostNameResolver
camel.rest.json-data-format	Name of specific json data format to use. By default <code>json-jackson</code> will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.port	The port number to use for exposing the REST service. Notice if you use servlet component then the port number configured here does not apply, as the port number in use is the actual port number the servlet component is using. eg if using Apache Tomcat its the tomcat http port, if using Apache Karaf its the HTTP service in Karaf that uses port 8181 by default etc. Though in those situations setting the port number here, allows tooling and JMX to know the port number, so its recommended to set the port number to the number that the servlet engine uses.		String
camel.rest.producer-api-doc	Sets the location of the api document (swagger api) the REST producer will use to validate the REST uri and query parameters are valid accordingly to the api document. This requires adding <code>camel-swagger-java</code> to the classpath, and any miss configuration will let Camel fail on startup and report the error(s). The location of the api document is loaded from classpath by default, but you can use <code>file:</code> or <code>http:</code> to refer to resources to load from file or http url.		String

Name	Description	Default	Type
camel.rest.producer-component	Sets the name of the Camel component to use as the REST producer.		String
camel.rest.scheme	The scheme to use for exposing the REST service. Usually http or https is supported. The default value is http.		String
camel.rest.skip-binding-on-error-code	Whether to skip binding on output if there is a custom HTTP error code header. This allows to build custom error messages that do not bind to json / xml etc, as success messages otherwise will do.	false	Boolean
camel.rest.use-x-forward-headers	Whether to use X-Forward headers for Host and related setting. The default value is true.	true	Boolean
camel.rest.xml-data-format	Name of specific XML data format to use. By default jaxb will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.api-context-id-pattern	Deprecated Sets an CamelContext id pattern to only allow Rest APIs from rest services within CamelContext's which name matches the pattern. The pattern name refers to the CamelContext name, to match on the current CamelContext only. For any other value, the pattern uses the rules from PatternHelper#matchPattern(String,String).		String
camel.rest.api-context-listing	Deprecated Sets whether listing of all available CamelContext's with REST services in the JVM is enabled. If enabled it allows to discover these contexts, if false then only the current CamelContext is in use.	false	Boolean

CHAPTER 29. CXF

Both producer and consumer are supported

The CXF component provides integration with [Apache CXF](#) for connecting to [JAX-WS](#) services hosted in CXF.

TIP

When using CXF in streaming modes (see `DataFormat` option), then also read about Stream caching.

29.1. DEPENDENCIES

When using **cxf** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-cxf-soap-starter</artifactId>
</dependency>
```

29.2. URI FORMAT

There are two URI formats for this endpoint: **cxfEndpoint** and **someAddress**.

```
cxf:bean:cxfEndpoint[?options]
```

Where **cxfEndpoint** represents a bean ID that references a bean in the Spring bean registry. With this URI format, most of the endpoint details are specified in the bean definition.

```
cxf://someAddress[?options]
```

Where **someAddress** specifies the CXF endpoint's address. With this URI format, most of the endpoint details are specified using options.

For either style above, you can append options to the URI as follows:

```
cxf:bean:cxfEndpoint?wsdlURL=wsdl/hello_world.wsdl&dataFormat=PAYLOAD
```

29.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

29.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings,

credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

29.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

29.4. COMPONENT OPTIONS

The CXF component supports 6 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
allowStreaming (advanced)	This option controls whether the CXF component, when running in PAYLOAD mode, will DOM parse the incoming messages into DOM Elements or keep the payload as a javax.xml.transform.Source object that would allow streaming in some cases.		Boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
headerFilterStrategy (filter)	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean

29.5. ENDPOINT OPTIONS

The CXF endpoint is configured using URI syntax:

```
cxf:beanId:address
```

with the following path and query parameters:

29.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
beanId (common)	To lookup an existing configured CxfEndpoint. Must used bean: as prefix.		String
address (service)	The service publish address.		String

29.5.2. Query Parameters (35 parameters)

Name	Description	Default	Type
dataFormat (common)	The data type messages supported by the CXF endpoint. Enum values: • PAYLOAD • RAW • MESSAGE • CXF_MESSAGE • POJO	POJO	DataFormat
wrappedStyle (common)	The WSDL style that describes how parameters are represented in the SOAP body. If the value is false, CXF will chose the document-literal unwrapped style, If the value is true, CXF will chose the document-literal wrapped style.		Boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
cookieHandler (producer)	Configure a cookie handler to maintain a HTTP session.		CookieHandler
defaultOperationName (producer)	This option will set the default operationName that will be used by the CxfProducer which invokes the remote service.		String
defaultOperationNamespace (producer)	This option will set the default operationNamespace that will be used by the CxfProducer which invokes the remote service.		String
hostnameVerifier (producer)	The hostname verifier to be used. Use the # notation to reference a HostnameVerifier from the registry.		HostnameVerifier
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
sslContextParameters (producer)	The Camel SSL setting reference. Use the # notation to reference the SSL Context.		SSLContextParameters
wrapped (producer)	Which kind of operation that CXF endpoint producer will invoke.	false	boolean
synchronous (producer (advanced))	Sets whether synchronous processing should be strictly used.	false	boolean

Name	Description	Default	Type
allowStreaming (advanced)	This option controls whether the CXF component, when running in PAYLOAD mode, will DOM parse the incoming messages into DOM Elements or keep the payload as a javax.xml.transform.Source object that would allow streaming in some cases.		Boolean
bus (advanced)	To use a custom configured CXF Bus.		Bus
continuationTimeout (advanced)	This option is used to set the CXF continuation timeout which could be used in CxfConsumer by default when the CXF server is using Jetty or Servlet transport.	30000	long
cxfBinding (advanced)	To use a custom CxfBinding to control the binding between Camel Message and CXF Message.		CxfBinding
cxfConfigurer (advanced)	This option could apply the implementation of org.apache.camel.component.cxf.CxfEndpointConfigurer which supports to configure the CXF endpoint in programmatic way. User can configure the CXF server and client by implementing configure{ServerClient} method of CxfEndpointConfigurer.		CxfConfigurer
defaultBus (advanced)	Will set the default bus when CXF endpoint create a bus by itself.	false	boolean
headerFilterStrategy (advanced)	To use a custom HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
mergeProtocolHeaders (advanced)	Whether to merge protocol headers. If enabled then propagating headers between Camel and CXF becomes more consistent and similar. For more details see CAMEL-6393.	false	boolean
mtomEnabled (advanced)	To enable MTOM (attachments). This requires to use POJO or PAYLOAD data format mode.	false	boolean
properties (advanced)	To set additional CXF options using the key/value pairs from the Map. For example to turn on stacktraces in SOAP faults, properties.faultStackTraceEnabled=true.		Map
skipPayloadMessagePartCheck (advanced)	Sets whether SOAP message validation should be disabled.	false	boolean

Name	Description	Default	Type
loggingFeatureEnabled (logging)	This option enables CXF Logging Feature which writes inbound and outbound SOAP messages to log.	false	boolean
loggingSizeLimit (logging)	To limit the total size of number of bytes the logger will output when logging feature has been enabled and -1 for no limit.	49152	int
skipFaultLogging (logging)	This option controls whether the PhaseInterceptorChain skips logging the Fault that it catches.	false	boolean
password (security)	This option is used to set the basic authentication information of password for the CXF client.		String
username (security)	This option is used to set the basic authentication information of username for the CXF client.		String
bindingId (service)	The bindingId for the service model to use.		String
portName (service)	The endpoint name this service is implementing, it maps to the wsdl:portname. In the format of ns:PORT_NAME where ns is a namespace prefix valid at this scope.		String
publishedEndpointUrl (service)	This option can override the endpointUrl that published from the WSDL which can be accessed with service address url plus wsdl.		String
serviceClass (service)	The class name of the SEI (Service Endpoint Interface) class which could have JSR181 annotation or not.		Class
serviceName (service)	The service name this service is implementing, it maps to the wsdl:servicename.		String
wsdlURL (service)	The location of the WSDL. Can be on the classpath, file system, or be hosted remotely.		String

The **serviceName** and **portName** are [QNames](#), so if you provide them be sure to prefix them with their {namespace} as shown in the examples above.

29.5.3. Descriptions of the dataformats

In Apache Camel, the Camel CXF component is the key to integrating routes with Web services. You can use the Camel CXF component to create a CXF endpoint, which can be used in either of the following ways:

- **Consumer** – (at the start of a route) represents a Web service instance, which integrates with the route. The type of payload injected into the route depends on the value of the endpoint's `dataFormat` option.
- **Producer** – (at other points in the route) represents a WS client proxy, which converts the current exchange object into an operation invocation on a remote Web service. The format of the current exchange must match the endpoint's `dataFormat` setting.

DataFormat	Description
POJO	POJOs (Plain old Java objects) are the Java parameters to the method being invoked on the target server. Both Protocol and Logical JAX-WS handlers are supported.
PAYLOAD	PAYLOAD is the message payload (the contents of the soap:body) after message configuration in the CXF endpoint is applied. Only Protocol JAX-WS handler is supported. Logical JAX-WS handler is not supported.
RAW	RAW mode provides the raw message stream that is received from the transport layer. It is not possible to touch or change the stream, some of the CXF interceptors will be removed if you are using this kind of DataFormat, so you can't see any soap headers after the camel-cxf consumer. JAX-WS handler is not supported.
CXF_MESSAGE	CXF_MESSAGE allows for invoking the full capabilities of CXF interceptors by converting the message from the transport layer into a raw SOAP message

You can determine the data format mode of an exchange by retrieving the exchange property, **CamelCXFDataFormat**. The exchange key constant is defined in **org.apache.camel.component.cxf.common.message.CxfConstants.DATA_FORMAT_PROPERTY**.

29.5.4. How to enable CXF's LoggingOutInterceptor in RAW mode

CXF's **LoggingOutInterceptor** outputs outbound message that goes on the wire to logging system (Java Util Logging). Since the **LoggingOutInterceptor** is in **PRE_STREAM** phase (but **PRE_STREAM** phase is removed in **RAW** mode), you have to configure **LoggingOutInterceptor** to be run during the **WRITE** phase. The following is an example.

```
@Bean
public CxfEndpoint serviceEndpoint(LoggingOutInterceptor loggingOutInterceptor) {
    CxfSpringEndpoint cxfEndpoint = new CxfSpringEndpoint();
    cxfEndpoint.setAddress("http://localhost:" + port
        + "/services" + SERVICE_ADDRESS);
    cxfEndpoint.setServiceClass(org.apache.camel.component.cxf.HelloService.class);
    Map<String, Object> properties = new HashMap<String, Object>();
    properties.put("dataFormat", "RAW");
    cxfEndpoint.setProperties(properties);
    cxfEndpoint.getOutInterceptors().add(loggingOutInterceptor);
}
```

```

    return cxfEndpoint;
}

@Bean
public LoggingOutInterceptor loggingOutInterceptor() {
    LoggingOutInterceptor logger = new LoggingOutInterceptor("write");
    return logger;
}

```

29.5.5. Description of relayHeaders option

There are *in-band* and *out-of-band* on-the-wire headers from the perspective of a JAXWS WSDL-first developer.

The *in-band* headers are headers that are explicitly defined as part of the WSDL binding contract for an endpoint such as SOAP headers.

The *out-of-band* headers are headers that are serialized over the wire, but are not explicitly part of the WSDL binding contract.

Headers relaying/filtering is bi-directional.

When a route has a CXF endpoint and the developer needs to have on-the-wire headers, such as SOAP headers, be relayed along the route to be consumed say by another JAXWS endpoint, then **relayHeaders** should be set to **true**, which is the default value.

29.5.6. Available only in POJO mode

The **relayHeaders=true** expresses an intent to relay the headers. The actual decision on whether a given header is relayed is delegated to a pluggable instance that implements the **MessageHeadersRelay** interface. A concrete implementation of **MessageHeadersRelay** will be consulted to decide if a header needs to be relayed or not. There is already an implementation of **SoapMessageHeadersRelay** which binds itself to well-known SOAP name spaces. Currently only out-of-band headers are filtered, and in-band headers will always be relayed when **relayHeaders=true**. If there is a header on the wire whose name space is unknown to the runtime, then a fall back **DefaultMessageHeadersRelay** will be used, which simply allows all headers to be relayed.

The **relayHeaders=false** setting specifies that all headers in-band and out-of-band should be dropped.

You can plugin your own **MessageHeadersRelay** implementations overriding or adding additional ones to the list of relays. In order to override a preloaded relay instance just make sure that your **MessageHeadersRelay** implementation services the same name spaces as the one you looking to override. Also note, that the overriding relay has to service all of the name spaces as the one you looking to override, or else a runtime exception on route start up will be thrown as this would introduce an ambiguity in name spaces to relay instance mappings.

```

<cxf:cxfEndpoint ...>
  <cxf:properties>
    <entry key="org.apache.camel.cxf.message.headers.relays">
      <list>
        <ref bean="customHeadersRelay"/>
      </list>
    </entry>
  </cxf:properties>

```

```

</cxf:cxfrEndpoint>
<bean id="customHeadersRelay"
class="org.apache.camel.component.cxf.soap.headers.CustomHeadersRelay"/>

```

Take a look at the tests that show how you'd be able to relay/drop headers here:

<https://github.com/apache/camel/blob/main/components/camel-cxf/camel-cxf-spring-soap/src/test/java/org/apache/camel/component/cxf/soap/headers/CxfMessageHeadersRelayTest.java>

- **POJO** and **PAYLOAD** modes are supported. In **POJO** mode, only out-of-band message headers are available for filtering as the in-band headers have been processed and removed from header list by CXF. The in-band headers are incorporated into the **MessageContentList** in POJO mode. The **camel-cxf** component does make any attempt to remove the in-band headers from the **MessageContentList**. If filtering of in-band headers is required, please use **PAYLOAD** mode or plug in a (pretty straightforward) CXF interceptor/JAXWS Handler to the CXF endpoint.
- The Message Header Relay mechanism has been merged into **CxfHeaderFilterStrategy**. The **relayHeaders** option, its semantics, and default value remain the same, but it is a property of **CxfHeaderFilterStrategy**. Here is an example of configuring it.

```

@Bean
public HeaderFilterStrategy dropAllMessageHeadersStrategy() {
    CxfHeaderFilterStrategy headerFilterStrategy = new CxfHeaderFilterStrategy();
    headerFilterStrategy.setRelayHeaders(false);
    return headerFilterStrategy;
}

```

Then, your endpoint can reference the **CxfHeaderFilterStrategy**.

```

@Bean
public CxfEndpoint routerNoRelayEndpoint(HeaderFilterStrategy dropAllMessageHeadersStrategy) {
    CxfSpringEndpoint cxfEndpoint = new CxfSpringEndpoint();
    cxfEndpoint.setServiceClass(org.apache.camel.component.cxf.soap.headers.HeaderTester.class);
    cxfEndpoint.setAddress("/CxfMessageHeadersRelayTest/HeaderService/routerNoRelayEndpoint");
    cxfEndpoint.setWsdURL("soap_header.wsdl");
    cxfEndpoint.setEndpointNameAsQName(
        QName.valueOf("{http://apache.org/camel/component/cxf/soap/headers}SoapPortNoRelay"));
    cxfEndpoint.setServiceNameAsQName(SERVICENAME);
    Map<String, Object> properties = new HashMap<String, Object>();
    properties.put("dataFormat", "PAYLOAD");
    cxfEndpoint.setProperties(properties);
    cxfEndpoint.setHeaderFilterStrategy(dropAllMessageHeadersStrategy);
    return cxfEndpoint;
}

```

```

@Bean
public CxfEndpoint serviceNoRelayEndpoint(HeaderFilterStrategy dropAllMessageHeadersStrategy)
{
    CxfSpringEndpoint cxfEndpoint = new CxfSpringEndpoint();
    cxfEndpoint.setServiceClass(org.apache.camel.component.cxf.soap.headers.HeaderTester.class);
    cxfEndpoint.setAddress("http://localhost:" + port +
"/services/CxfMessageHeadersRelayTest/HeaderService/routerNoRelayEndpointBackend");
    cxfEndpoint.setWsdURL("soap_header.wsdl");
    cxfEndpoint.setEndpointNameAsQName(

```

```

        QName.valueOf("{http://apache.org/camel/component/cxf/soap/headers}SoapPortNoRelay"));
        cxfEndpoint.setServiceNameAsQName(SERVICENAME);
        Map<String, Object> properties = new HashMap<String, Object>();
        properties.put("dataFormat", "PAYLOAD");
        cxfEndpoint.setProperties(properties);
        cxfEndpoint.setHeaderFilterStrategy(dropAllMessageHeadersStrategy);
        return cxfEndpoint;
    }

```

Then configure the route as follows:

```

from("cxf:bean:routerNoRelayEndpoint")
    .to("cxf:bean:serviceNoRelayEndpoint");

```

- The **MessageHeadersRelay** interface has changed slightly and has been renamed to **MessageHeaderFilter**. It is a property of **CxfHeaderFilterStrategy**. Here is an example of configuring user defined Message Header Filters:

```

@Bean
public HeaderFilterStrategy customMessageFilterStrategy() {
    CxfHeaderFilterStrategy headerFilterStrategy = new CxfHeaderFilterStrategy();
    List<MessageHeaderFilter> headerFilterList = new ArrayList<MessageHeaderFilter>();
    headerFilterList.add(new SoapMessageHeaderFilter());
    headerFilterList.add(new CustomHeaderFilter());
    headerFilterStrategy.setMessageHeaderFilters(headerFilterList);
    return headerFilterStrategy;
}

```

- In addition to **relayHeaders**, the following properties can be configured in **CxfHeaderFilterStrategy**.

Name	Required	Description
relayHeaders	No	All message headers will be processed by Message Header Filters <i>Type: boolean Default: true</i>
relayAllMessageHeaders	No	All message headers will be propagated (without processing by Message Header Filters) <i>Type: boolean Default: false</i>
allowFilterNamespaceClash	No	If two filters overlap in activation namespace, the property control how it should be handled. If the value is true , last one wins. If the value is false , it will throw an exception <i>Type: boolean Default: false</i>

29.6. CONFIGURE THE CXF ENDPOINTS WITH SPRING

You can configure the CXF endpoint with the Spring configuration file shown below, and you can also embed the endpoint into the **camelContext** tags. When you are invoking the service endpoint, you can set the **operationName** and **operationNamespace** headers to explicitly state which operation you are calling.

```

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:cxf="http://camel.apache.org/schema/cxf/jaxws"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://camel.apache.org/schema/cxf/jaxws http://camel.apache.org/schema/cxf/jaxws/camel-
    cxf.xsd
    http://camel.apache.org/schema/spring http://camel.apache.org/schema/spring/camel-
    spring.xsd">
  <cxf:cxfEndpoint id="routerEndpoint" address="http://localhost:9003/CamelContext/RouterPort"
    serviceClass="org.apache.hello_world_soap_http.GreeterImpl"/>
  <cxf:cxfEndpoint id="serviceEndpoint" address="http://localhost:9000/SoapContext/SoapPort"
    wsdlURL="testutils/hello_world.wsdl"
    serviceClass="org.apache.hello_world_soap_http.Greeter"
    endpointName="s:SoapPort"
    serviceName="s:SOAPService"
    xmlns:s="http://apache.org/hello_world_soap_http" />
  <camelContext id="camel" xmlns="http://camel.apache.org/schema/spring">
    <route>
      <from uri="cxf:bean:routerEndpoint" />
      <to uri="cxf:bean:serviceEndpoint" />
    </route>
  </camelContext>
</beans>

```

Be sure to include the JAX-WS **schemaLocation** attribute specified on the root beans element. This allows CXF to validate the file and is required. Also note the namespace declarations at the end of the **<cxf:cxfEndpoint/>** tag. These declarations are required because the combined **{namespace}localName** syntax is presently not supported for this tag's attribute values.

The **cxf:cxfEndpoint** element supports many additional attributes:

Name	Value
PortName	The endpoint name this service is implementing, it maps to the wsdl:port@name . In the format of ns:PORT_NAME where ns is a namespace prefix valid at this scope.
serviceName	The service name this service is implementing, it maps to the wsdl:service@name . In the format of ns:SERVICE_NAME where ns is a namespace prefix valid at this scope.
wsdlURL	The location of the WSDL. Can be on the classpath, file system, or be hosted remotely.
bindingId	The bindingId for the service model to use.
address	The service publish address.
bus	The bus name that will be used in the JAX-WS endpoint.

Name	Value
serviceClass	The class name of the SEI (Service Endpoint Interface) class which could have JSR181 annotation or not.

It also supports many child elements:

Name	Value
cxf:inInterceptors	The incoming interceptors for this endpoint. A list of <bean> or <ref> .
cxf:inFaultInterceptors	The incoming fault interceptors for this endpoint. A list of <bean> or <ref> .
cxf:outInterceptors	The outgoing interceptors for this endpoint. A list of <bean> or <ref> .
cxf:outFaultInterceptors	The outgoing fault interceptors for this endpoint. A list of <bean> or <ref> .
cxf:properties	A properties map which should be supplied to the JAX-WS endpoint. See below.
cxf:handlers	A JAX-WS handler list which should be supplied to the JAX-WS endpoint. See below.
cxf:dataBinding	You can specify the which DataBinding will be use in the endpoint. This can be supplied using the Spring <bean class="MyDataBinding"/> syntax.
cxf:binding	You can specify the BindingFactory for this endpoint to use. This can be supplied using the Spring <bean class="MyBindingFactory"/> syntax.
cxf:features	The features that hold the interceptors for this endpoint. A list of beans or refs
cxf:schemaLocations	The schema locations for endpoint to use. A list of schemaLocations
cxf:serviceFactory	The service factory for this endpoint to use. This can be supplied using the Spring <bean class="MyServiceFactory"/> syntax

You can find more advanced examples that show how to provide interceptors, properties and handlers on the CXF [JAX-WS Configuration page](#).

NOTE

You can use `cxf:properties` to set the camel-cxf endpoint's `dataFormat` and `setDefaultBus` properties from spring configuration file.

```
<cxf:cxfEndpoint id="testEndpoint" address="http://localhost:9000/router"
  serviceClass="org.apache.camel.component.cxf.HelloService"
  endpointName="s:PortName"
  serviceName="s:ServiceName"
  xmlns:s="http://www.example.com/test">
  <cxf:properties>
    <entry key="dataFormat" value="RAW"/>
    <entry key="setDefaultBus" value="true"/>
  </cxf:properties>
</cxf:cxfEndpoint>
```

NOTE

In SpringBoot, you can use Spring XML files to configure **camel-cxf** and use code similar to the following example to create XML configured beans:

```
@ImportResource({
  "classpath:spring-configuration.xml"
})
```

However, the use of Java code configured beans (as shown in other examples) is best practice in SpringBoot.

29.7. HOW TO MAKE THE CAMEL-CXF COMPONENT USE LOG4J INSTEAD OF JAVA.UTIL.LOGGING

CXF's default logger is **java.util.logging**. If you want to change it to log4j, proceed as follows. Create a file, in the classpath, named **META-INF/cxf/org.apache.cxf.logger**. This file should contain the fully-qualified name of the class, **org.apache.cxf.common.logging.Log4jLogger**, with no comments, on a single line.

29.8. HOW TO LET CAMEL-CXF RESPONSE START WITH XML PROCESSING INSTRUCTION

If you are using some SOAP client such as PHP, you will get this kind of error, because CXF doesn't add the XML processing instruction **<?xml version="1.0" encoding="utf-8"?>**:

```
Error:sendSms: SoapFault exception: [Client] looks like we got no XML document in [...]
```

To resolve this issue, you just need to tell `StaxOutInterceptor` to write the XML start document for you, as in the [WriteXmlDeclarationInterceptor](#) below:

```
public class WriteXmlDeclarationInterceptor extends AbstractPhaseInterceptor<SoapMessage> {
  public WriteXmlDeclarationInterceptor() {
    super(Phase.PRE_STREAM);
    addBefore(StaxOutInterceptor.class.getName());
  }
}
```



```

    public void handleMessage(SoapMessage message) throws Fault {
        message.put("org.apache.cxf.stax.force-start-document", Boolean.TRUE);
    }
}

```

As an alternative you can add a message header for it as demonstrated in [CxfConsumerTest](#):

```

// set up the response context which force start document
Map<String, Object> map = new HashMap<String, Object>();
map.put("org.apache.cxf.stax.force-start-document", Boolean.TRUE);
exchange.getOut().setHeader(Client.RESPONSE_CONTEXT, map);

```

29.9. HOW TO OVERRIDE THE CXF PRODUCER ADDRESS FROM MESSAGE HEADER

The **camel-cxf** producer supports to override the target service address by setting a message header **CamelDestinationOverrideUrl**.

```

// set up the service address from the message header to override the setting of CXF endpoint
exchange.getIn().setHeader(Exchange.DESTINATION_OVERRIDE_URL,
    constant(getServiceAddress()));

```

29.10. HOW TO CONSUME A MESSAGE FROM A CAMEL-CXF ENDPOINT IN POJO DATA FORMAT

The **camel-cxf** endpoint consumer POJO data format is based on the [CXF invoker](#), so the message header has a property with the name of **CxfConstants.OPERATION_NAME** and the message body is a list of the SEI method parameters.

Consider the [PersonProcessor](#) example code:

```

public class PersonProcessor implements Processor {

    private static final Logger LOG = LoggerFactory.getLogger(PersonProcessor.class);

    @Override
    @SuppressWarnings("unchecked")
    public void process(Exchange exchange) throws Exception {
        LOG.info("processing exchange in camel");

        BindingOperationInfo boi = (BindingOperationInfo)
exchange.getProperty(BindingOperationInfo.class.getName());
        if (boi != null) {
            LOG.info("boi.isUnwrapped" + boi.isUnwrapped());
        }
        // Get the parameters list which element is the holder.
        MessageContentsList msgList = (MessageContentsList) exchange.getIn().getBody();
        Holder<String> personId = (Holder<String>) msgList.get(0);
        Holder<String> ssn = (Holder<String>) msgList.get(1);
        Holder<String> name = (Holder<String>) msgList.get(2);
    }
}

```

```

        if (personId.value == null || personId.value.length() == 0) {
            LOG.info("person id 123, so throwing exception");
            // Try to throw out the soap fault message
            org.apache.camel.wsdl_first.types.UnknownPersonFault personFault
                = new org.apache.camel.wsdl_first.types.UnknownPersonFault();
            personFault.setPersonId("");
            org.apache.camel.wsdl_first.UnknownPersonFault fault
                = new org.apache.camel.wsdl_first.UnknownPersonFault("Get the null value of person
name", personFault);
            exchange.getMessage().setBody(fault);
            return;
        }

        name.value = "Bonjour";
        ssn.value = "123";
        LOG.info("setting Bonjour as the response");
        // Set the response message, first element is the return value of the operation,
        // the others are the holders of method parameters
        exchange.getMessage().setBody(new Object[] { null, personId, ssn, name });
    }
}

```

29.11. HOW TO PREPARE THE MESSAGE FOR THE CAMEL-CXF ENDPOINT IN POJO DATA FORMAT

The **camel-cxf** endpoint producer is based on the [CXF client API](#). First you need to specify the operation name in the message header, then add the method parameters to a list, and initialize the message with this parameter list. The response message's body is a `messageContentsList`, you can get the result from that list.

If you don't specify the operation name in the message header, **CxfProducer** will try to use the **defaultOperationName** from **CxfEndpoint**, if there is no **defaultOperationName** set on **CxfEndpoint**, it will pick up the first `operationName` from the `Operation` list.

If you want to get the object array from the message body, you can get the body using **message.getBody(Object[].class)**, as shown in [CxfProducerRouterTest.testInvokingSimpleServerWithParams](#):

```

Exchange senderExchange = new DefaultExchange(context, ExchangePattern.InOut);
final List<String> params = new ArrayList<>();
// Prepare the request message for the camel-cxf procedure
params.add(TEST_MESSAGE);
senderExchange.getIn().setBody(params);
senderExchange.getIn().setHeader(CxfConstants.OPERATION_NAME, ECHO_OPERATION);

Exchange exchange = template.send("direct:EndpointA", senderExchange);

org.apache.camel.Message out = exchange.getMessage();
// The response message's body is an MessageContentsList which first element is the return value of
the operation,
// If there are some holder parameters, the holder parameter will be filled in the rest of List.
// The result will be extract from the MessageContentsList with the String class type
MessageContentsList result = (MessageContentsList) out.getBody();
LOG.info("Received output text: " + result.get(0));

```

```

Map<String, Object> responseContext = CastUtils.cast((Map<?, ?>)
out.getHeader(Client.RESPONSE_CONTEXT));
assertNotNull(responseContext);
assertEquals("UTF-8", responseContext.get(org.apache.cxf.message.Message.ENCODING),
    "We should get the response context here");
assertEquals("echo " + TEST_MESSAGE, result.get(0), "Reply body on Camel is wrong");

```

29.12. HOW TO DEAL WITH THE MESSAGE FOR A CAMEL-CXF ENDPOINT IN PAYLOAD DATA FORMAT

PAYLOAD means that you process the payload from the SOAP envelope as a native `CxfPayload`.

Message.getBody() will return a **org.apache.camel.component.cxf.CxfPayload** object, with getters for SOAP message headers and the SOAP body.

See [CxfConsumerPayloadTest](#):

```

protected RouteBuilder createRouteBuilder() {
    return new RouteBuilder() {
        public void configure() {
            from(simpleEndpointURI + "&dataFormat=PAYLOAD").to("log:info").process(new Processor()
{
    @SuppressWarnings("unchecked")
    public void process(final Exchange exchange) throws Exception {
        CxfPayload<SoapHeader> requestPayload =
exchange.getIn().getBody(CxfPayload.class);
        List<Source> inElements = requestPayload.getBodySources();
        List<Source> outElements = new ArrayList<>();
        // You can use a customer toStringConverter to turn a CxfPayload message into String
as you want
        String request = exchange.getIn().getBody(String.class);
        XmlConverter converter = new XmlConverter();
        String documentString = ECHO_RESPONSE;

        Element in = new XmlConverter().toDOMElement(inElements.get(0));
        // Just check the element namespace
        if (!in.getNamespaceURI().equals(ELEMENT_NAMESPACE)) {
            throw new IllegalArgumentException("Wrong element namespace");
        }
        if (in.getLocalName().equals("echoBoolean")) {
            documentString = ECHO_BOOLEAN_RESPONSE;
            checkRequest("ECHO_BOOLEAN_REQUEST", request);
        } else {
            documentString = ECHO_RESPONSE;
            checkRequest("ECHO_REQUEST", request);
        }
        Document outDocument = converter.toDOMDocument(documentString, exchange);
        outElements.add(new DOMSource(outDocument.getDocumentElement()));
        // set the payload header with null
        CxfPayload<SoapHeader> responsePayload = new CxfPayload<>(null, outElements,
null);

        exchange.getMessage().setBody(responsePayload);
    }
});
}

```

```

    }
  };
}

```

29.13. HOW TO GET AND SET SOAP HEADERS IN POJO MODE

POJO means that the data format is a "list of Java objects" when the camel-cxf endpoint produces or consumes Camel exchanges. Even though Camel exposes the message body as POJOs in this mode, camel-cxf still provides access to read and write SOAP headers. However, since CXF interceptors remove in-band SOAP headers from the header list after they have been processed, only out-of-band SOAP headers are available to camel-cxf in POJO mode.

The following example illustrates how to get/set SOAP headers. Suppose we have a route that forwards from one Camel-cxf endpoint to another. That is, SOAP Client → Camel → CXF service. We can attach two processors to obtain/insert SOAP headers at (1) before a request goes out to the CXF service and (2) before the response comes back to the SOAP Client. Processor (1) and (2) in this example are `InsertRequestOutHeaderProcessor` and `InsertResponseOutHeaderProcessor`. Our route looks like this:

```

from("cxf:bean:routerRelayEndpointWithInsertion")
  .process(new InsertRequestOutHeaderProcessor())
  .to("cxf:bean:serviceRelayEndpointWithInsertion")
  .process(new InsertResponseOutHeaderProcessor());

```

The Bean **`routerRelayEndpointWithInsertion`** and **`serviceRelayEndpointWithInsertion`** are defined as follows:

```

@Bean
public CxfEndpoint routerRelayEndpointWithInsertion() {
    CxfSpringEndpoint cxfEndpoint = new CxfSpringEndpoint();
    cxfEndpoint.setServiceClass(org.apache.camel.component.cxf.soap.headers.HeaderTester.class);

    cxfEndpoint.setAddress("/CxfMessageHeadersRelayTest/HeaderService/routerRelayEndpointWithInsertion");
    cxfEndpoint.setWsdIURL("soap_header.wsdl");
    cxfEndpoint.setEndpointNameAsQName(
        QName.valueOf("
{http://apache.org/camel/component/cxf/soap/headers}SoapPortRelayWithInsertion"));
    cxfEndpoint.setServiceNameAsQName(SERVICENAME);
    cxfEndpoint.getFeatures().add(new LoggingFeature());
    return cxfEndpoint;
}

@Bean
public CxfEndpoint serviceRelayEndpointWithInsertion() {
    CxfSpringEndpoint cxfEndpoint = new CxfSpringEndpoint();
    cxfEndpoint.setServiceClass(org.apache.camel.component.cxf.soap.headers.HeaderTester.class);
    cxfEndpoint.setAddress("http://localhost:" + port +
"/services/CxfMessageHeadersRelayTest/HeaderService/routerRelayEndpointWithInsertionBackend");

    cxfEndpoint.setWsdIURL("soap_header.wsdl");
    cxfEndpoint.setEndpointNameAsQName(
        QName.valueOf("
{http://apache.org/camel/component/cxf/soap/headers}SoapPortRelayWithInsertion"));
    cxfEndpoint.setServiceNameAsQName(SERVICENAME);

```

```

    cxfEndpoint.getFeatures().add(new LoggingFeature());
    return cxfEndpoint;
}

```

SOAP headers are propagated to and from Camel Message headers. The Camel message header name is "org.apache.cxf.headers.Header.list" which is a constant defined in CXF (org.apache.cxf.headers.Header.HEADER_LIST). The header value is a List of CXF SoapHeader objects (org.apache.cxf.binding.soap.SoapHeader). The following snippet is the InsertResponseOutHeaderProcessor (that insert a new SOAP header in the response message). The way to access SOAP headers in both InsertResponseOutHeaderProcessor and InsertRequestOutHeaderProcessor are actually the same. The only difference between the two processors is setting the direction of the inserted SOAP header.

You can find the **InsertResponseOutHeaderProcessor** example in [CxfMessageHeadersRelayTest](#):

```

public static class InsertResponseOutHeaderProcessor implements Processor {

    public void process(Exchange exchange) throws Exception {
        List<SoapHeader> soapHeaders = CastUtils.cast((List<?
>)exchange.getIn().getHeader(Header.HEADER_LIST));

        // Insert a new header
        String xml = "<?xml version=\"1.0\" encoding=\"utf-8\"?><outofbandHeader "
            + "xmlns=\"http://cxf.apache.org/outofband/Header\" hdrAttribute=\"testHdrAttribute\" "
            + "xmlns:soap=\"http://schemas.xmlsoap.org/soap/envelope/\" soap:mustUnderstand=\"1\">"
            + "<name>New_testOobHeader</name><value>New_testOobHeaderValue</value>"
        </outofbandHeader>";
        SoapHeader newHeader = new SoapHeader(soapHeaders.get(0).getName(),
            DOMUtils.readXml(new StringReader(xml)).getDocumentElement());
        // make sure direction is OUT since it is a response message.
        newHeader.setDirection(Direction.DIRECTION_OUT);
        //newHeader.setMustUnderstand(false);
        soapHeaders.add(newHeader);
    }
}

```

29.14. HOW TO GET AND SET SOAP HEADERS IN PAYLOAD MODE

We've already shown how to access the SOAP message as CxfPayload object in PAYLOAD mode in the section [How to deal with the message for a camel-cxf endpoint in PAYLOAD data format](#) .

Once you obtain a CxfPayload object, you can invoke the CxfPayload.getHeaders() method that returns a List of DOM Elements (SOAP headers).

For an example see [CxfPayloadSoapHeaderTest](#):

```

from(getRouterEndpointURI()).process(new Processor() {
    @SuppressWarnings("unchecked")
    public void process(Exchange exchange) throws Exception {
        CxfPayload<SoapHeader> payload = exchange.getIn().getBody(CxfPayload.class);
        List<Source> elements = payload.getBodySources();
        assertNotNull(elements, "We should get the elements here");
        assertEquals(1, elements.size(), "Get the wrong elements size");
    }
}

```

```

Element el = new XmlConverter().toDOMElement(elements.get(0));
elements.set(0, new DOMSource(el));
assertEquals("http://camel.apache.org/pizza/types",
    el.getNamespaceURI(), "Get the wrong namespace URI");

List<SoapHeader> headers = payload.getHeaders();
assertNotNull(headers, "We should get the headers here");
assertEquals(1, headers.size(), "Get the wrong headers size");
assertEquals("http://camel.apache.org/pizza/types",
    ((Element) (headers.get(0).getObject())).getNamespaceURI(), "Get the wrong namespace
URI");
// alternatively you can also get the SOAP header via the camel header:
headers = exchange.getIn().getHeader(Header.HEADER_LIST, List.class);
assertNotNull(headers, "We should get the headers here");
assertEquals(1, headers.size(), "Get the wrong headers size");
assertEquals("http://camel.apache.org/pizza/types",
    ((Element) (headers.get(0).getObject())).getNamespaceURI(), "Get the wrong namespace
URI");
}

})
.to(getServiceEndpointURI());

```

You can also use the same way as described in sub-chapter "How to get and set SOAP headers in POJO mode" to set or get the SOAP headers. So, you can use the header "org.apache.cxf.headers.Header.list" to get and set a list of SOAP headers. This does also mean that if you have a route that forwards from one Camel-cxf endpoint to another (SOAP Client → Camel → CXF service), now also the SOAP headers sent by the SOAP client are forwarded to the CXF service. If you do not want that these headers are forwarded you have to remove them in the Camel header "org.apache.cxf.headers.Header.list".

29.15. SOAP HEADERS ARE NOT AVAILABLE IN RAW MODE

SOAP headers are not available in RAW mode as SOAP processing is skipped.

29.16. HOW TO THROW A SOAP FAULT FROM CAMEL

If you are using a **camel-cxf** endpoint to consume the SOAP request, you may need to throw the SOAP Fault from the camel context.

Basically, you can use the **throwFault** DSL to do that; it works for **POJO**, **PAYLOAD** and **MESSAGE** data format.

You can define the soap fault as shown in [CxfCustomizedExceptionTest](#):

```

SOAP_FAULT = new SoapFault(EXCEPTION_MESSAGE, SoapFault.FAULT_CODE_CLIENT);
Element detail = SOAP_FAULT.getOrCreateDetail();
Document doc = detail.getOwnerDocument();
Text tn = doc.createTextNode(DETAIL_TEXT);
detail.appendChild(tn);

```

Then throw it as you like

```

from(routerEndpointURI).setFaultBody(constant(SOAP_FAULT));

```

If your CXF endpoint is working in the **MESSAGE** data format, you could set the SOAP Fault message in the message body and set the response code in the message header as demonstrated by [CxfMessageStreamExceptionTest](#)

```
from(routerEndpointURI).process(new Processor() {

    public void process(Exchange exchange) throws Exception {
        Message out = exchange.getOut();
        // Set the message body with the
        out.setBody(this.getClass().getResourceAsStream("SoapFaultMessage.xml"));
        // Set the response code here
        out.setHeader(org.apache.cxf.message.Message.RESPONSE_CODE, new Integer(500));
    }

});
```

Same for using POJO data format. You can set the SOAPFault on the out body.

29.17. HOW TO PROPAGATE A CAMEL-CXF ENDPOINT'S REQUEST AND RESPONSE CONTEXT

[CXF client API](#) provides a way to invoke the operation with request and response context. If you are using a **camel-cxf** endpoint producer to invoke the outside web service, you can set the request context and get response context with the following code:

```
CxfExchange exchange = (CxfExchange)template.send(getJaxwsEndpointUri(), new
Processor() {
    public void process(final Exchange exchange) {
        final List<String> params = new ArrayList<String>();
        params.add(TEST_MESSAGE);
        // Set the request context to the inMessage
        Map<String, Object> requestContext = new HashMap<String, Object>();
        requestContext.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY,
JAXWS_SERVER_ADDRESS);
        exchange.getIn().setBody(params);
        exchange.getIn().setHeader(Client.REQUEST_CONTEXT, requestContext);
        exchange.getIn().setHeader(CxfConstants.OPERATION_NAME,
GREET_ME_OPERATION);
    }
});
org.apache.camel.Message out = exchange.getOut();
// The output is an object array, the first element of the array is the return value
Object[] output = out.getBody(Object[].class);
LOG.info("Received output text: " + output[0]);
// Get the response context form outMessage
Map<String, Object> responseContext =
CastUtils.cast((Map)out.getHeader(Client.RESPONSE_CONTEXT));
assertNotNull(responseContext);
assertEquals("Get the wrong wsdl operation name", "
{http://apache.org/hello_world_soap_http}greetMe",
    responseContext.get("javax.xml.ws.wsdl.operation").toString());
```

29.18. ATTACHMENT SUPPORT

POJO Mode: Both SOAP with Attachment and MTOM are supported (see example in Payload Mode for enabling MTOM). However, SOAP with Attachment is not tested. Since attachments are marshalled and unmarshalled into POJOs, users typically do not need to deal with the attachment themselves. Attachments are propagated to Camel message's attachments if the MTOM is not enabled. So, it is possible to retrieve attachments by Camel Message API

```
DataHandler Message.getAttachment(String id)
```

Payload Mode: MTOM is supported by the component. Attachments can be retrieved by Camel Message APIs mentioned above. SOAP with Attachment (SwA) is supported and attachments can be retrieved. SwA is the default (same as setting the CXF endpoint property "mtom-enabled" to false).

To enable MTOM, set the CXF endpoint property "mtom-enabled" to *true*.

```
@Bean
public CxfEndpoint routerEndpoint() {
    CxfSpringEndpoint cxfEndpoint = new CxfSpringEndpoint();
    cxfEndpoint.setServiceNameAsQName(SERVICE_QNAME);
    cxfEndpoint.setEndpointNameAsQName(PORT_QNAME);
    cxfEndpoint.setAddress("/") + getClass().getSimpleName() + "/jaxws-mtom/hello");
    cxfEndpoint.setWsdURL("mtom.wsdl");
    Map<String, Object> properties = new HashMap<String, Object>();
    properties.put("dataFormat", "PAYLOAD");
    properties.put("mtom-enabled", true);
    cxfEndpoint.setProperties(properties);
    return cxfEndpoint;
}
```

You can produce a Camel message with attachment to send to a CXF endpoint in Payload mode.

```
Exchange exchange = context.createProducerTemplate().send("direct:testEndpoint", new
Processor() {

    public void process(Exchange exchange) throws Exception {
        exchange.setPattern(ExchangePattern.InOut);
        List<Source> elements = new ArrayList<Source>();
        elements.add(new DOMSource(DOMUtils.readXml(new
StringReader(MtomTestHelper.REQ_MESSAGE)).getDocumentElement()));
        CxfPayload<SoapHeader> body = new CxfPayload<SoapHeader>(new ArrayList<SoapHeader>
()),
            elements, null);
        exchange.getIn().setBody(body);
        exchange.getIn().addAttachment(MtomTestHelper.REQ_PHOTO_CID,
            new DataHandler(new ByteArrayDataSource(MtomTestHelper.REQ_PHOTO_DATA,
"application/octet-stream")));

        exchange.getIn().addAttachment(MtomTestHelper.REQ_IMAGE_CID,
            new DataHandler(new ByteArrayDataSource(MtomTestHelper.requestJpeg, "image/jpeg")));
    }

});

// process response
```



```

CxfPayload<SoapHeader> out = exchange.getOut().getBody(CxfPayload.class);
Assert.assertEquals(1, out.getBody().size());

Map<String, String> ns = new HashMap<String, String>();
ns.put("ns", MtomTestHelper.SERVICE_TYPES_NS);
ns.put("xop", MtomTestHelper.XOP_NS);

XPathUtils xu = new XPathUtils(ns);
Element oute = new XmlConverter().toDOMElement(out.getBody().get(0));
Element ele = (Element)xu.getValue("//ns:DetailResponse/ns:photo/xop:Include", oute,
    XPathConstants.NODE);
String photold = ele.getAttribute("href").substring(4); // skip "cid:"

ele = (Element)xu.getValue("//ns:DetailResponse/ns:image/xop:Include", oute,
    XPathConstants.NODE);
String imageld = ele.getAttribute("href").substring(4); // skip "cid:"

DataHandler dr = exchange.getOut().getAttachment(photold);
Assert.assertEquals("application/octet-stream", dr.getContentType());
MtomTestHelper.assertEquals(MtomTestHelper.RESP_PHOTO_DATA,
    IOUtils.readBytesFromStream(dr.getInputStream()));

dr = exchange.getOut().getAttachment(imageld);
Assert.assertEquals("image/jpeg", dr.getContentType());

BufferedImage image = ImageIO.read(dr.getInputStream());
Assert.assertEquals(560, image.getWidth());
Assert.assertEquals(300, image.getHeight());

```

You can also consume a Camel message received from a CXF endpoint in Payload mode. The [CxfMtomConsumerPayloadModeTest](#) illustrates how this works:

```

public static class MyProcessor implements Processor {

    @SuppressWarnings("unchecked")
    public void process(Exchange exchange) throws Exception {
        CxfPayload<SoapHeader> in = exchange.getIn().getBody(CxfPayload.class);

        // verify request
        Assert.assertEquals(1, in.getBody().size());

        Map<String, String> ns = new HashMap<String, String>();
        ns.put("ns", MtomTestHelper.SERVICE_TYPES_NS);
        ns.put("xop", MtomTestHelper.XOP_NS);

        XPathUtils xu = new XPathUtils(ns);
        Element body = new XmlConverter().toDOMElement(in.getBody().get(0));
        Element ele = (Element)xu.getValue("//ns:Detail/ns:photo/xop:Include", body,
            XPathConstants.NODE);
        String photold = ele.getAttribute("href").substring(4); // skip "cid:"
        Assert.assertEquals(MtomTestHelper.REQ_PHOTO_CID, photold);

        ele = (Element)xu.getValue("//ns:Detail/ns:image/xop:Include", body,
            XPathConstants.NODE);
        String imageld = ele.getAttribute("href").substring(4); // skip "cid:"
        Assert.assertEquals(MtomTestHelper.REQ_IMAGE_CID, imageld);
    }
}

```

```

        DataHandler dr = exchange.getIn().getAttachment(photoid);
        Assert.assertEquals("application/octet-stream", dr.getContentType());
        MtomTestHelper.assertEquals(MtomTestHelper.REQ_PHOTO_DATA,
        IOUtils.readBytesFromStream(dr.getInputStream()));

        dr = exchange.getIn().getAttachment(imageld);
        Assert.assertEquals("image/jpeg", dr.getContentType());
        MtomTestHelper.assertEquals(MtomTestHelper.requestJpeg,
        IOUtils.readBytesFromStream(dr.getInputStream()));

        // create response
        List<Source> elements = new ArrayList<Source>();
        elements.add(new DOMSource(DOMUtils.readXml(new
        StringReader(MtomTestHelper.RESP_MESSAGE)).getDocumentElement()));
        CxfPayload<SoapHeader> sbody = new CxfPayload<SoapHeader>(new
        ArrayList<SoapHeader>(),
            elements, null);
        exchange.getOut().setBody(sbody);
        exchange.getOut().addAttachment(MtomTestHelper.RESP_PHOTO_CID,
            new DataHandler(new ByteArrayDataSource(MtomTestHelper.RESP_PHOTO_DATA,
            "application/octet-stream")));

        exchange.getOut().addAttachment(MtomTestHelper.RESP_IMAGE_CID,
            new DataHandler(new ByteArrayDataSource(MtomTestHelper.responseJpeg, "image/jpeg")));
    }
}

```

Raw Mode: Attachments are not supported as it does not process the message at all.

CXF_RAW Mode: MTOM is supported, and Attachments can be retrieved by Camel Message APIs mentioned above. Note that when receiving a multipart (i.e. MTOM) message the default SOAPMessage to String converter will provide the complete multipart payload on the body. If you require just the SOAP XML as a String, you can set the message body with `message.getSOAPPart()`, and Camel convert can do the rest of work for you.

29.19. STREAMING SUPPORT IN PAYLOAD MODE

The camel-cxf component now supports streaming of incoming messages when using PAYLOAD mode. Previously, the incoming messages would have been completely DOM parsed. For large messages, this is time consuming and uses a significant amount of memory. The incoming messages can remain as a `javax.xml.transform.Source` while being routed and, if nothing modifies the payload, can then be directly streamed out to the target destination. For common "simple proxy" use cases (example: `from("cxf:...").to("cxf:...")`), this can provide very significant performance increases as well as significantly lowered memory requirements.

However, there are cases where streaming may not be appropriate or desired. Due to the streaming nature, invalid incoming XML may not be caught until later in the processing chain. Also, certain actions may require the message to be DOM parsed anyway (like WS-Security or message tracing and such) in which case the advantages of the streaming is limited. At this point, there are two ways to control the streaming:

- Endpoint property: you can add `"allowStreaming=false"` as an endpoint property to turn the streaming on/off.

- Component property: the CxfComponent object also has an allowStreaming property that can set the default for endpoints created from that component.

Global system property: you can add a system property of "org.apache.camel.component.cxf.streaming" to "false" to turn it off. That sets the global default, but setting the endpoint property above will override this value for that endpoint.

29.20. USING THE GENERIC CXF DISPATCH MODE

The camel-cxf component supports the generic [CXF dispatch mode](#) that can transport messages of arbitrary structures (i.e., not bound to a specific XML schema). To use this mode, you simply omit specifying the wsdlURL and serviceClass attributes of the CXF endpoint.

```
<cxf:cxfEndpoint id="testEndpoint" address="http://localhost:9000/SoapContext/SoapAnyPort">
  <cxf:properties>
    <entry key="dataFormat" value="PAYLOAD"/>
  </cxf:properties>
</cxf:cxfEndpoint>
```

It is noted that the default CXF dispatch client does not send a specific SOAPAction header. Therefore, when the target service requires a specific SOAPAction value, it is supplied in the Camel header using the key SOAPAction (case-insensitive).

29.21. SPRING BOOT AUTO-CONFIGURATION

The component supports 13 options, which are listed below.

Name	Description	Default	Type
camel.component.cxf.allow-streaming	This option controls whether the CXF component, when running in PAYLOAD mode, will DOM parse the incoming messages into DOM Elements or keep the payload as a javax.xml.transform.Source object that would allow streaming in some cases.		Boolean
camel.component.cxf.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.cxf.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.cxf.enabled	Whether to enable auto configuration of the cxf component. This is enabled by default.		Boolean
camel.component.cxf.header-filter-strategy	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		HeaderFilterStrategy
camel.component.cxf.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.cxf.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean
camel.component.cxf.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.cxfrcs.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.cxfrcs.enabled	Whether to enable auto configuration of the cxfrcs component. This is enabled by default.		Boolean
camel.component.cxfrcs.header-filter-strategy	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		HeaderFilterStrategy
camel.component.cxfrcs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.cxfrcs.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean

CHAPTER 30. CXF-RS

Both producer and consumer are supported

The CXFRS component provides integration with [Apache CXF](#) for connecting to JAX-RS 1.1 and 2.0 services hosted in CXF.

30.1. DEPENDENCIES

When using **camel-cxf-rest** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-cxf-rest-starter</artifactId>
</dependency>
```

30.2. URI FORMAT

```
cxfrs://address?options
```

Where the **address** represents the address of the CXF endpoint.

```
cxfrs:bean:rsEndpoint
```

Where the **rsEndpoint** represents the name of the spring bean, which presents the CXFRS client or server.

For the formats above, you can append options to the URI as follows:

```
cxfrs:bean:cxfrEndpoint?resourceClasses=org.apache.camel.rs.Example
```

30.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

30.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).

- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

30.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

30.4. COMPONENT OPTIONS

The CXF-RS component supports 5 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
headerFilterStrategy (filter)	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean

30.5. ENDPOINT OPTIONS

The CXF-RS endpoint is configured using URI syntax:

```
cxfrs:beanId:address
```

With the following *path* and *query* parameters:

30.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
beanId (common)	To lookup an existing configured CxfRsEndpoint. Must use bean: as prefix.		String
address (common)	The service publish address.		String

30.5.2. Query Parameters (31 parameters)

Name	Description	Default	Type
features (common)	Set the feature list to the CxfRs endpoint.		List
loggingFeatureEnabled (common)	This option enables CXF Logging Feature which writes inbound and outbound REST messages to log.	false	boolean
loggingSizeLimit (common)	To limit the total size of number of bytes the logger will output when logging feature has been enabled.		int
modelRef (common)	This option is used to specify the model file which is useful for the resource class without annotation. When using this option, then the service class can be omitted, to emulate document-only endpoints.		String
providers (common)	Set custom JAX-RS provider(s) list to the CxfRs endpoint. You can specify a string with a list of providers to lookup in the registry separated by comma.		String
resourceClasses (common)	The resource classes which you want to export as REST service. Multiple classes can be separated by comma.		List
schemaLocations (common)	Sets the locations of the schema(s) which can be used to validate the incoming XML or JAXB-driven JSON.		List
skipFaultLogging (common)	This option controls whether the PhaseInterceptorChain skips logging the Fault that it catches.	false	boolean

Name	Description	Default	Type
bindingStyle (consumer)	Sets how requests and responses will be mapped to/from Camel. Two values are possible: SimpleConsumer: This binding style processes request parameters, multipart, etc. and maps them to IN headers, IN attachments and to the message body. It aims to eliminate low-level processing of <code>org.apache.cxf.message.MessageContentsList</code>. It also adds more flexibility and simplicity to the response mapping. Only available for consumers. Default: The default style. For consumers this passes on a <code>MessageContentsList</code> to the route, requiring low-level processing in the route. This is the traditional binding style, which simply dumps the <code>org.apache.cxf.message.MessageContentsList</code> coming in from the CXF stack onto the IN message body. The user is then responsible for processing it according to the contract defined by the JAX-RS method signature. Custom: allows you to specify a custom binding through the binding option. Enum values: ● SimpleConsumer ● Default ● Custom	Default	BindingStyle
publishedEndpointUrl (consumer)	This option can override the <code>endpointUrl</code> that published from the WADL which can be accessed with resource address url plus <code>_wadl</code> .		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which means any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore <code>bridgeErrorHandler</code> is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
serviceBeans (consumer (advanced))	The service beans (the bean ids to lookup in the registry) which you want to export as REST service. Multiple beans can be separated by comma.		<code>String</code>
cookieHandler (producer)	Configure a cookie handler to maintain a HTTP session.		<code>CookieHandler</code>
hostnameVerifier (producer)	The hostname verifier to be used. Use the <code>#</code> notation to reference a <code>HostnameVerifier</code> from the registry.		<code>HostnameVerifier</code>
sslContextParameters (producer)	The Camel SSL setting reference. Use the <code>#</code> notation to reference the SSL Context.		<code>SSLContextParameters</code>
throwExceptionOnFailure (producer)	This option tells the <code>CxfRsProducer</code> to inspect return codes and will generate an <code>Exception</code> if the return code is larger than 207.	<code>true</code>	<code>boolean</code>
httpClientAPI (producer (advanced))	If it is true, the <code>CxfRsProducer</code> will use the <code>HttpClientAPI</code> to invoke the service. If it is false, the <code>CxfRsProducer</code> will use the <code>ProxyClientAPI</code> to invoke the service.	<code>true</code>	<code>boolean</code>
ignoreDeleteMethodMessageBody (producer (advanced))	This option is used to tell <code>CxfRsProducer</code> to ignore the message body of the DELETE method when using HTTP API.	<code>false</code>	<code>boolean</code>

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
maxClientCacheSize (producer (advanced))	This option allows you to configure the maximum size of the cache. The implementation caches CXF clients or ClientFactoryBean in CxfProvider and CxfRsProvider.	10	int
synchronous (producer (advanced))	Sets whether synchronous processing should be strictly used.	false	boolean
binding (advanced)	To use a custom CxfBinding to control the binding between Camel Message and CXF Message.		CxfRsBinding
bus (advanced)	To use a custom configured CXF Bus.		Bus
continuationTimeout (advanced)	This option is used to set the CXF continuation timeout which could be used in CxfConsumer by default when the CXF server is using Jetty or Servlet transport.	30000	long
cxfRsConfigurer (advanced)	This option could apply the implementation of org.apache.camel.component.cxf.jaxrs.CxfRsEndpointConfigurer which supports to configure the CXF endpoint in programmatic way. User can configure the CXF server and client by implementing configure\\{Server/Client} method of CxfEndpointConfigurer.		CxfRsConfigurer
defaultBus (advanced)	Will set the default bus when CXF endpoint create a bus by itself.	false	boolean
headerFilterStrategy (advanced)	To use a custom HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy

Name	Description	Default	Type
performInvocation (advanced)	When the option is true, Camel will perform the invocation of the resource class instance and put the response object into the exchange for further processing.	false	boolean
propagateContexts (advanced)	When the option is true, JAXRS UriInfo, HttpHeaders, Request and SecurityContext contexts will be available to custom CXFRS processors as typed Camel exchange properties. These contexts can be used to analyze the current requests using JAX-RS API.	false	boolean

30.6. MESSAGE HEADERS

The CXF-RS component supports 16 message header(s), which is/are listed below:

Name	Description	Default	Type
operationName (common) Constant: OPERATION_NAME	The name of the operation.		String
CamelAuthentication (common) Constant: AUTHENTICATION	The authentication.		Subject
CamelHttpMethod (common) Constant: HTTP_METHOD	The http method to use.		String
CamelHttpPath (common) Constant: HTTP_PATH	The http path.		String

Name	Description	Default	Type
Content-Type (common) Constant: CONTENT_TYPE	The content type.		String
CamelHttpQuery (common) Constant: HTTP_QUERY	The http query.		String
CamelHttpResponseCode (common) Constant: HTTP_RESPONSE_CODE	The http response code.		Integer
Content-Encoding (common) Constant: CONTENT_ENCODING	The content encoding.		String
org.apache.cxf.message.Message. PROTOCOL_HEADERS (common) Constant: PROTOCOL_HEADERS	The protocol headers.		Map
CamelCxfMessage (common) Constant: CAMEL_CXF_MESSAGE	The CXF message.		Message

Name	Description	Default	Type
CamelCxfRsUsingHttpAPI (common) Constant: CAMEL_CXF_RS_USING_HTTP_API	If it is true, the CxfRsProducer will use the HttpClientAPI to invoke the service. If it is false, the CxfRsProducer will use the ProxyClientAPI to invoke the service.		Boolean
CamelCxfRsVarValues (common) Constant: CAMEL_CXF_RS_VAR_VALUES	The path values.		Object[]
CamelCxfRsResponseClass (common) Constant: CAMEL_CXF_RS_RESPONSE_CLASSES	The response class.		Class
CamelCxfRsResponseGenericType (common) Constant: CAMEL_CXF_RS_RESPONSE_GENERIC_TYPE	The response generic type.		Type
CamelCxfRsQueryMap (common) Constant: CAMEL_CXF_RS_QUERY_MAP	The query map.		Map
CamelCxfRsOperationResourceInfoStack (common) Constant: CAMEL_CXF_RS_OPERATION_RESOURCE_INFO_STACK	The stack of MethodInvocationInfo representing resources path when JAX-RS invocation looks for target.		OperationResourceInfoStack

You can also configure the CXF REST endpoint through the spring configuration.



NOTE

Avoid creating your own HTTP server instances in a Camel Spring Boot application by using either of the **`cxf-rt-transport-jetty`**, **`cxf-rt-transport-netty-server`**, **`cxf-rt-transport-undertow`** libraries. It is recommended to use the Spring Boot embedded HTTP server stack which is created when you use the **`spring-boot-starter-web`**, **`cxf-spring-boot-starter-jaxrs`**, or **`cxf-spring-boot-starter-jaxws`** dependencies.



NOTE

Since there are lots of differences between the CXF REST client and CXF REST Server, we provide different configuration for them.

+ Please check the following files for more details:

- the [schema file](#).
- [CXF JAX-RS documentation](#).

30.7. HOW TO CONFIGURE THE REST ENDPOINT IN CAMEL

In the [camel-cxf schema file](#), there are two elements for the REST endpoint definition:

- **`cxf:rsServer`** for REST consumer
- **`cxf:rsClient`** for REST producer.

You can find a Camel REST service route configuration example there.

30.8. HOW TO OVERRIDE THE CXF PRODUCER ADDRESS FROM MESSAGE HEADER

The **`camel-cxf:rs`** producer supports overriding the service address by setting the message with the key of **`CamelDestinationOverrideUrl`**.

```
// set up the service address from the message header to override the setting of CXF endpoint
exchange.getIn().setHeader(Exchange.DESTINATION_OVERRIDE_URL,
constant(getServiceAddress()));
```

30.9. CONSUMING A REST REQUEST - SIMPLE BINDING STYLE

Since Camel 2.11

The **Default** binding style is rather low-level, requiring the user to manually process the **`MessageContentsList`** object coming into the route. Thus, it tightly couples the route logic with the method signature and parameter indices of the JAX-RS operation which is somewhat inelegant, difficult and error-prone.

In contrast, the **`SimpleConsumer`** binding style performs the following mappings, to **make the request data more accessible** to you within the Camel Message:

- JAX-RS Parameters (**@HeaderParam**, **@QueryParam**, etc.) are injected as *IN* message headers. The header name matches the value of the annotation.
- The request entity (POJO or another type) becomes the *IN* message body. If a single entity cannot be identified in the JAX-RS method signature, it falls back to the original **MessageContentsList**.
- Binary **@Multipart** body parts become *IN* message attachments, supporting **DataHandler**, **InputStream**, **DataSource** and CXF's **Attachment** class.
- Non-binary **@Multipart** body parts are mapped as *IN* message headers. The header name matches the Body Part name.

Additionally, the following rules apply to the **Response** mapping:

- If the message body type is different to **javax.ws.rs.core.Response** (user-built response), a new **Response** is created and the message body is set as the entity (so long it's not null). The response status code is taken from the **Exchange.HTTP_RESPONSE_CODE** header, or defaults to 200 OK if not present.
- If the message body type is equal to **javax.ws.rs.core.Response**, it means that the user has built a custom response, and therefore it is respected, and it becomes the final response.
- In all cases, Camel headers permitted by custom or default **HeaderFilterStrategy** are added to the HTTP response.

30.9.1. Enabling the Simple Binding Style

This binding style can be activated by setting the **bindingStyle** parameter in the consumer endpoint to value **SimpleConsumer**:

```
from("cxfrs:bean:rsServer?bindingStyle=SimpleConsumer")
  .to("log:TEST?showAll=true");
```

30.9.2. Examples of request binding with different method signatures

Below is a list of method signatures along with the expected result from the simple binding:

- **public Response doAction(BusinessObject request);** the request payload is placed in the *IN* message body, replacing the original **MessageContentsList**.
- **public Response doAction(BusinessObject request, @HeaderParam("abcd") String abcd, @QueryParam("defg") String defg);** the request payload is placed in the *IN* message body, replacing the original **MessageContentsList**. Both request parameters are mapped as *IN* message headers with names "abcd" and "defg".
- **public Response doAction(@HeaderParam("abcd") String abcd, @QueryParam("defg") String defg);** both request parameters are mapped as the *IN* message headers with names "abcd" and "defg". The original **MessageContentsList** is preserved, even though it only contains the two parameters.
- **public Response doAction(@Multipart(value="body1") BusinessObject request, @Multipart(value="body2") BusinessObject request2);** the first parameter is transferred as a header with name "body1", and the second one is mapped as header "body2". The original

MessageContentsList is preserved as the *IN* message body.

- **public Response doAction(InputStream abcd);** the **InputStream** is unwrapped from the **MessageContentsList** and preserved as the *IN* message body.
- **public Response doAction(DataHandler abcd);** the *DataHandler* is unwrapped from the **MessageContentsList** and preserved as the *IN* message body.

30.9.3. Examples of the Simple Binding Style

Given a JAX-RS resource class with this method:

```
@POST @Path("/customers/{type}")
public Response newCustomer(Customer customer, @PathParam("type") String type,
    @QueryParam("active") @DefaultValue("true") boolean active) {
    return null;
}
```

Serviced by the following route:

```
from("cxfrs:bean:rsServer?bindingStyle=SimpleConsumer")
    .recipientList(simple("direct:${header.operationName}"));

from("direct:newCustomer")
    .log("Request: type=${header.type}, active=${header.active}, customerData=${body}");
```

The following HTTP request with XML payload (given that the Customer DTO is JAXB-annotated):

```
POST /customers/gold?active=true

Payload:
<Customer>
  <fullName>Raul Kripalani</fullName>
  <country>Spain</country>
  <project>Apache Camel</project>
</Customer>
```

Will print the message:

```
Request: type=gold, active=true, customerData=<Customer.toString() representation>
```

NOTE

More examples on how to process requests and write responses can be found [here](#).

30.10. CONSUMING A REST REQUEST - DEFAULT BINDING STYLE

The [CXF JAXRS front end](#) implements the [JAX-RS \(JSR-311\) API](#), so we can export the resource classes as a REST service. We leverage the [CXF Invoker API](#) to turn a REST request into a normal Java object method invocation. There is no need to specify the URI template within your endpoint. The CXF takes care of the REST request URI to resource class method mapping according to the JSR-311 specification. All you need to do in Camel is delegate this method request to the right processor or endpoint.

CXF-RS route example

```
private static final String CXF_RS_ENDPOINT_URI =
    "cxfrs://http://localhost:" + CXT + "/rest?";
resourceClasses=org.apache.camel.component.cxf.jaxrs.testbean.CustomerServiceResource";
private static final String CXF_RS_ENDPOINT_URI2 =
    "cxfrs://http://localhost:" + CXT + "/rest2?";
resourceClasses=org.apache.camel.component.cxf.jaxrs.testbean.CustomerService";
private static final String CXF_RS_ENDPOINT_URI3 =
    "cxfrs://http://localhost:" + CXT + "/rest3?";
+
"resourceClasses=org.apache.camel.component.cxf.jaxrs.testbean.CustomerServiceNoAnnotations&"

    + "modelRef=classpath:/org/apache/camel/component/cxf/jaxrs/CustomerServiceModel.xml";
private static final String CXF_RS_ENDPOINT_URI4 =
    "cxfrs://http://localhost:" + CXT + "/rest4?";
+
"modelRef=classpath:/org/apache/camel/component/cxf/jaxrs/CustomerServiceDefaultHandlerModel.xml";
private static final String CXF_RS_ENDPOINT_URI5 =
    "cxfrs://http://localhost:" + CXT + "/rest5?";
    + "propagateContexts=true&"
    +
"modelRef=classpath:/org/apache/camel/component/cxf/jaxrs/CustomerServiceDefaultHandlerModel.xml";
protected RouteBuilder createRouteBuilder() throws Exception {
    final Processor testProcessor = new TestProcessor();
    final Processor testProcessor2 = new TestProcessor2();
    final Processor testProcessor3 = new TestProcessor3();
    return new RouteBuilder() {
        public void configure() {
            errorHandler(new NoErrorHandlerBuilder());
            from(CXF_RS_ENDPOINT_URI).process(testProcessor);
            from(CXF_RS_ENDPOINT_URI2).process(testProcessor);
            from(CXF_RS_ENDPOINT_URI3).process(testProcessor);
            from(CXF_RS_ENDPOINT_URI4).process(testProcessor2);
            from(CXF_RS_ENDPOINT_URI5).process(testProcessor3);
        }
    };
}
```

And the corresponding resource class is used to configure the endpoint.

NOTE

By default, JAX-RS resource classes are **only** used to configure JAX-RS properties. Methods will **not** be executed during routing of messages to the endpoint. Instead, it is the responsibility of the route to do all processing.

It is sufficient to provide an interface only as opposed to a no-op service implementation class for the default mode.

If a **performInvocation** option is enabled, the service implementation will be invoked first, the response will be set on the Camel exchange, and the route execution will continue as usual. This can be useful for integrating the existing JAX-RS implementations into Camel routes and for post-processing JAX-RS Responses in custom processors.

```

@Path("/customerservice/")
public interface CustomerServiceResource {

    @GET
    @Path("/customers/{id}/")
    Customer getCustomer(@PathParam("id") String id);

    @PUT
    @Path("/customers/")
    Response updateCustomer(Customer customer);

    @Path("/{id}")
    @PUT()
    @Consumes({ "application/xml", "text/plain",
                "application/json" })
    @Produces({ "application/xml", "text/plain",
                "application/json" })
    Object invoke(@PathParam("id") String id,
                  String payload);
}

```

30.11. HOW TO INVOKE THE REST SERVICE THROUGH CAMEL-CXFRS PRODUCER

The [CXF JAXRS front end](#) implements a [proxy-based client API](#). With this API you can invoke the remote REST service through a proxy. The **camel-cxfrs** producer is based on this [proxy API](#). You can specify the operation name in the message header and prepare the parameter in the message body, the camel-cxfrs producer will generate the right REST request for you.

Example

```

Exchange exchange = template.send("direct://proxy", new Processor() {
    public void process(Exchange exchange) throws Exception {
        exchange.setPattern(ExchangePattern.InOut);
        Message inMessage = exchange.getIn();
        // set the operation name
        inMessage.setHeader(CxfConstants.OPERATION_NAME, "getCustomer");
        // using the proxy client API
        inMessage.setHeader(CxfConstants.CAMEL_CXF_RS_USING_HTTP_API, Boolean.FALSE);
        // set a customer header
        inMessage.setHeader("key", "value");
        // set up the accepted content type
        inMessage.setHeader(Exchange.ACCEPT_CONTENT_TYPE, "application/json");
        // set the parameters, if you just have one parameter,
        // camel will put this object into an Object[] itself
        inMessage.setBody("123");
    }
});

// get the response message
Customer response = (Customer) exchange.getMessage().getBody();

assertNotNull(response, "The response should not be null");
assertEquals(123, response.getId(), "Get a wrong customer id");

```

```

assertEquals("John", response.getName(), "Get a wrong customer name");
assertEquals(200, exchange.getMessage().getHeader(Exchange.HTTP_RESPONSE_CODE), "Get
a wrong response code");
assertEquals("value", exchange.getMessage().getHeader("key"), "Get a wrong header value");

```

The [CXF JAXRS front end](#) also provides an [HTTP centric client API](#). You can also invoke this API from camel-cxf-rs producer. You need to specify the [HTTP_PATH](#) and the [HTTP_METHOD](#) and let the producer use the http centric client API by using the URI option `httpClientAPI` or by setting the message header [CxfConstants.CAMEL_CXF_RS_USING_HTTP_API](#). You can turn the response object to the type class specified with the message header [CxfConstants.CAMEL_CXF_RS_RESPONSE_CLASS](#).

```

Exchange exchange = template.send("direct://http", new Processor() {
    public void process(Exchange exchange) throws Exception {
        exchange.setPattern(ExchangePattern.InOut)
        Message inMessage = exchange.getIn();
        // using the http central client API
        inMessage.setHeader(CxfConstants.CAMEL_CXF_RS_USING_HTTP_API, Boolean.TRUE);
        // set the Http method
        inMessage.setHeader(Exchange.HTTP_METHOD, "GET");
        // set the relative path
        inMessage.setHeader(Exchange.HTTP_PATH, "/customerservice/customers/123");
        // Specify the response class, cxfrs will use InputStream as the response object type
        inMessage.setHeader(CxfConstants.CAMEL_CXF_RS_RESPONSE_CLASS, Customer.class);
        // set a customer header
        inMessage.setHeader("key", "value");
        // since we use the Get method, so we don't need to set the message body
        inMessage.setBody(null);
    }
});

```

You can also specify the query parameters from cxfrs URI for the CXFRS http centric client.

```

Exchange exchange = template.send("cxfrs://http://localhost:9003/testQuery?
httpClientAPI=true&q1=12&q2=13"

```

To support the Dynamical routing, you can override the URI's query parameters by using the [CxfConstants.CAMEL_CXF_RS_QUERY_MAP](#) header to set the parameter map for it.

```

Map<String, String> queryMap = new LinkedHashMap<>();
queryMap.put("q1", "new");
queryMap.put("q2", "world");
inMessage.setHeader(CxfConstants.CAMEL_CXF_RS_QUERY_MAP, queryMap);

```

30.12. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
camel.component. cxfrs.autowired- enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component. cxfrs.bridge- error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component. cxfrs.enabled	Whether to enable auto configuration of the cxfrs component. This is enabled by default.		Boolean
camel.component. cxfrs.header- filter-strategy	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message. The option is a org.apache.camel.spi.HeaderFilterStrategy type.		HeaderFilterStrategy
camel.component. cxfrs.lazy-start- producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component. cxfrs.use-global- ssl-context- parameters	Enable usage of global SSL context parameters.	false	Boolean

CHAPTER 31. DATA FORMAT

Only producer is supported

The Dataformat component allows to use the [Data Format](#) as a Camel Component.

31.1. DEPENDENCIES

When using **dataformat** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-dataformat-starter</artifactId>
</dependency>
```

31.2. URI FORMAT

```
dataformat:name:(marshal|unmarshal)[?options]
```

Where **name** is the name of the Data Format. And then followed by the operation which must either be **marshal** or **unmarshal**. The options is used for configuring the [Data Format](#) in use. See the Data Format documentation for which options it support.

31.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

31.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

31.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

31.4. COMPONENT OPTIONS

The Data Format component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

31.5. ENDPOINT OPTIONS

The Data Format endpoint is configured using URI syntax:

```
dataformat:name:operation
```


with the following path and query parameters:

31.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
name (producer)	Required Name of data format.		String
operation (producer)	Required Operation to use either marshal or unmarshal. Enum values: • marshal • unmarshal		String

31.5.2. Query Parameters (1 parameters)

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

31.6. SAMPLES

For example to use the [JAXB Data Format](#) we can do as follows:

```
from("activemq:My.Queue").
  to("dataformat:jaxb:unmarshal?contextPath=com.acme.model").
  to("mqseries:Another.Queue");
```

And in XML DSL you do:

```
<camelContext id="camel" xmlns="http://camel.apache.org/schema/spring">
  <route>
    <from uri="activemq:My.Queue"/>
    <to uri="dataformat:jaxb:unmarshal?contextPath=com.acme.model"/>
  </route>
</camelContext>
```

```

    <to uri="mqseries:Another.Queue"/>
  </route>
</camelContext>

```

31.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.component.dataformat.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.dataformat.enabled	Whether to enable auto configuration of the dataformat component. This is enabled by default.		Boolean
camel.component.dataformat.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 32. DATASET

Both producer and consumer are supported

Testing of distributed and asynchronous processing is notoriously difficult. The [Mock](#), [Test](#) and [DataSet](#) endpoints work great with the Camel Testing Framework to simplify your unit and integration testing using [Enterprise Integration Patterns](#) and Camel's large range of Components together with the powerful Bean Integration.

The DataSet component provides a mechanism to easily perform load & soak testing of your system. It works by allowing you to create [DataSet instances](#) both as a source of messages and as a way to assert that the data set is received.

Camel will use the [throughput logger](#) when sending datasets.

32.1. DEPENDENCIES

When using **dataset** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-dataset-starter</artifactId>
</dependency>
```

32.2. URI FORMAT

```
dataset:name[?options]
```

Where **name** is used to find the [DataSet instance](#) in the Registry

Camel ships with a support implementation of **org.apache.camel.component.dataset.DataSet**, the **org.apache.camel.component.dataset.DataSetSupport** class, that can be used as a base for implementing your own DataSet. Camel also ships with some implementations that can be used for testing: **org.apache.camel.component.dataset.SimpleDataSet**, **org.apache.camel.component.dataset.ListDataSet** and **org.apache.camel.component.dataset.FileDataSet**, all of which extend **DataSetSupport**.

32.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

32.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a

few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

32.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

32.4. COMPONENT OPTIONS

The Dataset component supports 5 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
log (producer)	To turn on logging when the mock receives an incoming message. This will log only one time at INFO level for the incoming message. For more detailed logging then set the logger to DEBUG level for the org.apache.camel.component.mock.MockEndpoint class.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
exchangeFormatter (advanced)	Autowired Sets a custom ExchangeFormatter to convert the Exchange to a String suitable for logging. If not specified, we default to DefaultExchangeFormatter.		ExchangeFormatter

32.5. ENDPOINT OPTIONS

The Dataset endpoint is configured using URI syntax:

```
dataset:name
```

with the following path and query parameters:

32.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (common)	Required Name of DataSet to lookup in the registry.		DataSet

32.5.2. Query Parameters (21 parameters)

Name	Description	Default	Type
dataSetIndex (common)	Controls the behaviour of the CamelDataSetIndex header. For Consumers: - off = the header will not be set - strict/lenient = the header will be set For Producers: - off = the header value will not be verified, and will not be set if it is not present = strict = the header value must be present and will be verified = lenient = the header value will be verified if it is present, and will be set if it is not present. Enum values: • strict • lenient • off	lenient	String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
initialDelay (consumer)	Time period in millis to wait before starting sending messages.	1000	long
minRate (consumer)	Wait until the DataSet contains at least this number of messages.	0	int
preloadSize (consumer)	Sets how many messages should be preloaded (sent) before the route completes its initialization.	0	long
produceDelay (consumer)	Allows a delay to be specified which causes a delay when a message is sent by the consumer (to simulate slow processing).	3	long
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
assertPeriod (producer)	Sets a grace period after which the mock endpoint will re-assert to ensure the preliminary assertion is still valid. This is used for example to assert that exactly a number of messages arrives. For example if <code>expectedMessageCount(int)</code> was set to 5, then the assertion is satisfied when 5 or more message arrives. To ensure that exactly 5 messages arrives, then you would need to wait a little period to ensure no further message arrives. This is what you can use this method for. By default this period is disabled.		long
consumeDelay (producer)	Allows a delay to be specified which causes a delay when a message is consumed by the producer (to simulate slow processing).	0	long
expectedCount (producer)	Specifies the expected number of message exchanges that should be received by this endpoint. Beware: If you want to expect that 0 messages, then take extra care, as 0 matches when the tests starts, so you need to set a assert period time to let the test run for a while to make sure there are still no messages arrived; for that use <code>setAssertPeriod(long)</code>. An alternative is to use <code>NotifyBuilder</code>, and use the notifier to know when Camel is done routing some messages, before you call the <code>assertIsSatisfied()</code> method on the mocks. This allows you to not use a fixed assert period, to speedup testing times. If you want to assert that exactly n'th message arrives to this mock endpoint, then see also the <code>setAssertPeriod(long)</code> method for further details.	-1	int
failFast (producer)	Sets whether <code>assertIsSatisfied()</code> should fail fast at the first detected failed expectation while it may otherwise wait for all expected messages to arrive before performing expectations verifications. Is by default true. Set to false to use behavior as in Camel 2.x.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
log (producer)	To turn on logging when the mock receives an incoming message. This will log only one time at INFO level for the incoming message. For more detailed logging then set the logger to DEBUG level for the org.apache.camel.component.mock.MockEndpoint class.	false	boolean
reportGroup (producer)	A number that is used to turn on throughput logging based on groups of the size.		int
resultMinimumWaitTime (producer)	Sets the minimum expected amount of time (in millis) the assertIsSatisfied() will wait on a latch until it is satisfied.		long
resultWaitTime (producer)	Sets the maximum amount of time (in millis) the assertIsSatisfied() will wait on a latch until it is satisfied.		long
retainFirst (producer)	Specifies to only retain the first n'th number of received Exchanges. This is used when testing with big data, to reduce memory consumption by not storing copies of every Exchange this mock endpoint receives. Important: When using this limitation, then the getReceivedCounter() will still return the actual number of received Exchanges. For example if we have received 5000 Exchanges, and have configured to only retain the first 10 Exchanges, then the getReceivedCounter() will still return 5000 but there is only the first 10 Exchanges in the getExchanges() and getReceivedExchanges() methods. When using this method, then some of the other expectation methods is not supported, for example the expectedBodiesReceived(Object...) sets a expectation on the first number of bodies received. You can configure both setRetainFirst(int) and setRetainLast(int) methods, to limit both the first and last received.	-1	int

Name	Description	Default	Type
retainLast (producer)	Specifies to only retain the last n'th number of received Exchanges. This is used when testing with big data, to reduce memory consumption by not storing copies of every Exchange this mock endpoint receives. Important: When using this limitation, then the <code>getReceivedCounter()</code> will still return the actual number of received Exchanges. For example if we have received 5000 Exchanges, and have configured to only retain the last 20 Exchanges, then the <code>getReceivedCounter()</code> will still return 5000 but there is only the last 20 Exchanges in the <code>getExchanges()</code> and <code>getReceivedExchanges()</code> methods. When using this method, then some of the other expectation methods is not supported, for example the <code>expectedBodiesReceived(Object...)</code> sets a expectation on the first number of bodies received. You can configure both <code>setRetainFirst(int)</code> and <code>setRetainLast(int)</code> methods, to limit both the first and last received.	-1	int
sleepForEmptyTest (producer)	Allows a sleep to be specified to wait to check that this endpoint really is empty when <code>expectedMessageCount(int)</code> is called with zero.		long
copyOnExchange (producer (advanced))	Sets whether to make a deep copy of the incoming Exchange when received at this mock endpoint. Is by default true.	true	boolean

32.6. CONFIGURING DATASET

Camel will lookup in the Registry for a bean implementing the `DataSet` interface. So you can register your own `DataSet` as:

```
<bean id="myDataSet" class="com.mycompany.MyDataSet">
  <property name="size" value="100"/>
</bean>
```

32.7. EXAMPLE

For example, to test that a set of messages are sent to a queue and then consumed from the queue without losing any messages:

```
// send the dataset to a queue
from("dataset:foo").to("activemq:SomeQueue");

// now lets test that the messages are consumed correctly
from("activemq:SomeQueue").to("dataset:foo");
```

The above would look in the Registry to find the **foo** DataSet instance which is used to create the messages.

Then you create a DataSet implementation, such as using the **SimpleDataSet** as described below, configuring things like how big the data set is and what the messages look like etc.

32.8. DATASETSUPPORT (ABSTRACT CLASS)

The DataSetSupport abstract class is a nice starting point for new DataSets, and provides some useful features to derived classes.

32.8.1. Properties on DataSetSupport

Property	Type	Default	Description
defaultHeaders	Map<String, Object>	null	Specifies the default message body. For SimpleDataSet it is a constant payload; though if you want to create custom payloads per message, create your own derivation of DataSetSupport .
outputTransformer	org.apache.camel.Processor	null	
size	long	10	Specifies how many messages to send/consume.
reportCount	long	-1	Specifies the number of messages to be received before reporting progress. Useful for showing progress of a large load test. If < 0, then size / 5, if is 0 then size , else set to reportCount value.

32.9. SIMPLEDATASET

The **SimpleDataSet** extends **DataSetSupport**, and adds a default body.

32.9.1. Additional Properties on SimpleDataSet

Property	Type	Default	Description
----------	------	---------	-------------

Property	Type	Default	Description
defaultBody	Object	<hello>world! </hello>	Specifies the default message body. By default, the SimpleDataSet produces the same constant payload for each exchange. If you want to customize the payload for each exchange, create a Camel Processor and configure the SimpleDataSet to use it by setting the outputTransformer property.

32.10. LISTDATASET

The `ListDataSet` extends **DataSetSupport**, and adds a list of default bodies.

32.10.1. Additional Properties on ListDataSet

Property	Type	Default	Description
defaultBodies	List<Object>	empty LinkedList<Object>	Specifies the default message body. By default, the ListDataSet selects a constant payload from the list of defaultBodies using the CamelDataSetIndex . If you want to customize the payload, create a Camel Processor and configure the ListDataSet to use it by setting the outputTransformer property.
size	long	the size of the defaultBodies list	Specifies how many messages to send/consume. This value can be different from the size of the defaultBodies list. If the value is less than the size of the defaultBodies list, some of the list elements will not be used. If the value is greater than the size of the defaultBodies list, the payload for the exchange will be selected using the modulus of the CamelDataSetIndex and the size of the defaultBodies list (i.e. CamelDataSetIndex % defaultBodies.size())

32.11. FILEDATASET

The **FileDataSet** extends **ListDataSet**, and adds support for loading the bodies from a file.

32.11.1. Additional Properties on FileDataSet

Property	Type	Default	Description
sourceFile	File	null	Specifies the source file for payloads
delimiter	String	\z	Specifies the delimiter pattern used by a java.util.Scanner to split the file into multiple payloads.

32.12. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
camel.component.dataset-test.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.dataset-test.enabled	Whether to enable auto configuration of the dataset-test component. This is enabled by default.		Boolean
camel.component.dataset-test.exchange-formatter	Sets a custom ExchangeFormatter to convert the Exchange to a String suitable for logging. If not specified, we default to DefaultExchangeFormatter. The option is a <code>org.apache.camel.spi.ExchangeFormatter</code> type.		ExchangeFormatter
camel.component.dataset-test.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.dataset-test.log	To turn on logging when the mock receives an incoming message. This will log only one time at INFO level for the incoming message. For more detailed logging then set the logger to DEBUG level for the <code>org.apache.camel.component.mock.MockEndpoint</code> class.	false	Boolean
camel.component.dataset.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.dataset.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.dataset.enabled	Whether to enable auto configuration of the dataset component. This is enabled by default.		Boolean
camel.component.dataset.exchange-formatter	Sets a custom <code>ExchangeFormatter</code> to convert the Exchange to a String suitable for logging. If not specified, we default to <code>DefaultExchangeFormatter</code> . The option is a <code>org.apache.camel.spi.ExchangeFormatter</code> type.		<code>ExchangeFormatter</code>
camel.component.dataset.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.dataset.log	To turn on logging when the mock receives an incoming message. This will log only one time at INFO level for the incoming message. For more detailed logging then set the logger to DEBUG level for the <code>org.apache.camel.component.mock.MockEndpoint</code> class.	false	Boolean

CHAPTER 33. DIRECT

Both producer and consumer are supported

The Direct component provides direct, synchronous invocation of any consumers when a producer sends a message exchange.

This endpoint can be used to connect existing routes in the **same** camel context.



NOTE

Asynchronous

The [SEDA](#) component provides asynchronous invocation of any consumers when a producer sends a message exchange.

33.1. DEPENDENCIES

When using **direct** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-direct-starter</artifactId>
</dependency>
```

33.2. URI FORMAT

```
direct:someName[?options]
```

Where **someName** can be any string to uniquely identify the endpoint

33.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

33.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).

- directly in the Java code.

33.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

33.4. COMPONENT OPTIONS

The Direct component supports 5 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
block (producer)	If sending a message to a direct endpoint which has no active consumer, then we can tell the producer to block and wait for the consumer to become active.	true	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
timeout (producer)	The timeout value to use if block is enabled.	30000	long
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

33.5. ENDPOINT OPTIONS

The Direct endpoint is configured using URI syntax:

```
direct:name
```

with the following path and query parameters:

33.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (common)	Required Name of direct endpoint.		String

33.5.2. Query Parameters (8 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
block (producer)	If sending a message to a direct endpoint which has no active consumer, then we can tell the producer to block and wait for the consumer to become active.	true	boolean
failIfNoConsumers (producer)	Whether the producer should fail by throwing an exception, when sending to a <code>DIRECT</code> endpoint with no active consumers.	true	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
timeout (producer)	The timeout value to use if block is enabled.	30000	long
synchronous (advanced)	Whether synchronous processing is forced. If enabled then the producer thread, will be forced to wait until the message has been completed before the same thread will continue processing. If disabled (default) then the producer thread may be freed and can do other work while the message is continued processed by other threads (reactive).	false	boolean

33.6. SAMPLES

In the route below we use the direct component to link the two routes together:

```
from("activemq:queue:order.in")
  .to("bean:orderServer?method=validate")
  .to("direct:processOrder");

from("direct:processOrder")
  .to("bean:orderService?method=process")
  .to("activemq:queue:order.out");
```

And the sample using spring DSL:

```
<route>
  <from uri="activemq:queue:order.in"/>
  <to uri="bean:orderService?method=validate"/>
  <to uri="direct:processOrder"/>
</route>

<route>
  <from uri="direct:processOrder"/>
  <to uri="bean:orderService?method=process"/>
  <to uri="activemq:queue:order.out"/>
</route>
```

See also samples from the [SEDA](#) component, how they can be used together.

33.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
camel.component. .direct.autowired- enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component. .direct.block	If sending a message to a direct endpoint which has no active consumer, then we can tell the producer to block and wait for the consumer to become active.	true	Boolean
camel.component. .direct.bridge- error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component. .direct.enabled	Whether to enable auto configuration of the direct component. This is enabled by default.		Boolean
camel.component. .direct.lazy-start- producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component. .direct.timeout	The timeout value to use if block is enabled.	30000	Long

CHAPTER 34. ELASTICSEARCH

Since Camel 3.18.3

Only producer is supported

The ElasticSearch component allows you to interface with an [ElasticSearch](#) 8.x API using the Java API Client library.

34.1. DEPENDENCIES

When using **elasticsearch** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-elasticsearch-starter</artifactId>
</dependency>
```

34.2. URI FORMAT

```
elasticsearch://clusterName[?options]
```

34.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

34.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

34.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

34.4. COMPONENT OPTIONS

The Elasticsearch component supports 14 options, which are listed below.

Name	Description	Default	Type
connectionTimeout (producer)	The time in ms to wait before connection will timeout.	30000	int
hostAddresses (producer)	Comma separated list with ip:port formatted remote transport addresses to use. The ip and port options must be left blank for hostAddresses to be considered instead.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
maxRetryTimeout (producer)	The time in ms before retry.	30000	int
socketTimeout (producer)	The timeout in ms to wait before the socket will timeout.	30000	int

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
client (advanced)	Autowired To use an existing configured Elasticsearch client, instead of creating a client per endpoint. This allow to customize the client with specific settings.		RestClient
enableSniffer (advanced)	Enable automatically discover nodes from a running Elasticsearch cluster. If this option is used in conjunction with Spring Boot then it's managed by the Spring Boot configuration (see: Disable Sniffer in Spring Boot).	false	boolean
sniffAfterFailure Delay (advanced)	The delay of a sniff execution scheduled after a failure (in milliseconds).	60000	int
snifferInterval (advanced)	The interval between consecutive ordinary sniff executions in milliseconds. Will be honoured when sniffOnFailure is disabled or when there are no failures between consecutive sniff executions.	30000 0	int
certificatePath (security)	The path of the self-signed certificate to use to access to Elasticsearch.		String
enableSSL (security)	Enable SSL.	false	boolean
password (security)	Password for authenticate.		String
user (security)	Basic authenticate user.		String

34.5. ENDPOINT OPTIONS

The Elasticsearch endpoint is configured using URI syntax:

```
elasticsearch:clusterName
```

with the following path and query parameters:

34.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
clusterName (producer)	Required Name of the cluster.		String

34.5.2. Query Parameters (19 parameters)

Name	Description	Default	Type
connectionTimeout (producer)	The time in ms to wait before connection will timeout.	30000	int
disconnect (producer)	Disconnect after it finish calling the producer.	false	boolean
from (producer)	Starting index of the response.		Integer
hostAddresses (producer)	Comma separated list with ip:port formatted remote transport addresses to use.		String
indexName (producer)	The name of the index to act against.		String
maxRetryTimeout (producer)	The time in ms before retry.	30000	int

Name	Description	Default	Type
operation (producer)	What operation to perform. Enum values: • Index • Update • Bulk • GetById • MultiGet • MultiSearch • Delete • DeleteIndex • Search • Exists • Ping		ElasticsearchOperation
scrollKeepAliveMs (producer)	Time in ms during which elasticsearch will keep search context alive.	60000	int
size (producer)	Size of the response.		Integer
socketTimeout (producer)	The timeout in ms to wait before the socket will timeout.	30000	int
useScroll (producer)	Enable scroll usage.	false	boolean
waitForActiveShards (producer)	Index creation waits for the write consistency number of shards to be available.	1	int
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
documentClass (advanced)	The class to use when deserializing the documents.	Object Node	Class
enableSniffer (advanced)	Enable automatically discover nodes from a running Elasticsearch cluster. If this option is used in conjunction with Spring Boot then it's managed by the Spring Boot configuration (see: Disable Sniffer in Spring Boot).	false	boolean
sniffAfterFailure Delay (advanced)	The delay of a sniff execution scheduled after a failure (in milliseconds).	60000	int
snifferInterval (advanced)	The interval between consecutive ordinary sniff executions in milliseconds. Will be honoured when sniffOnFailure is disabled or when there are no failures between consecutive sniff executions.	30000 0	int
certificatePath (security)	The path of the self-signed certificate to use to access to Elasticsearch.		String
enableSSL (security)	Enable SSL.	false	boolean

34.6. MESSAGE HEADERS

The Elasticsearch component supports 9 message header(s), which is/are listed below:

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
operation (producer) Constant: PARAM_OPERATION	The operation to perform. Enum values: • Index • Update • Bulk • GetById • MultiGet • MultiSearch • Delete • DeleteIndex • Search • Exists • Ping		ElasticsearchOperation
indexId (producer) Constant: PARAM_INDEX_ID	The id of the indexed document.		String
indexName (producer) Constant: PARAM_INDEX_NAME	The name of the index to act against.		String
documentClass (producer) Constant: PARAM_DOCUMENT_CLASS	The full qualified name of the class of the document to unmarshall.	Object Node	Class
waitForActiveShards (producer) Constant: PARAM_WAIT_FOR_ACTIVE_SHARDS	The index creation waits for the write consistency number of shards to be available.		Integer

Name	Description	Default	Type
scrollKeepAliveMs (producer) Constant: PARAM_SCROLL_KEEP_ALIVE_MS	The starting index of the response.		Integer
useScroll (producer) Constant: PARAM_SCROLL	Set to true to enable scroll usage.		Boolean
size (producer) Constant: PARAM_SIZE	The size of the response.		Integer
from (producer) Constant: PARAM_FROM	The starting index of the response.		Integer

34.7. MESSAGE OPERATIONS

The following ElasticSearch operations are currently supported. Simply set an endpoint URI option or exchange header with a key of "operation" and a value set to one of the following. Some operations also require other parameters or the message body to be set.

operation	message body	description
Index	Map, String, byte[], Reader, InputStream or IndexRequest.Builder content to index	Adds content to an index and returns the content's indexId in the body. You can set the name of the target index by setting the message header with the key "indexName". You can set the indexId by setting the message header with the key "indexId".

operation	message body	description
GetById	String or GetRequest.Builder index id of content to retrieve	Retrieves the document corresponding to the given index id and returns a <code>GetResponse</code> object in the body. You can set the name of the target index by setting the message header with the key "indexName". You can set the type of document by setting the message header with the key "documentClass".
Delete	String or DeleteRequest.Builder index id of content to delete	Deletes the specified indexName and returns a <code>Result</code> object in the body. You can set the name of the target index by setting the message header with the key "indexName".
DeleteIndex	String or DeleteIndexRequest.Builder index name of the index to delete	Deletes the specified indexName and returns a status code in the body. You can set the name of the target index by setting the message header with the key "indexName".

operation	message body	description
Bulk	Iterable or BulkRequest.Builder of any type that is already accepted (DeleteOperation.Builder for delete operation, UpdateOperation.Builder for update operation, CreateOperation.Builder for create operation, byte[], InputStream, String, Reader, Map or any document type for index operation)	Adds/Updates/Deletes content from/to an index and returns a List<BulkResponseItem> object in the body You can set the name of the target index by setting the message header with the key "indexName".

operation	message body	description
Search	Map, String or SearchRequest.Builder	Search the content with the map of query string. You can set the name of the target index by setting the message header with the key "indexName". You can set the number of hits to return by setting the message header with the key "size". You can set the starting document offset by setting the message header with the key "from".
MultiSearch	MsearchRequest.Builder	Multiple search in one
MultiGet	Iterable<String> or MgetRequest.Builder the id of the document to retrieve	Multiple get in one You can set the name of the target index by setting the message header with the key "indexName".
Exists	None	Checks whether the index exists or not and returns a Boolean flag in the body. You must set the name of the target index by setting the message header with the key "indexName".
Update	byte[], InputStream, String, Reader, Map or any document type content to update	Updates content to an index and returns the content's indexId in the body. You can set the name of the target index by setting the message header with the key "indexName". You can set the indexId by setting the message header with the key "indexId".
Ping	None	Pings the Elasticsearch cluster and returns true if the ping succeeded, false otherwise

34.8. CONFIGURE THE COMPONENT AND ENABLE BASIC AUTHENTICATION

To use the Elasticsearch component it has to be configured with a minimum configuration.

```
ElasticsearchComponent elasticsearchComponent = new ElasticsearchComponent();
elasticsearchComponent.setHostAddresses("myelkhost:9200");
camelContext.addComponent("elasticsearch", elasticsearchComponent);
```

For basic authentication with elasticsearch or using reverse http proxy in front of the elasticsearch cluster, simply setup basic authentication and SSL on the component like the example below

```
ElasticsearchComponent elasticsearchComponent = new ElasticsearchComponent();
elasticsearchComponent.setHostAddresses("myelkhost:9200");
elasticsearchComponent.setUser("elkuser");
elasticsearchComponent.setPassword("secure!!");
elasticsearchComponent.setEnableSSL(true);
elasticsearchComponent.setCertificatePath(certPath);

camelContext.addComponent("elasticsearch", elasticsearchComponent);
```

34.9. INDEX EXAMPLE

Below is a simple INDEX example

```
from("direct:index")
  .to("elasticsearch://elasticsearch?operation=Index&indexName=twitter");

<route>
  <from uri="direct:index"/>
  <to uri="elasticsearch://elasticsearch?operation=Index&indexName=twitter"/>
</route>
```

For this operation you need to specify a `indexId` header.

A client would simply need to pass a body message containing a Map to the route. The result body contains the `indexId` created.

```
Map<String, String> map = new HashMap<String, String>();
map.put("content", "test");
String indexId = template.requestBody("direct:index", map, String.class);
```

34.10. SEARCH EXAMPLE

Searching on specific field(s) and value use the Operation `Search`. Pass in the query JSON String or the Map

```
from("direct:search")
  .to("elasticsearch://elasticsearch?operation=Search&indexName=twitter");

<route>
```



```

    <from uri="direct:search"/>
    <to uri="elasticsearch://elasticsearch?operation=Search&indexName=twitter"/>
</route>

```

```

String query = "{\"query\":{\"match\":{\"doc.content\":\"new release of ApacheCamel\"}}}";
HitsMetadata<?> response = template.requestBody("direct:search", query, HitsMetadata.class);

```

Search on specific field(s) using Map.

```

Map<String, Object> actualQuery = new HashMap<>();
actualQuery.put("doc.content", "new release of ApacheCamel");

Map<String, Object> match = new HashMap<>();
match.put("match", actualQuery);

Map<String, Object> query = new HashMap<>();
query.put("query", match);
HitsMetadata<?> response = template.requestBody("direct:search", query, HitsMetadata.class);

```

Search using Elasticsearch scroll api in order to fetch all results.

```

from("direct:search")
.to("elasticsearch://elasticsearch?
operation=Search&indexName=twitter&useScroll=true&scrollKeepAliveMs=30000");

```

```

<route>
  <from uri="direct:search"/>
  <to uri="elasticsearch://elasticsearch?
operation=Search&indexName=twitter&useScroll=true&scrollKeepAliveMs=30000"/>
</route>

```

```

String query = "{\"query\":{\"match\":{\"doc.content\":\"new release of ApacheCamel\"}}}";
try (ElasticsearchScrollRequestIterator response = template.requestBody("direct:search", query,
ElasticsearchScrollRequestIterator.class)) {
    // do something smart with results
}

```

can also be used.

```

from("direct:search")
.to("elasticsearch://elasticsearch?
operation=Search&indexName=twitter&useScroll=true&scrollKeepAliveMs=30000")
.split()
.body()
.streaming()
.to("mock:output")
.end();

```

34.11. MULTISEARCH EXAMPLE

MultiSearching on specific field(s) and value use the Operation `MultiSearch`. Pass in the `MultiSearchRequest` instance

```
from("direct:multiSearch")
  .to("elasticsearch://elasticsearch?operation=MultiSearch");
```

```
<route>
  <from uri="direct:multiSearch"/>
  <to uri="elasticsearch://elasticsearch?operation=MultiSearch"/>
</route>
```

MultiSearch on specific field(s)

```
MsearchRequest.Builder builder = new MsearchRequest.Builder().index("twitter").searches(
    new RequestItem.Builder().header(new MultisearchHeader.Builder().build())
        .body(new MultisearchBody.Builder().query(b -> b.matchAll(x -> x)).build()).build(),
    new RequestItem.Builder().header(new MultisearchHeader.Builder().build())
        .body(new MultisearchBody.Builder().query(b -> b.matchAll(x -> x)).build()).build());
List<MultiSearchResponseItem<?>> response = template.requestBody("direct:multiSearch", builder,
    List.class);
```

34.12. DOCUMENT TYPE

For all the search operations, it is possible to indicate the type of document to retrieve in order to get the result already unmarshalled with the expected type.

The document type can be set using the header "documentClass" or via the uri parameter of the same name.

34.13. USING CAMEL ELASTICSEARCH WITH SPRING BOOT

When you use **camel-elasticsearch-starter** with Spring Boot v2, then you must declare the following dependency in your own **pom.xml**.

```
<dependency>
  <groupId>jakarta.json</groupId>
  <artifactId>jakarta.json-api</artifactId>
  <version>2.0.2</version>
</dependency>
```

This is needed because Spring Boot v2 provides jakarta.json-api:1.1.6, and Elasticsearch requires to use json-api v2.

34.13.1. Use RestClient provided by Spring Boot

By default Spring Boot will auto configure an Elasticsearch RestClient that will be used by camel, it is possible to customize the client with the following basic properties:

```
spring.elasticsearch.uris=myelkhost:9200
spring.elasticsearch.username=elkuser
spring.elasticsearch.password=secure!!
```

More information can be found in [application-properties.data.spring.elasticsearch.connection-timeout](#).

34.13.2. Disable Sniffer when using Spring Boot

When Spring Boot is on the classpath the Sniffer client for Elasticsearch is enabled by default. This option can be disabled in the Spring Boot Configuration:

```
spring:
  autoconfigure:
    exclude:
      org.springframework.boot.autoconfigure.elasticsearch.ElasticsearchRestClientAutoConfiguration
```

34.14. SPRING BOOT AUTO-CONFIGURATION

The component supports 15 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.elasticsearch.automated-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.elasticsearch.certificate-path</code>	The path of the self-signed certificate to use to access to Elasticsearch.		String
<code>camel.component.elasticsearch.client</code>	To use an existing configured Elasticsearch client, instead of creating a client per endpoint. This allow to customize the client with specific settings. The option is a <code>org.elasticsearch.client.RestClient</code> type.		RestClient
<code>camel.component.elasticsearch.connection-timeout</code>	The time in ms to wait before connection will timeout.	30000	Integer
<code>camel.component.elasticsearch.enable-ssl</code>	Enable SSL.	false	Boolean
<code>camel.component.elasticsearch.enable-sniffer</code>	Enable automatically discover nodes from a running Elasticsearch cluster. If this option is used in conjunction with Spring Boot then it's managed by the Spring Boot configuration (see: Disable Sniffer in Spring Boot).	false	Boolean
<code>camel.component.elasticsearch.enabled</code>	Whether to enable auto configuration of the elasticsearch component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.elasticsearch.host-addresses	Comma separated list with ip:port formatted remote transport addresses to use. The ip and port options must be left blank for hostAddresses to be considered instead.		String
camel.component.elasticsearch.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.elasticsearch.max-retry-timeout	The time in ms before retry.	30000	Integer
camel.component.elasticsearch.password	Password for authenticate.		String
camel.component.elasticsearch.sniff-after-failure-delay	The delay of a sniff execution scheduled after a failure (in milliseconds).	60000	Integer
camel.component.elasticsearch.sniffer-interval	The interval between consecutive ordinary sniff executions in milliseconds. Will be honoured when sniffOnFailure is disabled or when there are no failures between consecutive sniff executions.	30000 0	Integer
camel.component.elasticsearch.socket-timeout	The timeout in ms to wait before the socket will timeout.	30000	Integer
camel.component.elasticsearch.user	Basic authenticate user.		String

CHAPTER 35. EXCHANGEPROPERTY

The ExchangeProperty Expression Language allows you to extract values of named exchange properties.

35.1. DEPENDENCIES

The ExchangeProperty language is part of **camel-core**.

When using **exchangeProperty** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-core-starter</artifactId>
</dependency>
```

35.2. EXCHANGE PROPERTY OPTIONS

The ExchangeProperty language supports 1 options, which are listed below.

Name	Default	Java Type	Description
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

35.3. EXAMPLE

The **recipientList** EIP can utilize a exchangeProperty like:

```
<route>
  <from uri="direct:a" />
  <recipientList>
    <exchangeProperty>myProperty</exchangeProperty>
  </recipientList>
</route>
```

In this case, the list of recipients are contained in the property 'myProperty'.

And the same example in Java DSL:

```
from("direct:a").recipientList(exchangeProperty("myProperty"));
```

35.4. SPRING BOOT AUTO-CONFIGURATION

The component supports 147 options, which are listed below.

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.acl-token</code>	Sets the ACL token to be used with Consul.		String
<code>camel.cloud.consul.service-discovery.block-seconds</code>	The seconds to wait for a watch event, default 10 seconds.	10	Integer
<code>camel.cloud.consul.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.consul.service-discovery.connect-timeout-millis</code>	Connect timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.datacenter</code>	The data center.		String
<code>camel.cloud.consul.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.consul.service-discovery.password</code>	Sets the password to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.consul.service-discovery.read-timeout-millis</code>	Read timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.url</code>	The Consul agent URL.		String

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.username</code>	Sets the username to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.write-timeout-millis</code>	Write timeout for OkHttpClient.		Long
<code>camel.cloud.dns.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.dns.service-discovery.domain</code>	The domain name;.		String
<code>camel.cloud.dns.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.dns.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.dns.service-discovery.proto</code>	The transport protocol of the desired service.	<code>_tcp</code>	String
<code>camel.cloud.etcd.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.etcd.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.etcd.service-discovery.password</code>	The password to use for basic authentication.		String

Name	Description	Default	Type
camel.cloud.etcd.service-discovery.properties	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
camel.cloud.etcd.service-discovery.service-path	The path to look for for service discovery.	<code>/services/</code>	String
camel.cloud.etcd.service-discovery.timeout	To set the maximum time an action could take to complete.		Long
camel.cloud.etcd.service-discovery.type	To set the discovery type, valid values are on-demand and watch.	on-demand	String
camel.cloud.etcd.service-discovery.uris	The URIs the client can connect to.		String
camel.cloud.etcd.service-discovery.user-name	The user name to use for basic authentication.		String
camel.cloud.kubernetes.service-discovery.api-version	Sets the API version when using client lookup.		String
camel.cloud.kubernetes.service-discovery.ca-cert-data	Sets the Certificate Authority data when using client lookup.		String
camel.cloud.kubernetes.service-discovery.ca-cert-file	Sets the Certificate Authority data that are loaded from the file when using client lookup.		String
camel.cloud.kubernetes.service-discovery.client-cert-data	Sets the Client Certificate data when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.client-cert-file</code>	Sets the Client Certificate data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-algo</code>	Sets the Client Keystore algorithm, such as RSA when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-data</code>	Sets the Client Keystore data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-file</code>	Sets the Client Keystore data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-passphrase</code>	Sets the Client Keystore passphrase when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.kubernetes.service-discovery.dns-domain</code>	Sets the DNS domain to use for DNS lookup.		String
<code>camel.cloud.kubernetes.service-discovery.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.lookup</code>	How to perform service lookup. Possible values: client, dns, environment. When using client, then the client queries the kubernetes master to obtain a list of active pods that provides the service, and then random (or round robin) select a pod. When using dns the service name is resolved as name.namespace.svc.dnsDomain. When using dnssrv the service name is resolved with SRV query forsvc... When using environment then environment variables are used to lookup the service. By default environment is used.	environment	String
<code>camel.cloud.kubernetes.service-discovery.master-url</code>	Sets the URL to the master when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.namespace</code>	Sets the namespace to use. Will by default use namespace from the ENV variable KUBERNETES_MASTER.		String
<code>camel.cloud.kubernetes.service-discovery.oauth-token</code>	Sets the OAUTH token for authentication (instead of username/password) when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.password</code>	Sets the password for authentication when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-name</code>	Sets the Port Name to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-protocol</code>	Sets the Port Protocol to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in com.netflix.client.config.CommonClientConfigKey.		Map

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.trust-certs</code>	Sets whether to turn on trust certificate check when using client lookup.	false	Boolean
<code>camel.cloud.kubernetes.service-discovery.username</code>	Sets the username for authentication when using client lookup.		String
<code>camel.cloud.ribbon.load-balancer.client-name</code>	Sets the Ribbon client name.		String
<code>camel.cloud.ribbon.load-balancer.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.ribbon.load-balancer.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.ribbon.load-balancer.namespace</code>	The namespace.		String
<code>camel.cloud.ribbon.load-balancer.password</code>	The password.		String
<code>camel.cloud.ribbon.load-balancer.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.ribbon.load-balancer.username</code>	The username.		String

Name	Description	Default	Type
camel.hystrix.allow-maximum-size-to-diverge-from-core-size	Allows the configuration for <code>maximumSize</code> to take effect. That value can then be equal to, or higher, than <code>coreSize</code> .	false	Boolean
camel.hystrix.circuit-breaker-enabled	Whether to use a <code>HystrixCircuitBreaker</code> or not. If false no circuit-breaker logic will be used and all requests permitted. This is similar in effect to <code>circuitBreakerForceClosed()</code> except that continues tracking metrics and knowing whether it should be open/closed, this property results in not even instantiating a circuit-breaker.	true	Boolean
camel.hystrix.circuit-breaker-error-threshold-percentage	Error percentage threshold (as whole number such as 50) at which point the circuit breaker will trip open and reject requests. It will stay tripped for the duration defined in <code>circuitBreakerSleepWindowInMilliseconds</code> ; The error percentage this is compared against comes from <code>HystrixCommandMetrics.getHealthCounts()</code> .	50	Integer
camel.hystrix.circuit-breaker-force-closed	If true the <code>HystrixCircuitBreaker#allowRequest()</code> will always return true to allow requests regardless of the error percentage from <code>HystrixCommandMetrics.getHealthCounts()</code> . The <code>circuitBreakerForceOpen()</code> property takes precedence so if it set to true this property does nothing.	false	Boolean
camel.hystrix.circuit-breaker-force-open	If true the <code>HystrixCircuitBreaker.allowRequest()</code> will always return false, causing the circuit to be open (tripped) and reject all requests. This property takes precedence over <code>circuitBreakerForceClosed()</code> ;	false	Boolean
camel.hystrix.circuit-breaker-request-volume-threshold	Minimum number of requests in the <code>metricsRollingStatisticalWindowInMilliseconds()</code> that must exist before the <code>HystrixCircuitBreaker</code> will trip. If below this number the circuit will not trip regardless of error percentage.	20	Integer
camel.hystrix.circuit-breaker-sleep-window-in-milliseconds	The time in milliseconds after a <code>HystrixCircuitBreaker</code> trips open that it should wait before trying requests again.	5000	Integer
camel.hystrix.configurations	Define additional configuration definitions.		Map

Name	Description	Default	Type
camel.hystrix.core-pool-size	Core thread-pool size that gets passed to <code>java.util.concurrent.ThreadPoolExecutor#setCorePoolSize(int)</code> .	10	Integer
camel.hystrix.enabled	Enable the component.	true	Boolean
camel.hystrix.execution-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.run()</code> . Requests beyond the concurrent limit will be rejected. Applicable only when <code>executionIsolationStrategy == SEMAPHORE</code> .	20	Integer
camel.hystrix.execution-isolation-strategy	What isolation strategy <code>HystrixCommand.run()</code> will be executed with. If <code>THREAD</code> then it will be executed on a separate thread and concurrent requests limited by the number of threads in the thread-pool. If <code>SEMAPHORE</code> then it will be executed on the calling thread and concurrent requests limited by the semaphore count.	THREAD	String
camel.hystrix.execution-isolation-thread-interrupt-on-timeout	Whether the execution thread should attempt an interrupt (using <code>Future#cancel()</code>) when a thread times out. Applicable only when <code>executionIsolationStrategy() == THREAD</code> .	true	Boolean
camel.hystrix.execution-timeout-enabled	Whether the timeout mechanism is enabled for this command.	true	Boolean
camel.hystrix.execution-timeout-in-milliseconds	Time in milliseconds at which point the command will timeout and halt execution. If <code>executionIsolationThreadInterruptOnTimeout == true</code> and the command is thread-isolated, the executing thread will be interrupted. If the command is semaphore-isolated and a <code>HystrixObservableCommand</code> , that command will get unsubscribed.	1000	Integer
camel.hystrix.fallback-ack-enabled	Whether <code>HystrixCommand.getFallback()</code> should be attempted when failure occurs.	true	Boolean
camel.hystrix.fallback-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.getFallback()</code> . Requests beyond the concurrent limit will fail-fast and not attempt retrieving a fallback.	10	Integer

Name	Description	Default	Type
camel.hystrix.group-key	Sets the group key to use. The default value is CamelHystrix.	Camel Hystrix	String
camel.hystrix.keep-alive-time	Keep-alive time in minutes that gets passed to <code>ThreadPoolExecutor#setKeepAliveTime(long, TimeUnit)</code> .	1	Integer
camel.hystrix.max-queue-size	Max queue size that gets passed to <code>BlockingQueue</code> in <code>HystrixConcurrencyStrategy.getBlockingQueue(int)</code> . This should only affect the instantiation of a threadpool - it is not eligible to change a queue size on the fly. For that, use <code>queueSizeRejectionThreshold()</code> .	-1	Integer
camel.hystrix.maximum-size	Maximum thread-pool size that gets passed to <code>ThreadPoolExecutor#setMaximumPoolSize(int)</code> . This is the maximum amount of concurrency that can be supported without starting to reject <code>HystrixCommands</code> . Please note that this setting only takes effect if you also set <code>allowMaximumSizeToDivergeFromCoreSize</code> .	10	Integer
camel.hystrix.metrics-health-snapshot-interval-in-milliseconds	Time in milliseconds to wait between allowing health snapshots to be taken that calculate success and error percentages and affect <code>HystrixCircuitBreaker.isOpen()</code> status. On high-volume circuits the continual calculation of error percentage can become CPU intensive thus this controls how often it is calculated.	500	Integer
camel.hystrix.metrics-rolling-percentile-bucket-size	Maximum number of values stored in each bucket of the rolling percentile. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-percentile-enabled	Whether percentile metrics should be captured using <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	true	Boolean
camel.hystrix.metrics-rolling-percentile-window-buckets	Number of buckets the rolling percentile window is broken into. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	6	Integer

Name	Description	Default	Type
camel.hystrix.metrics-rolling-percentile-window-in-milliseconds	Duration of percentile rolling window in milliseconds. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	10000	Integer
camel.hystrix.metrics-rolling-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside HystrixCommandMetrics.	10	Integer
camel.hystrix.metrics-rolling-statistical-window-in-milliseconds	This property sets the duration of the statistical rolling window, in milliseconds. This is how long metrics are kept for the thread pool. The window is divided into buckets and rolls by those increments.	10000	Integer
camel.hystrix.queue-size-rejection-threshold	Queue size rejection threshold is an artificial max size at which rejections will occur even if maxQueueSize has not been reached. This is done because the maxQueueSize of a BlockingQueue can not be dynamically changed and we want to support dynamically changing the queue size that affects rejections. This is used by HystrixCommand when queuing a thread for execution.	5	Integer
camel.hystrix.request-log-enabled	Whether HystrixCommand execution and events should be logged to HystrixRequestLog.	true	Boolean
camel.hystrix.thread-pool-key	Sets the thread pool key to use. Will by default use the same value as groupKey has been configured to use.	Camel Hystrix	String
camel.hystrix.thread-pool-rolling-number-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10	Integer
camel.hystrix.thread-pool-rolling-number-statistical-window-in-milliseconds	Duration of statistical rolling window in milliseconds. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10000	Integer
camel.language.constant.enabled	Whether to enable auto configuration of the constant language. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.language.constant.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.csimple.enabled</code>	Whether to enable auto configuration of the csimple language. This is enabled by default.		Boolean
<code>camel.language.csimple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.exchangeproperty.enabled</code>	Whether to enable auto configuration of the exchangeProperty language. This is enabled by default.		Boolean
<code>camel.language.exchangeproperty.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.file.enabled</code>	Whether to enable auto configuration of the file language. This is enabled by default.		Boolean
<code>camel.language.file.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.header.enabled</code>	Whether to enable auto configuration of the header language. This is enabled by default.		Boolean
<code>camel.language.header.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.ref.enabled</code>	Whether to enable auto configuration of the ref language. This is enabled by default.		Boolean
<code>camel.language.ref.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.simple.enabled</code>	Whether to enable auto configuration of the simple language. This is enabled by default.		Boolean
<code>camel.language.simple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.tokenize.enabled</code>	Whether to enable auto configuration of the tokenize language. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.language.tokenize.group-delimiter	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.		String
camel.language.tokenize.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.resilience4j.automatic-transition-from-open-to-half-open-enabled	Enables automatic transition from OPEN to HALF_OPEN state once the waitDurationInOpenState has passed.	false	Boolean
camel.resilience4j.circuit-breaker-ref	Refers to an existing io.github.resilience4j.circuitbreaker.CircuitBreaker instance to lookup and use from the registry. When using this, then any other circuit breaker options are not in use.		String
camel.resilience4j.config-ref	Refers to an existing io.github.resilience4j.circuitbreaker.CircuitBreakerConfig instance to lookup and use from the registry.		String
camel.resilience4j.configurations	Define additional configuration definitions.		Map
camel.resilience4j.enabled	Enable the component.	true	Boolean
camel.resilience4j.failure-rate-threshold	Configures the failure rate threshold in percentage. If the failure rate is equal or greater than the threshold the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 50 percentage.		Float
camel.resilience4j.minimum-number-of-calls	Configures the minimum number of calls which are required (per sliding window period) before the CircuitBreaker can calculate the error rate. For example, if minimumNumberOfCalls is 10, then at least 10 calls must be recorded, before the failure rate can be calculated. If only 9 calls have been recorded the CircuitBreaker will not transition to open even if all 9 calls have failed. Default minimumNumberOfCalls is 100.	100	Integer

Name	Description	Default	Type
camel.resilience4j.permitted-number-of-calls-in-half-open-state	Configures the number of permitted calls when the CircuitBreaker is half open. The size must be greater than 0. Default size is 10.	10	Integer
camel.resilience4j.sliding-window-size	Configures the size of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. slidingWindowSize configures the size of the sliding window. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. The slidingWindowSize must be greater than 0. The minimumNumberOfCalls must be greater than 0. If the slidingWindowType is COUNT_BASED, the minimumNumberOfCalls cannot be greater than slidingWindowSize. If the slidingWindowType is TIME_BASED, you can pick whatever you want. Default slidingWindowSize is 100.	100	Integer
camel.resilience4j.sliding-window-type	Configures the type of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. Default slidingWindowType is COUNT_BASED.	COUNT_BASED	String
camel.resilience4j.slow-call-duration-threshold	Configures the duration threshold (seconds) above which calls are considered as slow and increase the slow calls percentage. Default value is 60 seconds.	60	Integer
camel.resilience4j.slow-call-rate-threshold	Configures a threshold in percentage. The CircuitBreaker considers a call as slow when the call duration is greater than slowCallDurationThreshold. When the percentage of slow calls is equal or greater the threshold, the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 100 percentage which means that all recorded calls must be slower than slowCallDurationThreshold.		Float

Name	Description	Default	Type
camel.resilience4j.wait-duration-in-open-state	Configures the wait duration (in seconds) which specifies how long the CircuitBreaker should stay open, before it switches to half open. Default value is 60 seconds.	60	Integer
camel.resilience4j.writable-stack-trace-enabled	Enables writable stack traces. When set to false, <code>Exception.getStackTrace</code> returns a zero length array. This may be used to reduce log spam when the circuit breaker is open as the cause of the exceptions is already known (the circuit breaker is short-circuiting calls).	true	Boolean
camel.rest.api-component	The name of the Camel component to use as the REST API (such as swagger) If no API Component has been explicit configured, then Camel will lookup if there is a Camel component responsible for servicing and generating the REST API documentation, or if a <code>org.apache.camel.spi.RestApiProcessorFactory</code> is registered in the registry. If either one is found, then that is being used.		String
camel.rest.api-context-path	Sets a leading API context-path the REST API services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path.		String
camel.rest.api-context-route-id	Sets the route id to use for the route that services the REST API. The route will by default use an auto assigned route id.		String
camel.rest.api-host	To use an specific hostname for the API documentation (eg swagger) This can be used to override the generated host with this configured hostname.		String
camel.rest.api-property	Allows to configure as many additional properties for the api documentation (swagger). For example set property <code>api.title</code> to my cool stuff.		Map
camel.rest.api-vendor-extension	Whether vendor extension is enabled in the Rest APIs. If enabled then Camel will include additional information as vendor extension (eg keys starting with x-) such as route ids, class names etc. Not all 3rd party API gateways and tools supports vendor-extensions when importing your API docs.	false	Boolean
camel.rest.binding-mode	Sets the binding mode to use. The default value is off.		RestBindingMode

Name	Description	Default	Type
camel.rest.client-request-validation	Whether to enable validation of the client request to check whether the Content-Type and Accept headers from the client is supported by the Rest-DSL configuration of its consumes/produces settings. This can be turned on, to enable this check. In case of validation error, then HTTP Status codes 415 or 406 is returned. The default value is false.	false	Boolean
camel.rest.component	The Camel Rest component to use for the REST transport (consumer), such as netty-http, jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String
camel.rest.component-property	Allows to configure as many additional properties for the rest component in use.		Map
camel.rest.consumer-property	Allows to configure as many additional properties for the rest consumer in use.		Map
camel.rest.context-path	Sets a leading context-path the REST services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path. Or for components such as camel-jetty or camel-netty-http that includes a HTTP server.		String
camel.rest.cors-headers	Allows to configure custom CORS headers.		Map
camel.rest.data-format-property	Allows to configure as many additional properties for the data formats in use. For example set property prettyPrint to true to have json outputted in pretty mode. The properties can be prefixed to denote the option is only for either JSON or XML and for either the IN or the OUT. The prefixes are: json.in. json.out. xml.in. xml.out. For example a key with value xml.out.mustBeJAXBELEMENT is only for the XML data format for the outgoing. A key without a prefix is a common key for all situations.		Map
camel.rest.enable-cors	Whether to enable CORS headers in the HTTP response. The default value is false.	false	Boolean

Name	Description	Default	Type
camel.rest.endpoint-property	Allows to configure as many additional properties for the rest endpoint in use.		Map
camel.rest.host	The hostname to use for exposing the REST service.		String
camel.rest.hostname-resolver	If no hostname has been explicit configured, then this resolver is used to compute the hostname the REST service will be using.		RestHostNameResolver
camel.rest.json-data-format	Name of specific json data format to use. By default json-jackson will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.port	The port number to use for exposing the REST service. Notice if you use servlet component then the port number configured here does not apply, as the port number in use is the actual port number the servlet component is using. eg if using Apache Tomcat its the tomcat http port, if using Apache Karaf its the HTTP service in Karaf that uses port 8181 by default etc. Though in those situations setting the port number here, allows tooling and JMX to know the port number, so its recommended to set the port number to the number that the servlet engine uses.		String
camel.rest.producer-api-doc	Sets the location of the api document (swagger api) the REST producer will use to validate the REST uri and query parameters are valid accordingly to the api document. This requires adding camel-swagger-java to the classpath, and any miss configuration will let Camel fail on startup and report the error(s). The location of the api document is loaded from classpath by default, but you can use file: or http: to refer to resources to load from file or http url.		String
camel.rest.producer-component	Sets the name of the Camel component to use as the REST producer.		String
camel.rest.scheme	The scheme to use for exposing the REST service. Usually http or https is supported. The default value is http.		String
camel.rest.skip-binding-on-error-code	Whether to skip binding on output if there is a custom HTTP error code header. This allows to build custom error messages that do not bind to json / xml etc, as success messages otherwise will do.	false	Boolean

Name	Description	Default	Type
<code>camel.rest.use-x-forward-headers</code>	Whether to use X-Forward headers for Host and related setting. The default value is true.	true	Boolean
<code>camel.rest.xml-data-format</code>	Name of specific XML data format to use. By default jaxb will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
<code>camel.rest.api-context-id-pattern</code>	Deprecated Sets an CamelContext id pattern to only allow Rest APIs from rest services within CamelContext's which name matches the pattern. The pattern name refers to the CamelContext name, to match on the current CamelContext only. For any other value, the pattern uses the rules from PatternHelper#matchPattern(String,String).		String
<code>camel.rest.api-context-listing</code>	Deprecated Sets whether listing of all available CamelContext's with REST services in the JVM is enabled. If enabled it allows to discover these contexts, if false then only the current CamelContext is in use.	false	Boolean

CHAPTER 36. FHIR

Both producer and consumer are supported

The FHIR component integrates with the [HAPI-FHIR](#) library which is an open-source implementation of the [FHIR](#) (Fast Healthcare Interoperability Resources) specification in Java.

36.1. DEPENDENCIES

When using **fhir** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>  
  <groupId>org.apache.camel.springboot</groupId>  
  <artifactId>camel-fhir-starter</artifactId>  
</dependency>
```

36.2. URI FORMAT

The FHIR Component uses the following URI format:

```
fhir://endpoint-prefix/endpoint?[options]
```

Endpoint prefix can be one of:

- capabilities
- create
- delete
- history
- load-page
- meta
- operation
- patch
- read
- search
- transaction
- update
- validate

36.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

36.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

36.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

36.4. COMPONENT OPTIONS

The FHIR component supports 27 options, which are listed below.

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
encoding (common)	Encoding to use for all request. Enum values: • JSON • XML		String
fhirVersion (common)	The FHIR Version to use. Enum values: • DSTU2 • DSTU2_HL7ORG • DSTU2_1 • DSTU3 • R4 • R5	R4	String
log (common)	Will log every requests and responses.	false	boolean
prettyPrint (common)	Pretty print all request.	false	boolean
serverUrl (common)	The FHIR server base URL.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
client (advanced)	To use the custom client.		IGenericClient
clientFactory (advanced)	To use the custom client factory.		IRestfulClientFactory
compress (advanced)	Compresses outgoing (POST/PUT) contents to the GZIP format.	false	boolean
configuration (advanced)	To use the shared configuration.		FhirConfiguration
connectionTimeout (advanced)	How long to try and establish the initial TCP connection (in ms).	10000	Integer
deferModelScanning (advanced)	When this option is set, model classes will not be scanned for children until the child list for the given type is actually accessed.	false	boolean
fhirContext (advanced)	FhirContext is an expensive object to create. To avoid creating multiple instances, it can be set directly.		FhirContext
forceConformanceCheck (advanced)	Force conformance check.	false	boolean

Name	Description	Default	Type
sessionCookie (advanced)	HTTP session cookie to add to every request.		String
socketTimeout (advanced)	How long to block for individual read/write operations (in ms).	10000	Integer
summary (advanced)	Request that the server modify the response using the <code>_summary</code> param. Enum values: • COUNT • TEXT • DATA • TRUE • FALSE		String
validationMode (advanced)	When should Camel validate the FHIR Server's conformance statement. Enum values: • NEVER • ONCE	ONCE	String
proxyHost (proxy)	The proxy host.		String
proxyPassword (proxy)	The proxy password.		String
proxyPort (proxy)	The proxy port.		Integer
proxyUser (proxy)	The proxy username.		String
accessToken (security)	OAuth access token.		String
password (security)	Username to use for basic authentication.		String
username (security)	Username to use for basic authentication.		String

36.5. ENDPOINT OPTIONS

The FHIR endpoint is configured using URI syntax:

```
fhir:apiName/methodName
```

with the following path and query parameters:

36.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
apiName (common)	Required What kind of operation to perform. Enum values: • CAPABILITIES • CREATE • DELETE • HISTORY • LOAD_PAGE • META • OPERATION • PATCH • READ • SEARCH • TRANSACTION • UPDATE • VALIDATE		FhirApiName
methodName (common)	Required What sub operation to use for the selected operation.		String

36.5.2. Query Parameters (44 parameters)

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
encoding (common)	Encoding to use for all request. Enum values: • JSON • XML		String
fhirVersion (common)	The FHIR Version to use. Enum values: • DSTU2 • DSTU2_HL7ORG • DSTU2_1 • DSTU3 • R4 • R5	R4	String
inBody (common)	Sets the name of a parameter to be passed in the exchange In Body.		String
log (common)	Will log every requests and responses.	false	boolean
prettyPrint (common)	Pretty print all request.	false	boolean
serverUrl (common)	The FHIR server base URL.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an <code>Exchange</code> have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
client (advanced)	To use the custom client.		<code>IGenericClient</code>
clientFactory (advanced)	To use the custom client factory.		<code>IRestfulClientFactory</code>
compress (advanced)	Compresses outgoing (POST/PUT) contents to the GZIP format.	false	boolean
connectionTimeout (advanced)	How long to try and establish the initial TCP connection (in ms).	10000	Integer

Name	Description	Default	Type
deferModelScanning (advanced)	When this option is set, model classes will not be scanned for children until the child list for the given type is actually accessed.	false	boolean
fhirContext (advanced)	FhirContext is an expensive object to create. To avoid creating multiple instances, it can be set directly.		FhirContext
forceConformanceCheck (advanced)	Force conformance check.	false	boolean
sessionCookie (advanced)	HTTP session cookie to add to every request.		String
socketTimeout (advanced)	How long to block for individual read/write operations (in ms).	10000	Integer
summary (advanced)	Request that the server modify the response using the <code>_summary</code> param. Enum values: ● COUNT ● TEXT ● DATA ● TRUE ● FALSE		String
validationMode (advanced)	When should Camel validate the FHIR Server's conformance statement. Enum values: ● NEVER ● ONCE	ONCE	String
proxyHost (proxy)	The proxy host.		String
proxyPassword (proxy)	The proxy password.		String
proxyPort (proxy)	The proxy port.		Integer

Name	Description	Default	Type
proxyUser (proxy)	The proxy username.		String
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel

Name	Description	Default	Type
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
accessToken (security)	OAuth access token.		String
password (security)	Username to use for basic authentication.		String
username (security)	Username to use for basic authentication.		String

36.6. API PARAMETERS (13 APIS)

The @FHIR endpoint is an API based component and has additional parameters based on which API name and API method is used. The API name and API method is located in the endpoint URI as the **apiName/methodName** path parameters:

```
fhir:apiName/methodName
```

There are 13 API names as listed in the table below:

API Name	Type	Description
capabilities	Both	API to Fetch the capability statement for the server
create	Both	API for the create operation, which creates a new resource instance on the server
delete	Both	API for the delete operation, which performs a logical delete on a server resource
history	Both	API for the history method
load-page	Both	API that Loads the previous/next bundle of resources from a paged set, using the link specified in the link type=next tag within the atom bundle
meta	Both	API for the meta operations, which can be used to get, add and remove tags and other Meta elements from a resource or across the server
operation	Both	API for extended FHIR operations
patch	Both	API for the patch operation, which performs a logical patch on a server resource
read	Both	API method for read operations
search	Both	API to search for resources matching a given set of criteria
transaction	Both	API for sending a transaction (collection of resources) to the server to be executed as a single unit
update	Both	API for the update operation, which performs a logical delete on a server resource
validate	Both	API for validating resources

Each API is documented in the following sections to come.

36.6.1. API: capabilities

Both producer and consumer are supported

The capabilities API is defined in the syntax as follows:

```
fhir:capabilities/methodName?[parameters]
```

The method is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

Method	Description
ofType	Retrieve the conformance statement using the given model type

36.6.1.1. Method ofType

Signatures:

- ```
org.hl7.fhir.instance.model.api.IBaseConformance
ofType(Class<org.hl7.fhir.instance.model.api.IBaseConformance> type,
java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);
```

The fhir/ofType API method has the parameters listed in the table below:

| Parameter              | Description                                                                       | Type  |
|------------------------|-----------------------------------------------------------------------------------|-------|
| <b>extraParameters</b> | See ExtraParameters for a full list of parameters that can be passed, may be NULL | Map   |
| <b>type</b>            | The model type                                                                    | Class |

In addition to the parameters above, the fhir API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

### 36.6.2. API: create

#### Both producer and consumer are supported

The create API is defined in the syntax as follows:

```
fhir:create/methodName?[parameters]
```

The 1 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

| Method                   | Description                           |
|--------------------------|---------------------------------------|
| <a href="#">resource</a> | Creates a IBaseResource on the server |

### 36.6.2.1. Method resource

Signatures:

- `ca.uhn.fhir.rest.api.MethodOutcome resource(String resourceAsString, String url, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `ca.uhn.fhir.rest.api.MethodOutcome resource(org.hl7.fhir.instance.model.api.IBaseResource resource, String url, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/resource API method has the parameters listed in the table below:

| Parameter               | Description                                                                                                                                                                                                                                                                         | Type             |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>extraParameters</b>  | See ExtraParameters for a full list of parameters that can be passed, may be NULL                                                                                                                                                                                                   | Map              |
| <b>preferReturn</b>     | Add a Prefer header to the request, which requests that the server include or suppress the resource body as a part of the result. If a resource is returned by the server it will be parsed and accessible to the client via <code>MethodOutcome#getResource()</code> , may be null | PreferReturnEnum |
| <b>resource</b>         | The resource to create                                                                                                                                                                                                                                                              | IBaseResource    |
| <b>resourceAsString</b> | The resource to create                                                                                                                                                                                                                                                              | String           |
| <b>url</b>              | The search URL to use. The format of this URL should be of the form <code>ResourceTypeParameters</code> , for example:<br><code>Patientname=Smith&amp;identifier=13.2.4.11.4%7C847366</code> , may be null                                                                          | String           |

In addition to the parameters above, the fhir API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

### 36.6.3. API: delete

Both producer and consumer are supported

The delete API is defined in the syntax as follows:

```
fhir:delete/methodName?[parameters]
```

The 3 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

| Method                                   | Description                                                                                      |
|------------------------------------------|--------------------------------------------------------------------------------------------------|
| <a href="#">resource</a>                 | Deletes the given resource                                                                       |
| <a href="#">resourceById</a>             | Deletes the resource by resource type e                                                          |
| <a href="#">resourceConditionalByUrl</a> | Specifies that the delete should be performed as a conditional delete against a given search URL |

### 36.6.3.1. Method resource

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseOperationOutcome resource(org.hl7.fhir.instance.model.api.IBaseResource resource, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/resource` API method has the parameters listed in the table below:

| Parameter              | Description                                                                                    | Type                       |
|------------------------|------------------------------------------------------------------------------------------------|----------------------------|
| <b>extraParameters</b> | See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be NULL | Map                        |
| <b>resource</b>        | The <code>IBaseResource</code> to delete                                                       | <code>IBaseResource</code> |

### 36.6.3.2. Method resourceById

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseOperationOutcome resourceById(String type, String stringId, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `org.hl7.fhir.instance.model.api.IBaseOperationOutcome resourceById(org.hl7.fhir.instance.model.api.IIdType id, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/resourceById` API method has the parameters listed in the table below:

| Parameter              | Description                                                                       | Type    |
|------------------------|-----------------------------------------------------------------------------------|---------|
| <b>extraParameters</b> | See ExtraParameters for a full list of parameters that can be passed, may be NULL | Map     |
| <b>id</b>              | The IIdType referencing the resource                                              | IIdType |
| <b>stringId</b>        | It's id                                                                           | String  |
| <b>type</b>            | The resource type e.g Patient                                                     | String  |

### 36.6.3.3. Method resourceConditionalByUrl

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseOperationOutcome resourceConditionalByUrl(String url, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/resourceConditionalByUrl` API method has the parameters listed in the table below:

| Parameter              | Description                                                                                                                                                                                  | Type   |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <b>extraParameters</b> | See ExtraParameters for a full list of parameters that can be passed, may be NULL                                                                                                            | Map    |
| <b>url</b>             | The search URL to use. The format of this URL should be of the form <code>ResourceTypeParameters</code> , for example:<br><code>Patientname=Smith&amp;identifier=13.2.4.11.4%7C847366</code> | String |

In addition to the parameters above, the `fhir` API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

### 36.6.4. API: history

**Both producer and consumer are supported**

The history API is defined in the syntax as follows:

```
fhir:history/methodName?[parameters]
```

The 3 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

| Method                     | Description                                                                                     |
|----------------------------|-------------------------------------------------------------------------------------------------|
| <a href="#">onInstance</a> | Perform the operation across all versions of a specific resource (by ID and type) on the server |
| <a href="#">onServer</a>   | Perform the operation across all versions of all resources of all types on the server           |
| <a href="#">onType</a>     | Perform the operation across all versions of all resources of the given type on the server      |

### 36.6.4.1. Method onInstance

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseBundle onInstance(org.hl7.fhir.instance.model.api.IIdType id, Class<org.hl7.fhir.instance.model.api.IBaseBundle> returnType, Integer count, java.util.Date cutoff, org.hl7.fhir.instance.model.api.IPrimitiveType<java.util.Date> iCutoff, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/onInstance` API method has the parameters listed in the table below:

| Parameter              | Description                                                                                                                                                            | Type           |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>count</b>           | Request that the server return only up to theCount number of resources, may be NULL                                                                                    | Integer        |
| <b>cutoff</b>          | Request that the server return only resource versions that were created at or after the given time (inclusive), may be NULL                                            | Date           |
| <b>extraParameters</b> | See ExtraParameters for a full list of parameters that can be passed, may be NULL                                                                                      | Map            |
| <b>iCutoff</b>         | Request that the server return only resource versions that were created at or after the given time (inclusive), may be NULL                                            | IPrimitiveType |
| <b>id</b>              | The IIdType which must be populated with both a resource type and a resource ID at                                                                                     | IIdType        |
| <b>returnType</b>      | Request that the method return a Bundle resource (such as <code>ca.uhn.fhir.model.dstu2.resource.Bundle</code> ). Use this method if you are accessing a DSTU2 server. | Class          |

### 36.6.4.2. Method onServer

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseBundle`

```
onServer(Class<org.hl7.fhir.instance.model.api.IBaseBundle> returnType, Integer count,
java.util.Date cutoff, org.hl7.fhir.instance.model.api.IPrimitiveType<java.util.Date> iCutoff,
java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);
```

The `fhir/onServer` API method has the parameters listed in the table below:

| Parameter              | Description                                                                                                                                                            | Type           |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>count</b>           | Request that the server return only up to theCount number of resources, may be NULL                                                                                    | Integer        |
| <b>cutoff</b>          | Request that the server return only resource versions that were created at or after the given time (inclusive), may be NULL                                            | Date           |
| <b>extraParameters</b> | See ExtraParameters for a full list of parameters that can be passed, may be NULL                                                                                      | Map            |
| <b>iCutoff</b>         | Request that the server return only resource versions that were created at or after the given time (inclusive), may be NULL                                            | IPrimitiveType |
| <b>returnType</b>      | Request that the method return a Bundle resource (such as <code>ca.uhn.fhir.model.dstu2.resource.Bundle</code> ). Use this method if you are accessing a DSTU2 server. | Class          |

### 36.6.4.3. Method onType

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseBundle`  
`onType(Class<org.hl7.fhir.instance.model.api.IBaseResource> resourceType, Class<org.hl7.fhir.instance.model.api.IBaseBundle> returnType, Integer count, java.util.Date cutoff, org.hl7.fhir.instance.model.api.IPrimitiveType<java.util.Date> iCutoff, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/onType` API method has the parameters listed in the table below:

| Parameter              | Description                                                                                                                 | Type    |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------|
| <b>count</b>           | Request that the server return only up to theCount number of resources, may be NULL                                         | Integer |
| <b>cutoff</b>          | Request that the server return only resource versions that were created at or after the given time (inclusive), may be NULL | Date    |
| <b>extraParameters</b> | See ExtraParameters for a full list of parameters that can be passed, may be NULL                                           | Map     |



| Parameter           | Description                                                                                                                                                            | Type           |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>iCutoff</b>      | Request that the server return only resource versions that were created at or after the given time (inclusive), may be NULL                                            | IPrimitiveType |
| <b>resourceType</b> | The resource type to search for                                                                                                                                        | Class          |
| <b>returnType</b>   | Request that the method return a Bundle resource (such as <code>ca.uhn.fhir.model.dstu2.resource.Bundle</code> ). Use this method if you are accessing a DSTU2 server. | Class          |

In addition to the parameters above, the fhir API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

### 36.6.5. API: load-page

**Both producer and consumer are supported**

The load-page API is defined in the syntax as follows:

```
fhir:load-page/methodName?[parameters]
```

The 3 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

| Method                   | Description                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------|
| <a href="#">byUrl</a>    | Load a page of results using the given URL and bundle type and return a DSTU1 Atom bundle |
| <a href="#">next</a>     | Load the next page of results using the link with relation next in the bundle             |
| <a href="#">previous</a> | Load the previous page of results using the link with relation prev in the bundle         |

#### 36.6.5.1. Method byUrl

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseBundle byUrl(String url, Class<org.hl7.fhir.instance.model.api.IBaseBundle> returnType, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/byUrl` API method has the parameters listed in the table below:

| Parameter                    | Description                                                                                    | Type   |
|------------------------------|------------------------------------------------------------------------------------------------|--------|
| <code>extraParameters</code> | See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be NULL | Map    |
| <code>returnType</code>      | The return type                                                                                | Class  |
| <code>url</code>             | The search url                                                                                 | String |

### 36.6.5.2. Method next

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseBundle next(org.hl7.fhir.instance.model.api.IBaseBundle bundle, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/next` API method has the parameters listed in the table below:

| Parameter                    | Description                                                                                    | Type                     |
|------------------------------|------------------------------------------------------------------------------------------------|--------------------------|
| <code>bundle</code>          | The <code>IBaseBundle</code>                                                                   | <code>IBaseBundle</code> |
| <code>extraParameters</code> | See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be NULL | Map                      |

### 36.6.5.3. Method previous

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseBundle previous(org.hl7.fhir.instance.model.api.IBaseBundle bundle, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/previous` API method has the parameters listed in the table below:

| Parameter                    | Description                                                                                    | Type                     |
|------------------------------|------------------------------------------------------------------------------------------------|--------------------------|
| <code>bundle</code>          | The <code>IBaseBundle</code>                                                                   | <code>IBaseBundle</code> |
| <code>extraParameters</code> | See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be NULL | Map                      |

In addition to the parameters above, the `fhir` API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter

overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

### 36.6.6. API: meta

#### Both producer and consumer are supported

The meta API is defined in the syntax as follows:

```
fhir:meta/methodName?[parameters]
```

The 5 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

| Method                          | Description                                                                            |
|---------------------------------|----------------------------------------------------------------------------------------|
| <a href="#">add</a>             | Add the elements in the given metadata to the already existing set (do not remove any) |
| <a href="#">delete</a>          | Delete the elements in the given metadata from the given id                            |
| <a href="#">getFromResource</a> | Fetch the current metadata from a specific resource                                    |
| <a href="#">getFromServer</a>   | Fetch the current metadata from the whole Server                                       |
| <a href="#">getFromType</a>     | Fetch the current metadata from a specific type                                        |

#### 36.6.6.1. Method add

Signatures:

- ```
org.hl7.fhir.instance.model.api.IBaseMetaType
add(org.hl7.fhir.instance.model.api.IBaseMetaType meta, org.hl7.fhir.instance.model.api.IIdType
id, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object>
extraParameters);
```

The fhir/add API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
id	The id	IIdType
meta	The IBaseMetaType class	IBaseMetaType

36.6.6.2. Method delete

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseMetaType`
`delete(org.hl7.fhir.instance.model.api.IBaseMetaType meta,`
`org.hl7.fhir.instance.model.api.IIdType id,`
`java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/delete API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
id	The id	IIdType
meta	The IBaseMetaType class	IBaseMetaType

36.6.6.3. Method getFromResource

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseMetaType`
`getFromResource(Class<org.hl7.fhir.instance.model.api.IBaseMetaType> metaType,`
`org.hl7.fhir.instance.model.api.IIdType id,`
`java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/getFromResource API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
id	The id	IIdType
metaType	The IBaseMetaType class	Class

36.6.6.4. Method getFromServer

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseMetaType`
`getFromServer(Class<org.hl7.fhir.instance.model.api.IBaseMetaType> metaType,`
`java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/getFromServer API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
metaType	The type of the meta datatype for the given FHIR model version (should be MetaDt.class or MetaType.class)	Class

36.6.6.5. Method getFromType

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseMetaType
getFromType(Class<org.hl7.fhir.instance.model.api.IBaseMetaType> metaType, String
resourceType, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object>
extraParameters);`

The `fhir/getFromType` API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
metaType	The IBaseMetaType class	Class
resourceType	The resource type e.g Patient	String

In addition to the parameters above, the `fhir` API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

36.6.7. API: operation

Both producer and consumer are supported

The operation API is defined in the syntax as follows:

```
fhir:operation/methodName?[parameters]
```

The 5 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

Method	Description
onInstance	Perform the operation across all versions of a specific resource (by ID and type) on the server
onInstanceVersion	This operation operates on a specific version of a resource
onServer	Perform the operation across all versions of all resources of all types on the server
onType	Perform the operation across all versions of all resources of the given type on the server
processMessage	This operation is called \$process-message as defined by the FHIR specification

36.6.7.1. Method onInstance

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseResource onInstance(org.hl7.fhir.instance.model.api.IIdType id, String name, org.hl7.fhir.instance.model.api.IBaseParameters parameters, Class<org.hl7.fhir.instance.model.api.IBaseParameters> outputParameterType, boolean useHttpGet, Class<org.hl7.fhir.instance.model.api.IBaseResource> returnType, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/onInstance` API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be NULL	Map
id	Resource (version will be stripped)	IIdType
name	Operation name	String
outputParameterType	The type to use for the output parameters (this should be set to <code>Parameters.class</code> drawn from the version of the FHIR structures you are using), may be NULL	Class
parameters	The parameters to use as input. May also be null if the operation does not require any input parameters.	IBaseParameters

Parameter	Description	Type
returnType	If this operation returns a single resource body as its return type instead of a Parameters resource, use this method to specify that resource type. This is useful for certain operations (e.g. Patient/NNN/\$everything) which return a bundle instead of a Parameters resource, may be NULL	Class
useHttpGet	Use HTTP GET verb	Boolean

36.6.7.2. Method onInstanceVersion

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseResource
onInstanceVersion(org.hl7.fhir.instance.model.api.IIdType id, String name,
org.hl7.fhir.instance.model.api.IBaseParameters parameters,
Class<org.hl7.fhir.instance.model.api.IBaseParameters> outputParameterType, boolean
useHttpGet, Class<org.hl7.fhir.instance.model.api.IBaseResource> returnType,
java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/onInstanceVersion API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
id	Resource version	IIdType
name	Operation name	String
outputParameterType	The type to use for the output parameters (this should be set to Parameters.class drawn from the version of the FHIR structures you are using), may be NULL	Class
parameters	The parameters to use as input. May also be null if the operation does not require any input parameters.	IBaseParameters
returnType	If this operation returns a single resource body as its return type instead of a Parameters resource, use this method to specify that resource type. This is useful for certain operations (e.g. Patient/NNN/\$everything) which return a bundle instead of a Parameters resource, may be NULL	Class
useHttpGet	Use HTTP GET verb	Boolean

36.6.7.3. Method onServer

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseResource onServer(String name, org.hl7.fhir.instance.model.api.IBaseParameters parameters, Class<org.hl7.fhir.instance.model.api.IBaseParameters> outputParameterType, boolean useHttpGet, Class<org.hl7.fhir.instance.model.api.IBaseResource> returnType, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/onServer API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
name	Operation name	String
outputParameterType	The type to use for the output parameters (this should be set to Parameters.class drawn from the version of the FHIR structures you are using), may be NULL	Class
parameters	The parameters to use as input. May also be null if the operation does not require any input parameters.	IBaseParameters
returnType	If this operation returns a single resource body as its return type instead of a Parameters resource, use this method to specify that resource type. This is useful for certain operations (e.g. Patient/NNN/\$everything) which return a bundle instead of a Parameters resource, may be NULL	Class
useHttpGet	Use HTTP GET verb	Boolean

36.6.7.4. Method onType

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseResource onType(Class<org.hl7.fhir.instance.model.api.IBaseResource> resourceType, String name, org.hl7.fhir.instance.model.api.IBaseParameters parameters, Class<org.hl7.fhir.instance.model.api.IBaseParameters> outputParameterType, boolean useHttpGet, Class<org.hl7.fhir.instance.model.api.IBaseResource> returnType, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/onType API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
name	Operation name	String
outputParameterType	The type to use for the output parameters (this should be set to Parameters.class drawn from the version of the FHIR structures you are using), may be NULL	Class
parameters	The parameters to use as input. May also be null if the operation does not require any input parameters.	IBaseParameters
resourceType	The resource type to operate on	Class
returnType	If this operation returns a single resource body as its return type instead of a Parameters resource, use this method to specify that resource type. This is useful for certain operations (e.g. Patient/NNN/\$everything) which return a bundle instead of a Parameters resource, may be NULL	Class
useHttpGet	Use HTTP GET verb	Boolean

36.6.7.5. Method processMessage

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseBundle processMessage(String respondToUri, org.hl7.fhir.instance.model.api.IBaseBundle msgBundle, boolean asynchronous, Class<org.hl7.fhir.instance.model.api.IBaseBundle> responseClass, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/processMessage` API method has the parameters listed in the table below:

Parameter	Description	Type
asynchronous	Whether to process the message asynchronously or synchronously, defaults to synchronous.	Boolean
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
msgBundle	Set the Message Bundle to POST to the messaging server	IBaseBundle

Parameter	Description	Type
respondToUri	An optional query parameter indicating that responses from the receiving server should be sent to this URI, may be NULL	String
responseClass	The response class	Class

In addition to the parameters above, the fhir API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

36.6.8. API: patch

Both producer and consumer are supported

The patch API is defined in the syntax as follows:

```
fhir:patch/methodName?[parameters]
```

The 2 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

Method	Description
patchById	Applies the patch to the given resource ID
patchByUrl	Specifies that the update should be performed as a conditional create against a given search URL

36.6.8.1. Method patchById

Signatures:

- `ca.uhn.fhir.rest.api.MethodOutcome patchById(String patchBody, String stringId, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `ca.uhn.fhir.rest.api.MethodOutcome patchById(String patchBody, org.hl7.fhir.instance.model.api.IIdType id, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/patchById API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
id	The resource ID to patch	IdType
patchBody	The body of the patch document serialized in either XML or JSON which conforms to	String
preferReturn	Add a Prefer header to the request, which requests that the server include or suppress the resource body as a part of the result. If a resource is returned by the server it will be parsed and accessible to the client via <code>MethodOutcome#getResource()</code>	PreferReturnEnum
stringId	The resource ID to patch	String

36.6.8.2. Method patchByUrl

Signatures:

- `ca.uhn.fhir.rest.api.MethodOutcome patchByUrl(String patchBody, String url, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/patchByUrl` API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
patchBody	The body of the patch document serialized in either XML or JSON which conforms to	String
preferReturn	Add a Prefer header to the request, which requests that the server include or suppress the resource body as a part of the result. If a resource is returned by the server it will be parsed and accessible to the client via <code>MethodOutcome#getResource()</code>	PreferReturnEnum
url	The search URL to use. The format of this URL should be of the form <code>ResourceTypeParameters</code> , for example: <code>Patientname=Smith&identifier=13.2.4.11.4%7C847366</code>	String

In addition to the parameters above, the `fhir` API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

36.6.9. API: read

Both producer and consumer are supported

The read API is defined in the syntax as follows:

```
fhir:read/methodName?[parameters]
```

The 2 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

Method	Description
resourceById	Reads a IBaseResource on the server by id
resourceByUrl	Reads a IBaseResource on the server by url

36.6.9.1. Method resourceById

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseResource resourceById(Class<org.hl7.fhir.instance.model.api.IBaseResource> resource, Long longId, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `org.hl7.fhir.instance.model.api.IBaseResource resourceById(Class<org.hl7.fhir.instance.model.api.IBaseResource> resource, String stringId, String version, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `org.hl7.fhir.instance.model.api.IBaseResource resourceById(Class<org.hl7.fhir.instance.model.api.IBaseResource> resource, org.hl7.fhir.instance.model.api.IIdType id, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `org.hl7.fhir.instance.model.api.IBaseResource resourceById(String resourceClass, Long longId, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `org.hl7.fhir.instance.model.api.IBaseResource resourceById(String resourceClass, String stringId, String ifVersionMatches, String version, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

- `org.hl7.fhir.instance.model.api.IBaseResource resourceById(String resourceClass, org.hl7.fhir.instance.model.api.IIdType id, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/resourceById` API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be <code>NULL</code>	Map
id	The <code>IIdType</code> referencing the resource	<code>IIdType</code>
ifVersionMatches	A version to match against the newest version on the server	String
longId	The resource ID	Long
resource	The resource to read (e.g. Patient)	Class
resourceClass	The resource to read (e.g. Patient)	String
returnNull	Return null if version matches	Boolean
returnResource	Return the resource if version matches	<code>IBaseResource</code>
stringId	The resource ID	String
throwError	Throw error if the version matches	Boolean
version	The resource version	String

36.6.9.2. Method `resourceByUrl`

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseResource resourceByUrl(Class<org.hl7.fhir.instance.model.api.IBaseResource> resource, String url, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `org.hl7.fhir.instance.model.api.IBaseResource resourceByUrl(Class<org.hl7.fhir.instance.model.api.IBaseResource> resource, org.hl7.fhir.instance.model.api.IIdType iUrl, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `org.hl7.fhir.instance.model.api.IBaseResource resourceByUrl(String resourceClass, String url, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource`

```
returnResource, Boolean throwError,
java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);
```

- `org.hl7.fhir.instance.model.api.IBaseResource resourceByUrl(String resourceClass, org.hl7.fhir.instance.model.api.IIdType iUrl, String ifVersionMatches, Boolean returnNull, org.hl7.fhir.instance.model.api.IBaseResource returnResource, Boolean throwError, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/resourceByUrl` API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be <code>NULL</code>	Map
iUrl	The <code>IIdType</code> referencing the resource by absolute url	<code>IIdType</code>
ifVersionMatches	A version to match against the newest version on the server	String
resource	The resource to read (e.g. Patient)	Class
resourceClass	The resource to read (e.g. Patient.class)	String
returnNull	Return null if version matches	Boolean
returnResource	Return the resource if version matches	<code>IBaseResource</code>
throwError	Throw error if the version matches	Boolean
url	Referencing the resource by absolute url	String

In addition to the parameters above, the `fhir` API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

36.6.10. API: search

Both producer and consumer are supported

The search API is defined in the syntax as follows:

```
fhir:search/methodName?[parameters]
```

The 1 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

Method	Description
searchByUrl	Perform a search directly by URL

36.6.10.1. Method `searchByUrl`

Signatures:

- `org.hl7.fhir.instance.model.api.IBaseBundle searchByUrl(String url, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/searchByUrl` API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be NULL	Map
url	The URL to search for. Note that this URL may be complete (e.g.) in which case the client's base URL will be ignored. Or it can be relative (e.g. <code>Patientname=foo</code>) in which case the client's base URL will be used.	String

In addition to the parameters above, the `fhir` API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

36.6.11. API: transaction

Both producer and consumer are supported

The transaction API is defined in the syntax as follows:

```
fhir:transaction/methodName?[parameters]
```

The 2 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

Method	Description
withBundle	Use the given raw text (should be a Bundle resource) as the transaction input
withResources	Use a list of resources as the transaction input

36.6.11.1. Method withBundle

Signatures:

- `String withBundle(String stringBundle, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `org.hl7.fhir.instance.model.api.IBaseBundle withBundle(org.hl7.fhir.instance.model.api.IBaseBundle bundle, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/withBundle API method has the parameters listed in the table below:

Parameter	Description	Type
bundle	Bundle to use in the transaction	IBaseBundle
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
stringBundle	Bundle to use in the transaction	String

36.6.11.2. Method withResources

Signatures:

- `java.util.List<org.hl7.fhir.instance.model.api.IBaseResource> withResources(java.util.List<org.hl7.fhir.instance.model.api.IBaseResource> resources, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/withResources API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
resources	Resources to use in the transaction	List

In addition to the parameters above, the fhir API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

36.6.12. API: update

Both producer and consumer are supported

The update API is defined in the syntax as follows:

fhir:update/methodName?[parameters]

The 2 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

Method	Description
resource	Updates a IBaseResource on the server by id
resourceByUrl	Updates a IBaseResource on the server by search url

36.6.12.1. Method resource

Signatures:

- `ca.uhn.fhir.rest.api.MethodOutcome resource(String resourceAsString, String stringId, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `ca.uhn.fhir.rest.api.MethodOutcome resource(String resourceAsString, org.hl7.fhir.instance.model.api.IIdType id, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `ca.uhn.fhir.rest.api.MethodOutcome resource(org.hl7.fhir.instance.model.api.IBaseResource resource, String stringId, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `ca.uhn.fhir.rest.api.MethodOutcome resource(org.hl7.fhir.instance.model.api.IBaseResource resource, org.hl7.fhir.instance.model.api.IIdType id, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The fhir/resource API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See ExtraParameters for a full list of parameters that can be passed, may be NULL	Map
id	The IIdType referencing the resource	IIdType
preferReturn	Whether the server include or suppress the resource body as a part of the result	PreferReturnEnum
resource	The resource to update (e.g. Patient)	IBaseResource
resourceAsString	The resource body to update	String
stringId	The ID referencing the resource	String

36.6.12.2. Method resourceBySearchUrl

Signatures:

- `ca.uhn.fhir.rest.api.MethodOutcome resourceBySearchUrl(String resourceAsString, String url, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `ca.uhn.fhir.rest.api.MethodOutcome resourceBySearchUrl(org.hl7.fhir.instance.model.api.IBaseResource resource, String url, ca.uhn.fhir.rest.api.PreferReturnEnum preferReturn, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/resourceBySearchUrl` API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be NULL	Map
preferReturn	Whether the server include or suppress the resource body as a part of the result	PreferReturnEnum
resource	The resource to update (e.g. Patient)	IBaseResource
resourceAsString	The resource body to update	String
url	Specifies that the update should be performed as a conditional create against a given search URL	String

In addition to the parameters above, the `fhir` API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

36.6.13. API: validate

Both producer and consumer are supported

The `validate` API is defined in the syntax as follows:

```
fhir:validate/methodName?[parameters]
```

The 1 method(s) is listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand alias name which can be used in the syntax instead of the name)

Method	Description
resource	Validates the resource

36.6.13.1. Method resource

Signatures:

- `ca.uhn.fhir.rest.api.MethodOutcome resource(String resourceAsString, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`
- `ca.uhn.fhir.rest.api.MethodOutcome resource(org.hl7.fhir.instance.model.api.IBaseResource resource, java.util.Map<org.apache.camel.component.fhir.api.ExtraParameters, Object> extraParameters);`

The `fhir/resource` API method has the parameters listed in the table below:

Parameter	Description	Type
extraParameters	See <code>ExtraParameters</code> for a full list of parameters that can be passed, may be NULL	Map
resource	The <code>IBaseResource</code> to validate	<code>IBaseResource</code>
resourceAsString	Raw resource to validate	String

In addition to the parameters above, the `fhir` API can also use any of the [Query Parameters](#).

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelFhir.parameter**. The **inBody** parameter overrides message header, i.e. the endpoint parameter **inBody=myParameterNameHere** would override a **CamelFhir.myParameterNameHere** header.

36.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 56 options, which are listed below.

Name	Description	Default	Type
camel.component.fhir.access-token	OAuth access token.		String
camel.component.fhir.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.fhir.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.fhir.client	To use the custom client. The option is a <code>ca.uhn.fhir.rest.client.api.IGenericClient</code> type.		IGenericClient
camel.component.fhir.client-factory	To use the custom client factory. The option is a <code>ca.uhn.fhir.rest.client.api.IRestfulClientFactory</code> type.		IRestfulClientFactory
camel.component.fhir.compress	Compresses outgoing (POST/PUT) contents to the GZIP format.	false	Boolean
camel.component.fhir.configuration	To use the shared configuration. The option is a <code>org.apache.camel.component.fhir.FhirConfiguration</code> type.		FhirConfiguration
camel.component.fhir.connection-timeout	How long to try and establish the initial TCP connection (in ms).	10000	Integer
camel.component.fhir.defer-model-scanning	When this option is set, model classes will not be scanned for children until the child list for the given type is actually accessed.	false	Boolean
camel.component.fhir.enabled	Whether to enable auto configuration of the fhir component. This is enabled by default.		Boolean
camel.component.fhir.encoding	Encoding to use for all request.		String
camel.component.fhir.fhir-context	FhirContext is an expensive object to create. To avoid creating multiple instances, it can be set directly. The option is a <code>ca.uhn.fhir.context.FhirContext</code> type.		FhirContext
camel.component.fhir.fhir-version	The FHIR Version to use.	R4	String

Name	Description	Default	Type
<code>camel.component.fhir.force-conformance-check</code>	Force conformance check.	false	Boolean
<code>camel.component.fhir.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.fhir.log</code>	Will log every requests and responses.	false	Boolean
<code>camel.component.fhir.password</code>	Username to use for basic authentication.		String
<code>camel.component.fhir.pretty-print</code>	Pretty print all request.	false	Boolean
<code>camel.component.fhir.proxy-host</code>	The proxy host.		String
<code>camel.component.fhir.proxy-password</code>	The proxy password.		String
<code>camel.component.fhir.proxy-port</code>	The proxy port.		Integer
<code>camel.component.fhir.proxy-user</code>	The proxy username.		String
<code>camel.component.fhir.server-url</code>	The FHIR server base URL.		String
<code>camel.component.fhir.session-cookie</code>	HTTP session cookie to add to every request.		String

Name	Description	Default	Type
camel.component.fhir.socket-timeout	How long to block for individual read/write operations (in ms).	10000	Integer
camel.component.fhir.summary	Request that the server modify the response using the <code>_summary</code> param.		String
camel.component.fhir.username	Username to use for basic authentication.		String
camel.component.fhir.validation-mode	When should Camel validate the FHIR Server's conformance statement.	ONCE	String
camel.dataformat.fhirjson.content-type-header	Whether the data format should set the Content-Type header with the type from the data format. For example <code>application/xml</code> for data formats marshalling to XML, or <code>application/json</code> for data formats marshalling to JSON.	true	Boolean
camel.dataformat.fhirjson.dont-encode-elements	If provided, specifies the elements which should NOT be encoded. Valid values for this field would include: Patient - Don't encode patient and all its children Patient.name - Don't encode the patient's name Patient.name.family - Don't encode the patient's family name .text - Don't encode the text element on any resource (only the very first position may contain a wildcard) DSTU2 note: Note that values including meta, such as <code>Patient.meta</code> will work for DSTU2 parsers, but values with subelements on meta such as <code>Patient.meta.lastUpdated</code> will only work in DSTU3 mode.		Set
camel.dataformat.fhirjson.dont-strip-versions-from-references-at-paths	If supplied value(s), any resource references at the specified paths will have their resource versions encoded instead of being automatically stripped during the encoding process. This setting has no effect on the parsing process. This method provides a finer-grained level of control than <code>setStripVersionsFromReferences(String)</code> and any paths specified by this method will be encoded even if <code>setStripVersionsFromReferences(String)</code> has been set to true (which is the default).		List
camel.dataformat.fhirjson.enabled	Whether to enable auto configuration of the <code>fhirJson</code> data format. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.dataformat.fhirjson.encode-elements	If provided, specifies the elements which should be encoded, to the exclusion of all others. Valid values for this field would include: Patient - Encode patient and all its children Patient.name - Encode only the patient's name Patient.name.family - Encode only the patient's family name .text - Encode the text element on any resource (only the very first position may contain a wildcard) .(mandatory) - This is a special case which causes any mandatory fields (min 0) to be encoded.		Set
camel.dataformat.fhirjson.encode-elements-applies-to-child-resources-only	If set to true (default is false), the values supplied to setEncodeElements(Set) will not be applied to the root resource (typically a Bundle), but will be applied to any sub-resources contained within it (i.e. search result resources in that bundle).	false	Boolean
camel.dataformat.fhirjson.fhir-version	The version of FHIR to use. Possible values are: DSTU2,DSTU2_HL7ORG,DSTU2_1,DSTU3,R4.	DSTU3	String
camel.dataformat.fhirjson.omit-resource-id	If set to true (default is false) the ID of any resources being encoded will not be included in the output. Note that this does not apply to contained resources, only to root resources. In other words, if this is set to true, contained resources will still have local IDs but the outer/containing ID will not have an ID.	false	Boolean
camel.dataformat.fhirjson.override-resource-id-with-bundle-entry-full-url	If set to true (which is the default), the Bundle.entry.fullUrl will override the Bundle.entry.resource's resource id if the fullUrl is defined. This behavior happens when parsing the source data into a Bundle object. Set this to false if this is not the desired behavior (e.g. the client code wishes to perform additional validation checks between the fullUrl and the resource id).	false	Boolean
camel.dataformat.fhirjson.pretty-print	Sets the pretty print flag, meaning that the parser will encode resources with human-readable spacing and newlines between elements instead of condensing output as much as possible.	false	Boolean
camel.dataformat.fhirjson.server-base-url	Sets the server's base URL used by this parser. If a value is set, resource references will be turned into relative references if they are provided as absolute URLs but have a base matching the given base.		String

Name	Description	Default	Type
camel.dataformat.fhirjson.strip-versions-from-references	If set to true (which is the default), resource references containing a version will have the version removed when the resource is encoded. This is generally good behaviour because in most situations, references from one resource to another should be to the resource by ID, not by ID and version. In some cases though, it may be desirable to preserve the version in resource links. In that case, this value should be set to false. This method provides the ability to globally disable reference encoding. If finer-grained control is needed, use <code>setDontStripVersionsFromReferencesAtPath(List)</code> .	false	Boolean
camel.dataformat.fhirjson.summary-mode	If set to true (default is false) only elements marked by the FHIR specification as being summary elements will be included.	false	Boolean
camel.dataformat.fhirjson.suppress-narratives	If set to true (default is false), narratives will not be included in the encoded values.	false	Boolean
camel.dataformat.fhirxml.content-type-header	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.	true	Boolean
camel.dataformat.fhirxml.dont-encode-elements	If provided, specifies the elements which should NOT be encoded. Valid values for this field would include: Patient - Don't encode patient and all its children Patient.name - Don't encode the patient's name Patient.name.family - Don't encode the patient's family name .text - Don't encode the text element on any resource (only the very first position may contain a wildcard) DSTU2 note: Note that values including meta, such as Patient.meta will work for DSTU2 parsers, but values with subelements on meta such as Patient.meta.lastUpdated will only work in DSTU3 mode.		Set

Name	Description	Default	Type
<code>camel.dataformat.fhirxml.dont-strip-versions-from-references-at-paths</code>	If supplied value(s), any resource references at the specified paths will have their resource versions encoded instead of being automatically stripped during the encoding process. This setting has no effect on the parsing process. This method provides a finer-grained level of control than <code>setStripVersionsFromReferences(String)</code> and any paths specified by this method will be encoded even if <code>setStripVersionsFromReferences(String)</code> has been set to true (which is the default).		List
<code>camel.dataformat.fhirxml.enabled</code>	Whether to enable auto configuration of the fhirXml data format. This is enabled by default.		Boolean
<code>camel.dataformat.fhirxml.encode-elements</code>	If provided, specifies the elements which should be encoded, to the exclusion of all others. Valid values for this field would include: Patient - Encode patient and all its children Patient.name - Encode only the patient's name Patient.name.family - Encode only the patient's family name .text - Encode the text element on any resource (only the very first position may contain a wildcard) .(mandatory) - This is a special case which causes any mandatory fields (min 0) to be encoded.		Set
<code>camel.dataformat.fhirxml.encode-elements-applies-to-child-resources-only</code>	If set to true (default is false), the values supplied to <code>setEncodeElements(Set)</code> will not be applied to the root resource (typically a Bundle), but will be applied to any sub-resources contained within it (i.e. search result resources in that bundle).	false	Boolean
<code>camel.dataformat.fhirxml.fhir-version</code>	The version of FHIR to use. Possible values are: DSTU2,DSTU2_HL7ORG,DSTU2_1,DSTU3,R4.	DSTU3	String
<code>camel.dataformat.fhirxml.omit-resource-id</code>	If set to true (default is false) the ID of any resources being encoded will not be included in the output. Note that this does not apply to contained resources, only to root resources. In other words, if this is set to true, contained resources will still have local IDs but the outer/containing ID will not have an ID.	false	Boolean

Name	Description	Default	Type
<code>camel.dataformat.fhirxml.override-resource-id-with-bundle-entry-full-url</code>	If set to true (which is the default), the <code>Bundle.entry.fullUrl</code> will override the <code>Bundle.entry.resource</code> 's resource id if the <code>fullUrl</code> is defined. This behavior happens when parsing the source data into a <code>Bundle</code> object. Set this to false if this is not the desired behavior (e.g. the client code wishes to perform additional validation checks between the <code>fullUrl</code> and the resource id).	false	Boolean
<code>camel.dataformat.fhirxml.pretty-print</code>	Sets the pretty print flag, meaning that the parser will encode resources with human-readable spacing and newlines between elements instead of condensing output as much as possible.	false	Boolean
<code>camel.dataformat.fhirxml.server-base-url</code>	Sets the server's base URL used by this parser. If a value is set, resource references will be turned into relative references if they are provided as absolute URLs but have a base matching the given base.		String
<code>camel.dataformat.fhirxml.strip-versions-from-references</code>	If set to true (which is the default), resource references containing a version will have the version removed when the resource is encoded. This is generally good behaviour because in most situations, references from one resource to another should be to the resource by ID, not by ID and version. In some cases though, it may be desirable to preserve the version in resource links. In that case, this value should be set to false. This method provides the ability to globally disable reference encoding. If finer-grained control is needed, use <code>setDontStripVersionsFromReferencesAtPath(List)</code> .	false	Boolean
<code>camel.dataformat.fhirxml.summary-mode</code>	If set to true (default is false) only elements marked by the FHIR specification as being summary elements will be included.	false	Boolean
<code>camel.dataformat.fhirxml.suppress-narratives</code>	If set to true (default is false), narratives will not be included in the encoded values.	false	Boolean

CHAPTER 37. FILE

Both producer and consumer are supported

The File component provides access to file systems, allowing files to be processed by any other Camel Components or messages from other components to be saved to disk.

37.1. DEPENDENCIES

When using **file** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-file-starter</artifactId>
</dependency>
```

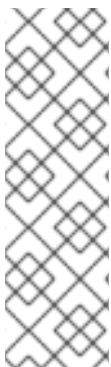
37.2. URI FORMAT

```
file:directoryName[?options]
```

Where **directoryName** represents the underlying file directory.

Only directories

Camel supports only endpoints configured with a starting directory. So the **directoryName** must be a directory. If you want to consume a single file only, you can use the **fileName** option, e.g. by setting **fileName=thefilename**. Also, the starting directory must not contain dynamic expressions with **\${ }** placeholders. Again use the **fileName** option to specify the dynamic part of the filename.



NOTE

Avoid reading files currently being written by another application

Beware the JDK File IO API is a bit limited in detecting whether another application is currently writing/copying a file. And the implementation can be different depending on OS platform as well. This could lead to that Camel thinks the file is not locked by another process and start consuming it. Therefore you have to do your own investigation what suits your environment. To help with this Camel provides different **readLock** options and **doneFileName** option that you can use. See also the section [Consuming files from folders where others drop files directly](#).

37.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

37.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the

endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

37.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

37.4. COMPONENT OPTIONS

The File component supports 3 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

37.5. ENDPOINT OPTIONS

The File endpoint is configured using URI syntax:

```
file:directoryName
```

with the following path and query parameters:

37.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
directoryName (common)	Required The starting directory.		File

37.5.2. Query Parameters (94 parameters)

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
charset (common)	This option is used to specify the encoding of the file. You can use this on the consumer, to specify the encodings of the files, which allow Camel to know the charset it should load the file content in case the file content is being accessed. Likewise when writing a file, you can use this option to specify which charset to write the file as well. Do mind that when writing the file Camel may have to read the message content into memory to be able to convert the data into the configured charset, so do not use this if you have big messages.		String
doneFileName (common)	Producer: If provided, then Camel will write a 2nd done file when the original file has been written. The done file will be empty. This option configures what file name to use. Either you can specify a fixed name. Or you can use dynamic placeholders. The done file will always be written in the same folder as the original file. Consumer: If provided, Camel will only consume files if a done file exists. This option configures what file name to use. Either you can specify a fixed name. Or you can use dynamic placeholders. The done file is always expected in the same folder as the original file. Only <code>\$\$\{file.name}</code> and <code>\$\$\{file.name.next}</code> is supported as dynamic placeholders.		String
fileName (common)	Use Expression such as File Language to dynamically set the filename. For consumers, it's used as a filename filter. For producers, it's used to evaluate the filename to write. If an expression is set, it take precedence over the CamelFileName header. (Note: The header itself can also be an Expression). The expression options support both String and Expression types. If the expression is a String type, it is always evaluated using the File Language. If the expression is an Expression type, the specified Expression type is used - this allows you, for instance, to use OGNL expressions. For the consumer, you can use it to filter filenames, so you can for instance consume today's file using the File Language syntax: <code>mydata-\$\$\{date:now:yyyyMMdd}.txt</code> . The producers support the CamelOverrideFileName header which takes precedence over any existing CamelFileName header; the CamelOverrideFileName is a header that is used only once, and makes it easier as this avoids to temporary store CamelFileName and have to restore it afterwards.		String

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
delete (consumer)	If true, the file will be deleted after it is processed successfully.	false	boolean
moveFailed (consumer)	Sets the move failure expression based on Simple language. For example, to move files into a .error subdirectory use: <code>.error</code> . Note: When moving the files to the fail location Camel will handle the error and will not pick up the file again.		String
noop (consumer)	If true, the file is not moved or deleted in any way. This option is good for readonly data, or for ETL type requirements. If <code>noop=true</code> , Camel will set <code>idempotent=true</code> as well, to avoid consuming the same files over and over again.	false	boolean
preMove (consumer)	Expression (such as File Language) used to dynamically set the filename when moving it before processing. For example to move in-progress files into the order directory set this value to <code>order</code> .		String
preSort (consumer)	When pre-sort is enabled then the consumer will sort the file and directory names during polling, that was retrieved from the file system. You may want to do this in case you need to operate on the files in a sorted order. The pre-sort is executed before the consumer starts to filter, and accept files to process by Camel. This option is default=false meaning disabled.	false	boolean
recursive (consumer)	If a directory, will look for files in all the sub-directories as well.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
directoryMustExist (consumer (advanced))	Similar to the <code>startingDirectoryMustExist</code> option but this applies during polling (after starting the consumer).	false	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
extendedAttributes (consumer (advanced))	To define which file attributes of interest. Like <code>posix:permissions, posix:owner, basic:lastAccessTime</code> , it supports basic wildcard like <code>posix:*, basic:lastAccessTime</code> .		<code>String</code>
inProgressRepository (consumer (advanced))	A pluggable in-progress repository <code>org.apache.camel.spi.IdempotentRepository</code> . The in-progress repository is used to account the current in progress files being consumed. By default a memory based repository is used.		<code>IdempotentRepository</code>
localWorkDirectory (consumer (advanced))	When consuming, a local work directory can be used to store the remote file content directly in local files, to avoid loading the content into memory. This is beneficial, if you consume a very big remote file and thus can conserve memory.		<code>String</code>
onCompletionExceptionHandler (consumer (advanced))	To use a custom <code>org.apache.camel.spi.ExceptionHandler</code> to handle any thrown exceptions that happens during the file on completion process where the consumer does either a commit or rollback. The default implementation will log any exception at <code>WARN</code> level and ignore.		<code>ExceptionHandler</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an <code>Exchange</code> have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>

Name	Description	Default	Type
probeContentType (consumer (advanced))	Whether to enable probing of the content type. If enable then the consumer uses <code>Files#probeContentType(java.nio.file.Path)</code> to determine the content-type of the file, and store that as a header with key <code>Exchange#FILE_CONTENT_TYPE</code> on the Message.	false	boolean
processStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.component.file.GenericFileProcessStrategy</code> allowing you to implement your own <code>readLock</code> option or similar. Can also be used when special conditions must be met before a file can be consumed, such as a special ready file exists. If this option is set then the <code>readLock</code> option does not apply.		<code>GenericFileProcessStrategy</code>
resumeStrategy (consumer (advanced))	Set a resume strategy for files. This makes it possible to define a strategy for resuming reading files after the last point before stopping the application. See the <code>FileConsumerResumeStrategy</code> for implementation details.		<code>FileConsumerResumeStrategy</code>
startingDirectoryMustExist (consumer (advanced))	Whether the starting directory must exist. Mind that the <code>autoCreate</code> option is default enabled, which means the starting directory is normally auto created if it doesn't exist. You can disable <code>autoCreate</code> and enable this to ensure the starting directory must exist. Will thrown an exception if the directory doesn't exist.	false	boolean
startingDirectoryMustHaveAccess (consumer (advanced))	Whether the starting directory has access permissions. Mind that the <code>startingDirectoryMustExist</code> parameter must be set to true in order to verify that the directory exists. Will thrown an exception if the directory doesn't have read and write permissions.	false	boolean
appendChars (producer)	Used to append characters (text) after writing files. This can for example be used to add new lines or other separators when writing and appending new files or existing files. To specify new-line (slash-n or slash-r) or tab (slash-t) characters then escape with an extra slash, eg slash-slash-n.		String

Name	Description	Default	Type
fileExist (producer)	What to do if a file already exists with the same name. Override, which is the default, replaces the existing file. - Append - adds content to the existing file. - Fail - throws a <code>GenericFileOperationException</code>, indicating that there is already an existing file. - Ignore - silently ignores the problem and does not override the existing file, but assumes everything is okay. - Move - option requires to use the <code>moveExisting</code> option to be configured as well. The option <code>eagerDeleteTargetFile</code> can be used to control what to do if an moving the file, and there exists already an existing file, otherwise causing the move operation to fail. The Move option will move any existing files, before writing the target file. - TryRename is only applicable if <code>tempFileName</code> option is in use. This allows to try renaming the file from the temporary name to the actual name, without doing any exists check. This check may be faster on some file systems and especially FTP servers. Enum values: ● Override ● Append ● Fail ● Ignore ● Move ● TryRename	Override	GenericFileExist
flatten (producer)	Flatten is used to flatten the file name path to strip any leading paths, so it's just the file name. This allows you to consume recursively into sub-directories, but when you eg write the files to another directory they will be written in a single directory. Setting this to true on the producer enforces that any file name in <code>CamelFileName</code> header will be stripped for any leading paths.	false	boolean
jailStartingDirectory (producer)	Used for jailing (restricting) writing files to the starting directory (and sub) only. This is enabled by default to not allow Camel to write files to outside directories (to be more secured out of the box). You can turn this off to allow writing files to directories outside the starting directory, such as parent or root folders.	true	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
moveExisting (producer)	Expression (such as File Language) used to compute file name to use when fileExist=Move is configured. To move files into a backup subdirectory just enter backup. This option only supports the following File Language tokens: file:name, file:name.ext, file:name.noext, file:onlyname, file:onlyname.noext, file:ext, and file:parent. Notice the file:parent is not supported by the FTP component, as the FTP component can only move any existing files to a relative directory based on current dir as base.		String
tempFileName (producer)	The same as tempPrefix option but offering a more fine grained control on the naming of the temporary filename as it uses the File Language. The location for tempFileName is relative to the final file location in the option 'fileName', not the target directory in the base uri. For example if option fileName includes a directory prefix: dir/finalFilename then tempFileName is relative to that subdirectory dir.		String
tempPrefix (producer)	This option is used to write the file using a temporary name and then, after the write is complete, rename it to the real name. Can be used to identify files being written and also avoid consumers (not using exclusive read locks) reading in progress files. Is often used by FTP when uploading big files.		String
allowNullBody (producer (advanced))	Used to specify if a null body is allowed during file writing. If set to true then an empty file will be created, when set to false, and attempting to send a null body to the file component, a GenericFileWriteException of 'Cannot write null body to file.' will be thrown. If the fileExist option is set to 'Override', then the file will be truncated, and if set to append the file will remain unchanged.	false	boolean

Name	Description	Default	Type
chmod (producer (advanced))	Specify the file permissions which is sent by the producer, the chmod value must be between 000 and 777; If there is a leading digit like in 0755 we will ignore it.		String
chmodDirectory (producer (advanced))	Specify the directory permissions used when the producer creates missing directories, the chmod value must be between 000 and 777; If there is a leading digit like in 0755 we will ignore it.		String
eagerDeleteTargetFile (producer (advanced))	Whether or not to eagerly delete any existing target file. This option only applies when you use fileExists=Override and the tempFileName option as well. You can use this to disable (set it to false) deleting the target file before the temp file is written. For example you may write big files and want the target file to exists during the temp file is being written. This ensure the target file is only deleted until the very last moment, just before the temp file is being renamed to the target filename. This option is also used to control whether to delete any existing files when fileExist=Move is enabled, and an existing file exists. If this option copyAndDeleteOnRenameFails false, then an exception will be thrown if an existing file existed, if its true, then the existing file is deleted before the move operation.	true	boolean
forceWrites (producer (advanced))	Whether to force syncing writes to the file system. You can turn this off if you do not want this level of guarantee, for example if writing to logs / audit logs etc; this would yield better performance.	true	boolean
keepLastModified (producer (advanced))	Will keep the last modified timestamp from the source file (if any). Will use the Exchange.FILE_LAST_MODIFIED header to located the timestamp. This header can contain either a java.util.Date or long with the timestamp. If the timestamp exists and the option is enabled it will set this timestamp on the written file. Note: This option only applies to the file producer. You cannot use this option with any of the ftp producers.	false	boolean
moveExistingFileStrategy (producer (advanced))	Strategy (Custom Strategy) used to move file with special naming token to use when fileExist=Move is configured. By default, there is an implementation used if no custom strategy is provided.		FileMoveExistingStrategy

Name	Description	Default	Type
autoCreate (advanced)	Automatically create missing directories in the file's pathname. For the file consumer, that means creating the starting directory. For the file producer, it means the directory the files should be written to.	true	boolean
bufferSize (advanced)	Buffer size in bytes used for writing files (or in case of FTP for downloading and uploading files).	131072	int
copyAndDeleteOnRenameFail (advanced)	Whether to fallback and do a copy and delete file, in case the file could not be renamed directly. This option is not available for the FTP component.	true	boolean
renameUsingCopy (advanced)	Perform rename operations using a copy and delete strategy. This is primarily used in environments where the regular rename operation is unreliable (e.g. across different file systems or networks). This option takes precedence over the <code>copyAndDeleteOnRenameFail</code> parameter that will automatically fall back to the copy and delete strategy, but only after additional delays.	false	boolean
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean
antExclude (filter)	Ant style filter exclusion. If both <code>antInclude</code> and <code>antExclude</code> are used, <code>antExclude</code> takes precedence over <code>antInclude</code> . Multiple exclusions may be specified in comma-delimited format.		String
antFilterCaseSensitive (filter)	Sets case sensitive flag on ant filter.	true	boolean
antInclude (filter)	Ant style filter inclusion. Multiple inclusions may be specified in comma-delimited format.		String
eagerMaxMessagesPerPoll (filter)	Allows for controlling whether the limit from <code>maxMessagesPerPoll</code> is eager or not. If eager then the limit is during the scanning of files. Where as false would scan all files, and then perform sorting. Setting this option to false allows for sorting all files first, and then limit the poll. Mind that this requires a higher memory usage as all file details are in memory to perform the sorting.	true	boolean

Name	Description	Default	Type
exclude (filter)	Is used to exclude files, if filename matches the regex pattern (matching is case in-sensitive). Notice if you use symbols such as plus sign and others you would need to configure this using the RAW() syntax if configuring this as an endpoint uri. See more details at configuring endpoint uris.		String
excludeExt (filter)	Is used to exclude files matching file extension name (case insensitive). For example to exclude bak files, then use excludeExt=bak. Multiple extensions can be separated by comma, for example to exclude bak and dat files, use excludeExt=bak,dat. Note that the file extension includes all parts, for example having a file named mydata.tar.gz will have extension as tar.gz. For more flexibility then use the include/exclude options.		String
filter (filter)	Pluggable filter as a org.apache.camel.component.file.GenericFileFilter class. Will skip files if filter returns false in its accept() method.		GenericFileFilter
filterDirectory (filter)	Filters the directory based on Simple language. For example to filter on current date, you can use a simple date pattern such as <code>\$\$\{date:now:yyyMMdd}</code> .		String
filterFile (filter)	Filters the file based on Simple language. For example to filter on file size, you can use <code>\$\$\{file:size} 5000</code> .		String
idempotent (filter)	Option to use the Idempotent Consumer EIP pattern to let Camel skip already processed files. Will by default use a memory based LRU Cache that holds 1000 entries. If noop=true then idempotent will be enabled as well to avoid consuming the same files over and over again.	false	Boolean
idempotentKey (filter)	To use a custom idempotent key. By default the absolute path of the file is used. You can use the File Language, for example to use the file name and file size, you can do: <code>idempotentKey=\$\$\{file:name}-\$\$\{file:size}</code> .		String
idempotentRepository (filter)	A pluggable repository org.apache.camel.spi.IdempotentRepository which by default use MemoryIdempotentRepository if none is specified and idempotent is true.		IdempotentRepository

Name	Description	Default	Type
include (filter)	Is used to include files, if filename matches the regex pattern (matching is case in-sensitive). Notice if you use symbols such as plus sign and others you would need to configure this using the RAW() syntax if configuring this as an endpoint uri. See more details at configuring endpoint uris.		String
includeExt (filter)	Is used to include files matching file extension name (case insensitive). For example to include txt files, then use includeExt=txt. Multiple extensions can be separated by comma, for example to include txt and xml files, use includeExt=txt,xml. Note that the file extension includes all parts, for example having a file named mydata.tar.gz will have extension as tar.gz. For more flexibility then use the include/exclude options.		String
maxDepth (filter)	The maximum depth to traverse when recursively processing a directory.	2147483647	int
maxMessagesPerPoll (filter)	To define a maximum messages to gather per poll. By default no maximum is set. Can be used to set a limit of e.g. 1000 to avoid when starting up the server that there are thousands of files. Set a value of 0 or negative to disabled it. Notice: If this option is in use then the File and FTP components will limit before any sorting. For example if you have 100000 files and use maxMessagesPerPoll=500, then only the first 500 files will be picked up, and then sorted. You can use the eagerMaxMessagesPerPoll option and set this to false to allow to scan all files first and then sort afterwards.		int
minDepth (filter)	The minimum depth to start processing when recursively processing a directory. Using minDepth=1 means the base directory. Using minDepth=2 means the first sub directory.		int
move (filter)	Expression (such as Simple Language) used to dynamically set the filename when moving it after processing. To move files into a .done subdirectory just enter .done.		String
exclusiveReadLockStrategy (lock)	Pluggable read-lock as a org.apache.camel.component.file.GenericFileExclusiveReadLockStrategy implementation.		GenericFileExclusiveReadLockStrategy

readLock (lock) Name	Description	none Default	String Type
	Used by consumer, to only poll the files if it has exclusive read-lock on the file (i.e. the file is not in-progress or being written). Camel will wait until the file lock is granted. This option provides the build in strategies: - none - No read lock is in use - markerFile - Camel creates a marker file (fileName.camellLock) and then holds a lock on it. This option is not available for the FTP component - changed - Changed is using file length/modification timestamp to detect whether the file is currently being copied or not. Will at least use 1 sec to determine this, so this option cannot consume files as fast as the others, but can be more reliable as the JDK IO API cannot always determine whether a file is currently being used by another process. The option readLockCheckInterval can be used to set the check frequency. - fileLock - is for using java.nio.channels.FileLock. This option is not avail for Windows OS and the FTP component. This approach should be avoided when accessing a remote file system via a mount/share unless that file system supports distributed file locks. - rename - rename is for using a try to rename the file as a test if we can get exclusive read-lock. - idempotent - (only for file component) idempotent is for using a idempotentRepository as the read-lock. This allows to use read locks that supports clustering if the idempotent repository implementation supports that. - idempotent-changed - (only for file component) idempotent-changed is for using a idempotentRepository and changed as the combined read-lock. This allows to use read locks that supports clustering if the idempotent repository implementation supports that. - idempotent-rename - (only for file component) idempotent-rename is for using a idempotentRepository and rename as the combined read-lock. This allows to use read locks that supports clustering if the idempotent repository implementation supports that. Notice: The various read locks is not all suited to work in clustered mode, where concurrent consumers on different nodes is competing for the same files on a shared file system. The markerFile using a close to atomic operation to create the empty marker file, but its not guaranteed to work in a cluster. The fileLock may work better but then the file system need to support distributed file locks, and so on. Using the idempotent read lock can support clustering if the idempotent repository supports clustering, such as Hazelcast Component or Infinispan. Enum values: ● none ● markerFile ● fileLock		

Name	● rename Description ● changed	Default	Type
	● idempotent ● idempotent-changed ● idempotent-rename		
readLockCheckInterval (lock)	Interval in millis for the read-lock, if supported by the read lock. This interval is used for sleeping between attempts to acquire the read lock. For example when using the changed read lock, you can set a higher interval period to cater for slow writes. The default of 1 sec. may be too fast if the producer is very slow writing the file. Notice: For FTP the default readLockCheckInterval is 5000. The readLockTimeout value must be higher than readLockCheckInterval, but a rule of thumb is to have a timeout that is at least 2 or more times higher than the readLockCheckInterval. This is needed to ensure that ample time is allowed for the read lock process to try to grab the lock before the timeout was hit.	1000	long
readLockDeleteOrphanLockFiles (lock)	Whether or not read lock with marker files should upon startup delete any orphan read lock files, which may have been left on the file system, if Camel was not properly shutdown (such as a JVM crash). If turning this option to false then any orphaned lock file will cause Camel to not attempt to pickup that file, this could also be due another node is concurrently reading files from the same shared directory.	true	boolean
readLockIdempotentReleaseAsync (lock)	Whether the delayed release task should be synchronous or asynchronous. See more details at the readLockIdempotentReleaseDelay option.	false	boolean
readLockIdempotentReleaseAsyncPoolSize (lock)	The number of threads in the scheduled thread pool when using asynchronous release tasks. Using a default of 1 core threads should be sufficient in almost all use-cases, only set this to a higher value if either updating the idempotent repository is slow, or there are a lot of files to process. This option is not in-use if you use a shared thread pool by configuring the readLockIdempotentReleaseExecutorService option. See more details at the readLockIdempotentReleaseDelay option.		int

Name	Description	Default	Type
readLockIdempotentReleaseDelay (lock)	Whether to delay the release task for a period of millis. This can be used to delay the release tasks to expand the window when a file is regarded as read-locked, in an active/active cluster scenario with a shared idempotent repository, to ensure other nodes cannot potentially scan and acquire the same file, due to race-conditions. By expanding the time-window of the release tasks helps prevents these situations. Note delaying is only needed if you have configured <code>readLockRemoveOnCommit</code> to true.		int
readLockIdempotentReleaseExecutorService (lock)	To use a custom and shared thread pool for asynchronous release tasks. See more details at the <code>readLockIdempotentReleaseDelay</code> option.		ScheduledExecutorService
readLockLoggingLevel (lock)	Logging level used when a read lock could not be acquired. By default a DEBUG is logged. You can change this level, for example to OFF to not have any logging. This option is only applicable for readLock of types: changed, fileLock, idempotent, idempotent-changed, idempotent-rename, rename. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	DEBUG	LoggingLevel
readLockMarkerFile (lock)	Whether to use marker file with the changed, rename, or exclusive read lock types. By default a marker file is used as well to guard against other processes picking up the same files. This behavior can be turned off by setting this option to false. For example if you do not want to write marker files to the file systems by the Camel application.	true	boolean
readLockMinAge (lock)	This option is applied only for readLock=changed. It allows to specify a minimum age the file must be before attempting to acquire the read lock. For example use <code>readLockMinAge=300s</code> to require the file is at last 5 minutes old. This can speedup the changed read lock as it will only attempt to acquire files which are at least that given age.	0	long

Name	Description	Default	Type
readLockMinLength (lock)	This option is applied only for readLock=changed. It allows you to configure a minimum file length. By default Camel expects the file to contain data, and thus the default value is 1. You can set this option to zero, to allow consuming zero-length files.	1	long
readLockRemoveOnCommit (lock)	This option is applied only for readLock=idempotent. It allows to specify whether to remove the file name entry from the idempotent repository when processing the file is succeeded and a commit happens. By default the file is not removed which ensures that any race-condition do not occur so another active node may attempt to grab the file. Instead the idempotent repository may support eviction strategies that you can configure to evict the file name entry after X minutes - this ensures no problems with race conditions. See more details at the readLockIdempotentReleaseDelay option.	false	boolean
readLockRemoveOnRollback (lock)	This option is applied only for readLock=idempotent. It allows to specify whether to remove the file name entry from the idempotent repository when processing the file failed and a rollback happens. If this option is false, then the file name entry is confirmed (as if the file did a commit).	true	boolean
readLockTimeout (lock)	Optional timeout in millis for the read-lock, if supported by the read-lock. If the read-lock could not be granted and the timeout triggered, then Camel will skip the file. At next poll Camel, will try the file again, and this time maybe the read-lock could be granted. Use a value of 0 or lower to indicate forever. Currently fileLock, changed and rename support the timeout. Notice: For FTP the default readLockTimeout value is 20000 instead of 10000. The readLockTimeout value must be higher than readLockCheckInterval, but a rule of thumb is to have a timeout that is at least 2 or more times higher than the readLockCheckInterval. This is needed to ensure that ample time is allowed for the read lock process to try to grab the lock before the timeout was hit.	10000	long
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int

Name	Description	Default	Type
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then <code>backoffIdleThreshold</code> and/or <code>backoffErrorThreshold</code> must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the <code>ScheduledPollConsumer</code> will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object

Name	Description	Default	Type
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
shuffle (sort)	To shuffle the list of files (sort in random order).	false	boolean
sortBy (sort)	Built-in sort by using the File Language. Supports nested sorts, so you can have a sort by file name and as a 2nd group sort by modified date.		String
sorter (sort)	Pluggable sorter as a java.util.Comparator class.		Comparator



NOTE

Default behavior for file producer

By default it will override any existing file, if one exist with the same name.

37.6. MOVE AND DELETE OPERATIONS

Any move or delete operations is executed after (post command) the routing has completed; so during processing of the **Exchange** the file is still located in the inbox folder.

Lets illustrate this with an example:

```
from("file://inbox?move=.done").to("bean:handleOrder");
```

When a file is dropped in the **inbox** folder, the file consumer notices this and creates a new **FileExchange** that is routed to the **handleOrder** bean. The bean then processes the **File** object. At this point in time the file is still located in the **inbox** folder. After the bean completes, and thus the route is completed, the file consumer will perform the move operation and move the file to the **.done** sub-folder.

The **move** and the **preMove** options are considered as a directory name (though if you use an expression such as [File Language](#), or [Simple](#) then the result of the expression evaluation is the file name to be used. For example, if you set:

```
move=../backup/copy-of-${file:name}
```

then that's using the [File](#) language which we use return the file name to be used), which can be either relative or absolute. If relative, the directory is created as a sub-folder from within the folder where the file was consumed.

By default, Camel will move consumed files to the **.camel** sub-folder relative to the directory where the file was consumed.

If you want to delete the file after processing, the route should be:

```
from("file://inbox?delete=true").to("bean:handleOrder");
```

We have introduced a **pre** move operation to move files **before** they are processed. This allows you to mark which files have been scanned as they are moved to this sub folder before being processed.

```
from("file://inbox?preMove=inprogress").to("bean:handleOrder");
```

You can combine the **pre** move and the regular move:

```
from("file://inbox?preMove=inprogress&move=.done").to("bean:handleOrder");
```

So in this situation, the file is in the **inprogress** folder when being processed and after it's processed, it's moved to the **.done** folder.

37.7. FINE GRAINED CONTROL OVER MOVE AND PREMOVE OPTION

The **move** and **preMove** options are Expression-based, so we have the full power of the [File Language](#) to do advanced configuration of the directory and name pattern.

Camel will, in fact, internally convert the directory name you enter into a [File Language](#) expression. So when we enter **move=.done** Camel will convert this into: **\${file:parent}/.done/\${file:onlyname}**. This is only done if Camel detects that you have not provided a **\$\{ }** in the option value yourself. So when you enter a **\$\{ }** Camel will **not** convert it and thus you have the full power.

So if we want to move the file into a backup folder with today's date as the pattern, we can do:

```
move=backup/${date:now:yyyyMMdd}/${file:name}
```

37.8. ABOUT MOVEFAILED

The **moveFailed** option allows you to move files that **could not** be processed successfully to another location such as an error folder of your choice. For example to move the files in an error folder with a

timestamp you can use `moveFailed=/error/${file:name.noext}-${date:now:yyyyMMddHHmmssSSS}.${file:ext}`.

37.9. MESSAGE HEADERS

The following headers are supported by this component:

37.9.1. File producer only

Header	Description
CamelFileName	Specifies the name of the file to write (relative to the endpoint directory). This name can be a String ; a String with a File Language or Simple language expression; or an Expression object. If it's null then Camel will auto-generate a filename based on the message unique ID.
CamelFileNameProduced	The actual absolute filepath (path + name) for the output file that was written. This header is set by Camel and its purpose is providing end-users with the name of the file that was written.
CamelOverruleFileName	Is used for overruling CamelFileName header and use the value instead (but only once, as the producer will remove this header after writing the file). The value can be only be a String. Notice that if the option fileName has been configured, then this is still being evaluated.

37.9.2. File consumer only

Header	Description
CamelFileName	Name of the consumed file as a relative file path with offset from the starting directory configured on the endpoint.
CamelFileNameOnly	Only the file name (the name with no leading paths).
CamelFileAbsolute	A boolean option specifying whether the consumed file denotes an absolute path or not. Should normally be false for relative paths. Absolute paths should normally not be used but we added to the move option to allow moving files to absolute paths. But can be used elsewhere as well.
CamelFileAbsolutePath	The absolute path to the file. For relative files this path holds the relative path instead.
CamelFilePath	The file path. For relative files this is the starting directory + the relative filename. For absolute files this is the absolute path.
CamelFileRelativePath	The relative path.
CamelFileParent	The parent path.

Header	Description
CamelFileLength	A long value containing the file size.
CamelFileLastModified	A Long value containing the last modified timestamp of the file.

37.10. BATCH CONSUMER

This component implements the Batch Consumer.

37.11. EXCHANGE PROPERTIES, FILE CONSUMER ONLY

As the file consumer implements the **BatchConsumer** it supports batching the files it polls. By batching we mean that Camel will add the following additional properties to the Exchange, so you know the number of files polled, the current index, and whether the batch is already completed.

Property	Description
CamelBatchSize	The total number of files that was polled in this batch.
CamelBatchIndex	The current index of the batch. Starts from 0.
CamelBatchComplete	A boolean value indicating the last Exchange in the batch. Is only true for the last entry.

This allows you for instance to know how many files exist in this batch and for instance let the `Aggregator2` aggregate this number of files.

37.12. USING CHARSET

The charset option allows for configuring an encoding of the files on both the consumer and producer endpoints. For example if you read utf-8 files, and want to convert the files to iso-8859-1, you can do:

```
from("file:inbox?charset=utf-8")
  .to("file:outbox?charset=iso-8859-1")
```

You can also use the **convertBodyTo** in the route. In the example below we have still input files in utf-8 format, but we want to convert the file content to a byte array in iso-8859-1 format. And then let a bean process the data. Before writing the content to the outbox folder using the current charset.

```
from("file:inbox?charset=utf-8")
  .convertBodyTo(byte[].class, "iso-8859-1")
  .to("bean:myBean")
  .to("file:outbox");
```

If you omit the charset on the consumer endpoint, then Camel does not know the charset of the file, and would by default use "UTF-8". However you can configure a JVM system property to override and use a different default encoding with the key **org.apache.camel.default.charset**.

In the example below this could be a problem if the files is not in UTF-8 encoding, which would be the default encoding for read the files.

In this example when writing the files, the content has already been converted to a byte array, and thus would write the content directly as is (without any further encodings).

```
from("file:inbox")
  .convertBodyTo(byte[].class, "iso-8859-1")
  .to("bean:myBean")
  .to("file:outbox");
```

You can also override and control the encoding dynamic when writing files, by setting a property on the exchange with the key **Exchange.CHARSET_NAME**. For example in the route below we set the property with a value from a message header.

```
from("file:inbox")
  .convertBodyTo(byte[].class, "iso-8859-1")
  .to("bean:myBean")
  .setProperty(Exchange.CHARSET_NAME, header("someCharsetHeader"))
  .to("file:outbox");
```

We suggest to keep things simpler, so if you pickup files with the same encoding, and want to write the files in a specific encoding, then favor to use the **charset** option on the endpoints.

Notice that if you have explicit configured a **charset** option on the endpoint, then that configuration is used, regardless of the **Exchange.CHARSET_NAME** property.

If you have some issues then you can enable DEBUG logging on **org.apache.camel.component.file**, and Camel logs when it reads/write a file using a specific charset. For example the route below will log the following:

```
from("file:inbox?charset=utf-8")
  .to("file:outbox?charset=iso-8859-1")
```

And the logs:

```
DEBUG GenericFileConverter      - Read file /Users/davsclaus/workspace/camel/camel-
core/target/charset/input/input.txt with charset utf-8
DEBUG FileOperations            - Using Reader to write file: target/charset/output.txt with charset:
iso-8859-1
```

37.13. COMMON GOTCHAS WITH FOLDER AND FILENAMES

When Camel is producing files (writing files) there are a few gotchas affecting how to set a filename of your choice. By default, Camel will use the message ID as the filename, and since the message ID is normally a unique generated ID, you will end up with filenames such as: **ID-MACHINENAME-2443-1211718892437-1-0**. If such a filename is not desired, then you must provide a filename in the **CamelFileName** message header. The constant, **Exchange.FILE_NAME**, can also be used.

The sample code below produces files using the message ID as the filename:

```
from("direct:report").to("file:target/reports");
```

To use **report.txt** as the filename you have to do:

```
from("direct:report").setHeader(Exchange.FILE_NAME, constant("report.txt")).to("file:target/reports");
```

- the same as above, but with **CamelFileName**:

```
from("direct:report").setHeader("CamelFileName", constant("report.txt")).to("file:target/reports");
```

And a syntax where we set the filename on the endpoint with the **fileName** URI option.

```
from("direct:report").to("file:target/reports/?fileName=report.txt");
```

37.14. FILENAME EXPRESSION

Filename can be set either using the **expression** option or as a string-based [File](#) language expression in the **CamelFileName** header. See the [File](#) language for syntax and samples.

37.15. CONSUMING FILES FROM FOLDERS WHERE OTHERS DROP FILES DIRECTLY

Beware if you consume files from a folder where other applications write files to directly. Take a look at the different readLock options to see what suits your use cases. The best approach is however to write to another folder and after the write move the file in the drop folder. However if you write files directly to the drop folder then the option changed could better detect whether a file is currently being written/copied as it uses a file changed algorithm to see whether the file size / modification changes over a period of time. The other readLock options rely on Java File API that sadly is not always very good at detecting this. You may also want to look at the doneFileName option, which uses a marker file (done file) to signal when a file is done and ready to be consumed.

37.16. USING DONE FILES

See also section *writing done files* below.

If you want only to consume files when a done file exists, then you can use the **doneFileName** option on the endpoint.

```
from("file:bar?doneFileName=done");
```

Will only consume files from the bar folder, if a done *file* exists in the same directory as the target files. Camel will automatically delete the *done file* when it's done consuming the files. Camel does not delete automatically the *done file* if **noop=true** is configured.

However it is more common to have one *done file* per target file. This means there is a 1:1 correlation. To do this you must use dynamic placeholders in the **doneFileName** option. Currently Camel supports the following two dynamic tokens: **file:name** and **file:name.noext** which must be enclosed in `${ }`. The consumer only supports the static part of the *done file* name as either prefix or suffix (not both).

```
from("file:bar?doneFileName=${file:name}.done");
```

In this example only files will be polled if there exists a done file with the name *file name.done*. For example

- **hello.txt** - is the file to be consumed

- **hello.txt.done** – is the associated done file

You can also use a prefix for the done file, such as:

```
from("file:bar?doneFileName=ready-${file:name}");
```

- **hello.txt** – is the file to be consumed
- **ready-hello.txt** – is the associated done file

37.17. WRITING DONE FILES

After you have written a file you may want to write an additional *donefile* as a kind of marker, to indicate to others that the file is finished and has been written. To do that you can use the **doneFileName** option on the file producer endpoint.

```
.to("file:bar?doneFileName=done");
```

Will simply create a file named **done** in the same directory as the target file.

However it is more common to have one done file per target file. This means there is a 1:1 correlation. To do this you must use dynamic placeholders in the **doneFileName** option. Currently Camel supports the following two dynamic tokens: **file:name** and **file:name.noext** which must be enclosed in `${ }`.

```
.to("file:bar?doneFileName=done-${file:name}");
```

Will for example create a file named **done-foo.txt** if the target file was **foo.txt** in the same directory as the target file.

```
.to("file:bar?doneFileName=${file:name}.done");
```

Will for example create a file named **foo.txt.done** if the target file was **foo.txt** in the same directory as the target file.

```
.to("file:bar?doneFileName=${file:name.noext}.done");
```

Will for example create a file named **foo.done** if the target file was **foo.txt** in the same directory as the target file.

37.18. SAMPLES

37.18.1. Read from a directory and write to another directory

```
from("file://inputdir/?delete=true").to("file://outputdir")
```

37.18.2. Read from a directory and write to another directory using a overrule dynamic name

```
from("file://inputdir/?delete=true").to("file://outputdir?overruleFile=copy-of-${file:name}")
```

Listen on a directory and create a message for each file dropped there. Copy the contents to the **outputdir** and delete the file in the **inputdir**.

37.18.3. Reading recursively from a directory and writing to another

```
from("file://inputdir/?recursive=true&delete=true").to("file://outputdir")
```

Listen on a directory and create a message for each file dropped there. Copy the contents to the **outputdir** and delete the file in the **inputdir**. Will scan recursively into sub-directories. Will lay out the files in the same directory structure in the **outputdir** as the **inputdir**, including any sub-directories.

```
inputdir/foo.txt  
inputdir/sub/bar.txt
```

Will result in the following output layout:

```
outputdir/foo.txt  
outputdir/sub/bar.txt
```

37.19. USING FLATTEN

If you want to store the files in the outputdir directory in the same directory, disregarding the source directory layout (e.g. to flatten out the path), you just add the **flatten=true** option on the file producer side:

```
from("file://inputdir/?recursive=true&delete=true").to("file://outputdir?flatten=true")
```

Will result in the following output layout:

```
outputdir/foo.txt  
outputdir/bar.txt
```

37.20. READING FROM A DIRECTORY AND THE DEFAULT MOVE OPERATION

Camel will by default move any processed file into a **.camel** subdirectory in the directory the file was consumed from.

```
from("file://inputdir/?recursive=true&delete=true").to("file://outputdir")
```

Affects the layout as follows:

before

```
inputdir/foo.txt  
inputdir/sub/bar.txt
```

after

```
inputdir/.camel/foo.txt  
inputdir/sub/.camel/bar.txt
```

```
outputdir/foo.txt
outputdir/sub/bar.txt
```

37.21. READ FROM A DIRECTORY AND PROCESS THE MESSAGE IN JAVA

```
from("file://inputdir/").process(new Processor() {
    public void process(Exchange exchange) throws Exception {
        Object body = exchange.getIn().getBody();
        // do some business logic with the input body
    }
});
```

The body will be a **File** object that points to the file that was just dropped into the **inputdir** directory.

37.22. WRITING TO FILES

Camel is of course also able to write files, i.e. produce files. In the sample below we receive some reports on the SEDA queue that we process before they are being written to a directory.

37.22.1. Write to subdirectory using **Exchange.FILE_NAME**

Using a single route, it is possible to write a file to any number of subdirectories. If you have a route setup as such:

```
<route>
  <from uri="bean:myBean"/>
  <to uri="file:/rootDirectory"/>
</route>
```

You can have **myBean** set the header **Exchange.FILE_NAME** to values such as:

```
Exchange.FILE_NAME = hello.txt => /rootDirectory/hello.txt
Exchange.FILE_NAME = foo/bye.txt => /rootDirectory/foo/bye.txt
```

This allows you to have a single route to write files to multiple destinations.

37.22.2. Writing file through the temporary directory relative to the final destination

Sometime you need to temporarily write the files to some directory relative to the destination directory. Such situation usually happens when some external process with limited filtering capabilities is reading from the directory you are writing to. In the example below files will be written to the **/var/myapp/filesInProgress** directory and after data transfer is done, they will be atomically moved to the **/var/myapp/finalDirectory** directory.

```
from("direct:start").
  to("file:///var/myapp/finalDirectory?tempPrefix=./filesInProgress/");
```

37.23. USING EXPRESSION FOR FILENAMES

In this sample we want to move consumed files to a backup folder using today's date as a sub-folder name:

```
from("file://inbox?move=backup/${date:now:yyyyMMdd}/${file:name}").to("...");
```

See [File language](#) for more samples.

37.24. AVOIDING READING THE SAME FILE MORE THAN ONCE (IDEMPOTENT CONSUMER)

Camel supports Idempotent Consumer directly within the component so it will skip already processed files. This feature can be enabled by setting the **idempotent=true** option.

```
from("file://inbox?idempotent=true").to("...");
```

Camel uses the absolute file name as the idempotent key, to detect duplicate files. You can customize this key by using an expression in the `idempotentKey` option. For example to use both the name and the file size as the key

```
<route>
  <from uri="file://inbox?idempotent=true&idempotentKey=${file:name}-${file:size}"/>
  <to uri="bean:processInbox"/>
</route>
```

By default Camel uses a in memory based store for keeping track of consumed files, it uses a least recently used cache holding up to 1000 entries. You can plugin your own implementation of this store by using the **idempotentRepository** option using the `#` sign in the value to indicate it's a referring to a bean in the Registry with the specified **id**.

```
<!-- define our store as a plain spring bean -->
<bean id="myStore" class="com.mycompany.MyIdempotentStore"/>

<route>
  <from uri="file://inbox?idempotent=true&idempotentRepository=#myStore"/>
  <to uri="bean:processInbox"/>
</route>
```

Camel will log at **DEBUG** level if it skips a file because it has been consumed before:

```
DEBUG FileConsumer is idempotent and the file has been consumed before. Will skip this file:
target\idempotent\report.txt
```

37.25. USING A FILE BASED IDEMPOTENT REPOSITORY

In this section we will use the file based idempotent repository **org.apache.camel.processor.idempotent.FileIdempotentRepository** instead of the in-memory based that is used as default.

This repository uses a 1st level cache to avoid reading the file repository. It will only use the file repository to store the content of the 1st level cache. Thereby the repository can survive server restarts. It will load the content of the file into the 1st level cache upon startup. The file structure is very simple as

it stores the key in separate lines in the file. By default, the file store has a size limit of 1mb. When the file grows larger Camel will truncate the file store, rebuilding the content by flushing the 1st level cache into a fresh empty file.

We configure our repository using Spring XML creating our file idempotent repository and define our file consumer to use our repository with the **idempotentRepository** using # sign to indicate Registry lookup:

37.26. USING A JPA BASED IDEMPOTENT REPOSITORY

In this section we will use the JPA based idempotent repository instead of the in-memory based that is used as default.

First we need a persistence-unit in **META-INF/persistence.xml** where we need to use the class **org.apache.camel.processor.idempotent.jpa.MessageProcessed** as model.

```
<persistence-unit name="idempotentDb" transaction-type="RESOURCE_LOCAL">
  <class>org.apache.camel.processor.idempotent.jpa.MessageProcessed</class>

  <properties>
    <property name="openjpa.ConnectionURL" value="jdbc:derby:target/idempotentTest;create=true"/>
    <property name="openjpa.ConnectionDriverName"
value="org.apache.derby.jdbc.EmbeddedDriver"/>
    <property name="openjpa.jdbc.SynchronizeMappings" value="buildSchema"/>
    <property name="openjpa.Log" value="DefaultLevel=WARN, Tool=INFO"/>
    <property name="openjpa.Multithreaded" value="true"/>
  </properties>
</persistence-unit>
```

Next, we can create our JPA idempotent repository in the spring XML file as well:

```
<!-- we define our jpa based idempotent repository we want to use in the file consumer -->
<bean id="jpaStore" class="org.apache.camel.processor.idempotent.jpa.JpaMessageIdRepository">
  <!-- Here we refer to the entityManagerFactory -->
  <constructor-arg index="0" ref="entityManagerFactory"/>
  <!-- This 2nd parameter is the name (= a category name).
      You can have different repositories with different names -->
  <constructor-arg index="1" value="FileConsumer"/>
</bean>
```

And yes then we just need to refer to the **jpaStore** bean in the file consumer endpoint using the **idempotentRepository** using the # syntax option:

```
<route>
  <from uri="file://inbox?idempotent=true&idempotentRepository=#jpaStore"/>
  <to uri="bean:processInbox"/>
</route>
```

37.27. FILTER USING ORG.APACHE.CAMEL.COMPONENT.FILE.GENERICFILEFILTER

Camel supports pluggable filtering strategies. You can then configure the endpoint with such a filter to skip certain files being processed.

In the sample we have built our own filter that skips files starting with **skip** in the filename:

And then we can configure our route using the **filter** attribute to reference our filter (using # notation) that we have defined in the spring XML file:

```
<!-- define our filter as a plain spring bean -->
<bean id="myFilter" class="com.mycompany.MyFileFilter"/>

<route>
  <from uri="file://inbox?filter=#myFilter"/>
  <to uri="bean:processInbox"/>
</route>
```

37.28. FILTERING USING ANT PATH MATCHER

The ANT path matcher is based on [AntPathMatcher](#).

The file paths is matched with the following rules:

- **?** matches one character
- ***** matches zero or more characters
- ****** matches zero or more directories in a path

The **antInclude** and **antExclude** options make it easy to specify ANT style include/exclude without having to define the filter. See the URI options above for more information. The sample below demonstrates how to use it.



NOTE

When using **minDepth/maxDepth** with the combination of **recursive=true**, **antExclude=...**, and **readLockDeleteOrphanLockFiles=true** results in scanning all the files/subfolders deeper than value mentioned in the **maxDepth**. The workaround is to configure **readLockDeleteOrphanLockFiles=false**.

37.28.1. Sorting using Comparator

Camel supports pluggable sorting strategies. This strategy it to use the build in **java.util.Comparator** in Java. You can then configure the endpoint with such a comparator and have Camel sort the files before being processed.

In the sample we have built our own comparator that just sorts by file name:

And then we can configure our route using the **sorter** option to reference to our sorter (**mySorter**) we have defined in the spring XML file:

```
<!-- define our sorter as a plain spring bean -->
<bean id="mySorter" class="com.mycompany.MyFileSorter"/>

<route>
  <from uri="file://inbox?sorter=#mySorter"/>
  <to uri="bean:processInbox"/>
</route>
```




NOTE

URI options can reference beans using the # syntax

In the Spring DSL route above notice that we can refer to beans in the Registry by prefixing the id with **#**. So writing **sorter=#mySorter**, will instruct Camel to go look in the Registry for a bean with the ID, **mySorter**.

37.28.2. Sorting using sortBy

Camel supports pluggable sorting strategies. This strategy it to use the [File](#) language to configure the sorting. The **sortBy** option is configured as follows:

```
sortBy=group 1;group 2;group 3;...
```

Where each group is separated with semi colon. In the simple situations you just use one group, so a simple example could be:

```
sortBy=file:name
```

This will sort by file name, you can reverse the order by prefixing **reverse:** to the group, so the sorting is now Z..A:

```
sortBy=reverse:file:name
```

As we have the full power of [File](#) language we can use some of the other parameters, so if we want to sort by file size we do:

```
sortBy=file:length
```

You can configure to ignore the case, using **ignoreCase:** for string comparison, so if you want to use file name sorting but to ignore the case then we do:

```
sortBy=ignoreCase:file:name
```

You can combine ignore case and reverse, however reverse must be specified first:

```
sortBy=reverse:ignoreCase:file:name
```

In the sample below we want to sort by last modified file, so we do:

```
sortBy=file:modified
```

And then we want to group by name as a 2nd option so files with same modification is sorted by name:

```
sortBy=file:modified;file:name
```

Now there is an issue here, can you spot it? Well the modified timestamp of the file is too fine as it will be in milliseconds, but what if we want to sort by date only and then subgroup by name?

Well as we have the true power of [File](#) language we can use its date command that supports patterns. So this can be solved as:

```
sortBy=date:file:yyyyMMdd;file:name
```

Yeah, that is pretty powerful, oh by the way you can also use reverse per group, so we could reverse the file names:

```
sortBy=date:file:yyyyMMdd;reverse:file:name
```

37.29. USING GENERICFILEPROCESSSTRATEGY

The option **processStrategy** can be used to use a custom **GenericFileProcessStrategy** that allows you to implement your own *begin*, *commit* and *rollback* logic.

For instance lets assume a system writes a file in a folder you should consume. But you should not start consuming the file before another *ready* file has been written as well.

So by implementing our own **GenericFileProcessStrategy** we can implement this as:

- In the **begin()** method we can test whether the special *ready* file exists. The begin method returns a **boolean** to indicate if we can consume the file or not.
- In the **abort()** method special logic can be executed in case the **begin** operation returned **false**, for example to cleanup resources etc.
- in the **commit()** method we can move the actual file and also delete the *ready* file.

37.30. USING FILTER

The **filter** option allows you to implement a custom filter in Java code by implementing the **org.apache.camel.component.file.GenericFileFilter** interface. This interface has an **accept** method that returns a boolean. Return **true** to include the file, and **false** to skip the file. There is a **isDirectory** method on **GenericFile** whether the file is a directory. This allows you to filter unwanted directories, to avoid traversing down unwanted directories.

For example to skip any directories which starts with **"skip"** in the name, can be implemented as follows:

37.31. USING BRIDGEERRORHANDLER

If you want to use the Camel Error Handler to deal with any exception occurring in the file consumer, then you can enable the **bridgeErrorHandler** option as shown below:

```
// to handle any IOException being thrown
onException(IOException.class)
    .handled(true)
    .log("IOException occurred due: ${exception.message}")
    .transform().simple("Error ${exception.message}")
    .to("mock:error");

// this is the file route that pickup files, notice how we bridge the consumer to use the Camel routing
// error handler
// the exclusiveReadLockStrategy is only configured because this is from an unit test, so we use that
// to simulate exceptions
from("file:target/nospace?bridgeErrorHandler=true")
    .convertBodyTo(String.class)
    .to("mock:result");
```

So all you have to do is to enable this option, and the error handler in the route will take it from there.



IMPORTANT

When using **bridgeErrorHandler**

When using `bridgeErrorHandler`, then interceptors, `OnCompletions` does **not** apply. The Exchange is processed directly by the Camel Error Handler, and does not allow prior actions such as interceptors, `onCompletion` to take action.

37.32. DEBUG LOGGING

This component has log level **TRACE** that can be helpful if you have problems.

37.33. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.file.acquire-lock-delay</code>	The time to wait before starting to try to acquire lock.		String
<code>camel.cluster.file.acquire-lock-interval</code>	The time to wait between attempts to try to acquire lock.		String
<code>camel.cluster.file.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.file.enabled</code>	Sets if the file cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.file.id</code>	Cluster Service ID.		String
<code>camel.cluster.file.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.file.root</code>	The root path.		String
<code>camel.component.file.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.file.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.file.enabled	Whether to enable auto configuration of the file component. This is enabled by default.		Boolean
camel.component.file.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 38. FILE LANGUAGE

The File Expression Language is an extension to the language, adding file related capabilities. These capabilities are related to common use cases working with file path and names. The goal is to allow expressions to be used with the

components for setting dynamic file patterns for both consumer and producer.



NOTE

The file language is merged with language which means you can use all the file syntax directly within the simple language.

38.1. DEPENDENCIES

The File language is part of **camel-core**.

When using **file** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-core-starter</artifactId>
</dependency>
```

38.2. FILE LANGUAGE OPTIONS

The File language supports 2 options, which are listed below.

Name	Default	Java Type	Description
resultType		String	Sets the class name of the result type (type from output).
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

38.3. SYNTAX

This language is an **extension** to the language so the syntax applies also. So the table below only lists the additional file related functions.

All the file tokens use the same expression name as the method on the **java.io.File** object, for instance **file:absolute** refers to the **java.io.File.getAbsolute()** method. Notice that not all expressions are supported by the current Exchange. For instance the component supports some options, whereas the File component supports all of them.

Expression	Type	File Consumer	File Producer	FTP Consumer	FTP Producer	Description
file:name	String	yes	no	yes	no	refers to the file name (is relative to the starting directory, see note below)
file:name.ext	String	yes	no	yes	no	refers to the file extension only
file:name.ext.single	String	yes	no	yes	no	refers to the file extension. If the file extension has multiple dots, then this expression strips and only returns the last part.
file:name.noext	String	yes	no	yes	no	refers to the file name with no extension (is relative to the starting directory, see note below)
file:name.noext.single	String	yes	no	yes	no	refers to the file name with no extension (is relative to the starting directory, see note below). If the file extension has multiple dots, then this expression strips only the last part, and keep the others.
file:only name	String	yes	no	yes	no	refers to the file name only with no leading paths.
file:only name.noext	String	yes	no	yes	no	refers to the file name only with no extension and with no leading paths.

Expression	Type	File Consumer	File Producer	FTP Consumer	FTP Producer	Description
file:onlyname.noext.single	String	yes	no	yes	no	refers to the file name only with no extension and with no leading paths. If the file extension has multiple dots, then this expression strips only the last part, and keep the others.
file:ext	String	yes	no	yes	no	refers to the file extension only
file:parent	String	yes	no	yes	no	refers to the file parent
file:path	String	yes	no	yes	no	refers to the file path
file:absolute	Boolean	yes	no	no	no	refers to whether the file is regarded as absolute or relative
file:absolute.path	String	yes	no	no	no	refers to the absolute file path
file:length	Long	yes	no	yes	no	refers to the file length returned as a Long type
file:size	Long	yes	no	yes	no	refers to the file length returned as a Long type
file:modified	Date	yes	no	yes	no	Refers to the file last modified returned as a Date type

Expression	Type	File Consumer	File Producer	FTP Consumer	FTP Producer	Description
date:command:pattern_	String	yes	yes	yes	yes	for date formatting using the java.text.SimpleDateFormat patterns. Is an extension to the language. Additional command is: file (consumers only) for the last modified timestamp of the file. Notice: all the commands from the language can also be used.

38.4. FILE TOKEN EXAMPLE

38.4.1. Relative paths

We have a **java.io.File** handle for the file **hello.txt** in the following **relative** directory: **.filelanguage\test**. And we configure our endpoint to use this starting directory **.filelanguage**. The file tokens will return as:

Expression	Returns
file:name	test\hello.txt
file:name.ext	txt
file:name.noext	test\hello
file:onlyname	hello.txt
file:onlyname.noext	hello
file:ext	txt
file:parent	filelanguage\test
file:path	filelanguage\test\hello.txt
file:absolute	false
file:absolute.path	\workspace\camel\camel-core\target\filelanguage\test\hello.txt

38.4.2. Absolute paths

We have a **java.io.File** handle for the file **hello.txt** in the following **absolute** directory: **\workspace\camel\camel-core\target\filelanguage\test**. And we configure out endpoint to use the absolute starting directory **\workspace\camel\camel-core\target\filelanguage**. The file tokens will return as:

Expression	Returns
file:name	test\hello.txt
file:name.ext	txt
file:name.noext	test\hello
file:onlyname	hello.txt
file:onlyname.noext	hello
file:ext	txt
file:parent	\workspace\camel\camel-core\target\filelanguage\test
file:path	\workspace\camel\camel-core\target\filelanguage\test\hello.txt
file:absolute	true
file:absolute.path	\workspace\camel\camel-core\target\filelanguage\test\hello.txt

38.5. SAMPLES

You can enter a fixed file name such as **myfile.txt**:

```
fileName="myfile.txt"
```

Let's assume we use the file consumer to read files and want to move the read files to back up folder with the current date as a sub folder. This can be done using an expression like:

```
fileName="backup/${date:now:yyyyMMdd}/${file:name.noext}.bak"
```

relative folder names are also supported so suppose the backup folder should be a sibling folder then you can append **..** as shown:

```
fileName="../backup/${date:now:yyyyMMdd}/${file:name.noext}.bak"
```

As this is an extension to the language we have access to all the goodies from this language also, so in this use case we want to use the `in.header.type` as a parameter in the dynamic expression:

```
fileName="../../backup/${date:now:yyyyMMdd}/type-${in.header.type}/backup-of-${file:name.noext}.bak"
```

If you have a custom date you want to use in the expression then Camel supports retrieving dates from the message header:

```
fileName="orders/order-${in.header.customerId}-${date:in.header.orderDate:yyyyMMdd}.xml"
```

And finally we can also use a bean expression to invoke a POJO class that generates some String output (or convertible to String) to be used:

```
fileName="uniquefile-${bean:myguidgenerator.generateid}.txt"
```

Of course all this can be combined in one expression where you can use the `and` language in one combined expression. This is pretty powerful for those common file path patterns.

38.6. SPRING BOOT AUTO-CONFIGURATION

The component supports 147 options, which are listed below.

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.acl-token</code>	Sets the ACL token to be used with Consul.		String
<code>camel.cloud.consul.service-discovery.block-seconds</code>	The seconds to wait for a watch event, default 10 seconds.	10	Integer
<code>camel.cloud.consul.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.consul.service-discovery.connect-timeout-millis</code>	Connect timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.datacenter</code>	The data center.		String

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.consul.service-discovery.password</code>	Sets the password to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.consul.service-discovery.read-timeout-millis</code>	Read timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.url</code>	The Consul agent URL.		String
<code>camel.cloud.consul.service-discovery.username</code>	Sets the username to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.write-timeout-millis</code>	Write timeout for OkHttpClient.		Long
<code>camel.cloud.dns.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.dns.service-discovery.domain</code>	The domain name;.		String
<code>camel.cloud.dns.service-discovery.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
<code>camel.cloud.dns.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.dns.service-discovery.proto</code>	The transport protocol of the desired service.	<code>_tcp</code>	String
<code>camel.cloud.etcd.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.etcd.service-discovery.enabled</code>	Enable the component.	<code>true</code>	Boolean
<code>camel.cloud.etcd.service-discovery.password</code>	The password to use for basic authentication.		String
<code>camel.cloud.etcd.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.etcd.service-discovery.service-path</code>	The path to look for for service discovery.	<code>/services/</code>	String
<code>camel.cloud.etcd.service-discovery.timeout</code>	To set the maximum time an action could take to complete.		Long
<code>camel.cloud.etcd.service-discovery.type</code>	To set the discovery type, valid values are on-demand and watch.	<code>on-demand</code>	String
<code>camel.cloud.etcd.service-discovery.uris</code>	The URIs the client can connect to.		String

Name	Description	Default	Type
<code>camel.cloud.etcd.service-discovery.user-name</code>	The user name to use for basic authentication.		String
<code>camel.cloud.kubernetes.service-discovery.api-version</code>	Sets the API version when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-data</code>	Sets the Certificate Authority data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-file</code>	Sets the Certificate Authority data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-data</code>	Sets the Client Certificate data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-file</code>	Sets the Client Certificate data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-algo</code>	Sets the Client Keystore algorithm, such as RSA when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-data</code>	Sets the Client Keystore data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-file</code>	Sets the Client Keystore data that are loaded from the file when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.client-key-passphrase</code>	Sets the Client Keystore passphrase when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.kubernetes.service-discovery.dns-domain</code>	Sets the DNS domain to use for DNS lookup.		String
<code>camel.cloud.kubernetes.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.kubernetes.service-discovery.lookup</code>	How to perform service lookup. Possible values: client, dns, environment. When using client, then the client queries the kubernetes master to obtain a list of active pods that provides the service, and then random (or round robin) select a pod. When using dns the service name is resolved as name.namespace.svc.dnsDomain. When using dnssrv the service name is resolved with SRV query forsvc... When using environment then environment variables are used to lookup the service. By default environment is used.	environment	String
<code>camel.cloud.kubernetes.service-discovery.master-url</code>	Sets the URL to the master when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.namespace</code>	Sets the namespace to use. Will by default use namespace from the ENV variable KUBERNETES_MASTER.		String
<code>camel.cloud.kubernetes.service-discovery.oauth-token</code>	Sets the OAUTH token for authentication (instead of username/password) when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.password</code>	Sets the password for authentication when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-name</code>	Sets the Port Name to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-protocol</code>	Sets the Port Protocol to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.kubernetes.service-discovery.trust-certs</code>	Sets whether to turn on trust certificate check when using client lookup.	false	Boolean
<code>camel.cloud.kubernetes.service-discovery.username</code>	Sets the username for authentication when using client lookup.		String
<code>camel.cloud.ribbon.load-balancer.client-name</code>	Sets the Ribbon client name.		String
<code>camel.cloud.ribbon.load-balancer.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.ribbon.load-balancer.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
<code>camel.cloud.ribbon.load-balancer.namespace</code>	The namespace.		String
<code>camel.cloud.ribbon.load-balancer.password</code>	The password.		String
<code>camel.cloud.ribbon.load-balancer.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.ribbon.load-balancer.username</code>	The username.		String
<code>camel.hystrix.allow-maximum-size-to-diverge-from-core-size</code>	Allows the configuration for <code>maximumSize</code> to take effect. That value can then be equal to, or higher, than <code>coreSize</code> .	false	Boolean
<code>camel.hystrix.circuit-breaker-enabled</code>	Whether to use a <code>HystrixCircuitBreaker</code> or not. If false no circuit-breaker logic will be used and all requests permitted. This is similar in effect to <code>circuitBreakerForceClosed()</code> except that continues tracking metrics and knowing whether it should be open/closed, this property results in not even instantiating a circuit-breaker.	true	Boolean
<code>camel.hystrix.circuit-breaker-error-threshold-percentage</code>	Error percentage threshold (as whole number such as 50) at which point the circuit breaker will trip open and reject requests. It will stay tripped for the duration defined in <code>circuitBreakerSleepWindowInMilliseconds</code> ; The error percentage this is compared against comes from <code>HystrixCommandMetrics.getHealthCounts()</code> .	50	Integer

Name	Description	Default	Type
camel.hystrix.circuit-breaker-force-closed	If true the <code>HystrixCircuitBreaker#allowRequest()</code> will always return true to allow requests regardless of the error percentage from <code>HystrixCommandMetrics.getHealthCounts()</code> . The <code>circuitBreakerForceOpen()</code> property takes precedence so if it set to true this property does nothing.	false	Boolean
camel.hystrix.circuit-breaker-force-open	If true the <code>HystrixCircuitBreaker.allowRequest()</code> will always return false, causing the circuit to be open (tripped) and reject all requests. This property takes precedence over <code>circuitBreakerForceClosed()</code> .	false	Boolean
camel.hystrix.circuit-breaker-request-volume-threshold	Minimum number of requests in the <code>metricsRollingStatisticalWindowInMilliseconds()</code> that must exist before the <code>HystrixCircuitBreaker</code> will trip. If below this number the circuit will not trip regardless of error percentage.	20	Integer
camel.hystrix.circuit-breaker-sleep-window-in-milliseconds	The time in milliseconds after a <code>HystrixCircuitBreaker</code> trips open that it should wait before trying requests again.	5000	Integer
camel.hystrix.configurations	Define additional configuration definitions.		Map
camel.hystrix.core-pool-size	Core thread-pool size that gets passed to <code>java.util.concurrent.ThreadPoolExecutor#setCorePoolSize(int)</code> .	10	Integer
camel.hystrix.enabled	Enable the component.	true	Boolean
camel.hystrix.execution-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.run()</code> . Requests beyond the concurrent limit will be rejected. Applicable only when <code>executionIsolationStrategy == SEMAPHORE</code> .	20	Integer
camel.hystrix.execution-isolation-strategy	What isolation strategy <code>HystrixCommand.run()</code> will be executed with. If <code>THREAD</code> then it will be executed on a separate thread and concurrent requests limited by the number of threads in the thread-pool. If <code>SEMAPHORE</code> then it will be executed on the calling thread and concurrent requests limited by the semaphore count.	THREAD	String

Name	Description	Default	Type
camel.hystrix.execution-isolation-thread-interrupt-on-timeout	Whether the execution thread should attempt an interrupt (using <code>Future#cancel()</code>) when a thread times out. Applicable only when <code>executionIsolationStrategy() == THREAD</code> .	true	Boolean
camel.hystrix.execution-timeout-enabled	Whether the timeout mechanism is enabled for this command.	true	Boolean
camel.hystrix.execution-timeout-in-milliseconds	Time in milliseconds at which point the command will timeout and halt execution. If <code>executionIsolationThreadInterruptOnTimeout == true</code> and the command is thread-isolated, the executing thread will be interrupted. If the command is semaphore-isolated and a <code>HystrixObservableCommand</code> , that command will get unsubscribed.	1000	Integer
camel.hystrix.fallback-enabled	Whether <code>HystrixCommand.getFallback()</code> should be attempted when failure occurs.	true	Boolean
camel.hystrix.fallback-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.getFallback()</code> . Requests beyond the concurrent limit will fail-fast and not attempt retrieving a fallback.	10	Integer
camel.hystrix.group-key	Sets the group key to use. The default value is <code>CamelHystrix</code> .	CamelHystrix	String
camel.hystrix.keep-alive-time	Keep-alive time in minutes that gets passed to <code>ThreadPoolExecutor#setKeepAliveTime(long, TimeUnit)</code> .	1	Integer
camel.hystrix.max-queue-size	Max queue size that gets passed to <code>BlockingQueue</code> in <code>HystrixConcurrencyStrategy.getBlockingQueue(int)</code> . This should only affect the instantiation of a threadpool - it is not eligible to change a queue size on the fly. For that, use <code>queueSizeRejectionThreshold()</code> .	-1	Integer
camel.hystrix.maximum-size	Maximum thread-pool size that gets passed to <code>ThreadPoolExecutor#setMaximumPoolSize(int)</code> . This is the maximum amount of concurrency that can be supported without starting to reject <code>HystrixCommands</code> . Please note that this setting only takes effect if you also set <code>allowMaximumSizeToDivergeFromCoreSize</code> .	10	Integer

Name	Description	Default	Type
camel.hystrix.metrics-health-snapshot-interval-in-milliseconds	Time in milliseconds to wait between allowing health snapshots to be taken that calculate success and error percentages and affect <code>HystrixCircuitBreaker.isOpen()</code> status. On high-volume circuits the continual calculation of error percentage can become CPU intensive thus this controls how often it is calculated.	500	Integer
camel.hystrix.metrics-rolling-percentile-bucket-size	Maximum number of values stored in each bucket of the rolling percentile. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-percentile-enabled	Whether percentile metrics should be captured using <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	true	Boolean
camel.hystrix.metrics-rolling-percentile-window-buckets	Number of buckets the rolling percentile window is broken into. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	6	Integer
camel.hystrix.metrics-rolling-percentile-window-in-milliseconds	Duration of percentile rolling window in milliseconds. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10000	Integer
camel.hystrix.metrics-rolling-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into <code>HystrixRollingNumber</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-statistical-window-in-milliseconds	This property sets the duration of the statistical rolling window, in milliseconds. This is how long metrics are kept for the thread pool. The window is divided into buckets and rolls by those increments.	10000	Integer
camel.hystrix.queue-size-rejection-threshold	Queue size rejection threshold is an artificial max size at which rejections will occur even if <code>maxQueueSize</code> has not been reached. This is done because the <code>maxQueueSize</code> of a <code>BlockingQueue</code> can not be dynamically changed and we want to support dynamically changing the queue size that affects rejections. This is used by <code>HystrixCommand</code> when queuing a thread for execution.	5	Integer

Name	Description	Default	Type
<code>camel.hystrix.request-log-enabled</code>	Whether HystrixCommand execution and events should be logged to HystrixRequestLog.	true	Boolean
<code>camel.hystrix.thread-pool-key</code>	Sets the thread pool key to use. Will by default use the same value as groupKey has been configured to use.	Camel Hystrix	String
<code>camel.hystrix.thread-pool-rolling-number-statistical-window-buckets</code>	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10	Integer
<code>camel.hystrix.thread-pool-rolling-number-statistical-window-in-milliseconds</code>	Duration of statistical rolling window in milliseconds. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10000	Integer
<code>camel.language.constant.enabled</code>	Whether to enable auto configuration of the constant language. This is enabled by default.		Boolean
<code>camel.language.constant.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.csimple.enabled</code>	Whether to enable auto configuration of the csimple language. This is enabled by default.		Boolean
<code>camel.language.csimple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.exchangeproperty.enabled</code>	Whether to enable auto configuration of the exchangeProperty language. This is enabled by default.		Boolean
<code>camel.language.exchangeproperty.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.file.enabled</code>	Whether to enable auto configuration of the file language. This is enabled by default.		Boolean
<code>camel.language.file.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

Name	Description	Default	Type
<code>camel.language.header.enabled</code>	Whether to enable auto configuration of the header language. This is enabled by default.		Boolean
<code>camel.language.header.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.ref.enabled</code>	Whether to enable auto configuration of the ref language. This is enabled by default.		Boolean
<code>camel.language.ref.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.simple.enabled</code>	Whether to enable auto configuration of the simple language. This is enabled by default.		Boolean
<code>camel.language.simple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.tokenize.enabled</code>	Whether to enable auto configuration of the tokenize language. This is enabled by default.		Boolean
<code>camel.language.tokenize.group-delimiter</code>	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.		String
<code>camel.language.tokenize.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.resilience4j.automatic-transition-from-open-to-half-open-enabled</code>	Enables automatic transition from OPEN to HALF_OPEN state once the <code>waitDurationInOpenState</code> has passed.	false	Boolean
<code>camel.resilience4j.circuit-breaker-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreaker</code> instance to lookup and use from the registry. When using this, then any other circuit breaker options are not in use.		String
<code>camel.resilience4j.config-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreakerConfig</code> instance to lookup and use from the registry.		String
<code>camel.resilience4j.configurations</code>	Define additional configuration definitions.		Map

Name	Description	Default	Type
camel.resilience4j.enabled	Enable the component.	true	Boolean
camel.resilience4j.failure-rate-threshold	Configures the failure rate threshold in percentage. If the failure rate is equal or greater than the threshold the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 50 percentage.		Float
camel.resilience4j.minimum-number-of-calls	Configures the minimum number of calls which are required (per sliding window period) before the CircuitBreaker can calculate the error rate. For example, if minimumNumberOfCalls is 10, then at least 10 calls must be recorded, before the failure rate can be calculated. If only 9 calls have been recorded the CircuitBreaker will not transition to open even if all 9 calls have failed. Default minimumNumberOfCalls is 100.	100	Integer
camel.resilience4j.permitted-number-of-calls-in-half-open-state	Configures the number of permitted calls when the CircuitBreaker is half open. The size must be greater than 0. Default size is 10.	10	Integer
camel.resilience4j.sliding-window-size	Configures the size of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. slidingWindowSize configures the size of the sliding window. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. The slidingWindowSize must be greater than 0. The minimumNumberOfCalls must be greater than 0. If the slidingWindowType is COUNT_BASED, the minimumNumberOfCalls cannot be greater than slidingWindowSize . If the slidingWindowType is TIME_BASED, you can pick whatever you want. Default slidingWindowSize is 100.	100	Integer

Name	Description	Default	Type
camel.resilience4j.sliding-window-type	Configures the type of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. Default slidingWindowType is COUNT_BASED.	COUNT_BASED	String
camel.resilience4j.slow-call-duration-threshold	Configures the duration threshold (seconds) above which calls are considered as slow and increase the slow calls percentage. Default value is 60 seconds.	60	Integer
camel.resilience4j.slow-call-rate-threshold	Configures a threshold in percentage. The CircuitBreaker considers a call as slow when the call duration is greater than slowCallDurationThreshold Duration. When the percentage of slow calls is equal or greater the threshold, the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 100 percentage which means that all recorded calls must be slower than slowCallDurationThreshold.		Float
camel.resilience4j.wait-duration-in-open-state	Configures the wait duration (in seconds) which specifies how long the CircuitBreaker should stay open, before it switches to half open. Default value is 60 seconds.	60	Integer
camel.resilience4j.writable-stack-trace-enabled	Enables writable stack traces. When set to false, Exception.getStackTrace returns a zero length array. This may be used to reduce log spam when the circuit breaker is open as the cause of the exceptions is already known (the circuit breaker is short-circuiting calls).	true	Boolean
camel.rest.api-component	The name of the Camel component to use as the REST API (such as swagger) If no API Component has been explicit configured, then Camel will lookup if there is a Camel component responsible for servicing and generating the REST API documentation, or if a org.apache.camel.spi.RestApiProcessorFactory is registered in the registry. If either one is found, then that is being used.		String

Name	Description	Default	Type
camel.rest.api-context-path	Sets a leading API context-path the REST API services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path.		String
camel.rest.api-context-route-id	Sets the route id to use for the route that services the REST API. The route will by default use an auto assigned route id.		String
camel.rest.api-host	To use an specific hostname for the API documentation (eg swagger) This can be used to override the generated host with this configured hostname.		String
camel.rest.api-property	Allows to configure as many additional properties for the api documentation (swagger). For example set property api.title to my cool stuff.		Map
camel.rest.api-vendor-extension	Whether vendor extension is enabled in the Rest APIs. If enabled then Camel will include additional information as vendor extension (eg keys starting with x-) such as route ids, class names etc. Not all 3rd party API gateways and tools supports vendor-extensions when importing your API docs.	false	Boolean
camel.rest.binding-mode	Sets the binding mode to use. The default value is off.		RestBindingMode
camel.rest.client-request-validation	Whether to enable validation of the client request to check whether the Content-Type and Accept headers from the client is supported by the Rest-DSL configuration of its consumes/produces settings. This can be turned on, to enable this check. In case of validation error, then HTTP Status codes 415 or 406 is returned. The default value is false.	false	Boolean
camel.rest.component	The Camel Rest component to use for the REST transport (consumer), such as netty-http, jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String

Name	Description	Default	Type
camel.rest.component-property	Allows to configure as many additional properties for the rest component in use.		Map
camel.rest.consumer-property	Allows to configure as many additional properties for the rest consumer in use.		Map
camel.rest.context-path	Sets a leading context-path the REST services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path. Or for components such as camel-jetty or camel-netty-http that includes a HTTP server.		String
camel.rest.cors-headers	Allows to configure custom CORS headers.		Map
camel.rest.data-format-property	Allows to configure as many additional properties for the data formats in use. For example set property prettyPrint to true to have json outputted in pretty mode. The properties can be prefixed to denote the option is only for either JSON or XML and for either the IN or the OUT. The prefixes are: json.in. json.out. xml.in. xml.out. For example a key with value xml.out.mustBeJAXBELEMENT is only for the XML data format for the outgoing. A key without a prefix is a common key for all situations.		Map
camel.rest.enable-cors	Whether to enable CORS headers in the HTTP response. The default value is false.	false	Boolean
camel.rest.endpoint-property	Allows to configure as many additional properties for the rest endpoint in use.		Map
camel.rest.host	The hostname to use for exposing the REST service.		String
camel.rest.host-name-resolver	If no hostname has been explicit configured, then this resolver is used to compute the hostname the REST service will be using.		RestHostNameResolver
camel.rest.json-data-format	Name of specific json data format to use. By default json-jackson will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String

Name	Description	Default	Type
camel.rest.port	The port number to use for exposing the REST service. Notice if you use servlet component then the port number configured here does not apply, as the port number in use is the actual port number the servlet component is using. eg if using Apache Tomcat its the tomcat http port, if using Apache Karaf its the HTTP service in Karaf that uses port 8181 by default etc. Though in those situations setting the port number here, allows tooling and JMX to know the port number, so its recommended to set the port number to the number that the servlet engine uses.		String
camel.rest.producer-api-doc	Sets the location of the api document (swagger api) the REST producer will use to validate the REST uri and query parameters are valid accordingly to the api document. This requires adding camel-swagger-java to the classpath, and any miss configuration will let Camel fail on startup and report the error(s). The location of the api document is loaded from classpath by default, but you can use file: or http: to refer to resources to load from file or http url.		String
camel.rest.producer-component	Sets the name of the Camel component to use as the REST producer.		String
camel.rest.scheme	The scheme to use for exposing the REST service. Usually http or https is supported. The default value is http.		String
camel.rest.skip-binding-on-error-code	Whether to skip binding on output if there is a custom HTTP error code header. This allows to build custom error messages that do not bind to json / xml etc, as success messages otherwise will do.	false	Boolean
camel.rest.use-x-forward-headers	Whether to use X-Forward headers for Host and related setting. The default value is true.	true	Boolean
camel.rest.xml-data-format	Name of specific XML data format to use. By default jaxb will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String

Name	Description	Default	Type
camel.rest.api-context-id-pattern	Deprecated Sets an CamelContext id pattern to only allow Rest APIs from rest services within CamelContext's which name matches the pattern. The pattern name refers to the CamelContext name, to match on the current CamelContext only. For any other value, the pattern uses the rules from PatternHelper#matchPattern(String,String).		String
camel.rest.api-context-listing	Deprecated Sets whether listing of all available CamelContext's with REST services in the JVM is enabled. If enabled it allows to discover these contexts, if false then only the current CamelContext is in use.	false	Boolean

CHAPTER 39. FLINK

Since Camel 2.18

Only producer is supported

This documentation page covers the [Flink](#) component for the Apache Camel. The **camel-flink** component provides a bridge between Camel components and Flink tasks. This component provides a way to route a message from various transports, dynamically choosing a flink task to execute, use an incoming message as input data for the task and finally deliver the results back to the Camel pipeline.

39.1. DEPENDENCIES

When using **camel-flink** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

Maven users will need to add the following dependency to their **pom.xml** for this component:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-flink-starter</artifactId>
</dependency>
```

39.2. URI FORMAT

Currently, the Flink Component supports only Producers. One can create DataSet, DataStream jobs.

```
flink:dataset?dataset=#myDataSet&dataSetCallback=#dataSetCallback
flink:datastream?datastream=#myDataStream&dataStreamCallback=#dataStreamCallback
```

39.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

39.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).

- directly in the Java code.

39.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

39.4. COMPONENT OPTIONS

The Flink component supports 5 options, which are listed below.

Name	Description	Default	Type
dataSetCallback (producer)	Function performing action against a DataSet.		DataSetCallback
dataStream (producer)	DataStream to compute against.		DataStream
dataStreamCallback (producer)	Function performing action against a DataStream.		DataStreamCallback
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

39.5. ENDPOINT OPTIONS

The Flink endpoint is configured using URI syntax:

```
flink:endpointType
```

With the following *path* and *query* parameters:

39.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
endpointType (producer)	Required Type of the endpoint (dataset, datastream). Enum values: - dataset - datastream		EndpointType

39.5.2. Query Parameters (6 parameters)

Name	Description	Default	Type
collect (producer)	Indicates if results should be collected or counted.	true	boolean
dataSet (producer)	DataSet to compute against.		DataSet
dataSetCallback (producer)	Function performing action against a DataSet.		DataSetCallback

Name	Description	Default	Type
dataStream (producer)	DataStream to compute against.		DataStream
dataStreamCallback (producer)	Function performing action against a DataStream.		DataStreamCallback
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

39.6. MESSAGE HEADERS

The Flink component supports 4 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelFlinkDataSet (producer) Constant: FLINK_DATASET_HEADER	The dataset.		Object
CamelFlinkDataSetCallback (producer) Constant: FLINK_DATASET_CALLBACK_HEADER	The dataset callback.		DataSetCallback
CamelFlinkDataStream (producer) Constant: FLINK_DATASTREAM_HEADER	The data stream.		Object

Name	Description	Default	Type
CamelFlinkDataStreamCallback (producer) Constant: FLINK_DATASTREAM_CALLBACK_HEADER	The data stream callback.		DataStreamCallback

39.7. FLINK DATASET CALLBACK

```

@Bean
public DataSetCallback<Long> dataSetCallback() {
    return new DataSetCallback<Long>() {
        public Long onDataSet(DataSet dataSet, Object... objects) {
            try {
                dataSet.print();
                return new Long(0);
            } catch (Exception e) {
                return new Long(-1);
            }
        }
    };
}

```

39.8. FLINK DATASTREAM CALLBACK

```

@Bean
public VoidDataStreamCallback dataStreamCallback() {
    return new VoidDataStreamCallback() {
        @Override
        public void doOnDataStream(DataStream dataStream, Object... objects) throws Exception {
            dataStream.flatMap(new Splitter()).print();

            environment.execute("data stream test");
        }
    };
}

```

39.9. CAMEL-FLINK PRODUCER CALL

```

CamelContext camelContext = new SpringCamelContext(context);

String pattern = "foo";

try {
    ProducerTemplate template = camelContext.createProducerTemplate();
    camelContext.start();
    Long count = template.requestBody("flink:dataSet?

```



```

dataSet=#myDataSet&dataSetCallback=#countLinesContaining", pattern, Long.class);
    } finally {
        camelContext.stop();
    }
}

```

39.10. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
camel.component.flink.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.flink.data-set-callback	Function performing action against a DataSet. The option is a <code>org.apache.camel.component.flink.DataSetCallback</code> type.		DataSetCallback
camel.component.flink.data-stream	DataStream to compute against. The option is a <code>org.apache.flink.streaming.api.datastream.DataStream</code> type.		DataStream
camel.component.flink.data-stream-callback	Function performing action against a DataStream. The option is a <code>org.apache.camel.component.flink.DataStreamCallback</code> type.		DataStreamCallback
camel.component.flink.enabled	Whether to enable auto configuration of the flink component. This is enabled by default.		Boolean
camel.component.flink.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 40. FTP

Both producer and consumer are supported

This component provides access to remote file systems over the FTP and SFTP protocols.

When consuming from remote FTP server make sure you read the section titled *Default when consuming files* further below for details related to consuming files.

Absolute path is **not** supported. Camel translates absolute path to relative by trimming all leading slashes from **directoryname**. There'll be WARN message printed in the logs.

40.1. DEPENDENCIES

When using **ftp** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-ftp-starter</artifactId>
</dependency>
```

40.2. URI FORMAT

```
ftp://[username@]hostname[:port]/directoryname[?options]
sftp://[username@]hostname[:port]/directoryname[?options]
ftps://[username@]hostname[:port]/directoryname[?options]
```

Where **directoryname** represents the underlying directory. The directory name is a relative path. Absolute path's is **not** supported. The relative path can contain nested folders, such as /inbox/us.

The **autoCreate** option is supported. When consumer starts, before polling is scheduled, there's additional FTP operation performed to create the directory configured for endpoint. The default value for **autoCreate** is **true**.

If no **username** is provided, then **anonymous** login is attempted using no password.

If no **port** number is provided, Camel will provide default values according to the protocol (ftp = 21, sftp = 22, ftps = 2222).

You can append query options to the URI in the following format, **?option=value&option=value&...**

This component uses two different libraries for the actual FTP work. FTP and FTPS uses [Apache Commons Net](#) while SFTP uses [JCraft JSCH](#).

FTPS (also known as FTP Secure) is an extension to FTP that adds support for the Transport Layer Security (TLS) and the Secure Sockets Layer (SSL) cryptographic protocols.

40.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level

- endpoint level

40.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

40.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

40.4. COMPONENT OPTIONS

The FTP component supports 3 options, which are listed below.

Name	Description	Default	Type
		t	

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

40.5. ENDPOINT OPTIONS

The FTP endpoint is configured using URI syntax:

```
ftp:host:port/directoryName
```

with the following path and query parameters:

40.5.1. Path Parameters (3 parameters)

Name	Description	Default	Type
host (common)	Required Hostname of the FTP server.		String
port (common)	Port of the FTP server.		int

Name	Description	Default	Type
directoryName (common)	The starting directory.		String

40.5.2. Query Parameters (111 parameters)

Name	Description	Default	Type
binary (common)	Specifies the file transfer mode, BINARY or ASCII. Default is ASCII (false).	false	boolean
charset (common)	This option is used to specify the encoding of the file. You can use this on the consumer, to specify the encodings of the files, which allow Camel to know the charset it should load the file content in case the file content is being accessed. Likewise when writing a file, you can use this option to specify which charset to write the file as well. Do mind that when writing the file Camel may have to read the message content into memory to be able to convert the data into the configured charset, so do not use this if you have big messages.		String
disconnect (common)	Whether or not to disconnect from remote FTP server right after use. Disconnect will only disconnect the current connection to the FTP server. If you have a consumer which you want to stop, then you need to stop the consumer/route instead.	false	boolean
doneFileName (common)	Producer: If provided, then Camel will write a 2nd done file when the original file has been written. The done file will be empty. This option configures what file name to use. Either you can specify a fixed name. Or you can use dynamic placeholders. The done file will always be written in the same folder as the original file. Consumer: If provided, Camel will only consume files if a done file exists. This option configures what file name to use. Either you can specify a fixed name. Or you can use dynamic placeholders. The done file is always expected in the same folder as the original file. Only <code>\${file.name}</code> and <code>\${file.name.next}</code> is supported as dynamic placeholders.		String

Name	Description	Default	Type
fileName (common)	Use Expression such as File Language to dynamically set the filename. For consumers, it's used as a filename filter. For producers, it's used to evaluate the filename to write. If an expression is set, it take precedence over the CamelFileName header. (Note: The header itself can also be an Expression). The expression options support both String and Expression types. If the expression is a String type, it is always evaluated using the File Language. If the expression is an Expression type, the specified Expression type is used - this allows you, for instance, to use OGNL expressions. For the consumer, you can use it to filter filenames, so you can for instance consume today's file using the File Language syntax: mydata-\${date:now:yyyyMMdd}.txt. The producers support the CamelOverrideFileName header which takes precedence over any existing CamelFileName header; the CamelOverrideFileName is a header that is used only once, and makes it easier as this avoids to temporary store CamelFileName and have to restore it afterwards.		String
passiveMode (common)	Sets passive mode connections. Default is active mode connections.	false	boolean
separator (common)	Sets the path separator to be used. UNIX = Uses unix style path separator Windows = Uses windows style path separator Auto = (is default) Use existing path separator in file name. Enum values: • UNIX • Windows • Auto	UNIX	PathSeparator
transferLoggingIntervalSeconds (common)	Configures the interval in seconds to use when logging the progress of upload and download operations that are in-flight. This is used for logging progress when operations takes longer time.	5	int

Name	Description	Default	Type
transferLoggingLevel (common)	Configure the logging level to use when logging the progress of upload and download operations. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	DEBUG	LogLevel
transferLoggingVerbose (common)	Configures whether the perform verbose (fine grained) logging of the progress of upload and download operations.	false	boolean
fastExistsCheck (common (advanced))	If set this option to be true, camel-ftp will use the list file directly to check if the file exists. Since some FTP server may not support to list the file directly, if the option is false, camel-ftp will use the old way to list the directory and check if the file exists. This option also influences readLock=changed to control whether it performs a fast check to update file information or not. This can be used to speed up the process if the FTP server has a lot of files.	false	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
delete (consumer)	If true, the file will be deleted after it is processed successfully.	false	boolean
moveFailed (consumer)	Sets the move failure expression based on Simple language. For example, to move files into a .error subdirectory use: .error. Note: When moving the files to the fail location Camel will handle the error and will not pick up the file again.		String

Name	Description	Default	Type
noop (consumer)	If true, the file is not moved or deleted in any way. This option is good for readonly data, or for ETL type requirements. If noop=true, Camel will set idempotent=true as well, to avoid consuming the same files over and over again.	false	boolean
preMove (consumer)	Expression (such as File Language) used to dynamically set the filename when moving it before processing. For example to move in-progress files into the order directory set this value to order.		String
preSort (consumer)	When pre-sort is enabled then the consumer will sort the file and directory names during polling, that was retrieved from the file system. You may want to do this in case you need to operate on the files in a sorted order. The pre-sort is executed before the consumer starts to filter, and accept files to process by Camel. This option is default=false meaning disabled.	false	boolean
recursive (consumer)	If a directory, will look for files in all the sub-directories as well.	false	boolean
resumeDownload (consumer)	Configures whether resume download is enabled. This must be supported by the FTP server (almost all FTP servers support it). In addition the options localWorkDirectory must be configured so downloaded files are stored in a local directory, and the option binary must be enabled, which is required to support resuming of downloads.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
streamDownload (consumer)	Sets the download method to use when not using a local working directory. If set to true, the remote files are streamed to the route as they are read. When set to false, the remote files are loaded into memory before being sent into the route. If enabling this option then you must set stepwise=false as both cannot be enabled at the same time.	false	boolean
download (consumer (advanced))	Whether the FTP consumer should download the file. If this option is set to false, then the message body will be null, but the consumer will still trigger a Camel Exchange that has details about the file such as file name, file size, etc. It's just that the file will not be downloaded.	false	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
handleDirectoryParserAbsoluteResult (consumer (advanced))	Allows you to set how the consumer will handle subfolders and files in the path if the directory parser results in with absolute paths. The reason for this is that some FTP servers may return file names with absolute paths, and if so then the FTP component needs to handle this by converting the returned path into a relative path.	<code>false</code>	<code>boolean</code>
ignoreFileNotFoundOrPermissionError (consumer (advanced))	Whether to ignore when (trying to list files in directories or when downloading a file), which does not exist or due to permission error. By default when a directory or file does not exist or insufficient permission, then an exception is thrown. Setting this option to <code>true</code> allows to ignore that instead.	<code>false</code>	<code>boolean</code>
inProgressRepository (consumer (advanced))	A pluggable in-progress repository <code>org.apache.camel.spi.IdempotentRepository</code> . The in-progress repository is used to account the current in progress files being consumed. By default a memory based repository is used.		<code>IdempotentRepository</code>
localWorkDirectory (consumer (advanced))	When consuming, a local work directory can be used to store the remote file content directly in local files, to avoid loading the content into memory. This is beneficial, if you consume a very big remote file and thus can conserve memory.		<code>String</code>

Name	Description	Default	Type
onCompletionExceptionHandler (consumer (advanced))	To use a custom <code>org.apache.camel.spi.ExceptionHandler</code> to handle any thrown exceptions that happens during the file on completion process where the consumer does either a commit or rollback. The default implementation will log any exception at WARN level and ignore.		ExceptionHandler
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
processStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.component.file.GenericFileProcessStrategy</code> allowing you to implement your own <code>readLock</code> option or similar. Can also be used when special conditions must be met before a file can be consumed, such as a special ready file exists. If this option is set then the <code>readLock</code> option does not apply.		GenericFileProcessStrategy
useList (consumer (advanced))	Whether to allow using LIST command when downloading a file. Default is true. In some use cases you may want to download a specific file and are not allowed to use the LIST command, and therefore you can set this option to false. Notice when using this option, then the specific file to download does not include meta-data information such as file size, timestamp, permissions etc, because those information is only possible to retrieve when LIST command is in use.	true	boolean

Name	Description	Default	Type
fileExist (producer)	What to do if a file already exists with the same name. Override, which is the default, replaces the existing file. – Append – adds content to the existing file. – Fail – throws a <code>GenericFileOperationException</code>, indicating that there is already an existing file. – Ignore – silently ignores the problem and does not override the existing file, but assumes everything is okay. – Move – option requires to use the <code>moveExisting</code> option to be configured as well. The option <code>eagerDeleteTargetFile</code> can be used to control what to do if an moving the file, and there exists already an existing file, otherwise causing the move operation to fail. The Move option will move any existing files, before writing the target file. – TryRename is only applicable if <code>tempFileName</code> option is in use. This allows to try renaming the file from the temporary name to the actual name, without doing any exists check. This check may be faster on some file systems and especially FTP servers. Enum values: • Override • Append • Fail • Ignore • Move • TryRename	Override	GenericFileExist
flatten (producer)	Flatten is used to flatten the file name path to strip any leading paths, so it's just the file name. This allows you to consume recursively into sub-directories, but when you eg write the files to another directory they will be written in a single directory. Setting this to true on the producer enforces that any file name in <code>CamelFileName</code> header will be stripped for any leading paths.	false	boolean
jailStartingDirectory (producer)	Used for jailing (restricting) writing files to the starting directory (and sub) only. This is enabled by default to not allow Camel to write files to outside directories (to be more secured out of the box). You can turn this off to allow writing files to directories outside the starting directory, such as parent or root folders.	true	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
moveExisting (producer)	Expression (such as File Language) used to compute file name to use when fileExist=Move is configured. To move files into a backup subdirectory just enter backup. This option only supports the following File Language tokens: file:name, file:name.ext, file:name.noext, file:onlyname, file:onlyname.noext, file:ext, and file:parent. Notice the file:parent is not supported by the FTP component, as the FTP component can only move any existing files to a relative directory based on current dir as base.		String
tempFileName (producer)	The same as tempPrefix option but offering a more fine grained control on the naming of the temporary filename as it uses the File Language. The location for tempFileName is relative to the final file location in the option 'fileName', not the target directory in the base uri. For example if option fileName includes a directory prefix: dir/finalFilename then tempFileName is relative to that subdirectory dir.		String
tempPrefix (producer)	This option is used to write the file using a temporary name and then, after the write is complete, rename it to the real name. Can be used to identify files being written and also avoid consumers (not using exclusive read locks) reading in progress files. Is often used by FTP when uploading big files.		String
allowNullBody (producer (advanced))	Used to specify if a null body is allowed during file writing. If set to true then an empty file will be created, when set to false, and attempting to send a null body to the file component, a GenericFileWriteException of 'Cannot write null body to file.' will be thrown. If the fileExist option is set to 'Override', then the file will be truncated, and if set to append the file will remain unchanged.	false	boolean

Name	Description	Default	Type
chmod (producer (advanced))	Allows you to set chmod on the stored file. For example chmod=640.		String
disconnectOnBatchComplete (producer (advanced))	Whether or not to disconnect from remote FTP server right after a Batch upload is complete. disconnectOnBatchComplete will only disconnect the current connection to the FTP server.	false	boolean
eagerDeleteTargetFile (producer (advanced))	Whether or not to eagerly delete any existing target file. This option only applies when you use fileExists=Override and the tempFileName option as well. You can use this to disable (set it to false) deleting the target file before the temp file is written. For example you may write big files and want the target file to exist during the temp file is being written. This ensures the target file is only deleted until the very last moment, just before the temp file is being renamed to the target filename. This option is also used to control whether to delete any existing files when fileExist=Move is enabled, and an existing file exists. If this option copyAndDeleteOnRenameFails false, then an exception will be thrown if an existing file existed, if its true, then the existing file is deleted before the move operation.	true	boolean
keepLastModified (producer (advanced))	Will keep the last modified timestamp from the source file (if any). Will use the Exchange.FILE_LAST_MODIFIED header to locate the timestamp. This header can contain either a java.util.Date or long with the timestamp. If the timestamp exists and the option is enabled it will set this timestamp on the written file. Note: This option only applies to the file producer. You cannot use this option with any of the ftp producers.	false	boolean
moveExistingFileStrategy (producer (advanced))	Strategy (Custom Strategy) used to move file with special naming token to use when fileExist=Move is configured. By default, there is an implementation used if no custom strategy is provided.		FileMoveExistingStrategy
sendNoop (producer (advanced))	Whether to send a noop command as a pre-write check before uploading files to the FTP server. This is enabled by default as a validation of the connection is still valid, which allows to silently re-connect to be able to upload the file. However if this causes problems, you can turn this option off.	true	boolean

Name	Description	Default	Type
activePortRange (advanced)	Set the client side port range in active mode. The syntax is: minPort-maxPort Both port numbers are inclusive, eg 10000-19999 to include all lxxxx ports.		String
autoCreate (advanced)	Automatically create missing directories in the file's pathname. For the file consumer, that means creating the starting directory. For the file producer, it means the directory the files should be written to.	true	boolean
bufferSize (advanced)	Buffer size in bytes used for writing files (or in case of FTP for downloading and uploading files).	131072	int
connectTimeout (advanced)	Sets the connect timeout for waiting for a connection to be established Used by both FTPClient and JSCH.	10000	int
ftpClient (advanced)	To use a custom instance of FTPClient.		FTPClient
ftpClientConfig (advanced)	To use a custom instance of FTPClientConfig to configure the FTP client the endpoint should use.		FTPClientConfig
ftpClientConfigParameters (advanced)	Used by FtpComponent to provide additional parameters for the FTPClientConfig.		Map
ftpClientParameters (advanced)	Used by FtpComponent to provide additional parameters for the FTPClient.		Map
maximumReconnectAttempts (advanced)	Specifies the maximum reconnect attempts Camel performs when it tries to connect to the remote FTP server. Use 0 to disable this behavior.		int
reconnectDelay (advanced)	Delay in millis Camel will wait before performing a reconnect attempt.	1000	long
siteCommand (advanced)	Sets optional site command(s) to be executed after successful login. Multiple site commands can be separated using a new line character.		String
soTimeout (advanced)	Sets the so timeout FTP and FTPS Is the SocketOptions.SO_TIMEOUT value in millis. Recommended option is to set this to 300000 so as not have a hanged connection. On SFTP this option is set as timeout on the JSCH Session instance.	300000	int

Name	Description	Default	Type
stepwise (advanced)	Sets whether we should stepwise change directories while traversing file structures when downloading files, or as well when uploading a file to a directory. You can disable this if you for example are in a situation where you cannot change directory on the FTP server due security reasons. Stepwise cannot be used together with streamDownload.	true	boolean
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean
throwExceptionOnConnectFailed (advanced)	Should an exception be thrown if connection failed (exhausted)By default exception is not thrown and a WARN is logged. You can use this to enable exception being thrown and handle the thrown exception from the org.apache.camel.spi.PollingConsumerPollStrategy rollback method.	false	boolean
timeout (advanced)	Sets the data timeout for waiting for reply Used only by FTPClient.	30000	int
antExclude (filter)	Ant style filter exclusion. If both antInclude and antExclude are used, antExclude takes precedence over antInclude. Multiple exclusions may be specified in comma-delimited format.		String
antFilterCaseSensitive (filter)	Sets case sensitive flag on ant filter.	true	boolean
antInclude (filter)	Ant style filter inclusion. Multiple inclusions may be specified in comma-delimited format.		String
eagerMaxMessagesPerPoll (filter)	Allows for controlling whether the limit from maxMessagesPerPoll is eager or not. If eager then the limit is during the scanning of files. Where as false would scan all files, and then perform sorting. Setting this option to false allows for sorting all files first, and then limit the poll. Mind that this requires a higher memory usage as all file details are in memory to perform the sorting.	true	boolean
exclude (filter)	Is used to exclude files, if filename matches the regex pattern (matching is case in-sensitive). Notice if you use symbols such as plus sign and others you would need to configure this using the RAW() syntax if configuring this as an endpoint uri. See more details at configuring endpoint uris.		String

Name	Description	Default	Type
excludeExt (filter)	Is used to exclude files matching file extension name (case insensitive). For example to exclude bak files, then use excludeExt=bak. Multiple extensions can be separated by comma, for example to exclude bak and dat files, use excludeExt=bak,dat. Note that the file extension includes all parts, for example having a file named mydata.tar.gz will have extension as tar.gz. For more flexibility then use the include/exclude options.		String
filter (filter)	Pluggable filter as a org.apache.camel.component.file.GenericFileFilter class. Will skip files if filter returns false in its accept() method.		GenericFileFilter
filterDirectory (filter)	Filters the directory based on Simple language. For example to filter on current date, you can use a simple date pattern such as <code>\$\$\{date:now:yyyMMdd}</code> .		String
filterFile (filter)	Filters the file based on Simple language. For example to filter on file size, you can use <code>\$\$\{file:size} 5000</code> .		String
idempotent (filter)	Option to use the Idempotent Consumer EIP pattern to let Camel skip already processed files. Will by default use a memory based LRUCache that holds 1000 entries. If noop=true then idempotent will be enabled as well to avoid consuming the same files over and over again.	false	Boolean
idempotentKey (filter)	To use a custom idempotent key. By default the absolute path of the file is used. You can use the File Language, for example to use the file name and file size, you can do: idempotentKey= <code>\$\$\{file:name}-\$\$\{file:size}</code> .		String
idempotentRepository (filter)	A pluggable repository org.apache.camel.spi.IdempotentRepository which by default use MemoryIdempotentRepository if none is specified and idempotent is true.		IdempotentRepository

Name	Description	Default	Type
include (filter)	Is used to include files, if filename matches the regex pattern (matching is case in-sensitive). Notice if you use symbols such as plus sign and others you would need to configure this using the RAW() syntax if configuring this as an endpoint uri. See more details at configuring endpoint uris.		String
includeExt (filter)	Is used to include files matching file extension name (case insensitive). For example to include txt files, then use includeExt=txt. Multiple extensions can be separated by comma, for example to include txt and xml files, use includeExt=txt,xml. Note that the file extension includes all parts, for example having a file named mydata.tar.gz will have extension as tar.gz. For more flexibility then use the include/exclude options.		String
maxDepth (filter)	The maximum depth to traverse when recursively processing a directory.	2147483647	int
maxMessagesPerPoll (filter)	To define a maximum messages to gather per poll. By default no maximum is set. Can be used to set a limit of e.g. 1000 to avoid when starting up the server that there are thousands of files. Set a value of 0 or negative to disabled it. Notice: If this option is in use then the File and FTP components will limit before any sorting. For example if you have 100000 files and use maxMessagesPerPoll=500, then only the first 500 files will be picked up, and then sorted. You can use the eagerMaxMessagesPerPoll option and set this to false to allow to scan all files first and then sort afterwards.		int
minDepth (filter)	The minimum depth to start processing when recursively processing a directory. Using minDepth=1 means the base directory. Using minDepth=2 means the first sub directory.		int
move (filter)	Expression (such as Simple Language) used to dynamically set the filename when moving it after processing. To move files into a .done subdirectory just enter .done.		String
exclusiveReadLockStrategy (lock)	Pluggable read-lock as a org.apache.camel.component.file.GenericFileExclusiveReadLockStrategy implementation.		GenericFileExclusiveReadLockStrategy

readLock (lock) Name	Description	none Default	String Type
	Used by consumer, to only poll the files if it has exclusive read-lock on the file (i.e. the file is not in-progress or being written). Camel will wait until the file lock is granted. This option provides the build in strategies: - none - No read lock is in use - markerFile - Camel creates a marker file (fileName.camellLock) and then holds a lock on it. This option is not available for the FTP component - changed - Changed is using file length/modification timestamp to detect whether the file is currently being copied or not. Will at least use 1 sec to determine this, so this option cannot consume files as fast as the others, but can be more reliable as the JDK IO API cannot always determine whether a file is currently being used by another process. The option readLockCheckInterval can be used to set the check frequency. - fileLock - is for using java.nio.channels.FileLock. This option is not avail for Windows OS and the FTP component. This approach should be avoided when accessing a remote file system via a mount/share unless that file system supports distributed file locks. - rename - rename is for using a try to rename the file as a test if we can get exclusive read-lock. - idempotent - (only for file component) idempotent is for using a idempotentRepository as the read-lock. This allows to use read locks that supports clustering if the idempotent repository implementation supports that. - idempotent-changed - (only for file component) idempotent-changed is for using a idempotentRepository and changed as the combined read-lock. This allows to use read locks that supports clustering if the idempotent repository implementation supports that. - idempotent-rename - (only for file component) idempotent-rename is for using a idempotentRepository and rename as the combined read-lock. This allows to use read locks that supports clustering if the idempotent repository implementation supports that. Notice: The various read locks is not all suited to work in clustered mode, where concurrent consumers on different nodes is competing for the same files on a shared file system. The markerFile using a close to atomic operation to create the empty marker file, but its not guaranteed to work in a cluster. The fileLock may work better but then the file system need to support distributed file locks, and so on. Using the idempotent read lock can support clustering if the idempotent repository supports clustering, such as Hazelcast Component or Infinispan. Enum values: ● none ● markerFile ● fileLock		

Name	● rename Description ● changed	Default	Type
	● idempotent ● idempotent-changed ● idempotent-rename		
readLockCheckInterval (lock)	Interval in millis for the read-lock, if supported by the read lock. This interval is used for sleeping between attempts to acquire the read lock. For example when using the changed read lock, you can set a higher interval period to cater for slow writes. The default of 1 sec. may be too fast if the producer is very slow writing the file. Notice: For FTP the default readLockCheckInterval is 5000. The readLockTimeout value must be higher than readLockCheckInterval, but a rule of thumb is to have a timeout that is at least 2 or more times higher than the readLockCheckInterval. This is needed to ensure that ample time is allowed for the read lock process to try to grab the lock before the timeout was hit.	1000	long
readLockDeleteOrphanLockFiles (lock)	Whether or not read lock with marker files should upon startup delete any orphan read lock files, which may have been left on the file system, if Camel was not properly shutdown (such as a JVM crash). If turning this option to false then any orphaned lock file will cause Camel to not attempt to pickup that file, this could also be due another node is concurrently reading files from the same shared directory.	true	boolean
readLockLoggingLevel (lock)	Logging level used when a read lock could not be acquired. By default a DEBUG is logged. You can change this level, for example to OFF to not have any logging. This option is only applicable for readLock of types: changed, fileLock, idempotent, idempotent-changed, idempotent-rename, rename. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	DEBUG	LoggingLevel

Name	Description	Default	Type
readLockMarkerFile (lock)	Whether to use marker file with the changed, rename, or exclusive read lock types. By default a marker file is used as well to guard against other processes picking up the same files. This behavior can be turned off by setting this option to false. For example if you do not want to write marker files to the file systems by the Camel application.	true	boolean
readLockMinAge (lock)	This option is applied only for readLock=changed. It allows to specify a minimum age the file must be before attempting to acquire the read lock. For example use readLockMinAge=300s to require the file is at last 5 minutes old. This can speedup the changed read lock as it will only attempt to acquire files which are at least that given age.	0	long
readLockMinLength (lock)	This option is applied only for readLock=changed. It allows you to configure a minimum file length. By default Camel expects the file to contain data, and thus the default value is 1. You can set this option to zero, to allow consuming zero-length files.	1	long
readLockRemoveOnCommit (lock)	This option is applied only for readLock=idempotent. It allows to specify whether to remove the file name entry from the idempotent repository when processing the file is succeeded and a commit happens. By default the file is not removed which ensures that any race-condition do not occur so another active node may attempt to grab the file. Instead the idempotent repository may support eviction strategies that you can configure to evict the file name entry after X minutes - this ensures no problems with race conditions. See more details at the readLockIdempotentReleaseDelay option.	false	boolean
readLockRemoveOnRollback (lock)	This option is applied only for readLock=idempotent. It allows to specify whether to remove the file name entry from the idempotent repository when processing the file failed and a rollback happens. If this option is false, then the file name entry is confirmed (as if the file did a commit).	true	boolean

Name	Description	Default	Type
readLockTimeout (lock)	Optional timeout in millis for the read-lock, if supported by the read-lock. If the read-lock could not be granted and the timeout triggered, then Camel will skip the file. At next poll Camel, will try the file again, and this time maybe the read-lock could be granted. Use a value of 0 or lower to indicate forever. Currently fileLock, changed and rename support the timeout. Notice: For FTP the default readLockTimeout value is 20000 instead of 10000. The readLockTimeout value must be higher than readLockCheckInterval, but a rule of thumb is to have a timeout that is at least 2 or more times higher than the readLockCheckInterval. This is needed to ensure that ample time is allowed for the read lock process to try to grab the lock before the timeout was hit.	10000	long
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long

Name	Description	Default	Type
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit

Name	Description	Default	Type
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See <code>ScheduledExecutorService</code> in JDK for details.	true	boolean
account (security)	Account to use for login.		String
password (security)	Password to use for login.		String
username (security)	Username to use for login.		String
shuffle (sort)	To shuffle the list of files (sort in random order).	false	boolean
sortBy (sort)	Built-in sort by using the File Language. Supports nested sorts, so you can have a sort by file name and as a 2nd group sort by modified date.		String
sorter (sort)	Pluggable sorter as a <code>java.util.Comparator</code> class.		Comparator

40.6. FTPS COMPONENT DEFAULT TRUST STORE

When using the **ftpClient**, properties related to SSL with the FTPS component, the trust store accept all certificates. If you only want trust selective certificates, you have to configure the trust store with the **ftpClient.trustStore.xxx** options or by configuring a custom **ftpClient**.

When using **sslContextParameters**, the trust store is managed by the configuration of the provided `SSLContextParameters` instance.

You can configure additional options on the **ftpClient** and **ftpClientConfig** from the URI directly by using the **ftpClient.** or **ftpClientConfig.** prefix.

For example to set the **setDataTimeout** on the **FTPClient** to 30 seconds you can do:

```
from("ftp://foo@myserver?password=secret&ftpClient.dataTimeout=30000").to("bean:foo");
```

You can mix and match and have use both prefixes, for example to configure date format or timezones.

```
from("ftp://foo@myserver?password=secret&ftpClient.dataTimeout=30000&ftpClientConfig.serverLanguageCode=fr").to("bean:foo");
```

You can have as many of these options as you like.

See the documentation of the Apache Commons FTP `FTPClientConfig` for possible options and more details. And as well for Apache Commons FTP `FTPClient`.

If you do not like having many and long configuration in the url you can refer to the **ftpClient** or **ftpClientConfig** to use by letting Camel lookup in the Registry for it.

For example:

```
<bean id="myConfig" class="org.apache.commons.net.ftp.FTPClientConfig">
  <property name="lenientFutureDates" value="true"/>
  <property name="serverLanguageCode" value="fr"/>
</bean>
```

And then let Camel lookup this bean when you use the # notation in the url.

```
from("ftp://foo@myserver?password=secret&ftpClientConfig=#myConfig").to("bean:foo");
```

40.7. EXAMPLES

```
ftp://someone@someftpserver.com/public/upload/images/holiday2008?password=secret&binary=true
ftp://someoneelse@someotherftpserver.co.uk:12049/reports/2008/password=secret&binary=false
ftp://publicftpserver.com/download
```

40.8. CONCURRENCY

FTP Consumer does not support concurrency

The FTP consumer (with the same endpoint) does not support concurrency (the backing FTP client is not thread safe).

You can use multiple FTP consumers to poll from different endpoints. It is only a single endpoint that does not support concurrent consumers.

The FTP producer does **not** have this issue, it supports concurrency.

40.9. MORE INFORMATION

This component is an extension of the File component. So there are more samples and details on the File component page.

40.10. DEFAULT WHEN CONSUMING FILES

The FTP consumer will by default leave the consumed files untouched on the remote FTP server. You have to configure it explicitly if you want it to delete the files or move them to another location. For example you can use **delete=true** to delete the files, or use **move=.done** to move the files into a hidden done sub directory.

The regular File consumer is different as it will by default move files to a **.camel** sub directory. The reason Camel does **not** do this by default for the FTP consumer is that it may lack permissions by default to be able to move or delete files.

40.10.1. limitations

The option **readLock** can be used to force Camel **not** to consume files that is currently in the progress of being written. However, this option is turned off by default, as it requires that the user has write access. See the options table at File2 for more details about read locks.

There are other solutions to avoid consuming files that are currently being written over FTP; for instance, you can write to a temporary destination and move the file after it has been written.

When moving files using **move** or **preMove** option the files are restricted to the FTP_ROOT folder. That prevents you from moving files outside the FTP area. If you want to move files to another area you can use soft links and move files into a soft linked folder.

40.11. MESSAGE HEADERS

The following message headers can be used to affect the behavior of the component

Header	Description
CamelFileName	Specifies the output file name (relative to the endpoint directory) to be used for the output message when sending to the endpoint. If this is not present and no expression either, then a generated message ID is used as the filename instead.
CamelFileNameProduced	The actual filepath (path + name) for the output file that was written. This header is set by Camel and its purpose is providing end-users the name of the file that was written.
CamelFileNameConsumed	The file name of the file consumed
CamelFileHost	The remote hostname.
CamelFileLocalWorkPath	Path to the local work file, if local work directory is used.

In addition the FTP/FTPS consumer and producer will enrich the Camel **Message** with the following headers

Header	Description
CamelFtpReplyCode	The FTP client reply code (the type is a integer)
CamelFtpReplyString	The FTP client reply string

40.11.1. Exchange Properties

Camel sets the following exchange properties

Header	Description
CamelBatchIndex	Current index out of total number of files being consumed in this batch.
CamelBatchSize	Total number of files being consumed in this batch.

Header	Description
CamelBatchComplete	True if there are no more files in this batch.

40.12. ABOUT TIMEOUTS

The two set of libraries (see top) has different API for setting timeout. You can use the **connectTimeout** option for both of them to set a timeout in millis to establish a network connection. An individual **soTimeout** can also be set on the FTP/FTPS, which corresponds to using **ftpClient.soTimeout**. Notice SFTP will automatically use **connectTimeout** as its **soTimeout**. The **timeout** option only applies for FTP/FTPS as the data timeout, which corresponds to the **ftpClient.dataTimeout** value. All timeout values are in millis.

40.13. USING LOCAL WORK DIRECTORY

Camel supports consuming from remote FTP servers and downloading the files directly into a local work directory. This avoids reading the entire remote file content into memory as it is streamed directly into the local file using **FileOutputStream**.

Camel will store to a local file with the same name as the remote file, though with **.inprogress** as extension while the file is being downloaded. Afterwards, the file is renamed to remove the **.inprogress** suffix. And finally, when the Exchange is complete the local file is deleted.

So if you want to download files from a remote FTP server and store it as files then you need to route to a file endpoint such as:

```
from("ftp://someone@someserver.com?password=secret&localWorkDirectory=/tmp").to("file://inbox");
```



NOTE

The route above is ultra efficient as it avoids reading the entire file content into memory. It will download the remote file directly to a local file stream. The **java.io.File** handle is then used as the Exchange body. The file producer leverages this fact and can work directly on the work file **java.io.File** handle and perform a **java.io.File.rename** to the target filename. As Camel knows it's a local work file, it can optimize and use a rename instead of a file copy, as the work file is meant to be deleted anyway.

40.14. STEPWISE CHANGING DIRECTORIES

Camel FTP can operate in two modes in terms of traversing directories when consuming files (eg downloading) or producing files (eg uploading)

- stepwise
- not stepwise

You may want to pick either one depending on your situation and security issues. Some Camel end users can only download files if they use stepwise, while others can only download if they do not.

You can use the **stepwise** option to control the behavior.

Note that stepwise changing of directory will in most cases only work when the user is confined to it's home directory and when the home directory is reported as "/".

The difference between the two of them is best illustrated with an example. Suppose we have the following directory structure on the remote FTP server we need to traverse and download files:

```
/
/one
/one/two
/one/two/sub-a
/one/two/sub-b
```

And that we have a file in each of sub-a (a.txt) and sub-b (b.txt) folder.

40.15. USING STEPWISE=TRUE (DEFAULT MODE)

```
TYPE A
200 Type set to A
PWD
257 "/" is current directory.
CWD one
250 CWD successful. "/one" is current directory.
CWD two
250 CWD successful. "/one/two" is current directory.
SYST
215 UNIX emulated by FileZilla
PORT 127,0,0,1,17,94
200 Port command successful
LIST
150 Opening data channel for directory list.
226 Transfer OK
CWD sub-a
250 CWD successful. "/one/two/sub-a" is current directory.
PORT 127,0,0,1,17,95
200 Port command successful
LIST
150 Opening data channel for directory list.
226 Transfer OK
CDUP
200 CDUP successful. "/one/two" is current directory.
CWD sub-b
250 CWD successful. "/one/two/sub-b" is current directory.
PORT 127,0,0,1,17,96
200 Port command successful
LIST
150 Opening data channel for directory list.
226 Transfer OK
CDUP
200 CDUP successful. "/one/two" is current directory.
CWD /
250 CWD successful. "/" is current directory.
PWD
257 "/" is current directory.
CWD one
250 CWD successful. "/one" is current directory.
```

```
CWD two
250 CWD successful. "/one/two" is current directory.
PORT 127,0,0,1,17,97
200 Port command successful
RETR foo.txt
150 Opening data channel for file transfer.
226 Transfer OK
CWD /
250 CWD successful. "/" is current directory.
PWD
257 "/" is current directory.
CWD one
250 CWD successful. "/one" is current directory.
CWD two
250 CWD successful. "/one/two" is current directory.
CWD sub-a
250 CWD successful. "/one/two/sub-a" is current directory.
PORT 127,0,0,1,17,98
200 Port command successful
RETR a.txt
150 Opening data channel for file transfer.
226 Transfer OK
CWD /
250 CWD successful. "/" is current directory.
PWD
257 "/" is current directory.
CWD one
250 CWD successful. "/one" is current directory.
CWD two
250 CWD successful. "/one/two" is current directory.
CWD sub-b
250 CWD successful. "/one/two/sub-b" is current directory.
PORT 127,0,0,1,17,99
200 Port command successful
RETR b.txt
150 Opening data channel for file transfer.
226 Transfer OK
CWD /
250 CWD successful. "/" is current directory.
QUIT
221 Goodbye
disconnected.
```

As you can see when stepwise is enabled, it will traverse the directory structure using CD xxx.

40.16. USING STEPWISE=FALSE

```
230 Logged on
TYPE A
200 Type set to A
SYST
215 UNIX emulated by FileZilla
PORT 127,0,0,1,4,122
200 Port command successful
LIST one/two
```

```

150 Opening data channel for directory list
226 Transfer OK
PORT 127,0,0,1,4,123
200 Port command successful
LIST one/two/sub-a
150 Opening data channel for directory list
226 Transfer OK
PORT 127,0,0,1,4,124
200 Port command successful
LIST one/two/sub-b
150 Opening data channel for directory list
226 Transfer OK
PORT 127,0,0,1,4,125
200 Port command successful
RETR one/two/foo.txt
150 Opening data channel for file transfer.
226 Transfer OK
PORT 127,0,0,1,4,126
200 Port command successful
RETR one/two/sub-a/a.txt
150 Opening data channel for file transfer.
226 Transfer OK
PORT 127,0,0,1,4,127
200 Port command successful
RETR one/two/sub-b/b.txt
150 Opening data channel for file transfer.
226 Transfer OK
QUIT
221 Goodbye
disconnected.

```

As you can see when not using stepwise, there are no CD operation invoked at all.

40.17. SAMPLES

In the sample below we set up Camel to download all the reports from the FTP server once every hour (60 min) as BINARY content and store it as files on the local file system.

And the route using XML DSL:

```

<route>
  <from uri="ftp://scott@localhost/public/reports?
password=tiger&binary=true&delay=60000"/>
  <to uri="file://target/test-reports"/>
</route>

```

40.17.1. Consuming a remote FTPS server (implicit SSL) and client authentication

```

from("ftps://admin@localhost:2222/public/camel?
password=admin&securityProtocol=SSL&implicit=true
  &ftpClient.keyStore.file=./src/test/resources/server.jks
  &ftpClient.keyStore.password=password&ftpClient.keyStore.keyPassword=password")
.to("bean:foo");

```

40.17.2. Consuming a remote FTPS server (explicit TLS) and a custom trust store configuration

```
from("ftps://admin@localhost:2222/public/camel?
password=admin&ftpClient.trustStore.file=./src/test/resources/server.jks&ftpClient.trustStore.password=
password")
.to("bean:foo");
```

40.18. CUSTOM FILTERING

Camel supports pluggable filtering strategies. This strategy it to use the build in **org.apache.camel.component.file.GenericFileFilter** in Java. You can then configure the endpoint with such a filter to skip certain filters before being processed.

In the sample we have built our own filter that only accepts files starting with report in the filename.

And then we can configure our route using the **filter** attribute to reference our filter (using # notation) that we have defined in the spring XML file:

```
<!-- define our sorter as a plain spring bean -->
<bean id="myFilter" class="com.mycompany.MyFileFilter"/>

<route>
  <from uri="ftp://someuser@someftpserver.com?password=secret&filter=#myFilter"/>
  <to uri="bean:processInbox"/>
</route>
```

40.19. FILTERING USING ANT PATH MATCHER

The ANT path matcher is a filter that is shipped out-of-the-box in the **camel-spring** jar. So you need to depend on **camel-spring** if you are using Maven.

The reason is that we leverage Spring's [AntPathMatcher](#) to do the actual matching.

The file paths are matched with the following rules:

- **?** matches one character
- ***** matches zero or more characters
- ****** matches zero or more directories in a path

The sample below demonstrates how to use it:

40.20. USING A PROXY WITH SFTP

To use an HTTP proxy to connect to your remote host, you can configure your route in the following way:

```
<!-- define our sorter as a plain spring bean -->
<bean id="proxy" class="com.jcraft.jsch.ProxyHTTP">
  <constructor-arg value="localhost"/>
  <constructor-arg value="7777"/>
</bean>
```

```
<route>
  <from uri="sftp://localhost:9999/root?username=admin&password=admin&proxy=#proxy"/>
  <to uri="bean:processFile"/>
</route>
```

You can also assign a user name and password to the proxy, if necessary. Please consult the documentation for **com.jcraft.jsch.Proxy** to discover all options.

40.21. SETTING PREFERRED SFTP AUTHENTICATION METHOD

If you want to explicitly specify the list of authentication methods that should be used by **sftp** component, use **preferredAuthentications** option. If for example you would like Camel to attempt to authenticate with private/public SSH key and fallback to user/password authentication in the case when no public key is available, use the following route configuration:

```
from("sftp://localhost:9999/root?
username=admin&password=admin&preferredAuthentications=publickey,password").
to("bean:processFile");
```

40.22. CONSUMING A SINGLE FILE USING A FIXED NAME

When you want to download a single file and knows the file name, you can use **fileName=myFileName.txt** to tell Camel the name of the file to download. By default the consumer will still do a FTP LIST command to do a directory listing and then filter these files based on the **fileName** option. Though in this use-case it may be desirable to turn off the directory listing by setting **useList=false**. For example the user account used to login to the FTP server may not have permission to do a FTP LIST command. So you can turn off this with **useList=false**, and then provide the fixed name of the file to download with **fileName=myFileName.txt**, then the FTP consumer can still download the file. If the file for some reason does not exist, then Camel will by default throw an exception, you can turn this off and ignore this by setting **ignoreFileNotFoundOrPermissionError=true**.

For example to have a Camel route that pickup a single file, and delete it after use you can do

```
from("ftp://admin@localhost:21/nolist/?
password=admin&stepwise=false&useList=false&ignoreFileNotFoundOrPermissionError=true&fileName
=report.txt&delete=true")
.to("activemq:queue:report");
```

Notice that we have used all the options we talked above.

You can also use this with **ConsumerTemplate**. For example to download a single file (if it exists) and grab the file content as a String type:

```
String data = template.retrieveBodyNoWait("ftp://admin@localhost:21/nolist/?
password=admin&stepwise=false&useList=false&ignoreFileNotFoundOrPermissionError=true&fileName
=report.txt&delete=true", String.class);
```

40.23. DEBUG LOGGING

This component has log level **TRACE** that can be helpful if you have problems.

40.24. SPRING BOOT AUTO-CONFIGURATION

The component supports 13 options, which are listed below.

Name	Description	Default	Type
camel.component.ftp.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.ftp.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.ftp.enabled	Whether to enable auto configuration of the ftp component. This is enabled by default.		Boolean
camel.component.ftp.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.ftp.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.ftp.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.ftp.enabled	Whether to enable auto configuration of the ftps component. This is enabled by default.		Boolean
camel.component.ftp.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.ftp.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean
camel.component.sftp.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.sftp.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.sftp.enabled	Whether to enable auto configuration of the sftp component. This is enabled by default.		Boolean
camel.component.sftp.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 41. GOOGLE BIGQUERY

Since Camel 2.20

Only producer is supported.

The Google Bigquery component provides access to the [Cloud BigQuery Infrastructure](https://developers.google.com/api-client-library/java/apis/bigquery/v2) via the link: <https://developers.google.com/api-client-library/java/apis/bigquery/v2> [Google Client Services API].

The current implementation does not use gRPC.

The current implementation does not support querying BigQuery, it is only a producer.

41.1. DEPENDENCIES

When using **google-bigquery** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-google-bigquery-starter</artifactId>
</dependency>
```

41.2. AUTHENTICATION CONFIGURATION

Google BigQuery component authentication is targeted for use with the GCP Service Accounts. For more information please refer to [Google Cloud Platform Auth Guide](#).

Google security credentials can be set explicitly by providing the path to the GCP credentials file location.

Or they are set implicitly, where the connection factory falls back on [Application Default Credentials](#).

When you have the **service account key** you can provide authentication credentials to your application code. Google security credentials can be set through the component endpoint:

```
String endpoint = "google-bigquery://project-id:datasetId[:tableId]?
serviceAccountKey=/home/user/Downloads/my-key.json";
```

You can also use the base64 encoded content of the authentication credentials file if you don't want to set a file system path.

```
String endpoint = "google-bigquery://project-id:datasetId[:tableId]?serviceAccountKey=base64:
<base64 encoded>";
```

Or by setting the environment variable **GOOGLE_APPLICATION_CREDENTIALS**:

```
export GOOGLE_APPLICATION_CREDENTIALS="/home/user/Downloads/my-key.json"
```

41.3. URI FORMAT

```
google-bigquery://project-id:datasetId[:tableId]?[options]
```

41.4. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

41.4.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

41.4.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

41.5. COMPONENT OPTIONS

The Google BigQuery component supports 5 options, which are listed below.

Name	Description	Default	Type
connectionFactory (producer)	Autowired ConnectionFactory to obtain connection to Bigquery Service. If not provided the default one will be used.		GoogleBigQueryConnectionFactory
datasetId (producer)	BigQuery Dataset Id.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
projectId (producer)	Google Cloud Project Id.		String
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

41.6. ENDPOINT OPTIONS

The Google BigQuery endpoint is configured using URI syntax:

```
google-bigquery:projectId:datasetId:tableId
```

with the following path and query parameters:

41.6.1. Path Parameters (3 parameters)

Name	Description	Default	Type
projectId (common)	Required Google Cloud Project Id.		String

Name	Description	Default	Type
datasetId (common)	Required BigQuery Dataset Id.		String
tableId (common)	BigQuery table id.		String

41.6.2. Query Parameters (4 parameters)

Name	Description	Default	Type
connectionFactory (producer)	Autowired ConnectionFactory to obtain connection to Bigquery Service. If not provided the default one will be used.		GoogleBigQueryConnectionFactory
useAsInsertId (producer)	Field name to use as insert id.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
serviceAccountKey (security)	Service account key in json format to authenticate an application as a service account to google cloud platform.		String

41.7. MESSAGE HEADERS

The Google BigQuery component supports 4 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelGoogleBigQueryTableSuffix (producer) Constant: TABLE_SUFFIX	Table suffix to use when inserting data.		String

Name	Description	Default	Type
CamelGoogleBigQueryTableId (producer) Constant: TABLE_ID	Table id where data will be submitted. If specified will override endpoint configuration.		String
CamelGoogleBigQueryInsertId (producer) Constant: INSERT_ID	InsertId to use when inserting data.		String
CamelGoogleBigQueryPartitionDecorator (producer) Constant: PARTITION_DECORATOR	Partition decorator to indicate partition to use when inserting data.		String

41.8. PRODUCER ENDPOINTS

Producer endpoints can accept and deliver to BigQuery individual and grouped exchanges alike. Grouped exchanges have **Exchange.GROUPED_EXCHANGE** property set.

Google BigQuery producer will send a grouped exchange in a single api call unless different table suffix or partition decorators are specified in which case it will break it down to ensure data is written with the correct suffix or partition decorator.

Google BigQuery endpoint expects the payload to be either a map or list of maps. A payload containing a map will insert a single row and a payload containing a list of map's will insert a row for each entry in the list.

41.9. TEMPLATE TABLES

Templated tables can be specified using the **GoogleBigQueryConstants.TABLE_SUFFIX** header. For example, the following route will create tables and insert records sharded on a per day basis:

```
from("direct:start")
  .header(GoogleBigQueryConstants.TABLE_SUFFIX, "_${date:now:yyyyMMdd}")
  .to("google-bigquery:sampleDataset:sampleTable")
```



NOTE

It is recommended to use partitioning for this use case.

For more information about Template table, see [Template Tables](#).

41.10. PARTITIONING

Partitioning is specified when creating a table and if set data will be automatically partitioned into separate tables. When inserting data a specific partition can be specified by setting the **GoogleBigQueryConstants.PARTITION_DECORATOR** header on the exchange.

For more information about partitioning, see [Creating partitioned tables](#).

41.11. ENSURING DATA CONSISTENCY

An insert id can be set on the exchange with the header **GoogleBigQueryConstants.INSERT_ID** or by specifying query parameter **useAsInsertId**. As an insert id need to be specified per row, the inserted the exchange header can't be used when the payload is a list. If the payload is a list then the **GoogleBigQueryConstants.INSERT_ID** will be ignored. In that case use the query parameter **useAsInsertId**.

For more information, see [Data consistency](#)

41.12. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.google-bigquery-sql.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.google-bigquery-sql.connection-factory</code>	ConnectionFactory to obtain connection to Bigquery Service. If not provided the default one will be used. The option is a <code>org.apache.camel.component.google.bigquery.GoogleBigQueryConnectionFactory</code> type.		GoogleBigQueryConnectionFactory
<code>camel.component.google-bigquery-sql.enabled</code>	Whether to enable auto configuration of the google-bigquery-sql component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.google-bigquery-sql.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.google-bigquery-sql.project-id	Google Cloud Project Id.		String
camel.component.google-bigquery.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.google-bigquery.connection-factory	ConnectionFactory to obtain connection to Bigquery Service. If not provided the default one will be used. The option is a org.apache.camel.component.google.bigquery.GoogleBigQueryConnectionFactory type.		GoogleBigQueryConnectionFactory
camel.component.google-bigquery.dataset-id	BigQuery Dataset Id.		String
camel.component.google-bigquery.enabled	Whether to enable auto configuration of the google-bigquery component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.google-bigquery.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.google-bigquery.project-id</code>	Google Cloud Project Id.		String

CHAPTER 42. GOOGLE PUBSUB

Since Camel 2.19

Both producer and consumer are supported.

The Google Pubsub component provides access to the [Cloud Pub/Sub Infrastructure](#) via the [Google Cloud Java Client for Google Cloud Pub/Sub](#).

42.1. DEPENDENCIES

When using **google-pubsub** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-google-pubsub-starter</artifactId>
</dependency>
```

42.2. URI FORMAT

The Google Pubsub Component uses the following URI format:

```
google-pubsub://project-id:destinationName?[options]
```

Destination Name can be either a topic or a subscription name.

42.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

42.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

42.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

42.4. COMPONENT OPTIONS

The Google Pubsub component supports 10 options, which are listed below.

Name	Description	Default	Type
authenticate (common)	Use Credentials when interacting with PubSub service (no authentication is required when using emulator).	true	boolean
endpoint (common)	Endpoint to use with local Pub/Sub emulator.		String
serviceAccountKey (common)	The Service account key that can be used as credentials for the PubSub publisher/subscriber. It can be loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
synchronousPullRetryableCodes (consumer)	Comma-separated list of additional retryable error codes for synchronous pull. By default the PubSub client library retries ABORTED, UNAVAILABLE, UNKNOWN.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
publisherCacheSize (producer)	Maximum number of producers to cache. This could be increased if you have producers for lots of different topics.		int
publisherCacheTimeout (producer)	How many milliseconds should each producer stay alive in the cache.		int
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
publisherTerminationTimeout (advanced)	How many milliseconds should a producer be allowed to terminate.		int

42.5. ENDPOINT OPTIONS

The Google Pubsub endpoint is configured using URI syntax:

```
google-pubsub:projectId:destinationName
```

with the following path and query parameters:

42.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
projectId (common)	Required The Google Cloud PubSub Project Id.		String
destinationName (common)	Required The Destination Name. For the consumer this will be the subscription name, while for the producer this will be the topic name.		String

42.5.2. Query Parameters (15 parameters)

Name	Description	Default	Type
authenticate (common)	Use Credentials when interacting with PubSub service (no authentication is required when using emulator).	true	boolean
loggerId (common)	Logger ID to use when a match to the parent route required.		String
serviceAccountKey (common)	The Service account key that can be used as credentials for the PubSub publisher/subscriber. It can be loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
ackMode (consumer)	AUTO = exchange gets ack'ed/nack'ed on completion. NONE = downstream process has to ack/nack explicitly. Enum values: ● AUTO ● NONE	AUTO	AckMode
concurrentConsumers (consumer)	The number of parallel streams consuming from the subscription.	1	Integer
maxAckExtensionPeriod (consumer)	Set the maximum period a message ack deadline will be extended. Value in seconds.	3600	int
maxMessagesPerPoll (consumer)	The max number of messages to receive from the server in a single API call.	1	Integer
synchronousPull (consumer)	Synchronously pull batches of messages.	false	boolean

Name	Description	Default	Type
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
messageOrderingEnabled (producer (advanced))	Should message ordering be enabled.	false	boolean
pubsubEndpoint (producer (advanced))	Pub/Sub endpoint to use. Required when using message ordering, and ensures that messages are received in order even when multiple publishers are used.		String

Name	Description	Default	Type
serializer (producer (advanced))	Autowired A custom GooglePubsubSerializer to use for serializing message payloads in the producer.		GooglePubsubSerializer

42.6. MESSAGE HEADERS

The Google Pubsub component supports 5 message header(s), which are listed below:

Name	Description	Default	Type
CamelGooglePubsubMessageId (common) Constant: MESSAGE_ID	The ID of the message, assigned by the server when the message is published.		String
CamelGooglePubsubMsgAckId (consumer) Constant: ACK_ID	The ID used to acknowledge the received message.		String
CamelGooglePubsubPublishTime (consumer) Constant: PUBLISH_TIME	The time at which the message was published.		Timestamp
CamelGooglePubsubAttributes (common) Constant: ATTRIBUTES	The attributes of the message.		Map
CamelGooglePubsubOrderingKey (producer) Constant: ORDERING_KEY	If non-empty, identifies related messages for which publish order should be respected.		String

42.7. PRODUCER ENDPOINTS

Producer endpoints can accept and deliver to PubSub individual and grouped exchanges alike. Grouped exchanges have **Exchange.GROUPED_EXCHANGE** property set.

Google PubSub expects the payload to be byte[] array, Producer endpoints will send:

- String body as byte[] encoded as UTF-8
- byte[] body as is
- Everything else will be serialised into byte[] array

A Map set as message header **GooglePubsubConstants.ATTRIBUTES** will be sent as PubSub attributes.

Google PubSub supports ordered message delivery.

To enable this set the options `messageOrderingEnabled` to true, and the `pubsubEndpoint` to a GCP region.

When producing messages set the message header **GooglePubsubConstants.ORDERING_KEY**. This will be set as the PubSub `orderingKey` for the message.

More information in [Ordering messages](#).

Once exchange has been delivered to PubSub the PubSub Message ID will be assigned to the header **GooglePubsubConstants.MESSAGE_ID**.

42.8. CONSUMER ENDPOINTS

Google PubSub will redeliver the message if it has not been acknowledged within the time period set as a configuration option on the subscription.

The component will acknowledge the message once exchange processing has been completed.

If the route throws an exception, the exchange is marked as failed and the component will NACK the message - it will be redelivered immediately.

To ack/nack the message the component uses Acknowledgement ID stored as header **GooglePubsubConstants.ACK_ID**. If the header is removed or tampered with, the ack will fail and the message will be redelivered again after the ack deadline.

42.9. MESSAGE BODY

The consumer endpoint returns the content of the message as byte[] - exactly as the underlying system sends it. It is up for the route to convert/unmarshall the contents.

42.10. AUTHENTICATION CONFIGURATION

By default this component acquires credentials using **GoogleCredentials.getDefault()**. This behavior can be disabled by setting *authenticate* option to **false**, in which case requests to Google API will be made without authentication details. This is only desirable when developing against an emulator. This behavior can be altered by supplying a path to a service account key file.

42.11. ROLLBACK AND REDELIVERY

The rollback for Google PubSub relies on the idea of the Acknowledgement Deadline - the time period where Google PubSub expects to receive the acknowledgement. If the acknowledgement has not been received, the message is redelivered.

Google provides an API to extend the deadline for a message.

More information in [Google PubSub Documentation](#).

So, rollback is effectively a deadline extension API call with zero value - i.e. deadline is reached now and message can be redelivered to the next consumer.

It is possible to delay the message redelivery by setting the acknowledgement deadline explicitly for the rollback by setting the message header **GooglePubsubConstants.ACK_DEADLINE** to the value in seconds.

42.12. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.google-pubsub.authenticate</code>	Use Credentials when interacting with PubSub service (no authentication is required when using emulator).	true	Boolean
<code>camel.component.google-pubsub.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.google-pubsub.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.google-pubsub.enabled</code>	Whether to enable auto configuration of the google-pubsub component. This is enabled by default.		Boolean
<code>camel.component.google-pubsub.endpoint</code>	Endpoint to use with local Pub/Sub emulator.		String

Name	Description	Default	Type
camel.component.google-pubsub.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.google-pubsub.publisher-cache-size	Maximum number of producers to cache. This could be increased if you have producers for lots of different topics.		Integer
camel.component.google-pubsub.publisher-cache-timeout	How many milliseconds should each producer stay alive in the cache.		Integer
camel.component.google-pubsub.publisher-termination-timeout	How many milliseconds should a producer be allowed to terminate.		Integer
camel.component.google-pubsub.service-account-key	The Service account key that can be used as credentials for the PubSub publisher/subscriber. It can be loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
camel.component.google-pubsub.synchronous-pull-retryable-codes	Comma-separated list of additional retryable error codes for synchronous pull. By default the PubSub client library retries ABORTED, UNAVAILABLE, UNKNOWN.		String

CHAPTER 43. GOOGLE SECRET MANAGER

Only producer is supported

The Google Secret Manager component provides access to [Google Cloud Secret Manager](#).

43.1. DEPENDENCIES

When using **camel-aws-secrets-manager** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-google-secret-manager-starter</artifactId>
</dependency>
```

43.2. AUTHENTICATION CONFIGURATION

Google Secret Manager component authentication is targeted for the use with the GCP Service Accounts. For more information, please refer to [Google Cloud Authentication](#).

When you have the **service account key**, you can provide authentication credentials to your application code. Google security credentials can be set through the component endpoint:

```
String endpoint = "google-secret-manager://myCamelFunction?
serviceAccountKey=/home/user/Downloads/my-key.json";
```

Or by setting the environment variable **GOOGLE_APPLICATION_CREDENTIALS**:

```
export GOOGLE_APPLICATION_CREDENTIALS="/home/user/Downloads/my-key.json"
```

43.3. URI FORMAT

```
google-secret-manager://functionName[?options]
```

You can append query options to the URI in the following format,

```
`?options=value&option2=value&...`
```

For example, in order to call the function **myCamelFunction** from the project **myProject** and location **us-central1**, use the following snippet:

```
from("google-secret-manager://myProject?serviceAccountKey=/home/user/Downloads/my-
key.json&operation=createSecret")
.to("direct:test");
```

43.4. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

43.4.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

43.4.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

43.5. COMPONENT OPTIONS

The Google Secret Manager component supports 2 options, which are listed below.

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

43.6. ENDPOINT OPTIONS

The Google Secret Manager endpoint is configured using URI syntax:

```
google-secret-manager:project
```

With the following *path* and *query* parameters:

43.6.1. Path Parameters (1 parameters)

Name	Description	Default	Type
project (common)	Required The Google Cloud Project Id name related to the Secret Manager.		String

43.6.2. Query Parameters (5 parameters)

Name	Description	Default	Type
serviceAccountKey (common)	Service account key to authenticate an application as a service account.		String

Name	Description	Default	Type
operation (producer)	The operation to perform on the producer. Enum values: • createSecret		GoogleSecretManagerOperations
pojoRequest (producer)	Specifies if the request is a pojo request.	false	boolean
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
client (advanced)	Autowired The client to use during service invocation.		SecretManagerServiceClient

43.7. MESSAGE HEADERS

The Google Secret Manager component supports 3 message header(s), which is/are listed below:

Name	Description	Default	Type
GoogleSecretManagerOperation (producer) Constant: OPERATION	The operation to perform. Enum values: • createSecret • getSecretVersion • deleteSecret • listSecrets		GoogleSecretManagerOperations

Name	Description	Default	Type
CamelGoogleSecretManagerSecretId (producer) Constant: SECRET_ID	The id of the secret.		String
CamelGoogleSecretManagerVersionId (producer) Constant: VERSION_ID	The version of the secret.	latest	String

43.7.1. Using GCP Secret Manager Properties Source

To use GCP Secret Manager, you need to provide **serviceAccountKey** file and GCP **projectId**. This can be done using environmental variables before starting the application:

```
export $CAMEL_VAULT_GCP_SERVICE_ACCOUNT_KEY=file://path/to/service.accountkey
export $CAMEL_VAULT_GCP_PROJECT_ID=projectId
```

You can also configure the credentials in the **application.properties** file as follows:

```
camel.vault.gcp.serviceAccountKey = serviceAccountKey
camel.vault.gcp.projectId = projectId
```

If you want to use the [GCP default client instance](#) , you will need to provide the following env variables:

```
export $CAMEL_VAULT_GCP_USE_DEFAULT_INSTANCE=true
export $CAMEL_VAULT_GCP_PROJECT_ID=projectId
```

You can also configure the credentials in the **application.properties** file as follows:

```
camel.vault.gcp.useDefaultInstance = true
camel.vault.gcp.projectId = region
```



NOTE

camel.vault.gcp configuration only applies to the Google Secret Manager properties function (for example, when resolving properties). When using the **operation** option to create, get, list secrets etc., you should provide the usual options for connecting to GCP Services.

At this point you can reference a property in the following way by using **gcp:** as prefix in the **{{ }}** syntax:


```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{gcp:route}}"/>
  </route>
</camelContext>
```

Where **route** will be the name of the secret stored in the GCP Secret Manager Service.

You could specify a default value in case the secret is not present on GCP Secret Manager:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{gcp:route:default}}"/>
  </route>
</camelContext>
```

In this case, if the secret doesn't exist, the property will fall back to **default** as value.

Also, you can get a particular field of the secret, if you have, for example, a secret named database of this form:

```
{
  "username": "admin",
  "password": "password123",
  "engine": "postgres",
  "host": "127.0.0.1",
  "port": "3128",
  "dbname": "db"
}
```

You can get a single secret value in your route, for example:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{gcp:database/username}}"/>
  </route>
</camelContext>
```

Or re-use the property as part of an endpoint.

You could specify a default value in case the particular field of secret is not present on GCP Secret Manager:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{gcp:database/username:admin}}"/>
  </route>
</camelContext>
```

In this case, if the secret doesn't exist or the secret exists, but the username field is not part of the secret, the property will fall back to "admin" as value.

There is also the syntax to get a particular version of the secret for both the approach, with field/default value specified or only with secret:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{gcp:route@1}}"/>
  </route>
</camelContext>
```

This approach will return the RAW route secret with version '1'.

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{gcp:route:default@1}}"/>
  </route>
</camelContext>
```

This approach will return the route secret value with version '1' or default value in case the secret doesn't exist or the version doesn't exist.

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{gcp:database/username:admin@1}}"/>
  </route>
</camelContext>
```

This approach will return the username field of the database secret with version '1' or admin in case the secret doesn't exist or the version doesn't exist.

There are only two requirements:

- Adding **camel-google-secret-manager** JAR to your Camel application.
- Give the service account used permissions to do operation at secret management level, (for example, accessing the secret payload, or being admin of secret manager service)

43.7.2. Automatic CamelContext reloading on Secret Refresh

Being able to reload Camel context on a Secret Refresh is achieved by specifying the usual credentials (the same used for Google Secret Manager Property Function).

With Environment variables:

```
export $CAMEL_VAULT_GCP_USE_DEFAULT_INSTANCE=true
export $CAMEL_VAULT_GCP_PROJECT_ID=projectId
```

or as plain Camel main properties:

```
camel.vault.gcp.useDefaultInstance = true
camel.vault.gcp.projectId = projectId
```

Or by specifying a path to a service account key file, instead of using the default instance.

To enable the automatic refresh, you will need to set the additional properties:

```
camel.vault.gcp.projectId= projectId
camel.vault.gcp.refreshEnabled=true
camel.vault.gcp.refreshPeriod=60000
camel.vault.gcp.secrets=hello*
camel.vault.gcp.subscriptionName=subscriptionName
camel.main.context-reload-enabled = true
```

where **camel.vault.gcp.refreshEnabled** will enable the automatic context reload, **camel.vault.gcp.refreshPeriod** is the interval of time between two different checks for update events and **camel.vault.gcp.secrets** is a regex representing the secrets we want to track for updates.

Note that **camel.vault.gcp.secrets** is not mandatory. If not specified the task responsible for checking updates events will take into accounts or the properties with an **gcp:** prefix.

The **camel.vault.gcp.subscriptionName** is the subscription name created in relation to the Google PubSub topic associated with the tracked secrets.

This mechanism while making use of the notification system related to Google Secret Manager: through this feature, every secret could be associated with one up to ten Google Pubsub Topics. These topics will receive events related to the life cycle of the secret.

There are only two requirements:

- Adding **camel-google-secret-manager** JAR to your Camel application.
- Give the service account used permissions to do operation at secret management level, (for example, accessing the secret payload, or being admin of secret manager service and also have permission over the Pubsub service).

43.7.3. Automatic CamelContext reloading on Secret Refresh - Required infrastructure's creation

you need to install the gcloud cli from the <https://cloud.google.com/sdk/docs/install>. Once the Cli has been installed we can proceed to log in and to set up the project with the following commands:

```
gcloud auth login
```

and

```
gcloud projects create <projectId> --name="GCP Secret Manager Refresh"
```

The project will need a service identity for using secret manager service which you can obtain through the command:

```
gcloud beta services identity create --service "secretmanager.googleapis.com" --project <project_id>
```

The following command will provide a service account name that we need to export:

```
export SM_SERVICE_ACCOUNT="service-...."
```

Since we want to have notifications about events related to a specific secret through a Google Pubsub topic we'll need to create a topic for this purpose with the following command:

```
gcloud pubsub topics create "projects/<project_id>/topics/pubsub-gcp-sec-refresh"
```

The service account will need Secret Manager authorization to publish messages on the topic just created, so we'll need to add an iam policy binding with the following command:

```
gcloud pubsub topics add-iam-policy-binding pubsub-gcp-sec-refresh --member  
"serviceAccount:${SM_SERVICE_ACCOUNT}" --role "roles/pubsub.publisher" --project <project_id>
```

We now need to create a subscription to the **pubsub-gcp-sec-refresh** just created and call it **sub-gcp-sec-refresh** with the following command:

```
gcloud pubsub subscriptions create "projects/<project_id>/subscriptions/sub-gcp-sec-refresh" --topic  
"projects/<project_id>/topics/pubsub-gcp-sec-refresh"
```

Now we need to create a service account for running our application:

```
gcloud iam service-accounts create gcp-sec-refresh-sa --description="GCP Sec Refresh SA" --  
project <project_id>
```

Let's give the SA an owner role:

```
gcloud projects add-iam-policy-binding <project_id> --member="serviceAccount:gcp-sec-refresh-  
sa@<project_id>.iam.gserviceaccount.com" --role="roles/owner"
```

Now we should create a Service account key file for the just create SA:

```
gcloud iam service-accounts keys create <project_id>.json --iam-account=gcp-sec-refresh-  
sa@<project_id>.iam.gserviceaccount.com
```

Let's enable the Secret Manager API for our project.

```
gcloud services enable secretmanager.googleapis.com --project <project_id>
```

Also enable the PubSub API as follows:

```
gcloud services enable pubsub.googleapis.com --project <project_id>
```

If needed enable also the Billing API.

Now it's time to create our secret, with topic notification:

```
gcloud secrets create <secret_name> --topics=projects/<project_id>/topics/pubsub-gcp-sec-refresh --  
project=<project_id>
```

And add the value

```
gcloud secrets versions add <secret_name> --data-file=<json_secret> --project=<project_id>
```

You could now use the `projectId` and the service account json file to recover the secret.

43.7.4. Google Secret Manager Producer operations

Google Functions component provides the following operation on the producer side:

- **createSecret**
- **getSecretVersion**
- **deleteSecret**
- **listSecrets**

If you don't specify an operation by default, the producer will use the **createSecret** operation.

43.7.5. Google Secret Manager Producer Operation examples

- **createSecret**: This operation will create a secret in the Secret Manager service

```
from("direct:start")
  .setHeader("GoogleSecretManagerConstants.SECRET_ID", constant("test"))
  .setBody(constant("hello"))
  .to("google-functions://myProject?serviceAccountKey=/home/user/Downloads/my-
key.json&operation=createSecret")
  .log("body:${body}")
```

- **getSecretVersion**: This operation will retrieve a secret value with the latest version in the Secret Manager service

```
from("direct:start")
  .setHeader("GoogleSecretManagerConstants.SECRET_ID", constant("test"))
  .to("google-functions://myProject?serviceAccountKey=/home/user/Downloads/my-
key.json&operation=getSecretVersion")
  .log("body:${body}")
```

This will log the value of the secret "test".

- **deleteSecret**: This operation will delete a secret

```
from("direct:start")
  .setHeader("GoogleSecretManagerConstants.SECRET_ID", constant("test"))
  .to("google-functions://myProject?serviceAccountKey=/home/user/Downloads/my-
key.json&operation=deleteSecret")
```

- **listSecrets**: This operation will return the secrets' list for the project `myProject`

```
from("direct:start")
  .setHeader("GoogleSecretManagerConstants.SECRET_ID", constant("test"))
  .to("google-functions://myProject?serviceAccountKey=/home/user/Downloads/my-
key.json&operation=listSecrets")
```

43.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.component.google-secret-manager.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.google-secret-manager.enabled	Whether to enable auto configuration of the google-secret-manager component. This is enabled by default.		Boolean
camel.component.google-secret-manager.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 44. GRAPHQL

Only producer is supported

The GraphQL component is a GraphQL client that communicates over HTTP and supports queries and mutations, but not subscriptions. It uses the [Apache HttpClient](#) library.

44.1. DEPENDENCIES

When using **camel-graphql** component starter with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-graphql-starter</artifactId>
</dependency>
```

44.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

44.2.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

44.2.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

44.3. COMPONENT OPTIONS

The GraphQL component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

44.4. ENDPOINT OPTIONS

The GraphQL endpoint is configured using following URI syntax:

```
graphql:httpUri
```

With the following *path* and *query* parameters:

44.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
httpUri (producer)	Required The GraphQL server URI.		URI

44.4.2. Query Parameters (12 parameters)

Name	Description	Default	Type
operationName (producer)	The query or mutation name.		String
proxyHost (producer)	The proxy host in the format hostname:port.		String
query (producer)	The query text.		String
queryFile (producer)	The query file name located in the classpath.		String
queryHeader (producer)	The name of a header containing the GraphQL query.		String
variables (producer)	The JsonObject instance containing the operation variables.		JsonObject
variablesHeader (producer)	The name of a header containing a JsonObject instance containing the operation variables.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
accessToken (security)	The access token sent in the Authorization header.		String
jwtAuthorizationType (security)	The JWT Authorization type. Default is Bearer.	Bearer	String
password (security)	The password for Basic authentication.		String
username (security)	The username for Basic authentication.		String

44.5. MESSAGE BODY

If the **variables** and **variablesHeader** parameters are not set and the IN body is a `JsonObject` instance, Camel uses it as the variables for the operation. If the **query** and **queryFile** parameters are not set and the IN body is a `String`, Camel uses it as the query. Camel stores the GraphQL response from the external server on the OUT message body. All headers from the IN message are copied to the OUT message, so the headers are preserved during routing. Additionally, Camel adds the HTTP response headers as well to the OUT message headers.

44.6. EXAMPLES

44.6.1. Queries

Simple queries can be defined directly in the URI:

```
from("direct:start")
  .to("graphql://http://example.com/graphql?query={books{id name}}")
```

The body can also be used for the query:

```
from("direct:start")
  .setBody(constant("{books{id name}}"))
  .to("graphql://http://example.com/graphql")
```

The query can come from a header:

```
from("direct:start")
  .setHeader("myQuery", constant("{books{id name}}"))
  .to("graphql://http://example.com/graphql?queryHeader=myQuery")
```

More complex queries can be stored in a file and referenced in the URI as follows.

booksQuery.graphql file:

```
query Books {
  books {
    id
    name
  }
}
```

```
from("direct:start")
  .to("graphql://http://example.com/graphql?queryFile=booksQuery.graphql")
```

When the query file defines multiple operations, it is required to specify which one should be executed:

```
from("direct:start")
  .to("graphql://http://example.com/graphql?
  queryFile=multipleQueries.graphql&operationName=Books")
```

Queries with variables need to reference a `JsonObject` instance from the registry:

bookByIdQuery.graphql file:

```

query BookById($id: Int!) {
  bookById(id: $id) {
    id
    name
    author
  }
}

```

```

@BindToRegistry("bookByIdQueryVariables")
public JsonObject bookByIdQueryVariables() {
  JsonObject variables = new JsonObject();
  variables.put("id", "book-1");
  return variables;
}

from("direct:start")
  .to("graphql://http://example.com/graphql?
queryFile=bookByIdQuery.graphql&variables=#bookByIdQueryVariables")

```

A query that accesses variables via the variablesHeader parameter:

```

from("direct:start")
  .setHeader("myVariables", () -> {
    JsonObject variables = new JsonObject();
    variables.put("id", "book-1");
    return variables;
  })
  .to("graphql://http://example.com/graphql?
queryFile=bookByIdQuery.graphql&variablesHeader=myVariables")

```

44.6.2. Mutations

Mutations are like queries with variables. They specify a query and a reference to a variables' bean as follows:

addBookMutation.graphql file:

```

mutation AddBook($bookInput: BookInput) {
  addBook(bookInput: $bookInput) {
    id
    name
    author {
      name
    }
  }
}

@BindToRegistry("addBookMutationVariables")
public JsonObject addBookMutationVariables() {
  JsonObject bookInput = new JsonObject();
  bookInput.put("name", "Typee");
  bookInput.put("authorId", "author-2");
  JsonObject variables = new JsonObject();
}

```

```

        variables.put("bookInput", bookInput);
        return variables;
    }

    from("direct:start")
        .to("graphql://http://example.com/graphql?
            queryFile=addBookMutation.graphql&variables=#addBookMutationVariables")

```

44.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.component.graphql.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.graphql.enabled	Whether to enable auto configuration of the graphql component. This is enabled by default.		Boolean
camel.component.graphql.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 45. GROOVY

Since Camel 1.3

Camel has support for using [Groovy](#). For example, you can use Groovy in a [Predicate](#) with the [Message Filter](#) EIP.

45.1. DEPENDENCIES

When using **camel-groovy** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-groovy-starter</artifactId>
</dependency>
```

45.2. URI FORMAT

The **camel-groovy** language component uses the following URI notation:

```
groovy("someGroovyExpression")
```

45.3. GROOVY OPTIONS

The Groovy language supports 2 options, which are listed below.

Name	Default	Java Type	Description
resultType		String	Sets the class of the result type (type from output).
trim	true	Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

45.4. EXAMPLES

Following example uses a groovy script as predicate in the message filter, to determine if any line items are over \$100:

Java

```
from("queue:foo")
  .filter(groovy("request.lineItems.any { i -> i.value > 100 }"))
  .to("queue:bar")
```

XML DSL

```
<route>
```

```

<from uri="queue:foo"/>
<filter>
  <groovy>request.lineItems.any { i -> i.value > 100 }</groovy>
  <to uri="queue:bar"/>
</filter>
</route>

```

45.5. GROOVY CONTEXT

Camel provides an exchange information in the Groovy context (just a **Map**). The **Exchange** is transferred as:

key	value
exchange	The Exchange itself.
exchangeProperties	The Exchange properties.
variables	The variables
headers	The headers of the In message.
camelContext	The Camel Context.
request	The In message.
body	The In message body.
response	The Out message (only for InOut message exchange pattern).

45.6. HOW TO GET THE RESULT FROM MULTIPLE STATEMENTS SCRIPT

As the Groovy script engine evaluate method returns a **Null** if it runs a multiple statements script. Camel looks up the value of script result by using the key of **result** from the value set. If you have multiple statements scripts, make sure to set the value of result variable as the script return value.

```

bar = "baz";
# some other statements ...
# camel take the result value as the script evaluation result
result = body * 2 + 1

```

45.7. CUSTOMIZING GROOVY SHELL

For very special use cases you may need to use a custom **GroovyShell** instance in your Groovy expressions. To provide the custom **GroovyShell**, add an implementation of the **org.apache.camel.language.groovy.GroovyShellFactory** SPI interface to the Camel registry.

```

public class CustomGroovyShellFactory implements GroovyShellFactory {

    public GroovyShell createGroovyShell(Exchange exchange) {
        ImportCustomizer importCustomizer = new ImportCustomizer();
        importCustomizer.addStaticStars("com.example.Utills");
        CompilerConfiguration configuration = new CompilerConfiguration();
        configuration.addCompilationCustomizers(importCustomizer);
        return new GroovyShell(configuration);
    }

}

```

Camel will then use your custom GroovyShell instance (containing your custom static imports), instead of the default one.

45.8. LOADING SCRIPT FROM EXTERNAL RESOURCE

You can externalize the script and have Camel load it from a resource such as **"classpath:"**, **"file:"**, or **"http:"**. You can achieve this by using the following syntax:

```
`"resource:scheme:location"`,
```

For example, to refer to a file on the classpath you can use the following:

```
.setHeader("myHeader").groovy("resource:classpath:mygroovy.groovy")
```

45.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 2 options, which are listed below.

Name	Description	Default	Type
camel.language.groovy.enabled	Whether to enable auto configuration of the groovy language. This is enabled by default.		Boolean
camel.language.groovy.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

CHAPTER 46. GRPC

Since Camel 2.19

Both producer and consumer are supported

The gRPC component allows you to call or expose Remote Procedure Call (RPC) services using [Protocol Buffers \(protobuf\)](#) exchange format over HTTP/2 transport.

46.1. DEPENDENCIES

When using **grpc** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-grpc-starter</artifactId>
</dependency>
```

46.2. URI FORMAT

```
grpc:host:port/service[?options]
```

46.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

46.3.1. Configuring component options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

46.3.2. Configuring endpoint options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

46.4. COMPONENT OPTIONS

The gRPC component supports 3 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

46.5. ENDPOINT OPTIONS

The gRPC endpoint is configured using URI syntax:

```
grpc:host:port/service
```

with the following path and query parameters:

46.5.1. Path Parameters (3 parameters)

Name	Description	Default	Type
host (common)	Required The gRPC server host name. This is localhost or 0.0.0.0 when being a consumer or remote server host name when using producer.		String
port (common)	Required The gRPC local or remote server port.		int
service (common)	Required Fully qualified service name from the protocol buffer descriptor file (package dot service definition name).		String

46.5.2. Query Parameters (29 parameters)

Name	Description	Default	Type
flowControlWindow (common)	The HTTP/2 flow control window size (MiB).	1048576	int
maxMessageSize (common)	The maximum message size allowed to be received/sent (MiB).	4194304	int
autoDiscoverServerInterceptors (consumer)	Setting the autoDiscoverServerInterceptors mechanism, if true, the component will look for a ServerInterceptor instance in the registry automatically otherwise it will skip that checking.	true	boolean
consumerStrategy (consumer)	This option specifies the top-level strategy for processing service requests and responses in streaming mode. If an aggregation strategy is selected, all requests will be accumulated in the list, then transferred to the flow, and the accumulated responses will be sent to the sender. If a propagation strategy is selected, request is sent to the stream, and the response will be immediately sent back to the sender. Enum values: ● AGGREGATION ● PROPAGATION	PROPAGATION	GrpcConsumerStrategy

Name	Description	Default	Type
forwardOnCompleted (consumer)	Determines if onCompleted events should be pushed to the Camel route.	false	boolean
forwardOnError (consumer)	Determines if onError events should be pushed to the Camel route. Exceptions will be set as message body.	false	boolean
maxConcurrentCallsPerConnection (consumer)	The maximum number of concurrent calls permitted for each incoming server connection.	2147483647	int
routeControlledStreamObserver (consumer)	Lets the route to take control over stream observer. If this value is set to true, then the response observer of gRPC call will be set with the name GrpcConstants.GRPC_RESPONSE_OBSERVER in the Exchange object. Please note that the stream observer's onNext(), onError(), onCompleted() methods should be called in the route.	false	boolean
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut		ExchangePattern
autoDiscoverClientInterceptors (producer)	Setting the autoDiscoverClientInterceptors mechanism, if true, the component will look for a ClientInterceptor instance in the registry automatically otherwise it will skip that checking.	true	boolean

Name	Description	Default	Type
method (producer)	gRPC method name.		String
producerStrategy (producer)	The mode used to communicate with a remote gRPC server. In SIMPLE mode a single exchange is translated into a remote procedure call. In STREAMING mode all exchanges will be sent within the same request (input and output of the recipient gRPC service must be of type 'stream'). Enum values: • SIMPLE • STREAMING	SIMPLE	GrpcProducerStrategy
streamRepliesTo (producer)	When using STREAMING client mode, it indicates the endpoint where responses should be forwarded.		String
userAgent (producer)	The user agent header passed to the server.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean
authenticationType (security)	Authentication method type in advance to the SSL/TLS negotiation. Enum values: • NONE • GOOGLE • JWT	NONE	GrpcAuthType

Name	Description	Default	Type
jwtAlgorithm (security)	JSON Web Token sign algorithm. Enum values: • HMAC256 • HMAC384 • HMAC512	HMAC256	JwtAlgorithm
jwtIssuer (security)	JSON Web Token issuer.		String
jwtSecret (security)	JSON Web Token secret.		String
jwtSubject (security)	JSON Web Token subject.		String
keyCertChainResource (security)	The X.509 certificate chain file resource in PEM format link.		String
keyPassword (security)	The PKCS#8 private key file password.		String
keyResource (security)	The PKCS#8 private key file resource in PEM format link.		String
negotiationType (security)	Identifies the security negotiation type used for HTTP/2 communication. Enum values: • TLS • PLAINTEXT_UPGRADE • PLAINTEXT	PLAINTEXT	NegotiationType
serviceAccountResource (security)	Service Account key file in JSON format resource link supported by the Google Cloud SDK.		String
trustCertCollectionResource (security)	The trusted certificates collection file resource in PEM format for verifying the remote endpoint's certificate.		String

46.6. MESSAGE HEADERS

The gRPC component supports 3 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelGrpcMethod dName (consumer) Constant: GRPC_METHOD_NAME_HEADER	Method name handled by the consumer service.		String
CamelGrpcUserAgent (consumer) Constant: GRPC_USER_AGENT_HEADER	If provided, the given agent will prepend the gRPC library's user agent information.		String
CamelGrpcEventType (consumer) Constant: GRPC_EVENT_TYPE_HEADER	Received event type from the sent request. Possible values: onNext onCompleted onError.		String

46.7. TRANSPORT SECURITY AND AUTHENTICATION SUPPORT

The following [authentication](#) mechanisms are built-in to gRPC and available in this component:

- **SSL/TLS:** gRPC has SSL/TLS integration and promotes the use of SSL/TLS to authenticate the server, and to encrypt all the data exchanged between the client and the server. Optional mechanisms are available for clients to provide certificates for mutual authentication.
- **Token-based authentication with Google:** gRPC provides a generic mechanism to attach metadata based credentials to requests and responses. Additional support for acquiring access tokens while accessing Google APIs through gRPC is provided. In general this mechanism must be used as well as SSL/TLS on the channel.

To enable these features the following component properties combinations must be configured:

Num.	Option	Parameter	Value	Required/Optional
1	SSL/TLS	negotiationType	TLS	Required
		keyCertChainResource		Required
		keyResource		Required
		keyPassword		Optional

Num.	Option	Parameter	Value	Required/Optional
		trustCertCollectionResource		Optional
2	Token-based authentication with Google API	authenticationType	GOOGLE	Required
		negotiationType	TLS	Required
		serviceAccountResource		Required
3	Custom JSON Web Token implementation authentication	authenticationType	JWT	Required
		negotiationType	NONE or TLS	Optional. The TLS/SSL not checking for this type, but strongly recommended.
		jwtAlgorithm	HMAC256(default) or (HMAC384, HMAC512)	Optional
		jwtSecret		Required
		jwtIssuer		Optional
		jwtSubject		Optional

46.8. GRPC PRODUCER RESOURCE TYPE MAPPING

The table below shows the types of objects in the message body, depending on the types (simple or stream) of incoming and outgoing parameters, as well as the invocation style (synchronous or asynchronous). Please note, that invocation of the procedures with incoming stream parameter in asynchronous style are not allowed.

Invocation style	Request type	Response type	Request Body Type	Result Body Type
synchronous	simple	simple	Object	Object
synchronous	simple	stream	Object	List<Object>

Invocation style	Request type	Response type	Request Body Type	Result Body Type
synchronous	stream	simple	not allowed	not allowed
synchronous	stream	stream	not allowed	not allowed
asynchronous	simple	simple	Object	List<Object>
asynchronous	simple	stream	Object	List<Object>
asynchronous	stream	simple	Object or List<Object>	List<Object>
asynchronous	stream	stream	Object or List<Object>	List<Object>

46.9. EXAMPLES

Below is a simple synchronous method invoke with host and port parameters:

```
from("direct:grpc-sync")
.to("grpc://remotehost:1101/org.apache.camel.component.grpc.PingPong?
method=sendPing&synchronous=true");

<route>
  <from uri="direct:grpc-sync" />
  <to uri="grpc://remotehost:1101/org.apache.camel.component.grpc.PingPong?
method=sendPing&synchronous=true"/>
</route>
```

An asynchronous method invokes

```
from("direct:grpc-async")
.to("grpc://remotehost:1101/org.apache.camel.component.grpc.PingPong?
method=pingAsyncResponse");
```

a gRPC service consumer with propagation consumer strategy:

```
from("grpc://localhost:1101/org.apache.camel.component.grpc.PingPong?
consumerStrategy=PROPAGATION")
.to("direct:grpc-service");
```

gRPC service producer with streaming producer strategy (requires a service that uses "stream" mode as input and output):

```
from("direct:grpc-request-stream")
.to("grpc://remotehost:1101/org.apache.camel.component.grpc.PingPong?
method=PingAsyncAsync&producerStrategy=STREAMING&streamRepliesTo=direct:grpc-response-stream");
```



```
from("direct:grpc-response-stream")
.log("Response received: ${body}");
```

gRPC service consumer TLS/SSL security negotiation enabled:

```
from("grpc://localhost:1101/org.apache.camel.component.grpc.PingPong?
consumerStrategy=PROPAGATION&negotiationType=TLS&keyCertChainResource=file:src/test/resour
ces/certs/server.pem&keyResource=file:src/test/resources/certs/server.key&trustCertCollectionResource
=file:src/test/resources/certs/ca.pem")
.to("direct:tls-enable")
```

gRPC service producer with custom JSON Web Token (JWT) implementation authentication:

```
from("direct:grpc-jwt")
.to("grpc://localhost:1101/org.apache.camel.component.grpc.PingPong?
method=pingSyncSync&synchronous=true&authenticationType=JWT&jwtSecret=supersecuredsecret"
);
```

46.10. CONFIGURATION

Use the **protobuf-maven-plugin**, which calls the Protocol Buffer Compiler (protoc) to generate Java source files from .proto (protocol buffer definition) files. This plugin will generate procedures request and response classes, their builders and gRPC procedures stubs classes as well.

Following steps are required:

Insert operating system and CPU architecture detection extension inside **<build>** tag of the project pom.xml or set `${os.detected.classifier}` parameter manually

```
<extensions>
<extension>
<groupId>kr.motd.maven</groupId>
<artifactId>os-maven-plugin</artifactId>
<version>1.7.1</version>
</extension>
</extensions>
```

Insert the gRPC and protobuf Java code generator plugins into the **<plugins>** tag of the project's pom.xml.

```
<plugin>
<groupId>org.xolstice.maven.plugins</groupId>
<artifactId>protobuf-maven-plugin</artifactId>
<version>0.6.1</version>
<configuration>
<protocArtifact>com.google.protobuf:protoc:${protobuf-
version}:exe:${os.detected.classifier}</protocArtifact>
<pluginId>grpc-java</pluginId>
<pluginArtifact>io.grpc:protoc-gen-grpc-java:${grpc-
version}:exe:${os.detected.classifier}</pluginArtifact>
</configuration>
<executions>
<execution>
```

```

    <goals>
    <goal>compile</goal>
    <goal>compile-custom</goal>
    <goal>test-compile</goal>
    <goal>test-compile-custom</goal>
    </goals>
  </execution>
</executions>
</plugin>

```

46.11. ADDITIONAL RESOURCES

See these resources:

- [gRPC project site](#)
- [Maven Protocol Buffers Plugin](#)

46.12. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.component.grpc.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.grpc.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.grpc.enabled	Whether to enable auto configuration of the grpc component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.grpc.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 47. HEADER

The Header Expression Language allows you to extract values of named headers.

47.1. DEPENDENCIES

The Header language is part of **camel-core**.

When using **header** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-core-starter</artifactId>
</dependency>
```

47.2. HEADER OPTIONS

The Header language supports 1 options, which are listed below.

Name	Default	Java Type	Description
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

47.3. EXAMPLE USAGE

The **recipientList** EIP can utilize a header:

```
<route>
  <from uri="direct:a" />
  <recipientList>
    <header>myHeader</header>
  </recipientList>
</route>
```

In this case, the list of recipients are contained in the header 'myHeader'.

And the same example in Java DSL:

```
from("direct:a").recipientList(header("myHeader"));
```

47.4. SPRING BOOT AUTO-CONFIGURATION

The component supports 147 options, which are listed below.

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.acl-token</code>	Sets the ACL token to be used with Consul.		String
<code>camel.cloud.consul.service-discovery.block-seconds</code>	The seconds to wait for a watch event, default 10 seconds.	10	Integer
<code>camel.cloud.consul.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.consul.service-discovery.connect-timeout-millis</code>	Connect timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.datacenter</code>	The data center.		String
<code>camel.cloud.consul.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.consul.service-discovery.password</code>	Sets the password to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.consul.service-discovery.read-timeout-millis</code>	Read timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.url</code>	The Consul agent URL.		String

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.username</code>	Sets the username to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.write-timeout-millis</code>	Write timeout for OkHttpClient.		Long
<code>camel.cloud.dns.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.dns.service-discovery.domain</code>	The domain name;.		String
<code>camel.cloud.dns.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.dns.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.dns.service-discovery.proto</code>	The transport protocol of the desired service.	<code>_tcp</code>	String
<code>camel.cloud.etcd.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.etcd.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.etcd.service-discovery.password</code>	The password to use for basic authentication.		String

Name	Description	Default	Type
<code>camel.cloud.etcd.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.etcd.service-discovery.service-path</code>	The path to look for for service discovery.	<code>/services/</code>	String
<code>camel.cloud.etcd.service-discovery.timeout</code>	To set the maximum time an action could take to complete.		Long
<code>camel.cloud.etcd.service-discovery.type</code>	To set the discovery type, valid values are on-demand and watch.	on-demand	String
<code>camel.cloud.etcd.service-discovery.uris</code>	The URIs the client can connect to.		String
<code>camel.cloud.etcd.service-discovery.user-name</code>	The user name to use for basic authentication.		String
<code>camel.cloud.kubernetes.service-discovery.api-version</code>	Sets the API version when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-data</code>	Sets the Certificate Authority data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-file</code>	Sets the Certificate Authority data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-data</code>	Sets the Client Certificate data when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.client-cert-file</code>	Sets the Client Certificate data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-algo</code>	Sets the Client Keystore algorithm, such as RSA when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-data</code>	Sets the Client Keystore data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-file</code>	Sets the Client Keystore data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-passphrase</code>	Sets the Client Keystore passphrase when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.kubernetes.service-discovery.dns-domain</code>	Sets the DNS domain to use for DNS lookup.		String
<code>camel.cloud.kubernetes.service-discovery.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
camel.cloud.kubernetes.service-discovery.lookup	How to perform service lookup. Possible values: client, dns, environment. When using client, then the client queries the kubernetes master to obtain a list of active pods that provides the service, and then random (or round robin) select a pod. When using dns the service name is resolved as name.namespace.svc.dnsDomain. When using dnssrv the service name is resolved with SRV query forsvc... When using environment then environment variables are used to lookup the service. By default environment is used.	environment	String
camel.cloud.kubernetes.service-discovery.master-url	Sets the URL to the master when using client lookup.		String
camel.cloud.kubernetes.service-discovery.namespace	Sets the namespace to use. Will by default use namespace from the ENV variable KUBERNETES_MASTER.		String
camel.cloud.kubernetes.service-discovery.oauth-token	Sets the OAUTH token for authentication (instead of username/password) when using client lookup.		String
camel.cloud.kubernetes.service-discovery.password	Sets the password for authentication when using client lookup.		String
camel.cloud.kubernetes.service-discovery.port-name	Sets the Port Name to use for DNS/DNSSRV lookup.		String
camel.cloud.kubernetes.service-discovery.port-protocol	Sets the Port Protocol to use for DNS/DNSSRV lookup.		String
camel.cloud.kubernetes.service-discovery.properties	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in com.netflix.client.config.CommonClientConfigKey.		Map

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.trust-certs</code>	Sets whether to turn on trust certificate check when using client lookup.	false	Boolean
<code>camel.cloud.kubernetes.service-discovery.username</code>	Sets the username for authentication when using client lookup.		String
<code>camel.cloud.ribbon.load-balancer.client-name</code>	Sets the Ribbon client name.		String
<code>camel.cloud.ribbon.load-balancer.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.ribbon.load-balancer.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.ribbon.load-balancer.namespace</code>	The namespace.		String
<code>camel.cloud.ribbon.load-balancer.password</code>	The password.		String
<code>camel.cloud.ribbon.load-balancer.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.ribbon.load-balancer.username</code>	The username.		String

Name	Description	Default	Type
camel.hystrix.allow-maximum-size-to-diverge-from-core-size	Allows the configuration for <code>maximumSize</code> to take effect. That value can then be equal to, or higher, than <code>coreSize</code> .	false	Boolean
camel.hystrix.circuit-breaker-enabled	Whether to use a <code>HystrixCircuitBreaker</code> or not. If false no circuit-breaker logic will be used and all requests permitted. This is similar in effect to <code>circuitBreakerForceClosed()</code> except that continues tracking metrics and knowing whether it should be open/closed, this property results in not even instantiating a circuit-breaker.	true	Boolean
camel.hystrix.circuit-breaker-error-threshold-percentage	Error percentage threshold (as whole number such as 50) at which point the circuit breaker will trip open and reject requests. It will stay tripped for the duration defined in <code>circuitBreakerSleepWindowInMilliseconds</code> ; The error percentage this is compared against comes from <code>HystrixCommandMetrics.getHealthCounts()</code> .	50	Integer
camel.hystrix.circuit-breaker-force-closed	If true the <code>HystrixCircuitBreaker#allowRequest()</code> will always return true to allow requests regardless of the error percentage from <code>HystrixCommandMetrics.getHealthCounts()</code> . The <code>circuitBreakerForceOpen()</code> property takes precedence so if it set to true this property does nothing.	false	Boolean
camel.hystrix.circuit-breaker-force-open	If true the <code>HystrixCircuitBreaker.allowRequest()</code> will always return false, causing the circuit to be open (tripped) and reject all requests. This property takes precedence over <code>circuitBreakerForceClosed()</code> ;	false	Boolean
camel.hystrix.circuit-breaker-request-volume-threshold	Minimum number of requests in the <code>metricsRollingStatisticalWindowInMilliseconds()</code> that must exist before the <code>HystrixCircuitBreaker</code> will trip. If below this number the circuit will not trip regardless of error percentage.	20	Integer
camel.hystrix.circuit-breaker-sleep-window-in-milliseconds	The time in milliseconds after a <code>HystrixCircuitBreaker</code> trips open that it should wait before trying requests again.	5000	Integer

Name	Description	Default	Type
camel.hystrix.configurations	Define additional configuration definitions.		Map
camel.hystrix.core-pool-size	Core thread-pool size that gets passed to <code>java.util.concurrent.ThreadPoolExecutor#setCorePoolSize(int)</code> .	10	Integer
camel.hystrix.enabled	Enable the component.	true	Boolean
camel.hystrix.execution-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.run()</code> . Requests beyond the concurrent limit will be rejected. Applicable only when <code>executionIsolationStrategy == SEMAPHORE</code> .	20	Integer
camel.hystrix.execution-isolation-strategy	What isolation strategy <code>HystrixCommand.run()</code> will be executed with. If <code>THREAD</code> then it will be executed on a separate thread and concurrent requests limited by the number of threads in the thread-pool. If <code>SEMAPHORE</code> then it will be executed on the calling thread and concurrent requests limited by the semaphore count.	THREAD	String
camel.hystrix.execution-isolation-thread-interrupt-on-timeout	Whether the execution thread should attempt an interrupt (using <code>Future#cancel()</code>) when a thread times out. Applicable only when <code>executionIsolationStrategy() == THREAD</code> .	true	Boolean
camel.hystrix.execution-timeout-enabled	Whether the timeout mechanism is enabled for this command.	true	Boolean
camel.hystrix.execution-timeout-in-milliseconds	Time in milliseconds at which point the command will timeout and halt execution. If <code>executionIsolationThreadInterruptOnTimeout == true</code> and the command is thread-isolated, the executing thread will be interrupted. If the command is semaphore-isolated and a <code>HystrixObservableCommand</code> , that command will get unsubscribed.	1000	Integer
camel.hystrix.fallback-enabled	Whether <code>HystrixCommand.getFallback()</code> should be attempted when failure occurs.	true	Boolean

Name	Description	Default	Type
camel.hystrix.fallback-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.getFallback()</code> . Requests beyond the concurrent limit will fail-fast and not attempt retrieving a fallback.	10	Integer
camel.hystrix.group-key	Sets the group key to use. The default value is <code>CamelHystrix</code> .	CamelHystrix	String
camel.hystrix.keep-alive-time	Keep-alive time in minutes that gets passed to <code>ThreadPoolExecutor#setKeepAliveTime(long, TimeUnit)</code> .	1	Integer
camel.hystrix.max-queue-size	Max queue size that gets passed to <code>BlockingQueue</code> in <code>HystrixConcurrencyStrategy.getBlockingQueue(int)</code> . This should only affect the instantiation of a threadpool - it is not eligible to change a queue size on the fly. For that, use <code>queueSizeRejectionThreshold()</code> .	-1	Integer
camel.hystrix.maximum-size	Maximum thread-pool size that gets passed to <code>ThreadPoolExecutor#setMaximumPoolSize(int)</code> . This is the maximum amount of concurrency that can be supported without starting to reject <code>HystrixCommands</code> . Please note that this setting only takes effect if you also set <code>allowMaximumSizeToDivergeFromCoreSize</code> .	10	Integer
camel.hystrix.metrics-health-snapshot-interval-in-milliseconds	Time in milliseconds to wait between allowing health snapshots to be taken that calculate success and error percentages and affect <code>HystrixCircuitBreaker.isOpen()</code> status. On high-volume circuits the continual calculation of error percentage can become CPU intensive thus this controls how often it is calculated.	500	Integer
camel.hystrix.metrics-rolling-percentile-bucket-size	Maximum number of values stored in each bucket of the rolling percentile. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-percentile-enabled	Whether percentile metrics should be captured using <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	true	Boolean

Name	Description	Default	Type
camel.hystrix.metrics-rolling-percentile-window-buckets	Number of buckets the rolling percentile window is broken into. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	6	Integer
camel.hystrix.metrics-rolling-percentile-window-in-milliseconds	Duration of percentile rolling window in milliseconds. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	10000	Integer
camel.hystrix.metrics-rolling-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside HystrixCommandMetrics.	10	Integer
camel.hystrix.metrics-rolling-statistical-window-in-milliseconds	This property sets the duration of the statistical rolling window, in milliseconds. This is how long metrics are kept for the thread pool. The window is divided into buckets and rolls by those increments.	10000	Integer
camel.hystrix.queue-size-rejection-threshold	Queue size rejection threshold is an artificial max size at which rejections will occur even if maxQueueSize has not been reached. This is done because the maxQueueSize of a BlockingQueue can not be dynamically changed and we want to support dynamically changing the queue size that affects rejections. This is used by HystrixCommand when queuing a thread for execution.	5	Integer
camel.hystrix.request-log-enabled	Whether HystrixCommand execution and events should be logged to HystrixRequestLog.	true	Boolean
camel.hystrix.thread-pool-key	Sets the thread pool key to use. Will by default use the same value as groupKey has been configured to use.	Camel Hystrix	String
camel.hystrix.thread-pool-rolling-number-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10	Integer

Name	Description	Default	Type
camel.hystrix.thread-pool-rolling-number-statistical-window-in-milliseconds	Duration of statistical rolling window in milliseconds. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10000	Integer
camel.language.constant.enabled	Whether to enable auto configuration of the constant language. This is enabled by default.		Boolean
camel.language.constant.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.language.csimple.enabled	Whether to enable auto configuration of the csimple language. This is enabled by default.		Boolean
camel.language.csimple.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.language.exchangeproperty.enabled	Whether to enable auto configuration of the exchangeProperty language. This is enabled by default.		Boolean
camel.language.exchangeproperty.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.language.file.enabled	Whether to enable auto configuration of the file language. This is enabled by default.		Boolean
camel.language.file.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.language.header.enabled	Whether to enable auto configuration of the header language. This is enabled by default.		Boolean
camel.language.header.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.language.ref.enabled	Whether to enable auto configuration of the ref language. This is enabled by default.		Boolean
camel.language.ref.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

Name	Description	Default	Type
<code>camel.language.simple.enabled</code>	Whether to enable auto configuration of the simple language. This is enabled by default.		Boolean
<code>camel.language.simple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.tokenize.enabled</code>	Whether to enable auto configuration of the tokenize language. This is enabled by default.		Boolean
<code>camel.language.tokenize.group-delimiter</code>	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.		String
<code>camel.language.tokenize.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.resilience4j.automatic-transition-from-open-to-half-open-enabled</code>	Enables automatic transition from OPEN to HALF_OPEN state once the <code>waitDurationInOpenState</code> has passed.	false	Boolean
<code>camel.resilience4j.circuit-breaker-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreaker</code> instance to lookup and use from the registry. When using this, then any other circuit breaker options are not in use.		String
<code>camel.resilience4j.config-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreakerConfig</code> instance to lookup and use from the registry.		String
<code>camel.resilience4j.configurations</code>	Define additional configuration definitions.		Map
<code>camel.resilience4j.enabled</code>	Enable the component.	true	Boolean
<code>camel.resilience4j.failure-rate-threshold</code>	Configures the failure rate threshold in percentage. If the failure rate is equal or greater than the threshold the <code>CircuitBreaker</code> transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 50 percentage.		Float

Name	Description	Default	Type
camel.resilience4j.minimum-number-of-calls	Configures the minimum number of calls which are required (per sliding window period) before the CircuitBreaker can calculate the error rate. For example, if <code>minimumNumberOfCalls</code> is 10, then at least 10 calls must be recorded, before the failure rate can be calculated. If only 9 calls have been recorded the CircuitBreaker will not transition to open even if all 9 calls have failed. Default <code>minimumNumberOfCalls</code> is 100.	100	Integer
camel.resilience4j.permitted-number-of-calls-in-half-open-state	Configures the number of permitted calls when the CircuitBreaker is half open. The size must be greater than 0. Default size is 10.	10	Integer
camel.resilience4j.sliding-window-size	Configures the size of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. <code>slidingWindowSize</code> configures the size of the sliding window. Sliding window can either be count-based or time-based. If <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the last <code>slidingWindowSize</code> calls are recorded and aggregated. If <code>slidingWindowType</code> is <code>TIME_BASED</code> , the calls of the last <code>slidingWindowSize</code> seconds are recorded and aggregated. The <code>slidingWindowSize</code> must be greater than 0. The <code>minimumNumberOfCalls</code> must be greater than 0. If the <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the <code>minimumNumberOfCalls</code> cannot be greater than <code>slidingWindowSize</code> . If the <code>slidingWindowType</code> is <code>TIME_BASED</code> , you can pick whatever you want. Default <code>slidingWindowSize</code> is 100.	100	Integer
camel.resilience4j.sliding-window-type	Configures the type of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. Sliding window can either be count-based or time-based. If <code>slidingWindowType</code> is <code>COUNT_BASED</code> , the last <code>slidingWindowSize</code> calls are recorded and aggregated. If <code>slidingWindowType</code> is <code>TIME_BASED</code> , the calls of the last <code>slidingWindowSize</code> seconds are recorded and aggregated. Default <code>slidingWindowType</code> is <code>COUNT_BASED</code> .	COUNT_BASED	String
camel.resilience4j.slow-call-duration-threshold	Configures the duration threshold (seconds) above which calls are considered as slow and increase the slow calls percentage. Default value is 60 seconds.	60	Integer

Name	Description	Default	Type
camel.resilience4j.slow-call-rate-threshold	Configures a threshold in percentage. The CircuitBreaker considers a call as slow when the call duration is greater than <code>slowCallDurationThreshold</code> Duration. When the percentage of slow calls is equal or greater the threshold, the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 100 percentage which means that all recorded calls must be slower than <code>slowCallDurationThreshold</code> .		Float
camel.resilience4j.wait-duration-in-open-state	Configures the wait duration (in seconds) which specifies how long the CircuitBreaker should stay open, before it switches to half open. Default value is 60 seconds.	60	Integer
camel.resilience4j.writable-stack-trace-enabled	Enables writable stack traces. When set to false, <code>Exception.getStackTrace</code> returns a zero length array. This may be used to reduce log spam when the circuit breaker is open as the cause of the exceptions is already known (the circuit breaker is short-circuiting calls).	true	Boolean
camel.rest.api-component	The name of the Camel component to use as the REST API (such as swagger) If no API Component has been explicit configured, then Camel will lookup if there is a Camel component responsible for servicing and generating the REST API documentation, or if a <code>org.apache.camel.spi.RestApiProcessorFactory</code> is registered in the registry. If either one is found, then that is being used.		String
camel.rest.api-context-path	Sets a leading API context-path the REST API services will be using. This can be used when using components such as <code>camel-servlet</code> where the deployed web application is deployed using a context-path.		String
camel.rest.api-context-route-id	Sets the route id to use for the route that services the REST API. The route will by default use an auto assigned route id.		String
camel.rest.api-host	To use an specific hostname for the API documentation (eg swagger) This can be used to override the generated host with this configured hostname.		String

Name	Description	Default	Type
camel.rest.api-property	Allows to configure as many additional properties for the api documentation (swagger). For example set property api.title to my cool stuff.		Map
camel.rest.api-vendor-extension	Whether vendor extension is enabled in the Rest APIs. If enabled then Camel will include additional information as vendor extension (eg keys starting with x-) such as route ids, class names etc. Not all 3rd party API gateways and tools supports vendor-extensions when importing your API docs.	false	Boolean
camel.rest.binding-mode	Sets the binding mode to use. The default value is off.		RestBindingMode
camel.rest.client-request-validation	Whether to enable validation of the client request to check whether the Content-Type and Accept headers from the client is supported by the Rest-DSL configuration of its consumes/produces settings. This can be turned on, to enable this check. In case of validation error, then HTTP Status codes 415 or 406 is returned. The default value is false.	false	Boolean
camel.rest.component	The Camel Rest component to use for the REST transport (consumer), such as netty-http, jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String
camel.rest.component-property	Allows to configure as many additional properties for the rest component in use.		Map
camel.rest.consumer-property	Allows to configure as many additional properties for the rest consumer in use.		Map
camel.rest.context-path	Sets a leading context-path the REST services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path. Or for components such as camel-jetty or camel-netty-http that includes a HTTP server.		String
camel.rest.cors-headers	Allows to configure custom CORS headers.		Map

Name	Description	Default	Type
camel.rest.data-format-property	Allows to configure as many additional properties for the data formats in use. For example set property prettyPrint to true to have json outputted in pretty mode. The properties can be prefixed to denote the option is only for either JSON or XML and for either the IN or the OUT. The prefixes are: json.in. json.out. xml.in. xml.out. For example a key with value xml.out.mustBeJAXBElement is only for the XML data format for the outgoing. A key without a prefix is a common key for all situations.		Map
camel.rest.enable-cors	Whether to enable CORS headers in the HTTP response. The default value is false.	false	Boolean
camel.rest.endpoint-property	Allows to configure as many additional properties for the rest endpoint in use.		Map
camel.rest.host	The hostname to use for exposing the REST service.		String
camel.rest.host-name-resolver	If no hostname has been explicit configured, then this resolver is used to compute the hostname the REST service will be using.		RestHostNameResolver
camel.rest.json-data-format	Name of specific json data format to use. By default json-jackson will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.port	The port number to use for exposing the REST service. Notice if you use servlet component then the port number configured here does not apply, as the port number in use is the actual port number the servlet component is using. eg if using Apache Tomcat its the tomcat http port, if using Apache Karaf its the HTTP service in Karaf that uses port 8181 by default etc. Though in those situations setting the port number here, allows tooling and JMX to know the port number, so its recommended to set the port number to the number that the servlet engine uses.		String

Name	Description	Default	Type
camel.rest.producer-api-doc	Sets the location of the api document (swagger api) the REST producer will use to validate the REST uri and query parameters are valid accordingly to the api document. This requires adding camel-swagger-java to the classpath, and any miss configuration will let Camel fail on startup and report the error(s). The location of the api document is loaded from classpath by default, but you can use file: or http: to refer to resources to load from file or http url.		String
camel.rest.producer-component	Sets the name of the Camel component to use as the REST producer.		String
camel.rest.scheme	The scheme to use for exposing the REST service. Usually http or https is supported. The default value is http.		String
camel.rest.skip-binding-on-error-code	Whether to skip binding on output if there is a custom HTTP error code header. This allows to build custom error messages that do not bind to json / xml etc, as success messages otherwise will do.	false	Boolean
camel.rest.use-x-forward-headers	Whether to use X-Forward headers for Host and related setting. The default value is true.	true	Boolean
camel.rest.xml-data-format	Name of specific XML data format to use. By default jaxb will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.api-context-id-pattern	Deprecated Sets an CamelContext id pattern to only allow Rest APIs from rest services within CamelContext's which name matches the pattern. The pattern name refers to the CamelContext name, to match on the current CamelContext only. For any other value, the pattern uses the rules from PatternHelper#matchPattern(String,String).		String
camel.rest.api-context-listing	Deprecated Sets whether listing of all available CamelContext's with REST services in the JVM is enabled. If enabled it allows to discover these contexts, if false then only the current CamelContext is in use.	false	Boolean

CHAPTER 48. HL7

The HL7 component is used for working with the HL7 MLLP protocol and [HL7 v2 messages](#) using the [HAPI library](#).

This component supports the following:

- HL7 MLLP codec for [Mina](#)
- HL7 MLLP codec for [Netty](#)
- Type Converter from/to HAPI and String
- HL7 DataFormat using the HAPI library

48.1. DEPENDENCIES

When using **hl7** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-hl7-starter</artifactId>
</dependency>
```

48.2. HL7 MLLP PROTOCOL

HL7 is often used with the HL7 MLLP protocol, which is a text based TCP socket based protocol. This component ships with a Mina and Netty Codec that conforms to the MLLP protocol so you can easily expose an HL7 listener accepting HL7 requests over the TCP transport layer. To expose a HL7 listener service, the [camel-mina](#) or link:[camel-netty](#) component is used with the HL7MLLPCodec (mina) or HL7MLLPNettyDecoder/HL7MLLPNettyEncoder (Netty).

HL7 MLLP codec can be configured as follows:

Name	Default Value	Description
startByte	0x0b	The start byte spanning the HL7 payload.
endByte1	0x1c	The first end byte spanning the HL7 payload.
endByte2	0x0d	The 2nd end byte spanning the HL7 payload.
charset	JVM Default	The encoding (a charset name) to use for the codec. If not provided, Camel will use the JVM default Charset .
produceString	true	If true, the codec creates a string using the defined charset. If false, the codec sends a plain byte array into the route, so that the HL7 Data Format can determine the actual charset from the HL7 message content.

Name	Default Value	Description
convertLFtoCR	false	Will convert \n to \r (0x0d, 13 decimal) as HL7 stipulates \r as segment terminators. The HAPI library requires the use of \r .

48.2.1. Exposing an HL7 listener using Mina

In the Spring XML file, we configure a mina endpoint to listen for HL7 requests using TCP on port **8888**:

```
<endpoint id="hl7MinaListener" uri="mina:tcp://localhost:8888?sync=true&codec=#hl7codec"/>
```

sync=true indicates that this listener is synchronous and therefore will return a HL7 response to the caller. The HL7 codec is setup with **codec=#hl7codec**. Note that **hl7codec** is just a Spring bean ID, so it could be named **mygreatcodecforhl7** or whatever. The codec is also set up in the Spring XML file:

```
<bean id="hl7codec" class="org.apache.camel.component.hl7.HL7MLLPCodec">
  <property name="charset" value="iso-8859-1"/>
</bean>
```

The endpoint **hl7MinaListener** can then be used in a route as a consumer, as this Java DSL example illustrates:

```
from("hl7MinaListener")
  .bean("patientLookupService");
```

This is a very simple route that will listen for HL7 and route it to a service named **patientLookupService**. This is also Spring bean ID, configured in the Spring XML as:

```
<bean id="patientLookupService"
class="com.mycompany.healthcare.service.PatientLookupService"/>
```

The business logic can be implemented in POJO classes that do not depend on Camel, as shown here:

```
import ca.uhn.hl7v2.HL7Exception;
import ca.uhn.hl7v2.model.Message;
import ca.uhn.hl7v2.model.v24.segment.QRD;

public class PatientLookupService {
  public Message lookupPatient(Message input) throws HL7Exception {
    QRD qrd = (QRD)input.get("QRD");
    String patientId = qrd.getWhoSubjectFilter(0).getIDNumber().getValue();

    // find patient data based on the patient id and create a HL7 model object with the response
    Message response = ... create and set response data
    return response
  }
}
```

48.2.2. Exposing an HL7 listener using Netty (available from Camel 2.15 onwards)

In the Spring XML file, we configure a netty endpoint to listen for HL7 requests using TCP on port **8888**:

```
<endpoint id="hl7NettyListener" uri="netty:tcp://localhost:8888?
sync=true&encoders=#hl7encoder&decoders=#hl7decoder"/>
```

sync=true indicates that this listener is synchronous and therefore will return a HL7 response to the caller. The HL7 codec is setup with **encoders=#hl7encoder*and*decoders=#hl7decoder**. Note that **hl7encoder** and **hl7decoder** are just bean IDs, so they could be named differently. The beans can be set in the Spring XML file:

```
<bean id="hl7decoder" class="org.apache.camel.component.hl7.HL7MLLPNettyDecoderFactory"/>
<bean id="hl7encoder" class="org.apache.camel.component.hl7.HL7MLLPNettyEncoderFactory"/>
```

The endpoint **hl7NettyListener** can then be used in a route as a consumer, as this Java DSL example illustrates:

```
from("hl7NettyListener")
    .bean("patientLookupService");
```

48.3. HL7 MODEL USING JAVA.LANG.STRING OR BYTE[]

The HL7 MLLP codec uses plain String as its data format. Camel uses its Type Converter to convert to/from strings to the HAPI HL7 model objects, but you can use the plain String objects if you prefer, for instance if you wish to parse the data yourself.

You can also let both the Mina and Netty codecs use a plain **byte[]** as its data format by setting the **produceString** property to false. The Type Converter is also capable of converting the **byte[]** to/from HAPI HL7 model objects.

48.4. HL7V2 MODEL USING HAPI

The HL7v2 model uses Java objects from the HAPI library. Using this library, you can encode and decode from the EDI format (ER7) that is mostly used with HL7v2.

The sample below is a request to lookup a patient with the patient ID **0101701234**.

```
MSH|^~\&|MYSENDER|MYRECEIVER|MYAPPLICATION||200612211200||QRY^A19|1234|P|2.4
QRD|200612211200|R||GetPatient||1^RD|0101701234|DEM||
```

Using the HL7 model you can work with a **ca.uhn.hl7v2.model.Message** object, e.g. to retrieve a patient ID:

```
Message msg = exchange.getIn().getBody(Message.class);
QRD qrd = (QRD)msg.get("QRD");
String patientId = qrd.getWhoSubjectFilter(0).getIDNumber().getValue(); // 0101701234
```

This is powerful when combined with the HL7 listener, because you don't have to work with **byte[]**, **String** or any other simple object formats. You can just use the HAPI HL7v2 model objects. If you know the message type in advance, you can be more type-safe:

```
QRY_A19 msg = exchange.getIn().getBody(QRY_A19.class);
String patientId = msg.getQRD().getWhoSubjectFilter(0).getIDNumber().getValue();
```


48.5. HL7 DATAFORMAT

The **camel-hl7** JAR ships with a HL7 data format that can be used to marshal or unmarshal HL7 model objects.

The HL7 dataformat supports 1 options, which are listed below.

Name	Default	Java Type	Description
validate		Boolean	Whether to validate the HL7 message is by default true.

- **marshal** = from Message to byte stream (can be used when responding using the HL7 MLLP codec)
- **unmarshal** = from byte stream to Message (can be used when receiving streamed data from the HL7 MLLP)

To use the data format, simply instantiate an instance and invoke the marshal or unmarshal operation in the route builder:

```
DataFormat hl7 = new HL7DataFormat();

from("direct:hl7in")
  .marshal(hl7)
  .to("jms:queue:hl7out");
```

In the sample above, the HL7 is marshalled from a HAPI Message object to a byte stream and put on a JMS queue.

The next example is the opposite:

```
DataFormat hl7 = new HL7DataFormat();

from("jms:queue:hl7out")
  .unmarshal(hl7)
  .to("patientLookupService");
```

Here we unmarshal the byte stream into a HAPI Message object that is passed to our patient lookup service.

48.5.1. Segment separators

Unmarshalling does not automatically fix segment separators anymore by converting `\n` to `\r`. If you need this conversion, **org.apache.camel.component.hl7.HL7#convertLFToCR** provides a handy **Expression** for this purpose.

48.5.2. Charset

Both **marshal** and **unmarshal** evaluate the charset provided in the field **MSH-18**. If this field is empty, by default the charset contained in the corresponding Camel charset property/header is assumed. You can even change this default behavior by overriding the **guessCharsetName** method when inheriting

from the **HL7DataFormat** class.

There is a shorthand syntax in Camel for well-known data formats that are commonly used. Then you don't need to create an instance of the **HL7DataFormat** object:

```
from("direct:hl7in")
  .marshal().hl7()
  .to("jms:queue:hl7out");

from("jms:queue:hl7out")
  .unmarshal().hl7()
  .to("patientLookupService");
```

48.6. MESSAGE HEADERS

The unmarshal operation adds these fields from the MSH segment as headers on the Camel message:

Key	MSH field	Example
CamelHL7SendingApplication	MSH-3	MYSERVER
CamelHL7SendingFacility	MSH-4	MYSERVERAPP
CamelHL7ReceivingApplication	MSH-5	MYCLIENT
CamelHL7ReceivingFacility	MSH-6	MYCLIENTAPP
CamelHL7Timestamp	MSH-7	20071231235900
CamelHL7Security	MSH-8	null
CamelHL7MessageType	MSH-9-1	ADT
CamelHL7TriggerEvent	MSH-9-2	A01
CamelHL7MessageControl	MSH-10	1234
CamelHL7ProcessingId	MSH-11	P

Key	MSH field	Example
CamelHL7VersionId	MSH-12	2.4
CamelHL7Context		contains the that was used to parse the message
CamelHL7Charset	MSH-18	UNICODE UTF-8

All headers except **CamelHL7Context** are **String** types. If a header value is missing, its value is **null**.

48.7. DEPENDENCIES

To use HL7 in your Camel routes you'll need to add a dependency on **camel-hl7** listed above, which implements this data format.

The HAPI library is split into a [base library](#) and several structure libraries, one for each HL7v2 message version:

- [v2.1 structures library](#)
- [v2.2 structures library](#)
- [v2.3 structures library](#)
- [v2.3.1 structures library](#)
- [v2.4 structures library](#)
- [v2.5 structures library](#)
- [v2.5.1 structures library](#)
- [v2.6 structures library](#)

By default **camel-hl7** only references the HAPI [base library](#). Applications are responsible for including structure libraries themselves. For example, if an application works with HL7v2 message versions 2.4 and 2.5 then the following dependencies must be added:

```
<dependency>
  <groupId>ca.uhn.hapi</groupId>
  <artifactId>hapi-structures-v24</artifactId>
  <version>2.2</version>
  <!-- use the same version as your hapi-base version -->
</dependency>
<dependency>
  <groupId>ca.uhn.hapi</groupId>
  <artifactId>hapi-structures-v25</artifactId>
```

```
<version>2.2</version>
<!-- use the same version as your hapi-base version -->
</dependency>
```

Alternatively, an OSGi bundle containing the base library, all structures libraries and required dependencies (on the bundle classpath) can be downloaded from the [central Maven repository](#).

```
<dependency>
  <groupId>ca.uhn.hapi</groupId>
  <artifactId>hapi-osgi-base</artifactId>
  <version>2.2</version>
</dependency>
```

48.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
<code>camel.dataformat.hl7.enabled</code>	Whether to enable auto configuration of the hl7 data format. This is enabled by default.		Boolean
<code>camel.dataformat.hl7.validate</code>	Whether to validate the HL7 message ls by default true.	true	Boolean
<code>camel.language.hl7terser.enabled</code>	Whether to enable auto configuration of the hl7terser language. This is enabled by default.		Boolean
<code>camel.language.hl7terser.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

CHAPTER 49. HTTP

Only producer is supported

The HTTP component provides HTTP based endpoints for calling external HTTP resources (as a client to call external servers using HTTP).

49.1. DEPENDENCIES

When using **http** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-http-starter</artifactId>
</dependency>
```

49.2. URI FORMAT

```
http:hostname[:port][/resourceUri][?options]
```

Will by default use port 80 for HTTP and 443 for HTTPS.

49.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

49.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

49.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

49.4. COMPONENT OPTIONS

The HTTP component supports 40 options, which are listed below.

Name	Description	Default	Type
cookieStore (producer)	To use a custom <code>org.apache.http.client.CookieStore</code> . By default the <code>org.apache.http.impl.client.BasicCookieStore</code> is used which is an in-memory only cookie store. Notice if <code>bridgeEndpoint=true</code> then the cookie store is forced to be a noop cookie store as cookie shouldn't be stored as we are just bridging (eg acting as a proxy).		<code>CookieStore</code>
copyHeaders (producer)	If this option is true then IN exchange headers will be copied to OUT exchange headers according to copy strategy. Setting this to false, allows to only include the headers from the HTTP response (not propagating IN headers).	true	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
responsePayloadStreamingThreshold (producer)	This threshold in bytes controls whether the response payload should be stored in memory as a byte array or be streaming based. Set this to -1 to always use streaming mode.	8192	int

Name	Description	Default	Type
skipRequestHeaders (producer (advanced))	Whether to skip mapping all the Camel headers as HTTP request headers. If there are no data from Camel headers needed to be included in the HTTP request then this can avoid parsing overhead with many object allocations for the JVM garbage collector.	false	boolean
skipResponseHeaders (producer (advanced))	Whether to skip mapping all the HTTP response headers to Camel headers. If there are no data needed from HTTP headers then this can avoid parsing overhead with many object allocations for the JVM garbage collector.	false	boolean
followRedirects (producer (advanced))	Whether the HTTP request should follow redirects. By default the HTTP request does not follow redirects.	false	boolean
allowJavaSerializedObject (advanced)	Whether to allow java serialization when a request uses context-type=application/x-java-serialized-object. This is by default turned off. If you enable this then be aware that Java will deserialize the incoming data from the request to Java and that can be a potential security risk.	false	boolean
authCachingDisabled (advanced)	Disables authentication scheme caching.	false	boolean
automaticRetriesDisabled (advanced)	Disables automatic request recovery and re-execution.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
clientConnectionManager (advanced)	To use a custom and shared HttpClientConnectionManager to manage connections. If this has been configured then this is always used for all endpoints created by this component.		HttpClientConnectionManager
connectionsPerRoute (advanced)	The maximum number of connections per route.	20	int

Name	Description	Default	Type
connectionStateDisabled (advanced)	Disables connection state tracking.	false	boolean
connectionTimeToLive (advanced)	The time for connection to live, the time unit is millisecond, the default value is always keep alive.		long
contentCompressionDisabled (advanced)	Disables automatic content decompression.	false	boolean
cookieManagementDisabled (advanced)	Disables state (cookie) management.	false	boolean
defaultUserAgentDisabled (advanced)	Disables the default user agent set by this builder if none has been provided by the user.	false	boolean
httpBinding (advanced)	To use a custom HttpBinding to control the mapping between Camel message and HttpClient.		HttpBinding
httpClientConfigurer (advanced)	To use the custom HttpClientConfigurer to perform configuration of the HttpClient that will be used.		HttpClientConfigurer
httpConfiguration (advanced)	To use the shared HttpConfiguration as base configuration.		HttpConfiguration
httpContext (advanced)	To use a custom org.apache.http.protocol.HttpContext when executing requests.		HttpContext
maxTotalConnections (advanced)	The maximum number of connections.	200	int
redirectHandlingDisabled (advanced)	Disables automatic redirect handling.	false	boolean
headerFilterStrategy (filter)	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
proxyAuthDomain (proxy)	Proxy authentication domain to use.		String

Name	Description	Default	Type
proxyAuthHost (proxy)	Proxy authentication host.		String
proxyAuthMethod (proxy)	Proxy authentication method to use. Enum values: • Basic • Digest • NTLM		String
proxyAuthNtHost (proxy)	Proxy authentication domain (workstation name) to use with NTLM.		String
proxyAuthPassword (proxy)	Proxy authentication password.		String
proxyAuthPort (proxy)	Proxy authentication port.		Integer
proxyAuthUsername (proxy)	Proxy authentication username.		String
sslContextParameters (security)	To configure security using SSLContextParameters. Important: Only one instance of org.apache.camel.support.jsse.SSLContextParameters is supported per HttpComponent. If you need to use 2 or more different instances, you need to define a new HttpComponent per instance you need.		SSLContextParameters
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean
x509HostnameVerifier (security)	To use a custom X509HostnameVerifier such as DefaultHostnameVerifier or NoopHostnameVerifier.		HostnameVerifier
connectionRequestTimeout (timeout)	The timeout in milliseconds used when requesting a connection from the connection manager. A timeout value of zero is interpreted as an infinite timeout. A timeout value of zero is interpreted as an infinite timeout. A negative value is interpreted as undefined (system default).	-1	int

Name	Description	Default	Type
connectTimeout (timeout)	Determines the timeout in milliseconds until a connection is established. A timeout value of zero is interpreted as an infinite timeout. A timeout value of zero is interpreted as an infinite timeout. A negative value is interpreted as undefined (system default).	-1	int
responseTimeout (timeout)	Determines the timeout until arrival of a response from the opposite endpoint. A timeout value of zero is interpreted as an infinite timeout. Please note that response timeout may be unsupported by HTTP transports with message multiplexing.	0	Timeout
soTimeout (timeout)	Determines the default socket timeout value for blocking I/O operations.	3 minute s	Timeout

49.5. ENDPOINT OPTIONS

The HTTP endpoint is configured using URI syntax:

```
http://httpUri
```

with the following path and query parameters:

49.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
httpUri (common)	Required The url of the HTTP endpoint to call.		URI

49.5.2. Query Parameters (54 parameters)

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
disableStreamCache (common)	Determines whether or not the raw input stream is cached or not. The Camel consumer (camel-servlet, camel-jetty etc.) will by default cache the input stream to support reading it multiple times to ensure it Camel can retrieve all data from the stream. However you can set this option to true when you for example need to access the raw stream, such as streaming it directly to a file or other persistent store. DefaultHttpBinding will copy the request input stream into a stream cache and put it into message body if this option is false to support reading the stream multiple times. If you use Servlet to bridge/proxy an endpoint then consider enabling this option to improve performance, in case you do not need to read the message payload multiple times. The producer (camel-http) will by default cache the response body stream. If setting this option to true, then the producers will not cache the response body stream but use the response stream as-is (the stream can only be read once) as the message body.	false	boolean
headerFilterStrategy (common (advanced))	To use a custom HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
bridgeEndpoint (producer)	If the option is true, HttpProducer will ignore the Exchange.HTTP_URI header, and use the endpoint's URI for request. You may also set the option <code>throwExceptionOnFailure</code> to be false to let the HttpProducer send all the fault response back.	false	boolean
connectionClose (producer)	Specifies whether a Connection Close header must be added to HTTP Request. By default <code>connectionClose</code> is false.	false	boolean

Name	Description	Default	Type
httpMethod (producer)	Configure the HTTP method to use. The HttpMethod header cannot override this option if set. Enum values: • GET • POST • PUT • DELETE • HEAD • OPTIONS • TRACE • PATCH		HttpMethods
skipRequestHeaders (producer)	Whether to skip mapping all the Camel headers as HTTP request headers. If there are no data from Camel headers needed to be included in the HTTP request then this can avoid parsing overhead with many object allocations for the JVM garbage collector.	false	boolean
skipResponseHeaders (producer)	Whether to skip mapping all the HTTP response headers to Camel headers. If there are no data needed from HTTP headers then this can avoid parsing overhead with many object allocations for the JVM garbage collector.	false	boolean
throwExceptionOnFailure (producer)	Option to disable throwing the <code>HttpOperationFailedException</code> in case of failed responses from the remote server. This allows you to get all responses regardless of the HTTP status code.	true	boolean
clearExpiredCookies (producer (advanced))	Whether to clear expired cookies before sending the HTTP request. This ensures the cookies store does not keep growing by adding new cookies which is never removed when they are expired. If the component has disabled cookie management then this option is disabled too.	true	boolean
cookieHandler (producer (advanced))	Configure a cookie handler to maintain a HTTP session.		CookieHandler

Name	Description	Default	Type
cookieStore (producer (advanced))	To use a custom CookieStore. By default the BasicCookieStore is used which is an in-memory only cookie store. Notice if bridgeEndpoint=true then the cookie store is forced to be a noop cookie store as cookie shouldn't be stored as we are just bridging (eg acting as a proxy). If a cookieHandler is set then the cookie store is also forced to be a noop cookie store as cookie handling is then performed by the cookieHandler.		CookieStore
copyHeaders (producer (advanced))	If this option is true then IN exchange headers will be copied to OUT exchange headers according to copy strategy. Setting this to false, allows to only include the headers from the HTTP response (not propagating IN headers).	true	boolean
customHostHeader (producer (advanced))	To use custom host header for producer. When not set in query will be ignored. When set will override host header derived from url.		String
deleteWithBody (producer (advanced))	Whether the HTTP DELETE should include the message body or not. By default HTTP DELETE do not include any HTTP body. However in some rare cases users may need to be able to include the message body.	false	boolean
followRedirects (producer (advanced))	Whether the HTTP request should follow redirects. By default the HTTP request does not follow redirects.	false	boolean
getWithBody (producer (advanced))	Whether the HTTP GET should include the message body or not. By default HTTP GET do not include any HTTP body. However in some rare cases users may need to be able to include the message body.	false	boolean
ignoreResponseBody (producer (advanced))	If this option is true, The http producer won't read response body and cache the input stream.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
okStatusCodeRange (producer (advanced))	The status codes which are considered a success response. The values are inclusive. Multiple ranges can be defined, separated by comma, e.g. 200-204,209,301-304. Each range must be a single number or from-to with the dash included.	200-299	String
preserveHostHeader (producer (advanced))	If the option is true, HttpProducer will set the Host header to the value contained in the current exchange Host header, useful in reverse proxy applications where you want the Host header received by the downstream server to reflect the URL called by the upstream client, this allows applications which use the Host header to generate accurate URL's for a proxied service.	false	boolean
userAgent (producer (advanced))	To set a custom HTTP User-Agent request header.		String
clientBuilder (advanced)	Provide access to the http client request parameters used on new RequestConfig instances used by producers or consumers of this endpoint.		HttpClientBuilder
clientConnectionManager (advanced)	To use a custom HttpClientConnectionManager to manage connections.		HttpClientConnectionManager
connectionsPerRoute (advanced)	The maximum number of connections per route.	20	int
httpClient (advanced)	Sets a custom HttpClient to be used by the producer.		HttpClient
httpClientConfigurer (advanced)	Register a custom configuration strategy for new HttpClient instances created by producers or consumers such as to configure authentication mechanisms etc.		HttpClientConfigurer

Name	Description	Default	Type
httpClientOptions (advanced)	To configure the HttpClient using the key/values from the Map.		Map
httpConnectionOptions (advanced)	To configure the connection and the socket using the key/values from the Map.		Map
httpContext (advanced)	To use a custom HttpContext instance.		HttpContext
maxTotalConnections (advanced)	The maximum number of connections.	200	int
useSystemProperties (advanced)	To use System Properties as fallback for configuration.	false	boolean
proxyAuthDomain (proxy)	Proxy authentication domain to use with NTLM.		String
proxyAuthHost (proxy)	Proxy authentication host.		String
proxyAuthMethod (proxy)	Proxy authentication method to use. Enum values: • Basic • Digest • NTLM		String
proxyAuthNtHost (proxy)	Proxy authentication domain (workstation name) to use with NTLM.		String
proxyAuthPassword (proxy)	Proxy authentication password.		String
proxyAuthPort (proxy)	Proxy authentication port.		int
proxyAuthScheme (proxy)	Proxy authentication scheme to use. Enum values: • http • https		String

Name	Description	Default	Type
proxyAuthUsername (proxy)	Proxy authentication username.		String
proxyHost (proxy)	Proxy hostname to use.		String
proxyPort (proxy)	Proxy port to use.		int
authDomain (security)	Authentication domain to use with NTLM.		String
authenticationPreemptive (security)	If this option is true, camel-http sends preemptive basic authentication to the server.	false	boolean
authHost (security)	Authentication host to use with NTLM.		String
authMethod (security)	Authentication methods allowed to use as a comma separated list of values Basic, Digest or NTLM.		String
authMethodPriority (security)	Which authentication method to prioritize to use, either as Basic, Digest or NTLM. Enum values: • Basic • Digest • NTLM		String
authPassword (security)	Authentication password.		String
authUsername (security)	Authentication username.		String
oauth2ClientId (security)	OAuth2 client id.		String
oauth2ClientSecret (security)	OAuth2 client secret.		String
oauth2Scope (security)	OAuth2 scope.		String

Name	Description	Default	Type
oauth2TokenEndpoint (security)	OAuth2 Token endpoint.		String
sslContextParameters (security)	To configure security using SSLContextParameters. Important: Only one instance of org.apache.camel.util.jsse.SSLContextParameters is supported per HttpComponent. If you need to use 2 or more different instances, you need to define a new HttpComponent per instance you need.		SSLContextParameters
x509HostnameVerifier (security)	To use a custom X509HostnameVerifier such as DefaultHostnameVerifier or NoopHostnameVerifier.		HostnameVerifier

49.6. MESSAGE HEADERS

Name	Description	Default	Type
Content-Encoding (producer) Constant: CONTENT_ENCODING	The HTTP content encoding. Is set on both the IN and OUT message to provide a content encoding, such as gzip.		String
CamelHttpResponseCode (producer) Constant: HTTP_RESPONSE_CODE	The HTTP response code from the external server. Is 200 for OK.		int
CamelHttpResponseText (producer) Constant: HTTP_RESPONSE_TEXT	The HTTP response text from the external server.		String
CamelHttpQuery (producer) Constant: HTTP_QUERY	URI parameters. Will override existing URI parameters set directly on the endpoint.		String

Name	Description	Default	Type
CamelHttpProtocolVersion (producer) Constant: HTTP_PROTOCOL_VERSION	The version of the http protocol used.		String
Host (producer) Constant: HTTP_HEADER_HOST	The target host.		String
CamelRestHttpUri (producer) Constant: REST_HTTP_URI	The rest http URI.		String
CamelHttpUri (producer) Constant: HTTP_URI	URI to call. Will override existing URI set directly on the endpoint. This uri is the uri of the http server to call. Its not the same as the Camel endpoint uri, where you can configure endpoint options such as security etc. This header does not support that, its only the uri of the http server.		String
CamelHttpPath (producer) Constant: HTTP_PATH	Request URI's path, the header will be used to build the request URI with the HTTP_URI.		String
CamelRestHttpQuery (producer) Constant: REST_HTTP_QUERY	The rest http query.		String
CamelHttpRawQuery (producer) Constant: HTTP_RAW_QUERY	The http raw query.		String

Name	Description	Default	Type
CamelHttpMethod (producer) Constant: HTTP_METHOD	The http method to use. Enum values: • GET • PATCH • POST • PUT • DELETE • HEAD • OPTIONS • TRACE		HttpMethods
CamelHttpCharacterEncoding (producer) Constant: HTTP_CHARACTER_ENCODING	The character encoding.		String
Content-Type (producer) Constant: CONTENT_TYPE	The HTTP content type. Is set on both the IN and OUT message to provide a content type, such as text/html.		String



NOTE

Avoid creating your own HTTP server instances in a Camel Spring Boot application by using either of the **cxfrt-transport-jetty**, **cxfrt-transport-netty-server**, **cxfrt-transport-underflow** libraries. It is recommended to use the Spring Boot embedded HTTP server stack which is created when you use the **spring-boot-starter-web**, **cxfrt-spring-boot-starter-jaxrs**, or **cxfrt-spring-boot-starter-jaxws** dependencies.

49.7. MESSAGE BODY

Camel will store the HTTP response from the external server on the OUT body. All headers from the IN message will be copied to the OUT message, so headers are preserved during routing. Additionally Camel will add the HTTP response headers as well to the OUT message headers.

49.8. USING SYSTEM PROPERTIES

When setting `useSystemProperties` to `true`, the HTTP Client will look for the following System Properties and it will use it:

- `ssl.TrustManagerFactory.algorithm`
- [javax.net.ssl.trustStoreType](#)
- [javax.net.ssl.trustStore](#)
- [javax.net.ssl.trustStoreProvider](#)
- [javax.net.ssl.trustStorePassword](#)
- `java.home`
- `ssl.KeyManagerFactory.algorithm`
- [javax.net.ssl.keyStoreType](#)
- [javax.net.ssl.keyStore](#)
- [javax.net.ssl.keyStoreProvider](#)
- [javax.net.ssl.keyStorePassword](#)
- `http.proxyHost`
- `http.proxyPort`
- `http.nonProxyHosts`
- `http.keepAlive`
- `http.maxConnections`

49.9. RESPONSE CODE

Camel will handle according to the HTTP response code:

- Response code is in the range 100..299, Camel regards it as a success response.
- Response code is in the range 300..399, Camel regards it as a redirection response and will throw a **HttpOperationFailedException** with the information.
- Response code is 400+, Camel regards it as an external server failure and will throw a **HttpOperationFailedException** with the information.

throwExceptionOnFailure The option, **throwExceptionOnFailure**, can be set to **false** to prevent the **HttpOperationFailedException** from being thrown for failed response codes. This allows you to get any response from the remote server.

There is a sample below demonstrating this.

49.10. EXCEPTIONS

HttpOperationFailedException exception contains the following information:

- The HTTP status code
- The HTTP status line (text of the status code)

- Redirect location, if server returned a redirect
- Response body as a **java.lang.String**, if server provided a body as response

49.11. WHICH HTTP METHOD WILL BE USED

The following algorithm is used to determine what HTTP method should be used:

1. Use method provided as endpoint configuration (**httpMethod**).
2. Use method provided in header (**Exchange.HTTP_METHOD**).
3. **GET** if query string is provided in header.
4. **GET** if endpoint is configured with a query string.
5. **POST** if there is data to send (body is not **null**).
6. **GET** otherwise.

49.12. HOW TO GET ACCESS TO HTTPServletRequest AND HttpServletResponse

You can get access to these two using the Camel type converter system using

```
HttpServletRequest request = exchange.getIn().getBody(HttpServletRequest.class);
HttpServletResponse response = exchange.getIn().getBody(HttpServletResponse.class);
```



NOTE

You can get the request and response not just from the processor after the camel-jetty or camel-cxf endpoint.

49.13. CONFIGURING URI TO CALL

You can set the HTTP producer's URI directly from the endpoint URI. In the route below, Camel will call out to the external server, **oldhost**, using HTTP.

```
from("direct:start")
    .to("http://oldhost");
```

And the equivalent Spring sample:

```
<camelContext xmlns="http://activemq.apache.org/camel/schema/spring">
  <route>
    <from uri="direct:start"/>
    <to uri="http://oldhost"/>
  </route>
</camelContext>
```

You can override the HTTP endpoint URI by adding a header with the key, **Exchange.HTTP_URI**, on the message.

```
from("direct:start")
    .setHeader(Exchange.HTTP_URI, constant("http://newhost"))
    .to("http://oldhost");
```

In the sample above Camel will call the <http://newhost/> despite the endpoint is configured with <http://oldhost/>.

If the http endpoint is working in bridge mode, it will ignore the message header of **Exchange.HTTP_URI**.

49.14. CONFIGURING URI PARAMETERS

The **http** producer supports URI parameters to be sent to the HTTP server. The URI parameters can either be set directly on the endpoint URI or as a header with the key **Exchange.HTTP_QUERY** on the message.

```
from("direct:start")
  .to("http://oldhost?order=123&detail=short");
```

Or options provided in a header:

```
from("direct:start")
  .setHeader(Exchange.HTTP_QUERY, constant("order=123&detail=short"))
  .to("http://oldhost");
```

49.15. HOW TO SET THE HTTP METHOD (GET/PATCH/POST/PUT/DELETE/HEAD/OPTIONS/TRACE) TO THE HTTP PRODUCER

The HTTP component provides a way to set the HTTP request method by setting the message header. Here is an example:

```
from("direct:start")
  .setHeader(Exchange.HTTP_METHOD,
    constant(org.apache.camel.component.http.HttpMethods.POST))
  .to("http://www.google.com")
  .to("mock:results");
```

The method can be written a bit shorter using the string constants:

```
.setHeader("CamelHttpMethod", constant("POST"))
```

And the equivalent Spring sample:

```
<camelContext xmlns="http://activemq.apache.org/camel/schema/spring">
  <route>
    <from uri="direct:start"/>
    <setHeader name="CamelHttpMethod">
      <constant>POST</constant>
    </setHeader>
    <to uri="http://www.google.com"/>
    <to uri="mock:results"/>
  </route>
</camelContext>
```

49.16. USING CLIENT TIMEOUT - SO_TIMEOUT

See the [HttpSOTimeoutTest](#) unit test.

49.17. CONFIGURING A PROXY

The HTTP component provides a way to configure a proxy.

```
from("direct:start")
  .to("http://oldhost?proxyAuthHost=www.myproxy.com&proxyAuthPort=80");
```

There is also support for proxy authentication via the **proxyAuthUsername** and **proxyAuthPassword** options.

49.17.1. Using proxy settings outside of URI

To avoid System properties conflicts, you can set proxy configuration only from the CamelContext or URI.

Java DSL :

```
context.getGlobalOptions().put("http.proxyHost", "172.168.18.9");
context.getGlobalOptions().put("http.proxyPort", "8080");
```

Spring XML

```
<camelContext>
  <properties>
    <property key="http.proxyHost" value="172.168.18.9"/>
    <property key="http.proxyPort" value="8080"/>
  </properties>
</camelContext>
```

Camel will first set the settings from Java System or CamelContext Properties and then the endpoint proxy options if provided.

So you can override the system properties with the endpoint options.

There is also a **http.proxyScheme** property you can set to explicit configure the scheme to use.

49.18. CONFIGURING CHARSET

If you are using **POST** to send data you can configure the **charset** using the **Exchange** property:

```
exchange.setProperty(Exchange.CHARSET_NAME, "ISO-8859-1");
```

49.18.1. Sample with scheduled poll

This sample polls the Google homepage every 10 seconds and write the page to the file **message.html**:

```
from("timer://foo?fixedRate=true&delay=0&period=10000")
  .to("http://www.google.com")
  .setHeader(FileComponent.HEADER_FILE_NAME, "message.html")
  .to("file:target/google");
```

49.18.2. URI Parameters from the endpoint URI

In this sample we have the complete URI endpoint that is just what you would have typed in a web browser. Multiple URI parameters can of course be set using the **&** character as separator, just as you would in the web browser. Camel does no tricks here.

```
// we query for Camel at the Google page
template.sendBody("http://www.google.com/search?q=Camel", null);
```

49.18.3. URI Parameters from the Message

```
Map headers = new HashMap();
headers.put(Exchange.HTTP_QUERY, "q=Camel&lr=lang_en");
// we query for Camel and English language at Google
template.sendBody("http://www.google.com/search", null, headers);
```

In the header value above notice that it should **not** be prefixed with **?** and you can separate parameters as usual with the **&** char.

49.18.4. Getting the Response Code

You can get the HTTP response code from the HTTP component by getting the value from the Out message header with **Exchange.HTTP_RESPONSE_CODE**.

```
Exchange exchange = template.send("http://www.google.com/search", new Processor() {
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(Exchange.HTTP_QUERY, constant("hl=en&q=activemq"));
    }
});
Message out = exchange.getOut();
int responseCode = out.getHeader(Exchange.HTTP_RESPONSE_CODE, Integer.class);
```

49.19. DISABLING COOKIES

To disable cookies you can set the HTTP Client to ignore cookies by adding the following URI option:

```
httpClient.cookieSpec=ignore
```

49.20. BASIC AUTH WITH THE STREAMING MESSAGE BODY

In order to avoid the **NonRepeatableRequestException**, you need to do the Preemptive Basic Authentication by adding the option:

authenticationPreemptive=true

49.21. ADVANCED USAGE

If you need more control over the HTTP producer you should use the **HttpComponent** where you can set various classes to give you custom behavior.

49.21.1. Setting up SSL for HTTP Client

Using the JSSE Configuration Utility

The HTTP component supports SSL/TLS configuration through the [Camel JSSE Configuration Utility](#). This utility greatly decreases the amount of component specific code you need to write and is configurable at the endpoint and component levels.

The following examples demonstrate how to use the utility with the HTTP component.

Programmatic configuration of the component

```
KeyStoreParameters ksp = new KeyStoreParameters();
ksp.setResource("file:/users/home/server/keystore.jks");
ksp.setPassword("keystorePassword");

KeyManagersParameters kmp = new KeyManagersParameters();
kmp.setKeyStore(ksp);
kmp.setKeyPassword("keyPassword");

SSLContextParameters scp = new SSLContextParameters();
scp.setKeyManagers(kmp);

HttpComponent httpComponent = getContext().getComponent("https", HttpComponent.class);
httpComponent.setSslContextParameters(scp);
```

Spring DSL based configuration of endpoint

```
<camel:sslContextParameters
  id="sslContextParameters">
  <camel:keyManagers
    keyPassword="keyPassword">
    <camel:keyStore
      resource="file:/users/home/server/keystore.jks"
      password="keystorePassword"/>
    </camel:keyManagers>
  </camel:sslContextParameters>

  <to uri="https://127.0.0.1/mail/?sslContextParameters=#sslContextParameters"/>
```

Configuring Apache HTTP Client Directly

Basically camel-http component is built on the top of [Apache HttpClient](#). Please refer to [SSL/TLS customization](#) for details or have a look into the

org.apache.camel.component.http.HttpsServerTestSupport unit test base class.

You can also implement a custom **org.apache.camel.component.http.HttpClientConfigurer** to do some configuration on the http client if you need full control of it.

However if you *just* want to specify the keystore and truststore you can do this with Apache HTTP **HttpClientConfigurer**, for example:

```
KeyStore keystore = ...;
KeyStore truststore = ...;

SchemeRegistry registry = new SchemeRegistry();
registry.register(new Scheme("https", 443, new SSLSocketFactory(keystore, "mypassword",
truststore)));
```

And then you need to create a class that implements **HttpClientConfigurer**, and registers https protocol providing a keystore or truststore per example above. Then, from your camel route builder class you can hook it up like so:

```
HttpComponent httpComponent = getContext().getComponent("http", HttpComponent.class);
httpComponent.setHttpClientConfigurer(new MyHttpClientConfigurer());
```

If you are doing this using the Spring DSL, you can specify your **HttpClientConfigurer** using the URI. For example:

```
<bean id="myHttpClientConfigurer"
class="my.https.HttpClientConfigurer">
</bean>

<to uri="https://myhostname.com:443/myURL?httpClientConfigurer=myHttpClientConfigurer"/>
```

As long as you implement the HttpClientConfigurer and configure your keystore and truststore as described above, it will work fine.

Using HTTPS to authenticate gotchas

An end user reported that he had problem with authenticating with HTTPS. The problem was eventually resolved by providing a custom configured **org.apache.http.protocol.HttpContext**:

1. Create a (Spring) factory for HttpContexts.

```
public class HttpContextFactory {

    private String httpHost = "localhost";
    private String httpPort = "9001";

    private BasicHttpContext httpContext = new BasicHttpContext();
    private BasicAuthCache authCache = new BasicAuthCache();
    private BasicScheme basicAuth = new BasicScheme();

    public HttpContext getObject() {
        authCache.put(new HttpHost(httpHost, httpPort), basicAuth);

        httpContext.setAttribute(ClientContext.AUTH_CACHE, authCache);

        return httpContext;
    }

    // getter and setter
}
```

2. Declare an HttpContext in the Spring application context file.

```
<bean id="myHttpContext" factory-bean="httpContextFactory" factory-method="getObject"/>
```

3. Reference the context in the http URL:

```
<to uri="https://myhostname.com:443/myURL?httpContext=myHttpContext"/>
```

Using different SSLContextParameters

The [HTTP](#) component only support one instance of **org.apache.camel.support.jsse.SSLContextParameters** per component. If you need to use 2 or more different instances, then you need to setup multiple [HTTP](#) components as shown below. Where we have 2 components, each using their own instance of **sslContextParameters** property.

```
<bean id="http-foo" class="org.apache.camel.component.http.HttpComponent">
  <property name="sslContextParameters" ref="sslContextParams1"/>
  <property name="x509HostnameVerifier" ref="hostnameVerifier"/>
</bean>

<bean id="http-bar" class="org.apache.camel.component.http.HttpComponent">
  <property name="sslContextParameters" ref="sslContextParams2"/>
  <property name="x509HostnameVerifier" ref="hostnameVerifier"/>
</bean>
```

For more information about how to configure SSL see [http-ssl](#) example.

49.22. SPRING BOOT AUTO-CONFIGURATION

The component supports 41 options, which are listed below.

Name	Description	Default	Type
camel.component.http.allow-java-serialized-object	Whether to allow java serialization when a request uses context-type=application/x-java-serialized-object. This is by default turned off. If you enable this then be aware that Java will deserialize the incoming data from the request to Java and that can be a potential security risk.	false	Boolean
camel.component.http.auth-caching-disabled	Disables authentication scheme caching.	false	Boolean
camel.component.http.automatic-retries-disabled	Disables automatic request recovery and re-execution.	false	Boolean
camel.component.http.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.http.client-connection-manager</code>	To use a custom and shared <code>HttpClientConnectionManager</code> to manage connections. If this has been configured then this is always used for all endpoints created by this component. The option is a <code>org.apache.http.conn.HttpClientConnectionManager</code> type.		<code>HttpClientConnectionManager</code>
<code>camel.component.http.connect-timeout</code>	Determines the timeout in milliseconds until a connection is established. A timeout value of zero is interpreted as an infinite timeout. A timeout value of zero is interpreted as an infinite timeout. A negative value is interpreted as undefined (system default).	-1	Integer
<code>camel.component.http.connection-request-timeout</code>	The timeout in milliseconds used when requesting a connection from the connection manager. A timeout value of zero is interpreted as an infinite timeout. A timeout value of zero is interpreted as an infinite timeout. A negative value is interpreted as undefined (system default).	-1	Integer
<code>camel.component.http.connection-state-disabled</code>	Disables connection state tracking.	false	Boolean
<code>camel.component.http.connection-time-to-live</code>	The time for connection to live, the time unit is millisecond, the default value is always keep alive.		Long
<code>camel.component.http.connections-per-route</code>	The maximum number of connections per route.	20	Integer
<code>camel.component.http.content-compression-disabled</code>	Disables automatic content decompression.	false	Boolean
<code>camel.component.http.cookie-management-disabled</code>	Disables state (cookie) management.	false	Boolean

Name	Description	Default	Type
camel.component.http.cookie-store	To use a custom <code>org.apache.http.client.CookieStore</code> . By default the <code>org.apache.http.impl.client.BasicCookieStore</code> is used which is an in-memory only cookie store. Notice if <code>bridgeEndpoint=true</code> then the cookie store is forced to be a noop cookie store as cookie shouldn't be stored as we are just bridging (eg acting as a proxy). The option is a <code>org.apache.http.client.CookieStore</code> type.		<code>CookieStore</code>
camel.component.http.copy-headers	If this option is true then IN exchange headers will be copied to OUT exchange headers according to copy strategy. Setting this to false, allows to only include the headers from the HTTP response (not propagating IN headers).	true	Boolean
camel.component.http.default-user-agent-disabled	Disables the default user agent set by this builder if none has been provided by the user.	false	Boolean
camel.component.http.enabled	Whether to enable auto configuration of the http component. This is enabled by default.		Boolean
camel.component.http.follow-redirects	Whether to the HTTP request should follow redirects. By default the HTTP request does not follow redirects.	false	Boolean
camel.component.http.header-filter-strategy	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		<code>HeaderFilterStrategy</code>
camel.component.http.http-binding	To use a custom <code>HttpBinding</code> to control the mapping between Camel message and <code>HttpClient</code> . The option is a <code>org.apache.camel.http.common.HttpBinding</code> type.		<code>HttpBinding</code>
camel.component.http.http-client-configurer	To use the custom <code>HttpClientConfigurer</code> to perform configuration of the <code>HttpClient</code> that will be used. The option is a <code>org.apache.camel.component.http.HttpClientConfigurer</code> type.		<code>HttpClientConfigurer</code>
camel.component.http.http-configuration	To use the shared <code>HttpConfiguration</code> as base configuration. The option is a <code>org.apache.camel.http.common.HttpConfiguration</code> type.		<code>HttpConfiguration</code>

Name	Description	Default	Type
camel.component.http.http-context	To use a custom <code>org.apache.http.protocol.HttpContext</code> when executing requests. The option is a <code>org.apache.http.protocol.HttpContext</code> type.		HttpContext
camel.component.http.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.http.max-total-connections	The maximum number of connections.	200	Integer
camel.component.http.proxy-auth-domain	Proxy authentication domain to use.		String
camel.component.http.proxy-auth-host	Proxy authentication host.		String
camel.component.http.proxy-auth-method	Proxy authentication method to use.		String
camel.component.http.proxy-auth-nt-host	Proxy authentication domain (workstation name) to use with NTLM.		String
camel.component.http.proxy-auth-password	Proxy authentication password.		String
camel.component.http.proxy-auth-port	Proxy authentication port.		Integer

Name	Description	Default	Type
<code>camel.component.http.proxy-auth-scheme</code>	Proxy authentication protocol scheme.		String
<code>camel.component.http.proxy-auth-username</code>	Proxy authentication username.		String
<code>camel.component.http.redirect-handling-disabled</code>	Disables automatic redirect handling.	false	Boolean
<code>camel.component.http.response-payload-streaming-threshold</code>	This threshold in bytes controls whether the response payload should be stored in memory as a byte array or be streaming based. Set this to -1 to always use streaming mode.	8192	Integer
<code>camel.component.http.response-timeout</code>	Determines the timeout until arrival of a response from the opposite endpoint. A timeout value of zero is interpreted as an infinite timeout. Please note that response timeout may be unsupported by HTTP transports with message multiplexing. The option is a <code>org.apache.hc.core5.util.Timeout</code> type.		Timeout
<code>camel.component.http.skip-request-headers</code>	Whether to skip mapping all the Camel headers as HTTP request headers. If there are no data from Camel headers needed to be included in the HTTP request then this can avoid parsing overhead with many object allocations for the JVM garbage collector.	false	Boolean
<code>camel.component.http.skip-response-headers</code>	Whether to skip mapping all the HTTP response headers to Camel headers. If there are no data needed from HTTP headers then this can avoid parsing overhead with many object allocations for the JVM garbage collector.	false	Boolean
<code>camel.component.http.so-timeout</code>	Determines the default socket timeout value for blocking I/O operations. The option is a <code>org.apache.hc.core5.util.Timeout</code> type.		Timeout

Name	Description	Default	Type
camel.component.http.ssl-context-parameters	To configure security using SSLContextParameters. Important: Only one instance of org.apache.camel.support.jsse.SSLContextParameters is supported per HttpComponent. If you need to use 2 or more different instances, you need to define a new HttpComponent per instance you need. The option is a org.apache.camel.support.jsse.SSLContextParameters type.		SSLContextParameters
camel.component.http.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean
camel.component.http.x509-hostname-verifier	To use a custom X509HostnameVerifier such as DefaultHostnameVerifier or NoopHostnameVerifier. The option is a javax.net.ssl.HostnameVerifier type.		HostnameVerifier

CHAPTER 50. HASHICORP VAULT

Since Camel 3.18

Only producer is supported

The hashicorp-vault component that integrates [Hashicorp Vault](#).

50.1. DEPENDENCIES

When using **hashicorp-vault** Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-hashicorp-vault-starter</artifactId>
</dependency>
```

50.2. CONFIGURING OPTIONS

Camel components are configured on two levels:

- Component level
- Endpoint level

50.2.1. Component Level Options

The **component level** is the highest level. The configurations you define at this level are inherited by all the endpoints. For example, a component can have security settings, credentials for authentication, urls for network connection, and so on.

Since components typically have pre-configured defaults for the most common cases, you may need to only configure a few component options, or maybe none at all.

You can configure components with

- [Component DSL](#)
- in a configuration file (**application.properties** or **yaml**),
- directly with Java code.

50.2.2. Endpoint Level Options

At the **Endpoint level** you have many options, which you can use to configure what you want the endpoint to do. The options are categorized according to whether the endpoint is used as a consumer (*from*) or as a producer (*to*) or used for both.

You can configure endpoints directly in the endpoint URI as **path** and **query** parameters. You can also use [Endpoint DSL](#) and [DataFormat DSL](#) as *type safe* ways of configuring endpoints and data formats in Java.

When configuring options, use [Property Placeholders](#) for urls, port numbers, sensitive information, and other settings.

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

50.3. COMPONENT OPTIONS

The Hashicorp Vault component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

50.4. ENDPOINT OPTIONS

The Hashicorp Vault endpoint is configured using URI syntax:

```
hashicorp-vault:secretsEngine
```

With the following *path* and *query* parameters:

50.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
secretsEngine (producer)	Vault Name to be used.		String

50.4.2. Query Parameters (8 parameters)

Name	Description	Default	Type
host (producer)	Hashicorp Vault instance host to be used.		String
operation (producer)	Operation to be performed. Enum values: • createSecret • getSecret • deleteSecret • listSecrets		Hashicorp Vault Operation
port (producer)	Hashicorp Vault instance port to be used.	8200	String
scheme (producer)	Hashicorp Vault instance scheme to be used.	https	String
secretPath (producer)	Hashicorp Vault instance secret Path to be used.		String
vaultTemplate (producer)	Autowired Instance of Vault template.		Vault Template

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
token (security)	Token to be used.		String

50.5. MESSAGE HEADERS

The Hashicorp Vault component supports 3 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelHashicorpVaultProducerOperation (producer) Constant: OPERATION	Overrides the desired operation to be used in the producer.		String
CamelHashicorpVaultSecretPath (producer) Constant: SECRET_PATH	Set the desired secret path as header.		String
CamelHashicorpVaultSecretVersion (producer) Constant: SECRET_VERSION	Set the desired secret version as header.		String

50.6. EXAMPLES

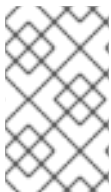
50.6.1. Using Hashicorp Vault Property Function

To use this function, you'll need to provide credentials for Hashicorp vault as environment variables:

```
export $CAMEL_VAULT_HASHICORP_TOKEN=token
export $CAMEL_VAULT_HASHICORP_HOST=host
export $CAMEL_VAULT_HASHICORP_PORT=port
export $CAMEL_VAULT_HASHICORP_SCHEME=http/https
```

You can also configure the credentials in the **application.properties** file such as:

```
camel.vault.hashicorp.token = token
camel.vault.hashicorp.host = host
camel.vault.hashicorp.port = port
camel.vault.hashicorp.scheme = scheme
```



NOTE

camel.vault.hashicorp configuration only applies to the Hashicorp Vault properties function (E.g when resolving properties). When using the **operation** option to create, get, list secrets etc., you should provide the **host**, **port**, **scheme** (if required) & **token** options.

At this point, you'll be able to reference a property in the following way:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{hashicorp:secret:route}}"/>
  </route>
</camelContext>
```

Where **route** is the name of the secret stored in the Hashicorp Vault instance, in the 'secret' engine.

You must specify a default value in case the secret is not present on Hashicorp Vault instance:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{hashicorp:secret:route:default}}"/>
  </route>
</camelContext>
```

In this case, if the secret doesn't exist in the 'secret' engine, the property will fall back to "default" as value.

Also, you are able to get a particular field of the secret, if you have, for example, a secret named database of this form:

```
{
  "username": "admin",
  "password": "password123",
  "engine": "postgres",
  "host": "127.0.0.1",
```

```
"port": "3128",
"dbname": "db"
}
```

You can have a single secret value in your route, in the 'secret' engine, for example:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{hashicorp:secret:database/username}}"/>
  </route>
</camelContext>
```

Or re-use the property as part of an endpoint.

Specify a default value in case the particular field of secret is not present on Hashicorp Vault instance, in the 'secret' engine:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="Username is {{hashicorp:secret:database/username:admin}}"/>
  </route>
</camelContext>
```

In this case, if the secret doesn't exist or the secret exists (in the 'secret' engine) but the username field is not part of the secret, the property will fall back to "admin" as value.

There is also the syntax to get a particular version of the secret for both the approach, with field/default value specified or only with secret:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{hashicorp:secret:route@2}}"/>
  </route>
</camelContext>
```

This approach returns the RAW route secret with version '2', in the 'secret' engine.

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <to uri="{{hashicorp:route:default@2}}"/>
  </route>
</camelContext>
```

This approach will return the route secret value with version '2' or default value in case the secret doesn't exist or the version doesn't exist (in the 'secret' engine).

```
<camelContext>
  <route>
    <from uri="direct:start"/>
```

```

    <log message="Username is {{hashicorp:secret:database/username:admin@2}}"/>
  </route>
</camelContext>

```

This approach will return the username field of the database secret with version '2' or admin in case the secret doesn't exist or the version doesn't exist (in the 'secret' engine).

The only requirement is adding the camel-hashicorp-vault jar to your Camel application.

50.7. SPRING BOOT AUTO-CONFIGURATION

When using hashicorp-vault with Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.component.hashicorp-vault.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.hashicorp-vault.enabled	Whether to enable auto configuration of the hashicorp-vault component. This is enabled by default.		Boolean
camel.component.hashicorp-vault.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 51. INFINISPAN

Both producer and consumer are supported

This component allows you to interact with [Infinispan](#) distributed data grid / cache using the Hot Rod protocol. Infinispan is an extremely scalable, highly available key/value data store and data grid platform written in Java.

51.1. DEPENDENCIES

When using **infinispan** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-infinispan-starter</artifactId>
</dependency>
```

51.2. URI FORMAT

```
infinispan://cacheName?[options]
```

The producer allows sending messages to a remote cache using the HotRod protocol. The consumer allows listening for events from a remote cache using the HotRod protocol.

51.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

51.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

51.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

51.4. COMPONENT OPTIONS

The Infinispan component supports 26 options, which are listed below.

Name	Description	Default	Type
configuration (common)	Component configuration.		InfinispanRemote Configuration
hosts (common)	Specifies the host of the cache on Infinispan instance.		String
queryBuilder (common)	Specifies the query builder.		InfinispanQueryBu ilder
secure (common)	Define if we are connecting to a secured Infinispan instance.	false	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
customListener (consumer)	Returns the custom listener in use, if provided.		InfinispanRemote CustomListener

Name	Description	Default	Type
eventTypes (consumer)	Specifies the set of event types to register by the consumer. Multiple event can be separated by comma. The possible event types are: CLIENT_CACHE_ENTRY_CREATED, CLIENT_CACHE_ENTRY_MODIFIED, CLIENT_CACHE_ENTRY_REMOVED, CLIENT_CACHE_ENTRY_EXPIRED, CLIENT_CACHE_FAILOVER.		String
defaultValue (producer)	Set a specific default value for some producer operations.		Object
key (producer)	Set a specific key for producer operations.		Object
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
oldValue (producer)	Set a specific old value for some producer operations.		Object

Name	Description	Default	Type
operation (producer)	The operation to perform. Enum values: • PUT • PUTASYNC • PUTALL • PUTALLASYNC • PUTIFABSENT • PUTIFABSENTASYNC • GET • GETORDEFAULT • CONTAINSKEY • CONTAINSVALUE • REMOVE • REMOVEASYNC • REPLACE • REPLACEASYNC • SIZE • CLEAR • CLEARASYNC • QUERY • STATS • COMPUTE • COMPUTEASYNC	PUT	InfinispanOperation
value (producer)	Set a specific value for producer operations.		Object
password (security)	Define the password to access the infinispan instance.		String
saslMechanism (security)	Define the SASL Mechanism to access the infinispan instance.		String

Name	Description	Default	Type
securityRealm (security)	Define the security realm to access the infinispn instance.		String
securityServerName (security)	Define the security server name to access the infinispn instance.		String
username (security)	Define the username to access the infinispn instance.		String
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
cacheContainer (advanced)	Autowired Specifies the cache Container to connect.		RemoteCacheManager
cacheContainerConfiguration (advanced)	Autowired The CacheContainer configuration. Used if the cacheContainer is not defined.		Configuration
configurationProperties (advanced)	Implementation specific properties for the CacheManager.		Map
configurationUri (advanced)	An implementation specific URI for the CacheManager.		String
flags (advanced)	A comma separated list of org.infinispn.client.hotrod.Flag to be applied by default on each cache invocation.		String
remappingFunction (advanced)	Set a specific remappingFunction to use in a compute operation.		BiFunction
resultHeader (advanced)	Store the operation result in a header instead of the message body. By default, resultHeader == null and the query result is stored in the message body, any existing content in the message body is discarded. If resultHeader is set, the value is used as the name of the header to store the query result and the original message body is preserved. This value can be overridden by an in message header named: CamelInfinispnOperationResultHeader.		String

51.5. ENDPOINT OPTIONS

The Infinispan endpoint is configured using URI syntax:

```
infinispan:cacheName
```

with the following path and query parameters:

51.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
cacheName (common)	Required The name of the cache to use. Use current to use the existing cache name from the currently configured cached manager. Or use default for the default cache manager name.		String

51.5.2. Query Parameters (26 parameters)

Name	Description	Default	Type
hosts (common)	Specifies the host of the cache on Infinispan instance.		String
queryBuilder (common)	Specifies the query builder.		InfinispanQueryBuilder
secure (common)	Define if we are connecting to a secured Infinispan instance.	false	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
customListener (consumer)	Returns the custom listener in use, if provided.		InfinispanRemoteCustomListener

Name	Description	Default	Type
eventTypes (consumer)	Specifies the set of event types to register by the consumer. Multiple event can be separated by comma. The possible event types are: CLIENT_CACHE_ENTRY_CREATED, CLIENT_CACHE_ENTRY_MODIFIED, CLIENT_CACHE_ENTRY_REMOVED, CLIENT_CACHE_ENTRY_EXPIRED, CLIENT_CACHE_FAILOVER.		String
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
defaultValue (producer)	Set a specific default value for some producer operations.		Object
key (producer)	Set a specific key for producer operations.		Object
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
oldValue (producer)	Set a specific old value for some producer operations.		Object

Name	Description	Default	Type
operation (producer)	The operation to perform. Enum values: • PUT • PUTASYNC • PUTALL • PUTALLASYNC • PUTIFABSENT • PUTIFABSENTASYNC • GET • GETORDEFAULT • CONTAINSKEY • CONTAINSVALUE • REMOVE • REMOVEASYNC • REPLACE • REPLACEASYNC • SIZE • CLEAR • CLEARASYNC • QUERY • STATS • COMPUTE • COMPUTEASYNC	PUT	InfinispanOperation
value (producer)	Set a specific value for producer operations.		Object
password (security)	Define the password to access the infinispan instance.		String
saslMechanism (security)	Define the SASL Mechanism to access the infinispan instance.		String

Name	Description	Default	Type
securityRealm (security)	Define the security realm to access the infinispn instance.		String
securityServerName (security)	Define the security server name to access the infinispn instance.		String
username (security)	Define the username to access the infinispn instance.		String
cacheContainer (advanced)	Autowired Specifies the cache Container to connect.		RemoteCacheManager
cacheContainerConfiguration (advanced)	Autowired The CacheContainer configuration. Used if the cacheContainer is not defined.		Configuration
configurationProperties (advanced)	Implementation specific properties for the CacheManager.		Map
configurationUri (advanced)	An implementation specific URI for the CacheManager.		String
flags (advanced)	A comma separated list of org.infinispn.client.hotrod.Flag to be applied by default on each cache invocation.		String
remappingFunction (advanced)	Set a specific remappingFunction to use in a compute operation.		BiFunction
resultHeader (advanced)	Store the operation result in a header instead of the message body. By default, resultHeader == null and the query result is stored in the message body, any existing content in the message body is discarded. If resultHeader is set, the value is used as the name of the header to store the query result and the original message body is preserved. This value can be overridden by an in message header named: CamelInfinispnOperationResultHeader.		String

51.6. CAMEL OPERATIONS

This section lists all available operations, along with their header information.

Table 51.1. Table 1. Put Operations

Operation Name	Description
InfinispanOperation.PUT	Puts a key/value pair in the cache, optionally with expiration
InfinispanOperation.PUTASYNC	Asynchronously puts a key/value pair in the cache, optionally with expiration
InfinispanOperation.PUTIFABSENT	Puts a key/value pair in the cache if it did not exist, optionally with expiration
InfinispanOperation.PUTIFABSENTASYNC	Asynchronously puts a key/value pair in the cache if it did not exist, optionally with expiration

- **Required Headers:**
 - CamelInfinispanKey
 - CamelInfinispanValue
- **Optional Headers:**
 - CamelInfinispanLifespanTime
 - CamelInfinispanLifespanTimeUnit
 - CamelInfinispanMaxIdleTime
 - CamelInfinispanMaxIdleTimeUnit
- **Result Header:**
 - CamelInfinispanOperationResult

Table 51.2. Table 2. Put All Operations

Operation Name	Description
InfinispanOperation.PUTALL	Adds multiple entries to a cache, optionally with expiration
CamelInfinispanOperation.PUTALLASYNC	Asynchronously adds multiple entries to a cache, optionally with expiration

- **Required Headers:**
 - CamelInfinispanMap
- **Optional Headers:**
 - CamelInfinispanLifespanTime
 - CamelInfinispanLifespanTimeUnit
 - CamelInfinispanMaxIdleTime

- `CamelInfinispanMaxIdleTimeUnit`

Table 51.3. Table 3. Get Operations

Operation Name	Description
<code>InfinispanOperation.GET</code>	Retrieves the value associated with a specific key from the cache
<code>InfinispanOperation.GETORDEFAULT</code>	Retrieves the value, or default value, associated with a specific key from the cache

- **Required Headers:**
 - `CamelInfinispanKey`

Table 51.4. Table 4. Contains Key Operation

Operation Name	Description
<code>InfinispanOperation.CONTAINSKEY</code>	Determines whether a cache contains a specific key

- **Required Headers**
 - `CamelInfinispanKey`
- **Result Header**
 - `CamelInfinispanOperationResult`

Table 51.5. Table 5. Contains Value Operation

Operation Name	Description
<code>InfinispanOperation.CONTAINSVALUE</code>	Determines whether a cache contains a specific value

- **Required Headers:**
 - `CamelInfinispanKey`

Table 51.6. Table 6. Remove Operations

Operation Name	Description
<code>InfinispanOperation.REMOVE</code>	Removes an entry from a cache, optionally only if the value matches a given one
<code>InfinispanOperation.REMOVEASYNC</code>	Asynchronously removes an entry from a cache, optionally only if the value matches a given one

- **Required Headers:**

- CamelInfinispanKey
- **Optional Headers:**
 - CamelInfinispanValue
- **Result Header:**
 - CamelInfinispanOperationResult

Table 51.7. Table 7. Replace Operations

Operation Name	Description
InfinispanOperation.REPLACE	Conditionally replaces an entry in the cache, optionally with expiration
InfinispanOperation.REPLACEASYNC	Asynchronously conditionally replaces an entry in the cache, optionally with expiration

- **Required Headers:**
 - CamelInfinispanKey
 - CamelInfinispanValue
 - CamelInfinispanOldValue
- **Optional Headers:**
 - CamelInfinispanLifespanTime
 - CamelInfinispanLifespanTimeUnit
 - CamelInfinispanMaxIdleTime
 - CamelInfinispanMaxIdleTimeUnit
- **Result Header:**
 - CamelInfinispanOperationResult

Table 51.8. Table 8. Clear Operations

Operation Name	Description
InfinispanOperation.CLEAR	Clears the cache
InfinispanOperation.CLEARASYNC	Asynchronously clears the cache

Table 51.9. Table 9. Size Operation

Operation Name	Description
InfinispanOperation.SIZE	Returns the number of entries in the cache

- **Result Header**
 - CamelInfinispanOperationResult

Table 51.10. Table 10. Stats Operation

Operation Name	Description
InfinispanOperation.STATS	Returns statistics about the cache

- **Result Header:**
 - CamelInfinispanOperationResult

Table 51.11. Table 11. Query Operation

Operation Name	Description
InfinispanOperation.QUERY	Executes a query on the cache

- **Required Headers:**
 - CamelInfinispanQueryBuilder
- **Result Header:**
 - CamelInfinispanOperationResult

**NOTE**

Write methods like `put(key, value)` and `remove(key)` do not return the previous value by default.

51.7. MESSAGE HEADERS

Name	Default Value	Type	Context	Description
CamelInfinispanCacheName	null	String	Shared	The cache participating in the operation or event.
CamelInfinispanOperation	PUT	InfinispanOperation	Producer	The operation to perform.

Name	Default Value	Type	Context	Description
CamelInfinispanMap	null	Map	Producer	A Map to use in case of CamelInfinispanOperationPutAll operation
CamelInfinispanKey	null	Object	Shared	The key to perform the operation to or the key generating the event.
CamelInfinispanValue	null	Object	Producer	The value to use for the operation.
CamelInfinispanEventType	null	String	Consumer	The type of the received event.
CamelInfinispanLifespanTime	null	long	Producer	The Lifespan time of a value inside the cache. Negative values are interpreted as infinity.
CamelInfinispanTimeUnit	null	String	Producer	The Time Unit of an entry Lifespan Time.
CamelInfinispanMaxIdleTime	null	long	Producer	The maximum amount of time an entry is allowed to be idle for before it is considered as expired.
CamelInfinispanMaxIdleTimeUnit	null	String	Producer	The Time Unit of an entry Max Idle Time.
CamelInfinispanQueryBuilder	null	InfinispanQueryBuilder	Producer	The QueryBuilde to use for QUERY command, if not present the command defaults to InifinispanConfiguration's one
CamelInfinispanOperationResultHeader	null	String	Producer	Store the operation result in a header instead of the message body

51.8. EXAMPLES

- Put a key/value into a named cache:

```
from("direct:start")
  .setHeader(InfinispanConstants.OPERATION).constant(InfinispanOperation.PUT) (1)
  .setHeader(InfinispanConstants.KEY).constant("123") (2)
  .to("infinispan:myCacheName&cacheContainer=#cacheContainer"); (3)
```

Where,

- 1 - Set the operation to perform
- 2 - Set the key used to identify the element in the cache
- 3 - Use the configured cache manager **cacheContainer** from the registry to put an element to the cache named **myCacheName**

It is possible to configure the lifetime and/or the idle time before the entry expires and gets evicted from the cache, as example:

```
from("direct:start")
    .setHeader(InfinispanConstants.OPERATION).constant(InfinispanOperation.GET)
    .setHeader(InfinispanConstants.KEY).constant("123")
    .setHeader(InfinispanConstants.LIFESPAN_TIME).constant(100L) (1)

    .setHeader(InfinispanConstants.LIFESPAN_TIME_UNIT).constant(TimeUnit.MILLISECONDS.toString()) (2)
    .to("infinispan:myCacheName");
```

where,

- 1 - Set the lifespan of the entry
- 2 - Set the time unit for the lifespan

Queries

```
from("direct:start")
    .setHeader(InfinispanConstants.OPERATION, InfinispanConstants.QUERY)
    .setHeader(InfinispanConstants.QUERY_BUILDER, new InfinispanQueryBuilder() {
        @Override
        public Query build(QueryFactory<Query> qf) {
            return qf.from(User.class).having("name").like("%abc%").build();
        }
    })
    .to("infinispan:myCacheName?cacheContainer=#cacheManager") ;
```



NOTE

The .proto descriptors for domain objects must be registered with the remote Data Grid server, see [Remote Query Example](#) in the official Infinispan documentation.

Custom Listeners

```
from("infinispan://?cacheContainer=#cacheManager&customListener=#myCustomListener")
    .to("mock:result");
```

The instance of **myCustomListener** must exist and Camel should be able to look it up from the **Registry**. Users are encouraged to extend the **org.apache.camel.component.infinispan.remote.InfinispanRemoteCustomListener** class and annotate the resulting class with **@ClientListener** which can be found in package **org.infinispan.client.hotrod.annotation**.

51.9. USING THE INFINISPAN BASED IDEMPOTENT REPOSITORY

In this section we will use the Infinispan based idempotent repository.

Java Example

```
InfinispanRemoteConfiguration conf = new InfinispanRemoteConfiguration(); (1)
conf.setHosts("localhost:11222")

InfinispanRemoteldempotentRepository repo = new
InfinispanRemoteldempotentRepository("idempotent"); (2)
repo.setConfiguration(conf);

context.addRoutes(new RouteBuilder() {
    @Override
    public void configure() {
        from("direct:start")
            .idempotentConsumer(header("MessageID"), repo) (3)
            .to("mock:result");
    }
});
```

where,

- 1 - Configure the cache
- 2 - Configure the repository bean
- 3 - Set the repository to the route

XML Example

```
<bean id="infinispanRepo"
class="org.apache.camel.component.infinispan.remote.InfinispanRemoteldempotentRepository"
destroy-method="stop">
    <constructor-arg value="idempotent"/> (1)
    <property name="configuration"> (2)
        <bean class="org.apache.camel.component.infinispan.remote.InfinispanRemoteConfiguration">
            <property name="hosts" value="localhost:11222"/>
        </bean>
    </property>
</bean>

<camelContext xmlns="http://camel.apache.org/schema/spring">
    <route>
        <from uri="direct:start" />
        <idempotentConsumer messageIdRepositoryRef="infinispanRepo"> (3)
            <header>MessageID</header>
            <to uri="mock:result" />
        </idempotentConsumer>
    </route>
</camelContext>
```

where,

- 1 - Set the name of the cache that will be used by the repository
- 2 - Configure the repository bean
- 3 - Set the repository to the route

51.10. USING THE INFINISPAN BASED AGGREGATION REPOSITORY

In this section we will use the Infinispan based aggregation repository.

Java Example

```
InfinispanRemoteConfiguration conf = new InfinispanRemoteConfiguration(); (1)
conf.setHosts("localhost:11222")

InfinispanRemoteAggregationRepository repo = new InfinispanRemoteAggregationRepository(); (2)
repo.setCacheName("aggregation");
repo.setConfiguration(conf);

context.addRoutes(new RouteBuilder() {
    @Override
    public void configure() {
        from("direct:start")
            .aggregate(header("MessageID"))
            .completionSize(3)
            .aggregationRepository(repo) (3)
            .aggregationStrategyRef("myStrategy")
            .to("mock:result");
    }
});
```

where,

- 1 - Configure the cache
- 2 - Create the repository bean
- 3 - Set the repository to the route

XML Example

```
<bean id="infinispanRepo"
class="org.apache.camel.component.infinispan.remote.InfinispanRemoteAggregationRepository"
destroy-method="stop">
    <constructor-arg value="aggregation"/> (1)
    <property name="configuration"> (2)
        <bean class="org.apache.camel.component.infinispan.remote.InfinispanRemoteConfiguration">
            <property name="hosts" value="localhost:11222"/>
        </bean>
    </property>
</bean>

<camelContext xmlns="http://camel.apache.org/schema/spring">
    <route>
        <from uri="direct:start" />
```



```

<aggregate strategyRef="myStrategy"
  completionSize="3"
  aggregationRepositoryRef="infinispanRepo"> (3)
  <correlationExpression>
    <header>MessageID</header>
  </correlationExpression>
  <to uri="mock:result"/>
</aggregate>
</route>
</camelContext>

```

where,

- 1 - Set the name of the cache that will be used by the repository
- 2 - Configure the repository bean
- 3 - Set the repository to the route



NOTE

With the release of Infinispan 11, it is required to set the encoding configuration on any cache created. This is critical for consuming events too. For more information have a look at [Data Encoding and MediaTypes](#) in the official Infinispan documentation.

51.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 23 options, which are listed below.

Name	Description	Default	Type
camel.component.infinispan.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.infinispan.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.infinispan.cache-container</code>	Specifies the cache Container to connect. The option is a <code>org.infinispan.client.hotrod.RemoteCacheManager</code> type.		<code>RemoteCacheManager</code>
<code>camel.component.infinispan.cache-container-configuration</code>	The <code>CacheContainer</code> configuration. Used if the <code>cacheContainer</code> is not defined. The option is a <code>org.infinispan.client.hotrod.configuration.Configuration</code> type.		<code>Configuration</code>
<code>camel.component.infinispan.configuration</code>	Component configuration. The option is a <code>org.apache.camel.component.infinispan.remote.InfinispanRemoteConfiguration</code> type.		<code>InfinispanRemoteConfiguration</code>
<code>camel.component.infinispan.configuration-properties</code>	Implementation specific properties for the <code>CacheManager</code> .		<code>Map</code>
<code>camel.component.infinispan.configuration-uri</code>	An implementation specific URI for the <code>CacheManager</code> .		<code>String</code>
<code>camel.component.infinispan.custom-listener</code>	Returns the custom listener in use, if provided. The option is a <code>org.apache.camel.component.infinispan.remote.InfinispanRemoteCustomListener</code> type.		<code>InfinispanRemoteCustomListener</code>
<code>camel.component.infinispan.enabled</code>	Whether to enable auto configuration of the <code>infinispan</code> component. This is enabled by default.		<code>Boolean</code>
<code>camel.component.infinispan.event-types</code>	Specifies the set of event types to register by the consumer. Multiple event can be separated by comma. The possible event types are: <code>CLIENT_CACHE_ENTRY_CREATED</code> , <code>CLIENT_CACHE_ENTRY_MODIFIED</code> , <code>CLIENT_CACHE_ENTRY_REMOVED</code> , <code>CLIENT_CACHE_ENTRY_EXPIRED</code> , <code>CLIENT_CACHE_FAILOVER</code> .		<code>String</code>
<code>camel.component.infinispan.flags</code>	A comma separated list of <code>org.infinispan.client.hotrod.Flag</code> to be applied by default on each cache invocation.		<code>String</code>
<code>camel.component.infinispan.hosts</code>	Specifies the host of the cache on <code>Infinispan</code> instance.		<code>String</code>

Name	Description	Default	Type
camel.component.infinispan.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.infinispan.operation	The operation to perform.		InfinispanOperation
camel.component.infinispan.password	Define the password to access the infinispan instance.		String
camel.component.infinispan.query-builder	Specifies the query builder. The option is a org.apache.camel.component.infinispan.InfinispanQueryBuilder type.		InfinispanQueryBuilder
camel.component.infinispan.remapping-function	Set a specific remappingFunction to use in a compute operation. The option is a java.util.function.BiFunction type.		BiFunction
camel.component.infinispan.result-header	Store the operation result in a header instead of the message body. By default, resultHeader == null and the query result is stored in the message body, any existing content in the message body is discarded. If resultHeader is set, the value is used as the name of the header to store the query result and the original message body is preserved. This value can be overridden by an in message header named: CamelInfinispanOperationResultHeader.		String
camel.component.infinispan.sasl-mechanism	Define the SASL Mechanism to access the infinispan instance.		String
camel.component.infinispan.secure	Define if we are connecting to a secured Infinispan instance.	false	Boolean

Name	Description	Default	Type
<code>camel.component.infinispan.security-realm</code>	Define the security realm to access the infinispan instance.		String
<code>camel.component.infinispan.security-server-name</code>	Define the security server name to access the infinispan instance.		String
<code>camel.component.infinispan.username</code>	Define the username to access the infinispan instance.		String

CHAPTER 52. INFINISPAN EMBEDDED

Since Camel 2.13

Both producer and consumer are supported

This component allows you to interact with [Infinispan](#) distributed data grid / cache. Infinispan is an extremely scalable, highly available key / value data store and data grid platform written in Java.

The **camel-infinispan-embedded** component includes the following features.

- **Local Camel Consumer** - Receives cache change notifications and sends them to be processed. This can be done synchronously or asynchronously, and is also supported with a replicated or distributed cache.
- **Local Camel Producer** - A producer creates and sends messages to an endpoint. The **camel-infinispan** producer uses **GET**, **PUT**, **REMOVE**, and **CLEAR** operations. The local producer is also supported with a replicated or distributed cache.

The events are processed asynchronously.

52.1. DEPENDENCIES

When using **infinispan-embedded** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-infinispan-embedded-starter</artifactId>
  <version>x.x.x</version>
  <!-- use the same version as your Camel core version -->
</dependency>
```

52.2. URI FORMAT

```
infinispan-embedded://cacheName?[options]
```

The producer allows sending messages to a local infinispan cache. The consumer allows listening for events from local infinispan cache.

If no cache configuration is provided, embedded cacheContainer is created directly in the component.

52.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

52.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the

endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

52.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

52.4. COMPONENT OPTIONS

The Infinispan Embedded component supports 20 options that are listed below.

Name	Description	Default	Type
configuration (common)	Component configuration.		InfinispanEmbeddedCon figuration
queryBuilder (common)	Specifies the query builder.		InfinispanQueryBuilder

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
clusteredListener (consumer)	If true, the listener will be installed for the entire cluster.	false	boolean
customListener (consumer)	Returns the custom listener in use, if provided.		<code>InfinispanEmbeddedCustomListener</code>

Name	Description	Default	Type
eventTypes (consumer)	Specifies the set of event types to register by the consumer. Multiple event can be separated by comma. The possible event types are: CACHE_ENTRY_ACTIVATED, CACHE_ENTRY_PASIVATED, CACHE_ENTRY_VISITED, CACHE_ENTRY_LOADED, CACHE_ENTRY_EVICTED, CACHE_ENTRY_CREATED, CACHE_ENTRY_REMOVED, CACHE_ENTRY_MODIFIED, TRANSACTION_COMPLETED, TRANSACTION_REGISTERED, CACHE_ENTRY_INVALIDATED, CACHE_ENTRY_EXPIRED, DATA_REHASHED, TOPOLOGY_CHANGED, PARTITION_STATUS_CHANGED, PERSISTENCE_AVAILABILITY_CHANGED.		String
sync (consumer)	If true, the consumer will receive notifications synchronously.	true	boolean
defaultValue (producer)	Set a specific default value for some producer operations.		Object
key (producer)	Set a specific key for producer operations.		Object

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
oldValue (producer)	Set a specific old value for some producer operations.		Object
operation (producer)	The operation to perform. Enum values: * PUT * PUTASYNC * PUTALL * PUTALLASYNC * PUTIFABSENT * PUTIFABSENTASYNC * GET * GETORDEFAULT * * CONTAINSKEY * CONTAINSVALUE * REMOVE * REMOVEASYNC * REPLACE * REPLACEASYNC * SIZE * * CLEAR * CLEARASYNC * QUERY * * STATS * COMPUTE * COMPUTEASYNC	PUT	InfinispanOperation

Name	Description	Default	Type
value* (producer)	Set a specific value for producer operations.		Object
autowiredEnabled (advanced)	Whether auto-wiring is enabled. This is used for automatic auto-wiring options (the option must be marked as auto-wired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
cacheContainer (advanced)	Autowired Specifies the cache Container to connect.		EmbeddedCacheManager
cacheContainerConfiguration (advanced)	Autowired The CacheContainer configuration. Used if the cacheContainer is not defined.		Configuration
configurationUri (advanced)	An implementation specific URI for the CacheManager.		String
flags (advanced)	A comma separated list of org.infinispan.context.Flag to be applied by default on each cache invocation.		String
remappingFunction (advanced)	Set a specific remappingFunction to use in a compute operation.		BiFunction

Name	Description	Default	Type
resultHeader (advanced)	Store the operation result in a header instead of the message body. By default, resultHeader == null and the query result is stored in the message body, any existing content in the message body is discarded. If resultHeader is set, the value is used as the name of the header to store the query result and the original message body is preserved. This value can be overridden by an in message header named: CamelInfinispanOperationResultHeader.		String

52.5. ENDPOINT OPTIONS

The Infinispan Embedded endpoint is configured using URI syntax.

`infinispan-embedded:cacheName`

Following are the path and query parameters.

52.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
cacheName (common)	Required The name of the cache to use. Use current to use the existing cache name from the currently configured cached manager. Or use default for the default cache manager name.		String

52.5.2. Query Parameters (20 parameters)

Name	Description	Default	Type
queryBuilder (common)	Specifies the query builder.		InfinispanQueryBuilder
clusteredListener (consumer)	If true, the listener will be installed for the entire cluster.	false	boolean
customListener (consumer)	Returns the custom listener in use, if provided.		InfinispanEmbeddedCustomListener
eventTypes (consumer)	Specifies the set of event types to register by the consumer. Multiple event can be separated by comma. The possible event types are: CACHE_ENTRY_ACTIVATED, CACHE_ENTRY_PASIVATED, CACHE_ENTRY_VISITED, CACHE_ENTRY_LOADED, CACHE_ENTRY_EVICTED, CACHE_ENTRY_CREATED, CACHE_ENTRY_REMOVED, CACHE_ENTRY_MODIFIED, TRANSACTION_COMPLETED, TRANSACTION_REGISTERED, CACHE_ENTRY_INVALIDATED, CACHE_ENTRY_EXPIRED, DATA_REHASHED, TOPOLOGY_CHANGED, PARTITION_STATUS_CHANGED, PERSISTENCE_AVAILABILITY_CHANGED.		String

Name	Description	Default	Type
sync (consumer)	If true, the consumer will receive notifications synchronously.	true	boolean
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: * InOnly * InOut * InOptionalOut		<code>ExchangePattern</code>

Name	Description	Default	Type
defaultValue (producer)	Set a specific default value for some producer operations.		Object
key (producer)	Set a specific key for producer operations.		Object
oldValue (producer)	Set a specific old value for some producer operations.		Object
operation (producer)	The operation to perform. Enum values: * PUT * PUTASYNC * PUTALL * PUTALLASYNC * PUTIFABSENT * PUTIFABSENTASYNC * GET * GETORDEFAULT * * CONTAINSKEY * CONTAINSVALUE * REMOVE * REMOVEASYNC * REPLACE * REPLACEASYNC * SIZE * CLEAR * CLEARASYNC * QUERY * STATS * COMPUTE * COMPUTEASYNC	PUT	InfinispanOperation
value (producer)	Set a specific value for producer operations.		Object

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
cacheContainer (advanced)	Autowired Specifies the cache Container to connect.		EmbeddedCacheManager
cacheContainerConfiguration (advanced)	Autowired The CacheContainer configuration. Used if the cacheContainer is not defined.		Configuration
configurationUri (advanced)	An implementation specific URI for the CacheManager.		String
flags (advanced)	A comma separated list of org.infinispan.context.Flag to be applied by default on each cache invocation.		String

Name	Description	Default	Type
remappingFunction (advanced)	Set a specific remappingFunction to use in a compute operation.		BiFunction
resultHeader (advanced)	Store the operation result in a header instead of the message body. By default, resultHeader == null and the query result is stored in the message body, any existing content in the message body is discarded. If resultHeader is set, the value is used as the name of the header to store the query result and the original message body is preserved. This value can be overridden by an in message header named: CamelInfinispanOperationResultHeader.		String

52.6. MESSAGE HEADERS

The Infinispan Embedded component supports 22 message headers that are listed below.

Name	Description	Default	Type
CamelInfinispanEventType (consumer) Constant: EVENT_TYPE	The type of the received event.		String
CamelInfinispanIsPre (consumer) Constant: IS_PRE	true if the notification is before the event has occurred, false if after the event has occurred.		boolean

Name	Description	Default	Type
CamelInfinispanCacheName (common) Constant: CACHE_NAME	The cache participating in the operation or event.		String
CamelInfinispanKey (common) Constant: KEY	The key to perform the operation to or the key generating the event.		Object
CamelInfinispanValue (producer) Constant: VALUE	The value to use for the operation.		Object
CamelInfinispanDefaultValue (producer) Constant: DEFAULT_VALUE	The default value to use for a getOrDefault.		Object
CamelInfinispanOldValue (producer) Constant: OLD_VALUE	The old value to use for a replace.		Object
CamelInfinispanMap (producer) Constant: MAP	A Map to use in case of CamelInfinispanOperationPutAll operation.		Map

Name	Description	Default	Type
CamelInfinispanOperation (producer) Constant: OPERATION	The operation to perform. Enum values: * PUT * PUTASYNC * PUTALL * PUTALLASYNC * PUTIFABSENT * PUTIFABSENTASYNC * GET * GETORDEFAULT * * CONTAINSKEY * CONTAINSVALUE * REMOVE * REMOVEASYNC * REPLACE * REPLACEASYNC * SIZE * * CLEAR * CLEARASYNC * QUERY * * STATS * COMPUTE * COMPUTEASYNC		InfinispanOperation
CamelInfinispanOperationResult (producer) Constant: RESULT	The name of the header whose value is the result.		String
CamelInfinispanOperationResultHeader (producer) Constant: RESULT_HEADER	Store the operation result in a header instead of the message body.		String
CamelInfinispanLifespanTime (producer) Constant: LIFESPAN_TIME	The Lifespan time of a value inside the cache. Negative values are interpreted as infinity.		long
CamelInfinispanTimeUnit (producer) Constant: LIFESPAN_TIME_UNIT	The Time Unit of an entry Lifespan Time. Enum values: * NANoseconds * MICROSECONDS * MILLISECONDs * SECONDS * MINUTES * HOURS * DAYS		TimeUnit

Name	Description	Default	Type
CamelInfinispanMaxIdleTime (producer) Constant: MAX_IDLE_TIME	The maximum amount of time an entry is allowed to be idle for before it is considered as expired.		long
CamelInfinispanMaxIdleTimeUnit (producer) Constant: MAX_IDLE_TIME_UNIT	The Time Unit of an entry Max Idle Time. Enum values: * NANoseconds * MICROseconds * MILLIseconds * SECONDS * MINUTES * HOURS * DAYS		TimeUnit
CamelInfinispanIgnoreReturnValues (consumer) Constant: IGNORE_RETURN_VALUES	Signals that write operation's return value are ignored, so reading the existing value from a store or from a remote node is not necessary.	false	boolean
CamelInfinispanEventData (consumer) Constant: EVENT_DATA	The event data.		Object
CamelInfinispanQueryBuilder (producer) Constant: QUERY_BUILDER	The QueryBuilder to use for QUERY command, if not present the command defaults to InfinispanConfiguration's one.		InfinispanQueryBuilder
CamelInfinispanCommandRetried (consumer) Constant: COMMAND_RETRIED	This will be true if the write command that caused this had to be retried again due to a topology change.		boolean
CamelInfinispanEntryCreated (consumer) Constant: ENTRY_CREATED	Indicates whether the cache entry modification event is the result of the cache entry being created.		boolean

Name	Description	Default	Type
CamelInfinispanOriginLocal (consumer) Constant: ORIGIN_LOCAL	true if the call originated on the local cache instance; false if originated from a remote one.		boolean
CamelInfinispanCurrentState (consumer) Constant: CURRENT_STATE	True if this event is generated from an existing entry as the listener has Listener.		boolean

52.7. CAMEL OPERATIONS

This section lists all available operations along with their header information.

Table 52.1. Table 1. Put Operations

Operation Name	Description
InfinispanOperation.PUT	Puts a key/value pair in the cache, optionally with expiration
InfinispanOperation.PUTASYNC	Asynchronously puts a key/value pair in the cache, optionally with expiration
InfinispanOperation.PUTIFABSENT	Puts a key/value pair in the cache if it did not exist, optionally with expiration
InfinispanOperation.PUTIFABSENTASYNC	Asynchronously puts a key/value pair in the cache if it did not exist, optionally with expiration

- **Required Headers:**
 - CamelInfinispanKey
 - CamelInfinispanValue
- **Optional Headers:**
 - CamelInfinispanLifespanTime
 - CamelInfinispanLifespanTimeUnit
 - CamelInfinispanMaxIdleTime
 - CamelInfinispanMaxIdleTimeUnit
- **Result Header:**
 - CamelInfinispanOperationResult

– CamelInfinispanOperationResult

Table 52.2. Table 2. Put All Operations

Operation Name	Description
InfinispanOperation.PUTALL	Adds multiple entries to a cache, optionally with expiration
CamelInfinispanOperation.PUTALLASYNC	Asynchronously adds multiple entries to a cache, optionally with expiration

- **Required Headers:**
 - CamelInfinispanMap
- **Optional Headers:**
 - CamelInfinispanLifespanTime
 - CamelInfinispanLifespanTimeUnit
 - CamelInfinispanMaxIdleTime
 - CamelInfinispanMaxIdleTimeUnit

Table 52.3. Table 3. Get Operations

Operation Name	Description
InfinispanOperation.GET	Retrieves the value associated with a specific key from the cache
InfinispanOperation.GETORDEFAULT	Retrieves the value, or default value, associated with a specific key from the cache

- **Required Headers:**
 - CamelInfinispanKey

Table 52.4. Table 4. Contains Key Operation

Operation Name	Description
InfinispanOperation.CONTAINSKEY	Determines whether a cache contains a specific key

- **Required Headers**
 - CamelInfinispanKey
- **Result Header**
 - CamelInfinispanOperationResult

Table 52.5. Table 5. Contains Value Operation

Operation Name	Description
InfinispanOperation.CONTAINSVALUE	Determines whether a cache contains a specific value

- **Required Headers:**
 - CamelInfinispanKey

Table 52.6. Table 6. Remove Operations

Operation Name	Description
InfinispanOperation.REMOVE	Removes an entry from a cache, optionally only if the value matches a given one
InfinispanOperation.REMOVEASYNC	Asynchronously removes an entry from a cache, optionally only if the value matches a given one

- **Required Headers:**
 - CamelInfinispanKey
- **Optional Headers:**
 - CamelInfinispanValue
- **Result Header:**
 - CamelInfinispanOperationResult

Table 52.7. Table 7. Replace Operations

Operation Name	Description
InfinispanOperation.REPLACE	Conditionally replaces an entry in the cache, optionally with expiration
InfinispanOperation.REPLACEASYNC	Asynchronously conditionally replaces an entry in the cache, optionally with expiration

- **Required Headers:**
 - CamelInfinispanKey
 - CamelInfinispanValue
 - CamelInfinispanOldValue
- **Optional Headers:**

- `CamellInfinispanLifespanTime`
- `CamellInfinispanLifespanTimeUnit`
- `CamellInfinispanMaxIdleTime`
- `CamellInfinispanMaxIdleTimeUnit`
- **Result Header:**
 - `CamellInfinispanOperationResult`

Table 52.8. Table 8. Clear Operations

Operation Name	Description
<code>InfinispanOperation.CLEAR</code>	Clears the cache
<code>InfinispanOperation.CLEARASYNC</code>	Asynchronously clears the cache

Table 52.9. Table 9. Size Operation

Operation Name	Description
<code>InfinispanOperation.SIZE</code>	Returns the number of entries in the cache

- **Result Header**
 - `CamellInfinispanOperationResult`

Table 52.10. Table 10. Stats Operation

Operation Name	Description
<code>InfinispanOperation.STATS</code>	Returns statistics about the cache

- **Result Header:**
 - `CamellInfinispanOperationResult`

Table 52.11. Table 11. Query Operation

Operation Name	Description
<code>InfinispanOperation.QUERY</code>	Executes a query on the cache

- **Required Headers:**
 - `CamellInfinispanQueryBuilder`
- **Result Header:**

- CamelInfinispanOperationResult



NOTE

Write methods like `put(key, value)` and `remove(key)` do not return the previous value by default.

52.8. EXAMPLES

- Put a key/value into a named cache:

```
from("direct:start")
  .setHeader(InfinispanConstants.OPERATION).constant(InfinispanOperation.PUT) (1)
  .setHeader(InfinispanConstants.KEY).constant("123") (2)
  .to("infinispan:myCacheName&cacheContainer=#cacheContainer"); (3)
```

- Set the operation to perform
- Set the key used to identify the element in the cache
- Use the configured cache manager **cacheContainer** from the registry to put an element to the cache named **myCacheName**
It is possible to configure the lifetime and/or the idle time before the entry expires and gets evicted from the cache, as example.

```
from("direct:start")
  .setHeader(InfinispanConstants.OPERATION).constant(InfinispanOperation.GET)
  .setHeader(InfinispanConstants.KEY).constant("123")
  .setHeader(InfinispanConstants.LIFESPAN_TIME).constant(100L) (1)

  .setHeader(InfinispanConstants.LIFESPAN_TIME_UNIT).constant(TimeUnit.MILLISECONDS.toString()) (2)
  .to("infinispan:myCacheName");
```

- Set the lifespan of the entry
- Set the time unit for the lifespan

- Queries

```
from("direct:start")
  .setHeader(InfinispanConstants.OPERATION, InfinispanConstants.QUERY)
  .setHeader(InfinispanConstants.QUERY_BUILDER, new InfinispanQueryBuilder() {
    @Override
    public Query build(QueryFactory<Query> qf) {
      return qf.from(User.class).having("name").like("%abc%").build();
    }
  })
  .to("infinispan:myCacheName?cacheContainer=#cacheManager");
```

- Custom Listeners

```
from("infinispan://?cacheContainer=#cacheManager&customListener=#myCustomListener")
  .to("mock:result");
```


- The instance of **myCustomListener** must exist and Camel should be able to look it up from the **Registry**. Users are encouraged to extend the **org.apache.camel.component.infinispan.embedded.InfinispanEmbeddedCustomListener** class and annotate the resulting class with **@Listener** which can be found in package **org.infinispan.notifications**.

52.9. USING THE INFINISPAN BASED IDEMPOTENT REPOSITORY

Java Example

```
InfinispanEmbeddedConfiguration conf = new InfinispanEmbeddedConfiguration(); (1)
conf.setConfigurationUri("classpath:infinispan.xml")

InfinispanEmbeddedIdempotentRepository repo = new
InfinispanEmbeddedIdempotentRepository("idempotent"); (2)
repo.setConfiguration(conf);

context.addRoutes(new RouteBuilder() {
    @Override
    public void configure() {
        from("direct:start")
            .idempotentConsumer(header("MessageID"), repo) (3)
            .to("mock:result");
    }
});
```

- Configure the cache
- Configure the repository bean
- Set the repository to the route

XML Example

```
<bean id="infinispanRepo"
class="org.apache.camel.component.infinispan.embedded.InfinispanEmbeddedIdempotentRepository"
destroy-method="stop">
    <constructor-arg value="idempotent"/> (1)
    <property name="configuration"> (2)
        <bean
class="org.apache.camel.component.infinispan.embedded.InfinispanEmbeddedConfiguration">
            <property name="configurationUri" value="classpath:infinispan.xml"/>
        </bean>
    </property>
</bean>

<camelContext xmlns="http://camel.apache.org/schema/spring">
    <route>
        <from uri="direct:start" />
        <idempotentConsumer idempotentRepository="infinispanRepo"> (3)
            <header>MessageID</header>
            <to uri="mock:result" />
        </idempotentConsumer>
    </route>
</camelContext>
```

- Set the name of the cache that will be used by the repository
- Configure the repository bean
- Set the repository to the route

52.10. USING THE INFINISPAN BASED AGGREGATION REPOSITORY

Java Example

```
InfinispanEmbeddedConfiguration conf = new InfinispanEmbeddedConfiguration(); (1)
conf.setConfigurationUri("classpath:infinispan.xml")

InfinispanEmbeddedAggregationRepository repo = new
InfinispanEmbeddedAggregationRepository("aggregation"); (2)
repo.setConfiguration(conf);

context.addRoutes(new RouteBuilder() {
    @Override
    public void configure() {
        from("direct:start")
            .aggregate(header("MessageID"))
            .completionSize(3)
            .aggregationRepository(repo) (3)
            .aggregationStrategy("myStrategy")
            .to("mock:result");
    }
});
```

- Configure the cache
- Create the repository bean
- Set the repository to the route

XML Example

```
<bean id="infinispanRepo"
class="org.apache.camel.component.infinispan.embedded.InfinispanEmbeddedAggregationRepository"
destroy-method="stop">
    <constructor-arg value="aggregation"/> (1)
    <property name="configuration"> (2)
        <bean
class="org.apache.camel.component.infinispan.embedded.InfinispanEmbeddedConfiguration">
            <property name="configurationUri" value="classpath:infinispan.xml"/>
        </bean>
    </property>
</bean>

<camelContext xmlns="http://camel.apache.org/schema/spring">
    <route>
        <from uri="direct:start" />
        <aggregate aggregationStrategy="myStrategy"
            completionSize="3">
```

```

        aggregationRepository="infinispanRepo"> (3)
    <correlationExpression>
        <header>MessageID</header>
    </correlationExpression>
    <to uri="mock:result"/>
</aggregate>
</route>
</camelContext>

```

- Set the name of the cache that will be used by the repository
- Configure the repository bean
- Set the repository to the route



NOTE

With the release of Infinispan 11, it is required to set the encoding configuration on any cache created. This is critical for consuming events too. For more information have a look at [Data Encoding and MediaTypes](#) in the official Infinispan documentation.

52.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 17 options that are listed below.

Name	Description	Default	Type
camel.component.infinispan-embedded.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.infinispan-embedded.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.infinispan-embedded.cache-container	Specifies the cache Container to connect. The option is a <code>org.infinispan.manager.EmbeddedCacheManager</code> type.		<code>EmbeddedCacheManager</code>
camel.component.infinispan-embedded.cache-container-configuration	The <code>CacheContainer</code> configuration. Used if the <code>cacheContainer</code> is not defined. The option is a <code>org.infinispan.configuration.cache.Configuration</code> type.		Configuration
camel.component.infinispan-embedded.clustered-listener	If true, the listener will be installed for the entire cluster.	false	Boolean
camel.component.infinispan-embedded.configuration	Component configuration. The option is a <code>org.apache.camel.component.infinispan.embedded.InfinispanEmbeddedConfiguration</code> type.		<code>InfinispanEmbeddedConfiguration</code>

Name	Description	Default	Type
<code>camel.component.infinispan-embedded.configuration-uri</code>	An implementation specific URI for the CacheManager.		String
<code>camel.component.infinispan-embedded.custom-listener</code>	Returns the custom listener in use, if provided. The option is a <code>org.apache.camel.component.infinispan.embedded.InfinispanEmbeddedCustomListener</code> type.		<code>InfinispanEmbeddedCustomListener</code>
<code>camel.component.infinispan-embedded.enabled</code>	Whether to enable auto configuration of the <code>infinispan-embedded</code> component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.infinispan-embedded.event-types	Specifies the set of event types to register by the consumer. Multiple event can be separated by comma. The possible event types are: CACHE_ENTRY_ACTIVATED, CACHE_ENTRY_PASIVATED, CACHE_ENTRY_VISITED, CACHE_ENTRY_LOADED, CACHE_ENTRY_EVICTED, CACHE_ENTRY_CREATED, CACHE_ENTRY_REMOVED, CACHE_ENTRY_MODIFIED, TRANSACTION_COMPLETED, TRANSACTION_REGISTERED, CACHE_ENTRY_INVALIDATED, CACHE_ENTRY_EXPIRED, DATA_REHASHED, TOPOLOGY_CHANGED, PARTITION_STATUS_CHANGED, PERSISTENCE_AVAILABILITY_CHANGED.		String
camel.component.infinispan-embedded.flags	A comma separated list of org.infinispan.context.Flag to be applied by default on each cache invocation.		String

Name	Description	Default	Type
camel.component.infinispan-embedded.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.infinispan-embedded.operation	The operation to perform.		InfinispanOperation
camel.component.infinispan-embedded.query-builder	Specifies the query builder. The option is a org.apache.camel.component.infinispan.InfinispanQueryBuilder type.		InfinispanQueryBuilder
camel.component.infinispan-embedded.remapping-function	Set a specific remappingFunction to use in a compute operation. The option is a java.util.function.BiFunction type.		BiFunction

Name	Description	Default	Type
camel.component.infinispan-embedded.result-header	Store the operation result in a header instead of the message body. By default, resultHeader == null and the query result is stored in the message body, any existing content in the message body is discarded. If resultHeader is set, the value is used as the name of the header to store the query result and the original message body is preserved. This value can be overridden by an in message header named: CamelInfinispanOperationResultHeader.		String
camel.component.infinispan-embedded.sync	If true, the consumer will receive notifications synchronously.	true	Boolean

CHAPTER 53. JACKSONXML

Jackson XML is a Data Format which uses the [Jackson library](#) with the [XMLMapper extension](#) to unmarshal an XML payload into Java objects or to marshal Java objects into an XML payload. NOTE: If you are familiar with Jackson, this XML data format behaves in the same way as its JSON counterpart, and thus can be used with classes annotated for JSON serialization/deserialization.

This extension also mimics [JAXB's "Code first" approach](#).

This data format relies on [Woodstox](#) (especially for features like pretty printing), a fast and efficient XML processor.

```
from("activemq:My.Queue").
    unmarshal().jacksonxml().
    to("mqseries:Another.Queue");
```

53.1. DEPENDENCIES

When using **jacksonxml** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jacksonxml-starter</artifactId>
</dependency>
```

53.2. JACKSONXML OPTIONS

The JacksonXML dataformat supports 15 options, which are listed below.

Name	Default	Java Type	Description
xmlMapper		String	Lookup and use the existing XmlMapper with the given id.
prettyPrint	false	Boolean	To enable pretty printing output nicely formatted. Is by default false.
unmarshalType		String	Class name of the java type to use when unmarshalling.
jsonView		String	When marshalling a POJO to JSON you might want to exclude certain fields from the JSON output. With Jackson you can use JSON views to accomplish this. This option is to refer to the class which has JsonView annotations.
include		String	If you want to marshal a pojo to JSON, and the pojo has some fields with null values. And you want to skip these null values, you can set this option to NON_NULL.

Name	Default	Java Type	Description
allowJmsType		Boolean	Used for JMS users to allow the JMSType header from the JMS spec to specify a FQN classname to use to unmarshal to.
collectionType		String	Refers to a custom collection type to lookup in the registry to use. This option should rarely be used, but allows to use different collection types than java.util.Collection based as default.
useList		Boolean	To unmarshal to a List of Map or a List of Pojo.
enableJaxbAnnotationModule		Boolean	Whether to enable the JAXB annotations module when using jackson. When enabled then JAXB annotations can be used by Jackson.
moduleClassNames		String	To use custom Jackson modules com.fasterxml.jackson.databind.Module specified as a String with FQN class names. Multiple classes can be separated by comma.
moduleRefs		String	To use custom Jackson modules referred from the Camel registry. Multiple modules can be separated by comma.
enableFeatures		String	Set of features to enable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature Multiple features can be separated by comma.
disableFeatures		String	Set of features to disable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature Multiple features can be separated by comma.
allowUnmarshalType		Boolean	If enabled then Jackson is allowed to attempt to use the CamelJacksonUnmarshalType header during the unmarshalling. This should only be enabled when desired to be used.
contentTypeHeader		Boolean	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.

53.2.1. Using Jackson XML in Spring DSL

When using Data Format in Spring DSL you need to declare the data formats first. This is done in the **DataFormats** XML tag.

```
<dataFormats>
  <!-- here we define a Xml data format with the id jack and that it should use the TestPojo as
the class type when
      doing unmarshal. The unmarshalType is optional, if not provided Camel will use a Map as
the type -->
  <jacksonxml id="jack" unmarshalType="org.apache.camel.component.jacksonxml.TestPojo"/>
</dataFormats>
```

And then you can refer to this id in the route:

```
<route>
  <from uri="direct:back"/>
  <unmarshal><custom ref="jack"/></unmarshal>
  <to uri="mock:reverse"/>
</route>
```

53.2.2. Excluding POJO fields from marshalling

When marshalling a POJO to XML you might want to exclude certain fields from the XML output. With Jackson you can use [JSON views](#) to accomplish this. First create one or more marker classes.

Use the marker classes with the **@JsonView** annotation to include/exclude certain fields. The annotation also works on getters.

Finally use the Camel **JacksonXMLDataFormat** to marshall the above POJO to XML.

Note that the weight field is missing in the resulting XML:

```
<pojo age="30" weight="70"/>
```

53.3. INCLUDE/EXCLUDE FIELDS USING THE JSONVIEW ATTRIBUTE WITH `JACKSONXML` DATAFORMAT

As an example of using this attribute you can instead of:

```
JacksonXMLDataFormat ageViewFormat = new JacksonXMLDataFormat(TestPojoView.class,
Views.Age.class);
from("direct:inPojoAgeView").
  marshal(ageViewFormat);
```

Directly specify your [JSON view](#) inside the Java DSL as:

```
from("direct:inPojoAgeView").
  marshal().jacksonxml(TestPojoView.class, Views.Age.class);
```

And the same in XML DSL:

```
<from uri="direct:inPojoAgeView"/>
<marshal>
  <jacksonxml unmarshalType="org.apache.camel.component.jacksonxml.TestPojoView"
  jsonView="org.apache.camel.component.jacksonxml.Views$Age"/>
</marshal>
```

53.4. SETTING SERIALIZATION INCLUDE OPTION

If you want to marshal a pojo to XML, and the pojo has some fields with null values. And you want to skip these null values, then you need to set either an annotation on the pojo,

```
@JsonInclude(Include.NON_NULL)
public class MyPojo {
    ...
}
```

But this requires you to include that annotation in your pojo source code. You can also configure the Camel JacksonXMLDataFormat to set the include option, as shown below:

```
JacksonXMLDataFormat format = new JacksonXMLDataFormat();
format.setInclude("NON_NULL");
```

Or from XML DSL you configure this as

```
<dataFormats>
  <jacksonxml id="jacksonxml" include="NON_NULL"/>
</dataFormats>
```

53.5. UNMARSHALLING FROM XML TO POJO WITH DYNAMIC CLASS NAME

If you use jackson to unmarshal XML to POJO, then you can now specify a header in the message that indicate which class name to unmarshal to.

The header has key **CamelJacksonUnmarshalType** if that header is present in the message, then Jackson will use that as FQN for the POJO class to unmarshal the XML payload as.

For JMS end users there is the JMSType header from the JMS spec that indicates that also. To enable support for JMSType you would need to turn that on, on the jackson data format as shown:

```
JacksonDataFormat format = new JacksonDataFormat();
format.setAllowJmsType(true);
```

Or from XML DSL you configure this as

```
<dataFormats>
  <jacksonxml id="jacksonxml" allowJmsType="true"/>
</dataFormats>
```

53.6. UNMARSHALLING FROM XML TO LIST<MAP> OR LIST<POJO>

If you are using Jackson to unmarshal XML to a list of map/pojo, you can now specify this by setting **useList="true"** or use the **org.apache.camel.component.jacksonxml.ListJacksonXMLDataFormat**. For example with Java you can do as shown below:

```
JacksonXMLDataFormat format = new ListJacksonXMLDataFormat();
// or
JacksonXMLDataFormat format = new JacksonXMLDataFormat();
format.useList();
// and you can specify the pojo class type also
format.setUnmarshalType(MyPojo.class);
```

And if you use XML DSL then you configure to use list using **useList** attribute as shown below:

```
<dataFormats>
  <jacksonxml id="jack" useList="true"/>
</dataFormats>
```

And you can specify the pojo type also

```
<dataFormats>
  <jacksonxml id="jack" useList="true" unmarshalType="com.foo.MyPojo"/>
</dataFormats>
```

53.7. USING CUSTOM JACKSON MODULES

You can use custom Jackson modules by specifying the class names of those using the `moduleClassNames` option as shown below.

```
<dataFormats>
  <jacksonxml id="jack" useList="true" unmarshalType="com.foo.MyPojo"
  moduleClassNames="com.foo.MyModule,com.foo.MyOtherModule"/>
</dataFormats>
```

When using `moduleClassNames` then the custom jackson modules are not configured, by created using default constructor and used as-is. If a custom module needs any custom configuration, then an instance of the module can be created and configured, and then use `modulesRefs` to refer to the module as shown below:

```
<bean id="myJacksonModule" class="com.foo.MyModule">
  ... // configure the module as you want
</bean>

<dataFormats>
  <jacksonxml id="jacksonxml" useList="true" unmarshalType="com.foo.MyPojo"
  moduleRefs="myJacksonModule"/>
</dataFormats>
```

Multiple modules can be specified separated by comma, such as
`moduleRefs="myJacksonModule,myOtherModule"`

53.8. ENABLING OR DISABLE FEATURES USING JACKSON

Jackson has a number of features you can enable or disable, which its `ObjectMapper` uses. For example to disable failing on unknown properties when marshalling, you can configure this using the `disableFeatures`:

```
<dataFormats>
  <jacksonxml id="jacksonxml" unmarshalType="com.foo.MyPojo"
  disableFeatures="FAIL_ON_UNKNOWN_PROPERTIES"/>
</dataFormats>
```

You can disable multiple features by separating the values using comma. The values for the features must be the name of the enums from Jackson from the following enum classes

- `com.fasterxml.jackson.databind.SerializationFeature`
- `com.fasterxml.jackson.databind.DeserializationFeature`
- `com.fasterxml.jackson.databind.MapperFeature`

To enable a feature use the `enableFeatures` options instead.

From Java code you can use the type safe methods from `camel-jackson` module:

```
JacksonDataFormat df = new JacksonDataFormat(MyPojo.class);
df.disableFeature(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
df.disableFeature(DeserializationFeature.FAIL_ON_NULL_FOR_PRIMITIVES);
```

53.9. CONVERTING MAPS TO POJO USING JACKSON

Jackson **ObjectMapper** can be used to convert maps to POJO objects. Jackson component comes with the data converter that can be used to convert `java.util.Map` instance to non-String, non-primitive and non-Number objects.

```
Map<String, Object> invoiceData = new HashMap<String, Object>();
invoiceData.put("netValue", 500);
producerTemplate.sendBody("direct:mapToInvoice", invoiceData);
...
// Later in the processor
Invoice invoice = exchange.getIn().getBody(Invoice.class);
```

If there is a single **ObjectMapper** instance available in the Camel registry, it will be used by the converter to perform the conversion. Otherwise the default mapper will be used.

53.10. FORMATTED XML MARSHALLING (PRETTY-PRINTING)

Using the **prettyPrint** option one can output a well formatted XML while marshalling:

```
<dataFormats>
  <jacksonxml id="jack" prettyPrint="true"/>
</dataFormats>
```

And in Java DSL:

```
from("direct:inPretty").marshal().jacksonxml(true);
```

Please note that there are 5 different overloaded **jacksonxml()** DSL methods which support the **prettyPrint** option in combination with other settings for **unmarshalType**, **jsonView** etc.

53.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 16 options, which are listed below.

Name	Description	Default	Type
<code>camel.dataformat.jacksonxml.allow-jms-type</code>	Used for JMS users to allow the JMSType header from the JMS spec to specify a FQN classname to use to unmarshal to.	false	Boolean
<code>camel.dataformat.jacksonxml.allow-unmarshall-type</code>	If enabled then Jackson is allowed to attempt to use the CamelJacksonUnmarshalType header during the unmarshalling. This should only be enabled when desired to be used.	false	Boolean
<code>camel.dataformat.jacksonxml.collection-type</code>	Refers to a custom collection type to lookup in the registry to use. This option should rarely be used, but allows to use different collection types than java.util.Collection based as default.		String
<code>camel.dataformat.jacksonxml.content-type-header</code>	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.	true	Boolean
<code>camel.dataformat.jacksonxml.disable-features</code>	Set of features to disable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature. Multiple features can be separated by comma.		String
<code>camel.dataformat.jacksonxml.enable-features</code>	Set of features to enable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature. Multiple features can be separated by comma.		String
<code>camel.dataformat.jacksonxml.enable-jaxb-annotation-module</code>	Whether to enable the JAXB annotations module when using jackson. When enabled then JAXB annotations can be used by Jackson.	false	Boolean

Name	Description	Default	Type
<code>camel.dataformat.jacksonxml.enabled</code>	Whether to enable auto configuration of the jacksonxml data format. This is enabled by default.		Boolean
<code>camel.dataformat.jacksonxml.include</code>	If you want to marshal a pojo to JSON, and the pojo has some fields with null values. And you want to skip these null values, you can set this option to NON_NULL.		String
<code>camel.dataformat.jacksonxml.json-view</code>	When marshalling a POJO to JSON you might want to exclude certain fields from the JSON output. With Jackson you can use JSON views to accomplish this. This option is to refer to the class which has JsonView annotations.		String
<code>camel.dataformat.jacksonxml.module-class-names</code>	To use custom Jackson modules <code>com.fasterxml.jackson.databind.Module</code> specified as a String with FQN class names. Multiple classes can be separated by comma.		String
<code>camel.dataformat.jacksonxml.module-refs</code>	To use custom Jackson modules referred from the Camel registry. Multiple modules can be separated by comma.		String
<code>camel.dataformat.jacksonxml.pretty-print</code>	To enable pretty printing output nicely formatted. Is by default false.	false	Boolean
<code>camel.dataformat.jacksonxml.unmarshal-type</code>	Class name of the java type to use when unmarshalling.		String
<code>camel.dataformat.jacksonxml.use-list</code>	To unmarshal to a List of Map or a List of Pojo.	false	Boolean
<code>camel.dataformat.jacksonxml.xml-mapper</code>	Lookup and use the existing XmlMapper with the given id.		String

CHAPTER 54. JAXB

Since Camel 1.0

JAXB is a Data Format which uses the JAXB XML marshalling standard to unmarshal an XML payload into Java objects or to marshal Java objects into an XML payload.

54.1. DEPENDENCIES

When using **jaxb** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jaxb-starter</artifactId>
</dependency>
```

54.2. OPTIONS

The JAXB dataformat supports 20 options, which are listed below.

Name	Default	Java Type	Description
contextPath		String	Required Package name where your JAXB classes are located.
contextPathsClassName	false	Boolean	This can be set to true to mark that the contextPath is referring to a classname and not a package name.
schema		String	To validate against an existing schema. You can use the prefix classpath:, file: or http: to specify how the resource should be resolved. You can separate multiple schema files by using the ',' character.
schemaSeverityLevel	0	Enum	Sets the schema severity level to use when validating against a schema. This level determines the minimum severity error that triggers JAXB to stop continue parsing. The default value of 0 (warning) means that any error (warning, error or fatal error) will trigger JAXB to stop. There are the following three levels: 0=warning, 1=error, 2=fatal error. Enum values: • 0 • 1 • 2
prettyPrint	false	Boolean	To enable pretty printing output nicely formatted. Is by default false.

Name	Default	Java Type	Description
objectFactory	false	Boolean	Whether to allow using ObjectFactory classes to create the POJO classes during marshalling. This only applies to POJO classes that has not been annotated with JAXB and providing jaxb.index descriptor files.
ignoreJAXBElement	false	Boolean	Whether to ignore JAXBElement elements - only needed to be set to false in very special use-cases.
mustBeJAXBElement	false	Boolean	Whether marshalling must be java objects with JAXB annotations. And if not then it fails. This option can be set to false to relax that, such as when the data is already in XML format.
filterNonXmlChars	false	Boolean	To ignore non xml characters and replace them with an empty space.
encoding		String	To overrule and use a specific encoding.
fragment	false	Boolean	To turn on marshalling XML fragment trees. By default JAXB looks for XmlRootElement annotation on given class to operate on whole XML tree. This is useful but not always - sometimes generated code does not have XmlRootElement annotation, sometimes you need unmarshall only part of tree. In that case you can use partial unmarshalling. To enable this behaviour you need set property partClass. Camel will pass this class to JAXB's unmarshaller.
partClass		String	Name of class used for fragment parsing. See more details at the fragment option.
partNamespace		String	XML namespace to use for fragment parsing. See more details at the fragment option.
namespacePrefixRef		String	When marshalling using JAXB or SOAP then the JAXB implementation will automatic assign namespace prefixes, such as ns2, ns3, ns4 etc. To control this mapping, Camel allows you to refer to a map which contains the desired mapping.
xmlStreamWriterWrapper		String	To use a custom xml stream writer.
schemaLocation		String	To define the location of the schema.
noNamespaceSchemaLocation		String	To define the location of the namespaceless schema.

Name	Default	Java Type	Description
<code>jaxbProviderProperties</code>		String	Refers to a custom <code>java.util.Map</code> to lookup in the registry containing custom JAXB provider properties to be used with the JAXB marshaller.
<code>contentTypeHeader</code>	true	Boolean	Whether the data format should set the Content-Type header with the type from the data format. For example <code>application/xml</code> for data formats marshalling to XML, or <code>application/json</code> for data formats marshalling to JSON.
<code>accessExternalSchemaProtocols</code>	false	String	Only in use if schema validation has been enabled. Restrict access to the protocols specified for external reference set by the <code>schemaLocation</code> attribute, <code>Import</code> and <code>Include</code> element. Examples of protocols are <code>file</code> , <code>http</code> , <code>jar:file</code> . <code>false</code> or <code>none</code> to deny all access to external references; a specific protocol, such as <code>file</code> , to give permission to only the protocol; the keyword <code>all</code> to grant permission to all protocols.

54.3. USING THE JAVA DSL

The following example uses a named `DataFormat` of **jaxb** which is configured with a Java package name to initialize the `JAXBContext`.

```
DataFormat jaxb = new JaxbDataFormat("com.acme.model");

from("activemq:My.Queue").
    unmarshal(jaxb).
    to("mqseries:Another.Queue");
```

You can use a named reference to a data format which can then be defined in your Registry such as via your Spring XML file.

```
from("activemq:My.Queue").
    unmarshal("myJaxbDataType").
    to("mqseries:Another.Queue");
```

54.4. USING SPRING XML

The following example shows how to configure the **JaxbDataFormat** and use it in multiple routes.

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://camel.apache.org/schema/spring http://camel.apache.org/schema/spring/camel-spring.xsd">

    <bean id="myJaxb" class="org.apache.camel.converter.jaxb.JaxbDataFormat">
```

```

    <property name="contextPath" value="org.apache.camel.example"/>
  </bean>

  <camelContext xmlns="http://camel.apache.org/schema/spring">
    <route>
      <from uri="direct:start"/>
      <marshal><custom ref="myJaxb"/></marshal>
      <to uri="direct:marshalled"/>
    </route>
    <route>
      <from uri="direct:marshalled"/>
      <unmarshal><custom ref="myJaxb"/></unmarshal>
      <to uri="mock:result"/>
    </route>
  </camelContext>

</beans>

```

54.5. MULTIPLE CONTEXT PATHS

It is possible to use this data format with more than one context path. You can specify multiple context paths using `:` as a separator, for example **com.mycompany:com.mycompany2**.

54.6. PARTIAL MARSHALLING / UNMARSHALLING

JAXB 2 supports marshalling and unmarshalling XML tree fragments. By default JAXB looks for the **@XmlRootElement** annotation on a given class to operate on whole XML tree. Sometimes the generated code does not have the **@XmlRootElement** annotation and sometimes you need to unmarshal only part of the tree.

In that case you can use partial unmarshalling. To enable this behaviour you need set property **partClass** on the **JaxbDataFormat**. Camel will pass this class to the JAXB unmarshaller. If **JaxbConstants.JAXB_PART_CLASS** is set as one of the exchange headers, its value is used to override the **partClass** property on the **JaxbDataFormat**.

For marshalling you have to add the **partNamespace** attribute with the **QName** of the destination namespace.

If **JaxbConstants.JAXB_PART_NAMESPACE** is set as one of the exchange headers, its value is used to override the **partNamespace** property on the **JaxbDataFormat**.

While setting **partNamespace** through **JaxbConstants.JAXB_PART_NAMESPACE**, please note that you need to specify its value in the format **{namespaceUri}localPart**, as per the example below.

```

.setHeader(JaxbConstants.JAXB_PART_NAMESPACE, constant(
  "{http://www.camel.apache.org/jaxb/example/address/1}address"));

```

54.7. FRAGMENT

JaxbDataFormat has a property named **fragment** which can set the **Marshaller.JAXB_FRAGMENT** property on the JAXB Marshaller. If you don't want the JAXB Marshaller to generate the XML declaration, you can set this option to be **true**. The default value of this property is **false**.

54.8. IGNORING NON-XML CHARACTERS

JaxbDataFormat supports ignoring [Non-XML Characters](#). Set the **filterNonXmlChars** property to **true**. The **JaxbDataFormat** will replace any non-XML character with a space character (" ") during message marshalling or unmarshalling. You can also set the Exchange property **Exchange.FILTER_NON_XML_CHARS**.

	JDK 1.5	JDK 1.6+
Filtering in use	StAX API and implementation	No
Filtering not in use	StAX API only	No

This feature has been tested with Woodstox 3.2.9 and Sun JDK 1.6 StAX implementation.

JaxbDataFormat now allows you to customize the **XMLStreamWriter** used to marshal the stream to XML. Using this configuration, you can add your own stream writer to completely remove, escape, or replace non-XML characters.

```
JaxbDataFormat customWriterFormat = new JaxbDataFormat("org.apache.camel.foo.bar");
customWriterFormat.setXmlStreamWriterWrapper(new TestXmlStreamWriter());
```

The following example shows using the Spring DSL and also enabling Camel's non-XML filtering:

```
<bean id="testXmlStreamWriterWrapper" class="org.apache.camel.jaxb.TestXmlStreamWriter"/>
<jaxb filterNonXmlChars="true" contextPath="org.apache.camel.foo.bar"
xmlStreamWriterWrapper="#testXmlStreamWriterWrapper" />
```

54.9. WORKING WITH THE OBJECTFACTORY

If you use XJC to create the java class from the schema, you will get an **ObjectFactory** for your JAXB context. Since the **ObjectFactory** uses the [JAXBElement](#) to hold the reference of the schema and element instance value, **JaxbDataformat** will ignore the **JAXBElement** by default and you will get the element instance value instead of the **JAXBElement** object from the unmarshaled message body.

If you want to get the **JAXBElement** object from the unmarshaled message body, you need to set the **JaxbDataFormat ignoreJAXBElement** property to be **false**.

54.10. SETTING THE ENCODING

You can set the **encoding** option on the **JaxbDataFormat** to configure the **Marshaller.JAXB_ENCODING** encoding property on the JAXB Marshaller.

You can setup which encoding to use when you declare the **JaxbDataFormat**. You can also provide the encoding in the Exchange property **Exchange.CHARSET_NAME**. This property will override the encoding set on the **JaxbDataFormat**.

54.11. CONTROLLING NAMESPACE PREFIX MAPPING

When marshalling using **JAXB** or **SOAP** then the JAXB implementation will automatic assign namespace prefixes, such as ns2, ns3, ns4 etc. To control this mapping, Camel allows you to refer to a map which contains the desired mapping.

For example, in Spring XML we can define a **Map** with the mapping. In the mapping file below, we map SOAP to use soap as a prefix. While our custom namespace <http://www.mycompany.com/foo/2> is not using any prefix.

```
<util:map id="myMap">
  <entry key="http://www.w3.org/2003/05/soap-envelope" value="soap"/>
  <!-- we don't want any prefix for our namespace -->
  <entry key="http://www.mycompany.com/foo/2" value=""/>
</util:map>
```

To use this in JAXB or SOAP data formats you refer to this map, using the **namespacePrefixRef** attribute as shown below. Then Camel will lookup in the Registry a **java.util.Map** with the id **myMap**, which was what we defined above.

```
<marshal>
  <soap version="1.2" contextPath="com.mycompany.foo" namespacePrefixRef="myMap"/>
</marshal>
```

54.12. SCHEMA VALIDATION

The **JaxbDataFormat** supports validation by marshalling and unmarshalling from / to XML. You can use the prefix **classpath:**, **file:** or **http:** to specify how the resource should be resolved. You can separate multiple schema files by using the , character.



NOTE

If the XSD schema files import/access other files, then you need to enable file protocol (or others to allow access).

Using the Java DSL, you can configure it in the following way:

```
JaxbDataFormat jaxbDataFormat = new JaxbDataFormat();
jaxbDataFormat.setContextPath(Person.class.getPackage().getName());
jaxbDataFormat.setSchema("classpath:person.xsd,classpath:address.xsd");
jaxbDataFormat.setAccessExternalSchemaProtocols("file");
```

You can do the same using the XML DSL:

```
<marshal>
  <jaxb id="jaxb" schema="classpath:person.xsd,classpath:address.xsd"
    accessExternalSchemaProtocols="file"/>
</marshal>
```

54.13. SCHEMA LOCATION

The **JaxbDataFormat** supports to specify the **SchemaLocation** when marshalling the XML.

Using the Java DSL, you can configure it in the following way:

```
JaxbDataFormat jaxbDataFormat = new JaxbDataFormat();
jaxbDataFormat.setContextPath(Person.class.getPackage().getName());
jaxbDataFormat.setSchemaLocation("schema/person.xsd");
```

You can do the same using the XML DSL:

```
<marshal>
  <jaxb id="jaxb" schemaLocation="schema/person.xsd"/>
</marshal>
```

54.14. MARSHAL DATA THAT IS ALREADY XML

The JAXB marshaller requires that the message body is JAXB compatible, e.g it is a **JAXBElement**, a java instance that has JAXB annotations, or extends **JAXBElement**. There can be situations where the message body is already in XML, e.g from a **String** type.

JaxbDataFormat has an option named **mustBeJAXBElement** which you can set to **false** to relax this check and have the JAXB marshaller only attempt marshalling on **JAXBElement** (**javax.xml.bind.JAXBIntrospector#isElement** returns **true**). In those situations the marshaller will fallback to marshal the message body as-is.

54.15. SPRING BOOT AUTO-CONFIGURATION

The component supports 21 options, which are listed below.

Name	Description	Default	Type
camel.dataformat.jaxb.access-external-schema-protocols	Only in use if schema validation has been enabled. Restrict access to the protocols specified for external reference set by the schemaLocation attribute, Import and Include element. Examples of protocols are file, http, jar:file. false or none to deny all access to external references; a specific protocol, such as file, to give permission to only the protocol; the keyword all to grant permission to all protocols.	false	String
camel.dataformat.jaxb.content-type-header	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.	true	Boolean

Name	Description	Default	Type
camel.dataformat.jaxb.context-path	Package name where your JAXB classes are located.		String
camel.dataformat.jaxb.context-path-is-class-name	This can be set to true to mark that the contextPath is referring to a classname and not a package name.	false	Boolean
camel.dataformat.jaxb.enabled	Whether to enable auto configuration of the jaxb data format. This is enabled by default.		Boolean
camel.dataformat.jaxb.encoding	To overrule and use a specific encoding.		String
camel.dataformat.jaxb.filter-non-xml-chars	To ignore non xml characheters and replace them with an empty space.	false	Boolean
camel.dataformat.jaxb.fragment	To turn on marshalling XML fragment trees. By default JAXB looks for XmlRootElement annotation on given class to operate on whole XML tree. This is useful but not always - sometimes generated code does not have XmlRootElement annotation, sometimes you need unmarshall only part of tree. In that case you can use partial unmarshalling. To enable this behaviours you need set property partClass. Camel will pass this class to JAXB's unmarshaller.	false	Boolean
camel.dataformat.jaxb.ignore-j-a-x-b-element	Whether to ignore JAXBELEMENT elements - only needed to be set to false in very special use-cases.	false	Boolean
camel.dataformat.jaxb.jaxb-provider-properties	Refers to a custom java.util.Map to lookup in the registry containing custom JAXB provider properties to be used with the JAXB marshaller.		String
camel.dataformat.jaxb.must-be-j-a-x-b-element	Whether marshalling must be java objects with JAXB annotations. And if not then it fails. This option can be set to false to relax that, such as when the data is already in XML format.	false	Boolean
camel.dataformat.jaxb.namespace-prefix-ref	When marshalling using JAXB or SOAP then the JAXB implementation will automatic assign namespace prefixes, such as ns2, ns3, ns4 etc. To control this mapping, Camel allows you to refer to a map which contains the desired mapping.		String

Name	Description	Default	Type
<code>camel.dataformat.jaxb.no-namespace-schema-location</code>	To define the location of the namespaceless schema.		String
<code>camel.dataformat.jaxb.object-factory</code>	Whether to allow using ObjectFactory classes to create the POJO classes during marshalling. This only applies to POJO classes that has not been annotated with JAXB and providing jaxb.index descriptor files.	false	Boolean
<code>camel.dataformat.jaxb.part-class</code>	Name of class used for fragment parsing. See more details at the fragment option.		String
<code>camel.dataformat.jaxb.part-namespace</code>	XML namespace to use for fragment parsing. See more details at the fragment option.		String
<code>camel.dataformat.jaxb.pretty-print</code>	To enable pretty printing output nicely formatted. Is by default false.	false	Boolean
<code>camel.dataformat.jaxb.schema</code>	To validate against an existing schema. You can use the prefix classpath:, file: or http: to specify how the resource should be resolved. You can separate multiple schema files by using the ',' character.		String
<code>camel.dataformat.jaxb.schema-location</code>	To define the location of the schema.		String
<code>camel.dataformat.jaxb.schema-severity-level</code>	Sets the schema severity level to use when validating against a schema. This level determines the minimum severity error that triggers JAXB to stop continue parsing. The default value of 0 (warning) means that any error (warning, error or fatal error) will trigger JAXB to stop. There are the following three levels: 0=warning, 1=error, 2=fatal error.	0	Integer
<code>camel.dataformat.jaxb.xml-stream-writer-wrapper</code>	To use a custom xml stream writer.		String

CHAPTER 55. JASYPT

Since Camel 2.5

Jasypt is a simplified encryption library which makes encryption and decryption easy. Camel integrates with Jasypt to allow sensitive information in [Properties](#) files to be encrypted. By dropping **camel-jasypt** on the classpath those encrypted values will automatically be decrypted on-the-fly by Camel. This ensures that human eyes can't easily spot sensitive information such as usernames and passwords.

55.1. DEPENDENCIES

When using **camel-jasypt** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jasypt-starter</artifactId>
</dependency>
```

55.2. TOOLING

The Jasypt component is a runnable JAR that provides a command line utility to encrypt or decrypt values. The usage documentation can be output to the console to describe the syntax and options it provides:

Apache Camel Jasypt takes the following options

```
-h or -help = Displays the help screen
-c or -command <command> = Command either encrypt or decrypt
-p or -password <password> = Password to use
-i or -input <input> = Text to encrypt or decrypt
-a or -algorithm <algorithm> = Optional algorithm to use
-rsga or -algorithm <algorithm> = Optional random salt generator algorithm to use
-riga or -algorithm <algorithm> = Optional random iv generator algorithm to use
```

A simple way of running the tool is with JBang. For example, to encrypt the value **tiger**, you can use the following parameters. Make sure to specify the version of **camel-jasypt** that you want to use.

```
$ jbang org.apache.camel:camel-jasypt:<camel version here> -c encrypt -p secret -i tiger
```

Which outputs the following result

```
Encrypted text: qaEEacuW7BUti8LcMgyjKw==
```

This means the encrypted representation **qaEEacuW7BUti8LcMgyjKw==** can be decrypted back to **tiger** if you know the master password which was **secret**.

If you run the tool again then the encrypted value will return a different result. But decrypting the value will always return the correct original value.

You can test decrypting the value by running the tooling using the following parameters:

```
$ jbang org.apache.camel:camel-jasypt:<camel version here> -c decrypt -p secret -i
qaEEacuW7BUti8LcMgyjKw==
```

Which outputs the following result:

```
Decrypted text: tiger
```

The idea is then to use those encrypted values in your [Properties](#) files. For example,

```
# Encrypted value for 'tiger'
my.secret = ENC(qaEEacuW7BUti8LcMgyjKw==)
```

55.3. PROTECTING THE MASTER PASSWORD

The master password used by Jasypt must be provided, so that it's capable of decrypting the values. However, having this master password out in the open may not be an ideal solution. Therefore, you could for example provide it as a JVM system property or as an OS environment setting. If you decide to do so then the **password** option supports prefixes that dictates this.

- **sysenv:** means to lookup the OS system environment with the given key.
- **sys:** means to lookup a JVM system property.

For example, you could provide the password before you start the application

```
$ export CAMEL_ENCRYPTION_PASSWORD=secret
```

Then start the application, such as running the start script.

When the application is up and running you can unset the environment

```
$ unset CAMEL_ENCRYPTION_PASSWORD
```

On runtimes like Spring Boot and Quarkus, you can configure a password property in the **application.properties** file as follows.

```
password=sysenv:CAMEL_ENCRYPTION_PASSWORD
```

Or if configuring **JasyptPropertiesParser** manually, you can set the password like this.

```
jasyptPropertiesParser.setPassword("sysenv:CAMEL_ENCRYPTION_PASSWORD");
```

55.4. EXAMPLE WITH JAVA DSL

On the Spring Boot and Quarkus runtimes, Camel Jasypt can be configured via configuration properties. Refer to their respective documentation pages for more information.

In Java DSL you need to configure Jasypt as a **JasyptPropertiesParser** instance and set the properties in the [Properties](#) component as shown below:

```
// create the jasypt properties parser
```

```

JasyptPropertiesParser jasypt = new JasyptPropertiesParser();
// set the master password (see above for how to do this in a secure way)
jasypt.setPassword("secret");

// create the properties' component
PropertiesComponent pc = new PropertiesComponent();
pc.setLocation("classpath:org/apache/camel/component/jasypt/secret.properties");
// and use the jasypt properties parser, so we can decrypt values
pc.setPropertiesParser(jasypt);
// end enable nested placeholder support
pc.setNestedPlaceholder(true);

// add properties component to camel context
context.setPropertiesComponent(pc);

```

It is possible to configure custom algorithms on the `JasyptPropertiesParser` like this.

```

JasyptPropertiesParser jasyptPropertiesParser = new JasyptPropertiesParser();

jasyptPropertiesParser.setAlgorithm("PBEWithHmacSHA256AndAES_256");
jasyptPropertiesParser.setRandomSaltGeneratorAlgorithm("PKCS11");
jasyptPropertiesParser.setRandomIvGeneratorAlgorithm("PKCS11");

```

The properties file **secret.properties** will contain your encrypted configuration values, such as shown below. Notice how the password value is encrypted and is surrounded like `ENC(value here)`.

```

my.secret.password=ENC(bsW9uV37gQ0QHfu7KO03Ww==)

```

55.5. EXAMPLE WITH SPRING XML

In Spring XML you need to configure the **JasyptPropertiesParser** which is shown below. Then the Camel `Properties` component is told to use **jasypt** as the properties parser, which means Jasypt has its chance to decrypt values looked up in the properties.

```

<!-- define the jasypt properties parser with the given password to be used -->
<bean id="jasypt" class="org.apache.camel.component.jasypt.JasyptPropertiesParser">
  <property name="password" value="secret"/>
</bean>

<!-- define the camel properties component -->
<bean id="properties" class="org.apache.camel.component.properties.PropertiesComponent">
  <!-- the properties file is in the classpath -->
  <property name="location"
value="classpath:org/apache/camel/component/jasypt/secret.properties"/>
  <!-- and let it leverage the jasypt parser -->
  <property name="propertiesParser" ref="jasypt"/>
  <!-- end enable nested placeholder -->
  <property name="nestedPlaceholder" value="true"/>
</bean>

```

The `Properties` component can also be inlined inside the **<camelContext>** tag which is shown below. Notice how we use the **propertiesParserRef** attribute to refer to Jasypt.

```

<!-- define the jasypt properties parser with the given password to be used -->

```

```

<bean id="jasypt" class="org.apache.camel.component.jasypt.JasyptPropertiesParser">
  <!-- password is mandatory, you can prefix it with sysenv: or sys: to indicate it should use
       an OS environment or JVM system property value, so you dont have the master password
       defined here -->
  <property name="password" value="secret"/>
</bean>

<camelContext xmlns="http://camel.apache.org/schema/spring">
  <!-- define the camel properties placeholder, and let it leverage jasypt -->
  <propertyPlaceholder id="properties"
    location="classpath:org/apache/camel/component/jasypt/myproperties.properties"
    nestedPlaceholder="true"
    propertiesParserRef="jasypt"/>

  <route>
    <from uri="direct:start"/>
    <to uri="{{cool.result}}"/>
  </route>
</camelContext>

```

55.6. SPRING BOOT AUTO-CONFIGURATION

The component supports 8 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.jasypt.algorithm</code>	The algorithm to be used for decryption.	PBEWithMD5AndDES	String
<code>camel.component.jasypt.enabled</code>	Enable the component.	false	Boolean
<code>camel.component.jasypt.iv-generator-class-name</code>	The initialization vector (IV) generator applied in decryption operations. Default: <code>org.jasypt.iv</code> .		String
<code>camel.component.jasypt.password</code>	The master password used by Jasypt for decrypting the values. This option supports prefixes which influence the master password lookup behaviour: <code>sysenv:</code> means to lookup the OS system environment with the given key. <code>sys:</code> means to lookup a JVM system property.		String
<code>camel.component.jasypt.provider-name</code>	The class name of the security provider to be used for obtaining the encryption algorithm.		String

Name	Description	Default	Type
<code>camel.component.jasypt.random-iv-generator-algorithm</code>	The algorithm for the random iv generator.	SHA1P RNG	String
<code>camel.component.jasypt.random-salt-generator-algorithm</code>	The algorithm for the salt generator.	SHA1P RNG	String
<code>camel.component.jasypt.salt-generator-class-name</code>	The salt generator applied in decryption operations. Default: <code>org.jasypt.salt.RandomSaltGenerator</code> .	<code>org.jasypt.salt.RandomSaltGenerator</code>	String

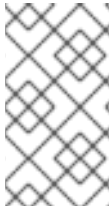
4.0// ParentAssemblies: assemblies/

CHAPTER 56. JDBC

Since Camel 1.2

Only producer is supported

The JDBC component enables you to access databases through JDBC, where SQL queries (SELECT) and operations (INSERT, UPDATE, etc) are sent in the message body. This component uses the standard JDBC API.



NOTE

Prerequisites

This component does not support transactions out of the box. For transactions, we recommend using the [Spring JDBC Component](#) instead.

56.1. DEPENDENCIES

When using **jdbc** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jdbc-starter</artifactId>
</dependency>
```

56.2. URI FORMAT

```
jdbc:dataSourceName[?options]
```

56.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

56.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example, a component may have security settings, credentials for authentication, URLs for network connection.

Components have pre-configured defaults that are commonly used. hence, you need to configure only a few options on a component or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

56.3.2. Configuring Endpoint Options

Endpoints have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from) or as a producer (to) or used for both.

Configuring endpoints is done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a *type safe* way of configuring endpoints and data formats in Java.

When configuring options is to use [Property Placeholders](#), which allows to not hard-code urls, port numbers, sensitive information, and other settings. Placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

56.4. COMPONENT OPTIONS

The JDBC component supports 4 options, which are listed below.

Name	Description	Default	Type
dataSource (producer)	To use the DataSource instance instead of looking up the data source by name from the registry.		DataSource
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
connectionStrategy (advanced)	To use a custom strategy for working with connections. Do not use a custom strategy when using the spring-jdbc component because a special Spring ConnectionStrategy is used by default to support Spring Transactions.		ConnectionStrategy

56.5. ENDPOINT OPTIONS

The JDBC endpoint is configured using URI syntax:

```
jdbc:dataSourceName
```

with the following path and query parameter:

56.5.1. Path Parameter (1 parameter)

Name	Description	Default	Type
dataSourceName (producer)	Required Name of DataSource to lookup in the Registry. If the name is dataSource or default, then Camel will attempt to lookup a default DataSource from the registry, meaning if there is a only one instance of DataSource found, then this DataSource will be used.		String

56.5.2. Query Parameters (14 parameters)

Name	Description	Default	Type
allowNamedParameters (producer)	Whether to allow using named parameters in the queries.	True	Boolean
outputClass (producer)	Specify the full package and class name to use as conversion when outputType=SelectOne or SelectList.		String
outputType (producer)	Determines the output the producer should use. Enum values: • SelectOne • SelectList • StreamList	Select List	JdbcOutputType
parameters (producer)	Optional parameters to the java.sql.Statement. For example to set maxRows, fetchSize etc.		Map
readSize (producer)	The default maximum number of rows that can be read by a polling query. The default value is 0.		int

Name	Description	Default	Type
resetAutoCommit (producer)	Camel will set the autoCommit on the JDBC connection to be false, commit the change after executed the statement and reset the autoCommit flag of the connection at the end, if the resetAutoCommit is true. If the JDBC connection doesn't support to reset the autoCommit flag, you can set the resetAutoCommit flag to be false, and Camel will not try to reset the autoCommit flag. When used with XA transactions you most likely need to set it to false so that the transaction manager is in charge of committing this tx.	True	Boolean
transacted (producer)	Whether transactions are in use.	False	Boolean
useGetBytesForBlob (producer)	To read BLOB columns as bytes instead of string data. This may be needed for certain databases such as Oracle where you must read BLOB columns as bytes.	False	Boolean
useHeadersAsParameters (producer)	Set this option to true to use the prepareStatementStrategy with named parameters. This allows to define queries with named placeholders, and use headers with the dynamic values for the query placeholders.	False	Boolean
useJDBC4ColumnNameAndLabelSemantics (producer)	Sets whether to use JDBC 4 or JDBC 3.0 or older semantic when retrieving column name. JDBC 4.0 uses columnName to get the column name where as JDBC 3.0 uses both columnName or columnLabel. Unfortunately JDBC drivers behave differently so you can use this option to work out issues around your JDBC driver if you get problem using this component. This option is default true.	True	Boolean
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	False	Boolean

Name	Description	Default	Type
beanRowMapper (advanced)	To use a custom <code>org.apache.camel.component.jdbc.BeanRowMapper</code> when using <code>outputClass</code> . The default implementation will lower case the row names and skip underscores, and dashes. For example <code>CUST_ID</code> is mapped as <code>custId</code> .		<code>BeanRowMapper</code>
connectionStrategy (advanced)	To use a custom strategy for working with connections. Do not use a custom strategy when using the <code>spring-jdbc</code> component because a special <code>Spring ConnectionStrategy</code> is used by default to support <code>Spring Transactions</code> .		<code>ConnectionStrategy</code>
prepareStatementStrategy (advanced)	Allows the plugin to use a custom <code>org.apache.camel.component.jdbc.JdbcPrepareStatementStrategy</code> to control preparation of the query and prepared statement.		<code>JdbcPrepareStatementStrategy</code>

56.6. MESSAGE HEADERS

The JBBC component supports 8 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelJdbcUpdateCount (producer) Constant: <code>JDBC_UPDATE_COUNT</code>	If the query is an <code>UPDATE</code> , query the update count is returned in this <code>OUT</code> header.		<code>int</code>
CamelJdbcRowCount (producer) Constant: <code>JDBC_ROW_COUNT</code>	If the query is a <code>SELECT</code> , query the row count is returned in this <code>OUT</code> header.		<code>int</code>
CamelJdbcColumnNames (producer) Constant: <code>JDBC_COLUMN_NAMES</code>	The column names from the <code>ResultSet</code> as a <code>java.util.Set</code> type.		<code>Set</code>

Name	Description	Default	Type
CamelJdbcParameters (producer) Constant: JDBC_PARAMETERS	A java.util.Map which has the headers to be used if <code>useHeadersAsParameters</code> has been enabled.		Map
CamelRetrieveGeneratedKeys (producer) Constant: JDBC_RETRIEVE_GENERATED_KEYS	Set its value to true to retrieve generated keys.	False	Boolean
CamelGeneratedColumns (producer) Constant: JDBC_GENERATED_COLUMNS	Set it to specify the expected generated columns.		String[] or int[]
CamelGeneratedKeysRowCount (producer) Constant: JDBC_GENERATED_KEYS_ROW_COUNT	The number of rows in the header that contains generated keys.		int
CamelGeneratedKeysRows (producer) Constant: JDBC_GENERATED_KEYS_DATA	Rows that contains the generated keys.		List

56.7. RESULT

By default the result is returned in the OUT body as an **ArrayList<HashMap<String, Object>>**. The List object contains the list of rows and the **Map** objects contain each row with the **String** key as the column name. You can use the option **outputType** to control the result.



NOTE

This component fetches **ResultSetMetaData** to be able to return the column name as the key in the **Map**.

56.8. GENERATED KEYS

If you insert data using *SQL INSERT*, then the *RDBMS* may support auto generated keys. You can instruct the JDBC producer to return the generated keys in headers. To do that set the header **CamelRetrieveGeneratedKeys=true**. Then the generated keys will be provided as headers with the keys listed in the table above.



NOTE

Using generated keys does not work together with named parameters.

56.9. USING NAMED PARAMETERS

In the given route below, we want to get all the projects from the projects table. Notice the SQL query has 2 named parameters, `:?lic` and `:?min`. Camel will then lookup these parameters from the message headers. Notice in the example above we set two headers with constant value for the named parameters:

```
from("direct:projects")
  .setHeader(":?lic", constant("ASF"))
  .setHeader(":?min", constant(123))
  .setBody(simple("select * from projects where license = :?lic and id > :?min order by id"))
  .to("jdbc:myDataSource?useHeadersAsParameters=true")
```

You can also store the header values in a **java.util.Map** and store the map on the headers with the key **CamelJdbcParameters**.

56.10. SAMPLES

In the following example, we setup the DataSource that **camel-jdbc** requires. First we register our datasource in the Camel registry as **testdb**:

```
EmbeddedDatabase db = new EmbeddedDatabaseBuilder()
  .setType(EmbeddedDatabaseType.DERBY).addScript("sql/init.sql").build();

CamelContext context = ...
context.getRegistry().bind("testdb", db);
```

Then we configure a route that routes to the JDBC component, so the SQL will be executed. Note how we refer to the **testdb** datasource that was bound in the previous step:

```
from("direct:hello")
  .to("jdbc:testdb");
```

We create an endpoint, add the SQL query to the body of the IN message, and then send the exchange. The result of the query is returned in the OUT body:

```
Endpoint endpoint = context.getEndpoint("direct:hello");
Exchange exchange = endpoint.createExchange();
// then we set the SQL on the in body
exchange.getMessage().setBody("select * from customer order by ID");
// now we send the exchange to the endpoint, and receives the response from Camel
Exchange out = template.send(endpoint, exchange);
```

If you want to work on the rows one by one instead of the entire ResultSet at once you need to use the Splitter EIP such as:

```
from("direct:hello")
// here we split the data from the testdb into new messages one by one
// so the mock endpoint will receive a message per row in the table
// the StreamList option allows to stream the result of the query without creating a List of rows
// and notice we also enable streaming mode on the splitter
```

```
.to("jdbc:testdb?outputType=StreamList")
  .split(body()).streaming()
  .to("mock:result");
```

56.11. SAMPLE - POLLING THE DATABASE EVERY MINUTE

If we want to poll a database using the JDBC component, we need to combine it with a polling scheduler such as the [Timer](#) or [Quartz](#) etc. In the following example, we retrieve data from the database every 60 seconds:

```
from("timer://foo?period=60000")
  .setBody(constant("select * from customer"))
  .to("jdbc:testdb")
  .to("activemq:queue:customers");
```

56.12. SAMPLE - MOVE DATA BETWEEN DATA SOURCES

A common use case is to query for data, process it and move it to another data source (ETL operations). In the following example, we retrieve new customer records from the source table every hour, filter/transform them and move them to a destination table:

```
from("timer://MoveNewCustomersEveryHour?period=3600000")
  .setBody(constant("select * from customer where create_time > (sysdate-1/24)"))
  .to("jdbc:testdb")
  .split(body())
    .process(new MyCustomerProcessor()) //filter/transform results as needed
  .setBody(simple("insert into processed_customer values('${body[ID]}','${body[NAME]}')"))
  .to("jdbc:testdb");
```

56.13. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.component.jdbc.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatically configure JDBC data sources, JMS connection factories, AWS Clients, etc.	True	Boolean
camel.component.jdbc.connection-strategy	To use a custom strategy for working with connections. Do not use a custom strategy when using the spring-jdbc component because a special Spring ConnectionStrategy is used by default to support Spring Transactions. The option is a <code>org.apache.camel.component.jdbc.ConnectionStrategy</code> type.		ConnectionStrategy

Name	Description	Default	Type
camel.component.jdbc.enabled	Whether to enable auto configuration of the jdbc component. This is enabled by default.		Boolean
camel.component.jdbc.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	False	Boolean

CHAPTER 57. JIRA

Both producer and consumer are supported

The JIRA component interacts with the JIRA API by encapsulating Atlassian's [REST Java Client for JIRA](#). It currently provides polling for new issues and new comments. It is also able to create new issues, add comments, change issues, add/remove watchers, add attachment and transition the state of an issue.

Rather than webhooks, this endpoint relies on simple polling. Reasons include:

- Concern for reliability/stability
- The types of payloads we're polling aren't typically large (plus, paging is available in the API)
- The need to support apps running somewhere not publicly accessible where a webhook would fail

Note that the JIRA API is fairly expansive. Therefore, this component could be easily expanded to provide additional interactions.

57.1. DEPENDENCIES

When using **jira** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>  
  <groupId>org.apache.camel.springboot</groupId>  
  <artifactId>camel-jira-starter</artifactId>  
</dependency>
```

57.2. URI FORMAT

```
jira://type[?options]
```

The Jira type accepts the following operations:

For consumers:

- newIssues: retrieve only new issues after the route is started
- newComments: retrieve only new comments after the route is started
- watchUpdates: retrieve only updated fields/issues based on provided jql

For producers:

- addIssue: add an issue
- addComment: add a comment on a given issue
- attach: add an attachment on a given issue
- deleteIssue: delete a given issue

- `updateIssue`: update fields of a given issue
- `transitionIssue`: transition a status of a given issue
- `watchers`: add/remove watchers of a given issue

As Jira is fully customizable, you must assure the fields IDs exists for the project and workflow, as they can change between different Jira servers.

57.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

57.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

57.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

57.4. COMPONENT OPTIONS

The Jira component supports 12 options, which are listed below.

Name	Description	Default	Type
delay (common)	Time in milliseconds to elapse for the next poll.	6000	Integer
jiraUrl (common)	Required The Jira server url, example: .		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
configuration (advanced)	To use a shared base jira configuration.		JiraConfiguration
accessToken (security)	(OAuth only) The access token generated by the Jira server.		String
consumerKey (security)	(OAuth only) The consumer key from Jira settings.		String

Name	Description	Default	Type
password (security)	(Basic authentication only) The password to authenticate to the Jira server. Use only if username basic authentication is used.		String
privateKey (security)	(OAuth only) The private key generated by the client to encrypt the conversation to the server.		String
username (security)	(Basic authentication only) The username to authenticate to the Jira server. Use only if OAuth is not enabled on the Jira server. Do not set the username and OAuth token parameter, if they are both set, the username basic authentication takes precedence.		String
verificationCode (security)	(OAuth only) The verification code from Jira generated in the first step of the authorization process.		String

57.5. ENDPOINT OPTIONS

The Jira endpoint is configured using URI syntax:

```
jira:type
```

with the following path and query parameters:

57.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
type (common)	Required Operation to perform. Consumers: NewIssues, NewComments. Producers: AddIssue, AttachFile, DeleteIssue, TransitionIssue, UpdateIssue, Watchers. See this class javadoc description for more information. Enum values: • ADDCOMMENT • ADDISSUE • ATTACH • DELETEISSUE • NEWISSUES • NEWCOMMENTS • WATCHUPDATES • UPDATEISSUE • TRANSITIONISSUE • WATCHERS • ADDISSUELINK • ADDWORKLOG • FETCHISSUE • FETCHCOMMENTS		JiraType

57.5.2. Query Parameters (16 parameters)

Name	Description	Default	Type
delay (common)	Time in milliseconds to elapse for the next poll.	6000	Integer
jiraUrl (common)	Required The Jira server url, example: .		String

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
jql (consumer)	JQL is the query language from JIRA which allows you to retrieve the data you want. For example <code>jql=project=MyProject</code> Where <code>MyProject</code> is the product key in Jira. It is important to use the <code>RAW()</code> and set the JQL inside it to prevent camel parsing it, example: <code>RAW(project in (MYP, COM) AND resolution = Unresolved)</code> .		String
maxResults (consumer)	Max number of issues to search for.	50	Integer
sendOnlyUpdatedField (consumer)	Indicator for sending only changed fields in exchange body or issue object. By default consumer sends only changed fields.	true	boolean
watchedFields (consumer)	Comma separated list of fields to watch for changes. Status,Priority are the defaults.	Status, Priority	String
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
accessToken (security)	(OAuth only) The access token generated by the Jira server.		String
consumerKey (security)	(OAuth only) The consumer key from Jira settings.		String
password (security)	(Basic authentication only) The password to authenticate to the Jira server. Use only if username basic authentication is used.		String
privateKey (security)	(OAuth only) The private key generated by the client to encrypt the conversation to the server.		String
username (security)	(Basic authentication only) The username to authenticate to the Jira server. Use only if OAuth is not enabled on the Jira server. Do not set the username and OAuth token parameter, if they are both set, the username basic authentication takes precedence.		String
verificationCode (security)	(OAuth only) The verification code from Jira generated in the first step of the authorization process.		String

57.6. CLIENT FACTORY

You can bind the **JiraRestClientFactory** with name **JiraRestClientFactory** in the registry to have it automatically set in the Jira endpoint.

57.7. AUTHENTICATION

Camel-jira supports [Basic Authentication](#) and [OAuth 3 legged authentication](#).

We recommend to use OAuth whenever possible, as it provides the best security for your users and system.

57.7.1. Basic authentication requirements:

- An username and password

57.7.2. OAuth authentication requirements:

Follow the tutorial in [Jira OAuth documentation](#) to generate the client private key, consumer key, verification code and access token.

- a private key, generated locally on your system.
- A verification code, generated by Jira server.
- The consumer key, set in the Jira server settings.
- An access token, generated by Jira server.

57.8. JQL

The JQL URI option is used by both consumer endpoints. Theoretically, items like "project key", etc. could be URI options themselves. However, by requiring the use of JQL, the consumers become much more flexible and powerful.

At the bare minimum, the consumers will require the following:

```
jira://[type]?[required options]&jql=project=[project key]
```

One important thing to note is that the newIssues consumer will automatically set the JQL as:

- append **ORDER BY key desc** to your JQL
- prepend **id > latestIssueId** to retrieve issues added after the camel route was started.

This is in order to optimize startup processing, rather than having to index every single issue in the project.

Another note is that, similarly, the newComments consumer will have to index every single issue **and** comment in the project. Therefore, for large projects, it's **vital** to optimize the JQL expression as much as possible. For example, the JIRA Toolkit Plugin includes a "Number of comments" custom field – use "Number of comments" > 0' in your query. Also try to minimize based on state (status=Open), increase the polling delay, etc. Example:

```
jira://[type]?[required options]&jql=RAW(project=[project key] AND status in (Open, \"Coding In Progress\") AND \"Number of comments\">0)
```

57.9. OPERATIONS

See a list of required headers to set when using the Jira operations. The author field for the producers is automatically set to the authenticated user in the Jira side.

If any required field is not set, then an `IllegalArgumentException` is throw.

There are operations that requires **id** for fields suchs as: issue type, priority, transition. Check the valid **id** on your jira project as they may differ on a jira installation and project workflow.

57.10. ADDISSUE

Required:

- **ProjectKey**: The project key, example: CAMEL, HHH, MYP.
- **IssueTypeId** or **IssueTypeName**: The **id** of the issue type or the name of the issue type, you can see the valid list in http://jira_server/rest/api/2/issue/createmeta?projectKeys=SAMPLE_KEY.
- **IssueSummary**: The summary of the issue.

Optional:

- **IssueAssignee**: the assignee user
- **IssuePriorityId** or **IssuePriorityName**: The priority of the issue, you can see the valid list in http://jira_server/rest/api/2/priority.
- **IssueComponents**: A list of string with the valid component names.
- **IssueWatchersAdd**: A list of strings with the usernames to add to the watcher list.
- **IssueDescription**: The description of the issue.

57.11. ADDCOMMENT

Required:

- **IssueKey**: The issue key identifier.
- body of the exchange is the description.

57.12. ATTACH

Only one file should attach per invocation.

Required:

- **IssueKey**: The issue key identifier.
- body of the exchange should be of type **File**

57.13. DELETEISSUE

Required:

- **IssueKey**: The issue key identifier.

57.14. TRANSITIONISSUE

Required:

Required:

- **IssueKey**: The issue key identifier.
- **IssueTransitionId**: The issue transition **id**.
- body of the exchange is the description.

57.15. UPDATEISSUE

- **IssueKey**: The issue key identifier.
- **IssueTypeId** or **IssueTypeName**: The **id** of the issue type or the name of the issue type, you can see the valid list in http://jira_server/rest/api/2/issue/createmeta?projectKeys=SAMPLE_KEY.
- **IssueSummary**: The summary of the issue.
- **IssueAssignee**: the assignee user
- **IssuePriorityId** or **IssuePriorityName**: The priority of the issue, you can see the valid list in http://jira_server/rest/api/2/priority.
- **IssueComponents**: A list of string with the valid component names.
- **IssueDescription**: The description of the issue.

57.16. WATCHER

- **IssueKey**: The issue key identifier.
- **IssueWatchersAdd**: A list of strings with the usernames to add to the watcher list.
- **IssueWatchersRemove**: A list of strings with the usernames to remove from the watcher list.

57.17. WATCHUPDATES (CONSUMER)

- **watchedFields** Comma separated list of fields to watch for changes i.e **Status,Priority,Assignee,Components** etc.
- **sendOnlyUpdatedField** By default only changed field is send as the body.

All messages also contain following headers that add additional info about the change:

- **issueKey**: Key of the updated issue
- **changed**: name of the updated field (i.e Status)
- **watchedIssues**: list of all issue keys that are watched in the time of update

57.18. SPRING BOOT AUTO-CONFIGURATION

The component supports 13 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.jira.access-token</code>	(OAuth only) The access token generated by the Jira server.		String
<code>camel.component.jira.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.jira.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.jira.configuration</code>	To use a shared base jira configuration. The option is a <code>org.apache.camel.component.jira.JiraConfiguration</code> type.		JiraConfiguration
<code>camel.component.jira.consumer-key</code>	(OAuth only) The consumer key from Jira settings.		String
<code>camel.component.jira.delay</code>	Time in milliseconds to elapse for the next poll.	6000	Integer
<code>camel.component.jira.enabled</code>	Whether to enable auto configuration of the jira component. This is enabled by default.		Boolean
<code>camel.component.jira.jira-url</code>	The Jira server url, example: http://my_jira.com:8081/ .		String
<code>camel.component.jira.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.jira.password	(Basic authentication only) The password to authenticate to the Jira server. Use only if username basic authentication is used.		String
camel.component.jira.private-key	(OAuth only) The private key generated by the client to encrypt the conversation to the server.		String
camel.component.jira.username	(Basic authentication only) The username to authenticate to the Jira server. Use only if OAuth is not enabled on the Jira server. Do not set the username and OAuth token parameter, if they are both set, the username basic authentication takes precedence.		String
camel.component.jira.verification-code	(OAuth only) The verification code from Jira generated in the first step of the authorization process.		String

CHAPTER 58. JMS

Both producer and consumer are supported

This component allows messages to be sent to (or consumed from) a [JMS](#) Queue or Topic. It uses Spring's JMS support for declarative transactions, including Spring's **JmsTemplate** for sending and a **MessageListenerContainer** for consuming.

58.1. DEPENDENCIES

When using **jms** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jms-starter</artifactId>
</dependency>
```



NOTE

Using ActiveMQ

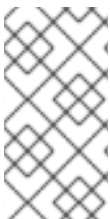
If you are using [Apache ActiveMQ](#), you should prefer the ActiveMQ component as it has been optimized for ActiveMQ. All of the options and samples on this page are also valid for the ActiveMQ component.



NOTE

Transacted and caching

See section **Transactions and Cache Levels** below if you are using transactions with [JMS](#) as it can impact performance.



NOTE

Request/Reply over JMS

Make sure to read the section *Request/reply over JMS* further below on this page for important notes about request/reply, as Camel offers a number of options to configure for performance, and clustered environments.

58.2. URI FORMAT

```
jms:[queue:]topic:destinationName[?options]
```

Where **destinationName** is a JMS queue or topic name. By default, the **destinationName** is interpreted as a queue name. For example, to connect to the queue, **FOO.BAR** use:

```
jms:FOO.BAR
```

You can include the optional **queue:** prefix, if you prefer:

```
jms:queue:FOO.BAR
```

To connect to a topic, you *must* include the **topic:** prefix. For example, to connect to the topic, **Stocks.Prices**, use:

```
jms:topic:Stocks.Prices
```

You append query options to the URI by using the following format,

?option=value&option=value&...

58.2.1. Using ActiveMQ

The JMS component reuses Spring 2's **JmsTemplate** for sending messages. This is not ideal for use in a non-J2EE container and typically requires some caching in the JMS provider to avoid [poor performance](#).

If you intend to use [Apache ActiveMQ](#) as your message broker, the recommendation is that you do one of the following:

- Use the ActiveMQ component, which is already optimized to use ActiveMQ efficiently
- Use the **PoolingConnectionFactory** in ActiveMQ.

58.2.2. Transactions and Cache Levels

If you are consuming messages and using transactions (**transacted=true**) then the default settings for cache level can impact performance.

If you are using XA transactions then you cannot cache as it can cause the XA transaction to not work properly.

If you are **not** using XA, then you should consider caching as it speeds up performance, such as setting **cacheLevelName=CACHE_CONSUMER**.

The default setting for **cacheLevelName** is **CACHE_AUTO**. This default auto detects the mode and sets the cache level accordingly to:

- **CACHE_CONSUMER** if **transacted=false**
- **CACHE_NONE** if **transacted=true**

So you can say the default setting is conservative. Consider using **cacheLevelName=CACHE_CONSUMER** if you are using non-XA transactions.

58.2.3. Durable Subscriptions with JMS 1.1

If you wish to use durable topic subscriptions, you need to specify both **clientId** and **durableSubscriptionName**. The value of the **clientId** must be unique and can only be used by a single JMS connection instance in your entire network.



NOTE

If you are using the [Apache ActiveMQ Classic](#), you may prefer to use a feature called Virtual Topic. This should remove the necessity of having a unique **clientId**. You can consult the specific documentation for [Artemis](#) or for [ActiveMQ Classic](#) for details about how to leverage this feature. You can find more details about durable messaging for ActiveMQ Classic [here](#).

58.2.3.1. Durable Subscriptions with JMS 2.0

If you wish to use durable topic subscriptions, you need to specify the **durableSubscriptionName**.

58.2.4. Message Header Mapping

When using message headers, the JMS specification states that header names must be valid Java identifiers. So try to name your headers to be valid Java identifiers. One benefit of doing this is that you can then use your headers inside a JMS Selector (whose SQL92 syntax mandates Java identifier syntax for headers).

A simple strategy for mapping header names is used by default. The strategy is to replace any dots and hyphens in the header name as shown below and to reverse the replacement when the header name is restored from a JMS message sent over the wire. What does this mean? No more losing method names to invoke on a bean component, no more losing the filename header for the File Component, and so on.

The current header name strategy for accepting header names in Camel is as follows:

- Dots are replaced by ``DOT`` and the replacement is reversed when Camel consume the message
- Hyphen is replaced by ``HYPHEN`` and the replacement is reversed when Camel consumes the message

You can configure many different properties on the JMS endpoint, which map to properties on the **JMSConfiguration** object.



NOTE

Mapping to Spring JMS

Many of these properties map to properties on Spring JMS, which Camel uses for sending and receiving messages. So you can get more information about these properties by consulting the relevant Spring documentation.

58.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

58.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a

few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

58.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

58.4. COMPONENT OPTIONS

The JMS component supports 98 options, which are listed below.

Name	Description	Default	Type
clientId (common)	Sets the JMS client ID to use. Note that this value, if specified, must be unique and can only be used by a single JMS connection instance. The clientId option is compulsory with JMS 1.1 durable topic subscriptions, because the client ID is used to control which client messages have to be stored for. With JMS 2.0 clients, clientId may be omitted, which creates a 'global' subscription. If using Apache ActiveMQ you may prefer to use Virtual Topics instead.		String

Name	Description	Default	Type
connectionFactory (common)	The connection factory to be use. A connection factory must be configured either on the component or endpoint.		ConnectionFactory
disableReplyTo (common)	Specifies whether Camel ignores the JMSReplyTo header in messages. If true, Camel does not send a reply back to the destination specified in the JMSReplyTo header. You can use this option if you want Camel to consume from a route and you do not want Camel to automatically send back a reply message because another component in your code handles the reply message. You can also use this option if you want to use Camel as a proxy between different message brokers and you want to route message from one system to another.	false	boolean
durableSubscriptionName (common)	The durable subscriber name for specifying durable topic subscriptions. The clientId option must be configured for a JMS 1.1 durable subscription, and may be configured for JMS 2.0, to create a private durable subscription.		String
jmsMessageType (common)	Allows you to force the use of a specific javax.jms.Message implementation for sending JMS messages. Possible values are: Bytes, Map, Object, Stream, Text. By default, Camel would determine which JMS message type to use from the In body type. This option allows you to specify it. Enum values: • Bytes • Map • Object • Stream • Text		JmsMessageType
replyTo (common)	Provides an explicit ReplyTo destination (overrides any incoming value of Message.getJMSReplyTo() in consumer).		String

Name	Description	Default	Type
testConnectionOnStartup (common)	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	boolean
acknowledgmentModeName (consumer)	The JMS acknowledgement name, which is one of: SESSION_TRANSACTED, CLIENT_ACKNOWLEDGE, AUTO_ACKNOWLEDGE, DUPS_OK_ACKNOWLEDGE. Enum values: • SESSION_TRANSACTED • CLIENT_ACKNOWLEDGE • AUTO_ACKNOWLEDGE • DUPS_OK_ACKNOWLEDGE	AUTO_ACKNOWLEDGE	String
artemisConsumerPriority (consumer)	Consumer priorities allow you to ensure that high priority consumers receive messages while they are active. Normally, active consumers connected to a queue receive messages from it in a round-robin fashion. When consumer priorities are in use, messages are delivered round-robin if multiple active consumers exist with the same high priority. Messages will only going to lower priority consumers when the high priority consumers do not have credit available to consume the message, or those high priority consumers have declined to accept the message (for instance because it does not meet the criteria of any selectors associated with the consumer).		int

Name	Description	Default	Type
asyncConsumer (consumer)	Whether the JmsConsumer processes the Exchange asynchronously. If enabled then the JmsConsumer may pickup the next message from the JMS queue, while the previous message is being processed asynchronously (by the Asynchronous Routing Engine). This means that messages may be processed not 100% strictly in order. If disabled (as default) then the Exchange is fully processed before the JmsConsumer picks up the next message from the JMS queue. Note if transacted has been enabled, then asyncConsumer=true does not run asynchronously, as transaction must be executed synchronously (Camel 3.0 may support async transactions).	false	boolean
autoStartup (consumer)	Specifies whether the consumer container should auto-startup.	true	boolean
cacheLevel (consumer)	Sets the cache level by ID for the underlying JMS resources. See cacheLevelName option for more details.		int
cacheLevelName (consumer)	Sets the cache level by name for the underlying JMS resources. Possible values are: CACHE_AUTO, CACHE_CONNECTION, CACHE_CONSUMER, CACHE_NONE, and CACHE_SESSION. The default setting is CACHE_AUTO. See the Spring documentation and Transactions Cache Levels for more information. Enum values: ● CACHE_AUTO ● CACHE_CONNECTION ● CACHE_CONSUMER ● CACHE_NONE ● CACHE_SESSION	CACHE_AUTO	String
concurrentConsumers (consumer)	Specifies the default number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the maxMessagesPerTask option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option replyToConcurrentConsumers is used to control number of concurrent consumers on the reply message listener.	1	int

Name	Description	Default	Type
maxConcurrentConsumers (consumer)	Specifies the maximum number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToMaxConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.		int
replyToDeliveryPersistent (consumer)	Specifies whether to use persistent delivery by default for replies.	true	boolean
selector (consumer)	Sets the JMS selector to use.		String
subscriptionDurable (consumer)	Set whether to make the subscription durable. The durable subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a durable subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well.	false	boolean
subscriptionName (consumer)	Set the name of a subscription to create. To be applied in case of a topic (pub-sub domain) with a shared or durable subscription. The subscription name needs to be unique within this client's JMS client id, if the client ID is configured. Default is the class name of the specified message listener. Note: Only 1 concurrent consumer (which is the default of this message listener container) is allowed for each subscription, except for a shared subscription (which requires JMS 2.0).		String

Name	Description	Default	Type
subscriptionShared (consumer)	Set whether to make the subscription shared. The shared subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a shared subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Note that shared subscriptions may also be durable, so this flag can (and often will) be combined with <code>subscriptionDurable</code> as well. Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well. Requires a JMS 2.0 compatible message broker.	false	boolean
acceptMessagesWhileStopping (consumer (advanced))	Specifies whether the consumer accept messages while it is stopping. You may consider enabling this option, if you start and stop JMS routes at runtime, while there are still messages enqueued on the queue. If this option is false, and you stop the JMS route, then messages may be rejected, and the JMS broker would have to attempt redeliveries, which yet again may be rejected, and eventually the message may be moved at a dead letter queue on the JMS broker. To avoid this its recommended to enable this option.	false	boolean
allowReplyManagerQuickStop (consumer (advanced))	Whether the <code>DefaultMessageListenerContainer</code> used in the reply managers for request/reply messaging allow the <code>DefaultMessageListenerContainer.runningAllowed</code> flag to quick stop in case <code>JmsConfiguration#isAcceptMessagesWhileStopping</code> is enabled, and <code>org.apache.camel.CamelContext</code> is currently being stopped. This quick stop ability is enabled by default in the regular JMS consumers but to enable for reply managers you must enable this flag.	false	boolean

Name	Description	Default	Type
consumerType (consumer (advanced))	The consumer type to use, which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use <code>org.springframework.jms.listener.DefaultMessageListenerContainer</code>, Simple will use <code>org.springframework.jms.listener.SimpleMessageListenerContainer</code>. When Custom is specified, the <code>MessageListenerContainerFactory</code> defined by the <code>messageListenerContainerFactory</code> option will determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use. Enum values: • Simple • Default • Custom	Default	ConsumerType
defaultTaskExecutorType (consumer (advanced))	Specifies what default <code>TaskExecutor</code> type to use in the <code>DefaultMessageListenerContainer</code>, for both consumer endpoints and the <code>ReplyTo</code> consumer of producer endpoints. Possible values: <code>SimpleAsync</code> (uses Spring's <code>SimpleAsyncTaskExecutor</code>) or <code>ThreadPool</code> (uses Spring's <code>ThreadPoolTaskExecutor</code> with optimal values - cached threadpool-like). If not set, it defaults to the previous behaviour, which uses a cached thread pool for consumer endpoints and <code>SimpleAsync</code> for reply consumers. The use of <code>ThreadPool</code> is recommended to reduce thread trash in elastic configurations with dynamically increasing and decreasing concurrent consumers. Enum values: • <code>ThreadPool</code> • <code>SimpleAsync</code>		DefaultTaskExecutorType
eagerLoadingOfProperties (consumer (advanced))	Enables eager loading of JMS properties and payload as soon as a message is loaded which generally is inefficient as the JMS properties may not be required but sometimes can catch early any issues with the underlying JMS provider and the use of JMS properties. See also the option <code>eagerPoisonBody</code>.	false	boolean

Name	Description	Default	Type
eagerPoisonBody (consumer (advanced))	If eagerLoadingOfProperties is enabled and the JMS message payload (JMS body or JMS properties) is poison (cannot be read/mapped), then set this text as the message body instead so the message can be processed (the cause of the poison are already stored as exception on the Exchange). This can be turned off by setting eagerPoisonBody=false. See also the option eagerLoadingOfProperties.	Poison JMS message due to <code>{exception.message}</code>	String
exposeListenerSession (consumer (advanced))	Specifies whether the listener session should be exposed when consuming messages.	false	boolean
replyToSameDestinationAllowed (consumer (advanced))	Whether a JMS consumer is allowed to send a reply message to the same destination that the consumer is using to consume from. This prevents an endless loop by consuming and sending back the same message to itself.	false	boolean
taskExecutor (consumer (advanced))	Allows you to specify a custom task executor for consuming messages.		TaskExecutor
deliveryDelay (producer)	Sets delivery delay to use for send calls for JMS. This option requires JMS 2.0 compliant broker.	-1	long
deliveryMode (producer)	Specifies the delivery mode to be used. Possible values are those defined by <code>javax.jms.DeliveryMode</code> . <code>NON_PERSISTENT</code> = 1 and <code>PERSISTENT</code> = 2. Enum values: • 1 • 2		Integer
deliveryPersistent (producer)	Specifies whether persistent delivery is used by default.	true	boolean

Name	Description	Default	Type
explicitQosEnabled (producer)	Set if the deliveryMode, priority or timeToLive qualities of service should be used when sending messages. This option is based on Spring's JmsTemplate. The deliveryMode, priority and timeToLive options are applied to the current endpoint. This contrasts with the preserveMessageQos option, which operates at message granularity, reading QoS properties exclusively from the Camel In message headers.	false	Boolean
formatDateHeadersToIso8601 (producer)	Sets whether JMS date properties should be formatted according to the ISO 8601 standard.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
preserveMessageQos (producer)	Set to true, if you want to send message using the QoS settings specified on the message, instead of the QoS settings on the JMS endpoint. The following three headers are considered JMSPriority, JMSDeliveryMode, and JMSExpiration. You can provide all or only some of them. If not provided, Camel will fall back to use the values from the endpoint instead. So, when using this option, the headers override the values from the endpoint. The explicitQosEnabled option, by contrast, will only use options set on the endpoint, and not values from the message header.	false	boolean

Name	Description	Default	Type
priority (producer)	Values greater than 1 specify the message priority when sending (where 1 is the lowest priority and 9 is the highest). The <code>explicitQosEnabled</code> option must also be enabled in order for this option to have any effect. Enum values: • 1 • 2 • 3 • 4 • 5 • 6 • 7 • 8 • 9	4	int
replyToConcurrentConsumers (producer)	Specifies the default number of concurrent consumers when doing request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.	1	int
replyToMaxConcurrentConsumers (producer)	Specifies the maximum number of concurrent consumers when using request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.		int
replyToOnTimeoutMaxConcurrentConsumers (producer)	Specifies the maximum number of concurrent consumers for continue routing when timeout occurred when using request/reply over JMS.	1	int
replyToOverride (producer)	Provides an explicit <code>ReplyTo</code> destination in the JMS message, which overrides the setting of <code>replyTo</code> . It is useful if you want to forward the message to a remote Queue and receive the reply message from the <code>ReplyTo</code> destination.		String

Name	Description	Default	Type
replyToType (producer)	Allows for explicitly specifying which kind of strategy to use for replyTo queues when doing request/reply over JMS. Possible values are: Temporary, Shared, or Exclusive. By default Camel will use temporary queues. However if replyTo has been configured, then Shared is used by default. This option allows you to use exclusive queues instead of shared ones. See Camel JMS documentation for more details, and especially the notes about the implications if running in a clustered environment, and the fact that Shared reply queues has lower performance than its alternatives Temporary and Exclusive. Enum values: • Temporary • Shared • Exclusive		ReplyToType
requestTimeout (producer)	The timeout for waiting for a reply when using the InOut Exchange Pattern (in milliseconds). The default is 20 seconds. You can include the header CamelJmsRequestTimeout to override this endpoint configured timeout value, and thus have per message individual timeout values. See also the requestTimeoutCheckerInterval option.	20000	long
timeToLive (producer)	When sending messages, specifies the time-to-live of the message (in milliseconds).	-1	long
allowAdditionalHeaders (producer (advanced))	This option is used to allow additional headers which may have values that are invalid according to JMS specification. For example some message systems such as WMQ do this with header names using prefix JMS_IBM_MQMD_ containing values with byte array or other invalid types. You can specify multiple header names separated by comma, and use as suffix for wildcard matching.		String
allowNullBody (producer (advanced))	Whether to allow sending messages with no body. If this option is false and the message body is null, then an JMSEException is thrown.	true	boolean

Name	Description	Default	Type
alwaysCopyMessage (producer (advanced))	If true, Camel will always make a JMS message copy of the message when it is passed to the producer for sending. Copying the message is needed in some situations, such as when a <code>replyToDestinationSelectorName</code> is set (incidentally, Camel will set the <code>alwaysCopyMessage</code> option to true, if a <code>replyToDestinationSelectorName</code> is set).	false	boolean
correlationProperty (producer (advanced))	When using InOut exchange pattern use this JMS property instead of <code>JMSCorrelationID</code> JMS property to correlate messages. If set messages will be correlated solely on the value of this property <code>JMSCorrelationID</code> property will be ignored and not set by Camel.		String
disableTimeToLive (producer (advanced))	Use this option to force disabling time to live. For example when you do request/reply over JMS, then Camel will by default use the <code>requestTimeout</code> value as time to live on the message being sent. The problem is that the sender and receiver systems have to have their clocks synchronized, so they are in sync. This is not always so easy to archive. So you can use <code>disableTimeToLive=true</code> to not set a time to live value on the sent message. Then the message will not expire on the receiver system. See below in section About time to live for more details.	false	boolean
forceSendOriginalMessage (producer (advanced))	When using <code>mapJmsMessage=false</code> Camel will create a new JMS message to send to a new JMS destination if you touch the headers (get or set) during the route. Set this option to true to force Camel to send the original JMS message that was received.	false	boolean
includeSentJMSMessageID (producer (advanced))	Only applicable when sending to JMS destination using InOnly (eg fire and forget). Enabling this option will enrich the Camel Exchange with the actual <code>JMSMessageID</code> that was used by the JMS client when the message was sent to the JMS destination.	false	boolean

Name	Description	Default	Type
replyToCacheLevelName (producer (advanced))	Sets the cache level by name for the reply consumer when doing request/reply over JMS. This option only applies when using fixed reply queues (not temporary). Camel will by default use: <code>CACHE_CONSUMER</code> for exclusive or shared w/ <code>replyToSelectorName</code>. And <code>CACHE_SESSION</code> for shared without <code>replyToSelectorName</code>. Some JMS brokers such as IBM WebSphere may require to set the <code>replyToCacheLevelName=CACHE_NONE</code> to work. Note: If using temporary queues then <code>CACHE_NONE</code> is not allowed, and you must use a higher value such as <code>CACHE_CONSUMER</code> or <code>CACHE_SESSION</code>. Enum values: ● <code>CACHE_AUTO</code> ● <code>CACHE_CONNECTION</code> ● <code>CACHE_CONSUMER</code> ● <code>CACHE_NONE</code> ● <code>CACHE_SESSION</code>		String
replyToSelectorName (producer (advanced))	Sets the JMS Selector using the fixed name to be used so you can filter out your own replies from the others when using a shared queue (that is, if you are not using a temporary reply queue).		String
streamMessageTypeEnabled (producer (advanced))	Sets whether <code>StreamMessage</code> type is enabled or not. Message payloads of streaming kind such as files, <code>InputStream</code> , etc will either be sent as <code>BytesMessage</code> or <code>StreamMessage</code> . This option controls which kind will be used. By default <code>BytesMessage</code> is used which enforces the entire message payload to be read into memory. By enabling this option the message payload is read into memory in chunks and each chunk is then written to the <code>StreamMessage</code> until no more data.	false	boolean
allowAutoWiredConnectionFactory (advanced)	Whether to auto-discover <code>ConnectionFactory</code> from the registry, if no connection factory has been configured. If only one instance of <code>ConnectionFactory</code> is found then it will be used. This is enabled by default.	true	boolean

Name	Description	Default	Type
allowAutoWiredDestinationResolver (advanced)	Whether to auto-discover DestinationResolver from the registry, if no destination resolver has been configured. If only one instance of DestinationResolver is found then it will be used. This is enabled by default.	true	boolean
allowSerializedHeaders (advanced)	Controls whether or not to include serialized headers. Applies only when transferExchange is true. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
artemisStreamingEnabled (advanced)	Whether optimizing for Apache Artemis streaming mode. This can reduce memory overhead when using Artemis with JMS StreamMessage types. This option must only be enabled if Apache Artemis is being used.	false	boolean
asyncStartListener (advanced)	Whether to startup the JmsConsumer message listener asynchronously, when starting a route. For example if a JmsConsumer cannot get a connection to a remote JMS broker, then it may block while retrying and/or failover. This will cause Camel to block while starting routes. By setting this option to true, you will let routes startup, while the JmsConsumer connects to the JMS broker using a dedicated thread in asynchronous mode. If this option is used, then beware that if the connection could not be established, then an exception is logged at WARN level, and the consumer will not be able to receive messages; You can then restart the route to retry.	false	boolean
asyncStopListener (advanced)	Whether to stop the JmsConsumer message listener asynchronously, when stopping a route.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
configuration (advanced)	To use a shared JMS configuration.		JmsConfiguration

Name	Description	Default	Type
destinationResolver (advanced)	A pluggable <code>org.springframework.jms.support.destination.DestinationResolver</code> that allows you to use your own resolver (for example, to lookup the real destination in a JNDI registry).		<code>DestinationResolver</code>
errorHandler (advanced)	Specifies a <code>org.springframework.util.ErrorHandler</code> to be invoked in case of any uncaught exceptions thrown while processing a <code>Message</code> . By default these exceptions will be logged at the <code>WARN</code> level, if no <code>errorHandler</code> has been configured. You can configure logging level and whether stack traces should be logged using <code>errorHandlerLoggingLevel</code> and <code>errorHandlerLogStackTrace</code> options. This makes it much easier to configure, than having to code a custom <code>ErrorHandler</code> .		<code>ErrorHandler</code>
exceptionListener (advanced)	Specifies the JMS Exception Listener that is to be notified of any underlying JMS exceptions.		<code>ExceptionListener</code>
idleConsumerLimit (advanced)	Specify the limit for the number of consumers that are allowed to be idle at any given time.	1	int
idleTaskExecutionLimit (advanced)	Specifies the limit for idle executions of a receive task, not having received any message within its execution. If this limit is reached, the task will shut down and leave receiving to other executing tasks (in the case of dynamic scheduling; see the <code>maxConcurrentConsumers</code> setting). There is additional doc available from Spring.	1	int
includeAllJMSXProperties (advanced)	Whether to include all <code>JMSXxxx</code> properties when mapping from JMS to Camel Message. Setting this to true will include properties such as <code>JMSXAppID</code> , and <code>JMSXUserID</code> etc. Note: If you are using a custom <code>headerFilterStrategy</code> then this option does not apply.	false	boolean

Name	Description	Default	Type
jmsKeyFormatStrategy (advanced)	Pluggable strategy for encoding and decoding JMS keys so they can be compliant with the JMS specification. Camel provides two implementations out of the box: default and passthrough. The default strategy will safely marshal dots and hyphens (. and -). The passthrough strategy leaves the key as is. Can be used for JMS brokers which do not care whether JMS header keys contain illegal characters. You can provide your own implementation of the <code>org.apache.camel.component.jms.JmsKeyFormatStrategy</code> and refer to it using the <code>#</code> notation. Enum values: • default • passthrough		JmsKeyFormatStrategy
mapJmsMessage (advanced)	Specifies whether Camel should auto map the received JMS message to a suited payload type, such as <code>javax.jms.TextMessage</code> to a <code>String</code> etc.	true	boolean
maxMessagesPerTask (advanced)	The number of messages per task. -1 is unlimited. If you use a range for concurrent consumers (eg min max), then this option can be used to set a value to eg 100 to control how fast the consumers will shrink when less work is required.	-1	int
messageConverter (advanced)	To use a custom Spring <code>org.springframework.jms.support.converter.MessageConverter</code> so you can be in control how to map to/from a <code>javax.jms.Message</code> .		MessageConverter
messageCreatedStrategy (advanced)	To use the given <code>MessageCreatedStrategy</code> which are invoked when Camel creates new instances of <code>javax.jms.Message</code> objects when Camel is sending a JMS message.		MessageCreatedStrategy
messageIdEnabled (advanced)	When sending, specifies whether message IDs should be added. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the message ID set to null; if the provider ignores the hint, the message ID must be set to its normal unique value.	true	boolean

Name	Description	Default	Type
messageListenerContainerFactory (advanced)	Registry ID of the MessageListenerContainerFactory used to determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use to consume messages. Setting this will automatically set <code>consumerType</code> to Custom.		MessageListenerContainerFactory
messageTimestampEnabled (advanced)	Specifies whether timestamps should be enabled by default on sending messages. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the timestamp set to zero; if the provider ignores the hint the timestamp must be set to its normal value.	true	boolean
pubSubNoLocal (advanced)	Specifies whether to inhibit the delivery of messages published by its own connection.	false	boolean
queueBrowseStrategy (advanced)	To use a custom QueueBrowseStrategy when browsing queues.		QueueBrowseStrategy
receiveTimeout (advanced)	The timeout for receiving messages (in milliseconds).	1000	long
recoveryInterval (advanced)	Specifies the interval between recovery attempts, i.e. when a connection is being refreshed, in milliseconds. The default is 5000 ms, that is, 5 seconds.	5000	long
requestTimeoutCheckerInterval (advanced)	Configures how often Camel should check for timed out Exchanges when doing request/reply over JMS. By default Camel checks once per second. But if you must react faster when a timeout occurs, then you can lower this interval, to check more frequently. The timeout is determined by the option <code>requestTimeout</code> .	1000	long
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean

Name	Description	Default	Type
transferException (advanced)	If enabled and you are using Request Reply messaging (InOut) and an Exchange failed on the consumer side, then the caused Exception will be send back in response as a javax.jms.ObjectMessage. If the client is Camel, the returned Exception is rethrown. This allows you to use Camel JMS as a bridge in your routing - for example, using persistent queues to enable robust routing. Notice that if you also have transferExchange enabled, this option takes precedence. The caught exception is required to be serializable. The original Exception on the consumer side can be wrapped in an outer exception such as org.apache.camel.RuntimeCamelException when returned to the producer. Use this with caution as the data is using Java Object serialization and requires the receiver to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumer!.	false	boolean
transferExchange (advanced)	You can transfer the exchange over the wire instead of just the body and headers. The following fields are transferred: In body, Out body, Fault body, In headers, Out headers, Fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level. You must enable this option on both the producer and consumer side, so Camel knows the payloads is an Exchange and not a regular payload. Use this with caution as the data is using Java Object serialization and requires the receiver to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumers having to use compatible Camel versions!.	false	boolean
useMessageIDAsCorrelationID (advanced)	Specifies whether JMSMessageID should always be used as JMSCorrelationID for InOut messages.	false	boolean
waitForProvisionCorrelationToBeUpdatedCounter (advanced)	Number of times to wait for provisional correlation id to be updated to the actual correlation id when doing request/reply over JMS and when the option useMessageIDAsCorrelationID is enabled.	50	int

Name	Description	Default	Type
waitForProvisionCorrelationToBeUpdatedThreadSleepingTime (advanced)	Interval in millis to sleep each time while waiting for provisional correlation id to be updated.	100	long
headerFilterStrategy (filter)	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message.		HeaderFilterStrategy
errorHandlerLoggingLevel (logging)	Allows to configure the default errorHandler logging level for logging uncaught exceptions. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	WARN	LoggingLevel
errorHandlerLogStackTrace (logging)	Allows to control whether stacktraces should be logged or not, by the default errorHandler.	true	boolean
password (security)	Password to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
username (security)	Username to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
transacted (transaction)	Specifies whether to use transacted mode.	false	boolean

Name	Description	Default	Type
transactedInOut (transaction)	Specifies whether InOut operations (request reply) default to using transacted mode. If this flag is set to true, then Spring JmsTemplate will have sessionTransacted set to true, and the acknowledgeMode as transacted on the JmsTemplate used for InOut operations. Note from Spring JMS: that within a JTA transaction, the parameters passed to createQueue, createTopic methods are not taken into account. Depending on the Java EE transaction context, the container makes its own decisions on these values. Analogously, these parameters are not taken into account within a locally managed transaction either, since Spring JMS operates on an existing JMS Session in this case. Setting this flag to true will use a short local JMS transaction when running outside of a managed transaction, and a synchronized local JMS transaction in case of a managed transaction (other than an XA transaction) being present. This has the effect of a local JMS transaction being managed alongside the main transaction (which might be a native JDBC transaction), with the JMS transaction committing right after the main transaction.	false	boolean
lazyCreateTransactionManager (transaction advanced))	If true, Camel will create a JmsTransactionManager, if there is no transactionManager injected when option transacted=true.	true	boolean
transactionManager (transaction advanced))	The Spring transaction manager to use.		PlatformTransactionManager
transactionName (transaction advanced))	The name of the transaction to use.		String
transactionTimeout (transaction advanced))	The timeout value of the transaction (in seconds), if using transacted mode.	-1	int

58.5. ENDPOINT OPTIONS

The JMS endpoint is configured using URI syntax:

```
jms:destinationType:destinationName
```

with the following path and query parameters:

58.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
destinationType (common)	The kind of destination to use. Enum values: • queue • topic • temp-queue • temp-topic	queue	String
destinationName (common)	Required Name of the queue or topic to use as destination.		String

58.5.2. Query Parameters (95 parameters)

Name	Description	Default	Type
clientId (common)	Sets the JMS client ID to use. Note that this value, if specified, must be unique and can only be used by a single JMS connection instance. It is typically only required for durable topic subscriptions with JMS 1.1. If using Apache ActiveMQ you may prefer to use Virtual Topics instead.		String
connectionFactory (common)	The connection factory to be use. A connection factory must be configured either on the component or endpoint.		ConnectionFactory
disableReplyTo (common)	Specifies whether Camel ignores the JMSReplyTo header in messages. If true, Camel does not send a reply back to the destination specified in the JMSReplyTo header. You can use this option if you want Camel to consume from a route and you do not want Camel to automatically send back a reply message because another component in your code handles the reply message. You can also use this option if you want to use Camel as a proxy between different message brokers and you want to route message from one system to another.	false	boolean
durableSubscriptionName (common)	The durable subscriber name for specifying durable topic subscriptions. The clientId option must be configured as well.		String

Name	Description	Default	Type
jmsMessageType (common)	Allows you to force the use of a specific <code>javax.jms.Message</code> implementation for sending JMS messages. Possible values are: Bytes, Map, Object, Stream, Text. By default, Camel would determine which JMS message type to use from the In body type. This option allows you to specify it. Enum values: • Bytes • Map • Object • Stream • Text		JmsMessageType
replyTo (common)	Provides an explicit ReplyTo destination (overrides any incoming value of <code>Message.getJMSReplyTo()</code> in consumer).		String
testConnectionOnStartup (common)	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	boolean
acknowledgmentModeName (consumer)	The JMS acknowledgement name, which is one of: <code>SESSION_TRANSACTED</code>, <code>CLIENT_ACKNOWLEDGE</code>, <code>AUTO_ACKNOWLEDGE</code>, <code>DUPS_OK_ACKNOWLEDGE</code>. Enum values: • <code>SESSION_TRANSACTED</code> • <code>CLIENT_ACKNOWLEDGE</code> • <code>AUTO_ACKNOWLEDGE</code> • <code>DUPS_OK_ACKNOWLEDGE</code>	AUTO_ACKNOWLEDGE	String

Name	Description	Default	Type
artemisConsumerPriority (consumer)	Consumer priorities allow you to ensure that high priority consumers receive messages while they are active. Normally, active consumers connected to a queue receive messages from it in a round-robin fashion. When consumer priorities are in use, messages are delivered round-robin if multiple active consumers exist with the same high priority. Messages will only go to lower priority consumers when the high priority consumers do not have credit available to consume the message, or those high priority consumers have declined to accept the message (for instance because it does not meet the criteria of any selectors associated with the consumer).		int
asyncConsumer (consumer)	Whether the JmsConsumer processes the Exchange asynchronously. If enabled then the JmsConsumer may pickup the next message from the JMS queue, while the previous message is being processed asynchronously (by the Asynchronous Routing Engine). This means that messages may be processed not 100% strictly in order. If disabled (as default) then the Exchange is fully processed before the JmsConsumer picks up the next message from the JMS queue. Note if transacted has been enabled, then asyncConsumer=true does not run asynchronously, as transaction must be executed synchronously (Camel 3.0 may support async transactions).	false	boolean
autoStartup (consumer)	Specifies whether the consumer container should auto-startup.	true	boolean
cacheLevel (consumer)	Sets the cache level by ID for the underlying JMS resources. See cacheLevelName option for more details.		int

Name	Description	Default	Type
cacheLevelName (consumer)	Sets the cache level by name for the underlying JMS resources. Possible values are: <code>CACHE_AUTO</code>, <code>CACHE_CONNECTION</code>, <code>CACHE_CONSUMER</code>, <code>CACHE_NONE</code>, and <code>CACHE_SESSION</code>. The default setting is <code>CACHE_AUTO</code>. See the Spring documentation and Transactions Cache Levels for more information. Enum values: • <code>CACHE_AUTO</code> • <code>CACHE_CONNECTION</code> • <code>CACHE_CONSUMER</code> • <code>CACHE_NONE</code> • <code>CACHE_SESSION</code>	<code>CACHE_AUTO</code>	String
concurrentConsumers (consumer)	Specifies the default number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.	1	int
maxConcurrentConsumers (consumer)	Specifies the maximum number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToMaxConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.		int
replyToDeliveryPersistent (consumer)	Specifies whether to use persistent delivery by default for replies.	true	boolean
selector (consumer)	Sets the JMS selector to use.		String

Name	Description	Default	Type
subscriptionDurable (consumer)	Set whether to make the subscription durable. The durable subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a durable subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well.	false	boolean
subscriptionName (consumer)	Set the name of a subscription to create. To be applied in case of a topic (pub-sub domain) with a shared or durable subscription. The subscription name needs to be unique within this client's JMS client id. Default is the class name of the specified message listener. Note: Only 1 concurrent consumer (which is the default of this message listener container) is allowed for each subscription, except for a shared subscription (which requires JMS 2.0).		String
subscriptionShared (consumer)	Set whether to make the subscription shared. The shared subscription name to be used can be specified through the <code>subscriptionName</code> property. Default is false. Set this to true to register a shared subscription, typically in combination with a <code>subscriptionName</code> value (unless your message listener class name is good enough as subscription name). Note that shared subscriptions may also be durable, so this flag can (and often will) be combined with <code>subscriptionDurable</code> as well. Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the <code>pubSubDomain</code> flag as well. Requires a JMS 2.0 compatible message broker.	false	boolean
acceptMessagesWhileStopping (consumer (advanced))	Specifies whether the consumer accept messages while it is stopping. You may consider enabling this option, if you start and stop JMS routes at runtime, while there are still messages enqueued on the queue. If this option is false, and you stop the JMS route, then messages may be rejected, and the JMS broker would have to attempt redeliveries, which yet again may be rejected, and eventually the message may be moved at a dead letter queue on the JMS broker. To avoid this its recommended to enable this option.	false	boolean

Name	Description	Default	Type
allowReplyManagerQuickStop (consumer (advanced))	Whether the DefaultMessageListenerContainer used in the reply managers for request/reply messaging allow the DefaultMessageListenerContainer.runningAllowed flag to quick stop in case JmsConfiguration#isAcceptMessagesWhileStopping is enabled, and org.apache.camel.CamelContext is currently being stopped. This quick stop ability is enabled by default in the regular JMS consumers but to enable for reply managers you must enable this flag.	false	boolean
consumerType (consumer (advanced))	The consumer type to use, which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use org.springframework.jms.listener.DefaultMessageListenerContainer, Simple will use org.springframework.jms.listener.SimpleMessageListenerContainer. When Custom is specified, the MessageListenerContainerFactory defined by the messageListenerContainerFactory option will determine what org.springframework.jms.listener.AbstractMessageListenerContainer to use. Enum values: • Simple • Default • Custom	Default	ConsumerType

Name	Description	Default	Type
defaultTaskExecutorType (consumer (advanced))	Specifies what default TaskExecutor type to use in the DefaultMessageListenerContainer, for both consumer endpoints and the ReplyTo consumer of producer endpoints. Possible values: SimpleAsync (uses Spring's SimpleAsyncTaskExecutor) or ThreadPool (uses Spring's ThreadPoolTaskExecutor with optimal values - cached threadpool-like). If not set, it defaults to the previous behaviour, which uses a cached thread pool for consumer endpoints and SimpleAsync for reply consumers. The use of ThreadPool is recommended to reduce thread trash in elastic configurations with dynamically increasing and decreasing concurrent consumers. Enum values: • ThreadPool • SimpleAsync		DefaultTaskExecutorType
eagerLoadingOfProperties (consumer (advanced))	Enables eager loading of JMS properties and payload as soon as a message is loaded which generally is inefficient as the JMS properties may not be required but sometimes can catch early any issues with the underlying JMS provider and the use of JMS properties. See also the option eagerPoisonBody.	false	boolean
eagerPoisonBody (consumer (advanced))	If eagerLoadingOfProperties is enabled and the JMS message payload (JMS body or JMS properties) is poison (cannot be read/mapped), then set this text as the message body instead so the message can be processed (the cause of the poison are already stored as exception on the Exchange). This can be turned off by setting eagerPoisonBody=false. See also the option eagerLoadingOfProperties.	Poison JMS message due to \${exception.message}	String
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern
exposeListenerSession (consumer (advanced))	Specifies whether the listener session should be exposed when consuming messages.	false	boolean
replyToSameDestinationAllowed (consumer (advanced))	Whether a JMS consumer is allowed to send a reply message to the same destination that the consumer is using to consume from. This prevents an endless loop by consuming and sending back the same message to itself.	false	boolean
taskExecutor (consumer (advanced))	Allows you to specify a custom task executor for consuming messages.		TaskExecutor
deliveryDelay (producer)	Sets delivery delay to use for send calls for JMS. This option requires JMS 2.0 compliant broker.	-1	long
deliveryMode (producer)	Specifies the delivery mode to be used. Possible values are those defined by <code>javax.jms.DeliveryMode</code>. <code>NON_PERSISTENT</code> = 1 and <code>PERSISTENT</code> = 2. Enum values: • 1 • 2		Integer
deliveryPersistent (producer)	Specifies whether persistent delivery is used by default.	true	boolean

Name	Description	Default	Type
explicitQosEnabled (producer)	Set if the deliveryMode, priority or timeToLive qualities of service should be used when sending messages. This option is based on Spring's JmsTemplate. The deliveryMode, priority and timeToLive options are applied to the current endpoint. This contrasts with the preserveMessageQos option, which operates at message granularity, reading QoS properties exclusively from the Camel In message headers.	false	Boolean
formatDateHeadersToIso8601 (producer)	Sets whether JMS date properties should be formatted according to the ISO 8601 standard.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
preserveMessageQos (producer)	Set to true, if you want to send message using the QoS settings specified on the message, instead of the QoS settings on the JMS endpoint. The following three headers are considered JMSPriority, JMSDeliveryMode, and JMSExpiration. You can provide all or only some of them. If not provided, Camel will fall back to use the values from the endpoint instead. So, when using this option, the headers override the values from the endpoint. The explicitQosEnabled option, by contrast, will only use options set on the endpoint, and not values from the message header.	false	boolean

Name	Description	Default	Type
priority (producer)	Values greater than 1 specify the message priority when sending (where 1 is the lowest priority and 9 is the highest). The <code>explicitQosEnabled</code> option must also be enabled in order for this option to have any effect. Enum values: • 1 • 2 • 3 • 4 • 5 • 6 • 7 • 8 • 9	4	int
replyToConcurrentConsumers (producer)	Specifies the default number of concurrent consumers when doing request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.	1	int
replyToMaxConcurrentConsumers (producer)	Specifies the maximum number of concurrent consumers when using request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.		int
replyToOnTimeoutMaxConcurrentConsumers (producer)	Specifies the maximum number of concurrent consumers for continue routing when timeout occurred when using request/reply over JMS.	1	int
replyToOverride (producer)	Provides an explicit <code>ReplyTo</code> destination in the JMS message, which overrides the setting of <code>replyTo</code> . It is useful if you want to forward the message to a remote Queue and receive the reply message from the <code>ReplyTo</code> destination.		String

Name	Description	Default	Type
replyToType (producer)	Allows for explicitly specifying which kind of strategy to use for replyTo queues when doing request/reply over JMS. Possible values are: Temporary, Shared, or Exclusive. By default Camel will use temporary queues. However if replyTo has been configured, then Shared is used by default. This option allows you to use exclusive queues instead of shared ones. See Camel JMS documentation for more details, and especially the notes about the implications if running in a clustered environment, and the fact that Shared reply queues has lower performance than its alternatives Temporary and Exclusive. Enum values: • Temporary • Shared • Exclusive		ReplyToType
requestTimeout (producer)	The timeout for waiting for a reply when using the InOut Exchange Pattern (in milliseconds). The default is 20 seconds. You can include the header CamelJmsRequestTimeout to override this endpoint configured timeout value, and thus have per message individual timeout values. See also the requestTimeoutCheckerInterval option.	20000	long
timeToLive (producer)	When sending messages, specifies the time-to-live of the message (in milliseconds).	-1	long
allowAdditionalHeaders (producer (advanced))	This option is used to allow additional headers which may have values that are invalid according to JMS specification. For example some message systems such as WMQ do this with header names using prefix JMS_IBM_MQMD_ containing values with byte array or other invalid types. You can specify multiple header names separated by comma, and use as suffix for wildcard matching.		String
allowNullBody (producer (advanced))	Whether to allow sending messages with no body. If this option is false and the message body is null, then an JMSEException is thrown.	true	boolean

Name	Description	Default	Type
alwaysCopyMessage (producer (advanced))	If true, Camel will always make a JMS message copy of the message when it is passed to the producer for sending. Copying the message is needed in some situations, such as when a <code>replyToDestinationSelectorName</code> is set (incidentally, Camel will set the <code>alwaysCopyMessage</code> option to true, if a <code>replyToDestinationSelectorName</code> is set).	false	boolean
correlationProperty (producer (advanced))	When using InOut exchange pattern use this JMS property instead of <code>JMSCorrelationID</code> JMS property to correlate messages. If set messages will be correlated solely on the value of this property <code>JMSCorrelationID</code> property will be ignored and not set by Camel.		String
disableTimeToLive (producer (advanced))	Use this option to force disabling time to live. For example when you do request/reply over JMS, then Camel will by default use the <code>requestTimeout</code> value as time to live on the message being sent. The problem is that the sender and receiver systems have to have their clocks synchronized, so they are in sync. This is not always so easy to archive. So you can use <code>disableTimeToLive=true</code> to not set a time to live value on the sent message. Then the message will not expire on the receiver system. See below in section About time to live for more details.	false	boolean
forceSendOriginalMessage (producer (advanced))	When using <code>mapJmsMessage=false</code> Camel will create a new JMS message to send to a new JMS destination if you touch the headers (get or set) during the route. Set this option to true to force Camel to send the original JMS message that was received.	false	boolean
includeSentJMSMessageID (producer (advanced))	Only applicable when sending to JMS destination using InOnly (eg fire and forget). Enabling this option will enrich the Camel Exchange with the actual <code>JMSMessageID</code> that was used by the JMS client when the message was sent to the JMS destination.	false	boolean

Name	Description	Default	Type
replyToCacheLevelName (producer (advanced))	Sets the cache level by name for the reply consumer when doing request/reply over JMS. This option only applies when using fixed reply queues (not temporary). Camel will by default use: <code>CACHE_CONSUMER</code> for exclusive or shared w/ <code>replyToSelectorName</code>. And <code>CACHE_SESSION</code> for shared without <code>replyToSelectorName</code>. Some JMS brokers such as IBM WebSphere may require to set the <code>replyToCacheLevelName=CACHE_NONE</code> to work. Note: If using temporary queues then <code>CACHE_NONE</code> is not allowed, and you must use a higher value such as <code>CACHE_CONSUMER</code> or <code>CACHE_SESSION</code>. Enum values: ● <code>CACHE_AUTO</code> ● <code>CACHE_CONNECTION</code> ● <code>CACHE_CONSUMER</code> ● <code>CACHE_NONE</code> ● <code>CACHE_SESSION</code>		String
replyToSelectorName (producer (advanced))	Sets the JMS Selector using the fixed name to be used so you can filter out your own replies from the others when using a shared queue (that is, if you are not using a temporary reply queue).		String
streamMessageTypeEnabled (producer (advanced))	Sets whether <code>StreamMessage</code> type is enabled or not. Message payloads of streaming kind such as files, <code>InputStream</code> , etc will either be sent as <code>BytesMessage</code> or <code>StreamMessage</code> . This option controls which kind will be used. By default <code>BytesMessage</code> is used which enforces the entire message payload to be read into memory. By enabling this option the message payload is read into memory in chunks and each chunk is then written to the <code>StreamMessage</code> until no more data.	false	boolean
allowSerializedHeaders (advanced)	Controls whether or not to include serialized headers. Applies only when <code>transferExchange</code> is true. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
artemisStreamingEnabled (advanced)	Whether optimizing for Apache Artemis streaming mode. This can reduce memory overhead when using Artemis with JMS <code>StreamMessage</code> types. This option must only be enabled if Apache Artemis is being used.	false	boolean

Name	Description	Default	Type
asyncStartListener (advanced)	Whether to startup the JmsConsumer message listener asynchronously, when starting a route. For example if a JmsConsumer cannot get a connection to a remote JMS broker, then it may block while retrying and/or failover. This will cause Camel to block while starting routes. By setting this option to true, you will let routes startup, while the JmsConsumer connects to the JMS broker using a dedicated thread in asynchronous mode. If this option is used, then beware that if the connection could not be established, then an exception is logged at WARN level, and the consumer will not be able to receive messages; You can then restart the route to retry.	false	boolean
asyncStopListener (advanced)	Whether to stop the JmsConsumer message listener asynchronously, when stopping a route.	false	boolean
destinationResolver (advanced)	A pluggable org.springframework.jms.support.destination.DestinationResolver that allows you to use your own resolver (for example, to lookup the real destination in a JNDI registry).		DestinationResolver
errorHandler (advanced)	Specifies a org.springframework.util.ErrorHandler to be invoked in case of any uncaught exceptions thrown while processing a Message. By default these exceptions will be logged at the WARN level, if no errorHandler has been configured. You can configure logging level and whether stack traces should be logged using errorHandlerLoggingLevel and errorHandlerLogStackTrace options. This makes it much easier to configure, than having to code a custom errorHandler.		ErrorHandler
exceptionListener (advanced)	Specifies the JMS Exception Listener that is to be notified of any underlying JMS exceptions.		ExceptionListener
headerFilterStrategy (advanced)	To use a custom HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
idleConsumerLimit (advanced)	Specify the limit for the number of consumers that are allowed to be idle at any given time.	1	int

Name	Description	Default	Type
idleTaskExecutionLimit (advanced)	Specifies the limit for idle executions of a receive task, not having received any message within its execution. If this limit is reached, the task will shut down and leave receiving to other executing tasks (in the case of dynamic scheduling; see the <code>maxConcurrentConsumers</code> setting). There is additional doc available from Spring.	1	int
includeAllJMSXProperties (advanced)	Whether to include all JMSXxxx properties when mapping from JMS to Camel Message. Setting this to true will include properties such as JMSXAppID, and JMSXUserID etc. Note: If you are using a custom <code>headerFilterStrategy</code> then this option does not apply.	false	boolean
jmsKeyFormatStrategy (advanced)	Pluggable strategy for encoding and decoding JMS keys so they can be compliant with the JMS specification. Camel provides two implementations out of the box: default and passthrough. The default strategy will safely marshal dots and hyphens (. and -). The passthrough strategy leaves the key as is. Can be used for JMS brokers which do not care whether JMS header keys contain illegal characters. You can provide your own implementation of the <code>org.apache.camel.component.jms.JmsKeyFormatStrategy</code> and refer to it using the # notation. Enum values: • default • passthrough		JmsKeyFormatStrategy
mapJmsMessage (advanced)	Specifies whether Camel should auto map the received JMS message to a suited payload type, such as <code>javax.jms.TextMessage</code> to a String etc.	true	boolean
maxMessagesPerTask (advanced)	The number of messages per task. -1 is unlimited. If you use a range for concurrent consumers (eg min max), then this option can be used to set a value to eg 100 to control how fast the consumers will shrink when less work is required.	-1	int
messageConverter (advanced)	To use a custom Spring <code>org.springframework.jms.support.converter.MessageConverter</code> so you can be in control how to map to/from a <code>javax.jms.Message</code> .		MessageConverter

Name	Description	Default	Type
messageCreatedStrategy (advanced)	To use the given MessageCreatedStrategy which are invoked when Camel creates new instances of javax.jms.Message objects when Camel is sending a JMS message.		MessageCreatedStrategy
messageIdEnabled (advanced)	When sending, specifies whether message IDs should be added. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the message ID set to null; if the provider ignores the hint, the message ID must be set to its normal unique value.	true	boolean
messageListenerContainerFactory (advanced)	Registry ID of the MessageListenerContainerFactory used to determine what org.springframework.jms.listener.AbstractMessageListenerContainer to use to consume messages. Setting this will automatically set consumerType to Custom.		MessageListenerContainerFactory
messageTimestampEnabled (advanced)	Specifies whether timestamps should be enabled by default on sending messages. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the timestamp set to zero; if the provider ignores the hint the timestamp must be set to its normal value.	true	boolean
pubSubNoLocal (advanced)	Specifies whether to inhibit the delivery of messages published by its own connection.	false	boolean
receiveTimeout (advanced)	The timeout for receiving messages (in milliseconds).	1000	long
recoveryInterval (advanced)	Specifies the interval between recovery attempts, i.e. when a connection is being refreshed, in milliseconds. The default is 5000 ms, that is, 5 seconds.	5000	long
requestTimeoutCheckerInterval (advanced)	Configures how often Camel should check for timed out Exchanges when doing request/reply over JMS. By default Camel checks once per second. But if you must react faster when a timeout occurs, then you can lower this interval, to check more frequently. The timeout is determined by the option requestTimeout.	1000	long
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean

Name	Description	Default	Type
transferException (advanced)	If enabled and you are using Request Reply messaging (InOut) and an Exchange failed on the consumer side, then the caused Exception will be send back in response as a <code>javax.jms.ObjectMessage</code> . If the client is Camel, the returned Exception is rethrown. This allows you to use Camel JMS as a bridge in your routing - for example, using persistent queues to enable robust routing. Notice that if you also have <code>transferExchange</code> enabled, this option takes precedence. The caught exception is required to be serializable. The original Exception on the consumer side can be wrapped in an outer exception such as <code>org.apache.camel.RuntimeCamelException</code> when returned to the producer. Use this with caution as the data is using Java Object serialization and requires the received to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumer!.	false	boolean
transferExchange (advanced)	You can transfer the exchange over the wire instead of just the body and headers. The following fields are transferred: In body, Out body, Fault body, In headers, Out headers, Fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level. You must enable this option on both the producer and consumer side, so Camel knows the payloads is an Exchange and not a regular payload. Use this with caution as the data is using Java Object serialization and requires the receiver to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumers having to use compatible Camel versions!.	false	boolean
useMessageIDAsCorrelationID (advanced)	Specifies whether <code>JMSMessageID</code> should always be used as <code>JMSCorrelationID</code> for InOut messages.	false	boolean
waitForProvisionCorrelationToBeUpdatedCounter (advanced)	Number of times to wait for provisional correlation id to be updated to the actual correlation id when doing request/reply over JMS and when the option <code>useMessageIDAsCorrelationID</code> is enabled.	50	int
waitForProvisionCorrelationToBeUpdatedThreadSleepingTime (advanced)	Interval in millis to sleep each time while waiting for provisional correlation id to be updated.	100	long

Name	Description	Default	Type
errorHandlerLoggingLevel (logging)	Allows to configure the default errorHandler logging level for logging uncaught exceptions. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	WARN	LogLevel
errorHandlerLogStackTrace (logging)	Allows to control whether stacktraces should be logged or not, by the default errorHandler.	true	boolean
password (security)	Password to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
username (security)	Username to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
transacted (transaction)	Specifies whether to use transacted mode.	false	boolean

Name	Description	Default	Type
transactedInOut (transaction)	Specifies whether InOut operations (request reply) default to using transacted mode. If this flag is set to true, then Spring JmsTemplate will have sessionTransacted set to true, and the acknowledgeMode as transacted on the JmsTemplate used for InOut operations. Note from Spring JMS: that within a JTA transaction, the parameters passed to createQueue, createTopic methods are not taken into account. Depending on the Java EE transaction context, the container makes its own decisions on these values. Analogously, these parameters are not taken into account within a locally managed transaction either, since Spring JMS operates on an existing JMS Session in this case. Setting this flag to true will use a short local JMS transaction when running outside of a managed transaction, and a synchronized local JMS transaction in case of a managed transaction (other than an XA transaction) being present. This has the effect of a local JMS transaction being managed alongside the main transaction (which might be a native JDBC transaction), with the JMS transaction committing right after the main transaction.	false	boolean
lazyCreateTransactionManager (transaction advanced))	If true, Camel will create a JmsTransactionManager, if there is no transactionManager injected when option transacted=true.	true	boolean
transactionManager (transaction advanced))	The Spring transaction manager to use.		PlatformTransactionManager
transactionName (transaction advanced))	The name of the transaction to use.		String
transactionTimeout (transaction advanced))	The timeout value of the transaction (in seconds), if using transacted mode.	-1	int

58.6. SAMPLES

JMS is used in many examples for other components as well. But we provide a few samples below to get started.

58.6.1. Receiving from JMS

In the following sample we configure a route that receives JMS messages and routes the message to a POJO:

```
from("jms:queue:foo").
  to("bean:myBusinessLogic");
```

You can of course use any of the EIP patterns so the route can be context based. For example, here's how to filter an order topic for the big spenders:

```
from("jms:topic:OrdersTopic").
  filter().method("myBean", "isGoldCustomer").
  to("jms:queue:BigSpendersQueue");
```

58.6.2. Sending to JMS

In the sample below we poll a file folder and send the file content to a JMS topic. As we want the content of the file as a **TextMessage** instead of a **BytesMessage**, we need to convert the body to a **String**:

```
from("file://orders").
  convertBodyTo(String.class).
  to("jms:topic:OrdersTopic");
```

58.6.3. Using Annotations

Camel also has annotations so you can use [POJO Consuming](#) and POJO Producing.

58.6.4. Spring DSL sample

The preceding examples use the Java DSL. Camel also supports Spring XML DSL. Here is the big spender sample using Spring DSL:

```
<route>
  <from uri="jms:topic:OrdersTopic"/>
  <filter>
    <method ref="myBean" method="isGoldCustomer"/>
    <to uri="jms:queue:BigSpendersQueue"/>
  </filter>
</route>
```

58.6.5. Other samples

JMS appears in many of the examples for other components and EIP patterns, as well in this Camel documentation. So feel free to browse the documentation.

58.6.6. Using JMS as a Dead Letter Queue storing Exchange

Normally, when using [JMS](#) as the transport, it only transfers the body and headers as the payload. If you want to use [JMS](#) with a [Dead Letter Channel](#), using a JMS queue as the Dead Letter Queue, then normally the caused Exception is not stored in the JMS message. You can, however, use the **transferExchange** option on the JMS dead letter queue to instruct Camel to store the entire Exchange in the queue as a **javax.jms.ObjectMessage** that holds a

org.apache.camel.support.DefaultExchangeHolder. This allows you to consume from the Dead Letter Queue and retrieve the caused exception from the Exchange property with the key **Exchange.EXCEPTION_CAUGHT**. The demo below illustrates this:

```
// setup error handler to use JMS as queue and store the entire Exchange
errorHandler(deadLetterChannel("jms:queue:dead?transferExchange=true"));
```

Then you can consume from the JMS queue and analyze the problem:

```
from("jms:queue:dead").to("bean:myErrorAnalyzer");

// and in our bean
String body = exchange.getIn().getBody();
Exception cause = exchange.getProperty(Exchange.EXCEPTION_CAUGHT, Exception.class);
// the cause message is
String problem = cause.getMessage();
```

58.6.7. Using JMS as a Dead Letter Channel storing error only

You can use JMS to store the cause error message or to store a custom body, which you can initialize yourself. The following example uses the Message Translator EIP to do a transformation on the failed exchange before it is moved to the [JMS](#) dead letter queue:

```
// we sent it to a seda dead queue first
errorHandler(deadLetterChannel("seda:dead"));

// and on the seda dead queue we can do the custom transformation before its sent to the JMS queue
from("seda:dead").transform(exceptionMessage()).to("jms:queue:dead");
```

Here we only store the original cause error message in the transform. You can, however, use any Expression to send whatever you like. For example, you can invoke a method on a Bean or use a custom processor.

58.7. MESSAGE MAPPING BETWEEN JMS AND CAMEL

Camel automatically maps messages between **javax.jms.Message** and **org.apache.camel.Message**.

When sending a JMS message, Camel converts the message body to the following JMS message types:

Body Type	JMS Message	Comment
String	javax.jms.TextMessage	
org.w3c.dom.Node	javax.jms.TextMessage	The DOM will be converted to String .
Map	javax.jms.MapMessage	
java.io.Serializable	javax.jms.ObjectMessage	

Body Type	JMS Message	Comment
<code>byte[]</code>	<code>javax.jms.BytesMessage</code>	
<code>java.io.File</code>	<code>javax.jms.BytesMessage</code>	
<code>java.io.Reader</code>	<code>javax.jms.BytesMessage</code>	
<code>java.io.InputStream</code>	<code>javax.jms.BytesMessage</code>	
<code>java.nio.ByteBuffer</code>	<code>javax.jms.BytesMessage</code>	

When receiving a JMS message, Camel converts the JMS message to the following body type:

JMS Message	Body Type
<code>javax.jms.TextMessage</code>	<code>String</code>
<code>javax.jms.BytesMessage</code>	<code>byte[]</code>
<code>javax.jms.MapMessage</code>	<code>Map<String, Object></code>
<code>javax.jms.ObjectMessage</code>	<code>Object</code>

58.7.1. Disabling auto-mapping of JMS messages

You can use the `mapJmsMessage` option to disable the auto-mapping above. If disabled, Camel will not try to map the received JMS message, but instead uses it directly as the payload. This allows you to avoid the overhead of mapping and let Camel just pass through the JMS message. For instance, it even allows you to route `javax.jms.ObjectMessage` JMS messages with classes you do **not** have on the classpath.

58.7.2. Using a custom MessageConverter

You can use the `messageConverter` option to do the mapping yourself in a Spring `org.springframework.jms.support.converter.MessageConverter` class.

For example, in the route below we use a custom message converter when sending a message to the JMS order queue:

```
from("file://inbox/order").to("jms:queue:order?messageConverter=#myMessageConverter");
```

You can also use a custom message converter when consuming from a JMS destination.

58.7.3. Controlling the mapping strategy selected

You can use the `jmsMessageType` option on the endpoint URL to force a specific message type for all messages.

In the route below, we poll files from a folder and send them as **javax.jms.TextMessage** as we have forced the JMS producer endpoint to use text messages:

```
from("file://inbox/order").to("jms:queue:order?jmsMessageType=Text");
```

You can also specify the message type to use for each message by setting the header with the key **CamelJmsMessageType**. For example:

```
from("file://inbox/order").setHeader("CamelJmsMessageType",
    JmsMessageType.Text).to("jms:queue:order");
```

The possible values are defined in the **enum** class, **org.apache.camel.jms.JmsMessageType**.

58.8. MESSAGE FORMAT WHEN SENDING

The exchange that is sent over the JMS wire must conform to the [JMS Message spec](#).

For the **exchange.in.header** the following rules apply for the header **keys**:

- Keys starting with **JMS** or **JMSX** are reserved.
- **exchange.in.headers** keys must be literals and all be valid Java identifiers (do not use dots in the key name).
- Camel replaces dots & hyphens and the reverse when consuming JMS messages:
 - . is replaced by ``DOT`` and the reverse replacement when Camel consumes the message.
 - is replaced by ``HYPHEN`` and the reverse replacement when Camel consumes the message.
- See also the option **jmsKeyFormatStrategy**, which allows use of your own custom strategy for formatting keys.

For the **exchange.in.header**, the following rules apply for the header **values**:

- The values must be primitives or their counter objects (such as **Integer**, **Long**, **Character**). The types, **String**, **CharSequence**, **Date**, **BigDecimal** and **BigInteger** are all converted to their **toString()** representation. All other types are dropped.

Camel will log with category **org.apache.camel.component.jms.JmsBinding** at **DEBUG** level if it drops a given header value. For example:

```
2008-07-09 06:43:04,046 [main          ] DEBUG JmsBinding
- Ignoring non primitive header: order of class:
org.apache.camel.component.jms.issues.DummyOrder with value: DummyOrder{orderId=333,
itemId=4444, quantity=2}
```

58.9. MESSAGE FORMAT WHEN RECEIVING

Camel adds the following properties to the **Exchange** when it receives a message:

Property	Type	Description
org.apache.camel.jms.replyDestination	javax.jms.Destination	The reply destination.

Camel adds the following JMS properties to the In message headers when it receives a JMS message:

Header	Type	Description
JMSCorrelationID	String	The JMS correlation ID.
JMSDeliveryMode	int	The JMS delivery mode.
JMSDestination	javax.jms.Destination	The JMS destination.
JMSExpiration	long	The JMS expiration.
JMSMessageID	String	The JMS unique message ID.
JMSPriority	int	The JMS priority (with 0 as the lowest priority and 9 as the highest).
JMSRedelivered	boolean	Is the JMS message redelivered.
JMSReplyTo	javax.jms.Destination	The JMS reply-to destination.
JMSTimestamp	long	The JMS timestamp.
JMSType	String	The JMS type.
JMSXGroupID	String	The JMS group ID.

As all the above information is standard JMS you can check the [JMS documentation](#) for further details.

58.10. ABOUT USING CAMEL TO SEND AND RECEIVE MESSAGES AND JMSREPLYTO

The JMS component is complex and you have to pay close attention to how it works in some cases. So this is a short summary of some of the areas/pitfalls to look for.

When Camel sends a message using its **JMSProducer**, it checks the following conditions:

- The message exchange pattern,
- Whether a **JMSReplyTo** was set in the endpoint or in the message headers,

- Whether any of the following options have been set on the JMS endpoint: **disableReplyTo**, **preserveMessageQos**, **explicitQosEnabled**.

All this can be a tad complex to understand and configure to support your use case.

58.10.1. JmsProducer

The **JmsProducer** behaves as follows, depending on configuration:

Exchange Pattern	Other options	Description
<i>InOut</i>	-	Camel will expect a reply, set a temporary JMSReplyTo , and after sending the message, it will start to listen for the reply message on the temporary queue.
<i>InOut</i>	JMSReplyTo is set	Camel will expect a reply and, after sending the message, it will start to listen for the reply message on the specified JMSReplyTo queue.
<i>InOnly</i>	-	Camel will send the message and not expect a reply.
<i>InOnly</i>	JMSReplyTo is set	By default, Camel discards the JMSReplyTo destination and clears the JMSReplyTo header before sending the message. Camel then sends the message and does not expect a reply. Camel logs this in the log at WARN level (changed to DEBUG level from Camel 2.6 onwards. You can use preserveMessageQos=true to instruct Camel to keep the JMSReplyTo . In all situations the JmsProducer does not expect any reply and thus continue after sending the message.

58.10.2. JmsConsumer

The **JmsConsumer** behaves as follows, depending on configuration:

Exchange Pattern	Other options	Description
<i>InOut</i>	-	Camel will send the reply back to the JMSReplyTo queue.
<i>InOnly</i>	-	Camel will not send a reply back, as the pattern is <i>InOnly</i> .
-	disableReplyTo=true	This option suppresses replies.

So pay attention to the message exchange pattern set on your exchanges.

If you send a message to a JMS destination in the middle of your route you can specify the exchange pattern to use, see more at [Request Reply](#).

This is useful if you want to send an **InOnly** message to a JMS topic:

```
from("activemq:queue:in")
  .to("bean:validateOrder")
  .to(ExchangePattern.InOnly, "activemq:topic:order")
  .to("bean:handleOrder");
```

58.11. REUSE ENDPOINT AND SEND TO DIFFERENT DESTINATIONS COMPUTED AT RUNTIME

If you need to send messages to a lot of different JMS destinations, it makes sense to reuse a JMS endpoint and specify the real destination in a message header. This allows Camel to reuse the same endpoint, but send to different destinations. This greatly reduces the number of endpoints created and economizes on memory and thread resources.

You can specify the destination in the following headers:

Header	Type	Description
CamelJmsDestination	javax.jms.Destination	A destination object.
CamelJmsDestinationName	String	The destination name.

For example, the following route shows how you can compute a destination at run time and use it to override the destination appearing in the JMS URL:

```
from("file://inbox")
  .to("bean:computeDestination")
  .to("activemq:queue:dummy");
```

The queue name, **dummy**, is just a placeholder. It must be provided as part of the JMS endpoint URL, but it will be ignored in this example.

In the **computeDestination** bean, specify the real destination by setting the **CamelJmsDestinationName** header as follows:

```
public void setJmsHeader(Exchange exchange) {
    String id = ....
    exchange.getIn().setHeader("CamelJmsDestinationName", "order:" + id);
}
```

Then Camel will read this header and use it as the destination instead of the one configured on the endpoint. So, in this example Camel sends the message to **activemq:queue:order:2**, assuming the **id** value was 2.

If both the **CamelJmsDestination** and the **CamelJmsDestinationName** headers are set, **CamelJmsDestination** takes priority. Keep in mind that the JMS producer removes both

CamelJmsDestination and **CamelJmsDestinationName** headers from the exchange and do not propagate them to the created JMS message in order to avoid the accidental loops in the routes (in scenarios when the message will be forwarded to the another JMS endpoint).

58.12. CONFIGURING DIFFERENT JMS PROVIDERS

You can configure your JMS provider in Spring XML as follows:

Basically, you can configure as many JMS component instances as you wish and give them a **unique name using the id attribute**. The preceding example configures an **activemq** component. You could do the same to configure MQSeries, TibCo, BEA, Sonic and so on.

Once you have a named JMS component, you can then refer to endpoints within that component using URIs. For example for the component name, **activemq**, you can then refer to destinations using the URI format, **activemq:[queue:]topic:]destinationName**. You can use the same approach for all other JMS providers.

This works by the SpringCamelContext lazily fetching components from the spring context for the scheme name you use for Endpoint URIs and having the Component resolve the endpoint URIs.

58.12.1. Using JNDI to find the ConnectionFactory

If you are using a J2EE container, you might need to look up JNDI to find the JMS **ConnectionFactory** rather than use the usual **<bean>** mechanism in Spring. You can do this using Spring's factory bean or the new Spring XML namespace. For example:

```
<bean id="weblogic" class="org.apache.camel.component.jms.JmsComponent">
  <property name="connectionFactory" ref="myConnectionFactory"/>
</bean>

<jee:jndi-lookup id="myConnectionFactory" jndi-name="jms/connectionFactory"/>
```

See [The jee schema](#) in the Spring reference documentation for more details about JNDI lookup.

58.13. CONCURRENT CONSUMING

A common requirement with JMS is to consume messages concurrently in multiple threads in order to make an application more responsive. You can set the **concurrentConsumers** option to specify the number of threads servicing the JMS endpoint, as follows:

```
from("jms:SomeQueue?concurrentConsumers=20").
  bean(MyClass.class);
```

You can configure this option in one of the following ways:

- On the **JmsComponent**,
- On the endpoint URI or,
- By invoking **setConcurrentConsumers()** directly on the **JmsEndpoint**.

58.13.1. Concurrent Consuming with async consumer

Notice that each concurrent consumer will only pickup the next available message from the JMS broker, when the current message has been fully processed. You can set the option **asyncConsumer=true** to let the consumer pickup the next message from the JMS queue, while the previous message is being processed asynchronously (by the Asynchronous Routing Engine). See more details in the table on top of the page about the **asyncConsumer** option.

```
from("jms:SomeQueue?concurrentConsumers=20&asyncConsumer=true").
    bean(MyClass.class);
```

58.14. REQUEST/REPLY OVER JMS

Camel supports request/reply over JMS. In essence the MEP of the Exchange should be **InOut** when you send a message to a JMS queue.

Camel offers a number of options to configure request/reply over JMS that influence performance and clustered environments. The table below summaries the options.

Option	Performance	Cluster	Description
Temporary	Fast	Yes	A temporary queue is used as reply queue, and automatic created by Camel. To use this do not specify a replyTo queue name. And you can optionally configure replyToType=Temporary to make it stand out that temporary queues are in use.
Shared	Slow	Yes	A shared persistent queue is used as reply queue. The queue must be created beforehand, although some brokers can create them on the fly such as Apache ActiveMQ. To use this you must specify the replyTo queue name. And you can optionally configure replyToType=Shared to make it stand out that shared queues are in use. A shared queue can be used in a clustered environment with multiple nodes running this Camel application at the same time. All using the same shared reply queue. This is possible because JMS Message selectors are used to correlate expected reply messages; this impacts performance though. JMS Message selectors is slower, and therefore not as fast as Temporary or Exclusive queues. See further below how to tweak this for better performance.

Option	Performance	Cluster	Description
Exclusive	Fast	No (*Yes)	An exclusive persistent queue is used as reply queue. The queue must be created beforehand, although some brokers can create them on the fly such as Apache ActiveMQ. To use this you must specify the <code>replyTo</code> queue name. And you must configure <code>replyToType=Exclusive</code> to instruct Camel to use exclusive queues, as Shared is used by default, if a <code>replyTo</code> queue name was configured. When using exclusive reply queues, then JMS Message selectors are not in use, and therefore other applications must not use this queue as well. An exclusive queue cannot be used in a clustered environment with multiple nodes running this Camel application at the same time; as we do not have control if the reply queue comes back to the same node that sent the request message; that is why shared queues use JMS Message selectors to make sure of this. Though if you configure each Exclusive reply queue with an unique name per node, then you can run this in a clustered environment. As then the reply message will be sent back to that queue for the given node, that awaits the reply message.
<code>replyToConcurrentConsumers</code>	Fast	Yes	Allows to process reply messages concurrently using concurrent message listeners in use. You can specify a range using the <code>replyToConcurrentConsumers</code> and <code>replyToMaxConcurrentConsumers</code> options. Notice: That using Shared reply queues may not work as well with concurrent listeners, so use this option with care.
<code>replyToMaxConcurrentConsumers</code>	Fast	Yes	Allows to process reply messages concurrently using concurrent message listeners in use. You can specify a range using the <code>replyToConcurrentConsumers</code> and <code>replyToMaxConcurrentConsumers</code> options. Notice: That using Shared reply queues may not work as well with concurrent listeners, so use this option with care.

The **JmsProducer** detects the **InOut** and provides a **JMSReplyTo** header with the reply destination to be used. By default Camel uses a temporary queue, but you can use the **`replyTo`** option on the endpoint to specify a fixed reply queue (see more below about fixed reply queue).

Camel will automatically setup a consumer which listen on the reply queue, so you should **not** do anything. This consumer is a Spring **DefaultMessageListenerContainer** which listen for replies. However it's fixed to 1 concurrent consumer. That means replies will be processed in sequence as there are only 1 thread to process the replies. You can configure the listener to use concurrent threads using the **`replyToConcurrentConsumers`** and **`replyToMaxConcurrentConsumers`** options. This allows you to easier configure this in Camel as shown below:

```
from(xxx)
.inOut().to("activemq:queue:foo?replyToConcurrentConsumers=5")
.to(yyy)
.to(zzz);
```

In this route we instruct Camel to route replies asynchronously using a thread pool with 5 threads.

58.14.1. Request/reply over JMS and using a shared fixed reply queue

You can use a fixed reply queue when doing request/reply over JMS as shown in the example below.

```
from(xxx)
.inOut().to("activemq:queue:foo?replyTo=bar")
.to(yyy)
```

In this example the fixed reply queue named "bar" is used. By default Camel assumes the queue is shared when using fixed reply queues, and therefore it uses a **JMSSelector** to only pickup the expected reply messages (eg based on the **JMSCorrelationID**). See next section for exclusive fixed reply queues. That means its not as fast as temporary queues. You can speedup how often Camel will pull for reply messages using the **receiveTimeout** option. By default its 1000 millis. So to make it faster you can set it to 250 millis to pull 4 times per second as shown:

```
from(xxx)
.inOut().to("activemq:queue:foo?replyTo=bar&receiveTimeout=250")
.to(yyy)
```

Notice this will cause the Camel to send pull requests to the message broker more frequent, and thus require more network traffic.

It is generally recommended to use temporary queues if possible.

58.14.2. Request/reply over JMS and using an exclusive fixed reply queue

In the previous example, Camel would anticipate the fixed reply queue named "bar" was shared, and thus it uses a **JMSSelector** to only consume reply messages which it expects. However there is a drawback doing this as the JMS selector is slower. Also the consumer on the reply queue is slower to update with new JMS selector ids. In fact it only updates when the **receiveTimeout** option times out, which by default is 1 second. So in theory the reply messages could take up till about 1 sec to be detected. On the other hand if the fixed reply queue is exclusive to the Camel reply consumer, then we can avoid using the JMS selectors, and thus be more performant. In fact as fast as using temporary queues. There is the **ReplyToType** option which you can configure to **Exclusive** to tell Camel that the reply queue is exclusive as shown in the example below:

```
from(xxx)
.inOut().to("activemq:queue:foo?replyTo=bar&replyToType=Exclusive")
.to(yyy)
```

Mind that the queue must be exclusive to each and every endpoint. So if you have two routes, then they each need an unique reply queue as shown in the next example:

```
from(xxx)
.inOut().to("activemq:queue:foo?replyTo=bar&replyToType=Exclusive")
.to(yyy)
```



```
from(aaa)
  .inOut().to("activemq:queue:order?replyTo=order.reply&replyToType=Exclusive")
  .to(bbb)
```

The same applies if you run in a clustered environment. Then each node in the cluster must use an unique reply queue name. As otherwise each node in the cluster may pickup messages which was intended as a reply on another node. For clustered environments its recommended to use shared reply queues instead.

58.15. SYNCHRONIZING CLOCKS BETWEEN SENDERS AND RECEIVERS

When doing messaging between systems, its desirable that the systems have synchronized clocks. For example when sending a [JMS](#) message, then you can set a time to live value on the message. Then the receiver can inspect this value, and determine if the message is already expired, and thus drop the message instead of consume and process it. However this requires that both sender and receiver have synchronized clocks. If you are using [ActiveMQ](#) then you can use the [timestamp plugin](#) to synchronize clocks.

58.16. ABOUT TIME TO LIVE

Read first above about synchronized clocks.

When you do request/reply (InOut) over [JMS](#) with Camel then Camel uses a timeout on the sender side, which is default 20 seconds from the **requestTimeout** option. You can control this by setting a higher/lower value. However the time to live value is still set on the message being send. So that requires the clocks to be synchronized between the systems. If they are not, then you may want to disable the time to live value being set. This is now possible using the **disableTimeToLive** option from **Camel 2.8** onwards. So if you set this option to **disableTimeToLive=true**, then Camel does **not** set any time to live value when sending [JMS](#) messages. **But** the request timeout is still active. So for example if you do request/reply over [JMS](#) and have disabled time to live, then Camel will still use a timeout by 20 seconds (the **requestTimeout** option). That option can of course also be configured. So the two options **requestTimeout** and **disableTimeToLive** gives you fine grained control when doing request/reply.

You can provide a header in the message to override and use as the request timeout value instead of the endpoint configured value. For example:

```
from("direct:someWhere")
  .to("jms:queue:foo?replyTo=bar&requestTimeout=30s")
  .to("bean:processReply");
```

In the route above we have a endpoint configured **requestTimeout** of 30 seconds. So Camel will wait up till 30 seconds for that reply message to come back on the bar queue. If no reply message is received then a **org.apache.camel.ExchangeTimedOutException** is set on the Exchange and Camel continues routing the message, which would then fail due the exception, and Camel's error handler reacts.

If you want to use a per message timeout value, you can set the header with key **org.apache.camel.component.jms.JmsConstants#JMS_REQUEST_TIMEOUT** which has constant value **"CamelJmsRequestTimeout"** with a timeout value as long type.

For example we can use a bean to compute the timeout value per individual message, such as calling the **"whatIsTheTimeout"** method on the service bean as shown below:

```
from("direct:someWhere")
```

```
.setHeader("CamelJmsRequestTimeout", method(ServiceBean.class, "whatIsTheTimeout"))
.to("jms:queue:foo?replyTo=bar&requestTimeout=30s")
.to("bean:processReply");
```

When you do fire and forget (InOut) over [JMS](#) with Camel then Camel by default does **not** set any time to live value on the message. You can configure a value by using the **timeToLive** option. For example to indicate a 5 sec., you set **timeToLive=5000**. The option **disableTimeToLive** can be used to force disabling the time to live, also for InOnly messaging. The **requestTimeout** option is not being used for InOnly messaging.

58.17. ENABLING TRANSACTED CONSUMPTION

A common requirement is to consume from a queue in a transaction and then process the message using the Camel route. To do this, just ensure that you set the following properties on the component/endpoint:

- **transacted** = true
- **transactionManager** = a *Transaction Manager* – typically the **JmsTransactionManager**

See the Transactional Client EIP pattern for further details.

Transactions and [Request Reply] over JMS

When using Request Reply over JMS you cannot use a single transaction; JMS will not send any messages until a commit is performed, so the server side won't receive anything at all until the transaction commits. Therefore to use [Request Reply](#) you must commit a transaction after sending the request and then use a separate transaction for receiving the response.

To address this issue the JMS component uses different properties to specify transaction use for oneway messaging and request reply messaging:

The **transacted** property applies **only** to the InOnly message Exchange Pattern (MEP).

You can leverage the [DMLC transacted session API](#) using the following properties on component/endpoint:

- **transacted** = true
- **lazyCreateTransactionManager** = false

The benefit of doing so is that the `cacheLevel` setting will be honored when using local transactions without a configured `TransactionManager`. When a `TransactionManager` is configured, no caching happens at DMLC level and it is necessary to rely on a pooled connection factory. For more details about this kind of setup, see [here](#) and [here](#).

58.18. USING JMSREPLYTO FOR LATE REPLIES

When using Camel as a JMS listener, it sets an Exchange property with the value of the `ReplyTo` **javax.jms.Destination** object, having the key **ReplyTo**. You can obtain this **Destination** as follows:

```
Destination replyDestination =
exchange.getIn().getHeader(JmsConstants.JMS_REPLY_DESTINATION, Destination.class);
```

And then later use it to send a reply using regular JMS or Camel.

```
// we need to pass in the JMS component, and in this sample we use ActiveMQ
JmsEndpoint endpoint = JmsEndpoint.newInstance(replyDestination, activeMQComponent);
// now we have the endpoint we can use regular Camel API to send a message to it
template.sendBody(endpoint, "Here is the late reply.");
```

A different solution to sending a reply is to provide the **replyDestination** object in the same Exchange property when sending. Camel will then pick up this property and use it for the real destination. The endpoint URI must include a dummy destination, however. For example:

```
// we pretend to send it to some non existing dummy queue
template.send("activemq:queue:dummy, new Processor() {
    public void process(Exchange exchange) throws Exception {
        // and here we override the destination with the ReplyTo destination object so the message is sent
        // to there instead of dummy
        exchange.getIn().setHeader(JmsConstants.JMS_DESTINATION, replyDestination);
        exchange.getIn().setBody("Here is the late reply.");
    }
}
```

58.19. USING A REQUEST TIMEOUT

In the sample below we send a Request Reply style message Exchange (we use the **requestBody** method = **InOut**) to the slow queue for further processing in Camel and we wait for a return reply:

58.20. SENDING AN INONLY MESSAGE AND KEEPING THE JMSREPLYTO HEADER

When sending to a [JMS](#) destination using **camel-jms** the producer will use the MEP to detect if its *InOnly* or *InOut* messaging. However there can be times where you want to send an *InOnly* message but keeping the **JMSReplyTo** header. To do so you have to instruct Camel to keep it, otherwise the **JMSReplyTo** header will be dropped.

For example to send an *InOnly* message to the foo queue, but with a **JMSReplyTo** with bar queue you can do as follows:

```
template.send("activemq:queue:foo?preserveMessageQos=true", new Processor() {
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setBody("World");
        exchange.getIn().setHeader("JMSReplyTo", "bar");
    }
});
```

Notice we use **preserveMessageQos=true** to instruct Camel to keep the **JMSReplyTo** header.

58.21. SETTING JMS PROVIDER OPTIONS ON THE DESTINATION

Some JMS providers, like IBM's WebSphere MQ need options to be set on the JMS destination. For example, you may need to specify the **targetClient** option. Since **targetClient** is a WebSphere MQ option and not a Camel URI option, you need to set that on the JMS destination name like so:

```
// ...
.setHeader("CamelJmsDestinationName", constant("queue:///MY_QUEUE?targetClient=1"))
.to("wmq:queue:MY_QUEUE?useMessageIDAsCorrelationID=true");
```

Some versions of WMQ won't accept this option on the destination name and you will get an exception like:

```
com.ibm.msg.client.jms.DetailedJMSEException: JMSCC0005: The specified
value 'MY_QUEUE?targetClient=1' is not allowed for
'XMSC_DESTINATION_NAME'
```

A workaround is to use a custom DestinationResolver:

```
JmsComponent wmq = new JmsComponent(connectionFactory);

wmq.setDestinationResolver(new DestinationResolver() {
    public Destination resolveDestinationName(Session session, String destinationName, boolean
    pubSubDomain) throws JMSEException {
        MQQueueSession wmqSession = (MQQueueSession) session;
        return wmqSession.createQueue("queue:/" + destinationName + "?targetClient=1");
    }
});
```

58.22. SPRING BOOT AUTO-CONFIGURATION

The component supports 99 options, which are listed below.

Name	Description	Default	Type
camel.component.jms.accept-messages-while-stopping	Specifies whether the consumer accept messages while it is stopping. You may consider enabling this option, if you start and stop JMS routes at runtime, while there are still messages enqueued on the queue. If this option is false, and you stop the JMS route, then messages may be rejected, and the JMS broker would have to attempt redeliveries, which yet again may be rejected, and eventually the message may be moved at a dead letter queue on the JMS broker. To avoid this its recommended to enable this option.	false	Boolean
camel.component.jms.acknowledgment-mode-name	The JMS acknowledgement name, which is one of: SESSION_TRANSACTED, CLIENT_ACKNOWLEDGE, AUTO_ACKNOWLEDGE, DUPS_OK_ACKNOWLEDGE.	AUTO_ACKNOWLEDGE	String

Name	Description	Default	Type
camel.component.jms.allow-additional-headers	This option is used to allow additional headers which may have values that are invalid according to JMS specification. For example some message systems such as WMQ do this with header names using prefix JMS_IBM_MQMD_ containing values with byte array or other invalid types. You can specify multiple header names separated by comma, and use as suffix for wildcard matching.		String
camel.component.jms.allow-auto-wired-connection-factory	Whether to auto-discover ConnectionFactory from the registry, if no connection factory has been configured. If only one instance of ConnectionFactory is found then it will be used. This is enabled by default.	true	Boolean
camel.component.jms.allow-auto-wired-destination-resolver	Whether to auto-discover DestinationResolver from the registry, if no destination resolver has been configured. If only one instance of DestinationResolver is found then it will be used. This is enabled by default.	true	Boolean
camel.component.jms.allow-null-body	Whether to allow sending messages with no body. If this option is false and the message body is null, then an JMSEException is thrown.	true	Boolean
camel.component.jms.allow-reply-manager-quick-stop	Whether the DefaultMessageListenerContainer used in the reply managers for request/reply messaging allow the DefaultMessageListenerContainer.runningAllowed flag to quick stop in case JmsConfiguration#isAcceptMessagesWhileStopping is enabled, and org.apache.camel.CamelContext is currently being stopped. This quick stop ability is enabled by default in the regular JMS consumers but to enable for reply managers you must enable this flag.	false	Boolean
camel.component.jms.allow-serialized-headers	Controls whether or not to include serialized headers. Applies only when transferExchange is true. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	Boolean

Name	Description	Default	Type
camel.component.jms.always-copy-message	If true, Camel will always make a JMS message copy of the message when it is passed to the producer for sending. Copying the message is needed in some situations, such as when a <code>replyToDestinationSelectorName</code> is set (incidentally, Camel will set the <code>alwaysCopyMessage</code> option to true, if a <code>replyToDestinationSelectorName</code> is set).	false	Boolean
camel.component.jms.artemis-consumer-priority	Consumer priorities allow you to ensure that high priority consumers receive messages while they are active. Normally, active consumers connected to a queue receive messages from it in a round-robin fashion. When consumer priorities are in use, messages are delivered round-robin if multiple active consumers exist with the same high priority. Messages will only going to lower priority consumers when the high priority consumers do not have credit available to consume the message, or those high priority consumers have declined to accept the message (for instance because it does not meet the criteria of any selectors associated with the consumer).		Integer
camel.component.jms.artemis-streaming-enabled	Whether optimizing for Apache Artemis streaming mode. This can reduce memory overhead when using Artemis with JMS <code>StreamMessage</code> types. This option must only be enabled if Apache Artemis is being used.	false	Boolean
camel.component.jms.async-consumer	Whether the <code>JmsConsumer</code> processes the Exchange asynchronously. If enabled then the <code>JmsConsumer</code> may pickup the next message from the JMS queue, while the previous message is being processed asynchronously (by the Asynchronous Routing Engine). This means that messages may be processed not 100% strictly in order. If disabled (as default) then the Exchange is fully processed before the <code>JmsConsumer</code> picks up the next message from the JMS queue. Note if <code>transacted</code> has been enabled, then <code>asyncConsumer=true</code> does not run asynchronously, as transaction must be executed synchronously (Camel 3.0 may support async transactions).	false	Boolean

Name	Description	Default	Type
camel.component.jms.async-start-listener	Whether to startup the JmsConsumer message listener asynchronously, when starting a route. For example if a JmsConsumer cannot get a connection to a remote JMS broker, then it may block while retrying and/or failover. This will cause Camel to block while starting routes. By setting this option to true, you will let routes startup, while the JmsConsumer connects to the JMS broker using a dedicated thread in asynchronous mode. If this option is used, then beware that if the connection could not be established, then an exception is logged at WARN level, and the consumer will not be able to receive messages; You can then restart the route to retry.	false	Boolean
camel.component.jms.async-stop-listener	Whether to stop the JmsConsumer message listener asynchronously, when stopping a route.	false	Boolean
camel.component.jms.auto-startup	Specifies whether the consumer container should auto-startup.	true	Boolean
camel.component.jms.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.jms.cache-level	Sets the cache level by ID for the underlying JMS resources. See cacheLevelName option for more details.		Integer
camel.component.jms.cache-level-name	Sets the cache level by name for the underlying JMS resources. Possible values are: CACHE_AUTO, CACHE_CONNECTION, CACHE_CONSUMER, CACHE_NONE, and CACHE_SESSION. The default setting is CACHE_AUTO. See the Spring documentation and Transactions Cache Levels for more information.	CACHE_AUTO	String
camel.component.jms.client-id	Sets the JMS client ID to use. Note that this value, if specified, must be unique and can only be used by a single JMS connection instance. It is typically only required for durable topic subscriptions with JMS 1.1		String

Name	Description	Default	Type
camel.component.jms.concurrent-consumers	Specifies the default number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option <code>replyToConcurrentConsumers</code> is used to control number of concurrent consumers on the reply message listener.	1	Integer
camel.component.jms.configuration	To use a shared JMS configuration. The option is a <code>org.apache.camel.component.jms.JmsConfiguration</code> type.		JmsConfiguration
camel.component.jms.connection-factory	The connection factory to be use. A connection factory must be configured either on the component or endpoint. The option is a <code>javax.jms.ConnectionFactory</code> type.		ConnectionFactory
camel.component.jms.consumer-type	The consumer type to use, which can be one of: Simple, Default, or Custom. The consumer type determines which Spring JMS listener to use. Default will use <code>org.springframework.jms.listener.DefaultMessageListenerContainer</code> , Simple will use <code>org.springframework.jms.listener.SimpleMessageListenerContainer</code> . When Custom is specified, the <code>MessageListenerContainerFactory</code> defined by the <code>messageListenerContainerFactory</code> option will determine what <code>org.springframework.jms.listener.AbstractMessageListenerContainer</code> to use.		ConsumerType
camel.component.jms.correlation-property	When using InOut exchange pattern use this JMS property instead of <code>JMSCorrelationID</code> JMS property to correlate messages. If set messages will be correlated solely on the value of this property <code>JMSCorrelationID</code> property will be ignored and not set by Camel.		String

Name	Description	Default	Type
camel.component.jms.default-task-executor-type	Specifies what default TaskExecutor type to use in the DefaultMessageListenerContainer, for both consumer endpoints and the ReplyTo consumer of producer endpoints. Possible values: SimpleAsync (uses Spring's SimpleAsyncTaskExecutor) or ThreadPool (uses Spring's ThreadPoolTaskExecutor with optimal values - cached threadpool-like). If not set, it defaults to the previous behaviour, which uses a cached thread pool for consumer endpoints and SimpleAsync for reply consumers. The use of ThreadPool is recommended to reduce thread trash in elastic configurations with dynamically increasing and decreasing concurrent consumers.		DefaultTaskExecutorType
camel.component.jms.delivery-delay	Sets delivery delay to use for send calls for JMS. This option requires JMS 2.0 compliant broker.	-1	Long
camel.component.jms.delivery-mode	Specifies the delivery mode to be used. Possible values are those defined by javax.jms.DeliveryMode. NON_PERSISTENT = 1 and PERSISTENT = 2.		Integer
camel.component.jms.delivery-persistent	Specifies whether persistent delivery is used by default.	true	Boolean
camel.component.jms.destination-resolver	A pluggable org.springframework.jms.support.destination.DestinationResolver that allows you to use your own resolver (for example, to lookup the real destination in a JNDI registry). The option is a org.springframework.jms.support.destination.DestinationResolver type.		DestinationResolver
camel.component.jms.disable-reply-to	Specifies whether Camel ignores the JMSReplyTo header in messages. If true, Camel does not send a reply back to the destination specified in the JMSReplyTo header. You can use this option if you want Camel to consume from a route and you do not want Camel to automatically send back a reply message because another component in your code handles the reply message. You can also use this option if you want to use Camel as a proxy between different message brokers and you want to route message from one system to another.	false	Boolean

Name	Description	Default	Type
camel.component.jms.disable-time-to-live	Use this option to force disabling time to live. For example when you do request/reply over JMS, then Camel will by default use the requestTimeout value as time to live on the message being sent. The problem is that the sender and receiver systems have to have their clocks synchronized, so they are in sync. This is not always so easy to archive. So you can use disableTimeToLive=true to not set a time to live value on the sent message. Then the message will not expire on the receiver system. See below in section About time to live for more details.	false	Boolean
camel.component.jms.durable-subscription-name	The durable subscriber name for specifying durable topic subscriptions. The clientId option must be configured as well.		String
camel.component.jms.eager-loading-of-properties	Enables eager loading of JMS properties and payload as soon as a message is loaded which generally is inefficient as the JMS properties may not be required but sometimes can catch early any issues with the underlying JMS provider and the use of JMS properties. See also the option eagerPoisonBody.	false	Boolean
camel.component.jms.eager-poison-body	If eagerLoadingOfProperties is enabled and the JMS message payload (JMS body or JMS properties) is poison (cannot be read/mapped), then set this text as the message body instead so the message can be processed (the cause of the poison are already stored as exception on the Exchange). This can be turned off by setting eagerPoisonBody=false. See also the option eagerLoadingOfProperties.	Poison JMS message due to \${exception.message}	String
camel.component.jms.enabled	Whether to enable auto configuration of the jms component. This is enabled by default.		Boolean
camel.component.jms.error-handler	Specifies a org.springframework.util.ErrorHandler to be invoked in case of any uncaught exceptions thrown while processing a Message. By default these exceptions will be logged at the WARN level, if no errorHandler has been configured. You can configure logging level and whether stack traces should be logged using errorHandlerLogLevel and errorHandlerLogStackTrace options. This makes it much easier to configure, than having to code a custom errorHandler. The option is a org.springframework.util.ErrorHandler type.		ErrorHandler

Name	Description	Default	Type
<code>camel.component.jms.error-handler-log-stack-trace</code>	Allows to control whether stacktraces should be logged or not, by the default errorHandler.	true	Boolean
<code>camel.component.jms.error-handler-logging-level</code>	Allows to configure the default errorHandler logging level for logging uncaught exceptions.		LogLevel
<code>camel.component.jms.exception-listener</code>	Specifies the JMS Exception Listener that is to be notified of any underlying JMS exceptions. The option is a <code>javax.jms.ExceptionListener</code> type.		ExceptionListener
<code>camel.component.jms.explicit-qos-enabled</code>	Set if the deliveryMode, priority or timeToLive qualities of service should be used when sending messages. This option is based on Spring's <code>JmsTemplate</code> . The deliveryMode, priority and timeToLive options are applied to the current endpoint. This contrasts with the <code>preserveMessageQos</code> option, which operates at message granularity, reading QoS properties exclusively from the Camel In message headers.	false	Boolean
<code>camel.component.jms.expose-listener-session</code>	Specifies whether the listener session should be exposed when consuming messages.	false	Boolean
<code>camel.component.jms.force-send-original-message</code>	When using <code>mapJmsMessage=false</code> Camel will create a new JMS message to send to a new JMS destination if you touch the headers (get or set) during the route. Set this option to true to force Camel to send the original JMS message that was received.	false	Boolean
<code>camel.component.jms.format-date-headers-to-iso8601</code>	Sets whether JMS date properties should be formatted according to the ISO 8601 standard.	false	Boolean
<code>camel.component.jms.header-filter-strategy</code>	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		HeaderFilterStrategy
<code>camel.component.jms.idle-consumer-limit</code>	Specify the limit for the number of consumers that are allowed to be idle at any given time.	1	Integer

Name	Description	Default	Type
camel.component.jms.idle-task-execution-limit	Specifies the limit for idle executions of a receive task, not having received any message within its execution. If this limit is reached, the task will shut down and leave receiving to other executing tasks (in the case of dynamic scheduling; see the <code>maxConcurrentConsumers</code> setting). There is additional doc available from Spring.	1	Integer
camel.component.jms.include-all-jms-x-properties	Whether to include all JMSXxxx properties when mapping from JMS to Camel Message. Setting this to true will include properties such as JMSXAppID, and JMSXUserID etc. Note: If you are using a custom <code>headerFilterStrategy</code> then this option does not apply.	false	Boolean
camel.component.jms.include-sent-jms-message-id	Only applicable when sending to JMS destination using <code>InOnly</code> (eg fire and forget). Enabling this option will enrich the Camel Exchange with the actual <code>JMSMessageID</code> that was used by the JMS client when the message was sent to the JMS destination.	false	Boolean
camel.component.jms.jms-key-format-strategy	Pluggable strategy for encoding and decoding JMS keys so they can be compliant with the JMS specification. Camel provides two implementations out of the box: <code>default</code> and <code>passthrough</code> . The default strategy will safely marshal dots and hyphens (<code>.</code> and <code>-</code>). The <code>passthrough</code> strategy leaves the key as is. Can be used for JMS brokers which do not care whether JMS header keys contain illegal characters. You can provide your own implementation of the <code>org.apache.camel.component.jms.JmsKeyFormatStrategy</code> and refer to it using the <code>#</code> notation.		<code>JmsKeyFormatStrategy</code>
camel.component.jms.jms-message-type	Allows you to force the use of a specific <code>javax.jms.Message</code> implementation for sending JMS messages. Possible values are: <code>Bytes</code> , <code>Map</code> , <code>Object</code> , <code>Stream</code> , <code>Text</code> . By default, Camel would determine which JMS message type to use from the <code>In</code> body type. This option allows you to specify it.		<code>JmsMessageType</code>
camel.component.jms.lazy-create-transaction-manager	If true, Camel will create a <code>JmsTransactionManager</code> , if there is no <code>transactionManager</code> injected when option <code>transacted=true</code> .	true	Boolean

Name	Description	Default	Type
camel.component.jms.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.jms.map-jms-message	Specifies whether Camel should auto map the received JMS message to a suited payload type, such as javax.jms.TextMessage to a String etc.	true	Boolean
camel.component.jms.max-concurrent-consumers	Specifies the maximum number of concurrent consumers when consuming from JMS (not for request/reply over JMS). See also the maxMessagesPerTask option to control dynamic scaling up/down of threads. When doing request/reply over JMS then the option replyToMaxConcurrentConsumers is used to control number of concurrent consumers on the reply message listener.		Integer
camel.component.jms.max-messages-per-task	The number of messages per task. -1 is unlimited. If you use a range for concurrent consumers (eg min max), then this option can be used to set a value to eg 100 to control how fast the consumers will shrink when less work is required.	-1	Integer
camel.component.jms.message-converter	To use a custom Spring org.springframework.jms.support.converter.Message Converter so you can be in control how to map to/from a javax.jms.Message. The option is a org.springframework.jms.support.converter.Message Converter type.		MessageConverter
camel.component.jms.message-created-strategy	To use the given MessageCreatedStrategy which are invoked when Camel creates new instances of javax.jms.Message objects when Camel is sending a JMS message. The option is a org.apache.camel.component.jms.MessageCreatedStrategy type.		MessageCreatedStrategy

Name	Description	Default	Type
camel.component.jms.message-id-enabled	When sending, specifies whether message IDs should be added. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the message ID set to null; if the provider ignores the hint, the message ID must be set to its normal unique value.	true	Boolean
camel.component.jms.message-listener-container-factory	Registry ID of the MessageListenerContainerFactory used to determine what org.springframework.jms.listener.AbstractMessageListenerContainer to use to consume messages. Setting this will automatically set consumerType to Custom. The option is a org.apache.camel.component.jms.MessageListenerContainerFactory type.		MessageListenerContainerFactory
camel.component.jms.message-timestamp-enabled	Specifies whether timestamps should be enabled by default on sending messages. This is just an hint to the JMS broker. If the JMS provider accepts this hint, these messages must have the timestamp set to zero; if the provider ignores the hint the timestamp must be set to its normal value.	true	Boolean
camel.component.jms.password	Password to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String
camel.component.jms.preserve-message-qos	Set to true, if you want to send message using the QoS settings specified on the message, instead of the QoS settings on the JMS endpoint. The following three headers are considered JMSPriority, JMSDeliveryMode, and JMSExpiration. You can provide all or only some of them. If not provided, Camel will fall back to use the values from the endpoint instead. So, when using this option, the headers override the values from the endpoint. The explicitQosEnabled option, by contrast, will only use options set on the endpoint, and not values from the message header.	false	Boolean
camel.component.jms.priority	Values greater than 1 specify the message priority when sending (where 1 is the lowest priority and 9 is the highest). The explicitQosEnabled option must also be enabled in order for this option to have any effect.	4	Integer
camel.component.jms.pub-sub-no-local	Specifies whether to inhibit the delivery of messages published by its own connection.	false	Boolean

Name	Description	Default	Type
camel.component.jms.queue-browse-strategy	To use a custom QueueBrowseStrategy when browsing queues. The option is a <code>org.apache.camel.component.jms.QueueBrowseStrategy</code> type.		QueueBrowseStrategy
camel.component.jms.receive-timeout	The timeout for receiving messages (in milliseconds). The option is a long type.	1000	Long
camel.component.jms.recovery-interval	Specifies the interval between recovery attempts, i.e. when a connection is being refreshed, in milliseconds. The default is 5000 ms, that is, 5 seconds. The option is a long type.	5000	Long
camel.component.jms.reply-to	Provides an explicit ReplyTo destination (overrides any incoming value of <code>Message.getJMSReplyTo()</code> in consumer).		String
camel.component.jms.reply-to-cache-level-name	Sets the cache level by name for the reply consumer when doing request/reply over JMS. This option only applies when using fixed reply queues (not temporary). Camel will by default use: <code>CACHE_CONSUMER</code> for exclusive or shared w/ <code>replyToSelectorName</code> . And <code>CACHE_SESSION</code> for shared without <code>replyToSelectorName</code> . Some JMS brokers such as IBM WebSphere may require to set the <code>replyToCacheLevelName=CACHE_NONE</code> to work. Note: If using temporary queues then <code>CACHE_NONE</code> is not allowed, and you must use a higher value such as <code>CACHE_CONSUMER</code> or <code>CACHE_SESSION</code> .		String
camel.component.jms.reply-to-concurrent-consumers	Specifies the default number of concurrent consumers when doing request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.	1	Integer
camel.component.jms.reply-to-delivery-persistent	Specifies whether to use persistent delivery by default for replies.	true	Boolean
camel.component.jms.reply-to-destination-selector-name	Sets the JMS Selector using the fixed name to be used so you can filter out your own replies from the others when using a shared queue (that is, if you are not using a temporary reply queue).		String

Name	Description	Default	Type
camel.component.jms.reply-to-max-concurrent-consumers	Specifies the maximum number of concurrent consumers when using request/reply over JMS. See also the <code>maxMessagesPerTask</code> option to control dynamic scaling up/down of threads.		Integer
camel.component.jms.reply-to-on-timeout-max-concurrent-consumers	Specifies the maximum number of concurrent consumers for continue routing when timeout occurred when using request/reply over JMS.	1	Integer
camel.component.jms.reply-to-override	Provides an explicit <code>ReplyTo</code> destination in the JMS message, which overrides the setting of <code>replyTo</code> . It is useful if you want to forward the message to a remote Queue and receive the reply message from the <code>ReplyTo</code> destination.		String
camel.component.jms.reply-to-same-destination-allowed	Whether a JMS consumer is allowed to send a reply message to the same destination that the consumer is using to consume from. This prevents an endless loop by consuming and sending back the same message to itself.	false	Boolean
camel.component.jms.reply-to-type	Allows for explicitly specifying which kind of strategy to use for <code>replyTo</code> queues when doing request/reply over JMS. Possible values are: <code>Temporary</code> , <code>Shared</code> , or <code>Exclusive</code> . By default Camel will use temporary queues. However if <code>replyTo</code> has been configured, then <code>Shared</code> is used by default. This option allows you to use exclusive queues instead of shared ones. See Camel JMS documentation for more details, and especially the notes about the implications if running in a clustered environment, and the fact that <code>Shared</code> reply queues has lower performance than its alternatives <code>Temporary</code> and <code>Exclusive</code> .		<code>ReplyToType</code>
camel.component.jms.request-timeout	The timeout for waiting for a reply when using the <code>InOut</code> Exchange Pattern (in milliseconds). The default is 20 seconds. You can include the header <code>CamelJmsRequestTimeout</code> to override this endpoint configured timeout value, and thus have per message individual timeout values. See also the <code>requestTimeoutCheckerInterval</code> option. The option is a long type.	20000	Long

Name	Description	Default	Type
camel.component.jms.request-timeout-checker-interval	Configures how often Camel should check for timed out Exchanges when doing request/reply over JMS. By default Camel checks once per second. But if you must react faster when a timeout occurs, then you can lower this interval, to check more frequently. The timeout is determined by the option requestTimeout. The option is a long type.	1000	Long
camel.component.jms.selector	Sets the JMS selector to use.		String
camel.component.jms.stream-message-type-enabled	Sets whether StreamMessage type is enabled or not. Message payloads of streaming kind such as files, InputStream, etc will either be sent as BytesMessage or StreamMessage. This option controls which kind will be used. By default BytesMessage is used which enforces the entire message payload to be read into memory. By enabling this option the message payload is read into memory in chunks and each chunk is then written to the StreamMessage until no more data.	false	Boolean
camel.component.jms.subscription-durable	Set whether to make the subscription durable. The durable subscription name to be used can be specified through the subscriptionName property. Default is false. Set this to true to register a durable subscription, typically in combination with a subscriptionName value (unless your message listener class name is good enough as subscription name). Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the pubSubDomain flag as well.	false	Boolean
camel.component.jms.subscription-name	Set the name of a subscription to create. To be applied in case of a topic (pub-sub domain) with a shared or durable subscription. The subscription name needs to be unique within this client's JMS client id. Default is the class name of the specified message listener. Note: Only 1 concurrent consumer (which is the default of this message listener container) is allowed for each subscription, except for a shared subscription (which requires JMS 2.0).		String

Name	Description	Default	Type
camel.component.jms.subscription-shared	Set whether to make the subscription shared. The shared subscription name to be used can be specified through the subscriptionName property. Default is false. Set this to true to register a shared subscription, typically in combination with a subscriptionName value (unless your message listener class name is good enough as subscription name). Note that shared subscriptions may also be durable, so this flag can (and often will) be combined with subscriptionDurable as well. Only makes sense when listening to a topic (pub-sub domain), therefore this method switches the pubSubDomain flag as well. Requires a JMS 2.0 compatible message broker.	false	Boolean
camel.component.jms.synchronous	Sets whether synchronous processing should be strictly used.	false	Boolean
camel.component.jms.task-executor	Allows you to specify a custom task executor for consuming messages. The option is a org.springframework.core.task.TaskExecutor type.		TaskExecutor
camel.component.jms.test-connection-on-startup	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	Boolean
camel.component.jms.time-to-live	When sending messages, specifies the time-to-live of the message (in milliseconds).	-1	Long
camel.component.jms.transacted	Specifies whether to use transacted mode.	false	Boolean

Name	Description	Default	Type
camel.component.jms.transacted-in-out	Specifies whether InOut operations (request reply) default to using transacted mode. If this flag is set to true, then Spring JmsTemplate will have sessionTransacted set to true, and the acknowledgeMode as transacted on the JmsTemplate used for InOut operations. Note from Spring JMS: that within a JTA transaction, the parameters passed to createQueue, createTopic methods are not taken into account. Depending on the Java EE transaction context, the container makes its own decisions on these values. Analogously, these parameters are not taken into account within a locally managed transaction either, since Spring JMS operates on an existing JMS Session in this case. Setting this flag to true will use a short local JMS transaction when running outside of a managed transaction, and a synchronized local JMS transaction in case of a managed transaction (other than an XA transaction) being present. This has the effect of a local JMS transaction being managed alongside the main transaction (which might be a native JDBC transaction), with the JMS transaction committing right after the main transaction.	false	Boolean
camel.component.jms.transaction-manager	The Spring transaction manager to use. The option is a org.springframework.transaction.PlatformTransactionManager type.		PlatformTransactionManager
camel.component.jms.transaction-name	The name of the transaction to use.		String
camel.component.jms.transaction-timeout	The timeout value of the transaction (in seconds), if using transacted mode.	-1	Integer

Name	Description	Default	Type
camel.component.jms.transfer-exception	If enabled and you are using Request Reply messaging (InOut) and an Exchange failed on the consumer side, then the caused Exception will be send back in response as a javax.jms.ObjectMessage. If the client is Camel, the returned Exception is rethrown. This allows you to use Camel JMS as a bridge in your routing - for example, using persistent queues to enable robust routing. Notice that if you also have transferExchange enabled, this option takes precedence. The caught exception is required to be serializable. The original Exception on the consumer side can be wrapped in an outer exception such as org.apache.camel.RuntimeCamelException when returned to the producer. Use this with caution as the data is using Java Object serialization and requires the received to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumer!.	false	Boolean
camel.component.jms.transfer-exchange	You can transfer the exchange over the wire instead of just the body and headers. The following fields are transferred: In body, Out body, Fault body, In headers, Out headers, Fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level. You must enable this option on both the producer and consumer side, so Camel knows the payloads is an Exchange and not a regular payload. Use this with caution as the data is using Java Object serialization and requires the receiver to be able to deserialize the data at Class level, which forces a strong coupling between the producers and consumers having to use compatible Camel versions!.	false	Boolean
camel.component.jms.use-message-i-d-as-correlation-i-d	Specifies whether JMSMessageID should always be used as JMSCorrelationID for InOut messages.	false	Boolean
camel.component.jms.username	Username to use with the ConnectionFactory. You can also configure username/password directly on the ConnectionFactory.		String

Name	Description	Default	Type
camel.component.jms.wait-for-provision-correlation-to-be-updated-counter	Number of times to wait for provisional correlation id to be updated to the actual correlation id when doing request/reply over JMS and when the option <code>useMessageIDAsCorrelationID</code> is enabled.	50	Integer
camel.component.jms.wait-for-provision-correlation-to-be-updated-thread-sleeping-time	Interval in millis to sleep each time while waiting for provisional correlation id to be updated. The option is a long type.	100	Long

CHAPTER 59. JOLOKIA

Jolokia starter integrates the Jolokia agent configuration in Spring Boot. It wraps the [Spring Support](#) by adding the default configurations to let the application work out-of-the-box without manually declaring Jolokia servers. This starter can be considered as an alternative to the [Jolokia JVM agent](#).

59.1. DEPENDENCIES

When using **camel-jolokia** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jolokia-starter</artifactId>
</dependency>
```

This exposes the Jolokia endpoint at <http://0.0.0.0:8778/jolokia/>. The agent can be configured according to the [Jolokia Spring support](#) page using the property prefix **camel.component.jolokia**. It is possible to configure **camel.component.jolokia.lookup-config=true**.

59.2. USING JOLOKIA AGENT

It is possible to configure the Jolokia server accessing the configuration map via properties according to the [Jolokia agent configuration](#) using **camel.component.jolokia.server-config** map configuration, that is, **camel.component.jolokia.server-config.port=8779** or **camel.component.jolokia.server-config.authMode=jaas**. The value of the **discoveryEnabled** configuration key is set to **true** on JVM local environment (not on Kubernetes), to allow the Hawtio process to discover the Jolokia agent. It is possible to disable the default configuration by setting the **camel.component.jolokia.server-config.discoveryEnabled=false**.

59.3. USING JOLOKIA RESTRICTOR

To avoid exposing all the JMX MBeans (see [Security](#) considerations), it provides a default Jolokia Restrictor that allows only Camel related data and some basic information from java. It is possible to avoid using the restrictor with the property **camel.component.jolokia.use-camel-restrictor=false** or to use your own custom one with the property **camel.component.jolokia.server-config.restrictorClass=org.example.MyRestrictor**.

An example to extend the provided restrictor is shown below.

Example

```
public class MyRestrictor extends CamelRestrictor {

  //getDefaultDomains() contains default domains, if you want to add some domain to the existing list
  @Override
  protected List<String> getAllowedDomains() {
    final List<String> newDomains = new ArrayList<>(getDefaultDomains());
    newDomains.add("my.domain");
    return newDomains;
  }
}
```

59.4. IMPLEMENTING SPRING JOLOKIA CONFIGURATION

The starter creates a default configuration according to the provided properties, that covers all the configuration for the Jolokia agent/server but if you want to use your custom **SpringJolokiaConfigHolder** implementation, it is possible to declare in the Spring context a Bean named **camelConfigHolder**.

Example of camelConfigHolder

```
@Bean("camelConfigHolder")
public SpringJolokiaConfigHolder myConfigHolder() {
    final SpringJolokiaConfigHolder myConfig = new SpringJolokiaConfigHolder();
    myConfig.setConfig(Map.of("threadNr", "5", "executor", "fixed"));
    return myConfig;
}
```

In this case, the executor configuration will be taken from the custom Bean if the same properties are not defined in the **application.properties**. This behaviour is also configurable with the property **camel.component.jolokia.config-from-properties-first=false**. It means that the configuration, in case the key is present in both **application.properties** and Bean implementation, uses the one from Bean. If the keys from properties and beans are not overridden each other, the simple merge will be applied. This way, the configuration properties allow to cover all the configurable aspect of the Jolokia server.

59.5. USING CAMEL LOG HANDLER HOLDER CONFIGURATION

There is a log configuration also provided in the starter that uses the slf4j implementation. You can change this configuration by providing a Bean named **camelLogHandlerHolder**.

Example of camelLogHandlerHolder

```
@Bean
@ConditionalOnMissingBean(name = "camelLogHandlerHolder")
public SpringJolokiaLogHandlerHolder myLogHandlerHolder() {
    final SpringJolokiaLogHandlerHolder stdoutHandlerHolder = new SpringJolokiaLogHandlerHolder();
    stdoutHandlerHolder.setType("stdout");
    return stdoutHandlerHolder;
}
```

The category for the Camel Spring Boot starter is **org.apache.camel.component.jolokia** while it is possible to configure level of the core Jolokia server with **org.jolokia**.

Example in application.properties

```
logging.level.org.apache.camel.component.jolokia = TRACE
logging.level.org.jolokia = TRACE
```

59.6. USING JOLOKIA CONFIGURATION IN KUBERNETES

The starter also provides some default configurations for a Kubernetes cluster environment. The starter checks the existence of a specific file to use as certification authority (**/var/run/secrets/kubernetes.io/serviceaccount/service-ca.crt**) and, if this file exists, it initializes the

server using TLS protocol and client authentication. So the endpoint become <https://0.0.0.0:8778/jolokia/>. It is possible to avoid this behaviour with the property **camel.component.jolokia.kubernetes-discover=false**.

The wrapped library Jolokia Spring Support provides the integration with the Spring Boot Actuator as per configuration where it is possible to retrieve information about Jolokia server. As any other actuator endpoint, it is possible to be excluded or disabled.

CHAPTER 60. JPA

Since Camel 1.0

Both producer and consumer are supported.

The JPA component enables you to store and retrieve Java objects from persistent storage using EJB 3's Java Persistence Architecture (JPA). Java Persistence Architecture (JPA) is a standard interface layer that wraps Object/Relational Mapping (ORM) products such as OpenJPA, Hibernate, TopLink.

60.1. DEPENDENCIES

When using **jpa** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jpa-starter</artifactId>
</dependency>
```

60.2. SENDING TO THE ENDPOINT

You can store a Java entity bean in a database by sending it to a JPA producer endpoint. The body of the **In** message is assumed to be an entity bean (that is, a POJO with an `@Entity` annotation on it) or a collection or array of entity beans.

If the body is a List of entities, use **entityType=java.util.List** as a configuration passed to the producer endpoint.

If the body does not contain one of the previous listed types, put a Message Translator before the endpoint to perform the necessary conversion first.

You can use **query**, **namedQuery** or **nativeQuery** for the producer as well. For the value of the **parameters**, you can use the Simple expression which allows you to retrieve parameter values from Message body, header and etc. Those query can be used for retrieving a set of data with using **SELECT** JPQL/SQL statement as well as executing bulk update/delete with using **UPDATE/DELETE** JPQL/SQL statement. Please note that you need to specify **useExecuteUpdate** to **true** if you execute **UPDATE/DELETE** with **namedQuery** as camel doesn't look into the named query unlike **query** and **nativeQuery**.

60.3. CONSUMING FROM THE ENDPOINT

Consuming messages from a JPA consumer endpoint removes (or updates) entity beans in the database. This allows you to use a database table as a logical queue: consumers take messages from the queue and then delete/update them to logically remove them from the queue.

If you do not wish to delete the entity bean when it has been processed (and when routing is done), you can specify **consumeDelete=false** on the URI. This will result in the entity being processed each poll.

If you would rather perform some update on the entity to mark it as processed (such as to exclude it from a future query) then you can annotate a method with `@Consumed` which will be invoked on your entity bean when the entity bean when it has been processed (and when routing is done).

You can use [@PreConsumed](#) which will be invoked on your entity bean before it has been processed (before routing).

If you are consuming a lot (100K+) of rows and experience OutOfMemory problems you should set the `maximumResults` to sensible value.

60.4. URI FORMAT

```
jpa:entityClassName[?options]
```

For sending to the endpoint, the **entityClassName** is optional. If specified, it helps the [Type Converter](#) to ensure the body is of the correct type.

For consuming, the **entityClassName** is mandatory.

60.5. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

60.5.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

60.5.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.

- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

60.6. COMPONENT OPTIONS

The JPA component supports 9 options, which are listed below.

Name	Description	Default	Type
aliases (common)	Maps an alias to a JPA entity class. The alias can then be used in the endpoint URI (instead of the fully qualified class name).		Map
entityManagerFactory (common)	To use the EntityManagerFactory. This is strongly recommended to configure.		EntityManagerFactory
joinTransaction (common)	The camel-jpa component will join transaction by default. You can use this option to turn this off, for example if you use LOCAL_RESOURCE and join transaction doesn't work with your JPA provider. This option can also be set globally on the JpaComponent, instead of having to set it on all endpoints.	true	boolean
sharedEntityManager (common)	Whether to use Spring's SharedEntityManager for the consumer/producer. Note in most cases joinTransaction should be set to false as this is not an EXTENDED EntityManager.	false	boolean
transactionManager (common)	To use the PlatformTransactionManager for managing transactions.		PlatformTransactionManager
transactionStrategy (common)	To use the TransactionStrategy for running the operations in a transaction.		TransactionStrategy
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

60.6.1. Endpoint Options

The JPA endpoint is configured using URI syntax:

```
jpa:entityType
```

with the following path and query parameters:

60.6.1.1. Path Parameters (1 parameters)

Name	Description	Default	Type
entityType (common)	Required Entity class name.		Class

60.6.1.2. Query Parameters (44 parameters)

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
joinTransaction (common)	The camel-jpa component will join transaction by default. You can use this option to turn this off, for example if you use LOCAL_RESOURCE and join transaction doesn't work with your JPA provider. This option can also be set globally on the JpaComponent, instead of having to set it on all endpoints.	true	boolean
maximumResults (common)	Set the maximum number of results to retrieve on the Query.	-1	int
namedQuery (common)	To use a named query.		String
nativeQuery (common)	To use a custom native query. You may want to use the option resultClass also when using native queries.		String
persistenceUnit (common)	Required The JPA persistence unit used by default.	camel	String
query (common)	To use a custom query.		String
resultClass (common)	Defines the type of the returned payload (we will call entityManager.createNativeQuery(nativeQuery, resultClass) instead of entityManager.createNativeQuery(nativeQuery)). Without this option, we will return an object array. Only has an affect when using in conjunction with native query when consuming data.		Class
consumeDelete (consumer)	If true, the entity is deleted after it is consumed; if false, the entity is not deleted.	true	boolean
consumeLockEntity (consumer)	Specifies whether or not to set an exclusive lock on each entity bean while processing the results from polling.	true	boolean
deleteHandler (consumer)	To use a custom DeleteHandler to delete the row after the consumer is done processing the exchange.		DeleteHandler

Name	Description	Default	Type
lockModeType (consumer)	To configure the lock mode on the consumer. Enum values: • READ • WRITE • OPTIMISTIC • OPTIMISTIC_FORCE_INCREMENT • PESSIMISTIC_READ • PESSIMISTIC_WRITE • PESSIMISTIC_FORCE_INCREMENT • NONE	PESSIMISTIC_WRITE	LockModeType
maxMessagesPerPoll (consumer)	An integer value to define the maximum number of messages to gather per poll. By default, no maximum is set. Can be used to avoid polling many thousands of messages when starting up the server. Set a value of 0 or negative to disable.		int
preDeleteHandler (consumer)	To use a custom Pre-DeleteHandler to delete the row after the consumer has read the entity.		DeleteHandler
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
skipLockedEntity (consumer)	To configure whether to use NOWAIT on lock and silently skip the entity.	false	boolean
transacted (consumer)	Whether to run the consumer in transacted mode, by which all messages will either commit or rollback, when the entire batch has been processed. The default behavior (false) is to commit all the previously successfully processed messages, and only rollback the last failed message.	false	boolean

Name	Description	Default	Type
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
parameters (consumer (advanced))	This key/value mapping is used for building the query parameters. It is expected to be of the generic type <code>java.util.Map</code> where the keys are the named parameters of a given JPA query and the values are their corresponding effective values you want to select for. When it's used for producer, Simple expression can be used as a parameter value. It allows you to retrieve parameter values from the message body, header and etc.		<code>Map</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
findEntity (producer)	If enabled then the producer will find a single entity by using the message body as key and <code>entityType</code> as the class type. This can be used instead of a query to find a single entity.	false	boolean

Name	Description	Default	Type
flushOnSend (producer)	Flushes the EntityManager after the entity bean has been persisted.	true	boolean
remove (producer)	Indicates to use entityManager.remove(entity).	false	boolean
useExecuteUpdate (producer)	To configure whether to use executeUpdate() when producer executes a query. When you use INSERT, UPDATE or DELETE statement as a named query, you need to specify this option to 'true'.		Boolean
usePersist (producer)	Indicates to use entityManager.persist(entity) instead of entityManager.merge(entity). Note: entityManager.persist(entity) doesn't work for detached entities (where the EntityManager has to execute an UPDATE instead of an INSERT query)!.	false	boolean
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
usePassedInEntityManager (producer (advanced))	If set to true, then Camel will use the EntityManager from the header JpaConstants.ENTITY_MANAGER instead of the configured entity manager on the component/endpoint. This allows end users to control which entity manager will be in use.	false	boolean
entityManagerProperties (advanced)	Additional properties for the entity manager to use.		Map
sharedEntityManager (advanced)	Whether to use Spring's SharedEntityManager for the consumer/producer. Note in most cases joinTransaction should be set to false as this is not an EXTENDED EntityManager.	false	boolean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int

Name	Description	Default	Type
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object

Name	Description	Default	Type
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANoseconds • MICROseconds • MILLIseconds • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean

60.7. MESSAGE HEADERS

The JPA component supports 2 message header(s), which are listed below:

Name	Description	Default	Type
CamelEntityManager (common) Constant: ENTITY_MANAGER	The JPA EntityManager object.		EntityManager
CamelJpaParameters (producer) Constant: link: JPA_PARAMETER_HEADERS_HEADER	Alternative way for passing query parameters as an Exchange header.		Map

60.8. CONFIGURING ENTITYMANAGERFACTORY

It is recommended to configure the JPA component to use a specific **EntityManagerFactory** instance. If failed to do so each **JpaEndpoint** will auto create their own instance of **EntityManagerFactory** which most often is not what you want.

For example, you can instantiate a JPA component that references the **myEMFactory** entity manager factory, as follows:

```
<bean id="jpa" class="org.apache.camel.component.jpa.JpaComponent">
  <property name="entityManagerFactory" ref="myEMFactory"/>
</bean>
```

The **JpaComponent** automatically looks up the **EntityManagerFactory** from the Registry which means you do not need to configure this on the **JpaComponent** as shown above. You only need to do so if there is ambiguity, in which case Camel will log a WARN.

60.9. CONFIGURING TRANSACTIONMANAGER

The **JpaComponent** automatically looks up the **TransactionManager** from the Registry. If Camel won't find any **TransactionManager** instance registered, it will also look up for the **TransactionTemplate** and try to extract **TransactionManager** from it.

If none **TransactionTemplate** is available in the registry, **JpaEndpoint** will auto create their own instance of **TransactionManager** which most often is not what you want.

If more than single instance of the **TransactionManager** is found, Camel will log a WARN. In such cases you might want to instantiate and explicitly configure a JPA component that references the **myTransactionManager** transaction manager, as follows:

```
<bean id="jpa" class="org.apache.camel.component.jpa.JpaComponent">
  <property name="entityManagerFactory" ref="myEMFactory"/>
  <property name="transactionManager" ref="myTransactionManager"/>
</bean>
```

60.10. USING A CONSUMER WITH A NAMED QUERY

For consuming only selected entities, you can use the **namedQuery** URI query option. First, you have to define the named query in the JPA Entity class:

```
@Entity
@NamedQuery(name = "step1", query = "select x from MultiSteps x where x.step = 1")
public class MultiSteps {
  ...
}
```

After that you can define a consumer uri as shown below:

```
from("jpa://org.apache.camel.examples.MultiSteps?namedQuery=step1")
.to("bean:myBusinessLogic");
```

60.11. USING A CONSUMER WITH A QUERY

For consuming only selected entities, you can use the **query** URI query option. You only have to define the query option:

```
from("jpa://org.apache.camel.examples.MultiSteps?query=select o from  
org.apache.camel.examples.MultiSteps o where o.step = 1")  
.to("bean:myBusinessLogic");
```

60.12. USING A CONSUMER WITH A NATIVE QUERY

For consuming only selected entities, you can use the **nativeQuery** URI query option. You only have to define the native query option:

```
from("jpa://org.apache.camel.examples.MultiSteps?nativeQuery=select * from MultiSteps where step  
= 1")  
.to("bean:myBusinessLogic");
```

If you use the native query option, you will receive an object array in the message body.

60.13. USING A PRODUCER WITH A NAMED QUERY

For retrieving selected entities or execute bulk update/delete, you can use the **namedQuery** URI query option. First, you have to define the named query in the JPA Entity class:

```
@Entity  
@NamedQuery(name = "step1", query = "select x from MultiSteps x where x.step = 1")  
public class MultiSteps {  
    ...  
}
```

After that you can define a producer uri as shown below:

```
from("direct:namedQuery")  
.to("jpa://org.apache.camel.examples.MultiSteps?namedQuery=step1");
```

Note that you need to specify **useExecuteUpdate** option to **true** to execute **UPDATE/DELETE** statement as a named query.

60.14. USING A PRODUCER WITH A QUERY

For retrieving selected entities or execute bulk update/delete, you can use the **query** URI query option. You only have to define the query option:

```
from("direct:query")  
.to("jpa://org.apache.camel.examples.MultiSteps?query=select o from  
org.apache.camel.examples.MultiSteps o where o.step = 1");
```

60.15. USING A PRODUCER WITH A NATIVE QUERY

For retrieving selected entities or execute bulk update/delete, you can use the **nativeQuery** URI query option. You only have to define the native query option:

```
from("direct:nativeQuery")
.to("jpa://org.apache.camel.examples.MultiSteps?
resultClass=org.apache.camel.examples.MultiSteps&nativeQuery=select * from MultiSteps where
step = 1");
```

If you use the native query option without specifying **resultClass**, you will receive an object array in the message body.

60.16. USING THE JPA-BASED IDEMPOTENT REPOSITORY

The Idempotent Consumer from the [EIP patterns](#) is used to filter out duplicate messages. A JPA-based idempotent repository is provided.

To use the JPA based idempotent repository.

Procedure

1. Set up a **persistence-unit** in the persistence.xml file.
2. Set up a **org.springframework.orm.jpa.JpaTemplate** which is used by the **org.apache.camel.processor.idempotent.jpa.JpaMessageIdRepository**.
3. Configure the error formatting macro: snippet: java.lang.IndexOutOfBoundsException: Index: 20, Size: 20
4. Configure the idempotent repository as **org.apache.camel.processor.idempotent.jpa.JpaMessageIdRepository**.
5. Create the JPA idempotent repository in the Spring XML file as shown below:

```
<camelContext xmlns="http://camel.apache.org/schema/spring">
  <route id="JpaMessageIdRepositoryTest">
    <from uri="direct:start" />
    <idempotentConsumer idempotentRepository="jpaStore">
      <header>messageId</header>
      <to uri="mock:result" />
    </idempotentConsumer>
  </route>
</camelContext>
```

When running this Camel component tests inside your IDE

If you run the [tests of this component](#) directly inside your IDE, and not through Maven, then you could see exceptions like these:

```
org.springframework.transaction.CannotCreateTransactionException: Could not open JPA
EntityManager for transaction; nested exception is
<openjpa-2.2.1-r422266:1396819 nonfatal user error>
org.apache.openjpa.persistence.ArgumentException: This configuration disallows runtime
optimization,
but the following listed types were not enhanced at build time or at class load time with a javaagent:
"org.apache.camel.examples.SendEmail".
```

```

    at
    org.springframework.orm.jpa.JpaTransactionManager.doBegin(JpaTransactionManager.java:427)
    at
    org.springframework.transaction.support.AbstractPlatformTransactionManager.getTransaction(Abstract
    PlatformTransactionManager.java:371)
    at
    org.springframework.transaction.support.TransactionTemplate.execute(TransactionTemplate.java:127)

    at org.apache.camel.processor.jpa.JpaRouteTest.cleanupRepository(JpaRouteTest.java:96)
    at org.apache.camel.processor.jpa.JpaRouteTest.createCamelContext(JpaRouteTest.java:67)
    at org.apache.camel.test.junit5.CamelTestSupport.doSetUp(CamelTestSupport.java:238)
    at org.apache.camel.test.junit5.CamelTestSupport.setUp(CamelTestSupport.java:208)

```

The problem here is that the source has been compiled or recompiled through your IDE and not through Maven, which would [enhance the byte-code at build time](#). To overcome this you need to enable [dynamic byte-code enhancement of OpenJPA](#). For example, assuming the current OpenJPA version being used in Camel is 2.2.1, to run the tests inside your IDE you would need to pass the following argument to the JVM:

```
-javaagent:<path_to_your_local_m2_cache>/org/apache/openjpa/openjpa/2.2.1/openjpa-2.2.1.jar
```

60.17. SPRING BOOT AUTO-CONFIGURATION

The component supports 10 options, which are listed below.

Name	Description	Default	Type
camel.component.jpa.aliaes	Maps an alias to a JPA entity class. The alias can then be used in the endpoint URI (instead of the fully qualified class name).		Map
camel.component.jpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.jpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.jpa.enabled	Whether to enable auto configuration of the jpa component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.jpa.entity-manager-factory	To use the EntityManagerFactory. This is strongly recommended to configure. The option is a <code>javax.persistence.EntityManagerFactory</code> type.		EntityManagerFactory
camel.component.jpa.join-transaction	The camel-jpa component will join transaction by default. You can use this option to turn this off, for example if you use <code>LOCAL_RESOURCE</code> and join transaction doesn't work with your JPA provider. This option can also be set globally on the <code>JpaComponent</code> , instead of having to set it on all endpoints.	true	Boolean
camel.component.jpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.jpa.shared-entity-manager	Whether to use Spring's <code>SharedEntityManager</code> for the consumer/producer. Note in most cases <code>joinTransaction</code> should be set to false as this is not an <code>EXTENDED EntityManager</code> .	false	Boolean
camel.component.jpa.transaction-manager	To use the <code>PlatformTransactionManager</code> for managing transactions. The option is a <code>org.springframework.transaction.PlatformTransactionManager</code> type.		PlatformTransactionManager
camel.component.jpa.transaction-strategy	To use the <code>TransactionStrategy</code> for running the operations in a transaction. The option is a <code>org.apache.camel.component.jpa.TransactionStrategy</code> type.		TransactionStrategy

CHAPTER 61. JQ

Since Camel 3.18

Camel supports [JQ](#) to allow using [Expression](#) or [Predicate](#) on JSON messages.

61.1. DEPENDENCIES

When using **jq with** Red Hat build of Camel Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jq-starter</artifactId>
</dependency>
```

61.2. JQ OPTIONS

The JQ language supports 4 options, which are listed below.

Name	Default	Java Type	Description
headerName		String	Name of header to use as input, instead of the message body It has as higher precedent than the propertyName if both are set.
propertyName		String	Name of property to use as input, instead of the message body. It has a lower precedent than the headerName if both are set.
resultType		String	Sets the class of the result type (type from output).
trim	true	Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

61.3. EXAMPLES

For example, you can use JQ in a [Predicate](#) with the [Content Based Router EIP](#) .

```
from("queue:books.new")
  .choice()
    .when().jq(".store.book.price < 10")
      .to("jms:queue:book.cheap")
    .when().jq(".store.book.price < 30")
      .to("jms:queue:book.average")
    .otherwise()
      .to("jms:queue:book.expensive");
```

61.4. MESSAGE BODY TYPES

Camel JQ leverages **camel-jackson** for type conversion. To enable camel-jackson POJO type conversion, refer to the [Camel Jackson](#) documentation.

61.5. USING HEADER AS INPUT

By default, JQ uses the message body as the input source. However, you can also use a header as input by specifying the **headerName** option.

For example to count the number of books from a JSON document that was stored in a header named **books** you can do:

```
from("direct:start")
  .setHeader("numberOfBooks")
  .jq(".store.books | length", int.class, "books")
  .to("mock:result");
```

61.6. CAMEL SUPPLIED JQ FUNCTIONS

The camel-jq adds the following functions:

- **header** – Allow to access the Message header in a JQ expression.

For example, to set the property foo with the value from the Message header `MyHeader`:

```
from("direct:start")
  .transform()
  .jq(".foo = header(\"MyHeader\")")
  .to("mock:result");
```

61.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.language.jq.enabled	Whether to enable auto configuration of the jq language. This is enabled by default.		Boolean
camel.language.jq.header-name	Name of header to use as input, instead of the message body It has as higher precedent than the propertyName if both are set.		String
camel.language.jq.property-name	Name of property to use as input, instead of the message body. It has a lower precedent than the headerName if both are set.		String
camel.language.jq.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

CHAPTER 62. JSLT

Since Camel 3.1

Only producer is supported

The JSLT component allows you to process a JSON messages using an [JSLT](#) expression. This can be ideal when doing JSON to JSON transformation or querying data.

62.1. DEPENDENCIES

When using **jslt** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jslt-starter</artifactId>
</dependency>
```

62.2. URI FORMAT

```
jslt:specName[?options]
```

Where **specName** is the classpath-local URI of the specification to invoke; or the complete URL of the remote specification (eg: [file://folder/myfile.vm](#)).

62.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

62.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

62.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

62.4. COMPONENT OPTIONS

The JSLT component supports 5 options, which are listed below.

Name	Description	Default	Type
allowTemplateFromHeader (producer)	Whether to allow to use resource template from header or not (default false). Enabling this allows to specify dynamic templates via message header. However this can be seen as a potential security vulnerability if the header is coming from a malicious user, so use this with care.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

Name	Description	Default	Type
functions (advanced)	JSLT can be extended by plugging in functions written in Java.		Collection
objectFilter (advanced)	JSLT can be extended by plugging in a custom jslt object filter.		JsonFilter

62.4.1. Endpoint Options

The JSLT endpoint is configured using URI syntax:

```
jslt:resourceUri
```

with the following path and query parameters:

62.4.1.1. Path Parameters (1 parameters)

Name	Description	Default	Type
resourceUri (producer)	Required Path to the resource. You can prefix with: classpath, file, http, ref, or bean. classpath, file and http loads the resource using these protocols (classpath is default). ref will lookup the resource in the registry. bean will call a method on a bean to be used as the resource. For bean you can specify the method name after dot, eg bean:myBean.myMethod.		String

62.4.1.2. Query Parameters (7 parameters)

Name	Description	Default	Type
allowContextMap All (producer)	Sets whether the context map should allow access to all details. By default only the message body and headers can be accessed. This option can be enabled for full access to the current Exchange and CamelContext. Doing so impose a potential security risk as this opens access to the full power of CamelContext API.	false	boolean

Name	Description	Default	Type
allowTemplateFromHeader (producer)	Whether to allow to use resource template from header or not (default false). Enabling this allows to specify dynamic templates via message header. However this can be seen as a potential security vulnerability if the header is coming from a malicious user, so use this with care.	false	boolean
contentCache (producer)	Sets whether to use resource content cache or not.	false	boolean
mapBigDecimalAsFloats (producer)	If true, the mapper will use the USE_BIG_DECIMAL_FOR_FLOATS in serialization features.	false	boolean
objectMapper (producer)	Setting a custom JSON Object Mapper to be used.		ObjectMapper
prettyPrint (common)	If true, JSON in output message is pretty printed.	false	boolean
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

62.5. MESSAGE HEADERS

The JSLT component supports 2 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelJsItString (producer) Constant: <code>HEADER_JSLT_STRING</code>	The JSLT Template as String.		String

Name	Description	Default	Type
CamelJsltResourceUri (producer) Constant: HEADER_JSLT_RESOURCE_URI	The resource URI.		String

62.6. PASSING VALUES TO JSLT

Camel can supply exchange information as variables when applying a JSLT expression on the body. The available variables from the **Exchange** are:

name	value
headers	The headers of the In message as a json object
exchange.properties	The Exchange properties as a json object. exchange is the name of the variable and <i>properties</i> is the path to the exchange properties. Available if allowContextMapAll option is true.

All the values that cannot be converted to json with Jackson are denied and will not be available in the jslt expression.

For example, the header named "type" and the exchange property "instance" can be accessed like

```
{
  "type": $headers.type,
  "instance": $exchange.properties.instance
}
```

62.7. SAMPLES

The sample example is as given below.

```
from("activemq:My.Queue").
  to("jslt:com/acme/MyResponse.json");
```

And a file based resource:

```
from("activemq:My.Queue").
  to("jslt:file://myfolder/MyResponse.json?contentCache=true").
  to("activemq:Another.Queue");
```

You can also specify which JSLT expression the component should use dynamically via a header, so for example:

```
from("direct:in").
```

```
setHeader("CamelJsltResourceUri").constant("path/to/my/spec.json").
to("jslt:dummy?allowTemplateFromHeader=true");
```

Or send whole jslt expression via header: (suitable for querying)

```
from("direct:in").
setHeader("CamelJsltString").constant(".published").
to("jslt:dummy?allowTemplateFromHeader=true");
```

Passing exchange properties to the jslt expression can be done like this

```
from("direct:in").
to("jslt:com/acme/MyResponse.json?allowContextMapAll=true");
```

62.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
camel.component.jslt.allow-template-from-header	Whether to allow to use resource template from header or not (default false). Enabling this allows to specify dynamic templates via message header. However this can be seen as a potential security vulnerability if the header is coming from a malicious user, so use this with care.	false	Boolean
camel.component.jslt.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.jslt.enabled	Whether to enable auto configuration of the jslt component. This is enabled by default.		Boolean
camel.component.jslt.functions	JSLT can be extended by plugging in functions written in Java.		Collection

Name	Description	Default	Type
camel.component.jslt.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.jslt.object-filter	JSLT can be extended by plugging in a custom jslt object filter. The option is a <code>com.schibsted.spt.data.jslt.filters.JsonFilter</code> type.		JsonFilter

CHAPTER 63. JSON GSON

Gson is a Data Format which uses the [Gson Library](#).

```
from("activemq:My.Queue").
  marshal().json(JsonLibrary.Gson).
  to("mqseries:Another.Queue");
```

63.1. DEPENDENCIES

When using **json-gson** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-gson-starter</artifactId>
</dependency>
```

63.2. GSON OPTIONS

The JSON Gson dataformat supports 3 options, which are listed below.

Name	Default	Java Type	Description
prettyPrint		Boolean	To enable pretty printing output nicely formatted. Is by default false.
unmarshalType		String	Class name of the java type to use when unmarshalling.
contentTypeHeader		Boolean	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.

63.3. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.dataformat.json-gson.content-type-header	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.	true	Boolean

Name	Description	Default	Type
<code>camel.dataformat.json-gson.enabled</code>	Whether to enable auto configuration of the json-gson data format. This is enabled by default.		Boolean
<code>camel.dataformat.json-gson.pretty-print</code>	To enable pretty printing output nicely formatted. Is by default false.	false	Boolean
<code>camel.dataformat.json-gson.unmarshal-type</code>	Class name of the java type to use when unmarshalling.		String

CHAPTER 64. JSON JACKSON

Jackson is a Data Format which uses the [Jackson Library](#)

```
from("activemq:My.Queue").
  marshal().json(JsonLibrary.Jackson).
  to("mqseries:Another.Queue");
```

64.1. DEPENDENCIES

When using **json-jackson** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jackson-starter</artifactId>
</dependency>
```

64.2. JACKSON OPTIONS

The JSON Jackson dataformat supports 20 options, which are listed below.

Name	Default	Java Type	Description
objectMapper		String	Lookup and use the existing ObjectMapper with the given id when using Jackson.
useDefaultObjectMapper		Boolean	Whether to lookup and use default Jackson ObjectMapper from the registry.
prettyPrint		Boolean	To enable pretty printing output nicely formatted. Is by default false.
unmarshalType		String	Class name of the java type to use when unmarshalling.

Name	Default	Java Type	Description
jsonView		String	When marshalling a POJO to JSON you might want to exclude certain fields from the JSON output. With Jackson you can use JSON views to accomplish this. This option is to refer to the class which has JsonView annotations.
include		String	If you want to marshal a pojo to JSON, and the pojo has some fields with null values. And you want to skip these null values, you can set this option to NON_NULL.
allowJmsType		Boolean	Used for JMS users to allow the JMSType header from the JMS spec to specify a FQN classname to use to unmarshal to.
collectionType		String	Refers to a custom collection type to lookup in the registry to use. This option should rarely be used, but allows to use different collection types than java.util.Collection based as default.

Name	Default	Java Type	Description
<code>useList</code>		Boolean	To unmarshal to a List of Map or a List of Pojo.
<code>moduleClassNames</code>		String	To use custom Jackson modules <code>com.fasterxml.jackson.databind.Module</code> specified as a String with FQN class names. Multiple classes can be separated by comma.
<code>moduleRefs</code>		String	To use custom Jackson modules referred from the Camel registry. Multiple modules can be separated by comma.
<code>enableFeatures</code>		String	Set of features to enable on the Jackson <code>com.fasterxml.jackson.databind.ObjectMapper</code> . The features should be a name that matches a enum from <code>com.fasterxml.jackson.databind.SerializationFeature</code> , <code>com.fasterxml.jackson.databind.DeserializationFeature</code> , or <code>com.fasterxml.jackson.databind.MapperFeature</code> . Multiple features can be separated by comma.

Name	Default	Java Type	Description
disableFeatures		String	Set of features to disable on the Jackson <code>com.fasterxml.jackson.databind.ObjectMapper</code> . The features should be a name that matches a enum from <code>com.fasterxml.jackson.databind.SerializationFeature</code> , <code>com.fasterxml.jackson.databind.DeserializationFeature</code> , or <code>com.fasterxml.jackson.databind.MapperFeature</code> . Multiple features can be separated by comma.
allowUnmarshalType		Boolean	If enabled then Jackson is allowed to attempt to use the <code>CamelJacksonUnmarshalType</code> header during the unmarshalling. This should only be enabled when desired to be used.
timezone		String	If set then Jackson will use the <code>Timezone</code> when marshalling/unmarshalling. This option will have no effect on the others <code>JsonDataFormat</code> , like <code>gson</code> , <code>fastjson</code> and <code>xstream</code> .

Name	Default	Java Type	Description
autoDiscoverObjectMapper		Boolean	If set to true then Jackson will lookup for an objectMapper into the registry.
contentTypeHeader		Boolean	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.
schemaResolver		String	Optional schema resolver used to lookup schemas for the data in transit.
autoDiscoverSchemaResolver		Boolean	When not disabled, the SchemaResolver will be looked up into the registry.
namingStrategy		String	If set then Jackson will use the the defined Property Naming Strategy. Possible values are: LOWER_CAMEL_CASE, LOWER_DOT_CASE, LOWER_CASE, KEBAB_CASE, SNAKE_CASE and UPPER_CAMEL_CASE.

64.3. USING CUSTOM OBJECTMAPPER

You can configure **JacksonDataFormat** to use a custom **ObjectMapper** in case you need more control of the mapping configuration.

If you setup a single **ObjectMapper** in the registry, then Camel will automatic lookup and use this **ObjectMapper**. For example if you use Spring Boot, then Spring Boot can provide a default **ObjectMapper** for you if you have Spring MVC enabled. And this would allow Camel to detect that there is one bean of **ObjectMapper** class type in the Spring Boot bean registry and then use it. When this happens you should set a **INFO** logging from Camel.

64.4. USING JACKSON FOR AUTOMATIC TYPE CONVERSION

The **camel-jackson** module allows integrating Jackson as a [Type Converter](#). This works in a similar way to [JAXB](#) that integrates with Camel's type converter.

To use this **camel-jackson** must be enabled, which is done by setting the following options on the **CamelContext** global options, as shown:

```
@Bean
CamelContextConfiguration contextConfiguration() {
    return new CamelContextConfiguration() {
        @Override
        public void beforeApplicationStart(CamelContext context) {
            // Enable Jackson JSON type converter.
            context.getGlobalOptions().put(JacksonConstants.ENABLE_TYPE_CONVERTER, "true");
            // Allow Jackson JSON to convert to pojo types also
            // (by default Jackson only converts to String and other simple types)
            getContext().getGlobalOptions().put(JacksonConstants.TYPE_CONVERTER_TO_POJO,
"true");
        }

        @Override
        public void afterApplicationStart(CamelContext camelContext) {

        }
    };
}
```

The **camel-jackson** type converter integrates with [JAXB](#) which means you can annotate POJO class with **JAXB** annotations that Jackson can use. You can also use Jackson's own annotations on your POJO classes.

64.5. SPRING BOOT AUTO-CONFIGURATION

The component supports 21 options, which are listed below.

Name	Description	Default	Type
<code>camel.dataformat.json-jackson.allow-jms-type</code>	Used for JMS users to allow the JMSType header from the JMS spec to specify a FQN classname to use to unmarshal to.	false	Boolean
<code>camel.dataformat.json-jackson.allow-unmarshall-type</code>	If enabled then Jackson is allowed to attempt to use the CamelJacksonUnmarshalType header during the unmarshalling. This should only be enabled when desired to be used.	false	Boolean
<code>camel.dataformat.json-jackson.auto-discover-object-mapper</code>	If set to true then Jackson will lookup for an objectMapper into the registry.	false	Boolean
<code>camel.dataformat.json-jackson.auto-discover-schema-resolver</code>	When not disabled, the SchemaResolver will be looked up into the registry.	true	Boolean
<code>camel.dataformat.json-jackson.collection-type</code>	Refers to a custom collection type to lookup in the registry to use. This option should rarely be used, but allows to use different collection types than java.util.Collection based as default.		String
<code>camel.dataformat.json-jackson.content-type-header</code>	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.	true	Boolean
<code>camel.dataformat.json-jackson.disable-features</code>	Set of features to disable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature. Multiple features can be separated by comma.		String
<code>camel.dataformat.json-jackson.enable-features</code>	Set of features to enable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature. Multiple features can be separated by comma.		String

Name	Description	Default	Type
<code>camel.dataformat.json-jackson.enabled</code>	Whether to enable auto configuration of the json-jackson data format. This is enabled by default.		Boolean
<code>camel.dataformat.json-jackson.include</code>	If you want to marshal a pojo to JSON, and the pojo has some fields with null values. And you want to skip these null values, you can set this option to <code>NON_NULL</code> .		String
<code>camel.dataformat.json-jackson.json-view</code>	When marshalling a POJO to JSON you might want to exclude certain fields from the JSON output. With Jackson you can use JSON views to accomplish this. This option is to refer to the class which has <code>JsonView</code> annotations.		String
<code>camel.dataformat.json-jackson.module-class-names</code>	To use custom Jackson modules <code>com.fasterxml.jackson.databind.Module</code> specified as a String with FQN class names. Multiple classes can be separated by comma.		String
<code>camel.dataformat.json-jackson.module-refs</code>	To use custom Jackson modules referred from the Camel registry. Multiple modules can be separated by comma.		String
<code>camel.dataformat.json-jackson.naming-strategy</code>	If set then Jackson will use the the defined Property Naming Strategy. Possible values are: <code>LOWER_CAMEL_CASE</code> , <code>LOWER_DOT_CASE</code> , <code>LOWER_CASE</code> , <code>KEBAB_CASE</code> , <code>SNAKE_CASE</code> and <code>UPPER_CAMEL_CASE</code> .		String
<code>camel.dataformat.json-jackson.object-mapper</code>	Lookup and use the existing ObjectMapper with the given id when using Jackson.		String
<code>camel.dataformat.json-jackson.pretty-print</code>	To enable pretty printing output nicely formatted. Is by default false.	false	Boolean
<code>camel.dataformat.json-jackson.schema-resolver</code>	Optional schema resolver used to lookup schemas for the data in transit.		String

Name	Description	Default	Type
<code>camel.dataformat.json-jackson.timezone</code>	If set then Jackson will use the Timezone when marshalling/unmarshalling. This option will have no effect on the others Json DataFormat, like gson, fastjson and xstream.		String
<code>camel.dataformat.json-jackson.unmarshalling-type</code>	Class name of the java type to use when unmarshalling.		String
<code>camel.dataformat.json-jackson.use-default-object-mapper</code>	Whether to lookup and use default Jackson ObjectMapper from the registry.	true	Boolean
<code>camel.dataformat.json-jackson.use-list</code>	To unmarshal to a List of Map or a List of Pojo.	false	Boolean

CHAPTER 65. JSONPATH

Camel supports [JSONPath](#) to allow using [Expression](#) or [Predicate](#) on JSON messages.

65.1. DEPENDENCIES

When using **jsonpath** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jsonpath-starter</artifactId>
</dependency>
```

65.2. JSONPATH OPTIONS

The JSONPath language supports 8 options, which are listed below.

Name	Default	Java Type	Description
resultType		String	Sets the class name of the result type (type from output).
suppressExceptions		Boolean	Whether to suppress exceptions such as PathNotFoundException.
allowSimple		Boolean	Whether to allow inlined Simple exceptions in the JSONPath expression.
allowEasyPredicate		Boolean	Whether to allow using the easy predicate parser to pre-parse predicates.
writeAsString		Boolean	Whether to write the output of each row/element as a JSON String value instead of a Map/POJO value.
headerName		String	Name of header to use as input, instead of the message body.

Name	Default	Java Type	Description
option		Enum	To configure additional options on JSONPath. Multiple values can be separated by comma. Enum values: • DEFAULT_PATH_LEAF_TO_NULL • ALWAYS_RETURN_LIST • AS_PATH_LIST • SUPPRESS_EXCEPTIONS • REQUIRE_PROPERTIES
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

65.3. EXAMPLES

For example, you can use JSONPath in a [Predicate](#) with the [Content Based Router EIP](#) .

```
from("queue:books.new")
  .choice()
    .when().jsonpath("$.store.book[?(@.price < 10)]")
      .to("jms:queue:book.cheap")
    .when().jsonpath("$.store.book[?(@.price < 30)]")
      .to("jms:queue:book.average")
    .otherwise()
      .to("jms:queue:book.expensive");
```

And in XML DSL:

```
<route>
  <from uri="direct:start"/>
  <choice>
    <when>
      <jsonpath>$.store.book[?(@.price < 10)]</jsonpath>
      <to uri="mock:cheap"/>
    </when>
    <when>
      <jsonpath>$.store.book[?(@.price < 30)]</jsonpath>
      <to uri="mock:average"/>
    </when>
    <otherwise>
      <to uri="mock:expensive"/>
    </otherwise>
  </choice>
</route>
```

65.4. JSONPATH SYNTAX

Using the JSONPath syntax takes some time to learn, even for basic predicates. So for example to find out all the cheap books you have to do:

```
$$.store.book[?(@.price < 20)]
```

65.4.1. Easy JSONPath Syntax

However, what if you could just write it as:

```
store.book.price < 20
```

And you can omit the path if you just want to look at nodes with a price key:

```
price < 20
```

To support this there is a **EasyPredicateParser** which kicks-in if you have defined the predicate using a basic style. That means the predicate must not start with the **\$** sign, and only include one operator.

The easy syntax is:

```
left OP right
```

You can use Camel simple language in the right operator, eg:

```
store.book.price < ${header.limit}
```

See the [JSONPath](#) project page for more syntax examples.

65.5. SUPPORTED MESSAGE BODY TYPES

Camel JSonPath supports message body using the following types:

Type	Comment
File	Reading from files
String	Plain strings
Map	Message bodies as java.util.Map types
List	Message bodies as java.util.List types
POJO	Optional If Jackson is on the classpath, then camel-jsonpath is able to use Jackson to read the message body as POJO and convert to java.util.Map which is supported by JSonPath. For example, you can add camel-jackson as dependency to include Jackson.

Type	Comment
InputStream	If none of the above types matches, then Camel will attempt to read the message body as a java.io.InputStream .

If a message body is of unsupported type then an exception is thrown by default, however you can configure JXPath to suppress exceptions (see below)

65.6. SUPPRESSING EXCEPTIONS

By default, jsonpath will throw an exception if the json payload does not have a valid path accordingly to the configured jsonpath expression. In some use-cases you may want to ignore this in case the json payload contains optional data. Therefore, you can set the option **suppressExceptions** to **true** to ignore this as shown:

```
from("direct:start")
  .choice()
    // use true to suppress exceptions
    .when().jsonpath("person.middlename", true)
      .to("mock:middle")
    .otherwise()
      .to("mock:other");
```

And in XML DSL:

```
<route>
  <from uri="direct:start"/>
  <choice>
    <when>
      <jsonpath suppressExceptions="true">person.middlename</jsonpath>
      <to uri="mock:middle"/>
    </when>
    <otherwise>
      <to uri="mock:other"/>
    </otherwise>
  </choice>
</route>
```

This option is also available on the **@JsonPath** annotation.

65.7. INLINE SIMPLE EXPRESSIONS

It's possible to inlined [Simple](#) language in the JXPath expression using the simple syntax **\${xxx}**.

An example is shown below:

```
from("direct:start")
  .choice()
    .when().jsonpath("$.store.book[?(@.price < ${header.cheap})]")
      .to("mock:cheap")
```

```
.when().jsonpath("$.store.book[?(@.price < ${header.average})]")
    .to("mock:average")
    .otherwise()
    .to("mock:expensive");
```

And in XML DSL:

```
<route>
  <from uri="direct:start"/>
  <choice>
    <when>
      <jsonpath>$.store.book[?(@.price < ${header.cheap})]</jsonpath>
      <to uri="mock:cheap"/>
    </when>
    <when>
      <jsonpath>$.store.book[?(@.price < ${header.average})]</jsonpath>
      <to uri="mock:average"/>
    </when>
    <otherwise>
      <to uri="mock:expensive"/>
    </otherwise>
  </choice>
</route>
```

You can turn off support for inlined Simple expression by setting the option **allowSimple** to **false** as shown:

```
.when().jsonpath("$.store.book[?(@.price < 10)]", false, false)
```

And in XML DSL:

```
<jsonpath allowSimple="false">$.store.book[?(@.price < 10)]</jsonpath>
```

65.8. JSONPATH INJECTION

You can use [Bean Integration](#) to invoke a method on a bean and use various languages such as JSONPath (via the **@JsonPath** annotation) to extract a value from the message and bind it to a method parameter, as shown below:

```
public class Foo {

    @Consume("activemq:queue:books.new")
    public void doSomething(@JsonPath("$.store.book[*].author") String author, @Body String json) {
        // process the inbound message here
    }
}
```

65.9. ENCODING DETECTION

The encoding of the JSON document is detected automatically, if the document is encoded in unicode (UTF-8, UTF-16LE, UTF-16BE, UTF-32LE, UTF-32BE) as specified in RFC-4627. If the encoding is a non-unicode encoding, you can either make sure that you enter the document in String format to

JSONPath, or you can specify the encoding in the header **CamelJsonPathJsonEncoding** which is defined as a constant in: **JsonpathConstants.HEADER_JSON_ENCODING**.

65.10. SPLIT JSON DATA INTO SUB ROWS AS JSON

You can use JSONPath to split a JSON document, such as:

```
from("direct:start")
  .split().jsonpath("$.store.book[*]")
  .to("log:book");
```

Then each book is logged, however the message body is a **Map** instance. Sometimes you may want to output this as plain String JSON value instead, which can be done with the **writeAsString** option as shown:

```
from("direct:start")
  .split().jsonpathWriteAsString("$.store.book[*]")
  .to("log:book");
```

Then each book is logged as a String JSON value.

65.11. USING HEADER AS INPUT

By default, JSONPath uses the message body as the input source. However, you can also use a header as input by specifying the **headerName** option.

For example to count the number of books from a JSON document that was stored in a header named **books** you can do:

```
from("direct:start")
  .setHeader("numberOfBooks")
  .jsonpath("$.store.book.length()", false, int.class, "books")
  .to("mock:result");
```

In the **jsonpath** expression above we specify the name of the header as **books**, and we also told that we wanted the result to be converted as an integer by **int.class**.

The same example in XML DSL would be:

```
<route>
  <from uri="direct:start"/>
  <setHeader name="numberOfBooks">
    <jsonpath headerName="books" resultType="int">$.store.book.length()</jsonpath>
  </setHeader>
  <to uri="mock:result"/>
</route>
```

65.12. SPRING BOOT AUTO-CONFIGURATION

The component supports 8 options, which are listed below.

Name	Description	Default	Type
<code>camel.language.js onpath.allow- easy-predicate</code>	Whether to allow using the easy predicate parser to pre-parse predicates.	true	Boolean
<code>camel.language.js onpath.allow- simple</code>	Whether to allow inlined Simple exceptions in the JSONPath expression.	true	Boolean
<code>camel.language.js onpath.enabled</code>	Whether to enable auto configuration of the jsonpath language. This is enabled by default.		Boolean
<code>camel.language.js onpath.header- name</code>	Name of header to use as input, instead of the message body.		String
<code>camel.language.js onpath.option</code>	To configure additional options on JSONPath. Multiple values can be separated by comma.		String
<code>camel.language.js onpath.suppress- exceptions</code>	Whether to suppress exceptions such as PathNotFoundException.	false	Boolean
<code>camel.language.js onpath.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.js onpath.write-as- string</code>	Whether to write the output of each row/element as a JSON String value instead of a Map/POJO value.	false	Boolean

CHAPTER 66. KAFKA

Both producer and consumer are supported

The Kafka component is used for communicating with [Apache Kafka](#) message broker.

66.1. DEPENDENCIES

When using **kafka** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>  
  <groupId>org.apache.camel.springboot</groupId>  
  <artifactId>camel-kafka-starter</artifactId>  
</dependency>
```

66.2. URI FORMAT

```
kafka:topic[?options]
```

66.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

66.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

66.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

66.4. COMPONENT OPTIONS

The Kafka component supports 116 options, which are listed below.

Name	Description	Default	Type
additionalProperties (common)	Sets additional properties for either kafka consumer or kafka producer in case they can't be set directly on the camel configurations (e.g: new Kafka properties that are not reflected yet in Camel configurations), the properties have to be prefixed with <code>additionalProperties..</code> E.g: <code>additionalProperties.transactional.id=12345&additionalProperties.schema.registry.url=http://localhost:8811/avro.</code>		Map
brokers (common)	URL of the Kafka brokers to use. The format is <code>host1:port1,host2:port2</code> , and the list can be a subset of brokers or a VIP pointing to a subset of brokers. This option is known as <code>bootstrap.servers</code> in the Kafka documentation.		String
clientId (common)	The client id is a user-specified string sent in each request to help trace calls. It should logically identify the application making the request.		String
configuration (common)	Allows to pre-configure the Kafka component with common options that the endpoints will reuse.		KafkaConfiguration
headerFilterStrategy (common)	To use a custom HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy

Name	Description	Default	Type
reconnectBackoffMaxMs (common)	The maximum amount of time in milliseconds to wait when reconnecting to a broker that has repeatedly failed to connect. If provided, the backoff per host will increase exponentially for each consecutive connection failure, up to this maximum. After calculating the backoff increase, 20% random jitter is added to avoid connection storms.	1000	Integer
retryBackoffMaxMs (common)	The maximum amount of time in milliseconds to wait when retrying a request to the broker that has repeatedly failed. If provided, the backoff per client will increase exponentially for each failed request, up to this maximum. To prevent all clients from being synchronized upon retry, a randomized jitter with a factor of 0.2 will be applied to the backoff, resulting in the backoff falling within a range between 20% below and 20% above the computed value. If <code>retry.backoff.ms</code> is set to be higher than <code>retry.backoff.max.ms</code> , then <code>retry.backoff.max.ms</code> will be used as a constant backoff from the beginning without any exponential increase.	1000	Integer
retryBackoffMs (common)	The amount of time to wait before attempting to retry a failed request to a given topic partition. This avoids repeatedly sending requests in a tight loop under some failure scenarios. This value is the initial backoff value and will increase exponentially for each failed request, up to the <code>retry.backoff.max.ms</code> value.	100	Integer
shutdownTimeout (common)	Timeout in milliseconds to wait gracefully for the consumer or producer to shutdown and terminate its worker threads.	30000	int
allowManualCommit (consumer)	Whether to allow doing manual commits via <code>KafkaManualCommit</code> . If this option is enabled then an instance of <code>KafkaManualCommit</code> is stored on the Exchange message header, which allows end users to access this API and perform manual offset commits via the Kafka consumer.	false	boolean
autoCommitEnable (consumer)	If true, periodically commit to ZooKeeper the offset of messages already fetched by the consumer. This committed offset will be used when the process fails as the position from which the new consumer will begin.	true	Boolean
autoCommitIntervalMs (consumer)	The frequency in ms that the consumer offsets are committed to zookeeper.	5000	Integer

Name	Description	Default	Type
autoCommitOnStop (consumer)	Whether to perform an explicit auto commit when the consumer stops to ensure the broker has a commit from the last consumed message. This requires the option <code>autoCommitEnable</code> is turned on. The possible values are: <code>sync</code>, <code>async</code>, or <code>none</code>. And <code>sync</code> is the default value. Enum values: • <code>sync</code> • <code>async</code> • <code>none</code>	<code>sync</code>	String
autoOffsetReset (consumer)	What to do when there is no initial offset in ZooKeeper or if an offset is out of range: <code>earliest</code> : automatically reset the offset to the earliest offset <code>latest</code> : automatically reset the offset to the latest offset <code>fail</code>: throw exception to the consumer. Enum values: • <code>latest</code> • <code>earliest</code> • <code>none</code>	<code>latest</code>	String
batching (consumer)	Whether to use batching for processing or streaming. The default is <code>false</code>, which uses streaming.	<code>false</code>	boolean
breakOnFirstError (consumer)	This options controls what happens when a consumer is processing an exchange and it fails. If the option is <code>false</code> then the consumer continues to the next message and processes it. If the option is <code>true</code> then the consumer breaks out, and will seek back to offset of the message that caused a failure, and then re-attempt to process this message. However this can lead to endless processing of the same message if its bound to fail every time, eg a poison message. Therefore its recommended to deal with that for example by using Camel's error handler.	<code>false</code>	boolean

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
checkCrcs (consumer)	Automatically check the CRC32 of the records consumed. This ensures no on-the-wire or on-disk corruption to the messages occurred. This check adds some overhead, so it may be disabled in cases seeking extreme performance.	true	Boolean
commitTimeoutMs (consumer)	The maximum time, in milliseconds, that the code will wait for a synchronous commit to complete.	5000	Long
consumerRequestTimeoutMs (consumer)	The configuration controls the maximum amount of time the client will wait for the response of a request. If the response is not received before the timeout elapses the client will resend the request if necessary or fail the request if retries are exhausted.	40000	Integer
consumersCount (consumer)	The number of consumers that connect to kafka server. Each consumer is run on a separate thread, that retrieves and process the incoming data.	1	int
fetchMaxBytes (consumer)	The maximum amount of data the server should return for a fetch request This is not an absolute maximum, if the first message in the first non-empty partition of the fetch is larger than this value, the message will still be returned to ensure that the consumer can make progress. The maximum message size accepted by the broker is defined via <code>message.max.bytes</code> (broker config) or <code>max.message.bytes</code> (topic config). Note that the consumer performs multiple fetches in parallel.	52428800	Integer
fetchMinBytes (consumer)	The minimum amount of data the server should return for a fetch request. If insufficient data is available the request will wait for that much data to accumulate before answering the request.	1	Integer

Name	Description	Default	Type
fetchWaitMaxMs (consumer)	The maximum amount of time the server will block before answering the fetch request if there isn't sufficient data to immediately satisfy fetch.min.bytes.	500	Integer
groupId (consumer)	A string that uniquely identifies the group of consumer processes to which this consumer belongs. By setting the same group id multiple processes indicate that they are all part of the same consumer group. This option is required for consumers.		String
groupInstanceId (consumer)	A unique identifier of the consumer instance provided by the end user. Only non-empty strings are permitted. If set, the consumer is treated as a static member, which means that only one instance with this ID is allowed in the consumer group at any time. This can be used in combination with a larger session timeout to avoid group rebalances caused by transient unavailability (e.g. process restarts). If not set, the consumer will join the group as a dynamic member, which is the traditional behavior.		String
headerDeserializer (consumer)	To use a custom <code>KafkaHeaderDeserializer</code> to deserialize kafka headers values.		<code>KafkaHeaderDeserializer</code>
heartbeatIntervalMs (consumer)	The expected time between heartbeats to the consumer coordinator when using Kafka's group management facilities. Heartbeats are used to ensure that the consumer's session stays active and to facilitate rebalancing when new consumers join or leave the group. The value must be set lower than <code>session.timeout.ms</code> , but typically should be set no higher than 1/3 of that value. It can be adjusted even lower to control the expected time for normal rebalances.	3000	Integer
keyDeserializer (consumer)	Deserializer class for key that implements the <code>Deserializer</code> interface.	<code>org.apache.kafka.common.serialization.StringDeserializer</code>	String

Name	Description	Default	Type
maxPartitionFetchBytes (consumer)	The maximum amount of data per-partition the server will return. The maximum total memory used for a request will be <code>#partitions max.partition.fetch.bytes</code> . This size must be at least as large as the maximum message size the server allows or else it is possible for the producer to send messages larger than the consumer can fetch. If that happens, the consumer can get stuck trying to fetch a large message on a certain partition.	1048576	Integer
maxPollIntervalMs (consumer)	The maximum delay between invocations of <code>poll()</code> when using consumer group management. This places an upper bound on the amount of time that the consumer can be idle before fetching more records. If <code>poll()</code> is not called before expiration of this timeout, then the consumer is considered failed and the group will rebalance in order to reassign the partitions to another member.		Long
maxPollRecords (consumer)	The maximum number of records returned in a single call to <code>poll()</code> .	500	Integer
offsetRepository (consumer)	The offset repository to use in order to locally store the offset of each partition of the topic. Defining one will disable the autocommit.		StateRepository
partitionAssignor (consumer)	The class name of the partition assignment strategy that the client will use to distribute partition ownership amongst consumer instances when group management is used.	<code>org.apache.kafka.clients.consumer.RangeAssignor</code>	String

Name	Description	Default	Type
pollOnError (consumer)	What to do if kafka threw an exception while polling for new messages. Will by default use the value from the component configuration unless an explicit value has been configured on the endpoint level. DISCARD will discard the message and continue to poll next message. ERROR_HANDLER will use Camel's error handler to process the exception, and afterwards continue to poll next message. RECONNECT will re-connect the consumer and try poll the message again. RETRY will let the consumer retry polling the same message again. STOP will stop the consumer (have to be manually started/restarted if the consumer should be able to consume messages again). Enum values: ● DISCARD ● ERROR_HANDLER ● RECONNECT ● RETRY ● STOP	ERROR_HANDLER	PollOnError
pollTimeoutMs (consumer)	The timeout used when polling the KafkaConsumer.	5000	Long
preValidateHostAndPort (consumer)	Whether to eager validate that broker host:port is valid and can be DNS resolved to known host during starting this consumer. If the validation fails, then an exception is thrown, which makes Camel fail fast. Disabling this will postpone the validation after the consumer is started, and Camel will keep re-connecting in case of validation or DNS resolution error.	true	boolean
seekTo (consumer)	Set if KafkaConsumer will read from beginning or end on startup: beginning : read from beginning end : read from end This is replacing the earlier property seekToBeginning. Enum values: ● beginning ● end		String
sessionTimeoutMs (consumer)	The timeout used to detect failures when using Kafka's group management facilities.	10000	Integer

Name	Description	Default	Type
specificAvroReader (consumer)	This enables the use of a specific Avro reader for use with the Confluent Platform schema registry and the <code>io.confluent.kafka.serializers.KafkaAvroDeserializer</code> . This option is only available in the Confluent Platform (not standard Apache Kafka).	false	boolean
topicsPattern (consumer)	Whether the topic is a pattern (regular expression). This can be used to subscribe to dynamic number of topics matching the pattern.	false	boolean
valueDeserializer (consumer)	Deserializer class for value that implements the <code>Deserializer</code> interface.	<code>org.apache.kafka.common.serialization.StringDeserializer</code>	String
createConsumerBackoffInterval (consumer (advanced))	The delay in millis seconds to wait before trying again to create the kafka consumer (kafka-client).	5000	long
createConsumerBackoffMaxAttempts (consumer (advanced))	Maximum attempts to create the kafka consumer (kafka-client), before eventually giving up and failing. Error during creating the consumer may be fatal due to invalid configuration and as such recovery is not possible. However, one part of the validation is DNS resolution of the bootstrap broker hostnames. This may be a temporary networking problem, and could potentially be recoverable. While other errors are fatal, such as some invalid kafka configurations. Unfortunately, kafka-client does not separate this kind of errors. Camel will by default retry forever, and therefore never give up. If you want to give up after many attempts then set this option and Camel will then when giving up terminate the consumer. To try again, you can manually restart the consumer by stopping, and starting the route.		int

Name	Description	Default	Type
isolationLevel (consumer (advanced))	Controls how to read messages written transactionally. If set to <code>read_committed</code>, <code>consumer.poll()</code> will only return transactional messages which have been committed. If set to <code>read_uncommitted</code> (the default), <code>consumer.poll()</code> will return all messages, even transactional messages which have been aborted. Non-transactional messages will be returned unconditionally in either mode. Messages will always be returned in offset order. Hence, in <code>read_committed</code> mode, <code>consumer.poll()</code> will only return messages up to the last stable offset (LSO), which is the one less than the offset of the first open transaction. In particular, any messages appearing after messages belonging to ongoing transactions will be withheld until the relevant transaction has been completed. As a result, <code>read_committed</code> consumers will not be able to read up to the high watermark when there are in flight transactions. Further, when in <code>read_committed</code> the <code>seekToEnd</code> method will return the LSO. Enum values: • <code>read_uncommitted</code> • <code>read_committed</code>		String
kafkaManualCommitFactory (consumer (advanced))	Autowired Factory to use for creating <code>KafkaManualCommit</code> instances. This allows to plugin a custom factory to create custom <code>KafkaManualCommit</code> instances in case special logic is needed when doing manual commits that deviates from the default implementation that comes out of the box.		<code>KafkaManualCommitFactory</code>
pollExceptionStrategy (consumer (advanced))	Autowired To use a custom strategy with the consumer to control how to handle exceptions thrown from the Kafka broker while pooling messages.		<code>PollExceptionStrategy</code>
subscribeConsumerBackoffInterval (consumer (advanced))	The delay in millis seconds to wait before trying again to subscribe to the kafka broker.	5000	long

Name	Description	Default	Type
subscribeConsumerBackoffMaxAttempts (consumer (advanced))	Maximum number the kafka consumer will attempt to subscribe to the kafka broker, before eventually giving up and failing. Error during subscribing the consumer to the kafka topic could be temporary errors due to network issues, and could potentially be recoverable. Camel will by default retry forever, and therefore never give up. If you want to give up after many attempts, then set this option and Camel will then when giving up terminate the consumer. You can manually restart the consumer by stopping and starting the route, to try again.		int
batchWithIndividualHeaders (producer)	If this feature is enabled and a single element of a batch is an Exchange or Message, the producer will generate individual kafka header values for it by using the batch Message to determine the values. Normal behavior consists of always using the same header values (which are determined by the parent Exchange which contains the Iterable or Iterator).	false	boolean
bufferMemorySize (producer)	The total bytes of memory the producer can use to buffer records waiting to be sent to the server. If records are sent faster than they can be delivered to the server the producer will either block or throw an exception based on the preference specified by <code>block.on.buffer.full</code> . This setting should correspond roughly to the total memory the producer will use, but is not a hard bound since not all memory the producer uses is used for buffering. Some additional memory will be used for compression (if compression is enabled) as well as for maintaining in-flight requests.	33554432	Integer
compressionCodec (producer)	This parameter allows you to specify the compression codec for all data generated by this producer. Valid values are none, gzip and snappy. Enum values: • none • gzip • snappy • lz4	none	String
connectionMaxIdleMs (producer)	Close idle connections after the number of milliseconds specified by this config.	540000	Integer

Name	Description	Default	Type
deliveryTimeoutMs (producer)	An upper bound on the time to report success or failure after a call to send() returns. This limits the total time that a record will be delayed prior to sending, the time to await acknowledgement from the broker (if expected), and the time allowed for retrievable send failures.	120000	Integer
enableIdempotence (producer)	If set to 'true' the producer will ensure that exactly one copy of each message is written in the stream. If 'false', producer retries may write duplicates of the retried message in the stream. If set to true this option will require max.in.flight.requests.per.connection to be set to 1 and retries cannot be zero and additionally acks must be set to 'all'.	false	boolean
headerSerializer (producer)	To use a custom KafkaHeaderSerializer to serialize kafka headers values.		KafkaHeaderSerializer
key (producer)	The record key (or null if no key is specified). If this option has been configured then it take precedence over header KafkaConstants#KEY.		String
keySerializer (producer)	The serializer class for keys (defaults to the same as for messages if nothing is given).	org.apache.kafka.common.serialization.StringSerializer	String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
lingerMs (producer)	The producer groups together any records that arrive in between request transmissions into a single batched request. Normally this occurs only under load when records arrive faster than they can be sent out. However in some circumstances the client may want to reduce the number of requests even under moderate load. This setting accomplishes this by adding a small amount of artificial delay that is, rather than immediately sending out a record the producer will wait for up to the given delay to allow other records to be sent so that the sends can be batched together. This can be thought of as analogous to Nagle's algorithm in TCP. This setting gives the upper bound on the delay for batching: once we get batch.size worth of records for a partition it will be sent immediately regardless of this setting, however if we have fewer than this many bytes accumulated for this partition we will 'linger' for the specified time waiting for more records to show up. This setting defaults to 0 (i.e. no delay). Setting <code>linger.ms=5</code> , for example, would have the effect of reducing the number of requests sent but would add up to 5ms of latency to records sent in the absense of load.	0	Integer
maxBlockMs (producer)	The configuration controls how long sending to kafka will block. These methods can be blocked for multiple reasons. For e.g: buffer full, metadata unavailable. This configuration imposes maximum limit on the total time spent in fetching metadata, serialization of key and value, partitioning and allocation of buffer memory when doing a <code>send()</code> . In case of <code>partitionsFor()</code> , this configuration imposes a maximum time threshold on waiting for metadata.	60000	Integer
maxInFlightRequest (producer)	The maximum number of unacknowledged requests the client will send on a single connection before blocking. Note that if this setting is set to be greater than 1 and there are failed sends, there is a risk of message re-ordering due to retries (i.e., if retries are enabled).	5	Integer
maxRequestSize (producer)	The maximum size of a request. This is also effectively a cap on the maximum record size. Note that the server has its own cap on record size which may be different from this. This setting will limit the number of record batches the producer will send in a single request to avoid sending huge requests.	1048576	Integer

Name	Description	Default	Type
metadataMaxAgeMs (producer)	The period of time in milliseconds after which we force a refresh of metadata even if we haven't seen any partition leadership changes to proactively discover any new brokers or partitions.	300000	Integer
metricReporters (producer)	A list of classes to use as metrics reporters. Implementing the MetricReporter interface allows plugging in classes that will be notified of new metric creation. The JmxReporter is always included to register JMX statistics.		String
metricsSampleWindowMs (producer)	The number of samples maintained to compute metrics.	30000	Integer
noOfMetricsSample (producer)	The number of samples maintained to compute metrics.	2	Integer
partitioner (producer)	The partitioner class for partitioning messages amongst sub-topics. The default partitioner is based on the hash of the key.	org.apache.kafka.clients.producer.internals.DefaultPartitioner	String
partitionerIgnoreKeys (producer)	Whether the message keys should be ignored when computing the partition. This setting has effect only when partitioner is not set.	false	boolean
partitionKey (producer)	The partition to which the record will be sent (or null if no partition was specified). If this option has been configured then it take precedence over header <code>KafkaConstants#PARTITION_KEY</code> .		Integer

Name	Description	Default	Type
producerBatchSize (producer)	The producer will attempt to batch records together into fewer requests whenever multiple records are being sent to the same partition. This helps performance on both the client and the server. This configuration controls the default batch size in bytes. No attempt will be made to batch records larger than this size. Requests sent to brokers will contain multiple batches, one for each partition with data available to be sent. A small batch size will make batching less common and may reduce throughput (a batch size of zero will disable batching entirely). A very large batch size may use memory a bit more wastefully as we will always allocate a buffer of the specified batch size in anticipation of additional records.	16384	Integer
queueBufferingMaxMessages (producer)	The maximum number of unsent messages that can be queued up the producer when using async mode before either the producer must be blocked or data must be dropped.	10000	Integer
receiveBufferBytes (producer)	The size of the TCP receive buffer (SO_RCVBUF) to use when reading data.	65536	Integer
reconnectBackoffMs (producer)	The amount of time to wait before attempting to reconnect to a given host. This avoids repeatedly connecting to a host in a tight loop. This backoff applies to all requests sent by the consumer to the broker.	50	Integer
recordMetadata (producer)	Whether the producer should store the RecordMetadata results from sending to Kafka. The results are stored in a List containing the RecordMetadata metadata's. The list is stored on a header with the key <code>KafkaConstants#KAFKA_RECORDMETA</code> .	true	boolean

Name	Description	Default	Type
requestRequiredAcks (producer)	The number of acknowledgments the producer requires the leader to have received before considering a request complete. This controls the durability of records that are sent. The following settings are common: <code>acks=0</code> If set to zero then the producer will not wait for any acknowledgment from the server at all. The record will be immediately added to the socket buffer and considered sent. No guarantee can be made that the server has received the record in this case, and the retries configuration will not take effect (as the client won't generally know of any failures). The offset given back for each record will always be set to <code>-1</code>. <code>acks=1</code> This will mean the leader will write the record to its local log but will respond without awaiting full acknowledgement from all followers. In this case should the leader fail immediately after acknowledging the record but before the followers have replicated it then the record will be lost. <code>acks=all</code> This means the leader will wait for the full set of in-sync replicas to acknowledge the record. This guarantees that the record will not be lost as long as at least one in-sync replica remains alive. This is the strongest available guarantee. Enum values: • <code>-1</code> • <code>0</code> • <code>1</code> • <code>all</code>	1	String
requestTimeoutMs (producer)	The amount of time the broker will wait trying to meet the <code>request.required.acks</code> requirement before sending back an error to the client.	30000	Integer
retries (producer)	Setting a value greater than zero will cause the client to resend any record whose send fails with a potentially transient error. Note that this retry is no different than if the client resent the record upon receiving the error. Allowing retries will potentially change the ordering of records because if two records are sent to a single partition, and the first fails and is retried but the second succeeds, then the second record may appear first.	0	Integer

Name	Description	Default	Type
retryBackoffMs (producer)	Before each retry, the producer refreshes the metadata of relevant topics to see if a new leader has been elected. Since leader election takes a bit of time, this property specifies the amount of time that the producer waits before refreshing the metadata.	100	Integer
sendBufferBytes (producer)	Socket write buffer size.	131072	Integer
useIterator (producer)	Sets whether sending to kafka should send the message body as a single record, or use a <code>java.util.Iterator</code> to send multiple records to kafka (if the message body can be iterated).	true	boolean
valueSerializer (producer)	The serializer class for messages.	<code>org.apache.kafka.common.serialization.StringSerializer</code>	String
workerPool (producer)	To use a custom worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from <code>KafkaProducer</code> using asynchronous non-blocking processing. If using this option then you must handle the lifecycle of the thread pool to shut the pool down when no longer needed.		<code>ExecutorService</code>
workerPoolCoreSize (producer)	Number of core threads for the worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from <code>KafkaProducer</code> using asynchronous non-blocking processing.	10	Integer
workerPoolMaxSize (producer)	Maximum number of threads for the worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from <code>KafkaProducer</code> using asynchronous non-blocking processing.	20	Integer

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
kafkaClientFactory (advanced)	Autowired Factory to use for creating org.apache.kafka.clients.consumer.KafkaConsumer and org.apache.kafka.clients.producer.KafkaProducer instances. This allows to configure a custom factory to create instances with logic that extends the vanilla Kafka clients.		KafkaClientFactory
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean
healthCheckConsumerEnabled (health)	Used for enabling or disabling all consumer based health checks from this component.	true	boolean
healthCheckProducerEnabled (health)	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting camel.health.producersEnabled=true.	true	boolean
interceptorClasses (monitoring)	Sets interceptors for producer or consumers. Producer interceptors have to be classes implementing org.apache.kafka.clients.producer.ProducerInterceptor or Consumer interceptors have to be classes implementing org.apache.kafka.clients.consumer.ConsumerInterceptor. Note that if you use Producer interceptor on a consumer it will throw a class cast exception in runtime.		String
schemaRegistryURL (confluent)	URL of the Confluent Platform schema registry servers to use. The format is host1:port1,host2:port2. This is known as schema.registry.url in the Confluent Platform documentation. This option is only available in the Confluent Platform (not standard Apache Kafka).		String
kerberosBeforeReloginMinTime (security)	Login thread sleep time between refresh attempts.	60000	Integer

Name	Description	Default	Type
kerberosInitCmd (security)	Kerberos kinit command path. Default is /usr/bin/kinit.	/usr/bin/kinit	String
kerberosPrincipalToLocalRules (security)	A list of rules for mapping from principal names to short names (typically operating system usernames). The rules are evaluated in order and the first rule that matches a principal name is used to map it to a short name. Any later rules in the list are ignored. By default, principal names of the form {username}/{hostname}{REALM} are mapped to {username}. For more details on the format please see the security authorization and acls documentation.. Multiple values can be separated by comma.	DEFAULT	String
kerberosRenewJitter (security)	Percentage of random jitter added to the renewal time.	0.05	Double
kerberosRenewWindowFactor (security)	Login thread will sleep until the specified window factor of time from last refresh to ticket's expiry has been reached, at which time it will try to renew the ticket.	0.8	Double
saslJaasConfig (security)	Expose the kafka sasl.jaas.config parameter Example: org.apache.kafka.common.security.plain.PlainLoginModule required username=USERNAME password=PASSWORD;.		String
saslKerberosServiceName (security)	The Kerberos principal name that Kafka runs as. This can be defined either in Kafka's JAAS config or in Kafka's config.		String
saslMechanism (security)	The Simple Authentication and Security Layer (SASL) Mechanism used. For the valid values see .	GSSAPI	String
securityProtocol (security)	Protocol used to communicate with brokers. SASL_PLAINTEXT, PLAINTEXT and SSL are supported.	PLAINTEXT	String
sslCipherSuites (security)	A list of cipher suites. This is a named combination of authentication, encryption, MAC and key exchange algorithm used to negotiate the security settings for a network connection using TLS or SSL network protocol.By default all the available cipher suites are supported.		String

Name	Description	Default	Type
sslContextParameters (security)	SSL configuration using a Camel SSLContextParameters object. If configured it's applied before the other SSL endpoint parameters. NOTE: Kafka only supports loading keystore from file locations, so prefix the location with file: in the KeyStoreParameters.resource option.		SSLContextParameters
sslEnabledProtocols (security)	The list of protocols enabled for SSL connections. TLSv1.2, TLSv1.1 and TLSv1 are enabled by default.		String
sslEndpointAlgorithm (security)	The endpoint identification algorithm to validate server hostname using server certificate.	https	String
sslKeymanagerAlgorithm (security)	The algorithm used by key manager factory for SSL connections. Default value is the key manager factory algorithm configured for the Java Virtual Machine.	SunX509	String
sslKeyPassword (security)	The password of the private key in the key store file. This is optional for client.		String
sslKeystoreLocation (security)	The location of the key store file. This is optional for client and can be used for two-way authentication for client.		String
sslKeystorePassword (security)	The store password for the key store file. This is optional for client and only needed if ssl.keystore.location is configured.		String
sslKeystoreType (security)	The file format of the key store file. This is optional for client. Default value is JKS.	JKS	String
sslProtocol (security)	The SSL protocol used to generate the SSLContext. Default setting is TLS, which is fine for most cases. Allowed values in recent JVMs are TLS, TLSv1.1 and TLSv1.2. SSL, SSLv2 and SSLv3 may be supported in older JVMs, but their usage is discouraged due to known security vulnerabilities.		String
sslProvider (security)	The name of the security provider used for SSL connections. Default value is the default security provider of the JVM.		String
sslTrustmanagerAlgorithm (security)	The algorithm used by trust manager factory for SSL connections. Default value is the trust manager factory algorithm configured for the Java Virtual Machine.	PKIX	String

Name	Description	Default	Type
sslTruststoreLocation (security)	The location of the trust store file.		String
sslTruststorePassword (security)	The password for the trust store file.		String
sslTruststoreType (security)	The file format of the trust store file. Default value is JKS.	JKS	String
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean

66.5. ENDPOINT OPTIONS

The Kafka endpoint is configured using URI syntax:

```
kafka:topic
```

with the following path and query parameters:

66.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
topic (common)	Required Name of the topic to use. On the consumer you can use comma to separate multiple topics. A producer can only send a message to a single topic.		String

66.5.2. Query Parameters (108 parameters)

Name	Description	Default	Type
additionalProperties (common)	Sets additional properties for either kafka consumer or kafka producer in case they can't be set directly on the camel configurations (e.g: new Kafka properties that are not reflected yet in Camel configurations), the properties have to be prefixed with <code>additionalProperties.</code> . E.g: <code>additionalProperties.transactional.id=12345&additionalProperties.schema.registry.url=http://localhost:8811/avro.</code>		Map

Name	Description	Default	Type
brokers (common)	URL of the Kafka brokers to use. The format is host1:port1,host2:port2, and the list can be a subset of brokers or a VIP pointing to a subset of brokers. This option is known as bootstrap.servers in the Kafka documentation.		String
clientId (common)	The client id is a user-specified string sent in each request to help trace calls. It should logically identify the application making the request.		String
headerFilterStrategy (common)	To use a custom HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
reconnectBackoffMaxMs (common)	The maximum amount of time in milliseconds to wait when reconnecting to a broker that has repeatedly failed to connect. If provided, the backoff per host will increase exponentially for each consecutive connection failure, up to this maximum. After calculating the backoff increase, 20% random jitter is added to avoid connection storms.	1000	Integer
shutdownTimeout (common)	Timeout in milliseconds to wait gracefully for the consumer or producer to shutdown and terminate its worker threads.	30000	int
allowManualCommit (consumer)	Whether to allow doing manual commits via KafkaManualCommit. If this option is enabled then an instance of KafkaManualCommit is stored on the Exchange message header, which allows end users to access this API and perform manual offset commits via the Kafka consumer.	false	boolean
autoCommitEnable (consumer)	If true, periodically commit to ZooKeeper the offset of messages already fetched by the consumer. This committed offset will be used when the process fails as the position from which the new consumer will begin.	true	Boolean
autoCommitIntervalMs (consumer)	The frequency in ms that the consumer offsets are committed to zookeeper.	5000	Integer

Name	Description	Default	Type
autoCommitOnStop (consumer)	Whether to perform an explicit auto commit when the consumer stops to ensure the broker has a commit from the last consumed message. This requires the option <code>autoCommitEnable</code> is turned on. The possible values are: <code>sync</code>, <code>async</code>, or <code>none</code>. And <code>sync</code> is the default value. Enum values: • <code>sync</code> • <code>async</code> • <code>none</code>	<code>sync</code>	String
autoOffsetReset (consumer)	What to do when there is no initial offset in ZooKeeper or if an offset is out of range: <code>earliest</code> : automatically reset the offset to the earliest offset <code>latest</code> : automatically reset the offset to the latest offset <code>fail</code>: throw exception to the consumer. Enum values: • <code>latest</code> • <code>earliest</code> • <code>none</code>	<code>latest</code>	String
batching (consumer)	Whether to use batching for processing or streaming. The default is <code>false</code> , which uses streaming.	<code>false</code>	boolean
breakOnFirstError (consumer)	This options controls what happens when a consumer is processing an exchange and it fails. If the option is <code>false</code> then the consumer continues to the next message and processes it. If the option is <code>true</code> then the consumer breaks out, and will seek back to offset of the message that caused a failure, and then re-attempt to process this message. However this can lead to endless processing of the same message if its bound to fail every time, eg a poison message. Therefore its recommended to deal with that for example by using Camel's error handler.	<code>false</code>	boolean

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
checkCrcs (consumer)	Automatically check the CRC32 of the records consumed. This ensures no on-the-wire or on-disk corruption to the messages occurred. This check adds some overhead, so it may be disabled in cases seeking extreme performance.	true	Boolean
commitTimeoutMs (consumer)	The maximum time, in milliseconds, that the code will wait for a synchronous commit to complete.	5000	Long
consumerRequestTimeoutMs (consumer)	The configuration controls the maximum amount of time the client will wait for the response of a request. If the response is not received before the timeout elapses the client will resend the request if necessary or fail the request if retries are exhausted.	40000	Integer
consumersCount (consumer)	The number of consumers that connect to kafka server. Each consumer is run on a separate thread, that retrieves and process the incoming data.	1	int
fetchMaxBytes (consumer)	The maximum amount of data the server should return for a fetch request This is not an absolute maximum, if the first message in the first non-empty partition of the fetch is larger than this value, the message will still be returned to ensure that the consumer can make progress. The maximum message size accepted by the broker is defined via <code>message.max.bytes</code> (broker config) or <code>max.message.bytes</code> (topic config). Note that the consumer performs multiple fetches in parallel.	52428800	Integer
fetchMinBytes (consumer)	The minimum amount of data the server should return for a fetch request. If insufficient data is available the request will wait for that much data to accumulate before answering the request.	1	Integer
fetchWaitMaxMs (consumer)	The maximum amount of time the server will block before answering the fetch request if there isn't sufficient data to immediately satisfy <code>fetch.min.bytes</code> .	500	Integer

Name	Description	Default	Type
groupId (consumer)	A string that uniquely identifies the group of consumer processes to which this consumer belongs. By setting the same group id multiple processes indicate that they are all part of the same consumer group. This option is required for consumers.		String
groupId (consumer)	A unique identifier of the consumer instance provided by the end user. Only non-empty strings are permitted. If set, the consumer is treated as a static member, which means that only one instance with this ID is allowed in the consumer group at any time. This can be used in combination with a larger session timeout to avoid group rebalances caused by transient unavailability (e.g. process restarts). If not set, the consumer will join the group as a dynamic member, which is the traditional behavior.		String
headerDeserializer (consumer)	To use a custom <code>KafkaHeaderDeserializer</code> to deserialize kafka headers values.		<code>KafkaHeaderDeserializer</code>
heartbeatIntervalMs (consumer)	The expected time between heartbeats to the consumer coordinator when using Kafka's group management facilities. Heartbeats are used to ensure that the consumer's session stays active and to facilitate rebalancing when new consumers join or leave the group. The value must be set lower than <code>session.timeout.ms</code> , but typically should be set no higher than 1/3 of that value. It can be adjusted even lower to control the expected time for normal rebalances.	3000	Integer
keyDeserializer (consumer)	Deserializer class for key that implements the <code>Deserializer</code> interface.	<code>org.apache.kafka.common.serialization.StringDeserializer</code>	String

Name	Description	Default	Type
maxPartitionFetchBytes (consumer)	The maximum amount of data per-partition the server will return. The maximum total memory used for a request will be <code>#partitions max.partition.fetch.bytes</code> . This size must be at least as large as the maximum message size the server allows or else it is possible for the producer to send messages larger than the consumer can fetch. If that happens, the consumer can get stuck trying to fetch a large message on a certain partition.	1048576	Integer
maxPollIntervalMs (consumer)	The maximum delay between invocations of <code>poll()</code> when using consumer group management. This places an upper bound on the amount of time that the consumer can be idle before fetching more records. If <code>poll()</code> is not called before expiration of this timeout, then the consumer is considered failed and the group will rebalance in order to reassign the partitions to another member.		Long
maxPollRecords (consumer)	The maximum number of records returned in a single call to <code>poll()</code> .	500	Integer
offsetRepository (consumer)	The offset repository to use in order to locally store the offset of each partition of the topic. Defining one will disable the autocommit.		StateRepository
partitionAssignor (consumer)	The class name of the partition assignment strategy that the client will use to distribute partition ownership amongst consumer instances when group management is used.	org.apache.kafka.clients.consumer.RangeAssignor	String

Name	Description	Default	Type
pollOnError (consumer)	What to do if kafka threw an exception while polling for new messages. Will by default use the value from the component configuration unless an explicit value has been configured on the endpoint level. DISCARD will discard the message and continue to poll next message. ERROR_HANDLER will use Camel's error handler to process the exception, and afterwards continue to poll next message. RECONNECT will reconnect the consumer and try poll the message again. RETRY will let the consumer retry polling the same message again. STOP will stop the consumer (have to be manually started/restarted if the consumer should be able to consume messages again). Enum values: ● DISCARD ● ERROR_HANDLER ● RECONNECT ● RETRY ● STOP	ERROR_HANDLER	PollOnError
pollTimeoutMs (consumer)	The timeout used when polling the KafkaConsumer.	5000	Long
preValidateHostAndPort (consumer)	Whether to eager validate that broker host:port is valid and can be DNS resolved to known host during starting this consumer. If the validation fails, then an exception is thrown, which makes Camel fail fast. Disabling this will postpone the validation after the consumer is started, and Camel will keep reconnecting in case of validation or DNS resolution error.	true	boolean
resumeStrategy (consumer)	This option allows the user to set a custom resume strategy. The resume strategy is executed when partitions are assigned (i.e.: when connecting or reconnecting). It allows implementations to customize how to resume operations and serve as more flexible alternative to the seekTo and the offsetRepository mechanisms. See the KafkaConsumerResumeStrategy for implementation details. This option does not affect the auto commit setting. It is likely that implementations using this setting will also want to evaluate using the manual commit option along with this.		KafkaConsumerResumeStrategy

Name	Description	Default	Type
seekTo (consumer)	Set if KafkaConsumer will read from beginning or end on startup: beginning : read from beginning end : read from end This is replacing the earlier property seekToBeginning. Enum values: - beginning - end		String
sessionTimeoutMs (consumer)	The timeout used to detect failures when using Kafka's group management facilities.	10000	Integer
specificAvroReader (consumer)	This enables the use of a specific Avro reader for use with the Confluent Platform schema registry and the io.confluent.kafka.serializers.KafkaAvroDeserializer. This option is only available in the Confluent Platform (not standard Apache Kafka).	false	boolean
topicsPattern (consumer)	Whether the topic is a pattern (regular expression). This can be used to subscribe to dynamic number of topics matching the pattern.	false	boolean
valueDeserializer (consumer)	Deserializer class for value that implements the Deserializer interface.	org.apache.kafka.common.serialization.StringDeserializer	String
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
isolationLevel (consumer (advanced))	Controls how to read messages written transactionally. If set to <code>read_committed</code>, <code>consumer.poll()</code> will only return transactional messages which have been committed. If set to <code>read_uncommitted</code> (the default), <code>consumer.poll()</code> will return all messages, even transactional messages which have been aborted. Non-transactional messages will be returned unconditionally in either mode. Messages will always be returned in offset order. Hence, in <code>read_committed</code> mode, <code>consumer.poll()</code> will only return messages up to the last stable offset (LSO), which is the one less than the offset of the first open transaction. In particular, any messages appearing after messages belonging to ongoing transactions will be withheld until the relevant transaction has been completed. As a result, <code>read_committed</code> consumers will not be able to read up to the high watermark when there are in flight transactions. Further, when in <code>read_committed</code> the <code>seekToEnd</code> method will return the LSO. Enum values: ● <code>read_uncommitted</code> ● <code>read_committed</code>		String
kafkaManualCommitFactory (consumer (advanced))	Factory to use for creating <code>KafkaManualCommit</code> instances. This allows to plugin a custom factory to create custom <code>KafkaManualCommit</code> instances in case special logic is needed when doing manual commits that deviates from the default implementation that comes out of the box.		KafkaManualCommitFactory

Name	Description	Default	Type
batchWithIndividualHeaders (producer)	If this feature is enabled and a single element of a batch is an Exchange or Message, the producer will generate individual kafka header values for it by using the batch Message to determine the values. Normal behavior consists of always using the same header values (which are determined by the parent Exchange which contains the Iterable or Iterator).	false	boolean
bufferMemorySize (producer)	The total bytes of memory the producer can use to buffer records waiting to be sent to the server. If records are sent faster than they can be delivered to the server the producer will either block or throw an exception based on the preference specified by <code>block.on.buffer.full</code> . This setting should correspond roughly to the total memory the producer will use, but is not a hard bound since not all memory the producer uses is used for buffering. Some additional memory will be used for compression (if compression is enabled) as well as for maintaining in-flight requests.	33554432	Integer
compressionCodec (producer)	This parameter allows you to specify the compression codec for all data generated by this producer. Valid values are none, gzip and snappy. Enum values: • none • gzip • snappy • lz4	none	String
connectionMaxIdleMs (producer)	Close idle connections after the number of milliseconds specified by this config.	540000	Integer
deliveryTimeoutMs (producer)	An upper bound on the time to report success or failure after a call to <code>send()</code> returns. This limits the total time that a record will be delayed prior to sending, the time to await acknowledgement from the broker (if expected), and the time allowed for retrievable send failures.	120000	Integer

Name	Description	Default	Type
enableIdempotence (producer)	If set to 'true' the producer will ensure that exactly one copy of each message is written in the stream. If 'false', producer retries may write duplicates of the retried message in the stream. If set to true this option will require <code>max.in.flight.requests.per.connection</code> to be set to 1 and retries cannot be zero and additionally acks must be set to 'all'.	false	boolean
headerSerializer (producer)	To use a custom <code>KafkaHeaderSerializer</code> to serialize kafka headers values.		<code>KafkaHeaderSerializer</code>
key (producer)	The record key (or null if no key is specified). If this option has been configured then it take precedence over header <code>KafkaConstants#KEY</code> .		String
keySerializer (producer)	The serializer class for keys (defaults to the same as for messages if nothing is given).	<code>org.apache.kafka.common.serialization.StringSerializer</code>	String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
lingerMs (producer)	The producer groups together any records that arrive in between request transmissions into a single batched request. Normally this occurs only under load when records arrive faster than they can be sent out. However in some circumstances the client may want to reduce the number of requests even under moderate load. This setting accomplishes this by adding a small amount of artificial delay that is, rather than immediately sending out a record the producer will wait for up to the given delay to allow other records to be sent so that the sends can be batched together. This can be thought of as analogous to Nagle's algorithm in TCP. This setting gives the upper bound on the delay for batching: once we get batch.size worth of records for a partition it will be sent immediately regardless of this setting, however if we have fewer than this many bytes accumulated for this partition we will 'linger' for the specified time waiting for more records to show up. This setting defaults to 0 (i.e. no delay). Setting linger.ms=5, for example, would have the effect of reducing the number of requests sent but would add up to 5ms of latency to records sent in the absense of load.	0	Integer
maxBlockMs (producer)	The configuration controls how long sending to kafka will block. These methods can be blocked for multiple reasons. For e.g: buffer full, metadata unavailable.This configuration imposes maximum limit on the total time spent in fetching metadata, serialization of key and value, partitioning and allocation of buffer memory when doing a send(). In case of partitionsFor(), this configuration imposes a maximum time threshold on waiting for metadata.	60000	Integer
maxInFlightRequest (producer)	The maximum number of unacknowledged requests the client will send on a single connection before blocking. Note that if this setting is set to be greater than 1 and there are failed sends, there is a risk of message re-ordering due to retries (i.e., if retries are enabled).	5	Integer
maxRequestSize (producer)	The maximum size of a request. This is also effectively a cap on the maximum record size. Note that the server has its own cap on record size which may be different from this. This setting will limit the number of record batches the producer will send in a single request to avoid sending huge requests.	1048576	Integer

Name	Description	Default	Type
metadataMaxAgeMs (producer)	The period of time in milliseconds after which we force a refresh of metadata even if we haven't seen any partition leadership changes to proactively discover any new brokers or partitions.	300000	Integer
metricReporters (producer)	A list of classes to use as metrics reporters. Implementing the MetricReporter interface allows plugging in classes that will be notified of new metric creation. The JmxReporter is always included to register JMX statistics.		String
metricsSampleWindowMs (producer)	The number of samples maintained to compute metrics.	30000	Integer
noOfMetricsSample (producer)	The number of samples maintained to compute metrics.	2	Integer
partitioner (producer)	The partitioner class for partitioning messages amongst sub-topics. The default partitioner is based on the hash of the key.	org.apache.kafka.clients.producer.internals.DefaultPartitioner	String
partitionKey (producer)	The partition to which the record will be sent (or null if no partition was specified). If this option has been configured then it take precedence over header <code>KafkaConstants#PARTITION_KEY</code> .		Integer
producerBatchSize (producer)	The producer will attempt to batch records together into fewer requests whenever multiple records are being sent to the same partition. This helps performance on both the client and the server. This configuration controls the default batch size in bytes. No attempt will be made to batch records larger than this size. Requests sent to brokers will contain multiple batches, one for each partition with data available to be sent. A small batch size will make batching less common and may reduce throughput (a batch size of zero will disable batching entirely). A very large batch size may use memory a bit more wastefully as we will always allocate a buffer of the specified batch size in anticipation of additional records.	16384	Integer

Name	Description	Default	Type
queueBufferingMaxMessages (producer)	The maximum number of unsent messages that can be queued up the producer when using async mode before either the producer must be blocked or data must be dropped.	10000	Integer
receiveBufferSize (producer)	The size of the TCP receive buffer (SO_RCVBUF) to use when reading data.	65536	Integer
reconnectBackoffMs (producer)	The amount of time to wait before attempting to reconnect to a given host. This avoids repeatedly connecting to a host in a tight loop. This backoff applies to all requests sent by the consumer to the broker.	50	Integer
recordMetadata (producer)	Whether the producer should store the RecordMetadata results from sending to Kafka. The results are stored in a List containing the RecordMetadata metadata's. The list is stored on a header with the key <code>KafkaConstants#KAFKA_RECORDMETA</code> .	true	boolean

Name	Description	Default	Type
requestRequiredAcks (producer)	The number of acknowledgments the producer requires the leader to have received before considering a request complete. This controls the durability of records that are sent. The following settings are common: <code>acks=0</code> If set to zero then the producer will not wait for any acknowledgment from the server at all. The record will be immediately added to the socket buffer and considered sent. No guarantee can be made that the server has received the record in this case, and the retries configuration will not take effect (as the client won't generally know of any failures). The offset given back for each record will always be set to <code>-1</code>. <code>acks=1</code> This will mean the leader will write the record to its local log but will respond without awaiting full acknowledgement from all followers. In this case should the leader fail immediately after acknowledging the record but before the followers have replicated it then the record will be lost. <code>acks=all</code> This means the leader will wait for the full set of in-sync replicas to acknowledge the record. This guarantees that the record will not be lost as long as at least one in-sync replica remains alive. This is the strongest available guarantee. Enum values: • <code>-1</code> • <code>0</code> • <code>1</code> • <code>all</code>	1	String
requestTimeoutMs (producer)	The amount of time the broker will wait trying to meet the <code>request.required.acks</code> requirement before sending back an error to the client.	30000	Integer
retries (producer)	Setting a value greater than zero will cause the client to resend any record whose send fails with a potentially transient error. Note that this retry is no different than if the client resent the record upon receiving the error. Allowing retries will potentially change the ordering of records because if two records are sent to a single partition, and the first fails and is retried but the second succeeds, then the second record may appear first.	0	Integer

Name	Description	Default	Type
retryBackoffMs (producer)	Before each retry, the producer refreshes the metadata of relevant topics to see if a new leader has been elected. Since leader election takes a bit of time, this property specifies the amount of time that the producer waits before refreshing the metadata.	100	Integer
sendBufferBytes (producer)	Socket write buffer size.	131072	Integer
valueSerializer (producer)	The serializer class for messages.	org.apache.kafka.common.serialization.StringSerializer	String
workerPool (producer)	To use a custom worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from KafkaProducer using asynchronous non-blocking processing. If using this option then you must handle the lifecycle of the thread pool to shut the pool down when no longer needed.		ExecutorService
workerPoolCoreSize (producer)	Number of core threads for the worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from KafkaProducer using asynchronous non-blocking processing.	10	Integer
workerPoolMaxSize (producer)	Maximum number of threads for the worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from KafkaProducer using asynchronous non-blocking processing.	20	Integer
kafkaClientFactory (advanced)	Factory to use for creating org.apache.kafka.clients.consumer.KafkaConsumer and org.apache.kafka.clients.producer.KafkaProducer instances. This allows to configure a custom factory to create instances with logic that extends the vanilla Kafka clients.		KafkaClientFactory
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean

Name	Description	Default	Type
schemaRegistryURL (confluent)	URL of the Confluent Platform schema registry servers to use. The format is host1:port1,host2:port2. This is known as schema.registry.url in the Confluent Platform documentation. This option is only available in the Confluent Platform (not standard Apache Kafka).		String
interceptorClasses (monitoring)	Sets interceptors for producer or consumers. Producer interceptors have to be classes implementing <code>org.apache.kafka.clients.producer.ProducerInterceptor</code> or Consumer interceptors have to be classes implementing <code>org.apache.kafka.clients.consumer.ConsumerInterceptor</code> . Note that if you use Producer interceptor on a consumer it will throw a class cast exception in runtime.		String
kerberosBeforeRefreshMinTime (security)	Login thread sleep time between refresh attempts.	60000	Integer
kerberosInitCmd (security)	Kerberos kinit command path. Default is <code>/usr/bin/kinit</code> .	<code>/usr/bin/kinit</code>	String
kerberosPrincipalToLocalRules (security)	A list of rules for mapping from principal names to short names (typically operating system usernames). The rules are evaluated in order and the first rule that matches a principal name is used to map it to a short name. Any later rules in the list are ignored. By default, principal names of the form <code>{username}/{hostname}{REALM}</code> are mapped to <code>{username}</code> . For more details on the format please see the security authorization and acls documentation.. Multiple values can be separated by comma.	DEFAULT	String
kerberosRenewJitter (security)	Percentage of random jitter added to the renewal time.	0.05	Double
kerberosRenewWindowFactor (security)	Login thread will sleep until the specified window factor of time from last refresh to ticket's expiry has been reached, at which time it will try to renew the ticket.	0.8	Double
saslJaasConfig (security)	Expose the kafka sasl.jaas.config parameter Example: <code>org.apache.kafka.common.security.plain.PlainLoginModule required username=USERNAME password=PASSWORD;</code> .		String

Name	Description	Default	Type
saslKerberosServiceName (security)	The Kerberos principal name that Kafka runs as. This can be defined either in Kafka's JAAS config or in Kafka's config.		String
saslMechanism (security)	The Simple Authentication and Security Layer (SASL) Mechanism used. For the valid values see .	GSSAPI	String
securityProtocol (security)	Protocol used to communicate with brokers. SASL_PLAINTEXT, PLAINTEXT and SSL are supported.	PLAINTEXT	String
sslCipherSuites (security)	A list of cipher suites. This is a named combination of authentication, encryption, MAC and key exchange algorithm used to negotiate the security settings for a network connection using TLS or SSL network protocol. By default all the available cipher suites are supported.		String
sslContextParameters (security)	SSL configuration using a Camel SSLContextParameters object. If configured it's applied before the other SSL endpoint parameters. NOTE: Kafka only supports loading keystore from file locations, so prefix the location with file: in the KeyStoreParameters.resource option.		SSLContextParameters
sslEnabledProtocols (security)	The list of protocols enabled for SSL connections. TLSv1.2, TLSv1.1 and TLSv1 are enabled by default.		String
sslEndpointAlgorithm (security)	The endpoint identification algorithm to validate server hostname using server certificate.	https	String
sslKeymanagerAlgorithm (security)	The algorithm used by key manager factory for SSL connections. Default value is the key manager factory algorithm configured for the Java Virtual Machine.	SunX509	String
sslKeyPassword (security)	The password of the private key in the key store file. This is optional for client.		String
sslKeystoreLocation (security)	The location of the key store file. This is optional for client and can be used for two-way authentication for client.		String
sslKeystorePassword (security)	The store password for the key store file. This is optional for client and only needed if ssl.keystore.location is configured.		String

Name	Description	Default	Type
sslKeystoreType (security)	The file format of the key store file. This is optional for client. Default value is JKS.	JKS	String
sslProtocol (security)	The SSL protocol used to generate the SSLContext. Default setting is TLS, which is fine for most cases. Allowed values in recent JVMs are TLS, TLSv1.1 and TLSv1.2. SSL, SSLv2 and SSLv3 may be supported in older JVMs, but their usage is discouraged due to known security vulnerabilities.		String
sslProvider (security)	The name of the security provider used for SSL connections. Default value is the default security provider of the JVM.		String
sslTrustmanagerAlgorithm (security)	The algorithm used by trust manager factory for SSL connections. Default value is the trust manager factory algorithm configured for the Java Virtual Machine.	PKIX	String
sslTruststoreLocation (security)	The location of the trust store file.		String
sslTruststorePassword (security)	The password for the trust store file.		String
sslTruststoreType (security)	The file format of the trust store file. Default value is JKS.	JKS	String

For more information about Producer/Consumer configuration see:

- <http://kafka.apache.org/documentation.html#newconsumerconfigs>
- <http://kafka.apache.org/documentation.html#producerconfigs>

66.6. MESSAGE HEADERS

The Kafka component supports 13 message header(s), which is/are listed below:

Name	Description	Default	Type
kafka.PARTITION_KEY (producer) Constant: PARTITION_KEY	Explicitly specify the partition.		Integer

Name	Description	Default	Type
kafka.PARTITION (consumer) Constant: PARTITION	The partition where the message was stored.		Integer
kafka.KEY (common) Constant: KEY	Required Producer: The key of the message in order to ensure that all related message goes in the same partition. Consumer: The key of the message if configured.		Object
kafka.TOPIC (consumer) Constant: TOPIC	The topic from where the message originated.		String
kafka.OVERRIDE_TOPIC (producer) Constant: OVERRIDE_TOPIC	The topic to which send the message (override and takes precedence), and the header is not preserved.		String
kafka.OFFSET (consumer) Constant: OFFSET	The offset of the message.		Long
kafka.HEADERS (consumer) Constant: HEADERS	The record headers.		Headers
kafka.LAST_RECORD_BEFORE_COMMIT (consumer) Constant: LAST_RECORD_BEFORE_COMMIT	Whether or not it's the last record before commit (only available if autoCommitEnable endpoint parameter is false).		Boolean
kafka.LAST_POLL_RECORD (consumer) Constant: LAST_POLL_RECORD	Indicates the last record within the current poll request (only available if autoCommitEnable endpoint parameter is false or allowManualCommit is true).		Boolean
kafka.TIMESTAMP (consumer) Constant: TIMESTAMP	The timestamp of the message.		Long

Name	Description	Default	Type
kafka.OVERRIDE_TIME_STAMP (producer) Constant: OVERRIDE_TIMESTAMP	The ProducerRecord also has an associated timestamp. If the user did provide a timestamp, the producer will stamp the record with the provided timestamp and the header is not preserved.		Long
kafka.RECORD_META (producer) Constant: KAFKA_RECORD_META	The metadata (only configured if recordMetadata endpoint parameter is true).		List
CamelKafkaManualCommit (consumer) Constant: MANUAL_COMMIT	Can be used for forcing manual offset commit when using Kafka consumer.		KafkaManualCommit

If you want to send a message to a dynamic topic then use `KafkaConstants.OVERRIDE_TOPIC` as it is used as a one-time header that is not sent along the message, and actually is removed in the producer.

66.7. USAGE

66.7.1. Consumer error handling

While kafka consumer is polling messages from the kafka broker, then errors can happen. This section describes what happens and what you can configure.

The consumer may throw exception when invoking the Kafka **poll** API. For example if the message cannot be de-serialized due invalid data, and many other kind of errors. Those errors are in the form of **KafkaException** which are either *retryable* or not. The exceptions which can be retried (**RetriableException**) will be retried again (with a poll timeout in between). All other kind of exceptions are handled according to the *pollOnError* configuration. This configuration has the following values:

- **DISCARD** will discard the message and continue to poll next message.
- **ERROR_HANDLER** will use Camel's error handler to process the exception, and afterwards continue to poll next message.
- **RECONNECT** will re-connect the consumer and try poll the message again.
- **RETRY** will let the consumer retry polling the same message again
- **STOP** will stop the consumer (have to be manually started/restarted if the consumer should be able to consume messages again).

The default is **ERROR_HANDLER** which will let Camel's error handler (if any configured) process the caused exception. And then afterwards continue to poll the next message. This behavior is similar to the *bridgeErrorHandler* option that Camel components have.

For advanced control then a custom implementation of **org.apache.camel.component.kafka.PollExceptionHandler** can be configured on the component level, which allows controlling which of the strategies to use for each exception.

66.7.2. Consumer error handling (advanced)

By default, Camel will poll using the **ERROR_HANDLER** to process exceptions. How Camel handles a message that results in an exception can be altered using the **breakOnFirstError** attribute in the configuration. Instead of continuing to poll the next message, Camel will instead commit the offset so that the message that caused the exception will be retried. This is similar to the **RETRY** polling strategy above.

```
KafkaComponent kafka = new KafkaComponent();
kafka.setBreakOnFirstError(true);
...
camelContext.addComponent("kafka", kafka);
```

It is recommended that you read the section below "Using manual commit with Kafka consumer" to understand how **breakOnFirstError** will work based on the **CommitManager** that is configured.

66.8. USING THE KAFKA IDEMPOTENT REPOSITORY

The **camel-kafka** library provides a Kafka topic-based idempotent repository.

This repository stores broadcasts all changes to idempotent state (add/remove) in a Kafka topic, and populates a local in-memory cache for each repository's process instance through event sourcing. The topic used must be unique per idempotent repository instance.

The mechanism does not have any requirements about the number of topic partitions; as the repository consumes from all partitions at the same time. It also does not have any requirements about the replication factor of the topic.

Each repository instance that uses the topic (e.g. typically on different machines running in parallel) controls its own consumer group, so in a cluster of 10 Camel processes using the same topic each will control its own offset.

On startup, the instance subscribes to the topic and rewinds the offset to the beginning, rebuilding the cache to the latest state. The cache will not be considered warmed up until one poll of **pollDurationMs** in length returns 0 records. Startup will not be completed until either the cache has warmed up, or 30 seconds go by; if the latter happens the idempotent repository may be in an inconsistent state until its consumer catches up to the end of the topic.

Be mindful of the format of the header used for the uniqueness check. By default, it uses Strings as the data types. When using primitive numeric formats, the header must be deserialized accordingly. Check the samples below for examples.

A **KafkaldempotentRepository** has the following properties:

Property	Description
topic	The name of the Kafka topic to use to broadcast changes. (required)
bootstrapServers	The bootstrap.servers property on the internal Kafka producer and consumer. Use this as shorthand if not setting consumerConfig and producerConfig . If used, this component will apply sensible default configurations for the producer and consumer.
producerConfig	Sets the properties that will be used by the Kafka producer that broadcasts changes. Overrides bootstrapServers , so must define the Kafka bootstrap.servers property itself
consumerConfig	Sets the properties that will be used by the Kafka consumer that populates the cache from the topic. Overrides bootstrapServers , so must define the Kafka bootstrap.servers property itself
maxCacheSize	How many of the most recently used keys should be stored in memory (default 1000).
pollDurationMs	The poll duration of the Kafka consumer. The local caches are updated immediately. This value will affect how far behind other peers that update their caches from the topic are relative to the idempotent consumer instance that sent the cache action message. The default value of this is 100 ms. If setting this value explicitly, be aware that there is a tradeoff between the remote cache liveness and the volume of network traffic between this repository's consumer and the Kafka brokers. The cache warmup process also depends on there being one poll that fetches nothing - this indicates that the stream has been consumed up to the current point. If the poll duration is excessively long for the rate at which messages are sent on the topic, there exists a possibility that the cache cannot be warmed up and will operate in an inconsistent state relative to its peers until it catches up.

The repository can be instantiated by defining the **topic** and **bootstrapServers**, or the **producerConfig** and **consumerConfig** property sets can be explicitly defined to enable features such as SSL/SASL. To use, this repository must be placed in the Camel registry, either manually or by registration as a bean in Spring/Blueprint, as it is **CamelContext** aware.

Sample usage is as follows:

```
KafkaldempotentRepository kafkaldempotentRepository = new
KafkaldempotentRepository("idempotent-db-inserts", "localhost:9091");

SimpleRegistry registry = new SimpleRegistry();
registry.put("insertDbIdemRepo", kafkaldempotentRepository); // must be registered in the registry, to
enable access to the CamelContext
CamelContext context = new CamelContext(registry);

// later in RouteBuilder...
from("direct:performInsert")
    .idempotentConsumer(header("id")).messageIdRepositoryRef("insertDbIdemRepo")
    // once-only insert into database
```

```
.end()
```

In XML:

```
<!-- simple -->
<bean id="insertDbIdemRepo"
  class="org.apache.camel.processor.idempotent.kafka.KafkaIdempotentRepository">
  <property name="topic" value="idempotent-db-inserts"/>
  <property name="bootstrapServers" value="localhost:9091"/>
</bean>

<!-- complex -->
<bean id="insertDbIdemRepo"
  class="org.apache.camel.processor.idempotent.kafka.KafkaIdempotentRepository">
  <property name="topic" value="idempotent-db-inserts"/>
  <property name="maxCacheSize" value="10000"/>
  <property name="consumerConfig">
    <props>
      <prop key="bootstrap.servers">localhost:9091</prop>
    </props>
  </property>
  <property name="producerConfig">
    <props>
      <prop key="bootstrap.servers">localhost:9091</prop>
    </props>
  </property>
</bean>
```

There are 3 alternatives to choose from when using idempotency with numeric identifiers. The first one is to use the static method **numericHeader** method from

org.apache.camel.component.kafka.serde.KafkaSerdeHelper to perform the conversion for you:

```
from("direct:performInsert")
  .idempotentConsumer(numericHeader("id")).messageIdRepositoryRef("insertDbIdemRepo")
  // once-only insert into database
  .end()
```

Alternatively, it is possible use a custom serializer configured via the route URL to perform the conversion:

```
public class CustomHeaderDeserializer extends DefaultKafkaHeaderDeserializer {
  private static final Logger LOG = LoggerFactory.getLogger(CustomHeaderDeserializer.class);

  @Override
  public Object deserialize(String key, byte[] value) {
    if (key.equals("id")) {
      BigInteger bi = new BigInteger(value);

      return String.valueOf(bi.longValue());
    } else {
      return super.deserialize(key, value);
    }
  }
}
```

Lastly, it is also possible to do so in a processor:

```
from(from).routeId("foo")
    .process(exchange -> {
        byte[] id = exchange.getIn().getHeader("id", byte[].class);

        BigInteger bi = new BigInteger(id);
        exchange.getIn().setHeader("id", String.valueOf(bi.longValue()));
    })
    .idempotentConsumer(header("id"))
    .messageIdRepositoryRef("kafkaIdempotentRepository")
    .to(to);
```

66.9. USING MANUAL COMMIT WITH KAFKA CONSUMER

By default the Kafka consumer will use auto commit, where the offset will be committed automatically in the background using a given interval.

In case you want to force manual commits, you can use **KafkaManualCommit** API from the Camel Exchange, stored on the message header. This requires to turn on manual commits by either setting the option **allowManualCommit** to **true** on the **KafkaComponent** or on the endpoint, for example:

```
KafkaComponent kafka = new KafkaComponent();
kafka.setAllowManualCommit(true);
...
camelContext.addComponent("kafka", kafka);
```

You can then use the **KafkaManualCommit** from Java code such as a Camel **Processor**:

```
public void process(Exchange exchange) {
    KafkaManualCommit manual =
        exchange.getIn().getHeader(KafkaConstants.MANUAL_COMMIT, KafkaManualCommit.class);
    manual.commit();
}
```

This will force a synchronous commit which will block until the commit is acknowledge on Kafka, or if it fails an exception is thrown. You can use an asynchronous commit as well by configuring the **KafkaManualCommitFactory** with the `DefaultKafkaManualAsyncCommitFactory` implementation.

The commit will then be done in the next consumer loop using the kafka asynchronous commit api. Be aware that records from a partition must be processed and committed by a unique thread. If not, this could lead with non consistent behaviors. This is mostly useful with aggregation's completion timeout strategies.

If you want to use a custom implementation of **KafkaManualCommit** then you can configure a custom **KafkaManualCommitFactory** on the **KafkaComponent** that creates instances of your custom implementation.

When configuring a consumer to use manual commit and a specific **CommitManager** it is important to understand how these influence the behavior of **breakOnFirstError**.

```
KafkaComponent kafka = new KafkaComponent();
kafka.setAutoCommitEnable(false);
kafka.setAllowManualCommit(true);
```

```
kafka.setBreakOnFirstError(true);
kafka.setKafkaManualCommitFactory(new DefaultKafkaManualCommitFactory());
...
camelContext.addComponent("kafka", kafka);
```

When the **CommitManager** is left to the default **NoopCommitManager** then **breakOnFirstError** will not automatically commit the offset so that the message with an error is retried. The consumer must manage that in the route using **KafkaManualCommit**.

When the **CommitManager** is changed to either the synchronous or asynchronous manager then **breakOnFirstError** will automatically commit the offset so that the message with an error is retried. This message will be continually retried until it can be processed without an error.

NOTE

- Records from a partition must be processed and committed by the same thread as the consumer. This means that certain EIPs, async or concurrent operations in the DSL may cause the commit to fail. In such circumstances, trying to commit the transaction will cause the Kafka client to throw a **java.util.ConcurrentModificationException** exception with the message **KafkaConsumer is not safe for multi-threaded access**. To prevent this from happening, redesign your route to avoid those operations.
- This is mostly useful with aggregation's completion timeout strategies.

66.10. PAUSABLE CONSUMERS

The Kafka component supports pausable consumers. This type of consumer can pause consuming data based on conditions external to the component itself, such as an external system being unavailable or other transient conditions.

```
from("kafka:topic")
    .pausable(new KafkaConsumerListener(), () -> canContinue()) // the pausable check gets called if
the exchange fails to be processed ...
    .routeId("pausable-route")
    .process(this::process) // Kafka consumer will be paused if this one throws an exception ...
    .to("some:destination"); // or this one
```

In this example, consuming messages can pause (by calling the Kafka's Consumer pause method) if the result from **canContinue** is false.



IMPORTANT

The pausable EIP is meant to be used as a support mechanism when **there is an exception** somewhere in the route that prevents the exchange from being processed. More specifically, the check called by the **pausable** EIP should be used to test for transient conditions preventing the exchange from being processed.



NOTE

Most users should prefer using the `routeId`, which offers better control of the route.

66.11. KAFKA HEADERS PROPAGATION

When consuming messages from Kafka, headers will be propagated to camel exchange headers automatically. Producing flow backed by same behaviour – camel headers of particular exchange will be propagated to kafka message headers.

Since kafka headers allows only **byte[]** values, in order camel exchange header to be propagated its value should be serialized to **bytes[]**, otherwise header will be skipped. Following header value types are supported: **String**, **Integer**, **Long**, **Double**, **Boolean**, **byte[]**. Note: all headers propagated **from** kafka **to** camel exchange will contain **byte[]** value by default. In order to override default functionality uri parameters can be set: **headerDeserializer** for **from** route and **headerSerializer** for **to** route. Example:

```
from("kafka:my_topic?headerDeserializer=#myDeserializer")
...
.to("kafka:my_topic?headerSerializer=#mySerializer")
```

By default all headers are being filtered by **KafkaHeaderFilterStrategy**. Strategy filters out headers which start with **Camel** or **org.apache.camel** prefixes. Default strategy can be overridden by using **headerFilterStrategy** uri parameter in both **to** and **from** routes:

```
from("kafka:my_topic?headerFilterStrategy=#myStrategy")
...
.to("kafka:my_topic?headerFilterStrategy=#myStrategy")
```

myStrategy object should be subclass of **HeaderFilterStrategy** and must be placed in the Camel registry, either manually or by registration as a bean in Spring/Blueprint, as it is **CamelContext** aware.

66.12. KAFKA TRANSACTION

You need to add **transactional.id**, **enable.idempotence** and **retries** in **additional-properties** to enable kafka transaction with the producer.

```
from("direct:transaction")
.to("kafka:my_topic?additional-properties[transactional.id]=1234&additional-
properties[enable.idempotence]=true&additional-properties[retries]=5");
```

At the end of exchange routing, the kafka producer would commit the transaction or abort it if there is an Exception throwing or the exchange is **RollbackOnly**. Since Kafka does not support transactions in multi threads, it will throw **ProducerFencedException** if there is another producer with the same **transaction.id** to make the transactional request.

It would work with JTA **camel-jta** by using **transacted()** and if it involves some resources (SQL or JMS), which supports XA, then they would work in tandem, where they both will either commit or rollback at the end of the exchange routing. In some cases, if the JTA transaction manager fails to commit (during the 2PC processing), but kafka transaction has been committed before and there is no chance to roll back the changes since the kafka transaction does not support JTA/XA spec. There is still a risk with the data consistency.

66.13. SETTING KERBEROS CONFIG FILE

Configure the 'krb5.conf' file directly through the API:

```
static {
    KafkaComponent.setKerberosConfigLocation("path/to/config/file");
}
```

66.14. BATCHING CONSUMER

To use a Kafka batching consumer with Camel, an application has to set the configuration **batching** to **true**.

The received records are stored in a list in the exchange used in the pipeline. As such, it is possible to commit individually every record or the whole batch at once by committing the last exchange on the list.

The size of the batch is controlled by the option **maxPollRecords**.

To avoid blocking for too long, waiting for the whole set of records to fill the batch, it is possible to use the **pollTimeoutMs** option to set a timeout for the polling. In this case, the batch may contain less messages than set in the **maxPollRecords**.

66.14.1. Automatic Commits

By default, Camel uses automatic commits when using batch processing. In this case, Camel automatically commits the records after they have been successfully processed by the application.

In case of failures, the records will not be processed. The code below provides an example of this approach:

```
public void configure() {
    from("kafka:topic?
groupId=myGroup&pollTimeoutMs=1000&batching=true&maxPollRecords=10&autoOffsetReset=earlies
").process(e -> {
    // The received records are stored as exchanges in a list. This gets the list of those exchanges
    final List<?> exchanges = e.getMessage().getBody(List.class);

    // Ensure we are actually receiving what we are asking for
    if (exchanges == null || exchanges.isEmpty()) {
        return;
    }

    // The records from the batch are stored in a list of exchanges in the original exchange. To
    process, we iterate over that list
    for (Object obj : exchanges) {
        if (obj instanceof Exchange exchange) {
            LOG.info("Processing exchange with body {}",
exchange.getMessage().getBody(String.class));
        }
    }
    }).to(KafkaTestUtil.MOCK_RESULT);
}
```

66.14.1.1. Handling Errors with Automatic Commits

When using automatic commits, Camel will not commit records if there is a failure in processing. Because of this, there is a risk that records could be reprocessed multiple times.

It is recommended to implement appropriate error handling mechanisms and patterns (i.e.; such as dead-letter queues), to prevent failed records from blocking processing progress.

The code below provides an example of handling errors with automatic commits:

```
public void configure() {
    /*
     * We want to use continued here, so that Camel auto-commits the batch even though part of it has
     * failed. In a
     * production scenario, applications should probably send these records to a separate topic or fix the
     * condition
     * that lead to the failure
     */
    onException(IllegalArgumentException.class).process(exchange -> {
        LOG.warn("Failed to process batch {}", exchange.getMessage().getBody());
        LOG.warn("Failed to process due to {}",
exchange.getProperty(Exchange.EXCEPTION_CAUGHT, Throwable.class).getMessage());
    }).continued(true);

    from("kafka:topic?
groupId=myGroup&pollTimeoutMs=1000&batching=true&maxPollRecords=10&autoOffsetReset=earlies
").process(e -> {
    // The received records are stored as exchanges in a list. This gets the list of those exchanges
    final List<?> exchanges = e.getMessage().getBody(List.class);

    // Ensure we are actually receiving what we are asking for
    if (exchanges == null || exchanges.isEmpty()) {
        return;
    }

    // The records from the batch are stored in a list of exchanges in the original exchange.
    int i = 0;
    for (Object o : exchanges) {
        if (o instanceof Exchange exchange) {
            i++;
            LOG.info("Processing exchange with body {}",
exchange.getMessage().getBody(String.class));

            if (i == 4) {
                throw new IllegalArgumentException("Failed to process record");
            }
        }
    }
    }).to(KafkaTestUtil.MOCK_RESULT);
}
```

66.14.2. Manual Commits

When working with batch processing with manual commits, it's up to the application to commit the records, and handle the outcome of potentially invalid records.

The code below provides an example of this approach:

```
public void configure() {
    from("kafka:topic?
batching=true&allowManualCommit=true&maxPollRecords=100&kafkaManualCommitFactory=#class:or
g.apache.camel.component.kafka.consumer.DefaultKafkaManualCommitFactory")
    .process(e -> {
```

```

// The received records are stored as exchanges in a list. This gets the list of those exchanges
final List<?> exchanges = e.getMessage().getBody(List.class);

// Ensure we are actually receiving what we are asking for
if (exchanges == null || exchanges.isEmpty()) {
    return;
}

/*
Every exchange in that list should contain a reference to the manual commit object. We use the
reference
for the last exchange in the list to commit the whole batch
*/
final Object tmp = exchanges.getLast();
if (tmp instanceof Exchange exchange) {
    KafkaManualCommit manual =
        exchange.getMessage().getHeader(KafkaConstants.MANUAL_COMMIT,
KafkaManualCommit.class);
    LOG.debug("Performing manual commit");
    manual.commit();
    LOG.debug("Done performing manual commit");
}
});
}

```

66.14.3. Dealing with long polling timeouts

In some cases, applications may want the polling process to have a long timeout (see: **pollTimeoutMs**).

To properly do so, first make sure to have a max polling interval that is higher than the polling timeout (see: **maxPollIntervalMs**).

Then, increase the shutdown timeout to ensure that committing, closing and other Kafka operations are not abruptly aborted. For instance:

```

public void configure() {
    // Note that this can be configured in other ways
    getCamelContext().getShutdownStrategy().setTimeout(10000);

    // route setup ...
}

```

66.15. CUSTOM SUBSCRIPTION ADAPTERS

Applications with complex subscription logic may provide a custom bean to handle the subscription process. To so, it is necessary to implement the interface **SubscribeAdapter**.

Example subscriber adapter that subscribes to a set of Kafka topics or patterns

```

public class CustomSubscribeAdapter implements SubscribeAdapter {
    @Override
    public void subscribe(Consumer<?, ?> consumer, ConsumerRebalanceListener reBalanceListener,
TopicInfo topicInfo) {
        if (topicInfo.isPattern()) {

```

```

        consumer.subscribe(topicInfo.getPattern(), reBalanceListener);
    } else {
        consumer.subscribe(topicInfo.getTopics(), reBalanceListener);
    }
}
}

```

Then, it is necessary to add it as named bean instance to the registry:

Add to registry example

```

context.getRegistry().bind(KafkaConstants.KAFKA_SUBSCRIBE_ADAPTER, new
CustomSubscribeAdapter());

```

66.16. INTEROPERABILITY

66.16.1. JMS

When interoperating Kafka and JMS, it may be necessary to coerce the JMS headers into their expected type.

For instance, when consuming messages from Kafka carrying JMS headers and then sending them to a JMS broker, those headers are first deserialized into a byte array. Then, the **camel-jms** component tries to coerce this byte array into the specific type used by. However, both the origin endpoint as well as how this was setup on the code itself may affect how the data is serialized and deserialized. As such, it is not feasible to naively assume the data type of the byte array.

To address this issue, we provide a custom header deserializer to force Kafka to de-serialize the JMS data according to the JMS specification. This approach ensures that the headers are properly interpreted and processed by the camel-jms component.

Due to the inherent complexity of each possible system and endpoint, it may not be possible for this deserializer to cover all possible scenarios. As such, it is provided as model that can be modified and/or adapted for the specific needs of each application.

To utilize this solution, you need to modify the route URI on the consumer end of the pipeline by including the **headerDeserializer** option. For example:

```

from("kafka:topic?
headerDeserializer=#class:org.apache.camel.component.kafka.consumer.support.interop.JMSDeserializer")
.to("...");

```

66.17. EXAMPLES

66.17.1. Consuming messages from Kafka

Here is the minimal route you need in order to read messages from Kafka.

```

from("kafka:test?brokers=localhost:9092")
.log("Message received from Kafka : ${body}")
.log("  on the topic ${headers[kafka.TOPIC]}")
.log("  on the partition ${headers[kafka.PARTITION]}")

```

```
.log("    with the offset ${headers[kafka.OFFSET]}")
.log("    with the key ${headers[kafka.KEY]}")
```

If you need to consume messages from multiple topics you can use a comma separated list of topic names.

```
from("kafka:test,test1,test2?brokers=localhost:9092")
.log("Message received from Kafka : ${body}")
.log("    on the topic ${headers[kafka.TOPIC]}")
.log("    on the partition ${headers[kafka.PARTITION]}")
.log("    with the offset ${headers[kafka.OFFSET]}")
.log("    with the key ${headers[kafka.KEY]}")
```

It's also possible to subscribe to multiple topics giving a pattern as the topic name and using the **topicsPattern** option.

```
from("kafka:test*?brokers=localhost:9092&topicsPattern=true")
.log("Message received from Kafka : ${body}")
.log("    on the topic ${headers[kafka.TOPIC]}")
.log("    on the partition ${headers[kafka.PARTITION]}")
.log("    with the offset ${headers[kafka.OFFSET]}")
.log("    with the key ${headers[kafka.KEY]}")
```

When consuming messages from Kafka you can use your own offset management and not delegate this management to Kafka. In order to keep the offsets the component needs a **StateRepository** implementation such as **FileStateRepository**. This bean should be available in the registry. Here how to use it :

```
// Create the repository in which the Kafka offsets will be persisted
FileStateRepository repository = FileStateRepository.fileStateRepository(new
File("/path/to/repo.dat"));

// Bind this repository into the Camel registry
Registry registry = createCamelRegistry();
registry.bind("offsetRepo", repository);

// Configure the camel context
DefaultCamelContext camelContext = new DefaultCamelContext(registry);
camelContext.addRoutes(new RouteBuilder() {
    @Override
    public void configure() throws Exception {
        from("kafka:" + TOPIC + "?brokers=localhost:{{kafkaPort}}" +
            // Setup the topic and broker address
            "&groupId=A" +
            // The consumer processor group ID
            "&autoOffsetReset=earliest" +
            // Ask to start from the beginning if we have unknown offset
            "&offsetRepository=#offsetRepo")
            // Keep the offsets in the previously configured repository
            .to("mock:result");
    }
});
```

66.17.2. Producing messages to Kafka

Here is the minimal route you need in order to write messages to Kafka.

```
from("direct:start")
    .setBody(constant("Message from Camel")) // Message to send
    .setHeader(KafkaConstants.KEY, constant("Camel")) // Key of the message
    .to("kafka:test?brokers=localhost:9092");
```

66.17.3. SSL configuration

You have 2 different ways to configure the SSL communication on the Kafka` component.

The first way is through the many SSL endpoint parameters

```
from("kafka:" + TOPIC + "?brokers=localhost:{{kafkaPort}}" +
    "&groupId=A" +
    "&sslKeystoreLocation=/path/to/keystore.jks" +
    "&sslKeystorePassword=changeit" +
    "&sslKeyPassword=changeit" +
    "&securityProtocol=SSL")
    .to("mock:result");
```

The second way is to use the **sslContextParameters** endpoint parameter.

```
// Configure the SSLContextParameters object
KeyStoreParameters ksp = new KeyStoreParameters();
ksp.setResource("/path/to/keystore.jks");
ksp.setPassword("changeit");
KeyManagersParameters kmp = new KeyManagersParameters();
kmp.setKeyStore(ksp);
kmp.setKeyPassword("changeit");
SSLContextParameters scp = new SSLContextParameters();
scp.setKeyManagers(kmp);

// Bind this SSLContextParameters into the Camel registry
Registry registry = createCamelRegistry();
registry.bind("ssl", scp);

// Configure the camel context
DefaultCamelContext camelContext = new DefaultCamelContext(registry);
camelContext.addRoutes(new RouteBuilder() {
    @Override
    public void configure() throws Exception {
        from("kafka:" + TOPIC + "?brokers=localhost:{{kafkaPort}}" +
            // Setup the topic and broker address
            "&groupId=A" +
            // The consumer processor group ID
            "&sslContextParameters=#ssl" +
            // The security protocol
            "&securityProtocol=SSL")
            // Reference the SSL configuration
            .to("mock:result");
    }
});
```

66.18. SPRING BOOT AUTO-CONFIGURATION

The component supports 117 options, which are listed below.

Name	Description	Default	Type
camel.component.kafka.additional-properties	Sets additional properties for either kafka consumer or kafka producer in case they can't be set directly on the camel configurations (e.g: new Kafka properties that are not reflected yet in Camel configurations), the properties have to be prefixed with <code>additionalProperties..</code> E.g: <code>additionalProperties.transactional.id=12345&additionalProperties.schema.registry.url=http://localhost:8811/avro.</code>		Map
camel.component.kafka.allow-manual-commit	Whether to allow doing manual commits via <code>KafkaManualCommit</code> . If this option is enabled then an instance of <code>KafkaManualCommit</code> is stored on the Exchange message header, which allows end users to access this API and perform manual offset commits via the Kafka consumer.	false	Boolean
camel.component.kafka.auto-commit-enable	If true, periodically commit to ZooKeeper the offset of messages already fetched by the consumer. This committed offset will be used when the process fails as the position from which the new consumer will begin.	true	Boolean
camel.component.kafka.auto-commit-interval-ms	The frequency in ms that the consumer offsets are committed to zookeeper.	5000	Integer
camel.component.kafka.auto-commit-on-stop	Whether to perform an explicit auto commit when the consumer stops to ensure the broker has a commit from the last consumed message. This requires the option <code>autoCommitEnable</code> is turned on. The possible values are: <code>sync</code> , <code>async</code> , or <code>none</code> . And <code>sync</code> is the default value.	sync	String
camel.component.kafka.auto-offset-reset	What to do when there is no initial offset in ZooKeeper or if an offset is out of range: <code>earliest</code> : automatically reset the offset to the earliest offset <code>latest</code> : automatically reset the offset to the latest offset <code>fail</code> : throw exception to the consumer.	latest	String

Name	Description	Default	Type
camel.component.kafka.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kafka.batch-with-individual-headers	If this feature is enabled and a single element of a batch is an Exchange or Message, the producer will generate individual kafka header values for it by using the batch Message to determine the values. Normal behavior consists of always using the same header values (which are determined by the parent Exchange which contains the Iterable or Iterator).	false	Boolean
camel.component.kafka.batching	Whether to use batching for processing or streaming. The default is false, which uses streaming.	false	Boolean
camel.component.kafka.break-on-first-error	This options controls what happens when a consumer is processing an exchange and it fails. If the option is false then the consumer continues to the next message and processes it. If the option is true then the consumer breaks out, and will seek back to offset of the message that caused a failure, and then re-attempt to process this message. However this can lead to endless processing of the same message if its bound to fail every time, eg a poison message. Therefore its recommended to deal with that for example by using Camel's error handler.	false	Boolean
camel.component.kafka.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kafka.brokers	URL of the Kafka brokers to use. The format is host1:port1,host2:port2, and the list can be a subset of brokers or a VIP pointing to a subset of brokers. This option is known as bootstrap.servers in the Kafka documentation.		String

Name	Description	Default	Type
camel.component.kafka.buffer-memory-size	The total bytes of memory the producer can use to buffer records waiting to be sent to the server. If records are sent faster than they can be delivered to the server the producer will either block or throw an exception based on the preference specified by <code>block.on.buffer.full</code> . This setting should correspond roughly to the total memory the producer will use, but is not a hard bound since not all memory the producer uses is used for buffering. Some additional memory will be used for compression (if compression is enabled) as well as for maintaining in-flight requests.	33554432	Integer
camel.component.kafka.check-crcs	Automatically check the CRC32 of the records consumed. This ensures no on-the-wire or on-disk corruption to the messages occurred. This check adds some overhead, so it may be disabled in cases seeking extreme performance.	true	Boolean
camel.component.kafka.client-id	The client id is a user-specified string sent in each request to help trace calls. It should logically identify the application making the request.		String
camel.component.kafka.commit-timeout-ms	The maximum time, in milliseconds, that the code will wait for a synchronous commit to complete. The option is a <code>java.lang.Long</code> type.	5000	Long
camel.component.kafka.compression-codec	This parameter allows you to specify the compression codec for all data generated by this producer. Valid values are <code>none</code> , <code>gzip</code> and <code>snappy</code> .	none	String
camel.component.kafka.configuration	Allows to pre-configure the Kafka component with common options that the endpoints will reuse. The option is a <code>org.apache.camel.component.kafka.KafkaConfiguration</code> type.		KafkaConfiguration
camel.component.kafka.connection-max-idle-ms	Close idle connections after the number of milliseconds specified by this config.	540000	Integer
camel.component.kafka.consumer-request-timeout-ms	The configuration controls the maximum amount of time the client will wait for the response of a request. If the response is not received before the timeout elapses the client will resend the request if necessary or fail the request if retries are exhausted.	40000	Integer

Name	Description	Default	Type
camel.component. .kafka.consumers- count	The number of consumers that connect to kafka server. Each consumer is run on a separate thread, that retrieves and process the incoming data.	1	Integer
camel.component. .kafka.create- consumer- backoff-interval	The delay in millis seconds to wait before trying again to create the kafka consumer (kafka-client).	5000	Long
camel.component. .kafka.create- consumer- backoff-max- attempts	Maximum attempts to create the kafka consumer (kafka-client), before eventually giving up and failing. Error during creating the consumer may be fatal due to invalid configuration and as such recovery is not possible. However, one part of the validation is DNS resolution of the bootstrap broker hostnames. This may be a temporary networking problem, and could potentially be recoverable. While other errors are fatal, such as some invalid kafka configurations. Unfortunately, kafka-client does not separate this kind of errors. Camel will by default retry forever, and therefore never give up. If you want to give up after many attempts then set this option and Camel will then when giving up terminate the consumer. To try again, you can manually restart the consumer by stopping, and starting the route.		Integer
camel.component. .kafka.delivery- timeout-ms	An upper bound on the time to report success or failure after a call to send() returns. This limits the total time that a record will be delayed prior to sending, the time to await acknowledgement from the broker (if expected), and the time allowed for retrievable send failures.	12000 0	Integer
camel.component. .kafka.enable- idempotence	If set to 'true' the producer will ensure that exactly one copy of each message is written in the stream. If 'false', producer retries may write duplicates of the retried message in the stream. If set to true this option will require max.in.flight.requests.per.connection to be set to 1 and retries cannot be zero and additionally acks must be set to 'all'.	false	Boolean
camel.component. .kafka.enabled	Whether to enable auto configuration of the kafka component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.kafka.fetch-max-bytes	The maximum amount of data the server should return for a fetch request This is not an absolute maximum, if the first message in the first non-empty partition of the fetch is larger than this value, the message will still be returned to ensure that the consumer can make progress. The maximum message size accepted by the broker is defined via <code>message.max.bytes</code> (broker config) or <code>max.message.bytes</code> (topic config). Note that the consumer performs multiple fetches in parallel.	52428800	Integer
camel.component.kafka.fetch-min-bytes	The minimum amount of data the server should return for a fetch request. If insufficient data is available the request will wait for that much data to accumulate before answering the request.	1	Integer
camel.component.kafka.fetch-wait-max-ms	The maximum amount of time the server will block before answering the fetch request if there isn't sufficient data to immediately satisfy <code>fetch.min.bytes</code> .	500	Integer
camel.component.kafka.group-id	A string that uniquely identifies the group of consumer processes to which this consumer belongs. By setting the same group id multiple processes indicate that they are all part of the same consumer group. This option is required for consumers.		String
camel.component.kafka.group-instance-id	A unique identifier of the consumer instance provided by the end user. Only non-empty strings are permitted. If set, the consumer is treated as a static member, which means that only one instance with this ID is allowed in the consumer group at any time. This can be used in combination with a larger session timeout to avoid group rebalances caused by transient unavailability (e.g. process restarts). If not set, the consumer will join the group as a dynamic member, which is the traditional behavior.		String
camel.component.kafka.header-deserializer	To use a custom <code>KafkaHeaderDeserializer</code> to deserialize kafka headers values. The option is a <code>org.apache.camel.component.kafka.serde.KafkaHeaderDeserializer</code> type.		<code>KafkaHeaderDeserializer</code>
camel.component.kafka.header-filter-strategy	To use a custom <code>HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		<code>HeaderFilterStrategy</code>

Name	Description	Default	Type
camel.component.kafka.header-serializer	To use a custom KafkaHeaderSerializer to serialize kafka headers values. The option is a <code>org.apache.camel.component.kafka.serde.KafkaHeaderSerializer</code> type.		KafkaHeaderSerializer
camel.component.kafka.health-check-consumer-enabled	Used for enabling or disabling all consumer based health checks from this component.	true	Boolean
camel.component.kafka.health-check-producer-enabled	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting <code>camel.health.producersEnabled=true</code> .	true	Boolean
camel.component.kafka.heartbeat-interval-ms	The expected time between heartbeats to the consumer coordinator when using Kafka's group management facilities. Heartbeats are used to ensure that the consumer's session stays active and to facilitate rebalancing when new consumers join or leave the group. The value must be set lower than <code>session.timeout.ms</code> , but typically should be set no higher than 1/3 of that value. It can be adjusted even lower to control the expected time for normal rebalances.	3000	Integer
camel.component.kafka.interceptor-classes	Sets interceptors for producer or consumers. Producer interceptors have to be classes implementing <code>org.apache.kafka.clients.producer.ProducerInterceptor</code> or Consumer interceptors have to be classes implementing <code>org.apache.kafka.clients.consumer.ConsumerInterceptor</code> Note that if you use Producer interceptor on a consumer it will throw a class cast exception in runtime.		String

Name	Description	Default	Type
camel.component.kafka.isolation-level	Controls how to read messages written transactionally. If set to <code>read_committed</code> , <code>consumer.poll()</code> will only return transactional messages which have been committed. If set to <code>read_uncommitted</code> (the default), <code>consumer.poll()</code> will return all messages, even transactional messages which have been aborted. Non-transactional messages will be returned unconditionally in either mode. Messages will always be returned in offset order. Hence, in <code>read_committed</code> mode, <code>consumer.poll()</code> will only return messages up to the last stable offset (LSO), which is the one less than the offset of the first open transaction. In particular, any messages appearing after messages belonging to ongoing transactions will be withheld until the relevant transaction has been completed. As a result, <code>read_committed</code> consumers will not be able to read up to the high watermark when there are in flight transactions. Further, when in <code>read_committed</code> the <code>seekToEnd</code> method will return the LSO.	<code>read_uncommitted</code>	String
camel.component.kafka.kafka-client-factory	Factory to use for creating <code>org.apache.kafka.clients.consumer.KafkaConsumer</code> and <code>org.apache.kafka.clients.producer.KafkaProducer</code> instances. This allows to configure a custom factory to create instances with logic that extends the vanilla Kafka clients. The option is a <code>org.apache.camel.component.kafka.KafkaClientFactory</code> type.		<code>KafkaClientFactory</code>
camel.component.kafka.kafka-manual-commit-factory	Factory to use for creating <code>KafkaManualCommit</code> instances. This allows to plugin a custom factory to create custom <code>KafkaManualCommit</code> instances in case special logic is needed when doing manual commits that deviates from the default implementation that comes out of the box. The option is a <code>org.apache.camel.component.kafka.KafkaManualCommitFactory</code> type.		<code>KafkaManualCommitFactory</code>
camel.component.kafka.kerberos-before-relogin-min-time	Login thread sleep time between refresh attempts.	60000	Integer
camel.component.kafka.kerberos-config-location	Location of the kerberos config file.		String

Name	Description	Default	Type
camel.component.kafka.kerberos-init-cmd	Kerberos kinit command path. Default is /usr/bin/kinit.	/usr/bin/kinit	String
camel.component.kafka.kerberos-principal-to-local-rules	A list of rules for mapping from principal names to short names (typically operating system usernames). The rules are evaluated in order and the first rule that matches a principal name is used to map it to a short name. Any later rules in the list are ignored. By default, principal names of the form {username}/{hostname}{REALM} are mapped to {username}. For more details on the format please see the security authorization and acls documentation.. Multiple values can be separated by comma.	DEFAULT	String
camel.component.kafka.kerberos-renew-jitter	Percentage of random jitter added to the renewal time.		Double
camel.component.kafka.kerberos-renew-window-factor	Login thread will sleep until the specified window factor of time from last refresh to ticket's expiry has been reached, at which time it will try to renew the ticket.		Double
camel.component.kafka.key	The record key (or null if no key is specified). If this option has been configured then it take precedence over header KafkaConstants#KEY.		String
camel.component.kafka.key-deserializer	Deserializer class for key that implements the Deserializer interface.	org.apache.kafka.common.serialization.StringDeserializer	String
camel.component.kafka.key-serializer	The serializer class for keys (defaults to the same as for messages if nothing is given).	org.apache.kafka.common.serialization.StringSerializer	String

Name	Description	Default	Type
camel.component.kafka.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kafka.linger-ms	The producer groups together any records that arrive in between request transmissions into a single batched request. Normally this occurs only under load when records arrive faster than they can be sent out. However in some circumstances the client may want to reduce the number of requests even under moderate load. This setting accomplishes this by adding a small amount of artificial delay that is, rather than immediately sending out a record the producer will wait for up to the given delay to allow other records to be sent so that the sends can be batched together. This can be thought of as analogous to Nagle's algorithm in TCP. This setting gives the upper bound on the delay for batching: once we get batch.size worth of records for a partition it will be sent immediately regardless of this setting, however if we have fewer than this many bytes accumulated for this partition we will 'linger' for the specified time waiting for more records to show up. This setting defaults to 0 (i.e. no delay). Setting linger.ms=5, for example, would have the effect of reducing the number of requests sent but would add up to 5ms of latency to records sent in the absense of load.	0	Integer
camel.component.kafka.max-block-ms	The configuration controls how long sending to kafka will block. These methods can be blocked for multiple reasons. For e.g: buffer full, metadata unavailable. This configuration imposes maximum limit on the total time spent in fetching metadata, serialization of key and value, partitioning and allocation of buffer memory when doing a send(). In case of partitionsFor(), this configuration imposes a maximum time threshold on waiting for metadata.	60000	Integer

Name	Description	Default	Type
camel.component.kafka.max-in-flight-request	The maximum number of unacknowledged requests the client will send on a single connection before blocking. Note that if this setting is set to be greater than 1 and there are failed sends, there is a risk of message re-ordering due to retries (i.e., if retries are enabled).	5	Integer
camel.component.kafka.max-partition-fetch-bytes	The maximum amount of data per-partition the server will return. The maximum total memory used for a request will be <code>#partitions max.partition.fetch.bytes</code> . This size must be at least as large as the maximum message size the server allows or else it is possible for the producer to send messages larger than the consumer can fetch. If that happens, the consumer can get stuck trying to fetch a large message on a certain partition.	1048576	Integer
camel.component.kafka.max-poll-interval-ms	The maximum delay between invocations of <code>poll()</code> when using consumer group management. This places an upper bound on the amount of time that the consumer can be idle before fetching more records. If <code>poll()</code> is not called before expiration of this timeout, then the consumer is considered failed and the group will rebalance in order to reassign the partitions to another member. The option is a <code>java.lang.Long</code> type.		Long
camel.component.kafka.max-poll-records	The maximum number of records returned in a single call to <code>poll()</code> .	500	Integer
camel.component.kafka.max-request-size	The maximum size of a request. This is also effectively a cap on the maximum record size. Note that the server has its own cap on record size which may be different from this. This setting will limit the number of record batches the producer will send in a single request to avoid sending huge requests.	1048576	Integer
camel.component.kafka.metadata-max-age-ms	The period of time in milliseconds after which we force a refresh of metadata even if we haven't seen any partition leadership changes to proactively discover any new brokers or partitions.	300000	Integer
camel.component.kafka.metric-reporters	A list of classes to use as metrics reporters. Implementing the <code>MetricReporter</code> interface allows plugging in classes that will be notified of new metric creation. The <code>JmxReporter</code> is always included to register JMX statistics.		String

Name	Description	Default	Type
camel.component.kafka.metrics-sample-window-ms	The number of samples maintained to compute metrics.	30000	Integer
camel.component.kafka.no-of-metrics-sample	The number of samples maintained to compute metrics.	2	Integer
camel.component.kafka.offset-repository	The offset repository to use in order to locally store the offset of each partition of the topic. Defining one will disable the autocommit. The option is a <code>org.apache.camel.spi.StateRepository<java.lang.String, java.lang.String></code> type.		StateRepository
camel.component.kafka.partition-assignor	The class name of the partition assignment strategy that the client will use to distribute partition ownership amongst consumer instances when group management is used.	<code>org.apache.kafka.clients.consumer.RangeAssignor</code>	String
camel.component.kafka.partition-key	The partition to which the record will be sent (or null if no partition was specified). If this option has been configured then it take precedence over header <code>KafkaConstants#PARTITION_KEY</code> .		Integer
camel.component.kafka.partitionner	The partitioner class for partitioning messages amongst sub-topics. The default partitioner is based on the hash of the key.	<code>org.apache.kafka.clients.producer.internals.DefaultPartitioner</code>	String
camel.component.kafka.partitionner-ignore-keys	Whether the message keys should be ignored when computing the partition. This setting has effect only when partitioner is not set.	false	Boolean
camel.component.kafka.poll-exception-strategy	To use a custom strategy with the consumer to control how to handle exceptions thrown from the Kafka broker while pooling messages. The option is a <code>org.apache.camel.component.kafka.PollExceptionHandler</code> type.		PollExceptionHandler

Name	Description	Default	Type
camel.component.kafka.poll-on-error	What to do if kafka threw an exception while polling for new messages. Will by default use the value from the component configuration unless an explicit value has been configured on the endpoint level. DISCARD will discard the message and continue to poll next message. ERROR_HANDLER will use Camel's error handler to process the exception, and afterwards continue to poll next message. RECONNECT will re-connect the consumer and try poll the message again. RETRY will let the consumer retry polling the same message again. STOP will stop the consumer (have to be manually started/restarted if the consumer should be able to consume messages again).		PollOnError
camel.component.kafka.poll-timeout-ms	The timeout used when polling the KafkaConsumer. The option is a java.lang.Long type.	5000	Long
camel.component.kafka.pre-validate-host-and-port	Whether to eager validate that broker host:port is valid and can be DNS resolved to known host during starting this consumer. If the validation fails, then an exception is thrown, which makes Camel fail fast. Disabling this will postpone the validation after the consumer is started, and Camel will keep re-connecting in case of validation or DNS resolution error.	true	Boolean
camel.component.kafka.producer-batch-size	The producer will attempt to batch records together into fewer requests whenever multiple records are being sent to the same partition. This helps performance on both the client and the server. This configuration controls the default batch size in bytes. No attempt will be made to batch records larger than this size. Requests sent to brokers will contain multiple batches, one for each partition with data available to be sent. A small batch size will make batching less common and may reduce throughput (a batch size of zero will disable batching entirely). A very large batch size may use memory a bit more wastefully as we will always allocate a buffer of the specified batch size in anticipation of additional records.	16384	Integer
camel.component.kafka.queue-buffering-max-messages	The maximum number of unsent messages that can be queued up the producer when using async mode before either the producer must be blocked or data must be dropped.	10000	Integer

Name	Description	Default	Type
camel.component. .kafka.receive- buffer-bytes	The size of the TCP receive buffer (SO_RCVBUF) to use when reading data.	65536	Integer
camel.component. .kafka.reconnect- backoff-max-ms	The maximum amount of time in milliseconds to wait when reconnecting to a broker that has repeatedly failed to connect. If provided, the backoff per host will increase exponentially for each consecutive connection failure, up to this maximum. After calculating the backoff increase, 20% random jitter is added to avoid connection storms.	1000	Integer
camel.component. .kafka.reconnect- backoff-ms	The amount of time to wait before attempting to reconnect to a given host. This avoids repeatedly connecting to a host in a tight loop. This backoff applies to all requests sent by the consumer to the broker.	50	Integer
camel.component. .kafka.record- metadata	Whether the producer should store the RecordMetadata results from sending to Kafka. The results are stored in a List containing the RecordMetadata metadata's. The list is stored on a header with the key <code>KafkaConstants#KAFKA_RECORDMETA</code> .	true	Boolean
camel.component. .kafka.request- required-acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. This controls the durability of records that are sent. The following settings are common: <code>acks=0</code> If set to zero then the producer will not wait for any acknowledgment from the server at all. The record will be immediately added to the socket buffer and considered sent. No guarantee can be made that the server has received the record in this case, and the retries configuration will not take effect (as the client won't generally know of any failures). The offset given back for each record will always be set to -1. <code>acks=1</code> This will mean the leader will write the record to its local log but will respond without awaiting full acknowledgement from all followers. In this case should the leader fail immediately after acknowledging the record but before the followers have replicated it then the record will be lost. <code>acks=all</code> This means the leader will wait for the full set of in-sync replicas to acknowledge the record. This guarantees that the record will not be lost as long as at least one in-sync replica remains alive. This is the strongest available guarantee.	1	String

Name	Description	Default	Type
camel.component.kafka.request-timeout-ms	The amount of time the broker will wait trying to meet the request.required.acks requirement before sending back an error to the client.	30000	Integer
camel.component.kafka.resume-strategy	This option allows the user to set a custom resume strategy. The resume strategy is executed when partitions are assigned (i.e.: when connecting or reconnecting). It allows implementations to customize how to resume operations and serve as more flexible alternative to the seekTo and the offsetRepository mechanisms. See the KafkaConsumerResumeStrategy for implementation details. This option does not affect the auto commit setting. It is likely that implementations using this setting will also want to evaluate using the manual commit option along with this. The option is a org.apache.camel.component.kafka.consumer.support.KafkaConsumerResumeStrategy type.		KafkaConsumerResumeStrategy
camel.component.kafka.retries	Setting a value greater than zero will cause the client to resend any record whose send fails with a potentially transient error. Note that this retry is no different than if the client resent the record upon receiving the error. Allowing retries will potentially change the ordering of records because if two records are sent to a single partition, and the first fails and is retried but the second succeeds, then the second record may appear first.	0	Integer
camel.component.kafka.retry-backoff-ms	Before each retry, the producer refreshes the metadata of relevant topics to see if a new leader has been elected. Since leader election takes a bit of time, this property specifies the amount of time that the producer waits before refreshing the metadata.	100	Integer
camel.component.kafka.sasl-jaas-config	Expose the kafka sasl.jaas.config parameter Example: org.apache.kafka.common.security.plain.PlainLoginModule required username=USERNAME password=PASSWORD;.		String
camel.component.kafka.sasl-kerberos-service-name	The Kerberos principal name that Kafka runs as. This can be defined either in Kafka's JAAS config or in Kafka's config.		String

Name	Description	Default	Type
camel.component .kafka.sasl-mechanism	The Simple Authentication and Security Layer (SASL) Mechanism used. For the valid values see .	GSSAPI	String
camel.component .kafka.schema-registry-u-r-l	URL of the Confluent Platform schema registry servers to use. The format is host1:port1,host2:port2. This is known as schema.registry.url in the Confluent Platform documentation. This option is only available in the Confluent Platform (not standard Apache Kafka).		String
camel.component .kafka.security-protocol	Protocol used to communicate with brokers. SASL_PLAINTEXT, PLAINTEXT and SSL are supported.	PLAINTEXT	String
camel.component .kafka.seek-to	Set if KafkaConsumer will read from beginning or end on startup: beginning : read from beginning end : read from end This is replacing the earlier property seekToBeginning.		String
camel.component .kafka.send-buffer-bytes	Socket write buffer size.	131072	Integer
camel.component .kafka.session-timeout-ms	The timeout used to detect failures when using Kafka's group management facilities.	10000	Integer
camel.component .kafka.shutdown-timeout	Timeout in milliseconds to wait gracefully for the consumer or producer to shutdown and terminate its worker threads.	30000	Integer
camel.component .kafka.specific-avro-reader	This enables the use of a specific Avro reader for use with the Confluent Platform schema registry and the io.confluent.kafka.serializers.KafkaAvroDeserializer. This option is only available in the Confluent Platform (not standard Apache Kafka).	false	Boolean
camel.component .kafka.ssl-cipher-suites	A list of cipher suites. This is a named combination of authentication, encryption, MAC and key exchange algorithm used to negotiate the security settings for a network connection using TLS or SSL network protocol.By default all the available cipher suites are supported.		String

Name	Description	Default	Type
camel.component.kafka.ssl-context-parameters	SSL configuration using a Camel SSLContextParameters object. If configured it's applied before the other SSL endpoint parameters. NOTE: Kafka only supports loading keystore from file locations, so prefix the location with file: in the KeyStoreParameters.resource option. The option is a org.apache.camel.support.jsse.SSLContextParameters type.		SSLContextParameters
camel.component.kafka.ssl-enabled-protocols	The list of protocols enabled for SSL connections. TLSv1.2, TLSv1.1 and TLSv1 are enabled by default.		String
camel.component.kafka.ssl-endpoint-algorithm	The endpoint identification algorithm to validate server hostname using server certificate.	https	String
camel.component.kafka.ssl-key-password	The password of the private key in the key store file. This is optional for client.		String
camel.component.kafka.ssl-keymanager-algorithm	The algorithm used by key manager factory for SSL connections. Default value is the key manager factory algorithm configured for the Java Virtual Machine.	SunX509	String
camel.component.kafka.ssl-keystore-location	The location of the key store file. This is optional for client and can be used for two-way authentication for client.		String
camel.component.kafka.ssl-keystore-password	The store password for the key store file. This is optional for client and only needed if ssl.keystore.location is configured.		String
camel.component.kafka.ssl-keystore-type	The file format of the key store file. This is optional for client. Default value is JKS.	JKS	String
camel.component.kafka.ssl-protocol	The SSL protocol used to generate the SSLContext. Default setting is TLS, which is fine for most cases. Allowed values in recent JVMs are TLS, TLSv1.1 and TLSv1.2. SSL, SSLv2 and SSLv3 may be supported in older JVMs, but their usage is discouraged due to known security vulnerabilities.		String

Name	Description	Default	Type
camel.component.kafka.ssl-provider	The name of the security provider used for SSL connections. Default value is the default security provider of the JVM.		String
camel.component.kafka.ssl-trustmanager-algorithm	The algorithm used by trust manager factory for SSL connections. Default value is the trust manager factory algorithm configured for the Java Virtual Machine.	PKIX	String
camel.component.kafka.ssl-truststore-location	The location of the trust store file.		String
camel.component.kafka.ssl-truststore-password	The password for the trust store file.		String
camel.component.kafka.ssl-truststore-type	The file format of the trust store file. Default value is JKS.	JKS	String
camel.component.kafka.subscribe-consumer-backoff-interval	The delay in millis seconds to wait before trying again to subscribe to the kafka broker.	5000	Long
camel.component.kafka.subscribe-consumer-backoff-max-attempts	Maximum number the kafka consumer will attempt to subscribe to the kafka broker, before eventually giving up and failing. Error during subscribing the consumer to the kafka topic could be temporary errors due to network issues, and could potentially be recoverable. Camel will by default retry forever, and therefore never give up. If you want to give up after many attempts, then set this option and Camel will then when giving up terminate the consumer. You can manually restart the consumer by stopping and starting the route, to try again.		Integer
camel.component.kafka.synchronous	Sets whether synchronous processing should be strictly used.	false	Boolean
camel.component.kafka.topic-is-pattern	Whether the topic is a pattern (regular expression). This can be used to subscribe to dynamic number of topics matching the pattern.	false	Boolean

Name	Description	Default	Type
camel.component.kafka.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean
camel.component.kafka.use-iterator	Sets whether sending to kafka should send the message body as a single record, or use a <code>java.util.Iterator</code> to send multiple records to kafka (if the message body can be iterated).	true	Boolean
camel.component.kafka.value-deserializer	Deserializer class for value that implements the <code>Deserializer</code> interface.	<code>org.apache.kafka.common.serialization.StringDeserializer</code>	String
camel.component.kafka.value-serializer	The serializer class for messages.	<code>org.apache.kafka.common.serialization.StringSerializer</code>	String
camel.component.kafka.worker-pool	To use a custom worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from <code>KafkaProducer</code> using asynchronous non-blocking processing. If using this option then you must handle the lifecycle of the thread pool to shut the pool down when no longer needed. The option is a <code>java.util.concurrent.ExecutorService</code> type.		<code>ExecutorService</code>
camel.component.kafka.worker-pool-core-size	Number of core threads for the worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from <code>KafkaProducer</code> using asynchronous non-blocking processing.	10	Integer
camel.component.kafka.worker-pool-max-size	Maximum number of threads for the worker pool for continue routing Exchange after kafka server has acknowledge the message that was sent to it from <code>KafkaProducer</code> using asynchronous non-blocking processing.	20	Integer

CHAPTER 67. KAMELET

Both producer and consumer are supported

The Kamelet Component provides support for interacting with the [Camel Route Template](#) engine using Endpoint semantic.

67.1. DEPENDENCIES

When using **kamelet** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kamelet-starter</artifactId>
</dependency>
```

67.2. URI FORMAT

```
kamelet:templateId/routeId[?options]
```

67.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

67.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

67.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

67.4. COMPONENT OPTIONS

The Kamelet component supports 9 options, which are listed below.

Name	Description	Default	Type
location (common)	The location(s) of the Kamelets on the file system. Multiple locations can be set separated by comma.	classpath:/kamelets	String
routeProperties (common)	Set route local parameters.		Map
templateProperties (common)	Set template local parameters.		Map
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
block (producer)	If sending a message to a kamelet endpoint which has no active consumer, then we can tell the producer to block and wait for the consumer to become active.	true	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
timeout (producer)	The timeout value to use if block is enabled.	30000	long
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
routeTemplateLoaderListener (advanced)	Autowired To plugin a custom listener for when the Kamelet component is loading Kamelets from external resources.		RouteTemplateLoaderListener

67.5. ENDPOINT OPTIONS

The Kamelet endpoint is configured using URI syntax:

```
kamelet:templateId/routeId
```

with the following path and query parameters:

67.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
templateId (common)	Required The Route Template ID.		String
routeId (common)	The Route ID. Default value notice: The ID will be auto-generated if not provided.		String

67.5.2. Query Parameters (8 parameters)

Name	Description	Default	Type
location (common)	Location of the Kamelet to use which can be specified as a resource from file system, classpath etc. The location cannot use wildcards, and must refer to a file including extension, for example file:/etc/foo-kamelet.xml.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
block (producer)	If sending a message to a direct endpoint which has no active consumer, then we can tell the producer to block and wait for the consumer to become active.	true	boolean
failIfNoConsumers (producer)	Whether the producer should fail by throwing an exception, when sending to a kamelet endpoint with no active consumers.	true	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
timeout (producer)	The timeout value to use if block is enabled.	30000	long

**NOTE**

The **kamelet** endpoint is **lenient**, which means that the endpoint accepts additional parameters that are passed to the engine and consumed upon route materialization.

67.6. DISCOVERY

If a [Route Template](#) is not found, the **kamelet** endpoint tries to load the related **kamelet** definition from the file system (by default **classpath:/kamelets**). The default resolution mechanism expects kamelet files to have the extension **.kamelet.yaml**.

67.7. SAMPLES

Kamelets can be used as if they were standard Camel components. For example, suppose that we have created a Route Template as follows:

```
routeTemplate("setMyBody")
  .templateParameter("bodyValue")
  .from("kamelet:source")
  .setBody().constant("#{bodyValue}");
```

**NOTE**

To let the **Kamelet** component wire the materialized route to the caller processor, we need to be able to identify the input and output endpoint of the route and this is done by using **kamelet:source** to mark the input endpoint and **kamelet:sink** for the output endpoint.

Then the template can be instantiated and invoked as shown below:

```
from("direct:setMyBody")
  .to("kamelet:setMyBody?bodyValue=myKamelet");
```

Behind the scenes, the **Kamelet** component does the following things:

1. It instantiates a route out of the Route Template identified by the given **templateId** path parameter (in this case **setBody**)
2. It will act like the **direct** component and connect the current route to the materialized one.

If you had to do it programmatically, it would have been something like:

```
routeTemplate("setMyBody")
    .templateParameter("bodyValue")
    .from("direct:{{foo}}")
    .setBody().constant("{{bodyValue}}");

TemplatedRouteBuilder.builder(context, "setMyBody")
    .parameter("foo", "bar")
    .parameter("bodyValue", "myKamelet")
    .add();

from("direct:template")
    .to("direct:bar");
```

67.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 10 options, which are listed below.

Name	Description	Default	Type
camel.component.kamelet.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kamelet.block	If sending a message to a kamelet endpoint which has no active consumer, then we can tell the producer to block and wait for the consumer to become active.	true	Boolean
camel.component.kamelet.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.kamelet.enabled	Whether to enable auto configuration of the kamelet component. This is enabled by default.		Boolean
camel.component.kamelet.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kamelet.location	The location(s) of the Kamelets on the file system. Multiple locations can be set separated by comma.	classpath:/kamelets	String
camel.component.kamelet.route-properties	Set route local parameters.		Map
camel.component.kamelet.route-template-loader-listener	To plugin a custom listener for when the Kamelet component is loading Kamelets from external resources. The option is a <code>org.apache.camel.spi.RouteTemplateLoaderListener</code> type.		RouteTemplateLoaderListener
camel.component.kamelet.template-properties	Set template local parameters.		Map
camel.component.kamelet.timeout	The timeout value to use if block is enabled.	30000	Long

CHAPTER 68. KAMELET MAIN

Since Camel 3.11

A **main** class that is opinionated to bootstrap and run Camel standalone with Kamelets (or plain YAML routes) for development and demo purposes.

68.1. INITIAL CONFIGURATION

The **KameletMain** is pre-configured with the following properties:

```
camel.component.kamelet.location = classpath:/kamelets,github:apache:camel-kamelets/kamelets
camel.component.rest.consumerComponentName = platform-http
camel.component.rest.producerComponentName = vertx-http
```

You can override these settings by updating the configuration in **application.properties**.

68.2. AUTOMATIC DEPENDENCIES DOWNLOADING

The Kamelet Main can automatically download Kamelet YAML files from a remote location over http/https, and from github as well.

The official Kamelets from the Apache Camel Kamelet Catalog is stored on github and they can be used out of the box as-is.

For example a Camel route can be **coded** in YAML which uses the Earthquake Kamelet from the catalog, as shown below:

```
- route:
  from: "kamelet:earthquake-source"
  steps:
    - unmarshal:
      json: {}
    - log: "Earthquake with magnitude ${body[properties][mag]} at ${body[properties][place]}"
```

In the above example, the earthquake kamelet will be downloaded from github, and as well its required dependencies.

For more information, see [Kamelet Main example](#)

CHAPTER 69. KUBERNETES

Since Camel 2.17

The Kubernetes components integrate your application with Kubernetes standalone or on top of Openshift.

69.1. KUBERNETES COMPONENTS

See the following for usage of each component:

[Kubernetes ConfigMap](#) Perform operations on Kubernetes ConfigMaps and get notified on ConfigMaps changes.

[Kubernetes Custom Resources](#) Perform operations on Kubernetes Custom Resources and get notified on Deployment changes.

[Kubernetes Deployments](#) Perform operations on Kubernetes Deployments and get notified on Deployment changes.

[Kubernetes Event](#) Perform operations on Kubernetes Events and get notified on Events changes.

[Kubernetes HPA](#) Perform operations on Kubernetes Horizontal Pod Autoscalers (HPA) and get notified on HPA changes.

[Kubernetes Job](#) Perform operations on Kubernetes Jobs.

[Kubernetes Namespaces](#) Perform operations on Kubernetes Namespaces and get notified on Namespace changes.

[Kubernetes Nodes](#) Perform operations on Kubernetes Nodes and get notified on Node changes.

[Kubernetes Persistent Volume](#) Perform operations on Kubernetes Persistent Volumes and get notified on Persistent Volume changes.

[Kubernetes Persistent Volume Claim](#) Perform operations on Kubernetes Persistent Volumes Claims and get notified on Persistent Volumes Claim changes.

[Kubernetes Pods](#) Perform operations on Kubernetes Pods and get notified on Pod changes.

[Kubernetes Replication Controller](#) Perform operations on Kubernetes Replication Controllers and get notified on Replication Controllers changes.

[Kubernetes Resources Quota](#) Perform operations on Kubernetes Resources Quotas.

[Kubernetes Secrets](#) Perform operations on Kubernetes Secrets.

[Kubernetes Service Account](#) Perform operations on Kubernetes Service Accounts.

[Kubernetes Services](#) Perform operations on Kubernetes Services and get notified on Service changes.

[Openshift Build Config](#) Perform operations on OpenShift Build Configs.

[Openshift Builds](#) Perform operations on OpenShift Builds.

[Openshift Deployment Configs](#) Perform operations on Openshift Deployment Configs and get notified on Deployment Config changes.

69.2. DEPENDENCIES

Add the following dependency to your **pom.xml** for this component:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

69.3. USAGE

69.3.1. Producer examples

Here we show some examples of producer using camel-kubernetes.

Create a pod

```
from("direct:createPod")
  .toF("kubernetes-pods://%s?oauthToken=%s&operation=createPod", host, authToken);
```

By using the **KubernetesConstants.KUBERNETES_POD_SPEC** header you can specify your PodSpec and pass it to this operation.

Delete a pod

```
from("direct:createPod")
  .toF("kubernetes-pods://%s?oauthToken=%s&operation=deletePod", host, authToken);
```

By using the **KubernetesConstants.KUBERNETES_POD_NAME** header you can specify your Pod name and pass it to this operation.

69.4. USING KUBERNETES CONFIGMAPS AND SECRETS

The **camel-kubernetes** component also provides Property Placeholder functions that loads the property values from Kubernetes `ConfigMaps` or **Secrets**.

For more information see [Property Placeholder](#).

69.5. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
camel.cluster.kubernetes.attributes	Custom service attributes.		Map

Name	Description	Default	Type
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long

Name	Description	Default	Type
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-deployments.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-events.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>Boolean</code>

CHAPTER 70. KUBERNETES CONFIGMAP

Since Camel 2.17

Both producer and consumer are supported

The Kubernetes ConfigMap component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes ConfigMap operations and a consumer to consume events related to ConfigMap objects.

70.1. DEPENDENCIES

When using **kubernetes-config-maps** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have the support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

70.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

70.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

70.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

70.3. COMPONENT OPTIONS

The Kubernetes ConfigMap component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

70.4. ENDPOINT OPTIONS

The Kubernetes ConfigMap endpoint is configured using URI syntax:

```
kubernetes-config-maps:masterUrl
```

with the following path and query parameters:

70.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

70.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

70.5. MESSAGE HEADERS

The Kubernetes ConfigMap component supports 7 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesConfigMapsLabels (producer) Constant: KUBERNETES_CONFIGMAPS_LABELS	The ConfigMap labels.		Map
CamelKubernetesConfigMapName (producer) Constant: KUBERNETES_CONFIGMAP_NAME	The ConfigMap name.		String
CamelKubernetesConfigData (producer) Constant: KUBERNETES_CONFIGMAP_DATA	The ConfigMap Data.		Map

Name	Description	Default	Type
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

70.6. SUPPORTED PRODUCER OPERATION

- listConfigMaps
- listConfigMapsByLabels
- getConfigMap
- createConfigMap
- updateConfigMap
- deleteConfigMap

70.7. KUBERNETES CONFIGMAPS PRODUCER EXAMPLES

- listConfigMaps: this operation list the configmaps

```
from("direct:list").
  to("kubernetes-config-maps:///?kubernetesClient=#kubernetesClient&operation=listConfigMaps").
  to("mock:result");
```

This operation returns a list of ConfigMaps from your cluster.

- listConfigMapsByLabels: this operation lists the configmaps selected by label.

```
from("direct:listByLabels").process(new Processor() {
```

```

@Override
public void process(Exchange exchange) throws Exception {
    Map<String, String> labels = new HashMap<>();
    labels.put("key1", "value1");
    labels.put("key2", "value2");

    exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_CONFIGMAPS_LABELS, labels);
}
});
to("kubernetes-config-maps:///?
kubernetesClient=#kubernetesClient&operation=listConfigMapsByLabels").
to("mock:result");

```

This operation returns a list of ConfigMaps from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

70.8. KUBERNETES CONFIGMAPS CONSUMER EXAMPLE

```

fromF("kubernetes-config-maps://%s?oauthToken=%s", host, authToken)
.setHeader(KubernetesConstants.KUBERNETES_NAMESPACE_NAME, constant("default"))
.setHeader(KubernetesConstants.KUBERNETES_CONFIGMAP_NAME, constant("test"))
.process(new KuberntesProcessor()).to("mock:result");

public class KuberntesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        ConfigMap cm = exchange.getIn().getBody(ConfigMap.class);
        log.info("Got event with configmap name: " + cm.getMetadata().getName() + " and action " +
in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}

```

This consumer returns a list of events on the namespace default for the config map test.

70.9. USING CONFIGMAP PROPERTIES FUNCTION WITH KUBERNETES

The **camel-kubernetes** component include the following configmap related functions:

- **configmap** - A function to lookup the string property from Kubernetes ConfigMaps.
- **configmap-binary** - A function to lookup the binary property from Kubernetes ConfigMaps.

Camel reads Configmaps from the Kubernetes API Server. When RBAC is enabled on the cluster, the ServiceAccount that is used to run the application must have the proper permissions for such access.

Before the Kubernetes property placeholder functions can be used, they need to be configured with either (or both):

- **path** - A mount path that must be mounted to the running pod, to load the configmaps or secrets from local disk.

- kubernetes client – Autowired. An **io.fabric8.kubernetes.client.KubernetesClient** instance to use for connecting to the Kubernetes API server.

Camel first uses the mount paths (if configured) to lookup, and then fallback to use the **KubernetesClient**.

70.9.1. Using configmap with Kubernetes

For example, a configmap named **myconfig** in Kubernetes has two entries:

```
drink = beer
first = Carlsberg
```

Then these values can be used in your Camel routes such as:

```
<camelContext>
  <route>
    <from uri="direct:start"/>
    <log message="What {{configmap:myconfig/drink}} do you want?"/>
    <log message="I want {{configmap:myconfig/first}}"/>
  </route>
</camelContext>
```

You can also provide a default value in case a key does not exist:

```
<log message="I want {{configmap:myconfig/second:Heineken}}"/>
```

70.10. AUTOMATIC CAMEL CONTEXT RELOADING ON CONFIGMAPS REFRESH

Being able to reload the Camel context on a Configmap Refresh is done by specifying the following properties:

```
camel.vault.kubernetescm.refreshEnabled=true
camel.vault.kubernetescm.configmaps=Configmap
camel.main.context-reload-enabled = true
```

where **camel.vault.kubernetescm.refreshEnabled** enables the automatic context reload and **camel.vault.kubernetescm.configmaps** is a regex representing or a comma separated lists of the configmaps we want to track for updates.

Whenever a configmap listed in the property, it will be updated in the same namespace of the running application, the Camel context will be reloaded, refreshing the configmap value.

70.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String

Name	Description	Default	Type
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-events.enabled</code>	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-events.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-events.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-hpa.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-hpa.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-namespaces.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-namespaces.bridg-e-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-namespaces.enabled</code>	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-namespaces.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-namespaces.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-nodes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-nodes.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-nodes</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-nodes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-persistent-volumes-claims</code> component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-pods.enabled	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
camel.component.kubernetes-pods.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-replication-controllers</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component. openshift- deploymentconfigs. lazy-start- producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 71. KUBERNETES CUSTOM RESOURCES

Since Camel 3.7

Both producer and consumer are supported

The Kubernetes Custom Resources component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Custom Resources operations and a consumer to consume events related to Node objects.

71.1. DEPENDENCIES

When using **kubernetes-custom-resources** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

71.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

71.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

71.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

71.3. COMPONENT OPTIONS

The Kubernetes Custom Resources component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

71.4. ENDPOINT OPTIONS

The Kubernetes Custom Resources endpoint is configured using URI syntax:

```
kubernetes-custom-resources:masterUrl
```

with the following path and query parameters:

71.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

71.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

71.5. MESSAGE HEADERS

The Kubernetes Custom Resources component supports 13 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesCRDInstanceName (producer) Constant: KUBERNETES_CRD_INSTANCE_NAME	The deployment name.		String
CamelKubernetesCRDEventTimestamp (consumer) Constant: KUBERNETES_CRD_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long
CamelKubernetesCRDEventAction (consumer) Constant: KUBERNETES_CRD_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action

Name	Description	Default	Type
CamelKubernetesCRDName (producer) Constant: KUBERNETES_CRD_NAME	The Consumer CRD Resource name we would like to watch.		String
CamelKubernetesCRDGroup (producer) Constant: KUBERNETES_CRD_GROUP	The Consumer CRD Resource Group we would like to watch.		String
CamelKubernetesCRDScope (producer) Constant: KUBERNETES_CRD_SCOPE	The Consumer CRD Resource Scope we would like to watch.		String
CamelKubernetesCRDVersion (producer) Constant: KUBERNETES_CRD_VERSION	The Consumer CRD Resource Version we would like to watch.		String
CamelKubernetesCRDPlural (producer) Constant: KUBERNETES_CRD_PLURAL	The Consumer CRD Resource Plural we would like to watch.		String
CamelKubernetesCRDLabels (producer) Constant: KUBERNETES_CRD_LABELS	The CRD resource labels.		Map

Name	Description	Default	Type
CamelKubernetesCRDInstance (producer) Constant: KUBERNETES_CRD_INSTANCE	The manifest of the CRD resource to create as JSON string.		String
CamelKubernetesDeleteResult (producer) Constant: KUBERNETES_DELETE_RESULT	The result of the delete operation.		boolean

71.6. SUPPORTED PRODUCER OPERATION

- `listCustomResources`
- `listCustomResourcesByLabels`
- `getCustomResource`
- `deleteCustomResource`
- `createCustomResource`
- `updateCustomResource`

71.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
camel.cluster.kubernetes.attributes	Custom service attributes.		Map
camel.cluster.kubernetes.cluster-labels	Set the labels used to identify the pods composing the cluster.		Map

Name	Description	Default	Type
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long

Name	Description	Default	Type
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-events.enabled</code>	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-events.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-events.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-hpa.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-hpa.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-job.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-namespaces.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-namespaces.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-namespaces.enabled</code>	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-namespaces.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-namespaces.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-nodes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-nodes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-persistent-volumes-claims.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes-claims.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-persistent-volumes</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-persistent-volumes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-pods.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-replication-controllers</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-resources-quota</code> component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.kubernetes-resources-quota.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-resources-quota.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-builds.enabled	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
camel.component.openshift-builds.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-builds.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-deploymentconfigs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 72. KUBERNETES DEPLOYMENTS

Since Camel 2.20

Both producer and consumer are supported

The Kubernetes Deployments component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Deployments operations and a consumer to consume events related to Deployments objects.

72.1. DEPENDENCIES

When using **kubernetes-deployments** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

72.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

72.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

72.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

72.3. COMPONENT OPTIONS

The Kubernetes Deployments component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

72.4. ENDPOINT OPTIONS

The Kubernetes Deployments endpoint is configured using URI syntax:

```
kubernetes-deployments:masterUrl
```

with the following path and query parameters:

72.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

72.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

72.5. MESSAGE HEADERS

The Kubernetes Deployments component supports 8 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesDeploymentsLabels (producer) Constant: KUBERNETES_DEPLOYMENTS_LABELS	The deployment labels.		Map
CamelKubernetesDeploymentName (producer) Constant: KUBERNETES_DEPLOYMENT_NAME	The deployment name.		String
CamelKubernetesDeploymentSpec (producer) Constant: KUBERNETES_DEPLOYMENT_SPEC	The spec for a deployment.		DeploymentSpec
CamelKubernetesDeploymentReplicas (producer) Constant: KUBERNETES_DEPLOYMENT_REPLICAS	The desired instance count.		Integer

Name	Description	Default	Type
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

72.6. SUPPORTED PRODUCER OPERATION

- listDeployments
- listDeploymentsByLabels
- getDeployment
- createDeployment
- updateDeployment
- deleteDeployment
- scaleDeployment

72.7. KUBERNETES DEPLOYMENTS PRODUCER EXAMPLES

- listDeployments: this operation lists the deployments on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-deployments:///?
kubernetesClient=#kubernetesClient&operation=listDeployments").
  to("mock:result");
```

This operation returns a List of Deployment from your cluster.

- listDeploymentsByLabels: this operation lists the deployments by labels on a kubernetes cluster.

```

from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_DEPLOYMENTS_LABELS,
            labels);
    }
});
toF("kubernetes-deployments:///?
kubernetesClient=#kubernetesClient&operation=listDeploymentsByLabels").
to("mock:result");

```

This operation returns a List of Deployments from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

72.7.1. Kubernetes Deployments Consumer Example

```

fromF("kubernetes-deployments://%s?oauthToken=%s&namespace=default&resourceName=test",
host, authToken).process(new KubernetesProcessor()).to("mock:result");
public class KubernetesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        Deployment dp = exchange.getIn().getBody(Deployment.class);
        log.info("Got event with configmap name: " + dp.getMetadata().getName() + " and action " +
            in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}

```

This consumer returns a list of events on the namespace default for the deployment test.

72.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String

Name	Description	Default	Type
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long

Name	Description	Default	Type
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the kubernetes-config-maps component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-deployments.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-events.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-nodes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-persistent-volumes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-pods.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-pods.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-build-configs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-build-configs.enabled</code>	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
<code>camel.component.openshift-build-configs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-build-configs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-builds.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the <code>openshift-deploymentconfigs</code> component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 73. KUBERNETES EVENT

Since Camel 3.20

Both producer and consumer are supported

The Kubernetes Event component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Event operations and a consumer to consume events related to Event objects.

73.1. DEPENDENCIES

When using **kubernetes-events** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

73.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

73.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

73.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

73.3. COMPONENT OPTIONS

The Kubernetes Event component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

73.4. ENDPOINT OPTIONS

The Kubernetes Event endpoint is configured using URI syntax:

```
kubernetes-events:masterUrl
```

with the following path and query parameters:

73.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

73.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

73.5. MESSAGE HEADERS

The Kubernetes Event component supports 14 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesEventsLabels (producer) Constant: KUBERNETES_EVENTS_LABELS	The event labels.		Map
CamelKubernetesEventTime (producer) Constant: KUBERNETES_EVENT_TIME	The event time in ISO-8601 extended offset date-time format, such as '2011-12-03T10:15:3001:00'.	server time	String
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action

Name	Description	Default	Type
CamelKubernetesEventType (producer) Constant: KUBERNETES_EVENT_TYPE	The event type.		String
CamelKubernetesEventReason (producer) Constant: KUBERNETES_EVENT_REASON	The event reason.		String
CamelKubernetesEventNote (producer) Constant: KUBERNETES_EVENT_NOTE	The event note.		String
CamelKubernetesEventRegarding (producer) Constant: KUBERNETES_EVENT_REGARDING	The event regarding.		ObjectReference
CamelKubernetesEventRelated (producer) Constant: KUBERNETES_EVENT_RELATED	The event related.		ObjectReference
CamelKubernetesEventReportingController (producer) Constant: KUBERNETES_EVENT_REPORTING_CONTROLLER	The event reporting controller.		String

Name	Description	Default	Type
CamelKubernetesEventReportingInstance (producer) Constant: KUBERNETES_EVENT_REPORTING_INSTANCE	The event reporting instance.		String
CamelKubernetesEventName (producer) Constant: KUBERNETES_EVENT_NAME	The event name.		String
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

73.6. SUPPORTED PRODUCER OPERATION

- `listEvents`
- `listEventsByLabels`
- `getEvent`
- `createEvent`
- `updateEvent`
- `deleteEvent`

73.7. KUBERNETES EVENTS PRODUCER EXAMPLES

- `listEvents`: this operation lists the events.

```
from("direct:list").
  to("kubernetes-events:///kubernetesClient=#kubernetesClient&operation=listEvents").
  to("mock:result");
```

This operation returns a list of events from your cluster. The type of the events is **io.fabric8.kubernetes.api.model.events.v1.Event**.

To indicate from which namespace the events are expected, it is possible to set the message header **CamelKubernetesNamespaceName**. By default, the events of all namespaces are returned.

- `listEventsByLabels`: this operation lists the events selected by labels.

```
from("direct:listByLabels").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENTS_LABELS,
labels);
    }
});
to("kubernetes-events:///?kubernetesClient=#kubernetesClient&operation=listEventsByLabels").
to("mock:result");
```

This operation returns a list of events from your cluster that occurred in any namespaces, using a label selector (in the example above only expect events which have the label "key1" set to "value1" and the label "key2" set to "value2"). The type of the events is **io.fabric8.kubernetes.api.model.events.v1.Event**.

This operation expects the message header **CamelKubernetesEventsLabels** to be set to a **Map<String, String>** where the key-value pairs represent the expected label names and values.

- `getEvent`: this operation gives a specific event.

```
from("direct:get").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_NAMESPACE_NAME,
"test");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENT_NAME,
"event1");
    }
});
to("kubernetes-events:///?kubernetesClient=#kubernetesClient&operation=getEvent").
to("mock:result");
```

This operation returns the event matching the criteria from your cluster. The type of the event is **io.fabric8.kubernetes.api.model.events.v1.Event**.

This operation expects two message headers which are **CamelKubernetesNamespaceName** and **CamelKubernetesEventName**, the first one needs to be set to the name of the target namespace and second one needs to be set to the target name of event.

If no matching event could be found, **null** is returned.

- `createEvent`: this operation creates a new event.


```

from("direct:get").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_NAMESPACE_NAME,
"default");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENT_NAME,
"test1");
        Map<String, String> labels = new HashMap<>();
        labels.put("this", "rocks");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENTS_LABELS,
labels);

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION_PRODUCER,
"Some Action");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENT_TYPE,
"Normal");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENT_REASON,
"Some Reason");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENT_REPORTING_CONTROLLER,
"Some-Reporting-Controller");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENT_REPORTING_INSTANCE,
"Some-Reporting-Instance");
    }
});
to("kubernetes-events:///kubernetesClient=#kubernetesClient&operation=createEvent");
to("mock:result");

```

This operation publishes a new event in your cluster. An event can be created in two ways either from message headers or directly from an **io.fabric8.kubernetes.api.model.events.v1.EventBuilder**.

Whatever the way used to create the event:

- The operation expects two message headers which are **CamelKubernetesNamespaceName** and **CamelKubernetesEventName**, to set respectively the name of namespace and the name of the produced event.
- The operation supports the message header **CamelKubernetesEventsLabels** to set the labels to the produced event.

The message headers that can be used to create an event are **CamelKubernetesEventTime**, **CamelKubernetesEventAction**, **CamelKubernetesEventType**, **CamelKubernetesEventReason**, **CamelKubernetesEventNote**, **CamelKubernetesEventRegarding**, **CamelKubernetesEventRelated**, **CamelKubernetesEventReportingController** and **CamelKubernetesEventReportingInstance**.

In case the supported message headers are not enough for a specific use case, it is still possible to set the message body with an object of type **io.fabric8.kubernetes.api.model.events.v1.EventBuilder** representing a prefilled builder to use when creating the event. Please note that the labels, name of event and name of namespace are always set from the message headers, even when the builder is provided.

- **updateEvent**: this operation updates an existing event.

The behavior is exactly the same as **createEvent**, only the name of the operation is different.

- `deleteEvent`: this operation deletes an existing event.

```
from("direct:get").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_NAMESPACE_NAME,
"default");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_EVENT_NAME,
"test1");
    }
});
to("kubernetes-events:///?kubernetesClient=#kubernetesClient&operation=deleteEvent").
to("mock:result");
```

This operation removes an existing event from your cluster. It returns a **boolean** to indicate whether the operation was successful or not.

This operation expects two message headers which are **CamelKubernetesNamespaceName** and **CamelKubernetesEventName**, the first one needs to be set to the name of the target namespace and second one needs to be set to the target name of event.

73.8. KUBERNETES EVENTS CONSUMER EXAMPLE

```
fromF("kubernetes-events://%s?oauthToken=%s", host, authToken)
.setHeader(KubernetesConstants.KUBERNETES_NAMESPACE_NAME, constant("default"))
.setHeader(KubernetesConstants.KUBERNETES_EVENT_NAME, constant("test"))
.process(new KuberntesProcessor()).to("mock:result");

public class KuberntesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        Event cm = exchange.getIn().getBody(Event.class);
        log.info("Got event with event name: " + cm.getMetadata().getName() + " and action " +
in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}
```

This consumer returns a message per event received on the namespace "default" for the event "test". It also set the action (**io.fabric8.kubernetes.client.Watcher.Action**) in the message header **CamelKubernetesEventAction** and the timestamp (**long**) in the message header **CamelKubernetesEventTimestamp**.

73.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long

Name	Description	Default	Type
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-deployments.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-events.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component. .openshift- deploymentconfi gs.kubernetes- client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component. .openshift- deploymentconfi gs.lazy-start- producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 74. KUBERNETES HPA

Since Camel 2.23

Both producer and consumer are supported

The Kubernetes HPA component is one of the [Kubernetes Components](#) which provides a producer to execute kubernetes Horizontal Pod Autoscaler operations and a consumer to consume events related to Horizontal Pod Autoscaler objects.

74.1. DEPENDENCIES

When using **kubernetes-hpa** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

74.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

74.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

74.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

74.3. COMPONENT OPTIONS

The Kubernetes HPA component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

74.4. ENDPOINT OPTIONS

The Kubernetes HPA endpoint is configured using URI syntax:

```
kubernetes-hpa:masterUrl
```

with the following path and query parameters:

74.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

74.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

74.5. MESSAGE HEADERS

The Kubernetes HPA component supports 7 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesHPAName (producer) Constant: KUBERNETES_HPA_NAME	The HPA name.		String
CamelKubernetesHPASpec (producer) Constant: KUBERNETES_HPA_SPEC	The spec for a HPA.		HorizontalPodAutoscalerSpec
CamelKubernetesHPALabels (producer) Constant: KUBERNETES_HPA_LABELS	The HPA labels.		Map

Name	Description	Default	Type
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

74.6. SUPPORTED PRODUCER OPERATION

- listHPA
- listHPAByLabels
- getHPA
- createHPA
- updateHPA
- deleteHPA

74.7. KUBERNETES HPA PRODUCER EXAMPLES

- listHPA: this operation lists the HPAs on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-hpa:///?kubernetesClient=#kubernetesClient&operation=listHPA").
  to("mock:result");
```

This operation returns a List of HPAs from your cluster.

- listDeploymentsByLabels: this operation lists the HPAs by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
```

```

@Override
public void process(Exchange exchange) throws Exception {
    Map<String, String> labels = new HashMap<>();
    labels.put("key1", "value1");
    labels.put("key2", "value2");
    exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_HPA_LABELS, labels);
}
});
toF("kubernetes-hpa:///?kubernetesClient=#kubernetesClient&operation=listHPAByLabels").
to("mock:result");

```

This operation returns a List of HPAs from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

74.8. KUBERNETES HPA CONSUMER EXAMPLE

```

fromF("kubernetes-hpa://%s?oauthToken=%s&namespace=default&resourceName=test", host,
authToken).process(new KubernetesProcessor()).to("mock:result");
public class KubernetesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        HorizontalPodAutoscaler hpa = exchange.getIn().getBody(HorizontalPodAutoscaler.class);
        log.info("Got event with hpa name: " + hpa.getMetadata().getName() + " and action " +
in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}

```

This consumer returns a list of events on the namespace default for the hpa test.

74.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer

Name	Description	Default	Type
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-config-maps.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-config-maps.enabled	Whether to enable auto configuration of the kubernetes-config-maps component. This is enabled by default.		Boolean
camel.component.kubernetes-config-maps.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-config-maps.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-custom-resources.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-nodes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-persistent-volumes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>Boolean</code>
<code>camel.component.kubernetes-pods.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	<code>true</code>	<code>Boolean</code>
<code>camel.component.kubernetes-pods.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	<code>false</code>	<code>Boolean</code>
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		<code>Boolean</code>
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-build-configs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-build-configs.enabled</code>	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
<code>camel.component.openshift-build-configs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-build-configs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-builds.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.openshift-builds.enabled	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
camel.component.openshift-builds.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.openshift-builds.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-deploymentconfigs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-deploymentconfigs.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 75. KUBERNETES JOB

Since Camel 2.23

Both producer and consumer are supported

The Kubernetes Job component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Job operations and a consumer to consume events related to Job objects.

75.1. DEPENDENCIES

When using **kubernetes-job** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

75.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

75.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

75.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections list all the options, firstly for the component followed by the endpoint.

75.3. COMPONENT OPTIONS

The Kubernetes Job component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

75.4. ENDPOINT OPTIONS

The Kubernetes Job endpoint is configured using URI syntax:

```
kubernetes-job:masterUrl
```

with the following path and query parameters:

75.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

75.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

75.5. MESSAGE HEADERS

The Kubernetes Job component supports 5 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesJobName (producer) Constant: KUBERNETES_JOB_NAME	The Job name.		String
CamelKubernetesJobSpec (producer) Constant: KUBERNETES_JOB_SPEC	The spec for a Job.		JobSpec
CamelKubernetesJobLabels (producer) Constant: KUBERNETES_JOB_LABELS	The Job labels.		Map

75.6. SUPPORTED PRODUCER OPERATION

- listJob
- listJobByLabels
- getJob
- createJob

- updateJob
- deleteJob

75.7. KUBERNETES JOB PRODUCER EXAMPLES

- listJob: this operation list the jobs on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-job:///?kubernetesClient=#kubernetesClient&operation=listJob").
  to("mock:result");
```

This operation returns a List of Job from your cluster.

- listJobByLabels: this operation list the jobs by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_JOB_LABELS, labels);
    }
});
toF("kubernetes-job:///?kubernetesClient=#kubernetesClient&operation=listJobByLabels").
to("mock:result");
```

This operation returns a List of Jobs from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

- createJob: This operation creates a job on a Kubernetes Cluster.

Example (see [Create Job example](#) for more information)

```
import java.util.ArrayList;
import java.util.Date;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import javax.inject.Inject;

import org.apache.camel.Endpoint;
import org.apache.camel.builder.RouteBuilder;
import org.apache.camel.cdi.Uri;
import org.apache.camel.component.kubernetes.KubernetesConstants;
import org.apache.camel.component.kubernetes.KubernetesOperations;

import io.fabric8.kubernetes.api.model.Container;
import io.fabric8.kubernetes.api.model.ObjectMeta;
import io.fabric8.kubernetes.api.model.PodSpec;
import io.fabric8.kubernetes.api.model.PodTemplateSpec;
import io.fabric8.kubernetes.api.model.batch.JobSpec;
```

```

public class KubernetesCreateJob extends RouteBuilder {

    @Inject
    @Uri("timer:foo?delay=1000&repeatCount=1")
    private Endpoint inputEndpoint;

    @Inject
    @Uri("log:output")
    private Endpoint resultEndpoint;

    @Override
    public void configure() {
        // you can configure the route rule with Java DSL here

        from(inputEndpoint)
            .routeId("kubernetes-jobcreate-client")
            .process(exchange -> {
                exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_JOB_NAME, "camel-
job"); //DNS-1123 subdomain must consist of lower case alphanumeric characters, '-' or '.', and must
start and end with an alphanumeric character (e.g. 'example.com', regex used for validation is '[a-z0-
9]([-a-z0-9]*[a-z0-9])?(\.[-a-z0-9]([-a-z0-9]*[a-z0-9])?)*')
                exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_NAMESPACE_NAME,
"default");

                Map<String, String> joblabels = new HashMap<String, String>();
                joblabels.put("jobLabelKey1", "value1");
                joblabels.put("jobLabelKey2", "value2");
                joblabels.put("app", "jobFromCamelApp");
                exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_JOB_LABELS,
joblabels);

                exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_JOB_SPEC,
generateJobSpec());
            })
            .toF("kubernetes-job:///{{kubernetes-master-url}}?oauthToken={{kubernetes-oauth-
token:}}&operation=" + KubernetesOperations.CREATE_JOB_OPERATION)
            .log("Job created:")
            .process(exchange -> {
                System.out.println(exchange.getIn().getBody());
            })
            .to(resultEndpoint);
    }

    private JobSpec generateJobSpec() {
        JobSpec js = new JobSpec();

        PodTemplateSpec pts = new PodTemplateSpec();

        PodSpec ps = new PodSpec();
        ps.setRestartPolicy("Never");
        ps.setContainers(generateContainers());
        pts.setSpec(ps);

        ObjectMeta metadata = new ObjectMeta();
        Map<String, String> annotations = new HashMap<String, String>();
        annotations.put("jobMetadataAnnotation1", "random value");
    }
}

```

```

metadata.setAnnotations(annotations);

Map<String, String> podlabels = new HashMap<String, String>();
podlabels.put("podLabelKey1", "value1");
podlabels.put("podLabelKey2", "value2");
podlabels.put("app", "podFromCamelApp");
metadata.setLabels(podlabels);

pts.setMetadata(metadata);
js.setTemplate(pts);
return js;
}

private List<Container> generateContainers() {
    Container container = new Container();
    container.setName("pi");
    container.setImage("perl");
    List<String> command = new ArrayList<String>();
    command.add("echo");
    command.add("Job created from Apache Camel code at " + (new Date()));
    container.setCommand(command);
    List<Container> containers = new ArrayList<Container>();
    containers.add(container);
    return containers;
}
}

```

75.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean

Name	Description	Default	Type
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-config-maps.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-config-maps.enabled	Whether to enable auto configuration of the kubernetes-config-maps component. This is enabled by default.		Boolean
camel.component.kubernetes-config-maps.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-config-maps.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-custom-resources.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>Boolean</code>
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	<code>true</code>	<code>Boolean</code>
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-resources-quota</code> component. This is enabled by default.		<code>Boolean</code>
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>

Name	Description	Default	Type
camel.component.kubernetes-resources-quota.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>Boolean</code>

CHAPTER 76. KUBERNETES NAMESPACES

Since Camel 2.17

Both producer and consumer are supported

The Kubernetes Namespaces component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Namespace operations and a consumer to consume events related to Namespace events.

76.1. DEPENDENCIES

When using **kubernetes-namespaces** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

76.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

76.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

76.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

76.3. COMPONENT OPTIONS

The Kubernetes Namespaces component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

76.4. ENDPOINT OPTIONS

The Kubernetes Namespaces endpoint is configured using URI syntax:

```
kubernetes-namespaces:masterUrl
```

with the following path and query parameters:

76.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

76.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

76.5. MESSAGE HEADERS

The Kubernetes Namespaces component supports 5 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesNamespaceLabels (producer) Constant: KUBERNETES_NAMESPACE_LABELS	The namespace labels.		Map
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

76.6. SUPPORTED PRODUCER OPERATION

- listNamespaces

- listNamespacesByLabels
- getNamespace
- createNamespace
- updateNamespace
- deleteNamespace

76.7. KUBERNETES NAMESPACES PRODUCER EXAMPLES

- listNamespaces: this operation list the namespaces on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-deployments:///?
kubernetesClient=#kubernetesClient&operation=listNamespaces").
  to("mock:result");
```

This operation returns a List of namespaces from your cluster.

- listNamespacesByLabels: this operation list the namespaces by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_NAMESPACES_LABELS,
        labels);
    }
});
toF("kubernetes-deployments:///?
kubernetesClient=#kubernetesClient&operation=listNamespacesByLabels").
  to("mock:result");
```

This operation returns a List of Namespaces from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

76.8. KUBERNETES NAMESPACES CONSUMER EXAMPLE

```
fromF("kubernetes-namespaces://%s?oauthToken=%s&namespace=default", host,
authToken).process(new KuberntesProcessor()).to("mock:result");
public class KuberntesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        Namespace ns = exchange.getIn().getBody(Namespace.class);
        log.info("Got event with configmap name: " + ns.getMetadata().getName() + " and action " +
in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}
```

This consumer returns a list of events on the namespace default.

76.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer

Name	Description	Default	Type
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-hpa.enabled	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
camel.component.kubernetes-hpa.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-job.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-namespaces.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-namespaces.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-namespaces.enabled</code>	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-namespaces.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-resources-quota</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-build-configs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-build-configs.enabled</code>	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
<code>camel.component.openshift-build-configs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-build-configs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-builds.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the <code>openshift-deploymentconfigs</code> component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 77. KUBERNETES NODES

Since Camel 2.17

Both producer and consumer are supported

The Kubernetes Nodes component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Node operations and a consumer to consume events related to Node objects.

77.1. DEPENDENCIES

When using **kubernetes-nodes** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
<groupId>org.apache.camel.springboot</groupId>
<artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

77.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

77.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

77.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

77.3. COMPONENT OPTIONS

The Kubernetes Nodes component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

77.4. ENDPOINT OPTIONS

The Kubernetes Nodes endpoint is configured using URI syntax:

```
kubernetes-nodes:masterUrl
```

with the following path and query parameters:

77.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

77.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

77.5. MESSAGE HEADERS

The Kubernetes Nodes component supports 6 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNodesLabels (producer) Constant: KUBERNETES_NODES_LABELS	The node labels.		Map
CamelKubernetesNodeName (producer) Constant: KUBERNETES_NODE_NAME	The node name.		String
CamelKubernetesNodeSpec (producer) Constant: KUBERNETES_NODE_SPEC	The spec for a node.		NodeSpec
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action

Name	Description	Default	Type
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

77.6. SUPPORTED PRODUCER OPERATION

- `listNodes`
- `listNodesByLabels`
- `getNode`
- `createNode`
- `updateNode`
- `deleteNode`

77.7. KUBERNETES NODES PRODUCER EXAMPLES

- `listNodes`: this operation list the nodes on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-nodes:///?kubernetesClient=#kubernetesClient&operation=listNodes").
  to("mock:result");
```

This operation returns a List of Nodes from your cluster.

- `listNodesByLabels`: this operation list the nodes by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_NODES_LABELS,
labels);
    }
});
toF("kubernetes-deployments:///?
kubernetesClient=#kubernetesClient&operation=listNodesByLabels").
to("mock:result");
```

This operation returns a List of Nodes from your cluster, using a label selector (with `key1` and `key2`, with value `value1` and `value2`).

77.8. KUBERNETES NODES CONSUMER EXAMPLE

```
fromF("kubernetes-nodes://%s?oauthToken=%s&resourceName=test", host,
authToken).process(new KubernetesProcessor()).to("mock:result");
public class KubernetesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        Node node = exchange.getIn().getBody(Node.class);
        log.info("Got event with configmap name: " + node.getMetadata().getName() + " and action " +
in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}
```

This consumer returns a list of events for the node test.

77.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String

Name	Description	Default	Type
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-events.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-hpa.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-hpa.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-namespaces.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-namespaces.bridg-e-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-namespaces.enabled</code>	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-namespaces.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-nodes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-pods.enabled	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
camel.component.kubernetes-pods.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-build-configs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-build-configs.enabled</code>	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
<code>camel.component.openshift-build-configs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-build-configs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-builds.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the <code>openshift-deploymentconfigs</code> component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component. openshift- deploymentconfigs. lazy-start- producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 78. KUBERNETES PERSISTENT VOLUME

Since Camel 2.17

Only producer is supported

The Kubernetes Persistent Volume component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Persistent Volume operations.

78.1. DEPENDENCIES

When using **kubernetes-persistent-volumes** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
<groupId>org.apache.camel.springboot</groupId>
<artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

78.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

78.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

78.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

78.3. COMPONENT OPTIONS

The Kubernetes Persistent Volume component supports 3 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (producer)	Autowired To use an existing kubernetes client.		KubernetesClient
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

78.4. ENDPOINT OPTIONS

The Kubernetes Persistent Volume endpoint is configured using URI syntax:

```
kubernetes-persistent-volumes:masterUrl
```

with the following path and query parameters:

78.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (producer)	Required Kubernetes Master url.		String

78.4.2. Query Parameters (21 parameters)

Name	Description	Default	Type
apiVersion (producer)	The Kubernetes API Version to use.		String
dnsDomain (producer)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (producer)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (producer)	The namespace.		String
operation (producer)	Producer operation to do on Kubernetes.		String
portName (producer)	The port name, used for ServiceCall EIP.		String
portProtocol (producer)	The port protocol, used for ServiceCall EIP.	tcp	String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String

Name	Description	Default	Type
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

78.5. MESSAGE HEADERS

The Kubernetes Persistent Volume component supports 3 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String

Name	Description	Default	Type
CamelKubernetesPersistentVolumeLabels (producer) Constant: KUBERNETES_PERSISTENT_VOLUMES_LABELS	The persistent volume labels.		Map
CamelKubernetesPersistentVolumeName (producer) Constant: KUBERNETES_PERSISTENT_VOLUME_NAME	The persistent volume name.		String

78.6. SUPPORTED PRODUCER OPERATION

- `listPersistentVolumes`
- `listPersistentVolumesByLabels`
- `getPersistentVolume`

78.7. KUBERNETES PERSISTENT VOLUMES PRODUCER EXAMPLES

- `listPersistentVolumes`: this operation lists the pv on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-persistent-volumes:///?
kubernetesClient=#kubernetesClient&operation=listPersistentVolumes").
  to("mock:result");
```

This operation returns a List of pv from your cluster.

- `listPersistentVolumesByLabels`: this operation lists the pv by labels on a kubernetes cluster

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_PERSISTENT_VOLUMES_LABELS, labels);
    }
})
```

```
});
toF("kubernetes-persistent-volumes:///?
kubernetesClient=#kubernetesClient&operation=listPersistentVolumesByLabels").
to("mock:result");
```

This operation returns a List of pv from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

78.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String

Name	Description	Default	Type
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-hpa.enabled	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
camel.component.kubernetes-hpa.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-namespaces.bridg e-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kube netes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-namespaces.lazy- start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired- enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-nodes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-persistent-volumes-claims.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes-claims.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-persistent-volumes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-pods.enabled	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
camel.component.kubernetes-pods.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-pods.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-replication-controllers.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-replication-controllers</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-resources-quota</code> component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.kubernetes-resources-quota.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-resources-quota.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-build-configs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-build-configs.enabled</code>	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
<code>camel.component.openshift-build-configs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-build-configs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-builds.enabled	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
camel.component.openshift-builds.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.openshift-builds.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-deploymentconfigs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-deploymentconfigs.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 79. KUBERNETES PERSISTENT VOLUME CLAIM

Since Camel 2.17

Only producer is supported

The Kubernetes Persistent Volume Claim component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Persistent Volume Claims operations.

79.1. DEPENDENCIES

When using **kubernetes-persistent-volumes-claims** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

79.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

79.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

79.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

79.3. COMPONENT OPTIONS

The Kubernetes Persistent Volume Claim component supports 3 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (producer)	Autowired To use an existing kubernetes client.		KubernetesClient
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

79.4. ENDPOINT OPTIONS

The Kubernetes Persistent Volume Claim endpoint is configured using URI syntax:

```
kubernetes-persistent-volumes-claims:masterUrl
```

with the following path and query parameters:

79.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (producer)	Required Kubernetes Master url.		String

79.4.2. Query Parameters (21 parameters)

Name	Description	Default	Type
apiVersion (producer)	The Kubernetes API Version to use.		String
dnsDomain (producer)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (producer)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (producer)	The namespace.		String
operation (producer)	Producer operation to do on Kubernetes.		String
portName (producer)	The port name, used for ServiceCall EIP.		String
portProtocol (producer)	The port protocol, used for ServiceCall EIP.	tcp	String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String

Name	Description	Default	Type
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

79.5. MESSAGE HEADERS

The Kubernetes Persistent Volume Claim component supports 5 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String

Name	Description	Default	Type
CamelKubernetesPersistentVolumeClaimsLabels (producer) Constant: KUBERNETES_PERSISTENT_VOLUME_CLAIMS_LABELS	The persistent volume claim labels.		Map
CamelKubernetesPersistentVolumeClaimName (producer) Constant: KUBERNETES_PERSISTENT_VOLUME_CLAIM_NAME	The persistent volume claim name.		String
CamelKubernetesPersistentVolumeClaimSpec (producer) Constant: KUBERNETES_PERSISTENT_VOLUME_CLAIM_SPEC	The spec for a persistent volume claim.		PersistentVolumeClaimSpec

79.6. SUPPORTED PRODUCER OPERATION

- `listPersistentVolumesClaims`
- `listPersistentVolumesClaimsByLabels`
- `getPersistentVolumeClaim`
- `createPersistentVolumeClaim`
- `updatePersistentVolumeClaim`
- `deletePersistentVolumeClaim`

79.7. KUBERNETES PERSISTENT VOLUME CLAIMS PRODUCER EXAMPLES

- `listPersistentVolumesClaims`: this operation lists the pvc on a kubernetes cluster.

```

from("direct:list").
  toF("kubernetes-persistent-volumes-claims:///?
kubernetesClient=#kubernetesClient&operation=listPersistentVolumesClaims").
  to("mock:result");

```

This operation returns a List of pvc from your cluster.

- `listPersistentVolumesClaimsByLabels`: this operation lists the pvc by labels on a kubernetes cluster.

```

from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_PERSISTENT_VOLUMES_CLAIM
S_LABELS, labels);
    }
});
toF("kubernetes-persistent-volumes-claims:///?
kubernetesClient=#kubernetesClient&operation=listPersistentVolumesClaimsByLabels").
to("mock:result");

```

This operation returns a List of pvc from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

79.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean

Name	Description	Default	Type
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-config-maps.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-config-maps.enabled	Whether to enable auto configuration of the kubernetes-config-maps component. This is enabled by default.		Boolean
camel.component.kubernetes-config-maps.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-config-maps.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-custom-resources.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-custom-resources.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-custom-resources.enabled	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
camel.component.kubernetes-custom-resources.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-custom-resources.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-deployments.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-hpa.enabled	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
camel.component.kubernetes-hpa.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-job.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-namespaces.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridg-e-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-nodes.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-nodes</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-nodes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-persistent-volumes-claims</code> component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-pods.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-replication-controllers</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-resources-quota</code> component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.kubernetes-resources-quota.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-resources-quota.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-build-configs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-builds.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the <code>openshift-deploymentconfigs</code> component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 80. KUBERNETES PODS

Since Camel 2.17

Both producer and consumer are supported

The Kubernetes Pods component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Pods operations and a consumer to consume events related to Pod Objects.

80.1. DEPENDENCIES

When using **kubernetes-pods** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

80.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

80.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

80.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

80.3. COMPONENT OPTIONS

The Kubernetes Pods component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

80.4. ENDPOINT OPTIONS

The Kubernetes Pods endpoint is configured using URI syntax:

```
kubernetes-pods:masterUrl
```

with the following path and query parameters:

80.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

80.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

80.5. MESSAGE HEADERS

The Kubernetes Pods component supports 7 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesPodsLabels (producer) Constant: KUBERNETES_PODS_LABELS	The pod labels.		Map
CamelKubernetesPodName (producer) Constant: KUBERNETES_POD_NAME	The pod name.		String
CamelKubernetesPodSpec (producer) Constant: KUBERNETES_POD_SPEC	The spec for a pod.		PodSpec

Name	Description	Default	Type
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

80.6. SUPPORTED PRODUCER OPERATION

- `listPods`
- `listPodsByLabels`
- `getPod`
- `createPod`
- `updatePod`
- `deletePod`

80.7. KUBERNETES PODS PRODUCER EXAMPLES

- `listPods`: this operation list the pods on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-pods:///?kubernetesClient=#kubernetesClient&operation=listPods").
  to("mock:result");
```

This operation returns a List of Pods from your cluster.

- `listPodsByLabels`: this operation list the pods by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
```

```

@Override
public void process(Exchange exchange) throws Exception {
    Map<String, String> labels = new HashMap<>();
    labels.put("key1", "value1");
    labels.put("key2", "value2");
    exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_PODS_LABELS,
labels);
}
});
toF("kubernetes-pods:///?kubernetesClient=#kubernetesClient&operation=listPodsByLabels").
to("mock:result");

```

This operation returns a List of Pods from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

80.8. KUBERNETES PODS CONSUMER EXAMPLE

```

fromF("kubernetes-pods://%s?oauthToken=%s&namespace=default&resourceName=test", host,
authToken).process(new KubernetesProcessor()).to("mock:result");
public class KubernetesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        Pod pod = exchange.getIn().getBody(Pod.class);
        log.info("Got event with configmap name: " + pod.getMetadata().getName() + " and action " +
in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}

```

This consumer returns a list of events on the namespace default for the pod test.

80.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer

Name	Description	Default	Type
camel.cluster.kubernetes.enabled	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
camel.cluster.kubernetes.id	Cluster Service ID.		String
camel.cluster.kubernetes.jitter-factor	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
camel.cluster.kubernetes.kubernetes-namespace	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
camel.cluster.kubernetes.lease-duration-millis	The default duration of the lease for the current leader.		Long
camel.cluster.kubernetes.master-url	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
camel.cluster.kubernetes.order	Service lookup order/priority.		Integer
camel.cluster.kubernetes.pod-name	Set the name of the current pod (autodetected from container host name by default).		String
camel.cluster.kubernetes.renew-deadline-millis	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
camel.cluster.kubernetes.retry-period-millis	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
camel.component.kubernetes-config-maps.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-config-maps.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-config-maps.enabled	Whether to enable auto configuration of the kubernetes-config-maps component. This is enabled by default.		Boolean
camel.component.kubernetes-config-maps.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-config-maps.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-custom-resources.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-deployments.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-events.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-hpa.enabled	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
camel.component.kubernetes-hpa.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-namespaces.enabled</code>	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-namespaces.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-namespaces.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-nodes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-nodes.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-nodes.enabled</code>	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-nodes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-build-configs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-build-configs.enabled</code>	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
<code>camel.component.openshift-build-configs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-build-configs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-builds.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 81. KUBERNETES REPLICATION CONTROLLER

Since Camel 2.17

Both producer and consumer are supported

The Kubernetes Replication Controller component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Replication controller operations and a consumer to consume events related to Replication Controller objects.

81.1. DEPENDENCIES

When using **kubernetes-replication-controllers** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

81.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

81.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

81.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

81.3. COMPONENT OPTIONS

The Kubernetes Replication Controller component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

81.4. ENDPOINT OPTIONS

The Kubernetes Replication Controller endpoint is configured using URI syntax:

```
kubernetes-replication-controllers:masterUrl
```

with the following path and query parameters:

81.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

81.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

81.5. MESSAGE HEADERS

The Kubernetes Replication Controller component supports 8 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesReplicationControllersLabels (producer) Constant: KUBERNETES_REPLICATION_CONTROLLERS_LABELS	The replication controller labels.		Map
CamelKubernetesReplicationControllerName (producer) Constant: KUBERNETES_REPLICATION_CONTROLLER_NAME	The replication controller name.		String
CamelKubernetesReplicationControllerSpec (producer) Constant: KUBERNETES_REPLICATION_CONTROLLER_SPEC	The spec for a replication controller.		ReplicationControllerSpec

Name	Description	Default	Type
CamelKubernetesReplicationControllerReplicas (producer) Constant: KUBERNETES_REPLICATION_CONTROLLER_REPLICAS	The number of replicas for a replication controller during the Scale operation.		Integer
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

81.6. SUPPORTED PRODUCER OPERATION

- `listReplicationControllers`
- `listReplicationControllersByLabels`
- `getReplicationController`
- `createReplicationController`
- `updateReplicationController`
- `deleteReplicationController`
- `scaleReplicationController`

81.7. KUBERNETES REPLICATION CONTROLLERS PRODUCER EXAMPLES

- `listReplicationControllers`: this operation list the RCs on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-replication-controllers:///?
kubernetesClient=#kubernetesClient&operation=listReplicationControllers").
  to("mock:result");
```

This operation returns a List of RCs from your cluster.

- `listReplicationControllersByLabels`: this operation list the RCs by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_REPLICATION_CONTROLLERS_
LABELS, labels);
    }
});
toF("kubernetes-replication-controllers:///?
kubernetesClient=#kubernetesClient&operation=listReplicationControllersByLabels").
  to("mock:result");
```

This operation returns a List of RCs from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

81.8. KUBERNETES REPLICATION CONTROLLERS CONSUMER EXAMPLE

```
fromF("kubernetes-replication-controllers://%s?
oauthToken=%s&namespace=default&resourceName=test", host, authToken).process(new
KubernertesProcessor()).to("mock:result");
public class KubernertesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        ReplicationController rc = exchange.getIn().getBody(ReplicationController.class);
        log.info("Got event with configmap name: " + rc.getMetadata().getName() + " and action " +
in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}
```

This consumer returns a list of events on the namespace default for the rc test.

81.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long

Name	Description	Default	Type
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-nodes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-pods.enabled	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
camel.component.kubernetes-pods.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-build-configs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-build-configs.enabled</code>	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
<code>camel.component.openshift-build-configs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-build-configs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-builds.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the <code>openshift-deploymentconfigs</code> component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component. openshift- deploymentconfigs. lazy-start- producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 82. KUBERNETES RESOURCES QUOTA

Since Camel 2.17

Only producer is supported

The Kubernetes Resources Quota component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Resource Quota operations.

82.1. DEPENDENCIES

When using **kubernetes-resources-quota** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

82.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

82.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

82.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

82.3. COMPONENT OPTIONS

The Kubernetes Resources Quota component supports 3 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (producer)	Autowired To use an existing kubernetes client.		KubernetesClient
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

82.4. ENDPOINT OPTIONS

The Kubernetes Resources Quota endpoint is configured using URI syntax:

```
kubernetes-resources-quota:masterUrl
```

with the following path and query parameters:

82.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (producer)	Required Kubernetes Master url.		String

82.4.2. Query Parameters (21 parameters)

Name	Description	Default	Type
apiVersion (producer)	The Kubernetes API Version to use.		String
dnsDomain (producer)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (producer)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (producer)	The namespace.		String
operation (producer)	Producer operation to do on Kubernetes.		String
portName (producer)	The port name, used for ServiceCall EIP.		String
portProtocol (producer)	The port protocol, used for ServiceCall EIP.	tcp	String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String

Name	Description	Default	Type
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

82.5. MESSAGE HEADERS

The Kubernetes Resources Quota component supports 5 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String

Name	Description	Default	Type
CamelKubernetesResourcesQuotaLabels (producer) Constant: KUBERNETES_RESOURCES_QUOTA_LABELS	The resource quota labels.		Map
CamelKubernetesResourcesQuotaName (producer) Constant: KUBERNETES_RESOURCES_QUOTA_NAME	The resource quota name.		String
CamelKubernetesResourceQuotaSpec (producer) Constant: KUBERNETES_RESOURCE_QUOTA_SPEC	The spec for a resource quota.		ResourceQuotaSpec

82.6. SUPPORTED PRODUCER OPERATION

- `listResourcesQuota`
- `listResourcesQuotaByLabels`
- `getResourcesQuota`
- `createResourcesQuota`
- `updateResourceQuota`
- `deleteResourcesQuota`

82.7. KUBERNETES RESOURCE QUOTA PRODUCER EXAMPLES

- `listResourcesQuota`: this operation list the Resource Quotas on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-resources-quota:///?
kubernetesClient=#kubernetesClient&operation=listResourcesQuota").
  to("mock:result");
```

This operation returns a List of Resource Quotas from your cluster.

- `listResourcesQuotaByLabels`: this operation list the Resource Quotas by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_RESOURCES_QUOTA_LABELS,
            labels);
    }
});
toF("kubernetes-resources-quota:///?
kubernetesClient=#kubernetesClient&operation=listResourcesQuotaByLabels").
to("mock:result");
```

This operation returns a List of Resource Quotas from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

82.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String

Name	Description	Default	Type
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-config-maps.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-config-maps.enabled	Whether to enable auto configuration of the kubernetes-config-maps component. This is enabled by default.		Boolean
camel.component.kubernetes-config-maps.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-config-maps.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-custom-resources.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-custom-resources.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-custom-resources.enabled	Whether to enable auto configuration of the kubernetes-custom-resources component. This is enabled by default.		Boolean
camel.component.kubernetes-custom-resources.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-custom-resources.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-deployments.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-hpa.enabled	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
camel.component.kubernetes-hpa.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-job.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-namespaces.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-namespaces.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-namespaces.enabled</code>	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-namespaces.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 83. KUBERNETES SECRETS

Since Camel 2.17

Only producer is supported

The Kubernetes Secrets component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Secrets operations.

83.1. DEPENDENCIES

When using **kubernetes-secrets** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
<groupId>org.apache.camel.springboot</groupId>
<artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

83.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

83.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

83.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

83.3. COMPONENT OPTIONS

The Kubernetes Secrets component supports 3 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (producer)	Autowired To use an existing kubernetes client.		KubernetesClient
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

83.4. ENDPOINT OPTIONS

The Kubernetes Secrets endpoint is configured using URI syntax:

```
kubernetes-secrets:masterUrl
```

with the following path and query parameters:

83.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (producer)	Required Kubernetes Master url.		String

83.4.2. Query Parameters (21 parameters)

Name	Description	Default	Type
apiVersion (producer)	The Kubernetes API Version to use.		String
dnsDomain (producer)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (producer)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (producer)	The namespace.		String
operation (producer)	Producer operation to do on Kubernetes.		String
portName (producer)	The port name, used for ServiceCall EIP.		String
portProtocol (producer)	The port protocol, used for ServiceCall EIP.	tcp	String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String

Name	Description	Default	Type
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

83.5. MESSAGE HEADERS

The Kubernetes Secrets component supports 5 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String

Name	Description	Default	Type
CamelKubernetesSecretsLabels (producer) Constant: KUBERNETES_SECRETS_LABELS	The secret labels.		Map
CamelKubernetesSecretName (producer) Constant: KUBERNETES_SECRET_NAME	The secret name.		String
CamelKubernetesSecret (producer) Constant: KUBERNETES_SECRET	A secret object.		Secret

83.6. SUPPORTED PRODUCER OPERATION

- `listSecrets`
- `listSecretsByLabels`
- `getSecret`
- `createSecret`
- `updateSecret`
- `deleteSecret`

83.7. KUBERNETES SECRETS PRODUCER EXAMPLES

- `listSecrets`: this operation list the secrets on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-secrets:///?kubernetesClient=#kubernetesClient&operation=listSecrets").
  to("mock:result");
```

This operation returns a List of secrets from your cluster.

- `listSecretsByLabels`: this operation list the Secrets by labels on a kubernetes cluster.

```

from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_SECRETS_LABELS,
labels);
    }
});
toF("kubernetes-secrets:///?kubernetesClient=#kubernetesClient&operation=listSecretsByLabels").
to("mock:result");

```

This operation returns a List of Secrets from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

83.8. USING SECRETS PROPERTIES FUNCTION WITH KUBERNETES

The **camel-kubernetes** component include the following secrets related functions:

- **secret** - A function to lookup the string property from Kubernetes Secrets.
- **secret-binary** - A function to lookup the binary property from Kubernetes Secrets.

Camel reads Secrets from the Kubernetes API Server. When RBAC is enabled on the cluster, the ServiceAccount that is used to run the application needs to have the proper permissions for such access.

Before the Kubernetes property placeholder functions can be used they need to be configured with either (or both)

- **path** - A mount path that must be mounted to the running pod, to load the configmaps or secrets from local disk.
- **kubernetes client** - Autowired. An **io.fabric8.kubernetes.client.KubernetesClient** instance to use for connecting to the Kubernetes API server.

Camel first uses the mount paths (if configured) to lookup, and then fallback to use the **KubernetesClient**.

For example, a secret named **mydb** could contain username and passwords to connect to a database such as:

```

myhost = killroy
myport = 5555
myuser = scott
mypass = tiger

```

This is used in Camel as shown in the sample Postgres Sink Kamelet:

```

<camelContext>
  <route>
    <from uri="direct:rome"/>
    <setBody>
      <constant>{ "username":"oscerd", "city":"Rome"}</constant>

```

```

</setBody>
<to uri="kamelet:postgresql-sink?serverName={{secret:mydb/myhost}}
    &serverPort={{secret:mydb/myport}}
    &username={{secret:mydb/myuser}}
    &password={{secret:mydb/mypass}}
    &databaseName=cities
    &query=INSERT INTO accounts (username,city) VALUES (:#username,:#city)"/>
</route>
</camelContext>

```

The **postgres-sink** Kamelet can also be configured in **application.properties** which reduces the configuration in the route above:

```

camel.component.kamelet.postgresql-sink.databaseName={{secret:mydb/myhost}}
camel.component.kamelet.postgresql-sink.serverPort={{secret:mydb/myport}}
camel.component.kamelet.postgresql-sink.username={{secret:mydb/myuser}}
camel.component.kamelet.postgresql-sink.password={{secret:mydb/mypass}}

```

Which reduces the route to:

```

<camelContext>
<route>
<from uri="direct:rome"/>
<setBody>
<constant>{ "username":"oscerd", "city":"Rome"}</constant>
</setBody>
<to uri="kamelet:postgresql-sink?databaseName=cities
    &query=INSERT INTO accounts (username,city) VALUES (:#username,:#city)"/>
</route>
</camelContext>

```

83.9. AUTOMATIC CAMEL CONTEXT RELOADING ON SECRET REFRESH

Being able to reload Camel context on a Secret Refresh is done by specifying the following properties:

```

camel.vault.kubernetes.refreshEnabled=true
camel.vault.kubernetes.secrets=Secret
camel.main.context-reload-enabled = true

```

where **camel.vault.kubernetes.refreshEnabled** enables the automatic context reload and **camel.vault.kubernetes.secrets** is a regex representing or a comma separated lists of the secrets we want to track for updates.

Whenever a secret is listed in the property, it will be updated in the same namespace of the running application, the Camel context will be reloaded, refreshing the secret value.

83.10. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long

Name	Description	Default	Type
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-deployments.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-events.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-builds.enabled	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
camel.component.openshift-builds.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the <code>openshift-deploymentconfigs</code> component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 84. KUBERNETES SERVICE ACCOUNT

Since Camel 2.17

Only producer is supported

The Kubernetes Service Account component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Service Account operations.

84.1. DEPENDENCIES

When using **kubernetes-service-accounts** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
<groupId>org.apache.camel.springboot</groupId>
<artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

84.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

84.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

84.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

84.3. COMPONENT OPTIONS

The Kubernetes Service Account component supports 3 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (producer)	Autowired To use an existing kubernetes client.		KubernetesClient
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

84.4. ENDPOINT OPTIONS

The Kubernetes Service Account endpoint is configured using URI syntax:

```
kubernetes-service-accounts:masterUrl
```

with the following path and query parameters:

84.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (producer)	Required Kubernetes Master url.		String

84.4.2. Query Parameters (21 parameters)

Name	Description	Default	Type
apiVersion (producer)	The Kubernetes API Version to use.		String
dnsDomain (producer)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (producer)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (producer)	The namespace.		String
operation (producer)	Producer operation to do on Kubernetes.		String
portName (producer)	The port name, used for ServiceCall EIP.		String
portProtocol (producer)	The port protocol, used for ServiceCall EIP.	tcp	String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String

Name	Description	Default	Type
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

84.5. MESSAGE HEADERS

The Kubernetes Service Account component supports 5 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String

Name	Description	Default	Type
CamelKubernetesServiceAccountsLabels (producer) Constant: KUBERNETES_SERVICE_ACCOUNTS_LABELS	The service account labels.		Map
CamelKubernetesServiceAccountName (producer) Constant: KUBERNETES_SERVICE_ACCOUNT_NAME	The service account name.		String
CamelKubernetesServiceAccount (producer) Constant: KUBERNETES_SERVICE_ACCOUNT	A service account object.		ServiceAccount

84.6. SUPPORTED PRODUCER OPERATION

- `listServiceAccounts`
- `listServiceAccountsByLabels`
- `getServiceAccount`
- `createServiceAccount`
- `updateServiceAccount`
- `deleteServiceAccount`

84.7. KUBERNETES SERVICEACCOUNTS PRODUCE EXAMPLES

- `listServiceAccounts`: this operation lists the service account on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-service-accounts:///?
kubernetesClient=#kubernetesClient&operation=listServiceAccounts").
  to("mock:result");
```

This operation returns a List of services from your cluster.

- `listServiceAccountsByLabels`: this operation lists the service account by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_SERVICE_ACCOUNTS_LABELS,
            labels);
    }
});
toF("kubernetes-service-accounts:///?
kubernetesClient=#kubernetesClient&operation=listServiceAccountsByLabels").
to("mock:result");
```

This operation returns a List of Services from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

84.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String

Name	Description	Default	Type
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-config-maps.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-config-maps.enabled	Whether to enable auto configuration of the kubernetes-config-maps component. This is enabled by default.		Boolean
camel.component.kubernetes-config-maps.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-config-maps.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-custom-resources.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-hpa.enabled	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
camel.component.kubernetes-hpa.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-job.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-namespaces.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-namespaces.bridg-e-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-namespaces.enabled</code>	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-namespaces.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-service-accounts.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-services.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-services.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-services.enabled	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
camel.component.kubernetes-services.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 85. KUBERNETES SERVICES

Since Camel 2.17

Both producer and consumer are supported

The Kubernetes Services component is one of the [Kubernetes Components](#) which provides a producer to execute Kubernetes Service operations and a consumer to consume events related to Service objects.

85.1. DEPENDENCIES

When using **kubernetes-services** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

85.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

85.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

85.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

85.3. COMPONENT OPTIONS

The Kubernetes Services component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

85.4. ENDPOINT OPTIONS

The Kubernetes Services endpoint is configured using URI syntax:

```
kubernetes-services:masterUrl
```

with the following path and query parameters:

85.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

85.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

85.5. MESSAGE HEADERS

The Kubernetes Services component supports 7 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesServiceLabels (producer) Constant: KUBERNETES_SERVICE_LABELS	The service labels.		Map
CamelKubernetesServiceName (producer) Constant: KUBERNETES_SERVICE_NAME	The service name.		String
CamelKubernetesServiceSpec (producer) Constant: KUBERNETES_SERVICE_SPEC	The spec of a service.		ServiceSpec

Name	Description	Default	Type
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

85.6. SUPPORTED PRODUCER OPERATION

- `listServices`
- `listServicesByLabels`
- `getService`
- `createService`
- `deleteService`

85.7. KUBERNETES SERVICES PRODUCER EXAMPLES

- `listServices`: this operation list the services on a kubernetes cluster.

```
from("direct:list").
  toF("kubernetes-services:///?kubernetesClient=#kubernetesClient&operation=listServices").
  to("mock:result");
```

This operation returns a List of services from your cluster.

- `listServicesByLabels`: this operation list the deployments by labels on a kubernetes cluster.

```
from("direct:listByLabels").process(new Processor() {
  @Override
  public void process(Exchange exchange) throws Exception {
```

```

        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_SERVICE_LABELS,
labels);
    }
});
toF("kubernetes-services:///?
kubernetesClient=#kubernetesClient&operation=listServicesByLabels").
to("mock:result");

```

This operation returns a List of Services from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

85.8. KUBERNETES SERVICES CONSUMER EXAMPLE

```

fromF("kubernetes-services://%s?oauthToken=%s&namespace=default&resourceName=test", host,
authToken).process(new KuberntesProcessor()).to("mock:result");

```

```

public class KuberntesProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        Service sv = exchange.getIn().getBody(Service.class);
        log.info("Got event with configmap name: " + sv.getMetadata().getName() + " and action " +
in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}

```

This consumer returns a list of events on the namespace default for the service test.

85.8.1. Spring Boot Auto-Configuration

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer

Name	Description	Default	Type
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-config-maps.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-config-maps.enabled	Whether to enable auto configuration of the kubernetes-config-maps component. This is enabled by default.		Boolean
camel.component.kubernetes-config-maps.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-config-maps.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-custom-resources.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-custom-resources.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-custom-resources.enabled	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
camel.component.kubernetes-custom-resources.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-custom-resources.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-deployments.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-events.enabled</code>	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-events.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-events.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-hpa.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-hpa.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-job.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-namespaces.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-namespaces.bridg-e-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-namespaces.enabled</code>	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-namespaces.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-nodes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-persistent-volumes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>Boolean</code>
<code>camel.component.kubernetes-pods.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	<code>true</code>	<code>Boolean</code>
<code>camel.component.kubernetes-pods.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	<code>false</code>	<code>Boolean</code>
<code>camel.component.kubernetes-pods.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-pods</code> component. This is enabled by default.		<code>Boolean</code>
<code>camel.component.kubernetes-pods.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-builds.enabled	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
camel.component.openshift-builds.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the <code>openshift-deploymentconfigs</code> component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 86. OPENSIFT BUILDS

Since Camel 2.17

Only producer is supported

The Openshift Builds component is one of the [Kubernetes Components](#) which provides a producer to execute Openshift builds operations.

86.1. DEPENDENCIES

When using **openshift-builds** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
<groupId>org.apache.camel.springboot</groupId>
<artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

86.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

86.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

86.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

86.3. COMPONENT OPTIONS

The Openshift Builds component supports 3 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (producer)	Autowired To use an existing kubernetes client.		KubernetesClient
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

86.4. ENDPOINT OPTIONS

The Openshift Builds endpoint is configured using URI syntax:

```
openshift-builds:masterUrl
```

with the following path and query parameters:

86.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (producer)	Required Kubernetes Master url.		String

86.4.2. Query Parameters (21 parameters)

Name	Description	Default	Type
apiVersion (producer)	The Kubernetes API Version to use.		String
dnsDomain (producer)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (producer)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (producer)	The namespace.		String
operation (producer)	Producer operation to do on Kubernetes.		String
portName (producer)	The port name, used for ServiceCall EIP.		String
portProtocol (producer)	The port protocol, used for ServiceCall EIP.	tcp	String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String

Name	Description	Default	Type
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

86.5. MESSAGE HEADERS

The Openshift Builds component supports 4 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String

Name	Description	Default	Type
CamelKubernetesBuildsLabels (producer) Constant: KUBERNETES_BUILDS_LABELS	The Openshift build labels.		Map
CamelKubernetesBuildName (producer) Constant: KUBERNETES_BUILD_NAME	The Openshift build name.		String

86.6. SUPPORTED PRODUCER OPERATION

- listBuilds
- listBuildsByLabels
- getBuild

86.7. OPENSIFT BUILDS PRODUCER EXAMPLES

- listBuilds: this operation list the Builds on an Openshift cluster.

```
from("direct:list").
  toF("openshift-builds:///?kubernetesClient=#kubernetesClient&operation=listBuilds").
  to("mock:result");
```

This operation returns a List of Builds from your Openshift cluster.

- listBuildsByLabels: this operation list the builds by labels on an Openshift cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");
        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_BUILDS_LABELS,
labels);
    }
});
toF("openshift-builds:///?kubernetesClient=#kubernetesClient&operation=listBuildsByLabels").
to("mock:result");
```

This operation returns a List of Builds from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

86.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer

Name	Description	Default	Type
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-custom-resources.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-deployments.auto-wired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-deployments.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-deployments.enabled</code>	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-deployments.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-hpa.enabled	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
camel.component.kubernetes-hpa.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridg-e-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-nodes.enabled	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
camel.component.kubernetes-nodes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-pods.enabled	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
camel.component.kubernetes-pods.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-pods.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-replication-controllers.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-replication-controllers.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-replication-controllers.enabled	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-secrets.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-secrets.enabled</code>	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-secrets.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-secrets.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-service-accounts.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-services.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-builds.enabled	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
camel.component.openshift-builds.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component. openshift- deploymentconfi gs.lazy-start- producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 87. OPENSIFT BUILD CONFIG

Since Camel 2.17

Only producer is supported

The OpenShift Build Config component is one of the [Kubernetes Components](#) which provides a producer to execute Openshift Build Configs operations.

87.1. DEPENDENCIES

When using **openshift-build-configs** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
<groupId>org.apache.camel.springboot</groupId>
<artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

87.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

87.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

87.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

87.3. COMPONENT OPTIONS

The Openshift Build Config component supports 3 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (producer)	Autowired To use an existing kubernetes client.		KubernetesClient
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

87.4. ENDPOINT OPTIONS

The Openshift Build Config endpoint is configured using URI syntax:

```
openshift-build-configs:masterUrl
```

with the following path and query parameters:

87.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (producer)	Required Kubernetes Master url.		String

87.4.2. Query Parameters (21 parameters)

Name	Description	Default	Type
apiVersion (producer)	The Kubernetes API Version to use.		String
dnsDomain (producer)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (producer)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (producer)	The namespace.		String
operation (producer)	Producer operation to do on Kubernetes.		String
portName (producer)	The port name, used for ServiceCall EIP.		String
portProtocol (producer)	The port protocol, used for ServiceCall EIP.	tcp	String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String

Name	Description	Default	Type
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

87.5. MESSAGE HEADERS

The Openshift Build Config component supports 4 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String

Name	Description	Default	Type
CamelKubernetesBuildConfigsLabels (producer) Constant: KUBERNETES_BUILD_CONFIGS_LABELS	The Openshift Config Build labels.		Map
CamelKubernetesBuildConfigName (producer) Constant: KUBERNETES_BUILD_CONFIG_NAME	The Openshift Config Build name.		String

87.6. SUPPORTED PRODUCER OPERATION

- `listBuildConfigs`
- `listBuildConfigsByLabels`
- `getBuildConfig`

87.7. OPENSIFT BUILD CONFIGS PRODUCER EXAMPLES

- `listBuilds`: this operation list the Build Configs on an Openshift cluster.

```
from("direct:list").
  toF("openshift-build-configs:///?kubernetesClient=#kubernetesClient&operation=listBuildConfigs").
  to("mock:result");
```

This operation returns a List of Builds from your Openshift cluster.

- `listBuildsByLabels`: this operation list the build configs by labels on an Openshift cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_BUILD_CONFIGS_LABELS,
        labels);
    }
});
```

```

    toF("openshift-build-configs:///?
    kubernetesClient=#kubernetesClient&operation=listBuildConfigsByLabels").
    to("mock:result");

```

This operation returns a List of Build configs from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

87.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String

Name	Description	Default	Type
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-custom-resources.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-custom-resources.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-custom-resources</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-custom-resources.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-custom-resources.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-deployments.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-events.enabled	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
camel.component.kubernetes-events.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-events.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-hpa.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-hpa.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-hpa.enabled	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
camel.component.kubernetes-hpa.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-hpa.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-job.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-job.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-job.enabled	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
camel.component.kubernetes-job.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-namespaces.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-nodes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-nodes.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-nodes.enabled</code>	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-nodes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-persistent-volumes.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-persistent-volumes.enabled	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-pods.enabled	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
camel.component.kubernetes-pods.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-pods.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-replication-controllers.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-replication-controllers.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-resources-quota.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.enabled</code>	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-service-accounts.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.openshift-builds.enabled	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
camel.component.openshift-builds.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.openshift-builds.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-deploymentconfigs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-deploymentconfigs.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.openshift-deploymentconfigs.enabled	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		<code>KubernetesClient</code>
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>Boolean</code>

CHAPTER 88. OPENSIFT DEPLOYMENT CONFIGS

Since Camel 3.18

Both producer and consumer are supported

The Openshift Deployment Configs component is one of the [Kubernetes Components](#) which provides a producer to execute Openshift Deployment Configs operations and a consumer to consume events related to Deployment Configs objects.

88.1. DEPENDENCIES

When using **openshift-deploymentconfigs** with Red Hat build of Apache Camel for Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kubernetes-starter</artifactId>
</dependency>
```

88.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

88.2.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

88.2.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

88.3. COMPONENT OPTIONS

The Openshift Deployment Configs component supports 4 options, which are listed below.

Name	Description	Default	Type
kubernetesClient (common)	Autowired To use an existing kubernetes client.		KubernetesClient
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

88.4. ENDPOINT OPTIONS

The Openshift Deployment Configs endpoint is configured using URI syntax:

```
openshift-deploymentconfigs:masterUrl
```

with the following path and query parameters:

88.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
masterUrl (common)	Required Kubernetes Master url.		String

88.4.2. Query Parameters (33 parameters)

Name	Description	Default	Type
apiVersion (common)	The Kubernetes API Version to use.		String
dnsDomain (common)	The dns domain, used for ServiceCall EIP.		String
kubernetesClient (common)	Default KubernetesClient to use if provided.		KubernetesClient
namespace (common)	The namespace.		String
portName (common)	The port name, used for ServiceCall EIP.		String
portProtocol (common)	The port protocol, used for ServiceCall EIP.	tcp	String
crdGroup (consumer)	The Consumer CRD Resource Group we would like to watch.		String
crdName (consumer)	The Consumer CRD Resource name we would like to watch.		String
crdPlural (consumer)	The Consumer CRD Resource Plural we would like to watch.		String
crdScope (consumer)	The Consumer CRD Resource Scope we would like to watch.		String
crdVersion (consumer)	The Consumer CRD Resource Version we would like to watch.		String
labelKey (consumer)	The Consumer Label key when watching at some resources.		String

Name	Description	Default	Type
labelValue (consumer)	The Consumer Label value when watching at some resources.		String
poolSize (consumer)	The Consumer pool size.	1	int
resourceName (consumer)	The Consumer Resource Name we would like to watch.		String
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code>		<code>ExchangePattern</code>
operation (producer)	Producer operation to do on Kubernetes.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
connectionTimeout (advanced)	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
caCertData (security)	The CA Cert Data.		String
caCertFile (security)	The CA Cert File.		String
clientCertData (security)	The Client Cert Data.		String
clientCertFile (security)	The Client Cert File.		String
clientKeyAlgo (security)	The Key Algorithm used by the client.		String
clientKeyData (security)	The Client Key data.		String
clientKeyFile (security)	The Client Key file.		String
clientKeyPassphrase (security)	The Client Key Passphrase.		String
oauthToken (security)	The Auth Token.		String
password (security)	Password to connect to Kubernetes.		String
trustCerts (security)	Define if the certs we used are trusted anyway or not.		Boolean
username (security)	Username to connect to Kubernetes.		String

88.5. MESSAGE HEADERS

The Openshift Deployment Configs component supports 8 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKubernetesOperation (producer) Constant: KUBERNETES_OPERATION	The Producer operation.		String
CamelKubernetesNamespaceName (producer) Constant: KUBERNETES_NAMESPACE_NAME	The namespace name.		String
CamelKubernetesDeploymentsLabels (producer) Constant: KUBERNETES_DEPLOYMENTS_LABELS	The deployment labels.		Map
CamelKubernetesDeploymentName (producer) Constant: KUBERNETES_DEPLOYMENT_NAME	The deployment name.		String
CamelKubernetesDeploymentReplicas (producer) Constant: KUBERNETES_DEPLOYMENT_REPLICAS	The desired instance count.		Integer
CamelKubernetesDeploymentConfigSpec (producer) Constant: KUBERNETES_DEPLOYMENT_CONFIG_SPEC	The spec for a deployment config.		DeploymentConfigSpec

Name	Description	Default	Type
CamelKubernetesEventAction (consumer) Constant: KUBERNETES_EVENT_ACTION	Action watched by the consumer. Enum values: • ADDED • MODIFIED • DELETED • ERROR • BOOKMARK		Action
CamelKubernetesEventTimestamp (consumer) Constant: KUBERNETES_EVENT_TIMESTAMP	Timestamp of the action watched by the consumer.		long

88.6. SUPPORTED PRODUCER OPERATION

- listDeploymentConfigs
- listDeploymentsConfigsByLabels
- getDeploymentConfig
- createDeploymentConfig
- updateDeploymentConfig
- deleteDeploymentConfig
- scaleDeploymentConfig

88.7. OPENSIFT DEPLOYMENT CONFIGS PRODUCER EXAMPLES

- listDeploymentConfigs: this operation list the deployments on a Openshift cluster.

```
from("direct:list").
  toF("openshift-deploymentconfigs:///?
kubernetesClient=#kubernetesClient&operation=listDeploymentConfigs").
  to("mock:result");
```

This operation returns a List of Deployment Configs from your cluster.

- `listDeploymentConfigsByLabels`: this operation list the deployment configs by labels on a Openshift cluster.

```
from("direct:listByLabels").process(new Processor() {
    @Override
    public void process(Exchange exchange) throws Exception {
        Map<String, String> labels = new HashMap<>();
        labels.put("key1", "value1");
        labels.put("key2", "value2");

        exchange.getIn().setHeader(KubernetesConstants.KUBERNETES_DEPLOYMENTS_LABELS,
            labels);
    }
});
toF("openshift-deploymentconfigs:///?
kubernetesClient=#kubernetesClient&operation=listDeploymentConfigsByLabels").
to("mock:result");
```

This operation returns a List of Deployment Configs from your cluster, using a label selector (with key1 and key2, with value value1 and value2).

88.8. OPENSIFT DEPLOYMENT CONFIGS CONSUMER EXAMPLE

```
fromF("openshift-deploymentconfigs://%s?
oauthToken=%s&namespace=default&resourceName=test", host, authToken).process(new
OpenshiftProcessor()).to("mock:result");
public class OpenshiftProcessor implements Processor {
    @Override
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        DeploymentConfig dp = exchange.getIn().getBody(DeploymentConfig.class);
        log.info("Got event with configmap name: " + dp.getMetadata().getName() + " and action " +
            in.getHeader(KubernetesConstants.KUBERNETES_EVENT_ACTION));
    }
}
```

This consumer returns a list of events on the namespace default for the deployment config test.

88.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 102 options, which are listed below.

Name	Description	Default	Type
<code>camel.cluster.kubernetes.attributes</code>	Custom service attributes.		Map
<code>camel.cluster.kubernetes.cluster-labels</code>	Set the labels used to identify the pods composing the cluster.		Map

Name	Description	Default	Type
<code>camel.cluster.kubernetes.config-map-name</code>	Set the name of the ConfigMap used to do optimistic locking (defaults to 'leaders').		String
<code>camel.cluster.kubernetes.connection-timeout-millis</code>	Connection timeout in milliseconds to use when making requests to the Kubernetes API server.		Integer
<code>camel.cluster.kubernetes.enabled</code>	Sets if the Kubernetes cluster service should be enabled or not, default is false.	false	Boolean
<code>camel.cluster.kubernetes.id</code>	Cluster Service ID.		String
<code>camel.cluster.kubernetes.jitter-factor</code>	A jitter factor to apply in order to prevent all pods to call Kubernetes APIs in the same instant.		Double
<code>camel.cluster.kubernetes.kubernetes-namespace</code>	Set the name of the Kubernetes namespace containing the pods and the configmap (autodetected by default).		String
<code>camel.cluster.kubernetes.lease-duration-millis</code>	The default duration of the lease for the current leader.		Long
<code>camel.cluster.kubernetes.master-url</code>	Set the URL of the Kubernetes master (read from Kubernetes client properties by default).		String
<code>camel.cluster.kubernetes.order</code>	Service lookup order/priority.		Integer
<code>camel.cluster.kubernetes.pod-name</code>	Set the name of the current pod (autodetected from container host name by default).		String
<code>camel.cluster.kubernetes.renew-deadline-millis</code>	The deadline after which the leader must stop its services because it may have lost the leadership.		Long
<code>camel.cluster.kubernetes.retry-period-millis</code>	The time between two subsequent attempts to check and acquire the leadership. It is randomized using the jitter factor.		Long

Name	Description	Default	Type
<code>camel.component.kubernetes-config-maps.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-config-maps.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-config-maps.enabled</code>	Whether to enable auto configuration of the <code>kubernetes-config-maps</code> component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-config-maps.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-config-maps.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-custom-resources.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-custom-resources.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-custom-resources.enabled	Whether to enable auto configuration of the kubernetes-custom-resources component. This is enabled by default.		Boolean
camel.component.kubernetes-custom-resources.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-custom-resources.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-deployments.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.kubernetes-deployments.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-deployments.enabled	Whether to enable auto configuration of the kubernetes-deployments component. This is enabled by default.		Boolean
camel.component.kubernetes-deployments.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-deployments.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-events.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-events.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-events.enabled</code>	Whether to enable auto configuration of the kubernetes-events component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-events.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-events.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-hpa.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-hpa.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-hpa.enabled</code>	Whether to enable auto configuration of the kubernetes-hpa component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-hpa.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-hpa.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-job.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-job.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-job.enabled</code>	Whether to enable auto configuration of the kubernetes-job component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-job.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-job.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-namespaces.auto-wired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-namespaces.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-namespaces.enabled	Whether to enable auto configuration of the kubernetes-namespaces component. This is enabled by default.		Boolean
camel.component.kubernetes-namespaces.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-namespaces.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-nodes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-nodes.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-nodes.enabled</code>	Whether to enable auto configuration of the kubernetes-nodes component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-nodes.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-nodes.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.kubernetes-persistent-volumes-claims.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes-claims component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-persistent-volumes-claims.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-persistent-volumes-claims.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-persistent-volumes.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-persistent-volumes.enabled</code>	Whether to enable auto configuration of the kubernetes-persistent-volumes component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.kubernetes-persistent-volumes.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-persistent-volumes.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-pods.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-pods.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.kubernetes-pods.enabled	Whether to enable auto configuration of the kubernetes-pods component. This is enabled by default.		Boolean
camel.component.kubernetes-pods.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-pods.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-replication-controllers.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-replication-controllers.enabled</code>	Whether to enable auto configuration of the kubernetes-replication-controllers component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-replication-controllers.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-replication-controllers.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-resources-quota.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-resources-quota.enabled</code>	Whether to enable auto configuration of the kubernetes-resources-quota component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-resources-quota.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.kubernetes-resources-quota.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.kubernetes-secrets.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-secrets.enabled	Whether to enable auto configuration of the kubernetes-secrets component. This is enabled by default.		Boolean
camel.component.kubernetes-secrets.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
camel.component.kubernetes-secrets.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.kubernetes-service-accounts.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kubernetes-service-accounts.enabled	Whether to enable auto configuration of the kubernetes-service-accounts component. This is enabled by default.		Boolean
camel.component.kubernetes-service-accounts.kubernetes-client	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.kubernetes-service-accounts.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.kubernetes-services.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.kubernetes-services.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.kubernetes-services.enabled</code>	Whether to enable auto configuration of the kubernetes-services component. This is enabled by default.		Boolean
<code>camel.component.kubernetes-services.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
camel.component.kubernetes-services.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-build-configs.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.openshift-build-configs.enabled	Whether to enable auto configuration of the openshift-build-configs component. This is enabled by default.		Boolean
camel.component.openshift-build-configs.kubernetes-client	To use an existing kubernetes client. The option is a io.fabric8.kubernetes.client.KubernetesClient type.		KubernetesClient
camel.component.openshift-build-configs.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.openshift-builds.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
<code>camel.component.openshift-builds.enabled</code>	Whether to enable auto configuration of the openshift-builds component. This is enabled by default.		Boolean
<code>camel.component.openshift-builds.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient
<code>camel.component.openshift-builds.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.openshift-deploymentconfigs.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.openshift-deploymentconfigs.enabled</code>	Whether to enable auto configuration of the openshift-deploymentconfigs component. This is enabled by default.		Boolean
<code>camel.component.openshift-deploymentconfigs.kubernetes-client</code>	To use an existing kubernetes client. The option is a <code>io.fabric8.kubernetes.client.KubernetesClient</code> type.		KubernetesClient

Name	Description	Default	Type
<code>camel.component.openshift-deploymentconfigs.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 89. KUDU

Since Camel 3.0

Only producer is supported

The Kudu component supports storing and retrieving data from/to [Apache Kudu](#), a free and open source column-oriented data store of the Apache Hadoop ecosystem.

89.1. DEPENDENCIES

When using **kudu** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-kudu-starter</artifactId>
</dependency>
```

89.2. PREREQUISITES

You must have a valid Kudu instance running. More information are available at [Apache Kudu](#).

89.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

89.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

89.3.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

89.4. COMPONENT OPTIONS

The Kudu component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). This allows CamelContext and routes to start in situations where a producer failing would cause the route to fail. With lazy startup, the startup failures can be handled during routing messages via Camel's routing error handlers.  NOTE When the first message is processed, it may take some time to create and start the producer which will increase the total processing time.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

89.5. ENDPOINT OPTIONS

The Kudu endpoint is configured using URI syntax:

```
kudu:host:port/tableName
```

with the following path and query parameters:

89.5.1. Path Parameters (3 parameters)

Name	Description	Default	Type
host (common)	Host of the server to connect to.		String
port (common)	Port of the server to connect to.		String
tableName (common)	Table to connect to.		String

89.5.2. Query Parameters (2 parameters)

Name	Description	Default	Type
operation (producer)	Operation to perform. Enum values: ● INSERT ● DELETE ● UPDATE ● UPSERT ● CREATE_TABLE ● SCAN		KuduOperations
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

89.6. MESSAGE HEADERS

The Kudu component supports 5 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelKuduSchema (producer) Constant: CAMEL_KUDU_SCHEMA	The schema.		Schema
CamelKuduTableOptions (producer) Constant: CAMEL_KUDU_TABLE_OPTIONS	The create table options.		CreateTableOptions
CamelKuduScanColumnNames (producer) Constant: CAMEL_KUDU_SCAN_COLUMN_NAMES	The projected column names for scan operation.		List
CamelKuduScanPredicate (producer) Constant: CAMEL_KUDU_SCAN_PREDICATE	The predicate for scan operation.		KuduPredicate
CamelKuduScanLimit (producer) Constant: CAMEL_KUDU_SCAN_LIMIT	The limit on the number of rows for scan operation.		long

89.7. INPUT BODY FORMATS

89.7.1. Insert, delete, update, and upsert

The input body format has to be a `java.util.Map<String, Object>`. This map will represent a row of the table whose elements are columns, where the key is the column name and the value is the value of the column.

89.8. OUTPUT BODY FORMATS

89.8.1. Scan

The output body format will be a `java.util.List<java.util.Map<String, Object>>`. Each element of the list will be a different row of the table. Each row is a `Map<String, Object>` whose elements will be each pair of column name and column value for that row.

89.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.component.kudu.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.kudu.enabled	Whether to enable auto configuration of the kudu component. This is enabled by default.		Boolean
camel.component.kudu.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 90. LANGUAGE

Only producer is supported

The Language component allows you to send Exchange to an endpoint which executes a script by any of the supported Languages in Camel. By having a component to execute language scripts, it allows more dynamic routing capabilities. For example by using the Routing Slip or [Dynamic Router](#) EIPs you can send messages to language endpoints where the script is dynamic defined as well.

This component is provided out of the box in camel-core and hence no additional JARs is needed. You only have to include additional Camel components if the language of choice mandates it, such as using [Groovy](#) or JavaScript languages.

90.1. DEPENDENCIES

When using **language** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-language-starter</artifactId>
</dependency>
```

90.2. URI FORMAT

```
language://languageName[:script][?options]
```

You can refer to an external resource for the script using same notation as supported by the other [Languages](#) in Camel.

```
language://languageName:resource:scheme:location][?options]
```

90.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

90.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).

- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

90.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

90.4. COMPONENT OPTIONS

The Language component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

90.5. ENDPOINT OPTIONS

The Language endpoint is configured using URI syntax:

```
language:languageName:resourceUri
```

with the following path and query parameters:

90.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
languageName (producer)	Required Sets the name of the language to use. Enum values: • bean • constant • exchangeProperty • file • groovy • header • javascript • jsonpath • mvel • ognl • ref • simple • spel • sql • terser • tokenize • xpath • xquery • xtokenize		String
resourceUri (producer)	Path to the resource, or a reference to lookup a bean in the Registry to use as the resource.		String

90.5.2. Query Parameters (7 parameters)

Name	Description	Default	Type
allowContextMapAll (producer)	Sets whether the context map should allow access to all details. By default only the message body and headers can be accessed. This option can be enabled for full access to the current Exchange and CamelContext. Doing so impose a potential security risk as this opens access to the full power of CamelContext API.	false	boolean
binary (producer)	Whether the script is binary content or text content. By default the script is read as text content (eg java.lang.String).	false	boolean
cacheScript (producer)	Whether to cache the compiled script and reuse. Notice reusing the script can cause side effects from processing one Camel org.apache.camel.Exchange to the next org.apache.camel.Exchange.	false	boolean
contentCache (producer)	Sets whether to use resource content cache or not.	true	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
script (producer)	Sets the script to execute.		String
transform (producer)	Whether or not the result of the script should be used as message body. This options is default true.	true	boolean

90.6. MESSAGE HEADERS

The following message headers can be used to affect the behavior of the component

Header	Description
CamelLanguageScript	The script to execute provided in the header. Takes precedence over script configured on the endpoint.

90.7. EXAMPLES

For example you can use the [Simple](#) language to Message Translator a message.

You can also provide the script as a header as shown below. Here we use XPath language to extract the text from the <foo> tag.

```
Object out = producer.requestBodyAndHeader("language:xpath", "<foo>Hello World</foo>",
Exchange.LANGUAGE_SCRIPT, "/foo/text()");
assertEquals("Hello World", out);
```

90.8. LOADING SCRIPTS FROM RESOURCES

You can specify a resource uri for a script to load in either the endpoint uri, or in the **Exchange.LANGUAGE_SCRIPT** header. The uri must start with one of the following schemes: file:, classpath:, or http:

By default the script is loaded once and cached. However you can disable the **contentCache** option and have the script loaded on each evaluation. For example if the file myscript.txt is changed on disk, then the updated script is used:

You can refer to the resource similar to the other [Languages](#) in Camel by prefixing with "resource:" as shown below.

90.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.component.language.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.language.enabled	Whether to enable auto configuration of the language component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component. .language.lazy- start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 91. LDAP

Since Camel 1.5

Only producer is supported

The LDAP component allows you to perform searches in LDAP servers using filters as the message payload. This component uses standard JNDI (**javax.naming** package) to access the server.

91.1. DEPENDENCIES

When using **ldap** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-ldap-starter</artifactId>
</dependency>
```

91.2. URI FORMAT

```
ldap:ldapServerBean[?options]
```

The **ldapServerBean** in the URI refers to a [DirContext](#) bean in the registry. The LDAP component only supports producer endpoints, which means that an **ldap** URI cannot appear in the **from** at the start of a route.

91.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

91.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

91.3.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

91.4. COMPONENT OPTIONS

The LDAP component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

91.5. ENDPOINT OPTIONS

The LDAP endpoint is configured using URI syntax:

```
ldap:dirContextName
```

with the following path and query parameters:

91.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
dirContextName (producer)	Required Name of either a <code>javax.naming.directory.DirContext</code> , or <code>java.util.Hashtable</code> , or <code>Map</code> bean to lookup in the registry. If the bean is either a <code>Hashtable</code> or <code>Map</code> then a new <code>javax.naming.directory.DirContext</code> instance is created for each use. If the bean is a <code>javax.naming.directory.DirContext</code> then the bean is used as given. The latter may not be possible in all situations where the <code>javax.naming.directory.DirContext</code> must not be shared, and in those situations it can be better to use <code>java.util.Hashtable</code> or <code>Map</code> instead.		String

91.5.2. Query Parameters (5 parameters)

Name	Description	Default	Type
base (producer)	The base DN for searches.	<code>ou=system</code>	String
pageSize (producer)	When specified the ldap module uses paging to retrieve all results (most LDAP Servers throw an exception when trying to retrieve more than 1000 entries in one query). To be able to use this a <code>LdapContext</code> (subclass of <code>DirContext</code>) has to be passed in as <code>LdapServerBean</code> (otherwise an exception is thrown).		Integer
returnedAttributes (producer)	Comma-separated list of attributes that should be set in each entry of the result.		String
scope (producer)	Specifies how deeply to search the tree of entries, starting at the base DN. Enum values: • <code>object</code> • <code>onelevel</code> • <code>subtree</code>	<code>subtree</code>	String

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

91.6. RESULT

The result is returned to Out body as a **List<javax.naming.directory.SearchResult>** object.

91.7. DIRCONTEXT

The URI, **ldap:ldapserver**, references a Spring bean with the ID, **ldapserver**. The **ldapserver** bean may be defined as follows:

```
<bean id="ldapserver" class="javax.naming.directory.InitialDirContext" scope="prototype">
  <constructor-arg>
    <props>
      <prop key="java.naming.factory.initial">com.sun.jndi.ldap.LdapCtxFactory</prop>
      <prop key="java.naming.provider.url">ldap://localhost:10389</prop>
      <prop key="java.naming.security.authentication">none</prop>
    </props>
  </constructor-arg>
</bean>
```

The preceding example declares a regular Sun based LDAP **DirContext** that connects anonymously to a locally hosted LDAP server.



NOTE

DirContext objects are **not** required to support concurrency by contract. It is therefore important that the directory context is declared with the setting, **scope="prototype"**, in the **bean** definition or that the context supports concurrency. In the Spring framework, **prototype** scoped objects are instantiated each time they are looked up.

91.8. SECURITY CONCERNS RELATED TO LDAP INJECTION



NOTE

The camel-ldap component uses the message body as filter the search results. Therefore, the message body should be protected from LDAP injection. To assist with this, you can use **org.apache.camel.component.ldap.LdapHelper** utility class that has method(s) to escape string values to be LDAP injection safe.

See [LDAP Injection](#) for more information.

91.9. SAMPLES

Following on from the Spring configuration above, the code sample below sends an LDAP request to filter search a group for a member. The Common Name is then extracted from the response.

```
ProducerTemplate template = exchange.getContext().createProducerTemplate();

Collection<SearchResult> results = template.requestBody(
    "ldap://ldapserver?base=ou=mygroup,ou=groups,ou=system",
    "(member=uid=huntc,ou=users,ou=system)", Collection.class);

if (results.size() > 0) {
    // Extract what we need from the device's profile

    Iterator resultIter = results.iterator();
    SearchResult searchResult = (SearchResult) resultIter.next();
    Attributes attributes = searchResult.getAttributes();
    Attribute deviceCNAttr = attributes.get("cn");
    String deviceCN = (String) deviceCNAttr.get();
    // ...
}
```

If no specific filter is required - for example, you just need to look up a single entry - specify a wildcard filter expression. For example, if the LDAP entry has a Common Name, use a filter expression like:

```
(cn=*)
```

91.9.1. Binding using credentials

A Camel end user donated this sample code he used to bind to the ldap server using credentials.

```
Properties props = new Properties();
props.setProperty(Context.INITIAL_CONTEXT_FACTORY, "com.sun.jndi.ldap.LdapCtxFactory");
props.setProperty(Context.PROVIDER_URL, "ldap://localhost:389");
props.setProperty(Context.URL_PKG_PREFIXES, "com.sun.jndi.url");
props.setProperty(Context.REFERRAL, "ignore");
props.setProperty(Context.SECURITY_AUTHENTICATION, "simple");
props.setProperty(Context.SECURITY_PRINCIPAL, "cn=Manager");
props.setProperty(Context.SECURITY_CREDENTIALS, "secret");

DefaultRegistry reg = new DefaultRegistry();
reg.bind("myldap", new InitialLdapContext(props, null));

CamelContext context = new DefaultCamelContext(reg);
context.addRoutes(
```

```

new RouteBuilder() {
    @Override
    public void configure() throws Exception {
        from("direct:start").to("ldap:myldap?base=ou=test");
    }
};
context.start();

ProducerTemplate template = context.createProducerTemplate();

Endpoint endpoint = context.getEndpoint("direct:start");
Exchange exchange = endpoint.createExchange();
exchange.getIn().setBody("uid=test");
Exchange out = template.send(endpoint, exchange);

Collection<SearchResult> data = out.getMessage().getBody(Collection.class);
assert data != null;
assert !data.isEmpty();

System.out.println(out.getMessage().getBody());

context.stop();

```

91.10. CONFIGURING SSL

All that is required is to create a custom socket factory and reference it in the InitialDirContext bean - see below sample.

SSL Configuration

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
    http://camel.apache.org/schema/spring http://camel.apache.org/schema/spring/camel-spring.xsd">

    <sslContextParameters xmlns="http://camel.apache.org/schema/spring"
id="sslContextParameters" >
        <keyManagers keyPassword="{{keystore.pwd}}">
            <keyStore resource="{{keystore.url}}" password="{{keystore.pwd}}"/>
        </keyManagers>
    </sslContextParameters>

    <bean id="customSocketFactory" class="com.example.ldap.CustomSocketFactory">
        <constructor-arg index="0" ref="sslContextParameters"/>
    </bean>

    <bean id="ldapservlet" class="javax.naming.directory.InitialDirContext" scope="prototype">
        <constructor-arg>
            <props>

```

```

        <prop key="java.naming.factory.initial">com.sun.jndi.Ldap.LdapCtxFactory</prop>
        <prop key="java.naming.provider.url">ldaps://127.0.0.1:10636</prop>
        <prop key="java.naming.security.protocol">ssl</prop>
        <prop key="java.naming.security.authentication">none</prop>
    </prop>
    key="java.naming.Ldap.factory.socket">com.example.Ldap.CustomSocketFactory</prop>
</props>
</constructor-arg>
</bean>
</beans>

```

Custom Socket Factory

```

package com.example.Ldap;

import java.io.IOException;
import java.net.InetAddress;
import java.net.Socket;
import java.security.KeyStore;

import javax.net.SocketFactory;
import javax.net.ssl.SSLContext;
import javax.net.ssl.SSLSocketFactory;
import javax.net.ssl.TrustManagerFactory;

import org.apache.camel.support.jsse.SSLContextParameters;

/**
 * The CustomSocketFactory. Loads the KeyStore and creates an instance of SSLSocketFactory
 */
public class CustomSocketFactory extends SSLSocketFactory {

    private static SSLSocketFactory socketFactory;

    /**
     * Called by the getDefault() method.
     */
    public CustomSocketFactory() {
    }

    /**
     * Called by Spring Boot DI to initialize an instance of SocketFactory
     */
    public CustomSocketFactory(SSLContextParameters sslContextParameters) {
        try {
            KeyStore keyStore =
sslContextParameters.getKeyManagers().getKeyStore().createKeyStore();
            TrustManagerFactory tmf = TrustManagerFactory.getInstance("SunX509");
            tmf.init(keyStore);
            SSLContext ctx = SSLContext.getInstance("TLS");
            ctx.init(null, tmf.getTrustManagers(), null);
            socketFactory = ctx.getSocketFactory();
        } catch (Exception ex) {
            ex.printStackTrace(System.err);
        }
    }
}

```

```

/**
 * Getter for the SocketFactory
 */
public static SocketFactory getDefault() {
    return new CustomSocketFactory();
}

@Override
public String[] getDefaultCipherSuites() {
    return socketFactory.getDefaultCipherSuites();
}

@Override
public String[] getSupportedCipherSuites() {
    return socketFactory.getSupportedCipherSuites();
}

@Override
public Socket createSocket(Socket socket, String string, int i, boolean bln) throws IOException {
    return socketFactory.createSocket(socket, string, i, bln);
}

@Override
public Socket createSocket(String string, int i) throws IOException {
    return socketFactory.createSocket(string, i);
}

@Override
public Socket createSocket(String string, int i, InetAddress ia, int i1) throws IOException {
    return socketFactory.createSocket(string, i, ia, i1);
}

@Override
public Socket createSocket(InetAddress ia, int i) throws IOException {
    return socketFactory.createSocket(ia, i);
}

@Override
public Socket createSocket(InetAddress ia, int i, InetAddress ia1, int i1) throws IOException {
    return socketFactory.createSocket(ia, i, ia1, i1);
}
}

```

91.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
		t	

Name	Description	Default	Type
camel.component. .ldap.autowired- enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component. .ldap.enabled	Whether to enable auto configuration of the ldap component. This is enabled by default.		Boolean
camel.component. .ldap.lazy-start- producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 92. LOG

Only producer is supported

The Log component logs message exchanges to the underlying logging mechanism.

Camel uses [SLF4J](#) which allows you to configure logging via, among others:

- Log4j
- Logback
- Java Util Logging

92.1. DEPENDENCIES

When using **log** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-log-starter</artifactId>
</dependency>
```

92.2. URI FORMAT

```
log:loggingCategory[?options]
```

Where **loggingCategory** is the name of the logging category to use. You can append query options to the URI in the following format,

?option=value&option=value&...



NOTE

Using Logger instance from the Registry

If there's single instance of **org.slf4j.Logger** found in the Registry, the **loggingCategory** is no longer used to create logger instance. The registered instance is used instead. Also it is possible to reference particular **Logger** instance using **?logger=#myLogger** URI parameter. Eventually, if there's no registered and URI **logger** parameter, the logger instance is created using **loggingCategory**.

For example, a log endpoint typically specifies the logging level using the **level** option, as follows:

```
log:org.apache.camel.example?level=DEBUG
```

The default logger logs every exchange (*regular logging*). But Camel also ships with the **Throughput** logger, which is used whenever the **groupSize** option is specified.

**NOTE****Also a log in the DSL**

There is also a **log** directly in the DSL, but it has a different purpose. Its meant for lightweight and human logs. See more details at LogEIP.

92.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

92.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

92.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

92.4. COMPONENT OPTIONS

The Log component supports 3 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
exchangeFormatter (advanced)	Autowired Sets a custom ExchangeFormatter to convert the Exchange to a String suitable for logging. If not specified, we default to DefaultExchangeFormatter.		ExchangeFormatter

92.5. ENDPOINT OPTIONS

The Log endpoint is configured using URI syntax:

```
log:loggerName
```

with the following path and query parameters:

92.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
loggerName (producer)	Required Name of the logging category to use.		String

92.5.2. Query Parameters (27 parameters)

Name	Description	Default	Type
groupActiveOnly (producer)	If true, will hide stats when no new messages have been received for a time interval, if false, show stats regardless of message traffic.	true	Boolean
groupDelay (producer)	Set the initial delay for stats (in millis).		Long
groupInterval (producer)	If specified will group message stats by this time interval (in millis).		Long
groupSize (producer)	An integer that specifies a group size for throughput logging.		Integer
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
level (producer)	Logging level to use. The default value is INFO. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	INFO	String
logMask (producer)	If true, mask sensitive information like password or passphrase in the log.		Boolean
marker (producer)	An optional Marker name to use.		String
exchangeFormatter (advanced)	To use a custom exchange formatter.		ExchangeFormatter

Name	Description	Default	Type
maxChars (formatting)	Limits the number of characters logged per line.	10000	int
multiline (formatting)	If enabled then each information is outputted on a newline.	false	boolean
showAll (formatting)	Quick option for turning all options on. (multiline, maxChars has to be manually set if to be used).	false	boolean
showAllProperties (formatting)	Show all of the exchange properties (both internal and custom).	false	boolean
showBody (formatting)	Show the message body.	true	boolean
showBodyType (formatting)	Show the body Java type.	true	boolean
showCaughtException (formatting)	If the exchange has a caught exception, show the exception message (no stack trace). A caught exception is stored as a property on the exchange (using the key <code>org.apache.camel.Exchange#EXCEPTION_CAUGHT</code>) and for instance a <code>doCatch</code> can catch exceptions.	false	boolean
showException (formatting)	If the exchange has an exception, show the exception message (no stacktrace).	false	boolean
showExchangeld (formatting)	Show the unique exchange ID.	false	boolean
showExchangePattern (formatting)	Shows the Message Exchange Pattern (or MEP for short).	true	boolean
showFiles (formatting)	If enabled Camel will output files.	false	boolean
showFuture (formatting)	If enabled Camel will on Future objects wait for it to complete to obtain the payload to be logged.	false	boolean
showHeaders (formatting)	Show the message headers.	false	boolean
showProperties (formatting)	Show the exchange properties (only custom). Use <code>showAllProperties</code> to show both internal and custom properties.	false	boolean

Name	Description	Default	Type
showStackTrace (formatting)	Show the stack trace, if an exchange has an exception. Only effective if one of showAll, showException or showCaughtException are enabled.	false	boolean
showStreams (formatting)	Whether Camel should show stream bodies or not (eg such as java.io.InputStream). Beware if you enable this option then you may not be able later to access the message body as the stream have already been read by this logger. To remedy this you will have to use Stream Caching.	false	boolean
skipBodyLineSeparator (formatting)	Whether to skip line separators when logging the message body. This allows to log the message body in one line, setting this option to false will preserve any line separators from the body, which then will log the body as is.	true	boolean
style (formatting)	Sets the outputs style to use. Enum values: • Default • Tab • Fixed	Default	OutputStyle

92.6. REGULAR LOGGER SAMPLE

In the route below we log the incoming orders at **DEBUG** level before the order is processed:

```
from("activemq:orders").to("log:com.mycompany.order?level=DEBUG").to("bean:processOrder");
```

Or using Spring XML to define the route:

```
<route>
  <from uri="activemq:orders"/>
  <to uri="log:com.mycompany.order?level=DEBUG"/>
  <to uri="bean:processOrder"/>
</route>
```

92.7. REGULAR LOGGER WITH FORMATTER SAMPLE

In the route below we log the incoming orders at **INFO** level before the order is processed.

```
from("activemq:orders").  
  to("log:com.mycompany.order?showAll=true&multiline=true").to("bean:processOrder");
```

92.8. THROUGHPUT LOGGER WITH GROUPSIZE SAMPLE

In the route below we log the throughput of the incoming orders at **DEBUG** level grouped by 10 messages.

```
from("activemq:orders").  
  to("log:com.mycompany.order?level=DEBUG&groupSize=10").to("bean:processOrder");
```

92.9. THROUGHPUT LOGGER WITH GROUPINTERVAL SAMPLE

This route will result in message stats logged every 10s, with an initial 60s delay and stats should be displayed even if there isn't any message traffic.

```
from("activemq:orders").  
  to("log:com.mycompany.order?  
    level=DEBUG&groupInterval=10000&groupDelay=60000&groupActiveOnly=false").to("bean:process  
    Order");
```

The following will be logged:

```
"Received: 1000 new messages, with total 2000 so far. Last group took: 10000 millis which is: 100  
messages per second. average: 100"
```

92.10. MASKING SENSITIVE INFORMATION LIKE PASSWORD

You can enable security masking for logging by setting **logMask** flag to **true**. Note that this option also affects Log EIP.

To enable mask in Java DSL at CamelContext level:

```
camelContext.setLogMask(true);
```

And in XML:

```
<camelContext logMask="true">
```

You can also turn it on/off at endpoint level. To enable mask in Java DSL at endpoint level, add `logMask=true` option in the URI for the log endpoint:

```
from("direct:start").to("log:foo?logMask=true");
```

And in XML:

```
<route>  
  <from uri="direct:foo"/>  
  <to uri="log:foo?logMask=true"/>  
</route>
```

org.apache.camel.support.processor.DefaultMaskingFormatter is used for the masking by default. If you want to use a custom masking formatter, put it into registry with the name **CamelCustomLogMask**. Note that the masking formatter must implement **org.apache.camel.spi.MaskingFormatter**.

92.11. FULL CUSTOMIZATION OF THE LOGGING OUTPUT

With the options outlined in the section, you can control much of the output of the logger. However, log lines will always follow this structure:

```
Exchange[Id:ID-machine-local-50656-1234567901234-1-2, ExchangePattern:InOut,
Properties:{CamelToEndpoint=log://org.apache.camel.component.log.TEST?showAll=true,
CamelCreatedTimestamp=Thu Mar 28 00:00:00 WET 2013},
Headers:{breadcrumbId=ID-machine-local-50656-1234567901234-1-1}, BodyType:String, Body:Hello
World, Out: null]
```

This format is unsuitable in some cases, perhaps because you need to...

- Filter the headers and properties that are printed, to strike a balance between insight and verbosity.
- Adjust the log message to whatever you deem most readable.
- Tailor log messages for digestion by log mining systems, e.g. Splunk.
- Print specific body types differently.

Whenever you require absolute customization, you can create a class that implements the interface. Within the **format(Exchange)** method you have access to the full Exchange, so you can select and extract the precise information you need, format it in a custom manner and return it. The return value will become the final log message.

You can have the Log component pick up your custom **ExchangeFormatter** in either of two ways:

Explicitly instantiating the LogComponent in your Registry:

```
<bean name="log" class="org.apache.camel.component.log.LogComponent">
  <property name="exchangeFormatter" ref="myCustomFormatter" />
</bean>
```

92.11.1. Convention over configuration

Simply by registering a bean with the name **logFormatter**; the Log Component is intelligent enough to pick it up automatically.

```
<bean name="logFormatter" class="com.xyz.MyCustomExchangeFormatter" />
```



NOTE

The **ExchangeFormatter** gets applied to **all Log endpoints within that Camel Context**. If you need different ExchangeFormatters for different endpoints, just instantiate the LogComponent as many times as needed, and use the relevant bean name as the endpoint prefix.

When using a custom log formatter, you can specify parameters in the log uri, which gets configured on the custom log formatter. Though when you do that you should define the "logFormatter" as prototype scoped so its not shared if you have different parameters, for example,

```
<bean name="logFormatter" class="com.xyz.MyCustomExchangeFormatter" scope="prototype"/>
```

And then we can have Camel routes using the log uri with different options:

```
<to uri="log:foo?param1=foo&param2=100"/>
```

```
<to uri="log:bar?param1=bar&param2=200"/>
```

92.12. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.component.log.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.log.enabled	Whether to enable auto configuration of the log component. This is enabled by default.		Boolean
camel.component.log.exchange-formatter	Sets a custom ExchangeFormatter to convert the Exchange to a String suitable for logging. If not specified, we default to DefaultExchangeFormatter. The option is a <code>org.apache.camel.spi.ExchangeFormatter</code> type.		ExchangeFormatter
camel.component.log.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 93. LRA

Since Camel 2.21

The LRA module provides bindings of the [Saga EIP](#) with any [MicroProfile compatible LRA Coordinator](#).

93.1. DEPENDENCIES

When using **lra** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-lra-starter</artifactId>
</dependency>
```

93.2. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options, which are listed below.

Name	Description	Default	Type
<code>camel.lra.coordinator-context-path</code>	The context path of the LRA coordinator service.		String
<code>camel.lra.coordinator-url</code>	The base URL of the LRA coordinator service (e.g.).		String
<code>camel.lra.enabled</code>	Global option to enable/disable component auto-configuration, default is true.	true	Boolean
<code>camel.lra.local-participant-context-path</code>	The context path of the local participant callback services.		String
<code>camel.lra.local-participant-url</code>	The local URL where the coordinator should send callbacks to (e.g.).		String

CHAPTER 94. MAIL

Both producer and consumer are supported

The Mail component provides access to Email via Spring's Mail support and the underlying JavaMail system.



NOTE

POP3 or IMAP

POP3 has some limitations and end users are encouraged to use IMAP if possible.



NOTE

Using mock-mail for testing

You can use a mock framework for unit testing, which allows you to test without the need for a real mail server. However you should remember to not include the mock-mail when you go into production or other environments where you need to send mails to a real mail server. Just the presence of the mock-javamail.jar on the classpath means that it will kick in and avoid sending the mails.

94.1. DEPENDENCIES

When using **camel-mail** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-mail-starter</artifactId>
</dependency>
```

94.2. URI FORMAT

Mail endpoints can have one of the following URI formats (for the protocols, SMTP, POP3, or IMAP, respectively):

```
smtp://[username@]host[:port][?options]
pop3://[username@]host[:port][?options]
imap://[username@]host[:port][?options]
```

The mail component also supports secure variants of these protocols (layered over SSL). You can enable the secure protocols by adding **s** to the scheme:

```
smtps://[username@]host[:port][?options]
pop3s://[username@]host[:port][?options]
imaps://[username@]host[:port][?options]
```

94.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level

- endpoint level

94.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

94.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

94.4. COMPONENT OPTIONS

The Mail component supports 43 options, which are listed below.

Name	Description	Default	Type
		t	

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
closeFolder (consumer)	Whether the consumer should close the folder after polling. Setting this option to false and having <code>disconnect=false</code> as well, then the consumer keep the folder open between polls.	true	boolean
copyTo (consumer)	After processing a mail message, it can be copied to a mail folder with the given name. You can override this configuration value, with a header with the key <code>copyTo</code> , allowing you to copy messages to folder names configured at runtime.		String
decodeFilename (consumer)	If set to true, the <code>MimeUtility.decodeText</code> method will be used to decode the filename. This is similar to setting JVM system property <code>mail.mime.encodefilename</code> .	false	boolean
delete (consumer)	Deletes the messages after they have been processed. This is done by setting the DELETED flag on the mail message. If false, the SEEN flag is set instead. As of Camel 2.10 you can override this configuration option by setting a header with the key <code>delete</code> to determine if the mail should be deleted or not.	false	boolean
disconnect (consumer)	Whether the consumer should disconnect after polling. If enabled this forces Camel to connect on each poll.	false	boolean
handleFailedMessage (consumer)	If the mail consumer cannot retrieve a given mail message, then this option allows to handle the caused exception by the consumer's error handler. By enable the bridge error handler on the consumer, then the Camel routing error handler can handle the exception instead. The default behavior would be the consumer throws an exception and no mails from the batch would be able to be routed by Camel.	false	boolean

Name	Description	Default	Type
mimeDecodeHeaders (consumer)	This option enables transparent MIME decoding and unfolding for mail headers.	false	boolean
moveTo (consumer)	After processing a mail message, it can be moved to a mail folder with the given name. You can override this configuration value, with a header with the key <code>moveTo</code> , allowing you to move messages to folder names configured at runtime.		String
peek (consumer)	Will mark the <code>javax.mail.Message</code> as peeked before processing the mail message. This applies to <code>IMAPMessage</code> messages types only. By using <code>peek</code> the mail will not be eager marked as SEEN on the mail server, which allows us to rollback the mail message if there is an error processing in Camel.	true	boolean
skipFailedMessage (consumer)	If the mail consumer cannot retrieve a given mail message, then this option allows to skip the message and move on to retrieve the next mail message. The default behavior would be the consumer throws an exception and no mails from the batch would be able to be routed by Camel.	false	boolean
unseen (consumer)	Whether to limit by unseen mails only.	true	boolean
fetchSize (consumer (advanced))	Sets the maximum number of messages to consume during a poll. This can be used to avoid overloading a mail server, if a mailbox folder contains a lot of messages. Default value of -1 means no fetch size and all messages will be consumed. Setting the value to 0 is a special corner case, where Camel will not consume any messages at all.	-1	int
folderName (consumer (advanced))	The folder to poll.	INBOX	String

Name	Description	Default	Type
mapMailMessage (consumer (advanced))	Specifies whether Camel should map the received mail message to Camel body/headers/attachments. If set to true, the body of the mail message is mapped to the body of the Camel IN message, the mail headers are mapped to IN headers, and the attachments to Camel IN attachment message. If this option is set to false then the IN message contains a raw <code>javax.mail.Message</code> . You can retrieve this raw message by calling <code>exchange.getIn().getBody(javax.mail.Message.class)</code> .	true	boolean
bcc (producer)	Sets the BCC email address. Separate multiple email addresses with comma.		String
cc (producer)	Sets the CC email address. Separate multiple email addresses with comma.		String
from (producer)	The from email address.	camel@local host	String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
replyTo (producer)	The Reply-To recipients (the receivers of the response mail). Separate multiple email addresses with a comma.		String
subject (producer)	The Subject of the message being sent. Note: Setting the subject in the header takes precedence over this option.		String
to (producer)	Sets the To email address. Separate multiple email addresses with comma.		String
javaMailSender (producer (advanced))	To use a custom <code>org.apache.camel.component.mail.JavaMailSender</code> for sending emails.		JavaMailSender

Name	Description	Default	Type
additionalJavaMailProperties (advanced)	Sets additional java mail properties, that will append/override any default properties that is set based on all the other options. This is useful if you need to add some special options but want to keep the others as is.		Properties
alternativeBodyHeader (advanced)	Specifies the key to an IN message header that contains an alternative email body. For example, if you send emails in text/html format and want to provide an alternative mail body for non-HTML email clients, set the alternative mail body with this key as a header.	Camel MailAlternativeBody	String
attachmentsContentTransferEncodingResolver (advanced)	To use a custom AttachmentsContentTransferEncodingResolver to resolve what content-type-encoding to use for attachments.		AttachmentsContentTransferEncodingResolver
authenticator (advanced)	The authenticator for login. If set then the password and username are ignored. Can be used for tokens which can expire and therefore must be read dynamically.		MailAuthenticator
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
configuration (advanced)	Sets the Mail configuration.		MailConfiguration
connectionTimeout (advanced)	The connection timeout in milliseconds.	30000	int
contentType (advanced)	The mail message content type. Use text/html for HTML mails.	text/plain	String
contentTypeResolver (advanced)	Resolver to determine Content-Type for file attachments.		ContentTypeResolver
debugMode (advanced)	Enable debug mode on the underlying mail framework. The SUN Mail framework logs the debug messages to System.out by default.	false	boolean

Name	Description	Default	Type
ignoreUnsupportedCharset (advanced)	Option to let Camel ignore unsupported charset in the local JVM when sending mails. If the charset is unsupported then charset=XXX (where XXX represents the unsupported charset) is removed from the content-type and it relies on the platform default instead.	false	boolean
ignoreUriScheme (advanced)	Option to let Camel ignore unsupported charset in the local JVM when sending mails. If the charset is unsupported then charset=XXX (where XXX represents the unsupported charset) is removed from the content-type and it relies on the platform default instead.	false	boolean
javaMailProperties (advanced)	Sets the java mail options. Will clear any default properties and only use the properties provided for this method.		Properties
session (advanced)	Specifies the mail session that camel should use for all mail interactions. Useful in scenarios where mail sessions are created and managed by some other resource, such as a JavaEE container. When using a custom mail session, then the hostname and port from the mail session will be used (if configured on the session).		Session
useInlineAttachments (advanced)	Whether to use disposition inline or attachment.	false	boolean
headerFilterStrategy (filter)	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
password (security)	The password for login. See also setAuthenticator(MailAuthenticator).		String
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean
username (security)	The username for login. See also setAuthenticator(MailAuthenticator).		String

94.5. ENDPOINT OPTIONS

The Mail endpoint is configured using URI syntax:

```
imap:host:port
```

with the following path and query parameters:

94.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
host (common)	Required The mail server host name.		String
port (common)	The port number of the mail server.		int

94.5.2. Query Parameters (66 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
closeFolder (consumer)	Whether the consumer should close the folder after polling. Setting this option to false and having <code>disconnect=false</code> as well, then the consumer keep the folder open between polls.	true	boolean
copyTo (consumer)	After processing a mail message, it can be copied to a mail folder with the given name. You can override this configuration value, with a header with the key <code>copyTo</code> , allowing you to copy messages to folder names configured at runtime.		String
decodeFilename (consumer)	If set to true, the <code>MimeUtility.decodeText</code> method will be used to decode the filename. This is similar to setting JVM system property <code>mail.mime.encodefilename</code> .	false	boolean

Name	Description	Default	Type
delete (consumer)	Deletes the messages after they have been processed. This is done by setting the DELETED flag on the mail message. If false, the SEEN flag is set instead. As of Camel 2.10 you can override this configuration option by setting a header with the key delete to determine if the mail should be deleted or not.	false	boolean
disconnect (consumer)	Whether the consumer should disconnect after polling. If enabled this forces Camel to connect on each poll.	false	boolean
handleFailedMessage (consumer)	If the mail consumer cannot retrieve a given mail message, then this option allows to handle the caused exception by the consumer's error handler. By enable the bridge error handler on the consumer, then the Camel routing error handler can handle the exception instead. The default behavior would be the consumer throws an exception and no mails from the batch would be able to be routed by Camel.	false	boolean
maxMessagesPerPoll (consumer)	Specifies the maximum number of messages to gather per poll. By default, no maximum is set. Can be used to set a limit of e.g. 1000 to avoid downloading thousands of files when the server starts up. Set a value of 0 or negative to disable this option.		int
mimeDecodeHeaders (consumer)	This option enables transparent MIME decoding and unfolding for mail headers.	false	boolean
moveTo (consumer)	After processing a mail message, it can be moved to a mail folder with the given name. You can override this configuration value, with a header with the key moveTo, allowing you to move messages to folder names configured at runtime.		String
peek (consumer)	Will mark the javax.mail.Message as peeked before processing the mail message. This applies to IMAPMessage messages types only. By using peek the mail will not be eager marked as SEEN on the mail server, which allows us to rollback the mail message if there is an error processing in Camel.	true	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean

Name	Description	Default	Type
skipFailedMessage (consumer)	If the mail consumer cannot retrieve a given mail message, then this option allows to skip the message and move on to retrieve the next mail message. The default behavior would be the consumer throws an exception and no mails from the batch would be able to be routed by Camel.	false	boolean
unseen (consumer)	Whether to limit by unseen mails only.	true	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
fetchSize (consumer (advanced))	Sets the maximum number of messages to consume during a poll. This can be used to avoid overloading a mail server, if a mailbox folder contains a lot of messages. Default value of -1 means no fetch size and all messages will be consumed. Setting the value to 0 is a special corner case, where Camel will not consume any messages at all.	-1	int
folderName (consumer (advanced))	The folder to poll.	INBOX	String
mailUidGenerator (consumer (advanced))	A pluggable MailUidGenerator that allows to use custom logic to generate UUID of the mail message.		MailUidGenerator

Name	Description	Default	Type
mapMailMessage (consumer (advanced))	Specifies whether Camel should map the received mail message to Camel body/headers/attachments. If set to true, the body of the mail message is mapped to the body of the Camel IN message, the mail headers are mapped to IN headers, and the attachments to Camel IN attachment message. If this option is set to false then the IN message contains a raw <code>javax.mail.Message</code> . You can retrieve this raw message by calling <code>exchange.getIn().getBody(javax.mail.Message.class)</code> .	true	boolean
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
postProcessAction (consumer (advanced))	Refers to an <code>MailBoxPostProcessAction</code> for doing post processing tasks on the mailbox once the normal processing ended.		<code>MailBoxPostProcessAction</code>
bcc (producer)	Sets the BCC email address. Separate multiple email addresses with comma.		String
cc (producer)	Sets the CC email address. Separate multiple email addresses with comma.		String
from (producer)	The from email address.	camel@localhost	String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
replyTo (producer)	The Reply-To recipients (the receivers of the response mail). Separate multiple email addresses with a comma.		String

Name	Description	Default	Type
subject (producer)	The Subject of the message being sent. Note: Setting the subject in the header takes precedence over this option.		String
to (producer)	Sets the To email address. Separate multiple email addresses with comma.		String
javaMailSender (producer (advanced))	To use a custom org.apache.camel.component.mail.JavaMailSender for sending emails.		JavaMailSender
additionalJavaMailProperties (advanced)	Sets additional java mail properties, that will append/override any default properties that is set based on all the other options. This is useful if you need to add some special options but want to keep the others as is.		Properties
alternativeBodyHeader (advanced)	Specifies the key to an IN message header that contains an alternative email body. For example, if you send emails in text/html format and want to provide an alternative mail body for non-HTML email clients, set the alternative mail body with this key as a header.	Camel MailAlternativeBody	String
attachmentsContentTransferEncodingResolver (advanced)	To use a custom AttachmentsContentTransferEncodingResolver to resolve what content-type-encoding to use for attachments.		AttachmentsContentTransferEncodingResolver
authenticator (advanced)	The authenticator for login. If set then the password and username are ignored. Can be used for tokens which can expire and therefore must be read dynamically.		MailAuthenticator
binding (advanced)	Sets the binding used to convert from a Camel message to and from a Mail message.		MailBinding
connectionTimeout (advanced)	The connection timeout in milliseconds.	30000	int
contentType (advanced)	The mail message content type. Use text/html for HTML mails.	text/plain	String
contentTypeResolver (advanced)	Resolver to determine Content-Type for file attachments.		ContentTypeResolver

Name	Description	Default	Type
debugMode (advanced)	Enable debug mode on the underlying mail framework. The SUN Mail framework logs the debug messages to System.out by default.	false	boolean
headerFilterStrategy (advanced)	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter headers.		HeaderFilterStrategy
ignoreUnsupportedCharset (advanced)	Option to let Camel ignore unsupported charset in the local JVM when sending mails. If the charset is unsupported then charset=XXX (where XXX represents the unsupported charset) is removed from the content-type and it relies on the platform default instead.	false	boolean
ignoreUriScheme (advanced)	Option to let Camel ignore unsupported charset in the local JVM when sending mails. If the charset is unsupported then charset=XXX (where XXX represents the unsupported charset) is removed from the content-type and it relies on the platform default instead.	false	boolean
javaMailProperties (advanced)	Sets the java mail options. Will clear any default properties and only use the properties provided for this method.		Properties
session (advanced)	Specifies the mail session that camel should use for all mail interactions. Useful in scenarios where mail sessions are created and managed by some other resource, such as a JavaEE container. When using a custom mail session, then the hostname and port from the mail session will be used (if configured on the session).		Session
useInlineAttachments (advanced)	Whether to use disposition inline or attachment.	false	boolean
idempotentRepository (filter)	A pluggable repository org.apache.camel.spi.IdempotentRepository which allows to cluster consuming from the same mailbox, and let the repository coordinate whether a mail message is valid for the consumer to process. By default no repository is in use.		IdempotentRepository

Name	Description	Default	Type
idempotentRepositoryRemoveOnCommit (filter)	When using idempotent repository, then when the mail message has been successfully processed and is committed, should the message id be removed from the idempotent repository (default) or be kept in the repository. By default its assumed the message id is unique and has no value to be kept in the repository, because the mail message will be marked as seen/moved or deleted to prevent it from being consumed again. And therefore having the message id stored in the idempotent repository has little value. However this option allows to store the message id, for whatever reason you may have.	true	boolean
searchTerm (filter)	Refers to a javax.mail.search.SearchTerm which allows to filter mails based on search criteria such as subject, body, from, sent after a certain date etc.		SearchTerm
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	60000	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long

Name	Description	Default	Type
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit

Name	Description	Default	Type
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See <code>ScheduledExecutorService</code> in JDK for details.	true	boolean
password (security)	The password for login. See also <code>setAuthenticator(MailAuthenticator)</code> .		String
sslContextParameters (security)	To configure security using <code>SSLContextParameters</code> .		<code>SSLContextParameters</code>
username (security)	The username for login. See also <code>setAuthenticator(MailAuthenticator)</code> .		String
sortTerm (sort)	Sorting order for messages. Only natively supported for IMAP. Emulated to some degree when using POP3 or when IMAP server does not have the SORT capability.		<code>SortTerm[]</code>

94.5.3. Sample endpoints

Typically, you specify a URI with login credentials as follows (taking SMTP as an example):

```
smtp://[username@]host[:port][?password=somepwd]
```

Alternatively, it is possible to specify both the user name and the password as query options:

```
smtp://host[:port]?password=somepwd&username=someuser
```

For example:

```
smtp://mycompany.mailserver:30?password=tiger&username=scott
```

94.5.4. Component alias names

- IMAP
- IMAPs
- POP3s
- SMTP
- SMTPs

94.5.5. Default ports

Default port numbers are supported. If the port number is omitted, Camel determines the port number to use based on the protocol.

Protocol	Default Port Number
SMTP	25
SMTPS	465
POP3	110
POP3S	995
IMAP	143
IMAPS	993

94.6. SSL SUPPORT

The underlying mail framework is responsible for providing SSL support. You may either configure SSL/TLS support by completely specifying the necessary Java Mail API configuration options, or you may provide a configured `SSLContextParameters` through the component or endpoint configuration.

94.6.1. Using the JSSE Configuration Utility

The mail component supports SSL/TLS configuration through the [Camel JSSE Configuration Utility](#). This utility greatly decreases the amount of component specific code you need to write and is configurable at the endpoint and component levels. The following examples demonstrate how to use the utility with the mail component.

Programmatic configuration of the endpoint

```

KeyStoreParameters ksp = new KeyStoreParameters();
ksp.setResource("/users/home/server/truststore.jks");
ksp.setPassword("keystorePassword");
TrustManagersParameters tmp = new TrustManagersParameters();
tmp.setKeyStore(ksp);
SSLContextParameters scp = new SSLContextParameters();
scp.setTrustManagers(tmp);
Registry registry = ...
registry.bind("sslContextParameters", scp);
...
from(...)
    .to("smtps://smtp.google.com?
        username=user@gmail.com&password=password&sslContextParameters=#sslContextParameters");

```

Spring DSL based configuration of endpoint

```

...
<camel:sslContextParameters id="sslContextParameters">

```

```

<camel:trustManagers>
  <camel:keyStore resource="/users/home/server/truststore.jks" password="keystorePassword"/>
</camel:trustManagers>
</camel:sslContextParameters>...
...
<to uri="smtps://smtp.google.com?
username=user@gmail.com&password=password&sslContextParameters=#sslContextParameters"/
>...

```

94.6.2. Configuring JavaMail Directly

Camel uses Jakarta JavaMail, which only trusts certificates issued by well known Certificate Authorities (the default JVM trust configuration). If you issue your own certificates, you have to import the CA certificates into the JVM's Java trust/key store files, override the default JVM trust/key store files (see **SSLNOTES.txt** in JavaMail for details).

94.7. MAIL MESSAGE CONTENT

Camel uses the message exchange's IN body as the [MimeMessage](#) text content. The body is converted to `String.class`.

Camel copies all of the exchange's IN headers to the [MimeMessage](#) headers.

The subject of the [MimeMessage](#) can be configured using a header property on the IN message. The code below demonstrates this:

The same applies for other [MimeMessage](#) headers such as recipients, so you can use a header property as To:

When using the `MailProducer` to send the mail to server, you should be able to get the message id of the [MimeMessage](#) with the key `CamelMailMessageId` from the Camel message header.

94.8. HEADERS TAKE PRECEDENCE OVER PRE-CONFIGURED RECIPIENTS

The recipients specified in the message headers always take precedence over recipients pre-configured in the endpoint URI. The idea is that if you provide any recipients in the message headers, that is what you get. The recipients pre-configured in the endpoint URI are treated as a fallback.

In the sample code below, the email message is sent to **davsclaus@apache.org**, because it takes precedence over the pre-configured recipient, **info@mycompany.com**. Any **CC** and **BCC** settings in the endpoint URI are also ignored and those recipients will not receive any mail. The choice between headers and pre-configured settings is all or nothing: the mail component *either* takes the recipients exclusively from the headers or exclusively from the pre-configured settings. It is not possible to mix and match headers and pre-configured settings.

```

Map<String, Object> headers = new HashMap<String, Object>();
headers.put("to", "davsclaus@apache.org");

template.sendBodyAndHeaders("smtp://admin@localhost?to=info@mycompany.com", "Hello World",
headers);

```

94.9. MULTIPLE RECIPIENTS FOR EASIER CONFIGURATION

It is possible to set multiple recipients using a comma-separated or a semicolon-separated list. This applies both to header settings and to settings in an endpoint URI. For example:

```
Map<String, Object> headers = new HashMap<String, Object>();
headers.put("to", "davsclaus@apache.org ; jstrachan@apache.org ; ningjiang@apache.org");
```

The preceding example uses a semicolon, `;`, as the separator character.

94.10. SETTING SENDER NAME AND EMAIL

You can specify recipients in the format, **name <email>**, to include both the name and the email address of the recipient.

For example, you define the following headers on the a Message:

```
Map headers = new HashMap();
map.put("To", "Claus Ibsen <davsclaus@apache.org>");
map.put("From", "James Strachan <jstrachan@apache.org>");
map.put("Subject", "Camel is cool");
```

94.11. JAVAMAIL API (EX SUN JAVAMAIL)

[JavaMail API](#) is used under the hood for consuming and producing mails. We encourage end-users to consult these references when using either POP3 or IMAP protocol. Note particularly that POP3 has a much more limited set of features than IMAP.

- [JavaMail POP3 API](#)
- [JavaMail IMAP API](#)
- And generally about the [MAIL Flags](#)

94.12. SAMPLES

We start with a simple route that sends the messages received from a JMS queue as emails. The email account is the **admin** account on **mymailserver.com**.

```
from("jms://queue:subscription").to("smtp://admin@mymailserver.com?password=secret");
```

In the next sample, we poll a mailbox for new emails once every minute.

```
from("imap://admin@mymailserver.com?password=secret&unseen=true&delay=60000")
.to("seda://mails");
```

94.13. SENDING MAIL WITH ATTACHMENT SAMPLE



NOTE

Attachments are not support by all Camel components

The *Attachments API* is based on the Java Activation Framework and is generally only used by the Mail API. Since many of the other Camel components do not support attachments, the attachments could potentially be lost as they propagate along the route. The rule of thumb, therefore, is to add attachments just before sending a message to the mail endpoint.

The mail component supports attachments. In the sample below, we send a mail message containing a plain text message with a logo file attachment.

94.14. SSL SAMPLE

In this sample, we want to poll our Google mail inbox for mails. To download mail onto a local mail client, Google mail requires you to enable and configure SSL. This is done by logging into your Google mail account and changing your settings to allow IMAP access. Google have extensive documentation on how to do this.

```
from("imaps://imap.gmail.com?
username=YOUR_USERNAME@gmail.com&password=YOUR_PASSWORD"
+ "&delete=false&unseen=true&delay=60000").to("log:newmail");
```

The preceding route polls the Google mail inbox for new mails once every minute and logs the received messages to the **newmail** logger category.

Running the sample with **DEBUG** logging enabled, we can monitor the progress in the logs:

```
2008-05-08 06:32:09,640 DEBUG MailConsumer - Connecting to MailStore
imaps://imap.gmail.com:993 (SSL enabled), folder=INBOX
2008-05-08 06:32:11,203 DEBUG MailConsumer - Polling mailfolder: imaps://imap.gmail.com:993
(SSL enabled), folder=INBOX
2008-05-08 06:32:11,640 DEBUG MailConsumer - Fetching 1 messages. Total 1 messages.
2008-05-08 06:32:12,171 DEBUG MailConsumer - Processing message: messageNumber=[332],
from=[James Bond <007@mi5.co.uk>], to=YOUR_USERNAME@gmail.com], subject=[...
2008-05-08 06:32:12,187 INFO newmail - Exchange[MailMessage: messageNumber=[332], from=
[James Bond <007@mi5.co.uk>], to=YOUR_USERNAME@gmail.com], subject=[...
```

94.15. CONSUMING MAILS WITH ATTACHMENT SAMPLE

In this sample we poll a mailbox and store all attachments from the mails as files. First, we define a route to poll the mailbox. As this sample is based on google mail, it uses the same route as shown in the SSL sample:

```
from("imaps://imap.gmail.com?
username=YOUR_USERNAME@gmail.com&password=YOUR_PASSWORD"
+ "&delete=false&unseen=true&delay=60000").process(new MyMailProcessor());
```

Instead of logging the mail we use a processor where we can process the mail from java code:

```
public void process(Exchange exchange) throws Exception {
    // the API is a bit clunky so we need to loop
    AttachmentMessage attachmentMessage = exchange.getMessage(AttachmentMessage.class);
    Map<String, DataHandler> attachments = attachmentMessage.getAttachments();
```

```

if (attachments.size() > 0) {
    for (String name : attachments.keySet()) {
        DataHandler dh = attachments.get(name);
        // get the file name
        String filename = dh.getName();

        // get the content and convert it to byte[]
        byte[] data = exchange.getContext().getTypeConverter()
            .convertTo(byte[].class, dh.getInputStream());

        // write the data to a file
        FileOutputStream out = new FileOutputStream(filename);
        out.write(data);
        out.flush();
        out.close();
    }
}
}

```

As you can see the API to handle attachments is a bit clunky but it's there so you can get the **javax.activation.DataHandler** so you can handle the attachments using standard API.

94.16. HOW TO SPLIT A MAIL MESSAGE WITH ATTACHMENTS

In this example we consume mail messages which may have a number of attachments. What we want to do is to use the Splitter EIP per individual attachment, to process the attachments separately. For example if the mail message has 5 attachments, we want the Splitter to process five messages, each having a single attachment. To do this we need to provide a custom Expression to the Splitter where we provide a List<Message> that contains the five messages with the single attachment.

The code is provided out of the box in Camel 2.10 onwards in the **camel-mail** component. The code is in the class: **org.apache.camel.component.mail.SplitAttachmentsExpression**, which you can find in the source code [here](#).

In the Camel route you then need to use this Expression in the route as shown below:

If you use XML DSL then you need to declare a method call expression in the Splitter as shown below

```

<split>
  <method beanType="org.apache.camel.component.mail.SplitAttachmentsExpression"/>
  <to uri="mock:split"/>
</split>

```

You can also split the attachments as byte[] to be stored as the message body. This is done by creating the expression with boolean true

```
SplitAttachmentsExpression split = SplitAttachmentsExpression(true);
```

And then use the expression with the splitter EIP.

94.17. USING CUSTOM SEARCHTERM

You can configure a **searchTerm** on the **MailEndpoint** which allows you to filter out unwanted mails.

For example to filter mails to contain Camel in either Subject or Text you can do as follows:

```
<route>
  <from uri="imaps://mymailseerver?
username=foo&password=secret&searchTerm.subjectOrBody=Camel"/>
  <to uri="bean:myBean"/>
</route>
```

Notice we use the "**searchTerm.subjectOrBody**" as parameter key to indicate that we want to search on mail subject or body, to contain the word "Camel".

The class **org.apache.camel.component.mail.SimpleSearchTerm** has a number of options you can configure:

Or to get the new unseen emails going 24 hours back in time you can do. Notice the "now-24h" syntax. See the table below for more details.

```
<route>
  <from uri="imaps://mymailseerver?
username=foo&password=secret&searchTerm.fromSentDate=now-24h"/>
  <to uri="bean:myBean"/>
</route>
```

You can have multiple searchTerm in the endpoint uri configuration. They would then be combined together using AND operator, eg so both conditions must match. For example to get the last unseen emails going back 24 hours which has Camel in the mail subject you can do:

```
<route>
  <from uri="imaps://mymailseerver?
username=foo&password=secret&searchTerm.subject=Camel&searchTerm.fromSentDate=now-
24h"/>
  <to uri="bean:myBean"/>
</route>
```

The **SimpleSearchTerm** is designed to be easily configurable from a POJO, so you can also configure it using a <bean> style in XML

```
<bean id="mySearchTerm" class="org.apache.camel.component.mail.SimpleSearchTerm">
  <property name="subject" value="Order"/>
  <property name="to" value="acme-order@acme.com"/>
  <property name="fromSentDate" value="now"/>
</bean>
```

You can then refer to this bean, using #beanId in your Camel route as shown:

```
<route>
  <from uri="imaps://mymailseerver?
username=foo&password=secret&searchTerm=#mySearchTerm"/>
  <to uri="bean:myBean"/>
</route>
```

In Java there is a builder class to build compound **SearchTerms** using the **org.apache.camel.component.mail.SearchTermBuilder** class. This allows you to build complex terms such as:

■

```
// we just want the unseen mails which is not spam
SearchTermBuilder builder = new SearchTermBuilder();

builder.unseen().body(Op.not, "Spam").subject(Op.not, "Spam")
// which was sent from either foo or bar
.from("foo@somewhere.com").from(Op.or, "bar@somewhere.com");
// .. and we could continue building the terms

SearchTerm term = builder.build();
```

94.18. POLLING OPTIMIZATION

The parameter `maxMessagePerPoll` and `fetchSize` allow you to restrict the number message that should be processed for each poll. These parameters should help to prevent bad performance when working with folders that contain a lot of messages. In previous versions these parameters have been evaluated too late, so that big mailboxes could still cause performance problems. With Camel 3.1 these parameters are evaluated earlier during the poll to avoid these problems.

94.19. USING HEADERS WITH ADDITIONAL JAVA MAIL SENDER PROPERTIES

When sending mails, then you can provide dynamic java mail properties for the **JavaMailSender** from the Exchange as message headers with keys starting with **java.smtp..**

You can set any of the **java.smtp** properties which you can find in the Java Mail documentation.

For example to provide a dynamic uuid in **java.smtp.from** (SMTP MAIL command):

```
.setHeader("from", constant("reply2me@foo.com"));
.setHeader("java.smtp.from", method(UUID.class, "randomUUID"));
.to("smtp://mymailserver:1234");
```



NOTE

This is only supported when **not** using a custom **JavaMailSender**.

94.20. SPRING BOOT AUTO-CONFIGURATION

The component supports 50 options, which are listed below.

Name	Description	Default	Type
camel.component-mail.additional-java-mail-properties	Sets additional java mail properties, that will append/override any default properties that is set based on all the other options. This is useful if you need to add some special options but want to keep the others as is. The option is a <code>java.util.Properties</code> type.		Properties

Name	Description	Default	Type
camel.component.mail.alternative-body-header	Specifies the key to an IN message header that contains an alternative email body. For example, if you send emails in text/html format and want to provide an alternative mail body for non-HTML email clients, set the alternative mail body with this key as a header.	Camel MailAlternativeBody	String
camel.component.mail.attachments-content-transfer-encoding-resolver	To use a custom AttachmentsContentTransferEncodingResolver to resolve what content-type-encoding to use for attachments. The option is a <code>org.apache.camel.component.mail.AttachmentsContentTransferEncodingResolver</code> type.		AttachmentsContentTransferEncodingResolver
camel.component.mail.authenticator	The authenticator for login. If set then the password and username are ignored. Can be used for tokens which can expire and therefore must be read dynamically. The option is a <code>org.apache.camel.component.mail.MailAuthenticator</code> type.		MailAuthenticator
camel.component.mail.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.mail.bcc	Sets the BCC email address. Separate multiple email addresses with comma.		String
camel.component.mail.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.mail.cc	Sets the CC email address. Separate multiple email addresses with comma.		String

Name	Description	Default	Type
camel.component.mail.close-folder	Whether the consumer should close the folder after polling. Setting this option to false and having disconnect=false as well, then the consumer keep the folder open between polls.	true	Boolean
camel.component.mail.configuration	Sets the Mail configuration. The option is a org.apache.camel.component.mail.MailConfiguration type.		MailConfiguration
camel.component.mail.connection-timeout	The connection timeout in milliseconds.	30000	Integer
camel.component.mail.content-type	The mail message content type. Use text/html for HTML mails.	text/plain	String
camel.component.mail.content-type-resolver	Resolver to determine Content-Type for file attachments. The option is a org.apache.camel.component.mail.ContentTypeResolver type.		ContentTypeResolver
camel.component.mail.copy-to	After processing a mail message, it can be copied to a mail folder with the given name. You can override this configuration value, with a header with the key copyTo, allowing you to copy messages to folder names configured at runtime.		String
camel.component.mail.debug-mode	Enable debug mode on the underlying mail framework. The SUN Mail framework logs the debug messages to System.out by default.	false	Boolean
camel.component.mail.decode-filename	If set to true, the MimeUtility.decodeText method will be used to decode the filename. This is similar to setting JVM system property mail.mime.encodefilename.	false	Boolean
camel.component.mail.delete	Deletes the messages after they have been processed. This is done by setting the DELETED flag on the mail message. If false, the SEEN flag is set instead. As of Camel 2.10 you can override this configuration option by setting a header with the key delete to determine if the mail should be deleted or not.	false	Boolean

Name	Description	Default	Type
camel.component.mail.disconnect	Whether the consumer should disconnect after polling. If enabled this forces Camel to connect on each poll.	false	Boolean
camel.component.mail.enabled	Whether to enable auto configuration of the mail component. This is enabled by default.		Boolean
camel.component.mail.fetch-size	Sets the maximum number of messages to consume during a poll. This can be used to avoid overloading a mail server, if a mailbox folder contains a lot of messages. Default value of -1 means no fetch size and all messages will be consumed. Setting the value to 0 is a special corner case, where Camel will not consume any messages at all.	-1	Integer
camel.component.mail.folder-name	The folder to poll.	INBOX	String
camel.component.mail.from	The from email address.	camel@localhost	String
camel.component.mail.handle-failed-message	If the mail consumer cannot retrieve a given mail message, then this option allows to handle the caused exception by the consumer's error handler. By enable the bridge error handler on the consumer, then the Camel routing error handler can handle the exception instead. The default behavior would be the consumer throws an exception and no mails from the batch would be able to be routed by Camel.	false	Boolean
camel.component.mail.header-filter-strategy	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message. The option is a org.apache.camel.spi.HeaderFilterStrategy type.		HeaderFilterStrategy
camel.component.mail.ignore-unsupported-charset	Option to let Camel ignore unsupported charset in the local JVM when sending mails. If the charset is unsupported then charset=XXX (where XXX represents the unsupported charset) is removed from the content-type and it relies on the platform default instead.	false	Boolean
camel.component.mail.ignore-uri-scheme	Option to let Camel ignore unsupported charset in the local JVM when sending mails. If the charset is unsupported then charset=XXX (where XXX represents the unsupported charset) is removed from the content-type and it relies on the platform default instead.	false	Boolean

Name	Description	Default	Type
camel.component.mail.java-mail-properties	Sets the java mail options. Will clear any default properties and only use the properties provided for this method. The option is a java.util.Properties type.		Properties
camel.component.mail.java-mail-sender	To use a custom org.apache.camel.component.mail.JavaMailSender for sending emails. The option is a org.apache.camel.component.mail.JavaMailSender type.		JavaMailSender
camel.component.mail.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.mail.map-mail-message	Specifies whether Camel should map the received mail message to Camel body/headers/attachments. If set to true, the body of the mail message is mapped to the body of the Camel IN message, the mail headers are mapped to IN headers, and the attachments to Camel IN attachment message. If this option is set to false then the IN message contains a raw javax.mail.Message. You can retrieve this raw message by calling exchange.getIn().getBody(javax.mail.Message.class).	true	Boolean
camel.component.mail.mime-decode-headers	This option enables transparent MIME decoding and unfolding for mail headers.	false	Boolean
camel.component.mail.move-to	After processing a mail message, it can be moved to a mail folder with the given name. You can override this configuration value, with a header with the key moveTo, allowing you to move messages to folder names configured at runtime.		String
camel.component.mail.password	The password for login. See also setAuthenticator(MailAuthenticator).		String

Name	Description	Default	Type
camel.component.mail.peek	Will mark the javax.mail.Message as peeked before processing the mail message. This applies to IMAPMessage messages types only. By using peek the mail will not be eager marked as SEEN on the mail server, which allows us to rollback the mail message if there is an error processing in Camel.	true	Boolean
camel.component.mail.reply-to	The Reply-To recipients (the receivers of the response mail). Separate multiple email addresses with a comma.		String
camel.component.mail.session	Specifies the mail session that camel should use for all mail interactions. Useful in scenarios where mail sessions are created and managed by some other resource, such as a JavaEE container. When using a custom mail session, then the hostname and port from the mail session will be used (if configured on the session). The option is a javax.mail.Session type.		Session
camel.component.mail.skip-failed-message	If the mail consumer cannot retrieve a given mail message, then this option allows to skip the message and move on to retrieve the next mail message. The default behavior would be the consumer throws an exception and no mails from the batch would be able to be routed by Camel.	false	Boolean
camel.component.mail.ssl-context-parameters	To configure security using SSLContextParameters. The option is a org.apache.camel.support.jsse.SSLContextParameters type.		SSLContextParameters
camel.component.mail.subject	The Subject of the message being sent. Note: Setting the subject in the header takes precedence over this option.		String
camel.component.mail.to	Sets the To email address. Separate multiple email addresses with comma.		String
camel.component.mail.unseen	Whether to limit by unseen mails only.	true	Boolean
camel.component.mail.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean

Name	Description	Default	Type
camel.component.mail.use-inline-attachments	Whether to use disposition inline or attachment.	false	Boolean
camel.component.mail.username	The username for login. See also <code>setAuthenticator(MailAuthenticator)</code> .		String
camel.dataformat.mime-multipart.binary-content	Defines whether the content of binary parts in the MIME multipart is binary (true) or Base-64 encoded (false). Default is false.	false	Boolean
camel.dataformat.mime-multipart.enabled	Whether to enable auto configuration of the mime-multipart data format. This is enabled by default.		Boolean
camel.dataformat.mime-multipart.headers-inline	Defines whether the MIME-Multipart headers are part of the message body (true) or are set as Camel headers (false). Default is false.	false	Boolean
camel.dataformat.mime-multipart.include-headers	A regex that defines which Camel headers are also included as MIME headers into the MIME multipart. This will only work if <code>headersInline</code> is set to true. Default is to include no headers.		String
camel.dataformat.mime-multipart.multipart-sub-type	Specify the subtype of the MIME Multipart. Default is mixed.	mixed	String
camel.dataformat.mime-multipart.multipart-without-attachment	Defines whether a message without attachment is also marshaled into a MIME Multipart (with only one body part). Default is false.	false	Boolean

CHAPTER 95. MAIL MICROSOFT OAUTH

Since Camel 3.18.4.

The Mail Microsoft OAuth2 provides an implementation of **org.apache.camel.component.mail.MailAuthenticator** to authenticate IMAP/POP/SMTP connections and access to Email via Spring's Mail support and the underlying JavaMail system.

95.1. DEPENDENCIES

Add the following dependency to your **pom.xml** for this component:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-mail-microsoft-oauth</artifactId>
</dependency>
```

Importing **camel-mail-microsoft-oauth** will automatically import camel-mail component.

95.2. MICROSOFT EXCHANGE ONLINE OAUTH2 MAIL AUTHENTICATOR IMAP SAMPLE

To use OAuth, an application must be registered with Azure Active Directory. Follow the instructions to register a new application.

Procedure

1. Enable the application to access Exchange mailboxes via client credentials flow. For more information, see [Authenticate an IMAP, POP or SMTP connection using OAuth](#)
2. Once everything is set up, declare and register in the registry, an instance of **org.apache.camel.component.mail.MicrosoftExchangeOnlineOAuth2MailAuthenticator**.
3. For Example, in a Spring Boot application:

```
@BindToRegistry("auth")
public MicrosoftExchangeOnlineOAuth2MailAuthenticator exchangeAuthenticator(){
    return new MicrosoftExchangeOnlineOAuth2MailAuthenticator(tenantId, clientId, clientSecret,
"jon@doe.com");
}
```

1. Then reference it in the Camel URI as follows:

```
from("imaps://outlook.office365.com:993"
    + "?authenticator=#auth"
    + "&mail.imaps.auth.mechanisms=XOAUTH2"
    + "&debugMode=true"
    + "&delete=false")
```

CHAPTER 96. MAPSTRUCT

Since Camel 3.19

Only producer is supported.

The camel-mapstruct component is used for converting POJOs using [MapStruct](#).

96.1. DEPENDENCIES

When using **mapstruct** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-mapstruct-starter</artifactId>
</dependency>
```

96.2. URI FORMAT

```
mapstruct:className[?options]
```

Where **className** is the fully qualified class name of the POJO to convert to.

96.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

96.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

96.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

96.4. COMPONENT OPTIONS

The MapStruct component supports 4 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
mapperPackageName (producer)	Required Package name(s) where Camel should discover Mapstruct mapping classes. Multiple package names can be separated by comma.		String
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
mapStructConverter (advanced)	Autowired To use a custom MapStructConverter such as adapting to a special runtime.		MapStructMapper Finder

96.5. ENDPOINT OPTIONS

The MapStruct endpoint is configured using URI syntax:

```
mapstruct:className
```

with the following path and query parameters:

96.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
className (producer)	Required The fully qualified class name of the POJO that mapstruct should convert to (target).		String

96.5.2. Query Parameters (2 parameters)

Name	Description	Default	Type
mandatory (producer)	Whether there must exist a mapstruct converter to convert to the POJO.	true	boolean
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

96.6. SETTING UP MAPSTRUCT

The camel-mapstruct component must be configured with one or more package names, for classpath scanning MapStruct **Mapper** classes. This is needed because the **Mapper** classes are to be used for converting POJOs with MapStruct.

For example, to set up two packages you can do as following:

```
MapstructComponent mc = context.getComponent("mapstruct", MapstructComponent.class);
mc.setMapperPackageName("com.foo.mapper,com.bar.mapper");
```

This can also be configured in **application.properties**:

```
camel.component.mapstruct.mapper-package-name = com.foo.mapper,com.bar.mapper
```

Camel will on startup scan these packages for classes which names ends with **Mapper**. These classes are then introspected to discover the mapping methods. These mapping methods are then registered into the Camel registry. This means that you can also use type converter to convert the POJOs with MapStruct, such as:

```
from("direct:foo")
    .convertBodyTo(MyFooDto.class);
```

Where **MyFooDto** is a POJO that MapStruct is able to convert to/from.

96.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.mapstruct.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.mapstruct.enabled</code>	Whether to enable auto configuration of the mapstruct component. This is enabled by default.		Boolean
<code>camel.component.mapstruct.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.mapstruct.mapstruct-converter</code>	To use a custom MapStructConverter such as adapting to a special runtime. The option is a <code>org.apache.camel.component.mapstruct.MapStructMapperFinder</code> type.		MapStructMapper Finder
<code>camel.component.mapstruct.mapstruct-package-name</code>	Package name(s) where Camel should discover Mapstruct mapping classes. Multiple package names can be separated by comma.		String

CHAPTER 97. MASTER

Only consumer is supported

The Camel-Master endpoint provides a way to ensure only a single consumer in a cluster consumes from a given endpoint; with automatic failover if that JVM dies.

This can be very useful if you need to consume from some legacy back end which either doesn't support concurrent consumption or due to commercial or stability reasons you can only have a single connection at any point in time.

97.1. DEPENDENCIES

When using **master** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-master-starter</artifactId>
</dependency>
```

97.2. USING THE MASTER ENDPOINT

Just prefix any camel endpoint with **master:someName:** where *someName* is a logical name and is used to acquire the master lock. e.g.

```
from("master:cheese:jms:foo").to("activemq:wine");
```

In this example, there master component ensures that the route is only active in one node, at any given time, in the cluster. So if there are 8 nodes in the cluster, then the master component will elect one route to be the leader, and only this route will be active, and hence only this route will consume messages from **jms:foo**. In case this route is stopped or unexpected terminated, then the master component will detect this, and re-elect another node to be active, which will then become active and start consuming messages from **jms:foo**.



NOTE

Apache ActiveMQ 5.x has such feature out of the box called [Exclusive Consumers](#).

97.3. URI FORMAT

```
master:namespace:endpoint[?options]
```

Where endpoint is any Camel endpoint you want to run in master/slave mode.

97.4. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

97.4.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

97.4.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

97.5. COMPONENT OPTIONS

The Master component supports 4 options, which are listed below.

Name	Description	Default	Type
		t	

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
service (advanced)	Inject the service to use.		CamelClusterService
serviceSelector (advanced)	Inject the service selector used to lookup the CamelClusterService to use.		Selector

97.6. ENDPOINT OPTIONS

The Master endpoint is configured using URI syntax:

```
master:namespace:delegateUri
```

with the following path and query parameters:

97.6.1. Path Parameters (2 parameters)

Name	Description	Default	Type
namespace (consumer)	Required The name of the cluster namespace to use.		String
delegateUri (consumer)	Required The endpoint uri to use in master/slave mode.		String

97.6.2. Query Parameters (3 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code> ● <code>InOptionalOut</code>		<code>ExchangePattern</code>

97.7. EXAMPLE

You can protect a clustered Camel application to only consume files from one active node.

```
// the file endpoint we want to consume from
String url = "file:target/inbox?delete=true";

// use the camel master component in the clustered group named myGroup
// to run a master/slave mode in the following Camel url
from("master:myGroup:" + url)
    .log(name + " - Received file: ${file:name}")
    .delay(delay)
    .log(name + " - Done file:  ${file:name}")
    .to("file:target/outbox");
```

The master component leverages `CamelClusterService` you can configure using

- **Java**

```
ZooKeeperClusterService service = new ZooKeeperClusterService();
service.setId("camel-node-1");
service.setNodes("myzk:2181");
```

```
service.setBasePath("/camel/cluster");

context.addService(service)
```

- **Xml (Spring/Blueprint)**

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://camel.apache.org/schema/spring
    http://camel.apache.org/schema/spring/camel-spring.xsd">

  <bean id="cluster"
class="org.apache.camel.component.zookeeper.cluster.ZooKeeperClusterService">
    <property name="id" value="camel-node-1"/>
    <property name="basePath" value="/camel/cluster"/>
    <property name="nodes" value="myzk:2181"/>
  </bean>

  <camelContext xmlns="http://camel.apache.org/schema/spring" autoStartup="false">
    ...
  </camelContext>

</beans>
```

- **Spring boot**

```
camel.component.zookeeper.cluster.service.enabled = true
camel.component.zookeeper.cluster.service.id      = camel-node-1
camel.component.zookeeper.cluster.service.base-path = /camel/cluster
camel.component.zookeeper.cluster.service.nodes    = myzk:2181
```

97.8. IMPLEMENTATIONS

Camel provides the following ClusterService implementations:

- camel-file
- camel-infinispan
- camel-kubernetes

97.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options, which are listed below.

Name	Description	Default	Type
camel.component. .master.autowired- enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component. .master.bridge- error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component. .master.enabled	Whether to enable auto configuration of the master component. This is enabled by default.		Boolean
camel.component. .master.service	Inject the service to use. The option is a <code>org.apache.camel.cluster.CamelClusterService</code> type.		CamelClusterService
camel.component. .master.service- selector	Inject the service selector used to lookup the <code>CamelClusterService</code> to use. The option is a <code>org.apache.camel.cluster.CamelClusterService.Selector</code> type.		CamelClusterService\$Selector

CHAPTER 98. MICROMETER

Since Camel 2.22

Only producer is supported

The [Micrometer](#) component allows to collect various metrics directly from Camel routes. Supported metric types are [counter](#), [summary](#), and [timer](#). [Micrometer](#) provides simple way to measure the behavior of an application. Configurable reporting backends (via Micrometer registries) enable different integration options for collecting and visualizing statistics.

The component also provides a **MicrometerRoutePolicyFactory** which allows to expose route statistics using Micrometer as well as **EventNotifier** implementations for counting routes and timing exchanges from their creation to their completion.

98.1. DEPENDENCIES

Maven users will need to add the following dependency to their **pom.xml** for this component:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-micrometer</artifactId>
</dependency>
```

At the same time update **dependencyManagement** section with:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.redhat.camel.springboot.platform</groupId>
      <artifactId>camel-spring-boot-bom</artifactId>
      <version>${camel-spring-boot-version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

98.2. URI FORMAT

```
micrometer:[ counter | summary | timer ]:metricname[?options]
```

98.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

98.3.1. Configuring component options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

98.3.2. Configuring endpoint options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

98.4. COMPONENT OPTIONS

The Micrometer component supports 3 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	False	Boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	True	Boolean

Name	Description	Default	Type
metricsRegistry (advanced)	To use a custom configured MetricRegistry.		MeterRegistry

98.5. ENDPOINT OPTIONS

The Micrometer endpoint is configured using URI syntax:

```
micrometer:metricsType:metricsName
```

with the following path and query parameters:

98.5.1. Path Parameters (3 parameters)

Name	Description	Default	Type
metricsType (producer)	Required Type of metrics. Enum values: • counter • summary • timer		Type
metricsName (producer)	Required Name of metrics.		String
tags (producer)	Tags of metrics.		Iterable

98.5.2. Query Parameters (6 parameters)

Name	Description	Default	Type
action (producer)	Action expression when using timer type. Enum values: • start • stop		String

Name	Description	Default	Type
decrement (producer)	Decrement value expression when using counter type.		String
increment (producer)	Increment value expression when using counter type.		boolean
metricsDescription (producer)	Description of metrics.		String
value (producer)	Value expression when using histogram type.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	False	Boolean

98.6. MESSAGE HEADERS

The Micrometer component supports 7 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelMetricsTimerAction (producer) Constant: HEADER_TIMER_ACTION	Override timer action in URI. Enum values: • start • stop		MicrometerTimerAction
CamelMetricsHistogramValue (producer) Constant: HEADER_HISTOGRAM_VALUE	Override histogram value in URI.		Long

Name	Description	Default	Type
CamelMetricsCounterDecrement (producer) Constant: HEADER_COUNTER_DECREMENT	Override decrement value in URI.		Double
CamelMetricsCounterIncrement (producer) Constant: HEADER_COUNTER_INCREMENT	Override increment value in URI.		Double
CamelMetricsName (producer) Constant: HEADER_METRIC_NAME	Override name value in URI.		String
CamelMetricsDescription (producer) Constant: HEADER_METRIC_DESCRIPTION	Override description value in URI.		String
CamelMetricsTags (producer) Constant: HEADER_METRIC_TAGS	To augment meter tags defined as URI parameters.		Iterable

98.7. METER REGISTRY

By default the Camel Micrometer component creates a **SimpleMeterRegistry** instance, suitable mainly for testing. You should define a dedicated registry by providing a **MeterRegistry** bean. Micrometer registries primarily determine the backend monitoring system to be used. A **CompositeMeterRegistry** can be used to address more than one monitoring target.

For example using Spring Java Configuration:

```
@Configuration
public static class MyConfig extends SingleRouteCamelConfiguration {

    @Bean
    @Override
    public RouteBuilder route() {
        return new RouteBuilder() {
            @Override
            public void configure() throws Exception {
                // define Camel routes here
            }
        };
    }
}
```

```

    };
}

@Bean(name = MicrometerConstants.METRICS_REGISTRY_NAME)
public MeterRegistry getMeterRegistry() {
    CompositeMeterRegistry registry = ...;
    registry.add(...);
    // ...
    return registry;
}
}

```

Or using CDI:

```

@Override
public void configure() {
    from(...)
        // Register the 'my-meter' meter in the MetricRegistry below
        .to("micrometer:meter:my-meter");
}

@Produces
// If multiple MetricRegistry beans
// @Named(MicrometerConstants.METRICS_REGISTRY_NAME)
MetricRegistry registry() {
    CompositeMeterRegistry registry = ...;
    registry.add(...);
    // ...
    return registry;
}
}

```

98.8. DEFAULT CAMEL METRICS

Some Camel specific metrics are available out of the box.

Name	Type	Description
camel.message.history	Timer	Sample of performance of each node in the route when message history is enabled
camel.routes.added	Gauge	Number of routes added
camel.routes.running	Gauge	Number of routes running
camel.exchanges.inflight	Gauge	Route inflight messages
camel.exchanges.total	Counter	Total number of processed exchanges
camel.exchanges.succeeded	Counter	Number of successfully completed exchange

Name	Type	Description
camel.exchanges.failed	Counter	Number of failed exchanges
camel.exchanges.failures.handled	Counter	Number of failures handled
camel.exchanges.external.redeliveries	Counter	Number of external initiated redeliveries (such as from JMS broker)
camel.exchange.event.notifier	Gauge + Summary	Metrics for message created, sent, completed, and failed events
camel.route.policy	Gauge + Summary	Route performance metrics
camel.route.policy.long.task	Gauge + Summary	Route long task metric

98.9. USAGE OF PRODUCERS

Each meter has type and name. Supported types are [counter](#), [distribution summary](#) and timer. If no type is provided then a counter is used by default.

The meter name is a string that is evaluated as **Simple** expression. In addition to using the **CamelMetricsName** header (see below), this allows to select the meter depending on exchange data.

The optional **tags** URI parameter is a comma-separated string, consisting of **key=value** expressions. Both **key** and **value** are strings that are also evaluated as **Simple** expression. E.g. the URI parameter **tags=X=\${header.Y}** would assign the current value of header **Y** to the key **X**.

98.9.1. Headers

The meter name defined in URI can be overridden by populating a header with name **CamelMetricsName**. The meter tags defined as URI parameters can be augmented by populating a header with name **CamelMetricsTags**.

For example:

```
from("direct:in")
    .setHeader(MicrometerConstants.HEADER_METRIC_NAME, constant("new.name"))
    .setHeader(MicrometerConstants.HEADER_METRIC_TAGS, constant(Tags.of("dynamic-key",
"dynamic-value")))
    .to("micrometer:counter:name.not.used?tags=key=value")
    .to("direct:out");
```

will update a counter with name **new.name** instead of **name.not.used** using the tag **dynamic-key** with value **dynamic-value** in addition to the tag **key** with value **value**.

All Metrics specific headers are removed from the message once the Micrometer endpoint finishes processing of exchange. While processing exchange Micrometer endpoint will catch all exceptions and write log entry using level **warn**.

98.10. COUNTER

```
micrometer:counter:name[?options]
```

98.10.1. Options

Name	Default	Description
increment		Double value to add to the counter
decrement		Double value to subtract from the counter

If neither **increment** or **decrement** is defined then counter value will be incremented by one. If **increment** and **decrement** are both defined only increment operation is called.

```
// update counter simple.counter by 7
from("direct:in")
  .to("micrometer:counter:simple.counter?increment=7")
  .to("direct:out");
```

```
// increment counter simple.counter by 1
from("direct:in")
  .to("micrometer:counter:simple.counter")
  .to("direct:out");
```

Both **increment** and **decrement** values are evaluated as **Simple** expressions with a Double result, e.g. if header **X** contains a value that evaluates to 3.0, the **simple.counter** counter is decremented by 3.0:

```
// decrement counter simple.counter by 3
from("direct:in")
  .to("micrometer:counter:simple.counter?decrement=${header.X}")
  .to("direct:out");
```

98.10.2. Headers

Like in camel-metrics, specific Message headers can be used to override increment and decrement values specified in the Micrometer endpoint URI.

Name	Description	Expected type
CamelMetricsCounterIncrement		Double value to add to the counter

Name	Description	Expected type
CamelMetricsCounterDecrement		Double value to subtract from the counter

```
from("direct:in")
  .setHeader(MicrometerConstants.HEADER_COUNTER_INCREMENT, constant(417.0D))
  .to("micrometer:counter:simple.counter?increment=7")
  .to("direct:out");
```

```
from("direct:in")
  .setHeader(MicrometerConstants.HEADER_COUNTER_INCREMENT, simple("${body.length}"))
  .to("micrometer:counter:body.length")
  .to("direct:out");
```

98.11. DISTRIBUTION SUMMARY

```
micrometer:summary:metricname[?options]
```

98.11.1. Options

Name	Default	Description
value		Value to use in histogram

If no **value** is not set, nothing is added to histogram and warning is logged.

```
// adds value 9923 to simple.histogram
from("direct:in")
  .to("micrometer:summary:simple.histogram?value=9923")
  .to("direct:out");
```

```
// nothing is added to simple.histogram; warning is logged
from("direct:in")
  .to("micrometer:summary:simple.histogram")
  .to("direct:out");
```

value is evaluated as **Simple** expressions with a Double result, e.g. if header **X** contains a value that evaluates to 3.0, this value is registered with the **simple.histogram**:

```
from("direct:in")
  .to("micrometer:summary:simple.histogram?value=${header.X}")
  .to("direct:out");
```

98.11.2. Headers

Like in **camel-metrics**, a specific Message header can be used to override the value specified in the Micrometer endpoint URI.

Name	Description	Expected type
CamelMetricsHistogramValue	Override histogram value in URI	Long

```
// adds value 992.0 to simple.histogram
from("direct:in")
  .setHeader(MicrometerConstants.HEADER_HISTOGRAM_VALUE, constant(992.0D))
  .to("micrometer:summary:simple.histogram?value=700")
  .to("direct:out")
```

98.12. TIMER

```
micrometer:timer:metricname[?options]
```

98.12.1. Options

Name	Default	Description
action		start or stop

If no **action** or invalid value is provided then warning is logged without any timer update. If action **start** is called on an already running timer or **stop** is called on an unknown timer, nothing is updated and warning is logged.

```
// measure time spent in route "direct:calculate"
from("direct:in")
  .to("micrometer:timer:simple.timer?action=start")
  .to("direct:calculate")
  .to("micrometer:timer:simple.timer?action=stop");
```

Timer.Sample objects are stored as Exchange properties between different Metrics component calls.

action is evaluated as a Simple expression returning a result of type **MicrometerTimerAction**.

98.12.2. Headers

Like in **camel-metrics**, a specific Message header can be used to override action value specified in the Micrometer endpoint URI.

Name	Description	Expected type
------	-------------	---------------

Name	Description	Expected type
CamelMetricsTimerAction	Override timer action in URI	org.apache.camel.component.micrometer.MicrometerTimerAction

```
// sets timer action using header
from("direct:in")
  .setHeader(MicrometerConstants.HEADER_TIMER_ACTION, MicrometerTimerAction.start)
  .to("micrometer:timer:simple.timer")
  .to("direct:out");
```

98.13. USING MICROMETER ROUTE POLICY FACTORY

MicrometerRoutePolicyFactory allows to add a RoutePolicy for each route in order to exposes route utilization statistics using Micrometer. This factory can be used in Java and XML as the examples below demonstrates.



NOTE

Instead of using the **MicrometerRoutePolicyFactory** you can define a dedicated **MicrometerRoutePolicy** per route you want to instrument, in case you only want to instrument a few selected routes.

From Java you just add the factory to the CamelContext as shown below:

```
context.addRoutePolicyFactory(new MicrometerRoutePolicyFactory());
```

And from XML DSL you define a <bean> as follows:

```
<!-- use camel-micrometer route policy to gather metrics for all routes -->
<bean id="metricsRoutePolicyFactory"
class="org.apache.camel.component.micrometer.routepolicy.MicrometerRoutePolicyFactory"/>
```

The **MicrometerRoutePolicyFactory** and **MicrometerRoutePolicy** supports the following options:

Name	Default	Description
prettyPrint	false	Whether to use pretty print when outputting statistics in json format
meterRegistry		Allow to use a shared MeterRegistry . If none is provided then Camel will create a shared instance used by the this CamelContext.
durationUnit	TimeUnit.MILLISECONDS	The unit to use for duration in when dumping the statistics as json.

Name	Default	Description
configuration	see below	MicrometerRoutePolicyConfiguration.class

The **MicrometerRoutePolicyConfiguration** supports the following options:

Name	Default	Description
additionalCounters	true	activates all additional counters
exchangesSucceeded	true	activates counter for succeeded exchanges
exchangesFailed	true	activates counter for failed exchanges
exchangesTotal	true	activates counter for total count of exchanges
externalRedeliveries	true	activates counter for redeliveries of exchanges
failuresHandled	true	activates counter for handled failures
longTask	false	activates long task timer (current processing time for micrometer)
timerInitiator	null	Consumer<Timer.Builder> for custom initialize Timer
longTaskInitiator	null	Consumer<LongTaskTimer.Builder> for custom initialize LongTaskTimer

If JMX is enabled in the CamelContext, the MBean is registered in the **type=services** tree with **name=MicrometerRoutePolicy**.

98.14. USING MICROMETER MESSAGE HISTORY FACTORY

MicrometerMessageHistoryFactory allows to use metrics to capture Message History performance statistics while routing messages. It works by using a Micrometer Timer for each node in all the routes. This factory can be used in Java and XML as the examples below demonstrates.

From Java you just set the factory to the CamelContext as shown below:

```
context.setMessageHistoryFactory(new MicrometerMessageHistoryFactory());
```

And from XML DSL you define a <bean> as follows:

```
<!-- use camel-micrometer message history to gather metrics for all messages being routed -->
```

```
<bean id="metricsMessageHistoryFactory"
class="org.apache.camel.component.micrometer.messagehistory.MicrometerMessageHistoryFactory"
/>
```

The following options is supported on the factory:

Name	Default	Description
prettyPrint	false	Whether to use pretty print when outputting statistics in json format
meterRegistry		Allow to use a shared MeterRegistry . If none is provided then Camel will create a shared instance used by the this CamelContext.
durationUnit	TimeUnit.MILLISE CONDS	The unit to use for duration in when dumping the statistics as json.

At runtime the metrics can be accessed from Java API or JMX which allows to gather the data as json output.

From Java code you can get the service from the CamelContext as shown:

```
MicrometerMessageHistoryService service =
context.hasService(MicrometerMessageHistoryService.class);
String json = service.dumpStatisticsAsJson();
```

If JMX is enabled in the CamelContext, the MBean is registered in the type=services tree with **name=MicrometerMessageHistory**.

98.15. MICROMETER EVENT NOTIFICATION

There is a **MicrometerRouteEventNotifier** (counting added and running routes) and a **MicrometerExchangeEventNotifier** (timing exchanges from their creation to their completion).

EventNotifiers can be added to the CamelContext, e.g.:

```
camelContext.getManagementStrategy().addEventNotifier(new MicrometerExchangeEventNotifier())
```

At runtime the metrics can be accessed from Java API or JMX which allows to gather the data as json output.

From Java code you can do get the service from the CamelContext as shown:

```
MicrometerEventNotifierService service = context.hasService(MicrometerEventNotifierService.class);
String json = service.dumpStatisticsAsJson();
```

If JMX is enabled in the CamelContext, the MBean is registered in the **type=services** tree with **name=MicrometerEventNotifier**.

98.16. INSTRUMENTING CAMEL THREAD POOLS

InstrumentedThreadPoolFactory allows you to gather performance information about Camel Thread Pools by injecting a **InstrumentedThreadPoolFactory** which collects information from inside of Camel. See more details at Advanced configuration of CamelContext using Spring.

98.17. EXPOSING MICROMETER STATISTICS IN JMX

Micrometer uses **MeterRegistry** implementations in order to publish statistics. While in production scenarios it is advisable to select a dedicated backend like Prometheus or Graphite, it may be sufficient for test or local deployments to publish statistics to JMX.

In order to achieve this, add the following dependency:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-jmx</artifactId>
  <version>${micrometer-version}</version>
</dependency>
```

and add a **JmxMeterRegistry** instance:

```
@Bean(name = MicrometerConstants.METRICS_REGISTRY_NAME)
public MeterRegistry getMeterRegistry() {
    CompositeMeterRegistry meterRegistry = new CompositeMeterRegistry();
    meterRegistry.add(...);
    meterRegistry.add(new JmxMeterRegistry(
        CamelJmxConfig.DEFAULT,
        Clock.SYSTEM,
        HierarchicalNameMapper.DEFAULT));
    return meterRegistry;
}
```

The **HierarchicalNameMapper** strategy determines how meter name and tags are assembled into an MBean name.

98.18. USING CAMEL MICROMETER WITH SPRING BOOT

When you use **camel-micrometer-starter** with Spring Boot, then Spring Boot auto configuration will automatically enable metrics capture if a **io.micrometer.core.instrument.MeterRegistry** is available.

For example to capture data with Prometheus, you can add the following dependency:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

See the following table for options to specify what metrics to capture, or to turn it off.

98.19. SPRING BOOT AUTO CONFIGURATION

Compared to the plain camel micrometer, micrometer component on Spring Boot provides 10 more options, which are listed below:

Name	Description	Default	Type
camel.component.micrometer.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	True	Boolean
camel.component.micrometer.enabled	Whether to enable auto configuration of the micrometer component. This is enabled by default.		Boolean
camel.component.micrometer.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	False	Boolean
camel.component.micrometer.metrics-registry	To use a custom configured MetricRegistry. The option is a io.micrometer.core.instrument.MeterRegistry type.		MeterRegistry
camel.metrics.enable-exchange-event-notifier	Set whether to enable the MicrometerExchangeEventNotifier for capturing metrics on exchange processing times.	True	Boolean

Name	Description	Default	Type
camel.metrics.enable-message-history	Set whether to enable the MicrometerMessageHistoryFactory for capturing metrics on individual route node processing times. Depending on the number of configured route nodes, there is the potential to create a large volume of metrics. Therefore, this option is disabled by default.	False	Boolean
camel.metrics.enable-route-event-notifier	Set whether to enable the MicrometerRouteEventNotifier for capturing metrics on the total number of routes and total number of routes running.	True	Boolean
camel.metrics.enable-route-policy	Set whether to enable the MicrometerRoutePolicyFactory for capturing metrics on route processing times.	True	Boolean
camel.metrics.uri-tag-dynamic	Whether to use static or dynamic values for URI tags in captured metrics. When using dynamic tags, then a REST service with base URL: /users/{id} will capture metrics with uri tag with the actual dynamic value such as: /users/123. However, this can lead to many tags as the URI is dynamic, so use this with care.	False	Boolean
camel.metrics.uri-tag-enabled	Whether HTTP uri tags should be enabled or not in captured metrics. If disabled then the uri tag, is likely not able to be resolved and will be marked as UNKNOWN.	True	Boolean

CHAPTER 99. MINIO

Since Camel 3.5

Both producer and consumer are supported

The Minio component supports storing and retrieving objects from/to [Minio](#) service.

99.1. PREREQUISITES

You must have valid credentials for authorized access to the buckets/folders. More information is available at [Minio](#).

99.2. DEPENDENCIES

When using **minio** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-minio-starter</artifactId>
</dependency>
```

99.3. URI FORMAT

```
minio://bucketName[?options]
```

The bucket will be created if it doesn't already exist. You can append query options to the URI in the following format,

```
?options=value&option2=value&...
```

For example in order to read file **hello.txt** from the bucket **helloBucket**, use the following snippet:

```
from("minio://helloBucket?accessKey=yourAccessKey&secretKey=yourSecretKey&prefix=hello.txt")
  .to("file:/var/downloaded");
```

99.4. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

99.4.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a

few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

99.4.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

99.5. COMPONENT OPTIONS

The Minio component supports 47 options, which are listed below.

Name	Description	Default	Type
autoCreateBucket (common)	Setting the autocreation of the bucket if bucket name not exist.	true	boolean
configuration (common)	The component configuration.		MinioConfiguration
customHttpClient (common)	Set custom HTTP client for authenticated access.		OkHttpClient
endpoint (common)	Endpoint can be an URL, domain name, IPv4 address or IPv6 address.		String

Name	Description	Default	Type
minioClient (common)	Autowired Reference to a Minio Client object in the registry.		MinioClient
objectLock (common)	Set when creating new bucket.	false	boolean
policy (common)	The policy for this queue to set in the method.		String
proxyPort (common)	TCP/IP port number. 80 and 443 are used as defaults for HTTP and HTTPS.		Integer
region (common)	The region in which Minio client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1). You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
secure (common)	Flag to indicate to use secure connection to minio service or not.	false	boolean
serverSideEncryption (common)	Server-side encryption.		ServerSideEncryption
serverSideEncryptionCustomerKey (common)	Server-side encryption for source object while copy/move objects.		ServerSideEncryptionCustomerKey
autoCloseBody (consumer)	If this option is true and includeBody is true, then the <code>MinioObject.close()</code> method will be called on exchange completion. This option is strongly related to includeBody option. In case of setting includeBody to true and autocloseBody to false, it will be up to the caller to close the <code>MinioObject</code> stream. Setting autocloseBody to true, will close the <code>MinioObject</code> stream automatically.	true	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
bypassGovernanceMode (consumer)	Set this flag if you want to bypassGovernanceMode when deleting a particular object.	false	boolean
deleteAfterRead (consumer)	Delete objects from Minio after they have been retrieved. The delete is only performed if the Exchange is committed. If a rollback occurs, the object is not deleted. If this option is false, then the same objects will be retrieve over and over again on the polls. Therefore you need to use the Idempotent Consumer EIP in the route to filter out duplicates. You can filter using the <code>MinioConstants#BUCKET_NAME</code> and <code>MinioConstants#OBJECT_NAME</code> headers, or only the <code>MinioConstants#OBJECT_NAME</code> header.	true	boolean
delimiter (consumer)	The delimiter which is used in the ListObjectsRequest to only consume objects we are interested in.		String
destinationBucketName (consumer)	Source bucket name.		String
destinationObjectName (consumer)	Source object name.		String
includeBody (consumer)	If it is true, the exchange body will be set to a stream to the contents of the file. If false, the headers will be set with the Minio object metadata, but the body will be null. This option is strongly related to autocloseBody option. In case of setting includeBody to true and autocloseBody to false, it will be up to the caller to close the MinioObject stream. Setting autocloseBody to true, will close the MinioObject stream automatically.	true	boolean
includeFolders (consumer)	The flag which is used in the ListObjectsRequest to set include folders.	false	boolean
includeUserMetadata (consumer)	The flag which is used in the ListObjectsRequest to get objects with user meta data.	false	boolean
includeVersions (consumer)	The flag which is used in the ListObjectsRequest to get objects with versioning.	false	boolean
length (consumer)	Number of bytes of object data from offset.		long

Name	Description	Default	Type
matchETag (consumer)	Set match ETag parameter for get object(s).		String
maxConnections (consumer)	Set the maxConnections parameter in the minio client configuration.	60	int
maxMessagesPerPoll (consumer)	Gets the maximum number of messages as a limit to poll at each polling. Gets the maximum number of messages as a limit to poll at each polling. The default value is 10. Use 0 or a negative number to set it as unlimited.	10	int
modifiedSince (consumer)	Set modified since parameter for get object(s).		ZonedDateTime
moveAfterRead (consumer)	Move objects from bucket to a different bucket after they have been retrieved. To accomplish the operation the destinationBucket option must be set. The copy bucket operation is only performed if the Exchange is committed. If a rollback occurs, the object is not moved.	false	boolean
notMatchETag (consumer)	Set not match ETag parameter for get object(s).		String
objectName (consumer)	To get the object from the bucket with the given object name.		String
offset (consumer)	Start byte position of object data.		long
prefix (consumer)	Object name starts with prefix.		String
recursive (consumer)	List recursively than directory structure emulation.	false	boolean
startAfter (consumer)	list objects in bucket after this object name.		String
unModifiedSince (consumer)	Set un modified since parameter for get object(s).		ZonedDateTime
useVersion1 (consumer)	when true, version 1 of REST API is used.	false	boolean

Name	Description	Default	Type
versionId (consumer)	Set specific version_ID of a object when deleting the object.		String
deleteAfterWrite (producer)	Delete file object after the Minio file has been uploaded.	false	boolean
keyName (producer)	Setting the key name for an element in the bucket through endpoint parameter.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
operation (producer)	The operation to do in case the user don't want to do only an upload. Enum values: ● copyObject ● listObjects ● deleteObject ● deleteObjects ● deleteBucket ● listBuckets ● getObject ● getObjectRange		MinioOperations
pojoRequest (producer)	If we want to use a POJO request as body or not.	false	boolean
storageClass (producer)	The storage class to set in the request.		String

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
accessKey (security)	Amazon AWS Secret Access Key or Minio Access Key. If not set camel will connect to service for anonymous access.		String
secretKey (security)	Amazon AWS Access Key Id or Minio Secret Key. If not set camel will connect to service for anonymous access.		String

99.6. ENDPOINT OPTIONS

The Minio endpoint is configured using URI syntax:

```
minio:bucketName
```

with the following path and query parameters:

99.6.1. Path Parameters (1 parameters)

Name	Description	Default	Type
bucketName (common)	Required Bucket name.		String

99.6.2. Query Parameters (63 parameters)

Name	Description	Default	Type
autoCreateBucket (common)	Setting the autocreation of the bucket if bucket name not exist.	true	boolean
customHttpClient (common)	Set custom HTTP client for authenticated access.		OkHttpClient
endpoint (common)	Endpoint can be an URL, domain name, IPv4 address or IPv6 address.		String

Name	Description	Default	Type
minioClient (common)	Autowired Reference to a Minio Client object in the registry.		MinioClient
objectLock (common)	Set when creating new bucket.	false	boolean
policy (common)	The policy for this queue to set in the method.		String
proxyPort (common)	TCP/IP port number. 80 and 443 are used as defaults for HTTP and HTTPS.		Integer
region (common)	The region in which Minio client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1). You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
secure (common)	Flag to indicate to use secure connection to minio service or not.	false	boolean
serverSideEncryption (common)	Server-side encryption.		ServerSideEncryption
serverSideEncryptionCustomerKey (common)	Server-side encryption for source object while copy/move objects.		ServerSideEncryptionCustomerKey
autoCloseBody (consumer)	If this option is true and includeBody is true, then the <code>MinioObject.close()</code> method will be called on exchange completion. This option is strongly related to includeBody option. In case of setting includeBody to true and autocloseBody to false, it will be up to the caller to close the MinioObject stream. Setting autocloseBody to true, will close the MinioObject stream automatically.	true	boolean
bypassGovernanceMode (consumer)	Set this flag if you want to bypassGovernanceMode when deleting a particular object.	false	boolean

Name	Description	Default	Type
deleteAfterRead (consumer)	Delete objects from Minio after they have been retrieved. The delete is only performed if the Exchange is committed. If a rollback occurs, the object is not deleted. If this option is false, then the same objects will be retrieve over and over again on the polls. Therefore you need to use the Idempotent Consumer EIP in the route to filter out duplicates. You can filter using the <code>MinioConstants#BUCKET_NAME</code> and <code>MinioConstants#OBJECT_NAME</code> headers, or only the <code>MinioConstants#OBJECT_NAME</code> header.	true	boolean
delimiter (consumer)	The delimiter which is used in the <code>ListObjectsRequest</code> to only consume objects we are interested in.		String
destinationBucketName (consumer)	Source bucket name.		String
destinationObjectName (consumer)	Source object name.		String
includeBody (consumer)	If it is true, the exchange body will be set to a stream to the contents of the file. If false, the headers will be set with the Minio object metadata, but the body will be null. This option is strongly related to <code>autocloseBody</code> option. In case of setting <code>includeBody</code> to true and <code>autocloseBody</code> to false, it will be up to the caller to close the <code>MinioObject</code> stream. Setting <code>autocloseBody</code> to true, will close the <code>MinioObject</code> stream automatically.	true	boolean
includeFolders (consumer)	The flag which is used in the <code>ListObjectsRequest</code> to set include folders.	false	boolean
includeUserMetadata (consumer)	The flag which is used in the <code>ListObjectsRequest</code> to get objects with user meta data.	false	boolean
includeVersions (consumer)	The flag which is used in the <code>ListObjectsRequest</code> to get objects with versioning.	false	boolean
length (consumer)	Number of bytes of object data from offset.		long
matchETag (consumer)	Set match ETag parameter for get object(s).		String

Name	Description	Default	Type
maxConnections (consumer)	Set the maxConnections parameter in the minio client configuration.	60	int
maxMessagesPer Poll (consumer)	Gets the maximum number of messages as a limit to poll at each polling. Gets the maximum number of messages as a limit to poll at each polling. The default value is 10. Use 0 or a negative number to set it as unlimited.	10	int
modifiedSince (consumer)	Set modified since parameter for get object(s).		ZonedDateTime
moveAfterRead (consumer)	Move objects from bucket to a different bucket after they have been retrieved. To accomplish the operation the destinationBucket option must be set. The copy bucket operation is only performed if the Exchange is committed. If a rollback occurs, the object is not moved.	false	boolean
notMatchETag (consumer)	Set not match ETag parameter for get object(s).		String
objectName (consumer)	To get the object from the bucket with the given object name.		String
offset (consumer)	Start byte position of object data.		long
prefix (consumer)	Object name starts with prefix.		String
recursive (consumer)	List recursively than directory structure emulation.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
startAfter (consumer)	list objects in bucket after this object name.		String
unModifiedSince (consumer)	Set un modified since parameter for get object(s).		ZonedDateTime
useVersion1 (consumer)	when true, version 1 of REST API is used.	false	boolean
versionId (consumer)	Set specific version_ID of a object when deleting the object.		String

Name	Description	Default	Type
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
deleteAfterWrite (producer)	Delete file object after the Minio file has been uploaded.	false	boolean
keyName (producer)	Setting the key name for an element in the bucket through endpoint parameter.		String

Name	Description	Default	Type
operation (producer)	The operation to do in case the user don't want to do only an upload. Enum values: • copyObject • listObjects • deleteObject • deleteObjects • deleteBucket • listBuckets • getObject • getObjectRange		MinioOperations
pojoRequest (producer)	If we want to use a POJO request as body or not.	false	boolean
storageClass (producer)	The storage class to set in the request.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int

Name	Description	Default	Type
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object

Name	Description	Default	Type
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
accessKey (security)	Amazon AWS Secret Access Key or Minio Access Key. If not set camel will connect to service for anonymous access.		String
secretKey (security)	Amazon AWS Access Key Id or Minio Secret Key. If not set camel will connect to service for anonymous access.		String

You have to provide the minioClient in the Registry or your accessKey and secretKey to access the [Minio](#).

99.7. BATCH CONSUMER

This component implements the Batch Consumer.

This allows you for instance to know how many messages exists in this batch and for instance let the Aggregator aggregate this number of messages.

99.8. MESSAGE HEADERS

The Minio component supports 21 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelMinioBucketName (common) Constant: BUCKET_NAME	Producer: The bucket Name which this object will be stored or which will be used for the current operation. Consumer: The name of the bucket in which this object is contained.		String
CamelMinioDestinationBucketName (producer) Constant: DESTINATION_BUCKET_NAME	The bucket Destination Name which will be used for the current operation.		String
CamelMinioContentControl (common) Constant: CACHE_CONTROL	Producer: The content control of this object. Consumer: The optional Cache-Control HTTP header which allows the user to specify caching behavior along the HTTP request/reply chain.		String
CamelMinioContentDisposition (common) Constant: CONTENT_DISPOSITION	Producer: The content disposition of this object. Consumer: The optional Content-Disposition HTTP header, which specifies presentational information such as the recommended filename for the object to be saved as.		String
CamelMinioContentEncoding (common) Constant: CONTENT_ENCODING	Producer: The content encoding of this object. Consumer: The optional Content-Encoding HTTP header specifying what content encodings have been applied to the object and what decoding mechanisms must be applied in order to obtain the media-type referenced by the Content-Type field.		String
CamelMinioContentLength (common) Constant: CONTENT_LENGTH	Producer: The content length of this object. Consumer: The Content-Length HTTP header indicating the size of the associated object in bytes.		Long

Name	Description	Default	Type
CamelMinioContentMD5 (common) Constant: CONTENT_MD5	Producer: The md5 checksum of this object. Consumer: The base64 encoded 128-bit MD5 digest of the associated object (content - not including headers) according to RFC 1864. This data is used as a message integrity check to verify that the data received by Minio is the same data that the caller sent.		String
CamelMinioContentType (common) Constant: CONTENT_TYPE	Producer: The content type of this object. Consumer: The Content-Type HTTP header, which indicates the type of content stored in the associated object. The value of this header is a standard MIME type.		String
CamelMinioETag (common) Constant: E_TAG	Producer: The ETag value for the newly uploaded object. Consumer: The hex encoded 128-bit MD5 digest of the associated object according to RFC 1864. This data is used as an integrity check to verify that the data received by the caller is the same data that was sent by Minio.		String
CamelMinioObjectName (common) Constant: OBJECT_NAME	Producer: The key under which this object will be stored or which will be used for the current operation. Consumer: The key under which this object is stored.		String
CamelMinioDestinationObjectName (producer) Constant: DESTINATION_OBJECT_NAME	The Destination key which will be used for the current operation.		String
CamelMinioLastModified (common) Constant: LAST_MODIFIED	Producer: The last modified timestamp of this object. Consumer: The value of the Last-Modified header, indicating the date and time at which Minio last recorded a modification to the associated object.		Date
CamelMinioStorageClass (producer) Constant: STORAGE_CLASS	The storage class of this object.		String

Name	Description	Default	Type
CamelMinioVersionId (common) Constant: VERSION_ID	Producer: The version Id of the object to be stored or returned from the current operation. Consumer: The version ID of the associated Minio object if available. Version IDs are only assigned to objects when an object is uploaded to an Minio bucket that has object versioning enabled.		String
CamelMinioCannedAcl (producer) Constant: CANNED_ACL	The canned acl that will be applied to the object. see com.amazonaws.services.s3.model.CannedAccessControlList for allowed values.		String
CamelMinioOperation (producer) Constant: MINIO_OPERATION	The operation to perform. Enum values: • copyObject • listObjects • deleteObject • deleteObjects • deleteBucket • listBuckets • getObject • getPartialObject		MinioOperations
CamelMinioServerSideEncryption (common) Constant: SERVER_SIDE_ENCRYPTION	Producer: Sets the server-side encryption algorithm when encrypting the object using Minio-managed keys. For example use AES256. Consumer: The server-side encryption algorithm when encrypting the object using Minio-managed keys.		String
CamelMinioExpirationTime (common) Constant: EXPIRATION_TIME	The expiration time.		String

Name	Description	Default	Type
CamelMinioReplicationStatus (common) Constant: REPLICATION_STATUS	The replication status.		String
CamelMinioOffset (producer) Constant: OFFSET	The offset.		String
CamelMinioLength (producer) Constant: LENGTH	The length.		String

99.8.1. Minio Producer operations

Camel-Minio component provides the following operation on the producer side:

- `copyObject`
- `deleteObject`
- `deleteObjects`
- `listBuckets`
- `deleteBucket`
- `listObjects`
- `getObject` (this will return a `MinioObject` instance)
- `getObjectRange` (this will return a `MinioObject` instance)

99.8.2. Advanced Minio configuration

If your Camel Application is running behind a firewall or if you need to have more control over the **MinioClient** instance configuration, you can create your own instance and refer to it in your Camel minio component configuration:

```
from("minio://MyBucket?minioClient=#client&delay=5000&maxMessagesPerPoll=5")
.to("mock:result");
```

99.8.3. Minio Producer Operation examples

- CopyObject: this operation copy an object from one bucket to a different one

```
from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(MinioConstants.DESTINATION_BUCKET_NAME,
"camelDestinationBucket");
        exchange.getIn().setHeader(MinioConstants.OBJECT_NAME, "camelKey");
        exchange.getIn().setHeader(MinioConstants.DESTINATION_OBJECT_NAME,
"camelDestinationKey");
    }
})
.to("minio://mycamelbucket?minioClient=#minioClient&operation=copyObject")
.to("mock:result");
```

This operation will copy the object with the name expressed in the header camelDestinationKey to the camelDestinationBucket bucket, from the bucket mycamelbucket.

- DeleteObject: this operation deletes an object from a bucket

```
from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(MinioConstants.OBJECT_NAME, "camelKey");
    }
})
.to("minio://mycamelbucket?minioClient=#minioClient&operation=deleteObject")
.to("mock:result");
```

This operation will delete the object camelKey from the bucket mycamelbucket.

- ListBuckets: this operation list the buckets for this account in this region

```
from("direct:start")
.to("minio://mycamelbucket?minioClient=#minioClient&operation=listBuckets")
.to("mock:result");
```

This operation will list the buckets for this account

- DeleteBucket: this operation delete the bucket specified as URI parameter or header

```
from("direct:start")
.to("minio://mycamelbucket?minioClient=#minioClient&operation=deleteBucket")
.to("mock:result");
```

This operation will delete the bucket mycamelbucket

- ListObjects: this operation list object in a specific bucket

```
from("direct:start")
.to("minio://mycamelbucket?minioClient=#minioClient&operation=listObjects")
.to("mock:result");
```

This operation will list the objects in the mycamelbucket bucket

- `GetObject`: this operation get a single object in a specific bucket

```
from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(MinioConstants.OBJECT_NAME, "camelKey");
    }
})
.to("minio://mycamelbucket?minioClient=#minioClient&operation=getObject")
.to("mock:result");
```

This operation will return an `MinioObject` instance related to the `camelKey` object in mycamelbucket bucket.

- `GetObjectRange`: this operation get a single object range in a specific bucket

```
from("direct:start").process(new Processor() {

    @Override
    public void process(Exchange exchange) throws Exception {
        exchange.getIn().setHeader(MinioConstants.OBJECT_NAME, "camelKey");
        exchange.getIn().setHeader(MinioConstants.OFFSET, "0");
        exchange.getIn().setHeader(MinioConstants.LENGTH, "9");
    }
})
.to("minio://mycamelbucket?minioClient=#minioClient&operation=getObjectRange")
.to("mock:result");
```

This operation will return an `MinioObject` instance related to the `camelKey` object in mycamelbucket bucket, containing bytes from 0 to 9.

99.9. BUCKET AUTOCREATION

With the option **`autoCreateBucket`** users are able to avoid the autocreation of a Minio Bucket in case it doesn't exist. The default for this option is **`true`**. If set to false any operation on a not-existent bucket in Minio won't be successful, and an error will be returned.

99.10. AUTOMATIC DETECTION OF MINIO CLIENT IN REGISTRY

The component is capable of detecting the presence of a Minio bean into the registry. If it's the only instance of that type it will be used as client, and you won't have to define it as uri parameter, like the example above. This may be really useful for smarter configuration of the endpoint.

99.11. MOVING STUFF BETWEEN A BUCKET AND ANOTHER BUCKET

Some users like to consume stuff from a bucket and move the content in a different one without using the `copyObject` feature of this component. If this is case for you, don't forget to remove the `bucketName` header from the incoming exchange of the consumer, otherwise the file will always be overwritten on the same original bucket.

99.12. MOVEAFTERREAD CONSUMER OPTION

In addition to `deleteAfterRead` it has been added another option, `moveAfterRead`. With this option enabled the consumed object will be moved to a target `destinationBucket` instead of being only deleted. This will require specifying the `destinationBucket` option. As example:

```
from("minio://mycamelbucket?
minioClient=#minioClient&moveAfterRead=true&destinationBucketName=myothercamelbucket")
.to("mock:result");
```

In this case the objects consumed will be moved to `myothercamelbucket` bucket and deleted from the original one (because of `deleteAfterRead` set to `true` as default).

99.13. USING A POJO AS BODY

Sometimes build a Minio Request can be complex, because of multiple options. We introduce the possibility to use a POJO as body. In Minio there are multiple operations you can submit, as an example for List brokers request, you can do something like:

```
from("direct:minio")
.setBody(ListObjectsArgs.builder()
.bucket(bucketName)
.recursive(getConfiguration().isRecursive()))
.to("minio://test?minioClient=#minioClient&operation=listObjects&pojoRequest=true")
```

In this way you'll pass the request directly without the need of passing headers and options specifically related to this operation.

99.14. SPRING BOOT AUTO-CONFIGURATION

The component supports 48 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.minio.access-key</code>	Amazon AWS Secret Access Key or Minio Access Key. If not set camel will connect to service for anonymous access.		String
<code>camel.component.minio.auto-close-body</code>	If this option is true and <code>includeBody</code> is true, then the <code>MinioObject.close()</code> method will be called on exchange completion. This option is strongly related to <code>includeBody</code> option. In case of setting <code>includeBody</code> to true and <code>autocloseBody</code> to false, it will be up to the caller to close the <code>MinioObject</code> stream. Setting <code>autocloseBody</code> to true, will close the <code>MinioObject</code> stream automatically.	true	Boolean
<code>camel.component.minio.auto-create-bucket</code>	Setting the autocreation of the bucket if bucket name not exist.	true	Boolean

Name	Description	Default	Type
camel.component.minio.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.minio.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.minio.bypass-governance-mode	Set this flag if you want to bypassGovernanceMode when deleting a particular object.	false	Boolean
camel.component.minio.configuration	The component configuration. The option is a <code>org.apache.camel.component.minio.MinioConfiguration</code> type.		MinioConfiguration
camel.component.minio.custom-http-client	Set custom HTTP client for authenticated access. The option is a <code>okhttp3.OkHttpClient</code> type.		OkHttpClient
camel.component.minio.delete-after-read	Delete objects from Minio after they have been retrieved. The delete is only performed if the Exchange is committed. If a rollback occurs, the object is not deleted. If this option is false, then the same objects will be retrieve over and over again on the polls. Therefore you need to use the Idempotent Consumer EIP in the route to filter out duplicates. You can filter using the <code>MinioConstants#BUCKET_NAME</code> and <code>MinioConstants#OBJECT_NAME</code> headers, or only the <code>MinioConstants#OBJECT_NAME</code> header.	true	Boolean
camel.component.minio.delete-after-write	Delete file object after the Minio file has been uploaded.	false	Boolean

Name	Description	Default	Type
camel.component.minio.delimiter	The delimiter which is used in the ListObjectsRequest to only consume objects we are interested in.		String
camel.component.minio.destination-bucket-name	Source bucket name.		String
camel.component.minio.destination-object-name	Source object name.		String
camel.component.minio.enabled	Whether to enable auto configuration of the minio component. This is enabled by default.		Boolean
camel.component.minio.endpoint	Endpoint can be an URL, domain name, IPv4 address or IPv6 address.		String
camel.component.minio.include-body	If it is true, the exchange body will be set to a stream to the contents of the file. If false, the headers will be set with the Minio object metadata, but the body will be null. This option is strongly related to autocloseBody option. In case of setting includeBody to true and autocloseBody to false, it will be up to the caller to close the MinioObject stream. Setting autocloseBody to true, will close the MinioObject stream automatically.	true	Boolean
camel.component.minio.include-folders	The flag which is used in the ListObjectsRequest to set include folders.	false	Boolean
camel.component.minio.include-user-metadata	The flag which is used in the ListObjectsRequest to get objects with user meta data.	false	Boolean
camel.component.minio.include-versions	The flag which is used in the ListObjectsRequest to get objects with versioning.	false	Boolean
camel.component.minio.key-name	Setting the key name for an element in the bucket through endpoint parameter.		String

Name	Description	Default	Type
camel.component.minio.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.minio.length	Number of bytes of object data from offset.		Long
camel.component.minio.match-e-tag	Set match ETag parameter for get object(s).		String
camel.component.minio.max-connections	Set the maxConnections parameter in the minio client configuration.	60	Integer
camel.component.minio.max-messages-per-poll	Gets the maximum number of messages as a limit to poll at each polling. Gets the maximum number of messages as a limit to poll at each polling. The default value is 10. Use 0 or a negative number to set it as unlimited.	10	Integer
camel.component.minio.minio-client	Reference to a Minio Client object in the registry. The option is a io.minio.MinioClient type.		MinioClient
camel.component.minio.modified-since	Set modified since parameter for get object(s). The option is a java.time.ZonedDateTime type.		ZonedDateTime
camel.component.minio.move-after-read	Move objects from bucket to a different bucket after they have been retrieved. To accomplish the operation the destinationBucket option must be set. The copy bucket operation is only performed if the Exchange is committed. If a rollback occurs, the object is not moved.	false	Boolean
camel.component.minio.not-match-e-tag	Set not match ETag parameter for get object(s).		String

Name	Description	Default	Type
<code>camel.component.minio.object-lock</code>	Set when creating new bucket.	false	Boolean
<code>camel.component.minio.object-name</code>	To get the object from the bucket with the given object name.		String
<code>camel.component.minio.offset</code>	Start byte position of object data.		Long
<code>camel.component.minio.operation</code>	The operation to do in case the user don't want to do only an upload.		MinioOperations
<code>camel.component.minio.pojo-request</code>	If we want to use a POJO request as body or not.	false	Boolean
<code>camel.component.minio.policy</code>	The policy for this queue to set in the method.		String
<code>camel.component.minio.prefix</code>	Object name starts with prefix.		String
<code>camel.component.minio.proxy-port</code>	TCP/IP port number. 80 and 443 are used as defaults for HTTP and HTTPS.		Integer
<code>camel.component.minio.recursive</code>	List recursively than directory structure emulation.	false	Boolean
<code>camel.component.minio.region</code>	The region in which Minio client needs to work. When using this parameter, the configuration will expect the lowercase name of the region (for example ap-east-1). You'll need to use the name <code>Region.EU_WEST_1.id()</code> .		String
<code>camel.component.minio.secret-key</code>	Amazon AWS Access Key Id or Minio Secret Key. If not set camel will connect to service for anonymous access.		String
<code>camel.component.minio.secure</code>	Flag to indicate to use secure connection to minio service or not.	false	Boolean
<code>camel.component.minio.server-side-encryption</code>	Server-side encryption. The option is a <code>io.minio.ServerSideEncryption</code> type.		ServerSideEncryption

Name	Description	Default	Type
<code>camel.component.minio.server-side-encryption-customer-key</code>	Server-side encryption for source object while copy/move objects. The option is a <code>io.minio.ServerSideEncryptionCustomerKey</code> type.		<code>ServerSideEncryptionCustomerKey</code>
<code>camel.component.minio.start-after</code>	list objects in bucket after this object name.		<code>String</code>
<code>camel.component.minio.storage-class</code>	The storage class to set in the request.		<code>String</code>
<code>camel.component.minio.unmodified-since</code>	Set un modified since parameter for get object(s). The option is a <code>java.time.ZonedDateTime</code> type.		<code>ZonedDateTime</code>
<code>camel.component.minio.use-version1</code>	when true, version 1 of REST API is used.	<code>false</code>	<code>Boolean</code>
<code>camel.component.minio.version-id</code>	Set specific version_ID of a object when deleting the object.		<code>String</code>

CHAPTER 100. MLLP

Both producer and consumer are supported

The MLLP component is specifically designed to handle the nuances of the MLLP protocol and provide the functionality required by Healthcare providers to communicate with other systems using the MLLP protocol.

The MLLP component provides a simple configuration URI, automated HL7 acknowledgment generation and automatic acknowledgment interrogation.

The MLLP protocol does not typically use a large number of concurrent TCP connections - a single active TCP connection is the normal case. Therefore, the MLLP component uses a simple thread-per-connection model based on standard Java Sockets. This keeps the implementation simple and eliminates the dependencies on only Camel itself.

The component supports the following:

- A Camel consumer using a TCP Server
- A Camel producer using a TCP Client

The MLLP component use **byte[]** payloads, and relies on Camel type conversion to convert **byte[]** to other types.

100.1. DEPENDENCIES

When using **mlp** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-mlp-starter</artifactId>
</dependency>
```

100.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

100.2.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

100.2.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

100.3. COMPONENT OPTIONS

The MLLP component supports 30 options, which are listed below.

Name	Description	Default	Type
autoAck (common)	Enable/Disable the automatic generation of a MLLP Acknowledgement MLLP Consumers only.	true	boolean
charsetName (common)	Sets the default charset to use.		String
configuration (common)	Sets the default configuration to use when creating MLLP endpoints.		MllpConfiguration
hl7Headers (common)	Enable/Disable the automatic generation of message headers from the HL7 Message MLLP Consumers only.	true	boolean

Name	Description	Default	Type
requireEndOfData (common)	Enable/Disable strict compliance to the MLLP standard. The MLLP standard specifies START_OF_BLOCK payload, however, some systems do not send the final END_OF_DATA byte. This setting controls whether or not the final END_OF_DATA byte is required or optional.	true	boolean
stringPayload (common)	Enable/Disable converting the payload to a String. If enabled, HL7 Payloads received from external systems will be validated converted to a String. If the charsetName property is set, that character set will be used for the conversion. If the charsetName property is not set, the value of MSH-18 will be used to determine the appropriate character set. If MSH-18 is not set, then the default ISO-8859-1 character set will be used.	true	boolean
validatePayload (common)	Enable/Disable the validation of HL7 Payloads. If enabled, HL7 Payloads received from external systems will be validated (see HL7Util.generateInvalidPayloadExceptionMessage for details on the validation). If an invalid payload is detected, a MllpInvalidMessageException (for consumers) or a MllpInvalidAcknowledgementException will be thrown.	false	boolean
acceptTimeout (consumer)	Timeout (in milliseconds) while waiting for a TCP connection. TCP Server Only.	60000	int
backlog (consumer)	The maximum queue length for incoming connection indications (a request to connect) is set to the backlog parameter. If a connection indication arrives when the queue is full, the connection is refused.	5	Integer
bindRetryInterval (consumer)	TCP Server Only - The number of milliseconds to wait between bind attempts.	5000	int
bindTimeout (consumer)	TCP Server Only - The number of milliseconds to retry binding to a server port.	30000	int

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to receive incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. If disabled, the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions by logging them at WARN or ERROR level and ignored.	true	boolean
lenientBind (consumer)	TCP Server Only - Allow the endpoint to start before the TCP ServerSocket is bound. In some environments, it may be desirable to allow the endpoint to start before the TCP ServerSocket is bound.	false	boolean
maxConcurrentConsumers (consumer)	The maximum number of concurrent MLLP Consumer connections that will be allowed. If a new connection is received and the maximum is number are already established, the new connection will be reset immediately.	5	int
reuseAddress (consumer)	Enable/disable the SO_REUSEADDR socket option.	false	Boolean
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut	InOut	ExchangePattern
connectTimeout (producer)	Timeout (in milliseconds) for establishing for a TCP connection TCP Client only.	30000	int

Name	Description	Default	Type
idleTimeoutStrategy (producer)	decide what action to take when idle timeout occurs. Possible values are : RESET: set SO_LINGER to 0 and reset the socket CLOSE: close the socket gracefully default is RESET. Enum values: ● RESET ● CLOSE	RESET	MllpIdleTimeoutStrategy
keepAlive (producer)	Enable/disable the SO_KEEPALIVE socket option.	true	Boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
tcpNoDelay (producer)	Enable/disable the TCP_NODELAY socket option.	true	Boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
defaultCharset (advanced)	Set the default character set to use for byte to/from String conversions.	ISO-8859-1	String
logPhi (advanced)	Whether to log PHI.	true	Boolean
logPhiMaxBytes (advanced)	Set the maximum number of bytes of PHI that will be logged in a log entry.	5120	Integer
readTimeout (advanced)	The SO_TIMEOUT value (in milliseconds) used after the start of an MLLP frame has been received.	5000	int

Name	Description	Default	Type
receiveBufferSize (advanced)	Sets the SO_RCVBUF option to the specified value (in bytes).	8192	Integer
receiveTimeout (advanced)	The SO_TIMEOUT value (in milliseconds) used when waiting for the start of an MLLP frame.	15000	int
sendBufferSize (advanced)	Sets the SO_SNDBUF option to the specified value (in bytes).	8192	Integer
idleTimeout (tcp)	The approximate idle time allowed before the Client TCP Connection will be reset. A null value or a value less than or equal to zero will disable the idle timeout.		Integer

100.4. ENDPOINT OPTIONS

The MLLP endpoint is configured using URI syntax:

```
mlp:hostname:port
```

with the following path and query parameters:

100.4.1. Path Parameters (2 parameters)

Name	Description	Default	Type
hostname (common)	Required Hostname or IP for connection for the TCP connection. The default value is null, which means any local IP address.		String
port (common)	Required Port number for the TCP connection.		int

100.4.2. Query Parameters (26 parameters)

Name	Description	Default	Type
autoAck (common)	Enable/Disable the automatic generation of a MLLP Acknowledgement MLLP Consumers only.	true	boolean
charsetName (common)	Sets the default charset to use.		String

Name	Description	Default	Type
hl7Headers (common)	Enable/Disable the automatic generation of message headers from the HL7 Message MLLP Consumers only.	true	boolean
requireEndOfData (common)	Enable/Disable strict compliance to the MLLP standard. The MLLP standard specifies START_OF_BLOCK payload END_OF_BLOCK END_OF_DATA, however, some systems do not send the final END_OF_DATA byte. This setting controls whether or not the final END_OF_DATA byte is required or optional.	true	boolean
stringPayload (common)	Enable/Disable converting the payload to a String. If enabled, HL7 Payloads received from external systems will be validated converted to a String. If the charsetName property is set, that character set will be used for the conversion. If the charsetName property is not set, the value of MSH-18 will be used to determine the appropriate character set. If MSH-18 is not set, then the default ISO-8859-1 character set will be used.	true	boolean
validatePayload (common)	Enable/Disable the validation of HL7 Payloads. If enabled, HL7 Payloads received from external systems will be validated (see <code>HL7Util.generateInvalidPayloadExceptionMessage</code> for details on the validation). If an invalid payload is detected, a <code>MllpInvalidMessageException</code> (for consumers) or a <code>MllpInvalidAcknowledgementException</code> will be thrown.	false	boolean
acceptTimeout (consumer)	Timeout (in milliseconds) while waiting for a TCP connection. TCP Server Only.	60000	int
backlog (consumer)	The maximum queue length for incoming connection indications (a request to connect) is set to the backlog parameter. If a connection indication arrives when the queue is full, the connection is refused.	5	Integer
bindRetryInterval (consumer)	TCP Server Only - The number of milliseconds to wait between bind attempts.	5000	int
bindTimeout (consumer)	TCP Server Only - The number of milliseconds to retry binding to a server port.	30000	int

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to receive incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. If disabled, the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions by logging them at WARN or ERROR level and ignored.	true	boolean
lenientBind (consumer)	TCP Server Only - Allow the endpoint to start before the TCP ServerSocket is bound. In some environments, it may be desirable to allow the endpoint to start before the TCP ServerSocket is bound.	false	boolean
maxConcurrentConsumers (consumer)	The maximum number of concurrent MLLP Consumer connections that will be allowed. If a new connection is received and the maximum is number are already established, the new connection will be reset immediately.	5	int
reuseAddress (consumer)	Enable/disable the SO_REUSEADDR socket option.	false	Boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut	InOut	ExchangePattern
connectTimeout (producer)	Timeout (in milliseconds) for establishing for a TCP connection TCP Client only.	30000	int

Name	Description	Default	Type
idleTimeoutStrategy (producer)	decide what action to take when idle timeout occurs. Possible values are : RESET: set SO_LINGER to 0 and reset the socket CLOSE: close the socket gracefully default is RESET. Enum values: ● RESET ● CLOSE	RESET	MllpIdleTimeoutStrategy
keepAlive (producer)	Enable/disable the SO_KEEPALIVE socket option.	true	Boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
tcpNoDelay (producer)	Enable/disable the TCP_NODELAY socket option.	true	Boolean
readTimeout (advanced)	The SO_TIMEOUT value (in milliseconds) used after the start of an MLLP frame has been received.	5000	int
receiveBufferSize (advanced)	Sets the SO_RCVBUF option to the specified value (in bytes).	8192	Integer
receiveTimeout (advanced)	The SO_TIMEOUT value (in milliseconds) used when waiting for the start of an MLLP frame.	15000	int
sendBufferSize (advanced)	Sets the SO_SNDBUF option to the specified value (in bytes).	8192	Integer
idleTimeout (tcp)	The approximate idle time allowed before the Client TCP Connection will be reset. A null value or a value less than or equal to zero will disable the idle timeout.		Integer

100.5. MLLP CONSUMER

The MLLP Consumer supports receiving MLLP-framed messages and sending HL7 Acknowledgements.

The MLLP Consumer can automatically generate the HL7 Acknowledgement (HL7 Application Acknowledgements only - AA, AE and AR), or the acknowledgement can be specified using the CamelMllpAcknowledgement exchange property. Additionally, the type of acknowledgement that will be generated can be controlled by setting the CamelMllpAcknowledgementType exchange property. The MLLP Consumer can read messages without sending any HL7 Acknowledgement if the automatic acknowledgement is disabled and exchange pattern is InOnly.

100.5.1. Message Headers

The MLLP Consumer adds these headers on the Camel message:

Key	Description
CamelMllpLocalAddress	The local TCP Address of the Socket
CamelMllpRemoteAddress	The local TCP Address of the Socket
CamelMllpSendingApplication	MSH-3 value
CamelMllpSendingFacility	MSH-4 value
CamelMllpReceivingApplication	MSH-5 value
CamelMllpReceivingFacility	MSH-6 value
CamelMllpTimestamp	MSH-7 value
CamelMllpSecurity	MSH-8 value
CamelMllpMessageType	MSH-9 value
CamelMllpEventType	MSH-9-1 value
CamelMllpTriggerEvent	MSH-9-2 value
CamelMllpMessageControllId	MSH-10 value
CamelMllpProcessingId	MSH-11 value
CamelMllpVersionId	MSH-12 value
CamelMllpCharset	MSH-18 value

All headers are String types. If a header value is missing, its value is null.

100.5.2. Exchange Properties

The type of acknowledgment the MLLP Consumer generates and state of the TCP Socket can be controlled by these properties on the Camel exchange:

Key	Type	Description
CamelMllpAcknowledgement	byte[]	If present, this property will be sent to client as the MLLP Acknowledgement
CamelMllpAcknowledgementString	String	If present and CamelMllpAcknowledgement is not present, this property will be sent to client as the MLLP Acknowledgement
CamelMllpAcknowledgementMsaText	String	If neither CamelMllpAcknowledgement or CamelMllpAcknowledgementString are present and autoAck is true, this property can be used to specify the contents of MSA-3 in the generated HL7 acknowledgement
CamelMllpAcknowledgementType	String	If neither CamelMllpAcknowledgement or CamelMllpAcknowledgementString are present and autoAck is true, this property can be used to specify the HL7 acknowledgement type (i.e. AA, AE, AR)
CamelMllpAutoAcknowledge	Boolean	Overrides the autoAck query parameter
CamelMllpCloseConnectionBeforeSend	Boolean	If true, the Socket will be closed before sending data
CamelMllpResetConnectionBeforeSend	Boolean	If true, the Socket will be reset before sending data
CamelMllpCloseConnectionAfterSend	Boolean	If true, the Socket will be closed immediately after sending data
CamelMllpResetConnectionAfterSend	Boolean	If true, the Socket will be reset immediately after sending any data

100.6. MLLP PRODUCER

The MLLP Producer supports sending MLLP-framed messages and receiving HL7 Acknowledgements. The MLLP Producer interrogates the HL7 Acknowledgments and raises exceptions if a negative acknowledgement is received. The received acknowledgement is interrogated and an exception is raised in the event of a negative acknowledgement. The MLLP Producer can ignore acknowledgements when configured with InOnly exchange pattern.

100.6.1. Message Headers

The MLLP Producer adds these headers on the Camel message:

Key	Description
CamelMllpLocalAddress	The local TCP Address of the Socket
CamelMllpRemoteAddress	The remote TCP Address of the Socket
CamelMllpAcknowledgement	The HL7 Acknowledgment byte[] received
CamelMllpAcknowledgementString	The HL7 Acknowledgment received, converted to a String

100.6.2. Exchange Properties

The state of the TCP Socket can be controlled by these properties on the Camel exchange:

Key	Type	Description
CamelMllpCloseConnectionBeforeSend	Boolean	If true, the Socket will be closed before sending data
CamelMllpResetConnectionBeforeSend	Boolean	If true, the Socket will be reset before sending data
CamelMllpCloseConnectionAfterSend	Boolean	If true, the Socket will be closed immediately after sending data
CamelMllpResetConnectionAfterSend	Boolean	If true, the Socket will be reset immediately after sending any data

100.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 31 options, which are listed below.

Name	Description	Default	Type
camel.component.mllp.accept-timeout	Timeout (in milliseconds) while waiting for a TCP connection TCP Server Only.	60000	Integer
camel.component.mllp.auto-ack	Enable/Disable the automatic generation of a MLLP Acknowledgement MLLP Consumers only.	true	Boolean

Name	Description	Default	Type
camel.component.mllp.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.mllp.backlog	The maximum queue length for incoming connection indications (a request to connect) is set to the backlog parameter. If a connection indication arrives when the queue is full, the connection is refused.	5	Integer
camel.component.mllp.bind-retry-interval	TCP Server Only - The number of milliseconds to wait between bind attempts.	5000	Integer
camel.component.mllp.bind-timeout	TCP Server Only - The number of milliseconds to retry binding to a server port.	30000	Integer
camel.component.mllp.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to receive incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. If disabled, the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions by logging them at WARN or ERROR level and ignored.	true	Boolean
camel.component.mllp.charset-name	Sets the default charset to use.		String
camel.component.mllp.configuration	Sets the default configuration to use when creating MLLP endpoints. The option is a org.apache.camel.component.mllp.MllpConfiguration type.		MllpConfiguration
camel.component.mllp.connect-timeout	Timeout (in milliseconds) for establishing for a TCP connection TCP Client only.	30000	Integer
camel.component.mllp.default-charset	Set the default character set to use for byte to/from String conversions.	ISO-8859-1	String

Name	Description	Default	Type
camel.component.mllp.enabled	Whether to enable auto configuration of the mllp component. This is enabled by default.		Boolean
camel.component.mllp.exchange-pattern	Sets the exchange pattern when the consumer creates an exchange.		ExchangePattern
camel.component.mllp.hl7-headers	Enable/Disable the automatic generation of message headers from the HL7 Message MLLP Consumers only.	true	Boolean
camel.component.mllp.idle-timeout	The approximate idle time allowed before the Client TCP Connection will be reset. A null value or a value less than or equal to zero will disable the idle timeout.		Integer
camel.component.mllp.idle-timeout-strategy	decide what action to take when idle timeout occurs. Possible values are : RESET: set SO_LINGER to 0 and reset the socket CLOSE: close the socket gracefully default is RESET.		MllpIdleTimeoutStrategy
camel.component.mllp.keep-alive	Enable/disable the SO_KEEPALIVE socket option.	true	Boolean
camel.component.mllp.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.mllp.lenient-bind	TCP Server Only - Allow the endpoint to start before the TCP ServerSocket is bound. In some environments, it may be desirable to allow the endpoint to start before the TCP ServerSocket is bound.	false	Boolean
camel.component.mllp.log-phi	Whether to log PHI.	true	Boolean
camel.component.mllp.log-phi-max-bytes	Set the maximum number of bytes of PHI that will be logged in a log entry.	5120	Integer

Name	Description	Default	Type
camel.component.mllp.max-concurrent-consumers	The maximum number of concurrent MLLP Consumer connections that will be allowed. If a new connection is received and the maximum is number are already established, the new connection will be reset immediately.	5	Integer
camel.component.mllp.read-timeout	The SO_TIMEOUT value (in milliseconds) used after the start of an MLLP frame has been received.	5000	Integer
camel.component.mllp.receive-buffer-size	Sets the SO_RCVBUF option to the specified value (in bytes).	8192	Integer
camel.component.mllp.receive-timeout	The SO_TIMEOUT value (in milliseconds) used when waiting for the start of an MLLP frame.	15000	Integer
camel.component.mllp.require-end-of-data	Enable/Disable strict compliance to the MLLP standard. The MLLP standard specifies START_OF_BLOCK payload, however, some systems do not send the final END_OF_DATA byte. This setting controls whether or not the final END_OF_DATA byte is required or optional.	true	Boolean
camel.component.mllp.reuse-address	Enable/disable the SO_REUSEADDR socket option.	false	Boolean
camel.component.mllp.send-buffer-size	Sets the SO_SNDBUF option to the specified value (in bytes).	8192	Integer
camel.component.mllp.string-payload	Enable/Disable converting the payload to a String. If enabled, HL7 Payloads received from external systems will be validated converted to a String. If the charsetName property is set, that character set will be used for the conversion. If the charsetName property is not set, the value of MSH-18 will be used to determine the appropriate character set. If MSH-18 is not set, then the default ISO-8859-1 character set will be used.	true	Boolean
camel.component.mllp.tcp-no-delay	Enable/disable the TCP_NODELAY socket option.	true	Boolean

Name	Description	Default	Type
camel.component .mllp.validate- payload	Enable/Disable the validation of HL7 Payloads If enabled, HL7 Payloads received from external systems will be validated (see <code>HL7Util.generateInvalidPayloadExceptionMessage</code> for details on the validation). If an invalid payload is detected, a <code>MllpInvalidMessageException</code> (for consumers) or a <code>MllpInvalidAcknowledgementException</code> will be thrown.	false	Boolean

CHAPTER 101. MOCK

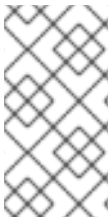
Only producer is supported

Testing of distributed and asynchronous processing is notoriously difficult. The [Mock](#), [Test](#) and [Dataset](#) endpoints work great with the Camel Testing Framework to simplify your unit and integration testing using [Enterprise Integration Patterns](#) and Camel's large range of Components together with the powerful Bean Integration.

The Mock component provides a powerful declarative testing mechanism, which is similar to [jMock](#) in that it allows declarative expectations to be created on any Mock endpoint before a test begins. Then the test is run, which typically fires messages to one or more endpoints, and finally the expectations can be asserted in a test case to ensure the system worked as expected.

This allows you to test various things like:

- The correct number of messages are received on each endpoint,
- The correct payloads are received, in the right order,
- Messages arrive on an endpoint in order, using some Expression to create an order testing function,
- Messages arrive match some kind of Predicate such as that specific headers have certain values, or that messages match some predicate, such as by evaluating an [XPath](#) or [XQuery](#) Expression.



NOTE

There is also the [Test endpoint](#) which is a Mock endpoint, but which uses a second endpoint to provide the list of expected message bodies and automatically sets up the Mock endpoint assertions. In other words, it's a Mock endpoint that automatically sets up its assertions from some sample messages in a File or [database](#), for example.



NOTE

Mock endpoints keep received Exchanges in memory indefinitely.

Remember that Mock is designed for testing. When you add Mock endpoints to a route, each Exchange sent to the endpoint will be stored (to allow for later validation) in memory until explicitly reset or the JVM is restarted. If you are sending high volume and/or large messages, this may cause excessive memory use. If your goal is to test deployable routes inline, consider using `NotifyBuilder` or `AdviceWith` in your tests instead of adding Mock endpoints to routes directly. There are two new options `retainFirst`, and `retainLast` that can be used to limit the number of messages the Mock endpoints keep in memory.

101.1. DEPENDENCIES

When using **mock** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-mock-starter</artifactId>
</dependency>
```

101.2. URI FORMAT

```
mock:someName[?options]
```

Where **someName** can be any string that uniquely identifies the endpoint.

101.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

101.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

101.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

101.4. COMPONENT OPTIONS

The Mock component supports 4 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
log (producer)	To turn on logging when the mock receives an incoming message. This will log only one time at INFO level for the incoming message. For more detailed logging then set the logger to DEBUG level for the org.apache.camel.component.mock.MockEndpoint class.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
exchangeFormatter (advanced)	Autowired Sets a custom ExchangeFormatter to convert the Exchange to a String suitable for logging. If not specified, we default to DefaultExchangeFormatter.		ExchangeFormatter

101.5. ENDPOINT OPTIONS

The Mock endpoint is configured using URI syntax:

```
mock:name
```

with the following path and query parameters:

101.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (producer)	Required Name of mock endpoint.		String

101.5.2. Query Parameters (12 parameters)

Name	Description	Default	Type
assertPeriod (producer)	Sets a grace period after which the mock endpoint will re-assert to ensure the preliminary assertion is still valid. This is used for example to assert that exactly a number of messages arrives. For example if <code>expectedMessageCount(int)</code> was set to 5, then the assertion is satisfied when 5 or more message arrives. To ensure that exactly 5 messages arrives, then you would need to wait a little period to ensure no further message arrives. This is what you can use this method for. By default this period is disabled.		long
expectedCount (producer)	Specifies the expected number of message exchanges that should be received by this endpoint. Beware: If you want to expect that 0 messages, then take extra care, as 0 matches when the tests starts, so you need to set a assert period time to let the test run for a while to make sure there are still no messages arrived; for that use <code>setAssertPeriod(long)</code> . An alternative is to use <code>NotifyBuilder</code> , and use the notifier to know when Camel is done routing some messages, before you call the <code>assertIsSatisfied()</code> method on the mocks. This allows you to not use a fixed assert period, to speedup testing times. If you want to assert that exactly n'th message arrives to this mock endpoint, then see also the <code>setAssertPeriod(long)</code> method for further details.	-1	int
failFast (producer)	Sets whether <code>assertIsSatisfied()</code> should fail fast at the first detected failed expectation while it may otherwise wait for all expected messages to arrive before performing expectations verifications. Is by default true. Set to false to use behavior as in Camel 2.x.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
log (producer)	To turn on logging when the mock receives an incoming message. This will log only one time at INFO level for the incoming message. For more detailed logging then set the logger to DEBUG level for the org.apache.camel.component.mock.MockEndpoint class.	false	boolean
reportGroup (producer)	A number that is used to turn on throughput logging based on groups of the size.		int
resultMinimumWaitTime (producer)	Sets the minimum expected amount of time (in millis) the assertIsSatisfied() will wait on a latch until it is satisfied.		long
resultWaitTime (producer)	Sets the maximum amount of time (in millis) the assertIsSatisfied() will wait on a latch until it is satisfied.		long
retainFirst (producer)	Specifies to only retain the first n'th number of received Exchanges. This is used when testing with big data, to reduce memory consumption by not storing copies of every Exchange this mock endpoint receives. Important: When using this limitation, then the getReceivedCounter() will still return the actual number of received Exchanges. For example if we have received 5000 Exchanges, and have configured to only retain the first 10 Exchanges, then the getReceivedCounter() will still return 5000 but there is only the first 10 Exchanges in the getExchanges() and getReceivedExchanges() methods. When using this method, then some of the other expectation methods is not supported, for example the expectedBodiesReceived(Object...) sets a expectation on the first number of bodies received. You can configure both setRetainFirst(int) and setRetainLast(int) methods, to limit both the first and last received.	-1	int

Name	Description	Default	Type
retainLast (producer)	Specifies to only retain the last n'th number of received Exchanges. This is used when testing with big data, to reduce memory consumption by not storing copies of every Exchange this mock endpoint receives. Important: When using this limitation, then the <code>getReceivedCounter()</code> will still return the actual number of received Exchanges. For example if we have received 5000 Exchanges, and have configured to only retain the last 20 Exchanges, then the <code>getReceivedCounter()</code> will still return 5000 but there is only the last 20 Exchanges in the <code>getExchanges()</code> and <code>getReceivedExchanges()</code> methods. When using this method, then some of the other expectation methods is not supported, for example the <code>expectedBodiesReceived(Object...)</code> sets a expectation on the first number of bodies received. You can configure both <code>setRetainFirst(int)</code> and <code>setRetainLast(int)</code> methods, to limit both the first and last received.	-1	int
sleepForEmptyTest (producer)	Allows a sleep to be specified to wait to check that this endpoint really is empty when <code>expectedMessageCount(int)</code> is called with zero.		long
copyOnExchange (producer (advanced))	Sets whether to make a deep copy of the incoming Exchange when received at this mock endpoint. Is by default true.	true	boolean

101.6. SIMPLE EXAMPLE

Here's a simple example of Mock endpoint in use. First, the endpoint is resolved on the context. Then we set an expectation, and then, after the test has run, we assert that our expectations have been met:

```
MockEndpoint resultEndpoint = context.getEndpoint("mock:foo", MockEndpoint.class);

// set expectations
resultEndpoint.expectedMessageCount(2);

// send some messages

// now lets assert that the mock:foo endpoint received 2 messages
resultEndpoint.assertIsSatisfied();
```

You typically always call the method to test that the expectations were met after running a test.

Camel will by default wait 10 seconds when the `assertIsSatisfied()` is invoked. This can be configured by setting the `setResultWaitTime(millis)` method.

101.7. USING ASSERTPERIOD

When the assertion is satisfied then Camel will stop waiting and continue from the **assertIsSatisfied** method. That means if a new message arrives on the mock endpoint, just a bit later, that arrival will not affect the outcome of the assertion. Suppose you do want to test that no new messages arrives after a period thereafter, then you can do that by setting the **setAssertPeriod** method, for example:

```
MockEndpoint resultEndpoint = context.getEndpoint("mock:foo", MockEndpoint.class);
resultEndpoint.setAssertPeriod(5000);
resultEndpoint.expectedMessageCount(2);

// send some messages

// now lets assert that the mock:foo endpoint received 2 messages
resultEndpoint.assertIsSatisfied();
```

101.8. SETTING EXPECTATIONS

You can see from the Javadoc of [MockEndpoint](#) the various helper methods you can use to set expectations. The main methods are as follows:

Method	Description
expectedMessageCount(int)	To define the expected message count on the endpoint.
expectedMinimumMessageCount(int)	To define the minimum number of expected messages on the endpoint.
expectedBodiesReceived(...)	To define the expected bodies that should be received (in order).
expectedHeaderReceived(...)	To define the expected header that should be received
expectsAscending(Expression)	To add an expectation that messages are received in order, using the given Expression to compare messages.
expectsDescending(Expression)	To add an expectation that messages are received in order, using the given Expression to compare messages.
expectsNoDuplicates(Expression)	To add an expectation that no duplicate messages are received; using an Expression to calculate a unique identifier for each message. This could be something like the JMSMessageID if using JMS, or some unique reference number within the message.

Here's another example:

```
resultEndpoint.expectedBodiesReceived("firstMessageBody", "secondMessageBody",
"thirdMessageBody");
```

101.9. ADDING EXPECTATIONS TO SPECIFIC MESSAGES

In addition, you can use the `message(int messageIndex)` method to add assertions about a specific message that is received.

For example, to add expectations of the headers or body of the first message (using zero-based indexing like `java.util.List`), you can use the following code:

```
resultEndpoint.message(0).header("foo").isEqualTo("bar");
```

There are some examples of the Mock endpoint in use in the [camel-core processor tests](#).

101.10. MOCKING EXISTING ENDPOINTS

Camel now allows you to automatically mock existing endpoints in your Camel routes.



NOTE

How it works

The endpoints are still in action. What happens differently is that a `Mock` endpoint is injected and receives the message first and then delegates the message to the target endpoint. You can view this as a kind of intercept and delegate or endpoint listener.

Suppose you have the given route below:

Route

```
@Override
protected RouteBuilder createRouteBuilder() throws Exception {
    return new RouteBuilder() {
        @Override
        public void configure() throws Exception {
            from("direct:start").routeId("start")
                .to("direct:foo").to("log:foo").to("mock:result");

            from("direct:foo").routeId("foo")
                .transform(constant("Bye World"));
        }
    };
}
```

You can then use the `adviceWith` feature in Camel to mock all the endpoints in a given route from your unit test, as shown below:

adviceWith mocking all endpoints

```
@Test
public void testAdvisedMockEndpoints() throws Exception {
    // advice the start route using the inlined AdviceWith lambda style route builder
```

```

// which has extended capabilities than the regular route builder
AdviceWith.adviceWith(context, "start", a ->
// mock all endpoints
a.mockEndpoints());

getMockEndpoint("mock:direct:start").expectedBodiesReceived("Hello World");
getMockEndpoint("mock:direct:foo").expectedBodiesReceived("Hello World");
getMockEndpoint("mock:log:foo").expectedBodiesReceived("Bye World");
getMockEndpoint("mock:result").expectedBodiesReceived("Bye World");

template.sendBody("direct:start", "Hello World");

assertMockEndpointsSatisfied();

// additional test to ensure correct endpoints in registry
assertNotNull(context.hasEndpoint("direct:start"));
assertNotNull(context.hasEndpoint("direct:foo"));
assertNotNull(context.hasEndpoint("log:foo"));
assertNotNull(context.hasEndpoint("mock:result"));
// all the endpoints was mocked
assertNotNull(context.hasEndpoint("mock:direct:start"));
assertNotNull(context.hasEndpoint("mock:direct:foo"));
assertNotNull(context.hasEndpoint("mock:log:foo"));
}

```

Notice that the mock endpoints is given the URI **mock:<endpoint>**, for example **mock:direct:foo**. Camel logs at **INFO** level the endpoints being mocked:

```
INFO  Advised endpoint [direct://foo] with mock endpoint [mock:direct:foo]
```



NOTE

Mocked endpoints are without parameters

Endpoints which are mocked will have their parameters stripped off. For example the endpoint **log:foo?showAll=true** will be mocked to the following endpoint **mock:log:foo**. Notice the parameters have been removed.

Its also possible to only mock certain endpoints using a pattern. For example to mock all **log** endpoints you do as shown:

adviceWith mocking only log endpoints using a pattern

```

@Test
public void testAdvisedMockEndpointsWithPattern() throws Exception {
// advice the start route using the inlined AdviceWith lambda style route builder
// which has extended capabilities than the regular route builder
AdviceWith.adviceWith(context, "start", a ->
// mock only log endpoints
a.mockEndpoints("log*"));

// now we can refer to log:foo as a mock and set our expectations
getMockEndpoint("mock:log:foo").expectedBodiesReceived("Bye World");

getMockEndpoint("mock:result").expectedBodiesReceived("Bye World");
}

```

```

template.sendBody("direct:start", "Hello World");

assertMockEndpointsSatisfied();

// additional test to ensure correct endpoints in registry
assertNotNull(context.hasEndpoint("direct:start"));
assertNotNull(context.hasEndpoint("direct:foo"));
assertNotNull(context.hasEndpoint("log:foo"));
assertNotNull(context.hasEndpoint("mock:result"));
// only the log:foo endpoint was mocked
assertNotNull(context.hasEndpoint("mock:log:foo"));
assertNull(context.hasEndpoint("mock:direct:start"));
assertNull(context.hasEndpoint("mock:direct:foo"));
}

```

The pattern supported can be a wildcard or a regular expression. See more details about this at Intercept as its the same matching function used by Camel.



NOTE

Mind that mocking endpoints causes the messages to be copied when they arrive on the mock. That means Camel will use more memory. This may not be suitable when you send in a lot of messages.

101.11. MOCKING EXISTING ENDPOINTS USING THE CAMEL-TEST COMPONENT

Instead of using the **adviceWith** to instruct Camel to mock endpoints, you can easily enable this behavior when using the **camel-test** Test Kit.

The same route can be tested as follows. Notice that we return **"*"** from the **isMockEndpoints** method, which tells Camel to mock all endpoints.

If you only want to mock all **log** endpoints you can return **"log*"** instead.

isMockEndpoints using camel-test kit

```

public class IsMockEndpointsJUnit4Test extends CamelTestSupport {

    @Override
    public String isMockEndpoints() {
        // override this method and return the pattern for which endpoints to mock.
        // use * to indicate all
        return "*";
    }

    @Test
    public void testMockAllEndpoints() throws Exception {
        // notice we have automatic mocked all endpoints and the name of the endpoints is "mock:uri"
        getMockEndpoint("mock:direct:start").expectedBodiesReceived("Hello World");
        getMockEndpoint("mock:direct:foo").expectedBodiesReceived("Hello World");
        getMockEndpoint("mock:log:foo").expectedBodiesReceived("Bye World");
    }
}

```

```

getMockEndpoint("mock:result").expectedBodiesReceived("Bye World");

template.sendBody("direct:start", "Hello World");

assertMockEndpointsSatisfied();

// additional test to ensure correct endpoints in registry
assertNotNull(context.hasEndpoint("direct:start"));
assertNotNull(context.hasEndpoint("direct:foo"));
assertNotNull(context.hasEndpoint("log:foo"));
assertNotNull(context.hasEndpoint("mock:result"));
// all the endpoints was mocked
assertNotNull(context.hasEndpoint("mock:direct:start"));
assertNotNull(context.hasEndpoint("mock:direct:foo"));
assertNotNull(context.hasEndpoint("mock:log:foo"));
}

@Override
protected RouteBuilder createRouteBuilder() throws Exception {
    return new RouteBuilder() {
        @Override
        public void configure() throws Exception {
            from("direct:start").to("direct:foo").to("log:foo").to("mock:result");

            from("direct:foo").transform(constant("Bye World"));
        }
    };
}
}

```

101.12. MOCKING EXISTING ENDPOINTS WITH XML DSL

If you do not use the **camel-test** component for unit testing (as shown above) you can use a different approach when using XML files for routes.

The solution is to create a new XML file used by the unit test and then include the intended XML file which has the route you want to test.

Suppose we have the route in the **camel-route.xml** file:

camel-route.xml

```

<!-- this camel route is in the camel-route.xml file -->
<camelContext xmlns="http://camel.apache.org/schema/spring">

    <route>
        <from uri="direct:start"/>
        <to uri="direct:foo"/>
        <to uri="log:foo"/>
        <to uri="mock:result"/>
    </route>

    <route>
        <from uri="direct:foo"/>
        <transform>
            <constant>Bye World</constant>

```

```

    </transform>
  </route>

</camelContext>

```

Then we create a new XML file as follows, where we include the **camel-route.xml** file and define a spring bean with the class **org.apache.camel.impl.InterceptSendToMockEndpointStrategy** which tells Camel to mock all endpoints:

test-camel-route.xml

```

<!-- the Camel route is defined in another XML file -->
<import resource="camel-route.xml"/>

<!-- bean which enables mocking all endpoints -->
<bean id="mockAllEndpoints"
class="org.apache.camel.component.mock.InterceptSendToMockEndpointStrategy"/>

```

Then in your unit test you load the new XML file (**test-camel-route.xml**) instead of **camel-route.xml**.

To only mock all [Log](#) endpoints you can define the pattern in the constructor for the bean:

```

<bean id="mockAllEndpoints"
class="org.apache.camel.impl.InterceptSendToMockEndpointStrategy">
  <constructor-arg index="0" value="log*" />
</bean>

```

101.13. MOCKING ENDPOINTS AND SKIP SENDING TO ORIGINAL ENDPOINT

Sometimes you want to easily mock and skip sending to a certain endpoints. So the message is detoured and send to the mock endpoint only. You can now use the **mockEndpointsAndSkip** method using **AdviceWith**. The example below will skip sending to the two endpoints **"direct:foo"**, and **"direct:bar"**.

adviceWith mock and skip sending to endpoints

```

@Test
public void testAdvisedMockEndpointsWithSkip() throws Exception {
    // advice the first route using the inlined AdviceWith route builder
    // which has extended capabilities than the regular route builder
    AdviceWith.adviceWith(context.getRouteDefinitions().get(0), context, new
AdviceWithRouteBuilder() {
        @Override
        public void configure() throws Exception {
            // mock sending to direct:foo and direct:bar and skip send to it
            mockEndpointsAndSkip("direct:foo", "direct:bar");
        }
    });

    getMockEndpoint("mock:result").expectedBodiesReceived("Hello World");
    getMockEndpoint("mock:direct:foo").expectedMessageCount(1);
    getMockEndpoint("mock:direct:bar").expectedMessageCount(1);
}

```

```

        template.sendBody("direct:start", "Hello World");

        assertMockEndpointsSatisfied();

        // the message was not send to the direct:foo route and thus not sent to
        // the seda endpoint
        SedaEndpoint seda = context.getEndpoint("seda:foo", SedaEndpoint.class);
        assertEquals(0, seda.getCurrentQueueSize());
    }

```

The same example using the Test Kit

isMockEndpointsAndSkip using camel-test kit

```

public class IsMockEndpointsAndSkipJUnit4Test extends CamelTestSupport {

    @Override
    public String isMockEndpointsAndSkip() {
        // override this method and return the pattern for which endpoints to mock,
        // and skip sending to the original endpoint.
        return "direct:foo";
    }

    @Test
    public void testMockEndpointAndSkip() throws Exception {
        // notice we have automatic mocked the direct:foo endpoints and the name of the endpoints is
        "mock:uri"
        getMockEndpoint("mock:result").expectedBodiesReceived("Hello World");
        getMockEndpoint("mock:direct:foo").expectedMessageCount(1);

        template.sendBody("direct:start", "Hello World");

        assertMockEndpointsSatisfied();

        // the message was not send to the direct:foo route and thus not sent to the seda endpoint
        SedaEndpoint seda = context.getEndpoint("seda:foo", SedaEndpoint.class);
        assertEquals(0, seda.getCurrentQueueSize());
    }

    @Override
    protected RouteBuilder createRouteBuilder() throws Exception {
        return new RouteBuilder() {
            @Override
            public void configure() throws Exception {
                from("direct:start").to("direct:foo").to("mock:result");

                from("direct:foo").transform(constant("Bye World")).to("seda:foo");
            }
        };
    }
}

```

101.14. LIMITING THE NUMBER OF MESSAGES TO KEEP

The [Mock](#) endpoints will by default keep a copy of every Exchange that it received. So if you test with a

lot of messages, then it will consume memory.

We have introduced two options **retainFirst** and **retainLast** that can be used to specify to only keep N'th of the first and/or last Exchanges.

For example in the code below, we only want to retain a copy of the first 5 and last 5 Exchanges the mock receives.

```
MockEndpoint mock = getMockEndpoint("mock:data");
mock.setRetainFirst(5);
mock.setRetainLast(5);
mock.expectedMessageCount(2000);

mock.assertIsSatisfied();
```

Using this has some limitations. The **getExchanges()** and **getReceivedExchanges()** methods on the **MockEndpoint** will return only the retained copies of the Exchanges. So in the example above, the list will contain 10 Exchanges; the first five, and the last five.

The **retainFirst** and **retainLast** options also have limitations on which expectation methods you can use. For example the **expectedXXX** methods that work on message bodies, headers, etc. will only operate on the retained messages. In the example above they can test only the expectations on the 10 retained messages.

101.15. TESTING WITH ARRIVAL TIMES

The **Mock** endpoint stores the arrival time of the message as a property on the Exchange

```
Date time = exchange.getProperty(Exchange.RECEIVED_TIMESTAMP, Date.class);
```

You can use this information to know when the message arrived on the mock. But it also provides foundation to know the time interval between the previous and next message arrived on the mock. You can use this to set expectations using the **arrives** DSL on the Mock endpoint.

For example to say that the first message should arrive between 0-2 seconds before the next you can do:

```
mock.message(0).arrives().noLaterThan(2).seconds().beforeNext();
```

You can also define this as that 2nd message (0 index based) should arrive no later than 0-2 seconds after the previous:

```
mock.message(1).arrives().noLaterThan(2).seconds().afterPrevious();
```

You can also use between to set a lower bound. For example suppose that it should be between 1-4 seconds:

```
mock.message(1).arrives().between(1, 4).seconds().afterPrevious();
```

You can also set the expectation on all messages, for example to say that the gap between them should be at most 1 second:

```
mock.allMessages().arrives().noLaterThan(1).seconds().beforeNext();
```



NOTE

Time units

In the example above we use **seconds** as the time unit, but Camel offers **milliseconds**, and **minutes** as well.

101.16. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.mock.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.mock.enabled</code>	Whether to enable auto configuration of the mock component. This is enabled by default.		Boolean
<code>camel.component.mock.exchange-formatter</code>	Sets a custom ExchangeFormatter to convert the Exchange to a String suitable for logging. If not specified, we default to DefaultExchangeFormatter. The option is a <code>org.apache.camel.spi.ExchangeFormatter</code> type.		ExchangeFormatter
<code>camel.component.mock.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.mock.log</code>	To turn on logging when the mock receives an incoming message. This will log only one time at INFO level for the incoming message. For more detailed logging then set the logger to DEBUG level for the <code>org.apache.camel.component.mock.MockEndpoint</code> class.	false	Boolean

CHAPTER 102. MONGODB

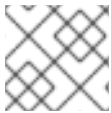
Both producer and consumer are supported

According to Wikipedia: "NoSQL is a movement promoting a loosely defined class of non-relational data stores that break with a long history of relational databases and ACID guarantees." NoSQL solutions have grown in popularity in the last few years, and major extremely-used sites and services such as Facebook, LinkedIn, Twitter, etc. are known to use them extensively to achieve scalability and agility.

Basically, NoSQL solutions differ from traditional RDBMS (Relational Database Management Systems) in that they don't use SQL as their query language and generally don't offer ACID-like transactional behaviour nor relational data. Instead, they are designed around the concept of flexible data structures and schemas (meaning that the traditional concept of a database table with a fixed schema is dropped), extreme scalability on commodity hardware and blazing-fast processing.

MongoDB is a very popular NoSQL solution and the camel-mongodb component integrates Camel with MongoDB allowing you to interact with MongoDB collections both as a producer (performing operations on the collection) and as a consumer (consuming documents from a MongoDB collection).

MongoDB revolves around the concepts of documents (not as is office documents, but rather hierarchical data defined in JSON/BSON) and collections. This component page will assume you are familiar with them. Otherwise, visit <http://www.mongodb.org/>.



NOTE

The MongoDB Camel component uses Mongo Java Driver 4.x.

102.1. DEPENDENCIES

When using **mongodb** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-mongodb-starter</artifactId>
</dependency>
```

102.2. URI FORMAT

```
mongodb:connectionBean?
database=databaseName&collection=collectionName&operation=operationName[&moreOptions...]
```

102.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

102.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

102.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

102.4. COMPONENT OPTIONS

The MongoDB component supports 4 options, which are listed below.

Name	Description	Default	Type
mongoConnection (common)	Autowired Shared client used for connection. All endpoints generated from the component will share this connection client.		MongoClient

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

102.5. ENDPOINT OPTIONS

The MongoDB endpoint is configured using URI syntax:

```
mongodb:connectionBean
```

with the following path and query parameters:

102.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
connectionBean (common)	Required Sets the connection bean reference used to lookup a client for connecting to a database.		String

102.5.2. Query Parameters (27 parameters)

Name	Description	Default	Type
collection (common)	Sets the name of the MongoDB collection to bind to this endpoint.		String
collectionIndex (common)	Sets the collection index (JSON FORMAT : <code>\{ field1 : order1, field2 : order2\}</code>).		String
createCollection (common)	Create collection during initialisation if it doesn't exist. Default is true.	true	boolean
database (common)	Sets the name of the MongoDB database to target.		String
hosts (common)	Host address of mongodb server in host:port format. It's possible also use more than one address, as comma separated list of hosts: host1:port1,host2:port2. If hosts parameter is specified, provided connectionBean is ignored.		String
mongoConnection (common)	Sets the connection bean used as a client for connecting to a database.		MongoClient

Name	Description	Default	Type
operation (common)	Sets the operation this endpoint will execute against MongoDB. Enum values: ● findById ● findOneByQuery ● findAll ● findDistinct ● insert ● save ● update ● remove ● bulkWrite ● aggregate ● getDbStats ● getColStats ● count ● command		MongoDbOperation
outputType (common)	Convert the output of the producer to the selected type : DocumentList Document or Mongolterable. DocumentList or Mongolterable applies to findAll and aggregate. Document applies to all other operations. Enum values: ● DocumentList ● Document ● Mongolterable		MongoDbOutputType

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
consumerType (consumer)	Consumer type.		String
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code> ● <code>InOptionalOut</code>		<code>ExchangePattern</code>
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
cursorRegenerationDelay (advanced)	MongoDB tailable cursors will block until new data arrives. If no new data is inserted, after some time the cursor will be automatically freed and closed by the MongoDB server. The client is expected to regenerate the cursor if needed. This value specifies the time to wait before attempting to fetch a new cursor, and if the attempt fails, how long before the next attempt is made. Default value is 1000ms.	1000	long
dynamicity (advanced)	Sets whether this endpoint will attempt to dynamically resolve the target database and collection from the incoming Exchange properties. Can be used to override at runtime the database and collection specified on the otherwise static endpoint URI. It is disabled by default to boost performance. Enabling it will take a minimal performance hit.	false	boolean
readPreference (advanced)	Configure how MongoDB clients route read operations to the members of a replica set. Possible values are PRIMARY, PRIMARY_PREFERRED, SECONDARY, SECONDARY_PREFERRED or NEAREST. Enum values: ● PRIMARY ● PRIMARY_PREFERRED ● SECONDARY ● SECONDARY_PREFERRED ● NEAREST	PRIMARY	String

Name	Description	Default	Type
writeConcern (advanced)	Configure the connection bean with the level of acknowledgment requested from MongoDB for write operations to a standalone mongod, replicaset or cluster. Possible values are ACKNOWLEDGED, W1, W2, W3, UNACKNOWLEDGED, JOURNALLED or MAJORITY. Enum values: • ACKNOWLEDGED • W1 • W2 • W3 • UNACKNOWLEDGED • JOURNALLED • MAJORITY	ACKNOWLEDGED	String
writeResultAsHeader (advanced)	In write operations, it determines whether instead of returning WriteResult as the body of the OUT message, we transfer the IN message to the OUT and attach the WriteResult as a header.	false	boolean
streamFilter (changeStream)	Filter condition for change streams consumer.		String
password (security)	User password for mongodb connection.		String
username (security)	Username for mongodb connection.		String
persistentId (tail)	One tail tracking collection can host many trackers for several tailable consumers. To keep them separate, each tracker should have its own unique persistentId.		String
persistentTailTracking (tail)	Enable persistent tail tracking, which is a mechanism to keep track of the last consumed message across system restarts. The next time the system is up, the endpoint will recover the cursor from the point where it last stopped slurping records.	false	boolean

Name	Description	Default	Type
tailTrackCollection (tail)	Collection where tail tracking information will be persisted. If not specified, <code>MongoDbTailTrackingConfig#DEFAULT_COLLECTION</code> will be used by default.		String
tailTrackDb (tail)	Indicates what database the tail tracking mechanism will persist to. If not specified, the current database will be picked by default. Dynamicity will not be taken into account even if enabled, i.e. the tail tracking database will not vary past endpoint initialisation.		String
tailTrackField (tail)	Field where the last tracked value will be placed. If not specified, <code>MongoDbTailTrackingConfig#DEFAULT_FIELD</code> will be used by default.		String
tailTrackIncreasingField (tail)	Correlation field in the incoming record which is of increasing nature and will be used to position the tailing cursor every time it is generated. The cursor will be (re)created with a query of type: <code>tailTrackIncreasingField greater than lastValue</code> (possibly recovered from persistent tail tracking). Can be of type Integer, Date, String, etc. NOTE: No support for dot notation at the current time, so the field should be at the top level of the document.		String

102.6. CONFIGURATION OF DATABASE IN SPRING XML

The following Spring XML creates a bean defining the connection to a MongoDB instance.

Since mongo java driver 3, the `WriteConcern` and `readPreference` options are not dynamically modifiable. They are defined in the `mongoClient` object

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:mongo="http://www.springframework.org/schema/data/mongo"
xsi:schemaLocation="http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/data/mongo
http://www.springframework.org/schema/data/mongo/spring-mongo.xsd
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

  <mongo:mongo-client id="mongoBean" host="${mongo.url}" port="${mongo.port}"
credentials="${mongo.user}:${mongo.pass}@${mongo.dbname}">
    <mongo:client-options write-concern="NORMAL" />
  </mongo:mongo-client>
```

</beans>

102.7. SAMPLE ROUTE

The following route defined in Spring XML executes the operation **getDbStats** on a collection.

Get DB stats for specified collection

```
<route>
  <from uri="direct:start" />
  <!-- using bean 'mongoBean' defined above -->
  <to uri="mongodb:mongoBean?
database=${mongodb.database}&collection=${mongodb.collection}&operation=getDbStats"
/>
  <to uri="direct:result" />
</route>
```

102.8. MONGODB OPERATIONS - PRODUCER ENDPOINTS

102.8.1. Query operations

102.8.1.1. findById

This operation retrieves only one element from the collection whose `_id` field matches the content of the IN message body. The incoming object can be anything that has an equivalent to a **Bson** type. See <http://bsonspec.org/spec.html> and <http://www.mongodb.org/display/DOCS/Java+Types>.

```
from("direct:findById")
  .to("mongodb:myDb?database=flights&collection=tickets&operation=findById")
  .to("mock:resultFindById");
```

Please, note that the default `_id` is treated by Mongo as and **ObjectId** type, so you may need to convert it properly.

```
from("direct:findById")
  .convertBodyTo(ObjectId.class)
  .to("mongodb:myDb?database=flights&collection=tickets&operation=findById")
  .to("mock:resultFindById");
```



NOTE

Supports optional parameters

This operation supports projection operators. See [Specifying a fields filter \(projection\)](#).

102.8.1.2. findOneByQuery

Retrieve the first element from a collection matching a MongoDB query selector. **If the CamelMongoDbCriteria header is set, then its value is used as the query selector**. If the **CamelMongoDbCriteria** header is *null*, then the IN message body is used as the query selector. In both cases, the query selector should be of type **Bson** or convertible to **Bson** (for instance, a JSON string or **HashMap**). See Type conversions for more info.

Create query selectors using the **Filters** provided by the MongoDB Driver.

102.8.1.3. Example without a query selector (returns the first document in a collection)

```
from("direct:findOneByQuery")
  .to("mongodb:myDb?database=flights&collection=tickets&operation=findOneByQuery")
  .to("mock:resultFindOneByQuery");
```

102.8.1.4. Example with a query selector (returns the first matching document in a collection):

```
from("direct:findOneByQuery")
  .setHeader(MongoDbConstants.CRITERIA, constant(Filters.eq("name", "Raul Kripalani")))
  .to("mongodb:myDb?database=flights&collection=tickets&operation=findOneByQuery")
  .to("mock:resultFindOneByQuery");
```



NOTE

Supports optional parameters

This operation supports projection operators and sort clauses. See [Specifying a fields filter \(projection\)](#), Specifying a sort clause.

102.8.1.5. findAll

The **findAll** operation returns all documents matching a query, or none at all, in which case all documents contained in the collection are returned. **The query object is extracted CamelMongoDbCriteria header.** if the CamelMongoDbCriteria header is null the query object is extracted message body, i.e. it should be of type **Bson** or convertible to **Bson**. It can be a JSON String or a Hashmap. See Type conversions for more info.

102.8.1.5.1. Example without a query selector (returns all documents in a collection)

```
from("direct:findAll")
  .to("mongodb:myDb?database=flights&collection=tickets&operation=findAll")
  .to("mock:resultFindAll");
```

102.8.1.5.2. Example with a query selector (returns all matching documents in a collection)

```
from("direct:findAll")
  .setHeader(MongoDbConstants.CRITERIA, Filters.eq("name", "Raul Kripalani"))
  .to("mongodb:myDb?database=flights&collection=tickets&operation=findAll")
  .to("mock:resultFindAll");
```

Paging and efficient retrieval is supported via the following headers:

Header key	Quick constant	Description (extracted from MongoDB API doc)	Expected type
------------	----------------	--	---------------

Header key	Quick constant	Description (extracted from MongoDB API doc)	Expected type
CamelMongoDb NumToSkip	MongoDbConstants.NUM_TO_SKIP	Discards a given number of elements at the beginning of the cursor.	int/Integer
CamelMongoDb Limit	MongoDbConstants.LIMIT	Limits the number of elements returned.	int/Integer
CamelMongoDb BatchSize	MongoDbConstants.BATCH_SIZE	Limits the number of elements returned in one batch. A cursor typically fetches a batch of result objects and store them locally. If batchSize is positive, it represents the size of each batch of objects retrieved. It can be adjusted to optimize performance and limit data transfer. If batchSize is negative, it will limit of number objects returned, that fit within the max batch size limit (usually 4MB), and cursor will be closed. For example if batchSize is -10, then the server will return a maximum of 10 documents and as many as can fit in 4MB, then close the cursor. Note that this feature is different from limit() in that documents must fit within a maximum size, and it removes the need to send a request to close the cursor server-side. The batch size can be changed even after a cursor is iterated, in which case the setting will apply on the next batch retrieval.	int/Integer
CamelMongoDb AllowDiskUse	MongoDbConstants.ALLOW_DISK_USE	Sets allowDiskUse MongoDB flag. This is supported since MongoDB Server 4.3.1. Using this header with older MongoDB Server version can cause query to fail.	boolean/Boolean

102.8.1.5.3. Example with option *outputType=Mongolterable* and batch size

```

from("direct:findAll")
    .setHeader(MongoDbConstants.BATCH_SIZE).constant(10)
    .setHeader(MongoDbConstants.CRITERIA, constant(Filters.eq("name", "Raul Kripalani")))
    .to("mongodb:myDb?
database=flights&collection=tickets&operation=findAll&outputType=Mongolterable")
    .to("mock:resultFindAll");

```

The **findAll** operation will also return the following OUT headers to enable you to iterate through result pages if you are using paging:

Header key	Quick constant	Description (extracted from MongoDB API doc)	Data type
Camel MongoDB resultTotalSize	MongoDbConstants.RESULT_TOTAL_SIZE	Number of objects matching the query. This does not take limit/skip into consideration.	int/Integer
Camel MongoDB resultPageSize	MongoDbConstants.RESULT_PAGE_SIZE	Number of objects matching the query. This does not take limit/skip into consideration.	int/Integer

**NOTE****Supports optional parameters**

This operation supports projection operators and sort clauses. See [Specifying a fields filter \(projection\)](#), Specifying a sort clause.

102.8.1.6. count

Returns the total number of objects in a collection, returning a Long as the OUT message body. The following example will count the number of records in the "dynamicCollectionName" collection. Notice how dynamicity is enabled, and as a result, the operation will not run against the "notableScientists" collection, but against the "dynamicCollectionName" collection.

```
// from("direct:count").to("mongodb:myDb?
database=tickets&collection=flights&operation=count&dynamicity=true");
Long result = template.requestBodyAndHeader("direct:count", "irrelevantBody",
MongoDbConstants.COLLECTION, "dynamicCollectionName");
assertTrue("Result is not of type Long", result instanceof Long);
```

You can provide a query **The query object is extracted CamelMongoDbCriteria header.** if the CamelMongoDbCriteria header is null the query object is extracted message body, i.e. it should be of type **Bson** or convertible to **Bson**, and operation will return the amount of documents matching this criteria.

```
Document query = ...
Long count = template.requestBodyAndHeader("direct:count", query,
MongoDbConstants.COLLECTION, "dynamicCollectionName");
```

102.8.1.7. Specifying a fields filter (projection)

Query operations will, by default, return the matching objects in their entirety (with all their fields). If your documents are large and you only require retrieving a subset of their fields, you can specify a field filter in all query operations, simply by setting the relevant **Bson** (or type convertible to **Bson**, such as a JSON String, Map, etc.) on the **CamelMongoDbFieldsProjection** header, constant shortcut:

MongoDbConstants.FIELDS_PROJECTION.

Here is an example that uses MongoDB's **Projections** to simplify the creation of Bson. It retrieves all fields except **_id** and **boringField**:

```
// route: from("direct:findAll").to("mongodb:myDb?
database=flights&collection=tickets&operation=findAll")
Bson fieldProjection = Projection.exclude("_id", "boringField");
Object result = template.requestBodyAndHeader("direct:findAll", ObjectUtils.NULL,
MongoDbConstants.FIELDS_PROJECTION, fieldProjection);
```

Here is an example that uses MongoDB's **Projections** to simplify the creation of Bson. It retrieves all fields except **_id** and **boringField**:

```
// route: from("direct:findAll").to("mongodb:myDb?
database=flights&collection=tickets&operation=findAll")
Bson fieldProjection = Projection.exclude("_id", "boringField");
Object result = template.requestBodyAndHeader("direct:findAll", ObjectUtils.NULL,
MongoDbConstants.FIELDS_PROJECTION, fieldProjection);
```

102.8.1.8. Specifying a sort clause

There is often a requirement to fetch the min/max record from a collection based on sorting by a particular field that uses MongoDB's **Sorts** to simplify the creation of Bson. It retrieves all fields except **_id** and **boringField**:

```
// route: from("direct:findAll").to("mongodb:myDb?
database=flights&collection=tickets&operation=findAll")
Bson sorts = Sorts.descending("_id");
Object result = template.requestBodyAndHeader("direct:findAll", ObjectUtils.NULL,
MongoDbConstants.SORT_BY, sorts);
```

In a Camel route the SORT_BY header can be used with the findOneByQuery operation to achieve the same result. If the FIELDS_PROJECTION header is also specified the operation will return a single field/value pair that can be passed directly to another component (for example, a parameterized MyBatis SELECT query). This example demonstrates fetching the temporally newest document from a collection and reducing the result to a single field, based on the **documentTimestamp** field:

```
.from("direct:someTriggeringEvent")
.setHeader(MongoDbConstants.SORT_BY).constant(Sorts.descending("documentTimestamp"))
.setHeader(MongoDbConstants.FIELDS_PROJECTION).constant(Projection.include("documentTime
stamp"))
.setBody().constant("{}")
.to("mongodb:myDb?database=local&collection=myDemoCollection&operation=findOneByQuery")
.to("direct:aMyBatisParameterizedSelect");
```

102.8.2. Create/update operations

102.8.2.1. insert

Inserts an new object into the MongoDB collection, taken from the IN message body. Type conversion is attempted to turn it into **Document** or a **List**.

Two modes are supported: single insert and multiple insert. For multiple insert, the endpoint will expect a List, Array or Collections of objects of any type, as long as they are - or can be converted to - **Document**. Example:

```
from("direct:insert")
  .to("mongodb:myDb?database=flights&collection=tickets&operation=insert");
```

The operation will return a **WriteResult**, and depending on the **WriteConcern** or the value of the **invokeGetLastError** option, **getLastError()** would have been called already or not. If you want to access the ultimate result of the write operation, you need to retrieve the **CommandResult** by calling **getLastError()** or **getCachedLastError()** on the **WriteResult**. Then you can verify the result by calling **CommandResult.ok()**, **CommandResult.getErrorMessage()** and/or **CommandResult.getException()**.

Note that the new object's **_id** must be unique in the collection. If you don't specify the value, MongoDB will automatically generate one for you. But if you do specify it and it is not unique, the insert operation will fail (and for Camel to notice, you will need to enable **invokeGetLastError** or set a **WriteConcern** that waits for the write result).

This is not a limitation of the component, but it is how things work in MongoDB for higher throughput. If you are using a custom **_id**, you are expected to ensure at the application level that is unique (and this is a good practice too).

OID(s) of the inserted record(s) is stored in the message header under **CamelMongoOid** key (**MongoDbConstants.OID** constant). The value stored is **org.bson.types.ObjectId** for single insert or **java.util.List<org.bson.types.ObjectId>** if multiple records have been inserted.

In MongoDB Java Driver 3.x the **insertOne** and **insertMany** operation return void. The Camel insert operation return the Document or List of Documents inserted. Note that each Documents are Updated by a new OID if need.

102.8.2.2. save

The save operation is equivalent to an *upsert* (UPdate, inSERT) operation, where the record will be updated, and if it doesn't exist, it will be inserted, all in one atomic operation. MongoDB will perform the matching based on the **_id** field.

Beware that in case of an update, the object is replaced entirely and the usage of [MongoDB's \\$modifiers](#) is not permitted. Therefore, if you want to manipulate the object if it already exists, you have two options:

1. perform a query to retrieve the entire object first along with all its fields (may not be efficient), alter it inside Camel and then save it.
2. use the update operation with [\\$modifiers](#), which will execute the update at the server-side instead. You can enable the upsert flag, in which case if an insert is required, MongoDB will apply the [\\$modifiers](#) to the filter query object and insert the result.

If the document to be saved does not contain the **_id** attribute, the operation will be an insert, and the new **_id** created will be placed in the **CamelMongoOid** header.

For example:

```
from("direct:insert")
  .to("mongodb:myDb?database=flights&collection=tickets&operation=save");

// route: from("direct:insert").to("mongodb:myDb?
// database=flights&collection=tickets&operation=save");
org.bson.Document docForSave = new org.bson.Document();
docForSave.put("key", "value");
Object result = template.requestBody("direct:insert", docForSave);
```

102.8.2.3. update

Update one or multiple records on the collection. Requires a filter query and a update rules.

You can define the filter using `MongoDBConstants.CRITERIA` header as **Bson** and define the update rules as **Bson** in Body.



NOTE

Update after enrich

While defining the filter by using `MongoDBConstants.CRITERIA` header as **Bson** to query mongodb before you do update, you should notice you need to remove it from the resulting camel exchange during aggregation if you use enrich pattern with a aggregation strategy and then apply mongodb update. If you don't remove this header during aggregation and/or redefine `MongoDBConstants.CRITERIA` header before sending camel exchange to mongodb producer endpoint, you may end up with invalid camel exchange payload while updating mongodb.

The second way Require a `List<Bson>` as the IN message body containing exactly 2 elements:

- Element 1 (index 0) ⇒ filter query ⇒ determines what objects will be affected, same as a typical query object
- Element 2 (index 1) ⇒ update rules ⇒ how matched objects will be updated. All [modifier operations](#) from MongoDB are supported.



NOTE

Multiupdates

By default, MongoDB will only update 1 object even if multiple objects match the filter query. To instruct MongoDB to update **all** matching records, set the **CamelMongoDbMultiUpdate** IN message header to **true**.

A header with key **CamelMongoDbRecordsAffected** will be returned (**MongoDbConstants.RECORDS_AFFECTED** constant) with the number of records updated (copied from **WriteResult.getN()**).

Supports the following IN message headers:

Header key	Quick constant	Description (extracted from MongoDB API doc)	Expected type
CamelMongoDb MultiUpdate	MongoDbConstants.MULTIUPDATE	If the update should be applied to all objects matching. See http://www.mongodb.org/display/DOCS/Atomic+Operations	boolean/Boolean
CamelMongoDb Upsert	MongoDbConstants.UPSERT	If the database should create the element if it does not exist	boolean/Boolean

For example, the following will update **all** records whose `filterField` field equals `true` by setting the value of the `"scientist"` field to `"Darwin"`:

```
// route: from("direct:update").to("mongodb:myDb?
database=science&collection=notableScientists&operation=update");
List<Bson> body = new ArrayList<>();
Bson filterField = Filters.eq("filterField", true);
body.add(filterField);
BsonDocument updateObj = new BsonDocument().append("$set", new BsonDocument("scientist",
new BsonString("Darwin")));
body.add(updateObj);
Object result = template.requestBodyAndHeader("direct:update", body,
MongoDbConstants.MULTIUPDATE, true);
```

```
// route: from("direct:update").to("mongodb:myDb?
database=science&collection=notableScientists&operation=update");
Maps<String, Object> headers = new HashMap<>(2);
headers.add(MongoDbConstants.MULTIUPDATE, true);
headers.add(MongoDbConstants.FIELDS_FILTER, Filters.eq("filterField", true));
String updateObj = Updates.set("scientist", "Darwin");
Object result = template.requestBodyAndHeaders("direct:update", updateObj, headers);
```

```
// route: from("direct:update").to("mongodb:myDb?
database=science&collection=notableScientists&operation=update");
String updateObj = "[{"filterField": true}, {"$set", {"scientist", "Darwin"}}]";
Object result = template.requestBodyAndHeader("direct:update", updateObj,
MongoDbConstants.MULTIUPDATE, true);
```

102.8.3. Delete operations

102.8.3.1. remove

Remove matching records from the collection. The IN message body will act as the removal filter query, and is expected to be of type **DBObject** or a type convertible to it.

The following example will remove all objects whose field `'conditionField'` equals `true`, in the `science` database, `notableScientists` collection:

```
// route: from("direct:remove").to("mongodb:myDb?
```

```

    database=science&collection=notableScientists&operation=remove");
    Bson conditionField = Filters.eq("conditionField", true);
    Object result = template.requestBody("direct:remove", conditionField);

```

A header with key **CamelMongoDbRecordsAffected** is returned (**MongoDbConstants.RECORDS_AFFECTED** constant) with type **int**, containing the number of records deleted (copied from **WriteResult.getN()**).

102.8.4. Bulk Write Operations

102.8.4.1. bulkWrite

Performs write operations in bulk with controls for order of execution. Requires a **List<WriteModel<Document>>** as the IN message body containing commands for insert, update, and delete operations.

The following example will insert a new scientist "Pierre Curie", update record with id "5" by setting the value of the "scientist" field to "Marie Curie" and delete record with id "3" :

```

// route: from("direct:bulkWrite").to("mongodb:myDb?
database=science&collection=notableScientists&operation=bulkWrite");
List<WriteModel<Document>> bulkOperations = Arrays.asList(
    new InsertOneModel<>(new Document("scientist", "Pierre Curie")),
    new UpdateOneModel<>(new Document("_id", "5"),
        new Document("$set", new Document("scientist", "Marie Curie"))),
    new DeleteOneModel<>(new Document("_id", "3")));

BulkWriteResult result = template.requestBody("direct:bulkWrite", bulkOperations,
BulkWriteResult.class);

```

By default, operations are executed in order and interrupted on the first write error without processing any remaining write operations in the list. To instruct MongoDB to continue to process remaining write operations in the list, set the **CamelMongoDbBulkOrdered** IN message header to **false**. Unordered operations are executed in parallel and this behavior is not guaranteed.

Header key	Quick constant	Description (extracted from MongoDB API doc)	Expected type
CamelMongoDbBulkOrdered	MongoDbConstants.BULK_ORDERED	Perform an ordered or unordered operation execution. Defaults to true.	boolean/Boolean

102.8.5. Other operations

102.8.5.1. aggregate

Perform an aggregation with the given pipeline contained in the body. **Aggregations could be long and heavy operations. Use with care.**

```

// route: from("direct:aggregate").to("mongodb:myDb?
database=science&collection=notableScientists&operation=aggregate");

```

```
List<Bson> aggregate = Arrays.asList(match(or(eq("scientist", "Darwin"), eq("scientist",
    group("$scientist", sum("count", 1))));
from("direct:aggregate")
    .setBody().constant(aggregate)
    .to("mongodb:myDb?database=science&collection=notableScientists&operation=aggregate")
    .to("mock:resultAggregate");
```

Supports the following IN message headers:

Header key	Quick constant	Description (extracted from MongoDB API doc)	Expected type
CamelMongoDbBatchSize	MongoDbConstants.BATCH_SIZE	Sets the number of documents to return per batch.	int/Integer
CamelMongoDbAllowDiskUse	MongoDbConstants.ALLOW_DISK_USE	Enable aggregation pipeline stages to write data to temporary files.	boolean/Boolean

By default a List of all results is returned. This can be heavy on memory depending on the size of the results. A safer alternative is to set your `outputType=MongolIterable`. The next Processor will see an iterable in the message body allowing it to step through the results one by one. Thus setting a batch size and returning an iterable allows for efficient retrieval and processing of the result.

An example would look like:

```
List<Bson> aggregate = Arrays.asList(match(or(eq("scientist", "Darwin"), eq("scientist",
    group("$scientist", sum("count", 1))));
from("direct:aggregate")
    .setHeader(MongoDbConstants.BATCH_SIZE).constant(10)
    .setBody().constant(aggregate)
    .to("mongodb:myDb?
database=science&collection=notableScientists&operation=aggregate&outputType=MongolIterable")
    .split(body())
    .streaming()
    .to("mock:resultAggregate");
```

Note that calling `.split(body())` is enough to send the entries down the route one-by-one, however it would still load all the entries into memory first. Calling `.streaming()` is thus required to load data into memory by batches.

102.8.5.2. getDbStats

Equivalent of running the `db.stats()` command in the MongoDB shell, which displays useful statistic figures about the database.

For example:

```
> db.stats();
{
  "db" : "test",
```

```

    "collections" : 7,
    "objects" : 719,
    "avgObjSize" : 59.73296244784423,
    "dataSize" : 42948,
    "storageSize" : 1000058880,
    "numExtents" : 9,
    "indexes" : 4,
    "indexSize" : 32704,
    "fileSize" : 1275068416,
    "nsSizeMB" : 16,
    "ok" : 1
  }

```

Usage example:

```

// from("direct:getDbStats").to("mongodb:myDb?
database=flights&collection=tickets&operation=getDbStats");
Object result = template.requestBody("direct:getDbStats", "irrelevantBody");
assertTrue("Result is not of type Document", result instanceof Document);

```

The operation will return a data structure similar to the one displayed in the shell, in the form of a **Document** in the OUT message body.

102.8.5.3. getColStats

Equivalent of running the **db.collection.stats()** command in the MongoDB shell, which displays useful statistic figures about the collection.

For example:

```

> db.camelTest.stats();
{
  "ns" : "test.camelTest",
  "count" : 100,
  "size" : 5792,
  "avgObjSize" : 57.92,
  "storageSize" : 20480,
  "numExtents" : 2,
  "nindexes" : 1,
  "lastExtentSize" : 16384,
  "paddingFactor" : 1,
  "flags" : 1,
  "totalIndexSize" : 8176,
  "indexSizes" : {
    "_id_" : 8176
  },
  "ok" : 1
}

```

Usage example:

```

// from("direct:getColStats").to("mongodb:myDb?
database=flights&collection=tickets&operation=getColStats");
Object result = template.requestBody("direct:getColStats", "irrelevantBody");
assertTrue("Result is not of type Document", result instanceof Document);

```

The operation will return a data structure similar to the one displayed in the shell, in the form of a **Document** in the OUT message body.

102.8.5.4. command

Run the body as a command on database. Useful for admin operation as getting host information, replication or sharding status.

Collection parameter is not use for this operation.

```
// route: from("command").to("mongodb:myDb?database=science&operation=command");
DBObject commandBody = new BasicDBObject("hostInfo", "1");
Object result = template.requestBody("direct:command", commandBody);
```

102.8.6. Dynamic operations

An Exchange can override the endpoint's fixed operation by setting the **CamelMongoDbOperation** header, defined by the **MongoDbConstants.OPERATION_HEADER** constant.

The values supported are determined by the **MongoDbOperation** enumeration and match the accepted values for the **operation** parameter on the endpoint URI.

For example:

```
// from("direct:insert").to("mongodb:myDb?database=flights&collection=tickets&operation=insert");
Object result = template.requestBodyAndHeader("direct:insert", "irrelevantBody",
MongoDbConstants.OPERATION_HEADER, "count");
assertTrue("Result is not of type Long", result instanceof Long);
```

102.9. CONSUMERS

There are several types of consumers:

1. Tailable Cursor Consumer
2. Change Streams Consumer

102.9.1. Tailable Cursor Consumer

MongoDB offers a mechanism to instantaneously consume ongoing data from a collection, by keeping the cursor open just like the **tail -f** command of *nix systems. This mechanism is significantly more efficient than a scheduled poll, due to the fact that the server pushes new data to the client as it becomes available, rather than making the client ping back at scheduled intervals to fetch new data. It also reduces otherwise redundant network traffic.

There is only one requisite to use tailable cursors: the collection must be a "capped collection", meaning that it will only hold N objects, and when the limit is reached, MongoDB flushes old objects in the same order they were originally inserted. For more information, please refer to <http://www.mongodb.org/display/DOCS/Tailable+Cursors>.

The Camel MongoDB component implements a tailable cursor consumer, making this feature available for you to use in your Camel routes. As new objects are inserted, MongoDB will push them as **Document** in natural order to your tailable cursor consumer, who will transform them to an Exchange and will trigger your route logic.

102.10. HOW THE TAILABLE CURSOR CONSUMER WORKS

To turn a cursor into a tailable cursor, a few special flags are to be signalled to MongoDB when first generating the cursor. Once created, the cursor will then stay open and will block upon calling the **MongoCursor.next()** method until new data arrives. However, the MongoDB server reserves itself the right to kill your cursor if new data doesn't appear after an indeterminate period. If you are interested to continue consuming new data, you have to regenerate the cursor. And to do so, you will have to remember the position where you left off or else you will start consuming from the top again.

The Camel MongoDB tailable cursor consumer takes care of all these tasks for you. You will just need to provide the key to some field in your data of increasing nature, which will act as a marker to position your cursor every time it is regenerated, e.g. a timestamp, a sequential ID, etc. It can be of any datatype supported by MongoDB. Date, Strings and Integers are found to work well. We call this mechanism "tail tracking" in the context of this component.

The consumer will remember the last value of this field and whenever the cursor is to be regenerated, it will run the query with a filter like: **increasingField > lastValue**, so that only unread data is consumed.

Setting the increasing field: Set the key of the increasing field on the endpoint URI **tailTrackingIncreasingField** option. In Camel 2.10, it must be a top-level field in your data, as nested navigation for this field is not yet supported. That is, the "timestamp" field is okay, but "nested.timestamp" will not work. Please open a ticket in the Camel JIRA if you do require support for nested increasing fields.

Cursor regeneration delay: One thing to note is that if new data is not already available upon initialisation, MongoDB will kill the cursor instantly. Since we don't want to overwhelm the server in this case, a **cursorRegenerationDelay** option has been introduced (with a default value of 1000ms.), which you can modify to suit your needs.

An example:

```
from("mongodb:myDb?
database=flights&collection=cancellations&tailTrackIncreasingField=departureTime")
  .id("tailableCursorConsumer1")
  .autoStartup(false)
  .to("mock:test");
```

The above route will consume from the "flights.cancellations" capped collection, using "departureTime" as the increasing field, with a default regeneration cursor delay of 1000ms.

102.11. PERSISTENT TAIL TRACKING

Standard tail tracking is volatile and the last value is only kept in memory. However, in practice you will need to restart your Camel container every now and then, but your last value would then be lost and your tailable cursor consumer would start consuming from the top again, very likely sending duplicate records into your route.

To overcome this situation, you can enable the **persistent tail tracking** feature to keep track of the last consumed increasing value in a special collection inside your MongoDB database too. When the consumer initialises again, it will restore the last tracked value and continue as if nothing happened.

The last read value is persisted on two occasions: every time the cursor is regenerated and when the consumer shuts down. We may consider persisting at regular intervals too in the future (flush every 5 seconds) for added robustness if the demand is there. To request this feature, please open a ticket in the Camel JIRA.

102.12. ENABLING PERSISTENT TAIL TRACKING

To enable this function, set at least the following options on the endpoint URI:

- **persistentTailTracking** option to **true**
- **persistentId** option to a unique identifier for this consumer, so that the same collection can be reused across many consumers

Additionally, you can set the **tailTrackDb**, **tailTrackCollection** and **tailTrackField** options to customise where the runtime information will be stored. Refer to the endpoint options table at the top of this page for descriptions of each option.

For example, the following route will consume from the "flights.cancellations" capped collection, using "departureTime" as the increasing field, with a default regeneration cursor delay of 1000ms, with persistent tail tracking turned on, and persisting under the "cancellationsTracker" id on the "flights.camelTailTracking", storing the last processed value under the "lastTrackingValue" field (**camelTailTracking** and **lastTrackingValue** are defaults).

```
from("mongodb:myDb?
database=flights&collection=cancellations&tailTrackIncreasingField=departureTime&persistentTailTrack
ng=true" +
"&persistentId=cancellationsTracker")
.id("tailableCursorConsumer2")
.autoStartup(false)
.to("mock:test");
```

Below is another example identical to the one above, but where the persistent tail tracking runtime information will be stored under the "trackers.camelTrackers" collection, in the "lastProcessedDepartureTime" field:

```
from("mongodb:myDb?
database=flights&collection=cancellations&tailTrackIncreasingField=departureTime&persistentTailTrack
ng=true" +
"&persistentId=cancellationsTracker&tailTrackDb=trackers&tailTrackCollection=camelTrackers" +
"&tailTrackField=lastProcessedDepartureTime")
.id("tailableCursorConsumer3")
.autoStartup(false)
.to("mock:test");
```

102.12.1. Change Streams Consumer

Change Streams allow applications to access real-time data changes without the complexity and risk of tailing the MongoDB oplog. Applications can use change streams to subscribe to all data changes on a collection and immediately react to them. Because change streams use the aggregation framework, applications can also filter for specific changes or transform the notifications at will. The exchange body will contain the full document of any change.

To configure Change Streams Consumer you need to specify **consumerType**, **database**, **collection** and optional JSON property **streamFilter** to filter events. That JSON property is standard MongoDB **\$match** aggregation. It could be easily specified using XML DSL configuration:

```
<route id="filterConsumer">
  <from uri="mongodb:myDb?"
```

```

consumerType=changeStreams&database=flights&collection=tickets&streamFilter={
'$match':{'$or':{'fullDocument.stringValue': 'specificValue'}} }"/>
<to uri="mock:test"/>
</route>

```

Java configuration:

```

from("mongodb:myDb?
consumerType=changeStreams&database=flights&collection=tickets&streamFilter={ '$match':{'$or':
[{'fullDocument.stringValue': 'specificValue'}} ]}")
.to("mock:test");

```



NOTE

You can externalize the `streamFilter` value into a property placeholder which allows the endpoint URI parameters to be *cleaner* and easier to read.

The **changeStreams** consumer type will also return the following OUT headers:

Header key	Quick constant	Description (extracted from MongoDB API doc)	Data type
CamelMongoDbStreamOperationType	MongoDbConstants.STREAM_OPERATION_TYPE	The type of operation that occurred. Can be any of the following values: insert, delete, replace, update, drop, rename, dropDatabase, invalidate.	String
_id	MongoDbConstants.MONGO_ID	A document that contains the <code>_id</code> of the document created or modified by the insert, replace, delete, update operations (i.e. CRUD operations). For sharded collections, also displays the full shard key for the document. The <code>_id</code> field is not repeated if it is already a part of the shard key.	ObjectId

102.13. TYPE CONVERSIONS

The **MongoDbBasicConverters** type converter included with the camel-mongodb component provides the following conversions:

Name	From type	To type	How?
fromMapToDocument	Map	Document	constructs a new Document via the new Document(Map m) constructor.
fromDocumentToMap	Document	Map	Document already implements Map .

Name	From type	To type	How?
fromStringToDocument	String	Document	uses com.mongodb.Document.parse(String s) .
fromStringToObjectId	String	ObjectId	constructs a new ObjectId via the new ObjectId(s)
fromFileToDocument	File	Document	uses fromInputStreamToDocument under the hood
fromInputStreamToDocument	InputStream	Document	converts the inputstream bytes to a Document
fromStringToList	String	List<Bson>	uses org.bson.codecs.configuration.CodecRegistries to convert to BsonArray then to List<Bson>.

This type converter is auto-discovered, so you don't need to configure anything manually.

102.14. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options, which are listed below.

Name	Description	Default	Type
camel.component.mongodb.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.mongodb.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.mongodb.enabled	Whether to enable auto configuration of the mongodb component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.mongodb.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.mongodb.mongo-connection	Shared client used for connection. All endpoints generated from the component will share this connection client. The option is a com.mongodb.client.MongoClient type.		MongoClient

CHAPTER 103. MYBATIS

Since Camel 2.7

Both producer and consumer are supported

The MyBatis component allows you to query, poll, insert, update and delete data in a relational database using [MyBatis](#).

103.1. DEPENDENCIES

When using **mybatis** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-mybatis-starter</artifactId>
</dependency>
```

103.2. URI FORMAT

```
mybatis:statementName[?options]
```

Where **statementName** is the statement name in the MyBatis XML mapping file which maps to the query, insert, update or delete operation you choose to evaluate.

You can append query options to the URI in the following format, **?option=value&option=value&...**

This component will by default load the MyBatis SqlMapConfig file from the root of the classpath with the expected name of **SqlMapConfig.xml**.

If the file is located in another location, you will need to configure the **configurationUri** option on the **MyBatisComponent** component.

103.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

103.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

103.3.2. Configuring Endpoint Options

Endpoints have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a *type safe* way of configuring endpoints and data formats in Java.

Use [Property Placeholders](#) to configure options that allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

103.4. COMPONENT OPTIONS

The MyBatis component supports 5 options, which are listed below.

Name	Description	Default	Type
configurationUri (common)	Location of MyBatis xml configuration file. The default value is: SqlMapConfig.xml loaded from the classpath.	SqlMapConfig.xml	String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
sqlSessionFactory (advanced)	To use the SqlSessionFactory.		SqlSessionFactory

103.5. ENDPOINT OPTIONS

The MyBatis endpoint is configured using URI syntax:

```
mybatis:statement
```

Following are the path and query parameters.

103.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
statement (common)	Required The statement name in the MyBatis XML mapping file which maps to the query, insert, update or delete operation you wish to evaluate.		String

103.5.2. Query Parameters (30 parameters)

Name	Description	Default	Type
maxMessagesPerPoll (consumer)	This option is intended to split results returned by the database pool into the batches and deliver them in multiple exchanges. This integer defines the maximum messages to deliver in single exchange. By default, no maximum is set. Can be used to set a limit of e.g. 1000 to avoid when starting up the server that there are thousands of files. Set a value of 0 or negative to disable it.	0	int
onConsume (consumer)	Statement to run after data has been processed in the route.		String
routeEmptyResultSet (consumer)	Whether allow empty resultset to be routed to the next hop.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean

Name	Description	Default	Type
transacted (consumer)	Enables or disables transaction. If enabled then if processing an exchange failed then the consumer breaks out processing any further exchanges to cause a rollback eager.	false	boolean
useliterator (consumer)	Process resultset individually or as a list.	true	boolean
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: * InOnly * InOut * InOptionalOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable org.apache.camel.PollingConsumerPollingStrategy allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
processingStrategy (consumer (advanced))	To use a custom MyBatisProcessingStrategy.		MyBatisProcessingStrategy
executorType (producer)	The executor type to be used while executing statements. simple - executor does nothing special. reuse - executor reuses prepared statements. batch - executor reuses statements and batches updates. Enum values: * SIMPLE * REUSE * BATCH	SIMPLE	ExecutorType

Name	Description	Default	Type
inputHeader (producer)	User the header value for input parameters instead of the message body. By default, inputHeader == null and the input parameters are taken from the message body. If outputHeader is set, the value is used and query parameters will be taken from the header instead of the body.		String
outputHeader (producer)	Store the query result in a header instead of the message body. By default, outputHeader == null and the query result is stored in the message body, any existing content in the message body is discarded. If outputHeader is set, the value is used as the name of the header to store the query result and the original message body is preserved. Setting outputHeader will also omit populating the default CamelMyBatisResult header since it would be the same as outputHeader all the time.		String
statementType (producer)	Mandatory to specify for the producer to control which kind of operation to invoke. Enum values: * SelectOne * SelectList * Insert * InsertList * Update * UpdateList * Delete * DeleteList		StatementType

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int

Name	Description	Default	Type
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then <code>backoffIdleThreshold</code> and/or <code>backoffErrorThreshold</code> must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the <code>ScheduledPollConsumer</code> will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long

Name	Description	Default	Type
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: * TRACE * DEBUG * INFO * WARN * ERROR * OFF	TRACE	LogLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: * NANoseconds * MICROseconds * MILLISECONDS * SECONDS * MINUTES * HOURS * DAYS	MILLISECONDS	TimeUnit

Name	Description	Default	Type
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See <code>ScheduledExecutorService</code> in JDK for details.	true	boolean

103.6. MESSAGE HEADERS

The MyBatis component supports 2 message headers that are listed below.

Name	Description	Default	Type
CamelMyBatisResult (producer) Constant: MYBATIS_RESULT	The response returned from MtBatis in any of the operations. For instance an INSERT could return the auto-generated key, or number of rows etc.		Object
CamelMyBatisStatementName (common) Constant: MYBATIS_STATEMENT_NAME	The statementName used (for example: <code>insertAccount</code>).		String

103.7. MESSAGE BODY

The response from MyBatis will only be set as the body if it is a **SELECT** statement. For example, for **INSERT** statements Camel will not replace the body. This allows you to continue routing and keep the original body. The response from MyBatis is always stored in the header with the key **CamelMyBatisResult**.

103.8. SAMPLES

For example if you wish to consume beans from a JMS queue and insert them into a database you could do the following:

```
from("activemq:queue:newAccount")
.to("mybatis:insertAccount?statementType=Insert");
```

You must specify the **statementType** as you need to instruct Camel which kind of operation to invoke.

Where **insertAccount** is the MyBatis ID in the SQL mapping file:

```
<!-- Insert example, using the Account parameter class -->
<insert id="insertAccount" parameterType="Account">
insert into ACCOUNT (
```

```

    ACC_ID,
    ACC_FIRST_NAME,
    ACC_LAST_NAME,
    ACC_EMAIL
  )
  values (
    #{id}, #{firstName}, #{lastName}, #{emailAddress}
  )
</insert>

```

103.9. USING STATEMENTTYPE FOR BETTER CONTROL OF MYBATIS

When routing to an MyBatis endpoint you will want more fine grained control so you can control whether the SQL statement to be executed is a **SELECT**, **UPDATE**, **DELETE** or **INSERT** etc. So for instance if we want to route to an MyBatis endpoint in which the IN body contains parameters to a **SELECT** statement we can do:

In the code above we can invoke the MyBatis statement **selectAccountById** and the IN body should contain the account id we want to retrieve, such as an **Integer** type.

You can do the same for some of the other operations, such as **SelectList**:

And the same for **UPDATE**, where you can send an **Account** object as the IN body to MyBatis:

103.9.1. Using InsertList StatementType

MyBatis allows you to insert multiple rows using its for-each batch driver. To use this, you need to use the `<foreach>` in the mapper XML file. For example as shown below:

Then you can insert multiple rows, by sending a Camel message to the **mybatis** endpoint which uses the **InsertList** statement type, as shown below:

103.9.2. Using UpdateList StatementType

MyBatis allows you to update multiple rows using its for-each batch driver. To use this, you need to use the `<foreach>` in the mapper XML file. For example as shown below:

```

<update id="batchUpdateAccount" parameterType="java.util.Map">
  update ACCOUNT set
  ACC_EMAIL = #{emailAddress}
  where
  ACC_ID in
  <foreach item="Account" collection="list" open="(" close=")" separator=",">
    #{Account.id}
  </foreach>
</update>

```

Then you can update multiple rows, by sending a Camel message to the mybatis endpoint which uses the UpdateList statement type, as shown below:

```

from("direct:start")
  .to("mybatis:batchUpdateAccount?statementType=UpdateList")
  .to("mock:result");

```


103.9.3. Using DeleteList StatementType

MyBatis allows you to delete multiple rows using its for-each batch driver. To use this, you need to use the `<foreach>` in the mapper XML file. For example as shown below:

```
<delete id="batchDeleteAccountById" parameterType="java.util.List">
  delete from ACCOUNT
  where
  ACC_ID in
  <foreach item="AccountID" collection="list" open="(" close=")" separator=",">
    #{AccountID}
  </foreach>
</delete>
```

Then you can delete multiple rows, by sending a Camel message to the mybatis endpoint which uses the DeleteList statement type, as shown below:

```
from("direct:start")
  .to("mybatis:batchDeleteAccount?statementType=DeleteList")
  .to("mock:result");
```

103.9.4. Notice on InsertList, UpdateList and DeleteList StatementTypes

Parameter of any type (List, Map, etc.) can be passed to mybatis and an end user is responsible for handling it as required with the help of [mybatis dynamic queries](#) capabilities.

103.9.5. cheduled polling example

This component supports scheduled polling and can therefore be used as a Polling Consumer. For example to poll the database every minute:

```
from("mybatis:selectAllAccounts?delay=60000")
  .to("activemq:queue:allAccounts");
```

See "ScheduledPollConsumer Options" on Polling Consumer for more options.

Alternatively you can use another mechanism for triggering the scheduled polls, such as the [Timer](#) or [Quartz](#) components. In the sample below we poll the database, every 30 seconds using the [Timer](#) component and send the data to the JMS queue:

```
from("timer://pollTheDatabase?delay=30000")
  .to("mybatis:selectAllAccounts")
  .to("activemq:queue:allAccounts");
```

And the MyBatis SQL mapping file used:

```
<!-- Select with no parameters using the result map for Account class. -->
<select id="selectAllAccounts" resultMap="AccountResult">
  select * from ACCOUNT
</select>
```

103.9.6. Using onConsume

This component supports executing statements **after** data have been consumed and processed by Camel. This allows you to do post updates in the database. Notice all statements must be **UPDATE** statements. Camel supports executing multiple statements whose names should be separated by commas.

The route below illustrates we execute the **consumeAccount** statement data is processed. This allows us to change the status of the row in the database to processed, so we avoid consuming it twice or more.

And the statements in the sqlmap file:

103.9.7. Participating in transactions

Setting up a transaction manager under camel-mybatis can be a little bit fiddly, as it involves externalizing the database configuration outside the standard MyBatis **SqlMapConfig.xml** file.

The first part requires the setup of a **DataSource**. This is typically a pool (either DBCP, or c3p0), which needs to be wrapped in a Spring proxy. This proxy enables non-Spring use of the **DataSource** to participate in Spring transactions (the MyBatis **SqlSessionFactory** does just this).

```
<bean id="dataSource"
class="org.springframework.jdbc.datasource.TransactionAwareDataSourceProxy">
  <constructor-arg>
    <bean class="com.mchange.v2.c3p0.ComboPooledDataSource">
      <property name="driverClass" value="org.postgresql.Driver"/>
      <property name="jdbcUrl" value="jdbc:postgresql://localhost:5432/myDatabase"/>
      <property name="user" value="myUser"/>
      <property name="password" value="myPassword"/>
    </bean>
  </constructor-arg>
</bean>
```

This has the additional benefit of enabling the database configuration to be externalized using property placeholders.

A transaction manager is then configured to manage the outermost **DataSource**:

```
<bean id="txManager"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
  <property name="dataSource" ref="dataSource"/>
</bean>
```

A mybatis-spring **SqlSessionFactoryBean** then wraps that same **DataSource**:

```
<bean id="sqlSessionFactory" class="org.mybatis.spring.SqlSessionFactoryBean">
  <property name="dataSource" ref="dataSource"/>
  <!-- standard mybatis config file -->
  <property name="configLocation" value="/META-INF/SqlMapConfig.xml"/>
  <!-- externalised mappers -->
  <property name="mapperLocations" value="classpath*:META-INF/mappers/**/*.xml"/>
</bean>
```

The camel-mybatis component is then configured with that factory:

```
<bean id="mybatis" class="org.apache.camel.component.mybatis.MyBatisComponent">
  <property name="sqlSessionFactory" ref="sqlSessionFactory"/>
</bean>
```

Finally, a transaction policy is defined over the top of the transaction manager, which can then be used as usual:

```
<bean id="PROPAGATION_REQUIRED"
class="org.apache.camel.spring.spi.SpringTransactionPolicy">
  <property name="transactionManager" ref="txManager"/>
  <property name="propagationBehaviorName" value="PROPAGATION_REQUIRED"/>
</bean>

<camelContext id="my-model-context" xmlns="http://camel.apache.org/schema/spring">
  <route id="insertModel">
    <from uri="direct:insert"/>
    <transacted ref="PROPAGATION_REQUIRED"/>
    <to uri="mybatis:myModel.insert?statementType=Insert"/>
  </route>
</camelContext>
```

103.10. MYBATIS SPRING BOOT STARTER INTEGRATION

Spring Boot users can use [mybatis-spring-boot-starter](#) artifact provided by the mybatis team

```
<dependency>
  <groupId>org.mybatis.spring.boot</groupId>
  <artifactId>mybatis-spring-boot-starter</artifactId>
  <version>2.3.0</version>
</dependency>
```

in particular AutoConfigured beans from mybatis-spring-boot-starter can be used as follow:

```
#application.properties
camel.component.mybatis.sql-session-factory=#sqlSessionFactory
```

103.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
camel.component.mybatis-bean.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.mybatis-bean.configuration-uri	Location of MyBatis xml configuration file. The default value is: SqlMapConfig.xml loaded from the classpath.	SqlMapConfig.xml	String
camel.component.mybatis-bean.enabled	Whether to enable auto configuration of the mybatis-bean component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.mybatis-bean.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.mybatis-bean.sql-session-factory	To use the SqlSessionFactory. The option is a org.apache.ibatis.session.SqlSessionFactory type.		SqlSessionFactory
camel.component.mybatis.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.mybatis.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.mybatis.configuration-uri	Location of MyBatis xml configuration file. The default value is: <code>SqlMapConfig.xml</code> loaded from the classpath.	<code>SqlMapConfig.xml</code>	String
camel.component.mybatis.enabled	Whether to enable auto configuration of the mybatis component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.mybatis.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.mybatis.sql-session-factory	To use the SqlSessionFactory. The option is a org.apache.ibatis.session.SqlSessionFactory type.		SqlSessionFactory

CHAPTER 104. NETTY

Both producer and consumer are supported

The Netty component in Camel is a socket communication component, based on the [Netty](#) project version 4.

Netty is a NIO client server framework which enables quick and easy development of networkServerInitializerFactory applications such as protocol servers and clients.

Netty greatly simplifies and streamlines network programming such as TCP and UDP socket server.

This camel component supports both producer and consumer endpoints.

The Netty component has several options and allows fine-grained control of a number of TCP/UDP communication parameters (buffer sizes, keepAlives, tcpNoDelay, etc) and facilitates both In-Only and In-Out communication on a Camel route.

104.1. DEPENDENCIES

When using **netty** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-netty-starter</artifactId>
</dependency>
```

104.2. URI FORMAT

The URI scheme for a netty component is as follows

```
netty:tcp://0.0.0.0:99999[?options]
netty:udp://remotehost:99999/[?options]
```

This component supports producer and consumer endpoints for both TCP and UDP.

104.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

104.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

104.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

104.4. COMPONENT OPTIONS

The Netty component supports 73 options, which are listed below.

Name	Description	Default	Type
configuration (common)	To use the NettyConfiguration as configuration when creating endpoints.		NettyConfiguration
disconnect (common)	Whether or not to disconnect(close) from Netty Channel right after use. Can be used for both consumer and producer.	false	boolean
keepAlive (common)	Setting to ensure socket is not closed due to inactivity.	true	boolean
reuseAddress (common)	Setting to facilitate socket multiplexing.	true	boolean

Name	Description	Default	Type
reuseChannel (common)	This option allows producers and consumers (in client mode) to reuse the same Netty Channel for the lifecycle of processing the Exchange. This is useful if you need to call a server multiple times in a Camel route and want to use the same network connection. When using this, the channel is not returned to the connection pool until the Exchange is done; or disconnected if the disconnect option is set to true. The reused Channel is stored on the Exchange as an exchange property with the key <code>NettyConstants#NETTY_CHANNEL</code> which allows you to obtain the channel during routing and use it as well.	false	boolean
sync (common)	Setting to set endpoint as one-way or request-response.	true	boolean
tcpNoDelay (common)	Setting to improve TCP protocol performance.	true	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
broadcast (consumer)	Setting to choose Multicast over UDP.	false	boolean
clientMode (consumer)	If the clientMode is true, netty consumer will connect the address as a TCP client.	false	boolean
reconnect (consumer)	Used only in clientMode in consumer, the consumer will attempt to reconnect on disconnection if this is enabled.	true	boolean
reconnectInterval (consumer)	Used if reconnect and clientMode is enabled. The interval in milli seconds to attempt reconnection.	10000	int

Name	Description	Default	Type
backlog (consumer (advanced))	Allows to configure a backlog for netty consumer (server). Note the backlog is just a best effort depending on the OS. Setting this option to a value such as 200, 500 or 1000, tells the TCP stack how long the accept queue can be. If this option is not configured, then the backlog depends on OS setting.		int
bossCount (consumer (advanced))	When netty works on nio mode, it uses default bossCount parameter from Netty, which is 1. User can use this option to override the default bossCount from Netty.	1	int
bossGroup (consumer (advanced))	Set the BossGroup which could be used for handling the new connection of the server side across the NettyEndpoint.		EventLoopGroup
disconnectOnNoReply (consumer (advanced))	If sync is enabled then this option dictates NettyConsumer if it should disconnect where there is no reply to send back.	true	boolean
executorService (consumer (advanced))	To use the given EventExecutorGroup.		EventExecutorGroup
maximumPoolSize (consumer (advanced))	Sets a maximum thread pool size for the netty consumer ordered thread pool. The default size is 2 x cpu_core plus 1. Setting this value to eg 10 will then use 10 threads unless 2 x cpu_core plus 1 is a higher value, which then will override and be used. For example if there are 8 cores, then the consumer thread pool will be 17. This thread pool is used to route messages received from Netty by Camel. We use a separate thread pool to ensure ordering of messages and also in case some messages will block, then netty's worker threads (event loop) won't be affected.		int
nettyServerBootstrapFactory (consumer (advanced))	To use a custom NettyServerBootstrapFactory.		NettyServerBootstrapFactory
networkInterface (consumer (advanced))	When using UDP then this option can be used to specify a network interface by its name, such as eth0 to join a multicast group.		String

Name	Description	Default	Type
noReplyLogLevel (consumer (advanced))	If sync is enabled this option dictates NettyConsumer which logging level to use when logging a there is no reply to send back. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	WARN	LoggingLevel
serverClosedChannelExceptionCaughtLogLevel (consumer (advanced))	If the server (NettyConsumer) catches an <code>java.nio.channels.ClosedChannelException</code> then its logged using this logging level. This is used to avoid logging the closed channel exceptions, as clients can disconnect abruptly and then cause a flood of closed exceptions in the Netty server. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	DEBUG	LoggingLevel

Name	Description	Default	Type
serverExceptionHandlerLogLevel (consumer (advanced))	If the server (NettyConsumer) catches an exception then its logged using this logging level. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	WARN	LogLevel
serverInitializerFactory (consumer (advanced))	To use a custom ServerInitializerFactory.		ServerInitializerFactory
usingExecutorService (consumer (advanced))	Whether to use ordered thread pool, to ensure events are processed orderly on the same channel.	true	boolean
connectTimeout (producer)	Time to wait for a socket connection to be available. Value is in milliseconds.	10000	int
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
requestTimeout (producer)	Allows to use a timeout for the Netty producer when calling a remote server. By default no timeout is in use. The value is in milli seconds, so eg 30000 is 30 seconds. The requestTimeout is using Netty's ReadTimeoutHandler to trigger the timeout.		long
clientInitializerFactory (producer (advanced))	To use a custom ClientInitializerFactory.		ClientInitializerFactory

Name	Description	Default	Type
correlationManager (producer (advanced))	To use a custom correlation manager to manage how request and reply messages are mapped when using request/reply with the netty producer. This should only be used if you have a way to map requests together with replies such as if there is correlation ids in both the request and reply messages. This can be used if you want to multiplex concurrent messages on the same channel (aka connection) in netty. When doing this you must have a way to correlate the request and reply messages so you can store the right reply on the inflight Camel Exchange before its continued routed. We recommend extending the TimeoutCorrelationManagerSupport when you build custom correlation managers. This provides support for timeout and other complexities you otherwise would need to implement as well. See also the producerPoolEnabled option for more details.		NettyCamelStateCorrelationManager
lazyChannelCreation (producer (advanced))	Channels can be lazily created to avoid exceptions, if the remote server is not up and running when the Camel producer is started.	true	boolean
producerPoolEnabled (producer (advanced))	Whether producer pool is enabled or not. Important: If you turn this off then a single shared connection is used for the producer, also if you are doing request/reply. That means there is a potential issue with interleaved responses if replies comes back out-of-order. Therefore you need to have a correlation id in both the request and reply messages so you can properly correlate the replies to the Camel callback that is responsible for continue processing the message in Camel. To do this you need to implement NettyCamelStateCorrelationManager as correlation manager and configure it via the correlationManager option. See also the correlationManager option for more details.	true	boolean
producerPoolMaxIdle (producer (advanced))	Sets the cap on the number of idle instances in the pool.	100	int
producerPoolMaxTotal (producer (advanced))	Sets the cap on the number of objects that can be allocated by the pool (checked out to clients, or idle awaiting checkout) at a given time. Use a negative value for no limit.	-1	int

Name	Description	Default	Type
producerPoolMinEvictableIdle (producer (advanced))	Sets the minimum amount of time (value in millis) an object may sit idle in the pool before it is eligible for eviction by the idle object evictor.	300000	long
producerPoolMinIdle (producer (advanced))	Sets the minimum number of instances allowed in the producer pool before the evictor thread (if active) spawns new objects.		int
udpConnectionlessSending (producer (advanced))	This option supports connection less udp sending which is a real fire and forget. A connected udp send receive the PortUnreachableException if no one is listen on the receiving port.	false	boolean
useByteBuf (producer (advanced))	If the useByteBuf is true, netty producer will turn the message body into ByteBuf before sending it out.	false	boolean
hostnameVerification (security)	To enable/disable hostname verification on SSLEngine.	false	boolean
allowSerializedHeaders (advanced)	Only used for TCP when transferExchange is true. When set to true, serializable objects in headers and properties will be added to the exchange. Otherwise Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
channelGroup (advanced)	To use a explicit ChannelGroup.		ChannelGroup
nativeTransport (advanced)	Whether to use native transport instead of NIO. Native transport takes advantage of the host operating system and is only supported on some platforms. You need to add the netty JAR for the host operating system you are using. See more details at: .	false	boolean

Name	Description	Default	Type
options (advanced)	Allows to configure additional netty options using option. as prefix. For example option.child.keepAlive=false to set the netty option child.keepAlive=false. See the Netty documentation for possible options that can be used.		Map
receiveBufferSize (advanced)	The TCP/UDP buffer sizes to be used during inbound communication. Size is bytes.	65536	int
receiveBufferSize Predictor (advanced)	Configures the buffer size predictor. See details at Jetty documentation and this mail thread.		int
sendBufferSize (advanced)	The TCP/UDP buffer sizes to be used during outbound communication. Size is bytes.	65536	int
transferExchange (advanced)	Only used for TCP. You can transfer the exchange over the wire instead of just the body. The following fields are transferred: In body, Out body, fault body, In headers, Out headers, fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
udpByteArrayCodec (advanced)	For UDP only. If enabled the using byte array codec instead of Java serialization protocol.	false	boolean
workerCount (advanced)	When netty works on nio mode, it uses default workerCount parameter from Netty (which is cpu_core_threads x 2). User can use this option to override the default workerCount from Netty.		int
workerGroup (advanced)	To use a explicit EventLoopGroup as the boss thread pool. For example to share a thread pool with multiple consumers or producers. By default each consumer or producer has their own worker pool with 2 x cpu count core threads.		EventLoopGroup
allowDefaultCodec (codec)	The netty component installs a default codec if both, encoder/decoder is null and textline is false. Setting allowDefaultCodec to false prevents the netty component from installing a default codec as the first element in the filter chain.	true	boolean
autoAppendDelimiter (codec)	Whether or not to auto append missing end delimiter when sending using the textline codec.	true	boolean

Name	Description	Default	Type
decoderMaxLineLength (codec)	The max line length to use for the textline codec.	1024	int
decoders (codec)	A list of decoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		List
delimiter (codec)	The delimiter to use for the textline codec. Possible values are LINE and NULL. Enum values: • LINE • NULL	LINE	TextLineDelimiter
encoders (codec)	A list of encoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		List
encoding (codec)	The encoding (a charset name) to use for the textline codec. If not provided, Camel will use the JVM default Charset.		String
textline (codec)	Only used for TCP. If no codec is specified, you can use this flag to indicate a text line based codec; if not specified or the value is false, then Object Serialization is assumed over TCP – however only Strings are allowed to be serialized by default.	false	boolean
enabledProtocols (security)	Which protocols to enable when using SSL.	TLSv1, TLSv1.1, TLSv1.2	String
keyStoreFile (security)	Client side certificate keystore to be used for encryption.		File
keyStoreFormat (security)	Keystore format to be used for payload encryption. Defaults to JKS if not set.		String
keyStoreResource (security)	Client side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String

Name	Description	Default	Type
needClientAuth (security)	Configures whether the server needs client authentication when using SSL.	false	boolean
passphrase (security)	Password setting to use in order to encrypt/decrypt payloads sent using SSH.		String
securityProvider (security)	Security provider to be used for payload encryption. Defaults to SunX509 if not set.		String
ssl (security)	Setting to specify whether SSL encryption is applied to this endpoint.	false	boolean
sslClientCertHeaders (security)	When enabled and in SSL mode, then the Netty consumer will enrich the Camel Message with headers having information about the client certificate such as subject name, issuer name, serial number, and the valid date range.	false	boolean
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters
sslHandler (security)	Reference to a class that could be used to return an SSL Handler.		SslHandler
trustStoreFile (security)	Server side certificate keystore to be used for encryption.		File
trustStoreResource (security)	Server side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean

104.5. ENDPOINT OPTIONS

The Netty endpoint is configured using URI syntax:

```
netty:protocol://host:port
```

with the following path and query parameters:

104.5.1. Path Parameters (3 parameters)

Name	Description	Default	Type
protocol (common)	Required The protocol to use which can be tcp or udp. Enum values: • tcp • udp		String
host (common)	Required The hostname. For the consumer the hostname is localhost or 0.0.0.0. For the producer the hostname is the remote host to connect to.		String
port (common)	Required The host port number.		int

104.5.2. Query Parameters (71 parameters)

Name	Description	Default	Type
disconnect (common)	Whether or not to disconnect(close) from Netty Channel right after use. Can be used for both consumer and producer.	false	boolean
keepAlive (common)	Setting to ensure socket is not closed due to inactivity.	true	boolean
reuseAddress (common)	Setting to facilitate socket multiplexing.	true	boolean
reuseChannel (common)	This option allows producers and consumers (in client mode) to reuse the same Netty Channel for the lifecycle of processing the Exchange. This is useful if you need to call a server multiple times in a Camel route and want to use the same network connection. When using this, the channel is not returned to the connection pool until the Exchange is done; or disconnected if the disconnect option is set to true. The reused Channel is stored on the Exchange as an exchange property with the key <code>NettyConstants#NETTY_CHANNEL</code> which allows you to obtain the channel during routing and use it as well.	false	boolean
sync (common)	Setting to set endpoint as one-way or request-response.	true	boolean

Name	Description	Default	Type
tcpNoDelay (common)	Setting to improve TCP protocol performance.	true	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
broadcast (consumer)	Setting to choose Multicast over UDP.	false	boolean
clientMode (consumer)	If the clientMode is true, netty consumer will connect the address as a TCP client.	false	boolean
reconnect (consumer)	Used only in clientMode in consumer, the consumer will attempt to reconnect on disconnection if this is enabled.	true	boolean
reconnectInterval (consumer)	Used if reconnect and clientMode is enabled. The interval in milli seconds to attempt reconnection.	10000	int
backlog (consumer (advanced))	Allows to configure a backlog for netty consumer (server). Note the backlog is just a best effort depending on the OS. Setting this option to a value such as 200, 500 or 1000, tells the TCP stack how long the accept queue can be. If this option is not configured, then the backlog depends on OS setting.		int
bossCount (consumer (advanced))	When netty works on nio mode, it uses default bossCount parameter from Netty, which is 1. User can use this option to override the default bossCount from Netty.	1	int
bossGroup (consumer (advanced))	Set the BossGroup which could be used for handling the new connection of the server side across the NettyEndpoint.		EventLoopGroup
disconnectOnNoReply (consumer (advanced))	If sync is enabled then this option dictates NettyConsumer if it should disconnect where there is no reply to send back.	true	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code> ● <code>InOptionalOut</code>		<code>ExchangePattern</code>
nettyServerBootstrapFactory (consumer (advanced))	To use a custom <code>NettyServerBootstrapFactory</code> .		<code>NettyServerBootstrapFactory</code>
networkInterface (consumer (advanced))	When using UDP then this option can be used to specify a network interface by its name, such as <code>eth0</code> to join a multicast group.		<code>String</code>
noReplyLogLevel (consumer (advanced))	If <code>sync</code> is enabled this option dictates <code>NettyConsumer</code> which logging level to use when logging a there is no reply to send back. Enum values: ● <code>TRACE</code> ● <code>DEBUG</code> ● <code>INFO</code> ● <code>WARN</code> ● <code>ERROR</code> ● <code>OFF</code>	<code>WARN</code>	<code>LogLevel</code>

Name	Description	Default	Type
serverClosedChannelExceptionHandlerLogLevel (consumer (advanced))	If the server (NettyConsumer) catches an <code>java.nio.channels.ClosedChannelException</code> then its logged using this logging level. This is used to avoid logging the closed channel exceptions, as clients can disconnect abruptly and then cause a flood of closed exceptions in the Netty server. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	DEBUG	LogLevel
serverExceptionHandlerLogLevel (consumer (advanced))	If the server (NettyConsumer) catches an exception then its logged using this logging level. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	WARN	LogLevel
serverInitializerFactory (consumer (advanced))	To use a custom <code>ServerInitializerFactory</code> .		<code>ServerInitializerFactory</code>
usingExecutorService (consumer (advanced))	Whether to use ordered thread pool, to ensure events are processed orderly on the same channel.	true	boolean
connectTimeout (producer)	Time to wait for a socket connection to be available. Value is in milliseconds.	10000	int

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
requestTimeout (producer)	Allows to use a timeout for the Netty producer when calling a remote server. By default no timeout is in use. The value is in milli seconds, so eg 30000 is 30 seconds. The requestTimeout is using Netty's ReadTimeoutHandler to trigger the timeout.		long
clientInitializerFactory (producer (advanced))	To use a custom ClientInitializerFactory.		ClientInitializerFactory
correlationManager (producer (advanced))	To use a custom correlation manager to manage how request and reply messages are mapped when using request/reply with the netty producer. This should only be used if you have a way to map requests together with replies such as if there is correlation ids in both the request and reply messages. This can be used if you want to multiplex concurrent messages on the same channel (aka connection) in netty. When doing this you must have a way to correlate the request and reply messages so you can store the right reply on the inflight Camel Exchange before its continued routed. We recommend extending the TimeoutCorrelationManagerSupport when you build custom correlation managers. This provides support for timeout and other complexities you otherwise would need to implement as well. See also the producerPoolEnabled option for more details.		NettyCamelStateCorrelationManager
lazyChannelCreation (producer (advanced))	Channels can be lazily created to avoid exceptions, if the remote server is not up and running when the Camel producer is started.	true	boolean

Name	Description	Default	Type
producerPoolEnabled (producer (advanced))	Whether producer pool is enabled or not. Important: If you turn this off then a single shared connection is used for the producer, also if you are doing request/reply. That means there is a potential issue with interleaved responses if replies comes back out-of-order. Therefore you need to have a correlation id in both the request and reply messages so you can properly correlate the replies to the Camel callback that is responsible for continue processing the message in Camel. To do this you need to implement <code>NettyCamelStateCorrelationManager</code> as correlation manager and configure it via the <code>correlationManager</code> option. See also the <code>correlationManager</code> option for more details.	true	boolean
producerPoolMaxIdle (producer (advanced))	Sets the cap on the number of idle instances in the pool.	100	int
producerPoolMaxTotal (producer (advanced))	Sets the cap on the number of objects that can be allocated by the pool (checked out to clients, or idle awaiting checkout) at a given time. Use a negative value for no limit.	-1	int
producerPoolMinEvictableIdle (producer (advanced))	Sets the minimum amount of time (value in millis) an object may sit idle in the pool before it is eligible for eviction by the idle object evictor.	30000 0	long
producerPoolMinIdle (producer (advanced))	Sets the minimum number of instances allowed in the producer pool before the evictor thread (if active) spawns new objects.		int
udpConnectionlessSending (producer (advanced))	This option supports connection less udp sending which is a real fire and forget. A connected udp send receive the <code>PortUnreachableException</code> if no one is listen on the receiving port.	false	boolean
useByteBuf (producer (advanced))	If the <code>useByteBuf</code> is true, netty producer will turn the message body into <code>ByteBuf</code> before sending it out.	false	boolean
hostnameVerification (security)	To enable/disable hostname verification on <code>SSL</code> Engine.	false	boolean

Name	Description	Default	Type
allowSerializedHeaders (advanced)	Only used for TCP when <code>transferExchange</code> is true. When set to true, serializable objects in headers and properties will be added to the exchange. Otherwise Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
channelGroup (advanced)	To use a explicit ChannelGroup.		ChannelGroup
nativeTransport (advanced)	Whether to use native transport instead of NIO. Native transport takes advantage of the host operating system and is only supported on some platforms. You need to add the netty JAR for the host operating system you are using. See more details at: .	false	boolean
options (advanced)	Allows to configure additional netty options using option. as prefix. For example <code>option.child.keepAlive=false</code> to set the netty option <code>child.keepAlive=false</code> . See the Netty documentation for possible options that can be used.		Map
receiveBufferSize (advanced)	The TCP/UDP buffer sizes to be used during inbound communication. Size is bytes.	65536	int
receiveBufferSizePredictor (advanced)	Configures the buffer size predictor. See details at Jetty documentation and this mail thread.		int
sendBufferSize (advanced)	The TCP/UDP buffer sizes to be used during outbound communication. Size is bytes.	65536	int
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean
transferExchange (advanced)	Only used for TCP. You can transfer the exchange over the wire instead of just the body. The following fields are transferred: In body, Out body, fault body, In headers, Out headers, fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
udpByteArrayCodec (advanced)	For UDP only. If enabled the using byte array codec instead of Java serialization protocol.	false	boolean

Name	Description	Default	Type
workerCount (advanced)	When netty works on nio mode, it uses default workerCount parameter from Netty (which is <code>cpu_core_threads x 2</code>). User can use this option to override the default workerCount from Netty.		int
workerGroup (advanced)	To use a explicit EventLoopGroup as the boss thread pool. For example to share a thread pool with multiple consumers or producers. By default each consumer or producer has their own worker pool with <code>2 x cpu count</code> core threads.		EventLoopGroup
allowDefaultCodec (codec)	The netty component installs a default codec if both, encoder/decoder is null and textline is false. Setting <code>allowDefaultCodec</code> to false prevents the netty component from installing a default codec as the first element in the filter chain.	true	boolean
autoAppendDelimiter (codec)	Whether or not to auto append missing end delimiter when sending using the textline codec.	true	boolean
decoderMaxLineLength (codec)	The max line length to use for the textline codec.	1024	int
decoders (codec)	A list of decoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with <code>#</code> so Camel knows it should lookup.		List
delimiter (codec)	The delimiter to use for the textline codec. Possible values are LINE and NULL. Enum values: • LINE • NULL	LINE	TextLineDelimiter
encoders (codec)	A list of encoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with <code>#</code> so Camel knows it should lookup.		List
encoding (codec)	The encoding (a charset name) to use for the textline codec. If not provided, Camel will use the JVM default Charset.		String

Name	Description	Default	Type
textline (codec)	Only used for TCP. If no codec is specified, you can use this flag to indicate a text line based codec; if not specified or the value is false, then Object Serialization is assumed over TCP - however only Strings are allowed to be serialized by default.	false	boolean
enabledProtocols (security)	Which protocols to enable when using SSL.	TLSv1, TLSv1.1, TLSv1.2	String
keyStoreFile (security)	Client side certificate keystore to be used for encryption.		File
keyStoreFormat (security)	Keystore format to be used for payload encryption. Defaults to JKS if not set.		String
keyStoreResource (security)	Client side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
needClientAuth (security)	Configures whether the server needs client authentication when using SSL.	false	boolean
passphrase (security)	Password setting to use in order to encrypt/decrypt payloads sent using SSH.		String
securityProvider (security)	Security provider to be used for payload encryption. Defaults to SunX509 if not set.		String
ssl (security)	Setting to specify whether SSL encryption is applied to this endpoint.	false	boolean
sslClientCertHeaders (security)	When enabled and in SSL mode, then the Netty consumer will enrich the Camel Message with headers having information about the client certificate such as subject name, issuer name, serial number, and the valid date range.	false	boolean
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters
sslHandler (security)	Reference to a class that could be used to return an SSL Handler.		SslHandler

Name	Description	Default	Type
trustStoreFile (security)	Server side certificate keystore to be used for encryption.		File
trustStoreResource (security)	Server side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String

104.6. REGISTRY BASED OPTIONS

Codec Handlers and SSL Keystores can be enlisted in the Registry, such as in the Spring XML file. The values that could be passed in, are the following:

Name	Description
passphrase	password setting to use in order to encrypt/decrypt payloads sent using SSH
keyStoreFormat	keystore format to be used for payload encryption. Defaults to "JKS" if not set
securityProvider	Security provider to be used for payload encryption. Defaults to "SunX509" if not set.
keyStoreFile	deprecated: Client side certificate keystore to be used for encryption
trustStoreFile	deprecated: Server side certificate keystore to be used for encryption
keyStoreResource	Client side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with " classpath: ", " file: ", or " http: " to load the resource from different systems.
trustStoreResource	Server side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with " classpath: ", " file: ", or " http: " to load the resource from different systems.
sslHandler	Reference to a class that could be used to return an SSL Handler
encoder	A custom ChannelHandler class that can be used to perform special marshalling of outbound payloads. Must override io.netty.channel.ChannelInboundHandlerAdapter.
encoders	A list of encoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.
decoder	A custom ChannelHandler class that can be used to perform special marshalling of inbound payloads. Must override io.netty.channel.ChannelOutboundHandlerAdapter.

Name	Description
decoders	A list of decoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.

**NOTE**

Read below about using non shareable encoders/decoders.

104.6.1. Using non shareable encoders or decoders

If your encoders or decoders are not shareable (e.g. they don't have the `@Shareable` class annotation), then your encoder/decoder must implement the **`org.apache.camel.component.netty.ChannelHandlerFactory`** interface, and return a new instance in the **`newChannelHandler`** method. This is to ensure the encoder/decoder can safely be used. If this is not the case, then the Netty component will log a WARN when an endpoint is created.

The Netty component offers a **`org.apache.camel.component.netty.ChannelHandlerFactories`** factory class, that has a number of commonly used methods.

104.7. SENDING MESSAGES TO/FROM A NETTY ENDPOINT

104.7.1. Netty Producer

In Producer mode, the component provides the ability to send payloads to a socket endpoint using either TCP or UDP protocols (with optional SSL support).

The producer mode supports both one-way and request-response based operations.

104.7.2. Netty Consumer

In Consumer mode, the component provides the ability to:

- listen on a specified socket using either TCP or UDP protocols (with optional SSL support),
- receive requests on the socket using text/xml, binary and serialized object based payloads and
- send them along on a route as message exchanges.

The consumer mode supports both one-way and request-response based operations.

104.8. EXAMPLES

104.8.1. A UDP Netty endpoint using Request-Reply and serialized object payload

Note that Object serialization is not allowed by default, and so a decoder must be configured.

■

```

@BindToRegistry("decoder")
public ChannelHandler getDecoder() throws Exception {
    return new DefaultChannelHandlerFactory() {
        @Override
        public ChannelHandler newChannelHandler() {
            return new DatagramPacketObjectDecoder(ClassResolvers.weakCachingResolver(null));
        }
    };
}

RouteBuilder builder = new RouteBuilder() {
    public void configure() {
        from("netty:udp://0.0.0.0:5155?sync=true&decoders=#decoder")
            .process(new Processor() {
                public void process(Exchange exchange) throws Exception {
                    Poetry poetry = (Poetry) exchange.getIn().getBody();
                    // Process poetry in some way
                    exchange.getOut().setBody("Message received");
                }
            })
    }
};

```

104.8.2. A TCP based Netty consumer endpoint using One-way communication

```

RouteBuilder builder = new RouteBuilder() {
    public void configure() {
        from("netty:tcp://0.0.0.0:5150")
            .to("mock:result");
    }
};

```

104.8.3. An SSL/TCP based Netty consumer endpoint using Request-Reply communication

Using the JSSE Configuration Utility

The Netty component supports SSL/TLS configuration through the [Camel JSSE Configuration Utility](#). This utility greatly decreases the amount of component specific code you need to write and is configurable at the endpoint and component levels. The following examples demonstrate how to use the utility with the Netty component.

Programmatic configuration of the component

```

KeyStoreParameters ksp = new KeyStoreParameters();
ksp.setResource("/users/home/server/keystore.jks");
ksp.setPassword("keystorePassword");

KeyManagersParameters kmp = new KeyManagersParameters();
kmp.setKeyStore(ksp);
kmp.setKeyPassword("keyPassword");

SSLContextParameters scp = new SSLContextParameters();
scp.setKeyManagers(kmp);

```

```
NettyComponent nettyComponent = getContext().getComponent("netty", NettyComponent.class);
nettyComponent.setSslContextParameters(scp);
```

Spring DSL based configuration of endpoint

```
...
<camel:sslContextParameters
  id="sslContextParameters">
  <camel:keyManagers
    keyPassword="keyPassword">
    <camel:keyStore
      resource="/users/home/server/keystore.jks"
      password="keystorePassword"/>
    </camel:keyManagers>
  </camel:sslContextParameters>...
...
<to uri="netty:tcp://0.0.0.0:5150?
sync=true&ssl=true&sslContextParameters=#sslContextParameters"/>
...

```

Using Basic SSL/TLS configuration on the Jetty Component

```
Registry registry = context.getRegistry();
registry.bind("password", "changeit");
registry.bind("ksf", new File("src/test/resources/keystore.jks"));
registry.bind("tsf", new File("src/test/resources/keystore.jks"));

context.addRoutes(new RouteBuilder() {
  public void configure() {
    String netty_ssl_endpoint =
      "netty:tcp://0.0.0.0:5150?sync=true&ssl=true&passphrase=#password"
      + "&keyStoreFile=#ksf&trustStoreFile=#tsf";
    String return_string =
      "When You Go Home, Tell Them Of Us And Say,"
      + "For Your Tomorrow, We Gave Our Today.";

    from(netty_ssl_endpoint)
      .process(new Processor() {
        public void process(Exchange exchange) throws Exception {
          exchange.getOut().setBody(return_string);
        }
      })
  }
});
```

Getting access to SSLSession and the client certificate

You can get access to the **javax.net.ssl.SSLSession** if you eg need to get details about the client certificate. When **ssl=true** then the Netty component will store the **SSLSession** as a header on the Camel Message as shown below:

```
SSLSession session = exchange.getIn().getHeader(NettyConstants.NETTY_SSL_SESSION,
SSLSession.class);
// get the first certificate which is client certificate
```

```
javax.security.cert.X509Certificate cert = session.getPeerCertificateChain()[0];
Principal principal = cert.getSubjectDN();
```

Remember to set **needClientAuth=true** to authenticate the client, otherwise **SSLSession** cannot access information about the client certificate, and you may get an exception **javax.net.ssl.SSLPeerUnverifiedException: peer not authenticated**. You may also get this exception if the client certificate is expired or not valid etc.



NOTE

The option **sslClientCertHeaders** can be set to **true** which then enriches the Camel Message with headers having details about the client certificate. For example the subject name is readily available in the header **CamelNettySSLClientCertSubjectName**.

104.8.4. Using Multiple Codecs

In certain cases it may be necessary to add chains of encoders and decoders to the netty pipeline. To add multiple codecs to a camel netty endpoint the 'encoders' and 'decoders' uri parameters should be used. Like the 'encoder' and 'decoder' parameters they are used to supply references (lists of ChannelUpstreamHandlers and ChannelDownstreamHandlers) that should be added to the pipeline. Note that if encoders is specified then the encoder param will be ignored, similarly for decoders and the decoder param.



NOTE

Read further above about using non shareable encoders/decoders.

The lists of codecs need to be added to the Camel's registry so they can be resolved when the endpoint is created.

```
ChannelHandlerFactory lengthDecoder =
ChannelHandlerFactories.newLengthFieldBasedFrameDecoder(1048576, 0, 4, 0, 4);

StringDecoder stringDecoder = new StringDecoder();
registry.bind("length-decoder", lengthDecoder);
registry.bind("string-decoder", stringDecoder);

LengthFieldPrepender lengthEncoder = new LengthFieldPrepender(4);
StringEncoder stringEncoder = new StringEncoder();
registry.bind("length-encoder", lengthEncoder);
registry.bind("string-encoder", stringEncoder);

List<ChannelHandler> decoders = new ArrayList<ChannelHandler>();
decoders.add(lengthDecoder);
decoders.add(stringDecoder);

List<ChannelHandler> encoders = new ArrayList<ChannelHandler>();
encoders.add(lengthEncoder);
encoders.add(stringEncoder);

registry.bind("encoders", encoders);
registry.bind("decoders", decoders);
```

Spring's native collections support can be used to specify the codec lists in an application context


```

<util:list id="decoders" list-class="java.util.LinkedList">
  <bean class="org.apache.camel.component.netty.ChannelHandlerFactories" factory-
method="newLengthFieldBasedFrameDecoder">
    <constructor-arg value="1048576"/>
    <constructor-arg value="0"/>
    <constructor-arg value="4"/>
    <constructor-arg value="0"/>
    <constructor-arg value="4"/>
  </bean>
  <bean class="io.netty.handler.codec.string.StringDecoder"/>
</util:list>

<util:list id="encoders" list-class="java.util.LinkedList">
  <bean class="io.netty.handler.codec.LengthFieldPrepender">
    <constructor-arg value="4"/>
  </bean>
  <bean class="io.netty.handler.codec.string.StringEncoder"/>
</util:list>

<bean id="length-encoder" class="io.netty.handler.codec.LengthFieldPrepender">
  <constructor-arg value="4"/>
</bean>
<bean id="string-encoder" class="io.netty.handler.codec.string.StringEncoder"/>

<bean id="length-decoder" class="org.apache.camel.component.netty.ChannelHandlerFactories"
factory-method="newLengthFieldBasedFrameDecoder">
  <constructor-arg value="1048576"/>
  <constructor-arg value="0"/>
  <constructor-arg value="4"/>
  <constructor-arg value="0"/>
  <constructor-arg value="4"/>
</bean>
<bean id="string-decoder" class="io.netty.handler.codec.string.StringDecoder"/>

```

The bean names can then be used in netty endpoint definitions either as a comma separated list or contained in a List e.g.

```

from("direct:multiple-codec").to("netty:tcp://0.0.0.0:{{port}}?encoders=#encoders&sync=false");

from("netty:tcp://0.0.0.0:{{port}}?decoders=#length-decoder,#string-
decoder&sync=false").to("mock:multiple-codec");

```

or via XML.

```

<camelContext id="multiple-netty-codecs-context" xmlns="http://camel.apache.org/schema/spring">
  <route>
    <from uri="direct:multiple-codec"/>
    <to uri="netty:tcp://0.0.0.0:5150?encoders=#encoders&sync=false"/>
  </route>
  <route>
    <from uri="netty:tcp://0.0.0.0:5150?decoders=#length-decoder,#string-
decoder&sync=false"/>
    <to uri="mock:multiple-codec"/>
  </route>
</camelContext>

```

104.9. CLOSING CHANNEL WHEN COMPLETE

When acting as a server you sometimes want to close the channel when, for example, a client conversion is finished.

You can do this by simply setting the endpoint option **disconnect=true**.

However you can also instruct Camel on a per message basis as follows.

To instruct Camel to close the channel, you should add a header with the key

CamelNettyCloseChannelWhenComplete set to a boolean **true** value.

For instance, the example below will close the channel after it has written the bye message back to the client:

```
from("netty:tcp://0.0.0.0:8080").process(new Processor() {
    public void process(Exchange exchange) throws Exception {
        String body = exchange.getIn().getBody(String.class);
        exchange.getOut().setBody("Bye " + body);
        // some condition which determines if we should close
        if (close) {

exchange.getOut().setHeader(NettyConstants.NETTY_CLOSE_CHANNEL_WHEN_COMPLETE,
true);
        }
    }
});
```

Adding custom channel pipeline factories to gain complete control over a created pipeline.

104.10. CUSTOM PIPELINE

Custom channel pipelines provide complete control to the user over the handler/interceptor chain by inserting custom handler(s), encoder(s) & decoder(s) without having to specify them in the Netty Endpoint URL in a very simple way.

In order to add a custom pipeline, a custom channel pipeline factory must be created and registered with the context via the context registry (Registry, or the camel-spring ApplicationContextRegistry etc).

A custom pipeline factory must be constructed as follows

- A Producer linked channel pipeline factory must extend the abstract class **ClientPipelineFactory**.
- A Consumer linked channel pipeline factory must extend the abstract class **ServerInitializerFactory**.
- The classes should override the `initChannel()` method in order to insert custom handler(s), encoder(s) and decoder(s). Not overriding the **initChannel()** method creates a pipeline with no handlers, encoders or decoders wired to the pipeline.

The example below shows how `ServerInitializerFactory` factory may be created

104.10.1. Using custom pipeline factory

```
public class SampleServerInitializerFactory extends ServerInitializerFactory {
    private int maxLineSize = 1024;
```

```

protected void initChannel(Channel ch) throws Exception {
    ChannelPipeline channelPipeline = ch.pipeline();

    channelPipeline.addLast("encoder-SD", new StringEncoder(CharsetUtil.UTF_8));
    channelPipeline.addLast("decoder-DELIM", new DelimiterBasedFrameDecoder(maxLineSize,
true, Delimiters.lineDelimiter()));
    channelPipeline.addLast("decoder-SD", new StringDecoder(CharsetUtil.UTF_8));
    // here we add the default Camel ServerChannelHandler for the consumer, to allow Camel to
    route the message etc.
    channelPipeline.addLast("handler", new ServerChannelHandler(consumer));
}
}

```

The custom channel pipeline factory can then be added to the registry and instantiated/utilized on a camel route in the following way

```

Registry registry = camelContext.getRegistry();
ServerInitializerFactory factory = new TestServerInitializerFactory();
registry.bind("spf", factory);
context.addRoutes(new RouteBuilder() {
    public void configure() {
        String netty_ssl_endpoint =
            "netty:tcp://0.0.0.0:5150?serverInitializerFactory=#spf"
        String return_string =
            "When You Go Home, Tell Them Of Us And Say,"
            + "For Your Tomorrow, We Gave Our Today.";

        from(netty_ssl_endpoint)
            .process(new Processor() {
                public void process(Exchange exchange) throws Exception {
                    exchange.getOut().setBody(return_string);
                }
            })
    }
});

```

104.11. REUSING NETTY BOSS AND WORKER THREAD POOLS

Netty has two kind of thread pools: boss and worker. By default each Netty consumer and producer has their private thread pools. If you want to reuse these thread pools among multiple consumers or producers then the thread pools must be created and enlisted in the Registry.

For example using Spring XML we can create a shared worker thread pool using the **NettyWorkerPoolBuilder** with 2 worker threads as shown below:

```

<!-- use the worker pool builder to help create the shared thread pool -->
<bean id="poolBuilder" class="org.apache.camel.component.netty.NettyWorkerPoolBuilder">
    <property name="workerCount" value="2"/>
</bean>

<!-- the shared worker thread pool -->
<bean id="sharedPool" class="org.jboss.netty.channel.socket.nio.WorkerPool"
    factory-bean="poolBuilder" factory-method="build" destroy-method="shutdown">
</bean>

```



NOTE

For boss thread pool there is a **org.apache.camel.component.netty.NettyServerBossPoolBuilder** builder for Netty consumers, and a **org.apache.camel.component.netty.NettyClientBossPoolBuilder** for the Netty producers.

Then in the Camel routes we can refer to this worker pools by configuring the **workerPool** option in the URI as shown below:

```
<route>
  <from uri="netty:tcp://0.0.0.0:5021?
textline=true&sync=true&workerPool=#sharedPool&usingExecutorService=false"/>
  <to uri="log:result"/>
  ...
</route>
```

And if we have another route we can refer to the shared worker pool:

```
<route>
  <from uri="netty:tcp://0.0.0.0:5022?
textline=true&sync=true&workerPool=#sharedPool&usingExecutorService=false"/>
  <to uri="log:result"/>
  ...
</route>
```

and so forth.

104.12. MULTIPLEXING CONCURRENT MESSAGES OVER A SINGLE CONNECTION WITH REQUEST/REPLY

When using Netty for request/reply messaging via the netty producer then by default each message is sent via a non-shared connection (pooled). This ensures that replies are automatic being able to map to the correct request thread for further routing in Camel. In other words correlation between request/reply messages happens out-of-the-box because the replies comes back on the same connection that was used for sending the request; and this connection is not shared with others. When the response comes back, the connection is returned back to the connection pool, where it can be reused by others.

However if you want to multiplex concurrent request/responses on a single shared connection, then you need to turn off the connection pooling by setting **producerPoolEnabled=false**. Now this means there is a potential issue with interleaved responses if replies comes back out-of-order. Therefore you need to have a correlation id in both the request and reply messages so you can properly correlate the replies to the Camel callback that is responsible for continue processing the message in Camel. To do this you need to implement **NettyCamelStateCorrelationManager** as correlation manager and configure it via the **correlationManager=#myManager** option.



NOTE

We recommend extending the **TimeoutCorrelationManagerSupport** when you build custom correlation managers. This provides support for timeout and other complexities you otherwise would need to implement as well.

You can find an example with the Apache Camel source code in the examples directory under the **camel-example-netty-custom-correlation** directory.

104.13. SPRING BOOT AUTO-CONFIGURATION

The component supports 74 options, which are listed below.

Name	Description	Default	Type
camel.component.netty.allow-default-codec	The netty component installs a default codec if both, encoder/decoder is null and textline is false. Setting allowDefaultCodec to false prevents the netty component from installing a default codec as the first element in the filter chain.	true	Boolean
camel.component.netty.allow-serialized-headers	Only used for TCP when transferExchange is true. When set to true, serializable objects in headers and properties will be added to the exchange. Otherwise Camel will exclude any non-serializable objects and log it at WARN level.	false	Boolean
camel.component.netty.auto-append-delimiter	Whether or not to auto append missing end delimiter when sending using the textline codec.	true	Boolean
camel.component.netty.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.netty.backlog	Allows to configure a backlog for netty consumer (server). Note the backlog is just a best effort depending on the OS. Setting this option to a value such as 200, 500 or 1000, tells the TCP stack how long the accept queue can be. If this option is not configured, then the backlog depends on OS setting.		Integer
camel.component.netty.boss-count	When netty works on nio mode, it uses default bossCount parameter from Netty, which is 1. User can use this option to override the default bossCount from Netty.	1	Integer
camel.component.netty.boss-group	Set the BossGroup which could be used for handling the new connection of the server side across the NettyEndpoint. The option is a io.netty.channel.EventLoopGroup type.		EventLoopGroup

Name	Description	Default	Type
camel.component.netty.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.netty.broadcast	Setting to choose Multicast over UDP.	false	Boolean
camel.component.netty.channel-group	To use a explicit ChannelGroup. The option is a <code>io.netty.channel.group.ChannelGroup</code> type.		ChannelGroup
camel.component.netty.client-initializer-factory	To use a custom ClientInitializerFactory. The option is a <code>org.apache.camel.component.netty.ClientInitializerFactory</code> type.		ClientInitializerFactory
camel.component.netty.client-mode	If the <code>clientMode</code> is true, netty consumer will connect the address as a TCP client.	false	Boolean
camel.component.netty.configuration	To use the <code>NettyConfiguration</code> as configuration when creating endpoints. The option is a <code>org.apache.camel.component.netty.NettyConfiguration</code> type.		NettyConfiguration
camel.component.netty.connect-timeout	Time to wait for a socket connection to be available. Value is in milliseconds.	10000	Integer

Name	Description	Default	Type
camel.component.netty.correlation-manager	To use a custom correlation manager to manage how request and reply messages are mapped when using request/reply with the netty producer. This should only be used if you have a way to map requests together with replies such as if there is correlation ids in both the request and reply messages. This can be used if you want to multiplex concurrent messages on the same channel (aka connection) in netty. When doing this you must have a way to correlate the request and reply messages so you can store the right reply on the inflight Camel Exchange before its continued routed. We recommend extending the TimeoutCorrelationManagerSupport when you build custom correlation managers. This provides support for timeout and other complexities you otherwise would need to implement as well. See also the producerPoolEnabled option for more details. The option is a <code>org.apache.camel.component.netty.NettyCamelStateCorrelationManager</code> type.		NettyCamelStateCorrelationManager
camel.component.netty.decoder-max-line-length	The max line length to use for the textline codec.	1024	Integer
camel.component.netty.decoders	A list of decoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		String
camel.component.netty.delimiter	The delimiter to use for the textline codec. Possible values are LINE and NULL.		TextLineDelimiter
camel.component.netty.disconnect	Whether or not to disconnect(close) from Netty Channel right after use. Can be used for both consumer and producer.	false	Boolean
camel.component.netty.disconnect-on-no-reply	If sync is enabled then this option dictates NettyConsumer if it should disconnect where there is no reply to send back.	true	Boolean
camel.component.netty.enabled	Whether to enable auto configuration of the netty component. This is enabled by default.		Boolean
camel.component.netty.enabled-protocols	Which protocols to enable when using SSL.	TLSv1, TLSv1.1, TLSv1.2	String

Name	Description	Default	Type
camel.component.netty.encoders	A list of encoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		String
camel.component.netty.encoding	The encoding (a charset name) to use for the textline codec. If not provided, Camel will use the JVM default Charset.		String
camel.component.netty.executor-service	To use the given EventExecutorGroup. The option is a io.netty.util.concurrent.EventExecutorGroup type.		EventExecutorGroup
camel.component.netty.hostname-verification	To enable/disable hostname verification on SSLEngine.	false	Boolean
camel.component.netty.keep-alive	Setting to ensure socket is not closed due to inactivity.	true	Boolean
camel.component.netty.key-store-file	Client side certificate keystore to be used for encryption.		File
camel.component.netty.key-store-format	Keystore format to be used for payload encryption. Defaults to JKS if not set.		String
camel.component.netty.key-store-resource	Client side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
camel.component.netty.lazy-channel-creation	Channels can be lazily created to avoid exceptions, if the remote server is not up and running when the Camel producer is started.	true	Boolean

Name	Description	Default	Type
camel.component.netty.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.netty.maximum-pool-size	Sets a maximum thread pool size for the netty consumer ordered thread pool. The default size is 2 x cpu_core plus 1. Setting this value to eg 10 will then use 10 threads unless 2 x cpu_core plus 1 is a higher value, which then will override and be used. For example if there are 8 cores, then the consumer thread pool will be 17. This thread pool is used to route messages received from Netty by Camel. We use a separate thread pool to ensure ordering of messages and also in case some messages will block, then netty's worker threads (event loop) won't be affected.		Integer
camel.component.netty.native-transport	Whether to use native transport instead of NIO. Native transport takes advantage of the host operating system and is only supported on some platforms. You need to add the netty JAR for the host operating system you are using. See more details at: .	false	Boolean
camel.component.netty.need-client-auth	Configures whether the server needs client authentication when using SSL.	false	Boolean
camel.component.netty.netty-server-bootstrap-factory	To use a custom NettyServerBootstrapFactory. The option is a org.apache.camel.component.netty.NettyServerBootstrapFactory type.		NettyServerBootstrapFactory
camel.component.netty.network-interface	When using UDP then this option can be used to specify a network interface by its name, such as eth0 to join a multicast group.		String
camel.component.netty.no-reply-log-level	If sync is enabled this option dictates NettyConsumer which logging level to use when logging a there is no reply to send back.		LogLevel

Name	Description	Default	Type
camel.component.netty.options	Allows to configure additional netty options using option. as prefix. For example option.child.keepAlive=false to set the netty option child.keepAlive=false. See the Netty documentation for possible options that can be used.		Map
camel.component.netty.passphrase	Password setting to use in order to encrypt/decrypt payloads sent using SSH.		String
camel.component.netty.producer-pool-enabled	Whether producer pool is enabled or not. Important: If you turn this off then a single shared connection is used for the producer, also if you are doing request/reply. That means there is a potential issue with interleaved responses if replies comes back out-of-order. Therefore you need to have a correlation id in both the request and reply messages so you can properly correlate the replies to the Camel callback that is responsible for continue processing the message in Camel. To do this you need to implement NettyCamelStateCorrelationManager as correlation manager and configure it via the correlationManager option. See also the correlationManager option for more details.	true	Boolean
camel.component.netty.producer-pool-max-idle	Sets the cap on the number of idle instances in the pool.	100	Integer
camel.component.netty.producer-pool-max-total	Sets the cap on the number of objects that can be allocated by the pool (checked out to clients, or idle awaiting checkout) at a given time. Use a negative value for no limit.	-1	Integer
camel.component.netty.producer-pool-min-evictable-idle	Sets the minimum amount of time (value in millis) an object may sit idle in the pool before it is eligible for eviction by the idle object evictor.	30000 0	Long
camel.component.netty.producer-pool-min-idle	Sets the minimum number of instances allowed in the producer pool before the evictor thread (if active) spawns new objects.		Integer
camel.component.netty.receive-buffer-size	The TCP/UDP buffer sizes to be used during inbound communication. Size is bytes.	65536	Integer

Name	Description	Default	Type
camel.component.netty.receive-buffer-size-predictor	Configures the buffer size predictor. See details at Jetty documentation and this mail thread.		Integer
camel.component.netty.reconnect	Used only in clientMode in consumer, the consumer will attempt to reconnect on disconnection if this is enabled.	true	Boolean
camel.component.netty.reconnect-interval	Used if reconnect and clientMode is enabled. The interval in milli seconds to attempt reconnection.	10000	Integer
camel.component.netty.request-timeout	Allows to use a timeout for the Netty producer when calling a remote server. By default no timeout is in use. The value is in milli seconds, so eg 30000 is 30 seconds. The requestTimeout is using Netty's ReadTimeoutHandler to trigger the timeout.		Long
camel.component.netty.reuse-address	Setting to facilitate socket multiplexing.	true	Boolean
camel.component.netty.reuse-channel	This option allows producers and consumers (in client mode) to reuse the same Netty Channel for the lifecycle of processing the Exchange. This is useful if you need to call a server multiple times in a Camel route and want to use the same network connection. When using this, the channel is not returned to the connection pool until the Exchange is done; or disconnected if the disconnect option is set to true. The reused Channel is stored on the Exchange as an exchange property with the key <code>NettyConstants#NETTY_CHANNEL</code> which allows you to obtain the channel during routing and use it as well.	false	Boolean
camel.component.netty.security-provider	Security provider to be used for payload encryption. Defaults to SunX509 if not set.		String
camel.component.netty.send-buffer-size	The TCP/UDP buffer sizes to be used during outbound communication. Size is bytes.	65536	Integer

Name	Description	Default	Type
camel.component.netty.server-closed-channel-exception-caught-log-level	If the server (NettyConsumer) catches an <code>java.nio.channels.ClosedChannelException</code> then its logged using this logging level. This is used to avoid logging the closed channel exceptions, as clients can disconnect abruptly and then cause a flood of closed exceptions in the Netty server.		LogLevel
camel.component.netty.server-exception-caught-log-level	If the server (NettyConsumer) catches an exception then its logged using this logging level.		LogLevel
camel.component.netty.server-initializer-factory	To use a custom <code>ServerInitializerFactory</code> . The option is a <code>org.apache.camel.component.netty.ServerInitializerFactory</code> type.		<code>ServerInitializerFactory</code>
camel.component.netty.ssl	Setting to specify whether SSL encryption is applied to this endpoint.	false	Boolean
camel.component.netty.ssl-client-cert-headers	When enabled and in SSL mode, then the Netty consumer will enrich the Camel Message with headers having information about the client certificate such as subject name, issuer name, serial number, and the valid date range.	false	Boolean
camel.component.netty.ssl-context-parameters	To configure security using <code>SSLContextParameters</code> . The option is a <code>org.apache.camel.support.jsse.SSLContextParameters</code> type.		<code>SSLContextParameters</code>
camel.component.netty.ssl-handler	Reference to a class that could be used to return an SSL Handler. The option is a <code>io.netty.handler.ssl.SslHandler</code> type.		<code>SslHandler</code>
camel.component.netty.sync	Setting to set endpoint as one-way or request-response.	true	Boolean
camel.component.netty.tcp-no-delay	Setting to improve TCP protocol performance.	true	Boolean
camel.component.netty.textline	Only used for TCP. If no codec is specified, you can use this flag to indicate a text line based codec; if not specified or the value is false, then Object Serialization is assumed over TCP - however only Strings are allowed to be serialized by default.	false	Boolean

Name	Description	Default	Type
camel.component.netty.transfer-exchange	Only used for TCP. You can transfer the exchange over the wire instead of just the body. The following fields are transferred: In body, Out body, fault body, In headers, Out headers, fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	Boolean
camel.component.netty.trust-store-file	Server side certificate keystore to be used for encryption.		File
camel.component.netty.trust-store-resource	Server side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
camel.component.netty.udp-byte-array-codec	For UDP only. If enabled the using byte array codec instead of Java serialization protocol.	false	Boolean
camel.component.netty.udp-connectionless-sending	This option supports connection less udp sending which is a real fire and forget. A connected udp send receive the PortUnreachableException if no one is listen on the receiving port.	false	Boolean
camel.component.netty.use-byte-buf	If the useByteBuf is true, netty producer will turn the message body into ByteBuf before sending it out.	false	Boolean
camel.component.netty.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean
camel.component.netty.using-executor-service	Whether to use ordered thread pool, to ensure events are processed orderly on the same channel.	true	Boolean
camel.component.netty.worker-count	When netty works on nio mode, it uses default workerCount parameter from Netty (which is <code>cpu_core_threads x 2</code>). User can use this option to override the default workerCount from Netty.		Integer

Name	Description	Default	Type
camel.component.netty.worker-group	To use a explicit EventLoopGroup as the boss thread pool. For example to share a thread pool with multiple consumers or producers. By default each consumer or producer has their own worker pool with 2 x cpu count core threads. The option is a io.netty.channel.EventLoopGroup type.		EventLoopGroup

CHAPTER 105. NETTY HTTP

Since Camel 2.14

Both producer and consumer are supported

The Netty HTTP component is an extension to [Netty](#) component to facilitate HTTP transport with Netty.

NOTE

Stream

Netty is stream based, which means the input it receives is submitted to Camel as a stream. That means you will only be able to read the content of the stream **once**. If you find a situation where the message body appears to be empty or you need to access the data multiple times (eg: doing multicasting, or redelivery error handling) you should use Stream caching or convert the message body to a **String** which is safe to be re-read multiple times. Notice Netty HTTP reads the entire stream into memory using **io.netty.handler.codec.http.HttpObjectAggregator** to build the entire full http message. But the resulting message is still a stream based message which is readable once.

105.1. DEPENDENCIES

When using **netty-http** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-netty-http-starter</artifactId>
</dependency>
```

105.2. URI FORMAT

The URI scheme for a netty component is as follows

```
netty-http:http://0.0.0.0:8080[?options]
```

NOTE

Query parameters vs endpoint options

You may be wondering how Camel recognizes URI query parameters and endpoint options. For example you might create endpoint URI as follows: **netty-http:http://example.com?**

myParam=myValue&compression=true. In this example **myParam** is the HTTP parameter, while **compression** is the Camel endpoint option. The strategy used by Camel in such situations is to resolve available endpoint options and remove them from the URI. It means that for the discussed example, the HTTP request sent by Netty HTTP producer to the endpoint will look as follows: **http://example.com?myParam=myValue**, because **compression** endpoint option will be resolved and removed from the target URL. Keep also in mind that you cannot specify endpoint options using dynamic headers (like **CamelHttpRequest**). Endpoint options can be specified only at the endpoint URI definition level (like **to** or **from** DSL elements).



IMPORTANT

This component inherits all the options from [Netty](#). Notice that some options from Netty are not applicable when using this Netty HTTP component, such as options related to UDP transport.

105.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

105.3.1. Configuring component options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

105.3.2. Configuring endpoint options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

105.4. COMPONENT OPTIONS

The Netty HTTP component supports 80 options, which are listed below.

Name	Description	Default	Type
configuration (common)	To use the NettyConfiguration as configuration when creating endpoints.		NettyConfiguratio n

Name	Description	Default	Type
disconnect (common)	Whether or not to disconnect(close) from Netty Channel right after use. Can be used for both consumer and producer.	false	boolean
keepAlive (common)	Setting to ensure socket is not closed due to inactivity.	true	boolean
reuseAddress (common)	Setting to facilitate socket multiplexing.	true	boolean
reuseChannel (common)	This option allows producers and consumers (in client mode) to reuse the same Netty Channel for the lifecycle of processing the Exchange. This is useful if you need to call a server multiple times in a Camel route and want to use the same network connection. When using this, the channel is not returned to the connection pool until the Exchange is done; or disconnected if the disconnect option is set to true. The reused Channel is stored on the Exchange as an exchange property with the key <code>NettyConstants#NETTY_CHANNEL</code> which allows you to obtain the channel during routing and use it as well.	false	boolean
sync (common)	Setting to set endpoint as one-way or request-response.	true	boolean
tcpNoDelay (common)	Setting to improve TCP protocol performance.	true	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
broadcast (consumer)	Setting to choose Multicast over UDP.	false	boolean
clientMode (consumer)	If the clientMode is true, netty consumer will connect the address as a TCP client.	false	boolean

Name	Description	Default	Type
muteException (consumer)	If enabled and an Exchange failed processing on the consumer side the response's body won't contain the exception's stack trace.	false	boolean
reconnect (consumer)	Used only in clientMode in consumer, the consumer will attempt to reconnect on disconnection if this is enabled.	true	boolean
reconnectInterval (consumer)	Used if reconnect and clientMode is enabled. The interval in milli seconds to attempt reconnection.	10000	int
backlog (consumer (advanced))	Allows to configure a backlog for netty consumer (server). Note the backlog is just a best effort depending on the OS. Setting this option to a value such as 200, 500 or 1000, tells the TCP stack how long the accept queue can be. If this option is not configured, then the backlog depends on OS setting.		int
bossCount (consumer (advanced))	When netty works on nio mode, it uses default bossCount parameter from Netty, which is 1. User can use this option to override the default bossCount from Netty.	1	int
bossGroup (consumer (advanced))	Set the BossGroup which could be used for handling the new connection of the server side across the NettyEndpoint.		EventLoopGroup
disconnectOnNoReply (consumer (advanced))	If sync is enabled then this option dictates NettyConsumer if it should disconnect where there is no reply to send back.	true	boolean
executorService (consumer (advanced))	To use the given EventExecutorGroup.		EventExecutorGroup
maximumPoolSize (consumer (advanced))	Sets a maximum thread pool size for the netty consumer ordered thread pool. The default size is 2 x cpu_core plus 1. Setting this value to eg 10 will then use 10 threads unless 2 x cpu_core plus 1 is a higher value, which then will override and be used. For example if there are 8 cores, then the consumer thread pool will be 17. This thread pool is used to route messages received from Netty by Camel. We use a separate thread pool to ensure ordering of messages and also in case some messages will block, then netty's worker threads (event loop) won't be affected.		int

Name	Description	Default	Type
nettyServerBootstrapFactory (consumer (advanced))	To use a custom NettyServerBootstrapFactory.		NettyServerBootstrapFactory
networkInterface (consumer (advanced))	When using UDP then this option can be used to specify a network interface by its name, such as eth0 to join a multicast group.		String
noReplyLogLevel (consumer (advanced))	If sync is enabled this option dictates NettyConsumer which logging level to use when logging a there is no reply to send back. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	WARN	LogLevel
serverClosedChannelExceptionCaughtLogLevel (consumer (advanced))	If the server (NettyConsumer) catches an java.nio.channels.ClosedChannelException then its logged using this logging level. This is used to avoid logging the closed channel exceptions, as clients can disconnect abruptly and then cause a flood of closed exceptions in the Netty server. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	DEBUG	LogLevel

Name	Description	Default	Type
serverExceptionHandlerLogLevel (consumer (advanced))	If the server (NettyConsumer) catches an exception then its logged using this logging level. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	WARN	LogLevel
serverInitializerFactory (consumer (advanced))	To use a custom ServerInitializerFactory.		ServerInitializerFactory
usingExecutorService (consumer (advanced))	Whether to use ordered thread pool, to ensure events are processed orderly on the same channel.	true	boolean
connectTimeout (producer)	Time to wait for a socket connection to be available. Value is in milliseconds.	10000	int
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
requestTimeout (producer)	Allows to use a timeout for the Netty producer when calling a remote server. By default no timeout is in use. The value is in milli seconds, so eg 30000 is 30 seconds. The requestTimeout is using Netty's ReadTimeoutHandler to trigger the timeout.		long
clientInitializerFactory (producer (advanced))	To use a custom ClientInitializerFactory.		ClientInitializerFactory

Name	Description	Default	Type
correlationManager (producer (advanced))	To use a custom correlation manager to manage how request and reply messages are mapped when using request/reply with the netty producer. This should only be used if you have a way to map requests together with replies such as if there is correlation ids in both the request and reply messages. This can be used if you want to multiplex concurrent messages on the same channel (aka connection) in netty. When doing this you must have a way to correlate the request and reply messages so you can store the right reply on the inflight Camel Exchange before its continued routed. We recommend extending the TimeoutCorrelationManagerSupport when you build custom correlation managers. This provides support for timeout and other complexities you otherwise would need to implement as well. See also the producerPoolEnabled option for more details.		NettyCamelStateCorrelationManager
lazyChannelCreation (producer (advanced))	Channels can be lazily created to avoid exceptions, if the remote server is not up and running when the Camel producer is started.	true	boolean
producerPoolBlockWhenExhausted (producer (advanced))	Sets the value for the blockWhenExhausted configuration attribute. It determines whether to block when the borrowObject() method is invoked when the pool is exhausted (the maximum number of active objects has been reached).	true	boolean
producerPoolEnabled (producer (advanced))	Whether producer pool is enabled or not. Important: If you turn this off then a single shared connection is used for the producer, also if you are doing request/reply. That means there is a potential issue with interleaved responses if replies comes back out-of-order. Therefore you need to have a correlation id in both the request and reply messages so you can properly correlate the replies to the Camel callback that is responsible for continue processing the message in Camel. To do this you need to implement NettyCamelStateCorrelationManager as correlation manager and configure it via the correlationManager option. See also the correlationManager option for more details.	true	boolean
producerPoolMaxIdle (producer (advanced))	Sets the cap on the number of idle instances in the pool.	100	int

Name	Description	Default	Type
producerPoolMaxTotal (producer (advanced))	Sets the cap on the number of objects that can be allocated by the pool (checked out to clients, or idle awaiting checkout) at a given time. Use a negative value for no limit.	-1	int
producerPoolMaxWait (producer (advanced))	Sets the maximum duration (value in millis) the borrowObject() method should block before throwing an exception when the pool is exhausted and producerPoolBlockWhenExhausted is true. When less than 0, the borrowObject() method may block indefinitely.	-1	long
producerPoolMinEvictableIdle (producer (advanced))	Sets the minimum amount of time (value in millis) an object may sit idle in the pool before it is eligible for eviction by the idle object evictor.	30000 0	long
producerPoolMinIdle (producer (advanced))	Sets the minimum number of instances allowed in the producer pool before the evictor thread (if active) spawns new objects.		int
udpConnectionlessSending (producer (advanced))	This option supports connection less udp sending which is a real fire and forget. A connected udp send receive the PortUnreachableException if no one is listen on the receiving port.	false	boolean
useByteBuf (producer (advanced))	If the useByteBuf is true, netty producer will turn the message body into ByteBuf before sending it out.	false	boolean
hostnameVerification (security)	To enable/disable hostname verification on SSLEngine.	false	boolean
allowSerializedHeaders (advanced)	Only used for TCP when transferExchange is true. When set to true, serializable objects in headers and properties will be added to the exchange. Otherwise Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

Name	Description	Default	Type
channelGroup (advanced)	To use a explicit ChannelGroup.		ChannelGroup
headerFilterStrategy (advanced)	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter headers.		HeaderFilterStrategy
nativeTransport (advanced)	Whether to use native transport instead of NIO. Native transport takes advantage of the host operating system and is only supported on some platforms. You need to add the netty JAR for the host operating system you are using. See more details at: .	false	boolean
nettyHttpBinding (advanced)	To use a custom org.apache.camel.component.netty.http.NettyHttpBinding for binding to/from Netty and Camel Message API.		NettyHttpBinding
options (advanced)	Allows to configure additional netty options using option. as prefix. For example option.child.keepAlive=false to set the netty option child.keepAlive=false. See the Netty documentation for possible options that can be used.		Map
receiveBufferSize (advanced)	The TCP/UDP buffer sizes to be used during inbound communication. Size is bytes.	65536	int
receiveBufferSizePredictor (advanced)	Configures the buffer size predictor. See details at Jetty documentation and this mail thread.		int
sendBufferSize (advanced)	The TCP/UDP buffer sizes to be used during outbound communication. Size is bytes.	65536	int
transferExchange (advanced)	Only used for TCP. You can transfer the exchange over the wire instead of just the body. The following fields are transferred: In body, Out body, fault body, In headers, Out headers, fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
udpByteArrayCodec (advanced)	For UDP only. If enabled the using byte array codec instead of Java serialization protocol.	false	boolean

Name	Description	Default	Type
unixDomainSocketPath (advanced)	Path to unix domain socket to use instead of inet socket. Host and port parameters will not be used, however required. It is ok to set dummy values for them. Must be used with nativeTransport=true and clientMode=false.		String
workerCount (advanced)	When netty works on nio mode, it uses default workerCount parameter from Netty (which is cpu_core_threads x 2). User can use this option to override the default workerCount from Netty.		int
workerGroup (advanced)	To use a explicit EventLoopGroup as the boss thread pool. For example to share a thread pool with multiple consumers or producers. By default each consumer or producer has their own worker pool with 2 x cpu count core threads.		EventLoopGroup
allowDefaultCodec (codec)	The netty component installs a default codec if both, encoder/decoder is null and textline is false. Setting allowDefaultCodec to false prevents the netty component from installing a default codec as the first element in the filter chain.	true	boolean
autoAppendDelimiter (codec)	Whether or not to auto append missing end delimiter when sending using the textline codec.	true	boolean
decoderMaxLineLength (codec)	The max line length to use for the textline codec.	1024	int
decoders (codec)	A list of decoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		String
delimiter (codec)	The delimiter to use for the textline codec. Possible values are LINE and NULL. Enum values: • LINE • NULL	LINE	TextLineDelimiter

Name	Description	Default	Type
encoders (codec)	A list of encoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		String
encoding (codec)	The encoding (a charset name) to use for the textline codec. If not provided, Camel will use the JVM default Charset.		String
textline (codec)	Only used for TCP. If no codec is specified, you can use this flag to indicate a text line based codec; if not specified or the value is false, then Object Serialization is assumed over TCP - however only Strings are allowed to be serialized by default.	false	boolean
enabledProtocols (security)	Which protocols to enable when using SSL.	TLSv1.2, TLSv1.3	String
keyStoreFile (security)	Client side certificate keystore to be used for encryption.		File
keyStoreFormat (security)	Keystore format to be used for payload encryption. Defaults to JKS if not set.		String
keyStoreResource (security)	Client side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
needClientAuth (security)	Configures whether the server needs client authentication when using SSL.	false	boolean
passphrase (security)	Password setting to use in order to encrypt/decrypt payloads sent using SSH.		String
securityConfiguration (security)	Refers to a org.apache.camel.component.netty.http.NettyHttpSecurityConfiguration for configuring secure web resources.		NettyHttpSecurityConfiguration
securityProvider (security)	Security provider to be used for payload encryption. Defaults to SunX509 if not set.		String
ssl (security)	Setting to specify whether SSL encryption is applied to this endpoint.	false	boolean

Name	Description	Default	Type
sslClientCertHeaders (security)	When enabled and in SSL mode, then the Netty consumer will enrich the Camel Message with headers having information about the client certificate such as subject name, issuer name, serial number, and the valid date range.	false	boolean
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters
sslHandler (security)	Reference to a class that could be used to return an SSL Handler.		SslHandler
trustStoreFile (security)	Server side certificate keystore to be used for encryption.		File
trustStoreResource (security)	Server side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean

105.5. ENDPOINT OPTIONS

The Netty HTTP endpoint is configured using URI syntax:

```
netty-http:protocol://host:port/path
```

with the following path and query parameters:

105.5.1. Path Parameters (4 parameters)

Name	Description	Default	Type
protocol (common)	Required The protocol to use which is either http, https or proxy – a consumer only option. Enum values: • http • https		String

Name	Description	Default	Type
host (common)	Required The local hostname such as localhost, or 0.0.0.0 when being a consumer. The remote HTTP server hostname when using producer.		String
port (common)	The host port number.		int
path (common)	Resource path.		String

105.5.2. Query Parameters (85 parameters)

Name	Description	Default	Type
bridgeEndpoint (common)	If the option is true, the producer will ignore the <code>NettyHttpConstants.HTTP_URI</code> header, and use the endpoint's URI for request. You may also set the <code>throwExceptionOnFailure</code> to be false to let the producer send all the fault response back. The consumer working in the bridge mode will skip the gzip compression and WWW URL form encoding (by adding the <code>Exchange.SKIP_GZIP_ENCODING</code> and <code>Exchange.SKIP_WWW_FORM_URL_ENCODED</code> headers to the consumed exchange).	false	boolean
disconnect (common)	Whether or not to disconnect(close) from Netty Channel right after use. Can be used for both consumer and producer.	false	boolean
keepAlive (common)	Setting to ensure socket is not closed due to inactivity.	true	boolean
reuseAddress (common)	Setting to facilitate socket multiplexing.	true	boolean
reuseChannel (common)	This option allows producers and consumers (in client mode) to reuse the same Netty Channel for the lifecycle of processing the Exchange. This is useful if you need to call a server multiple times in a Camel route and want to use the same network connection. When using this, the channel is not returned to the connection pool until the Exchange is done; or disconnected if the disconnect option is set to true. The reused Channel is stored on the Exchange as an exchange property with the key <code>NettyConstants#NETTY_CHANNEL</code> which allows you to obtain the channel during routing and use it as well.	false	boolean

Name	Description	Default	Type
sync (common)	Setting to set endpoint as one-way or request-response.	true	boolean
tcpNoDelay (common)	Setting to improve TCP protocol performance.	true	boolean
matchOnUriPrefix (consumer)	Whether or not Camel should try to find a target consumer by matching the URI prefix if no exact match is found.	false	boolean
muteException (consumer)	If enabled and an Exchange failed processing on the consumer side the response's body won't contain the exception's stack trace.	false	boolean
send503whenSuspended (consumer)	Whether to send back HTTP status code 503 when the consumer has been suspended. If the option is false then the Netty Acceptor is unbound when the consumer is suspended, so clients cannot connect anymore.	true	boolean
backlog (consumer (advanced))	Allows to configure a backlog for netty consumer (server). Note the backlog is just a best effort depending on the OS. Setting this option to a value such as 200, 500 or 1000, tells the TCP stack how long the accept queue can be. If this option is not configured, then the backlog depends on OS setting.		int
bossCount (consumer (advanced))	When netty works on nio mode, it uses default bossCount parameter from Netty, which is 1. User can use this option to override the default bossCount from Netty.	1	int
bossGroup (consumer (advanced))	Set the BossGroup which could be used for handling the new connection of the server side across the NettyEndpoint.		EventLoopGroup
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
chunkedMaxContentLength (consumer (advanced))	Value in bytes the max content length per chunked frame received on the Netty HTTP server.	1048576	int
compression (consumer (advanced))	Allow using gzip/deflate for compression on the Netty HTTP server if the client supports it from the HTTP headers.	false	boolean
disconnectOnNoReply (consumer (advanced))	If sync is enabled then this option dictates NettyConsumer if it should disconnect where there is no reply to send back.	true	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut		ExchangePattern
httpMethodRestrict (consumer (advanced))	To disable HTTP methods on the Netty HTTP consumer. You can specify multiple separated by comma.		String
logWarnOnBadRequest (consumer (advanced))	Whether Netty HTTP server should log a WARN if decoding the HTTP request failed and a HTTP Status 400 (bad request) is returned.	true	boolean
mapHeaders (consumer (advanced))	If this option is enabled, then during binding from Netty to Camel Message then the headers will be mapped as well (eg added as header to the Camel Message as well). You can turn off this option to disable this. The headers can still be accessed from the org.apache.camel.component.netty.http.NettyHttpRequestMessage message with the method getHttpRequest() that returns the Netty HTTP request io.netty.handler.codec.http.HttpRequest instance.	true	boolean

Name	Description	Default	Type
maxChunkSize (consumer (advanced))	The maximum length of the content or each chunk. If the content length (or the length of each chunk) exceeds this value, the content or chunk will be split into multiple <code>io.netty.handler.codec.http.HttpContents</code> whose length is <code>maxChunkSize</code> at maximum. See <code>io.netty.handler.codec.http.HttpObjectDecoder</code> .	8192	int
maxHeaderSize (consumer (advanced))	The maximum length of all headers. If the sum of the length of each header exceeds this value, a <code>io.netty.handler.codec.TooLongFrameException</code> will be raised.	8192	int
maxInitialLineLength (consumer (advanced))	The maximum length of the initial line (e.g. <code>GET / HTTP/1.0</code> or <code>HTTP/1.0 200 OK</code>) If the length of the initial line exceeds this value, a <code>TooLongFrameException</code> will be raised. See <code>io.netty.handler.codec.http.HttpObjectDecoder</code> .	4096	int
nettyServerBootstrapFactory (consumer (advanced))	To use a custom <code>NettyServerBootstrapFactory</code> .		<code>NettyServerBootstrapFactory</code>
nettySharedHttpServer (consumer (advanced))	To use a shared Netty HTTP server. See Netty HTTP Server Example for more details.		<code>NettySharedHttpServer</code>
noReplyLogLevel (consumer (advanced))	If sync is enabled this option dictates <code>NettyConsumer</code> which logging level to use when logging a there is no reply to send back. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	WARN	LogLevel

Name	Description	Default	Type
serverClosedChannelExceptionHandlerLogLevel (consumer (advanced))	If the server (NettyConsumer) catches an <code>java.nio.channels.ClosedChannelException</code> then its logged using this logging level. This is used to avoid logging the closed channel exceptions, as clients can disconnect abruptly and then cause a flood of closed exceptions in the Netty server. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	DEBUG	LoggingLevel
serverExceptionHandlerLogLevel (consumer (advanced))	If the server (NettyConsumer) catches an exception then its logged using this logging level. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	WARN	LoggingLevel
serverInitializerFactory (consumer (advanced))	To use a custom <code>ServerInitializerFactory</code> .		<code>ServerInitializerFactory</code>
traceEnabled (consumer (advanced))	Specifies whether to enable HTTP TRACE for this Netty HTTP consumer. By default TRACE is turned off.	false	boolean

Name	Description	Default	Type
urlDecodeHeaders (consumer (advanced))	If this option is enabled, then during binding from Netty to Camel Message then the header values will be URL decoded (eg %20 will be a space character. Notice this option is used by the default <code>org.apache.camel.component.netty.http.NettyHttpBinding</code> and therefore if you implement a custom <code>org.apache.camel.component.netty.http.NettyHttpBinding</code> then you would need to decode the headers accordingly to this option.	false	boolean
usingExecutorService (consumer (advanced))	Whether to use ordered thread pool, to ensure events are processed orderly on the same channel.	true	boolean
connectTimeout (producer)	Time to wait for a socket connection to be available. Value is in milliseconds.	10000	int
cookieHandler (producer)	Configure a cookie handler to maintain a HTTP session.		CookieHandler
requestTimeout (producer)	Allows to use a timeout for the Netty producer when calling a remote server. By default no timeout is in use. The value is in milli seconds, so eg 30000 is 30 seconds. The requestTimeout is using Netty's <code>ReadTimeoutHandler</code> to trigger the timeout.		long
throwExceptionOnFailure (producer)	Option to disable throwing the <code>HttpOperationFailedException</code> in case of failed responses from the remote server. This allows you to get all responses regardless of the HTTP status code.	true	boolean
clientInitializerFactory (producer (advanced))	To use a custom <code>ClientInitializerFactory</code> .		ClientInitializerFactory
lazyChannelCreation (producer (advanced))	Channels can be lazily created to avoid exceptions, if the remote server is not up and running when the Camel producer is started.	true	boolean

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
okStatusCodeRange (producer (advanced))	The status codes which are considered a success response. The values are inclusive. Multiple ranges can be defined, separated by comma, e.g. 200-204,209,301-304. Each range must be a single number or from-to with the dash included. The default range is 200-299.	200-299	String
producerPoolBlockWhenExhausted (producer (advanced))	Sets the value for the blockWhenExhausted configuration attribute. It determines whether to block when the borrowObject() method is invoked when the pool is exhausted (the maximum number of active objects has been reached).	true	boolean
producerPoolEnabled (producer (advanced))	Whether producer pool is enabled or not. Important: If you turn this off then a single shared connection is used for the producer, also if you are doing request/reply. That means there is a potential issue with interleaved responses if replies comes back out-of-order. Therefore you need to have a correlation id in both the request and reply messages so you can properly correlate the replies to the Camel callback that is responsible for continue processing the message in Camel. To do this you need to implement NettyCamelStateCorrelationManager as correlation manager and configure it via the correlationManager option. See also the correlationManager option for more details.	true	boolean
producerPoolMaxIdle (producer (advanced))	Sets the cap on the number of idle instances in the pool.	100	int
producerPoolMaxTotal (producer (advanced))	Sets the cap on the number of objects that can be allocated by the pool (checked out to clients, or idle awaiting checkout) at a given time. Use a negative value for no limit.	-1	int

Name	Description	Default	Type
producerPoolMaxWait (producer (advanced))	Sets the maximum duration (value in millis) the borrowObject() method should block before throwing an exception when the pool is exhausted and producerPoolBlockWhenExhausted is true. When less than 0, the borrowObject() method may block indefinitely.	-1	long
producerPoolMinEvictableIdle (producer (advanced))	Sets the minimum amount of time (value in millis) an object may sit idle in the pool before it is eligible for eviction by the idle object evictor.	30000 0	long
producerPoolMinIdle (producer (advanced))	Sets the minimum number of instances allowed in the producer pool before the evictor thread (if active) spawns new objects.		int
useRelativePath (producer (advanced))	Sets whether to use a relative path in HTTP requests.	true	boolean
hostnameVerification (security)	To enable/disable hostname verification on SSLEngine.	false	boolean
allowSerializedHeaders (advanced)	Only used for TCP when transferExchange is true. When set to true, serializable objects in headers and properties will be added to the exchange. Otherwise Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
channelGroup (advanced)	To use a explicit ChannelGroup.		ChannelGroup
configuration (advanced)	To use a custom configured NettyHttpConfiguration for configuring this endpoint.		NettyHttpConfiguration

Name	Description	Default	Type
disableStreamCache (advanced)	Determines whether or not the raw input stream from <code>Netty HttpRequest#getContent()</code> or <code>HttpResponset#getContent()</code> is cached or not (Camel will read the stream into a in light-weight memory based Stream caching) cache. By default Camel will cache the Netty input stream to support reading it multiple times to ensure it Camel can retrieve all data from the stream. However you can set this option to true when you for example need to access the raw stream, such as streaming it directly to a file or other persistent store. Mind that if you enable this option, then you cannot read the Netty stream multiple times out of the box, and you would need manually to reset the reader index on the Netty raw stream. Also Netty will auto-close the Netty stream when the Netty HTTP server/HTTP client is done processing, which means that if the asynchronous routing engine is in use then any asynchronous thread that may continue routing the <code>org.apache.camel.Exchange</code> may not be able to read the Netty stream, because Netty has closed it.	false	boolean
headerFilterStrategy (advanced)	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter headers.		HeaderFilterStrategy
nativeTransport (advanced)	Whether to use native transport instead of NIO. Native transport takes advantage of the host operating system and is only supported on some platforms. You need to add the netty JAR for the host operating system you are using. See more details at: .	false	boolean
nettyHttpBinding (advanced)	To use a custom <code>org.apache.camel.component.netty.http.NettyHttpBinding</code> for binding to/from Netty and Camel Message API.		NettyHttpBinding
options (advanced)	Allows to configure additional netty options using option. as prefix. For example <code>option.child.keepAlive=false</code> to set the netty option <code>child.keepAlive=false</code> . See the Netty documentation for possible options that can be used.		Map
receiveBufferSize (advanced)	The TCP/UDP buffer sizes to be used during inbound communication. Size is bytes.	65536	int

Name	Description	Default	Type
receiveBufferSizePredictor (advanced)	Configures the buffer size predictor. See details at Jetty documentation and this mail thread.		int
sendBufferSize (advanced)	The TCP/UDP buffer sizes to be used during outbound communication. Size is bytes.	65536	int
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean
transferException (advanced)	If enabled and an Exchange failed processing on the consumer side, and if the caused Exception was send back serialized in the response as a application/x-java-serialized-object content type. On the producer side the exception will be deserialized and thrown as is, instead of the <code>HttpOperationFailedException</code> . The caused exception is required to be serialized. This is by default turned off. If you enable this then be aware that Java will deserialize the incoming data from the request to Java and that can be a potential security risk.	false	boolean
transferExchange (advanced)	Only used for TCP. You can transfer the exchange over the wire instead of just the body. The following fields are transferred: In body, Out body, fault body, In headers, Out headers, fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	boolean
unixDomainSocketPath (advanced)	Path to unix domain socket to use instead of inet socket. Host and port parameters will not be used, however required. It is ok to set dummy values for them. Must be used with <code>nativeTransport=true</code> and <code>clientMode=false</code> .		String
workerCount (advanced)	When netty works on nio mode, it uses default workerCount parameter from Netty (which is <code>cpu_core_threads x 2</code>). User can use this option to override the default workerCount from Netty.		int
workerGroup (advanced)	To use a explicit EventLoopGroup as the boss thread pool. For example to share a thread pool with multiple consumers or producers. By default each consumer or producer has their own worker pool with 2 x cpu count core threads.		EventLoopGroup

Name	Description	Default	Type
decoders (codec)	A list of decoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		String
encoders (codec)	A list of encoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		String
enabledProtocols (security)	Which protocols to enable when using SSL.	TLSv1.2, TLSv1.3	String
keyStoreFile (security)	Client side certificate keystore to be used for encryption.		File
keyStoreFormat (security)	Keystore format to be used for payload encryption. Defaults to JKS if not set.		String
keyStoreResource (security)	Client side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
needClientAuth (security)	Configures whether the server needs client authentication when using SSL.	false	boolean
passphrase (security)	Password setting to use in order to encrypt/decrypt payloads sent using SSH.		String
securityConfiguration (security)	Refers to a org.apache.camel.component.netty.http.NettyHttpSecurityConfiguration for configuring secure web resources.		NettyHttpSecurityConfiguration
securityOptions (security)	To configure NettyHttpSecurityConfiguration using key/value pairs from the map.		Map
securityProvider (security)	Security provider to be used for payload encryption. Defaults to SunX509 if not set.		String
ssl (security)	Setting to specify whether SSL encryption is applied to this endpoint.	false	boolean

Name	Description	Default	Type
sslClientCertHeaders (security)	When enabled and in SSL mode, then the Netty consumer will enrich the Camel Message with headers having information about the client certificate such as subject name, issuer name, serial number, and the valid date range.	false	boolean
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters
sslHandler (security)	Reference to a class that could be used to return an SSL Handler.		SslHandler
trustStoreFile (security)	Server side certificate keystore to be used for encryption.		File
trustStoreResource (security)	Server side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String

105.6. MESSAGE HEADERS

The Netty HTTP component supports 23 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelHttpAuthentication (common) Constant: HTTP_AUTHENTICATION	If the user was authenticated using HTTP Basic then this header is added with the value Basic.		String
Content-Type (common) Constant: CONTENT_TYPE	To set the content-type of the HTTP body. For example: text/plain; charset=UTF-8.		String
connection (common) Constant: CONNECTION	The value of the HTTP header connection to use.		String

Name	Description	Default	Type
CamelNettyCloseChannelWhenComplete (common) Constant: NETTY_CLOSE_CHANNEL_WHEN_COMPLETE	Indicates whether the channel should be closed after complete.		Boolean
CamelHttpResponseCode (common) Constant: HTTP_RESPONSE_CODE	Allows to set the HTTP Status code to use. By default 200 is used for success, and 500 for failure.		Integer
CamelHttpProtocolVersion (common) Constant: HTTP_PROTOCOL_VERSION	The version of the HTTP protocol.	HTTP/1.1	String
CamelHttpMethod (common) Constant: HTTP_METHOD	The HTTP method used, such as GET, POST, TRACE etc.	GET	String
CamelHttpQuery (common) Constant: HTTP_QUERY	Any query parameters, such as foo=bar&beer=yes.		String
CamelHttpPath (common) Constant: HTTP_PATH	Allows to provide URI context-path and query parameters as a String value that overrides the endpoint configuration. This allows to reuse the same producer for calling same remote http server, but using a dynamic context-path and query parameters.		String
CamelHttpRawQuery (common) Constant: HTTP_RAW_QUERY	Any query parameters, such as foo=bar&beer=yes. Stored in the raw form, as they arrived to the consumer (i.e. before URL decoding).		String

Name	Description	Default	Type
CamelHttpUrl (common) Constant: HTTP_URL	The URL including protocol, host and port, etc: http://0.0.0.0:8080/myapp .		String
CamelHttpCharacterEncoding (common) Constant: HTTP_CHARACTER_ENCODING	The charset from the content-type header.		String
CamelHttpUri (common) Constant: HTTP_URI	The URI without protocol, host and port, etc: /myapp.		String
CamelNettyChannelHandlerContext (common) Constant: NETTY_CHANNEL_HANDLER_CONTEXT	The channel handler context.		ChannelHandlerContext
CamelNettyRemoteAddress (common) Constant: NETTY_REMOTE_ADDRESS	The remote address.		SocketAddress
CamelNettyLocalAddress (common) Constant: NETTY_LOCAL_ADDRESS	The local address.		SocketAddress

Name	Description	Default	Type
CamelNettySSLSession (common) Constant: NETTY_SSL_SESSION	The SSL session.		SSLSession
CamelNettySSLClientCertSubjectName (common) Constant: NETTY_SSL_CLIENT_CERT_SUBJECT_NAME	The SSL client certificate subject name.		String
CamelNettySSLClientCertIssuerName (common) Constant: NETTY_SSL_CLIENT_CERT_ISSUER_NAME	The SSL client certificate issuer name.		String
CamelNettySSLClientCertSerialNumber (common) Constant: NETTY_SSL_CLIENT_CERT_SERIAL_NO	The SSL client certificate serial number.		String
CamelNettySSLClientCertNotBefore (common) Constant: NETTY_SSL_CLIENT_CERT_NOT_BEFORE	The SSL client certificate not before.		Date

Name	Description	Default	Type
CamelNettySSLClientCertNotAfter (common) Constant: NETTY_SSL_CLIENT_CERT_NOT_AFTER	The SSL client certificate not after.		Date
CamelNettyRequestTimeout (common) Constant: NETTY_REQUEST_TIMEOUT	The read timeout.		Long

105.7. ACCESS TO NETTY TYPES

This component uses the **org.apache.camel.component.netty.http.NettyHttpRequestMessage** as the message implementation on the Exchange. This allows end users to get access to the original Netty request/response instances if needed, as shown below. Mind that the original response may not be accessible at all times.

```
io.netty.handler.codec.http.HttpRequest request =
exchange.getIn(NettyHttpRequestMessage.class).getHttpRequest();
```

105.8. EXAMPLES

In the route below we use Netty HTTP as a HTTP server, which returns back a hardcoded "Bye World" message.

```
from("netty-http:http://0.0.0.0:8080/foo")
    .transform().constant("Bye World");
```

And we can call this HTTP server using Camel also, with the `ProducerTemplate` as shown below:

```
String out = template.requestBody("netty-http:http://0.0.0.0:8080/foo", "Hello World", String.class);
System.out.println(out);
```

And we get back "Bye World" as the output.

105.8.1. How do I let Netty match wildcards

By default Netty HTTP will only match on exact uri's. But you can instruct Netty to match prefixes. For example

```
from("netty-http:http://0.0.0.0:8123/foo").to("mock:foo");
```

In the route above Netty HTTP will only match if the uri is an exact match, so it will match if you enter <http://0.0.0.0:8123/foo> but not match if you do <http://0.0.0.0:8123/foo/bar>.

So if you want to enable wildcard matching you do as follows:

```
from("netty-http:http://0.0.0.0:8123/foo?matchOnUriPrefix=true").to("mock:foo");
```

So now Netty matches any endpoints with starts with **foo**.

To match **any** endpoint you can do:

```
from("netty-http:http://0.0.0.0:8123?matchOnUriPrefix=true").to("mock:foo");
```

105.8.2. Using multiple routes with same port

In the same CamelContext you can have multiple routes from Netty HTTP that shares the same port (eg a **io.netty.bootstrap.ServerBootstrap** instance). Doing this requires a number of bootstrap options to be identical in the routes, as the routes will share the same **io.netty.bootstrap.ServerBootstrap** instance. The instance will be configured with the options from the first route created.

The options the routes must be identical configured is all the options defined in the **org.apache.camel.component.netty.NettyServerBootstrapConfiguration** configuration class. If you have configured another route with different options, Camel will throw an exception on startup, indicating the options is not identical. To mitigate this ensure all options is identical.

Here is an example with two routes that share the same port.

Two routes sharing the same port

```
from("netty-http:http://0.0.0.0:{{port}}/foo")
    .to("mock:foo")
    .transform().constant("Bye World");

from("netty-http:http://0.0.0.0:{{port}}/bar")
    .to("mock:bar")
    .transform().constant("Bye Camel");
```

And here is an example of a misconfigured 2nd route that do not have identical **org.apache.camel.component.netty.NettyServerBootstrapConfiguration** option as the 1st route. This will cause Camel to fail on startup.

Two routes sharing the same port, but the 2nd route is misconfigured

and will fail on starting

```
from("netty-http:http://0.0.0.0:{{port}}/foo")
    .to("mock:foo")
    .transform().constant("Bye World");

// we cannot have a 2nd route on same port with SSL enabled, when the 1st route is NOT
from("netty-http:http://0.0.0.0:{{port}}/bar?ssl=true")
    .to("mock:bar")
    .transform().constant("Bye Camel");
```

105.8.3. Reusing same server bootstrap configuration with multiple routes

By configuring the common server bootstrap option in a single instance of a **org.apache.camel.component.netty.NettyServerBootstrapConfiguration** type, we can use the **bootstrapConfiguration** option on the Netty HTTP consumers to refer and reuse the same options across all consumers.

```
<bean id="nettyHttpBootstrapOptions"
class="org.apache.camel.component.netty.NettyServerBootstrapConfiguration">
  <property name="backlog" value="200"/>
  <property name="connectionTimeout" value="20000"/>
  <property name="workerCount" value="16"/>
</bean>
```

And in the routes you refer to this option as shown below

```
<route>
  <from uri="netty-http:http://0.0.0.0:{{port}}/foo?
bootstrapConfiguration=#nettyHttpBootstrapOptions"/>
  ...
</route>

<route>
  <from uri="netty-http:http://0.0.0.0:{{port}}/bar?
bootstrapConfiguration=#nettyHttpBootstrapOptions"/>
  ...
</route>

<route>
  <from uri="netty-http:http://0.0.0.0:{{port}}/beer?
bootstrapConfiguration=#nettyHttpBootstrapOptions"/>
  ...
</route>
```

105.8.4. Reusing same server bootstrap configuration with multiple routes across multiple bundles in OSGi container

See the above Netty HTTP Server Example for more details and example how to do that.

105.8.5. Implementing a reverse proxy

Netty HTTP component can act as a reverse proxy, in that case **Exchange.HTTP_SCHEME**, **Exchange.HTTP_HOST** and **Exchange.HTTP_PORT** headers are populated from the absolute URL received on the request line of the HTTP request.

Here's an example of a HTTP proxy that simply transforms the response from the origin server to uppercase.

```
from("netty-http:proxy://0.0.0.0:8080")
.toD("netty-http:"
  + "${headers." + Exchange.HTTP_SCHEME + "}://"
  + "${headers." + Exchange.HTTP_HOST + "}:"
  + "${headers." + Exchange.HTTP_PORT +}")
.process(this::processResponse);
```

```

void processResponse(final Exchange exchange) {
    final NettyHttpMessage message = exchange.getIn(NettyHttpMessage.class);
    final FullHttpResponse response = message.getHttpResponse();

    final ByteBuf buf = response.content();
    final String string = buf.toString(StandardCharsets.UTF_8);

    buf.resetWriterIndex();
    ByteBufUtil.writeUtf8(buf, string.toUpperCase(Locale.US));
}

```

105.9. USING HTTP BASIC AUTHENTICATION

The Netty HTTP consumer supports HTTP basic authentication by specifying the security realm name to use, as shown below

```

<route>
  <from uri="netty-http:http://0.0.0.0:{{port}}/foo?securityConfiguration.realm=karaf"/>
  ...
</route>

```

The realm name is mandatory to enable basic authentication. By default the JAAS based authenticator is used, which will use the realm name specified (karaf in the example above) and use the JAAS realm and the JAAS `\\{{LoginModule}}`s of this realm for authentication.

End user of Apache Karaf / ServiceMix has a karaf realm out of the box, and hence why the example above would work out of the box in these containers.

105.9.1. Specifying ACL on web resources

The **org.apache.camel.component.netty.http.SecurityConstraint** allows to define constraints on web resources. The **org.apache.camel.component.netty.http.SecurityConstraintMapping** is provided out of the box, allowing to easily define inclusions and exclusions with roles.

For example as shown below in the XML DSL, we define the constraint bean:

```

<bean id="constraint" class="org.apache.camel.component.netty.http.SecurityConstraintMapping">
  <!-- inclusions defines url -> roles restrictions -->
  <!-- a * should be used for any role accepted (or even no roles) -->
  <property name="inclusions">
    <map>
      <entry key="/*" value="*" />
      <entry key="/admin/*" value="admin" />
      <entry key="/guest/*" value="admin,guest" />
    </map>
  </property>
  <!-- exclusions is used to define public urls, which requires no authentication -->
  <property name="exclusions">
    <set>
      <value>/public/*</value>
    </set>
  </property>
</bean>

```

The constraint above is define so that

- access to /* is restricted and any roles is accepted (also if user has no roles)
- access to /admin/* requires the admin role
- access to /guest/* requires the admin or guest role
- access to /public/* is an exclusion which means no authentication is needed, and is therefore public for everyone without logging in

To use this constraint we just need to refer to the bean id as shown below:

```
<route>
  <from uri="netty-http:http://0.0.0.0:{{port}}/foo?
matchOnUriPrefix=true&securityConfiguration.realm=karaf&securityConfiguration.securityCon
straint=#constraint"/>
  ...
</route>
```

105.10. SPRING BOOT AUTO-CONFIGURATION

The component supports 67 options, which are listed below.

Name	Description	Default	Type
camel.component.netty-http.allow-serialized-headers	Only used for TCP when transferExchange is true. When set to true, serializable objects in headers and properties will be added to the exchange. Otherwise Camel will exclude any non-serializable objects and log it at WARN level.	false	Boolean
camel.component.netty-http.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.netty-http.backlog	Allows to configure a backlog for netty consumer (server). Note the backlog is just a best effort depending on the OS. Setting this option to a value such as 200, 500 or 1000, tells the TCP stack how long the accept queue can be. If this option is not configured, then the backlog depends on OS setting.		Integer
camel.component.netty-http.boss-count	When netty works on nio mode, it uses default bossCount parameter from Netty, which is 1. User can use this option to override the default bossCount from Netty.	1	Integer

Name	Description	Default	Type
camel.component.netty-http.boss-group	Set the BossGroup which could be used for handling the new connection of the server side across the NettyEndpoint. The option is a <code>io.netty.channel.EventLoopGroup</code> type.		<code>EventLoopGroup</code>
camel.component.netty-http.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.netty-http.channel-group	To use a explicit ChannelGroup. The option is a <code>io.netty.channel.group.ChannelGroup</code> type.		<code>ChannelGroup</code>
camel.component.netty-http.client-initializer-factory	To use a custom ClientInitializerFactory. The option is a <code>org.apache.camel.component.netty.ClientInitializerFactory</code> type.		<code>ClientInitializerFactory</code>
camel.component.netty-http.configuration	To use the NettyConfiguration as configuration when creating endpoints. The option is a <code>org.apache.camel.component.netty.NettyConfiguration</code> type.		<code>NettyConfiguration</code>
camel.component.netty-http.connect-timeout	Time to wait for a socket connection to be available. Value is in milliseconds.	10000	Integer

Name	Description	Default	Type
camel.component.netty-http.correlation-manager	To use a custom correlation manager to manage how request and reply messages are mapped when using request/reply with the netty producer. This should only be used if you have a way to map requests together with replies such as if there is correlation ids in both the request and reply messages. This can be used if you want to multiplex concurrent messages on the same channel (aka connection) in netty. When doing this you must have a way to correlate the request and reply messages so you can store the right reply on the inflight Camel Exchange before its continued routed. We recommend extending the TimeoutCorrelationManagerSupport when you build custom correlation managers. This provides support for timeout and other complexities you otherwise would need to implement as well. See also the producerPoolEnabled option for more details. The option is a <code>org.apache.camel.component.netty.NettyCamelStateCorrelationManager</code> type.		NettyCamelStateCorrelationManager
camel.component.netty-http.decoders	A list of decoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		String
camel.component.netty-http.disconnect	Whether or not to disconnect(close) from Netty Channel right after use. Can be used for both consumer and producer.	false	Boolean
camel.component.netty-http.disconnect-on-no-reply	If sync is enabled then this option dictates NettyConsumer if it should disconnect where there is no reply to send back.	true	Boolean
camel.component.netty-http.enabled	Whether to enable auto configuration of the netty-http component. This is enabled by default.		Boolean
camel.component.netty-http.enabled-protocols	Which protocols to enable when using SSL.	TLSv1.2, TLSv1.3	String

Name	Description	Default	Type
camel.component.netty-http.encoders	A list of encoders to be used. You can use a String which have values separated by comma, and have the values be looked up in the Registry. Just remember to prefix the value with # so Camel knows it should lookup.		String
camel.component.netty-http.executor-service	To use the given EventExecutorGroup. The option is a io.netty.util.concurrent.EventExecutorGroup type.		EventExecutorGroup
camel.component.netty-http.header-filter-strategy	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter headers. The option is a org.apache.camel.spi.HeaderFilterStrategy type.		HeaderFilterStrategy
camel.component.netty-http.hostname-verification	To enable/disable hostname verification on SSLEngine.	false	Boolean
camel.component.netty-http.keep-alive	Setting to ensure socket is not closed due to inactivity.	true	Boolean
camel.component.netty-http.key-store-file	Client side certificate keystore to be used for encryption.		File
camel.component.netty-http.key-store-format	Keystore format to be used for payload encryption. Defaults to JKS if not set.		String
camel.component.netty-http.key-store-resource	Client side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
camel.component.netty-http.lazy-channel-creation	Channels can be lazily created to avoid exceptions, if the remote server is not up and running when the Camel producer is started.	true	Boolean

Name	Description	Default	Type
camel.component.netty-http.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.netty-http.maximum-pool-size	Sets a maximum thread pool size for the netty consumer ordered thread pool. The default size is 2 x cpu_core plus 1. Setting this value to eg 10 will then use 10 threads unless 2 x cpu_core plus 1 is a higher value, which then will override and be used. For example if there are 8 cores, then the consumer thread pool will be 17. This thread pool is used to route messages received from Netty by Camel. We use a separate thread pool to ensure ordering of messages and also in case some messages will block, then netty's worker threads (event loop) won't be affected.		Integer
camel.component.netty-http.mute-exception	If enabled and an Exchange failed processing on the consumer side the response's body won't contain the exception's stack trace.	false	Boolean
camel.component.netty-http.native-transport	Whether to use native transport instead of NIO. Native transport takes advantage of the host operating system and is only supported on some platforms. You need to add the netty JAR for the host operating system you are using. See more details at: .	false	Boolean
camel.component.netty-http.need-client-auth	Configures whether the server needs client authentication when using SSL.	false	Boolean
camel.component.netty-http.netty-binding	To use a custom org.apache.camel.component.netty.http.NettyHttpBinding for binding to/from Netty and Camel Message API. The option is a org.apache.camel.component.netty.http.NettyHttpBinding type.		NettyHttpBinding

Name	Description	Default	Type
camel.component.netty-http.netty-server-bootstrap-factory	To use a custom NettyServerBootstrapFactory. The option is a <code>org.apache.camel.component.netty.NettyServerBootstrapFactory</code> type.		NettyServerBootstrapFactory
camel.component.netty-http.no-reply-log-level	If sync is enabled this option dictates NettyConsumer which logging level to use when logging a there is no reply to send back.		LogLevel
camel.component.netty-http.options	Allows to configure additional netty options using option. as prefix. For example <code>option.child.keepAlive=false</code> to set the netty option <code>child.keepAlive=false</code> . See the Netty documentation for possible options that can be used.		Map
camel.component.netty-http.passphrase	Password setting to use in order to encrypt/decrypt payloads sent using SSH.		String
camel.component.netty-http.producer-pool-block-when-exhausted	Sets the value for the <code>blockWhenExhausted</code> configuration attribute. It determines whether to block when the <code>borrowObject()</code> method is invoked when the pool is exhausted (the maximum number of active objects has been reached).	true	Boolean
camel.component.netty-http.producer-pool-enabled	Whether producer pool is enabled or not. Important: If you turn this off then a single shared connection is used for the producer, also if you are doing request/reply. That means there is a potential issue with interleaved responses if replies comes back out-of-order. Therefore you need to have a correlation id in both the request and reply messages so you can properly correlate the replies to the Camel callback that is responsible for continue processing the message in Camel. To do this you need to implement <code>NettyCamelStateCorrelationManager</code> as correlation manager and configure it via the <code>correlationManager</code> option. See also the <code>correlationManager</code> option for more details.	true	Boolean
camel.component.netty-http.producer-pool-max-idle	Sets the cap on the number of idle instances in the pool.	100	Integer

Name	Description	Default	Type
<code>camel.component.netty-http.producer-pool-max-total</code>	Sets the cap on the number of objects that can be allocated by the pool (checked out to clients, or idle awaiting checkout) at a given time. Use a negative value for no limit.	-1	Integer
<code>camel.component.netty-http.producer-pool-max-wait</code>	Sets the maximum duration (value in millis) the <code>borrowObject()</code> method should block before throwing an exception when the pool is exhausted and <code>producerPoolBlockWhenExhausted</code> is true. When less than 0, the <code>borrowObject()</code> method may block indefinitely.	-1	Long
<code>camel.component.netty-http.producer-pool-min-evictable-idle</code>	Sets the minimum amount of time (value in millis) an object may sit idle in the pool before it is eligible for eviction by the idle object evictor.	30000 0	Long
<code>camel.component.netty-http.producer-pool-min-idle</code>	Sets the minimum number of instances allowed in the producer pool before the evictor thread (if active) spawns new objects.		Integer
<code>camel.component.netty-http.receive-buffer-size</code>	The TCP/UDP buffer sizes to be used during inbound communication. Size is bytes.	65536	Integer
<code>camel.component.netty-http.receive-buffer-size-predictor</code>	Configures the buffer size predictor. See details at Jetty documentation and this mail thread.		Integer
<code>camel.component.netty-http.request-timeout</code>	Allows to use a timeout for the Netty producer when calling a remote server. By default no timeout is in use. The value is in milli seconds, so eg 30000 is 30 seconds. The <code>requestTimeout</code> is using Netty's <code>ReadTimeoutHandler</code> to trigger the timeout.		Long
<code>camel.component.netty-http.reuse-address</code>	Setting to facilitate socket multiplexing.	true	Boolean

Name	Description	Default	Type
camel.component.netty-http.reuse-channel	This option allows producers and consumers (in client mode) to reuse the same Netty Channel for the lifecycle of processing the Exchange. This is useful if you need to call a server multiple times in a Camel route and want to use the same network connection. When using this, the channel is not returned to the connection pool until the Exchange is done; or disconnected if the disconnect option is set to true. The reused Channel is stored on the Exchange as an exchange property with the key <code>NettyConstants#NETTY_CHANNEL</code> which allows you to obtain the channel during routing and use it as well.	false	Boolean
camel.component.netty-http.security-configuration	Refers to a <code>org.apache.camel.component.netty.http.NettyHttpSecurityConfiguration</code> for configuring secure web resources. The option is a <code>org.apache.camel.component.netty.http.NettyHttpSecurityConfiguration</code> type.		NettyHttpSecurityConfiguration
camel.component.netty-http.security-provider	Security provider to be used for payload encryption. Defaults to SunX509 if not set.		String
camel.component.netty-http.send-buffer-size	The TCP/UDP buffer sizes to be used during outbound communication. Size is bytes.	65536	Integer
camel.component.netty-http.server-closed-channel-exception-caught-log-level	If the server (NettyConsumer) catches an <code>java.nio.channels.ClosedChannelException</code> then its logged using this logging level. This is used to avoid logging the closed channel exceptions, as clients can disconnect abruptly and then cause a flood of closed exceptions in the Netty server.		LogLevel
camel.component.netty-http.server-exception-caught-log-level	If the server (NettyConsumer) catches an exception then its logged using this logging level.		LogLevel
camel.component.netty-http.server-initializer-factory	To use a custom <code>ServerInitializerFactory</code> . The option is a <code>org.apache.camel.component.netty.ServerInitializerFactory</code> type.		ServerInitializerFactory

Name	Description	Default	Type
camel.component.netty-http.ssl	Setting to specify whether SSL encryption is applied to this endpoint.	false	Boolean
camel.component.netty-http.ssl-client-cert-headers	When enabled and in SSL mode, then the Netty consumer will enrich the Camel Message with headers having information about the client certificate such as subject name, issuer name, serial number, and the valid date range.	false	Boolean
camel.component.netty-http.ssl-context-parameters	To configure security using SSLContextParameters. The option is a <code>org.apache.camel.support.jsse.SSLContextParameters</code> type.		SSLContextParameters
camel.component.netty-http.ssl-handler	Reference to a class that could be used to return an SSL Handler. The option is a <code>io.netty.handler.ssl.SslHandler</code> type.		SslHandler
camel.component.netty-http.sync	Setting to set endpoint as one-way or request-response.	true	Boolean
camel.component.netty-http.tcp-no-delay	Setting to improve TCP protocol performance.	true	Boolean
camel.component.netty-http.transfer-exchange	Only used for TCP. You can transfer the exchange over the wire instead of just the body. The following fields are transferred: In body, Out body, fault body, In headers, Out headers, fault headers, exchange properties, exchange exception. This requires that the objects are serializable. Camel will exclude any non-serializable objects and log it at WARN level.	false	Boolean
camel.component.netty-http.trust-store-file	Server side certificate keystore to be used for encryption.		File
camel.component.netty-http.trust-store-resource	Server side certificate keystore to be used for encryption. Is loaded by default from classpath, but you can prefix with <code>classpath:</code> , <code>file:</code> , or <code>http:</code> to load the resource from different systems.		String

Name	Description	Default	Type
camel.component.netty-http.unix-domain-socket-path	Path to unix domain socket to use instead of inet socket. Host and port parameters will not be used, however required. It is ok to set dummy values for them. Must be used with nativeTransport=true and clientMode=false.		String
camel.component.netty-http.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean
camel.component.netty-http.using-executor-service	Whether to use ordered thread pool, to ensure events are processed orderly on the same channel.	true	Boolean
camel.component.netty-http.worker-count	When netty works on nio mode, it uses default workerCount parameter from Netty (which is cpu_core_threads x 2). User can use this option to override the default workerCount from Netty.		Integer
camel.component.netty-http.worker-group	To use a explicit EventLoopGroup as the boss thread pool. For example to share a thread pool with multiple consumers or producers. By default each consumer or producer has their own worker pool with 2 x cpu count core threads. The option is a io.netty.channel.EventLoopGroup type.		EventLoopGroup

CHAPTER 106. OBSERVABILITY SERVICES

The Camel Observability Services component provides a set of opinionated components and configuration which simplify operations such as observability on cloud environments. Although the component is mainly targeted for cloud, it can be used in any other environment, giving to the Camel application the ability to expose a set of observability features by default.

106.1. DEPENDENCIES

When using **camel-observability-services** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-observability-services-starter</artifactId>
</dependency>
```

106.2. USAGE

106.2.1. Auto-detection from classpath

All you need to do is to add the **camel-observability-services** dependency to the classpath. There's no need to add any further configuration. Each component will be configured using each own default setting, except the endpoint which will be exposed in **/observe/<service>** by default.

If you need to customize each of the different components provided within this service, then, you can specify in the **application.properties** each of the configuration as it would be done normally when you provide the individual component.

NOTE: The customization of the configuration for this component is not available for Spring Boot runtime due to a [known limitation](#). You can use this component in Spring Boot runtime with the default settings only. If you need to provide any customization, you'll need to configure each component separately.

106.3. COMPONENTS AVAILABLE

The presence of this dependency will provide the following components:

- camel-health
- camel-management
- camel-micrometer-prometheus
- camel-opentelemetry

106.3.1. List of known endpoints

The presence of this dependency will expose the following endpoints:

Endpoint	Description
/observe/health	startup probe endpoint
/observe/health/live	liveness probe endpoint
/observe/health/ready	readiness probe endpoint
/observe/metrics	metrics exposed as in Micrometer Prometheus Registry

106.4. OPENTELEMETRY CONFIGURATION

The presence of this component will provide the required instrumentation to easily enable the collection of OpenTelemetry metrics.

106.4.1. Testing OpenTelemetry collector

If you need a quick way to verify the configuration of OpenTelemetry traces, you can start a local collector by running a Docker service:

```
docker run -p 4318:4318 otel/opentelemetry-collector-contrib:0.113.0
```

This service will expose the port **4318** to **localhost** which is the default setting expected by the agent. You can change this configuration accordingly.

106.4.2. OpenTelemetry Agent configuration

To collect the metrics exposed by the application, and, depending on the runtime of choice, you will need to start your Camel application with a Java agent. The Java agent goal is to push those metrics to the OpenTelemetry compatible collector server. Follow the instructions provided in [OpenTelemetry Java Agent configuration](#).

NOTE: Due to a [flaw in the alignment of OpenTelemetry dependencies in Spring Boot](#) runtime, you may hit a runtime error until the Spring Boot BOM dependencies is aligning the OpenTelemetry dependencies. Be mindful of the required workaround required for this to work until the issue is fixed.

106.5. JMX CONFIGURATION

The presence of this component implies the presence of **camel-management** component. This is in charge to include information about Camel application status in JMX format.

NOTE: The presence of this component automatically enables the collection of the JMX metrics. This should be negligible from performance point of view. However, you may want to disable that running the application with **-Dorg.apache.camel.jmx.disabled=true** JVM option.

106.5.1. Testing JMX

An easy way to test the JMX configuration is to run a JMX client such as **jconsole**. Execute it from the same machine where the Camel application you want to monitor is running, and you can quickly verify the

status of the application (selecting the local process). There is the possibility to [expose the JMX remotely](#) as well.

Exposing remote JMX would require some security setting in place to avoid disclosing sensitive information. Also mind that it may not be suitable for cloud development, i.e., Kubernetes, due to possible limitations within the binary protocols used by the JMX technology. In this case, it is recommended the usage of a JSR 160 compatible Java agent.

106.5.2. JMX Agent configuration

When dealing with JMX, you may want to expose the information available via HTTP(S) protocol which would make JMX more suitable to cloud-based development. A possible way to expose the information is to use a JSR 160 compatible Java agent, whose goal is to interact as an adapter interface towards JMX, exposing an HTTP-based service instead. Follow the instructions provided in [JMX Java Agent configuration](#) .

CHAPTER 107. OLINGO4

Since Camel 2.19

Both producer and consumer are supported

The Olingo4 component uses [Apache Olingo](#) version 4.0 APIs to interact with OData 4.0 compliant service. Since version 4.0, OData is an OASIS standard and a number of popular open source and commercial vendors and products support this protocol. A sample list of supporting products can be found on the [OData website](#).

The Olingo4 component supports reading entity sets, entities, simple and complex properties, counts, using custom and OData system query parameters. It supports updating entities and properties. It also supports submitting queries and change requests as a single OData batch operation.

The component supports configuring HTTP connection parameters and headers for OData service connection. This allows configuring use of SSL, OAuth2.0, etc. as required by the target OData service.

107.1. DEPENDENCIES

When using **camel-olingo4** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-olingo4-starter</artifactId>
</dependency>
```

107.2. URI FORMAT

```
olingo4://endpoint/<resource-path>?[options]
```

107.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

107.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).

- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

107.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

107.4. COMPONENT OPTIONS

The Olingo4 component supports 16 options, which are listed below.

Name	Description	Default	Type
configuration (common)	To use the shared configuration.		Olingo4Configuration
connectTimeout (common)	HTTP connection creation timeout in milliseconds, defaults to 30,000 (30 seconds).	30000	int
contentType (common)	Content-Type header value can be used to specify JSON or XML message format, defaults to application/json; charset=utf-8.	application/json; charset=utf-8	String
filterAlreadySeen (common)	Set this to true to filter out results that have already been communicated by this component.	false	boolean
httpHeaders (common)	Custom HTTP headers to inject into every request, this could include OAuth tokens, etc.		Map

Name	Description	Default	Type
proxy (common)	HTTP proxy server configuration.		HttpHost
serviceUri (common)	Target OData service base URI, for example, http://services.odata.org/OData/OData.svc .		String
socketTimeout (common)	HTTP request timeout in milliseconds, defaults to 30,000 (30 seconds).	30000	int
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
splitResult (consumer)	For endpoints that return an array or collection, a consumer endpoint will map every element to distinct messages, unless splitResult is set to false.	true	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
httpAsyncClientBuilder (advanced)	Custom HTTP async client builder for more complex HTTP client configuration, overrides connectionTimeout, socketTimeout, proxy and sslContext. Note that a socketTimeout MUST be specified in the builder, otherwise OData requests could block indefinitely.		HttpAsyncClientBuilder

Name	Description	Default	Type
httpClientBuilder (advanced)	Custom HTTP client builder for more complex HTTP client configuration, overrides connectionTimeout, socketTimeout, proxy and sslContext. Note that a socketTimeout MUST be specified in the builder, otherwise OData requests could block indefinitely.		HttpCli entBuil der
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLC ontextP aramet ers
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolea n

107.5. ENDPOINT OPTIONS

The Olingo4 endpoint is configured using URI syntax:

```
olingo4:apiName/methodName
```

With the following *path* and *query* parameters:

107.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
apiName (common)	Required What kind of operation to perform. Enum values: • DEFAULT		Olingo 4ApiNa me
methodName (common)	Required What sub operation to use for the selected operation.		String

107.5.2. Query Parameters (32 parameters)

Name	Description	Default	Type
connectTimeout (common)	HTTP connection creation timeout in milliseconds, defaults to 30,000 (30 seconds).	30000	int

Name	Description	Default	Type
contentType (common)	Content-Type header value can be used to specify JSON or XML message format, defaults to application/json; charset=utf-8.	application/json; charset=utf-8	String
filterAlreadySeen (common)	Set this to true to filter out results that have already been communicated by this component.	false	boolean
httpHeaders (common)	Custom HTTP headers to inject into every request, this could include OAuth tokens, etc.		Map
inBody (common)	Sets the name of a parameter to be passed in the exchange In Body.		String
proxy (common)	HTTP proxy server configuration.		HttpHost
serviceUri (common)	Target OData service base URI, for example, http://services.odata.org/OData/OData.svc .		String
socketTimeout (common)	HTTP request timeout in milliseconds, defaults to 30,000 (30 seconds).	30000	int
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
splitResult (consumer)	For endpoints that return an array or collection, a consumer endpoint will map every element to distinct messages, unless splitResult is set to false.	true	boolean
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an <code>Exchange</code> have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
httpAsyncClientBuilder (advanced)	Custom HTTP async client builder for more complex HTTP client configuration, overrides <code>connectionTimeout</code> , <code>socketTimeout</code> , <code>proxy</code> and <code>sslContext</code> . Note that a <code>socketTimeout</code> MUST be specified in the builder, otherwise OData requests could block indefinitely.		<code>HttpAsyncClientBuilder</code>
httpClientBuilder (advanced)	Custom HTTP client builder for more complex HTTP client configuration, overrides <code>connectionTimeout</code> , <code>socketTimeout</code> , <code>proxy</code> and <code>sslContext</code> . Note that a <code>socketTimeout</code> MUST be specified in the builder, otherwise OData requests could block indefinitely.		<code>HttpClientBuilder</code>
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the <code>backoffMultiplier</code> should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the <code>backoffMultiplier</code> should kick-in.		int

Name	Description	Default	Type
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map

Name	Description	Default	Type
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters

107.6. API PARAMETERS (1 APIS)

The Olingo4 endpoint is an API-based component and has additional parameters based on which API name and API method is used. The API name and API method is located in the endpoint URI as the **apiName/methodName** path parameters:

```
olingo4:apiName/methodName
```

There are 1 API names as listed in the table below:

API Name	Type	Description
API:DEFAULT	Both	Olingo4 Client Api Interface

Each API is documented in the following sections to come.

107.6.1. API: DEFAULT

Both producer and consumer are supported

The DEFAULT API is defined in the syntax as follows:

```
olingo4:DEFAULT/methodName?[parameters]
```

The 9 method(s) is(are) listed in the table below, followed by detailed syntax for each method. (API methods can have a shorthand *alias* name which can be used in the syntax instead of the name)

Method	Alias	Description
action		Calls a OData action
batch		Executes a batch request
create		Creates a new OData resource
delete		Deletes an OData resource and invokes callback with org
merge		Patches/merges an OData resource using HTTP MERGE
patch		Patches/merges an OData resource using HTTP PATCH
read		Reads an OData resource and invokes callback with appropriate result
update		Updates an OData resource
uread		Reads an OData resource and invokes callback with the unparsed input stream

107.6.1.1. Method action

Signatures:

- `void action(org.apache.olingo.commons.api.edm.Edm edm, String resourcePath, java.util.Map<String, String> endpointHttpHeaders, Object data, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler responseHandler);`

The olingo4/action API method has the parameters listed in the table below:

Parameter	Description	Type
data	Action data	Object
edm	Service Edm	Edm
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map

Parameter	Description	Type
resourcePath	Resource path to action	String
responseHandler	Org.apache.olingo.client.api.domain.ClientEntity callback handler	Olingo4ResponseHandler

107.6.1.2. Method batch

Signatures:

- `void batch(org.apache.olingo.commons.api.edm.Edm edm, java.util.Map<String, String> endpointHttpHeaders, Object data, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler<java.util.List<org.apache.came responseHandler>);`

The olingo4/batch API method has the parameters listed in the table below:

Parameter	Description	Type
data	Ordered org.apache.camel.component.olingo4.api.batch.Olingo4BatchRequest list	Object
edm	Service Edm	Edm
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map
responseHandler	Callback handler	Olingo4ResponseHandler

107.6.1.3. Method create

Signatures:

- `void create(org.apache.olingo.commons.api.edm.Edm edm, String resourcePath, java.util.Map<String, String> endpointHttpHeaders, Object data, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler responseHandler);`

The olingo4/create API method has the parameters listed in the table below:

Parameter	Description	Type
data	Request data	Object
edm	Service Edm	Edm

Parameter	Description	Type
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map
resourcePath	Resource path to create	String
responseHandler	Callback handler	Olingo4ResponseHandler

107.6.1.4. Method delete

Signatures:

- `void delete(String resourcePath, java.util.Map<String, String> endpointHttpHeaders, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler<org.apache.olingo.commons.api.http.HttpStatusCode> responseHandler);`

The olingo4/delete API method has the parameters listed in the table below:

Parameter	Description	Type
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map
resourcePath	Resource path for Entry	String
responseHandler	Org.apache.olingo.commons.api.http.HttpStatusCode callback handler	Olingo4ResponseHandler

107.6.1.5. Method merge

Signatures:

- `void merge(org.apache.olingo.commons.api.edm.Edm edm, String resourcePath, java.util.Map<String, String> endpointHttpHeaders, Object data, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler responseHandler);`

The olingo4/merge API method has the parameters listed in the table below:

Parameter	Description	Type
data	Patch/merge data	Object
edm	Service Edm	Edm
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map

Parameter	Description	Type
resourcePath	Resource path to update	String
responseHandler	Org.apache.olingo.client.api.domain.ClientEntity callback handler	Olingo4ResponseHandler

107.6.1.6. Method patch

Signatures:

- `void patch(org.apache.olingo.commons.api.edm.Edm edm, String resourcePath, java.util.Map<String, String> endpointHttpHeaders, Object data, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler responseHandler);`

The olingo4/patch API method has the parameters listed in the table below:

Parameter	Description	Type
data	Patch/merge data	Object
edm	Service Edm	Edm
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map
resourcePath	Resource path to update	String
responseHandler	Org.apache.olingo.client.api.domain.ClientEntity callback handler	Olingo4ResponseHandler

107.6.1.7. Method read

Signatures:

- `void read(org.apache.olingo.commons.api.edm.Edm edm, String resourcePath, java.util.Map<String, String> queryParams, java.util.Map<String, String> endpointHttpHeaders, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler responseHandler);`

The olingo4/read API method has the parameters listed in the table below:

Parameter	Description	Type
edm	Service Edm, read from calling read(null, \$metdata, null, responseHandler)	Edm

Parameter	Description	Type
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map
queryParams	OData query params http://docs.oasis-open.org/odata/odata/v4.0/odata-v4.0-part1-protocol.html#_Toc453752288	Map
resourcePath	OData Resource path	String
responseHandler	Callback handler	Olingo4ResponseHandler

107.6.1.8. Method update

Signatures:

- `void update(org.apache.olingo.commons.api.edm.Edm edm, String resourcePath, java.util.Map<String, String> endpointHttpHeaders, Object data, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler responseHandler);`

The `olingo4/update` API method has the parameters listed in the table below:

Parameter	Description	Type
data	Updated data	Object
edm	Service Edm	Edm
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map
resourcePath	Resource path to update	String
responseHandler	Org.apache.olingo.client.api.domain.ClientEntity callback handler	Olingo4ResponseHandler

107.6.1.9. Method uread

Signatures:

- `void uread(org.apache.olingo.commons.api.edm.Edm edm, String resourcePath, java.util.Map<String, String> queryParams, java.util.Map<String, String> endpointHttpHeaders, org.apache.camel.component.olingo4.api.Olingo4ResponseHandler<java.io.InputStream> responseHandler);`

The `olingo4/uread` API method has the parameters listed in the table below:

Parameter	Description	Type
edm	Service Edm, read from calling read(null, \$metdata, null, responseHandler)	Edm
endpointHttpHeaders	HTTP Headers to add/override the component versions	Map
queryParams	OData query params http://docs.oasis-open.org/odata/odata/v4.0/odata-v4.0-part1-protocol.html#_Toc453752288	Map
resourcePath	OData Resource path	String
responseHandler	Callback handler	Olingo4ResponseHandler

In addition to the parameters above, the olingo4 API can also use any of the Query Parameters listed above.

Any of the parameters can be provided in either the endpoint URI, or dynamically in a message header. The message header name must be of the format **CamelOlingo4.parameter**. The **inBody** parameter overrides message header, i.e., the endpoint parameter **inBody=myParameterNameHere** would override a **CamelOlingo4.myParameterNameHere** header.

107.7. MESSAGE HEADERS

The Olingo4 component supports 1 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelOlingo4.responseHttpHeaders (producer) Constant: FULL_RESPONSE_HTTP_HEADERS	The response Http headers.		Map

107.8. ENDPOINT HTTP HEADERS

The component level configuration property **httpHeaders** supplies static HTTP header information. However, some systems require dynamic header information to be passed to and received from the endpoint. A sample use case would be systems that require dynamic security tokens. The **endpointHttpHeaders** and **responseHttpHeaders** endpoint properties provide this capability. Set headers that need to be passed to the endpoint in the **CamelOlingo4.endpointHttpHeaders** property and the response headers will be returned in a **CamelOlingo4.responseHttpHeaders** property. Both properties are of the type **java.util.Map<String, String>**.

107.9. ODATA RESOURCE TYPE MAPPING

The result of **read** endpoint and data type of **data** option depends on the OData resource being queried, created or modified.

OData Resource Type	Resource URI from resourcePath and keyPredicate	In or Out Body Type
Entity data model	\$metadata	org.apache.olingo.commons.api.edm.Edm
Service document	/	org.apache.olingo.client.api.domain.ClientServiceDocument
OData entity set	<entity-set>	org.apache.olingo.client.api.domain.ClientEntitySet
OData entity	<entity-set>(<key-predicate>)	org.apache.olingo.client.api.domain.ClientEntity for Out body (response) java.util.Map<String, Object> for In body (request)
Simple property	<entity-set>(<key-predicate>)/<simple-property>	org.apache.olingo.client.api.domain.ClientPrimitiveValue
Simple property value	<entity-set>(<key-predicate>)/<simple-property>/\$value	org.apache.olingo.client.api.domain.ClientPrimitiveValue
Complex property	<entity-set>(<key-predicate>)/<complex-property>	org.apache.olingo.client.api.domain.ClientComplexValue
Count	<resource-uri>/\$count	java.lang.Long

107.10. SAMPLES

The following route reads top 5 entries from the People entity ordered by ascending FirstName property.

```
from("direct:...")
  .setHeader("CamelOlingo4.$top", "5");
  .to("olingo4://read/People?orderBy=FirstName%20asc");
```

The following route reads Airports entity using the key property value in incoming **id** header.

```
from("direct:...")
  .setHeader("CamelOlingo4.keyPredicate", header("id"))
  .to("olingo4://read/Airports");
```

The following route creates People entity using the **ClientEntity** in body message.

```
from("direct:...")
  .to("olingo4://create/People");
```

The following route calls an odata action using the **ClientEntity** in the body message. The body message may be null for actions that do not expect an input.

```
from("direct:...")
  .to("olingo4://action/People");
```

107.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 17 options, which are listed below.

Name	Description	Default	Type
camel.component.olingo4.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.olingo4.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.olingo4.configuration	To use the shared configuration. The option is a org.apache.camel.component.olingo4.Olingo4Configuration type.		Olingo4Configuration
camel.component.olingo4.connection-timeout	HTTP connection creation timeout in milliseconds, defaults to 30,000 (30 seconds).	30000	Integer

Name	Description	Default	Type
camel.component.olingo4.content-type	Content-Type header value can be used to specify JSON or XML message format, defaults to application/json;charset=utf-8.	application/json;charset=utf-8	String
camel.component.olingo4.enabled	Whether to enable auto configuration of the olingo4 component. This is enabled by default.		Boolean
camel.component.olingo4.filter-already-seen	Set this to true to filter out results that have already been communicated by this component.	false	Boolean
camel.component.olingo4.http-async-client-builder	Custom HTTP async client builder for more complex HTTP client configuration, overrides connectionTimeout, socketTimeout, proxy and sslContext. Note that a socketTimeout MUST be specified in the builder, otherwise OData requests could block indefinitely. The option is a org.apache.http.impl.nio.client.HttpAsyncClientBuilder type.		HttpAsyncClientBuilder
camel.component.olingo4.http-client-builder	Custom HTTP client builder for more complex HTTP client configuration, overrides connectionTimeout, socketTimeout, proxy and sslContext. Note that a socketTimeout MUST be specified in the builder, otherwise OData requests could block indefinitely. The option is a org.apache.http.impl.client.HttpClientBuilder type.		HttpClientBuilder
camel.component.olingo4.http-headers	Custom HTTP headers to inject into every request, this could include OAuth tokens, etc.		Map
camel.component.olingo4.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.olingo4.proxy</code>	HTTP proxy server configuration. The option is a <code>org.apache.http.HttpHost</code> type.		<code>HttpHost</code>
<code>camel.component.olingo4.service-uri</code>	Target OData service base URI, e.g. .		<code>String</code>
<code>camel.component.olingo4.socket-timeout</code>	HTTP request timeout in milliseconds, defaults to 30,000 (30 seconds).	30000	<code>Integer</code>
<code>camel.component.olingo4.split-result</code>	For endpoints that return an array or collection, a consumer endpoint will map every element to distinct messages, unless <code>splitResult</code> is set to false.	true	<code>Boolean</code>
<code>camel.component.olingo4.ssl-context-parameters</code>	To configure security using <code>SSLContextParameters</code> . The option is a <code>org.apache.camel.support.jsse.SSLContextParameters</code> type.		<code>SSLContextParameters</code>
<code>camel.component.olingo4.use-global-ssl-context-parameters</code>	Enable usage of global SSL context parameters.	false	<code>Boolean</code>

CHAPTER 108. OPENAPI JAVA

The Rest DSL can be integrated with the **camel-openapi-java** module which is used for exposing the REST services and their APIs using [OpenApi](#).

The camel-openapi-java module can be used from the REST components (without the need for servlet).

108.1. DEPENDENCIES

When using **openapi-java** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-openapi-java-starter</artifactId>
</dependency>
```

108.2. USING OPENAPI IN REST-DSL

You can enable the OpenApi api from the rest-dsl by configuring the **apiContextPath** dsl as shown below:

```
public class UserRouteBuilder extends RouteBuilder {
    @Override
    public void configure() throws Exception {
        // configure we want to use servlet as the component for the rest DSL
        // and we enable json binding mode
        restConfiguration().component("netty-http").bindingMode(RestBindingMode.json)
            // and output using pretty print
            .dataFormatProperty("prettyPrint", "true")
            // setup context path and port number that netty will use
            .contextPath("/").port(8080)
            // add OpenApi api-doc out of the box
            .apiContextPath("/api-doc")
            .apiProperty("api.title", "User API").apiProperty("api.version", "1.2.3")
            // and enable CORS
            .apiProperty("cors", "true");

        // this user REST service is json only
        rest("/user").description("User rest service")
            .consumes("application/json").produces("application/json")
            .get("/{id}").description("Find user by id").outType(User.class)
            .param().name("id").type(path).description("The id of the user to
get").dataType("int").endParam()
            .to("bean:userService?method=getUser(${header.id})")
            .put().description("Updates or create a user").type(User.class)
            .param().name("body").type(body).description("The user to update or create").endParam()
            .to("bean:userService?method=updateUser")
            .get("/findAll").description("Find all users").outType(User[].class)
            .to("bean:userService?method=listUsers");
    }
}
```

108.3. OPTIONS

The OpenApi module can be configured using the following options. To configure using a servlet you use the `init-param` as shown above. When configuring directly in the `rest-dsl`, you use the appropriate method, such as **`enableCORS`**, **`host`**, **`contextPath`**, `dsl`. The options with **`api.xxx`** is configured using **`apiProperty`** `dsl`.

Option	Type	Description
<code>cors</code>	Boolean	Whether to enable CORS. Notice this only enables CORS for the api browser, and not the actual access to the REST services. Is default false.
<code>openapi.version</code>	String	OpenApi spec version. Is default 3.0.
<code>host</code>	String	To setup the hostname. If not configured camel-openapi-java will calculate the name as localhost based.
<code>schemes</code>	String	The protocol schemes to use. Multiple values can be separated by comma such as "http,https". The default value is "http".
<code>base.path</code>	String	Required: To setup the base path where the REST services is available. The path is relative (eg do not start with http/https) and camel-openapi-java will calculate the absolute base path at runtime, which will be <code>protocol://host:port/context-path/base.path</code>
<code>api.path</code>	String	To setup the path where the API is available (eg /api-docs). The path is relative (eg do not start with http/https) and camel-openapi-java will calculate the absolute base path at runtime, which will be <code>protocol://host:port/context-path/api.path</code> So using relative paths is much easier. See above for an example.
<code>api.version</code>	String	The version of the api. Is default 0.0.0.
<code>api.title</code>	String	The title of the application.
<code>api.description</code>	String	A short description of the application.
<code>api.termsOfService</code>	String	A URL to the Terms of Service of the API.
<code>api.contact.name</code>	String	Name of person or organization to contact
<code>api.contact.email</code>	String	An email to be used for API-related correspondence.
<code>api.contact.url</code>	String	A URL to a website for more contact information.
<code>api.license.name</code>	String	The license name used for the API.
<code>api.license.url</code>	String	A URL to the license used for the API.

108.4. ADDING SECURITY DEFINITIONS IN API DOC

The Rest DSL now supports declaring OpenApi **securityDefinitions** in the generated API document. For example as shown below:

```
rest("/user").tag("dude").description("User rest service")
    // setup security definitions
    .securityDefinitions()
        .oauth2("petstore_auth").authorizationUrl("http://petstore.swagger.io/oauth/dialog").end()
        .apiKey("api_key").withHeader("myHeader").end()
    .end()
    .consumes("application/json").produces("application/json")
```

Here we have setup two security definitions

- OAuth2 - with implicit authorization with the provided url
- Api Key - using an api key that comes from HTTP header named *myHeader*

Then you need to specify on the rest operations which security to use by referring to their key (petstore_auth or api_key).

```
.get("/{id}/{date}").description("Find user by id and date").outType(User.class)
    .security("api_key")

...

.put().description("Updates or create a user").type(User.class)
    .security("petstore_auth", "write:pets,read:pets")
```

Here the get operation is using the Api Key security and the put operation is using OAuth security with permitted scopes of read and write pets.

108.5. JSON OR YAML

The camel-openapi-java module supports both JSon and Yaml out of the box. You can specify in the request url what you want returned by using /openapi.json or /openapi.yaml for either one. If none is specified then the HTTP Accept header is used to detect if json or yaml can be accepted. If either both is accepted or none was set as accepted then json is returned as the default format.

108.6. USEXFORWARDHEADERS AND API URL RESOLUTION

The OpenApi specification allows you to specify the host, port & path that is serving the API. In OpenApi V2 this is done via the **host** field and in OpenAPI V3 it is part of the **servers** field.

By default, the value for these fields is determined by **X-Forwarded** headers, **X-Forwarded-Host** & **X-Forwarded-Proto**.

This can be overridden by disabling the lookup of **X-Forwarded** headers and by specifying your own host, port & scheme on the REST configuration.

```
restConfiguration().component("netty-http")
    .useXForwardHeaders(false)
    .apiProperty("schemes", "https");
```

```
.host("localhost")
.port(8080);
```

108.7. EXAMPLES

In the Apache Camel distribution we ship the **camel-example-openapi-cdi** and **camel-example-spring-boot-rest-openapi-simple** which demonstrates using this OpenApi component.

108.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 1 options, which are listed below.

Name	Description	Default	Type
camel.openapi.enabled	Enables Camel Rest DSL to automatic register its OpenAPI (eg swagger doc) in Spring Boot which allows tooling such as SpringDoc to integrate with Camel.	true	Boolean

CHAPTER 109. OPENSEARCH

Since Camel 4.0

Only producer is supported

The OpenSearch component allows you to interface with an [OpenSearch 2.8.x](#) API using the Java API Client library.

109.1. DEPENDENCIES

When using **bean** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-opensearch-starter</artifactId>
</dependency>
```

109.2. URI FORMAT

```
opensearch://clusterName[?options]
```

109.3. CONFIGURING OPTIONS

Camel components are configured on two levels:

- Component level
- Endpoint level

109.3.1. Component Level Options

The **component level** is the highest level. The configurations you define at this level are inherited by all the endpoints. For example, a component can have security settings, credentials for authentication, urls for network connection, and so on.

Since components typically have pre-configured defaults for the most common cases, you may need to only configure a few component options, or maybe none at all.

You can configure components with

- [Component DSL](#)
- in a configuration file (**application.properties** or **yaml**),
- directly with Java code.

109.3.2. Endpoint Level Options

At the **Endpoint level** you have many options, which you can use to configure what you want the endpoint to do. The options are categorized according to whether the endpoint is used as a consumer (*from*) or as a producer (*to*) or used for both.

You can configure endpoints directly in the endpoint URI as **path** and **query** parameters. You can also use [Endpoint DSL](#) and [DataFormat DSL](#) as *type safe* ways of configuring endpoints and data formats in Java.

When configuring options, use [Property Placeholders](#) for urls, port numbers, sensitive information, and other settings.

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

109.4. COMPONENT OPTIONS

The OpenSearch component supports 13 options, which are listed below.

Name	Description	Default	Type
connectionTimeout (producer)	The time in ms to wait before connection will time out.	30000	int
hostAddresses (producer)	Comma separated list with ip:port formatted remote transport addresses to use. The ip and port options must be left blank for hostAddresses to be considered instead.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
maxRetryTimeout (producer)	The time in ms before retry.	30000	int
socketTimeout (producer)	The timeout in ms to wait before the socket will time out.	30000	int

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
client (advanced)	Autowired To use an existing configured OpenSearch client, instead of creating a client per endpoint. This allows customizing the client with specific settings.		Rest Client
enableSniffer (advanced)	Enable automatically discover nodes from a running OpenSearch cluster. If this option is used in conjunction with Spring Boot, then it's managed by the Spring Boot configuration (see: Disable Sniffer in Spring Boot).	false	boolean
sniffAfterFailureDelay (advanced)	The delay of a sniff execution scheduled after a failure (in milliseconds).	60000	int
snifferInterval (advanced)	The interval between consecutive ordinary sniff executions in milliseconds. Will be honoured when sniffOnFailure is disabled or when there are no failures between consecutive sniff executions.	30000	int
enableSSL (security)	Enable SSL.	false	boolean
password (security)	Password for authenticating.		String
user (security)	Basic authenticate user.		String

109.5. ENDPOINT OPTIONS

The OpenSearch endpoint is configured using URI syntax:

```
opensearch:clusterName
```

With the following *path* and *query* parameters:

109.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
clusterName (producer)	Required Name of the cluster.		String

109.5.2. Query Parameters (19 parameters)

Name	Description	Default	Type
connectionTimeout (producer)	The time in ms to wait before connection will time out.	30000	int
disconnect (producer)	Disconnect after it finishes calling the producer.	false	boolean
from (producer)	Starting index of the response.		Integer
hostAddresses (producer)	Comma separated list with ip:port formatted remote transport addresses to use.		String
indexName (producer)	The name of the index to act against.		String
maxRetryTimeout (producer)	The time in ms before retry.	30000	int

Name	Description	Default	Type
operation (producer)	What operation to perform. Enum values: • Index • Update • Bulk • GetById • MultiGet • MultiSearch • Delete • DeleteIndex • Search • Exists • Ping		OpenSearch Operation
scrollKeepAliveMs (producer)	Time in ms during which OpenSearch will keep search context alive.	60000	int
size (producer)	Size of the response.		Integer
socketTimeout (producer)	The timeout in ms to wait before the socket will time out.	30000	int
useScroll (producer)	Enable scroll usage.	false	boolean
waitForActiveShards (producer)	Index creation waits for the right consistency number of shards to be available.	1	int

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
documentClass (advanced)	The class to use when deserializing the documents.	ObjectNode	Class
enableSniffer (advanced)	Enable automatically discover nodes from a running OpenSearch cluster. If this option is used in conjunction with Spring Boot, then it's managed by the Spring Boot configuration (see: Disable Sniffer in Spring Boot).	false	boolean
sniffAfterFailureDelay (advanced)	The delay of a sniff execution scheduled after a failure (in milliseconds).	60000	int
snifferInterval (advanced)	The interval between consecutive ordinary sniff executions in milliseconds. Will be honoured when sniffOnFailure is disabled or when there are no failures between consecutive sniff executions.	30000	int
certificatePath (security)	The certificate that can be used to access the ES Cluster. It can be loaded by default from classpath, but you can prefix with classpath:, file:, or http: to load the resource from different systems.		String
enableSSL (security)	Enable SSL.	false	boolean

109.6. MESSAGE HEADERS

The OpenSearch component supports 9 message header(s), listed below:

Name	Description	Default	Type
operation (producer) Constant: PARAM_OPERATION	The operation to perform. Enum values: • Index • Update • Bulk • GetById • MultiGet • MultiSearch • Delete • DeleteIndex • Search • Exists • Ping		Operation
indexId (producer) Constant: PARAM_INDEX_ID	The id of the indexed document.		String
indexName (producer) Constant: PARAM_INDEX_NAME	The name of the index to act against.		String
documentClass (producer) Constant: PARAM_DOCUMENT_CLASS	The full qualified name of the class of the document to unmarshall.	ObjectNode	Class

Name	Description	Default	Type
waitForActiveShards (producer) Constant: PARAM_WAIT_FOR_ACTIVE_SHARDS	The index creation waits for the write consistency number of shards to be available.		Integer
scrollKeepAliveMs (producer) Constant: PARAM_SCROLL_KEEP_ALIVE_MS	The starting index of the response.		Integer
useScroll (producer) Constant: PARAM_SCROLL	Set to true to enable scroll usage.		Boolean
size (producer) Constant: PARAM_SIZE	The size of the response.		Integer
from (producer) Constant: PARAM_FROM	The starting index of the response.		Integer

109.7. USAGE

109.7.1. Message Operations

The following [OpenSearch](#) operations are currently supported. Set an endpoint URI option or exchange header with a key of "operation" and a value set to one of the following. Some operations also require other parameters or the message body to be set.

operation	message body	description
-----------	--------------	-------------

operation	message body	description
Index	Map, String, byte[], Reader, InputStream or IndexRequest.Builder content to index	Adds content to an index and returns the content's indexId in the body. You can set the name of the target index by setting the message header with the key "indexName". You can set the indexId by setting the message header with the key "indexId".
GetById	String or GetRequest.Builder index id of content to retrieve	Retrieves the document corresponding to the given index id and returns a GetResponse object in the body. You can set the name of the target index by setting the message header with the key "indexName". You can set the type of document by setting the message header with the key "documentClass".
Delete	String or DeleteRequest.Builder index id of content to delete	Deletes the specified indexName and returns a Result object in the body. You can set the name of the target index by setting the message header with the key "indexName".
DeleteIndex	String or DeleteIndexRequest.Builder index name of the index to delete	Deletes the specified indexName and returns a status code in the body. You can set the name of the target index by setting the message header with the key "indexName".
Bulk Iterable or BulkRequest.Builder of any type that is already accepted (DeleteOperation.Builder for delete operation, UpdateOperation.Builder for update operation, CreateOperation.Builder for create operation, byte[], InputStream, String, Reader, Map or any document type for index operation)	Adds/Updates/Deletes content from/to an index and returns a List<BulkResponseItem> object in the body You can set the name of the target index by setting the message header with the key "indexName".	Search

operation	message body	description
Map, String or SearchRequest.Builder	Search the content with the map of query string. You can set the name of the target index by setting the message header with the key "indexName". You can set the number of hits to return by setting the message header with the key "size". You can set the starting document offset by setting the message header with the key "from".	MultiSearch
MsearchRequest.Builder	Multiple search in one	MultiGet
Iterable<String> or MgetRequest.Builder the id of the document to retrieve	Multiple get in one You can set the name of the target index by setting the message header with the key "indexName".	Exists
None	Checks whether the index exists or not and returns a Boolean flag in the body. You must set the name of the target index by setting the message header with the key "indexName".	Update
byte[], InputStream, String, Reader, Map or any document type content to update	Updates content to an index and returns the content's indexId in the body. You can set the name of the target index by setting the message header with the key "indexName". You can set the indexId by setting the message header with the key "indexId".	Ping

109.7.2. Configure the component and enable basic authentication

To use the OpenSearch component, it has to be configured with a minimum configuration.

```
OpensearchComponent opensearchComponent = new OpensearchComponent();
opensearchComponent.setHostAddresses("opensearch-host:9200");
camelContext.addComponent("opensearch", opensearchComponent);
```

For basic authentication with OpenSearch or using reverse http proxy in front of the OpenSearch cluster, simply setup basic authentication and SSL on the component like the example below

```

OpenSearchComponent opensearchComponent = new OpenSearchComponent();
opensearchComponent.setHostAddresses("opensearch-host:9200");
opensearchComponent.setUser("opensearchuser");
opensearchComponent.setPassword("secure!!");

camelContext.addComponent("opensearch", opensearchComponent);

```

109.7.3. Document type

For all the search operations, it is possible to indicate the type of document to retrieve to get the result already unmarshalled with the expected type.

The document type can be set using the header "documentClass" or via the uri parameter of the same name.

109.8. EXAMPLES

109.8.1. Index Example

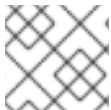
Below is a simple INDEX example

```

from("direct:index")
.to("opensearch://opensearch?operation=Index&indexName=twitter");

<route>
  <from uri="direct:index"/>
  <to uri="opensearch://opensearch?operation=Index&indexName=twitter"/>
</route>

```



NOTE

For this operation, you'll need to specify an indexId header.

A client only needs to pass a body message containing a **Map** to the route. The result body contains the **indexId** created.

```

Map<String, String> map = new HashMap<String, String>();
map.put("content", "test");
String indexId = template.requestBody("direct:index", map, String.class);

```

109.8.2. Search Example

Searching on specific field(s) and value use the Operation 'Search'. Pass in the query JSON String or the Map

```

from("direct:search")
.to("opensearch://opensearch?operation=Search&indexName=twitter");

<route>
  <from uri="direct:search"/>

```

```
<to uri="opensearch://opensearch?operation=Search&indexName=twitter"/>
</route>
```

```
String query = "{\"query\":{\"match\":{\"doc.content\":\"new release of ApacheCamel\"}}}\";
HitsMetadata<?> response = template.requestBody("direct:search", query, HitsMetadata.class);
```

Search on specific field(s) using Map.

```
Map<String, Object> actualQuery = new HashMap<>();
actualQuery.put("doc.content", "new release of ApacheCamel");

Map<String, Object> match = new HashMap<>();
match.put("match", actualQuery);

Map<String, Object> query = new HashMap<>();
query.put("query", match);
HitsMetadata<?> response = template.requestBody("direct:search", query, HitsMetadata.class);
```

Search using OpenSearch scroll api to fetch all results.

```
from("direct:search")
  .to("opensearch://opensearch?
operation=Search&indexName=twitter&useScroll=true&scrollKeepAliveMs=30000");
```

```
<route>
  <from uri="direct:search"/>
  <to uri="opensearch://opensearch?
operation=Search&indexName=twitter&useScroll=true&scrollKeepAliveMs=30000"/>
</route>
```

```
String query = "{\"query\":{\"match\":{\"doc.content\":\"new release of ApacheCamel\"}}}\";
try (OpenSearchScrollRequestIterator response = template.requestBody("direct:search", query,
OpenSearchScrollRequestIterator.class)) {
  // do something smart with results
}
```

[Split EIP](#) can also be used.

```
from("direct:search")
  .to("opensearch://opensearch?
operation=Search&indexName=twitter&useScroll=true&scrollKeepAliveMs=30000")
  .split()
  .body()
  .streaming()
  .to("mock:output")
  .end();
```

109.8.3. MultiSearch Example

MultiSearching on specific field(s) and value uses the Operation **MultiSearch**. Pass in the MultiSearchRequest instance

```
from("direct:multiSearch")
```

```
.to("opensearch://opensearch?operation=MultiSearch");
```

```
<route>
  <from uri="direct:multiSearch"/>
  <to uri="opensearch://opensearch?operation=MultiSearch"/>
</route>
```

MultiSearch on specific field(s)

```
MsearchRequest.Builder builder = new MsearchRequest.Builder().index("twitter").searches(
    new RequestItem.Builder().header(new MultisearchHeader.Builder().build())
        .body(new MultisearchBody.Builder().query(b -> b.matchAll(x -> x)).build()).build(),
    new RequestItem.Builder().header(new MultisearchHeader.Builder().build())
        .body(new MultisearchBody.Builder().query(b -> b.matchAll(x -> x)).build()).build());
List<MultiSearchResponseItem<?>> response = template.requestBody("direct:multiSearch", builder,
    List.class);
```

CHAPTER 110. OPENTELEMETRY

Since Camel 3.5

The OpenTelemetry component is used for tracing and timing the incoming and outgoing Camel messages using [OpenTelemetry](#).

Events (spans) are captured for incoming and outgoing messages that are sent to/from Camel.

110.1. DEPENDENCIES

Add the following dependency to your **pom.xml** for this component:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-opentelemetry-starter</artifactId>
</dependency>
```

110.2. CONFIGURATION PROPERTIES

The configuration properties for the OpenTelemetry tracer are:

Option	Default	Description
excludePatterns		Sets exclude pattern(s) that will disable tracing for Camel messages that matches the pattern. The content is a Set<String> where the key is a pattern. The pattern uses the rules from Intercept.
encoding	false	Sets whether the header keys need to be encoded (connector specific) or not. The value is a boolean. Dashes need for instances to be encoded for JMS property keys.

110.2.1. Using Camel OpenTelemetry

Add the **camel-opentelemetry** component in your POM, in addition to any specific dependencies associated with the chosen OpenTelemetry compliant Tracer.

To explicitly configure OpenTelemetry support, instantiate the **OpenTelemetryTracer** and initialize the camel context. You can optionally specify a **Tracer**, or alternatively it can be implicitly discovered using the **Registry**

```
import io.opentelemetry.api.OpenTelemetry;
import io.opentelemetry.api.trace.Tracer;
```

```
OpenTelemetryTracer otelTracer = new OpenTelemetryTracer();
// By default it uses the DefaultTracer, but you can override it with a specific OpenTelemetry Tracer
// implementation.
otelTracer.setTracer(...);
// And then initialize the context
otelTracer.init(camelContext);
```

110.3. SPRING BOOT

Add the **camel-opentelemetry-starter** dependency, and then turn on the OpenTracing by annotating the main class with **@CamelOpenTelemetry**.

The **OpenTelemetryTracer** is implicitly obtained from the camel context's **Registry**, unless a **OpenTelemetryTracer** bean has been defined by the application.

110.4. USING TRACING STRATEGY

The **camel-opentelemetry** component starter allows you to trace not only the from/to Camel endpoints but also the Java Beans invoked from Camel processor/bean too. By default, Java beans invoked from Camel Processor or bean are categorized as another "span", that is, if user writes **.to("bean:beanName?method=methodName")**, it will be categorized under the same "span" with the from/to Camel endpoints.

To categorize the Java beans invoked from Camel processor/bean under the same "span", you can use the **org.apache.camel.opentelemetry.OpenTelemetryTracingStrategy** class with the **setTracingStrategy()** option.

```
import io.opentelemetry.api.OpenTelemetry;
import io.opentelemetry.api.trace.Tracer;

@Configuration
public class CamelOtelConfiguration {
    public CamelOtelConfiguration(CamelContext camelContext, OpenTelemetryTracer tracer) {
        var strategy = new OpenTelemetryTracingStrategy(tracer);
        tracer.setTracingStrategy(strategy);
        camelContext.getCamelContextExtension().addInterceptStrategy(strategy);
    }
}
```

In case of too much data, you can filter the data by **camel.opentelemetry.exclude-patterns** property.

```
camel:
  opentelemetry:
    exclude-patterns:
      - ## Set some ID here to filter ##
```

110.5. JAVA AGENT

Download the [Java agent](#).

This package includes the instrumentation agent as well as instrumentations for all supported libraries and all available data exporters. The package provides a completely automatic, out-of-the-box experience.

Enable the instrumentation agent using the **-javaagent** flag to the JVM.

```
java -javaagent:path/to/opentelemetry-javaagent.jar \
-jar myapp.jar
```

By default, the OpenTelemetry Java agent uses [OTLP exporter](#) configured to send data to [OpenTelemetry collector](#) at <http://localhost:4317>.

Configuration parameters are passed as Java system properties (**-D** flags) or as environment variables. See [Configuring the agent](#) and [OpenTelemetry auto-configuration](#) for the full list of configuration items. For example:

```
java -javaagent:path/to/opentelemetry-javaagent.jar \
  -Dotel.service.name=your-service-name \
  -Dotel.traces.exporter=jaeger \
  -jar myapp.jar
```

110.6. ADDING THE ROUTEID ATTRIBUTE TO THE SPAN

To identify the origin of the Route, it is necessary to add the RouteID attribute to the span. The workaround is to use the Span decorator from Camel. For this, add a custom decorator for each component you want to enrich with the route id. For example, if you want to print the route id in the direct component, the implementation is as follows:

```
import org.apache.camel.Endpoint;
import org.apache.camel.Exchange;
import org.apache.camel.tracing.SpanAdapter;
import org.apache.camel.tracing.decorators.DirectSpanDecorator;

public class RouteIDSpanDecorator extends DirectSpanDecorator {

    @Override
    public void pre(SpanAdapter span, Exchange exchange, Endpoint endpoint) {
        super.pre(span, exchange, endpoint);
        span.setTag("camel.route.id", exchange.getFromRouteId());
    }
}
```

Add the file named **org.apache.camel.tracing.SpanDecorator** to **src/main/resources/META-INF/services** folder which contains the reference of the class . (for example, `com.redhat.xxxx.RouteIDSpanDecorator`). This allows the Camel tracing feature to load the custom Span decorator with the tag applied in the trace.

110.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 2 options, which are listed below.

Name	Description	Default	Type
camel.opentelemetry.encoding	Activate or deactivate the dash encoding in headers (required by JMS) for messaging.		Boolean
camel.opentelemetry.exclude-patterns	Sets exclude pattern(s) that will disable the tracing for the Camel messages that matches the pattern.		Set

110.8. MDC LOGGING

When MDC Logging is enabled for the active Camel context, the Trace ID and Span ID are added and removed from the MDC for each route, where the keys are **trace_id** and **span_id**, respectively.

CHAPTER 111. PAHO

Both producer and consumer are supported

Paho component provides connector for the MQTT messaging protocol using the [Eclipse Paho library](#). Paho is one of the most popular MQTT libraries, so if you would like to integrate it with your Java project – Camel Paho connector is a way to go.

111.1. DEPENDENCIES

When using **paho** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-paho-starter</artifactId>
</dependency>
```

111.2. URI FORMAT

```
paho:topic[?options]
```

Where **topic** is the name of the topic.

111.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

111.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

111.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

111.4. COMPONENT OPTIONS

The Paho component supports 31 options, which are listed below.

Name	Description	Default	Type
automaticReconnect (common)	Sets whether the client will automatically attempt to reconnect to the server if the connection is lost. If set to false, the client will not attempt to automatically reconnect to the server in the event that the connection is lost. If set to true, in the event that the connection is lost, the client will attempt to reconnect to the server. It will initially wait 1 second before it attempts to reconnect, for every failed reconnect attempt, the delay will double until it is at 2 minutes at which point the delay will stay at 2 minutes.	true	boolean
brokerUrl (common)	The URL of the MQTT broker.	tcp://localhost:1883	String

Name	Description	Default	Type
cleanSession (common)	Sets whether the client and server should remember state across restarts and reconnects. If set to false both the client and server will maintain state across restarts of the client, the server and the connection. As state is maintained: Message delivery will be reliable meeting the specified QOS even if the client, server or connection are restarted. The server will treat a subscription as durable. If set to true the client and server will not maintain state across restarts of the client, the server or the connection. This means Message delivery to the specified QOS cannot be maintained if the client, server or connection are restarted The server will treat a subscription as non-durable.	true	boolean
clientId (common)	MQTT client identifier. The identifier must be unique.		String
configuration (common)	To use the shared Paho configuration.		PahoConfiguration
connectionTimeout (common)	Sets the connection timeout value. This value, measured in seconds, defines the maximum time interval the client will wait for the network connection to the MQTT server to be established. The default timeout is 30 seconds. A value of 0 disables timeout processing meaning the client will wait until the network connection is made successfully or fails.	30	int
filePersistenceDirectory (common)	Base directory used by file persistence. Will by default use user directory.		String
keepAliveInterval (common)	Sets the keep alive interval. This value, measured in seconds, defines the maximum time interval between messages sent or received. It enables the client to detect if the server is no longer available, without having to wait for the TCP/IP timeout. The client will ensure that at least one message travels across the network within each keep alive period. In the absence of a data-related message during the time period, the client sends a very small ping message, which the server will acknowledge. A value of 0 disables keepalive processing in the client. The default value is 60 seconds.	60	int
maxInflight (common)	Sets the max inflight. please increase this value in a high traffic environment. The default value is 10.	10	int

Name	Description	Default	Type
maxReconnectDelay (common)	Get the maximum time (in millis) to wait between reconnects.	128000	int
mqttVersion (common)	Sets the MQTT version. The default action is to connect with version 3.1.1, and to fall back to 3.1 if that fails. Version 3.1.1 or 3.1 can be selected specifically, with no fall back, by using the MQTT_VERSION_3_1_1 or MQTT_VERSION_3_1 options respectively.		int
persistence (common)	Client persistence to be used - memory or file. Enum values: • FILE • MEMORY	MEMORY	PahoPersistence
qos (common)	Client quality of service level (0-2).	2	int
retained (common)	Retain option.	false	boolean

Name	Description	Default	Type
serverURIs (common)	Set a list of one or more serverURIs the client may connect to. Multiple servers can be separated by comma. Each serverURI specifies the address of a server that the client may connect to. Two types of connection are supported tcp:// for a TCP connection and ssl:// for a TCP connection secured by SSL/TLS. For example: tcp://localhost:1883 ssl://localhost:8883 If the port is not specified, it will default to 1883 for tcp:// URIs, and 8883 for ssl:// URIs. If serverURIs is set then it overrides the serverURI parameter passed in on the constructor of the MQTT client. When an attempt to connect is initiated the client will start with the first serverURI in the list and work through the list until a connection is established with a server. If a connection cannot be made to any of the servers then the connect attempt fails. Specifying a list of servers that a client may connect to has several uses: High Availability and reliable message delivery Some MQTT servers support a high availability feature where two or more equal MQTT servers share state. An MQTT client can connect to any of the equal servers and be assured that messages are reliably delivered and durable subscriptions are maintained no matter which server the client connects to. The cleansession flag must be set to false if durable subscriptions and/or reliable message delivery is required. Hunt List A set of servers may be specified that are not equal (as in the high availability option). As no state is shared across the servers reliable message delivery and durable subscriptions are not valid. The cleansession flag must be set to true if the hunt list mode is used.		String
willPayload (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		String
willQos (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		int

Name	Description	Default	Type
willRetained (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.	false	boolean
willTopic (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
client (advanced)	To use a shared Paho client.		MqttClient

Name	Description	Default	Type
customWebSocketHeaders (advanced)	Sets the Custom WebSocket Headers for the WebSocket Connection.		Properties
executorServiceTimeout (advanced)	Set the time in seconds that the executor service should wait when terminating before forcefully terminating. It is not recommended to change this value unless you are absolutely sure that you need to.	1	int
httpsHostnameVerificationEnabled (security)	Whether SSL HostnameVerifier is enabled or not. The default value is true.	true	boolean
password (security)	Password to be used for authentication against the MQTT broker.		String
socketFactory (security)	Sets the SocketFactory to use. This allows an application to apply its own policies around the creation of network sockets. If using an SSL connection, an SSLSocketFactory can be used to supply application-specific security settings.		SocketFactory
sslClientProps (security)	Sets the SSL properties for the connection. Note that these properties are only valid if an implementation of the Java Secure Socket Extensions (JSSE) is available. These properties are not used if a custom SocketFactory has been set. The following properties can be used: - com.ibm.ssl.protocol One of: SSL, SSLv3, TLS, TLSv1, SSL_TLS. - com.ibm.ssl.contextProvider Underlying JSSE provider. For example IBMJSSE2 or SunJSSE - com.ibm.ssl.keyStore The name of the file that contains the KeyStore object that you want the KeyManager to use. For example /mydir/etc/key.p12 - com.ibm.ssl.keyStorePassword The password for the KeyStore object that you want the KeyManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. - com.ibm.ssl.keyStoreType Type of key store, for example PKCS12, JKS, or JCEKS. - com.ibm.ssl.keyStoreProvider Key store provider, for example IBMJCE or IBMJCEFIPS. - com.ibm.ssl.trustStore The name of the file that contains the KeyStore object that you want the TrustManager to use.		Properties

Name	Description	Default	Type
	<code>com.ibm.ssl.trustStorePassword</code> The password for the TrustStore object that you want the TrustManager to use. The password can either be in plain-text, or may be obfuscated using the static method: <code>com.ibm.micro.security.Password.obfuscate(char password)</code>. This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. <code>com.ibm.ssl.trustStoreType</code> The type of KeyStore object that you want the default TrustManager to use. Same possible values as <code>keyStoreType</code>. <code>com.ibm.ssl.trustStoreProvider</code> Trust store provider, for example IBMJCE or IBMJCEFIPS. <code>com.ibm.ssl.enabledCipherSuites</code> A list of which ciphers are enabled. Values are dependent on the provider, for example: <code>SSL_RSA_WITH_AES_128_CBC_SHA;SSL_RSA_WITH_3DES_EDE_CBC_SHA</code>. <code>com.ibm.ssl.keyManager</code> Sets the algorithm that will be used to instantiate a KeyManagerFactory object instead of using the default algorithm available in the platform. Example values: <code>IbmX509</code> or <code>IBMJ9X509</code>. <code>com.ibm.ssl.trustManager</code> Sets the algorithm that will be used to instantiate a TrustManagerFactory object instead of using the default algorithm available in the platform. Example values: <code>PKIX</code> or <code>IBMJ9X509</code>.		
sslHostnameVerifier (security)	Sets the HostnameVerifier for the SSL connection. Note that it will be used after handshake on a connection and you should do actions by yourself when hostname is verified error. There is no default HostnameVerifier.		HostnameVerifier
userName (security)	Username to be used for authentication against the MQTT broker.		String

111.5. ENDPOINT OPTIONS

The Paho endpoint is configured using URI syntax:

```
paho:topic
```

with the following path and query parameters:

111.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
topic (common)	Required Name of the topic.		String

11.5.2. Query Parameters (31 parameters)

Name	Description	Default	Type
automaticReconnect (common)	Sets whether the client will automatically attempt to reconnect to the server if the connection is lost. If set to false, the client will not attempt to automatically reconnect to the server in the event that the connection is lost. If set to true, in the event that the connection is lost, the client will attempt to reconnect to the server. It will initially wait 1 second before it attempts to reconnect, for every failed reconnect attempt, the delay will double until it is at 2 minutes at which point the delay will stay at 2 minutes.	true	boolean
brokerUrl (common)	The URL of the MQTT broker.	tcp://localhost:1883	String
cleanSession (common)	Sets whether the client and server should remember state across restarts and reconnects. If set to false both the client and server will maintain state across restarts of the client, the server and the connection. As state is maintained: Message delivery will be reliable meeting the specified QOS even if the client, server or connection are restarted. The server will treat a subscription as durable. If set to true the client and server will not maintain state across restarts of the client, the server or the connection. This means Message delivery to the specified QOS cannot be maintained if the client, server or connection are restarted The server will treat a subscription as non-durable.	true	boolean
clientId (common)	MQTT client identifier. The identifier must be unique.		String
connectionTimeout (common)	Sets the connection timeout value. This value, measured in seconds, defines the maximum time interval the client will wait for the network connection to the MQTT server to be established. The default timeout is 30 seconds. A value of 0 disables timeout processing meaning the client will wait until the network connection is made successfully or fails.	30	int

Name	Description	Default	Type
filePersistenceDirectory (common)	Base directory used by file persistence. Will by default use user directory.		String
keepAliveInterval (common)	Sets the keep alive interval. This value, measured in seconds, defines the maximum time interval between messages sent or received. It enables the client to detect if the server is no longer available, without having to wait for the TCP/IP timeout. The client will ensure that at least one message travels across the network within each keep alive period. In the absence of a data-related message during the time period, the client sends a very small ping message, which the server will acknowledge. A value of 0 disables keepalive processing in the client. The default value is 60 seconds.	60	int
maxInflight (common)	Sets the max inflight. please increase this value in a high traffic environment. The default value is 10.	10	int
maxReconnectDelay (common)	Get the maximum time (in millis) to wait between reconnects.	128000	int
mqttVersion (common)	Sets the MQTT version. The default action is to connect with version 3.1.1, and to fall back to 3.1 if that fails. Version 3.1.1 or 3.1 can be selected specifically, with no fall back, by using the MQTT_VERSION_3_1_1 or MQTT_VERSION_3_1 options respectively.		int
persistence (common)	Client persistence to be used - memory or file. Enum values: • FILE • MEMORY	MEMORY	PahoPersistence
qos (common)	Client quality of service level (0-2).	2	int
retained (common)	Retain option.	false	boolean

Name	Description	Default	Type
serverURIs (common)	Set a list of one or more serverURIs the client may connect to. Multiple servers can be separated by comma. Each serverURI specifies the address of a server that the client may connect to. Two types of connection are supported tcp:// for a TCP connection and ssl:// for a TCP connection secured by SSL/TLS. For example: tcp://localhost:1883 ssl://localhost:8883 If the port is not specified, it will default to 1883 for tcp:// URIs, and 8883 for ssl:// URIs. If serverURIs is set then it overrides the serverURI parameter passed in on the constructor of the MQTT client. When an attempt to connect is initiated the client will start with the first serverURI in the list and work through the list until a connection is established with a server. If a connection cannot be made to any of the servers then the connect attempt fails. Specifying a list of servers that a client may connect to has several uses: High Availability and reliable message delivery Some MQTT servers support a high availability feature where two or more equal MQTT servers share state. An MQTT client can connect to any of the equal servers and be assured that messages are reliably delivered and durable subscriptions are maintained no matter which server the client connects to. The cleansession flag must be set to false if durable subscriptions and/or reliable message delivery is required. Hunt List A set of servers may be specified that are not equal (as in the high availability option). As no state is shared across the servers reliable message delivery and durable subscriptions are not valid. The cleansession flag must be set to true if the hunt list mode is used.		String
willPayload (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		String
willQos (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		int

Name	Description	Default	Type
willRetained (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.	false	boolean
willTopic (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code> ● <code>InOptionalOut</code>		<code>ExchangePattern</code>

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
client (advanced)	To use an existing mqtt client.		MqttClient
customWebSocketHeaders (advanced)	Sets the Custom WebSocket Headers for the WebSocket Connection.		Properties
executorServiceTimeout (advanced)	Set the time in seconds that the executor service should wait when terminating before forcefully terminating. It is not recommended to change this value unless you are absolutely sure that you need to.	1	int
httpsHostnameVerificationEnabled (security)	Whether SSL HostnameVerifier is enabled or not. The default value is true.	true	boolean
password (security)	Password to be used for authentication against the MQTT broker.		String
socketFactory (security)	Sets the SocketFactory to use. This allows an application to apply its own policies around the creation of network sockets. If using an SSL connection, an SSLSocketFactory can be used to supply application-specific security settings.		SocketFactory
sslClientProps (security)	Sets the SSL properties for the connection. Note that these properties are only valid if an implementation of the Java Secure Socket Extensions (JSSE) is available. These properties are not used if a custom SocketFactory has been set. The following properties can be used: com.ibm.ssl.protocol One of: SSL, SSLv3, TLS, TLSv1, SSL_TLS. com.ibm.ssl.contextProvider Underlying JSSE provider. For example IBMJSSE2 or SunJSSE com.ibm.ssl.keyStore The name of the file that contains the KeyStore object that you want the KeyManager to use. For example /mydir/etc/key.p12 com.ibm.ssl.keyStorePassword The password for the KeyStore object that you want the KeyManager to		Properties

Name	Description	Default	Type
	use. The password can either be in plain-text, or may be obfuscated using the static method: <code>com.ibm.micro.security.Password.obfuscate(char password)</code>. This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. <code>com.ibm.ssl.keyStoreType</code> Type of key store, for example PKCS12, JKS, or JCEKS. <code>com.ibm.ssl.keyStoreProvider</code> Key store provider, for example IBMJCE or IBMJCEFIPS. <code>com.ibm.ssl.trustStore</code> The name of the file that contains the KeyStore object that you want the TrustManager to use. <code>com.ibm.ssl.trustStorePassword</code> The password for the TrustStore object that you want the TrustManager to use. The password can either be in plain-text, or may be obfuscated using the static method: <code>com.ibm.micro.security.Password.obfuscate(char password)</code>. This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. <code>com.ibm.ssl.trustStoreType</code> The type of KeyStore object that you want the default TrustManager to use. Same possible values as <code>keyStoreType</code>. <code>com.ibm.ssl.trustStoreProvider</code> Trust store provider, for example IBMJCE or IBMJCEFIPS. <code>com.ibm.ssl.enabledCipherSuites</code> A list of which ciphers are enabled. Values are dependent on the provider, for example: <code>SSL_RSA_WITH_AES_128_CBC_SHA;SSL_RSA_WITH_3DES_EDE_CBC_SHA</code>. <code>com.ibm.ssl.keyManager</code> Sets the algorithm that will be used to instantiate a KeyManagerFactory object instead of using the default algorithm available in the platform. Example values: <code>IbmX509</code> or <code>IBMJ9X509</code>. <code>com.ibm.ssl.trustManager</code> Sets the algorithm that will be used to instantiate a TrustManagerFactory object instead of using the default algorithm available in the platform. Example values: <code>PKIX</code> or <code>IBMJ9X509</code>.		
sslHostnameVerifier (security)	Sets the HostnameVerifier for the SSL connection. Note that it will be used after handshake on a connection and you should do actions by yourself when hostname is verified error. There is no default HostnameVerifier.		HostnameVerifier
userName (security)	Username to be used for authentication against the MQTT broker.		String

111.6. HEADERS

The following headers are recognized by the Paho component:

Header	Java constant	Endpoint type	Value type	Description
CamelMqttTopic	PahoConstants.MQTT_TOPIC	Consumer	String	The name of the topic
CamelMqttQoS	PahoConstants.MQTT_QOS	Consumer	Integer	QualityOfService of the incoming message
CamelPahoOverrideTopic	PahoConstants.CAMEL_PAHO_OVERRIDE_TOPIC	Producer	String	Name of topic to override and send to instead of topic specified on endpoint

11.7. DEFAULT PAYLOAD TYPE

By default Camel Paho component operates on the binary payloads extracted out of (or put into) the MQTT message:

```
// Receive payload
byte[] payload = (byte[]) consumerTemplate.receiveBody("paho:topic");

// Send payload
byte[] payload = "message".getBytes();
producerTemplate.sendBody("paho:topic", payload);
```

But of course Camel build-in [type conversion API](#) can perform the automatic data type transformations for you. In the example below Camel automatically converts binary payload into **String** (and conversely):

```
// Receive payload
String payload = consumerTemplate.receiveBody("paho:topic", String.class);

// Send payload
String payload = "message";
producerTemplate.sendBody("paho:topic", payload);
```

11.8. SAMPLES

For example the following snippet reads messages from the MQTT broker installed on the same host as the Camel router:

```
from("paho:some/queue")
    .to("mock:test");
```

While the snippet below sends message to the MQTT broker:

```
from("direct:test")
    .to("paho:some/target/queue");
```

For example this is how to read messages from the remote MQTT broker:


```
from("paho:some/queue?brokerUrl=tcp://iot.eclipse.org:1883")
    .to("mock:test");
```

And here we override the default topic and set to a dynamic topic

```
from("direct:test")
    .setHeader(PahoConstants.CAMEL_PAHO_OVERRIDE_TOPIC, simple("${header.customerId}"))
    .to("paho:some/target/queue");
```

111.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 32 options, which are listed below.

Name	Description	Default	Type
camel.component.paho.automatic-reconnect	Sets whether the client will automatically attempt to reconnect to the server if the connection is lost. If set to false, the client will not attempt to automatically reconnect to the server in the event that the connection is lost. If set to true, in the event that the connection is lost, the client will attempt to reconnect to the server. It will initially wait 1 second before it attempts to reconnect, for every failed reconnect attempt, the delay will double until it is at 2 minutes at which point the delay will stay at 2 minutes.	true	Boolean
camel.component.paho.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.paho.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.paho.broker-url	The URL of the MQTT broker.	tcp://localhost:1883	String

Name	Description	Default	Type
camel.component.paho.clean-session	Sets whether the client and server should remember state across restarts and reconnects. If set to false both the client and server will maintain state across restarts of the client, the server and the connection. As state is maintained: Message delivery will be reliable meeting the specified QOS even if the client, server or connection are restarted. The server will treat a subscription as durable. If set to true the client and server will not maintain state across restarts of the client, the server or the connection. This means Message delivery to the specified QOS cannot be maintained if the client, server or connection are restarted The server will treat a subscription as non-durable.	true	Boolean
camel.component.paho.client	To use a shared Paho client. The option is a <code>org.eclipse.paho.client.mqttv3.MqttClient</code> type.		MqttClient
camel.component.paho.client-id	MQTT client identifier. The identifier must be unique.		String
camel.component.paho.configuration	To use the shared Paho configuration. The option is a <code>org.apache.camel.component.paho.PahoConfiguration</code> type.		PahoConfiguration
camel.component.paho.connection-timeout	Sets the connection timeout value. This value, measured in seconds, defines the maximum time interval the client will wait for the network connection to the MQTT server to be established. The default timeout is 30 seconds. A value of 0 disables timeout processing meaning the client will wait until the network connection is made successfully or fails.	30	Integer
camel.component.paho.custom-web-socket-headers	Sets the Custom WebSocket Headers for the WebSocket Connection. The option is a <code>java.util.Properties</code> type.		Properties
camel.component.paho.enabled	Whether to enable auto configuration of the paho component. This is enabled by default.		Boolean
camel.component.paho.executor-service-timeout	Set the time in seconds that the executor service should wait when terminating before forcefully terminating. It is not recommended to change this value unless you are absolutely sure that you need to.	1	Integer

Name	Description	Default	Type
camel.component.paho.file-persistence-directory	Base directory used by file persistence. Will by default use user directory.		String
camel.component.paho.https-hostname-verification-enabled	Whether SSL HostnameVerifier is enabled or not. The default value is true.	true	Boolean
camel.component.paho.keep-alive-interval	Sets the keep alive interval. This value, measured in seconds, defines the maximum time interval between messages sent or received. It enables the client to detect if the server is no longer available, without having to wait for the TCP/IP timeout. The client will ensure that at least one message travels across the network within each keep alive period. In the absence of a data-related message during the time period, the client sends a very small ping message, which the server will acknowledge. A value of 0 disables keepalive processing in the client. The default value is 60 seconds.	60	Integer
camel.component.paho.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.paho.max-inflight	Sets the max inflight. please increase this value in a high traffic environment. The default value is 10.	10	Integer
camel.component.paho.max-reconnect-delay	Get the maximum time (in millis) to wait between reconnects.	128000	Integer
camel.component.paho.mqtt-version	Sets the MQTT version. The default action is to connect with version 3.1.1, and to fall back to 3.1 if that fails. Version 3.1.1 or 3.1 can be selected specifically, with no fall back, by using the MQTT_VERSION_3_1_1 or MQTT_VERSION_3_1 options respectively.		Integer

Name	Description	Default	Type
<code>camel.component.paho.password</code>	Password to be used for authentication against the MQTT broker.		String
<code>camel.component.paho.persistence</code>	Client persistence to be used - memory or file.		PahoPersistence
<code>camel.component.paho.qos</code>	Client quality of service level (0-2).	2	Integer
<code>camel.component.paho.retained</code>	Retain option.	false	Boolean
<code>camel.component.paho.server-uris</code>	Set a list of one or more serverURIs the client may connect to. Multiple servers can be separated by comma. Each serverURI specifies the address of a server that the client may connect to. Two types of connection are supported <code>tcp://</code> for a TCP connection and <code>ssl://</code> for a TCP connection secured by SSL/TLS. For example: <code>tcp://localhost:1883</code> <code>ssl://localhost:8883</code> If the port is not specified, it will default to 1883 for <code>tcp://</code> URIs, and 8883 for <code>ssl://</code> URIs. If <code>serverURIs</code> is set then it overrides the <code>serverURI</code> parameter passed in on the constructor of the MQTT client. When an attempt to connect is initiated the client will start with the first serverURI in the list and work through the list until a connection is established with a server. If a connection cannot be made to any of the servers then the connect attempt fails. Specifying a list of servers that a client may connect to has several uses: High Availability and reliable message delivery Some MQTT servers support a high availability feature where two or more equal MQTT servers share state. An MQTT client can connect to any of the equal servers and be assured that messages are reliably delivered and durable subscriptions are maintained no matter which server the client connects to. The <code>cleansession</code> flag must be set to false if durable subscriptions and/or reliable message delivery is required. Hunt List A set of servers may be specified that are not equal (as in the high availability option). As no state is shared across the servers reliable message delivery and durable subscriptions are not valid. The <code>cleansession</code> flag must be set to true if the hunt list mode is used.		String

Name	Description	Default	Type
camel.component.paho.socket-factory	Sets the SocketFactory to use. This allows an application to apply its own policies around the creation of network sockets. If using an SSL connection, an SSLSocketFactory can be used to supply application-specific security settings. The option is a javax.net.SocketFactory type.		SocketFactory
camel.component.paho.ssl-client-props	Sets the SSL properties for the connection. Note that these properties are only valid if an implementation of the Java Secure Socket Extensions (JSSE) is available. These properties are not used if a custom SocketFactory has been set. The following properties can be used: com.ibm.ssl.protocol One of: SSL, SSLv3, TLS, TLSv1, SSL_TLS. com.ibm.ssl.contextProvider Underlying JSSE provider. For example IBMJSSE2 or SunJSSE com.ibm.ssl.keyStore The name of the file that contains the KeyStore object that you want the KeyManager to use. For example /mydir/etc/key.p12 com.ibm.ssl.keyStorePassword The password for the KeyStore object that you want the KeyManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. com.ibm.ssl.keyStoreType Type of key store, for example PKCS12, JKS, or JCEKS. com.ibm.ssl.keyStoreProvider Key store provider, for example IBMJCE or IBMJCEFIPS. com.ibm.ssl.trustStore The name of the file that contains the KeyStore object that you want the TrustManager to use. com.ibm.ssl.trustStorePassword The password for the TrustStore object that you want the TrustManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. com.ibm.ssl.trustStoreType The type of KeyStore object that you want the default TrustManager to use. Same possible values as keyStoreType. com.ibm.ssl.trustStoreProvider Trust store provider, for example IBMJCE or IBMJCEFIPS. com.ibm.ssl.enabledCipherSuites A list of which ciphers are enabled. Values are dependent on the		Properties

Name	Description	Default	Type
	provider, for example: <code>SSL_RSA_WITH_AES_128_CBC_SHA;SSL_RSA_WITH_3DES_EDE_CBC_SHA</code>. <code>com.ibm.ssl.keyManager</code> Sets the algorithm that will be used to instantiate a <code>KeyManagerFactory</code> object instead of using the default algorithm available in the platform. Example values: <code>IbmX509</code> or <code>IBMJ9X509</code>. <code>com.ibm.ssl.trustManager</code> Sets the algorithm that will be used to instantiate a <code>TrustManagerFactory</code> object instead of using the default algorithm available in the platform. Example values: <code>PKIX</code> or <code>IBMJ9X509</code>. The option is a <code>java.util.Properties</code> type.		
camel.component.paho.ssl-hostname-verifier	Sets the <code>HostnameVerifier</code> for the SSL connection. Note that it will be used after handshake on a connection and you should do actions by yourself when hostname is verified error. There is no default <code>HostnameVerifier</code> . The option is a <code>javax.net.ssl.HostnameVerifier</code> type.		<code>HostnameVerifier</code>
camel.component.paho.user-name	Username to be used for authentication against the MQTT broker.		<code>String</code>
camel.component.paho.will-payload	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		<code>String</code>
camel.component.paho.will-qos	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		<code>Integer</code>
camel.component.paho.will-retained	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.	<code>false</code>	<code>Boolean</code>

Name	Description	Default	Type
camel.component.paho.will-topic	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to The byte payload for the message. The quality of service to publish the message at (0, 1 or 2). Whether or not the message should be retained.		String

CHAPTER 112. PAHO MQTT 5

Both producer and consumer are supported

Paho MQTT5 component provides connector for the MQTT messaging protocol using the [Eclipse Paho](#) library with MQTT v5. Paho is one of the most popular MQTT libraries, so if you would like to integrate it with your Java project – Camel Paho connector is a way to go.

112.1. DEPENDENCIES

When using **paho-mqtt5** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-paho-mqtt5-starter</artifactId>
</dependency>
```

112.2. URI FORMAT

```
paho-mqtt5:topic[?options]
```

Where **topic** is the name of the topic.

112.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

112.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

112.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

112.4. COMPONENT OPTIONS

The Paho MQTT 5 component supports 32 options, which are listed below.

Name	Description	Default	Type
automaticReconnect (common)	Sets whether the client will automatically attempt to reconnect to the server if the connection is lost. If set to false, the client will not attempt to automatically reconnect to the server in the event that the connection is lost. If set to true, in the event that the connection is lost, the client will attempt to reconnect to the server. It will initially wait 1 second before it attempts to reconnect, for every failed reconnect attempt, the delay will double until it is at 2 minutes at which point the delay will stay at 2 minutes.	true	boolean
brokerUrl (common)	The URL of the MQTT broker.	tcp://localhost:1883	String

Name	Description	Default	Type
cleanStart (common)	Sets whether the client and server should remember state across restarts and reconnects. If set to false both the client and server will maintain state across restarts of the client, the server and the connection. As state is maintained: Message delivery will be reliable meeting the specified QOS even if the client, server or connection are restarted. The server will treat a subscription as durable. If set to true the client and server will not maintain state across restarts of the client, the server or the connection. This means Message delivery to the specified QOS cannot be maintained if the client, server or connection are restarted The server will treat a subscription as non-durable.	true	boolean
clientId (common)	MQTT client identifier. The identifier must be unique.		String
configuration (common)	To use the shared Paho configuration.		PahoMqtt5Configuration
connectionTimeout (common)	Sets the connection timeout value. This value, measured in seconds, defines the maximum time interval the client will wait for the network connection to the MQTT server to be established. The default timeout is 30 seconds. A value of 0 disables timeout processing meaning the client will wait until the network connection is made successfully or fails.	30	int
filePersistenceDirectory (common)	Base directory used by file persistence. Will by default use user directory.		String
keepAliveInterval (common)	Sets the keep alive interval. This value, measured in seconds, defines the maximum time interval between messages sent or received. It enables the client to detect if the server is no longer available, without having to wait for the TCP/IP timeout. The client will ensure that at least one message travels across the network within each keep alive period. In the absence of a data-related message during the time period, the client sends a very small ping message, which the server will acknowledge. A value of 0 disables keepalive processing in the client. The default value is 60 seconds.	60	int
maxReconnectDelay (common)	Get the maximum time (in millis) to wait between reconnects.	128000	int

Name	Description	Default	Type
persistence (common)	Client persistence to be used - memory or file. Enum values: • FILE • MEMORY	MEMORY	PahoMqtt5Persistence
qos (common)	Client quality of service level (0-2).	2	int
receiveMaximum (common)	Sets the Receive Maximum. This value represents the limit of QoS 1 and QoS 2 publications that the client is willing to process concurrently. There is no mechanism to limit the number of QoS 0 publications that the Server might try to send. The default value is 65535.	65535	int
retained (common)	Retain option.	false	boolean

Name	Description	Default	Type
serverURIs (common)	Set a list of one or more serverURIs the client may connect to. Multiple servers can be separated by comma. Each serverURI specifies the address of a server that the client may connect to. Two types of connection are supported tcp:// for a TCP connection and ssl:// for a TCP connection secured by SSL/TLS. For example: tcp://localhost:1883 ssl://localhost:8883 If the port is not specified, it will default to 1883 for tcp:// URIs, and 8883 for ssl:// URIs. If serverURIs is set then it overrides the serverURI parameter passed in on the constructor of the MQTT client. When an attempt to connect is initiated the client will start with the first serverURI in the list and work through the list until a connection is established with a server. If a connection cannot be made to any of the servers then the connect attempt fails. Specifying a list of servers that a client may connect to has several uses: High Availability and reliable message delivery Some MQTT servers support a high availability feature where two or more equal MQTT servers share state. An MQTT client can connect to any of the equal servers and be assured that messages are reliably delivered and durable subscriptions are maintained no matter which server the client connects to. The cleansession flag must be set to false if durable subscriptions and/or reliable message delivery is required. Hunt List A set of servers may be specified that are not equal (as in the high availability option). As no state is shared across the servers reliable message delivery and durable subscriptions are not valid. The cleansession flag must be set to true if the hunt list mode is used.		String
sessionExpiryInterval (common)	Sets the Session Expiry Interval. This value, measured in seconds, defines the maximum time that the broker will maintain the session for once the client disconnects. Clients should only connect with a long Session Expiry interval if they intend to connect to the server at some later point in time. By default this value is -1 and so will not be sent, in this case, the session will not expire. If a 0 is sent, the session will end immediately once the Network Connection is closed. When the client has determined that it has no longer any use for the session, it should disconnect with a Session Expiry Interval set to 0.	-1	long
willMqttProperties (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The MQTT properties set for the message.		MqttProperties

Name	Description	Default	Type
willPayload (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The byte payload for the message.		String
willQos (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The quality of service to publish the message at (0, 1 or 2).	1	int
willRetained (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. Whether or not the message should be retained.	false	boolean
willTopic (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
client (advanced)	To use a shared Paho client.		MqttClient
customWebSocketHeaders (advanced)	Sets the Custom WebSocket Headers for the WebSocket Connection.		Map
executorServiceTimeout (advanced)	Set the time in seconds that the executor service should wait when terminating before forcefully terminating. It is not recommended to change this value unless you are absolutely sure that you need to.	1	int
httpsHostnameVerificationEnabled (security)	Whether SSL HostnameVerifier is enabled or not. The default value is true.	true	boolean
password (security)	Password to be used for authentication against the MQTT broker.		String
socketFactory (security)	Sets the SocketFactory to use. This allows an application to apply its own policies around the creation of network sockets. If using an SSL connection, an SSLSocketFactory can be used to supply application-specific security settings.		SocketFactory
sslClientProps (security)	Sets the SSL properties for the connection. Note that these properties are only valid if an implementation of the Java Secure Socket Extensions (JSSE) is available. These properties are not used if a custom SocketFactory has been set. The following properties can be used: com.ibm.ssl.protocol One of: SSL, SSLv3, TLS, TLSv1, SSL_TLS. com.ibm.ssl.contextProvider Underlying JSSE provider. For example IBMJSSE2 or SunJSSE com.ibm.ssl.keyStore The name of the file that contains the KeyStore object that you want the KeyManager to use. For example /mydir/etc/key.p12 com.ibm.ssl.keyStorePassword The password for the KeyStore object that you want the KeyManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a		Properties

Name	Description	Default	Type
	simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. com.ibm.ssl.keyStoreType Type of key store, for example PKCS12, JKS, or JCEKS. com.ibm.ssl.keyStoreProvider Key store provider, for example IBMJCE or IBMJCEFIPS. com.ibm.ssl.trustStore The name of the file that contains the KeyStore object that you want the TrustManager to use. com.ibm.ssl.trustStorePassword The password for the TrustStore object that you want the TrustManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. com.ibm.ssl.trustStoreType The type of KeyStore object that you want the default TrustManager to use. Same possible values as keyStoreType. com.ibm.ssl.trustStoreProvider Trust store provider, for example IBMJCE or IBMJCEFIPS. com.ibm.ssl.enabledCipherSuites A list of which ciphers are enabled. Values are dependent on the provider, for example: SSL_RSA_WITH_AES_128_CBC_SHA;SSL_RSA_WITH_3DES_EDE_CBC_SHA. com.ibm.ssl.keyManager Sets the algorithm that will be used to instantiate a KeyManagerFactory object instead of using the default algorithm available in the platform. Example values: IbmX509 or IBMJ9X509. com.ibm.ssl.trustManager Sets the algorithm that will be used to instantiate a TrustManagerFactory object instead of using the default algorithm available in the platform. Example values: PKIX or IBMJ9X509.		
sslHostnameVerifier (security)	Sets the HostnameVerifier for the SSL connection. Note that it will be used after handshake on a connection and you should do actions by yourself when hostname is verified error. There is no default HostnameVerifier.		HostnameVerifier
userName (security)	Username to be used for authentication against the MQTT broker.		String

112.5. ENDPOINT OPTIONS

The Paho MQTT 5 endpoint is configured using URI syntax:

```
paho-mqtt5:topic
```

with the following path and query parameters:

112.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
topic (common)	Required Name of the topic.		String

112.5.2. Query Parameters (32 parameters)

Name	Description	Default	Type
automaticReconnect (common)	Sets whether the client will automatically attempt to reconnect to the server if the connection is lost. If set to false, the client will not attempt to automatically reconnect to the server in the event that the connection is lost. If set to true, in the event that the connection is lost, the client will attempt to reconnect to the server. It will initially wait 1 second before it attempts to reconnect, for every failed reconnect attempt, the delay will double until it is at 2 minutes at which point the delay will stay at 2 minutes.	true	boolean
brokerUrl (common)	The URL of the MQTT broker.	tcp://localhost:1883	String
cleanStart (common)	Sets whether the client and server should remember state across restarts and reconnects. If set to false both the client and server will maintain state across restarts of the client, the server and the connection. As state is maintained: Message delivery will be reliable meeting the specified QOS even if the client, server or connection are restarted. The server will treat a subscription as durable. If set to true the client and server will not maintain state across restarts of the client, the server or the connection. This means Message delivery to the specified QOS cannot be maintained if the client, server or connection are restarted The server will treat a subscription as non-durable.	true	boolean
clientId (common)	MQTT client identifier. The identifier must be unique.		String

Name	Description	Default	Type
connectionTimeout (common)	Sets the connection timeout value. This value, measured in seconds, defines the maximum time interval the client will wait for the network connection to the MQTT server to be established. The default timeout is 30 seconds. A value of 0 disables timeout processing meaning the client will wait until the network connection is made successfully or fails.	30	int
filePersistenceDirectory (common)	Base directory used by file persistence. Will by default use user directory.		String
keepAliveInterval (common)	Sets the keep alive interval. This value, measured in seconds, defines the maximum time interval between messages sent or received. It enables the client to detect if the server is no longer available, without having to wait for the TCP/IP timeout. The client will ensure that at least one message travels across the network within each keep alive period. In the absence of a data-related message during the time period, the client sends a very small ping message, which the server will acknowledge. A value of 0 disables keepalive processing in the client. The default value is 60 seconds.	60	int
maxReconnectDelay (common)	Get the maximum time (in millis) to wait between reconnects.	128000	int
persistence (common)	Client persistence to be used - memory or file. Enum values: ● FILE ● MEMORY	MEMORY	PahoMqtt5Persistence
qos (common)	Client quality of service level (0-2).	2	int
receiveMaximum (common)	Sets the Receive Maximum. This value represents the limit of QoS 1 and QoS 2 publications that the client is willing to process concurrently. There is no mechanism to limit the number of QoS 0 publications that the Server might try to send. The default value is 65535.	65535	int
retained (common)	Retain option.	false	boolean

Name	Description	Default	Type
serverURIs (common)	Set a list of one or more serverURIs the client may connect to. Multiple servers can be separated by comma. Each serverURI specifies the address of a server that the client may connect to. Two types of connection are supported tcp:// for a TCP connection and ssl:// for a TCP connection secured by SSL/TLS. For example: tcp://localhost:1883 ssl://localhost:8883 If the port is not specified, it will default to 1883 for tcp:// URIs, and 8883 for ssl:// URIs. If serverURIs is set then it overrides the serverURI parameter passed in on the constructor of the MQTT client. When an attempt to connect is initiated the client will start with the first serverURI in the list and work through the list until a connection is established with a server. If a connection cannot be made to any of the servers then the connect attempt fails. Specifying a list of servers that a client may connect to has several uses: High Availability and reliable message delivery Some MQTT servers support a high availability feature where two or more equal MQTT servers share state. An MQTT client can connect to any of the equal servers and be assured that messages are reliably delivered and durable subscriptions are maintained no matter which server the client connects to. The cleansession flag must be set to false if durable subscriptions and/or reliable message delivery is required. Hunt List A set of servers may be specified that are not equal (as in the high availability option). As no state is shared across the servers reliable message delivery and durable subscriptions are not valid. The cleansession flag must be set to true if the hunt list mode is used.		String
sessionExpiryInterval (common)	Sets the Session Expiry Interval. This value, measured in seconds, defines the maximum time that the broker will maintain the session for once the client disconnects. Clients should only connect with a long Session Expiry interval if they intend to connect to the server at some later point in time. By default this value is -1 and so will not be sent, in this case, the session will not expire. If a 0 is sent, the session will end immediately once the Network Connection is closed. When the client has determined that it has no longer any use for the session, it should disconnect with a Session Expiry Interval set to 0.	-1	long

Name	Description	Default	Type
willMqttProperties (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The MQTT properties set for the message.		MqttProperties
willPayload (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The byte payload for the message.		String
willQos (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The quality of service to publish the message at (0, 1 or 2).	1	int
willRetained (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. Whether or not the message should be retained.	false	boolean
willTopic (common)	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to.		String
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
client (advanced)	To use an existing mqtt client.		MqttClient
customWebSocketHeaders (advanced)	Sets the Custom WebSocket Headers for the WebSocket Connection.		Map
executorServiceTimeout (advanced)	Set the time in seconds that the executor service should wait when terminating before forcefully terminating. It is not recommended to change this value unless you are absolutely sure that you need to.	1	int
httpsHostnameVerificationEnabled (security)	Whether SSL HostnameVerifier is enabled or not. The default value is true.	true	boolean
password (security)	Password to be used for authentication against the MQTT broker.		String
socketFactory (security)	Sets the SocketFactory to use. This allows an application to apply its own policies around the creation of network sockets. If using an SSL connection, an SSLSocketFactory can be used to supply application-specific security settings.		SocketFactory

sslClientProps Name (security)	Description	Default	Properties Type
	Sets the SSL properties for the connection. Note that these properties are only valid if an implementation of the Java Secure Socket Extensions (JSSE) is available. These properties are not used if a custom SocketFactory has been set. The following properties can be used: com.ibm.ssl.protocol One of: SSL, SSLv3, TLS, TLSv1, SSL_TLS. com.ibm.ssl.contextProvider Underlying JSSE provider. For example IBMJSSE2 or SunJSSE com.ibm.ssl.keyStore The name of the file that contains the KeyStore object that you want the KeyManager to use. For example /mydir/etc/key.p12 com.ibm.ssl.keyStorePassword The password for the KeyStore object that you want the KeyManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. com.ibm.ssl.keyStoreType Type of key store, for example PKCS12, JKS, or JCEKS. com.ibm.ssl.keyStoreProvider Key store provider, for example IBMJCE or IBMJCEFIPS. com.ibm.ssl.trustStore The name of the file that contains the KeyStore object that you want the TrustManager to use. com.ibm.ssl.trustStorePassword The password for the TrustStore object that you want the TrustManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. com.ibm.ssl.trustStoreType The type of KeyStore object that you want the default TrustManager to use. Same possible values as keyStoreType. com.ibm.ssl.trustStoreProvider Trust store provider, for example IBMJCE or IBMJCEFIPS. com.ibm.ssl.enabledCipherSuites A list of which ciphers are enabled. Values are dependent on the provider, for example: SSL_RSA_WITH_AES_128_CBC_SHA;SSL_RSA_WITH_3DES_EDE_CBC_SHA. com.ibm.ssl.keyManager Sets the algorithm that will be used to instantiate a KeyManagerFactory object instead of using the default algorithm available in the platform. Example values: IbmX509 or IBMJ9X509. com.ibm.ssl.trustManager Sets the algorithm that will be used to instantiate a TrustManagerFactory object instead of using the default algorithm available in the platform. Example values: PKIX or IBMJ9X509.		

Name	Description	Default	Type
<code>sslHostnameVerifier</code> (security)	Sets the <code>HostnameVerifier</code> for the SSL connection. Note that it will be used after handshake on a connection and you should do actions by yourself when hostname is verified error. There is no default <code>HostnameVerifier</code> .		
<code>userName</code> (security)	Username to be used for authentication against the MQTT broker.		String

112.6. HEADERS

The following headers are recognized by the Paho component:

Header	Java constant	Endpoint type	Value type	Description
<code>CamelMqttTopic</code>	<code>PahoConstants.MQTT_TOPIC</code>	Consumer	String	The name of the topic
<code>CamelMqttQoS</code>	<code>PahoConstants.MQTT_QOS</code>	Consumer	Integer	QualityOfService of the incoming message
<code>CamelPahoOverrideTopic</code>	<code>PahoConstants.CAMEL_PAHO_OVERRIDE_TOPIC</code>	Producer	String	Name of topic to override and send to instead of topic specified on endpoint

112.7. DEFAULT PAYLOAD TYPE

By default Camel Paho component operates on the binary payloads extracted out of (or put into) the MQTT message:

```
// Receive payload
byte[] payload = (byte[]) consumerTemplate.receiveBody("paho:topic");

// Send payload
byte[] payload = "message".getBytes();
producerTemplate.sendBody("paho:topic", payload);
```

But of course Camel build-in [type conversion API](#) can perform the automatic data type transformations for you. In the example below Camel automatically converts binary payload into **String** (and conversely):

```
// Receive payload
String payload = consumerTemplate.receiveBody("paho:topic", String.class);

// Send payload
String payload = "message";
producerTemplate.sendBody("paho:topic", payload);
```

112.8. SAMPLES

For example the following snippet reads messages from the MQTT broker installed on the same host as the Camel router:

```
from("paho:some/queue")
.to("mock:test");
```

While the snippet below sends message to the MQTT broker:

```
from("direct:test")
.to("paho:some/target/queue");
```

For example this is how to read messages from the remote MQTT broker:

```
from("paho:some/queue?brokerUrl=tcp://iot.eclipse.org:1883")
.to("mock:test");
```

And here we override the default topic and set to a dynamic topic

```
from("direct:test")
.setHeader(PahoConstants.CAMEL_PAHO_OVERRIDE_TOPIC, simple("${header.customerId}"))
.to("paho:some/target/queue");
```

112.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 33 options, which are listed below.

Name	Description	Default	Type
camel.component.paho-mqtt5.automatic-reconnect	Sets whether the client will automatically attempt to reconnect to the server if the connection is lost. If set to false, the client will not attempt to automatically reconnect to the server in the event that the connection is lost. If set to true, in the event that the connection is lost, the client will attempt to reconnect to the server. It will initially wait 1 second before it attempts to reconnect, for every failed reconnect attempt, the delay will double until it is at 2 minutes at which point the delay will stay at 2 minutes.	true	Boolean
camel.component.paho-mqtt5.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.paho-mqtt5.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.paho-mqtt5.broker-url	The URL of the MQTT broker.	tcp://localhost:1883	String
camel.component.paho-mqtt5.clean-start	Sets whether the client and server should remember state across restarts and reconnects. If set to false both the client and server will maintain state across restarts of the client, the server and the connection. As state is maintained: Message delivery will be reliable meeting the specified QOS even if the client, server or connection are restarted. The server will treat a subscription as durable. If set to true the client and server will not maintain state across restarts of the client, the server or the connection. This means Message delivery to the specified QOS cannot be maintained if the client, server or connection are restarted The server will treat a subscription as non-durable.	true	Boolean
camel.component.paho-mqtt5.client	To use a shared Paho client. The option is a <code>org.eclipse.paho.mqttv5.client.MqttClient</code> type.		MqttClient
camel.component.paho-mqtt5.client-id	MQTT client identifier. The identifier must be unique.		String
camel.component.paho-mqtt5.configuration	To use the shared Paho configuration. The option is a <code>org.apache.camel.component.paho.mqtt5.PahoMqtt5Configuration</code> type.		PahoMqtt5Configuration

Name	Description	Default	Type
<code>camel.component.paho-mqtt5.connection-timeout</code>	Sets the connection timeout value. This value, measured in seconds, defines the maximum time interval the client will wait for the network connection to the MQTT server to be established. The default timeout is 30 seconds. A value of 0 disables timeout processing meaning the client will wait until the network connection is made successfully or fails.	30	Integer
<code>camel.component.paho-mqtt5.custom-web-socket-headers</code>	Sets the Custom WebSocket Headers for the WebSocket Connection.		Map
<code>camel.component.paho-mqtt5.enabled</code>	Whether to enable auto configuration of the paho-mqtt5 component. This is enabled by default.		Boolean
<code>camel.component.paho-mqtt5.executor-service-timeout</code>	Set the time in seconds that the executor service should wait when terminating before forcefully terminating. It is not recommended to change this value unless you are absolutely sure that you need to.	1	Integer
<code>camel.component.paho-mqtt5.file-persistence-directory</code>	Base directory used by file persistence. Will by default use user directory.		String
<code>camel.component.paho-mqtt5.https-hostname-verification-enabled</code>	Whether SSL HostnameVerifier is enabled or not. The default value is true.	true	Boolean
<code>camel.component.paho-mqtt5.keep-alive-interval</code>	Sets the keep alive interval. This value, measured in seconds, defines the maximum time interval between messages sent or received. It enables the client to detect if the server is no longer available, without having to wait for the TCP/IP timeout. The client will ensure that at least one message travels across the network within each keep alive period. In the absence of a data-related message during the time period, the client sends a very small ping message, which the server will acknowledge. A value of 0 disables keepalive processing in the client. The default value is 60 seconds.	60	Integer

Name	Description	Default	Type
camel.component.paho-mqtt5.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.paho-mqtt5.max-reconnect-delay	Get the maximum time (in millis) to wait between reconnects.	128000	Integer
camel.component.paho-mqtt5.password	Password to be used for authentication against the MQTT broker.		String
camel.component.paho-mqtt5.persistence	Client persistence to be used - memory or file.		PahoMqtt5Persistence
camel.component.paho-mqtt5.qos	Client quality of service level (0-2).	2	Integer
camel.component.paho-mqtt5.receive-maximum	Sets the Receive Maximum. This value represents the limit of QoS 1 and QoS 2 publications that the client is willing to process concurrently. There is no mechanism to limit the number of QoS 0 publications that the Server might try to send. The default value is 65535.	65535	Integer
camel.component.paho-mqtt5.retained	Retain option.	false	Boolean

Name	Description	Default	Type
camel.component.paho-mqtt5.server-uris	Set a list of one or more serverURIs the client may connect to. Multiple servers can be separated by comma. Each serverURI specifies the address of a server that the client may connect to. Two types of connection are supported tcp:// for a TCP connection and ssl:// for a TCP connection secured by SSL/TLS. For example: tcp://localhost:1883 ssl://localhost:8883 If the port is not specified, it will default to 1883 for tcp:// URIs, and 8883 for ssl:// URIs. If serverURIs is set then it overrides the serverURI parameter passed in on the constructor of the MQTT client. When an attempt to connect is initiated the client will start with the first serverURI in the list and work through the list until a connection is established with a server. If a connection cannot be made to any of the servers then the connect attempt fails. Specifying a list of servers that a client may connect to has several uses: High Availability and reliable message delivery Some MQTT servers support a high availability feature where two or more equal MQTT servers share state. An MQTT client can connect to any of the equal servers and be assured that messages are reliably delivered and durable subscriptions are maintained no matter which server the client connects to. The cleansession flag must be set to false if durable subscriptions and/or reliable message delivery is required. Hunt List A set of servers may be specified that are not equal (as in the high availability option). As no state is shared across the servers reliable message delivery and durable subscriptions are not valid. The cleansession flag must be set to true if the hunt list mode is used.		String
camel.component.paho-mqtt5.session-expiry-interval	Sets the Session Expiry Interval. This value, measured in seconds, defines the maximum time that the broker will maintain the session for once the client disconnects. Clients should only connect with a long Session Expiry interval if they intend to connect to the server at some later point in time. By default this value is -1 and so will not be sent, in this case, the session will not expire. If a 0 is sent, the session will end immediately once the Network Connection is closed. When the client has determined that it has no longer any use for the session, it should disconnect with a Session Expiry Interval set to 0.	-1	Long

Name	Description	Default	Type
camel.component.paho-mqtt5.socket-factory	Sets the SocketFactory to use. This allows an application to apply its own policies around the creation of network sockets. If using an SSL connection, an SSLSocketFactory can be used to supply application-specific security settings. The option is a javax.net.SocketFactory type.		SocketFactory
camel.component.paho-mqtt5.ssl-client-props	Sets the SSL properties for the connection. Note that these properties are only valid if an implementation of the Java Secure Socket Extensions (JSSE) is available. These properties are not used if a custom SocketFactory has been set. The following properties can be used: - com.ibm.ssl.protocol One of: SSL, SSLv3, TLS, TLSv1, SSL_TLS. - com.ibm.ssl.contextProvider Underlying JSSE provider. For example IBMJSSE2 or SunJSSE - com.ibm.ssl.keyStore The name of the file that contains the KeyStore object that you want the KeyManager to use. For example /mydir/etc/key.p12 - com.ibm.ssl.keyStorePassword The password for the KeyStore object that you want the KeyManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. - com.ibm.ssl.keyStoreType Type of key store, for example PKCS12, JKS, or JCEKS. - com.ibm.ssl.keyStoreProvider Key store provider, for example IBMJCE or IBMJCEFPIS. - com.ibm.ssl.trustStore The name of the file that contains the KeyStore object that you want the TrustManager to use. - com.ibm.ssl.trustStorePassword The password for the TrustStore object that you want the TrustManager to use. The password can either be in plain-text, or may be obfuscated using the static method: com.ibm.micro.security.Password.obfuscate(char password). This obfuscates the password using a simple and insecure XOR and Base64 encoding mechanism. Note that this is only a simple scrambler to obfuscate clear-text passwords. - com.ibm.ssl.trustStoreType The type of KeyStore object that you want the default TrustManager to use. Same possible values as keyStoreType. - com.ibm.ssl.trustStoreProvider Trust store provider, for example IBMJCE or IBMJCEFPIS. - com.ibm.ssl.enabledCipherSuites A list of which ciphers are enabled. Values are dependent on the		Properties

Name	Description	Default	Type
	provider, for example: <code>SSL_RSA_WITH_AES_128_CBC_SHA;SSL_RSA_WITH_3DES_EDE_CBC_SHA</code>. <code>com.ibm.ssl.keyManager</code> Sets the algorithm that will be used to instantiate a <code>KeyManagerFactory</code> object instead of using the default algorithm available in the platform. Example values: <code>IbmX509</code> or <code>IBMJ9X509</code>. <code>com.ibm.ssl.trustManager</code> Sets the algorithm that will be used to instantiate a <code>TrustManagerFactory</code> object instead of using the default algorithm available in the platform. Example values: <code>PKIX</code> or <code>IBMJ9X509</code>. The option is a <code>java.util.Properties</code> type.		
<code>camel.component.paho-mqtt5.ssl-hostname-verifier</code>	Sets the <code>HostnameVerifier</code> for the SSL connection. Note that it will be used after handshake on a connection and you should do actions by yourself when hostname is verified error. There is no default <code>HostnameVerifier</code> . The option is a <code>javax.net.ssl.HostnameVerifier</code> type.		<code>HostnameVerifier</code>
<code>camel.component.paho-mqtt5.user-name</code>	Username to be used for authentication against the MQTT broker.		<code>String</code>
<code>camel.component.paho-mqtt5.will-mqtt-properties</code>	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The MQTT properties set for the message. The option is a <code>org.eclipse.paho.mqttv5.common.packet.MqttProperties</code> type.		<code>MqttProperties</code>
<code>camel.component.paho-mqtt5.will-payload</code>	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The byte payload for the message.		<code>String</code>
<code>camel.component.paho-mqtt5.will-qos</code>	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The quality of service to publish the message at (0, 1 or 2).	1	<code>Integer</code>
<code>camel.component.paho-mqtt5.will-retained</code>	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. Whether or not the message should be retained.	false	<code>Boolean</code>

Name	Description	Default	Type
camel.component.paho-mqtt5.will-topic	Sets the Last Will and Testament (LWT) for the connection. In the event that this client unexpectedly loses its connection to the server, the server will publish a message to itself using the supplied details. The topic to publish to.		String

CHAPTER 113. PLATFORM HTTP

Since Camel 3.0

Only consumer is supported

The Platform HTTP is used to allow Camel to use the existing HTTP server from the runtime, for example when running Camel on Spring Boot, Quarkus, or other runtimes.

113.1. DEPENDENCIES

When using **platform-http** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-platform-http-starter</artifactId>
</dependency>
```

113.2. PLATFORM HTTP PROVIDER

To use Platform HTTP a provider (engine) is required to be available on the classpath. The purpose is to have drivers for different runtimes such as Quarkus, VertX, or Spring Boot.

At this moment there is only support for Quarkus and VertX by **camel-platform-http-vertx**. This JAR must be on the classpath otherwise the Platform HTTP component cannot be used and an exception will be thrown on startup.

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-platform-http-vertx</artifactId>
  <version>4.10.3.redhat-00025</version>
  <!-- use the same version as your Camel core version -->
</dependency>
```

113.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

113.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

113.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

113.4. COMPONENT OPTIONS

The Platform HTTP component supports 5 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
handleWriteResponseError (consumer)	When Camel is complete processing the message, and the HTTP server is writing response. This option controls whether Camel should catch any failure during writing response and store this on the Exchange, which allows <code>onCompletion/UnitOfWork</code> to regard the Exchange as failed and have access to the caused exception from the HTTP server.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
engine (advanced)	An HTTP Server engine implementation to serve the requests.		PlatformHttpEngine
headerFilterStrategy (filter)	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy

113.4.1. Endpoint Options

The Platform HTTP endpoint is configured using URI syntax:

```
platform-http:path
```

with the following path and query parameters:

113.4.1.1. Path Parameters (1 parameters)

Name	Description	Default	Type
path (consumer)	Required The path under which this endpoint serves the HTTP requests, for proxy use 'proxy'.		String

113.4.1.2. Query Parameters (21 parameters)

Name	Description	Default	Type
consumes (consumer)	The content type this endpoint accepts as an input, such as application/xml or application/json. null or / mean no restriction.		String
cookieDomain (consumer)	Sets which server can receive cookies.		String

Name	Description	Default	Type
cookieHttpOnly (consumer)	Sets whether to prevent client side scripts from accessing created cookies.	false	boolean
cookieMaxAge (consumer)	Sets the maximum cookie age in seconds.		Long
cookiePath (consumer)	Sets the URL path that must exist in the requested URL in order to send the Cookie.	/	String
cookieSameSite (consumer)	Sets whether to prevent the browser from sending cookies along with cross-site requests. Enum values: • STRICT • LAX • NONE	Lax	CookieSameSite
cookieSecure (consumer)	Sets whether the cookie is only sent to the server with an encrypted request over HTTPS.	false	boolean
handleWriteResponseError (consumer)	When Camel is complete processing the message, and the HTTP server is writing response. This option controls whether Camel should catch any failure during writing response and store this on the Exchange, which allows onCompletion/UnitOfWork to regard the Exchange as failed and have access to the caused exception from the HTTP server.	false	boolean
httpMethodRestrict (consumer)	A comma separated list of HTTP methods to serve, e.g. GET,POST . If no methods are specified, all methods will be served.		String
matchOnUriPrefix (consumer)	Whether or not the consumer should try to find a target consumer by matching the URI prefix if no exact match is found.	false	boolean
muteException (consumer)	If enabled and an Exchange failed processing on the consumer side the response's body won't contain the exception's stack trace.	true	boolean
produces (consumer)	The content type this endpoint produces, such as application/xml or application/json.		String

Name	Description	Default	Type
returnHttpRequestHeaders (consumer)	Whether to include HTTP request headers (Accept, User-Agent, etc.) into HTTP response produced by this endpoint.	false	boolean
useCookieHandler (consumer)	Whether to enable the Cookie Handler that allows Cookie addition, expiry, and retrieval (currently only supported by camel-platform-http-vertx).	false	boolean
useStreaming (consumer)	Whether to use streaming for large requests and responses (currently only supported by camel-platform-http-vertx).	false	boolean
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
fileNameExtWhitelist (consumer (advanced))	A comma or whitespace separated list of file extensions. Uploads having these extensions will be stored locally. Null value or asterisk (*) will allow all files.		String
headerFilterStrategy (advanced)	To use a custom HeaderFilterStrategy to filter headers to and from Camel message.		HeaderFilterStrategy

Name	Description	Default	Type
platformHttpEngine (advanced)	An HTTP Server engine implementation to serve the requests of this endpoint.		PlatformHttpEngine

113.5. IMPLEMENTING A REVERSE PROXY

Platform HTTP component can act as a reverse proxy, in that case some headers are populated from the absolute URL received on the request line of the HTTP request. Those headers are specific to the underlining platform.

At this moment, this feature is only supported for Vert.x in **camel-platform-http-vertx** component.

113.6. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
camel.component. platform- http.autowired- enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component. platform- http.bridge- error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component. platform- http.enabled	Whether to enable auto configuration of the platform-http component. This is enabled by default.		Boolean
camel.component. platform- http.engine	An HTTP Server engine implementation to serve the requests. The option is a <code>org.apache.camel.component.platform.http.spi.PlatformHttpEngine</code> type.		PlatformHttpEngine

Name	Description	Default	Type
<code>camel.component. platform- http.handle- write-response- error</code>	When Camel is complete processing the message, and the HTTP server is writing response. This option controls whether Camel should catch any failure during writing response and store this on the Exchange, which allows onCompletion/UnitOfWork to regard the Exchange as failed and have access to the caused exception from the HTTP server.	false	Boolean
<code>camel.component. platform- http.header- filter-strategy</code>	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		HeaderFilterStrategy

CHAPTER 114. PROTOBUF JACKSON

Jackson Protobuf is a Data Format which uses the [Jackson library](#) with the [Protobuf extension](#) to unmarshal a Protobuf payload into Java objects or to marshal Java objects into a Protobuf payload.



NOTE

If you are familiar with Jackson, this Protobuf data format behaves in the same way as its JSON counterpart, and thus can be used with classes annotated for JSON serialization/deserialization.

```
from("kafka:topic").
  unmarshal().protobuf(ProtobufLibrary.Jackson, JsonNode.class).
  to("log:info");
```

114.1. DEPENDENCIES

When using **protobuf-jackson** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-jackson-protobuf-starter</artifactId>
</dependency>
```

114.2. CONFIGURING THE SCHEMARESOLVER

Since Protobuf serialization is schema-based, this data format requires that you provide a SchemaResolver object that is able to lookup the schema for each exchange that is going to be marshalled/unmarshalled.

You can add a single SchemaResolver to the registry and it will be looked up automatically. Or you can explicitly specify the reference to a custom SchemaResolver.

114.3. PROTOBUF JACKSON OPTIONS

The Protobuf Jackson dataformat supports 18 options, which are listed below.

Name	Default	Java Type	Description
contentTypeHeader		Boolean	Whether the data format should set the Content-Type header with the type from the data format. For example application/xml for data formats marshalling to XML, or application/json for data formats marshalling to JSON.
objectMapper		String	Lookup and use the existing ObjectMapper with the given id when using Jackson.

Name	Default	Java Type	Description
<code>useDefaultObjectMapper</code>		Boolean	Whether to lookup and use default Jackson ObjectMapper from the registry.
<code>unmarshalType</code>		String	Class name of the java type to use when unmarshalling.
<code>jsonView</code>		String	When marshalling a POJO to JSON you might want to exclude certain fields from the JSON output. With Jackson you can use JSON views to accomplish this. This option is to refer to the class which has JsonView annotations.
<code>include</code>		String	If you want to marshal a pojo to JSON, and the pojo has some fields with null values. And you want to skip these null values, you can set this option to NON_NULL.
<code>allowJmsType</code>		Boolean	Used for JMS users to allow the JMSType header from the JMS spec to specify a FQN classname to use to unmarshal to.
<code>collectionType</code>		String	Refers to a custom collection type to lookup in the registry to use. This option should rarely be used, but allows to use different collection types than java.util.Collection based as default.
<code>useList</code>		Boolean	To unmarshal to a List of Map or a List of Pojo.
<code>moduleClassNames</code>		String	To use custom Jackson modules com.fasterxml.jackson.databind.Module specified as a String with FQN class names. Multiple classes can be separated by comma.
<code>moduleRefs</code>		String	To use custom Jackson modules referred from the Camel registry. Multiple modules can be separated by comma.
<code>enableFeatures</code>		String	Set of features to enable on the Jackson com.fasterxml.jackson.databind.ObjectMapper. The features should be a name that matches a enum from com.fasterxml.jackson.databind.SerializationFeature, com.fasterxml.jackson.databind.DeserializationFeature, or com.fasterxml.jackson.databind.MapperFeature Multiple features can be separated by comma.

Name	Default	Java Type	Description
disableFeatures		String	Set of features to disable on the Jackson <code>com.fasterxml.jackson.databind.ObjectMapper</code> . The features should be a name that matches a enum from <code>com.fasterxml.jackson.databind.SerializationFeature</code> , <code>com.fasterxml.jackson.databind.DeserializationFeature</code> , or <code>com.fasterxml.jackson.databind.MapperFeature</code> . Multiple features can be separated by comma.
allowUnmarshalType		Boolean	If enabled then Jackson is allowed to attempt to use the <code>CamelJacksonUnmarshalType</code> header during the unmarshalling. This should only be enabled when desired to be used.
timezone		String	If set then Jackson will use the Timezone when marshalling/unmarshalling.
autoDiscoverObjectMapper		Boolean	If set to true then Jackson will lookup for an <code>objectMapper</code> into the registry.
schemaResolver		String	Optional schema resolver used to lookup schemas for the data in transit.
autoDiscoverSchemaResolver		Boolean	When not disabled, the <code>SchemaResolver</code> will be looked up into the registry.

114.4. USING CUSTOM PROTOBUFMAPPER

You can configure **JacksonProtobufDataFormat** to use a custom **ProtobufMapper** in case you need more control of the mapping configuration.

If you setup a single **ProtobufMapper** in the registry, then Camel will automatic lookup and use this **ProtobufMapper**.

114.5. SPRING BOOT AUTO-CONFIGURATION

The component supports 19 options, which are listed below.

Name	Description	Default	Type
camel.dataformat.protobuf-jackson.allow-jms-type	Used for JMS users to allow the <code>JMSType</code> header from the JMS spec to specify a FQN classname to use to unmarshal to.	false	Boolean

Name	Description	Default	Type
<code>camel.dataformat.protobuf-jackson.allow-unmarshall-type</code>	If enabled then Jackson is allowed to attempt to use the <code>CamelJacksonUnmarshalType</code> header during the unmarshalling. This should only be enabled when desired to be used.	false	Boolean
<code>camel.dataformat.protobuf-jackson.auto-discover-object-mapper</code>	If set to true then Jackson will lookup for an <code>objectMapper</code> into the registry.	false	Boolean
<code>camel.dataformat.protobuf-jackson.auto-discover-schema-resolver</code>	When not disabled, the <code>SchemaResolver</code> will be looked up into the registry.	true	Boolean
<code>camel.dataformat.protobuf-jackson.collection-type</code>	Refers to a custom collection type to lookup in the registry to use. This option should rarely be used, but allows to use different collection types than <code>java.util.Collection</code> based as default.		String
<code>camel.dataformat.protobuf-jackson.content-type-header</code>	Whether the data format should set the <code>Content-Type</code> header with the type from the data format. For example <code>application/xml</code> for data formats marshalling to XML, or <code>application/json</code> for data formats marshalling to JSON.	true	Boolean
<code>camel.dataformat.protobuf-jackson.disable-features</code>	Set of features to disable on the Jackson <code>com.fasterxml.jackson.databind.ObjectMapper</code> . The features should be a name that matches a enum from <code>com.fasterxml.jackson.databind.SerializationFeature</code> , <code>com.fasterxml.jackson.databind.DeserializationFeature</code> , or <code>com.fasterxml.jackson.databind.MapperFeature</code> . Multiple features can be separated by comma.		String
<code>camel.dataformat.protobuf-jackson.enable-features</code>	Set of features to enable on the Jackson <code>com.fasterxml.jackson.databind.ObjectMapper</code> . The features should be a name that matches a enum from <code>com.fasterxml.jackson.databind.SerializationFeature</code> , <code>com.fasterxml.jackson.databind.DeserializationFeature</code> , or <code>com.fasterxml.jackson.databind.MapperFeature</code> . Multiple features can be separated by comma.		String
<code>camel.dataformat.protobuf-jackson.enabled</code>	Whether to enable auto configuration of the <code>protobuf-jackson</code> data format. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.dataformat.protobuf-jackson.include</code>	If you want to marshal a pojo to JSON, and the pojo has some fields with null values. And you want to skip these null values, you can set this option to <code>NON_NULL</code> .		String
<code>camel.dataformat.protobuf-jackson.json-view</code>	When marshalling a POJO to JSON you might want to exclude certain fields from the JSON output. With Jackson you can use JSON views to accomplish this. This option is to refer to the class which has <code>JsonView</code> annotations.		String
<code>camel.dataformat.protobuf-jackson.module-class-names</code>	To use custom Jackson modules <code>com.fasterxml.jackson.databind.Module</code> specified as a String with FQN class names. Multiple classes can be separated by comma.		String
<code>camel.dataformat.protobuf-jackson.module-refs</code>	To use custom Jackson modules referred from the Camel registry. Multiple modules can be separated by comma.		String
<code>camel.dataformat.protobuf-jackson.object-mapper</code>	Lookup and use the existing ObjectMapper with the given id when using Jackson.		String
<code>camel.dataformat.protobuf-jackson.schema-resolver</code>	Optional schema resolver used to lookup schemas for the data in transit.		String
<code>camel.dataformat.protobuf-jackson.timezone</code>	If set then Jackson will use the Timezone when marshalling/unmarshalling.		String
<code>camel.dataformat.protobuf-jackson.unmarshal-type</code>	Class name of the java type to use when unmarshalling.		String
<code>camel.dataformat.protobuf-jackson.use-default-object-mapper</code>	Whether to lookup and use default Jackson ObjectMapper from the registry.	true	Boolean

Name	Description	Default	Type
<code>camel.dataformat.protobuf-jackson.use-list</code>	To unmarshal to a List of Map or a List of Pojo.	false	Boolean

CHAPTER 115. QUARTZ

Only consumer is supported

The Quartz component provides a scheduled delivery of messages using the [Quartz Scheduler 2.x](#). Each endpoint represents a different timer (in Quartz terms, a Trigger and JobDetail).

115.1. DEPENDENCIES

When using **quartz** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-quartz-starter</artifactId>
</dependency>
```

115.2. URI FORMAT

```
quartz://timerName?options
quartz://groupName/timerName?options
quartz://groupName/timerName?cron=expression
quartz://timerName?cron=expression
```

The component uses either a **CronTrigger** or a **SimpleTrigger**. If no cron expression is provided, the component uses a simple trigger. If no **groupName** is provided, the quartz component uses the **Camel** group name.

115.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

115.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

115.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

115.4. COMPONENT OPTIONS

The Quartz component supports 13 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
enableJmx (consumer)	Whether to enable Quartz JMX which allows to manage the Quartz scheduler from JMX. This options is default true.	true	boolean
prefixInstanceName (consumer)	Whether to prefix the Quartz Scheduler instance name with the CamelContext name. This is enabled by default, to let each CamelContext use its own Quartz scheduler instance by default. You can set this option to false to reuse Quartz scheduler instances between multiple CamelContext's.	true	boolean
prefixJobNameWithEndpointId (consumer)	Whether to prefix the quartz job with the endpoint id. This option is default false.	false	boolean

Name	Description	Default	Type
properties (consumer)	Properties to configure the Quartz scheduler.		Map
propertiesFile (consumer)	File name of the properties to load from the classpath.		String
propertiesRef (consumer)	References to an existing Properties or Map to lookup in the registry to use for configuring quartz.		String
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
scheduler (advanced)	To use the custom configured Quartz scheduler, instead of creating a new Scheduler.		Scheduler
schedulerFactory (advanced)	To use the custom SchedulerFactory which is used to create the Scheduler.		SchedulerFactory
autoStartScheduler (scheduler)	Whether or not the scheduler should be auto started. This options is default true.	true	boolean
interruptJobsOnShutdown (scheduler)	Whether to interrupt jobs on shutdown which forces the scheduler to shutdown quicker and attempt to interrupt any running jobs. If this is enabled then any running jobs can fail due to being interrupted. When a job is interrupted then Camel will mark the exchange to stop continue routing and set <code>java.util.concurrent.RejectedExecutionException</code> as caused exception. Therefore use this with care, as its often better to allow Camel jobs to complete and shutdown gracefully.	false	boolean
startDelayedSeconds (scheduler)	Seconds to wait before starting the quartz scheduler.		int

115.5. ENDPOINT OPTIONS

The Quartz endpoint is configured using URI syntax:

```
quartz:groupName/triggerName
```

with the following path and query parameters:

115.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
groupName (consumer)	The quartz group name to use. The combination of group name and trigger name should be unique.	Camel	String
triggerName (consumer)	Required The quartz trigger name to use. The combination of group name and trigger name should be unique.		String

115.5.2. Query Parameters (17 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
cron (consumer)	Specifies a cron expression to define when to trigger.		String
deleteJob (consumer)	If set to true, then the trigger automatically delete when route stop. Else if set to false, it will remain in scheduler. When set to false, it will also mean user may reuse pre-configured trigger with camel Uri. Just ensure the names match. Notice you cannot have both <code>deleteJob</code> and <code>pauseJob</code> set to true.	true	boolean
durableJob (consumer)	Whether or not the job should remain stored after it is orphaned (no triggers point to it).	false	boolean
pauseJob (consumer)	If set to true, then the trigger automatically pauses when route stop. Else if set to false, it will remain in scheduler. When set to false, it will also mean user may reuse pre-configured trigger with camel Uri. Just ensure the names match. Notice you cannot have both <code>deleteJob</code> and <code>pauseJob</code> set to true.	false	boolean
recoverableJob (consumer)	Instructs the scheduler whether or not the job should be re-executed if a 'recovery' or 'fail-over' situation is encountered.	false	boolean

Name	Description	Default	Type
stateful (consumer)	Uses a Quartz PersistJobDataAfterExecution and DisallowConcurrentExecution instead of the default job.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern
customCalendar (advanced)	Specifies a custom calendar to avoid specific range of date.		Calendar
jobParameters (advanced)	To configure additional options on the job.		Map
prefixJobNameWithEndpointId (advanced)	Whether the job name should be prefixed with endpoint id.	false	boolean
triggerParameters (advanced)	To configure additional options on the trigger.		Map
usingFixedCamelContextName (advanced)	If it is true, JobDataMap uses the CamelContext name directly to reference the CamelContext, if it is false, JobDataMap uses use the CamelContext management name which could be changed during the deploy time.	false	boolean
autoStartScheduler (scheduler)	Whether or not the scheduler should be auto started.	true	boolean
startDelayedSeconds (scheduler)	Seconds to wait before starting the quartz scheduler.		int

Name	Description	Default	Type
triggerStartDelay (scheduler)	In case of scheduler has already started, we want the trigger start slightly after current time to ensure endpoint is fully started before the job kicks in. Negative value shifts trigger start time in the past.	500	long

115.5.3. Configuring quartz.properties file

By default Quartz will look for a **quartz.properties** file in the **org/quartz** directory of the classpath. If you are using WAR deployments this means just drop the quartz.properties in **WEB-INF/classes/org/quartz**.

However the Camel [Quartz](#) component also allows you to configure properties:

Parameter	Default	Type	Description
properties	null	Properties	You can configure a java.util.Properties instance.
propertiesFile	null	String	File name of the properties to load from the classpath

To do this you can configure this in Spring XML as follows

```
<bean id="quartz" class="org.apache.camel.component.quartz.QuartzComponent">
  <property name="propertiesFile" value="com/mycompany/myquartz.properties"/>
</bean>
```

115.6. ENABLING QUARTZ SCHEDULER IN JMX

You need to configure the quartz scheduler properties to enable JMX. That is typically setting the option "**org.quartz.scheduler.jmx.export**" to a **true** value in the configuration file.

This option is set to true by default, unless explicitly disabled.

115.7. STARTING THE QUARTZ SCHEDULER

The [Quartz](#) component offers an option to let the Quartz scheduler be started delayed, or not auto started at all.

This is an example:

```
<bean id="quartz" class="org.apache.camel.component.quartz.QuartzComponent">
  <property name="startDelayedSeconds" value="5"/>
</bean>
```

115.8. CLUSTERING

If you use Quartz in clustered mode, e.g. the **JobStore** is clustered. Then the [Quartz](#) component will not pause/remove triggers when a node is being stopped/shutdown. This allows the trigger to keep running on the other nodes in the cluster.



NOTE

When running in clustered node no checking is done to ensure unique job name/group for endpoints.

115.9. MESSAGE HEADERS

Camel adds the getters from the Quartz Execution Context as header values. The following headers are added:

calendar, fireTime, jobDetail, jobInstance, jobRunTime, mergedJobDataMap, nextFireTime, previousFireTime, refireCount, result, scheduledFireTime, scheduler, trigger, triggerName, triggerGroup.

The **fireTime** header contains the **java.util.Date** of when the exchange was fired.

115.10. USING CRON TRIGGERS

Quartz supports [Cron-like expressions](#) for specifying timers in a handy format. You can use these expressions in the **cron** URI parameter; though to preserve valid URI encoding we allow **+** to be used instead of spaces.

For example, the following will fire a message every five minutes starting at 12pm (noon) to 6pm on weekdays:

```
from("quartz://myGroup/myTimerName?cron=0+0/5+12-18+?+*+MON-FRI")
.to("activemq:Totally.Rocks");
```

which is equivalent to using the cron expression

```
0 0/5 12-18 ? * MON-FRI
```

The following table shows the URI character encodings we use to preserve valid URI syntax:

URI Character	Cron character
+	Space

115.11. SPECIFYING TIME ZONE

The Quartz Scheduler allows you to configure time zone per trigger. For example to use a timezone of your country, then you can do as follows:

```
quartz://groupName/timerName?cron=0+0/5+12-18+?+*+MON-FRI&trigger.timeZone=Europe/Stockholm
```

The `timeZone` value is the values accepted by `java.util.TimeZone`.

115.12. CONFIGURING MISFIRE INSTRUCTIONS

The quartz scheduler can be configured with a misfire instruction to handle misfire situations for the trigger. The concrete trigger type that you are using will have defined a set of additional **MISFIRE_INSTRUCTION_XXX** constants that may be set as this property's value.

For example to configure the simple trigger to use misfire instruction 4:

```
quartz://myGroup/myTimerName?trigger.repeatInterval=2000&trigger.misfireInstruction=4
```

And likewise you can configure the cron trigger with one of its misfire instructions as well:

```
quartz://myGroup/myTimerName?cron=0/2+*+*+*+*?&trigger.misfireInstruction=2
```

The simple and cron triggers has the following misfire instructions representative:

115.12.1. SimpleTrigger.MISFIRE_INSTRUCTION_FIRE_NOW = 1 (default)

Instructs the Scheduler that upon a mis-fire situation, the SimpleTrigger wants to be fired now by Scheduler.

This instruction should typically only be used for 'one-shot' (non-repeating) Triggers. If it is used on a trigger with a repeat count > 0 then it is equivalent to the instruction `MISFIRE_INSTRUCTION_RESCHEDULE_NOW_WITH_REMAINING_REPEAT_COUNT`.

115.12.2. SimpleTrigger.MISFIRE_INSTRUCTION_RESCHEDULE_NOW_WITH_EXISTING_REPEAT_COUNT = 2

Instructs the Scheduler that upon a mis-fire situation, the SimpleTrigger wants to be re-scheduled to 'now' (even if the associated Calendar excludes 'now') with the repeat count left as-is. This does obey the Trigger end-time however, so if 'now' is after the end-time the Trigger will not fire again.

Use of this instruction causes the trigger to 'forget' the start-time and repeat-count that it was originally setup with (this is only an issue if you for some reason wanted to be able to tell what the original values were at some later time).

115.12.3. SimpleTrigger.MISFIRE_INSTRUCTION_RESCHEDULE_NOW_WITH_REMAINING_REPEAT_COUNT = 3

Instructs the Scheduler that upon a mis-fire situation, the SimpleTrigger wants to be re-scheduled to 'now' (even if the associated Calendar excludes 'now') with the repeat count set to what it would be, if it had not missed any firings. This does obey the Trigger end-time however, so if 'now' is after the end-time the Trigger will not fire again.

Use of this instruction causes the trigger to 'forget' the start-time and repeat-count that it was originally setup with. Instead, the repeat count on the trigger will be changed to whatever the remaining repeat count is (this is only an issue if you for some reason wanted to be able to tell what the original values were at some later time).

This instruction could cause the Trigger to go to the 'COMPLETE' state after firing 'now', if all the repeat-fire-times were missed.

115.12.4. SimpleTrigger.MISFIRE_INSTRUCTION_RESCHEDULE_NEXT_WITH_REMAIN = 4

Instructs the Scheduler that upon a mis-fire situation, the SimpleTrigger wants to be re-scheduled to the next scheduled time after 'now' - taking into account any associated Calendar and with the repeat count set to what it would be, if it had not missed any firings.



NOTE

This instruction could cause the Trigger to go directly to the 'COMPLETE' state if all fire-times where missed.

115.12.5. SimpleTrigger.MISFIRE_INSTRUCTION_RESCHEDULE_NEXT_WITH_EXISTIN = 5

Instructs the Scheduler that upon a mis-fire situation, the SimpleTrigger wants to be re-scheduled to the next scheduled time after 'now' - taking into account any associated Calendar, and with the repeat count left unchanged.



NOTE

This instruction could cause the Trigger to go directly to the 'COMPLETE' state if the end-time of the trigger has arrived.

115.12.6. CronTrigger.MISFIRE_INSTRUCTION_FIRE_ONCE_NOW = 1 (default)

Instructs the Scheduler that upon a mis-fire situation, the CronTrigger wants to be fired now by Scheduler.

115.12.7. CronTrigger.MISFIRE_INSTRUCTION_DO_NOTHING = 2

Instructs the Scheduler that upon a mis-fire situation, the CronTrigger wants to have it's next-fire-time updated to the next time in the schedule after the current time (taking into account any associated Calendar but it does not want to be fired now).

115.13. USING QUARTZSCHEDULEDPOLLCONSUMERSCHEDULER

The [Quartz](#) component provides a Polling Consumer scheduler which allows to use cron based scheduling for Polling Consumer such as the File and FTP consumers.

For example to use a cron based expression to poll for files every 2nd second, then a Camel route can be define simply as:

```
from("file:inbox?scheduler=quartz&scheduler.cron=0/2+*+*+*+*+*")
.to("bean:process");
```

Notice we define the **scheduler=quartz** to instruct Camel to use the [Quartz](#) based scheduler. Then we use **scheduler.xxx** options to configure the scheduler. The [Quartz](#) scheduler requires the cron option to be set.

The following options is supported:

Parameter	Default	Type	Description
quartzScheduler	null	org.quartz.Scheduler	To use a custom Quartz scheduler. If none configure then the shared scheduler from the component is used.
cron	null	String	Mandatory: To define the cron expression for triggering the polls.
triggerId	null	String	To specify the trigger id. If none provided then an UUID is generated and used.
triggerGroup	QuartzScheduledPollConsumerScheduler	String	To specify the trigger group.
timeZone	Default	TimeZone	The time zone to use for the CRON trigger.



IMPORTANT

Remember configuring these options from the endpoint URLs must be prefixed with **scheduler**.

For example to configure the trigger id and group:

```
from("file:inbox?scheduler=quartz&scheduler.cron=0/2+*+*+*+*?
&scheduler.triggerId=myId&scheduler.triggerGroup=myGroup")
.to("bean:process");
```

There is also a CRON scheduler in Spring, so you can use the following as well:

```
from("file:inbox?scheduler=spring&scheduler.cron=0/2+*+*+*+*?")
.to("bean:process");
```

115.14. CRON COMPONENT SUPPORT

The Quartz component can be used as implementation of the Camel Cron component.

Maven users will need to add the following additional dependency to their **pom.xml**:

```
<dependency>
  <groupId>org.apache.camel</groupId>
```

```

<artifactId>camel-cron</artifactId>
<version>{CamelSBVersion}</version>
<!-- use the same version as your Camel core version -->
</dependency>

```

Users can then use the cron component instead of the quartz component, as in the following route:

```

from("cron://name?schedule=0+0/5+12-18+?+*+MON-FRI")
.to("activemq:Totally.Rocks");

```

115.15. SPRING BOOT AUTO-CONFIGURATION

The component supports 14 options, which are listed below.

Name	Description	Default	Type
camel.component.quartz.auto-start-scheduler	Whether or not the scheduler should be auto started. This options is default true.	true	Boolean
camel.component.quartz.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.quartz.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.quartz.enable-jmx	Whether to enable Quartz JMX which allows to manage the Quartz scheduler from JMX. This options is default true.	true	Boolean
camel.component.quartz.enabled	Whether to enable auto configuration of the quartz component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.quartz.interrupt-jobs-on-shutdown	Whether to interrupt jobs on shutdown which forces the scheduler to shutdown quicker and attempt to interrupt any running jobs. If this is enabled then any running jobs can fail due to being interrupted. When a job is interrupted then Camel will mark the exchange to stop continue routing and set <code>java.util.concurrent.RejectedExecutionException</code> as caused exception. Therefore use this with care, as its often better to allow Camel jobs to complete and shutdown gracefully.	false	Boolean
camel.component.quartz.prefix-instance-name	Whether to prefix the Quartz Scheduler instance name with the CamelContext name. This is enabled by default, to let each CamelContext use its own Quartz scheduler instance by default. You can set this option to false to reuse Quartz scheduler instances between multiple CamelContext's.	true	Boolean
camel.component.quartz.prefix-job-name-with-endpoint-id	Whether to prefix the quartz job with the endpoint id. This option is default false.	false	Boolean
camel.component.quartz.properties	Properties to configure the Quartz scheduler.		Map
camel.component.quartz.properties-file	File name of the properties to load from the classpath.		String
camel.component.quartz.properties-ref	References to an existing Properties or Map to lookup in the registry to use for configuring quartz.		String
camel.component.quartz.scheduler	To use the custom configured Quartz scheduler, instead of creating a new Scheduler. The option is a <code>org.quartz.Scheduler</code> type.		Scheduler
camel.component.quartz.scheduler-factory	To use the custom SchedulerFactory which is used to create the Scheduler. The option is a <code>org.quartz.SchedulerFactory</code> type.		SchedulerFactory
camel.component.quartz.start-delayed-seconds	Seconds to wait before starting the quartz scheduler.		Integer

CHAPTER 116. REF

Both producer and consumer are supported

The Ref component is used for lookup of existing endpoints bound in the Registry.

116.1. DEPENDENCIES

When using **ref** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-ref-starter</artifactId>
</dependency>
```

116.2. URI FORMAT

```
ref:someName[?options]
```

Where **someName** is the name of an endpoint in the Registry (usually, but not always, the Spring registry). If you are using the Spring registry, **someName** would be the bean ID of an endpoint in the Spring registry.

116.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

116.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

116.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

116.4. COMPONENT OPTIONS

The Ref component supports 3 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

116.5. ENDPOINT OPTIONS

The Ref endpoint is configured using URI syntax:

```
ref:name
```

with the following path and query parameters:

116.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (common)	Required Name of endpoint to lookup in the registry.		String

116.5.2. Query Parameters (4 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

116.6. RUNTIME LOOKUP

This component can be used when you need dynamic discovery of endpoints in the Registry where you can compute the URI at runtime. Then you can look up the endpoint using the following code:

```
// lookup the endpoint
String myEndpointRef = "bigspenderOrder";
Endpoint endpoint = context.getEndpoint("ref:" + myEndpointRef);

Producer producer = endpoint.createProducer();
Exchange exchange = producer.createExchange();
exchange.getIn().setBody(payloadToSend);
// send the exchange
producer.process(exchange);
```

And you could have a list of endpoints defined in the Registry such as:

```
<camelContext id="camel" xmlns="http://activemq.apache.org/camel/schema/spring">
  <endpoint id="normalOrder" uri="activemq:order.slow"/>
  <endpoint id="bigspenderOrder" uri="activemq:order.high"/>
</camelContext>
```

116.7. SAMPLE

In the sample below we use the **ref:** in the URI to reference the endpoint with the spring ID, **endpoint2**:

You could, of course, have used the **ref** attribute instead:

```
<to uri="ref:endpoint2"/>
```

Which is the more common way to write it.

116.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.component.ref.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.ref.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.ref.enabled	Whether to enable auto configuration of the ref component. This is enabled by default.		Boolean
camel.component.ref.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 117. REF

The Ref Expression Language is really just a way to lookup a custom **Expression** or **Predicate** from the [Registry](#).

This is particular usable in XML DSLs.

117.1. DEPENDENCIES

The Ref language is part of **camel-core**. When using **ref** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-core-starter</artifactId>
</dependency>
```

117.2. REF LANGUAGE OPTIONS

The Ref language supports 1 options, which are listed below.

Name	Default	Java Type	Description
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

117.3. EXAMPLE USAGE

The Splitter EIP in XML DSL can utilize a custom expression using **<ref>** like:

```
<bean id="myExpression" class="com.mycompany.MyCustomExpression"/>

<route>
  <from uri="seda:a"/>
  <split>
    <ref>myExpression</ref>
    <to uri="mock:b"/>
  </split>
</route>
```

in this case, the message coming from the seda:a endpoint will be splitted using a custom **Expression** which has the id **myExpression** in the [Registry](#).

And the same example using Java DSL:

```
from("seda:a").split().ref("myExpression").to("seda:b");
```

117.4. SPRING BOOT AUTO-CONFIGURATION

The component supports 147 options, which are listed below.

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.acl-token</code>	Sets the ACL token to be used with Consul.		String
<code>camel.cloud.consul.service-discovery.block-seconds</code>	The seconds to wait for a watch event, default 10 seconds.	10	Integer
<code>camel.cloud.consul.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.consul.service-discovery.connect-timeout-millis</code>	Connect timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.datacenter</code>	The data center.		String
<code>camel.cloud.consul.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.consul.service-discovery.password</code>	Sets the password to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.consul.service-discovery.read-timeout-millis</code>	Read timeout for OkHttpClient.		Long

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.url</code>	The Consul agent URL.		String
<code>camel.cloud.consul.service-discovery.username</code>	Sets the username to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.write-timeout-millis</code>	Write timeout for OkHttpClient.		Long
<code>camel.cloud.dns.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.dns.service-discovery.domain</code>	The domain name;.		String
<code>camel.cloud.dns.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.dns.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.dns.service-discovery.protocol</code>	The transport protocol of the desired service.	<code>_tcp</code>	String
<code>camel.cloud.etcd.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.etcd.service-discovery.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
<code>camel.cloud.etcd.service-discovery.password</code>	The password to use for basic authentication.		String
<code>camel.cloud.etcd.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.etcd.service-discovery.service-path</code>	The path to look for for service discovery.	<code>/services/</code>	String
<code>camel.cloud.etcd.service-discovery.timeout</code>	To set the maximum time an action could take to complete.		Long
<code>camel.cloud.etcd.service-discovery.type</code>	To set the discovery type, valid values are on-demand and watch.	on-demand	String
<code>camel.cloud.etcd.service-discovery.uris</code>	The URIs the client can connect to.		String
<code>camel.cloud.etcd.service-discovery.user-name</code>	The user name to use for basic authentication.		String
<code>camel.cloud.kubernetes.service-discovery.api-version</code>	Sets the API version when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-data</code>	Sets the Certificate Authority data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-file</code>	Sets the Certificate Authority data that are loaded from the file when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.client-cert-data</code>	Sets the Client Certificate data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-file</code>	Sets the Client Certificate data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-algo</code>	Sets the Client Keystore algorithm, such as RSA when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-data</code>	Sets the Client Keystore data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-file</code>	Sets the Client Keystore data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-passphrase</code>	Sets the Client Keystore passphrase when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.kubernetes.service-discovery.dns-domain</code>	Sets the DNS domain to use for DNS lookup.		String
<code>camel.cloud.kubernetes.service-discovery.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.lookup</code>	How to perform service lookup. Possible values: client, dns, environment. When using client, then the client queries the kubernetes master to obtain a list of active pods that provides the service, and then random (or round robin) select a pod. When using dns the service name is resolved as name.namespace.svc.dnsDomain. When using dnssrv the service name is resolved with SRV query forsvc... When using environment then environment variables are used to lookup the service. By default environment is used.	environment	String
<code>camel.cloud.kubernetes.service-discovery.master-url</code>	Sets the URL to the master when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.namespace</code>	Sets the namespace to use. Will by default use namespace from the ENV variable KUBERNETES_MASTER.		String
<code>camel.cloud.kubernetes.service-discovery.oauth-token</code>	Sets the OAUTH token for authentication (instead of username/password) when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.password</code>	Sets the password for authentication when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-name</code>	Sets the Port Name to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-protocol</code>	Sets the Port Protocol to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in com.netflix.client.config.CommonClientConfigKey.		Map

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.trust-certs</code>	Sets whether to turn on trust certificate check when using client lookup.	false	Boolean
<code>camel.cloud.kubernetes.service-discovery.username</code>	Sets the username for authentication when using client lookup.		String
<code>camel.cloud.ribbon.load-balancer.client-name</code>	Sets the Ribbon client name.		String
<code>camel.cloud.ribbon.load-balancer.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.ribbon.load-balancer.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.ribbon.load-balancer.namespace</code>	The namespace.		String
<code>camel.cloud.ribbon.load-balancer.password</code>	The password.		String
<code>camel.cloud.ribbon.load-balancer.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.ribbon.load-balancer.username</code>	The username.		String

Name	Description	Default	Type
camel.hystrix.allow-maximum-size-to-diverge-from-core-size	Allows the configuration for <code>maximumSize</code> to take effect. That value can then be equal to, or higher, than <code>coreSize</code> .	false	Boolean
camel.hystrix.circuit-breaker-enabled	Whether to use a <code>HystrixCircuitBreaker</code> or not. If false no circuit-breaker logic will be used and all requests permitted. This is similar in effect to <code>circuitBreakerForceClosed()</code> except that continues tracking metrics and knowing whether it should be open/closed, this property results in not even instantiating a circuit-breaker.	true	Boolean
camel.hystrix.circuit-breaker-error-threshold-percentage	Error percentage threshold (as whole number such as 50) at which point the circuit breaker will trip open and reject requests. It will stay tripped for the duration defined in <code>circuitBreakerSleepWindowInMilliseconds</code> ; The error percentage this is compared against comes from <code>HystrixCommandMetrics.getHealthCounts()</code> .	50	Integer
camel.hystrix.circuit-breaker-force-closed	If true the <code>HystrixCircuitBreaker#allowRequest()</code> will always return true to allow requests regardless of the error percentage from <code>HystrixCommandMetrics.getHealthCounts()</code> . The <code>circuitBreakerForceOpen()</code> property takes precedence so if it set to true this property does nothing.	false	Boolean
camel.hystrix.circuit-breaker-force-open	If true the <code>HystrixCircuitBreaker.allowRequest()</code> will always return false, causing the circuit to be open (tripped) and reject all requests. This property takes precedence over <code>circuitBreakerForceClosed()</code> ;	false	Boolean
camel.hystrix.circuit-breaker-request-volume-threshold	Minimum number of requests in the <code>metricsRollingStatisticalWindowInMilliseconds()</code> that must exist before the <code>HystrixCircuitBreaker</code> will trip. If below this number the circuit will not trip regardless of error percentage.	20	Integer
camel.hystrix.circuit-breaker-sleep-window-in-milliseconds	The time in milliseconds after a <code>HystrixCircuitBreaker</code> trips open that it should wait before trying requests again.	5000	Integer

Name	Description	Default	Type
camel.hystrix.configurations	Define additional configuration definitions.		Map
camel.hystrix.core-pool-size	Core thread-pool size that gets passed to <code>java.util.concurrent.ThreadPoolExecutor#setCorePoolSize(int)</code> .	10	Integer
camel.hystrix.enabled	Enable the component.	true	Boolean
camel.hystrix.execution-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.run()</code> . Requests beyond the concurrent limit will be rejected. Applicable only when <code>executionIsolationStrategy == SEMAPHORE</code> .	20	Integer
camel.hystrix.execution-isolation-strategy	What isolation strategy <code>HystrixCommand.run()</code> will be executed with. If <code>THREAD</code> then it will be executed on a separate thread and concurrent requests limited by the number of threads in the thread-pool. If <code>SEMAPHORE</code> then it will be executed on the calling thread and concurrent requests limited by the semaphore count.	THREAD	String
camel.hystrix.execution-isolation-thread-interrupt-on-timeout	Whether the execution thread should attempt an interrupt (using <code>Future#cancel()</code>) when a thread times out. Applicable only when <code>executionIsolationStrategy() == THREAD</code> .	true	Boolean
camel.hystrix.execution-timeout-enabled	Whether the timeout mechanism is enabled for this command.	true	Boolean
camel.hystrix.execution-timeout-in-milliseconds	Time in milliseconds at which point the command will timeout and halt execution. If <code>executionIsolationThreadInterruptOnTimeout == true</code> and the command is thread-isolated, the executing thread will be interrupted. If the command is semaphore-isolated and a <code>HystrixObservableCommand</code> , that command will get unsubscribed.	1000	Integer
camel.hystrix.fallback-enabled	Whether <code>HystrixCommand.getFallback()</code> should be attempted when failure occurs.	true	Boolean

Name	Description	Default	Type
camel.hystrix.fallback-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.getFallback()</code> . Requests beyond the concurrent limit will fail-fast and not attempt retrieving a fallback.	10	Integer
camel.hystrix.group-key	Sets the group key to use. The default value is <code>CamelHystrix</code> .	CamelHystrix	String
camel.hystrix.keep-alive-time	Keep-alive time in minutes that gets passed to <code>ThreadPoolExecutor#setKeepAliveTime(long, TimeUnit)</code> .	1	Integer
camel.hystrix.max-queue-size	Max queue size that gets passed to <code>BlockingQueue</code> in <code>HystrixConcurrencyStrategy.getBlockingQueue(int)</code> . This should only affect the instantiation of a threadpool - it is not eligible to change a queue size on the fly. For that, use <code>queueSizeRejectionThreshold()</code> .	-1	Integer
camel.hystrix.maximum-size	Maximum thread-pool size that gets passed to <code>ThreadPoolExecutor#setMaximumPoolSize(int)</code> . This is the maximum amount of concurrency that can be supported without starting to reject <code>HystrixCommands</code> . Please note that this setting only takes effect if you also set <code>allowMaximumSizeToDivergeFromCoreSize</code> .	10	Integer
camel.hystrix.metrics-health-snapshot-interval-in-milliseconds	Time in milliseconds to wait between allowing health snapshots to be taken that calculate success and error percentages and affect <code>HystrixCircuitBreaker.isOpen()</code> status. On high-volume circuits the continual calculation of error percentage can become CPU intensive thus this controls how often it is calculated.	500	Integer
camel.hystrix.metrics-rolling-percentile-bucket-size	Maximum number of values stored in each bucket of the rolling percentile. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-percentile-enabled	Whether percentile metrics should be captured using <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	true	Boolean

Name	Description	Default	Type
camel.hystrix.metrics-rolling-percentile-window-buckets	Number of buckets the rolling percentile window is broken into. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	6	Integer
camel.hystrix.metrics-rolling-percentile-window-in-milliseconds	Duration of percentile rolling window in milliseconds. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	10000	Integer
camel.hystrix.metrics-rolling-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside HystrixCommandMetrics.	10	Integer
camel.hystrix.metrics-rolling-statistical-window-in-milliseconds	This property sets the duration of the statistical rolling window, in milliseconds. This is how long metrics are kept for the thread pool. The window is divided into buckets and rolls by those increments.	10000	Integer
camel.hystrix.queue-size-rejection-threshold	Queue size rejection threshold is an artificial max size at which rejections will occur even if maxQueueSize has not been reached. This is done because the maxQueueSize of a BlockingQueue can not be dynamically changed and we want to support dynamically changing the queue size that affects rejections. This is used by HystrixCommand when queuing a thread for execution.	5	Integer
camel.hystrix.request-log-enabled	Whether HystrixCommand execution and events should be logged to HystrixRequestLog.	true	Boolean
camel.hystrix.thread-pool-key	Sets the thread pool key to use. Will by default use the same value as groupKey has been configured to use.	Camel Hystrix	String
camel.hystrix.thread-pool-rolling-number-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10	Integer

Name	Description	Default	Type
<code>camel.hystrix.thread-pool-rolling-number-statistical-window-in-milliseconds</code>	Duration of statistical rolling window in milliseconds. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10000	Integer
<code>camel.language.constant.enabled</code>	Whether to enable auto configuration of the constant language. This is enabled by default.		Boolean
<code>camel.language.constant.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.csimple.enabled</code>	Whether to enable auto configuration of the csimple language. This is enabled by default.		Boolean
<code>camel.language.csimple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.exchangeproperty.enabled</code>	Whether to enable auto configuration of the exchangeProperty language. This is enabled by default.		Boolean
<code>camel.language.exchangeproperty.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.file.enabled</code>	Whether to enable auto configuration of the file language. This is enabled by default.		Boolean
<code>camel.language.file.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.header.enabled</code>	Whether to enable auto configuration of the header language. This is enabled by default.		Boolean
<code>camel.language.header.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.ref.enabled</code>	Whether to enable auto configuration of the ref language. This is enabled by default.		Boolean
<code>camel.language.ref.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.simple.enabled</code>	Whether to enable auto configuration of the simple language. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.language.simple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.tokenize.enabled</code>	Whether to enable auto configuration of the tokenize language. This is enabled by default.		Boolean
<code>camel.language.tokenize.group-delimiter</code>	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.		String
<code>camel.language.tokenize.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.resilience4j.automatic-transition-from-open-to-half-open-enabled</code>	Enables automatic transition from OPEN to HALF_OPEN state once the <code>waitDurationInOpenState</code> has passed.	false	Boolean
<code>camel.resilience4j.circuit-breaker-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreaker</code> instance to lookup and use from the registry. When using this, then any other circuit breaker options are not in use.		String
<code>camel.resilience4j.config-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreakerConfig</code> instance to lookup and use from the registry.		String
<code>camel.resilience4j.configurations</code>	Define additional configuration definitions.		Map
<code>camel.resilience4j.enabled</code>	Enable the component.	true	Boolean
<code>camel.resilience4j.failure-rate-threshold</code>	Configures the failure rate threshold in percentage. If the failure rate is equal or greater than the threshold the <code>CircuitBreaker</code> transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 50 percentage.		Float

Name	Description	Default	Type
camel.resilience4j.minimum-number-of-calls	Configures the minimum number of calls which are required (per sliding window period) before the CircuitBreaker can calculate the error rate. For example, if minimumNumberOfCalls is 10, then at least 10 calls must be recorded, before the failure rate can be calculated. If only 9 calls have been recorded the CircuitBreaker will not transition to open even if all 9 calls have failed. Default minimumNumberOfCalls is 100.	100	Integer
camel.resilience4j.permitted-number-of-calls-in-half-open-state	Configures the number of permitted calls when the CircuitBreaker is half open. The size must be greater than 0. Default size is 10.	10	Integer
camel.resilience4j.sliding-window-size	Configures the size of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. slidingWindowSize configures the size of the sliding window. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. The slidingWindowSize must be greater than 0. The minimumNumberOfCalls must be greater than 0. If the slidingWindowType is COUNT_BASED, the minimumNumberOfCalls cannot be greater than slidingWindowSize. If the slidingWindowType is TIME_BASED, you can pick whatever you want. Default slidingWindowSize is 100.	100	Integer
camel.resilience4j.sliding-window-type	Configures the type of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. Default slidingWindowType is COUNT_BASED.	COUNT_BASED	String
camel.resilience4j.slow-call-duration-threshold	Configures the duration threshold (seconds) above which calls are considered as slow and increase the slow calls percentage. Default value is 60 seconds.	60	Integer

Name	Description	Default	Type
camel.resilience4j.slow-call-rate-threshold	Configures a threshold in percentage. The CircuitBreaker considers a call as slow when the call duration is greater than <code>slowCallDurationThreshold</code> . When the percentage of slow calls is equal or greater the threshold, the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 100 percentage which means that all recorded calls must be slower than <code>slowCallDurationThreshold</code> .		Float
camel.resilience4j.wait-duration-in-open-state	Configures the wait duration (in seconds) which specifies how long the CircuitBreaker should stay open, before it switches to half open. Default value is 60 seconds.	60	Integer
camel.resilience4j.writable-stack-trace-enabled	Enables writable stack traces. When set to false, <code>Exception.getStackTrace</code> returns a zero length array. This may be used to reduce log spam when the circuit breaker is open as the cause of the exceptions is already known (the circuit breaker is short-circuiting calls).	true	Boolean
camel.rest.api-component	The name of the Camel component to use as the REST API (such as swagger) If no API Component has been explicit configured, then Camel will lookup if there is a Camel component responsible for servicing and generating the REST API documentation, or if a <code>org.apache.camel.spi.RestApiProcessorFactory</code> is registered in the registry. If either one is found, then that is being used.		String
camel.rest.api-context-path	Sets a leading API context-path the REST API services will be using. This can be used when using components such as <code>camel-servlet</code> where the deployed web application is deployed using a context-path.		String
camel.rest.api-context-route-id	Sets the route id to use for the route that services the REST API. The route will by default use an auto assigned route id.		String
camel.rest.api-host	To use an specific hostname for the API documentation (eg swagger) This can be used to override the generated host with this configured hostname.		String

Name	Description	Default	Type
camel.rest.api-property	Allows to configure as many additional properties for the api documentation (swagger). For example set property api.title to my cool stuff.		Map
camel.rest.api-vendor-extension	Whether vendor extension is enabled in the Rest APIs. If enabled then Camel will include additional information as vendor extension (eg keys starting with x-) such as route ids, class names etc. Not all 3rd party API gateways and tools supports vendor-extensions when importing your API docs.	false	Boolean
camel.rest.binding-mode	Sets the binding mode to use. The default value is off.		RestBindingMode
camel.rest.client-request-validation	Whether to enable validation of the client request to check whether the Content-Type and Accept headers from the client is supported by the Rest-DSL configuration of its consumes/produces settings. This can be turned on, to enable this check. In case of validation error, then HTTP Status codes 415 or 406 is returned. The default value is false.	false	Boolean
camel.rest.component	The Camel Rest component to use for the REST transport (consumer), such as netty-http, jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String
camel.rest.component-property	Allows to configure as many additional properties for the rest component in use.		Map
camel.rest.consumer-property	Allows to configure as many additional properties for the rest consumer in use.		Map
camel.rest.context-path	Sets a leading context-path the REST services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path. Or for components such as camel-jetty or camel-netty-http that includes a HTTP server.		String
camel.rest.cors-headers	Allows to configure custom CORS headers.		Map

Name	Description	Default	Type
camel.rest.data-format-property	Allows to configure as many additional properties for the data formats in use. For example set property <code>prettyPrint</code> to <code>true</code> to have json outputted in pretty mode. The properties can be prefixed to denote the option is only for either JSON or XML and for either the IN or the OUT. The prefixes are: <code>json.in</code> , <code>json.out</code> , <code>xml.in</code> , <code>xml.out</code> . For example a key with value <code>xml.out.mustBeJAXBElement</code> is only for the XML data format for the outgoing. A key without a prefix is a common key for all situations.		Map
camel.rest.enable-cors	Whether to enable CORS headers in the HTTP response. The default value is <code>false</code> .	<code>false</code>	Boolean
camel.rest.endpoint-property	Allows to configure as many additional properties for the rest endpoint in use.		Map
camel.rest.host	The hostname to use for exposing the REST service.		String
camel.rest.host-name-resolver	If no hostname has been explicit configured, then this resolver is used to compute the hostname the REST service will be using.		RestHostNameResolver
camel.rest.json-data-format	Name of specific json data format to use. By default <code>json-jackson</code> will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.port	The port number to use for exposing the REST service. Notice if you use servlet component then the port number configured here does not apply, as the port number in use is the actual port number the servlet component is using. eg if using Apache Tomcat its the tomcat http port, if using Apache Karaf its the HTTP service in Karaf that uses port 8181 by default etc. Though in those situations setting the port number here, allows tooling and JMX to know the port number, so its recommended to set the port number to the number that the servlet engine uses.		String
camel.rest.producer-api-doc	Sets the location of the api document (swagger api) the REST producer will use to validate the REST uri and query parameters are valid accordingly to the api document. This requires adding <code>camel-swagger-java</code> to the classpath, and any miss configuration will let Camel fail on startup and report the error(s). The location of the api document is loaded from classpath by default, but you can use <code>file:</code> or <code>http:</code> to refer to resources to load from file or http url.		String

Name	Description	Default	Type
<code>camel.rest.producer-component</code>	Sets the name of the Camel component to use as the REST producer.		String
<code>camel.rest.scheme</code>	The scheme to use for exposing the REST service. Usually http or https is supported. The default value is http.		String
<code>camel.rest.skip-binding-on-error-code</code>	Whether to skip binding on output if there is a custom HTTP error code header. This allows to build custom error messages that do not bind to json / xml etc, as success messages otherwise will do.	false	Boolean
<code>camel.rest.use-x-forward-headers</code>	Whether to use X-Forward headers for Host and related setting. The default value is true.	true	Boolean
<code>camel.rest.xml-data-format</code>	Name of specific XML data format to use. By default jaxb will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
<code>camel.rest.api-context-id-pattern</code>	Deprecated Sets an CamelContext id pattern to only allow Rest APIs from rest services within CamelContext's which name matches the pattern. The pattern name refers to the CamelContext name, to match on the current CamelContext only. For any other value, the pattern uses the rules from <code>PatternHelper#matchPattern(String,String)</code> .		String
<code>camel.rest.api-context-listing</code>	Deprecated Sets whether listing of all available CamelContext's with REST services in the JVM is enabled. If enabled it allows to discover these contexts, if false then only the current CamelContext is in use.	false	Boolean

CHAPTER 118. REST

Both producer and consumer are supported

The REST component allows to define REST endpoints (consumer) using the Rest DSL and plugin to other Camel components as the REST transport.

The rest component can also be used as a client (producer) to call REST services.

118.1. DEPENDENCIES

When using **rest** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-rest-starter</artifactId>
</dependency>
```

118.2. URI FORMAT

```
rest://method:path[:uriTemplate]?[options]
```

118.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

118.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

118.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

118.4. COMPONENT OPTIONS

The REST component supports 8 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
consumerComponentName (consumer)	The Camel Rest component to use for (consumer) the REST transport, such as jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a <code>org.apache.camel.spi.RestConsumerFactory</code> is registered in the registry. If either one is found, then that is being used.		String
apiDoc (producer)	The swagger api doc resource to use. The resource is loaded from classpath by default and must be in JSON format.		String

Name	Description	Default	Type
componentName (producer)	Deprecated The Camel Rest component to use for (producer) the REST transport, such as http, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a <code>org.apache.camel.spi.RestProducerFactory</code> is registered in the registry. If either one is found, then that is being used.		String
host (producer)	Host and port of HTTP service to use (override host in swagger schema).		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
producerComponentName (producer)	The Camel Rest component to use for (producer) the REST transport, such as http, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a <code>org.apache.camel.spi.RestProducerFactory</code> is registered in the registry. If either one is found, then that is being used.		String
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

118.5. ENDPOINT OPTIONS

The REST endpoint is configured using URI syntax:

```
rest:method:path:uriTemplate
```

with the following path and query parameters:

118.5.1. Path Parameters (3 parameters)

Name	Description	Default	Type
method (common)	Required HTTP method to use. Enum values: • get • post • put • delete • patch • head • trace • connect • options		String
path (common)	Required The base path.		String
uriTemplate (common)	The uri template.		String

118.5.2. Query Parameters (16 parameters)

Name	Description	Default	Type
consumes (common)	Media type such as: 'text/xml', or 'application/json' this REST service accepts. By default we accept all kinds of types.		String
inType (common)	To declare the incoming POJO binding type as a FQN class name.		String
outType (common)	To declare the outgoing POJO binding type as a FQN class name.		String
produces (common)	Media type such as: 'text/xml', or 'application/json' this REST service returns.		String
routeId (common)	Name of the route this REST services creates.		String

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
consumerComponentName (consumer)	The Camel Rest component to use for (consumer) the REST transport, such as jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a <code>org.apache.camel.spi.RestConsumerFactory</code> is registered in the registry. If either one is found, then that is being used.		String
description (consumer)	Human description to document this REST service.		String
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code> ● <code>InOptionalOut</code>		<code>ExchangePattern</code>
apiDoc (producer)	The openapi api doc resource to use. The resource is loaded from classpath by default and must be in JSON format.		String

Name	Description	Default	Type
bindingMode (producer)	Configures the binding mode for the producer. If set to anything other than 'off' the producer will try to convert the body of the incoming message from inType to the json or xml, and the response from json or xml to outType. Enum values: • auto • off • json • xml • json_xml		RestBindingMode
host (producer)	Host and port of HTTP service to use (override host in openapi schema).		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
producerComponentName (producer)	The Camel Rest component to use for (producer) the REST transport, such as http, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestProducerFactory is registered in the registry. If either one is found, then that is being used.		String
queryParameters (producer)	Query parameters for the HTTP service to call. The query parameters can contain multiple parameters separated by ampersand such such as foo=123&bar=456.		String

118.6. SUPPORTED REST COMPONENTS

The following components support rest consumer (Rest DSL):

- camel-servlet
- camel-platform-http

The following components support rest producer:

- camel-http

118.7. PATH AND URITEMPLATE SYNTAX

The path and uriTemplate option is defined using a REST syntax where you define the REST context path using support for parameters.



NOTE

If no uriTemplate is configured then path option works the same way. It does not matter if you configure only path or if you configure both options. Though configuring both a path and uriTemplate is a more common practice with REST.

The following is a Camel route using a path only

```
from("rest:get:hello")
  .transform().constant("Bye World");
```

And the following route uses a parameter which is mapped to a Camel header with the key "me".

```
from("rest:get:hello/{me}")
  .transform().simple("Bye ${header.me}");
```

The following examples have configured a base path as "hello" and then have two REST services configured using uriTemplates.

```
from("rest:get:hello/{me}")
  .transform().simple("Hi ${header.me}");

from("rest:get:hello/french/{me}")
  .transform().simple("Bonjour ${header.me}");
```



NOTE

The Rest endpoint path does not accept escaped characters, for example, the **plus** sign. This is default behavior of Apache Camel 3.

118.8. REST PRODUCER EXAMPLES

You can use the rest component to call REST services like any other Camel component.

For example to call a REST service on using **hello/{me}** you can do

```
from("direct:start")
  .to("rest:get:hello/{me}");
```

And then the dynamic value **{me}** is mapped to Camel message with the same name. So to call this REST service you can send an empty message body and a header as shown:

```
template.sendBodyAndHeader("direct:start", null, "me", "Donald Duck");
```

The Rest producer needs to know the hostname and port of the REST service, which you can configure using the host option as shown:

```
from("direct:start")
  .to("rest:get:hello/{me}?host=myserver:8080/foo");
```

Instead of using the host option, you can configure the host on the **restConfiguration** as shown:

```
restConfiguration().host("myserver:8080/foo");

from("direct:start")
  .to("rest:get:hello/{me}");
```

You can use the **producerComponent** to select which Camel component to use as the HTTP client, for example to use http you can do:

```
restConfiguration().host("myserver:8080/foo").producerComponent("http");

from("direct:start")
  .to("rest:get:hello/{me}");
```

118.9. REST PRODUCER BINDING

The REST producer supports binding using JSON or XML like the rest-dsl does.

For example to use jetty with json binding mode turned on you can configure this in the rest configuration:

```
restConfiguration().component("jetty").host("localhost").port(8080).bindingMode(RestBindingMode.json)
;

from("direct:start")
  .to("rest:post:user");
```

Then when calling the REST service using rest producer it will automatic bind any POJOs to json before calling the REST service:

```
UserPojo user = new UserPojo();
user.setId(123);
user.setName("Donald Duck");

template.sendBody("direct:start", user);
```

In the example above we send a POJO instance **UserPojo** as the message body. And because we have turned on JSON binding in the rest configuration, then the POJO will be marshalled from POJO to JSON before calling the REST service.

However if you want to also perform binding for the response message (eg what the REST service send back as response) you would need to configure the **outType** option to specify what is the classname of the POJO to unmarshal from JSon to POJO.

For example if the REST service returns a JSon payload that binds to **com.foo.MyResponsePojo** you can configure this as shown:

```
restConfiguration().component("jetty").host("localhost").port(8080).bindingMode(RestBindingMode.json)
;

from("direct:start")
.to("rest:post:user?outType=com.foo.MyResponsePojo");
```



NOTE

You must configure **outType** option if you want POJO binding to happen for the response messages received from calling the REST service.

118.10. MORE EXAMPLES

See Rest DSL which offers more examples and how you can use the Rest DSL to define those in a nicer RESTful way.

There is a **camel-example-servlet-rest-tomcat** example in the Apache Camel distribution, that demonstrates how to use the Rest DSL with SERVLET as transport that can be deployed on Apache Tomcat, or similar web containers.

118.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 12 options, which are listed below.

Name	Description	Default	Type
camel.component.rest-api.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.rest-api.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.rest-api.enabled	Whether to enable auto configuration of the rest-api component. This is enabled by default.		Boolean
camel.component.rest.api-doc	The swagger api doc resource to use. The resource is loaded from classpath by default and must be in JSON format.		String
camel.component.rest.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.rest.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.rest.consumer-component-name	The Camel Rest component to use for (consumer) the REST transport, such as jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String
camel.component.rest.enabled	Whether to enable auto configuration of the rest component. This is enabled by default.		Boolean
camel.component.rest.host	Host and port of HTTP service to use (override host in swagger schema).		String

Name	Description	Default	Type
<code>camel.component.rest.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.rest.producer-component-name</code>	The Camel Rest component to use for (producer) the REST transport, such as http, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a <code>org.apache.camel.spi.RestProducerFactory</code> is registered in the registry. If either one is found, then that is being used.		String
<code>camel.component.rest.component-name</code>	Deprecated The Camel Rest component to use for (producer) the REST transport, such as http, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a <code>org.apache.camel.spi.RestProducerFactory</code> is registered in the registry. If either one is found, then that is being used.		String

CHAPTER 119. SAGA

Only producer is supported

The Saga component provides a bridge to execute custom actions within a route using the Saga EIP.

The component should be used for advanced tasks, such as deciding to complete or compensate a Saga with completionMode set to **MANUAL**.

Refer to the Saga EIP documentation for help on using sagas in common scenarios.

119.1. DEPENDENCIES

When using **saga** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-saga-starter</artifactId>
</dependency>
```

119.2. URI FORMAT

```
saga:action
```

119.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

119.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

119.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

119.4. COMPONENT OPTIONS

The Saga component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

119.5. ENDPOINT OPTIONS

The Saga endpoint is configured using URI syntax:

```
saga:action
```

with the following path and query parameters:

119.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
action (producer)	Required Action to execute (complete or compensate). Enum values: • COMPLETE • COMPENSATE		SagaEndpointAction

119.5.2. Query Parameters (1 parameters)

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

119.6. USING CAMEL-SAGA WITH SPRING BOOT AND LRA COORDINATOR

This example shows how to work with Apache Camel Saga using Spring Boot and Narayana LRA Coordinator to manage long running actions. See [Saga example](#) for more information.

119.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
camel.component.saga.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.saga.enabled	Whether to enable auto configuration of the saga component. This is enabled by default.		Boolean
camel.component.saga.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 120. SALESFORCE

Both producer and consumer are supported

This component supports producer and consumer endpoints to communicate with Salesforce using Java DTOs.

There is a companion maven plugin Camel Salesforce Plugin that generates these DTOs (see further below).

120.1. DEPENDENCIES

When using **salesforce** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-salesforce-starter</artifactId>
</dependency>
```

By default, **camel-salesforce-maven-plugin** uses TLSv1.3 to interact with salesforce. TLS version is configurable on the plugin. FIPS users can configure the property **sslContextParameters.secureSocketProtocol**. To use the **maven-plugin** you must add the following dependency to the **pom.xml** file.

```
<plugin>
  <groupId>org.apache.camel.maven</groupId>
  <artifactId>camel-salesforce-maven-plugin</artifactId>
  <version>${camel-community.version}</version>
  <executions>
    <execution>
      <goals>
        <goal>generate</goal>
      </goals>
      <configuration>
        <clientId>${camelSalesforce.clientId}</clientId>
        <clientSecret>${camelSalesforce.clientSecret}</clientSecret>
        <userName>${camelSalesforce.userName}</userName>
        <password>${camelSalesforce.password}</password>
        <sslContextParameters>
          <secureSocketProtocol>TLSv1.2</secureSocketProtocol>
        </sslContextParameters>
      </configuration>
      <includes>
        <include>Contact</include>
      </includes>
    </execution>
  </executions>
</plugin>
```

Where **camel-community.version** refers to the corresponding Camel community version that you use when working with **camel-salesforce-maven-plugin**. For example, for Red Hat build of Camel Spring Boot version 4.4.0 you can use '4.4.0' version of Apache Camel.

120.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

120.2.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

120.2.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

120.3. COMPONENT OPTIONS

The Salesforce component supports 90 options, which are listed below.

Name	Description	Default	Type
apexMethod (common)	APEX method name.		String
apexQueryParams (common)	Query params for APEX method.		Map
apiVersion (common)	Salesforce API version.	53.0	String
backoffIncrement (common)	Backoff interval increment for Streaming connection restart attempts for failures beyond CometD auto-reconnect.	1000	long
batchId (common)	Bulk API Batch ID.		String
contentType (common)	Bulk API content type, one of XML, CSV, ZIP_XML, ZIP_CSV. Enum values: • XML • CSV • JSON • ZIP_XML • ZIP_CSV • ZIP_JSON		ContentType
defaultReplayId (common)	Default replayId setting if no value is found in initialReplayIdMap.	-1	Long
fallBackReplayId (common)	ReplayId to fall back to after an Invalid Replay Id response.	-1	Long
format (common)	Payload format to use for Salesforce API calls, either JSON or XML, defaults to JSON. As of Camel 3.12, this option only applies to the Raw operation. Enum values: • JSON • XML		PayloadFormat

Name	Description	Default	Type
httpClient (common)	Custom Jetty Http Client to use to connect to Salesforce.		SalesforceHttpClient
httpClientConnectionTimeout (common)	Connection timeout used by the HttpClient when connecting to the Salesforce server.	60000	long
httpClientIdleTimeout (common)	Timeout used by the HttpClient when waiting for response from the Salesforce server.	10000	long
httpMaxContentLength (common)	Max content length of an HTTP response.		Integer
httpRequestBufferSize (common)	HTTP request buffer size. May need to be increased for large SOQL queries.	8192	Integer
includeDetails (common)	Include details in Salesforce1 Analytics report, defaults to false.		Boolean
initialReplayIdMap (common)	Replay IDs to start from per channel name.		Map
instanceId (common)	Salesforce1 Analytics report execution instance ID.		String
jobId (common)	Bulk API Job ID.		String
limit (common)	Limit on number of returned records. Applicable to some of the API, check the Salesforce documentation.		Integer
locator (common)	Locator provided by salesforce Bulk 2.0 API for use in getting results for a Query job.		String
maxBackoff (common)	Maximum backoff interval for Streaming connection restart attempts for failures beyond CometD auto-reconnect.	30000	long

Name	Description	Default	Type
maxRecords (common)	The maximum number of records to retrieve per set of results for a Bulk 2.0 Query. The request is still subject to the size limits. If you are working with a very large number of query results, you may experience a timeout before receiving all the data from Salesforce. To prevent a timeout, specify the maximum number of records your client is expecting to receive in the maxRecords parameter. This splits the results into smaller sets with this value as the maximum size.		Integer
notFoundBehaviour (common)	Sets the behaviour of 404 not found status received from Salesforce API. Should the body be set to NULL NotFoundBehaviour#NULL or should a exception be signaled on the exchange NotFoundBehaviour#EXCEPTION - the default. Enum values: ● EXCEPTION ● NULL	EXCEPTION	NotFoundBehaviour
notifyForFields (common)	Notify for fields, options are ALL, REFERENCED, SELECT, WHERE. Enum values: ● ALL ● REFERENCED ● SELECT ● WHERE		NotifyForFieldsEnum
notifyForOperationCreate (common)	Notify for create operation, defaults to false (API version = 29.0).		Boolean
notifyForOperationDelete (common)	Notify for delete operation, defaults to false (API version = 29.0).		Boolean

Name	Description	Default	Type
notifyForOperations (common)	Notify for operations, options are ALL, CREATE, EXTENDED, UPDATE (API version 29.0). Enum values: • ALL • CREATE • EXTENDED • UPDATE		NotifyForOperationsEnum
notifyForOperationUndelete (common)	Notify for un-delete operation, defaults to false (API version = 29.0).		Boolean
notifyForOperationUpdate (common)	Notify for update operation, defaults to false (API version = 29.0).		Boolean
objectMapper (common)	Custom Jackson ObjectMapper to use when serializing/deserializing Salesforce objects.		ObjectMapper
packages (common)	In what packages are the generated DTO classes. Typically the classes would be generated using camel-salesforce-maven-plugin. Set it if using the generated DTOs to gain the benefit of using short SOBJ names in parameters/header values. Multiple packages can be separated by comma.		String
pkChunking (common)	Use PK Chunking. Only for use in original Bulk API. Bulk 2.0 API performs PK chunking automatically, if necessary.		Boolean
pkChunkingChunkSize (common)	Chunk size for use with PK Chunking. If unspecified, salesforce default is 100,000. Maximum size is 250,000.		Integer
pkChunkingParent (common)	Specifies the parent object when you're enabling PK chunking for queries on sharing objects. The chunks are based on the parent object's records rather than the sharing object's records. For example, when querying on AccountShare, specify Account as the parent object. PK chunking is supported for sharing objects as long as the parent object is supported.		String

Name	Description	Default	Type
pkChunkingStartRow (common)	Specifies the 15-character or 18-character record ID to be used as the lower boundary for the first chunk. Use this parameter to specify a starting ID when restarting a job that failed between batches.		String
queryLocator (common)	Query Locator provided by salesforce for use when a query results in more records than can be retrieved in a single call. Use this value in a subsequent call to retrieve additional records.		String
rawPayload (common)	Use raw payload String for request and response (either JSON or XML depending on format), instead of DTOs, false by default.	false	boolean
reportId (common)	Salesforce1 Analytics report Id.		String
reportMetadata (common)	Salesforce1 Analytics report metadata for filtering.		ReportMetadata
resultId (common)	Bulk API Result ID.		String
sObjectBlobFieldName (common)	SObject blob field name.		String
sObjectClass (common)	Fully qualified SObject class name, usually generated using camel-salesforce-maven-plugin.		String
sObjectFields (common)	SObject fields to retrieve.		String
sObjectId (common)	SObject ID if required by API.		String
sObjectIdName (common)	SObject external ID field name.		String
sObjectIdValue (common)	SObject external ID field value.		String
sObjectName (common)	SObject name if required or supported by API.		String
sObjectQuery (common)	Salesforce SOQL query string.		String

Name	Description	Default	Type
sObjectSearch (common)	Salesforce SOSL search string.		String
updateTopic (common)	Whether to update an existing Push Topic when using the Streaming API, defaults to false.	false	boolean
config (common (advanced))	Global endpoint configuration - use to set values that are common to all endpoints.		SalesforceEndpointConfig
httpClientProperties (common (advanced))	Used to set any properties that can be configured on the underlying HTTP client. Have a look at properties of SalesforceHttpClient and the Jetty HttpClient for all available options.		Map
longPollingTransportProperties (common (advanced))	Used to set any properties that can be configured on the LongPollingTransport used by the BayeuxClient (CometD) used by the streaming api.		Map
workerPoolMaxSize (common (advanced))	Maximum size of the thread pool used to handle HTTP responses.	20	int
workerPoolSize (common (advanced))	Size of the thread pool used to handle HTTP responses.	10	int
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
allOrNone (producer)	Composite API option to indicate to rollback all records if any are not successful.	false	boolean
apexUrl (producer)	APEX method URL.		String
compositeMethod (producer)	Composite (raw) method.		String

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
rawHttpHeaders (producer)	Comma separated list of message headers to include as HTTP parameters for Raw operation.		String
rawMethod (producer)	HTTP method to use for the Raw operation.		String
rawPath (producer)	The portion of the endpoint URL after the domain name. E.g., '/services/data/v52.0/subjects/Account/'.		String
rawQueryParameters (producer)	Comma separated list of message headers to include as query parameters for Raw operation. Do not url-encode values as this will be done automatically.		String
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
httpProxyExcludedAddresses (proxy)	A list of addresses for which HTTP proxy server should not be used.		Set
httpProxyHost (proxy)	Hostname of the HTTP proxy server to use.		String
httpProxyIncludedAddresses (proxy)	A list of addresses for which HTTP proxy server should be used.		Set
httpProxyPort (proxy)	Port number of the HTTP proxy server to use.		Integer

Name	Description	Default	Type
httpProxySocks4 (proxy)	If set to true the configures the HTTP proxy to use as a SOCKS4 proxy.	false	boolean
authenticationType (security)	Explicit authentication method to be used, one of USERNAME_PASSWORD, REFRESH_TOKEN or JWT. Salesforce component can auto-determine the authentication method to use from the properties set, set this property to eliminate any ambiguity. Enum values: • USERNAME_PASSWORD • REFRESH_TOKEN • JWT		AuthenticationType
clientId (security)	Required OAuth Consumer Key of the connected app configured in the Salesforce instance setup. Typically a connected app needs to be configured but one can be provided by installing a package.		String
clientSecret (security)	OAuth Consumer Secret of the connected app configured in the Salesforce instance setup.		String
httpProxyAuthUri (security)	Used in authentication against the HTTP proxy server, needs to match the URI of the proxy server in order for the httpProxyUsername and httpProxyPassword to be used for authentication.		String
httpProxyPassword (security)	Password to use to authenticate against the HTTP proxy server.		String
httpProxyRealm (security)	Realm of the proxy server, used in preemptive Basic/Digest authentication methods against the HTTP proxy server.		String
httpProxySecure (security)	If set to false disables the use of TLS when accessing the HTTP proxy.	true	boolean
httpProxyUseDigestAuth (security)	If set to true Digest authentication will be used when authenticating to the HTTP proxy, otherwise Basic authorization method will be used.	false	boolean
httpProxyUsername (security)	Username to use to authenticate against the HTTP proxy server.		String

Name	Description	Default	Type
instanceUrl (security)	URL of the Salesforce instance used after authentication, by default received from Salesforce on successful authentication.		String
jwtAudience (security)	Value to use for the Audience claim (aud) when using OAuth JWT flow. If not set, the login URL will be used, which is appropriate in most cases.		String
keystore (security)	KeyStore parameters to use in OAuth JWT flow. The KeyStore should contain only one entry with private key and certificate. Salesforce does not verify the certificate chain, so this can easily be a selfsigned certificate. Make sure that you upload the certificate to the corresponding connected app.		KeyStoreParameters
lazyLogin (security)	If set to true prevents the component from authenticating to Salesforce with the start of the component. You would generally set this to the (default) false and authenticate early and be immediately aware of any authentication issues.	false	boolean
loginConfig (security)	All authentication configuration in one nested bean, all properties set there can be set directly on the component as well.		SalesforceLoginConfig
loginUrl (security)	Required URL of the Salesforce instance used for authentication, by default set to https://login.salesforce.com .	https://login.salesforce.com	String
password (security)	Password used in OAuth flow to gain access to access token. It's easy to get started with password OAuth flow, but in general one should avoid it as it is deemed less secure than other flows. Make sure that you append security token to the end of the password if using one.		String
refreshToken (security)	Refresh token already obtained in the refresh token OAuth flow. One needs to setup a web application and configure a callback URL to receive the refresh token, or configure using the builtin callback at https://login.salesforce.com/services/oauth2/success or https://test.salesforce.com/services/oauth2/success and then retrieve the refresh_token from the URL at the end of the flow. Note that in development organizations Salesforce allows hosting the callback web application at localhost.		String

Name	Description	Default	Type
sslContextParameters (security)	SSL parameters to use, see SSLContextParameters class for all available options.		SSLContextParameters
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean
userName (security)	Username used in OAuth flow to gain access to access token. It's easy to get started with password OAuth flow, but in general one should avoid it as it is deemed less secure than other flows.		String

120.4. ENDPOINT OPTIONS

The Salesforce endpoint is configured using URI syntax:

```
salesforce:operationName:topicName
```

with the following path and query parameters:

120.4.1. Path Parameters (2 parameters)

Name	Description	Default	Type
operationName (producer)	The operation to use. Enum values: • getVersions • getResources • getGlobalObjects • getBasicInfo • getDescription • getObject • createSObject • updateSObject • deleteSObject • getObjectWithId • upsertSObject • deleteObjectWithId		OperationName

Name	• getBlobField • query Description	Default	Type
	• queryMore • queryAll • search • apexCall • recent • createJob • getJob • closeJob • abortJob • createBatch • getBatch • getAllBatches • getRequest • getResults • createBatchQuery • getQueryResultIds • getQueryResult • getRecentReports • getReportDescription • executeSyncReport • executeAsyncReport • getReportInstances • getReportResults • limits • approval • approvals • composite-tree • composite-batch • composite • compositeRetrieveSObjectCollections • compositeCreateSObjectCollections		

Name	• compositeUpdateSOBJECTCollections • compositeUpsertSOBJECTCollections Description	Default	Type
	• compositeDeleteSOBJECTCollections • bulk2GetAllJobs • bulk2CreateJob • bulk2GetJob • bulk2CreateBatch • bulk2CloseJob • bulk2AbortJob • bulk2DeleteJob • bulk2GetSuccessfulResults • bulk2GetFailedResults • bulk2GetUnprocessedRecords • bulk2CreateQueryJob • bulk2GetQueryJob • bulk2GetAllQueryJobs • bulk2GetQueryJobResults • bulk2AbortQueryJob • bulk2DeleteQueryJob • raw		
topicName (consumer)	The name of the topic/channel to use.		String

120.4.2. Query Parameters (57 parameters)

Name	Description	Default	Type
apexMethod (common)	APEX method name.		String
apexQueryParams (common)	Query params for APEX method.		Map
apiVersion (common)	Salesforce API version.	53.0	String

Name	Description	Default	Type
backoffIncrement (common)	Backoff interval increment for Streaming connection restart attempts for failures beyond CometD auto-reconnect.	1000	long
batchId (common)	Bulk API Batch ID.		String
contentType (common)	Bulk API content type, one of XML, CSV, ZIP_XML, ZIP_CSV. Enum values: • XML • CSV • JSON • ZIP_XML • ZIP_CSV • ZIP_JSON		ContentType
defaultReplayId (common)	Default replayId setting if no value is found in initialReplayIdMap.	-1	Long
fallBackReplayId (common)	ReplayId to fall back to after an Invalid Replay Id response.	-1	Long
format (common)	Payload format to use for Salesforce API calls, either JSON or XML, defaults to JSON. As of Camel 3.12, this option only applies to the Raw operation. Enum values: • JSON • XML		PayloadFormat
httpClient (common)	Custom Jetty Http Client to use to connect to Salesforce.		SalesforceHttpClient
includeDetails (common)	Include details in Salesforce Analytics report, defaults to false.		Boolean
initialReplayIdMap (common)	Replay IDs to start from per channel name.		Map

Name	Description	Default	Type
instanceId (common)	Salesforce1 Analytics report execution instance ID.		String
jobId (common)	Bulk API Job ID.		String
limit (common)	Limit on number of returned records. Applicable to some of the API, check the Salesforce documentation.		Integer
locator (common)	Locator provided by salesforce Bulk 2.0 API for use in getting results for a Query job.		String
maxBackoff (common)	Maximum backoff interval for Streaming connection restart attempts for failures beyond CometD auto-reconnect.	30000	long
maxRecords (common)	The maximum number of records to retrieve per set of results for a Bulk 2.0 Query. The request is still subject to the size limits. If you are working with a very large number of query results, you may experience a timeout before receiving all the data from Salesforce. To prevent a timeout, specify the maximum number of records your client is expecting to receive in the maxRecords parameter. This splits the results into smaller sets with this value as the maximum size.		Integer
notFoundBehaviour (common)	Sets the behaviour of 404 not found status received from Salesforce API. Should the body be set to NULL NotFoundBehaviour#NULL or should a exception be signaled on the exchange NotFoundBehaviour#EXCEPTION - the default. Enum values: ● EXCEPTION ● NULL	EXCEPTION	NotFoundBehaviour

Name	Description	Default	Type
notifyForFields (common)	Notify for fields, options are ALL, REFERENCED, SELECT, WHERE. Enum values: • ALL • REFERENCED • SELECT • WHERE		NotifyForFieldsEnum
notifyForOperationCreate (common)	Notify for create operation, defaults to false (API version = 29.0).		Boolean
notifyForOperationDelete (common)	Notify for delete operation, defaults to false (API version = 29.0).		Boolean
notifyForOperations (common)	Notify for operations, options are ALL, CREATE, EXTENDED, UPDATE (API version 29.0). Enum values: • ALL • CREATE • EXTENDED • UPDATE		NotifyForOperationsEnum
notifyForOperationUndelete (common)	Notify for un-delete operation, defaults to false (API version = 29.0).		Boolean
notifyForOperationUpdate (common)	Notify for update operation, defaults to false (API version = 29.0).		Boolean
objectMapper (common)	Custom Jackson ObjectMapper to use when serializing/deserializing Salesforce objects.		ObjectMapper
pkChunking (common)	Use PK Chunking. Only for use in original Bulk API. Bulk 2.0 API performs PK chunking automatically, if necessary.		Boolean

Name	Description	Default	Type
pkChunkingChunkSize (common)	Chunk size for use with PK Chunking. If unspecified, salesforce default is 100,000. Maximum size is 250,000.		Integer
pkChunkingParent (common)	Specifies the parent object when you're enabling PK chunking for queries on sharing objects. The chunks are based on the parent object's records rather than the sharing object's records. For example, when querying on AccountShare, specify Account as the parent object. PK chunking is supported for sharing objects as long as the parent object is supported.		String
pkChunkingStartRow (common)	Specifies the 15-character or 18-character record ID to be used as the lower boundary for the first chunk. Use this parameter to specify a starting ID when restarting a job that failed between batches.		String
queryLocator (common)	Query Locator provided by salesforce for use when a query results in more records than can be retrieved in a single call. Use this value in a subsequent call to retrieve additional records.		String
rawPayload (common)	Use raw payload String for request and response (either JSON or XML depending on format), instead of DTOs, false by default.	false	boolean
reportId (common)	Salesforce1 Analytics report Id.		String
reportMetadata (common)	Salesforce1 Analytics report metadata for filtering.		ReportMetadata
resultId (common)	Bulk API Result ID.		String
sObjectBlobFieldName (common)	SObject blob field name.		String
sObjectClass (common)	Fully qualified SObject class name, usually generated using camel-salesforce-maven-plugin.		String
sObjectFields (common)	SObject fields to retrieve.		String
sObjectId (common)	SObject ID if required by API.		String

Name	Description	Default	Type
sObjectIdName (common)	SObject external ID field name.		String
sObjectIdValue (common)	SObject external ID field value.		String
sObjectName (common)	SObject name if required or supported by API.		String
sObjectQuery (common)	Salesforce SOQL query string.		String
sObjectSearch (common)	Salesforce SOSL search string.		String
updateTopic (common)	Whether to update an existing Push Topic when using the Streaming API, defaults to false.	false	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
replayId (consumer)	The replayId value to use when subscribing.		Long
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>

Name	Description	Default	Type
allOrNone (producer)	Composite API option to indicate to rollback all records if any are not successful.	false	boolean
apexUrl (producer)	APEX method URL.		String
compositeMethod (producer)	Composite (raw) method.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
rawHttpHeaders (producer)	Comma separated list of message headers to include as HTTP parameters for Raw operation.		String
rawMethod (producer)	HTTP method to use for the Raw operation.		String
rawPath (producer)	The portion of the endpoint URL after the domain name. E.g., '/services/data/v52.0/subjects/Account/'.		String
rawQueryParameters (producer)	Comma separated list of message headers to include as query parameters for Raw operation. Do not url-encode values as this will be done automatically.		String

120.5. AUTHENTICATING TO SALESFORCE

The component supports three OAuth authentication flows:

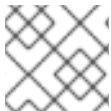
- [OAuth 2.0 Username-Password Flow](#)
- [OAuth 2.0 Refresh Token Flow](#)
- [OAuth 2.0 JWT Bearer Token Flow](#)

For each of the flow different set of properties needs to be set:

Table 120.1. Table 1. Properties to set for each authentication flow

Property	Where to find it on Salesforce	Flow
clientId	Connected App, Consumer Key	All flows
clientSecret	Connected App, Consumer Secret	Username-Password, Refresh Token
userName	Salesforce user username	Username-Password, JWT Bearer Token
password	Salesforce user password	Username-Password
refreshToken	From OAuth flow callback	Refresh Token
keystore	Connected App, Digital Certificate	JWT Bearer Token

The component auto determines what flow you're trying to configure, to be remove ambiguity set the **authenticationType** property.



NOTE

Using Username-Password Flow in production is not encouraged.



NOTE

The certificate used in JWT Bearer Token Flow can be a selfsigned certificate. The KeyStore holding the certificate and the private key must contain only single certificate-private key entry.

120.6. URI FORMAT

When used as a consumer, receiving streaming events, the URI scheme is:

```
salesforce:topic?options
```

When used as a producer, invoking the Salesforce REST APIs, the URI scheme is:

```
salesforce:operationName?options
```

120.7. PASSING IN SALESFORCE HEADERS AND FETCHING SALESFORCE RESPONSE HEADERS

There is support to pass [Salesforce headers](#) via inbound message headers, header names that start with **Sforce** or **x-sfdc** on the Camel message will be passed on in the request, and response headers that start with **Sforce** will be present in the outbound message headers.

For example to fetch API limits you can specify:

```
// in your Camel route set the header before Salesforce endpoint
//...
.setHeader("Sforce-Limit-Info", constant("api-usage"))
.to("salesforce:getGlobalObjects")
.to(myProcessor);

// myProcessor will receive `Sforce-Limit-Info` header on the outbound
// message
class MyProcessor implements Processor {
    public void process(Exchange exchange) throws Exception {
        Message in = exchange.getIn();
        String apiLimits = in.getHeader("Sforce-Limit-Info", String.class);
    }
}
```

In addition, HTTP response status code and text are available as headers **Exchange.HTTP_RESPONSE_CODE** and **Exchange.HTTP_RESPONSE_TEXT**.

120.8. SUPPORTED SALESFORCE APIS

The component supports the following Salesforce APIs

Producer endpoints can use the following APIs. Most of the APIs process one record at a time, the Query API can retrieve multiple Records.

120.8.1. Rest API

You can use the following for **operationName**:

- `getVersions` - Gets supported Salesforce REST API versions
- `getResources` - Gets available Salesforce REST Resource endpoints
- `getGlobalObjects` - Gets metadata for all available SObject types
- `getBasicInfo` - Gets basic metadata for a specific SObject type
- `getDescription` - Gets comprehensive metadata for a specific SObject type
- `getSObject` - Gets an SObject using its Salesforce Id
- `createSObject` - Creates an SObject
- `updateSObject` - Updates an SObject using Id
- `deleteSObject` - Deletes an SObject using Id
- `getSObjectWithId` - Gets an SObject using an external (user defined) id field
- `upsertSObject` - Updates or inserts an SObject using an external id
- `deleteSObjectWithId` - Deletes an SObject using an external id
- `query` - Runs a Salesforce SOQL query

- queryMore - Retrieves more results (in case of large number of results) using result link returned from the 'query' API
- search - Runs a Salesforce SOSL query
- limits - fetching organization API usage limits
- recent - fetching recent items
- approval - submit a record or records (batch) for approval process
- approvals - fetch a list of all approval processes
- composite - submit up to 25 possibly related REST requests and receive individual responses. It's also possible to use "raw" composite without limitation.
- composite-tree - create up to 200 records with parent-child relationships (up to 5 levels) in one go
- composite-batch - submit a composition of requests in batch
- compositeRetrieveObjectCollections - Retrieve one or more records of the same object type.
- compositeCreateObjectCollections - Add up to 200 records, returning a list of SaveObjectResult objects.
- compositeUpdateObjectCollections - Update up to 200 records, returning a list of SaveObjectResult objects.
- compositeUpsertObjectCollections - Create or update (upsert) up to 200 records based on an external ID field. Returns a list of UpsertObjectResult objects.
- compositeDeleteObjectCollections - Delete up to 200 records, returning a list of SaveObjectResult objects.
- queryAll - Runs a SOQL query. It returns the results that are deleted because of a merge (merges up to three records into one of the records, deletes the others, and reparents any related records) or delete. Also returns the information about archived Task and Event records.
- getBlobField - Retrieves the specified blob field from an individual record.
- apexCall - Executes a user defined APEX REST API call.
- raw - Send requests to salesforce and have full, raw control over endpoint, parameters, body, etc.

For example, the following producer endpoint uses the upsertSObject API, with the sObjectIdName parameter specifying 'Name' as the external id field. The request message body should be an SObject DTO generated using the maven plugin. The response message will either be **null** if an existing record was updated, or **CreateSObjectResult** with an id of the new record, or a list of errors while creating the new object.

```
...to("salesforce:upsertSObject?sObjectIdName=Name")...
```

120.8.2. Bulk 2.0 API

The Bulk 2.0 API has a simplified model over the original Bulk API. Use it to quickly load a large amount of data into salesforce, or query a large amount of data out of salesforce. Data must be provided in CSV format. The minimum API version for Bulk 2.0 is v41.0. The minimum API version for Bulk Queries is v47.0. DTO classes mentioned below are from the **org.apache.camel.component.salesforce.api.dto.bulkv2** package. The following operations are supported:

- **bulk2CreateJob** – Create a bulk job. Supply an instance of **Job** in the message body.
- **bulk2GetJob** – Get an existing Job. **jobId** parameter is required.
- **bulk2CreateBatch** – Add a Batch of CSV records to a job. Supply CSV data in the message body. The first row must contain headers. **jobId** parameter is required.
- **bulk2CloseJob** – Close a job. You must close the job in order for it to be processed or aborted/deleted. **jobId** parameter is required.
- **bulk2AbortJob** – Abort a job. **jobId** parameter is required.
- **bulk2DeleteJob** – Delete a job. **jobId** parameter is required.
- **bulk2GetSuccessfulResults** – Get successful results for a job. Returned message body will contain an InputStream of CSV data. **jobId** parameter is required.
- **bulk2GetFailedResults** – Get failed results for a job. Returned message body will contain an InputStream of CSV data. **jobId** parameter is required.
- **bulk2GetUnprocessedRecords** – Get unprocessed records for a job. Returned message body will contain an InputStream of CSV data. **jobId** parameter is required.
- **bulk2GetAllJobs** – Get all jobs. Response body is an instance of **Jobs**. If the **done** property is false, there are additional pages to fetch, and the **nextRecordsUrl** property contains the value to be set in the **queryLocator** parameter on subsequent calls.
- **bulk2CreateQueryJob** – Create a bulk query job. Supply an instance of **QueryJob** in the message body.
- **bulk2GetQueryJob** – Get a bulk query job. **jobId** parameter is required.
- **bulk2GetQueryJobResults** – Get bulk query job results. **jobId** parameter is required. Accepts **maxRecords** and **locator** parameters. Response message headers include **Sforce-NumberOfRecords** and **Sforce-Locator** headers. The value of **Sforce-Locator** can be passed into subsequent calls via the **locator** parameter.
- **bulk2AbortQueryJob** – Abort a bulk query job. **jobId** parameter is required.
- **bulk2DeleteQueryJob** – Delete a bulk query job. **jobId** parameter is required.
- **bulk2GetAllQueryJobs** – Get all jobs. Response body is an instance of **QueryJobs**. If the **done** property is false, there are additional pages to fetch, and the **nextRecordsUrl** property contains the value to be set in the **queryLocator** parameter on subsequent calls.

120.8.3. Rest Bulk (original) API

Producer endpoints can use the following APIs. All Job data formats, i.e. xml, csv, zip/xml, and zip/csv

are supported.

The request and response have to be marshalled/unmarshalled by the route. Usually the request will be some stream source like a CSV file, and the response may also be saved to a file to be correlated with the request.

You can use the following for **operationName**:

- **createJob** - Creates a Salesforce Bulk Job. Must supply a **JobInfo** instance in body. PK Chunking is supported via the `pkChunking*` options. See an explanation [here](#).
- **getJob** - Gets a Job using its Salesforce Id
- **closeJob** - Closes a Job
- **abortJob** - Aborts a Job
- **createBatch** - Submits a Batch within a Bulk Job
- **getBatch** - Gets a Batch using Id
- **getAllBatches** - Gets all Batches for a Bulk Job Id
- **getRequest** - Gets Request data (XML/CSV) for a Batch
- **getResults** - Gets the results of the Batch when its complete
- **createBatchQuery** - Creates a Batch from an SOQL query
- **getQueryResultIds** - Gets a list of Result Ids for a Batch Query
- **getQueryResult** - Gets results for a Result Id
- **getRecentReports** - Gets up to 200 of the reports you most recently viewed by sending a GET request to the Report List resource.
- **getReportDescription** - Retrieves the report, report type, and related metadata for a report, either in a tabular or summary or matrix format.
- **executeSyncReport** - Runs a report synchronously with or without changing filters and returns the latest summary data.
- **executeAsyncReport** - Runs an instance of a report asynchronously with or without filters and returns the summary data with or without details.
- **getReportInstances** - Returns a list of instances for a report that you requested to be run asynchronously. Each item in the list is treated as a separate instance of the report.
- **getReportResults**: Contains the results of running a report.

For example, the following producer endpoint uses the `createBatch` API to create a Job Batch. The in message must contain a body that can be converted into an **InputStream** (usually UTF-8 CSV or XML content from a file, etc.) and header fields `'jobId'` for the Job and `'contentType'` for the Job content type, which can be XML, CSV, ZIP_XML or ZIP_CSV. The put message body will contain **BatchInfo** on success, or throw a **SalesforceException** on error.

```
...to("salesforce:createBatch")..
```

120.8.4. Rest Streaming API

Consumer endpoints can use the following syntax for streaming endpoints to receive Salesforce notifications on create/update.

To create and subscribe to a topic

```
from("salesforce:CamelTestTopic?
  notifyForFields=ALL&notifyForOperations=ALL&sObjectName=Merchandise__c&updateTopic=true&sObjectQuery=SELECT Id, Name FROM Merchandise__c")...
```

To subscribe to an existing topic

```
from("salesforce:CamelTestTopic&sObjectName=Merchandise__c")...
```

120.8.5. Platform events

To emit a platform event use **createSObject** operation. And set the message body can be JSON string or InputStream with key-value data – in that case **sObjectName** needs to be set to the API name of the event, or a class that extends from AbstractDTOBase with the appropriate class name for the event.

For example using a DTO:

```
class Order_Event__e extends AbstractDTOBase {
  @JsonProperty("OrderNumber")
  private String orderNumber;
  // ... other properties and getters/setters
}

from("timer:tick")
  .process(exchange -> {
    final Message in = exchange.getIn();
    String orderNumber = "ORD" + exchange.getProperty(Exchange.TIMER_COUNTER);
    Order_Event__e event = new Order_Event__e();
    event.setOrderNumber(orderNumber);
    in.setBody(event);
  })
  .to("salesforce:createSObject");
```

Or using JSON event data:

```
from("timer:tick")
  .process(exchange -> {
    final Message in = exchange.getIn();
    String orderNumber = "ORD" + exchange.getProperty(Exchange.TIMER_COUNTER);
    in.setBody("{\"OrderNumber\":\"" + orderNumber + "\"}");
  })
  .to("salesforce:createSObject?sObjectName=Order_Event__e");
```

To receive platform events use the consumer endpoint with the API name of the platform event prefixed with **event/** (or **/event/**), e.g.: **salesforce:events/Order_Event__e**. Processor consuming from that endpoint will receive either **org.apache.camel.component.salesforce.api.dto.PlatformEvent** object or **org.cometd.bayeux.Message** in the body depending on the **rawPayload** being **false** or **true** respectively.

For example, in the simplest form to consume one event:

```
PlatformEvent event = consumer.receiveBody("salesforce:event/Order_Event__e",
PlatformEvent.class);
```

120.8.6. Change data capture events

On the one hand, Salesforce could be configured to emit notifications for record changes of select objects. On the other hand, the Camel Salesforce component could react to such notifications, allowing for instance to [synchronize those changes into an external system](#) .

The notifications of interest could be specified in the **from("salesforce:XXX")** clause of a Camel route via the subscription channel, e.g:

```
from("salesforce:data/ChangeEvents?replayId=-1").log("being notified of all change events")
from("salesforce:data/AccountChangeEvent?replayId=-1").log("being notified of change events for
Account records")
from("salesforce:data/Employee__ChangeEvent?replayId=-1").log("being notified of change events
for Employee__c custom object")
```

The received message contains either **java.util.Map<String,Object>** or **org.cometd.bayeux.Message** in the body depending on the **rawPayload** being **false** or **true** respectively. The **CamelSalesforceChangeType** header could be valued to one of **CREATE**, **UPDATE**, **DELETE** or **UNDELETE**.

More details about how to use the Camel Salesforce component change data capture capabilities could be found in the [ChangeEventsConsumerIntegrationTest](#).

The [Salesforce developer guide](#) is a good fit to better know the subtleties of implementing a change data capture integration application. The dynamic nature of change event body fields, high level replication steps as well as security considerations could be of interest.

120.9. EXAMPLES

120.9.1. Uploading a document to a ContentWorkspace

Create the ContentVersion in Java, using a Processor instance:

```
public class ContentProcessor implements Processor {
    public void process(Exchange exchange) throws Exception {
        Message message = exchange.getIn();

        ContentVersion cv = new ContentVersion();
        ContentWorkspace cw = getWorkspace(exchange);
        cv.setFirstPublishLocationId(cw.getId());
        cv.setTitle("test document");
        cv.setPathOnClient("test_doc.html");
        byte[] document = message.getBody(byte[].class);
        ObjectMapper mapper = new ObjectMapper();
        String enc = mapper.convertValue(document, String.class);
        cv.setVersionDataUrl(enc);
        message.setBody(cv);
    }
}
```



```
protected ContentWorkspace getWorkSpace(Exchange exchange) {
    // Look up the content workspace somehow, maybe use enrich() to add it to a
    // header that can be extracted here
    ----
}
}
```

Give the output from the processor to the Salesforce component:

```
from("file:///home/camel/library")
.to(new ContentProcessor()) // convert bytes from the file into a ContentVersion SObject
                             // for the salesforce component
.to("salesforce:createSObject");
```

120.10. USING SALESFORCE LIMITS API

With **salesforce:limits** operation you can fetch of API limits from Salesforce and then act upon that data received. The result of **salesforce:limits** operation is mapped to **org.apache.camel.component.salesforce.api.dto.Limits** class and can be used in a custom processors or expressions.

For instance, consider that you need to limit the API usage of Salesforce so that 10% of daily API requests is left for other routes. The body of output message contains an instance of **org.apache.camel.component.salesforce.api.dto.Limits** object that can be used in conjunction with Content Based Router and Content Based Router and [Spring Expression Language \(SpEL\)](#) to choose when to perform queries.

Notice how multiplying **1.0** with the integer value held in **body.dailyApiRequests.remaining** makes the expression evaluate as with floating point arithmetic, without it – it would end up making integral division which would result with either **0** (some API limits consumed) or **1** (no API limits consumed).

```
from("direct:querySalesforce")
.to("salesforce:limits")
.choice()
.when(spel("#{1.0 * body.dailyApiRequests.remaining / body.dailyApiRequests.max < 0.1}"))
.to("salesforce:query?...")
.otherwise()
.setBody(constant("Used up Salesforce API limits, leaving 10% for critical routes"))
.endChoice()
```

120.11. WORKING WITH APPROVALS

All the properties are named exactly the same as in the Salesforce REST API prefixed with **approval..** You can set approval properties by setting **approval.PropertyName** of the Endpoint these will be used as template – meaning that any property not present in either body or header will be taken from the Endpoint configuration. Or you can set the approval template on the Endpoint by assigning **approval** property to a reference onto a bean in the Registry.

You can also provide header values using the same **approval.PropertyName** in the incoming message headers.

And finally body can contain one **ApprovalRequest** or an **Iterable** of **ApprovalRequest** objects to process as a batch.

The important thing to remember is the priority of the values specified in these three mechanisms:

1. value in body takes precedence before any other
2. value in message header takes precedence before template value
3. value in template is set if no other value in header or body was given

For example to send one record for approval using values in headers use:

Given a route:

```
from("direct:example1")//
    .setHeader("approval.ContextId", simple("${body['contextId']}"))
    .setHeader("approval.NextApproverIds", simple("${body['nextApproverIds']}"))
    .to("salesforce:approval?"//
        + "approval.actionType=Submit"//
        + "&approval.comments=this is a test"//
        + "&approval.processDefinitionNameOrId=Test_Account_Process"//
        + "&approval.skipEntryCriteria=true");
```

You could send a record for approval using:

```
final Map<String, String> body = new HashMap<>();
body.put("contextId", accountIds.iterator().next());
body.put("nextApproverIds", userId);

final ApprovalResult result = template.requestBody("direct:example1", body, ApprovalResult.class);
```

120.12. USING SALESFORCE RECENT ITEMS API

To fetch the recent items use **salesforce:recent** operation. This operation returns an **java.util.List** of **org.apache.camel.component.salesforce.api.dto.RecentItem** objects (**List<RecentItem>**) that in turn contain the **Id**, **Name** and **Attributes** (with **type** and **url** properties). You can limit the number of returned items by specifying **limit** parameter set to maximum number of records to return. For example:

```
from("direct:fetchRecentItems")
    to("salesforce:recent")
    .split().body()
    .log("${body.name} at ${body.attributes.url}");
```

120.13. USING SALESFORCE COMPOSITE API TO SUBMIT SUBJECT TREE

To create up to 200 records including parent-child relationships use **salesforce:composite-tree** operation. This requires an instance of **org.apache.camel.component.salesforce.api.dto.composite.SObjectTree** in the input message and returns the same tree of objects in the output message. The **org.apache.camel.component.salesforce.api.dto.AbstractObjectBase** instances within the tree get

updated with the identifier values (**Id** property) or their corresponding **org.apache.camel.component.salesforce.api.dto.composite.SObjectNode** is populated with **errors** on failure.

Note that for some records operation can succeed and for some it can fail – so you need to manually check for errors.

Easiest way to use this functionality is to use the DTOs generated by the **camel-salesforce-maven-plugin**, but you also have the option of customizing the references that identify the each object in the tree, for instance primary keys from your database.

Lets look at an example:

```
Account account = ...
Contact president = ...
Contact marketing = ...

Account anotherAccount = ...
Contact sales = ...
Asset someAsset = ...

// build the tree
SObjectTree request = new SObjectTree();
request.addObject(account).addChildren(president, marketing);
request.addObject(anotherAccount).addChild(sales).addChild(someAsset);

final SObjectTree response = template.requestBody("salesforce:composite-tree", tree,
SObjectTree.class);
final Map<Boolean, List<SObjectNode>> result = response.allNodes()
.collect(Collectors.groupingBy(SObjectNode::hasErrors));

final List<SObjectNode> withErrors = result.get(true);
final List<SObjectNode> succeeded = result.get(false);

final String firstId = succeeded.get(0).getId();
```

120.14. USING SALESFORCE COMPOSITE API TO SUBMIT MULTIPLE REQUESTS IN A BATCH

The Composite API batch operation (**composite-batch**) allows you to accumulate multiple requests in a batch and then submit them in one go, saving the round trip cost of multiple individual requests. Each response is then received in a list of responses with the order preserved, so that the n-th requests response is in the n-th place of the response.



NOTE

The results can vary from API to API so the result of the request is given as a **java.lang.Object**. In most cases the result will be a **java.util.Map** with string keys and values or other **java.util.Map** as value. Requests are made in JSON format and hold some type information (i.e. it is known what values are strings and what values are numbers).

Lets look at an example:

```

final String accountId = ...
final ObjectBatch batch = new ObjectBatch("38.0");

final Account updates = new Account();
updates.setName("NewName");
batch.addUpdate("Account", accountId, updates);

final Account newAccount = new Account();
newAccount.setName("Account created from Composite batch API");
batch.addCreate(newAccount);

batch.addGet("Account", accountId, "Name", "BillingPostalCode");

batch.addDelete("Account", accountId);

final ObjectBatchResponse response = template.requestBody("salesforce:composite-batch", batch,
ObjectBatchResponse.class);

boolean hasErrors = response.hasErrors(); // if any of the requests has resulted in either 4xx or 5xx
HTTP status
final List<ObjectBatchResult> results = response.getResults(); // results of three operations sent in
batch

final ObjectBatchResult updateResult = results.get(0); // update result
final int updateStatus = updateResult.getStatusCode(); // probably 204
final Object updateResultData = updateResult.getResult(); // probably null

final ObjectBatchResult createResult = results.get(1); // create result
@SuppressWarnings("unchecked")
final Map<String, Object> createData = (Map<String, Object>) createResult.getResult();
final String newAccountId = createData.get("id"); // id of the new account, this is for JSON, for XML it
would be createData.get("Result").get("id")

final ObjectBatchResult retrieveResult = results.get(2); // retrieve result
@SuppressWarnings("unchecked")
final Map<String, Object> retrieveData = (Map<String, Object>) retrieveResult.getResult();
final String accountName = retrieveData.get("Name"); // Name of the retrieved account, this is for
JSON, for XML it would be createData.get("Account").get("Name")
final String accountBillingPostalCode = retrieveData.get("BillingPostalCode"); // Name of the retrieved
account, this is for JSON, for XML it would be createData.get("Account").get("BillingPostalCode")

final ObjectBatchResult deleteResult = results.get(3); // delete result
final int updateStatus = deleteResult.getStatusCode(); // probably 204
final Object updateResultData = deleteResult.getResult(); // probably null

```

120.15. USING SALESFORCE COMPOSITE API TO SUBMIT MULTIPLE CHAINED REQUESTS

The **composite** operation allows submitting up to 25 requests that can be chained together, for instance identifier generated in previous request can be used in subsequent request. Individual requests and responses are linked with the provided *reference*.

**NOTE**

Composite API supports only JSON payloads.

**NOTE**

As with the batch API the results can vary from API to API so the result of the request is given as a **java.lang.Object**. In most cases the result will be a **java.util.Map** with string keys and values or other **java.util.Map** as value. Requests are made in JSON format hold some type information (i.e. it is known what values are strings and what values are numbers).

Lets look at an example:

```
SObjectComposite composite = new SObjectComposite("38.0", true);

// first insert operation via an external id
final Account updateAccount = new TestAccount();
updateAccount.setName("Salesforce");
updateAccount.setBillingStreet("Landmark @ 1 Market Street");
updateAccount.setBillingCity("San Francisco");
updateAccount.setBillingState("California");
updateAccount.setIndustry(Account_IndustryEnum.TECHNOLOGY);
composite.addUpdate("Account", "001xx000003DlpcAAG", updateAccount, "UpdatedAccount");

final Contact newContact = new TestContact();
newContact.setLastName("John Doe");
newContact.setPhone("1234567890");
composite.addCreate(newContact, "NewContact");

final AccountContactJunction__c junction = new AccountContactJunction__c();
junction.setAccount__c("001xx000003DlpcAAG");
junction.setContactId__c("@{NewContact.id}");
composite.addCreate(junction, "JunctionRecord");

final SObjectCompositeResponse response = template.requestBody("salesforce:composite",
composite, SObjectCompositeResponse.class);
final List<SObjectCompositeResult> results = response.getCompositeResponse();

final SObjectCompositeResult accountUpdateResult = results.stream().filter(r ->
"UpdatedAccount".equals(r.getReferenceId())).findFirst().get()
final int statusCode = accountUpdateResult.getStatusCode(); // should be 200
final Map<String, ?> accountUpdateBody = accountUpdateResult.getBody();

final SObjectCompositeResult contactCreationResult = results.stream().filter(r ->
"JunctionRecord".equals(r.getReferenceId())).findFirst().get()
```

120.16. USING "RAW" SALESFORCE COMPOSITE

It's possible to directly call Salesforce composite by preparing the Salesforce JSON request in the route thanks to the **rawPayload** option.

For instance, you can have the following route:

```

from("timer:fire?period=2000").setBody(constant("{\n" +
    "  \"allOrNone\" : true,\n" +
    "  \"records\" : [ {\n" +
    "    \"attributes\" : {\"type\" : \"FOO\"},\n" +
    "    \"Name\" : \"123456789\",\n" +
    "    \"FOO\" : \"XXXX\",\n" +
    "    \"ACCOUNT\" : 2100.0\n" +
    "  }]\n" +
    "}")
.to("salesforce:composite?rawPayload=true")
.log("${body}");

```

The route directly creates the body as JSON and directly submit to salesforce endpoint using **rawPayload=true** option.

With this approach, you have the complete control on the Salesforce request.

POST is the default HTTP method used to send raw Composite requests to salesforce. Use the **compositeMethod** option to override to the other supported value, **GET**, which returns a list of other available composite resources.

120.17. USING RAW OPERATION

Send HTTP requests to salesforce with full, raw control of all aspects of the call. Any serialization or deserialization of request and response bodies must be performed in the route. The **Content-Type** HTTP header will be automatically set based on the **format** option, but this can be overridden with the **rawHttpHeaders** option.

Parameter	Type	Description	Default	Required
request body	String or InputStream	Body of the HTTP request		
rawPath	String	The portion of the endpoint URL after the domain name, e.g., '/services/data/v51.0/subjects/Account/'		x
rawMethod	String	The HTTP method		x
rawQueryParameters	String	Comma separated list of message headers to include as query parameters. Do not url-encode values as this will be done automatically.		

Parameter	Type	Description	Default	Required
rawHttpHeaders	String	Comma separated list of message headers to include as HTTP headers		

120.17.1. Query example

In this example we'll send a query to the REST API. The query must be passed in a URL parameter called "q", so we'll create a message header called q and tell the raw operation to include that message header as a URL parameter:

```
from("direct:queryExample")
  .setHeader("q", "SELECT Id, LastName FROM Contact")
  .to("salesforce:raw?
format=JSON&rawMethod=GET&rawQueryParameters=q&rawPath=/services/data/v51.0/query")
  // deserialize JSON results or handle in some other way
```

120.17.2. SOBJect example

In this example, we'll pass a Contact the REST API in a **create** operation. Since the **raw** operation does not perform any serialization, we make sure to pass XML in the message body

```
from("direct:createAContact")
  .setBody(constant("<Contact><LastName>TestLast</LastName></Contact>"))
  .to("salesforce:raw?
format=XML&rawMethod=POST&rawPath=/services/data/v51.0/subjects/Contact")
```

The response is:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Result>
  <id>0034x00000RnV6zAAF</id>
  <success>true</success>
</Result>
```

120.18. USING COMPOSITE SUBJECT COLLECTIONS

The SOBJect Collections API executes actions on multiple records in one request. Use sObject Collections to reduce the number of round-trips between the client and server. The entire request counts as a single call toward your API limits. This resource is available in API version 42.0 and later.

SObject records (aka DTOs) supplied to these operations must be instances of subclasses of **AbstractDescribedObjectBase**. See the Maven Plugin section for information on generating these DTO classes. These operations serialize supplied DTOs to JSON.

120.18.1. compositeRetrieveSObjectCollections

Retrieve one or more records of the same object type.

Parameter	Type	Description	Default	Required
ids	List of String or comma-separated string	A list of one or more IDs of the objects to return. All IDs must belong to the same object type.		x
fields	List of String or comma-separated string	A list of fields to include in the response. The field names you specify must be valid, and you must have read-level permissions to each field.		x
sObjectName	String	Type of SObject, e.g. Account		x
sObjectClass	String	Fully-qualified class name of DTO class to use for deserializing the response.		Required if sObjectName parameter does not resolve to a class that exists in the package specified by the package option.

120.18.2. compositeCreateObjectCollections

Add up to 200 records, returning a list of SaveSObjectResult objects. Mixed SObject types is supported.

Parameter	Type	Description	Default	Required
request body	List of SObject	A list of SObjects to create		x

Parameter	Type	Description	Default	Required
allOrNone	boolean	Indicates whether to roll back the entire request when the creation of any object fails (true) or to continue with the independent creation of other objects in the request.	false	

120.18.3. compositeUpdateObjectCollections

Update up to 200 records, returning a list of SaveObjectResult objects. Mixed SObject types is supported.

Parameter	Type	Description	Default	Required
request body	List of SObject	A list of SObjects to update		x
allOrNone	boolean	Indicates whether to roll back the entire request when the update of any object fails (true) or to continue with the independent update of other objects in the request.	false	

120.18.4. compositeUpsertObjectCollections

Create or update (upsert) up to 200 records based on an external ID field, returning a list of UpsertObjectResult objects. Mixed SObject types is not supported.

Parameter	Type	Description	Default	Required
request body	List of SObject	A list of SObjects to upsert		x
allOrNone	boolean	Indicates whether to roll back the entire request when the upsert of any object fails (true) or to continue with the independent upsert of other objects in the request.	false	
sObjectName	String	Type of SObject, e.g. Account		x
sObjectIdName	String	Name of External ID field		x

120.18.5. compositeDeleteObjectCollections

Delete up to 200 records, returning a list of DeleteObjectResult objects. Mixed SObject types is supported.

Parameter	Type	Description	Default	Required
sObjectIds or request body	List of String or comma-separated string	A list of up to 200 IDs of objects to be deleted.		x
allOrNone	boolean	Indicates whether to roll back the entire request when the deletion of any object fails (true) or to continue with the independent deletion of other objects in the request.	false	

120.19. SENDING NULL VALUES TO SALESFORCE

By default, SObject fields with null values are not sent to salesforce. In order to send null values to salesforce, use the **fieldsToNull** property, as follows:

```
accountSObject.getFieldsToNull().add("Site");
```

120.20. GENERATING SOQL QUERY STRINGS

org.apache.camel.component.salesforce.api.utils.QueryHelper contains helper methods to generate SOQL queries. For instance to fetch all custom fields from *Account* SObject you can simply generate the SOQL SELECT by invoking:

```
String allCustomFieldsQuery = QueryHelper.queryToFetchFilteredFieldsOf(new Account(),
    SObjectField::isCustom);
```

120.21. CAMEL SALESFORCE MAVEN PLUGIN

This Maven plugin generates DTOs for the Camel.

For obvious security reasons it is recommended that the clientId, clientSecret, userName and password fields be not set in the pom.xml. The plugin should be configured for the rest of the properties, and can be executed using the following command:

```
mvn camel-salesforce:generate -DcamelSalesforce.clientId=<clientId> -
DcamelSalesforce.clientSecret=<clientsecret> \
-DcamelSalesforce.userName=<username> -DcamelSalesforce.password=<password>
```

The generated DTOs use Jackson annotations. All Salesforce field types are supported. Date and time fields are mapped to **java.time.ZonedDateTime** by default, and picklist fields are mapped to generated Java Enumerations.

Please refer to [README.md](#) for details on how to generate the DTO.

120.22. SPRING BOOT AUTO-CONFIGURATION

The component supports 91 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.salesforce.all-or-none</code>	Composite API option to indicate to rollback all records if any are not successful.	false	Boolean
<code>camel.component.salesforce.apex-method</code>	APEX method name.		String
<code>camel.component.salesforce.apex-query-params</code>	Query params for APEX method.		Map
<code>camel.component.salesforce.apex-url</code>	APEX method URL.		String
<code>camel.component.salesforce.api-version</code>	Salesforce API version.	53.0	String
<code>camel.component.salesforce.authentication-type</code>	Explicit authentication method to be used, one of USERNAME_PASSWORD, REFRESH_TOKEN or JWT. Salesforce component can auto-determine the authentication method to use from the properties set, set this property to eliminate any ambiguity.		AuthenticationType
<code>camel.component.salesforce.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.salesforce.backoff-increment</code>	Backoff interval increment for Streaming connection restart attempts for failures beyond CometD auto-reconnect. The option is a long type.	1000	Long
<code>camel.component.salesforce.batch-id</code>	Bulk API Batch ID.		String

Name	Description	Default	Type
camel.component.salesforce.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.salesforce.client-id	OAuth Consumer Key of the connected app configured in the Salesforce instance setup. Typically a connected app needs to be configured but one can be provided by installing a package.		String
camel.component.salesforce.client-secret	OAuth Consumer Secret of the connected app configured in the Salesforce instance setup.		String
camel.component.salesforce.composite-method	Composite (raw) method.		String
camel.component.salesforce.config	Global endpoint configuration - use to set values that are common to all endpoints. The option is a org.apache.camel.component.salesforce.SalesforceEndpointConfig type.		SalesforceEndpointConfig
camel.component.salesforce.content-type	Bulk API content type, one of XML, CSV, ZIP_XML, ZIP_CSV.		ContentType
camel.component.salesforce.default-replay-id	Default replayId setting if no value is found in initialReplayIdMap.	-1	Long
camel.component.salesforce.enabled	Whether to enable auto configuration of the salesforce component. This is enabled by default.		Boolean
camel.component.salesforce.fall-back-replay-id	ReplayId to fall back to after an Invalid Replay Id response.	-1	Long
camel.component.salesforce.format	Payload format to use for Salesforce API calls, either JSON or XML, defaults to JSON. As of Camel 3.12, this option only applies to the Raw operation.		PayloadFormat

Name	Description	Default	Type
<code>camel.component.salesforce.http-client</code>	Custom Jetty Http Client to use to connect to Salesforce. The option is a <code>org.apache.camel.component.salesforce.SalesforceHttpClient</code> type.		<code>SalesforceHttpClient</code>
<code>camel.component.salesforce.http-client-connection-timeout</code>	Connection timeout used by the <code>HttpClient</code> when connecting to the Salesforce server.	60000	Long
<code>camel.component.salesforce.http-client-idle-timeout</code>	Timeout used by the <code>HttpClient</code> when waiting for response from the Salesforce server.	10000	Long
<code>camel.component.salesforce.http-client-properties</code>	Used to set any properties that can be configured on the underlying HTTP client. Have a look at properties of <code>SalesforceHttpClient</code> and the Jetty <code>HttpClient</code> for all available options.		Map
<code>camel.component.salesforce.http-max-content-length</code>	Max content length of an HTTP response.		Integer
<code>camel.component.salesforce.http-proxy-auth-uri</code>	Used in authentication against the HTTP proxy server, needs to match the URI of the proxy server in order for the <code>httpProxyUsername</code> and <code>httpProxyPassword</code> to be used for authentication.		String
<code>camel.component.salesforce.http-proxy-excluded-addresses</code>	A list of addresses for which HTTP proxy server should not be used.		Set
<code>camel.component.salesforce.http-proxy-host</code>	Hostname of the HTTP proxy server to use.		String
<code>camel.component.salesforce.http-proxy-included-addresses</code>	A list of addresses for which HTTP proxy server should be used.		Set
<code>camel.component.salesforce.http-proxy-password</code>	Password to use to authenticate against the HTTP proxy server.		String

Name	Description	Default	Type
<code>camel.component.salesforce.http-proxy-port</code>	Port number of the HTTP proxy server to use.		Integer
<code>camel.component.salesforce.http-proxy-realm</code>	Realm of the proxy server, used in preemptive Basic/Digest authentication methods against the HTTP proxy server.		String
<code>camel.component.salesforce.http-proxy-secure</code>	If set to false disables the use of TLS when accessing the HTTP proxy.	true	Boolean
<code>camel.component.salesforce.http-proxy-socks4</code>	If set to true the configures the HTTP proxy to use as a SOCKS4 proxy.	false	Boolean
<code>camel.component.salesforce.http-proxy-use-digest-auth</code>	If set to true Digest authentication will be used when authenticating to the HTTP proxy, otherwise Basic authorization method will be used.	false	Boolean
<code>camel.component.salesforce.http-proxy-username</code>	Username to use to authenticate against the HTTP proxy server.		String
<code>camel.component.salesforce.http-request-buffer-size</code>	HTTP request buffer size. May need to be increased for large SOQL queries.	8192	Integer
<code>camel.component.salesforce.include-details</code>	Include details in Salesforce Analytics report, defaults to false.		Boolean
<code>camel.component.salesforce.initial-replay-id-map</code>	Replay IDs to start from per channel name.		Map
<code>camel.component.salesforce.instance-id</code>	Salesforce Analytics report execution instance ID.		String
<code>camel.component.salesforce.instance-url</code>	URL of the Salesforce instance used after authentication, by default received from Salesforce on successful authentication.		String
<code>camel.component.salesforce.job-id</code>	Bulk API Job ID.		String

Name	Description	Default	Type
camel.component.salesforce.jwt-audience	Value to use for the Audience claim (aud) when using OAuth JWT flow. If not set, the login URL will be used, which is appropriate in most cases.		String
camel.component.salesforce.keystore	KeyStore parameters to use in OAuth JWT flow. The KeyStore should contain only one entry with private key and certificate. Salesforce does not verify the certificate chain, so this can easily be a selfsigned certificate. Make sure that you upload the certificate to the corresponding connected app. The option is a <code>org.apache.camel.support.jsse.KeyStoreParameters</code> type.		KeyStoreParameters
camel.component.salesforce.lazy-login	If set to true prevents the component from authenticating to Salesforce with the start of the component. You would generally set this to the (default) false and authenticate early and be immediately aware of any authentication issues.	false	Boolean
camel.component.salesforce.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.salesforce.limit	Limit on number of returned records. Applicable to some of the API, check the Salesforce documentation.		Integer
camel.component.salesforce.locator	Locator provided by salesforce Bulk 2.0 API for use in getting results for a Query job.		String
camel.component.salesforce.login-config	All authentication configuration in one nested bean, all properties set there can be set directly on the component as well. The option is a <code>org.apache.camel.component.salesforce.SalesforceLoginConfig</code> type.		SalesforceLoginConfig

Name	Description	Default	Type
camel.component.salesforce.login-url	URL of the Salesforce instance used for authentication, by default set to .		String
camel.component.salesforce.long-polling-transport-properties	Used to set any properties that can be configured on the LongPollingTransport used by the BayeuxClient (CometD) used by the streaming api.		Map
camel.component.salesforce.max-backoff	Maximum backoff interval for Streaming connection restart attempts for failures beyond CometD auto-reconnect. The option is a long type.	30000	Long
camel.component.salesforce.max-records	The maximum number of records to retrieve per set of results for a Bulk 2.0 Query. The request is still subject to the size limits. If you are working with a very large number of query results, you may experience a timeout before receiving all the data from Salesforce. To prevent a timeout, specify the maximum number of records your client is expecting to receive in the maxRecords parameter. This splits the results into smaller sets with this value as the maximum size.		Integer
camel.component.salesforce.not-found-behaviour	Sets the behaviour of 404 not found status received from Salesforce API. Should the body be set to NULL NotFoundBehaviour#NULL or should a exception be signaled on the exchange NotFoundBehaviour#EXCEPTION - the default.		NotFoundBehaviour
camel.component.salesforce.notify-for-fields	Notify for fields, options are ALL, REFERENCED, SELECT, WHERE.		NotifyForFieldsEnum
camel.component.salesforce.notify-for-operation-create	Notify for create operation, defaults to false (API version = 29.0).		Boolean
camel.component.salesforce.notify-for-operation-delete	Notify for delete operation, defaults to false (API version = 29.0).		Boolean
camel.component.salesforce.notify-for-operation-undelete	Notify for un-delete operation, defaults to false (API version = 29.0).		Boolean

Name	Description	Default	Type
<code>camel.component.salesforce.notify-for-operation-update</code>	Notify for update operation, defaults to false (API version = 29.0).		Boolean
<code>camel.component.salesforce.notify-for-operations</code>	Notify for operations, options are ALL, CREATE, EXTENDED, UPDATE (API version 29.0).		NotifyForOperationsEnum
<code>camel.component.salesforce.object-mapper</code>	Custom Jackson ObjectMapper to use when serializing/deserializing Salesforce objects. The option is a <code>com.fasterxml.jackson.databind.ObjectMapper</code> type.		ObjectMapper
<code>camel.component.salesforce.packages</code>	In what packages are the generated DTO classes. Typically the classes would be generated using <code>camel-salesforce-maven-plugin</code> . Set it if using the generated DTOs to gain the benefit of using short SObject names in parameters/header values. Multiple packages can be separated by comma.		String
<code>camel.component.salesforce.password</code>	Password used in OAuth flow to gain access to access token. It's easy to get started with password OAuth flow, but in general one should avoid it as it is deemed less secure than other flows. Make sure that you append security token to the end of the password if using one.		String
<code>camel.component.salesforce.pk-chunking</code>	Use PK Chunking. Only for use in original Bulk API. Bulk 2.0 API performs PK chunking automatically, if necessary.		Boolean
<code>camel.component.salesforce.pk-chunking-chunk-size</code>	Chunk size for use with PK Chunking. If unspecified, salesforce default is 100,000. Maximum size is 250,000.		Integer
<code>camel.component.salesforce.pk-chunking-parent</code>	Specifies the parent object when you're enabling PK chunking for queries on sharing objects. The chunks are based on the parent object's records rather than the sharing object's records. For example, when querying on AccountShare, specify Account as the parent object. PK chunking is supported for sharing objects as long as the parent object is supported.		String
<code>camel.component.salesforce.pk-chunking-start-row</code>	Specifies the 15-character or 18-character record ID to be used as the lower boundary for the first chunk. Use this parameter to specify a starting ID when restarting a job that failed between batches.		String

Name	Description	Default	Type
<code>camel.component.salesforce.query-locator</code>	Query Locator provided by salesforce for use when a query results in more records than can be retrieved in a single call. Use this value in a subsequent call to retrieve additional records.		String
<code>camel.component.salesforce.raw-http-headers</code>	Comma separated list of message headers to include as HTTP parameters for Raw operation.		String
<code>camel.component.salesforce.raw-method</code>	HTTP method to use for the Raw operation.		String
<code>camel.component.salesforce.raw-path</code>	The portion of the endpoint URL after the domain name. E.g., '/services/data/v52.0/subjects/Account/'.		String
<code>camel.component.salesforce.raw-payload</code>	Use raw payload String for request and response (either JSON or XML depending on format), instead of DTOs, false by default.	false	Boolean
<code>camel.component.salesforce.raw-query-parameters</code>	Comma separated list of message headers to include as query parameters for Raw operation. Do not url-encode values as this will be done automatically.		String
<code>camel.component.salesforce.refresh-token</code>	Refresh token already obtained in the refresh token OAuth flow. One needs to setup a web application and configure a callback URL to receive the refresh token, or configure using the builtin callback at and then retrieve the refresh_token from the URL at the end of the flow. Note that in development organizations Salesforce allows hosting the callback web application at localhost.		String
<code>camel.component.salesforce.report-id</code>	Salesforce1 Analytics report Id.		String
<code>camel.component.salesforce.report-metadata</code>	Salesforce1 Analytics report metadata for filtering. The option is a <code>org.apache.camel.component.salesforce.api.dto.analytics.reports.ReportMetadata</code> type.		ReportMetadata
<code>camel.component.salesforce.result-id</code>	Bulk API Result ID.		String

Name	Description	Default	Type
<code>camel.component.salesforce.s-object-blob-field-name</code>	SObject blob field name.		String
<code>camel.component.salesforce.s-object-class</code>	Fully qualified SObject class name, usually generated using camel-salesforce-maven-plugin.		String
<code>camel.component.salesforce.s-object-fields</code>	SObject fields to retrieve.		String
<code>camel.component.salesforce.s-object-id</code>	SObject ID if required by API.		String
<code>camel.component.salesforce.s-object-id-name</code>	SObject external ID field name.		String
<code>camel.component.salesforce.s-object-id-value</code>	SObject external ID field value.		String
<code>camel.component.salesforce.s-object-name</code>	SObject name if required or supported by API.		String
<code>camel.component.salesforce.s-object-query</code>	Salesforce SOQL query string.		String
<code>camel.component.salesforce.s-object-search</code>	Salesforce SOSL search string.		String
<code>camel.component.salesforce.ssl-context-parameters</code>	SSL parameters to use, see <code>SSLContextParameters</code> class for all available options. The option is a <code>org.apache.camel.support.jsse.SSLContextParameters</code> type.		<code>SSLContextParameters</code>
<code>camel.component.salesforce.update-topic</code>	Whether to update an existing Push Topic when using the Streaming API, defaults to false.	false	Boolean

Name	Description	Default	Type
<code>camel.component.salesforce.use-global-ssl-context-parameters</code>	Enable usage of global SSL context parameters.	false	Boolean
<code>camel.component.salesforce.username</code>	Username used in OAuth flow to gain access to access token. It's easy to get started with password OAuth flow, but in general one should avoid it as it is deemed less secure than other flows.		String
<code>camel.component.salesforce.worker-pool-max-size</code>	Maximum size of the thread pool used to handle HTTP responses.	20	Integer
<code>camel.component.salesforce.worker-pool-size</code>	Size of the thread pool used to handle HTTP responses.	10	Integer

CHAPTER 121. SAP COMPONENT

The SAP component is a package consisting of ten different SAP components. There are remote function call (RFC) components that support the sRFC, tRFC, and qRFC protocols and there are IDoc components that facilitate communication using messages in IDoc format. The component uses the SAP Java Connector (SAP JCo) library to facilitate bidirectional communication with SAP and the SAP IDoc library to transmit the documents in the Intermediate Document (IDoc) format.

121.1. DEPENDENCIES

Add the following dependency to your **pom.xml** for this component:

```
<dependency>
  <groupId>org.fusesource</groupId>
  <artifactId>camel-sap-starter</artifactId>
  <version>4.10.3.redhat-00025</version>
</dependency>
```

121.1.1. Additional platform restrictions for the SAP component

Because the SAP component depends on the third-party JCo 3 and IDoc 3 libraries, it can only be installed on the platforms that these libraries support.

121.1.2. SAP JCo and SAP IDoc libraries

A prerequisite for using the SAP component is that the SAP Java Connector (SAP JCo) libraries and the SAP IDoc library are installed into the **lib/** directory of the Java runtime. You must make sure that you download the appropriate set of SAP libraries for your target operating system from the SAP Service Marketplace.

The names of the library files vary depending on the target operating system, as shown below.

Table 121.1. Required SAP Libraries

SAP Component	Linux and UNIX	Windows
SAP JCo 3	sapjco3.jar libsapjco3.so	sapjco3.jar sapjco3.dll
SAP IDoc	sapidoc3.jar	sapidoc3.jar

121.1.2.1. Installing SAP JCo and SAP IDoc libraries

The instructions to install the SAP JCo and SAP IDoc libraries are as follows. The instructions are different for running on OpenShift Container Platform and for running locally. It is recommended to create two Maven profiles such as **openshift** and **local**.

Procedure

1. Install the artifacts into the local maven repository. Navigate to your project's **lib** directory and save the SAP libraries.
2. Define the dependencies in the **pom.xml**.

```
<dependency>
  <groupId>com.sap.conn.jco</groupId>
  <artifactId>sapjco3</artifactId>
  <version>${sapjco3-version}</version>
  <scope>system</scope>
  <systemPath>${basedir}/lib/sapjco3.jar</systemPath>
</dependency>

<dependency>
  <groupId>com.sap.conn.idoc</groupId>
  <artifactId>sapidoc3</artifactId>
  <version>${sapidoc3-version}</version>
  <scope>system</scope>
  <systemPath>${basedir}/lib/sapidoc3.jar</systemPath>
</dependency>
```

3. Define the **spring-boot-maven-plugin**.

For local profile

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <version>${spring.boot-version}</version>
  <executions>
    <execution>
      <goals>
        <goal>repackage</goal>
      </goals>
    </execution>
  </executions>
  <configuration>
    <additionalClasspathElements>${basedir}/lib</additionalClasspathElements>
  </configuration>
</plugin>
```

For openshift profile

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <version>${spring.boot-version}</version>
  <executions>
    <execution>
      <goals>
        <goal>repackage</goal>
      </goals>
    </execution>
  </executions>
  <configuration>
```

```

    <!-- <includeSystemScope>false</includeSystemScope> -->
    <layout>ZIP</layout>
    <excludeGroupIds>com.sap.conn.jco,com.sap.conn.idoc</excludeGroupIds>
  </configuration>
</plugin>

```

4. Define **openshift-maven-plugin**.

```

<plugin>
  <groupId>org.eclipse.jkube</groupId>
  <artifactId>openshift-maven-plugin</artifactId>
  <version>1.16.2.redhat-00013</version>
  <configuration>
    <images>
      <image>
        <name>${project.build.finalName}</name>
        <alias>${project.build.finalName}</alias>
        <build>
          <from>${jkube.generator.from}</from>
          <assembly>
            <targetDir>/deployments</targetDir>
            <layers>
              <layer>
                <id>lib</id>
                <fileSets>
                  <fileSet>
                    <directory>lib</directory>
                    <outputDirectory>data</outputDirectory>
                  </fileSet>
                </fileSets>
              </layer>
            </layers>
          </assembly>
        </build>
      </image>
    </images>
    <resources>
      <env>
        <JAVA_OPTS_APPEND>-
Dloader.path=/deployments/data</JAVA_OPTS_APPEND>
      </env>
    </resources>
  </configuration>
  <executions>
    <execution>
      <goals>
        <goal>resource</goal>
        <goal>build</goal>
        <goal>apply</goal>
      </goals>
    </execution>
  </executions>
</plugin>

```

5. Define **openshift-maven-plugin** for the **openshift** profile.

```
<jkube.enricher.jkube-service.port>8080</jkube.enricher.jkube-service.port>
```

If you want the **openshift-maven-plugin** to use **Deployemnt** instead of **DeploymentConfig**, follow extra steps.

6. Add **jkube.build.switchToDeployment** property in the **pom.xml** file.

```
<jkube.build.switchToDeployment>true</jkube.build.switchToDeployment>
<jkube.enricher.jkube-service.port>8080</jkube.enricher.jkube-service.port>
</properties>
```

7. Define the filename in the **openshift** profile.

```
<profile>
  <id>openshift</id>
  <build>
    <finalName>${project.artifactId}</finalName>
    <defaultGoal>install</defaultGoal>
```

If you want to switch **application.properties** between **openshift** and **local** profiles, use the following steps.

8. Create the **application-openshift.properties** and the **application-local.properties** files under the same directory with **application.properties**. The **application-openshift.properties** file is used by **Openshift** profile and the **application-local.properties** file is used by **local** profile.
9. Edit the **application.properties** file as follows.

```
spring.profiles.active=@spring.profiles.active@
```

121.2. URI FORMAT

There are two different kinds of endpoint provided by the SAP component: the Remote Function Call (RFC) endpoints, and the Intermediate Document (IDoc) endpoints.

The URI formats for the RFC endpoints are as follows:

```
sap-srfc-destination:destinationName:rfcName
sap-trfc-destination:destinationName:rfcName
sap-qrfc-destination:destinationName:queueName:rfcName
sap-srfc-server:serverName:rfcName[?options]
sap-trfc-server:serverName:rfcName[?options]
```

The URI formats for the IDoc endpoints are as follows:

```
sap-idoc-
destination:destinationName:idocType[:idocTypeExtension[:systemRelease[:applicationRelease]]]
sap-idoclist-
destination:destinationName:idocType[:idocTypeExtension[:systemRelease[:applicationRelease]]]
sap-qidoc-
destination:destinationName:queueName:idocType[:idocTypeExtension[:systemRelease[:applicationRelease]]]
sap-qidoclist-
```



```
destination:destinationName:queueName:docType[:docTypeExtension[:systemRelease[:applicationRelease]]]
sap-idoclist-server:serverName:docType[:docTypeExtension[:systemRelease[:applicationRelease]]]
[?options]
```

The URI formats prefixed by `sap-endpointKind-destination` are used to define destination endpoints (in other words, Camel producer endpoints) and `destinationName` is the name of a specific outbound connection to an SAP instance. Outbound connections are named and configured at the component level.

The URI formats prefixed by `sap-endpointKind-server` are used to define server endpoints (in other words, Camel consumer endpoints) and `serverName` is the name of a specific inbound connection from an SAP instance. Inbound connections are named and configured at the component level.

The other components of an RFC endpoint URI are as follows:

rfcName

(Required) In a destination endpoint URI, is the name of the RFC invoked by the endpoint in the connected SAP instance. In a server endpoint URI, is the name of the RFC handled by the endpoint when invoked from the connected SAP instance.

queueName

Specifies the queue this endpoint sends an SAP request to.

The other components of an IDoc endpoint URI are as follows:

docType

(Required) Specifies the Basic IDoc Type of an IDoc produced by this endpoint.

docTypeExtension

Specifies the IDoc Type Extension, if any, of an IDoc produced by this endpoint.

systemRelease

Specifies the associated SAP Basis Release, if any, of an IDoc produced by this endpoint.

applicationRelease

Specifies the associated Application Release, if any, of an IDoc produced by this endpoint.

queueName

Specifies the queue this endpoint sends an SAP request to.

121.2.1. Options for RFC destination endpoints

The RFC destination endpoints (**sap-srfc-destination**, **sap-trfc-destination**, and **sap-qrfc-destination**) support the following URI options:

Name	Default	Description
stateful	false	If true , specifies that this endpoint initiates an SAP stateful session
transacted	false	If true , specifies that this endpoint initiates an SAP transaction

121.2.2. Options for RFC server endpoints

The SAP RFC server endpoints (**sap-srfc-server** and **sap-trfc-server**) support the following URI options:

Name	Default	Description
stateful	false	If true , specifies that this endpoint initiates an SAP stateful session.
propagateExceptions	false	(sap-trfc-server endpoint only) If true , specifies that this endpoint propagates exceptions back to the caller in SAP, instead of the exchange's exception handler.

121.2.3. Options for the IDoc List Server endpoint

The SAP IDoc List Server endpoint (**sap-idoclist-server**) supports the following URI options:

Name	Default	Description
stateful	false	If true , specifies that this endpoint initiates an SAP stateful session.
propagateExceptions	false	If true , specifies that this endpoint propagates exceptions back to the caller in SAP, instead of the exchange's exception handler.

121.2.4. Summary of the RFC and IDoc endpoints

The SAP component package provides the following RFC and IDoc endpoints:

sap-srfc-destination

Camel SAP Synchronous Remote Function Call Destination Camel component. This endpoint should be used in cases where Camel routes require synchronous delivery of requests to and responses from an SAP system.

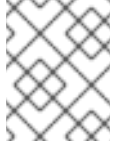


NOTE

The sRFC protocol used by this component delivers requests and responses to and from an SAP system with **best effort**. In case of a communication error while sending a request, the completion status of a remote function call in the receiving SAP system remains **in doubt**.

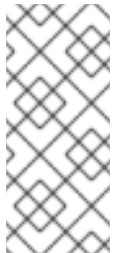
sap-trfc-destination

Camel SAP Transactional Remote Function Call Destination Camel component. This endpoint should be used in cases where requests must be delivered to the receiving SAP system **at most once**. To accomplish this, the component generates a transaction ID, **tid**, which accompanies every request sent through the component in a route's exchange. The receiving SAP system records the **tid** accompanying a request before delivering the request; if the SAP system receives the request again with the same **tid** it will not deliver the request. Thus if a route encounters a communication error when sending a request through an endpoint of this component, it can retry sending the request within the same exchange knowing it will be delivered and executed only once.



NOTE

The tRFC protocol used by this component is asynchronous and does not return a response. Thus the endpoints of this component do not return a response message.



NOTE

This component does not guarantee the order of a series of requests through its endpoints, and the delivery and execution order of these requests may differ on the receiving SAP system due to communication errors and resends of a request. For guaranteed delivery order, please see the Camel SAP Queued Remote Function Call Destination Camel component.

sap-qrfc-destination

Camel SAP Queued Remote Function Call Destination Camel component. This component extends the capabilities of the Transactional Remote Function Call Destination camel component by adding **in order** delivery guarantees to the delivery of requests through its endpoints. This endpoint should be used in cases where a series of requests depend on each other and must be delivered to the receiving SAP system **at most once** and **in order**. The component accomplishes the **at most once** delivery guarantees using the same mechanisms as the Camel SAP Transactional Remote Function Call Destination Camel component. The ordering guarantee is accomplished by serializing the requests in the order they are received by the SAP system to an *inbound queue*. Inbound queues are processed by the *QIN scheduler* within SAP. When the inbound queue is **activated**, the QIN Scheduler will execute the queue requests in order.



NOTE

The qRFC protocol used by this component is asynchronous and does not return a response. Thus the endpoints of this component do not return a response message.

sap-srfc-server

Camel SAP Synchronous Remote Function Call Server Camel component. This component and its endpoints should be used in cases where a Camel route is required to synchronously handle requests from and responses to an SAP system.

sap-trfc-server

Camel SAP Transactional Remote Function Call Server Camel component. This endpoint should be used in cases where the sending SAP system requires **at most once** delivery of its requests to a Camel route. To accomplish this, the sending SAP system generates a transaction ID, **tid**, which accompanies every request it sends to the component's endpoints. The sending SAP system will first check with the component whether a given **tid** has been received by it before sending a series of requests associated with the **tid**. The component will check the list of received **tids** it maintains,

record the sent **tid** if it is not in that list, and then respond to the sending SAP system, indicating whether or not the **tid** has already been recorded. The sending SAP system will only then send the series of requests, if the **tid** has not been previously recorded. This enables a sending SAP system to reliably send a series of requests once to a camel route.

sap-idoc-destination

Camel SAP IDoc Destination Camel component. This endpoint should be used in cases where a Camel route sends a list of Intermediate Documents (IDocs) to an SAP system.

sap-idoclist-destination

Camel SAP IDoc List Destination Camel component. This endpoint should be used in cases where a Camel route sends a list of Intermediate documents (IDocs) list to an SAP system.

sap-qidoc-destination

Camel SAP Queued IDoc Destination Camel component. This component and its endpoints should be used in cases where a Camel route is required to send a list of Intermediate documents (IDocs) to an SAP system in order.

sap-qidoclist-destination

Camel SAP Queued IDoc List Destination Camel component. This component and its endpoints are used in cases where a camel route sends the Intermediate documents (IDocs) list to an SAP system in order.

sap-idoclist-server

Camel SAP IDoc List Server Camel component. This endpoint should be used in cases where a sending SAP system requires delivery of Intermediate Document lists to a Camel route. This component uses the tRFC protocol to communicate with SAP as described in the **sap-trfc-server-standalone** quick start.

121.2.5. SAP RFC destination endpoint

An RFC destination endpoint supports outbound communication to SAP, which enable these endpoints to make RFC calls out to ABAP function modules in SAP. An RFC destination endpoint is configured to make an RFC call to a specific ABAP function over a specific connection to an SAP instance. An RFC destination is a logical designation for an outbound connection and has a unique name. An RFC destination is specified by a set of connection parameters called *destination data*.

An RFC destination endpoint will extract an RFC request from the input message of the IN-OUT exchanges it receives and dispatch that request in a function call to SAP. The response from the function call will be returned in the output message of the exchange. Since SAP RFC destination endpoints only support outbound communication, an RFC destination endpoint only supports the creation of producers.

121.2.6. SAP RFC server endpoint

An RFC server endpoint supports inbound communication from SAP, which enables ABAP applications in SAP to make RFC calls into server endpoints. An ABAP application interacts with an RFC server endpoint as if it were a remote function module. An RFC server endpoint is configured to receive an RFC call to a specific RFC function over a specific connection from an SAP instance. An RFC server is a logical designation for an inbound connection and has a unique name. An RFC server is specified by a set of connection parameters called *server data*.

An RFC server endpoint will handle an incoming RFC request and dispatch it as the input message of an IN-OUT exchange. The output message of the exchange will be returned as the response of the RFC call. Since SAP RFC server endpoints only support inbound communication, an RFC server endpoint only supports the creation of consumers.

121.2.7. SAP IDoc and IDoc list destination endpoints

An IDoc destination endpoint supports outbound communication to SAP, which can then perform further processing on the IDoc message. An IDoc document represents a business transaction, which can easily be exchanged with non-SAP systems. An IDoc destination is specified by a set of connection parameters called *destination data*.

An IDoc list destination endpoint is similar to an IDoc destination endpoint, except that the messages it handles consist of a **list** of IDoc documents.

121.2.8. SAP IDoc list server endpoint

An IDoc list server endpoint supports inbound communication from SAP, enabling a Camel route to receive a list of IDoc documents from an SAP system. An IDoc list server is specified by a set of connection parameters called *server data*.

121.2.9. Metadata repositories

A metadata repository is used to store the following kinds of metadata:

Interface descriptions of function modules

This metadata is used by the JCo and ABAP runtimes to check RFC calls to ensure the type-safe transfer of data between communication partners before dispatching those calls. A repository is populated with repository data. Repository data is a map of named function templates. A function template contains the metadata describing all the parameters and their typing information passed to and from a function module and has the unique name of the function module it describes.

IDoc type descriptions

This metadata is used by the IDoc runtime to ensure that the IDoc documents are correctly formatted before being sent to a communication partner. A basic IDoc type consists of a name, a list of permitted segments, and a description of the hierarchical relationship between the segments. Some additional constraints can be imposed on the segments: a segment can be mandatory or optional; and it is possible to specify a minimum/maximum range for each segment (defining the number of allowed repetitions of that segment).

SAP destination and server endpoints thus require access to a repository, in order to send and receive RFC calls and in order to send and receive IDoc documents. For RFC calls, the metadata for all function modules invoked and handled by the endpoints must reside within the repository; and for IDoc endpoints, the metadata for all IDoc types and IDoc type extensions handled by the endpoints must reside within the repository. The location of the repository used by a destination and server endpoint is specified in the destination data and the server data of their respective connections.

In the case of an SAP destination endpoint, the repository it uses typically resides in an SAP system and it defaults to the SAP system it is connected to. This default requires no explicit configuration in the destination data. Furthermore, the metadata for the remote function call that a destination endpoint makes will already exist in a repository for any existing function module that it calls. The metadata for calls made by destination endpoints thus require no configuration in the SAP component.

On the other hand, the metadata for function calls handled by server endpoints do not typically reside in the repository of an SAP system and must instead be provided by a repository residing in the SAP component. The SAP component maintains a map of named metadata repositories. The name of a repository corresponds to the name of the server to which it provides metadata.

121.3. CONFIGURATION

The SAP component maintains three maps to store destination data, server data, and repository data. The *destination data store* and the *server data store* are configured on a special configuration object, **SapConnectionConfiguration**, which automatically gets injected into the SAP component (in the context of Blueprint XML configuration or Spring XML configuration files). The *repository data store* must be configured directly on the relevant SAP component.

121.3.1. Configuration Overview

The SAP component maintains three maps to store destination data, server data, and repository data. The component's property, **destinationDataStore**, stores destination data keyed by destination name, the property, **serverDataStore**, stores server data keyed by server name and the property, **repositoryDataStore**, stores repository data keyed by repository name. These configurations must be passed to the component during its initialization.

Example

The following example shows how to configure a sample destination data store and a sample server data store in a Blueprint XML file. The **sap-configuration** bean (of type **SapConnectionConfiguration**) will automatically be injected into any SAP component that is used in this XML file.

```
<?xml version="1.0" encoding="UTF-8"?>
<blueprint ... >
  ...
  <!-- Configures the Inbound and Outbound SAP Connections -->
  <bean id="sap-configuration"
    class="org.fusesource.camel.component.sap.SapConnectionConfiguration">
    <property name="destinationDataStore">
      <map>
        <entry key="quickstartDest" value-ref="quickstartDestinationData" />
      </map>
    </property>
    <property name="serverDataStore">
      <map>
        <entry key="quickstartServer" value-ref="quickstartServerData" />
      </map>
    </property>
  </bean>

  <!-- Configures an Outbound SAP Connection -->
  <!-- *** Please enter the connection property values for your environment *** -->
  <bean id="quickstartDestinationData"
    class="org.fusesource.camel.component.sap.model.rfc.impl.DestinationDataImpl">
    <property name="ashost" value="example.com" />
    <property name="sysnr" value="00" />
    <property name="client" value="000" />
    <property name="user" value="username" />
    <property name="passwd" value="passowrd" />
    <property name="lang" value="en" />
  </bean>

  <!-- Configures an Inbound SAP Connection -->
  <!-- *** Please enter the connection property values for your environment ** -->
  <bean id="quickstartServerData"
    class="org.fusesource.camel.component.sap.model.rfc.impl.ServerDataImpl">
    <property name="gwhost" value="example.com" />
    <property name="gwserv" value="3300" />
  </bean>
```

```

<!-- The following property values should not be changed -->
<property name="progid" value="QUICKSTART" />
<property name="repositoryDestination" value="quickstartDest" />
<property name="connectionCount" value="2" />
</bean>
</blueprint>

```

121.3.2. Destination Configuration

The configurations for destinations are maintained in the **destinationDataStore** property of the SAP component. Each entry in this map configures a distinct outbound connection to an SAP instance. The key for each entry is the name of the outbound connection and is used in the *destinationName* component of a destination endpoint URI as described in the URI format section.

The value for each entry is a destination data configuration object –

org.fusesource.camel.component.sap.model.rfc.impl.DestinationDataImpl – that specifies the configuration of an outbound SAP connection.

Sample destination configuration

The following Blueprint XML code shows how to configure a sample destination with the name, **quickstartDest**.

```

<?xml version="1.0" encoding="UTF-8"?>
<blueprint ... >
    ...
    <!-- Create interceptor to support tRFC processing -->
    <bean id="currentProcessorDefinitionInterceptor"
        class="org.fusesource.camel.component.sap.CurrentProcessorDefinitionInterceptStrategy" />

    <!-- Configures the Inbound and Outbound SAP Connections -->
    <bean id="sap-configuration"
        class="org.fusesource.camel.component.sap.SapConnectionConfiguration">
        <property name="destinationDataStore">
            <map>
                <entry key="quickstartDest" value-ref="quickstartDestinationData" />
            </map>
        </property>
    </bean>

    <!-- Configures an Outbound SAP Connection -->
    <!-- *** Please enter the connection property values for your environment *** -->
    <bean id="quickstartDestinationData"
        class="org.fusesource.camel.component.sap.model.rfc.impl.DestinationDataImpl">
        <property name="ashost" value="example.com" />
        <property name="sysnr" value="00" />
        <property name="client" value="000" />
        <property name="user" value="username" />
        <property name="passwd" value="password" />
        <property name="lang" value="en" />
    </bean>

</blueprint>

```

For example, after configuring the destination as shown in the preceding Blueprint XML file, you could invoke the **BAPI_FLCUST_GETLIST** remote function call on the **quickstartDest** destination using the following URI:

sap-srfc-destination:quickstartDest:BAPI_FLCUST_GETLIST

121.3.2.1. Interceptor for tRFC and qRFC destinations

The preceding sample destination configuration shows the instantiation of a **CurrentProcessorDefinitionInterceptStrategy** object. This object installs an interceptor in the Camel runtime, which enables the Camel SAP component to keep track of its position within a Camel route while it is handling RFC transactions.



IMPORTANT

This interceptor is critically important for transactional RFC destination endpoints (such as **sap-trfc-destination** and **sap-qrfc-destination**) and must be installed in the Camel runtime for outbound transactional RFC communication to be properly managed. The Destination RFC Transaction Handlers issues warnings into the Camel log if the strategy is not found at runtime. In this situation the Camel runtime will need to be re-provisioned and restarted to properly manage outbound transactional RFC communication.

121.3.2.2. Log on and authentication options

The following table lists the **log on and authentication** options for configuring a destination in the SAP destination data store:

Name	Default Value	Description
client		SAP client, mandatory log on parameter.
user		log on user, log on parameter for password based authentication.
aliasUser		log on user alias, can be used instead of log on user.
userId		User identity used for log on to the ABAP AS. Used by the JCo runtime, if the destination configuration uses SSO/assertion ticket, certificate, current user ,or SNC environment for authentication. The user ID is mandatory, if neither user nor user alias is set. This ID will never be sent to the SAP backend, it will be used by the JCo runtime locally.

passwd		log on password, log on parameter for password based authentication.
lang		log on language, if not defined, the default user language is used.
mysapso2		Use the specified SAP Cookie Version 2 as a log on ticket for SSO based authentication.
x509cert		Use the specified X509 certificate for certificate based authentication.
lcheck		Postpone the authentication until the first call - 1 (enable). Used in special cases only.
useSapGui		Use a visible, hidden, or do not use SAP GUI
codePage		Additional log on parameter to define the codepage used to convert the log on parameters. Used in special cases only.
getsso2		Order an SSO ticket after log on, the obtained ticket is available in the destination attributes.
denyInitialPassword		If set to 1 , using initial passwords will lead to an exception (default is 0).

121.3.2.3. Connection options

The following table lists the **connection** options for configuring a destination in the SAP destination data store:

Name	Default Value	Description
------	---------------	-------------

saprouter		SAP Router string for connection to systems behind a SAP Router. SAP Router string contains the chain of SAP Routers and theirs port numbers and has the form: (/H/<host>[/S/<port>])+ .
sysnr		System number of the SAP ABAP application server, mandatory for a direct connection.
ashost		SAP ABAP application server, mandatory for a direct connection.
mshost		SAP message server, mandatory property for a load balancing connection.
msserv		SAP message server port, optional property for a load balancing connection. In order to resolve the service names <code>sapmsXXX</code> a lookup in etc/services is performed by the network layer of the operating system. If using port numbers instead of symbolic service names, no lookups are performed and no additional entries are needed.
gwhost		Allows specifying a concrete gateway, which should be used for establishing the connection to an application server. If not specified, the gateway on the application server is used.
gwserv		Should be set, when using <code>gwhost</code> . Allows specifying the port used on that gateway. If not specified, the port of the gateway on the application server is used. In order to resolve the service names <code>sapgwXXX</code> a lookup in etc/services is performed by the network layer of the operating system. If using port numbers instead of symbolic service names, no lookups are performed and no additional entries are needed.

r3name		System ID of the SAP system, mandatory property for a load balancing connection.
group		Group of SAP application servers, mandatory property for a load balancing connection.
network	LAN	Set this value depending on the network quality between JCo and your target system to optimize performance. The valid values are LAN or WAN (which is relevant for fast serialization only). If you set the network configuration option to WAN , a slower but more efficient compression algorithm is used and the data is analyzed for further compression options. If you set the network configuration to LAN a very fast compression algorithm is used and data analysis is performed only at a very basic level. When you set the LAN option, the compression ratio is not as efficient but the network transfer time is considered to be less significant. The default setting is LAN .
serializationFormat	rowBased	The valid values are rowBased or columnBased . For fast serialization columnBased must be set. The default serialization setting is rowBased .

121.3.2.4. Connection pool options

The following table lists the **connection pool** options for configuring a destination in the SAP destination data store:

Name	Default Value	Description
------	---------------	-------------

peakLimit	0	Maximum number of active outbound connections that can be created for a destination simultaneously. A value of 0 allows an unlimited number of active connections. Otherwise, if the value is less than the value of jpoolCapacity , it will be automatically increased to this value. Default setting is the value of poolCapacity , or in case of poolCapacity not being specified as well, the default is 0 (unlimited).
poolCapacity	1	Maximum number of idle outbound connections kept open by the destination. A value of 0 has the effect that there is no connection pooling (default is 1).
expirationTime		Time in milliseconds after which a free connection held internally by the destination can be closed.
expirationPeriod		Period in milliseconds after which the destination checks the released connections for expiration.
maxGetTime		Maximum time in milliseconds to wait for a connection, if the maximum allowed number of connections has already been allocated by the application.

121.3.2.5. Secure network connection options

The following table lists the **secure network** options for configuring a destination in the SAP destination data store:

Name	Default Value	Description
sncMode		Secure network connection (SNC) mode, 0 (off) or 1 (on).
sncPartnername		SNC partner, for example: p:CN=R3, O=XYZ-INC, C=EN.

sncQop		SNC level of security: 1 to 9 .
sncMyname		Own SNC name. Overrides the environment settings.
sncLibrary		Path to the library that provides the SNC service.

121.3.2.6. Repository options

The following table lists the **repository** options for configuring a destination in the SAP destination data store:

Name	Default Value	Description
repositoryDest		Specifies the destination which is used as a repository.
repositoryUser		If a repository destination is not set, and this property is set, it is used as user for repository calls. This enables you to use a different user for repository lookups.
repositoryPasswd		The password for a repository user. Mandatory, if a repository user is used.
repositorySnc		(Optional) If SNC is used for this destination, it is possible to turn it off for repository connections, if this property is set to 0 . Default setting is the value of jco.client.snc_mode . For special cases only.

repositoryRoundtripOptimization		Enable the RFC_METADATA_GET API, which provides the repository data in one single round trip. 1 Activates use of RFC_METADATA_GET in ABAP System. 0 Deactivates RFC_METADATA_GET in ABAP System. If the property is not set, the destination initially does a remote call to check whether RFC_METADATA_GET is available. If it is available, the destination will use it. Note: If the repository is already initialized (for example, because it is used by some other destination), this property does not have any effect. Generally, this property is related to the ABAP System, and should have the same value on all destinations pointing to the same ABAP System. See note 1456826 for backend prerequisites.
--	--	---

121.3.2.7. Trace configuration options

The following table lists the **trace configuration** options for configuring a destination in the SAP destination data store:

Name	Default Value	Description
trace		Enable/disable RFC trace (0 or 1).
cpicTrace		Enable/disable CPIC trace [0..3].

121.3.3. Server Configuration

The configurations for servers are maintained in the **serverDataStore** property of the SAP component. Each entry in this map configures a distinct inbound connection from an SAP instance. The key for each entry is the name of the outbound connection and is used in the **serverName** component of a server endpoint URI as described in the URI format section.

The value for each entry is a *server data configuration object*, **org.fusesource.camel.component.sap.model.rfc.impl.ServerDataImpl**, which defines the configuration of an inbound SAP connection.

Sample server configuration

The following Blueprint XML code shows how to create a sample server configuration with the name, **quickstartServer**.

```
<?xml version="1.0" encoding="UTF-8"?>
<blueprint ... >
    ...
    <!-- Configures the Inbound and Outbound SAP Connections -->
    <bean id="sap-configuration"
        class="org.fusesource.camel.component.sap.SapConnectionConfiguration">
        <property name="destinationDataStore">
            <map>
                <entry key="quickstartDest" value-ref="quickstartDestinationData" />
            </map>
        </property>
        <property name="serverDataStore">
            <map>
                <entry key="quickstartServer" value-ref="quickstartServerData" />
            </map>
        </property>
    </bean>

    <!-- Configures an Outbound SAP Connection -->
    <!-- *** Please enter the connection property values for your environment *** -->
    <bean id="quickstartDestinationData"
        class="org.fusesource.camel.component.sap.model.rfc.impl.DestinationDataImpl">
        <property name="ashost" value="example.com" />
        <property name="sysnr" value="00" />
        <property name="client" value="000" />
        <property name="user" value="username" />
        <property name="passwd" value="passowrd" />
        <property name="lang" value="en" />
    </bean>

    <!-- Configures an Inbound SAP Connection -->
    <!-- *** Please enter the connection property values for your environment ** -->
    <bean id="quickstartServerData"
        class="org.fusesource.camel.component.sap.model.rfc.impl.ServerDataImpl">
        <property name="gwhost" value="example.com" />
        <property name="gwserv" value="3300" />
        <!-- The following property values should not be changed -->
        <property name="progid" value="QUICKSTART" />
        <property name="repositoryDestination" value="quickstartDest" />
        <property name="connectionCount" value="2" />
    </bean>
</blueprint>
```

Notice how this example also configures a destination connection, **quickstartDest**, which the server uses to retrieve metadata from a remote SAP instance. This destination is configured in the server data through the **repositoryDestination** option. If you do not configure this option, you must create a local metadata repository instead.

For example, after configuring the destination as shown in the preceding Blueprint XML file, you could handle the **BAPI_FLCUST_GETLIST** remote function call from an invoking client, using the following URI:

```
sap-srfc-server:quickstartServer:BAPI_FLCUST_GETLIST
```

121.3.3.1. Required options

The required options for the server data configuration object are, as follows:

Name	Default Value	Description
gwhost		Gateway host on which the server connection should be registered.
gwserv		Gateway service, which is the port on which a registration can be done. In order to resolve the service names sapgwXXX , a lookup in etc/services is performed by the network layer of the operating system. If using port numbers instead of symbolic service names, no lookups are performed and no additional entries are needed.
progid		The program ID with which the registration is done. Serves as an identifier on the gateway and in the destination in the ABAP system.
repositoryDestination		Specifies a destination name that the server can use in order to retrieve metadata from a metadata repository hosted in a remote SAP server.
connectionCount		The number of connections that should be registered at the gateway.

121.3.3.2. Secure network connection options

The secure network connection options for the server data configuration object are as follows:

Name	Default Value	Description
------	---------------	-------------

sncMode		Secure network connection (SNC) mode, 0 (off) or 1 (on).
sncQop		SNC level of security, 1 to 9 .
sncMyname		SNC name of your server. Overrides the default SNC name. Typically something like p:CN=JCoServer, O=ACompany, C=EN.
sncLib		Path to library which provides SNC service. If this property is not provided, the value of the jco.middleware.snc_lib property is used instead.

121.3.3.3. Other options

The other options for the server data configuration object are, as follows:

Name	Default Value	Description
saprouter		SAP router string to use for a system protected by a firewall, which can therefore only be reached through a SAProuter, when registering the server at the gateway of that ABAP System. A typical router string is /H/firewall.hostname/H/ .
maxStartupDelay		The maximum time (in seconds) between two start-up attempts in case of failures. The waiting time is doubled from initially 1 second after each start-up failure until either the maximum value is reached or the server could be started successfully.
trace		Enable/disable RFC trace (0 or 1)
workerThreadCount		The maximum number of threads used by the server connection. If not set, the value for the connectionCount is used as the workerThreadCount . The maximum number of threads can not exceed 99.

workerThreadMinCount		The minimum number of threads used by server connection. If not set, the value for connectionCount is used as the workerThreadMinCount .
-----------------------------	--	--

121.3.4. Repository Configuration

The configurations for repositories are maintained in the **repositoryDataStore** property of the SAP Component. Each entry in this map configures a distinct repository. The key for each entry is the name of the repository and this key also corresponds to the name of the server to which this repository is attached.

The value of each entry is a repository data configuration object, **org.fusesource.camel.component.sap.model.rfc.impl.RepositoryDataImpl**, that defines the contents of a metadata repository. A repository data object is a map of function template configuration objects, **org.fusesource.camel.component.sap.model.rfc.impl.FunctionTemplateImpl**. Each entry in this map specifies the interface of a function module and the key for each entry is the name of the function module specified.

Repository data example

The following code shows a simple example of configuring a metadata repository:

```
<?xml version="1.0" encoding="UTF-8"?>
<blueprint ... >
  ...
  <!-- Configures the sap-srfc-server component -->
  <bean id="sap-configuration"
    class="org.fusesource.camel.component.sap.SapConnectionConfiguration">
    <property name="repositoryDataStore">
      <map>
        <entry key="nplServer" value-ref="nplRepositoryData" />
      </map>
    </property>
  </bean>

  <!-- Configures a Metadata Repository -->
  <bean id="nplRepositoryData"
    class="org.fusesource.camel.component.sap.model.rfc.impl.RepositoryDataImpl">
    <property name="functionTemplates">
      <map>
        <entry key="BOOK_FLIGHT" value-ref="bookFlightFunctionTemplate" />
      </map>
    </property>
  </bean>
  ...
</blueprint>
```

121.3.4.1. Function template properties

The interface of a function module consists of four parameter lists by which data is transferred back and forth to the function module in an RFC call. Each parameter list consists of one or more fields, each of which is a named parameter transferred in an RFC call. The following parameter lists and exception list

are supported:

- The *import parameter list* contains parameter values sent to a function module in an RFC call;
- The *export parameter list* contains parameter values that are returned by a function module in an RFC call;
- The *changing parameter list* contains parameter values sent to and returned by a function module in an RFC call;
- The *table parameter list* contains internal table values sent to and returned by a function module in an RFC call.
- The interface of a function module also consists of an *exception list* of ABAP exceptions that may be raised when the module is invoked in an RFC call.

A function template describes the name and type of parameters in each parameter list of a function interface and the ABAP exceptions thrown by the function. A function template object maintains five property lists of metadata objects, as described in the following table.

Property	Description
importParameterList	A list of list field metadata objects, org.fusesource.camel.component.sap.model.rfc.impl.ListFieldMeataDataImpl . Specifies the parameters sent in an RFC call to a function module.
changingParameterList	A list of list field metadata objects, org.fusesource.camel.component.sap.model.rfc.impl.ListFieldMeataDataImpl . Specifies the parameters sent and returned in an RFC call to and from a function module.
exportParameterList	A list of list field metadata objects, org.fusesource.camel.component.sap.model.rfc.impl.ListFieldMeataDataImpl . Specifies the parameters returned in an RFC call from a function module.
tableParameterList	A list of list field metadata objects, org.fusesource.camel.component.sap.model.rfc.impl.ListFieldMeataDataImpl . Specifies the table parameters that are sent and returned in an RFC call to and from a function module.
exceptionList	A list of ABAP exception metadata objects, org.fusesource.camel.component.sap.model.rfc.impl.AbapExceptionImpl . Specifies the ABAP exceptions potentially raised in an RFC call of the function module.

Function template example

The following example shows an outline of how to configure a function template:

```
<bean id="bookFlightFunctionTemplate"
  class="org.fusesource.camel.component.sap.model.rfc.impl.FunctionTemplateImpl">
  <property name="importParameterList">
    <list>
      ...
    </list>
  </property>
  <property name="changingParameterList">
    <list>
      ...
    </list>
  </property>
  <property name="exportParameterList">
    <list>
      ...
    </list>
  </property>
  <property name="tableParameterList">
    <list>
      ...
    </list>
  </property>
  <property name="exceptionList">
    <list>
      ...
    </list>
  </property>
</bean>
```

121.3.4.2. List field metadata properties

A list field metadata object,

org.fusesource.camel.component.sap.model.rfc.impl.ListFieldMeataDataImpl, specifies the name and type of a field in a parameter list. For an elementary parameter field (**CHAR**, **DATE**, **BCD**, **TIME**, **BYTE**, **NUM**, **FLOAT**, **INT**, **INT1**, **INT2**, **DECF16**, **DECF34**, **STRING**, **XSTRING**), the following table lists the configuration properties that may be set on a list field metadata object:

Name	Default Value	Description
name	-	The name of the parameter field.
type	-	The parameter type of the field.
byteLength	-	The field length in bytes for a non-Unicode layout. This value depends on the parameter type.
unicodeByteLength	-	The field length in bytes for a Unicode layout. This value depends on the parameter type.

decimals	0	The number of decimals in field value. Required for parameter types BCD and FLOAT.
optional	false	If true , the field is optional and need not be set in an RFC call.

Note that all elementary parameter fields require that the **name**, **type**, **byteLength**, and **unicodeByteLength** properties be specified in the field metadata object. In addition, the **BCD**, **FLOAT**, **DECF16**, and **DECF34** fields require the decimal property to be specified in the field metadata object.

For a complex parameter field of type **TABLE** or **STRUCTURE**, the following table lists the configuration properties that may be set on a list field metadata object:

Name	Default Value	Description
name	-	The name of the parameter field.
type	-	The parameter type of the field.
recordMetaData	-	The metadata for the structure or table. A record metadata object, org.fusesource.camel.component.sap.model.rfc.impl.RecordMetaDataImpl , is passed to specify the fields in the structure or table rows.
optional	false	If true , the field is optional and need not be set in a RFC call.



NOTE

All complex parameter fields require that the **name**, **type**, and **recordMetaData** properties be specified in the field metadata object. The value of the **recordMetaData** property is a record field metadata object, **org.fusesource.camel.component.sap.model.rfc.impl.RecordMetaDataImpl**, which specifies the structure of a nested structure or the structure of a table row.

Elementary list field metadata example

The following metadata configuration specifies an optional, 24-digit packed BCD number parameter with two decimal places named **TICKET_PRICE**:

```
<bean class="org.fusesource.camel.component.sap.model.rfc.impl.ListFieldMetaDataImpl">
  <property name="name" value="TICKET_PRICE" />
  <property name="type" value="BCD" />
  <property name="byteLength" value="12" />
  <property name="unicodeByteLength" value="24" />
</bean>
```

```

    <property name="decimals" value="2" />
    <property name="optional" value="true" />
  </bean>

```

Complex list field metadata example

The following metadata configuration specifies a required **TABLE** parameter named **CONNINFO** with a row structure specified by the **connectionInfo** record metadata object:

```

<bean class="org.fusesource.camel.component.sap.model.rfc.impl.ListFieldMetaDatumImpl">
  <property name="name" value="CONNINFO" />
  <property name="type" value="TABLE" />
  <property name="recordMetaData" ref="connectionInfo" />
</bean>

```

121.3.4.3. Record metadata properties

A record metadata object, **org.fusesource.camel.component.sap.model.rfc.impl.RecordMetaDatumImpl**, specifies the name and contents of a nested **STRUCTURE** or the row of a **TABLE** parameter. A record metadata object maintains a list of record field metadata objects, **org.fusesource.camel.component.sap.model.rfc.impl.FieldMetaDatumImpl**, which specifies the parameters that reside in the nested structure or table row.

The following table lists configuration properties that may be set on a record metadata object:

Name	Default Value	Description
name	-	The name of the record.
recordFieldMetaData	-	The list of record field metadata objects, org.fusesource.camel.component.sap.model.rfc.impl.FieldMetaDatumImpl . Specifies the fields contained within the structure.



NOTE

All properties of the record metadata object are required.

Record metadata example

The following example shows how to configure a record metadata object:

```

<bean id="connectionInfo"
  class="org.fusesource.camel.component.sap.model.rfc.impl.RecordMetaDatumImpl">
  <property name="name" value="CONNECTION_INFO" />
  <property name="recordFieldMetaData">
    <list>
      ...
    </list>
  </property>
</bean>

```

```

    </list>
  </property>
</bean>

```

121.3.4.4. Record field metadata properties

A record field metadata object,

org.fusesource.camel.component.sap.model.rfc.impl.FieldMetaDataImpl, specifies the name and type of a parameter field within a structure.

A record field metadata object is similar to a parameter field metadata object, except that the offsets of the individual field locations within the nested structure or table row must be additionally specified. The non-Unicode and Unicode offsets of an individual field must be calculated and specified from the sum of non-Unicode and Unicode byte lengths of the preceding fields in the structure or row.



NOTE

The failure to properly specify the offsets of fields in nested structures and table rows will cause the field storage of parameters in the underlying JCo and ABAP runtimes to overlap and prevent the proper transfer of values in RFC calls.

For an elementary parameter field (**CHAR**, **DATE**, **BCD**, **TIME**, **BYTE**, **NUM**, **FLOAT**, **INT**, **INT1**, **INT2**, **DECF16**, **DECF34**, **STRING**, **XSTRING**), the following table lists the configuration properties that may be set on a record field metadata object:

Name	Default Value	Description
name	-	The name of the parameter field.
type	-	The parameter type of the field.
byteLength	-	The field length in bytes for a non-Unicode layout. This value depends on the parameter type.
unicodeByteLength	-	The field length in bytes for a Unicode layout. This value depends on the parameter type.
byteOffset	-	The field offset in bytes for non-Unicode layout. This offset is the byte location of the field within the enclosing structure.
unicodeByteOffset	-	The field offset in bytes for Unicode layout. This offset is the byte location of the field within the enclosing structure.

decimals	0	The number of decimals in field value; only required for parameter types BCD and FLOAT .
-----------------	----------	--

For a complex parameter field of type **TABLE** or **STRUCTURE**, the following table lists the configuration properties that may be set on a record field metadata object:

Name	Default Value	Description
name	-	The name of the parameter field.
type	-	The parameter type of the field.
byteOffset	-	The field offset in bytes for non-Unicode layout. This offset is the byte location of the field within the enclosing structure.
unicodeByteOffset	-	The field offset in bytes for Unicode layout. This offset is the byte location of the field within the enclosing structure.
recordMetaData	-	The metadata for the structure or table. A record metadata object, org.fusesource.camel.component.sap.model.rfc.impl.RecordMetaDataImpl , is passed to specify the fields in the structure or table rows.

Elementary record field metadata example

The following metadata configuration specifies a **DATE** field parameter named **ARRDATE** located 85 bytes into the enclosing structure in the case of a non-Unicode layout and located 170 bytes into the enclosing structure in the case of a Unicode layout.

```
<bean class="org.fusesource.camel.component.sap.model.rfc.impl.FieldMetaDataImpl">
  <property name="name" value="ARRDATE" />
  <property name="type" value="DATE" />
  <property name="byteLength" value="8" />
  <property name="unicodeByteLength" value="16" />
  <property name="byteOffset" value="85" />
  <property name="unicodeByteOffset" value="170" />
</bean>
```

Complex record field metadata example

The following metadata configuration specifies a **STRUCTURE** field parameter named **FLTINFO** with a structure specified by the **flightInfo** record metadata object. The parameter is located at the beginning of the enclosing structure in both the case of a non-Unicode and Unicode layout.


```

<bean class="org.fusesource.camel.component.sap.model.rfc.impl.FieldMetaDataImpl">
  <property name="name" value="FLTINFO" />
  <property name="type" value="STRUCTURE" />
  <property name="byteOffset" value="0" />
  <property name="unicodeByteOffset" value="0" />
  <property name="recordMetaData" ref="flightInfo" />
</bean>

```

121.4. MESSAGE HEADERS

The SAP component supports the following message headers:

Header	Description
CamelSap.scheme	The URI scheme of the last endpoint to process the message. Use one of the following values: sap-srfc-destination sap-trfc-destination sap-qrfc-destination sap-srfc-server sap-trfc-server sap-idoc-destination sap-idoclist-destination sap-qidoc-destination sap-qidoclist-destination sap-idoclist-server
CamelSap.destinationName	The destination name of the last destination endpoint to process the message.
CamelSap.serverName	The server name of the last server endpoint to process the message.
CamelSap.queueName	The queue name of the last queuing endpoint to process the message.
CamelSap.rfcName	The RFC name of the last RFC endpoint to process the message.
CamelSap.idocType	The IDoc type of the last IDoc endpoint to process the message.

CamelSap.idocTypeExtension	The IDoc type extension, if any, of the last IDoc endpoint to process the message.
CamelSap.systemRelease	The system release, if any, of the last IDoc endpoint to process the message.
CamelSap.applicationRelease	The application release, if any, of the last IDoc endpoint to process the message.

121.5. EXCHANGE PROPERTIES

The SAP component adds the following exchange properties:

Property	Description
CamelSap.destinationPropertiesMap	A map containing the properties of each SAP destination encountered by the exchange. The map is keyed by destination name and each entry is a java.util.Properties object containing the configuration properties of that destination.
CamelSap.serverPropertiesMap	A map containing the properties of each SAP server encountered by the exchange. The map is keyed by server name and each entry is a java.util.Properties object containing the configuration properties of that server.

121.6. MESSAGE BODY FOR RFC

121.6.1. Request and response objects

An SAP endpoint expects to receive a message with a message body containing an SAP request object and will return a message with a message body containing an SAP response object. SAP requests and responses are fixed map data structures containing named fields with each field having a predefined data type.

Note that the named fields in an SAP request and response are specific to an SAP endpoint, with each endpoint defining the parameters in the SAP request and response it will accept. An SAP endpoint provides factory methods to create the request and response objects that are specific to it.

```
public class SAPEndpoint ... {
    ...
    public Structure getRequest() throws Exception;

    public Structure getResponse() throws Exception;
    ...
}
```

121.6.2. Structure objects

Both SAP request and response objects are represented in Java as a structure object which supports the **org.fusesource.camel.component.sap.model.rfc.Structure** interface. This interface extends both the **java.util.Map** and **org.eclipse.emf.ecore.EObject** interfaces.

```
public interface Structure extends org.eclipse.emf.ecore.EObject,
    java.util.Map<String, Object> {

    <T> T get(Object key, Class<T> type);

}
```

The field values in a structure object are accessed through the field's getter methods in the map interface. In addition, the structure interface provides a type-restricted method to retrieve field values.

Structure objects are implemented in the component runtime using the Eclipse Modeling Framework (EMF) and support that framework's **EObject** interface. Instances of a structure object have attached metadata which define and restrict the structure and contents of the map of fields it provides. This metadata can be accessed and introspected using the standard methods provided by EMF. Please refer to the EMF documentation for further details.



NOTE

Attempts to get a parameter not defined on a structure object will return null. Attempts to set a parameter not defined on a structure will throw an exception as well as attempts to set the value of a parameter with an incorrect type.

As discussed in the following sections, structure objects can contain fields that contain values of the complex field types, **STRUCTURE**, and **TABLE**.



NOTE

It is unnecessary to create instances of these types and add them to the structure. Instances of these field values are created on demand if necessary when accessed in the enclosing structure.

121.6.3. Field types

The fields that reside within the structure object of an SAP request or response may be either *elementary* or *complex*. An elementary field contains a single scalar value, whereas a complex field will contain one or more fields of either an elementary or complex type.

121.6.3.1. Elementary field types

An elementary field may be a character, numeric, hexadecimal or string field type. The following table summarizes the types of elementary fields that may reside in a structure object:

Field Type	Corresponding Java Type	Byte Length	Unicode Byte Length	Number Decimals Digits	Description

CHAR	java.lang.String	1 to 65535	1 to 65535	-	ABAP Type 'C': Fixed sized character string
DATE	java.util.Date	8	16	-	ABAP Type 'D': Date (format: YYYYMMDD)
BCD	java.math.BigDecimal	1 to 16	1 to 16	0 to 14	ABAP Type 'P': Packed BCD number. A BCD number contains two digits per byte.
TIME	java.util.Date	6	12	-	ABAP Type 'T': Time (format: HHMMSS)
BYTE	byte[]	1 to 65535	1 to 65535	-	ABAP Type 'X':Fixed sized byte array
NUM	java.lang.String	1 to 65535	1 to 65535	-	ABAP Type 'N': Fixed sized numeric character string
FLOAT	java.lang.Double	8	8	0 to 15	ABAP Type 'F': Floating point number
INT	java.lang.Integer	4	4	-	ABAP Type 'I': 4-byte Integer
INT2	java.lang.Integer	2	2	-	ABAP Type 'S': 2-byte Integer
INT1	java.lang.Integer	1	1	-	ABAP Type 'B': 1-byte Integer
DECF16	java.math.BigDecimal	8	8	16	ABAP Type 'decfloat16': 8 -byte Decimal Floating Point Number

DECF34	java.math.BigDecimal	16	16	34	ABAP Type 'decfloat34': 16-byte Decimal Floating Point Number
STRING	java.lang.String	8	8	-	ABAP Type 'G': Variable length character string
XSTRING	byte[]	8	8	-	ABAP Type 'Y': Variable length byte array

121.6.3.2. Character field types

A character field contains a fixed sized character string that may use either a non-Unicode or Unicode character encoding in the underlying JCo and ABAP runtimes. Non-Unicode character strings encode one character per byte. Unicode character strings are encoded in two bytes using UTF-16 encoding. Character field values are represented in Java as **java.lang.String** objects and the underlying JCo runtime is responsible for the conversion to their ABAP representation.

A character field declares its field length in its associated **byteLength** and **unicodeByteLength** properties, which determine the length of the field's character string in each encoding system.

CHAR

A **CHAR** character field is a text field containing alphanumeric characters and corresponds to the ABAP type C.

NUM

A **NUM** character field is a numeric text field containing numeric characters only and corresponds to the ABAP type N.

DATE

A **DATE** character field is an 8 character date field with the year, month and day formatted as **YYYYMMDD** and corresponds to the ABAP type D.

TIME

A **TIME** character field is a 6 character time field with the hours, minutes and seconds formatted as **HHMMSS** and corresponds to the ABAP type T.

121.6.3.3. Numeric field types

A numeric field contains a number. The following numeric field types are supported:

INT

An **INT** numeric field is an integer field stored as a 4-byte integer value in the underlying JCo and ABAP runtimes and corresponds to the ABAP type I. An **INT** field value is represented in Java as a **java.lang.Integer** object.

INT2

An **INT2** numeric field is an integer field stored as a 2-byte integer value in the underlying JCo and ABAP runtimes and corresponds to the ABAP type S. An **INT2** field value is represented in Java as a **java.lang.Integer** object.

INT1

An **INT1** field is an integer field stored as a 1-byte integer value in the underlying JCo and ABAP runtimes value and corresponds to the ABAP type B. An **INT1** field value is represented in Java as a **java.lang.Integer** object.

FLOAT

A **FLOAT** field is a binary floating point number field stored as an 8-byte double value in the underlying JCo and ABAP runtimes and corresponds to the ABAP type F. A **FLOAT** field declares the number of decimal digits that the field's value contains in its associated decimal property. In the case of a **FLOAT** field, this decimal property can have a value between 1 and 15 digits. A **FLOAT** field value is represented in Java as a **java.lang.Double** object.

BCD

A **BCD** field is a binary coded decimal field stored as a 1 to 16 byte packed number in the underlying JCo and ABAP runtimes and corresponds to the ABAP type P. A packed number stores two decimal digits per byte. A **BCD** field declares its field length in its associated **byteLength** and **unicodeByteLength** properties. In the case of a **BCD** field, these properties can have a value between 1 and 16 bytes, and both properties will have the same value. A **BCD** field declares the number of decimal digits that the field's value contains in its associated decimal property. In the case of a **BCD** field, this decimal property can have a value between 1 and 14 digits. A **BCD** field value is represented in Java as a **java.math.BigDecimal**.

DECF16

A **DECF16** field is a decimal floating point stored as an 8-byte IEEE 754 decimal64 floating point value in the underlying JCo and ABAP runtimes and corresponds to the ABAP type **decfloat16**. The value of a **DECF16** field has 16 decimal digits. The value of a **DECF16** field is represented in Java as **java.math.BigDecimal**.

DECF34

A **DECF34** field is a decimal floating point stored as a 16-byte IEEE 754 decimal128 floating point value in the underlying JCo and ABAP runtimes and corresponds to the ABAP type **decfloat34**. The value of a **DECF34** field has 34 decimal digits. The value of a **DECF34** field is represented in Java as **java.math.BigDecimal**.

121.6.3.4. Hexadecimal field types

A hexadecimal field contains raw binary data. The following hexadecimal field types are supported:

BYTE

A **BYTE** field is a fixed sized byte string stored as a byte array in the underlying JCo and ABAP runtimes and corresponds to the ABAP type X. A **BYTE** field declares its field length in its associated **byteLength** and **unicodeByteLength** properties. In the case of a **BYTE** field, these properties can have a value between 1 and 65535 bytes and both properties will have the same value. The value of a **BYTE** field is represented in Java as a **byte[]** object.

121.6.3.5. String field types

A string field references a variable length string value. The length of that string value is not fixed until runtime. The storage for the string value is dynamically created in the underlying JCo and ABAP runtimes. The storage for the string field itself is fixed and contains only a string header.

STRING

A **STRING** field refers to a character string stored in the underlying JCo and ABAP runtimes as an 8-byte value. It corresponds to the ABAP type G. The value of the **STRING** field is represented in Java as a **java.lang.String** object.

XSTRING

An **XSTRING** field refers to a byte string stored in the underlying JCo and ABAP runtimes as an 8-byte value. It corresponds to the ABAP type Y. The value of the **STRING** field is represented in Java as a **byte[]** object.

121.6.3.6. Complex field types

A complex field may be either a structure or table field type. The following table summarizes these complex field types.

Field Type	Corresponding Java Type	Byte Length	Unicode Byte Length	Number Decimals Digits	Description
STRUCTURE	org.fusesource.camel.component.sap.model.rfc.Structure	Total of individual field byte lengths	Total of individual field Unicode byte lengths	-	ABAP Type 'u' & 'v': Heterogeneous Structure
TABLE	org.fusesource.camel.component.sap.model.rfc.Table	Byte length of row structure	Unicode byte length of row structure	-	ABAP Type 'h': Table

121.6.3.7. Structure field types

A **STRUCTURE** field contains a structure object and is stored in the underlying JCo and ABAP runtimes as an ABAP structure record. It corresponds to either an ABAP type **u** or **v**. The value of a **STRUCTURE** field is represented in Java as a structure object with the interface **org.fusesource.camel.component.sap.model.rfc.Structure**.

121.6.3.8. Table field types

A **TABLE** field contains a table object and is stored in the underlying JCo and ABAP runtimes as an ABAP internal table. It corresponds to the ABAP type **h**. The value of the field is represented in Java by a table object with the interface **org.fusesource.camel.component.sap.model.rfc.Table**.

121.6.3.9. Table objects

A table object is a homogeneous list data structure containing rows of structure objects with the same structure. This interface extends both the **java.util.List** and **org.eclipse.emf.ecore.EObject** interfaces.

```
public interface Table<S extends Structure>
    extends org.eclipse.emf.ecore.EObject,
        java.util.List<S> {

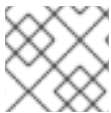
    /**
     * Creates and adds a table row at the end of the row list
     */
    S add();

    /**
     * Creates and adds a table row at the index in the row list
     */
    S add(int index);

}
```

The list of rows in a table object is accessed and managed using the standard methods defined in the list interface. In addition, the table interface provides two factory methods for creating and adding structure objects to the row list.

Table objects are implemented in the component runtime using the Eclipse Modeling Framework (EMF) and support that framework's EObject interface. Instances of a table object have attached metadata which define and restrict the structure and contents of the rows it provides. This metadata can be accessed and introspected using the standard methods provided by EMF. Please refer to the EMF documentation for further details.



NOTE

Attempts to add or set a row structure value of the wrong type will throw an exception.

121.7. MESSAGE BODY FOR IDOC

121.7.1. IDoc message type

When using one of the IDoc Camel SAP endpoints, the type of the message body depends on which particular endpoint you are using.

For a **sap-idoc-destination** endpoint or a **sap-qidoc-destination** endpoint, the message body is of **Document** type:

```
org.fusesource.camel.component.sap.model.idoc.Document
```

For a **sap-idoclist-destination** endpoint, a **sap-qidoclist-destination** endpoint, or a **sap-idoclist-server** endpoint, the message body is of **DocumentList** type:

```
org.fusesource.camel.component.sap.model.idoc.DocumentList
```

121.7.2. The IDoc document model

For the Camel SAP component, an IDoc document is modeled using the Eclipse Modeling Framework (EMF), which provides a wrapper API around the underlying SAP IDoc API. The most important types in this model are:

```
org.fusesource.camel.component.sap.model.idoc.Document
org.fusesource.camel.component.sap.model.idoc.Segment
```

The **Document** type represents an IDoc document instance. In outline, the **Document** interface exposes the following methods:

```
// Java
package org.fusesource.camel.component.sap.model.idoc;
...
public interface Document extends EObject {
    // Access the field values from the IDoc control record
    String getArchiveKey();
    void setArchiveKey(String value);
    String getClient();
    void setClient(String value);
    ...

    // Access the IDoc document contents
    Segment getRootSegment();
}
```

The following kinds of method are exposed by the **Document** interface:

Methods for accessing the control record

Most of the methods are for accessing or modifying field values of the IDoc control record. These methods are of the form *AttributeName*, where *AttributeName* is the name of a field value.

Method for accessing the document contents

The **getRootSegment** method provides access to the document contents (IDoc data records), returning the contents as a **Segment** object. Each **Segment** object can contain an arbitrary number of child segments, and the segments can be nested to an arbitrary degree.

Note, however, that the precise layout of the segment hierarchy is defined by the particular *IDoc* type of the document. When creating (or reading) a segment hierarchy, therefore, you must be sure to follow the exact structure as defined by the IDoc type.

The **Segment** type is used to access the data records of the IDoc document, where the segments are laid out in accordance with the structure defined by the document's IDoc type. In outline, the **Segment** interface exposes the following methods:

```
// Java
package org.fusesource.camel.component.sap.model.idoc;
...
public interface Segment extends EObject, java.util.Map<String, Object> {
    // Returns the value of the '<em><b>Parent</b></em>' reference.
    Segment getParent();

    // Return an immutable list of all child segments
    <S extends Segment> EList<S> getChildren();

    // Returns a list of child segments of the specified segment type.
```

```

<S extends Segment> SegmentList<S> getChildren(String segmentType);

EList<String> getTypes();

Document getDocument();

String getDescription();

String getType();

String getDefinition();

int getHierarchyLevel();

String getIdocType();

String getIdocTypeExtension();

String getSystemRelease();

String getApplicationRelease();

int getNumFields();

long getMaxOccurrence();

long getMinOccurrence();

boolean isMandatory();

boolean isQualified();

int getRecordLength();

<T> T get(Object key, Class<T> type);
}

```

The **getChildren(String segmentType)** method is particularly useful for adding new (nested) children to a segment. It returns an object of type, **SegmentList**, which is defined as follows:

```

// Java
package org.fusesource.camel.component.sap.model.idoc;
...
public interface SegmentList<S extends Segment> extends EObject, EList<S> {
    S add();

    S add(int index);
}

```

Hence, to create a data record of **E1SCU_CRE** type, you could use Java code like the following:

```

Segment rootSegment = document.getRootSegment();

Segment E1SCU_CRE_Segment = rootSegment.getChildren("E1SCU_CRE").add();

```

121.7.3. How an IDoc is related to a Document object

According to the SAP documentation, an IDoc document consists of the following main parts:

Control record

The control record (which contains the metadata for the IDoc document) is represented by the attributes on the **Document** object.

Data records

The data records are represented by the **Segment** objects, which are constructed as a nested hierarchy of segments. You can access the root segment through the **Document.getRootSegment** method.

Status records

In the Camel SAP component, the status records are **not** represented by the document model. But you do have access to the latest status value through the **status** attribute on the control record.

Example of creating a Document instance

The following example shows how to create an IDoc document with the IDoc type, **FLCUSTOMER_CREATEFROMDATA01**, using the IDoc model API in Java.

Example 121.1. Creating an IDoc Document in Java

```
// Java
import org.fusesource.camel.component.sap.model.idoc.Document;
import org.fusesource.camel.component.sap.model.idoc.Segment;
import org.fusesource.camel.component.sap.util.IDocUtil;

import org.fusesource.camel.component.sap.model.idoc.Document;
import org.fusesource.camel.component.sap.model.idoc.DocumentList;
import org.fusesource.camel.component.sap.model.idoc.IdocFactory;
import org.fusesource.camel.component.sap.model.idoc.IdocPackage;
import org.fusesource.camel.component.sap.model.idoc.Segment;
import org.fusesource.camel.component.sap.model.idoc.SegmentChildren;
...
//
// Create a new IDoc instance using the modeling classes
//

// Get the SAP Endpoint bean from the Camel context.
// In this example, it's a 'sap-idoc-destination' endpoint.
SapTransactionalIdocDestinationEndpoint endpoint =
    exchange.getContext().getEndpoint(
        "bean:SapEndpointBeanID",
        SapTransactionalIdocDestinationEndpoint.class
    );

// The endpoint automatically populates some required control record attributes
Document document = endpoint.createDocument()

// Initialize additional control record attributes
document.setMessageType("FLCUSTOMER_CREATEFROMDATA");
document.setRecipientPartnerNumber("QUICKCLNT");
document.setRecipientPartnerType("LS");
document.setSenderPartnerNumber("QUICKSTART");
```

```

document.setSenderPartnerType("LS");

Segment rootSegment = document.getRootSegment();

Segment E1SCU_CRE_Segment = rootSegment.getChildren("E1SCU_CRE").add();

Segment E1BPSCUNEW_Segment =
E1SCU_CRE_Segment.getChildren("E1BPSCUNEW").add();
E1BPSCUNEW_Segment.put("CUSTNAME", "Fred Flintstone");
E1BPSCUNEW_Segment.put("FORM", "Mr.");
E1BPSCUNEW_Segment.put("STREET", "123 Rubble Lane");
E1BPSCUNEW_Segment.put("POSTCODE", "01234");
E1BPSCUNEW_Segment.put("CITY", "Bedrock");
E1BPSCUNEW_Segment.put("COUNTR", "US");
E1BPSCUNEW_Segment.put("PHONE", "800-555-1212");
E1BPSCUNEW_Segment.put("EMAIL", "fred@bedrock.com");
E1BPSCUNEW_Segment.put("CUSTTYPE", "P");
E1BPSCUNEW_Segment.put("DISCOUNT", "005");
E1BPSCUNEW_Segment.put("LANGU", "E");

```

121.8. DOCUMENT ATTRIBUTES

IDoc Document Attributes table shows the control record attributes that you can set on the **Document** object.

Table 121.2. IDoc Document Attributes

Attribute	Length	SAP Field	Description
archiveKey	70	ARCKEY	EDI archive key
client	3	MANDT	Client
creationDate	8	CREDAT	Date IDoc was created
creationTime	6	CRETIM	Time IDoc was created
direction	1	DIRECT	Direction
eDIMessage	14	REFMES	Reference to message
eDIMessageGroup	14	REFGRP	Reference to message group
eDIMessageType	6	STDMES	EDI message type
eDIStandardFlag	1	STD	EDI standard
eDIStandardVersion	6	STDVRS	Version of EDI standard

Attribute	Length	SAP Field	Description
eDITransmissionFile	14	REFINT	Reference to interchange file
iDocCompoundType	8	DOCTYP	IDoc type
iDocNumber	16	DOCNUM	IDoc number
iDocSAPRelease	4	DOCREL	SAP Release of IDoc
iDocType	30	IDOCTP	Name of basic IDoc type
iDocTypeExtension	30	CIMTYP	Name of extension type
messageCode	3	MESCOD	Logical message code
messageFunction	3	MESFCT	Logical message function
messageType	30	MESTYP	Logical message type
outputMode	1	OUTMOD	Output mode
recipientAddress	10	RCVSAD	Receiver address (SADR)
recipientLogicalAddress	70	RCVLAD	Logical address of receiver
recipientPartnerFunction	2	RCVPFC	Partner function of receiver
recipientPartnerNumber	10	RCVPRN	Partner number of receiver
recipientPartnerType	2	RCVPRT	Partner type of receiver
recipientPort	10	RCVPOR	Receiver port (SAP System, EDI subsystem)
senderAddress		SNDSAD	Sender address (SADR)
senderLogicalAddress	70	SNDLAD	Logical address of sender

Attribute	Length	SAP Field	Description
senderPartnerFunction	2	SNDPFC	Partner function of sender
senderPartnerNumber	10	SNDPRN	Partner number of sender
senderPartnerType	2	SNDPRT	Partner type of sender
senderPort	10	SNDPOR	Sender port (SAP System, EDI subsystem)
serialization	20	SERIAL	EDI/ALE: Serialization field
status	2	STATUS	Status of IDoc
testFlag	1	TEST	Test flag

121.8.1. Setting document attributes in Java

When setting the control record attributes in Java, the usual convention for Java bean properties is followed. That is, a **name** attribute can be accessed through the **getName** and **setName** methods, for getting and setting the attribute value. For example, the **iDocType**, **iDocTypeExtension**, and **messageType** attributes can be set as follows on a **Document** object:

```
// Java
document.setIDocType("FLCUSTOMER_CREATEFROMDATA01");
document.setIDocTypeExtension("");
document.setMessageType("FLCUSTOMER_CREATEFROMDATA");
```

121.8.2. Setting document attributes in XML

When setting the control record attributes in XML, the attributes must be set on the **idoc:Document** element. For example, the **iDocType**, **iDocTypeExtension**, and **messageType** attributes can be set as follows:

```
<?xml version="1.0" encoding="ASCII"?>
<idoc:Document ...
    iDocType="FLCUSTOMER_CREATEFROMDATA01"
    iDocTypeExtension=""
    messageType="FLCUSTOMER_CREATEFROMDATA" ... >
    ...
</idoc:Document>
```

121.9. TRANSACTION SUPPORT

121.9.1. BAPI transaction model

The SAP Component supports the BAPI transaction model for outbound communication with SAP. A destination endpoint with a URL containing the transacted option set to **true** will, if necessary, initiate a stateful session on the outbound connection of the endpoint and register a Camel Synchronization object with the exchange. This synchronization object will call the BAPI service method **BAPI_TRANSACTION_COMMIT** and end the stateful session when the processing of the message exchange is complete. If the processing of the message exchange fails, the synchronization object will call the BAPI server method **BAPI_TRANSACTION_ROLLBACK** and end the stateful session.

121.9.2. RFC transaction model

The tRFC protocol accomplishes an AT-MOST-ONCE delivery and processing guarantee by identifying each transactional request with a unique transaction identifier (TID). A TID accompanies each request sent in the protocol. A sending application using the tRFC protocol must identify each instance of a request with a unique TID when sending the request. An application may send a request with a given TID multiple times, but the protocol ensures that the request is delivered and processed in the receiving system at most once. An application may choose to resend a request with a given TID when encountering a communication or system error when sending the request, and is thus in doubt as to whether that request was delivered and processed in the receiving system. By resending a request when encountering a communication error, a client application using the tRFC protocol can thus ensure EXACTLY-ONCE delivery and processing guarantees for its request.

121.9.3. Which transaction model to use?

A BAPI transaction is an application level transaction, in the sense that it imposes ACID guarantees on the persistent data changes performed by a BAPI method or RFC function in the SAP database. An RFC transaction is a communication transaction, in the sense that it imposes delivery guarantees (AT-MOST-ONCE, EXACTLY-ONCE, EXACTLY-ONCE-IN-ORDER) on requests to a BAPI method and/or RFC function.

121.9.4. Transactional RFC destination endpoints

The following destination endpoints support RFC transactions:

- **sap-trfc-destination**
- **sap-qrfc-destination**

A single Camel route can include multiple transactional RFC destination endpoints, sending messages to multiple RFC destinations and even sending messages to the same RFC destination multiple times. This implies that the Camel SAP component potentially needs to keep track of **many** transaction IDs (TIDs) for each **Exchange** object passing along a route. Now if the route processing fails and must be retried, the situation gets quite complicated. The RFC transaction semantics demand that each RFC destination along the route must be invoked using the **same** TID that was used the first time around (and where the TIDs for each destinations are distinct from each other). In other words, the Camel SAP component must keep track of which TID was used at which point along the route, and remember this information, so that the TIDs can be replayed in the correct order.

By default, Camel does not provide a mechanism that enables an **Exchange** to know where it is in a route. To provide such a mechanism, it is necessary to install the **CurrentProcessorDefinitionInterceptStrategy** interceptor into the Camel runtime. This interceptor must be installed into the Camel runtime, in order for the Camel SAP component to keep track of the TIDs in a route.

121.9.5. Transactional RFC server endpoints

The following server endpoints support RFC transactions:

- **sap-trfc-server**

When a Camel exchange processing a transactional request encounters a processing error, Camel handles the processing error through its standard error handling mechanisms. If the Camel route processing the exchange is configured to propagate the error back to the caller, the SAP server endpoint that initiated the exchange takes note of the failure and the sending SAP system is notified of the error. The sending SAP system can then respond by sending another transaction request with the same TID to process the request again.

121.10. XML SERIALIZATION FOR RFC

SAP request and response objects support an XML serialization format which enable these objects to be serialized to and from an XML document.

121.10.1. XML namespace

Each RFC in a repository defines a specific XML namespace for the elements which compose the serialized forms of its Request and Response objects. The form of this namespace URL is as follows:

```
http://sap.fusesource.org/rfc/<Repository Name>/<RFC Name>
```

RFC namespace URLs have a common <http://sap.fusesource.org/rfc> prefix followed by the name of the repository in which the RFC's metadata is defined. The final component in the URL is the name of the RFC itself.

121.10.2. Request and response XML documents

An SAP request object will be serialized into an XML document with the root element of that document named Request and scoped by the namespace of the request's RFC.

```
<?xml version="1.0" encoding="ASCII"?>
<BOOK_FLIGHT:Request
  xmlns:BOOK_FLIGHT="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT">
  ...
</BOOK_FLIGHT:Request>
```

An SAP response object will be serialized into an XML document with the root element of that document named Response and scoped by the namespace of the response's RFC.

```
<?xml version="1.0" encoding="ASCII"?>
<BOOK_FLIGHT:Response
  xmlns:BOOK_FLIGHT="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT">
  ...
</BOOK_FLIGHT:Response>
```

121.10.3. Structure fields

Structure fields in parameter lists or nested structures are serialized as elements. The element name of the serialized structure corresponds to the field name of the structure within the enclosing parameter list, structure or table row entry it resides.

```
<BOOK_FLIGHT:FLTINFO
  xmlns:BOOK_FLIGHT="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT">
  ...
</BOOK_FLIGHT:FLTINFO>
```

Note that the type name of the structure element in the RFC namespace will correspond to the name of the record metadata object which defines the structure, as in the following example:

```
<xs:schema
  targetNamespace="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT">
  xmlns:xs="http://www.w3.org/2001/XMLSchema">
  ...
  <xs:complexType name="FLTINFO_STRUCTURE">
  ...
  </xs:complexType>
  ...
</xs:schema>
```

This distinction will be important when specifying a JAXB bean to marshal and unmarshal the structure.

121.10.4. Table fields

Table fields in parameter lists or nested structures are serialized as elements. The element name of the serialized structure will correspond to the field name of the table within the enclosing parameter list, structure, or table row entry it resides. The table element will contain a series of row elements to hold the serialized values of the table's row entries.

```
<BOOK_FLIGHT:CONNINFO
  xmlns:BOOK_FLIGHT="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT">
  <row ... > ... </row>
  ...
  <row ... > ... </row>
</BOOK_FLIGHT:CONNINFO>
```

Note that the type name of the table element in the RFC namespace corresponds to the name of the record metadata object which defines the row structure of the table suffixed by **_TABLE**. The type name of the table row element in the RFC name corresponds to the name of the record metadata object which defines the row structure of the table, as in the following example:

```
<xs:schema
  targetNamespace="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT">
  xmlns:xs="http://www.w3.org/2001/XMLSchema">
  ...
  <xs:complexType name="CONNECTION_INFO_STRUCTURE_TABLE">
    <xs:sequence>
      <xs:element
        name="row"
        minOccurs="0"
        maxOccurs="unbounded"
        type="CONNECTION_INFO_STRUCTURE"/>
    </xs:sequence>
  </xs:complexType>
```

```
...
    <xs:sequence>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="CONNECTION_INFO_STRUCTURE">
  ...
</xs:complexType>
...
</xs:schema>
```

This distinction will be important when specifying a JAXB bean to marshal and unmarshal the structure.

121.10.5. Elementary fields

Elementary fields in parameter lists or nested structures are serialized as attributes on the element of the enclosing parameter list or structure. The attribute name of the serialized field corresponds to the field name of the field within the enclosing parameter list, structure, or table row entry it resides, as in the following example:

```
<?xml version="1.0" encoding="ASCII"?>
<BOOK_FLIGHT:Request
  xmlns:BOOK_FLIGHT="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT"
  CUSTNAME="James Legrand"
  PASSFORM="Mr"
  PASSNAME="Travelin Joe"
  PASSBIRTH="1990-03-17T00:00:00.000-0500"
  FLIGHTDATE="2014-03-19T00:00:00.000-0400"
  TRAVELAGENCYNUMBER="00000110"
  DESTINATION_FROM="SFO"
  DESTINATION_TO="FRA"/>
```

121.10.6. Date and time formats

Date and Time fields are serialized into attribute values using the following format:

```
yyyy-MM-dd'T'HH:mm:ss.SSSZ
```

Date fields will be serialized with only the year, month, day and timezone components set:

```
DEPDATE="2014-03-19T00:00:00.000-0400"
```

Time fields will be serialized with only the hour, minute, second, millisecond and timezone components set:

```
DEPTIME="1970-01-01T16:00:00.000-0500"
```

121.11. XML SERIALIZATION FOR IDOC

An IDoc message body can be serialized into an XML string format, with the help of a built-in type converter.

121.11.1. XML namespace

Each serialized IDoc is associated with an XML namespace, which has the following general format:

```
http://sap.fusesource.org/idoc/repositoryName/idocType/idocTypeExtension/systemRelease/applicationRelease
```

Both the *repositoryName* (name of the remote SAP metadata repository) and the *idocType* (IDoc document type) are mandatory, but the other components of the namespace can be left blank. For example, you could have an XML namespace like the following:

```
http://sap.fusesource.org/idoc/MY_REPO/FLCUSTOMER_CREATEFROMDATA01///
```

121.11.2. Built-in type converter

The Camel SAP component has a built-in type converter, which is capable of converting a **Document** object or a **DocumentList** object to and from a **String** type.

For example, to serialize a **Document** object to an XML string, you can simply add the following line to a route in XML DSL:

```
<convertBodyTo type="java.lang.String"/>
```

You can also use this approach to a serialized XML message into a **Document** object. For example, given that the current message body is a serialized XML string, you can convert it back into a **Document** object by adding the following line to a route in XML DSL:

```
<convertBodyTo type="org.fusesource.camel.component.sap.model.idoc.Document"/>
```

121.11.3. Sample IDoc message body in XML format

When you convert an IDoc message to a **String**, it is serialized into an XML document, where the root element is either **idoc:Document** (for a single document) or **idoc:DocumentList** (for a list of documents). It shows that a single IDoc document that has been serialized to an **idoc:Document** element.

Example 121.2. IDoc Message Body in XML

```
<?xml version="1.0" encoding="ASCII"?>
<idoc:Document
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:FLCUSTOMER_CREATEFROMDATA01---
="http://sap.fusesource.org/idoc/XXX/FLCUSTOMER_CREATEFROMDATA01///"
  xmlns:idoc="http://sap.fusesource.org/idoc"
  creationDate="2015-01-28T12:39:13.980-0500"
  creationTime="2015-01-28T12:39:13.980-0500"
  iDocType="FLCUSTOMER_CREATEFROMDATA01"
  iDocTypeExtension=""
  messageType="FLCUSTOMER_CREATEFROMDATA"
  recipientPartnerNumber="QUICKCLNT"
  recipientPartnerType="LS"
  senderPartnerNumber="QUICKSTART"
  senderPartnerType="LS">
```

```

<rootSegment xsi:type="FLCUSTOMER_CREATEFROMDATA01---:ROOT" document="/">
  <segmentChildren parent="//@rootSegment">
    <E1SCU_CRE parent="//@rootSegment" document="/">
      <segmentChildren parent="//@rootSegment/@segmentChildren/@E1SCU_CRE.0">
        <E1BPSCUNEW parent="//@rootSegment/@segmentChildren/@E1SCU_CRE.0"
          document="/"
          CUSTNAME="Fred Flintstone" FORM="Mr."
          STREET="123 Rubble Lane"
          POSTCODE="01234"
          CITY="Bedrock"
          COUNTR="US"
          PHONE="800-555-1212"
          EMAIL="fred@bedrock.com"
          CUSTTYPE="P"
          DISCOUNT="005"
          LANGU="E"/>
      </segmentChildren>
    </E1SCU_CRE>
  </segmentChildren>
</rootSegment>
</idoc:Document>

```

121.12. EXAMPLE 1: READING DATA FROM SAP

This example demonstrates a route that reads **FlightCustomer** business object data from SAP. The route invokes the **FlightCustomer** BAPI method, **BAPI_FLCUST_GETLIST**, using an SAP synchronous RFC destination endpoint to retrieve the data.

121.12.1. Java DSL for route

The Java DSL for the example route is as follows:

```

from("direct:getFlightCustomerInfo")
  .to("bean:createFlightCustomerGetListRequest")
  .to("sap-srfc-destination:nplDest:BAPI_FLCUST_GETLIST")
  .to("bean:returnFlightCustomerInfo");

```

121.12.2. XML DSL for route

And the Spring DSL for the same route is as follows:

```

<route>
  <from uri="direct:getFlightCustomerInfo"/>
  <to uri="bean:createFlightCustomerGetListRequest"/>
  <to uri="sap-srfc-destination:nplDest:BAPI_FLCUST_GETLIST"/>
  <to uri="bean:returnFlightCustomerInfo"/>
</route>

```

121.12.3. createFlightCustomerGetListRequest bean

The **createFlightCustomerGetListRequest** bean is responsible for building an SAP request object in its exchange method that is used in the RFC call of the subsequent SAP endpoint. The following code snippet demonstrates the sequence of operations to build the request object:

```
public void create(Exchange exchange) throws Exception {

    // Get SAP Endpoint to be called from context.
    SapSynchronousRfcDestinationEndpoint endpoint =
        exchange.getContext().getEndpoint("sap-srfc-destination:nplDest:BAPI_FLCUST_GETLIST",
            SapSynchronousRfcDestinationEndpoint.class);

    // Retrieve bean from message containing Flight Customer name to
    // look up.
    BookFlightRequest bookFlightRequest =
        exchange.getIn().getBody(BookFlightRequest.class);

    // Create SAP Request object from target endpoint.
    Structure request = endpoint.getRequest();

    // Add Customer Name to request if set
    if (bookFlightRequest.getCustomerName() != null &&
        bookFlightRequest.getCustomerName().length() > 0) {
        request.put("CUSTOMER_NAME",
            bookFlightRequest.getCustomerName());
    }
    else {
        throw new Exception("No Customer Name");
    }

    // Put request object into body of exchange message.
    exchange.getIn().setBody(request);
}
```

121.12.4. returnFlightCustomerInfo bean

The **returnFlightCustomerInfo** bean is responsible for extracting data from the SAP response object in its exchange method that it receives from the previous SAP endpoint. The following code snippet demonstrates the sequence of operations to extract the data from the response object:

```
public void createFlightCustomerInfo(Exchange exchange) throws Exception {

    // Retrieve SAP response object from body of exchange message.
    Structure flightCustomerGetListResponse =
        exchange.getIn().getBody(Structure.class);

    if (flightCustomerGetListResponse == null) {
        throw new Exception("No Flight Customer Get List Response");
    }

    // Check BAPI return parameter for errors
    @SuppressWarnings("unchecked")
    Table<Structure> bapiReturn =
        flightCustomerGetListResponse.get("RETURN", Table.class);
    Structure bapiReturnEntry = bapiReturn.get(0);
    if (bapiReturnEntry.get("TYPE", String.class) != "S") {
```

```

        String message = bapiReturnEntry.get("MESSAGE", String.class);
        throw new Exception("BAPI call failed: " + message);
    }

    // Get customer list table from response object.
    @SuppressWarnings("unchecked")
    Table<? extends Structure> customerList =
        flightCustomerGetListResponse.get("CUSTOMER_LIST", Table.class);

    if (customerList == null || customerList.size() == 0) {
        throw new Exception("No Customer Info.");
    }

    // Get Flight Customer data from first row of table.
    Structure customer = customerList.get(0);

    // Create bean to hold Flight Customer data.
    FlightCustomerInfo flightCustomerInfo = new FlightCustomerInfo();

    // Get customer id from Flight Customer data and add to bean.
    String customerId = customer.get("CUSTOMERID", String.class);
    if (customerId != null) {
        flightCustomerInfo.setCustomerNumber(customerId);
    }

    ...

    // Put bean into body of exchange message.
    exchange.getIn().setHeader("flightCustomerInfo", flightCustomerInfo);
}

```

121.13. EXAMPLE 2: WRITING DATA TO SAP

This example demonstrates a route that creates a **FlightTrip** business object instance in SAP. The route invokes the **FlightTrip** BAPI method, **BAPI_FLTRIP_CREATE**, using a destination endpoint to create the object.

121.13.1. Java DSL for route

The Java DSL for the example route is as follows:

```

from("direct:createFlightTrip")
    .to("bean:createFlightTripRequest")
    .to("sap-srfc-destination:nplDest:BAPI_FLTRIP_CREATE?transacted=true")
    .to("bean:returnFlightTripResponse");

```

121.13.2. XML DSL for route

And the Spring DSL for the same route is as follows:

```

<route>
  <from uri="direct:createFlightTrip"/>
  <to uri="bean:createFlightTripRequest"/>

```

```

    <to uri="sap-srfc-destination:nplDest:BAPI_FLTRIP_CREATE?transacted=true"/>
    <to uri="bean:returnFlightTripResponse"/>
</route>

```

121.13.3. Transaction support

Note that the URL for the SAP endpoint has the **transacted** option set to **true**. When this option is enabled, the endpoint ensures that an SAP transaction session has been initiated before invoking the RFC call. Because this endpoint's RFC creates new data in SAP, this option is necessary to make the route's changes permanent in SAP.

121.13.4. Populating request parameters

The **createFlightTripRequest** and **returnFlightTripResponse** beans are responsible for populating request parameters into the SAP request and extracting response parameters from the SAP response respectively, following the same sequence of operations as demonstrated in the previous example.

121.14. EXAMPLE 3: HANDLING REQUESTS FROM SAP

This example demonstrates a route which handles a request from SAP to the **BOOK_FLIGHT** RFC, which is implemented by the route. In addition, it demonstrates the component's XML serialization support, using JAXB to unmarshal and marshal SAP request objects and response objects to custom beans.

This route creates a **FlightTrip** business object on behalf of a travel agent, **FlightCustomer**. The route first unmarshals the SAP request object received by the SAP server endpoint into a custom JAXB bean. This custom bean is then multicasted in the exchange to three sub-routes, which gather the travel agent, flight connection, and passenger information required to create the flight trip. The final sub-route creates the flight trip object in SAP, as demonstrated in the previous example. The final sub-route also creates and returns a custom JAXB bean which is marshaled into an SAP response object and returned by the server endpoint.

121.14.1. Java DSL for route

The Java DSL for the example route is as follows:

```

DataFormat jaxb = new JaxbDataFormat("org.fusesource.sap.example.jaxb");

from("sap-srfc-server:nplserver:BOOK_FLIGHT")
    .unmarshal(jaxb)
    .multicast()
    .to("direct:getFlightConnectionInfo",
        "direct:getFlightCustomerInfo",
        "direct:getPassengerInfo")
    .end()
    .to("direct:createFlightTrip")
    .marshal(jaxb);

```

121.14.2. XML DSL for route

And the XML DSL for the same route is as follows:

```

<route>

```

```

<from uri="sap-srfc-server:nplserver:BOOK_FLIGHT"/>
<unmarshal>
  <jaxb contextPath="org.fusesource.sap.example.jaxb"/>
</unmarshal>
<multicast>
  <to uri="direct:getFlightConnectionInfo"/>
  <to uri="direct:getFlightCustomerInfo"/>
  <to uri="direct:getPassengerInfo"/>
</multicast>
<to uri="direct:createFlightTrip"/>
<marshal>
  <jaxb contextPath="org.fusesource.sap.example.jaxb"/>
</marshal>
</route>

```

121.14.3. BookFlightRequest bean

The following listing illustrates a JAXB bean which unmarshals from the serialized form of an SAP **BOOK_FLIGHT** request object:

```

@XmlRootElement(name="Request",
namespace="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT")
@XmlAccessorType(XmlAccessType.FIELD)
public class BookFlightRequest {

    @XmlAttribute(name="CUSTNAME")
    private String customerName;

    @XmlAttribute(name="FLIGHTDATE")
    @XmlJavaTypeAdapter(DateAdapter.class)
    private Date flightDate;

    @XmlAttribute(name="TRAVELAGENCYNUMBER")
    private String travelAgencyNumber;

    @XmlAttribute(name="DESTINATION_FROM")
    private String startAirportCode;

    @XmlAttribute(name="DESTINATION_TO")
    private String endAirportCode;

    @XmlAttribute(name="PASSFORM")
    private String passengerFormOfAddress;

    @XmlAttribute(name="PASSNAME")
    private String passengerName;

    @XmlAttribute(name="PASSBIRTH")
    @XmlJavaTypeAdapter(DateAdapter.class)
    private Date passengerDateOfBirth;

    @XmlAttribute(name="CLASS")
    private String flightClass;
}

```



```
...
}
```

121.14.4. BookFlightResponse bean

The following listing illustrates a JAXB bean which marshals to the serialized form of an SAP **BOOK_FLIGHT** response object:

```
@XmlElement(name="Response",
namespace="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT")
@XmlAccessorType(XmlAccessType.FIELD)
public class BookFlightResponse {

    @XmlAttribute(name="TRIPNUMBER")
    private String tripNumber;

    @XmlAttribute(name="TICKET_PRICE")
    private BigDecimal ticketPrice;

    @XmlAttribute(name="TICKET_TAX")
    private BigDecimal ticketTax;

    @XmlAttribute(name="CURRENCY")
    private String currency;

    @XmlAttribute(name="PASSFORM")
    private String passengerFormOfAddress;

    @XmlAttribute(name="PASSNAME")
    private String passengerName;

    @XmlAttribute(name="PASSBIRTH")
    @XmlJavaTypeAdapter(DateAdapter.class)
    private Date passengerDateOfBirth;

    @XmlElement(name="FLTINFO")
    private FlightInfo flightInfo;

    @XmlElement(name="CONNINFO")
    private ConnectionInfoTable connectionInfo;

    ...
}
```



NOTE

The complex parameter fields of the response object are serialized as child elements of the response.

121.14.5. FlightInfo bean

The following listing illustrates a JAXB bean which marshals to the serialized form of the complex structure parameter **FLTINFO**:

```

@XmlRootElement(name="FLTINFO",
namespace="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT")
@XmlAccessorType(XmlAccessType.FIELD)
public class FlightInfo {

    @XmlAttribute(name="FLIGHTTIME")
    private String flightTime;

    @XmlAttribute(name="CITYFROM")
    private String cityFrom;

    @XmlAttribute(name="DEPDATE")
    @XmlJavaTypeAdapter(DateAdapter.class)
    private Date departureDate;

    @XmlAttribute(name="DEPTIME")
    @XmlJavaTypeAdapter(DateAdapter.class)
    private Date departureTime;

    @XmlAttribute(name="CITYTO")
    private String cityTo;

    @XmlAttribute(name="ARRDATE")
    @XmlJavaTypeAdapter(DateAdapter.class)
    private Date arrivalDate;

    @XmlAttribute(name="ARRTIME")
    @XmlJavaTypeAdapter(DateAdapter.class)
    private Date arrivalTime;

    ...
}

```

121.14.6. ConnectionInfoTable bean

The following listing illustrates a JAXB bean which marshals to the serialized form of the complex table parameter, **CONNINFO**. Note that the name of the root element type of the JAXB bean corresponds to the name of the row structure type suffixed with **_TABLE** and the bean contains a list of row elements.

```

@XmlRootElement(name="CONNINFO_TABLE",
namespace="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT")
@XmlAccessorType(XmlAccessType.FIELD)
public class ConnectionInfoTable {

    @XmlElement(name="row")
    List<ConnectionInfo> rows;

    ...
}

```

121.14.7. ConnectionInfo bean

The following listing illustrates a JAXB bean, which marshals to the serialized form of the above tables row elements:

```

@XmlRootElement(name="CONNINFO",
namespace="http://sap.fusesource.org/rfc/nplServer/BOOK_FLIGHT")
@XmlAccessorType(XmlAccessType.FIELD)
public class ConnectionInfo {

    @XmlAttribute(name="CONNID")
    String connectionId;

    @XmlAttribute(name="AIRLINE")
    String airline;

    @XmlAttribute(name="PLANETYPE")
    String planeType;

    @XmlAttribute(name="CITYFROM")
    String cityFrom;

    @XmlAttribute(name="DEPDATE")
    @XmlJavaTypeAdapter(DateAdapter.class)
    Date departureDate;

    @XmlAttribute(name="DEPTIME")
    @XmlJavaTypeAdapter(DateAdapter.class)
    Date departureTime;

    @XmlAttribute(name="CITYTO")
    String cityTo;

    @XmlAttribute(name="ARRDATE")
    @XmlJavaTypeAdapter(DateAdapter.class)
    Date arrivalDate;

    @XmlAttribute(name="ARRTIME")
    @XmlJavaTypeAdapter(DateAdapter.class)
    Date arrivalTime;

    ...
}

```

CHAPTER 122. XQUERY

Camel supports [XQuery](#) to allow an [Expression](#) or [Predicate](#) to be used in the [DSL](#).

For example, you could use XQuery to create a predicate in a [Message Filter](#) or as an expression for a [Recipient List](#).

122.1. DEPENDENCIES

To use XQuery in your camel routes you need to add the a dependency on **camel-saxon** which implements the XQuery language. When using **xquery** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-saxon-starter</artifactId>
</dependency>
```

122.2. XQUERY LANGUAGE OPTIONS

The XQuery language supports 4 options, which are listed below.

Name	Default	Java Type	Description
type		String	Sets the class name of the result type (type from output) The default result type is NodeSet.
headerName		String	Name of header to use as input, instead of the message body.
configurationRef		String	Reference to a saxon configuration instance in the registry to use for xquery (requires camel-saxon). This may be needed to add custom functions to a saxon configuration, so these custom functions can be used in xquery expressions.
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

122.3. VARIABLES

The message body will be set as the **contextItem**. And the following variables are available as well:

Variable	Type	Description
exchange	Exchange	The current Exchange
in.body	Object	The message body

Variable	Type	Description
out.body	Object	deprecated The OUT message body (if any)
in.headers.*	Object	You can access the value of exchange.in.headers with key foo by using the variable which name is in.headers.foo
out.headers.*	Object	deprecated You can access the value of exchange.out.headers with key foo by using the variable which name is out.headers.foo variable
key name	Object	Any exchange.properties and exchange.in.headers and any additional parameters set using setParameters(Map) . These parameters are added with they own key name, for instance if there is an IN header with the key name foo then its added as foo .

122.4. EXAMPLE

```
from("queue:foo")
  .filter().xquery("//foo")
  .to("queue:bar")
```

You can also use functions inside your query, in which case you need an explicit type conversion, or you will get an **org.w3c.dom.DOMException: HIERARCHY_REQUEST_ERR**). You need to pass in the expected output type of the function. For example the concat function returns a **String** which is done as shown:

```
from("direct:start")
  .recipientList().xquery("concat('mock:foo.', /person/@city)", String.class);
```

And in XML DSL:

```
<route>
  <from uri="direct:start"/>
  <recipientList>
    <xquery type="java.lang.String">concat('mock:foo.', /person/@city</xquery>
  </recipientList>
</route>
```

122.4.1. Using namespaces

If you have a standard set of namespaces you wish to work with and wish to share them across many XQuery expressions you can use the **org.apache.camel.support.builder.Namespaces** when using Java DSL as shown:

```
Namespaces ns = new Namespaces("c", "http://acme.com/cheese");

from("direct:start")
  .filter().xquery("/c:person[@name='James']", ns)
  .to("mock:result");
```

Notice how the namespaces are provided to **xquery** with the **ns** variable that are passed in as the 2nd parameter.

Each namespace is a key=value pair, where the prefix is the key. In the XQuery expression then the namespace is used by its prefix, eg:

```
/c:person[@name='James']
```

The namespace builder supports adding multiple namespaces as shown:

```
Namespaces ns = new Namespaces("c", "http://acme.com/cheese")
    .add("w", "http://acme.com/wine")
    .add("b", "http://acme.com/beer");
```

When using namespaces in XML DSL then its different, as you setup the namespaces in the XML root tag (or one of the **camelContext**, **routes**, **route** tags).

In the XML example below we use Spring XML where the namespace is declared in the root tag **beans**, in the line with **xmlns:foo="http://example.com/person"**:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:foo="http://example.com/person"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://camel.apache.org/schema/spring http://camel.apache.org/schema/spring/camel-
    spring.xsd">

  <camelContext id="camel" xmlns="http://activemq.apache.org/camel/schema/spring">
    <route>
      <from uri="activemq:MyQueue"/>
      <filter>
        <xquery>/foo:person[@name='James']</xquery>
        <to uri="mqseries:SomeOtherQueue"/>
      </filter>
    </route>
  </camelContext>
</beans>
```

This namespace uses **foo** as prefix, so the **<xquery>** expression uses **/foo:** to use this namespace.

122.5. USING XQUERY AS TRANSFORMATION

We can do a message translation using transform or setBody in the route, as shown below:

```
from("direct:start").
  transform().xquery("/people/person");
```

Notice that xquery will use DOMResult by default, so if we want to grab the value of the person node, using **text()** we need to tell XQuery to use String as result type, as shown:

```
from("direct:start").
  transform().xquery("/people/person/text()", String.class);
```

If you want to use Camel variables like headers, you have to explicitly declare them in the XQuery expression.

```
<transform>
  <xquery>
    declare variable $in.headers.foo external;
    element item {$in.headers.foo}
  </xquery>
</transform>
```

122.6. LOADING SCRIPT FROM EXTERNAL RESOURCE

You can externalize the script and have Camel load it from a resource such as **"classpath:"**, **"file:"**, or **"http:"**. This is done using the following syntax: **"resource:scheme:location"**, e.g. to refer to a file on the classpath you can do:

```
.setHeader("myHeader").xquery("resource:classpath:myxquery.txt", String.class)
```

122.7. LEARNING XQUERY

XQuery is a very powerful language for querying, searching, sorting and returning XML. For help learning XQuery try these tutorials

- Mike Kay's [XQuery Primer](#)
- The W3Schools [XQuery Tutorial](#)

122.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
camel.component.xquery.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.xquery.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.xquery.configuration	To use a custom Saxon configuration. The option is a <code>net.sf.saxon.Configuration</code> type.		Configuration
camel.component.xquery.configuration-properties	To set custom Saxon configuration properties.		Map
camel.component.xquery.enabled	Whether to enable auto configuration of the xquery component. This is enabled by default.		Boolean
camel.component.xquery.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.xquery.module-uri-resolver	To use the custom ModuleURIResolver. The option is a <code>net.sf.saxon.lib.ModuleURIResolver</code> type.		ModuleURIResolver
camel.language.xquery.configuration-ref	Reference to a saxon configuration instance in the registry to use for xquery (requires camel-saxon). This may be needed to add custom functions to a saxon configuration, so these custom functions can be used in xquery expressions.		String
camel.language.xquery.enabled	Whether to enable auto configuration of the xquery language. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.language.xquery.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.xquery.type</code>	Sets the class name of the result type (type from output) The default result type is NodeSet.		String

CHAPTER 123. SCHEDULER

Only consumer is supported

The Scheduler component is used to generate message exchanges when a scheduler fires. This component is similar to the [Timer](#) component, but it offers more functionality in terms of scheduling. Also this component uses JDK **ScheduledExecutorService**. Where as the timer uses a JDK **Timer**.

You can only consume events from this endpoint.

123.1. DEPENDENCIES

When using **scheduler** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-scheduler-starter</artifactId>
</dependency>
```

123.2. URI FORMAT

```
scheduler:name[?options]
```

Where **name** is the name of the scheduler, which is created and shared across endpoints. So if you use the same name for all your scheduler endpoints, only one scheduler thread pool and thread will be used - but you can configure the thread pool to allow more concurrent threads.



NOTE

The IN body of the generated exchange is **null**. So **exchange.getIn().getBody()** returns **null**.

123.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

123.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

123.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

123.4. COMPONENT OPTIONS

The Scheduler component supports 3 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

Name	Description	Default	Type
poolSize (scheduler)	Number of core threads in the thread pool used by the scheduling thread pool. Is by default using a single thread.	1	int

123.5. ENDPOINT OPTIONS

The Scheduler endpoint is configured using URI syntax:

```
scheduler:name
```

with the following path and query parameters:

123.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (consumer)	Required The name of the scheduler.		String

123.5.2. Query Parameters (21 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the <code>backoffMultiplier</code> should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the <code>backoffMultiplier</code> should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then <code>backoffIdleThreshold</code> and/or <code>backoffErrorThreshold</code> must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the <code>ScheduledPollConsumer</code> will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
poolSize (scheduler)	Number of core threads in the thread pool used by the scheduling thread pool. Is by default using a single thread.	1	int

Name	Description	Default	Type
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean

Name	Description	Default	Type
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANoseconds • MICROseconds • MILLIseconds • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean

123.6. MORE INFORMATION

This component is a scheduler [Polling Consumer](#) where you can find more information about the options above, and examples at the [Polling Consumer](#) page.

123.7. EXCHANGE PROPERTIES

When the timer is fired, it adds the following information as properties to the **Exchange**:

Name	Type	Description
Exchange.TIMER_NAME	String	The value of the name option.
Exchange.TIMER_FIRED_TIME	Date	The time when the consumer fired.

123.8. SAMPLE

To set up a route that generates an event every 60 seconds:

```
from("scheduler://foo?delay=60000").to("bean:myBean?method=someMethodName");
```

■

The above route will generate an event and then invoke the **someMethodName** method on the bean called **myBean** in the Registry such as JNDI or Spring.

And the route in Spring DSL:

```
<route>
  <from uri="scheduler://foo?delay=60000"/>
  <to uri="bean:myBean?method=someMethodName"/>
</route>
```

123.9. FORCING THE SCHEDULER TO TRIGGER IMMEDIATELY WHEN COMPLETED

To let the scheduler trigger as soon as the previous task is complete, you can set the option **greedy=true**. But beware then the scheduler will keep firing all the time. So use this with caution.

123.10. FORCING THE SCHEDULER TO BE IDLE

There can be use cases where you want the scheduler to trigger and be greedy. But sometimes you want "tell the scheduler" that there was no task to poll, so the scheduler can change into idle mode using the backoff options. To do this you would need to set a property on the exchange with the key **Exchange.SCHEDULER_POLLED_MESSAGES** to a boolean value of false. This will cause the consumer to indicate that there was no messages polled.

The consumer will otherwise as by default return 1 message polled to the scheduler, every time the consumer has completed processing the exchange.

123.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.component.scheduler.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.scheduler.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.scheduler.enabled	Whether to enable auto configuration of the scheduler component. This is enabled by default.		Boolean
camel.component.scheduler.pool-size	Number of core threads in the thread pool used by the scheduling thread pool. Is by default using a single thread.	1	Integer

CHAPTER 124. SEDA

Both producer and consumer are supported

The SEDA component provides asynchronous [SEDA](#) behavior, so that messages are exchanged on a [BlockingQueue](#) and consumers are invoked in a separate thread from the producer.

Note that queues are only visible within a *single* CamelContext. If you want to communicate across **CamelContext** instances (for example, communicating between Web applications), see the component.

This component does not implement any kind of persistence or recovery, if the VM terminates while messages are yet to be processed. If you need persistence, reliability or distributed SEDA, try using either [JMS](#) or ActiveMQ.



NOTE

Synchronous

The [Direct](#) component provides synchronous invocation of any consumers when a producer sends a message exchange.

124.1. DEPENDENCIES

When using **seda** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-seda-starter</artifactId>
</dependency>
```

124.2. URI FORMAT

```
seda:someName[?options]
```

Where **someName** can be any string that uniquely identifies the endpoint within the current CamelContext.

124.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

124.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a

few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

124.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

124.4. COMPONENT OPTIONS

The SEDA component supports 10 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
concurrentConsumers (consumer)	Sets the default number of concurrent threads processing exchanges.	1	int

Name	Description	Default	Type
defaultPollTimeout (consumer (advanced))	The timeout (in milliseconds) used when polling. When a timeout occurs, the consumer can check whether it is allowed to continue running. Setting a lower value allows the consumer to react more quickly upon shutdown.	1000	int
defaultBlockWhenFull (producer)	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will instead block and wait until the message can be accepted.	false	boolean
defaultDiscardWhenFull (producer)	Whether a thread that sends messages to a full SEDA queue will be discarded. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will give up sending and continue, meaning that the message was not sent to the SEDA queue.	false	boolean
defaultOfferTimeout (producer)	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, where a configured timeout can be added to the block case. Utilizing the <code>.offer(timeout)</code> method of the underlining java queue.		long
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

Name	Description	Default	Type
defaultQueueFactory (advanced)	Sets the default queue factory.		BlockingQueueFactory
queueSize (advanced)	Sets the default maximum capacity of the SEDA queue (i.e., the number of messages it can hold).	1000	int

124.5. ENDPOINT OPTIONS

The SEDA endpoint is configured using URI syntax:

```
seda:name
```

with the following path and query parameters:

124.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (common)	Required Name of queue.		String

124.5.2. Query Parameters (18 parameters)

Name	Description	Default	Type
size (common)	The maximum capacity of the SEDA queue (i.e., the number of messages it can hold). Will by default use the <code>defaultSize</code> set on the SEDA component.	1000	int
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
concurrentConsumers (consumer)	Number of concurrent threads processing exchanges.	1	int

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
limitConcurrentConsumers (consumer (advanced))	Whether to limit the number of <code>concurrentConsumers</code> to the maximum of 500. By default, an exception will be thrown if an endpoint is configured with a greater number. You can disable that check by turning this option off.	<code>true</code>	<code>boolean</code>
multipleConsumers (consumer (advanced))	Specifies whether multiple consumers are allowed. If enabled, you can use SEDA for Publish-Subscribe messaging. That is, you can send a message to the SEDA queue and have each consumer receive a copy of the message. When enabled, this option should be specified on every consumer endpoint.	<code>false</code>	<code>boolean</code>
pollTimeout (consumer (advanced))	The timeout (in milliseconds) used when polling. When a timeout occurs, the consumer can check whether it is allowed to continue running. Setting a lower value allows the consumer to react more quickly upon shutdown.	<code>1000</code>	<code>int</code>
purgeWhenStopping (consumer (advanced))	Whether to purge the task queue when stopping the consumer/route. This allows to stop faster, as any pending messages on the queue is discarded.	<code>false</code>	<code>boolean</code>
blockWhenFull (producer)	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will instead block and wait until the message can be accepted.	<code>false</code>	<code>boolean</code>

Name	Description	Default	Type
discardIfNoConsumers (producer)	Whether the producer should discard the message (do not add the message to the queue), when sending to a queue with no active consumers. Only one of the options <code>discardIfNoConsumers</code> and <code>failIfNoConsumers</code> can be enabled at the same time.	false	boolean
discardWhenFull (producer)	Whether a thread that sends messages to a full SEDA queue will be discarded. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will give up sending and continue, meaning that the message was not sent to the SEDA queue.	false	boolean
failIfNoConsumers (producer)	Whether the producer should fail by throwing an exception, when sending to a queue with no active consumers. Only one of the options <code>discardIfNoConsumers</code> and <code>failIfNoConsumers</code> can be enabled at the same time.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
offerTimeout (producer)	Offer timeout (in milliseconds) can be added to the block case when queue is full. You can disable timeout by using 0 or a negative value.		long
timeout (producer)	Timeout (in milliseconds) before a SEDA producer will stop waiting for an asynchronous task to complete. You can disable timeout by using 0 or a negative value.	30000	long

Name	Description	Default	Type
waitForTaskToComplete (producer)	Option to specify whether the caller should wait for the async task to complete or not before continuing. The following three options are supported: Always, Never or IfReplyExpected. The first two values are self-explanatory. The last value, IfReplyExpected, will only wait if the message is Request Reply based. The default option is IfReplyExpected. Enum values: • Never • IfReplyExpected • Always	IfReplyExpected	WaitForTaskToComplete
queue (advanced)	Define the queue instance which will be used by the endpoint.		BlockingQueue

124.6. CHOOSING BLOCKINGQUEUE IMPLEMENTATION

By default, the SEDA component always instantiates `LinkedBlockingQueue`, but you can use different implementation, you can reference your own `BlockingQueue` implementation, in this case the `size` option is not used

```
<bean id="arrayQueue" class="java.util.ArrayBlockingQueue">
  <constructor-arg index="0" value="10" ><!-- size -->
  <constructor-arg index="1" value="true" ><!-- fairness -->
</bean>

<!-- ... and later -->
<from>seda:array?queue=#arrayQueue</from>
```

Or you can reference a `BlockingQueueFactory` implementation, 3 implementations are provided `LinkedBlockingQueueFactory`, `ArrayBlockingQueueFactory` and `PriorityBlockingQueueFactory`:

```
<bean id="priorityQueueFactory"
class="org.apache.camel.component.seda.PriorityBlockingQueueFactory">
  <property name="comparator">
    <bean class="org.apache.camel.demo.MyExchangeComparator" />
  </property>
</bean>

<!-- ... and later -->
<from>seda:priority?queueFactory=#priorityQueueFactory&size=100</from>
```

124.7. USE OF REQUEST REPLY

The [SEDA](#) component supports using Request Reply, where the caller will wait for the Async route to complete. For instance:

```
from("mina:tcp://0.0.0.0:9876?textline=true&sync=true").to("seda:input");

from("seda:input").to("bean:processInput").to("bean:createResponse");
```

In the route above, we have a TCP listener on port 9876 that accepts incoming requests. The request is routed to the **seda:input** queue. As it is a Request Reply message, we wait for the response. When the consumer on the **seda:input** queue is complete, it copies the response to the original message response.

124.8. CONCURRENT CONSUMERS

By default, the SEDA endpoint uses a single consumer thread, but you can configure it to use concurrent consumer threads. So instead of thread pools you can use:

```
from("seda:stageName?concurrentConsumers=5").process(...)
```

As for the difference between the two, note a *thread pool* can increase/shrink dynamically at runtime depending on load, whereas the number of concurrent consumers is always fixed.

124.9. THREAD POOLS

Be aware that adding a thread pool to a SEDA endpoint by doing something like:

```
from("seda:stageName").thread(5).process(...)
```

Can wind up with two **BlockQueues**: one from the SEDA endpoint, and one from the workqueue of the thread pool, which may not be what you want. Instead, you might wish to configure a Direct endpoint with a thread pool, which can process messages both synchronously and asynchronously. For example:

```
from("direct:stageName").thread(5).process(...)
```

You can also directly configure number of threads that process messages on a SEDA endpoint using the **concurrentConsumers** option.

124.10. SAMPLE

In the route below we use the SEDA queue to send the request to this async queue to be able to send a fire-and-forget message for further processing in another thread, and return a constant reply in this thread to the original caller.

We send a Hello World message and expects the reply to be OK.

```
@Test
public void testSendAsync() throws Exception {
    MockEndpoint mock = getMockEndpoint("mock:result");
    mock.expectedBodiesReceived("Hello World");

    // START SNIPPET: e2
    Object out = template.requestBody("direct:start", "Hello World");
```

```

    assertEquals("OK", out);
    // END SNIPPET: e2

    assertMockEndpointsSatisfied();
}

@Override
protected RouteBuilder createRouteBuilder() throws Exception {
    return new RouteBuilder() {
        // START SNIPPET: e1
        public void configure() throws Exception {
            from("direct:start")
                // send it to the seda queue that is async
                .to("seda:next")
                // return a constant response
                .transform(constant("OK"));

            from("seda:next").to("mock:result");
        }
        // END SNIPPET: e1
    };
}

```

The "Hello World" message will be consumed from the SEDA queue from another thread for further processing. Since this is from a unit test, it will be sent to a **mock** endpoint where we can do assertions in the unit test.

124.11. USING MULTIPLECONSUMERS

In this example we have defined two consumers.

```

@Test
public void testSameOptionsProducerStillOkay() throws Exception {
    getMockEndpoint("mock:foo").expectedBodiesReceived("Hello World");
    getMockEndpoint("mock:bar").expectedBodiesReceived("Hello World");

    template.sendBody("seda:foo", "Hello World");

    assertMockEndpointsSatisfied();
}

@Override
protected RouteBuilder createRouteBuilder() throws Exception {
    return new RouteBuilder() {
        @Override
        public void configure() throws Exception {
            from("seda:foo?multipleConsumers=true").routeId("foo").to("mock:foo");
            from("seda:foo?multipleConsumers=true").routeId("bar").to("mock:bar");
        }
    };
}

```

Since we have specified **multipleConsumers=true** on the seda foo endpoint we can have those two consumers receive their own copy of the message as a kind of pub-sub style messaging.

As the beans are part of an unit test they simply send the message to a mock endpoint.

124.12. EXTRACTING QUEUE INFORMATION

If needed, information such as queue size, etc. can be obtained without using JMX in this fashion:

```
SedaEndpoint seda = context.getEndpoint("seda:xxx");
int size = seda.getExchanges().size();
```

124.13. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
camel.component.seda.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.seda.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.seda.concurrent-consumers	Sets the default number of concurrent threads processing exchanges.	1	Integer
camel.component.seda.default-block-when-full	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will instead block and wait until the message can be accepted.	false	Boolean
camel.component.seda.default-discard-when-full	Whether a thread that sends messages to a full SEDA queue will be discarded. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will give up sending and continue, meaning that the message was not sent to the SEDA queue.	false	Boolean

Name	Description	Default	Type
camel.component.seda.default-offer-timeout	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, where a configured timeout can be added to the block case. Utilizing the <code>.offer(timeout)</code> method of the underlining java queue.		Long
camel.component.seda.default-poll-timeout	The timeout (in milliseconds) used when polling. When a timeout occurs, the consumer can check whether it is allowed to continue running. Setting a lower value allows the consumer to react more quickly upon shutdown.	1000	Integer
camel.component.seda.default-queue-factory	Sets the default queue factory. The option is a <code>org.apache.camel.component.seda.BlockingQueueFactory<org.apache.camel.Exchange></code> type.		BlockingQueueFactory
camel.component.seda.enabled	Whether to enable auto configuration of the seda component. This is enabled by default.		Boolean
camel.component.seda.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.seda.queue-size	Sets the default maximum capacity of the SEDA queue (i.e., the number of messages it can hold).	1000	Integer

CHAPTER 125. SERVLET

Only consumer is supported

The Servlet component provides HTTP based endpoints for consuming HTTP requests that arrive at a HTTP endpoint that is bound to a published Servlet.



NOTE

Stream

Servlet is stream based, which means the input it receives is submitted to Camel as a stream. That means you will only be able to read the content of the stream **once**. If you find a situation where the message body appears to be empty or you need to access the data multiple times (eg: doing multicasting, or redelivery error handling) you should use Stream caching or convert the message body to a **String** which is safe to be read multiple times.

125.1. DEPENDENCIES

When using **servlet** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-servlet-starter</artifactId>
</dependency>
```

125.2. URI FORMAT

```
servlet://relative_path[?options]
```

125.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

125.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).

- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

125.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

125.4. COMPONENT OPTIONS

The Servlet component supports 11 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which means any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
muteException (consumer)	If enabled and an Exchange failed processing on the consumer side the response's body won't contain the exception's stack trace.	false	boolean
servletName (consumer)	Default name of servlet to use. The default name is CamelServlet.	CamelServlet	String

Name	Description	Default	Type
attachmentMultipartBinding (consumer (advanced))	Whether to automatic bind multipart/form-data as attachments on the Camel Exchange. The options <code>attachmentMultipartBinding=true</code> and <code>disableStreamCache=false</code> cannot work together. Remove <code>disableStreamCache</code> to use <code>AttachmentMultipartBinding</code> . This is turned off by default as this may require servlet specific configuration to enable this when using Servlets.	false	boolean
fileNameExtWhitelist (consumer (advanced))	Whitelist of accepted filename extensions for accepting uploaded files. Multiple extensions can be separated by comma, such as <code>txt,xml</code> .		String
httpRegistry (consumer (advanced))	To use a custom <code>org.apache.camel.component.servlet.HttpRegistry</code> .		HttpRegistry
allowJavaSerializedObject (advanced)	Whether to allow java serialization when a request uses <code>context-type=application/x-java-serialized-object</code> . This is by default turned off. If you enable this then be aware that Java will deserialize the incoming data from the request to Java and that can be a potential security risk.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
httpBinding (advanced)	To use a custom <code>HttpBinding</code> to control the mapping between Camel message and <code>HttpClient</code> .		HttpBinding
httpConfiguration (advanced)	To use the shared <code>HttpConfiguration</code> as base configuration.		HttpConfiguration
headerFilterStrategy (filter)	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message.		HeaderFilterStrategy

125.5. ENDPOINT OPTIONS

The Servlet endpoint is configured using URI syntax:

```
servlet:contextPath
```

with the following path and query parameters:

125.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
contextPath (consumer)	Required The context-path to use.		String

125.5.2. Query Parameters (22 parameters)

Name	Description	Default	Type
chunked (consumer)	If this option is false the Servlet will disable the HTTP streaming and set the content-length header on the response.	true	boolean
disableStreamCache (common)	Determines whether or not the raw input stream from Servlet is cached or not (Camel will read the stream into a in memory/overflow to file, Stream caching) cache. By default Camel will cache the Servlet input stream to support reading it multiple times to ensure Camel can retrieve all data from the stream. However you can set this option to true when you for example need to access the raw stream, such as streaming it directly to a file or other persistent store. DefaultHttpBinding will copy the request input stream into a stream cache and put it into message body if this option is false to support reading the stream multiple times. If you use Servlet to bridge/proxy an endpoint then consider enabling this option to improve performance, in case you do not need to read the message payload multiple times. The http producer will by default cache the response body stream. If this option is set to true, then the producers will not cache the response body stream but use the response stream as-is as the message body.	false	boolean
headerFilterStrategy (common)	To use a custom HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
httpBinding (common (advanced))	To use a custom HttpBinding to control the mapping between Camel message and HttpClient.		HttpBinding
async (consumer)	Configure the consumer to work in async mode.	false	boolean

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which means any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
httpMethodRestrict (consumer)	Used to only allow consuming if the HttpMethod matches, such as GET/POST/PUT etc. Multiple methods can be specified separated by comma.		String
matchOnUriPrefix (consumer)	Whether or not the consumer should try to find a target consumer by matching the URI prefix if no exact match is found.	false	boolean
muteException (consumer)	If enabled and an Exchange failed processing on the consumer side the response's body won't contain the exception's stack trace.	false	boolean
responseBufferSize (consumer)	To use a custom buffer size on the <code>javax.servlet.ServletResponse</code> .		Integer
servletName (consumer)	Name of the servlet to use.	Camel Servlet	String
transferException (consumer)	If enabled and an Exchange failed processing on the consumer side, and if the caused Exception was sent back serialized in the response as an <code>application/x-java-serialized-object</code> content type. On the producer side the exception will be deserialized and thrown as is, instead of the <code>HttpOperationFailedException</code> . The caused exception is required to be serialized. This is by default turned off. If you enable this then be aware that Java will deserialize the incoming data from the request to Java and that can be a potential security risk.	false	boolean
attachmentMultipartBinding (consumer (advanced))	Whether to automatic bind multipart/form-data as attachments on the Camel Exchange. The options <code>attachmentMultipartBinding=true</code> and <code>disableStreamCache=false</code> cannot work together. Remove <code>disableStreamCache</code> to use <code>AttachmentMultipartBinding</code> . This is turned off by default as this may require servlet specific configuration to enable this when using Servlets.	false	boolean

Name	Description	Default	Type
eagerCheckContentAvailable (consumer (advanced))	Whether to eager check whether the HTTP requests has content if the content-length header is 0 or not present. This can be turned on in case HTTP clients do not send streamed data.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
fileNameExtWhitelist (consumer (advanced))	Whitelist of accepted filename extensions for accepting uploaded files. Multiple extensions can be separated by comma, such as txt,xml.		String
mapHttpRequestBody (consumer (advanced))	If this option is true then IN exchange Body of the exchange will be mapped to HTTP body. Setting this to false will avoid the HTTP mapping.	true	boolean
mapHttpRequestFormUrlEncodedBody (consumer (advanced))	If this option is true then IN exchange Form Encoded body of the exchange will be mapped to HTTP. Setting this to false will avoid the HTTP Form Encoded body mapping.	true	boolean
mapHttpRequestHeaders (consumer (advanced))	If this option is true then IN exchange Headers of the exchange will be mapped to HTTP headers. Setting this to false will avoid the HTTP Headers mapping.	true	boolean

Name	Description	Default	Type
optionsEnabled (consumer (advanced))	Specifies whether to enable HTTP OPTIONS for this Servlet consumer. By default OPTIONS is turned off.	false	boolean
traceEnabled (consumer (advanced))	Specifies whether to enable HTTP TRACE for this Servlet consumer. By default TRACE is turned off.	false	boolean

125.6. MESSAGE HEADERS

Camel will apply the same Message Headers as the [HTTP](#) component.

Camel will also populate all **request.parameter** and **request.headers**. For example, if a client request has the URL, <http://myserver/myserver?orderid=123>, the exchange will contain a header named **orderid** with the value 123.

125.7. USAGE

You can consume only **from** endpoints generated by the Servlet component. Therefore, it should be used only as input into your Camel routes. To issue HTTP requests against other HTTP endpoints, use the [HTTP](#) component.

125.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 15 options, which are listed below.

Name	Description	Default	Type
camel.component.servlet.allow-java-serialized-object	Whether to allow java serialization when a request uses context-type=application/x-java-serialized-object. This is by default turned off. If you enable this then be aware that Java will deserialize the incoming data from the request to Java and that can be a potential security risk.	false	Boolean
camel.component.servlet.attachment-multipart-binding	Whether to automatic bind multipart/form-data as attachments on the Camel Exchange. The options attachmentMultipartBinding=true and disableStreamCache=false cannot work together. Remove disableStreamCache to use AttachmentMultipartBinding. This is turned off by default as this may require servlet specific configuration to enable this when using Servlet's.	false	Boolean

Name	Description	Default	Type
camel.component.servlet.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.servlet.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which means any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.servlet.enabled	Whether to enable auto configuration of the servlet component. This is enabled by default.		Boolean
camel.component.servlet.file-name-ext-whitelist	Whitelist of accepted filename extensions for accepting uploaded files. Multiple extensions can be separated by comma, such as txt,xml.		String
camel.component.servlet.header-filter-strategy	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message. The option is a org.apache.camel.spi.HeaderFilterStrategy type.		HeaderFilterStrategy
camel.component.servlet.http-binding	To use a custom HttpBinding to control the mapping between Camel message and HttpClient. The option is a org.apache.camel.http.common.HttpBinding type.		HttpBinding
camel.component.servlet.http-configuration	To use the shared HttpConfiguration as base configuration. The option is a org.apache.camel.http.common.HttpConfiguration type.		HttpConfiguration
camel.component.servlet.http-registry	To use a custom org.apache.camel.component.servlet.HttpRegistry. The option is a org.apache.camel.http.common.HttpRegistry type.		HttpRegistry

Name	Description	Default	Type
camel.component.servlet.mute-exception	If enabled and an Exchange failed processing on the consumer side the response's body won't contain the exception's stack trace.	false	Boolean
camel.component.servlet.servlet-name	Default name of servlet to use. The default name is CamelServlet.	CamelServlet	String
camel.servlet.mapping.context-path	Context path used by the servlet component for automatic mapping.	/camel/*	String
camel.servlet.mapping.enabled	Enables the automatic mapping of the servlet component into the Spring web context.	true	Boolean
camel.servlet.mapping.servlet-name	The name of the Camel servlet.	CamelServlet	String

CHAPTER 126. SIMPLE

The Simple Expression Language was a really simple language when it was created, but has since grown more powerful. It is primarily intended for being a very small and simple language for evaluating **Expression** or **Predicate** without requiring any new dependencies or knowledge of other scripting languages such as Groovy.

The simple language is designed with intend to cover almost all the common use cases when little need for scripting in your Camel routes.

However, for much more complex use cases then a more powerful language is recommended such as:

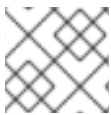
- [Groovy](#)
- [MVEL](#)
- [OGNL](#)



NOTE

The simple language requires **camel-bean** JAR as classpath dependency if the simple language uses OGNL expressions, such as calling a method named **myMethod** on the message body: **`${body.myMethod()}`**. At runtime the simple language will then us its built-in OGNL support which requires the **camel-bean** component.

The simple language uses **`${body}`** placeholders for complex expressions or functions.



NOTE

See also the [CSimple](#) language which is compiled.



NOTE

Alternative syntax

You can also use the alternative syntax which uses **`$simple{ }`** as placeholders. This can be used in situations to avoid clashes when using for example Spring property placeholder together with Camel.

126.1. DEPENDENCIES

When using **simple** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-core-starter</artifactId>
</dependency>
```

126.2. SIMPLE LANGUAGE OPTIONS

The Simple language supports 2 options, which are listed below.

Name	Default	Java Type	Description
resultType		String	Sets the class name of the result type (type from output).
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

126.3. VARIABLES

Variable	Type	Description
camelId	String	the CamelContext name
camelContext. OGNL	Object	the CamelContext invoked using a Camel OGNL expression.
exchange	Exchange	the Exchange
exchange. OGNL	Object	the Exchange invoked using a Camel OGNL expression.
exchangeId	String	the exchange id
id	String	the message id
messageTimestamp	String	the message timestamp (millis since epoch) that this message originates from. Some systems like JMS, Kafka, AWS have a timestamp on the event/message, that Camel received. This method returns the timestamp, if a timestamp exists. The message timestamp and exchange created are not the same. An exchange always have a created timestamp which is the local timestamp when Camel created the exchange. The message timestamp is only available in some Camel components when the consumer is able to extract the timestamp from the source event. If the message has no timestamp then 0 is returned.
body	Object	the body
body. OGNL	Object	the body invoked using a Camel OGNL expression.
bodyAs(<i>type</i>)	Type	Converts the body to the given type determined by its classname. The converted body can be null.
bodyAs(<i>type</i>). OGNL	Object	Converts the body to the given type determined by its classname and then invoke methods using a Camel OGNL expression. The converted body can be null.

Variable	Type	Description
bodyOneLine	String	Converts the body to a String and removes all line-breaks so the string is in one line.
mandatoryBodyAs(<i>type</i>)	Type	Converts the body to the given type determined by its classname, and expects the body to be not null.
mandatoryBodyAs(<i>type</i>). OGNL	Object	Converts the body to the given type determined by its classname and then invoke methods using a Camel OGNL expression.
header.foo	Object	refer to the foo header
header[foo]	Object	refer to the foo header
headers.foo	Object	refer to the foo header
headers:foo	Object	refer to the foo header
headers[foo]	Object	refer to the foo header
header.foo[bar]	Object	regard foo header as a map and perform lookup on the map with bar as key
header.foo. OGNL	Object	refer to the foo header and invoke its value using a Camel OGNL expression.
headerAs(<i>key,type</i>)	Type	converts the header to the given type determined by its classname
headers	Map	refer to the headers
exchangeProperty.foo	Object	refer to the foo property on the exchange
exchangeProperty[foo]	Object	refer to the foo property on the exchange
exchangeProperty.foo. OGNL	Object	refer to the foo property on the exchange and invoke its value using a Camel OGNL expression.
sys.foo	String	refer to the JVM system property
sysenv.foo	String	refer to the system environment variable
env.foo	String	refer to the system environment variable

Variable	Type	Description
exception	Object	refer to the exception object on the exchange, is null if no exception set on exchange. Will fallback and grab caught exceptions (Exchange.EXCEPTION_CAUGHT) if the Exchange has any.
exception.OGNL	Object	refer to the exchange exception invoked using a Camel OGNL expression object
exception.message	String	refer to the exception.message on the exchange, is null if no exception set on exchange. Will fallback and grab caught exceptions (Exchange.EXCEPTION_CAUGHT) if the Exchange has any.
exception.stacktrace	String	refer to the exception.stracktrace on the exchange, is null if no exception set on exchange. Will fallback and grab caught exceptions (Exchange.EXCEPTION_CAUGHT) if the Exchange has any.
date:_command_	Date	evaluates to a Date object. Supported commands are: now for current timestamp, exchangeCreated for the timestamp when the current exchange was created, header.xxx to use the Long/Date object header with the key xxx. exchangeProperty.xxx to use the Long/Date object in the exchange property with the key xxx. file for the last modified timestamp of the file (available with a File consumer). Command accepts offsets such as: now-24h or header.xxx+1h or even now+1h30m-100 .
date:_command;pattern_	String	Date formatting using java.text.SimpleDateFormat patterns.
date-with-timezone:_command;timezone;pattern_	String	Date formatting using java.text.SimpleDateFormat timezones and patterns.
bean:_bean expression_	Object	Invoking a bean expression using the language. Specifying a method name you must use dot as separator. We also support the ?method=methodname syntax that is used by the component. Camel will by default lookup a bean by the given name. However if you need to refer to a bean class (such as calling a static method) then you can prefix with type, such as bean:type:fqnClassName .
properties:key:default	String	Lookup a property with the given key. If the key does not exists or has no value, then an optional default value can be specified.
routeld	String	Returns the id of the current route the Exchange is being routed.
stepld	String	Returns the id of the current step the Exchange is being routed.

Variable	Type	Description
threadName	String	Returns the name of the current thread. Can be used for logging purpose.
hostname	String	Returns the local hostname (may be empty if not possible to resolve).
ref:xxx	Object	To lookup a bean from the Registry with the given id.
type:name.field	Object	To refer to a type or field by its FQN name. To refer to a field you can append <code>.FIELD_NAME</code> . For example, you can refer to the constant field from Exchange as: org.apache.camel.Exchange.FILE_NAME
null	null	represents a null
random(value)	Integer	returns a random Integer between 0 (included) and <i>value</i> (excluded)
random(min,max)	Integer	returns a random Integer between <i>min</i> (included) and <i>max</i> (excluded)
collate(group)	List	The collate function iterates the message body and groups the data into sub lists of specified size. This can be used with the Splitter EIP to split a message body and group/batch the splitted sub message into a group of N sub lists. This method works similar to the collate method in Groovy.
skip(number)	Iterator	The skip function iterates the message body and skips the first number of items. This can be used with the Splitter EIP to split a message body and skip the first N number of items.
messageHistory	String	The message history of the current exchange how it has been routed. This is similar to the route stack-trace message history the error handler logs in case of an unhandled exception.
messageHistory(false)	String	As messageHistory but without the exchange details (only includes the route stack-trace). This can be used if you do not want to log sensitive data from the message itself.

126.4. OGNL EXPRESSION SUPPORT

When using **OGNL** then **camel-bean** JAR is required to be on the classpath.

Camel's OGNL support is for invoking methods only. You cannot access fields. Camel support accessing the length field of Java arrays.

The [Simple](#) and Bean language now supports a Camel OGNL notation for invoking beans in a chain like fashion. Suppose the Message IN body contains a POJO which has a **getAddress()** method.

Then you can use Camel OGNL notation to access the address object:

```
simple("${body.address}")
simple("${body.address.street}")
simple("${body.address.zip}")
```

Camel understands the shorthand names for getters, but you can invoke any method or use the real name such as:

```
simple("${body.address}")
simple("${body.getAddress.getStreet}")
simple("${body.address.getZip}")
simple("${body.doSomething}")
```

You can also use the null safe operator (**?.**) to avoid NPE if for example the body does NOT have an address

```
simple("${body?.address?.street}")
```

It is also possible to index in **Map** or **List** types, so you can do:

```
simple("${body[foo].name}")
```

To assume the body is **Map** based and lookup the value with **foo** as key, and invoke the **getName** method on that value.

If the key has space, then you **must** enclose the key with quotes, for example 'foo bar':

```
simple("${body['foo bar'].name}")
```

You can access the **Map** or **List** objects directly using their key name (with or without dots) :

```
simple("${body[foo]}")
simple("${body[this.is.foo]}")
```

Suppose there was no value with the key **foo** then you can use the null safe operator to avoid the NPE as shown:

```
simple("${body[foo]?.name}")
```

You can also access **List** types, for example to get lines from the address you can do:

```
simple("${body.address.lines[0]}")
simple("${body.address.lines[1]}")
simple("${body.address.lines[2]}")
```

There is a special **last** keyword which can be used to get the last value from a list.

```
simple("${body.address.lines[last]}")
```

And to get the 2nd last you can subtract a number, so we can use **last-1** to indicate this:

```
simple("${body.address.lines[last-1]}")
```

And the 3rd last is of course:

```
simple("${body.address.lines[last-2]}")
```

And you can call the size method on the list with

```
simple("${body.address.lines.size}")
```

Camel supports the length field for Java arrays as well, eg:

```
String[] lines = new String[]{"foo", "bar", "cat"};
exchange.getIn().setBody(lines);
```

```
simple("There are ${body.length} lines")
```

And yes you can combine this with the operator support as shown below:

```
simple("${body.address.zip} > 1000")
```

126.5. OPERATOR SUPPORT

The parser is limited to only support a single operator.

To enable it the left value must be enclosed in `${ }`. The syntax is:

```
${leftValue} OP rightValue
```

Where the **rightValue** can be a String literal enclosed in `' '`, **null**, a constant value or another expression enclosed in `${ }`.



NOTE

There **must** be spaces around the operator.

Camel will automatically type convert the rightValue type to the leftValue type, so it is able to eg. convert a string into a numeric, so you can use `>` comparison for numeric values.

The following operators are supported:

Operator	Description
<code>==</code>	equals
<code>=~</code>	equals ignore case (will ignore case when comparing String values)
<code>></code>	greater than

Operator	Description
>=	greater than or equals
<	less than
≤	less than or equals
!=	not equals
!=~	not equals ignore case (will ignore case when comparing String values)
contains	For testing if contains in a string based value
!contains	For testing if not contains in a string based value
~~	For testing if contains by ignoring case sensitivity in a string based value
!~~	For testing if not contains by ignoring case sensitivity in a string based value
regex	For matching against a given regular expression pattern defined as a String value
!regex	For not matching against a given regular expression pattern defined as a String value
in	For matching if in a set of values, each element must be separated by comma. If you want to include an empty value, then it must be defined using double comma, eg '„bronze,silver,gold', which is a set of four values with an empty value and then the three medals.
!in	For matching if not in a set of values, each element must be separated by comma. If you want to include an empty value, then it must be defined using double comma, eg '„bronze,silver,gold', which is a set of four values with an empty value and then the three medals.
is	For matching if the left hand side type is an instance of the value.
!is	For matching if the left hand side type is not an instance of the value.
range	For matching if the left hand side is within a range of values defined as numbers: from..to..
!range	For matching if the left hand side is not within a range of values defined as numbers: from..to..

Operator	Description
startsWith	For testing if the left hand side string starts with the right hand string.
starts with	Same as the startsWith operator.
endsWith	For testing if the left hand side string ends with the right hand string.
ends with	Same as the endsWith operator.

And the following unary operators can be used:

Operator	Description
++	To increment a number by one. The left hand side must be a function, otherwise parsed as literal.
–	To decrement a number by one. The left hand side must be a function, otherwise parsed as literal.
\n	To use newline character.
\t	To use tab character.
\r	To use carriage return character.
\}	To use the } character as text. This may be needed when building a JSon structure with the simple language.

And the following logical operators can be used to group expressions:

Operator	Description
&&	The logical and operator is used to group two expressions.
	The logical or operator is used to group two expressions.

The syntax for AND is:

```
`${leftValue} OP rightValue && ${leftValue} OP rightValue
```

And the syntax for OR is:

```
`${leftValue} OP rightValue || ${leftValue} OP rightValue
```

Some examples:

```
// exact equals match
simple("${header.foo} == 'foo'")

// ignore case when comparing, so if the header has value FOO this will match
simple("${header.foo} =~ 'foo'")

// here Camel will type convert '100' into the type of header.bar and if it is an Integer '100' will also be
// converted to an Integer
simple("${header.bar} == '100'")

simple("${header.bar} == 100")

// 100 will be converted to the type of header.bar so we can do > comparison
simple("${header.bar} > 100")
```

126.5.1. Comparing with different types

When you compare with different types such as String and int, then you have to take a bit care. Camel will use the type from the left hand side as 1st priority. And fallback to the right hand side type if both values couldn't be compared based on that type.

This means you can flip the values to enforce a specific type. Suppose the bar value above is a String. Then you can flip the equation:

```
simple("100 < ${header.bar}")
```

which then ensures the int type is used as 1st priority.

This may change in the future if the Camel team improves the binary comparison operations to prefer numeric types to String based. It's most often the String type which causes problem when comparing with numbers.

```
// testing for null
simple("${header.baz} == null")

// testing for not null
simple("${header.baz} != null")
```

And a bit more advanced example where the right value is another expression

```
simple("${header.date} == ${date:now:yyyyMMdd}")

simple("${header.type} == ${bean:orderService?method=getOrderType}")
```

And an example with contains, testing if the title contains the word Camel

```
simple("${header.title} contains 'Camel'")
```

And an example with regex, testing if the number header is a 4 digit value:

```
simple("${header.number} regex '\\d{4}')
```

And finally an example if the header equals any of the values in the list. Each element must be separated by comma, and no space around.

This also works for numbers etc, as Camel will convert each element into the type of the left hand side.

```
simple("${header.type} in 'gold,silver'")
```

And for all the last 3 we also support the negate test using not:

```
simple("${header.type} !in 'gold,silver'")
```

And you can test if the type is a certain instance, eg for instance a String

```
simple("${header.type} is 'java.lang.String'")
```

We have added a shorthand for all **java.lang** types so you can write it as:

```
simple("${header.type} is 'String'")
```

Ranges are also supported. The range interval requires numbers and both from and end are inclusive. For instance to test whether a value is between 100 and 199:

```
simple("${header.number} range 100..199")
```

Notice we use `..` in the range without spaces. It is based on the same syntax as Groovy.

```
simple("${header.number} range '100..199'")
```

As the XML DSL does not have all the power as the Java DSL with all its various builder methods, you have to resort to use some other languages for testing with simple operators. Now you can do this with the simple language. In the sample below we want to test if the header is a widget order:

```
<from uri="seda:orders">
  <filter>
    <simple>${header.type} == 'widget'</simple>
    <to uri="bean:orderService?method=handleWidget"/>
  </filter>
</from>
```

126.5.2. Using and / or

If you have two expressions you can combine them with the **&&** or **||** operator.

For instance:

```
simple("${header.title} contains 'Camel' && ${header.type} == 'gold'")
```

And of course the **||** is also supported. The sample would be:

```
simple("${header.title} contains 'Camel' || ${header.type} == 'gold'")
```

126.6. EXAMPLES

In the XML DSL sample below we filter based on a header value:

```
<from uri="seda:orders">
  <filter>
    <simple>${header.foo}</simple>
    <to uri="mock:fooOrders"/>
  </filter>
</from>
```

The Simple language can be used for the predicate test above in the Message Filter pattern, where we test if the in message has a **foo** header (a header with the key **foo** exists). If the expression evaluates to **true** then the message is routed to the **mock:fooOrders** endpoint, otherwise the message is dropped.

The same example in Java DSL:

```
from("seda:orders")
  .filter().simple("${header.foo}")
  .to("seda:fooOrders");
```

You can also use the simple language for simple text concatenations such as:

```
from("direct:hello")
  .transform().simple("Hello ${header.user} how are you?")
  .to("mock:reply");
```

Notice that we must use `${}` placeholders in the expression now to allow Camel to parse it correctly.

And this sample uses the date command to output current date.

```
from("direct:hello")
  .transform().simple("The today is ${date:now:yyyyMMdd} and it is a great day.")
  .to("mock:reply");
```

And in the sample below we invoke the bean language to invoke a method on a bean to be included in the returned string:

```
from("direct:order")
  .transform().simple("OrderId: ${bean:orderIdGenerator}")
  .to("mock:reply");
```

Where **orderIdGenerator** is the id of the bean registered in the Registry. If using Spring then it is the Spring bean id.

If we want to declare which method to invoke on the order id generator bean we must prepend **.methodName** such as below where we invoke the **generateId** method.

```
from("direct:order")
  .transform().simple("OrderId: ${bean:orderIdGenerator.generateId}")
  .to("mock:reply");
```

We can use the **?method=methodName** option that we are familiar with the Bean component itself:

```
from("direct:order")
  .transform().simple("OrderId: ${bean:orderIdGenerator?method=generateId}")
  .to("mock:reply");
```

You can also convert the body to a given type, for example to ensure that it is a String you can do:

```
<transform>
  <simple>Hello ${bodyAs(String)} how are you?</simple>
</transform>
```

There are a few types which have a shorthand notation, so we can use **String** instead of **java.lang.String**. These are: **byte[]**, **String**, **Integer**, **Long**. All other types must use their FQN name, e.g. **org.w3c.dom.Document**.

It is also possible to lookup a value from a header **Map**:

```
<transform>
  <simple>The gold value is ${header.type[gold]}</simple>
</transform>
```

In the code above we lookup the header with name **type** and regard it as a **java.util.Map** and we then lookup with the key **gold** and return the value. If the header is not convertible to Map an exception is thrown. If the header with name **type** does not exist **null** is returned.

You can nest functions, such as shown below:

```
<setHeader name="myHeader">
  <simple>${properties:${header.someKey}}</simple>
</setHeader>
```

126.7. SETTING RESULT TYPE

You can now provide a result type to the [Simple](#) expression, which means the result of the evaluation will be converted to the desired type. This is most usable to define types such as booleans, integers, etc.

For example to set a header as a boolean type you can do:

```
.setHeader("cool", simple("true", Boolean.class))
```

And in XML DSL

```
<setHeader name="cool">
  <!-- use resultType to indicate that the type should be a java.lang.Boolean -->
  <simple resultType="java.lang.Boolean">true</simple>
</setHeader>
```

126.8. USING NEW LINES OR TABS IN XML DSLS

It is easier to specify new lines or tabs in XML DSLs as you can escape the value now

```
<transform>
  <simple>The following text\nis on a new line</simple>
```

```
</transform>
```

126.9. LEADING AND TRAILING WHITESPACE HANDLING

The trim attribute of the expression can be used to control whether the leading and trailing whitespace characters are removed or preserved. The default value is true, which removes the whitespace characters.

```
<setBody>
  <simple trim="false">You get some trailing whitespace characters. </simple>
</setBody>
```

126.10. LOADING SCRIPT FROM EXTERNAL RESOURCE

You can externalize the script and have Camel load it from a resource such as **"classpath:"**, **"file:"**, or **"http:"**. This is done using the following syntax: **"resource:scheme:location"**, e.g. to refer to a file on the classpath you can do:

```
.setHeader("myHeader").simple("resource:classpath:mysimple.txt")
```

126.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 147 options, which are listed below.

Name	Description	Default	Type
camel.cloud.consul.service-discovery.acl-token	Sets the ACL token to be used with Consul.		String
camel.cloud.consul.service-discovery.block-seconds	The seconds to wait for a watch event, default 10 seconds.	10	Integer
camel.cloud.consul.service-discovery.configurations	Define additional configuration definitions.		Map
camel.cloud.consul.service-discovery.connect-timeout-millis	Connect timeout for OkHttpClient.		Long

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.datacenter</code>	The data center.		String
<code>camel.cloud.consul.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.consul.service-discovery.password</code>	Sets the password to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.consul.service-discovery.read-timeout-millis</code>	Read timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.url</code>	The Consul agent URL.		String
<code>camel.cloud.consul.service-discovery.username</code>	Sets the username to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.write-timeout-millis</code>	Write timeout for OkHttpClient.		Long
<code>camel.cloud.dns.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.dns.service-discovery.domain</code>	The domain name;.		String

Name	Description	Default	Type
<code>camel.cloud.dns.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.dns.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.dns.service-discovery.proto</code>	The transport protocol of the desired service.	<code>_tcp</code>	String
<code>camel.cloud.etcd.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.etcd.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.etcd.service-discovery.password</code>	The password to use for basic authentication.		String
<code>camel.cloud.etcd.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.etcd.service-discovery.service-path</code>	The path to look for for service discovery.	<code>/services/</code>	String
<code>camel.cloud.etcd.service-discovery.timeout</code>	To set the maximum time an action could take to complete.		Long
<code>camel.cloud.etcd.service-discovery.type</code>	To set the discovery type, valid values are on-demand and watch.	on-demand	String

Name	Description	Default	Type
<code>camel.cloud.etcd.service-discovery.uris</code>	The URIs the client can connect to.		String
<code>camel.cloud.etcd.service-discovery.user-name</code>	The user name to use for basic authentication.		String
<code>camel.cloud.kubernetes.service-discovery.api-version</code>	Sets the API version when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-data</code>	Sets the Certificate Authority data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-file</code>	Sets the Certificate Authority data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-data</code>	Sets the Client Certificate data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-file</code>	Sets the Client Certificate data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-algo</code>	Sets the Client Keystore algorithm, such as RSA when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-data</code>	Sets the Client Keystore data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-file</code>	Sets the Client Keystore data that are loaded from the file when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.client-key-passphrase</code>	Sets the Client Keystore passphrase when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.kubernetes.service-discovery.dns-domain</code>	Sets the DNS domain to use for DNS lookup.		String
<code>camel.cloud.kubernetes.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.kubernetes.service-discovery.lookup</code>	How to perform service lookup. Possible values: client, dns, environment. When using client, then the client queries the kubernetes master to obtain a list of active pods that provides the service, and then random (or round robin) select a pod. When using dns the service name is resolved as name.namespace.svc.dnsDomain. When using dnssrv the service name is resolved with SRV query forsvc... When using environment then environment variables are used to lookup the service. By default environment is used.	environment	String
<code>camel.cloud.kubernetes.service-discovery.master-url</code>	Sets the URL to the master when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.namespace</code>	Sets the namespace to use. Will by default use namespace from the ENV variable KUBERNETES_MASTER.		String
<code>camel.cloud.kubernetes.service-discovery.oauth-token</code>	Sets the OAUTH token for authentication (instead of username/password) when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.password</code>	Sets the password for authentication when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-name</code>	Sets the Port Name to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-protocol</code>	Sets the Port Protocol to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.kubernetes.service-discovery.trust-certs</code>	Sets whether to turn on trust certificate check when using client lookup.	false	Boolean
<code>camel.cloud.kubernetes.service-discovery.username</code>	Sets the username for authentication when using client lookup.		String
<code>camel.cloud.ribbon.load-balancer.client-name</code>	Sets the Ribbon client name.		String
<code>camel.cloud.ribbon.load-balancer.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.ribbon.load-balancer.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
<code>camel.cloud.ribbon.load-balancer.namespace</code>	The namespace.		String
<code>camel.cloud.ribbon.load-balancer.password</code>	The password.		String
<code>camel.cloud.ribbon.load-balancer.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.ribbon.load-balancer.username</code>	The username.		String
<code>camel.hystrix.allow-maximum-size-to-diverge-from-core-size</code>	Allows the configuration for <code>maximumSize</code> to take effect. That value can then be equal to, or higher, than <code>coreSize</code> .	false	Boolean
<code>camel.hystrix.circuit-breaker-enabled</code>	Whether to use a <code>HystrixCircuitBreaker</code> or not. If false no circuit-breaker logic will be used and all requests permitted. This is similar in effect to <code>circuitBreakerForceClosed()</code> except that continues tracking metrics and knowing whether it should be open/closed, this property results in not even instantiating a circuit-breaker.	true	Boolean
<code>camel.hystrix.circuit-breaker-error-threshold-percentage</code>	Error percentage threshold (as whole number such as 50) at which point the circuit breaker will trip open and reject requests. It will stay tripped for the duration defined in <code>circuitBreakerSleepWindowInMilliseconds</code> ; The error percentage this is compared against comes from <code>HystrixCommandMetrics.getHealthCounts()</code> .	50	Integer
<code>camel.hystrix.circuit-breaker-force-closed</code>	If true the <code>HystrixCircuitBreaker#allowRequest()</code> will always return true to allow requests regardless of the error percentage from <code>HystrixCommandMetrics.getHealthCounts()</code> . The <code>circuitBreakerForceOpen()</code> property takes precedence so if it set to true this property does nothing.	false	Boolean

Name	Description	Default	Type
camel.hystrix.circuit-breaker-force-open	If true the <code>HystrixCircuitBreaker.allowRequest()</code> will always return false, causing the circuit to be open (tripped) and reject all requests. This property takes precedence over <code>circuitBreakerForceClosed()</code> ;	false	Boolean
camel.hystrix.circuit-breaker-request-volume-threshold	Minimum number of requests in the <code>metricsRollingStatisticalWindowInMilliseconds()</code> that must exist before the <code>HystrixCircuitBreaker</code> will trip. If below this number the circuit will not trip regardless of error percentage.	20	Integer
camel.hystrix.circuit-breaker-sleep-window-in-milliseconds	The time in milliseconds after a <code>HystrixCircuitBreaker</code> trips open that it should wait before trying requests again.	5000	Integer
camel.hystrix.configurations	Define additional configuration definitions.		Map
camel.hystrix.core-pool-size	Core thread-pool size that gets passed to <code>java.util.concurrent.ThreadPoolExecutor#setCorePoolSize(int)</code> .	10	Integer
camel.hystrix.enabled	Enable the component.	true	Boolean
camel.hystrix.execution-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.run()</code> . Requests beyond the concurrent limit will be rejected. Applicable only when <code>executionIsolationStrategy == SEMAPHORE</code> .	20	Integer
camel.hystrix.execution-isolation-strategy	What isolation strategy <code>HystrixCommand.run()</code> will be executed with. If <code>THREAD</code> then it will be executed on a separate thread and concurrent requests limited by the number of threads in the thread-pool. If <code>SEMAPHORE</code> then it will be executed on the calling thread and concurrent requests limited by the semaphore count.	THREAD	String
camel.hystrix.execution-isolation-thread-interrupt-on-timeout	Whether the execution thread should attempt an interrupt (using <code>Future#cancel</code>) when a thread times out. Applicable only when <code>executionIsolationStrategy() == THREAD</code> .	true	Boolean

Name	Description	Default	Type
camel.hystrix.execution-timeout-enabled	Whether the timeout mechanism is enabled for this command.	true	Boolean
camel.hystrix.execution-timeout-in-milliseconds	Time in milliseconds at which point the command will timeout and halt execution. If <code>executionIsolationThreadInterruptOnTimeout == true</code> and the command is thread-isolated, the executing thread will be interrupted. If the command is semaphore-isolated and a <code>HystrixObservableCommand</code> , that command will get unsubscribed.	1000	Integer
camel.hystrix.fallback-enabled	Whether <code>HystrixCommand.getFallback()</code> should be attempted when failure occurs.	true	Boolean
camel.hystrix.fallback-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.getFallback()</code> . Requests beyond the concurrent limit will fail-fast and not attempt retrieving a fallback.	10	Integer
camel.hystrix.group-key	Sets the group key to use. The default value is <code>CamelHystrix</code> .	CamelHystrix	String
camel.hystrix.keep-alive-time	Keep-alive time in minutes that gets passed to <code>ThreadPoolExecutor#setKeepAliveTime(long, TimeUnit)</code> .	1	Integer
camel.hystrix.max-queue-size	Max queue size that gets passed to <code>BlockingQueue</code> in <code>HystrixConcurrencyStrategy.getBlockingQueue(int)</code> . This should only affect the instantiation of a threadpool - it is not eligible to change a queue size on the fly. For that, use <code>queueSizeRejectionThreshold()</code> .	-1	Integer
camel.hystrix.maximum-size	Maximum thread-pool size that gets passed to <code>ThreadPoolExecutor#setMaximumPoolSize(int)</code> . This is the maximum amount of concurrency that can be supported without starting to reject <code>HystrixCommands</code> . Please note that this setting only takes effect if you also set <code>allowMaximumSizeToDivergeFromCoreSize</code> .	10	Integer

Name	Description	Default	Type
camel.hystrix.metrics-health-snapshot-interval-in-milliseconds	Time in milliseconds to wait between allowing health snapshots to be taken that calculate success and error percentages and affect <code>HystrixCircuitBreaker.isOpen()</code> status. On high-volume circuits the continual calculation of error percentage can become CPU intensive thus this controls how often it is calculated.	500	Integer
camel.hystrix.metrics-rolling-percentile-bucket-size	Maximum number of values stored in each bucket of the rolling percentile. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-percentile-enabled	Whether percentile metrics should be captured using <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	true	Boolean
camel.hystrix.metrics-rolling-percentile-window-buckets	Number of buckets the rolling percentile window is broken into. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	6	Integer
camel.hystrix.metrics-rolling-percentile-window-in-milliseconds	Duration of percentile rolling window in milliseconds. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10000	Integer
camel.hystrix.metrics-rolling-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into <code>HystrixRollingNumber</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-statistical-window-in-milliseconds	This property sets the duration of the statistical rolling window, in milliseconds. This is how long metrics are kept for the thread pool. The window is divided into buckets and rolls by those increments.	10000	Integer

Name	Description	Default	Type
camel.hystrix.queue-size-rejection-threshold	Queue size rejection threshold is an artificial max size at which rejections will occur even if maxQueueSize has not been reached. This is done because the maxQueueSize of a BlockingQueue can not be dynamically changed and we want to support dynamically changing the queue size that affects rejections. This is used by HystrixCommand when queuing a thread for execution.	5	Integer
camel.hystrix.request-log-enabled	Whether HystrixCommand execution and events should be logged to HystrixRequestLog.	true	Boolean
camel.hystrix.thread-pool-key	Sets the thread pool key to use. Will by default use the same value as groupKey has been configured to use.	Camel Hystrix	String
camel.hystrix.thread-pool-rolling-number-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10	Integer
camel.hystrix.thread-pool-rolling-number-statistical-window-in-milliseconds	Duration of statistical rolling window in milliseconds. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10000	Integer
camel.language.constant.enabled	Whether to enable auto configuration of the constant language. This is enabled by default.		Boolean
camel.language.constant.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.language.csimple.enabled	Whether to enable auto configuration of the csimple language. This is enabled by default.		Boolean
camel.language.csimple.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.language.exchangeproperty.enabled	Whether to enable auto configuration of the exchangeProperty language. This is enabled by default.		Boolean
camel.language.exchangeproperty.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

Name	Description	Default	Type
<code>camel.language.file.enabled</code>	Whether to enable auto configuration of the file language. This is enabled by default.		Boolean
<code>camel.language.file.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.header.enabled</code>	Whether to enable auto configuration of the header language. This is enabled by default.		Boolean
<code>camel.language.header.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.ref.enabled</code>	Whether to enable auto configuration of the ref language. This is enabled by default.		Boolean
<code>camel.language.ref.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.simple.enabled</code>	Whether to enable auto configuration of the simple language. This is enabled by default.		Boolean
<code>camel.language.simple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.tokenize.enabled</code>	Whether to enable auto configuration of the tokenize language. This is enabled by default.		Boolean
<code>camel.language.tokenize.group-delimiter</code>	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.		String
<code>camel.language.tokenize.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.resilience4j.automatic-transition-from-open-to-half-open-enabled</code>	Enables automatic transition from OPEN to HALF_OPEN state once the <code>waitDurationInOpenState</code> has passed.	false	Boolean
<code>camel.resilience4j.circuit-breaker-ref</code>	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreaker</code> instance to lookup and use from the registry. When using this, then any other circuit breaker options are not in use.		String

Name	Description	Default	Type
camel.resilience4j.config-ref	Refers to an existing <code>io.github.resilience4j.circuitbreaker.CircuitBreakerConfig</code> instance to lookup and use from the registry.		String
camel.resilience4j.configurations	Define additional configuration definitions.		Map
camel.resilience4j.enabled	Enable the component.	true	Boolean
camel.resilience4j.failure-rate-threshold	Configures the failure rate threshold in percentage. If the failure rate is equal or greater than the threshold the <code>CircuitBreaker</code> transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 50 percentage.		Float
camel.resilience4j.minimum-number-of-calls	Configures the minimum number of calls which are required (per sliding window period) before the <code>CircuitBreaker</code> can calculate the error rate. For example, if <code>minimumNumberOfCalls</code> is 10, then at least 10 calls must be recorded, before the failure rate can be calculated. If only 9 calls have been recorded the <code>CircuitBreaker</code> will not transition to open even if all 9 calls have failed. Default <code>minimumNumberOfCalls</code> is 100.	100	Integer
camel.resilience4j.permitted-number-of-calls-in-half-open-state	Configures the number of permitted calls when the <code>CircuitBreaker</code> is half open. The size must be greater than 0. Default size is 10.	10	Integer

Name	Description	Default	Type
camel.resilience4j.sliding-window-size	Configures the size of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. slidingWindowSize configures the size of the sliding window. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. The slidingWindowSize must be greater than 0. The minimumNumberOfCalls must be greater than 0. If the slidingWindowType is COUNT_BASED, the minimumNumberOfCalls cannot be greater than slidingWindowSize. If the slidingWindowType is TIME_BASED, you can pick whatever you want. Default slidingWindowSize is 100.	100	Integer
camel.resilience4j.sliding-window-type	Configures the type of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. Default slidingWindowType is COUNT_BASED.	COUNT_BASED	String
camel.resilience4j.slow-call-duration-threshold	Configures the duration threshold (seconds) above which calls are considered as slow and increase the slow calls percentage. Default value is 60 seconds.	60	Integer
camel.resilience4j.slow-call-rate-threshold	Configures a threshold in percentage. The CircuitBreaker considers a call as slow when the call duration is greater than slowCallDurationThreshold Duration. When the percentage of slow calls is equal or greater the threshold, the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 100 percentage which means that all recorded calls must be slower than slowCallDurationThreshold.		Float
camel.resilience4j.wait-duration-in-open-state	Configures the wait duration (in seconds) which specifies how long the CircuitBreaker should stay open, before it switches to half open. Default value is 60 seconds.	60	Integer

Name	Description	Default	Type
camel.resilience4j.writable-stack-trace-enabled	Enables writable stack traces. When set to false, <code>Exception.getStackTrace</code> returns a zero length array. This may be used to reduce log spam when the circuit breaker is open as the cause of the exceptions is already known (the circuit breaker is short-circuiting calls).	true	Boolean
camel.rest.api-component	The name of the Camel component to use as the REST API (such as swagger) If no API Component has been explicit configured, then Camel will lookup if there is a Camel component responsible for servicing and generating the REST API documentation, or if a <code>org.apache.camel.spi.RestApiProcessorFactory</code> is registered in the registry. If either one is found, then that is being used.		String
camel.rest.api-context-path	Sets a leading API context-path the REST API services will be using. This can be used when using components such as <code>camel-servlet</code> where the deployed web application is deployed using a context-path.		String
camel.rest.api-context-route-id	Sets the route id to use for the route that services the REST API. The route will by default use an auto assigned route id.		String
camel.rest.api-host	To use an specific hostname for the API documentation (eg swagger) This can be used to override the generated host with this configured hostname.		String
camel.rest.api-property	Allows to configure as many additional properties for the api documentation (swagger). For example set property <code>api.title</code> to my cool stuff.		Map
camel.rest.api-vendor-extension	Whether vendor extension is enabled in the Rest APIs. If enabled then Camel will include additional information as vendor extension (eg keys starting with <code>x-</code>) such as route ids, class names etc. Not all 3rd party API gateways and tools supports vendor-extensions when importing your API docs.	false	Boolean
camel.rest.binding-mode	Sets the binding mode to use. The default value is off.		RestBindingMode

Name	Description	Default	Type
camel.rest.client-request-validation	Whether to enable validation of the client request to check whether the Content-Type and Accept headers from the client is supported by the Rest-DSL configuration of its consumes/produces settings. This can be turned on, to enable this check. In case of validation error, then HTTP Status codes 415 or 406 is returned. The default value is false.	false	Boolean
camel.rest.component	The Camel Rest component to use for the REST transport (consumer), such as netty-http, jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String
camel.rest.component-property	Allows to configure as many additional properties for the rest component in use.		Map
camel.rest.consumer-property	Allows to configure as many additional properties for the rest consumer in use.		Map
camel.rest.context-path	Sets a leading context-path the REST services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path. Or for components such as camel-jetty or camel-netty-http that includes a HTTP server.		String
camel.rest.cors-headers	Allows to configure custom CORS headers.		Map
camel.rest.data-format-property	Allows to configure as many additional properties for the data formats in use. For example set property prettyPrint to true to have json outputted in pretty mode. The properties can be prefixed to denote the option is only for either JSON or XML and for either the IN or the OUT. The prefixes are: json.in. json.out. xml.in. xml.out. For example a key with value xml.out.mustBeJAXBELEMENT is only for the XML data format for the outgoing. A key without a prefix is a common key for all situations.		Map
camel.rest.enable-cors	Whether to enable CORS headers in the HTTP response. The default value is false.	false	Boolean

Name	Description	Default	Type
camel.rest.endpoint-property	Allows to configure as many additional properties for the rest endpoint in use.		Map
camel.rest.host	The hostname to use for exposing the REST service.		String
camel.rest.hostname-resolver	If no hostname has been explicit configured, then this resolver is used to compute the hostname the REST service will be using.		RestHostNameResolver
camel.rest.json-data-format	Name of specific json data format to use. By default json-jackson will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.port	The port number to use for exposing the REST service. Notice if you use servlet component then the port number configured here does not apply, as the port number in use is the actual port number the servlet component is using. eg if using Apache Tomcat its the tomcat http port, if using Apache Karaf its the HTTP service in Karaf that uses port 8181 by default etc. Though in those situations setting the port number here, allows tooling and JMX to know the port number, so its recommended to set the port number to the number that the servlet engine uses.		String
camel.rest.producer-api-doc	Sets the location of the api document (swagger api) the REST producer will use to validate the REST uri and query parameters are valid accordingly to the api document. This requires adding camel-swagger-java to the classpath, and any miss configuration will let Camel fail on startup and report the error(s). The location of the api document is loaded from classpath by default, but you can use file: or http: to refer to resources to load from file or http url.		String
camel.rest.producer-component	Sets the name of the Camel component to use as the REST producer.		String
camel.rest.scheme	The scheme to use for exposing the REST service. Usually http or https is supported. The default value is http.		String
camel.rest.skip-binding-on-error-code	Whether to skip binding on output if there is a custom HTTP error code header. This allows to build custom error messages that do not bind to json / xml etc, as success messages otherwise will do.	false	Boolean

Name	Description	Default	Type
<code>camel.rest.use-x-forward-headers</code>	Whether to use X-Forward headers for Host and related setting. The default value is true.	true	Boolean
<code>camel.rest.xml-data-format</code>	Name of specific XML data format to use. By default jaxb will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
<code>camel.rest.api-context-id-pattern</code>	Deprecated Sets an CamelContext id pattern to only allow Rest APIs from rest services within CamelContext's which name matches the pattern. The pattern name refers to the CamelContext name, to match on the current CamelContext only. For any other value, the pattern uses the rules from PatternHelper#matchPattern(String,String).		String
<code>camel.rest.api-context-listing</code>	Deprecated Sets whether listing of all available CamelContext's with REST services in the JVM is enabled. If enabled it allows to discover these contexts, if false then only the current CamelContext is in use.	false	Boolean

CHAPTER 127. SLACK

Both producer and consumer are supported

The Slack component allows you to connect to an instance of [Slack](#) and delivers a message contained in the message body via a pre established [Slack incoming webhook](#).

127.1. DEPENDENCIES

When using **slack** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>  
  <groupId>org.apache.camel.springboot</groupId>  
  <artifactId>camel-slack-starter</artifactId>  
</dependency>
```

127.2. URI FORMAT

To send a message to a channel.

```
slack:#channel[?options]
```

To send a direct message to a slackuser.

```
slack:@userID[?options]
```

127.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

127.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

127.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

127.4. COMPONENT OPTIONS

The Slack component supports 5 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
token (token)	The token to use.		String
webhookUrl (webhook)	The incoming webhook URL.		String

127.5. ENDPOINT OPTIONS

The Slack endpoint is configured using URI syntax:

```
slack:channel
```

with the following path and query parameters:

127.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
channel (common)	Required The channel name (syntax #name) or slackuser (syntax userName) to send a message directly to an user.		String

127.5.2. Query Parameters (29 parameters)

Name	Description	Default	Type
token (common)	The token to use.		String

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
conversationType (consumer)	Type of conversation. Enum values: • PUBLIC_CHANNEL • PRIVATE_CHANNEL • MPIM • IM	PUBLIC_CHANNEL	ConversationType
maxResults (consumer)	The Max Result for the poll.	10	String
naturalOrder (consumer)	Create exchanges in natural order (oldest to newest) or not.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
serverUrl (consumer)	The Server URL of the Slack instance.		String
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
iconEmoji (producer)	Deprecated Use a Slack emoji as an avatar.		String
iconUrl (producer)	Deprecated The avatar that the component will use when sending message to a channel or user.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
username (producer)	Deprecated This is the username that the bot will have when sending messages to a channel or user.		String
webhookUrl (producer)	The incoming webhook URL.		String
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int

Name	Description	Default	Type
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object

Name	Description	Default	Type
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean

127.6. CONFIGURING IN SPRINT XML

The Slack component with XML must be configured as a Spring or Blueprint bean that contains the incoming webhook url or the app token for the integration as a parameter.

```
<bean id="slack" class="org.apache.camel.component.slack.SlackComponent">
  <property name="webhookUrl"
value="https://hooks.slack.com/services/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"/>
  <property name="token" value="xoxb-12345678901-1234567890123-
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"/>
</bean>
```

For Java you can configure this using Java code.

127.7. EXAMPLE

A CamelContext with Blueprint could be as:

```
<?xml version="1.0" encoding="UTF-8"?>
<blueprint xmlns="http://www.osgi.org/xmlns/blueprint/v1.0.0" default-activation="lazy">

  <bean id="slack" class="org.apache.camel.component.slack.SlackComponent">
```

```

    <property name="webhookUrl"
value="https://hooks.slack.com/services/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"/>
    </bean>

    <camelContext xmlns="http://camel.apache.org/schema/blueprint">
        <route>
            <from uri="direct:test"/>
            <to uri="slack:#channel?iconEmoji=:camel:&username=CamelTest"/>
        </route>
    </camelContext>

</blueprint>

```

127.8. PRODUCER

You can now use a token to send a message instead of WebhookUrl.

```

from("direct:test")
    .to("slack:#random?token=RAW(<YOUR_TOKEN>)");

```

You can now use the Slack API model to create blocks. You can read more about it here <https://api.slack.com/block-kit>.

```

public void testSlackAPIModelMessage() {
    Message message = new Message();
    message.setBlocks(Collections.singletonList(SectionBlock
        .builder()
        .text(MarkdownTextObject
            .builder()
            .text("**Hello from Camel!**")
            .build())
        .build()));

    template.sendBody(test, message);
}

```

127.9. CONSUMER

You can use also a consumer for messages in channel.

```

from("slack://general?token=RAW(<YOUR_TOKEN>)&maxResults=1")
    .to("mock:result");

```

In this way you'll get the last message from general channel. The consumer will take track of the timestamp of the last message consumed and in the next poll it will check from that timestamp.

You'll need to create a Slack app and use it on your workspace.

Use the 'Bot User OAuth Access Token' as token for the consumer endpoint.

**NOTE**

Add the corresponding history (**channels:history** or **groups:history** or **mpim:history** or **im:history**) and read (**channels:read** or **groups:read** or **mpim:read** or **im:read**) user token scope to your app to grant it permission to view messages in the corresponding channel. You will need to use the `conversationType` option to set it up too (**PUBLIC_CHANNEL**, **PRIVATE_CHANNEL**, **MPIM**, **IM**)

The `naturalOrder` option allows consuming messages from the oldest to the newest. Originally you would get the newest first and consume backward (message 3 ⇒ message 2 ⇒ message 1)

**NOTE**

You can use the `conversationType` option to read history and messages from a channel that is not only public (**PUBLIC_CHANNEL**, **PRIVATE_CHANNEL**, **MPIM**, **IM**)

127.10. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.slack.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.slack.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.slack.enabled</code>	Whether to enable auto configuration of the slack component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.slack.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.slack.token</code>	The token to use.		String
<code>camel.component.slack.webhook-url</code>	The incoming webhook URL.		String

CHAPTER 128. SMB

Since Camel 4.3

Only consumer is supported.

The Server Message Block (SMB) component provides a way to connect natively to SMB file shares, such as those provided by Microsoft Windows or [Samba](#).

128.1. DEPENDENCIES

When using **camel-smb** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-smb-starter</artifactId>
</dependency>
```

128.2. URI FORMAT

```
smb:address[:port]/shareName[?options]
```

128.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

128.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

128.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

128.4. COMPONENT OPTIONS

The SMB component supports 2 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

128.5. ENDPOINT OPTIONS

The SMB endpoint is configured using URI syntax:

```
smb:hostname:port/shareName
```

With the following *path* and *query* parameters:

128.5.1. Path Parameters (3 parameters)

Name	Description	Default	Type
hostname (consumer)	Required The share hostname or IP address.		String
port (consumer)	The share port number.	445	int
shareName (consumer)	The name of the share to connect to.		String

128.6. QUERY PARAMETERS (26 PARAMETERS)

Name	Description	Default	Type
idempotentRepository (consumer)	A pluggable repository <code>org.apache.camel.spi.IdempotentRepository</code> which by default use <code>MemoryIdempotentRepository</code> if none is specified.		<code>IdempotentRepository</code>
path (consumer)	Required The path, within the share, to consume the files from.		String
searchPattern (consumer)	The search pattern used to list the files.	*.txt	String
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean

Name	Description	Default	Type
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
smbIoBean (advanced)	An optional SMB I/O bean to use to setup the file access attributes when reading/writing a file.		SmbIoBean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the <code>backoffMultiplier</code> should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the <code>backoffMultiplier</code> should kick-in.		int

Name	Description	Default	Type
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object

Name	Description	Default	Type
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANoseconds • MICROseconds • MILLIseconds • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
domain (security)	The user domain.		String
password (security)	The password to access the share.		String
username (security)	The username required to access the share.		String

128.7. EXAMPLES

Following example shows how to poll all the files from an SMB file share and read the contents of these files:

```
private void process(Exchange exchange) throws IOException {
    final File file = exchange.getMessage().getBody(File.class);
    try (InputStream inputStream = file.getInputStream()) {
        LOG.debug("Read exchange: {}, with contents: {}", file.getFileInformation(), new
String(inputStream.readAllBytes()));
    }
}

public void configure() {
```

```
fromF("smb:%s/%s?username=%s&password=%s&path=", service.address(),  
service.shareName(), service.userName(), service.password())  
    .process(this::process)  
    .to("mock:result");  
}
```



NOTE

The File object provided is not a **java.io.File** instance, but, instead a **com.hierynomus.smbj.share.File** instance.

CHAPTER 129. SMOOKS COMPONENT

Since Camel 4.7

Only producer is supported

The Camel Smooks component uses [Smooks](#) to break up the structured data (EDI, CSV, POJO, and so on) of a Camel message body into fragments. These fragments can be processed independently of one another from within Smooks.

Common applications of Smooks include:

- transformation: EDI to CSV, POJO to EDI, POJO to XML, and so on.
- scalable processing: process huge payloads while keeping a small memory footprint.
- split, transform, and route fragments to destinations such as JMS queues, file systems, and databases.
- enrichment: enrich fragments with data from a database or other data sources.
- Java binding: populate POJOs from a source such as CSV, EDI, XML, and other POJOs.

Use the [Smooks Data Format](#) instead of this component when you are primarily interested in transformation and binding. This should be used for other Smooks features, like routing.

129.1. DEPENDENCIES

When using **camel-smooks** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-smooks-starter</artifactId>
</dependency>
```

129.2. URI FORMAT

```
smooks://smooks-config-path[?options]
```

129.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

129.3.1. Configuring component options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

129.3.2. Configuring endpoint options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

129.4. COMPONENT OPTIONS

The Smooks component supports 3 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
smooksFactory (advanced)	Autowired To use a custom factory for creating Smooks.		SmooksFactory

129.5. ENDPOINT OPTIONS

The Smooks endpoint is configured using URI syntax:

```
smooks:smooksConfig
```

With the following *path* and *query* parameters:

129.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
smooksConfig (producer)	Required Path to the Smooks configuration file.		String

129.5.2. Query Parameters (3 parameters)

Name	Description	Default	Type
reportPath (producer)	File path to place the generated HTML execution report. The report is a useful tool in the developers arsenal for diagnosing issues or comprehending a transformation. Do not set in production since this is a major performance drain.		String

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
allowExecutionContextFromHeader (advanced)	Allow execution context to be set from the CamelSmooksExecutionContext header.	false	Boolean

129.6. MESSAGE HEADERS

The Smooks component supports 1 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelSmooksExecutionContext (advanced) Constant: SMOOKS_EXECUTION_CONTEXT	The Smooks execution context.		ExecutionContext

129.7. USAGE

Using the Smooks component lets you leverage all the features of Smooks, such as transformation and fragment-driven routing, from within Camel. You can take an existing Smooks configuration and reference it from your Camel routes as shown below:

Java

```
from("file:inputDir?noop=true")
  .to("smooks:smooks-config.xml")
  .to("jms:queue:order");
```

YAML

```
- from:
  uri: file:inputDir?noop=true
  steps:
    - to: smooks:smooks-config.xml
    - to: jms:queue:order
```

The Smooks component is configured with a mandatory configuration file, which is **smooks-config.xml** in the example above. It is not clear what type of output the component is producing from looking at the above route. By default, the message body output is a stream but the type of output can be changed by configuring **{<https://www.smooks.org/xsd/smooks/smooks-core-1.6.xsd>}exports** in the Smooks configuration as shown next:

smooks-config.xml

```
<smooks-resource-list xmlns="https://www.smooks.org/xsd/smooks-2.0.xsd"
  xmlns:edi="https://www.smooks.org/xsd/smooks/edi-2.0.xsd"
  xmlns:core="https://www.smooks.org/xsd/smooks/smooks-core-1.6.xsd">

  <core:exports>
    <core:result type="org.smooks.io.sink.StringSink"/>
  </core:exports>

  <edi:parser schemaUri="/edi-mapping-model.dfdl.xsd" segmentTerminator="%NL;"
dataElementSeparator="*"
    compositeDataElementSeparator="^"/>

</smooks-resource-list>
```

The **{<https://www.smooks.org/xsd/smooks/smooks-core-1.6.xsd>}exports** element in this example configures Smooks to export the execution result to Camel as a string. Keep in mind that exporting the result as string means that the whole result will be kept in-memory which could cause unexpected performance issues for large payloads.

129.7.1. Bean routing

Smooks is capable of routing fragments to Camel endpoints using the **{<https://www.smooks.org/xsd/smooks/camel-1.5.xsd>}route** element from the Smooks configuration. As an example, you can route to an endpoint by declaring the following in your Smooks configuration:

smooks-config.xml

```
<smooks-resource-list xmlns="https://www.smooks.org/xsd/smooks-2.0.xsd"
  xmlns:jb="https://www.smooks.org/xsd/smooks/javabean-1.6.xsd"
  xmlns:camel="https://www.smooks.org/xsd/smooks/camel-1.5.xsd">
```

```

<!-- Create some bean instances from the input source... -->
<jb:bean beanId="orderItem" ...>
  <!-- etc... See Smooks Java Binding docs -->
</jb:bean>

<!-- Route bean to camel endpoints... -->
<camel:route beanId="orderItem">
  <camel:to endpoint="direct:slow" if="orderItem.priority = 'Normal'"/>
  <camel:to endpoint="direct:express" if="orderItem.priority = 'High'"/>
</camel:route>

</smooks-resource-list>

```

The file above configures Smooks to route the Java bean **orderItem** in the **bean context** to the endpoints **direct:slow** and **direct:express**, depending on whether the **priority** field of the **orderItem** instance is equal to **Normal** or **High**. It is possible to route based on an event selector rather than a condition thanks to the **routeOnElement** XML attribute:

smooks-config.xml

```

<smooks-resource-list xmlns="https://www.smooks.org/xsd/smooks-2.0.xsd"
  xmlns:jb="https://www.smooks.org/xsd/smooks/javabean-1.6.xsd"
  xmlns:camel="https://www.smooks.org/xsd/smooks/camel-1.5.xsd">

  <!-- Create some bean instances from the input source... -->
  <jb:bean beanId="orderItem" ...>
    <!-- etc... See Smooks Java Binding docs -->
  </jb:bean>

  <!-- Route bean to camel endpoints... -->
  <camel:route beanId="orderItem" routeOnElement="order">
    <camel:to endpoint="direct:all"/>
  </camel:route>

</smooks-resource-list>

```



NOTE

Instead of routing complex objects to Camel, a **pipeline** allows you to have a template (e.g., FreeMarker) reference the beans and then route the evaluated template as string (e.g., XML, CSV, etc.) to Camel.

CHAPTER 130. SMOOKS DATA FORMAT

The Smooks Data Format uses [Smooks](#) to transform from one data format to another and back again. A configuration for a Smooks Data Format should not allocate system resources because the data format does not close those resources. Use this data format when you are primarily interested in transformation and binding. For other Smooks features like routing use the [Smooks](#) component starter.

130.1. DEPENDENCIES

When using **camel-smooks** component starter with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-smooks-starter</artifactId>
</dependency>
```

130.2. USAGE

Below is an example of using the Smooks Data Format to unmarshal a CSV into a **java.util.List** of **org.smooks.example.Customer** instances:

Java

```
from("direct:unmarshal")
  .unmarshal().smooks("csv-smooks-unmarshal-config.xml")
  .log("Unmarshalled customers: ${body}");
```

YAML

```
- from:
  uri: direct:unmarshal
  steps:
    - unmarshal:
      smooks:
        smooksConfig: csv-smooks-unmarshal-config.xml
    - log: "Unmarshalled customers: ${body}"
```

The Smooks configuration in **csv-smooks-unmarshal-config.xml** is as follows:

```
<smooks-resource-list xmlns="https://www.smooks.org/xsd/smooks-2.0.xsd"
  xmlns:core="https://www.smooks.org/xsd/smooks/smooks-core-1.6.xsd"
  xmlns:csv="https://www.smooks.org/xsd/smooks/csv-1.7.xsd">

  <core:exports>
    <core:result type="org.smooks.io.sink.JavaSink" extract="result"/>
  </core:exports>

  <csv:reader fields="firstName,lastName,gender,age,country">
    <csv:listBinding beanId="result" class="org.smooks.example.Customer"/>
  </csv:reader>
</smooks-resource-list>
```

</csv:reader>

</smooks-resource-list>

130.3. SMOOKS DATA FORMAT OPTIONS

The Smooks dataformat supports 1 option:

Name	Description	Default	Java Type
smooksConfig (common)a	Required Path to the Smooks configuration file.		String

130.4. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
camel.component.smooks.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.smooks.enabled	Whether to enable auto configuration of the smooks component. This is enabled by default.		Boolean
camel.component.smooks.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.smooks.smooks-factory	To use a custom factory for creating Smooks. The option is a org.smooks.SmooksFactory type.		SmooksFactory
camel.dataformat.smooks.enabled	Whether to enable auto configuration of the smooks data format. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.dataformat.smooks.smooks-config</code>	Path to the Smooks configuration file.		String

CHAPTER 131. SNMP

Since Camel 2.1

Both producer and consumer are supported

The SNMP component gives you the ability to poll SNMP capable devices or receiving traps

131.1. DEPENDENCIES

When using **snmp** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-snmpp-starter</artifactId>
</dependency>
```

131.2. URI FORMAT

```
snmp://hostname[:port][?Options]
```

The component supports polling OID values from an SNMP enabled device and receiving traps.

131.3. SNMP PRODUCER

It can also be used to request information using GET method. The response body type is **org.apache.camel.component.snmp.SnmpMessage**.

131.4. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

131.4.1. Configuring component options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties/yaml), or directly with Java code.

131.4.2. Configuring endpoint options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

131.5. COMPONENT OPTIONS

The SNMP component supports 5 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

Name	Description	Default	Type
healthCheckConsumerEnabled (health)	Used for enabling or disabling all consumer based health checks from this component.	true	boolean
healthCheckProducerEnabled (health)	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting camel.health.producersEnabled=true.	true	boolean

131.6. ENDPOINT OPTIONS

The SNMP endpoint is configured using URI syntax:

```
snmp:host:port
```

with the following path and query parameters:

131.6.1. Path Parameters (2 parameters)

Name	Description	Default	Type
host (common)	Required Hostname of the SNMP enabled device.		String
port (common)	Required Port number of the SNMP enabled device.		Integer

131.6.2. Query Parameters (36 parameters)

Name	Description	Default	Type
oids (common)	Defines which values you are interested in. Please have a look at the Wikipedia to get a better understanding. You may provide a single OID or a coma separated list of OIDs. Example: oids=1.3.6.1.2.1.1.3.0,1.3.6.1.2.1.25.3.2.1.5.1,1.3.6.1.2.1.25.3.5.1.1,1.3.6.1.2.1.43.5.1.1.1.1.		OIDList

Name	Description	Default	Type
protocol (common)	Here you can select which protocol to use. You can use either udp or tcp. Enum values: • tcp • udp	udp	String
retries (common)	Defines how often a retry is made before canceling the request.	2	int
snmpCommunity (common)	Sets the community octet string for the snmp request.	public	String
snmpContextEngineId (common)	Sets the context engine ID field of the scoped PDU.		String
snmpContextName (common)	Sets the context name field of this scoped PDU.		String
snmpVersion (common)	Sets the snmp version for the request. The value 0 means SNMPv1, 1 means SNMPv2c, and the value 3 means SNMPv3. Enum values: • 0 • 1 • 3	0	int
timeout (common)	Sets the timeout value for the request in millis.	1500	int
type (common)	Which operation to perform such as poll, trap, etc. Enum values: • TRAP • POLL • GET_NEXT		SnmpActionType
delay (consumer)	Sets update rate in seconds.	60000	long

Name	Description	Default	Type
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
treeList (consumer)	Sets the flag whether the scoped PDU will be displayed as the list if it has child elements in its tree.	false	boolean
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● <code>InOnly</code> ● <code>InOut</code>		<code>ExchangePattern</code>
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		<code>PollingConsumerPollStrategy</code>

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long

Name	Description	Default	Type
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: ● NANOSECONDS ● MICROSECONDS ● MILLISECONDS ● SECONDS ● MINUTES ● HOURS ● DAYS	MILLIS ECON DS	TimeUnit

Name	Description	Default	Type
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See <code>ScheduledExecutorService</code> in JDK for details.	true	boolean
authenticationPassphrase (security)	The authentication passphrase. If not null, <code>authenticationProtocol</code> must also be not null. RFC3414 11.2 requires passphrases to have a minimum length of 8 bytes. If the length of <code>authenticationPassphrase</code> is less than 8 bytes an <code>IllegalArgumentException</code> is thrown.		String
authenticationProtocol (security)	Authentication protocol to use if security level is set to enable authentication The possible values are: MD5, SHA1. Enum values: • MD5 • SHA1		String
privacyPassphrase (security)	The privacy passphrase. If not null, <code>privacyProtocol</code> must also be not null. RFC3414 11.2 requires passphrases to have a minimum length of 8 bytes. If the length of <code>authenticationPassphrase</code> is less than 8 bytes an <code>IllegalArgumentException</code> is thrown.		String
privacyProtocol (security)	The privacy protocol ID to be associated with this user. If set to null, this user only supports unencrypted messages.		String

Name	Description	Default	Type
securityLevel (security)	Sets the security level for this target. The supplied security level must be supported by the security model dependent information associated with the security name set for this target. The value 1 means: No authentication and no encryption. Anyone can create and read messages with this security level. The value 2 means: Authentication and no encryption. Only the one with the right authentication key can create messages with this security level, but anyone can read the contents of the message. The value 3 means: Authentication and encryption. Only the one with the right authentication key can create messages with this security level, and only the one with the right encryption/decryption key can read the contents of the message. Enum values: • 1 • 2 • 3	3	int
securityName (security)	Sets the security name to be used with this target.		String

131.7. THE RESULT OF A POLL

Given the situation, that I poll for the following OIDs:

OIDs

```
1.3.6.1.2.1.1.3.0
1.3.6.1.2.1.25.3.2.1.5.1
1.3.6.1.2.1.25.3.5.1.1.1
1.3.6.1.2.1.43.5.1.1.11.1
```

The result will be the following:

Result of toString conversion

```
<?xml version="1.0" encoding="UTF-8"?>
<snmp>
  <entry>
    <oid>1.3.6.1.2.1.1.3.0</oid>
    <value>6 days, 21:14:28.00</value>
  </entry>
  <entry>
    <oid>1.3.6.1.2.1.25.3.2.1.5.1</oid>
```

```

    <value>2</value>
  </entry>
  <entry>
    <oid>1.3.6.1.2.1.25.3.5.1.1.1</oid>
    <value>3</value>
  </entry>
  <entry>
    <oid>1.3.6.1.2.1.43.5.1.1.11.1</oid>
    <value>6</value>
  </entry>
  <entry>
    <oid>1.3.6.1.2.1.1.1.0</oid>
    <value>My Very Special Printer Of Brand Unknown</value>
  </entry>
</snmp>

```

As you maybe recognized there is one more result than requested....1.3.6.1.2.1.1.0.

This one is filled in by the device automatically in this special case. So it may absolutely happen, that you receive more than you requested...be prepared.

OID starting with dot representation

```
.1.3.6.1.4.1.6527.3.1.2.21.2.1.50
```

As you may notice, default snmpVersion is 0 which means **version1** in the endpoint if it is not set explicitly. Make sure you explicitly set snmpVersion which is not default value, of course in a case of where you are able to query SNMP tables with different versions. Other possible values are **version2c** and **version3**.

131.8. EXAMPLES

Polling a remote device:

```
snmp:192.168.178.23:161?protocol=udp&type=POLL&oids=1.3.6.1.2.1.1.5.0
```

Setting up a trap receiver (**Note that no OID info is needed here!**):

```
snmp:127.0.0.1:162?protocol=udp&type=TRAP
```

You can get the community of SNMP TRAP with message header 'securityName', peer address of the SNMP TRAP with message header 'peerAddress'.

Routing example in Java: (converts the SNMP PDU to XML String)

```

from("snmp:192.168.178.23:161?protocol=udp&type=POLL&oids=1.3.6.1.2.1.1.5.0").
  convertBodyTo(String.class).
  to("activemq:snmp.states");

```

131.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
camel.component.snmp.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.snmp.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.snmp.enabled	Whether to enable auto configuration of the snmp component. This is enabled by default.		Boolean
camel.component.snmp.health-check-consumer-enabled	Used for enabling or disabling all consumer based health checks from this component.	true	Boolean
camel.component.snmp.health-check-producer-enabled	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting <code>camel.health.producersEnabled=true</code> .	true	Boolean
camel.component.snmp.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 132. SOAP

SOAP is a Data Format which uses JAXB2 and JAX-WS annotations to marshal and unmarshal SOAP payloads. It provides the basic features of Apache CXF without need for the CXF Stack.

Namespace prefix mapping

See [JAXB](#) for details how you can control namespace prefix mappings when marshalling using SOAP data format.

132.1. DEPENDENCIES

When using **soap** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-soap-starter</artifactId>
</dependency>
```

132.2. SOAP OPTIONS

The SOAP dataformat supports 6 options, which are listed below.

Name	Default	Java Type	Description
contextPath		String	Required Package name where your JAXB classes are located.
encoding		String	To overrule and use a specific encoding.
elementNameStrategyRef		String	Refers to an element strategy to lookup from the registry. An element name strategy is used for two purposes. The first is to find a xml element name for a given object and soap action when marshaling the object into a SOAP message. The second is to find an Exception class for a given soap fault name. The following three element strategy class name is provided out of the box. QNameStrategy - Uses a fixed qName that is configured on instantiation. Exception lookup is not supported TypeNameStrategy - Uses the name and namespace from the XMLType annotation of the given type. If no namespace is set then package-info is used. Exception lookup is not supported ServiceInterfaceStrategy - Uses information from a webservice interface to determine the type name and to find the exception class for a SOAP fault All three classes is located in the package name org.apache.camel.dataformat.soap.name If you have generated the web service stub code with cxf-codegen or a similar tool then you probably will want to use the ServiceInterfaceStrategy. In the case you have no annotated service interface you should use QNameStrategy or TypeNameStrategy.

Name	Default	Java Type	Description
version		String	SOAP version should either be 1.1 or 1.2. Is by default 1.1.
namespacePrefixRef		String	When marshalling using JAXB or SOAP then the JAXB implementation will automatic assign namespace prefixes, such as ns2, ns3, ns4 etc. To control this mapping, Camel allows you to refer to a map which contains the desired mapping.
schema		String	To validate against an existing schema. Your can use the prefix classpath:, file: or http: to specify how the resource should by resolved. You can separate multiple schema files by using the ',' character.

132.3. ELEMENTNAMESTRATEGY

An element name strategy is used for two purposes. The first is to find a xml element name for a given object and soap action when marshaling the object into a SOAP message. The second is to find an Exception class for a given soap fault name.

Strate gy	Usage
QName Strategy	Uses a fixed qName that is configured on instantiation. Exception lookup is not supported
TypeN ameStr ategy	Uses the name and namespace from the @XMLType annotation of the given type. If no namespace is set then package-info is used. Exception lookup is not supported
Service Interfa ceStrat egy	Uses information from a webservice interface to determine the type name and to find the exception class for a SOAP fault

If you have generated the web service stub code with cxf-codegen or a similar tool then you probably will want to use the ServiceInterfaceStrategy. In the case you have no annotated service interface you should use QNameStrategy or TypeNameStrategy.

132.4. USING THE JAVA DSL

The following example uses a named DataFormat of soap which is configured with the package com.example.customerservice to initialize the [JAXBContext](#). The second parameter is the ElementNameStrategy. The route is able to marshal normal objects as well as exceptions. (Note the below just sends a SOAP Envelope to a queue. A web service provider would actually need to be listening to the queue for a SOAP call to actually occur, in which case it would be a one way SOAP request. If you need request reply then you should look at the next example.)

■

```
SoapJaxbDataFormat soap = new SoapJaxbDataFormat("com.example.customerservice", new
ServiceInterfaceStrategy(CustomerService.class));
from("direct:start")
    .marshal(soap)
    .to("jms:myQueue");
```



NOTE

See also

As the SOAP dataformat inherits from the JAXB dataformat most settings apply here as well.

132.4.1. Using SOAP 1.2

Since Camel 2.11

```
SoapJaxbDataFormat soap = new SoapJaxbDataFormat("com.example.customerservice", new
ServiceInterfaceStrategy(CustomerService.class));
soap.setVersion("1.2");
from("direct:start")
    .marshal(soap)
    .to("jms:myQueue");
```

When using XML DSL there is a version attribute you can set on the `<soapjxb>` element.

```
<!-- Defining a ServiceInterfaceStrategy for retrieving the element name when marshalling -->
<bean id="myNameStrategy"
class="org.apache.camel.dataformat.soap.name.ServiceInterfaceStrategy">
    <constructor-arg value="com.example.customerservice.CustomerService"/>
    <constructor-arg value="true"/>
</bean>
```

And in the Camel route

```
<route>
    <from uri="direct:start"/>
    <marshal>
        <soapjxb contentPath="com.example.customerservice" version="1.2"
elementNameStrategyRef="myNameStrategy"/>
    </marshal>
    <to uri="jms:myQueue"/>
</route>
```

132.5. MULTI-PART MESSAGES

Multi-part SOAP messages are supported by the `ServiceInterfaceStrategy`. The `ServiceInterfaceStrategy` must be initialized with a service interface definition that is annotated in accordance with JAX-WS 2.2 and meets the requirements of the Document Bare style. The target method must meet the following criteria, as per the JAX-WS specification: 1) it must have at most one **in** or **in/out** non-header parameter, 2) if it has a return type other than **void** it must have no **in/out** or **out** non-header parameters, 3) if it has a return type of **void** it must have at most one **in/out** or **out** non-header parameter.

The `ServiceInterfaceStrategy` should be initialized with a boolean parameter that indicates whether the mapping strategy applies to the request parameters or response parameters.

```
ServiceInterfaceStrategy strat = new
ServiceInterfaceStrategy(com.example.customerservice.multipart.MultiPartCustomerService.class,
true);
SoapJaxbDataFormat soapDataFormat = new
SoapJaxbDataFormat("com.example.customerservice.multipart", strat);
```

132.5.1. Holder Object mapping

JAX-WS specifies the use of a type-parameterized `javax.xml.ws.Holder` object for **In/Out** and **Out** parameters. You may use an instance of the parameterized-type directly. The camel-soap DataFormat marshals Holder values in accordance with the JAXB mapping for the class of the **Holder's value**. **No mapping is provided for 'Holder** objects in an unmarshalled response.

132.6. EXAMPLES

132.6.1. Webservice client

The following route supports marshalling the request and unmarshalling a response or fault.

```
String WS_URI = "cxf://http://myserver/customerservice?
serviceClass=com.example.customerservice&dataFormat=RAW";
SoapJaxbDataFormat soapDF = new SoapJaxbDataFormat("com.example.customerservice", new
ServiceInterfaceStrategy(CustomerService.class));
from("direct:customerServiceClient")
.onException(Exception.class)
.handled(true)
.unmarshal(soapDF)
.end()
.marshall(soapDF)
.to(WS_URI)
.unmarshal(soapDF);
```

The below snippet creates a proxy for the service interface and makes a SOAP call to the above route.

```
import org.apache.camel.Endpoint;
import org.apache.camel.component.bean.ProxyHelper;
...

Endpoint startEndpoint = context.getEndpoint("direct:customerServiceClient");
ClassLoader classLoader = Thread.currentThread().getContextClassLoader();
// CustomerService below is the service endpoint interface, *not* the javax.xml.ws.Service subclass
CustomerService proxy = ProxyHelper.createProxy(startEndpoint, classLoader,
CustomerService.class);
GetCustomersByNameResponse response = proxy.getCustomersByName(new
GetCustomersByName());
```

132.6.2. Webservice Server

Using the following route sets up a webservice server that listens on jms queue `customerServiceQueue`

and processes requests using the class `CustomerServiceImpl`. The `customerServiceImpl` of course should implement the interface `CustomerService`. Instead of directly instantiating the server class it could be defined in a spring context as a regular bean.

```
SoapJaxbDataFormat soapDF = new SoapJaxbDataFormat("com.example.customerservice", new
ServiceInterfaceStrategy(CustomerService.class));
CustomerService serverBean = new CustomerServiceImpl();
from("jms://queue:customerServiceQueue")
    .onException(Exception.class)
    .handled(true)
    .marshal(soapDF)
    .end()
    .unmarshal(soapDF)
    .bean(serverBean)
    .marshal(soapDF);
```

132.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 7 options, which are listed below.

Name	Description	Default	Type
<code>camel.dataformat.soapjaxb.context-path</code>	Package name where your JAXB classes are located.		String
<code>camel.dataformat.soapjaxb.element-name-strategy-ref</code>	Refers to an element strategy to lookup from the registry. An element name strategy is used for two purposes. The first is to find a xml element name for a given object and soap action when marshaling the object into a SOAP message. The second is to find an Exception class for a given soap fault name. The following three element strategy class name is provided out of the box. <code>QNameStrategy</code> - Uses a fixed qName that is configured on instantiation. Exception lookup is not supported <code>TypeNameStrategy</code> - Uses the name and namespace from the XMLType annotation of the given type. If no namespace is set then package-info is used. Exception lookup is not supported <code>ServiceInterfaceStrategy</code> - Uses information from a webservice interface to determine the type name and to find the exception class for a SOAP fault All three classes is located in the package name <code>org.apache.camel.dataformat.soap.name</code> If you have generated the web service stub code with cxf-codegen or a similar tool then you probably will want to use the <code>ServiceInterfaceStrategy</code> . In the case you have no annotated service interface you should use <code>QNameStrategy</code> or <code>TypeNameStrategy</code> .		String

Name	Description	Default	Type
camel.dataformat.soapjaxb.enabled	Whether to enable auto configuration of the soapjaxb data format. This is enabled by default.		Boolean
camel.dataformat.soapjaxb.encoding	To overrule and use a specific encoding.		String
camel.dataformat.soapjaxb.namespace-prefix-ref	When marshalling using JAXB or SOAP then the JAXB implementation will automatic assign namespace prefixes, such as ns2, ns3, ns4 etc. To control this mapping, Camel allows you to refer to a map which contains the desired mapping.		String
camel.dataformat.soapjaxb.schema	To validate against an existing schema. Your can use the prefix classpath:, file: or http: to specify how the resource should by resolved. You can separate multiple schema files by using the ',' character.		String
camel.dataformat.soapjaxb.version	SOAP version should either be 1.1 or 1.2. Is by default 1.1.	1.1	String

CHAPTER 133. SPLUNK

Since Camel 2.13

Both producer and consumer are supported

The Splunk component provides access to [Splunk](#) using the Splunk provided [client](#) api, and it enables you to publish and search for events in Splunk.

133.1. DEPENDENCIES

When using **splunk** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-splunk-starter</artifactId>
</dependency>
```

133.2. URI FORMAT

```
splunk://[endpoint]?[options]
```

133.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

133.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

133.3.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

133.4. COMPONENT OPTIONS

The Splunk component supports 6 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
splunkConfigurationFactory (advanced)	To use the SplunkConfigurationFactory.		SplunkConfigurationFactory
healthCheckConsumerEnabled (health)	Used for enabling or disabling all consumer based health checks from this component.	true	boolean

Name	Description	Default	Type
healthCheckProducerEnabled (health)	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting camel.health.producersEnabled=true.	true	boolean

133.5. ENDPOINT OPTIONS

The Splunk endpoint is configured using URI syntax:

```
splunk:name
```

with the following path and query parameters:

133.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (common)	Required Name has no purpose.		String

133.5.2. Query Parameters (45 parameters)

Name	Description	Default	Type
app (common)	Splunk app.		String
connectionTimeout (common)	Timeout in MS when connecting to Splunk server.	5000	int
host (common)	Splunk host.	localhost	String
owner (common)	Splunk owner.		String
port (common)	Splunk port.	8089	int
scheme (common)	Splunk scheme.	https	String

Name	Description	Default	Type
count (consumer)	A number that indicates the maximum number of entities to return.		int
earliestTime (consumer)	Earliest time of the search time window.		String
initEarliestTime (consumer)	Initial start offset of the first search.		String
latestTime (consumer)	Latest time of the search time window.		String
savedSearch (consumer)	The name of the query saved in Splunk to run.		String
search (consumer)	The Splunk query to run.		String
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
streaming (consumer)	Sets streaming mode. Streaming mode sends exchanges as they are received, rather than in a batch.	false	boolean
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		<code>ExceptionHandler</code>

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
eventHost (producer)	Override the default Splunk event host field.		String
index (producer)	Splunk index to write to.		String
raw (producer)	Should the payload be inserted raw.	false	boolean
source (producer)	Splunk source argument.		String
sourceType (producer)	Splunk sourcetype argument.		String
tcpReceiverLocalPort (producer)	Splunk tcp receiver port defined locally on splunk server. (For example if splunk port 9997 is mapped to 12345, tcpReceiverLocalPort has to be 9997).		Integer
tcpReceiverPort (producer)	Splunk tcp receiver port.		int
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel

Name	Description	Default	Type
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
password (security)	Password for Splunk.		String
sslProtocol (security)	Set the ssl protocol to use. Enum values: • TLSv1.2 • TLSv1.1 • TLSv1 • SSLv3	TLSv1.2	SSLSecurityProtocol

Name	Description	Default	Type
token (security)	User's token for Splunk. This takes precedence over password when both are set.		String
username (security)	Username for Splunk.		String
useSunHttpsHandler (security)	Use sun.net.www.protocol.https.Handler Https handler to establish the Splunk Connection. Can be useful when running in application servers to avoid app. server https handling.	false	boolean

133.6. PRODUCER ENDPOINTS

Endpoint	Description
stream	Streams data to a named index or the default if not specified. When using stream mode be aware of that Splunk has some internal buffer (about 1MB or so) before events gets to the index. If you need realtime, better use submit or tcp mode.
submit	submit mode. Uses Splunk rest api to publish events to a named index or the default if not specified.
tcp	tcp mode. Streams data to a tcp port, and requires a open receiver port in Splunk.

When publishing events the message body should contain a SplunkEvent. See comment under message body.

Example

```
from("direct:start").convertBodyTo(SplunkEvent.class)
    .to("splunk://submit?
username=user&password=123&index=myindex&sourceType=someSourceType&source=mySource"
)...
```

In this example a converter is required to convert to a SplunkEvent class.

133.7. CONSUMER ENDPOINTS:

Endpoint	Description
normal	Performs normal search and requires a search query in the search option.

Endpoint	Description
realtime	Performs realtime search on live data and requires a search query in the search option.
savedsearch	Performs search based on a search query saved in splunk and requires the name of the query in the savedSearch option.

Example

```
from("splunk://normal?delay=5000&username=user&password=123&initEarliestTime=-10s&search=search index=myindex sourcetype=someSourcetype")
    .to("direct:search-result");
```

The **camel-splunk** component creates a route exchange per search result with a `SplunkEvent` in the body.

133.8. MESSAGE BODY

Splunk operates on data in key/value pairs. The `SplunkEvent` class is a placeholder for such data, and should be in the message body for the producer. Likewise it will be returned in the body per search result for the consumer.

You can send raw data to Splunk by setting the `raw` option on the producer endpoint. This is useful for, for example, json/xml and other payloads where Splunk has build in support.

133.9. USE CASES

Search Twitter for tweets with music and publish events to Splunk:

```
from("twitter://search?
type=polling&keywords=music&delay=10&consumerKey=abc&consumerSecret=def&accessToken=hij&
ccessTokenSecret=xxx")
    .convertBodyTo(SplunkEvent.class)
    .to("splunk://submit?username=foo&password=bar&index=camel-
tweets&sourceType=twitter&source=music-tweets");
```

To convert a Tweet to a `SplunkEvent` you could use a converter like:

```
@Converter
public class Tweet2SplunkEvent {
    @Converter
    public static SplunkEvent convertTweet(Status status) {
        SplunkEvent data = new SplunkEvent("twitter-message", null);
        //data.addPair("source", status.getSource());
        data.addPair("from_user", status.getUser().getScreenName());
        data.addPair("in_reply_to", status.getInReplyToScreenName());
        data.addPair(SplunkEvent.COMMON_START_TIME, status.getCreatedAt());
        data.addPair(SplunkEvent.COMMON_EVENT_ID, status.getId());
    }
}
```



```

data.addPair("text", status.getText());
data.addPair("retweet_count", status.getRetweetCount());
if (status.getPlace() != null) {
    data.addPair("place_country", status.getPlace().getCountry());
    data.addPair("place_name", status.getPlace().getName());
    data.addPair("place_street", status.getPlace().getStreetAddress());
}
if (status.getGeoLocation() != null) {
    data.addPair("geo_latitude", status.getGeoLocation().getLatitude());
    data.addPair("geo_longitude", status.getGeoLocation().getLongitude());
}
return data;
}
}

```

Search Splunk for tweets:

```

from("splunk://normal?username=foo&password=bar&initEarliestTime=-2m&search=search
index=camel-tweets sourcetype=twitter")
.log("${body}");

```

133.10. OTHER COMMENTS

Splunk comes with a variety of options for leveraging machine generated data with prebuilt apps for analyzing and displaying this. For example the jmx application could be used to publish jmx attributes, such as, route and jvm metrics to Splunk, and displaying this on a dashboard.

133.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 7 options, which are listed below.

Name	Description	Default	Type
camel.component.splunk.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.splunk.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.splunk.enabled	Whether to enable auto configuration of the splunk component. This is enabled by default.		Boolean
camel.component.splunk.health-check-consumer-enabled	Used for enabling or disabling all consumer based health checks from this component.	true	Boolean
camel.component.splunk.health-check-producer-enabled	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting camel.health.producersEnabled=true.	true	Boolean
camel.component.splunk.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.splunk.splunk-configuration-factory	To use the SplunkConfigurationFactory. The option is a org.apache.camel.component.splunk.SplunkConfigurationFactory type.		SplunkConfigurationFactory

CHAPTER 134. SPRING BATCH

Since Camel 2.10

Only producer is supported

The Spring Batch component and support classes provide integration bridge between Camel and [Spring Batch](#) infrastructure.

134.1. DEPENDENCIES

When using **spring-batch** with Red Hat build of Camel Spring Boot, use the following Maven dependency to enable support for auto-configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-spring-batch-starter</artifactId>
</dependency>
```

134.2. URI FORMAT

```
spring-batch:jobName[?options]
```

Where, **jobName** represents the name of the Spring Batch job located in the Camel registry. If a JobRegistry is provided is used to locate the job.

This component is only used to define producer endpoints, that means you cannot use the Spring Batch component in a **from()** statement.

134.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

134.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

134.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

134.4. COMPONENT OPTIONS

The Spring Batch component supports 4 options, which are listed below.

Name	Description	Default	Type
jobLauncher (producer)	Explicitly specifies a JobLauncher to be used.		JobLauncher
jobRegistry (producer)	Explicitly specifies a JobRegistry to be used.		JobRegistry

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

134.5. ENDPOINT OPTIONS

The Spring Batch endpoint is configured using URI syntax:

```
spring-batch:jobName
```

Following are the path and query parameters:

134.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
jobName (producer)	Required The name of the Spring Batch job located in the registry.		String

134.5.2. Query Parameters (4 parameters)

Name	Description	Default	Type
jobFromHeader (producer)	Explicitly defines if the jobName should be taken from the headers instead of the URI.	false	boolean
jobLauncher (producer)	Explicitly specifies a JobLauncher to be used.		JobLauncher
jobRegistry (producer)	Explicitly specifies a JobRegistry to be used.		JobRegistry
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

134.6. USAGE

When Spring Batch component receives the message, it triggers the job execution. The job is executed using the **org.springframework.batch.core.launch.JobLauncher** instance resolved according to the following algorithm.

- if **JobLauncher** is manually set on the component, then use it.
- if **jobLauncherRef** option is set on the component, then search Camel Registry for the **JobLauncher** with the given name.
- if there is **JobLauncher** registered in the Camel Registry under **jobLauncher** name, then use it.
- if none of the steps above allow to resolve the **JobLauncher** and there is exactly one **JobLauncher** instance in the Camel Registry, then use it.

All headers found in the message are passed to the **JobLauncher** as job parameters. **String**, **Long**, **Double** and **java.util.Date** values are copied to the **org.springframework.batch.core.JobParametersBuilder** and other data types are converted to Strings.

134.7. EXAMPLES

Triggering the Spring Batch job execution:

```
from("direct:startBatch").to("spring-batch:myJob");
```

Triggering the Spring Batch job execution with the **JobLauncher** set explicitly.

```
from("direct:startBatch").to("spring-batch:myJob?jobLauncherRef=myJobLauncher");
```

A **JobExecution** instance returned by the **JobLauncher** is forwarded by the **SpringBatchProducer** as the output message. You can use the **JobExecution** instance to perform some operations using the Spring Batch API directly.

```
from("direct:startBatch").to("spring-batch:myJob").to("mock:JobExecutions");
...
MockEndpoint mockEndpoint = ...;
JobExecution jobExecution =
mockEndpoint.getExchanges().get(0).getIn().getBody(JobExecution.class);
BatchStatus currentJobStatus = jobExecution.getStatus();
```

134.8. SUPPORT CLASSES

Apart from the component, Camel Spring Batch also provides support classes that you can use to hook into Spring Batch infrastructure.

134.8.1. CamelItemReader

CamelItemReader can be used to read batch data directly from the Camel infrastructure.

For example, the snippet below configures Spring Batch to read data from JMS queue:

```

<bean id="camelReader"
class="org.apache.camel.component.spring.batch.support.CamelItemReader">
  <constructor-arg ref="consumerTemplate"/>
  <constructor-arg value="jms:dataQueue"/>
</bean>

<batch:job id="myJob">
  <batch:step id="step">
    <batch:tasklet>
      <batch:chunk reader="camelReader" writer="someWriter" commit-interval="100"/>
    </batch:tasklet>
  </batch:step>
</batch:job>

```

134.8.2. CamelItemWriter

CamelItemWriter has similar purpose as **CamelItemReader**, but it is dedicated to write chunk of the processed data.

For example the snippet below configures Spring Batch to read data from JMS queue.

```

<bean id="camelwriter"
class="org.apache.camel.component.spring.batch.support.CamelItemWriter">
  <constructor-arg ref="producerTemplate"/>
  <constructor-arg value="jms:dataQueue"/>
</bean>

<batch:job id="myJob">
  <batch:step id="step">
    <batch:tasklet>
      <batch:chunk reader="someReader" writer="camelwriter" commit-interval="100"/>
    </batch:tasklet>
  </batch:step>
</batch:job>

```

134.8.3. CamelItemProcessor

CamelItemProcessor is the implementation of Spring Batch **org.springframework.batch.item.ItemProcessor** interface. The latter implementation relays on [Request Reply pattern](#) to delegate the processing of the batch item to the Camel infrastructure. The item to process is sent to the Camel endpoint as the body of the message.

For example the snippet below performs simple processing of the batch item using the [Direct endpoint](#) and the [Simple expression language](#).

```

<camel:camelContext>
  <camel:route>
    <camel:from uri="direct:processor"/>
    <camel:setExchangePattern pattern="InOut"/>
    <camel:setBody>
      <camel:simple>Processed ${body}</camel:simple>
    </camel:setBody>
  </camel:route>
</camel:camelContext>

```



```

<bean id="camelProcessor"
class="org.apache.camel.component.spring.batch.support.CamelItemProcessor">
  <constructor-arg ref="producerTemplate"/>
  <constructor-arg value="direct:processor"/>
</bean>

<batch:job id="myJob">
  <batch:step id="step">
    <batch:tasklet>
      <batch:chunk reader="someReader" writer="someWriter" processor="camelProcessor" commit-
interval="100"/>
    </batch:tasklet>
  </batch:step>
</batch:job>

```

134.8.4. CamelJobExecutionListener

CamelJobExecutionListener is the implementation of the **org.springframework.batch.core.JobExecutionListener** interface sending job execution events to the Camel endpoint.

The **org.springframework.batch.core.JobExecution** instance produced by the Spring Batch is sent as a body of the message. To distinguish between before- and after-callbacks

SPRING_BATCH_JOB_EVENT_TYPE header is set to the **BEFORE** or **AFTER** value.

The example snippet below sends Spring Batch job execution events to the JMS queue.

```

<bean id="camelJobExecutionListener"
class="org.apache.camel.component.spring.batch.support.CamelJobExecutionListener">
  <constructor-arg ref="producerTemplate"/>
  <constructor-arg value="jms:batchEventsBus"/>
</bean>

<batch:job id="myJob">
  <batch:step id="step">
    <batch:tasklet>
      <batch:chunk reader="someReader" writer="someWriter" commit-interval="100"/>
    </batch:tasklet>
  </batch:step>
  <batch:listeners>
    <batch:listener ref="camelJobExecutionListener"/>
  </batch:listeners>
</batch:job>

```

134.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options, which are listed below.

Name	Description	Default	Type
camel.component.spring-batch.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.spring-batch.enabled	Whether to enable auto-configuration of the spring-batch component. This is enabled by default.		Boolean
camel.component.spring-batch.job-launcher	Explicitly specifies a JobLauncher to be used. The option is a <code>org.springframework.batch.core.launch.JobLauncher</code> type.		JobLauncher
camel.component.spring-batch.job-registry	Explicitly specifies a JobRegistry to be used. The option is a <code>org.springframework.batch.core.configuration.JobRegistry</code> type.		JobRegistry

Name	Description	Default	Type
<code>camel.component.spring-batch.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 135. SPRING JDBC

Since Camel 3.10

Only producer is supported

The Spring JDBC component is an extension of the JDBC component with one additional feature to integrate with Spring Transaction Manager.

135.1. DEPENDENCIES

When using **spring-jdbc** with Red Hat build of Camel Spring Boot use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-spring-jdbc-starter</artifactId>
</dependency>
```

The version is specified using BOM in the following way.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.redhat.camel.springboot.platform</groupId>
      <artifactId>camel-spring-boot-bom</artifactId>
      <version>${camel-spring-boot-version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

135.2. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

135.2.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).

- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

135.2.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

135.3. COMPONENT OPTIONS

The Spring JDBC component supports 4 options that are listed below.

Name	Description	Default	Type
dataSource (producer)	To use the DataSource instance instead of looking up the data source by name from the registry.		DataSource

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

Name	Description	Default	Type
connectionStrategy (advanced)	To use a custom strategy for working with connections. Do not use a custom strategy when using the spring-jdbc component because a special Spring ConnectionStrategy is used by default to support Spring Transactions.		ConnectionStrategy

135.4. ENDPOINT OPTIONS

The Spring JDBC endpoint is configured using URI syntax:

```
spring-jdbc:dataSourceName
```

Following are the path and query parameters:

135.4.1. Path Parameters (1 parameters)

Name	Description	Default	Type
dataSourceName (producer)	Required Name of DataSource to lookup in the Registry. If the name is dataSource or default, then Camel will attempt to lookup a default DataSource from the registry, meaning if there is a only one instance of DataSource found, then this DataSource will be used.		String

135.4.2. Query Parameters (14 parameters)

Name	Description	Default	Type
allowNamedParameters (producer)	Whether to allow using named parameters in the queries.	true	boolean

Name	Description	Default	Type
outputClass (producer)	Specify the full package and class name to use as conversion when <code>outputType=SelectOne</code> or <code>SelectList</code> .		String
outputType (producer)	Determines the output the producer should use. Enum values: • <code>SelectOne</code> • <code>SelectList</code> • <code>StreamList</code>	SelectList	JdbcOutputType
parameters (producer)	Optional parameters to the <code>java.sql.Statement</code> . For example to set <code>maxRows</code> , <code>fetchSize</code> etc.		Map
readSize (producer)	The default maximum number of rows that can be read by a polling query. The default value is 0.		int

Name	Description	Default	Type
resetAutoCommit (producer)	Camel will set the autoCommit on the JDBC connection to be false, commit the change after executed the statement and reset the autoCommit flag of the connection at the end, if the resetAutoCommit is true. If the JDBC connection doesn't support to reset the autoCommit flag, you can set the resetAutoCommit flag to be false, and Camel will not try to reset the autoCommit flag. When used with XA transactions you most likely need to set it to false so that the transaction manager is in charge of committing this tx.	true	boolean
transacted (producer)	Whether transactions are in use.	false	boolean
useGetBytesForBlob (producer)	To read BLOB columns as bytes instead of string data. This may be needed for certain databases such as Oracle where you must read BLOB columns as bytes.	false	boolean
useHeadersAsParameters (producer)	Set this option to true to use the prepareStatementStrategy with named parameters. This allows to define queries with named placeholders, and use headers with the dynamic values for the query placeholders.	false	boolean

Name	Description	Default	Type
useJDBC4ColumnNameAndLabelSemantics (producer)	Sets whether to use JDBC 4 or JDBC 3.0 or older semantic when retrieving column name. JDBC 4.0 uses columnName to get the column name where as JDBC 3.0 uses both columnName or columnLabel. Unfortunately JDBC drivers behave differently so you can use this option to work out issues around your JDBC driver if you get problem using this component This option is default true.	true	boolean
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
beanRowMapper (advanced)	To use a custom <code>org.apache.camel.component.jdbc.BeanRowMapper</code> when using <code>outputClass</code> . The default implementation will lower case the row names and skip underscores, and dashes. For example <code>CUST_ID</code> is mapped as <code>custId</code> .		<code>BeanRowMapper</code>
connectionStrategy (advanced)	To use a custom strategy for working with connections. Do not use a custom strategy when using the <code>spring-jdbc</code> component because a special Spring <code>ConnectionStrategy</code> is used by default to support Spring Transactions.		<code>ConnectionStrategy</code>
prepareStatementStrategy (advanced)	Allows the plugin to use a custom <code>org.apache.camel.component.jdbc.JdbcPrepareStatementStrategy</code> to control preparation of the query and prepared statement.		<code>JdbcPrepareStatementStrategy</code>

135.5. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options that are listed below.

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
camel.component.spring-jdbc.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.spring-jdbc.connection-strategy	To use a custom strategy for working with connections. Do not use a custom strategy when using the spring-jdbc component because a special Spring ConnectionStrategy is used by default to support Spring Transactions. The option is a <code>org.apache.camel.component.jdbc.ConnectionStrategy</code> type.		ConnectionStrategy
camel.component.spring-jdbc.enabled	Whether to enable auto configuration of the spring-jdbc component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.spring-jdbc.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 136. SPRING LDAP

Since Camel 2.11

Only producer is supported

The Spring LDAP component provides a Camel wrapper for [Spring LDAP](#).

136.1. DEPENDENCIES

When using **spring-ldap** with Red Hat build of Camel Spring Boot, use the following Maven dependency to enable support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-spring-ldap-starter</artifactId>
</dependency>
```

136.2. URI FORMAT

```
spring-ldap:springLdapTemplate[?options]
```

Where **springLdapTemplate** is the name of the [Spring LDAP Template bean](#). In this bean, you configure the URL and the credentials for your LDAP access.

136.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

136.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

136.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

136.4. COMPONENT OPTIONS

The Spring LDAP component supports 2 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

136.5. ENDPOINT OPTIONS

The Spring LDAP endpoint is configured using URI syntax:

```
spring-ldap:templateName
```

Following are the path and query parameters:

136.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
templateName (producer)	Required Name of the Spring LDAP Template bean.		String

136.5.2. Query Parameters (3 parameters)

Name	Description	Default	Type
------	-------------	---------	------

Name	Description	Default	Type
operation (producer)	Required The LDAP operation to be performed. Enum values: • SEARCH • BIND • UNBIND • AUTHENTICAT E • MODIFY_ATTR IBUTES • FUNCTION_DR IVEN		LdapOperation
scope (producer)	The scope of the search operation. Enum values: • object • onelevel • subtree	subtree	String

Name	Description	Default	Type
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

136.6. USAGE

The component supports producer endpoints only. An attempt to create a consumer endpoint can result in an **UnsupportedOperationException**.

The body of the message must be a map (an instance of **java.util.Map**). Unless a base DN is specified in the configuration of your ContextSource, this map must contain at least an entry with the key **dn** (not needed for function_driven operation) that specifies the root node for the LDAP operation to be performed. Other entries of the map are operation-specific.

The body of the message remains unchanged for the **bind** and **unbind** operations. For the **search** and **function_driven** operations, the body is set to the result of the search, see <http://static.springsource.org/spring-ldap/site/apidocs/org/springframework/ldap/core/LdapTemplate.html#search%28java.lang.String,%20ja>

136.6.1. Search

The message body must have an entry with the key **filter**. The value must be a **String** representing a valid LDAP filter, see http://en.wikipedia.org/wiki/Lightweight_Directory_Access_Protocol#Search_and_Compare.

136.6.2. Bind

The message body must have an entry with the key **attributes**. The value must be an instance of [javax.naming.directory.Attributes](http://en.wikipedia.org/wiki/Lightweight_Directory_Access_Protocol#Search_and_Compare) This entry specifies the LDAP node to be created.

136.6.3. Unbind

No further entries are necessary, the node with the specified **dn** is deleted.

136.6.4. Authenticate

The message body must have entries with the keys **filter** and **password**. The values must be an instance of **String** representing a valid LDAP filter and a user password, respectively.

136.6.5. Modify Attributes

The message body must have an entry with the key **modificationItems**. The value must be an instance of any array of type [javax.naming.directory.ModificationItem](#)

136.6.6. Function-Driven

The message body must have entries with the keys **function** and **request**. The **function** value must be of type **java.util.function.BiFunction<L, Q, S>**. The **L** type parameter must be of type **org.springframework.ldap.core.LdapOperations**. The **request** value must be the same type as the **Q** type parameter in the **function** and it must encapsulate the parameters expected by the **LdapTemplate** method being invoked within the **function**. The **S** type parameter represents the response type as returned by the **LdapTemplate** method being invoked. This operation allows dynamic invocation of **LdapTemplate** methods that are not covered by the operations mentioned above.

Key definitions

In order to avoid spelling errors, the following constants are defined in **org.apache.camel.springldap.SpringLdapProducer**:

- `public static final String DN = "dn"`
- `public static final String FILTER = "filter"`
- `public static final String ATTRIBUTES = "attributes"`
- `public static final String PASSWORD = "password";`
- `public static final String MODIFICATION_ITEMS = "modificationItems";`
- `public static final String FUNCTION = "function";`
- `public static final String REQUEST = "request";`

Following is an example of `createMap` function:

```
from("direct:start")
    .setBody(constant(createMap()))
    .to("spring-ldap:ldapTemplate?operation=BIND");
```

Here, `createMap` function returns `Map` object that contains information about attributes and domain name of ldap server.

```
private static Map<String, Object> createMap() {
    BasicAttributes basicAttributes = new BasicAttributes();
    basicAttributes.put("cn", "Name Surname");
}
```

```

basicAttributes.put("sn", "Surname");
basicAttributes.put("objectClass", "person");
Map<String, Object> map = new HashMap<>();
map.put(SpringLdapProducer.DN, "cn=LdapDN,dc=example,dc=org");
map.put(SpringLdapProducer.ATTRIBUTES, basicAttributes);
return map;
}

```

You must also configure ldap connection using Spring Boot auto-configuration or LdapTemplate Bean for the above example.

Example for Spring Boot auto-configuration:

```

spring.ldap.password=passwordforldapserver
spring.ldap.urls=urlForLdapServer
spring.ldap.username=usernameForLdapServer

```

136.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options that are listed below.

Name	Description	Default	Type
camel.component.spring-ldap.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.spring-ldap.enabled	Whether to enable auto configuration of the spring-ldap component. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.component.spring-ldap.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 137. SPRING RABBITMQ

Since Camel 3.8

Both producer and consumer are supported

The Spring RabbitMQ component allows you to produce and consume messages from [RabbitMQ](#) instances using the Spring RabbitMQ client.

137.1. DEPENDENCIES

When using **spring-rabbitmq** with Red Hat build of Camel Spring Boot use the following Maven dependency to enable support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-spring-rabbitmq-starter</artifactId>
</dependency>
```

The version is specified using BOM in the following way.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.redhat.camel.springboot.platform</groupId>
      <artifactId>camel-spring-boot-bom</artifactId>
      <version>${camel-spring-boot-version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

137.2. URI FORMAT

```
spring-rabbitmq:exchangeName?[options]
```

The **exchangeName** determines the exchange to which the produced messages are sent to. In the case of consumers, the **exchangeName** determines the exchange the queue is bound to.

137.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

137.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings,

credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

137.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

137.4. COMPONENT OPTIONS

The Spring RabbitMQ component supports 29 options that are listed below.

Name	Description	Default	Type
amqpAdmin (common)	Autowired Optional AMQP Admin service to use for auto declaring elements (queues, exchanges, bindings).		AmqpAdmin
connectionFactory (common)	Autowired The connection factory to be use. A connection factory must be configured either on the component or endpoint.		ConnectionFactory

Name	Description	Default	Type
testConnectionOnStartup (common)	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	boolean
autoDeclare (consumer)	Specifies whether the consumer should auto declare binding between exchange, queue and routing key when starting. Enabling this can be good for development to make it easy to standup exchanges, queues and bindings on the broker.	false	boolean
autoStartup (consumer)	Specifies whether the consumer container should auto-startup.	true	boolean

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
deadLetterExchange (consumer)	The name of the dead letter exchange.		String
deadLetterExchangeType (consumer)	The type of the dead letter exchange. Enum values: • direct • fanout • headers • topic	direct	String
deadLetterQueue (consumer)	The name of the dead letter queue.		String
deadLetterRoutingKey (consumer)	The routing key for the dead letter exchange.		String
maximumRetryAttempts (consumer)	How many times a Rabbitmq consumer will retry the same message if Camel failed to process the message.	5	int

Name	Description	Default	Type
rejectAndDontRequeue (consumer)	Whether a Rabbitmq consumer should reject the message without requeuing. This enables failed messages to be sent to a Dead Letter Exchange/Queue, if the broker is so configured.	true	boolean
retryDelay (consumer)	Delay in msec a Rabbitmq consumer will wait before redelivering a message that Camel failed to process.	1000	int
concurrentConsumers (consumer (advanced))	The number of consumers.	1	int
errorHandler (consumer (advanced))	To use a custom ErrorHandler for handling exceptions from the message listener (consumer).		ErrorHandler
listenerContainerFactory (consumer (advanced))	To use a custom factory for creating and configuring ListenerContainer to be used by the consumer for receiving messages.		ListenerContainerFactory
maxConcurrentConsumers (consumer (advanced))	The maximum number of consumers (available only with SMLC).		Integer
messageListenerContainerType (consumer (advanced))	The type of the MessageListenerContainer. Enum values: • DMLC • SMLC	DMLC	String

Name	Description	Default	Type
prefetchCount (consumer (advanced))	Tell the broker how many messages to send to each consumer in a single request. Often this can be set quite high to improve throughput.	250	int
retry (consumer (advanced))	Custom retry configuration to use. If this is configured then the other settings such as <code>maximumRetryAttempts</code> for retry are not in use.		RetryOperationsInterceptor
shutdownTimeout (consumer (advanced))	The time to wait for workers in milliseconds after the container is stopped. If any workers are active when the shutdown signal comes they will be allowed to finish processing as long as they can finish within this timeout.	5000	long
allowNullBody (producer)	Whether to allow sending messages with no body. If this option is false and the message body is null, then an <code>MessageConversionException</code> is thrown.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
replyTimeout (producer)	Specify the timeout in milliseconds to be used when waiting for a reply message when doing request/reply messaging. The default value is 5 seconds. A negative value indicates an indefinite timeout.	5000	long

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
ignoreDeclarationExceptions (advanced)	Switch on ignore exceptions such as mismatched properties when declaring.	false	boolean
messageConverter (advanced)	To use a custom MessageConverter so you can be in control how to map to/from a <code>org.springframework.amqp.core.Message</code> .		MessageConverter
messagePropertiesConverter (advanced)	To use a custom MessagePropertiesConverter so you can be in control how to map to/from a <code>org.springframework.amqp.core.MessageProperties</code> .		MessagePropertiesConverter
headerFilterStrategy (filter)	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message.		HeaderFilterStrategy

137.5. ENDPOINT OPTIONS

The Spring RabbitMQ endpoint is configured using URI syntax:

```
spring-rabbitmq:exchangeName
```

Following are the path and query parameters:

137.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
exchangeName (common)	Required The exchange name determines the exchange to which the produced messages will be sent to. In the case of consumers, the exchange name determines the exchange the queue will be bound to. Note: to use default exchange then do not use empty name, but use default instead.		String

137.5.2. Query Parameters (34 parameters)

Name	Description	Default	Type
connectionFactory (common)	The connection factory to be use. A connection factory must be configured either on the component or endpoint.		ConnectionFactory

Name	Description	Default	Type
disableReplyTo (common)	Specifies whether Camel ignores the ReplyTo header in messages. If true, Camel does not send a reply back to the destination specified in the ReplyTo header. You can use this option if you want Camel to consume from a route and you do not want Camel to automatically send back a reply message because another component in your code handles the reply message. You can also use this option if you want to use Camel as a proxy between different message brokers and you want to route message from one system to another.	false	boolean
routingKey (common)	The value of a routing key to use. Default is empty which is not helpful when using the default (or any direct) exchange, but fine if the exchange is a headers exchange for instance.		String
testConnectionOnStartup (common)	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	boolean

Name	Description	Default	Type
acknowledgeMode (consumer)	Flag controlling the behaviour of the container with respect to message acknowledgement. The most common usage is to let the container handle the acknowledgements (so the listener doesn't need to know about the channel or the message). Set to <code>AcknowledgeMode.MANUAL</code> if the listener will send the acknowledgements itself using <code>Channel.basicAck(long, boolean)</code>. Manual acks are consistent with either a transactional or non-transactional channel, but if you are doing no other work on the channel at the same other than receiving a single message then the transaction is probably unnecessary. Set to <code>AcknowledgeMode.NONE</code> to tell the broker not to expect any acknowledgements, and it will assume all messages are acknowledged as soon as they are sent (this is <code>autoack</code> in native Rabbit broker terms). If <code>AcknowledgeMode.NONE</code> then the channel cannot be transactional (so the container will fail on start up if that flag is accidentally set). Enum values: • <code>NONE</code> • <code>MANUAL</code> • <code>AUTO</code>		<code>AcknowledgeMode</code>

Name	Description	Default	Type
asyncConsumer (consumer)	Whether the consumer processes the Exchange asynchronously. If enabled then the consumer may pickup the next message from the queue, while the previous message is being processed asynchronously (by the Asynchronous Routing Engine). This means that messages may be processed not 100% strictly in order. If disabled (as default) then the Exchange is fully processed before the consumer will pickup the next message from the queue.	false	boolean
autoDeclare (consumer)	Specifies whether the consumer should auto declare binding between exchange, queue and routing key when starting.	true	boolean
autoStartup (consumer)	Specifies whether the consumer container should auto-startup.	true	boolean
deadLetterExchange (consumer)	The name of the dead letter exchange.		String
deadLetterExchangeType (consumer)	The type of the dead letter exchange. Enum values: • direct • fanout • headers • topic	direct	String

Name	Description	Default	Type
deadLetterQueue (consumer)	The name of the dead letter queue.		String
deadLetterRoutingKey (consumer)	The routing key for the dead letter exchange.		String
exchangeType (consumer)	The type of the exchange. Enum values: • direct • fanout • headers • topic	direct	String
exclusive (consumer)	Set to true for an exclusive consumer.	false	boolean
maximumRetryAttempts (consumer)	How many times a Rabbitmq consumer will retry the same message if Camel failed to process the message.	5	int
noLocal (consumer)	Set to true for an no-local consumer.	false	boolean
queues (consumer)	The queue(s) to use for consuming messages. Multiple queue names can be separated by comma. If none has been configured then Camel will generate an unique id as the queue name for the consumer.		String
rejectAndDontRequeue (consumer)	Whether a Rabbitmq consumer should reject the message without requeuing. This enables failed messages to be sent to a Dead Letter Exchange/Queue, if the broker is so configured.	true	boolean

Name	Description	Default	Type
retryDelay (consumer)	Delay in msec a Rabbitmq consumer will wait before redelivering a message that Camel failed to process.	1000	int
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
concurrentConsumers (consumer (advanced))	The number of consumers.		Integer
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern
maxConcurrentConsumers (consumer (advanced))	The maximum number of consumers (available only with SMLC).		Integer
messageListenerContainerType (consumer (advanced))	The type of the MessageListenerContainer. Enum values: • DMLC • SMLC	DMLC	String
prefetchCount (consumer (advanced))	Tell the broker how many messages to send in a single request. Often this can be set quite high to improve throughput.		Integer
retry (consumer (advanced))	Custom retry configuration to use. If this is configured then the other settings such as <code>maximumRetryAttempts</code> for retry are not in use.		RetryOperationsInterceptor
replyTimeout (producer)	Specify the timeout in milliseconds to be used when waiting for a reply message when doing request/reply messaging. The default value is 5 seconds. A negative value indicates an indefinite timeout.	5000	long

Name	Description	Default	Type
usePublisherConnection (producer)	Use a separate connection for publishers and consumers.	false	boolean
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
args (advanced)	Specify arguments for configuring the different RabbitMQ concepts, a different prefix is required for each element: arg.consumer. arg.exchange. arg.queue. arg.binding. arg.dlq.exchange. arg.dlq.queue. arg.dlq.binding. For example to declare a queue with message ttl argument: args=arg.queue.x-message-ttl=60000.		Map
messageConverter (advanced)	To use a custom MessageConverter so you can be in control how to map to/from a org.springframework.amqp.core.Message.		MessageConverter
messagePropertiesConverter (advanced)	To use a custom MessagePropertiesConverter so you can be in control how to map to/from a org.springframework.amqp.core.MessageProperties.		MessagePropertiesConverter
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	false	boolean

137.6. MESSAGE HEADERS

The Spring RabbitMQ component supports 2 message headers that are listed below:

Name	Description	Default	Type
CamelSpringRabbitmqRoutingOverrideKey (common) Constant: ROUTING_OVERRIDE_KEY	The exchange key.		String
CamelSpringRabbitmqExchangeOverrideName (common) Constant: EXCHANGE_OVERRIDE_NAME	The exchange name.		String

137.7. USING A CONNECTION FACTORY

To connect to RabbitMQ you must setup a **ConnectionFactory** (same as JMS) with the login details as described below.

It is recommended to use **CachingConnectionFactory** from spring-rabbit as it comes with connection pooling out of the box.

```
<bean id="rabbitConnectionFactory"
class="org.springframework.amqp.rabbit.connection.CachingConnectionFactory">
  <property name="uri" value="amqp://localhost:5672"/>
</bean>
```

The **ConnectionFactory** is auto-detected by default, so you can just execute it.

```
<camelContext>
  <route>
    <from uri="direct:cheese"/>
    <to uri="spring-rabbitmq:foo?routingKey=cheese"/>
  </route>
</camelContext>
```

137.8. DEFAULT EXCHANGE NAME

To use default exchange name (which would be an empty exchange name in RabbitMQ) you must use **default** as name in the endpoint uri, like:

```
to("spring-rabbitmq:default?routingKey=foo")
```

137.9. AUTO DECLARE EXCHANGES, QUEUES AND BINDINGS

Before you can send or receive messages from RabbitMQ, you must first set up the exchanges, queues and bindings.

In development mode, Camel can automatically do this. You can enable this by setting **autoDeclare=true** on the **SpringRabbitMQComponent**.

Then Spring RabbitMQ automatically declares the elements and sets up the binding between the exchange, queue and routing keys.

The elements can be configured using the multi-valued **args** option.

For example to specify the queue as durable and exclusive, you can configure the endpoint uri with **arg.queue.durable=true&arg.queue.exclusive=true**.

Exchanges

Option	Type	Description	Default
autoDelete	boolean	True if the server should delete the exchange when it is no longer in use (if all bindings are deleted).	false
durable	boolean	A durable exchange will survive a server restart.	true

You can also configure any additional **x-** arguments. See details in the RabbitMQ documentation.

Queues

Option	Type	Description	Default
autoDelete	boolean	True if the server should delete the exchange when it is no longer in use (if all bindings are deleted).	false
durable	boolean	A durable queue will survive a server restart.	false
exclusive	boolean	Whether the queue is exclusive	false
x-dead-letter-exchange	String	The name of the dead letter exchange. If none configured then the component configured value is used.	

Option	Type	Description	Default
x-dead-letter-routing-key	String	The routing key for the dead letter exchange. If none configured then the component configured value is used.	

You can also configure any additional **x-** arguments, such as the message time to live using **x-message-ttl**, and many others. See details in the RabbitMQ documentation.

137.10. MAPPING FROM CAMEL TO RABBITMQ

The message body is mapped from Camel Message body to a **byte[]** which is the type that RabbitMQ uses for message body. Camel uses its type converter to convert the message body to byte array.

Spring Rabbit comes out of the box with support for mapping Java serialized objects but Camel Spring RabbitMQ does **not** support this due to security vulnerabilities and using Java objects is a bad design as it enforces strong coupling.

Custom message headers is mapped from Camel Message headers to RabbitMQ headers. This behaviour can be customized by configuring a new implementation of **HeaderFilterStrategy** on the Camel component.

137.11. REQUEST / REPLY

Request and reply messaging is supported using [RabbitMQ direct reply-to](#).

The example below does request/reply, where the message is sent using the cheese exchange name and routing key *foo.bar*, which is being consumed by the 2nd Camel route, that prepends the message with ``Hello ``, and then sends back the message.

So if we send **World** as message body to *direct:start* then, we can see the message being logged

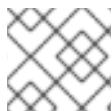
- log:request ⇒ World
- log:input ⇒ World
- log:response ⇒ Hello World

```
from("direct:start")
    .to("log:request")
    .to(ExchangePattern.InOut, "spring-rabbitmq:cheese?routingKey=foo.bar")
    .to("log:response");

from("spring-rabbitmq:cheese?queues=myqueue&routingKey=foo.bar")
    .to("log:input")
    .transform(body().prepend("Hello "));
```

137.12. REUSE ENDPOINT AND SEND TO DIFFERENT DESTINATIONS COMPUTED AT RUNTIME

If you need to send messages to a lot of different RabbitMQ exchanges, you must reuse an endpoint and specify the real destination in a message header. This allows Camel to reuse the same endpoint, but send to different exchanges. This greatly reduces the number of endpoints created and economizes on memory and thread resources.



NOTE

Using `toD` is easier than specifying the dynamic destination with headers.

You can specify using the following headers:

Header	Type	Description
CamelSpringRabbitmqExchangeOverrideName	String	The exchange name.
CamelSpringRabbitmqRoutingOverrideKey	String	The routing key.

For example, the following route shows how you can compute a destination at run time and use it to override the exchange appearing in the endpoint URL:

```
from("file://inbox")
  .to("bean:computeDestination")
  .to("spring-rabbitmq:dummy");
```

The exchange name, **dummy**, is just a placeholder. It must be provided as part of the RabbitMQ endpoint URL, but it is ignored in this example.

In the **computeDestination** bean, specify the real destination by setting the **CamelRabbitmqExchangeOverrideName** header as follows:

```
public void setExchangeHeader(Exchange exchange) {
    String region = ....
    exchange.getIn().setHeader("CamelSpringRabbitmqExchangeOverrideName", "order-" + region);
}
```

Camel reads this header and uses it as the exchange name instead of the one configured on the endpoint. So, in this example Camel sends the message to **spring-rabbitmq:order-emea**, assuming the **region** value was **emea**.

The producer removes both **CamelSpringRabbitmqExchangeOverrideName** and **CamelSpringRabbitmqRoutingOverrideKey** headers from the exchange and do not propagate them to the created Rabbitmq message in order to avoid the accidental loops in the routes (in scenarios when the message is forwarded to another RabbitMQ endpoint).

137.13. USING TOD

If you need to send messages to a lot of different exchanges, you must reuse an endpoint and specify the dynamic destinations with simple language using [toD](#).

For example, you need to send messages to the exchange with order types, then you can use **toD** as follows:

```
from("direct:order")
.toD("spring-rabbit:order-${header.orderType}");
```

137.14. SPRING BOOT AUTO-CONFIGURATION

The component supports 30 options that are listed below.

Name	Description	Default	Type
camel.component.spring-rabbitmq.allow-null-body	Whether to allow sending messages with no body. If this option is false and the message body is null, then an <code>MessageConversionException</code> is thrown.	false	Boolean
camel.component.spring-rabbitmq.amqp-admin	Optional AMQP Admin service to use for auto declaring elements (queues, exchanges, bindings). The option is a <code>org.springframework.amqp.core.AmqpAdmin</code> type.		<code>AmqpAdmin</code>
camel.component.spring-rabbitmq.auto-declare	Specifies whether the consumer should auto declare binding between exchange, queue and routing key when starting. Enabling this can be good for development to make it easy to standup exchanges, queues and bindings on the broker.	false	Boolean
camel.component.spring-rabbitmq.auto-startup	Specifies whether the consumer container should auto-startup.	true	Boolean

Name	Description	Default	Type
camel.component.spring-rabbitmq.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.spring-rabbitmq.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.spring-rabbitmq.concurrent-consumers	The number of consumers.	1	Integer

Name	Description	Default	Type
camel.component.spring-rabbitmq.connection-factory	The connection factory to be use. A connection factory must be configured either on the component or endpoint. The option is a <code>org.springframework.amqp.rabbit.connection.ConnectionFactory</code> type.		ConnectionFactory
camel.component.spring-rabbitmq.dead-letter-exchange	The name of the dead letter exchange.		String
camel.component.spring-rabbitmq.dead-letter-exchange-type	The type of the dead letter exchange.	direct	String
camel.component.spring-rabbitmq.dead-letter-queue	The name of the dead letter queue.		String
camel.component.spring-rabbitmq.dead-letter-routing-key	The routing key for the dead letter exchange.		String
camel.component.spring-rabbitmq.enabled	Whether to enable auto configuration of the spring-rabbitmq component. This is enabled by default.		Boolean
camel.component.spring-rabbitmq.error-handler	To use a custom ErrorHandler for handling exceptions from the message listener (consumer). The option is a <code>org.springframework.util.ErrorHandler</code> type.		ErrorHandler

Name	Description	Default	Type
camel.component.spring-rabbitmq.header-filter-strategy	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		<code>HeaderFilterStrategy</code>
camel.component.spring-rabbitmq.ignore-declaration-exceptions	Switch on ignore exceptions such as mismatched properties when declaring.	false	Boolean
camel.component.spring-rabbitmq.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.spring-rabbitmq.listener-container-factory	To use a custom factory for creating and configuring <code>ListenerContainer</code> to be used by the consumer for receiving messages. The option is a <code>org.apache.camel.component.springrabbit.ListenerContainerFactory</code> type.		<code>ListenerContainerFactory</code>

Name	Description	Default	Type
camel.component.spring-rabbitmq.max-concurrent-consumers	The maximum number of consumers (available only with SMLC).		Integer
camel.component.spring-rabbitmq.maximum-retry-attempts	How many times a Rabbitmq consumer will retry the same message if Camel failed to process the message.	5	Integer
camel.component.spring-rabbitmq.message-converter	To use a custom MessageConverter so you can be in control how to map to/from a org.springframework.amqp.core.Message. The option is a org.springframework.amqp.support.converter.MessageConverter type.		MessageConverter
camel.component.spring-rabbitmq.message-listener-container-type	The type of the MessageListenerContainer.	DMLC	String
camel.component.spring-rabbitmq.message-properties-converter	To use a custom MessagePropertiesConverter so you can be in control how to map to/from a org.springframework.amqp.core.MessageProperties. The option is a org.apache.camel.component.springrabbit.MessagePropertiesConverter type.		MessagePropertiesConverter
camel.component.spring-rabbitmq.prefetch-count	Tell the broker how many messages to send to each consumer in a single request. Often this can be set quite high to improve throughput.	250	Integer

Name	Description	Default	Type
camel.component.spring-rabbitmq.reject-and-dont-requeue	Whether a Rabbitmq consumer should reject the message without requeuing. This enables failed messages to be sent to a Dead Letter Exchange/Queue, if the broker is so configured.	true	Boolean
camel.component.spring-rabbitmq.reply-timeout	Specify the timeout in milliseconds to be used when waiting for a reply message when doing request/reply messaging. The default value is 5 seconds. A negative value indicates an indefinite timeout. The option is a long type.	5000	Long
camel.component.spring-rabbitmq.retry	Custom retry configuration to use. If this is configured then the other settings such as <code>maximumRetryAttempts</code> for retry are not in use. The option is a <code>org.springframework.retry.interceptor.RetryOperationsInterceptor</code> type.		<code>RetryOperationsInterceptor</code>
camel.component.spring-rabbitmq.retry-delay	Delay in msec a Rabbitmq consumer will wait before redelivering a message that Camel failed to process.	1000	Integer

Name	Description	Default	Type
camel.component.spring-rabbitmq.shutdown-timeout	The time to wait for workers in milliseconds after the container is stopped. If any workers are active when the shutdown signal comes they will be allowed to finish processing as long as they can finish within this timeout. The option is a long type.	5000	Long
camel.component.spring-rabbitmq.test-connection-on-startup	Specifies whether to test the connection on startup. This ensures that when Camel starts that all the JMS consumers have a valid connection to the JMS broker. If a connection cannot be granted then Camel throws an exception on startup. This ensures that Camel is not started with failed connections. The JMS producers is tested as well.	false	Boolean

CHAPTER 138. SPRING REDIS

The producer and consumer are supported.

This component allows sending and receiving messages from [Redis](#). Redis is an advanced key-value store where keys can contain strings, hashes, lists, sets and sorted sets. In addition Redis provides pub/sub functionality for inter-app communications. Camel provides a producer for executing commands, a consumer for subscribing to pub/sub messages, and an idempotent repository for filtering out duplicate messages.

Prerequisites

To use this component, you must have a Redis server running.

138.1. DEPENDENCIES

When using **spring-redis** with Red Hat build of Camel Spring Boot, ensure you use the following Maven dependency to have support for auto-configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-spring-redis-starter</artifactId>
</dependency>
```

Use the BOM to get the version.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.redhat.camel.springboot.platform</groupId>
      <artifactId>camel-spring-boot-bom</artifactId>
      <version>${camel-spring-boot-version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

138.2. URI FORMAT

```
spring-redis://host:port[?options]
```

138.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

138.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example, a component may have security settings, credentials for authentication, URLs for network connection.

Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (`application.properties`|`yaml`), or directly with Java code.

138.3.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a *type safe* way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#), which allows you to not hardcode URLs, port numbers, sensitive information, and other settings. In other words, placeholders allow you to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

138.4. COMPONENT OPTIONS

The Spring Redis component supports 4 options, which are listed below.

Name	Description	Default	Type
redisTemplate (common)	Autowired Reference to a pre-configured RedisTemplate instance to use.		RedisTemplate
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer uses the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy, you can use this to allow CamelContext and routes to start up in situations where a producer may otherwise fail during starting and cause the route to fail being started. By starting lazy, Camel's routing error handlers handle any startup failures while routing messages. Beware that when the first message is processed, creating, and starting, the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring of JDBC data sources, JMS connection factories, and AWS Clients.	true	boolean

138.5. ENDPOINT OPTIONS

The Spring Redis endpoint is configured using URI syntax:

```
spring-redis:host:port
```

with the following path and query parameters:

138.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
host (common)	Required The host where the Redis server is running.		String
port (common)	Required Redis server port number.		Integer

138.5.2. Query Parameters (10 parameters)

Name	Description	Default	Type
channels (common)	List of topic names or name patterns to subscribe to. Multiple names can be separated by a comma.		String
	Default command, which can be overridden by	SET	Command

command Name (common)	message header. Notice that the consumer only supports the following commands only: PSUBSCRIBE and SUBSCRIBE. Description	Default	Type
	Enum values: ● PING ● SET ● GET ● QUIT ● EXISTS ● DEL ● TYPE ● FLUSHDB ● KEYS ● RANDOMKEY ● RENAME ● RENAMENX ● RENAMEX ● DBSIZE ● EXPIRE ● EXPIREAT ● TTL ● SELECT ● MOVE ● FLUSHALL ● GETSET ● MGET ● SETNX ● SETEX ● MSET ● MSETNX ● DECRBY ● DECR ● INCRBY ● INCR		

Name	● APPEND ● SUBSTR Description	Default	Type
	● HSET ● HGET ● HSETNX ● HMSET ● HMGET ● HINCRBY ● HEXISTS ● HDEL ● HLEN ● HKEYS ● HVALS ● HGETALL ● RPUSH ● LPUSH ● LLEN ● LRANGE ● LTRIM ● LINDEX ● LSET ● LREM ● LPOP ● RPOP ● RPOPLPUSH ● SADD ● SMEMBERS ● SREM ● SPOP ● SMOVE ● SCARD ● SISMEMBER ● SINTER		

Name	Description • SINTERSTORE • SUNION	Default	Type
	• SUNIONSTORE • SDIFF • SDIFFSTORE • SRANDMEMBER • ZADD • ZRANGE • ZREM • ZINCRBY • ZRANK • ZREVRANK • ZREVRANGE • ZCARD • ZSCORE • MULTI • DISCARD • EXEC • WATCH • UNWATCH • SORT • BLPOP • BRPOP • AUTH • SUBSCRIBE • PUBLISH • UNSUBSCRIBE • PSUBSCRIBE • PUNSUBSCRIBE • ZCOUNT • ZRANGEBYSCORE • ZREVRANGEBYSCORE • ZREMRANGEBYRANK		

Name	• ZREMRANGEBYSCORE • ZUNIONSTORE	Default	Type
	• ZINTERSTORE • SAVE • BGSAVE • BGREWRITEAOF • LASTSAVE • SHUTDOWN • INFO • MONITOR • SLAVEOF • CONFIG • STRLEN • SYNC • LPUSHX • PERSIST • RPUSHX • ECHO • LINSERT • DEBUG • BRPOPLPUSH • SETBIT • GETBIT • SETRANGE • GETRANGE • PEXPIRE • PEXPIREAT • GEOADD • GEODIST • GEOHASH • GEOPOS • GEORADIUS • GEORADIUSBYMEMBER		

Name	Description	Default	Type
connectionFactory (common)	Reference to a pre-configured RedisConnectionFactory instance to use.		RedisConnectionFactory
redisTemplate (common)	Reference to a pre-configured RedisTemplate instance to use.		RedisTemplate
serializer (common)	Reference to a pre-configured RedisSerializer instance to use.		RedisSerializer
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which means any exceptions that occurred while the consumer is trying to pick up incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. The consumer defaults to use the org.apache.camel.spi.ExceptionHandler to deal with exceptions. These exceptions log at WARN or ERROR level and ignored.	False	Boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. If you enable the bridgeErrorHandler option, this option is not used. By default, the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern
listenerContainer (consumer (advanced))	Reference to a pre-configured RedisMessageListenerContainer instance to use.		RedisMessageListenerContainer

Name	Description	Default	Type
lazyStartProducer (Producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy, you can use this to allow CamelContext and routes to start up in situations where a producer may otherwise fail during starting and cause the route startup to fail. By deferring this startup to be lazy, the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	False	Boolean

138.6. MESSAGE HEADERS

The Spring Redis component supports 29 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelRedis.Command (producer) Constant: COMMAND	The command to perform.		String
CamelRedis.Key (common) Constant: KEY	The key.		String
CamelRedis.Keys (common) Constant: KEYS	The keys.		Collection
CamelRedis.Field (common) Constant: FIELD	The field.		String
CamelRedis.Fields (common) Constant: FIELDS	The fields.		Collection

Name	Description	Default	Type
CamelRedis.Value (common) Constant: VALUE	The value.		Object
CamelRedis.Values (common) Constant: VALUES	The values.		Map
CamelRedis.Start (common) Constant: START	Start.		Long
CamelRedis.End (common) Constant: END	End.		Long
CamelRedis.Timeout (common) Constant: TIMEOUT	The timeout.		Long
CamelRedis.Offset (common) Constant: OFFSET	The offset.		Long
CamelRedis.Destination (common) Constant: DESTINATION	The destination.		String
CamelRedis.Channel (common) Constant: CHANNEL	The channel.		byte[] or String
CamelRedis.Message (common) Constant: MESSAGE	The message.		Object

Name	Description	Default	Type
CamelRedis.Index (common) Constant: INDEX	The index.		Long
CamelRedis.Position (common) Constant: POSITION	The position.		String
CamelRedis.Pivot (common) Constant: PIVOT	The pivot.		String
CamelRedis.Count (common) Constant: COUNT	Count.		Long
CamelRedis.Timestamp (common) Constant: TIMESTAMP	The timestamp.		Long
CamelRedis.Pattern (common) Constant: PATTERN	The pattern.		byte[] or String
CamelRedis.Db (common) Constant: DB	The db.		Integer
CamelRedis.Score (common) Constant: SCORE	The score.		Double
CamelRedis.Min (common) Constant: MIN	The min.		Double

Name	Description	Default	Type
CamelRedis.Max (common) Constant: MAX	The max.		Double
CamelRedis.Increment (common) Constant: INCREMENT	Increment.		Double
CamelRedis.WithScore (common) Constant: WITHSCORE	WithScore.		Boolean
CamelRedis.Latitude (common) Constant: LATITUDE	Latitude.		Double
CamelRedis.Longitude (common) Constant: LONGITUDE	Latitude.		Double
CamelRedis.Radius (common) Constant: RADIUS	Radius.		Double

138.7. USAGE

Also, see the available [unit tests](#).

Redis Producer

```
from("direct:start")
    .setHeader("CamelRedis.Key", constant(key))
    .setHeader("CamelRedis.Value", constant(value))
    .to("spring-redis://host:port?command=SET&redisTemplate=#redisTemplate");
```

Redis Consumer

```
from("spring-redis://host:port?command=SUBSCRIBE&channels=myChannel")
    .log("Received message: ${body}");
```

**NOTE**

Where '//'host:port' is URL address for running Redis server.

138.7.1. Message headers evaluated by the Redis producer

The producer issues commands to the server and each command has a different set of parameters with specific types. The result from the command execution is returned in the message body.

Hash Commands	Description	Parameters	Result
HSET	Set the string value of a hash field	RedisConstants.KEY /"Ca melRedis.Key" (String), RedisConstants.FIELD /"C amelRedis.Field" (String), RedisConstants.VALUE /" CamelRedis.Value" (Object)	Void
HGET	Get the value of a hash field	RedisConstants.KEY /"Ca melRedis.Key" (String), RedisConstants.FIELD /"C amelRedis.Field" (String)	String
HSETNX	Set the value of a hash field, only if the field does not exist	RedisConstants.KEY /"Ca melRedis.Key" (String), RedisConstants.FIELD /"C amelRedis.Field" (String), RedisConstants.VALUE /" CamelRedis.Value" (Object)	Void
HMSET	Set multiple hash fields to multiple values	RedisConstants.KEY /"Ca melRedis.Key" (String), RedisConstants.VALUES /"CamelRedis.Values" (Map<String, Object>)	Void
HMGET	Get the values of all the given hash fields	RedisConstants.KEY /"Ca melRedis.Key" (String), RedisConstants.FIELDS /" CamelRedis.Filds" (Collection<String>)	Collect ion<Ob ject>
HINCRBY	Increment the integer value of a hash field by the given number	RedisConstants.KEY /"Ca melRedis.Key" (String), RedisConstants.FIELD /"C amelRedis.Field" (String), RedisConstants.VALUE /" CamelRedis.Value" (Long)	Long

Hash Commands	Description	Parameters	Result
HEXISTS	Determine if a hash field exists	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.FIELD /"CamelRedis.Field" (String)	Boolean
HDEL	Delete one or more hash fields	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.FIELD /"CamelRedis.Field" (String)	Void
HLEN	Get the number of fields in a hash	RedisConstants.KEY /"CamelRedis.Key" (String)	Long
HKEYS	Get all the fields in a hash	RedisConstants.KEY /"CamelRedis.Key" (String)	Set<String>
HVALS	Get all the values in a hash	RedisConstants.KEY /"CamelRedis.Key" (String)	Collection<Object>
HGETALL	Get all the fields and values in a hash	RedisConstants.KEY /"CamelRedis.Key" (String)	Map<String, Object>

List Commands	Description	Parameters	Result
RPUSH	Append one or multiple values to a list	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Long
RPUSHX	Append a value to a list, only if the list exists	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Long
LPUSH	Prepend one or multiple values to a list	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Long
LLEN	Get the length of a list	RedisConstants.KEY /"CamelRedis.Key" (String)	Long

List Commands	Description	Parameters	Result
LRange	Get a range of elements from a list	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.START /"CamelRedis.Start"Long), RedisConstants.END /"CamelRedis.End" (Long)	List<Object>
LTrim	Trim a list to the specified range	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.START /"CamelRedis.Start"Long), RedisConstants.END /"CamelRedis.End" (Long)	Void
LIndex	Get an element from a list by its index	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.INDEX /"CamelRedis.Index" (Long)	String
LInsert	Insert an element before or after another element in a list	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.PIVOT /"CamelRedis.Pivot" (String), RedisConstants.POSITION /"CamelRedis.Position" (String)	Long
LSet	Set the value of an element in a list by its index	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.INDEX /"CamelRedis.Index" (Long)	Void
LRem	Remove elements from a list	RedisConstants.KEY / RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.COUNT /"CamelRedis.Count" (Long)	Long
LPop	Remove and get the first element in a list	RedisConstants.KEY /"CamelRedis.Key" (String)	Object

List Commands	Description	Parameters	Result
RPOP	Remove and get the last element in a list	RedisConstants.KEY /"CamelRedis.Key" (String)	String
RPOPLPUSH	Remove the last element in a list, append it to another list and return it	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.DESTINATION /"CamelRedis.Destination" (String)	Object
BRPOPLPUSH	Pop a value from a list, push it to another list and return it; or block until one is available	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.DESTINATION /"CamelRedis.Destination" (String), RedisConstants.TIMEOUT /"CamelRedis.Timeout" (Long)	Object
BLPOP	Remove and get the first element in a list, or block until one is available	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.TIMEOUT /"CamelRedis.Timeout" (Long)	Object
BRPOP	Remove and get the last element in a list, or block until one is available	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.TIMEOUT /"CamelRedis.Timeout" (Long)	String

Set Commands	Description	Parameters	Result
SADD	Add one or more members to a set	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Boolean
SMEMBERS	Get all the members in a set	RedisConstants.KEY /"CamelRedis.Key" (String)	Set<Object>
SREM	Remove one or more members from a set	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Boolean
SPOP	Remove and return a random member from a set	RedisConstants.KEY /"CamelRedis.Key" (String)	String

Set Commands	Description	Parameters	Result
SMOVE	Move a member from one set to another	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.DESTINATION /"CamelRedis.Destination" (String)	Boolean
SCARD	Get the number of members in a set	RedisConstants.KEY /"CamelRedis.Key" (String)	Long
SISMEMBER	Determine if a given value is a member of a set	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Boolean
SINTER	Intersect multiple sets	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.KEYS /"CamelRedis.Keys" (String)	Set<Object>
SINTERSTORE	Intersect multiple sets and store the resulting set in a key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.KEYS /"CamelRedis.Keys" (String), RedisConstants.DESTINATION /"CamelRedis.Destination" (String)	Void
SUNION	Add multiple sets	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.KEYS /"CamelRedis.Keys" (String)	Set<Object>
SUNIONSTORE	Add multiple sets and store the resulting set in a key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.KEYS /"CamelRedis.Keys" (String), RedisConstants.DESTINATION /"CamelRedis.Destination" (String)	Void
SDIFF	Subtract multiple sets	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.KEYS /"CamelRedis.Keys" (String)	Set<Object>

Set Commands	Description	Parameters	Result
SDIFFSTORE	Subtract multiple sets and store the resulting set in a key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.KEYS /"CamelRedis.Keys" (String), RedisConstants.DESTINATION /"CamelRedis.Destination" (String)	Void
SRANDMEMBER	Get one or multiple random members from a set	RedisConstants.KEY /"CamelRedis.Key" (String)	String

Ordered set Commands	Description	Parameters	Result
ZADD	Add one or more members to a sorted set, or update its score if it already exists	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.SCORE /"CamelRedis.Score" (Double)	Boolean
ZRANGE	Return a range of members in a sorted set, by index	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.START /"CamelRedis.Start" (Long), RedisConstants.END /"CamelRedis.End" (Long), RedisConstants.WITHSCORE /"CamelRedis.WithScore" (Boolean)	Object
ZREM	Remove one or more members from a sorted set	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Boolean
ZINCRBY	Increment the score of a member in a sorted set	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.INCREMENT /"CamelRedis.Increment" (Double)	Double
ZRANK	Determine the index of a member in a sorted set	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Long

Ordered set Commands	Description	Parameters	Result
ZREVRANK	Determine the index of a member in a sorted set, with scores ordered from high to low	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Long
ZREVRANGE	Return a range of members in a sorted set, by index, with scores ordered from high to low	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.START /"CamelRedis.Start" Long), RedisConstants.END /"CamelRedis.End" (Long), RedisConstants.WITHSCORE /"CamelRedis.WithScore" (Boolean)	Object
ZCARD	Get the number of members in a sorted set	RedisConstants.KEY /"CamelRedis.Key" (String)	Long
ZCOUNT	Count the members in a sorted set with scores within the given values	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.MIN /"CamelRedis.Min" (Double), RedisConstants.MAX /"CamelRedis.Max" (Double)	Long
ZRANGEBYSCORE	Return a range of members in a sorted set, by score	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.MIN /"CamelRedis.Min" (Double), RedisConstants.MAX /"CamelRedis.Max" (Double)	Set<Object>
ZREVRANGEBYSCORE	Return a range of members in a sorted set, by score, with scores ordered from high to low	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.MIN /"CamelRedis.Min" (Double), RedisConstants.MAX /"CamelRedis.Max" (Double)	Set<Object>
ZREMRANGEBYRANK	Remove all members in a sorted set within the given indexes	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.START /"CamelRedis.Start" (Long), RedisConstants.END /"CamelRedis.End" (Long)	Void

Ordered set Commands	Description	Parameters	Result
ZREMRANGEBYSCORE	Remove all members in a sorted set within the given scores	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.START /"CamelRedis.Start" (Long), RedisConstants.END /"CamelRedis.End" (Long)	Void
ZUNIONSTORE	Add multiple sorted sets and store the resulting sorted set in a new key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.KEYS /"CamelRedis.Keys" (String), RedisConstants.DESTINATION /"CamelRedis.Destination" (String)	Void
ZINTERSTORE	Intersect multiple sorted sets and store the resulting sorted set in a new key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.KEYS /"CamelRedis.Keys" (String), RedisConstants.DESTINATION /"CamelRedis.Destination" (String)	Void

String Commands	Description	Parameters	Result
SET	Set the string value of a key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Void
GET	Get the value of a key	RedisConstants.KEY /"CamelRedis.Key" (String)	Object
STRLEN	Get the length of the value stored in a key	RedisConstants.KEY /"CamelRedis.Key" (String)	Long
APPEND	Append a value to a key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (String)	Integer

String Commands	Description	Parameters	Result
SETBIT	Sets or clears the bit at offset in the string value stored at key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.OFFSET /"CamelRedis.Offset" (Long), RedisConstants.VALUE /"CamelRedis.Value" (Boolean)	Void
GETBIT	Returns the bit value at offset in the string value stored at key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.OFFSET /"CamelRedis.Offset" (Long)	Boolean
SETRANGE	Overwrite part of a string at key starting at the specified offset	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.OFFSET /"CamelRedis.Offset" (Long)	Void
GETRANGE	Get a substring of the string stored at a key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.START /"CamelRedis.Start" (Long), RedisConstants.END /"CamelRedis.End" (Long)	String
SETNX	Set the value of a key, only if the key does not exist	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Boolean
SETEX	Set the value and expiration of a key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.TIMEOUT /"CamelRedis.Timeout" (Long), SECONDS	Void
DECRBY	Decrement the integer value of a key by the given number	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Long)	Long
DECR	Decrement the integer value of a key by one	RedisConstants.KEY /"CamelRedis.Key" (String),	Long

String Commands	Description	Parameters	Result
INCRBY	Increment the integer value of a key by the given amount	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Long)	Long
INCR	Increment the integer value of a key by one	RedisConstants.KEY /"CamelRedis.Key" (String)	Long
MGET	Get the values of all the given keys	RedisConstants.FIELDS /"CamelRedis.Fields" (Collection<String>)	List<Object>
MSET	Set multiple keys to multiple values	RedisConstants.VALUES /"CamelRedis.Values" (Map<String, Object>)	Void
MSETNX	Set multiple keys to multiple values, only if none of the keys exist	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Void
GETSET	Set the string value of a key and return its old value	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Object

Key Commands	Description	Parameters	Result
EXISTS	Determine if a key exists	RedisConstants.KEY /"CamelRedis.Key" (String)	Boolean
DEL	Delete a key	RedisConstants.KEYS /"CamelRedis.Keys" (String)	Void
TYPE	Determine the type stored at key	RedisConstants.KEY /"CamelRedis.Key" (String)	DataType
KEYS	Find all keys matching the given pattern	RedisConstants.PATTERN /"CamelRedis.Pattern" (String)	Collection<String>
RANDOMKEY	Return a random key from the keyspace	RedisConstants.PATTERN /"CamelRedis.Pattern" (String), RedisConstants.VALUE /"CamelRedis.Value" (String)	String
RENAME	Rename a key	RedisConstants.KEY /"CamelRedis.Key" (String)	Void

Key Commands	Description	Parameters	Result
RENAMENX	Rename a key, only if the new key does not exist	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (String)	Boolean
EXPIRE	Set a key's time to live in seconds	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.TIMEOUT /"CamelRedis.Timeout" (Long)	Boolean
SORT	Sort the elements in a list, set or sorted set	RedisConstants.KEY /"CamelRedis.Key" (String)	List<Object>
PERSIST	Remove the expiration from a key	RedisConstants.KEY /"CamelRedis.Key" (String)	Boolean
EXPIREAT	Set the expiration for a key as a UNIX timestamp	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.TIMESTAMP /"CamelRedis.Timestamp" (Long)	Boolean
PEXPIRE	Set a key's time to live in milliseconds	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.TIMEOUT /"CamelRedis.Timeout" (Long)	Boolean
PEXPIREAT	Set the expiration for a key as a UNIX timestamp specified in milliseconds	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.TIMESTAMP /"CamelRedis.Timestamp" (Long)	Boolean
TTL	Get the time to live for a key	RedisConstants.KEY /"CamelRedis.Key" (String)	Long
MOVE	Move a key to another database	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.DB /"CamelRedis.Db" (Integer)	Boolean

Geo Commands	Description	Parameters	Result
--------------	-------------	------------	--------

Geo Commands	Description	Parameters	Result
GEOADD	Adds the specified geospatial items (latitude, longitude, name) to the specified key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.LATITUDE /"CamelRedis.Latitude" (Double), RedisConstants.LONGITUDE /"CamelRedis.Longitude" (Double), RedisConstants.VALUE /"CamelRedis.Value" (Object)	Long
GEODIST	Return the distance between two members in the geospatial index for the specified key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUES /"CamelRedis.Values" (Object[])	Distance
GEOHASH	Return valid Geohash strings representing the position of an element in the geospatial index for the specified key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	List<String>
GEOPOS	Return the positions (longitude, latitude) of an element in the geospatial index for the specified key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object)	List<Point>
GEORADIUS	Return the element in the geospatial index for the specified key, which is within the borders of the area specified with the central location and the maximum distance from the center (the radius)	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.LATITUDE /"CamelRedis.Latitude" (Double), RedisConstants.LONGITUDE /"CamelRedis.Longitude" (Double), RedisConstants.RADIUS /"CamelRedis.Radius" (Double), RedisConstants.COUNT /"CamelRedis.Count" (Integer)	GeoResults
GEORADIUSBYMEMBER	This command is exactly like GEORADIUS with the sole difference that instead of taking, as the center of the area to query, a longitude and latitude value, it takes the name of a member already existing inside the geospatial index for the specified key	RedisConstants.KEY /"CamelRedis.Key" (String), RedisConstants.VALUE /"CamelRedis.Value" (Object), RedisConstants.RADIUS /"CamelRedis.Radius" (Double), RedisConstants.COUNT /"CamelRedis.Count" (Integer)	GeoResults

Other Commands	Description	Parameters	Result
MULTI	Mark the start of a transaction block	none	Void
DISCARD	Discard all commands issued after MULTI	none	Void
EXEC	Execute all commands issued after MULTI	none	Void
WATCH	Watch the given keys to determine the execution of the MULTI/EXEC block	RedisConstants.KEYS /"CamelRedis.Keys" (String)	Void
UNWATCH	Forget about all watched keys	none	Void
ECHO	Echo the given string	RedisConstants.VALUE /"CamelRedis.Value" (String)	String
PING	Ping the server	none	String
QUIT	Close the connection	none	Void
PUBLISH	Post a message to a channel	RedisConstants.CHANNEL /"CamelRedis.Channel" (String), RedisConstants.MESSAGE /"CamelRedis.Message" (Object)	Void

138.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options, which are listed below.

Name	Description	Default	Type
camel.component.spring-redis.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatically configure JDBC data sources, JMS connection factories, AWS Clients, etc.	True	Boolean

Name	Description	Default	Type
camel.component.spring-redis.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which means any exceptions that occur while the consumer is trying to pick up incoming messages, or the likes, will be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	False	Boolean
camel.component.spring-redis.enabled	Whether to enable auto configuration of the spring-redis component. This is enabled by default.		Boolean
camel.component.spring-redis.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy, you can allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail start up. By deferring this startup to be lazy, the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	False	Boolean
camel.component.spring-redis.redis-template	Reference to a pre-configured RedisTemplate instance to use. The option is an <code>org.springframework.data.redis.core.RedisTemplate</code> type.		RedisTemplate

CHAPTER 139. SPRING SECURITY

Since Camel 2.3

The Camel Spring Security component provides role-based authorization for Camel routes. It leverages the authentication and user services provided by [Spring Security](#) (formerly Acegi Security) and adds a declarative, role-based policy system to control whether a route can be executed by a given principal.

If you are not familiar with the Spring Security authentication and authorization system, please review the current reference documentation on the [SpringSource](#) web site linked above.

139.1. DEPENDENCIES

When using **spring-security** with Red Hat build of Camel Spring Boot, use the following Maven dependency to enable support for auto-configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-spring-security-starter</artifactId>
</dependency>
```

139.2. CREATING AUTHORIZATION POLICIES

Access to a route is controlled by an instance of a **SpringSecurityAuthorizationPolicy** object. A policy object contains the name of the Spring Security authority (role) required to run a set of endpoints and references to Spring Security **AuthenticationManager** and **AccessDecisionManager** objects used to determine whether the current principal has been assigned that role. Policy objects may be configured as Spring beans or by using an **<authorizationPolicy>** element in Spring XML.

The **<authorizationPolicy>** element can contain the following attributes:

Name	Default Value	Description
id	null	The unique Spring bean identifier which is used to reference the policy in routes (required)
access	null	The Spring Security authority name that is passed to the access decision manager (required)
authenticationManager	authenticationManager	The name of the Spring Security AuthenticationManager object in the context
accessDecisionManager	accessDecisionManager	The name of the Spring Security AccessDecisionManager object in the context

Name	Default Value	Description
authenticationAdapter	DefaultAuthenticationAdapter	The name of a camel-spring-security AuthenticationAdapter object in the context that is used to convert a javax.security.auth.Subject into a Spring Security Authentication instance.
useThreadSecurityContext	true	If a javax.security.auth.Subject cannot be found in the In message header under <code>Exchange.AUTHENTICATION</code> , check the Spring Security SecurityContextHolder for an Authentication object.
alwaysReauthenticate	false	If set to true, the SpringSecurityAuthorizationPolicy will always call AuthenticationManager.authenticate() each time the policy is accessed.

139.3. CONTROLLING ACCESS TO CAMEL ROUTES

A Spring Security **AuthenticationManager** and **AccessDecisionManager** are required to use this component. Here is an example of how to configure these objects in Spring XML using the Spring Security namespace:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:spring-security="http://www.springframework.org/schema/security"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/security
http://www.springframework.org/schema/security/spring-security.xsd">

  <bean id="accessDecisionManager"
class="org.springframework.security.access.vote.AffirmativeBased">
    <property name="allowIfAllAbstainDecisions" value="true"/>
    <property name="decisionVoters">
      <list>
        <bean class="org.springframework.security.access.vote.RoleVoter"/>
      </list>
    </property>
  </bean>

  <spring-security:authentication-manager alias="authenticationManager">
```

```

    <spring-security:authentication-provider user-service-ref="userDetailsService"/>
</spring-security:authentication-manager>

    <spring-security:user-service id="userDetailsService">
    <spring-security:user name="jim" password="jimspassword" authorities="ROLE_USER,
ROLE_ADMIN"/>
    <spring-security:user name="bob" password="bobspassword" authorities="ROLE_USER"/>
    </spring-security:user-service>

</beans>

```

Now that the underlying security objects are set up, we can use them to configure an authorization policy and use that policy to control access to a route:

```

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:spring-security="http://www.springframework.org/schema/security"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
  http://camel.apache.org/schema/spring http://camel.apache.org/schema/spring/camel-spring.xsd
  http://camel.apache.org/schema/spring-security http://camel.apache.org/schema/spring-
security/camel-spring-security.xsd
  http://www.springframework.org/schema/security
http://www.springframework.org/schema/security/spring-security.xsd">

  <!-- import the Spring security configuration -->
  <import resource=
"classpath:org/apache/camel/component/spring/security/commonSecurity.xml"/>

  <authorizationPolicy id="admin" access="ROLE_ADMIN"
  authenticationManager="authenticationManager"
  accessDecisionManager="accessDecisionManager"
  xmlns="http://camel.apache.org/schema/spring-security"/>

  <camelContext id="myCamelContext" xmlns="http://camel.apache.org/schema/spring">
    <route>
      <from uri="direct:start"/>
      <!-- The exchange should be authenticated with the role -->
      <!-- of ADMIN before it is send to mock:endpoint -->
      <policy ref="admin">
        <to uri="mock:end"/>
      </policy>
    </route>
  </camelContext>
</beans>

```

In this example, the endpoint **mock:end** is only executed if:

- The *admin* **SpringSecurityAuthorizationPolicy** can locate a Spring Security **Authentication** object, that:
 - can be authenticated or is possible to authenticate
 - contains the **ROLE_ADMIN** authority

139.4. AUTHENTICATION

The process of obtaining security credentials that are used for authorization is not specified by this component. You can write your own processors or components which get authentication information from the exchange depending on your needs. For example, you can create a processor that gets credentials from an HTTP request header originating in the [Jetty](#) component. No matter how the credentials are collected, they need to be placed in the **Inmessage** or the **SecurityContextHolder** so the Camel [Spring Security](#) component can access them.

```
import javax.security.auth.Subject;
import org.apache.camel.*;
import org.apache.commons.codec.binary.Base64;
import org.springframework.security.authentication.*;

public class MyAuthService implements Processor {
    public void process(Exchange exchange) throws Exception {
        // get the username and password from the HTTP header
        // http://en.wikipedia.org/wiki/Basic_access_authentication
        String userpass = new
String(Base64.decodeBase64(exchange.getIn().getHeader("Authorization", String.class)));
        String[] tokens = userpass.split(":");

        // create an Authentication object
        UsernamePasswordAuthenticationToken authToken = new
UsernamePasswordAuthenticationToken(tokens[0], tokens[1]);

        // wrap it in a Subject
        Subject subject = new Subject();
        subject.getPrincipals().add(authToken);

        // place the Subject in the In message
        exchange.getIn().setHeader(Exchange.AUTHENTICATION, subject);

        // you could also do this if useThreadSecurityContext is set to true
        // SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}
```

The **SpringSecurityAuthorizationPolicy** automatically authenticates the **Authentication** object if necessary.

NOTE

Be aware of these two issues when you use the **SecurityContextHolder** instead of or in addition to the **Exchange.AUTHENTICATION** header:

1. The context holder uses a thread-local variable to hold the **Authentication** object. Any routes that cross thread boundaries, like **seda** or **jms**, lose the **Authentication** object.
2. The Spring Security system expects that an **Authentication** object in the context is already authenticated and has roles.
See the [Spring Technical Overview](#), section 5.3.1: "What is authentication in Spring Security?" for more details.

The default behavior of **camel-spring-security** is to look for a **Subject** in the **Exchange.AUTHENTICATION** header. This **Subject** must contain at least one principal, which must be a subclass of **org.springframework.security.core.Authentication**.

You can customize the mapping of **Subject** to **Authentication** object by providing an implementation of the **org.apache.camel.component.spring.security.AuthenticationAdapter** to your **<authorizationPolicy>** bean.

This can be useful if you are working with components that do not use Spring Security but do provide a **Subject**.

At this time, only the **CXF** component populates the **Exchange.AUTHENTICATION** header.

139.5. HANDLING AUTHENTICATION AND AUTHORIZATION ERRORS

If authentication or authorization fails in the **SpringSecurityAuthorizationPolicy**, a **CamelAuthorizationException** is thrown. This can be handled using Camel's standard exception handling methods, like the Exception Clause. The **CamelAuthorizationException** has a reference to the ID of the policy which threw the exception so you can handle errors based on the policy as well as the type of exception.

```
<onException>
  <exception>org.springframework.security.authentication.AccessDeniedException</exception>
  <choice>
    <when>
      <simple>${exception.policyId} == 'user'</simple>
      <transform>
        <constant>You do not have ROLE_USER access!</constant>
      </transform>
    </when>
    <when>
      <simple>${exception.policyId} == 'admin'</simple>
      <transform>
        <constant>You do not have ROLE_ADMIN access!</constant>
      </transform>
    </when>
  </choice>
</onException>
```

139.6. SPRING BOOT AUTO-CONFIGURATION

The component has no Spring Boot auto configuration options.

CHAPTER 140. SPRING WEBSERVICE

Since Camel 2.6

Both producer and consumer are supported

The Spring WS component allows you to integrate with [Spring Web Services](#). It offers both, *client*-side support for accessing web services, and *server*-side support for creating your own contract-first web services.

140.1. DEPENDENCIES

When using **spring-ws** with Red Hat build of Camel Spring Boot use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-spring-ws</artifactId>
  <!-- use the same version as your Camel core version -->
</dependency>
```

Use the BOM to get the version.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.redhat.camel.springboot.platform</groupId>
      <artifactId>camel-spring-boot-bom</artifactId>
      <version>${camel-spring-boot-version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

140.2. URI FORMAT

The URI scheme for this component is as follows

```
spring-ws:[mapping-type:]address[?options]
```

To expose a web service **mapping-type** needs to be set to any of the following:

Mapping type	Description
rootqname	Offers the option to map web service requests based on the qualified name of the root element contained in the message.
soapaction	Used to map web service requests based on the SOAP action specified in the header of the message.

Mapping type	Description
uri	In order to map web service requests that target a specific URI.
xpathresult	Used to map web service requests based on the evaluation of an XPath expression against the incoming message. The result of the evaluation should match the XPath result specified in the endpoint URI.
beannname	Allows you to reference an org.apache.camel.component.spring.ws.bean.CamelEndpointDispatcher object in order to integrate with existing (legacy) endpoint mappings like PayloadRootQNameEndpointMapping , SoapActionEndpointMapping , etc

As a consumer the **address** should contain a value relevant to the specified mapping-type (e.g. a SOAP action, XPath expression). As a producer the address should be set to the URI of the web service your calling upon.

140.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

140.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

140.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

140.4. COMPONENT OPTIONS

The Spring WebService component supports 4 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean

140.5. ENDPOINT OPTIONS

The Spring WebService endpoint is configured using URI syntax:

-

spring-ws:type:lookupKey:webServiceEndpointUri

Following are the path and query parameters:

140.5.1. Path Parameters (4 parameters)

Name	Description	Default	Type
type (consumer)	Endpoint mapping type if endpoint mapping is used. rootqname - Offers the option to map web service requests based on the qualified name of the root element contained in the message. soapaction - Used to map web service requests based on the SOAP action specified in the header of the message. uri - In order to map web service requests that target a specific URI. xpathresult - Used to map web service requests based on the evaluation of an XPath expression against the incoming message. The result of the evaluation should match the XPath result specified in the endpoint URI. beanname - Allows you to reference an org.apache.camel.component.spring.ws.bean.CamelEndpointDispatcher object in order to integrate with existing (legacy) endpoint mappings like PayloadRootQNameEndpointMapping, SoapActionEndpointMapping, etc. Enum values: ● ROOT_QNAME ● ACTION		EndpointMappingType

Name	Description	Default	Type
	• TO • SOAP_ACTION • XPATHRESULT • URI • URI_PATH • BEANNAME		
lookupKey (consumer)	Endpoint mapping key if endpoint mapping is used.		String
webServiceEndpointUri (producer)	The default Web Service endpoint uri to use for the producer.		String
expression (consumer)	The XPath expression to use when option type=xpathresult. Then this option is required to be configured.		String

140.5.2. Query Parameters (21 parameters)

Name	Description	Default	Type
messageFilter (common)	Option to provide a custom MessageFilter. For example when you want to process your headers or attachments by your own.		MessageFilter
messageIdStrategy (common)	Option to provide a custom MessageIdStrategy to control generation of WS-Addressing unique message ids.		MessageIdStrategy

Name	Description	Default	Type
endpointDispatcher (consumer)	Spring org.springframework.ws. server.endpoint.Message Endpoint for dispatching messages received by Spring-WS to a Camel endpoint, to integrate with existing (legacy) endpoint mappings like PayloadRootQNameEnd pointMapping, SoapActionEndpointMa pping, etc.		CamelEndpointDispatch er
endpointMapping (consumer)	Reference to an instance of org.apache.camel.comp onent.spring.ws.bean.Ca melEndpointMapping in the Registry/ApplicationCon text. Only one bean is required in the registry to serve all Camel/Spring-WS endpoints. This bean is auto-discovered by the MessageDispatcher and used to map requests to Camel endpoints based on characteristics specified on the endpoint (like root QName, SOAP action, etc).		CamelSpringWSEndpoin tMapping

Name	Description	Default	Type
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern

Name	Description	Default	Type
allowResponseAttachmentOverride (producer)	Option to override soap response attachments in in/out exchange with attachments from the actual service layer. If the invoked service appends or rewrites the soap attachments this option when set to true, allows the modified soap attachments to be overwritten in in/out message attachments.	false	boolean
allowResponseHeaderOverride (producer)	Option to override soap response header in in/out exchange with header info from the actual service layer. If the invoked service appends or rewrites the soap header this option when set to true, allows the modified soap header to be overwritten in in/out message headers.	false	boolean
faultAction (producer)	Signifies the value for the faultAction response WS-Addressing Fault Action header that is provided by the method. See org.springframework.ws.soap.addressing.server.annotation.Action annotation for more details.		URI
faultTo (producer)	Signifies the value for the faultAction response WS-Addressing FaultTo header that is provided by the method. See org.springframework.ws.soap.addressing.server.annotation.Action annotation for more details.		URI

Name	Description	Default	Type
messageFactory (producer)	Option to provide a custom <code>WebServiceMessageFactory</code> . For example when you want Apache Axiom to handle web service messages instead of SAAJ.		<code>WebServiceMessageFactory</code>
messageSender (producer)	Option to provide a custom <code>WebServiceMessageSender</code> . For example to perform authentication or use alternative transports.		<code>WebServiceMessageSender</code>
outputAction (producer)	Signifies the value for the response WS-Addressing Action header that is provided by the method. See <code>org.springframework.ws.soap.addressing.server.annotation.Action</code> annotation for more details.		URI
replyTo (producer)	Signifies the value for the replyTo response WS-Addressing ReplyTo header that is provided by the method. See <code>org.springframework.ws.soap.addressing.server.annotation.Action</code> annotation for more details.		URI
soapAction (producer)	SOAP action to include inside a SOAP request when accessing remote web services.		String

Name	Description	Default	Type
timeout (producer)	Sets the socket read timeout (in milliseconds) while invoking a webservice using the producer, see <code>URLConnection.setReadTimeout()</code> and <code>CommonsHttpMessageSender.setReadTimeout()</code> . This option works when using the built-in message sender implementations: <code>CommonsHttpMessageSender</code> and <code>HttpURLConnectionMessageSender</code> . One of these implementations will be used by default for HTTP based services unless you customize the Spring WS configuration options supplied to the component. If you are using a non-standard sender, it is assumed that you will handle your own timeout configuration. The built-in message sender <code>HttpComponentsMessageSender</code> is considered instead of <code>CommonsHttpMessageSender</code> which has been deprecated, see <code>HttpComponentsMessageSender.setReadTimeout()</code> .		int
webServiceTemplate (producer)	Option to provide a custom <code>WebServiceTemplate</code> . This allows for full control over client-side web services handling; like adding a custom interceptor or specifying a fault resolver, message sender or message factory.		<code>WebServiceTemplate</code>

Name	Description	Default	Type
wsAddressingAction (producer)	WS-Addressing 1.0 action header to include when accessing web services. The To header is set to the address of the web service as specified in the endpoint URI (default Spring-WS behavior).		URI
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters

140.6. MESSAGE HEADERS

The Spring WebService component supports 7 message headers that are listed below:

Name	Description	Default	Type
CamelSpringWebserviceEndpointUri (producer) Constant: SPRING_WS_ENDPOINT_URI	The endpoint URI.		String
CamelSpringWebserviceSoapAction (producer) Constant: SPRING_WS_SOAP_ACTION	SOAP action to include inside a SOAP request when accessing remote web services.		String
CamelSpringWebserviceSoapHeader (producer) Constant: SPRING_WS_SOAP_HEADER	The soap header source.		Source
CamelSpringWebserviceAddressingAction (producer) Constant: SPRING_WS_ADDRESSING_ACTION	WS-Addressing 1.0 action header to include when accessing web services. The To header is set to the address of the web service as specified in the endpoint URI (default Spring-WS behavior).		URI
CamelSpringWebserviceAddressingFaultTo (producer) Constant: SPRING_WS_ADDRESSING_PRODUCER_FAULT_TO	Signifies the value for the faultAction response WS-Addressing FaultTo header that is provided by the method. See org.springframework.ws.soap.addressing.server.annotation.Action annotation for more details.		URI

Name	Description	Default	Type
CamelSpringWebserviceAddressingReplyTo (producer) Constant: SPRING_WS_ADDRESSING_PRODUCER_REPLY_TO	Signifies the value for the replyTo response WS-Addressing ReplyTo header that is provided by the method. See org.springframework.ws.soap.addressing.server.annotation.Action annotation for more details.		URI
breadcrumbId (consumer) Constant: BREADCRUMB_ID	The breadcrumb id.		String

140.7. ACCESSING WEB SERVICES

To call a web service at <http://foo.com/bar> simply define a route:

```
from("direct:example").to("spring-ws:http://foo.com/bar")
```

and send a message:

```
template.requestBody("direct:example", "<foobar xmlns='http://foo.com'><msg>test message</msg></foobar>");
```

If you are calling a SOAP service, you must not include SOAP tags. Spring-WS performs the XML-to-SOAP marshaling.

140.8. SENDING SOAP AND WS-ADDRESSING ACTION HEADERS

When a remote web service requires a SOAP action or use of the WS-Addressing standard, you define your route as:

```
from("direct:example")
.to("spring-ws:http://foo.com/bar?soapAction=http://foo.com&wsAddressingAction=http://bar.com")
```

You can also override the endpoint options with header values.

```
template.requestBodyAndHeader("direct:example",
"<foobar xmlns='http://foo.com'><msg>test message</msg></foobar>",
SpringWebserviceConstants.SPRING_WS_SOAP_ACTION, "http://baz.com");
```

140.9. USING SOAP HEADERS

You can provide the SOAP header as a Camel Message header when sending a message to a spring-ws endpoint. For example given the following SOAP header in a String:

```
String body = ...
String soapHeader = "<h:Header xmlns:h=\"http://www.webserviceX.NET/\">
<h:MessageID>1234567890</h:MessageID><h:Nested><h:NestedID>1111</h:NestedID>
</h:Nested></h:Header>";
```

We can set the body and header on the Camel Message as follows:

```
exchange.getIn().setBody(body);
exchange.getIn().setHeader(SpringWebserviceConstants.SPRING_WS_SOAP_HEADER,
soapHeader);
```

And then send the Exchange to a **spring-ws** endpoint to call the Web Service.

Similarly, the spring-ws consumer also enriches the Camel Message with the SOAP header.

For an example see this [unit test](#).

140.10. THE HEADER AND ATTACHMENT PROPAGATION

Spring WS Camel supports propagation of the headers and attachments into Spring-WS WebServiceMessage response. The endpoint uses a "hook" with the MessageFilter (default implementation is provided by BasicMessageFilter) to propagate the exchange headers and attachments into WebServiceMessage response.

```
exchange.getOut().getHeaders().put("myCustom","myHeaderValue")
exchange.getIn().addAttachment("myAttachment", new DataHandler(...))
```

If the exchange header in the pipeline contains text, it generates QName(key)=value attribute in the soap header. You must create a QName class directly and put any key into header.

140.11. HOW TO TRANSFORM THE SOAP HEADER USING A STYLESHEET

The header transformation filter (HeaderTransformationMessageFilter.java) can be used to transform the soap header for a soap request. If you want to use the header transformation filter, see the below example:

```
<bean id="headerTransformationFilter"
class="org.apache.camel.component.spring.ws.filter.impl.HeaderTransformationMessageFilter">
  <constructor-arg index="0" value="org/apache/camel/component/spring/ws/soap-header-
transform.xslt"/>
</bean>
```

Use the bead defined above in the camel endpoint

```
<route>
  <from uri="direct:stockQuoteWebserviceHeaderTransformation"/>
  <to uri="spring-ws:http://localhost?"
```

```
webServiceTemplate=#webServiceTemplate&soapAction=http://www.stockquotes.edu/GetQuote&
&messageFilter=#headerTransformationFilter"/>
</route>
```

140.12. HOW TO USE MTOM ATTACHMENTS

The `BasicMessageFilter` provides all required information for Apache Axiom in order to produce MTOM message. If you want to use Apache Camel Spring WS within Apache Axiom, here is an example: – Define the **messageFactory** as shown below, and Spring-WS populates your SOAP message with optimized attachments through an MTOM strategy.

```
<bean id="axiomMessageFactory"
class="org.springframework.ws.soap.axiom.AxiomSoapMessageFactory">
<property name="payloadCaching" value="false" />
<property name="attachmentCaching" value="true" />
<property name="attachmentCacheThreshold" value="1024" />
</bean>
```

- Add into your `pom.xml` the following dependencies

```
<dependency>
<groupId>org.apache.ws.commons.axiom</groupId>
<artifactId>axiom-api</artifactId>
<version>1.2.13</version>
</dependency>
<dependency>
<groupId>org.apache.ws.commons.axiom</groupId>
<artifactId>axiom-impl</artifactId>
<version>1.2.13</version>
<scope>runtime</scope>
</dependency>
```

- Add your attachment into the pipeline, for example using a `Processor` implementation.

```
private class Attachement implements Processor {
public void process(Exchange exchange) throws Exception
{ exchange.getOut().copyFrom(exchange.getIn()); File file = new File("testAttachment.txt");
exchange.getOut().addAttachment("test", new DataHandler(new FileDataSource(file))); }
}
```

- Define endpoint (producer) as usual, for example like this:

```
from("direct:send")
.process(new Attachement())
.to("spring-ws:http://localhost:8089/mySoapService?
soapAction=mySoap&messageFactory=axiomMessageFactory");
```

- Your producer now generates MTOM messages with optimized attachments.

140.13. THE CUSTOM HEADER AND ATTACHMENT FILTERING

If you need to provide your custom processing of either headers or attachments, extend existing `BasicMessageFilter` and override the appropriate methods or write a brand new implementation of the

MessageFilter interface.

To use your custom filter, add either a global or a local message filter into your spring context.

- a) the global custom filter that provides the global configuration for all Spring-WS endpoints

```
<bean id="messageFilter" class="your.domain.myMessageFilter" scope="singleton" />
```

or

- b) the local messageFilter directly on the endpoint as follows:

```
to("spring-ws:http://yourdomain.com?messageFilter=#myEndpointSpecificMessageFilter");
```

For more information see [CAMEL-5724](#)

If you want to create your own MessageFilter, consider overriding the following methods in the default implementation of MessageFilter in class BasicMessageFilter:

```
protected void doProcessSoapHeader(Message inOrOut, SoapMessage soapMessage)
{your code /*no need to call super*/ }
```

```
protected void doProcessSoapAttachements(Message inOrOut, SoapMessage response)
{ your code /*no need to call super*/ }
```

140.14. USING A CUSTOM MESSAGESENDER AND MESSAGEFACTORY

A custom message sender or factory in the registry can be referenced like this:

```
from("direct:example")
.to("spring-ws:http://foo.com/bar?
messageFactory=#messageFactory&messageSender=#messageSender")
```

Spring configuration:

```
<!-- authenticate using HTTP Basic Authentication -->
<bean id="messageSender"
class="org.springframework.ws.transport.http.HttpComponentsMessageSender">
  <property name="credentials">
    <bean class="org.apache.commons.httpclient.UsernamePasswordCredentials">
      <constructor-arg index="0" value="admin"/>
      <constructor-arg index="1" value="secret"/>
    </bean>
  </property>
</bean>

<!-- force use of Sun SAAJ implementation, http://static.springsource.org/spring-
ws/sites/1.5/faq.html#saaj-jboss -->
<bean id="messageFactory" class="org.springframework.ws.soap.saaj.SaajSoapMessageFactory">
  <property name="messageFactory">
    <bean class="com.sun.xml.messaging.saaj.soap.ver1_1.SOAPMessageFactory1_1Impl"/>
  </property>
</bean>
```

140.15. EXPOSING WEB SERVICES

To expose a web service using this component, you first must set up a [MessageDispatcher](#) to look for endpoint mappings in a Spring XML file. If you want to run inside a servlet container, you must use a **MessageDispatcherServlet** configured in **web.xml**.

By default the **MessageDispatcherServlet** will look for a Spring XML named **/WEB-INF/spring-ws-servlet.xml**. To use Camel with Spring-WS the only mandatory bean in that XML file is **CamelEndpointMapping**. This bean allows the **MessageDispatcher** to dispatch web service requests to your routes.

web.xml

```
<web-app>
  <servlet>
    <servlet-name>spring-ws</servlet-name>
    <servlet-class>org.springframework.ws.transport.http.MessageDispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>spring-ws</servlet-name>
    <url-pattern>/*</url-pattern>
  </servlet-mapping>
</web-app>
```

spring-ws-servlet.xml

```
<bean id="endpointMapping"
class="org.apache.camel.component.spring.ws.bean.CamelEndpointMapping" />

<bean id="wsdl" class="org.springframework.ws.wsdl.wsdl11.DefaultWsdl11Definition">
  <property name="schema">
    <bean class="org.springframework.xml.xsd.SimpleXsdSchema">
      <property name="xsd" value="/WEB-INF/foobar.xsd"/>
    </bean>
  </property>
  <property name="portTypeName" value="FooBar"/>
  <property name="locationUri" value="/" />
  <property name="targetNamespace" value="http://example.com/" />
</bean>
```

More information on setting up Spring-WS can be found in [Writing Contract-First Web Services](#). Basically paragraph 3.6 "Implementing the Endpoint" is handled by this component (specifically paragraph 3.6.2 "Routing the Message to the Endpoint" is where **CamelEndpointMapping** comes in). See the Spring Web Services Example included in the Camel distribution.

140.16. ENDPOINT MAPPING IN ROUTES

With the XML configuration in-place, you can now use Camel's DSL to define what web service requests are handled by your endpoint:

The following route receives all web service requests that have a root element named "GetFoo" within the <http://example.com/> namespace.

```
from("spring-ws:rootqname:{http://example.com/}GetFoo?endpointMapping=#endpointMapping")
  .convertBodyTo(String.class).to(mock:example)
```

The following route receives web service requests containing the <http://example.com/GetFoo> SOAP action.

```
from("spring-ws:soapaction:http://example.com/GetFoo?endpointMapping=#endpointMapping")
  .convertBodyTo(String.class).to(mock:example)
```

The following route receives all requests sent to <http://example.com/foobar>.

```
from("spring-ws:uri:http://example.com/foobar?endpointMapping=#endpointMapping")
  .convertBodyTo(String.class).to(mock:example)
```

The route below receives requests that contain the element `<foobar>abc</foobar>` anywhere inside the message (and the default namespace).

```
from("spring-ws:xpathresult:abc?expression=//foobar&endpointMapping=#endpointMapping")
  .convertBodyTo(String.class).to(mock:example)
```

140.16.1. Alternative configuration, using existing endpoint mappings

For every endpoint with mapping-type **beanname** one bean of type **CamelEndpointDispatcher** with a corresponding name is required in the Registry/ApplicationContext. This bean acts as a bridge between the Camel endpoint and an existing [endpoint mapping](#) like **PayloadRootQNameEndpointMapping**.

The use of the **beanname** mapping-type is primarily meant for (legacy) situations where you are already using Spring-WS and have endpoint mappings defined in a Spring XML file. The **beanname** mapping-type allows you to wire your Camel route into an existing endpoint mapping. When you are starting from the beginning, you must define your endpoint mappings as Camel URI's (as illustrated above with **endpointMapping**) since it requires less configuration and is more expressive. You can also use vanilla Spring-WS with the help of annotations.

An example of a route using **beanname**:

```
<camelContext xmlns="http://camel.apache.org/schema/spring">
  <route>
    <from uri="spring-ws:beanname:QuoteEndpointDispatcher" />
    <to uri="mock:example" />
  </route>
</camelContext>

<bean id="legacyEndpointMapping"
class="org.springframework.ws.server.endpoint.mapping.PayloadRootQNameEndpointMapping">
  <property name="mappings">
    <props>
      <prop key="{http://example.com/}GetFuture">FutureEndpointDispatcher</prop>
      <prop key="{http://example.com/}GetQuote">QuoteEndpointDispatcher</prop>
    </props>
  </property>
</bean>

<bean id="QuoteEndpointDispatcher"
```

```
class="org.apache.camel.component.spring.ws.bean.CamelEndpointDispatcher" />
<bean id="FutureEndpointDispatcher"
class="org.apache.camel.component.spring.ws.bean.CamelEndpointDispatcher" />
```

140.17. POJO (UN)MARSHALLING

Camel's pluggable data formats offer support for pojo/xml marshalling using libraries such as JAXB, XStream, JibX, Castor and XMLBeans. You can use these data formats in your route to sent and receive pojo's to and from web services.

When *accessing* web services you can marshal the request and unmarshal the response message:

```
JaxbDataFormat jaxb = new JaxbDataFormat(false);
jaxb.setContextPath("com.example.model");

from("direct:example").marshal(jaxb).to("spring-ws:http://foo.com/bar").unmarshal(jaxb);
```

Similarly, when *providing* web services, you can unmarshal XML requests to POJOs and marshal the response message back to XML:

```
from("spring-ws:rootname:{http://example.com}/GetFoo?
endpointMapping=#endpointMapping").unmarshal(jaxb)
.to("mock:example").marshal(jaxb);
```

140.18. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options that are listed below.

Name	Description	Default	Type
camel.component.spring-ws.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.spring-ws.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.spring-ws.enabled	Whether to enable auto configuration of the spring-ws component. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.component.spring-ws.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages using Camel's routing error handlers. When the first message is processed then creating and starting the producer takes time and prolong the total processing time of the processing.	false	Boolean
camel.component.spring-ws.use-global-ssl-context-parameters	Enable usage of global SSL context parameters.	false	Boolean

CHAPTER 141. SQL

Both producer and consumer are supported

The SQL component allows you to work with databases using JDBC queries. The difference between this component and [JDBC](#) component is that in case of SQL the query is a property of the endpoint and it uses message payload as parameters passed to the query.

This component uses **spring-jdbc** behind the scenes for the actual SQL handling.

The SQL component also supports:

- a JDBC based repository for the Idempotent Consumer EIP pattern. See further below.
- a JDBC based repository for the Aggregator EIP pattern. See further below.

141.1. DEPENDENCIES

When using **sql** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-sql-starter</artifactId>
</dependency>
```

141.2. URI FORMAT



NOTE

This component can be used as a [Transactional Client](#).

The SQL component uses the following endpoint URI notation:

```
sql:select * from table where id=# order by name[?options]
```

You can use named parameters by using ``#name_of_the_parameter`` style as shown:

```
sql:select * from table where id=:#myId order by name[?options]
```

When using named parameters, Camel will lookup the names from, in the given precedence:

1. from message body if its a **java.util.Map**
2. from message headers

If a named parameter cannot be resolved, then an exception is thrown.

You can use Simple expressions as parameters as shown:

```
sql:select * from table where id=:${exchangeProperty.myId} order by name[?options]
```

Notice that the standard `?` symbol that denotes the parameters to an SQL query is substituted with the `#` symbol, because the `?` symbol is used to specify options for the endpoint. The `?` symbol replacement can be configured on endpoint basis.

You can externalize your SQL queries to files in the classpath or file system as shown:

```
sql:classpath:sql/myquery.sql[?options]
```

And the `myquery.sql` file is in the classpath and is just a plain text

```
-- this is a comment
select *
from table
where
  id = :#{exchangeProperty.myId}
order by
  name
```

In the file you can use multilines and format the SQL as you wish. And also use comments such as the `--` dash line.

141.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

141.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

141.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

141.4. COMPONENT OPTIONS

The SQL component supports 5 options, which are listed below.

Name	Description	Default	Type
dataSource (common)	Autowired Sets the DataSource to use to communicate with the database.		DataSource
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

Name	Description	Default	Type
usePlaceholder (advanced)	Sets whether to use placeholder and replace all placeholder characters with sign in the SQL queries. This option is default true.	true	boolean

141.5. ENDPOINT OPTIONS

The SQL endpoint is configured using URI syntax:

```
sql:query
```

with the following path and query parameters:

141.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
query (common)	Required Sets the SQL query to perform. You can externalize the query by using file: or classpath: as prefix and specify the location of the file.		String

141.5.2. Query Parameters (45 parameters)

Name	Description	Default	Type
allowNamedParameters (common)	Whether to allow using named parameters in the queries.	true	boolean
dataSource (common)	Autowired Sets the DataSource to use to communicate with the database at endpoint level.		DataSource
outputClass (common)	Specify the full package and class name to use as conversion when outputType=SelectOne.		String
outputHeader (common)	Store the query result in a header instead of the message body. By default, outputHeader == null and the query result is stored in the message body, any existing content in the message body is discarded. If outputHeader is set, the value is used as the name of the header to store the query result and the original message body is preserved.		String

Name	Description	Default	Type
outputType (common)	Make the output of consumer or producer to SelectList as List of Map, or SelectOne as single Java object in the following way: a) If the query has only single column, then that JDBC Column object is returned. (such as SELECT COUNT() FROM PROJECT will return a Long object. b) If the query has more than one column, then it will return a Map of that result. c) If the outputClass is set, then it will convert the query result into an Java bean object by calling all the setters that match the column names. It will assume your class has a default constructor to create instance with. d) If the query resulted in more than one rows, it throws an non-unique result exception. StreamList streams the result of the query using an Iterator. This can be used with the Splitter EIP in streaming mode to process the ResultSet in streaming fashion. Enum values: ● SelectOne ● SelectList ● StreamList	Select List	SqlOutputType
separator (common)	The separator to use when parameter values is taken from message body (if the body is a String type), to be inserted at # placeholders. Notice if you use named parameters, then a Map type is used instead. The default value is comma.	,	char
breakBatchOnConsumeFail (consumer)	Sets whether to break batch if onConsume failed.	false	boolean
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
expectedUpdateCount (consumer)	Sets an expected update count to validate when using onConsume.	-1	int

Name	Description	Default	Type
maxMessagesPerPoll (consumer)	Sets the maximum number of messages to poll.		int
onConsume (consumer)	After processing each row then this query can be executed, if the Exchange was processed successfully, for example to mark the row as processed. The query can have parameter.		String
onConsumeBatchComplete (consumer)	After processing the entire batch, this query can be executed to bulk update rows etc. The query cannot have parameters.		String
onConsumeFailed (consumer)	After processing each row then this query can be executed, if the Exchange failed, for example to mark the row as failed. The query can have parameter.		String
routeEmptyResultSet (consumer)	Sets whether empty resultset should be allowed to be sent to the next hop. Defaults to false. So the empty resultset will be filtered out.	false	boolean
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
transacted (consumer)	Enables or disables transaction. If enabled then if processing an exchange failed then the consumer breaks out processing any further exchanges to cause a rollback eager.	false	boolean
useIterator (consumer)	Sets how resultset should be delivered to route. Indicates delivery as either a list or individual object. defaults to true.	true	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
processingStrategy (consumer (advanced))	Allows to plugin to use a custom <code>org.apache.camel.component.sql.SqlProcessingStrategy</code> to execute queries when the consumer has processed the rows/batch.		SqlProcessingStrategy
batch (producer)	Enables or disables batch mode.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
noop (producer)	If set, will ignore the results of the SQL query and use the existing IN message as the OUT message for the continuation of processing.	false	boolean
useMessageBodyForSql (producer)	Whether to use the message body as the SQL and then headers for parameters. If this option is enabled then the SQL in the uri is not used. Note that query parameters in the message body are represented by a question mark instead of a # symbol.	false	boolean

Name	Description	Default	Type
alwaysPopulateStatement (advanced)	If enabled then the populateStatement method from org.apache.camel.component.sql.SqlPrepareStatementStrategy is always invoked, also if there is no expected parameters to be prepared. When this is false then the populateStatement is only invoked if there is 1 or more expected parameters to be set; for example this avoids reading the message body/headers for SQL queries with no parameters.	false	boolean
parametersCount (advanced)	If set greater than zero, then Camel will use this count value of parameters to replace instead of querying via JDBC metadata API. This is useful if the JDBC vendor could not return correct parameters count, then user may override instead.		int
placeholder (advanced)	Specifies a character that will be replaced to in SQL query. Notice, that it is simple String.replaceAll() operation and no SQL parsing is involved (quoted strings will also change).	#	String
prepareStatementStrategy (advanced)	Allows to plugin to use a custom org.apache.camel.component.sql.SqlPrepareStatementStrategy to control preparation of the query and prepared statement.		SqlPrepareStatementStrategy
templateOptions (advanced)	Configures the Spring JdbcTemplate with the key/values from the Map.		Map
usePlaceholder (advanced)	Sets whether to use placeholder and replace all placeholder characters with sign in the SQL queries.	true	boolean
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long

Name	Description	Default	Type
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: ● TRACE ● DEBUG ● INFO ● WARN ● ERROR ● OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean

Name	Description	Default	Type
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANoseconds • MICROseconds • MILLIseconds • SECONDS • MINUTES • HOURS • DAYS	MILLIS ECON DS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean

141.6. TREATMENT OF THE MESSAGE BODY

The SQL component tries to convert the message body to an object of **java.util.Iterator** type and then uses this iterator to fill the query parameters (where each query parameter is represented by a **#** symbol (or configured placeholder) in the endpoint URI). If the message body is not an array or collection, the conversion results in an iterator that iterates over only one object, which is the body itself.

For example, if the message body is an instance of **java.util.List**, the first item in the list is substituted into the first occurrence of **#** in the SQL query, the second item in the list is substituted into the second occurrence of **#**, and so on.

If **batch** is set to **true**, then the interpretation of the inbound message body changes slightly – instead of an iterator of parameters, the component expects an iterator that contains the parameter iterators; the size of the outer iterator determines the batch size.

You can use the option **useMessageBodyForSql** that allows to use the message body as the SQL statement, and then the SQL parameters must be provided in a header with the key **SqlConstants.SQL_PARAMETERS**. This allows the SQL component to work more dynamically as the SQL query is from the message body. Use templating (such as [Velocity](#), [Freemarker](#)) for conditional processing, e.g. to include or exclude **where** clauses depending on the presence of query parameters.

141.7. RESULT OF THE QUERY

For **select** operations, the result is an instance of **List<Map<String, Object>>** type, as returned by the [JdbcTemplate.queryForList\(\)](#) method. For **update** operations, a **NULL** body is returned as the **update** operation is only set as a header and never as a body.



NOTE

See [Header Values](#) for more information on the **update** operation.

By default, the result is placed in the message body. If the `outputHeader` parameter is set, the result is placed in the header. This is an alternative to using a full message enrichment pattern to add headers, it provides a concise syntax for querying a sequence or some other small value into a header. It is convenient to use `outputHeader` and `outputType` together:

```
from("jms:order.inbox")
  .to("sql:select order_seq.nextval from dual?outputHeader=OrderId&outputType=SelectOne")
  .to("jms:order.booking");
```

141.8. USING STREAMLIST

The producer supports `outputType=StreamList` that uses an iterator to stream the output of the query. This allows to process the data in a streaming fashion which for example can be used by the Splitter EIP to process each row one at a time, and load data from the database as needed.

```
from("direct:withSplitModel")
  .to("sql:select * from projects order by id?
outputType=StreamList&outputClass=org.apache.camel.component.sql.ProjectModel")
  .to("log:stream")
  .split(body()).streaming()
  .to("log:row")
  .to("mock:result")
  .end();
```

141.9. HEADER VALUES

When performing **update** operations, the SQL Component stores the update count in the following message headers:

Header	Description
CamelSqlUpdateCount	The number of rows updated for update operations, returned as an Integer object. This header is not provided when using <code>outputType=StreamList</code> .
CamelSqlRowCount	The number of rows returned for select operations, returned as an Integer object. This header is not provided when using <code>outputType=StreamList</code> .
CamelSqlQuery	Query to execute. This query takes precedence over the query specified in the endpoint URI. Note that query parameters in the header are represented by a ? instead of a # symbol

When performing **insert** operations, the SQL Component stores the rows with the generated keys and number of these rows in the following message headers:

Header	Description
CamelSqlGeneratedKeysRowCount	The number of rows in the header that contains generated keys.
CamelSqlGeneratedKeysRows	Rows that contains the generated keys (a list of maps of keys).

141.10. GENERATED KEYS

If you insert data using SQL INSERT, then the RDBMS may support auto generated keys. You can instruct the SQL producer to return the generated keys in headers.

To do that set the header **CamelSqlRetrieveGeneratedKeys=true**. Then the generated keys will be provided as headers with the keys listed in the table above.

To specify which generated columns should be retrieved, set the header **CamelSqlGeneratedColumns** to a **String[]** or **int[]**, indicating the column names or indexes, respectively. Some databases requires this, such as Oracle. It may also be necessary to use the **parametersCount** option if the driver cannot correctly determine the number of parameters.

You can see more details in this [unit test](#).

141.11. DATASOURCE

You can set a reference to a **DataSource** in the URI directly:

```
sql:select * from table where id=# order by name?dataSource=#myDS
```

141.12. USING NAMED PARAMETERS

In the given route below, we want to get all the projects from the projects table. Notice the SQL query has 2 named parameters, `:#lic` and `:#min`.

Camel will then lookup for these parameters from the message body or message headers. Notice in the example above we set two headers with constant value for the named parameters:

```
from("direct:projects")
  .setHeader("lic", constant("ASF"))
  .setHeader("min", constant(123))
  .to("sql:select * from projects where license = :#lic and id > :#min order by id")
```

Though if the message body is a **java.util.Map** then the named parameters will be taken from the body.

```
from("direct:projects")
  .to("sql:select * from projects where license = :#lic and id > :#min order by id")
```

141.13. USING EXPRESSION PARAMETERS IN PRODUCERS

In the given route below, we want to get all the project from the database. It uses the body of the exchange for defining the license and uses the value of a property as the second parameter.

```
from("direct:projects")
  .setBody(constant("ASF"))
  .setProperty("min", constant(123))
  .to("sql:select * from projects where license = :${body} and id > :${exchangeProperty.min} order
    by id")
```

141.13.1. Using expression parameters in consumers

When using the SQL component as consumer, you can now also use expression parameters (simple language) to build dynamic query parameters, such as calling a method on a bean to retrieve an id, date or something.

For example in the sample below we call the nextId method on the bean myIdGenerator:

```
from("sql:select * from projects where id = :${bean:myIdGenerator.nextId}")
  .to("mock:result");
```

And the bean has the following method:

```
public static class MyIdGenerator {

    private int id = 1;

    public int nextId() {
        return id++;
    }
}
```

Notice that there is no existing **Exchange** with message body and headers, so the simple expression you can use in the consumer are most useable for calling bean methods as in this example.

141.14. USING IN QUERIES WITH DYNAMIC VALUES

The SQL producer allows to use SQL queries with IN statements where the IN values is dynamic computed. For example from the message body or a header etc.

To use IN you need to:

- prefix the parameter name with **in:**
- add () around the parameter

An example explains this better. The following query is used:

```
-- this is a comment
select *
from projects
where project in (:#in:names)
order by id
```

In the following route:

-

```
from("direct:query")
  .to("sql:classpath:sql/selectProjectsIn.sql")
  .to("log:query")
  .to("mock:query");
```

Then the IN query can use a header with the key names with the dynamic values such as:

```
// use an array
template.requestBodyAndHeader("direct:query", "Hi there!", "names", new String[]{"Camel", "AMQ"});

// use a list
List<String> names = new ArrayList<String>();
names.add("Camel");
names.add("AMQ");

template.requestBodyAndHeader("direct:query", "Hi there!", "names", names);

// use a string separated values with comma
template.requestBodyAndHeader("direct:query", "Hi there!", "names", "Camel,AMQ");
```

The query can also be specified in the endpoint instead of being externalized (notice that externalizing makes maintaining the SQL queries easier)

```
from("direct:query")
  .to("sql:select * from projects where project in (:#in:names) order by id")
  .to("log:query")
  .to("mock:query");
```

141.15. USING THE JDBC BASED IDEMPOTENT REPOSITORY

In this section we will use the JDBC based idempotent repository.



NOTE

Abstract class

There is an abstract class

org.apache.camel.processor.idempotent.jdbc.AbstractJdbcMessageIdRepository
you can extend to build custom JDBC idempotent repository.

First we have to create the database table which will be used by the idempotent repository. We use the following schema:

```
CREATE TABLE CAMEL_MESSAGEPROCESSED ( processorName VARCHAR(255),
  messageId VARCHAR(100) )
```

We added the createdAt column:

```
CREATE TABLE CAMEL_MESSAGEPROCESSED ( processorName VARCHAR(255),
  messageId VARCHAR(100), createdAt TIMESTAMP )
```

**NOTE**

The SQL Server **TIMESTAMP** type is a fixed-length binary-string type. It does not map to any of the JDBC time types: **DATE**, **TIME**, or **TIMESTAMP**.

When working with concurrent consumers it is crucial to create a unique constraint on the columns `processorName` and `messageId`. Because the syntax for this constraint differs from database to database, we do not show it here.

141.15.1. Customize the JDBC idempotency repository

You have a few options to tune the `org.apache.camel.processor.idempotent.jdbc.JdbcMessageIdRepository` for your needs:

Parameter	Default Value	Description
<code>createTableIfNotExists</code>	<code>true</code>	Defines whether or not Camel should try to create the table if it doesn't exist.
<code>tableName</code>	<code>CAMEL_MESSAGEPROCESSED</code>	To use a custom table name instead of the default name: <code>CAMEL_MESSAGEPROCESSED</code> .
<code>tableExistsString</code>	<code>SELECT 1 FROM CAMEL_MESSAGEPROCESSED WHERE 1 = 0</code>	This query is used to figure out whether the table already exists or not. It must throw an exception to indicate the table doesn't exist.
<code>createString</code>	<code>CREATE TABLE CAMEL_MESSAGEPROCESSED (processorName VARCHAR(255), messageId VARCHAR(100), createdAt TIMESTAMP)</code>	The statement which is used to create the table.
<code>queryString</code>	<code>SELECT COUNT(*) FROM CAMEL_MESSAGEPROCESSED WHERE processorName = ? AND messageId = ?</code>	The query which is used to figure out whether the message already exists in the repository (the result is not equals to '0'). It takes two parameters. This first one is the processor name (String) and the second one is the message id (String).
<code>insertString</code>	<code>INSERT INTO CAMEL_MESSAGEPROCESSED (processorName, messageId, createdAt) VALUES (?, ?, ?)</code>	The statement which is used to add the entry into the table. It takes three parameter. The first one is the processor name (String), the second one is the message id (String) and the third one is the timestamp (java.sql.Timestamp) when this entry was added to the repository.

Parameter	Default Value	Description
deleteString	DELETE FROM CAMEL_MESSAGEPRO CESSED WHERE processorName = ? AND messageId = ?	The statement which is used to delete the entry from the database. It takes two parameter. This first one is the processor name (String) and the second one is the message id (String).

The option **tableName** can be used to use the default SQL queries but with a different table name. However if you want to customize the SQL queries then you can configure each of them individually.

141.15.2. Orphan Lock aware Jdbc IdempotentRepository

One of the limitations of **org.apache.camel.processor.idempotent.jdbc.JdbcMessageIdRepository** is that it does not handle orphan locks resulting from JVM crash or non graceful shutdown. This can result in unprocessed files/messages if this implementation is used with camel-file, camel-ftp etc. if you need to address orphan locks processing then use **org.apache.camel.processor.idempotent.jdbc.JdbcOrphanLockAwareIdempotentRepository**. This repository keeps track of the locks held by an instance of the application. For each lock held, the application will send keep alive signals to the lock repository resulting in updating the createdAt column with the current Timestamp. When an application instance tries to acquire a lock if the, then there are three possibilities exist :

- lock entry does not exist then the lock is provided using the base implementation of **JdbcMessageIdRepository**.
- lock already exists and the `createdAt < System.currentTimeMillis() - lockMaxAgeMillis`. In this case it is assumed that an active instance has the lock and the lock is not provided to the new instance requesting the lock
- lock already exists and the `createdAt >= System.currentTimeMillis() - lockMaxAgeMillis`. In this case it is assumed that there is no active instance which has the lock and the lock is provided to the requesting instance. The reason behind is that if the original instance which had the lock, if it was still running, it would have updated the Timestamp on createdAt using its keepAlive mechanism

This repository has two additional configuration parameters

Parameter	Description
lockMaxAgeMillis	This refers to the duration after which the lock is considered orphaned i.e. if the <code>currentTimestamp - createdAt >= lockMaxAgeMillis</code> then lock is orphaned.
lockKeepAliveIntervalMillis	The frequency at which keep alive updates are done to createdAt Timestamp column.

141.15.3. Caching Jdbc IdempotentRepository

Some SQL implementations are not fast on a per query basis. The **JdbcMessageIdRepository**

implementation does its idempotent checks individually within SQL transactions. Checking a mere 100 keys can take minutes. The **JdbcCachedMessageIdRepository** preloads an in-memory cache on start with the entire list of keys. This cache is then checked first before passing through to the original implementation.

As with all cache implementations, there are considerations that should be made with regard to stale data and your specific usage.

141.16. USING THE JDBC BASED AGGREGATION REPOSITORY

JdbcAggregationRepository is an **AggregationRepository** which on the fly persists the aggregated messages. This ensures that you will not lose messages, as the default aggregator will use an in-memory only **AggregationRepository**. The **JdbcAggregationRepository** allows together with Camel to provide persistent support for the Aggregator.

Only when an Exchange has been successfully processed it will be marked as complete which happens when the **confirm** method is invoked on the **AggregationRepository**. This means if the same Exchange fails again it will be kept retried until it success.

You can use option **maximumRedeliveries** to limit the maximum number of redelivery attempts for a given recovered Exchange. You must also set the **deadLetterUri** option so Camel knows where to send the Exchange when the **maximumRedeliveries** was hit.

You can see some examples in the unit tests of camel-sql, for example

JdbcAggregateRecoverDeadLetterChannelTest.java

141.16.1. Database

To be operational, each aggregator uses two table: the aggregation and completed one. By convention the completed has the same name as the aggregation one suffixed with "**_COMPLETED**". The name must be configured in the Spring bean with the **RepositoryName** property. In the following example aggregation will be used.

The table structure definition of both table are identical: in both case a String value is used as key (**id**) whereas a Blob contains the exchange serialized in byte array.

However one difference should be remembered: the **id** field does not have the same content depending on the table.

In the aggregation table **id** holds the correlation Id used by the component to aggregate the messages. In the completed table, **id** holds the id of the exchange stored in corresponding the blob field.

Here is the SQL query used to create the tables, just replace "**aggregation**" with your aggregator repository name.

```
CREATE TABLE aggregation (
  id varchar(255) NOT NULL,
  exchange blob NOT NULL,
  version BIGINT NOT NULL,
  constraint aggregation_pk PRIMARY KEY (id)
);
CREATE TABLE aggregation_completed (
  id varchar(255) NOT NULL,
  exchange blob NOT NULL,
  version BIGINT NOT NULL,
  constraint aggregation_completed_pk PRIMARY KEY (id)
);
```

141.17. STORING BODY AND HEADERS AS TEXT

You can configure the **JdbcAggregationRepository** to store message body and select(ed) headers as String in separate columns. For example to store the body, and the following two headers **companyName** and **accountName** use the following SQL:

```
CREATE TABLE aggregationRepo3 (
  id varchar(255) NOT NULL,
  exchange blob NOT NULL,
  version BIGINT NOT NULL,
  body varchar(1000),
  companyName varchar(1000),
  accountName varchar(1000),
  constraint aggregationRepo3_pk PRIMARY KEY (id)
);
CREATE TABLE aggregationRepo3_completed (
  id varchar(255) NOT NULL,
  exchange blob NOT NULL,
  version BIGINT NOT NULL,
  body varchar(1000),
  companyName varchar(1000),
  accountName varchar(1000),
  constraint aggregationRepo3_completed_pk PRIMARY KEY (id)
);
```

And then configure the repository to enable this behavior as shown below:

```
<bean id="repo3"
  class="org.apache.camel.processor.aggregate.jdbc.JdbcAggregationRepository">
  <property name="repositoryName" value="aggregationRepo3"/>
  <property name="transactionManager" ref="txManager3"/>
  <property name="dataSource" ref="dataSource3"/>
  <!-- configure to store the message body and following headers as text in the repo -->
  <property name="storeBodyAsText" value="true"/>
  <property name="headersToStoreAsText">
    <list>
      <value>companyName</value>
      <value>accountName</value>
    </list>
  </property>
</bean>
```

141.17.1. Codec (Serialization)

Since they can contain any type of payload, Exchanges are not serializable by design. It is converted into a byte array to be stored in a database BLOB field. All those conversions are handled by the **JdbcCodec** class. One detail of the code requires your attention: the **ClassLoadingAwareObjectInputStream**.

The **ClassLoadingAwareObjectInputStream** has been reused from the [Apache ActiveMQ](#) project. It wraps an **ObjectInputStream** and use it with the **ContextClassLoader** rather than the **currentThread** one. The benefit is to be able to load classes exposed by other bundles. This allows the exchange body and headers to have custom types object references.

141.17.2. Transaction

A Spring **PlatformTransactionManager** is required to orchestrate transaction.

141.17.2.1. Service (Start/Stop)

The **start** method verify the connection of the database and the presence of the required tables. If anything is wrong it will fail during starting.

141.17.3. Aggregator configuration

Depending on the targeted environment, the aggregator might need some configuration. As you already know, each aggregator should have its own repository (with the corresponding pair of table created in the database) and a data source. If the default lobHandler is not adapted to your database system, it can be injected with the **lobHandler** property.

Here is the declaration for Oracle:

```
<bean id="lobHandler" class="org.springframework.jdbc.support.lob.OracleLobHandler">
  <property name="nativeJdbcExtractor" ref="nativeJdbcExtractor"/>
</bean>
<bean id="nativeJdbcExtractor"
  class="org.springframework.jdbc.support.nativejdbc.CommonsDbcpNativeJdbcExtractor"/>
<bean id="repo"
  class="org.apache.camel.processor.aggregate.jdbc.JdbcAggregationRepository">
  <property name="transactionManager" ref="transactionManager"/>
  <property name="repositoryName" value="aggregation"/>
  <property name="dataSource" ref="dataSource"/>
  <!-- Only with Oracle, else use default -->
  <property name="lobHandler" ref="lobHandler"/>
</bean>
```

141.17.4. Optimistic locking

You can turn on **optimisticLocking** and use this JDBC based aggregation repository in a clustered environment where multiple Camel applications shared the same database for the aggregation repository. If there is a race condition there JDBC driver will throw a vendor specific exception which the **JdbcAggregationRepository** can react upon. To know which caused exceptions from the JDBC driver is regarded as an optimistic locking error we need a mapper to do this. Therefore there is a **org.apache.camel.processor.aggregate.jdbc.JdbcOptimisticLockingExceptionMapper** allows you to implement your custom logic if needed. There is a default implementation **org.apache.camel.processor.aggregate.jdbc.DefaultJdbcOptimisticLockingExceptionMapper** which works as follows:

The following check is done:

- If the caused exception is an **SQLException** then the SQLState is checked if starts with 23.
- If the caused exception is a **DataIntegrityViolationException**
- If the caused exception class name has "ConstraintViolation" in its name.
- Optional checking for FQN class name matches if any class names has been configured.

You can in addition add FQN classnames, and if any of the caused exception (or any nested) equals any of the FQN class names, then its an optimistick locking error.

Here is an example, where we define 2 extra FQN class names from the JDBC vendor.

```
<bean id="repo"
class="org.apache.camel.processor.aggregate.jdbc.JdbcAggregationRepository">
  <property name="transactionManager" ref="transactionManager"/>
  <property name="repositoryName" value="aggregation"/>
  <property name="dataSource" ref="dataSource"/>
  <property name="jdbcOptimisticLockingExceptionMapper" ref="myExceptionMapper"/>
</bean>
<!-- use the default mapper with extraFQN class names from our JDBC driver -->
<bean id="myExceptionMapper"
class="org.apache.camel.processor.aggregate.jdbc.DefaultJdbcOptimisticLockingExceptionMapper">
  <property name="classNames">
    <util:set>
      <value>com.foo.sql.MyViolationExceptoion</value>
      <value>com.foo.sql.MyOtherViolationExceptoion</value>
    </util:set>
  </property>
</bean>
```

141.17.5. Propagation behavior

JdbcAggregationRepository uses two distinct *transaction templates* from Spring-TX. One is read-only and one is used for read-write operations.

However, when using **JdbcAggregationRepository** within a route that itself uses **<transacted />** and there's common **PlatformTransactionManager** used, there may be a need to configure *propagation behavior* used by transaction templates inside **JdbcAggregationRepository**.

Here's a way to do it:

```
<bean id="repo"
class="org.apache.camel.processor.aggregate.jdbc.JdbcAggregationRepository">
  <property name="propagationBehaviorName" value="PROPAGATION_NESTED" />
</bean>
```

Propagation is specified by constants of **org.springframework.transaction.TransactionDefinition** interface, so **propagationBehaviorName** is convenient setter that allows to use names of the constants.

141.17.6. PostgreSQL case

There's special database that may cause problems with optimistic locking used by **JdbcAggregationRepository**. PostgreSQL marks connection as invalid in case of data integrity violation exception (the one with SQLState 23505). This makes the connection effectively unusable within nested transaction. Details can be found in the [document](#)..

org.apache.camel.processor.aggregate.jdbc.PostgresAggregationRepository extends **JdbcAggregationRepository** and uses special **INSERT .. ON CONFLICT ..** statement to provide optimistic locking behavior.

This statement is (with default aggregation table definition):

```
INSERT INTO aggregation (id, exchange) values (?, ?) ON CONFLICT DO NOTHING
```

Details can be found in [PostgreSQL documentation](#).

When this clause is used, `java.sql.PreparedStatement.executeUpdate()` call returns **0** instead of throwing `SQLException` with `SQLState=23505`. Further handling is exactly the same as with generic **JdbcAggregationRepository**, but without marking PostgreSQL connection as invalid.

141.18. CAMEL SQL STARTER

A starter module is available to spring-boot users. When using the starter, the **DataSource** can be directly configured using spring-boot properties.

```
# Example for a mysql datasource
spring.datasource.url=jdbc:mysql://localhost/test
spring.datasource.username=dbuser
spring.datasource.password=dbpass
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

To use this feature, add the following dependencies to your spring boot pom.xml file:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-sql-starter</artifactId>
  <version>${camel.version}</version> <!-- use the same version as your Camel core version -->
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
  <version>${spring-boot-version}</version>
</dependency>
```

You should also include the specific database driver, if needed.

141.19. SPRING BOOT AUTO-CONFIGURATION

The component supports 8 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.sql-stored.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.sql-stored.enabled	Whether to enable auto configuration of the sql-stored component. This is enabled by default.		Boolean
camel.component.sql-stored.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.sql.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.sql.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.sql.enabled	Whether to enable auto configuration of the sql component. This is enabled by default.		Boolean
camel.component.sql.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.sql.use-placeholder	Sets whether to use placeholder and replace all placeholder characters with sign in the SQL queries. This option is default true.	true	Boolean

CHAPTER 142. SQL STORED PROCEDURE

Since Camel 2.17

Only producer is supported

The SQL Stored component allows you to work with databases using JDBC Stored Procedure queries. This component is an extension to the [SQL](#) component but specialized for calling stored procedures.

This component uses **spring-jdbc** behind the scenes for the actual SQL handling.

142.1. DEPENDENCIES

When using **camel-sql** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-sql-starter</artifactId>
</dependency>
```

142.2. URI FORMAT

The SQL component uses the following endpoint URI notation:

```
sql-stored:template[?options]
```

Where template is the stored procedure template, where you declare the name of the stored procedure and the IN, INOUT, and OUT arguments.

You can also refer to the template in an external file on the file system or classpath such as:

```
sql-stored:classpath:sql/myprocedure.sql[?options]
```

Where sql/myprocedure.sql is a plain text file in the classpath with the template, as show:

```
SUBNUMBERS(
  INTEGER ${headers.num1},
  INTEGER ${headers.num2},
  INOUT INTEGER ${headers.num3} out1,
  OUT INTEGER out2
)
```

142.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

142.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

142.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

142.4. COMPONENT OPTIONS

The SQL Stored Procedure component supports 3 options, which are listed below.

Name	Description	Default	Type
dataSource (producer)	Autowired Sets the DataSource to use to communicate with the database.		DataSource

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

142.5. ENDPOINT OPTIONS

The SQL Stored Procedure endpoint is configured using URI syntax:

```
sql-stored:template
```

With the following *path* and *query* parameters:

142.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
template (producer)	Required Sets the stored procedure template to perform. You can externalize the template by using file: or classpath: as prefix and specify the location of the file.		String

142.5.2. Query Parameters (8 parameters)

Name	Description	Default	Type
batch (producer)	Enables or disables batch mode.	false	boolean

Name	Description	Default	Type
dataSource (producer)	Sets the DataSource to use to communicate with the database.		DataSource
function (producer)	Whether this call is for a function.	false	boolean
noop (producer)	If set, will ignore the results of the stored procedure template and use the existing IN message as the OUT message for the continuation of processing.	false	boolean
outputHeader (producer)	Store the template result in a header instead of the message body. By default, outputHeader == null and the template result is stored in the message body, any existing content in the message body is discarded. If outputHeader is set, the value is used as the name of the header to store the template result and the original message body is preserved.		String
useMessageBodyForTemplate (producer)	Whether to use the message body as the stored procedure template and then headers for parameters. If this option is enabled then the template in the uri is not used.	false	boolean
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
templateOptions (advanced)	Configures the Spring JdbcTemplate with the key/values from the Map.		Map

142.6. MESSAGE HEADERS

The SQL Stored Procedure component supports 3 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelSqlStoredTemplate (producer) Constant: SQL_STORED_TEMPLATE	The template.		String
CamelSqlStoredParameters (producer) Constant: SQL_STORED_PARAMETERS	The parameters.		Iterator
CamelSqlStoredUpdateCount (producer) Constant: SQL_STORED_UPDATE_COUNT	The update count.		Integer

142.7. DECLARING THE STORED PROCEDURE TEMPLATE

The template is declared using a syntax that would be similar to a Java method signature. The name of the stored procedure, and then the arguments enclosed in parentheses. An example explains this well:

```
<to uri="sql-stored:STOREDSAMPLE(INTEGER ${headers.num1},INTEGER
${headers.num2},INOUT INTEGER ${headers.num3} result1,OUT INTEGER result2)"/>
```

The arguments are declared by a type and then a mapping to the Camel message using simple expression. So, in this example, the first two parameters are IN values of INTEGER type, mapped to the message headers. The third parameter is INOUT, meaning it accepts an INTEGER and then returns a different INTEGER result. The last parameter is the OUT value, also an INTEGER type.

In SQL terms, the stored procedure could be declared as:

```
CREATE PROCEDURE STOREDSAMPLE(VALUE1 INTEGER, VALUE2 INTEGER, INOUT
RESULT1 INTEGER, OUT RESULT2 INTEGER)
```

142.7.1. IN Parameters

IN parameters take four parts separated by a space: parameter name, SQL type (with scale), type name, and value source.

Parameter name is optional and will be auto generated if not provided. It must be given between quotes('').

SQL type is required and can be an integer (positive or negative) or reference to integer field in some class. If SQL type contains a dot, then the component tries to resolve that class and read the given field. For example, SQL type **com.Foo.INTEGER** is read from the field **INTEGER** of class **com.Foo**. If the type doesn't contain comma then class to resolve the integer value will be **java.sql.Types**. Type can be postfixed by scale for example **DECIMAL(10)** would mean **java.sql.Types.DECIMAL** with scale 10.

Type name is optional and must be given between quotes(').

Value source is required. Value source populates the parameter value from the Exchange. It can be either a Simple expression or header location i.e. **:#<header name>**. For example, the Simple expression **\${header.val}** would mean that parameter value will be read from the header **val**. Header location expression **:#val** would have identical effect.

```
<to uri="sql-stored:MYFUNC('param1' org.example.Types.INTEGER(10) ${header.srcValue})"/>
```

URI means that the stored procedure will be called with parameter name *param1*, it's SQL type is read from field **INTEGER** of class **org.example.Types** and scale will be set to 10. Input value for the parameter is passed from the header *srcValue*.

```
<to uri="sql-stored:MYFUNC('param1' 100 'mytypename' ${header.srcValue})"/>
```

URI is identical to previous on except SQL-type is 100 and type name is *mytypename*.

Actual call will be done using **org.springframework.jdbc.core.SqlParameter**.

142.7.2. OUT Parameters

OUT parameters work similarly IN parameters and contain three parts: SQL type(with scale), type name, and output parameter name.

SQL type works the same as IN parameters.

Type name is optional and also works the same as IN parameters.

Output parameter name is used for the OUT parameter name, as well as the header name where the result will be stored.

```
<to uri="sql-stored:MYFUNC(OUT org.example.Types.DECIMAL(10) outheader1)"/>
```

URI means that the OUT parameter's name is **outheader1** and result will be but into header **outheader1**.

```
<to uri="sql-stored:MYFUNC(OUT org.example.Types.NUMERIC(10) 'mytype' outheader1)"/>
```

This is identical to previous one but type name will be **mytype**.

Actual call will be done using **org.springframework.jdbc.core.SqlOutParameter**.

142.7.3. INOUT Parameters

INOUT parameters are a combination of all of the above. They receive a value from the exchange, as well as store a result as a message header. The only caveat is that the IN parameter's "name" is skipped. Instead, the OUT parameter's *name* defines both the SQL parameter name, and the result header name.

```
<to uri="sql-stored:MYFUNC(INOUT DECIMAL(10) ${headers.inheader} outheader)"/>
```

Actual call will be done using **org.springframework.jdbc.core.SqlInOutParameter**.

142.7.4. Query Timeout

You can configure query timeout (via **template.queryTimeout**) on statements used for query processing as shown:

```
<to uri="sql-stored:MYFUNC(INOUT DECIMAL(10) ${headers.inheader} outheader)?  
template.queryTimeout=5000"/>
```

This will be overridden by the remaining transaction timeout when executing within a transaction that has a timeout specified at the transaction level.

142.8. CAMEL SQL STARTER

A starter module is available to spring boot users. When using the starter, the **DataSource** can be directly configured using spring-boot properties.

```
# Example for a mysql datasource  
spring.datasource.url=jdbc:mysql://localhost/test  
spring.datasource.username=dbuser  
spring.datasource.password=dbpass  
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

To use this feature, add the following dependencies to your spring boot pom.xml file:

```
<dependency>  
  <groupId>org.apache.camel.springboot</groupId>  
  <artifactId>camel-sql-starter</artifactId>  
</dependency>  
  
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-jdbc</artifactId>  
  <version>${spring-boot-version}</version>  
</dependency>
```

You can also include the specific database driver, if needed.

142.9. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
camel.component.sql-stored.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.sql-stored.enabled	Whether to enable auto configuration of the sql-stored component. This is enabled by default.		Boolean
camel.component.sql-stored.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.sql.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.sql.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean

Name	Description	Default	Type
camel.component.sql.enabled	Whether to enable auto configuration of the sql component. This is enabled by default.		Boolean
camel.component.sql.health-check-consumer-enabled	Used for enabling or disabling all consumer based health checks from this component.	true	Boolean
camel.component.sql.health-check-producer-enabled	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting camel.health.producersEnabled=true.	true	Boolean
camel.component.sql.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.sql.row-mapper-factory	Factory for creating RowMapper. The option is a org.apache.camel.component.sql.RowMapperFactory type.		RowMapperFactory
camel.component.sql.use-placeholder	Sets whether to use placeholder and replace all placeholder characters with sign in the SQL queries. This option is default true.	true	Boolean

CHAPTER 143. SSH

Since Camel 2.10

Both producer and consumer are supported

The SSH component enables access to SSH servers so that you can send an SSH command and process the response.

143.1. DEPENDENCIES

When using **camel-ssh** with Red Hat build of Camel Spring Boot, add the following Maven dependency to your **pom.xml** to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-ssh-starter</artifactId>
</dependency>
```

143.2. URI FORMAT

```
ssh:[username[:password]@]host[:port][?options]
```

143.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

143.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

143.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

143.4. COMPONENT OPTIONS

The SSH component supports 25 options, which are listed below.

Name	Description	Default	Type
failOnUnknownHost (common)	Specifies whether a connection to an unknown host should fail or not. This value is only checked when the property knownHosts is set.	false	boolean
knownHostsResource (common)	Sets the resource path for a known_hosts file.		String
timeout (common)	Sets the timeout in milliseconds to wait in establishing the remote SSH server connection. Defaults to 30000 milliseconds.	30000	long
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
pollCommand (consumer)	Sets the command string to send to the remote SSH server during every poll cycle. Only works with camel-ssh component being used as a consumer, i.e. from(ssh://...) You may need to end your command with a newline, and that must be URL encoded %0A.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
channelType (advanced)	Sets the channel type to pass to the Channel as part of command execution. Defaults to exec.	exec	String
clientBuilder (advanced)	Autowired Instance of ClientBuilder used by the producer or consumer to create a new SshClient.		ClientBuilder

Name	Description	Default	Type
compressions (advanced)	Whether to use compression, and if so which.		String
configuration (advanced)	Component configuration.		SshConfiguration
shellPrompt (advanced)	Sets the shellPrompt to be dropped when response is read after command execution.		String
sleepForShellPrompt (advanced)	Sets the sleep period in milliseconds to wait reading response from shell prompt. Defaults to 100 milliseconds.	100	long
healthCheckConsumerEnabled (health)	Used for enabling or disabling all consumer based health checks from this component.	true	boolean
healthCheckProducerEnabled (health)	Used for enabling or disabling all producer based health checks from this component.  NOTE By default all producer based health-checks are disabled. You can turn on producer checks globally by setting camel.health.producersEnabled=true.	true	boolean
certResource (security)	Sets the resource path of the certificate to use for Authentication. Will use ResourceHelperKeyPairProvider to resolve file based certificate, and depends on keyType setting.		String
certResourcePassword (security)	Sets the password to use in loading certResource, if certResource is an encrypted key.		String
ciphers (security)	Comma-separated list of allowed/supported ciphers in their order of preference.		String
kex (security)	Comma-separated list of allowed/supported key exchange algorithms in their order of preference.		String
keyPairProvider (security)	Sets the KeyPairProvider reference to use when connecting using Certificates to the remote SSH Server.		KeyPairProvider

Name	Description	Default	Type
keyType (security)	Sets the key type to pass to the KeyPairProvider as part of authentication. KeyPairProvider.loadKey(...) will be passed this value. From Camel 3.0.0 / 2.25.0, by default Camel will select the first available KeyPair that is loaded. Prior to this, a KeyType of 'ssh-rsa' was enforced by default.		String
macs (security)	Comma-separated list of allowed/supported message authentication code algorithms in their order of preference. The MAC algorithm is used for data integrity protection.		String
password (security)	Sets the password to use in connecting to remote SSH server. Requires keyPairProvider to be set to null.		String
signatures (security)	Comma-separated list of allowed/supported signature algorithms in their order of preference.		String
username (security)	Sets the username to use in logging into the remote SSH server.		String

143.5. ENDPOINT OPTIONS

The SSH endpoint is configured using URI syntax:

```
ssh:host:port
```

With the following *path* and *query* parameters:

143.5.1. Path Parameters (2 parameters)

Name	Description	Default	Type
host (common)	Required Sets the hostname of the remote SSH server.		String
port (common)	Sets the port number for the remote SSH server.	22	int

143.5.2. Query Parameters (39 parameters)

Name	Description	Default	Type
failOnUnknownHost (common)	Specifies whether a connection to an unknown host should fail or not. This value is only checked when the property knownHosts is set.	false	boolean
knownHostsResource (common)	Sets the resource path for a known_hosts file.		String
timeout (common)	Sets the timeout in milliseconds to wait in establishing the remote SSH server connection. Defaults to 30000 milliseconds.	30000	long
pollCommand (consumer)	Sets the command string to send to the remote SSH server during every poll cycle. Only works with camel-ssh component being used as a consumer, i.e. from(ssh://...) You may need to end your command with a newline, and that must be URL encoded %0A.		String
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
channelType (advanced)	Sets the channel type to pass to the Channel as part of command execution. Defaults to exec.	exec	String
clientBuilder (advanced)	Autowired Instance of ClientBuilder used by the producer or consumer to create a new SshClient.		ClientBuilder
compressions (advanced)	Whether to use compression, and if so which.		String
shellPrompt (advanced)	Sets the shellPrompt to be dropped when response is read after command execution.		String
sleepForShellPrompt (advanced)	Sets the sleep period in milliseconds to wait reading response from shell prompt. Defaults to 100 milliseconds.	100	long
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int

Name	Description	Default	Type
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService

Name	Description	Default	Type
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See ScheduledExecutorService in JDK for details.	true	boolean
certResource (security)	Sets the resource path of the certificate to use for Authentication. Will use ResourceHelperKeyPairProvider to resolve file based certificate, and depends on keyType setting.		String
certResourcePassword (security)	Sets the password to use in loading certResource, if certResource is an encrypted key.		String
ciphers (security)	Comma-separated list of allowed/supported ciphers in their order of preference.		String
kex (security)	Comma-separated list of allowed/supported key exchange algorithms in their order of preference.		String
keyPairProvider (security)	Sets the KeyPairProvider reference to use when connecting using Certificates to the remote SSH Server.		KeyPairProvider

Name	Description	Default	Type
keyType (security)	Sets the key type to pass to the KeyPairProvider as part of authentication. KeyPairProvider.loadKey(...) will be passed this value. From Camel 3.0.0 / 2.25.0, by default Camel will select the first available KeyPair that is loaded. Prior to this, a KeyType of 'ssh-rsa' was enforced by default.		String
macs (security)	Comma-separated list of allowed/supported message authentication code algorithms in their order of preference. The MAC algorithm is used for data integrity protection.		String
password (security)	Sets the password to use in connecting to remote SSH server. Requires keyPairProvider to be set to null.		String
signatures (security)	Comma-separated list of allowed/supported signature algorithms in their order of preference.		String
username (security)	Sets the username to use in logging into the remote SSH server.		String

143.6. MESSAGE HEADERS

The SSH component supports 4 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelSshUsername (common) Constant: USERNAME_HEADER	The user name.		String
CamelSshPassword (common) Constant: PASSWORD_HEADER	The password.		String
CamelSshStderr (common) Constant: STDERR	The value of this header is a InputStream with the standard error stream of the executable.		InputStream

Name	Description	Default	Type
CamelSshExitValue (common) Constant: EXIT_VALUE	The value of this header is the exit value that is returned, after the execution. By convention a non-zero status exit value indicates abnormal termination. Note that the exit value is OS dependent.		Integer

143.7. USAGE AS A PRODUCER ENDPOINT

When the SSH Component is used as a Producer (`.to("ssh://...")`), it sends the message body as the command to execute on the remote SSH server.

XML DSL example

Note that the command has an XML encoded newline (`
`).

```
<route id="camel-example-ssh-producer">
  <from uri="direct:exampleSshProducer"/>
  <setBody>
    <constant>features:list&#10;</constant>
  </setBody>
  <to uri="ssh://karaf:karaf@localhost:8101"/>
  <log message="${body}"/>
</route>
```

143.8. AUTHENTICATION

The SSH Component can authenticate against the remote SSH server using one of two mechanisms:

- Public Key certificate
- Username/password

Configuring how the SSH Component does authentication is based on how and which options are set.

1. First, it will look to see if the **certResource** option has been set, and if so, use it to locate the referenced Public Key certificate and use that for authentication.
2. If **certResource** is not set, it will look to see if a **keyPairProvider** has been set, and if so, it will use that for certificate-based authentication.
3. If neither **certResource** nor **keyPairProvider** are set, it will use the **username** and **password** options for authentication. Even though the **username** and **password** are provided in the endpoint configuration and headers set with **SshConstants.USERNAME_HEADER** (**CamelSshUsername**) and **SshConstants.PASSWORD_HEADER** (**CamelSshPassword**), the endpoint configuration is surpassed and credentials set in the headers are used.

The following route fragment shows an SSH polling consumer using a certificate from the classpath.

XML DSL

■

```
<route>
  <from uri="ssh://scott@localhost:8101?
certResource=classpath:test_rsa&useFixedDelay=true&delay=5000&pollCommand=features:list%0A"/>
  <log message="${body}"/>
</route>
```

Java DSL

```
from("ssh://scott@localhost:8101?
certResource=classpath:test_rsa&useFixedDelay=true&delay=5000&pollCommand=features:list%0A"
)
.log("${body}");
```

An example of using Public Key authentication is provided in [examples/camel-example-ssh-security](#).

143.9. CERTIFICATE DEPENDENCIES

You need to add some additional runtime dependencies if you use certificate-based authentication. You may need to use later versions depending on what version of Camel you are using.

The component uses **sshd-core** library which is based on either **bouncycastle** or **eddsa** security providers. **camel-ssh** is picking explicitly **bouncycastle** as security provider.

```
<dependency>
  <groupId>org.apache.sshd</groupId>
  <artifactId>sshd-core</artifactId>
  <version>2.8.0</version>
</dependency>
<dependency>
  <groupId>org.bouncycastle</groupId>
  <artifactId>bcpkg-jdk18on</artifactId>
  <version>1.71</version>
</dependency>
<dependency>
  <groupId>org.bouncycastle</groupId>
  <artifactId>bcpkix-jdk18on</artifactId>
  <version>1.71</version>
</dependency>
```

143.10. SPRING BOOT AUTO-CONFIGURATION

The component supports 26 options, which are listed below.

Name	Description	Default	Type
		t	

Name	Description	Default	Type
camel.component.ssh.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.ssh.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions (if possible) occurred while the Camel consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. Important: This is only possible if the 3rd party component allows Camel to be alerted if an exception was thrown. Some components handle this internally only, and therefore bridgeErrorHandler is not possible. In other situations we may improve the Camel component to hook into the 3rd party component and make this possible for future releases. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.ssh.cert-resource	Sets the resource path of the certificate to use for Authentication. Will use ResourceHelperKeyPairProvider to resolve file based certificate, and depends on keyType setting.		String
camel.component.ssh.cert-resource-password	Sets the password to use in loading certResource, if certResource is an encrypted key.		String
camel.component.ssh.channel-type	Sets the channel type to pass to the Channel as part of command execution. Defaults to exec.	exec	String
camel.component.ssh.ciphers	Comma-separated list of allowed/supported ciphers in their order of preference.		String
camel.component.ssh.client-builder	Instance of ClientBuilder used by the producer or consumer to create a new SshClient. The option is a org.apache.sshd.client.ClientBuilder type.		ClientBuilder
camel.component.ssh.compressions	Whether to use compression, and if so which.		String

Name	Description	Default	Type
camel.component.ssh.configuration	Component configuration. The option is a <code>org.apache.camel.component.ssh.SshConfiguration</code> type.		SshConfiguration
camel.component.ssh.enabled	Whether to enable auto configuration of the ssh component. This is enabled by default.		Boolean
camel.component.ssh.fail-on-unknown-host	Specifies whether a connection to an unknown host should fail or not. This value is only checked when the property <code>knownHosts</code> is set.	false	Boolean
camel.component.ssh.health-check-consumer-enabled	Used for enabling or disabling all consumer based health checks from this component.	true	Boolean
camel.component.ssh.health-check-producer-enabled	Used for enabling or disabling all producer based health checks from this component. Notice: Camel has by default disabled all producer based health-checks. You can turn on producer checks globally by setting <code>camel.health.producersEnabled=true</code> .	true	Boolean
camel.component.ssh.kex	Comma-separated list of allowed/supported key exchange algorithms in their order of preference.		String
camel.component.ssh.key-pair-provider	Sets the <code>KeyPairProvider</code> reference to use when connecting using Certificates to the remote SSH Server. The option is a <code>org.apache.sshd.common.keyprovider.KeyPairProvider</code> type.		KeyPairProvider
camel.component.ssh.key-type	Sets the key type to pass to the <code>KeyPairProvider</code> as part of authentication. <code>KeyPairProvider.loadKey(...)</code> will be passed this value. From Camel 3.0.0 / 2.25.0, by default Camel will select the first available <code>KeyPair</code> that is loaded. Prior to this, a <code>KeyType</code> of 'ssh-rsa' was enforced by default.		String
camel.component.ssh.known-hosts-resource	Sets the resource path for a <code>known_hosts</code> file.		String

Name	Description	Default	Type
camel.component.ssh.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.ssh.macs	Comma-separated list of allowed/supported message authentication code algorithms in their order of preference. The MAC algorithm is used for data integrity protection.		String
camel.component.ssh.password	Sets the password to use in connecting to remote SSH server. Requires keyPairProvider to be set to null.		String
camel.component.ssh.poll-command	Sets the command string to send to the remote SSH server during every poll cycle. Only works with camel-ssh component being used as a consumer, i.e. from(ssh://...) You may need to end your command with a newline, and that must be URL encoded %0A.		String
camel.component.ssh.shell-prompt	Sets the shellPrompt to be dropped when response is read after command execution.		String
camel.component.ssh.signatures	Comma-separated list of allowed/supported signature algorithms in their order of preference.		String
camel.component.ssh.sleep-for-shell-prompt	Sets the sleep period in milliseconds to wait reading response from shell prompt. Defaults to 100 milliseconds.	100	Long
camel.component.ssh.timeout	Sets the timeout in milliseconds to wait in establishing the remote SSH server connection. Defaults to 30000 milliseconds.	30000	Long
camel.component.ssh.username	Sets the username to use in logging into the remote SSH server.		String

CHAPTER 144. STUB

Both producer and consumer are supported

The Stub component provides a simple way to stub out any physical endpoints while in development or testing, allowing you for example to run a route without needing to actually connect to a specific [SMTP](#) or [HTTP](#) endpoint. Just add **stub:** in front of any endpoint URI to stub out the endpoint.

Internally the Stub component creates [VM](#) endpoints. The main difference between Stub and [VM](#) is that VM will validate the URI and parameters you give it, so putting vm: in front of a typical URI with query arguments will usually fail. Stub won't though, as it basically ignores all query parameters to let you quickly stub out one or more endpoints in your route temporarily.

144.1. DEPENDENCIES

When using **stub** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-stub-starter</artifactId>
</dependency>
```

144.2. URI FORMAT

```
stub:someUri
```

Where **someUri** can be any URI with any query parameters.

144.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

144.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

144.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

144.4. COMPONENT OPTIONS

The Stub component supports 10 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
concurrentConsumers (consumer)	Sets the default number of concurrent threads processing exchanges.	1	int
defaultPollTimeout (consumer (advanced))	The timeout (in milliseconds) used when polling. When a timeout occurs, the consumer can check whether it is allowed to continue running. Setting a lower value allows the consumer to react more quickly upon shutdown.	1000	int

Name	Description	Default	Type
defaultBlockWhenFull (producer)	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will instead block and wait until the message can be accepted.	false	boolean
defaultDiscardWhenFull (producer)	Whether a thread that sends messages to a full SEDA queue will be discarded. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will give up sending and continue, meaning that the message was not sent to the SEDA queue.	false	boolean
defaultOfferTimeout (producer)	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, where a configured timeout can be added to the block case. Utilizing the <code>.offer(timeout)</code> method of the underlying java queue.		long
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
defaultQueueFactory (advanced)	Sets the default queue factory.		BlockingQueueFactory
queueSize (advanced)	Sets the default maximum capacity of the SEDA queue (i.e., the number of messages it can hold).	1000	int

144.5. ENDPOINT OPTIONS

The Stub endpoint is configured using URI syntax:

```
stub:name
```

with the following path and query parameters:

144.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
name (common)	Required Name of queue.		String

144.5.2. Query Parameters (18 parameters)

Name	Description	Default	Type
size (common)	The maximum capacity of the SEDA queue (i.e., the number of messages it can hold). Will by default use the <code>defaultSize</code> set on the SEDA component.	1000	int
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
concurrentConsumers (consumer)	Number of concurrent threads processing exchanges.	1	int
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • InOnly • InOut • InOptionalOut		ExchangePattern
limitConcurrentConsumers (consumer (advanced))	Whether to limit the number of concurrentConsumers to the maximum of 500. By default, an exception will be thrown if an endpoint is configured with a greater number. You can disable that check by turning this option off.	true	boolean
multipleConsumers (consumer (advanced))	Specifies whether multiple consumers are allowed. If enabled, you can use SEDA for Publish-Subscribe messaging. That is, you can send a message to the SEDA queue and have each consumer receive a copy of the message. When enabled, this option should be specified on every consumer endpoint.	false	boolean
pollTimeout (consumer (advanced))	The timeout (in milliseconds) used when polling. When a timeout occurs, the consumer can check whether it is allowed to continue running. Setting a lower value allows the consumer to react more quickly upon shutdown.	1000	int
purgeWhenStopping (consumer (advanced))	Whether to purge the task queue when stopping the consumer/route. This allows to stop faster, as any pending messages on the queue is discarded.	false	boolean
blockWhenFull (producer)	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will instead block and wait until the message can be accepted.	false	boolean
discardIfNoConsumers (producer)	Whether the producer should discard the message (do not add the message to the queue), when sending to a queue with no active consumers. Only one of the options discardIfNoConsumers and failIfNoConsumers can be enabled at the same time.	false	boolean

Name	Description	Default	Type
discardWhenFull (producer)	Whether a thread that sends messages to a full SEDA queue will be discarded. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will give up sending and continue, meaning that the message was not sent to the SEDA queue.	false	boolean
failIfNoConsumers (producer)	Whether the producer should fail by throwing an exception, when sending to a queue with no active consumers. Only one of the options <code>discardIfNoConsumers</code> and <code>failIfNoConsumers</code> can be enabled at the same time.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
offerTimeout (producer)	Offer timeout (in milliseconds) can be added to the block case when queue is full. You can disable timeout by using 0 or a negative value.		long
timeout (producer)	Timeout (in milliseconds) before a SEDA producer will stop waiting for an asynchronous task to complete. You can disable timeout by using 0 or a negative value.	30000	long
waitForTaskToComplete (producer)	Option to specify whether the caller should wait for the async task to complete or not before continuing. The following three options are supported: Always, Never or <code>IfReplyExpected</code>. The first two values are self-explanatory. The last value, <code>IfReplyExpected</code>, will only wait if the message is Request Reply based. The default option is <code>IfReplyExpected</code>. Enum values: • Never • <code>IfReplyExpected</code> • Always	<code>IfReplyExpected</code>	<code>WaitForTaskToComplete</code>

Name	Description	Default	Type
queue (advanced)	Define the queue instance which will be used by the endpoint.		BlockingQueue

144.6. EXAMPLES

Here are a few samples of stubbing endpoint uris

```
stub:smtp://somehost.foo.com?user=whatnot&something=else
stub:http://somehost.bar.com/something
```

144.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 11 options, which are listed below.

Name	Description	Default	Type
camel.component. .stub.autowired- enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component. .stub.bridge- error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component. .stub.concurrent- consumers	Sets the default number of concurrent threads processing exchanges.	1	Integer
camel.component. .stub.default- block-when-full	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will instead block and wait until the message can be accepted.	false	Boolean

Name	Description	Default	Type
camel.component.stub.default-discard-when-full	Whether a thread that sends messages to a full SEDA queue will be discarded. By default, an exception will be thrown stating that the queue is full. By enabling this option, the calling thread will give up sending and continue, meaning that the message was not sent to the SEDA queue.	false	Boolean
camel.component.stub.default-offer-timeout	Whether a thread that sends messages to a full SEDA queue will block until the queue's capacity is no longer exhausted. By default, an exception will be thrown stating that the queue is full. By enabling this option, where a configured timeout can be added to the block case. Utilizing the <code>.offer(timeout)</code> method of the underlining java queue.		Long
camel.component.stub.default-poll-timeout	The timeout (in milliseconds) used when polling. When a timeout occurs, the consumer can check whether it is allowed to continue running. Setting a lower value allows the consumer to react more quickly upon shutdown.	1000	Integer
camel.component.stub.default-queue-factory	Sets the default queue factory. The option is a <code>org.apache.camel.component.seda.BlockingQueueFactory<org.apache.camel.Exchange></code> type.		BlockingQueueFactory
camel.component.stub.enabled	Whether to enable auto configuration of the stub component. This is enabled by default.		Boolean
camel.component.stub.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.stub.queue-size	Sets the default maximum capacity of the SEDA queue (i.e., the number of messages it can hold).	1000	Integer

CHAPTER 145. TELEGRAM

Both producer and consumer are supported

The Telegram component provides access to the [Telegram Bot API](#). It allows a Camel-based application to send and receive messages by acting as a Bot, participating in direct conversations with normal users, private and public groups or channels.

A Telegram Bot must be created before using this component, following the instructions at the [Telegram Bot developers](#) home. When a new Bot is created, the [BotFather](#) provides an **authorization token** corresponding to the Bot. The authorization token is a mandatory parameter for the camel-telegram endpoint.



NOTE

In order to allow the Bot to receive all messages exchanged within a group or channel (not just the ones starting with a '/' character), ask the BotFather to **disable the privacy mode**, using the `/setprivacy` command.

145.1. DEPENDENCIES

When using **telegram** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-telegram-starter</artifactId>
</dependency>
```

145.2. URI FORMAT

```
telegram:type[?options]
```

145.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

145.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

145.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

145.4. COMPONENT OPTIONS

The Telegram component supports 7 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
baseUri (advanced)	Can be used to set an alternative base URI, e.g. when you want to test the component against a mock Telegram API.		String
client (advanced)	To use a custom AsyncHttpClient.		AsyncHttpClient
clientConfig (advanced)	To configure the AsyncHttpClient to use a custom com.ning.http.client.AsyncHttpClientConfig instance.		AsyncHttpClientConfig
authorizationToken (security)	The default Telegram authorization token to be used when the information is not provided in the endpoints.		String

145.5. ENDPOINT OPTIONS

The Telegram endpoint is configured using URI syntax:

```
telegram:type
```

with the following path and query parameters:

145.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
type (common)	Required The endpoint type. Currently, only the 'bots' type is supported. Enum values: • bots		String

145.5.2. Query Parameters (30 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
limit (consumer)	Limit on the number of updates that can be received in a single polling request.	100	Integer
sendEmptyMessageWhenIdle (consumer)	If the polling consumer did not poll any files, you can enable this option to send an empty message (no body) instead.	false	boolean
timeout (consumer)	Timeout in seconds for long polling. Put 0 for short polling or a bigger number for long polling. Long polling produces shorter response time.	30	Integer
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler

Name	Description	Default	Type
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern
pollStrategy (consumer (advanced))	A pluggable <code>org.apache.camel.PollingConsumerPollingStrategy</code> allowing you to provide your custom implementation to control error handling usually occurred during the poll operation before an Exchange have been created and being routed in Camel.		PollingConsumerPollStrategy
chatId (producer)	The identifier of the chat that will receive the produced messages. Chat ids can be first obtained from incoming messages (eg. when a telegram user starts a conversation with a bot, its client sends automatically a '/start' message containing the chat id). It is an optional parameter, as the chat id can be set dynamically for each outgoing message (using body or headers).		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
baseUri (advanced)	Can be used to set an alternative base URI, e.g. when you want to test the component against a mock Telegram API.		String
bufferSize (advanced)	The initial in-memory buffer size used when transferring data between Camel and AHC Client.	4096	int
clientConfig (advanced)	To configure the AsyncHttpClient to use a custom <code>com.ning.http.client.AsyncHttpClientConfig</code> instance.		AsyncHttpClientConfig

Name	Description	Default	Type
proxyHost (proxy)	HTTP proxy host which could be used when sending out the message.		String
proxyPort (proxy)	HTTP proxy port which could be used when sending out the message.		Integer
proxyType (proxy)	HTTP proxy type which could be used when sending out the message. Enum values: • HTTP • SOCKS4 • SOCKS5	HTTP	TelegramProxyType
backoffErrorThreshold (scheduler)	The number of subsequent error polls (failed due some error) that should happen before the backoffMultiplier should kick-in.		int
backoffIdleThreshold (scheduler)	The number of subsequent idle polls that should happen before the backoffMultiplier should kick-in.		int
backoffMultiplier (scheduler)	To let the scheduled polling consumer backoff if there has been a number of subsequent idles/errors in a row. The multiplier is then the number of polls that will be skipped before the next actual attempt is happening again. When this option is in use then backoffIdleThreshold and/or backoffErrorThreshold must also be configured.		int
delay (scheduler)	Milliseconds before the next poll.	500	long
greedy (scheduler)	If greedy is enabled, then the ScheduledPollConsumer will run immediately again, if the previous run polled 1 or more messages.	false	boolean
initialDelay (scheduler)	Milliseconds before the first poll starts.	1000	long
repeatCount (scheduler)	Specifies a maximum limit of number of fires. So if you set it to 1, the scheduler will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.	0	long

Name	Description	Default	Type
runLoggingLevel (scheduler)	The consumer logs a start/complete log line when it polls. This option allows you to configure the logging level for that. Enum values: • TRACE • DEBUG • INFO • WARN • ERROR • OFF	TRACE	LoggingLevel
scheduledExecutorService (scheduler)	Allows for configuring a custom/shared thread pool to use for the consumer. By default each consumer has its own single threaded thread pool.		ScheduledExecutorService
scheduler (scheduler)	To use a cron scheduler from either camel-spring or camel-quartz component. Use value spring or quartz for built in scheduler.	none	Object
schedulerProperties (scheduler)	To configure additional properties when using a custom scheduler or any of the Quartz, Spring based scheduler.		Map
startScheduler (scheduler)	Whether the scheduler should be auto started.	true	boolean
timeUnit (scheduler)	Time unit for initialDelay and delay options. Enum values: • NANOSECONDS • MICROSECONDS • MILLISECONDS • SECONDS • MINUTES • HOURS • DAYS	MILLISECONDS	TimeUnit

Name	Description	Default	Type
useFixedDelay (scheduler)	Controls if fixed delay or fixed rate is used. See <code>ScheduledExecutorService</code> in JDK for details.	true	boolean
authorizationToken (security)	Required The authorization token for using the bot (ask the BotFather).		String

145.5.3. Message Headers

Name	Description
CamelTelegramChatId	This header is used by the producer endpoint in order to resolve the chat id that will receive the message. The recipient chat id can be placed (in order of priority) in message body, in the CamelTelegramChatId header or in the endpoint configuration (chatId option). This header is also present in all incoming messages.
CamelTelegramMediaType	This header is used to identify the media type when the outgoing message is composed of pure binary data. Possible values are strings or enum values belonging to the org.apache.camel.component.telegram.TelegramMediaType enumeration.
CamelTelegramMediaTitleCaption	This header is used to provide a caption or title for outgoing binary messages.
CamelTelegramParseMode	This header is used to format text messages using HTML or Markdown (see org.apache.camel.component.telegram.TelegramParseMode).

145.6. USAGE

The Telegram component supports both consumer and producer endpoints. It can also be used in **reactive chat-bot mode** (to consume, then produce messages).

145.7. PRODUCER EXAMPLE

The following is a basic example of how to send a message to a Telegram chat through the Telegram Bot API.

in Java DSL

```
from("direct:start").to("telegram:bots?
authorizationToken=123456789:insertYourAuthorizationTokenHere");
```

or in Spring XML

```
<route>
  <from uri="direct:start"/>
  <to uri="telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere"/>
</route>
```

 `<route>`

The code **123456789:insertYourAuthorizationTokenHere** is the **authorization token** corresponding to the Bot.

When using the producer endpoint without specifying the **chat id** option, the target chat will be identified using information contained in the body or headers of the message. The following message bodies are allowed for a producer endpoint (messages of type **OutgoingXXXMessage** belong to the package **org.apache.camel.component.telegram.model**)

Java Type	Description
OutgoingTextMessage	To send a text message to a chat
OutgoingPhotoMessage	To send a photo (JPG, PNG) to a chat
OutgoingAudioMessage	To send a mp3 audio to a chat
OutgoingVideoMessage	To send a mp4 video to a chat
OutgoingDocumentMessage	To send a file to a chat (any media type)
OutgoingStickerMessage	To send a sticker to a chat (WEBP)
OutgoingAnswerInlineQuery	To send answers to an inline query
EditMessageTextMessage	To edit text and game messages (editMessageText)
EditMessageCaptionMessage	To edit captions of messages (editMessageCaption)
EditMessageMediaMessage	To edit animation, audio, document, photo, or video messages. (editMessageMedia)
EditMessageReplyMarkupMessage	To edit only the reply markup of message. (editMessageReplyMarkup)
EditMessageDelete	To delete a message, including service messages. (deleteMessage)
SendLocationMessage	To send a location (setSendLocation)
EditMessageLiveLocationMessage	To send changes to a live location (editMessageLiveLocation)
StopMessageLiveLocationMessage	To stop updating a live location message sent by the bot or via the bot (for inline bots) before live_period expires (stopMessageLiveLocation)
SendVenueMessage	To send information about a venue (sendVenue)

Java Type	Description
byte[]	To send any media type supported. It requires the CamelTelegramMediaType header to be set to the appropriate media type
String	To send a text message to a chat. It gets converted automatically into a OutgoingTextMessage

145.8. CONSUMER EXAMPLE

The following is a basic example of how to receive all messages that telegram users are sending to the configured Bot. In Java DSL

```
from("telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere")
.bean(ProcessorBean.class)
```

or in Spring XML

```
<route>
  <from uri="telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere"/>
  <bean ref="myBean" />
</route>

<bean id="myBean" class="com.example.MyBean"/>
```

The **MyBean** is a simple bean that will receive the messages

```
public class MyBean {

    public void process(String message) {
        // or Exchange, or org.apache.camel.component.telegram.model.IncomingMessage (or both)

        // do process
    }

}
```

Supported types for incoming messages are

Java Type	Description
IncomingMessage	The full object representation of an incoming message
String	The content of the message, for text messages only

145.9. REACTIVE CHAT-BOT EXAMPLE

The reactive chat-bot mode is a simple way of using the Camel component to build a simple chat bot that replies directly to chat messages received from the Telegram users.

The following is a basic configuration of the chat-bot in Java DSL

```
from("telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere")
  .bean(ChatBotLogic.class)
  .to("telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere");
```

or in Spring XML

```
<route>
  <from uri="telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere"/>
  <bean ref="chatBotLogic" />
  <to uri="telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere"/>
</route>

<bean id="chatBotLogic" class="com.example.ChatBotLogic"/>
```

The **ChatBotLogic** is a simple bean that implements a generic String-to-String method.

```
public class ChatBotLogic {

    public String chatBotProcess(String message) {
        if( "do-not-reply".equals(message) ) {
            return null; // no response in the chat
        }

        return "echo from the bot: " + message; // echoes the message
    }

}
```

Every non-null string returned by the **chatBotProcess** method is automatically routed to the chat that originated the request (as the **CamelTelegramChatId** header is used to route the message).

145.10. GETTING THE CHAT ID

If you want to push messages to a specific Telegram chat when an event occurs, you need to retrieve the corresponding chat ID. The chat ID is not currently shown in the telegram client, but you can obtain it using a simple route.

First, add the bot to the chat where you want to push messages, then run a route like the following one.

```
from("telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere")
  .to("log:INFO?showHeaders=true");
```

Any message received by the bot will be dumped to your log together with information about the chat (**CamelTelegramChatId** header).

Once you get the chat ID, you can use the following sample route to push message to it.

```
from("timer:tick")
  .setBody().constant("Hello")
```



```
to("telegram:bots?
authorizationToken=123456789:insertYourAuthorizationTokenHere&chatId=123456")
```

Note that the corresponding URI parameter is simply **chatId**.

145.11. CUSTOMIZING KEYBOARD

You can customize the user keyboard instead of asking him to write an option. **OutgoingTextMessage** has the property **ReplyMarkup** which can be used for such thing.

```
from("telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere")
    .process(exchange -> {

        OutgoingTextMessage msg = new OutgoingTextMessage();
        msg.setText("Choose one option!");

        InlineKeyboardButton buttonOptionOneI = InlineKeyboardButton.builder()
            .text("Option One - I").build();

        InlineKeyboardButton buttonOptionOneII = InlineKeyboardButton.builder()
            .text("Option One - II").build();

        InlineKeyboardButton buttonOptionTwoI = InlineKeyboardButton.builder()
            .text("Option Two - I").build();

        ReplyKeyboardMarkup replyMarkup = ReplyKeyboardMarkup.builder()
            .keyboard()
            .addRow(Arrays.asList(buttonOptionOneI, buttonOptionOneII))
            .addRow(Arrays.asList(buttonOptionTwoI))
            .close()
            .oneTimeKeyboard(true)
            .build();

        msg.setReplyMarkup(replyMarkup);

        exchange.getIn().setBody(msg);
    })
    .to("telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere");
```

If you want to disable it the next message must have the property **removeKeyboard** set on **ReplyKeyboardMarkup** object.

```
from("telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere")
    .process(exchange -> {

        OutgoingTextMessage msg = new OutgoingTextMessage();
        msg.setText("Your answer was accepted!");

        ReplyKeyboardMarkup replyMarkup = ReplyKeyboardMarkup.builder()
            .removeKeyboard(true)
            .build();

        msg.setReplyKeyboardMarkup(replyMarkup);

        exchange.getIn().setBody(msg);
```

```

    })
    .to("telegram:bots?authorizationToken=123456789:insertYourAuthorizationTokenHere");

```

145.12. WEBHOOK MODE

The Telegram component supports usage in the **webhook mode** using the **camel-webhook** component.

In order to enable webhook mode, users need first to add a REST implementation to their application. Maven users, for example, can add **netty-http** to their **pom.xml** file:

```

<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-netty-http</artifactId>
  <version>{CamelSBVersion}</version>
  <!-- use the same version as your Camel core version -->
</dependency>

```

Once done, you need to prepend the webhook URI to the telegram URI you want to use.

In Java DSL:

```

from("webhook:telegram:bots?
authorizationToken=123456789:insertYourAuthorizationTokenHere").to("log:info");

```

Some endpoints will be exposed by your application and Telegram will be configured to send messages to them. You need to ensure that your server is exposed to the internet and to pass the right value of the **camel.component.webhook.configuration.webhook-external-url** property.

Refer to the **camel-webhook** component documentation for instructions on how to set it.

145.13. SPRING BOOT AUTO-CONFIGURATION

The component supports 8 options, which are listed below.

Name	Description	Default	Type
camel.component.telegram.authorization-token	The default Telegram authorization token to be used when the information is not provided in the endpoints.		String
camel.component.telegram.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.telegram.base-uri	Can be used to set an alternative base URI, e.g. when you want to test the component against a mock Telegram API.		String
camel.component.telegram.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.telegram.client	To use a custom <code>AsyncHttpClient</code> . The option is a <code>org.apache.camel.component.telegram.AsyncHttpClient</code> type.		<code>AsyncHttpClient</code>
camel.component.telegram.client-config	To configure the <code>AsyncHttpClient</code> to use a custom <code>com.ning.http.client.AsyncHttpClientConfig</code> instance. The option is a <code>org.apache.camel.component.telegram.AsyncHttpClientConfig</code> type.		<code>AsyncHttpClientConfig</code>
camel.component.telegram.enabled	Whether to enable auto configuration of the telegram component. This is enabled by default.		Boolean
camel.component.telegram.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

CHAPTER 146. TIMER

Only consumer is supported

The Timer component is used to generate message exchanges when a timer fires. You can only consume events from this endpoint.

146.1. DEPENDENCIES

When using **timer** with Red Hat build of Camel Spring Boot, make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-timer-starter</artifactId>
</dependency>
```

146.2. URI FORMAT

```
timer:name[?options]
```

Where **name** is the name of the **Timer** object, which is created and shared across endpoints. So if you use the same name for all your timer endpoints, only one **Timer** object and thread will be used.



NOTE

The IN body of the generated exchange is **null**. So `exchange.getIn().getBody()` returns **null**.



NOTE

Advanced Scheduler

See also the [Quartz](#) component that supports much more advanced scheduling.

146.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

146.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

146.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

146.4. COMPONENT OPTIONS

The Timer component supports 2 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

146.5. ENDPOINT OPTIONS

The Timer endpoint is configured using URI syntax:

```
timer:timerName
```

with the following path and query parameters:

146.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
timerName (consumer)	Required The name of the timer.		String

146.5.2. Query Parameters (13 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
delay (consumer)	Delay before first event is triggered.	1000	long
fixedRate (consumer)	Events take place at approximately regular intervals, separated by the specified period.	false	boolean
includeMetadata (consumer)	Whether to include metadata in the exchange such as fired time, timer name, timer count etc. This information is default included.	true	boolean
period (consumer)	If greater than 0, generate periodic events every period.	1000	long
repeatCount (consumer)	Specifies a maximum limit of number of fires. So if you set it to 1, the timer will only fire once. If you set it to 5, it will only fire five times. A value of zero or negative means fire forever.		long

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code> • <code>InOptionalOut</code>		<code>ExchangePattern</code>
daemon (advanced)	Specifies whether or not the thread associated with the timer endpoint runs as a daemon. The default value is <code>true</code> .	<code>true</code>	<code>boolean</code>
pattern (advanced)	Allows you to specify a custom Date pattern to use for setting the time option using URI syntax.		<code>String</code>
synchronous (advanced)	Sets whether synchronous processing should be strictly used.	<code>false</code>	<code>boolean</code>
time (advanced)	A <code>java.util.Date</code> the first event should be generated. If using the URI, the pattern expected is: <code>yyyy-MM-dd HH:mm:ss</code> or <code>yyyy-MM-dd'T'HH:mm:ss</code> .		<code>Date</code>
timer (advanced)	To use a custom <code>Timer</code> .		<code>Timer</code>

146.6. EXCHANGE PROPERTIES

When the timer is fired, it adds the following information as properties to the **Exchange**:

Name	Type	Description
Exchange.TIMER_NAME	String	The value of the name option.
Exchange.TIMER_TIME	Date	The value of the time option.

Name	Type	Description
Exchange.TIMER_PERIOD	long	The value of the period option.
Exchange.TIMER_FIRED_TIME	Date	The time when the consumer fired.
Exchange.TIMER_COUNTER	Long	The current fire counter. Starts from 1.

146.7. SAMPLE

To set up a route that generates an event every 60 seconds:

```
from("timer://foo?fixedRate=true&period=60000").to("bean:myBean?method=someMethodName");
```

The above route will generate an event and then invoke the **someMethodName** method on the bean called **myBean** in the Registry.

And the route in Spring DSL:

```
<route>
  <from uri="timer://foo?fixedRate=true&period=60000"/>
  <to uri="bean:myBean?method=someMethodName"/>
</route>
```

146.8. FIRING AS SOON AS POSSIBLE

Since Camel 2.17

You may want to fire messages in a Camel route as soon as possible you can use a negative delay:

```
<route>
  <from uri="timer://foo?delay=-1"/>
  <to uri="bean:myBean?method=someMethodName"/>
</route>
```

In this way the timer will fire messages immediately.

You can also specify a `repeatCount` parameter in conjunction with a negative delay to stop firing messages after a fixed number has been reached.

If you don't specify a `repeatCount` then the timer will continue firing messages until the route will be stopped.

146.9. FIRING ONLY ONCE

You may want to fire a message in a Camel route only once, such as when starting the route. To do that you use the `repeatCount` option as shown:


```

<route>
  <from uri="timer://foo?repeatCount=1"/>
  <to uri="bean:myBean?method=someMethodName"/>
</route>

```

146.10. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.component.timer.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.timer.bridge-error-handler	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
camel.component.timer.enabled	Whether to enable auto configuration of the timer component. This is enabled by default.		Boolean

CHAPTER 147. TOKENIZE

The tokenizer language is a built-in language in **camel-core**, which is most often used with the [Split](#) EIP to split a message using a token-based strategy.

The tokenizer language is intended to tokenize text documents using a specified delimiter pattern. It can also be used to tokenize XML documents with some limited capability. For a truly XML-aware tokenization, the use of the [XML Tokenize](#) language is recommended as it offers a faster, more efficient tokenization specifically for XML documents.

147.1. DEPENDENCIES

When using **tokenize** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-core-starter</artifactId>
</dependency>
```

147.2. TOKENIZE OPTIONS

The Tokenize language supports 11 options, which are listed below.

Name	Default	Java Type	Description
token		String	Required The (start) token to use as tokenizer, for example you can use the new line token. You can use simple language as the token to support dynamic tokens.
endToken		String	The end token to use as tokenizer if using start/end token pairs. You can use simple language as the token to support dynamic tokens.
inheritNamespace TagName		String	To inherit namespaces from a root/parent tag name when using XML You can use simple language as the tag name to support dynamic names.
headerName		String	Name of header to tokenize instead of using the message body.
regex		Boolean	If the token is a regular expression pattern. The default value is false.
xml		Boolean	Whether the input is XML messages. This option must be set to true if working with XML payloads.
includeTokens		Boolean	Whether to include the tokens in the parts when using pairs The default value is false.

Name	Default	Java Type	Description
group		String	To group N parts together, for example to split big files into chunks of 1000 lines. You can use simple language as the group to support dynamic group sizes.
groupDelimiter		String	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.
skipFirst		Boolean	To skip the very first element.
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

147.3. EXAMPLE

The following example shows how to take a request from the `direct:a` endpoint then split it into pieces using an [Expression](#), then forward each piece to `direct:b`:

```
<route>
  <from uri="direct:a"/>
  <split>
    <tokenize token="\n"/>
    <to uri="direct:b"/>
  </split>
</route>
```

And in Java DSL:

```
from("direct:a")
  .split(body().tokenize("\n"))
  .to("direct:b");
```

147.4. SEE ALSO

For more examples see [Split](#) EIP.

147.5. SPRING BOOT AUTO-CONFIGURATION

The component supports 147 options, which are listed below.

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.acl-token</code>	Sets the ACL token to be used with Consul.		String
<code>camel.cloud.consul.service-discovery.block-seconds</code>	The seconds to wait for a watch event, default 10 seconds.	10	Integer
<code>camel.cloud.consul.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.consul.service-discovery.connect-timeout-millis</code>	Connect timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.datacenter</code>	The data center.		String
<code>camel.cloud.consul.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.consul.service-discovery.password</code>	Sets the password to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.consul.service-discovery.read-timeout-millis</code>	Read timeout for OkHttpClient.		Long
<code>camel.cloud.consul.service-discovery.url</code>	The Consul agent URL.		String

Name	Description	Default	Type
<code>camel.cloud.consul.service-discovery.username</code>	Sets the username to be used for basic authentication.		String
<code>camel.cloud.consul.service-discovery.write-timeout-millis</code>	Write timeout for OkHttpClient.		Long
<code>camel.cloud.dns.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.dns.service-discovery.domain</code>	The domain name;.		String
<code>camel.cloud.dns.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.dns.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.dns.service-discovery.proto</code>	The transport protocol of the desired service.	<code>_tcp</code>	String
<code>camel.cloud.etcd.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.etcd.service-discovery.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.etcd.service-discovery.password</code>	The password to use for basic authentication.		String

Name	Description	Default	Type
<code>camel.cloud.etcd.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.etcd.service-discovery.service-path</code>	The path to look for for service discovery.	<code>/services/</code>	String
<code>camel.cloud.etcd.service-discovery.timeout</code>	To set the maximum time an action could take to complete.		Long
<code>camel.cloud.etcd.service-discovery.type</code>	To set the discovery type, valid values are on-demand and watch.	on-demand	String
<code>camel.cloud.etcd.service-discovery.uris</code>	The URIs the client can connect to.		String
<code>camel.cloud.etcd.service-discovery.user-name</code>	The user name to use for basic authentication.		String
<code>camel.cloud.kubernetes.service-discovery.api-version</code>	Sets the API version when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-data</code>	Sets the Certificate Authority data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.ca-cert-file</code>	Sets the Certificate Authority data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-cert-data</code>	Sets the Client Certificate data when using client lookup.		String

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.client-cert-file</code>	Sets the Client Certificate data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-algo</code>	Sets the Client Keystore algorithm, such as RSA when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-data</code>	Sets the Client Keystore data when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-file</code>	Sets the Client Keystore data that are loaded from the file when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.client-key-passphrase</code>	Sets the Client Keystore passphrase when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.kubernetes.service-discovery.dns-domain</code>	Sets the DNS domain to use for DNS lookup.		String
<code>camel.cloud.kubernetes.service-discovery.enabled</code>	Enable the component.	true	Boolean

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.lookup</code>	How to perform service lookup. Possible values: client, dns, environment. When using client, then the client queries the kubernetes master to obtain a list of active pods that provides the service, and then random (or round robin) select a pod. When using dns the service name is resolved as name.namespace.svc.dnsDomain. When using dnssrv the service name is resolved with SRV query forsvc... When using environment then environment variables are used to lookup the service. By default environment is used.	environment	String
<code>camel.cloud.kubernetes.service-discovery.master-url</code>	Sets the URL to the master when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.namespace</code>	Sets the namespace to use. Will by default use namespace from the ENV variable KUBERNETES_MASTER.		String
<code>camel.cloud.kubernetes.service-discovery.oauth-token</code>	Sets the OAUTH token for authentication (instead of username/password) when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.password</code>	Sets the password for authentication when using client lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-name</code>	Sets the Port Name to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.port-protocol</code>	Sets the Port Protocol to use for DNS/DNSSRV lookup.		String
<code>camel.cloud.kubernetes.service-discovery.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in com.netflix.client.config.CommonClientConfigKey.		Map

Name	Description	Default	Type
<code>camel.cloud.kubernetes.service-discovery.trust-certs</code>	Sets whether to turn on trust certificate check when using client lookup.	false	Boolean
<code>camel.cloud.kubernetes.service-discovery.username</code>	Sets the username for authentication when using client lookup.		String
<code>camel.cloud.ribbon.load-balancer.client-name</code>	Sets the Ribbon client name.		String
<code>camel.cloud.ribbon.load-balancer.configurations</code>	Define additional configuration definitions.		Map
<code>camel.cloud.ribbon.load-balancer.enabled</code>	Enable the component.	true	Boolean
<code>camel.cloud.ribbon.load-balancer.namespace</code>	The namespace.		String
<code>camel.cloud.ribbon.load-balancer.password</code>	The password.		String
<code>camel.cloud.ribbon.load-balancer.properties</code>	Set client properties to use. These properties are specific to what service call implementation are in use. For example if using ribbon, then the client properties are define in <code>com.netflix.client.config.CommonClientConfigKey</code> .		Map
<code>camel.cloud.ribbon.load-balancer.username</code>	The username.		String

Name	Description	Default	Type
camel.hystrix.allow-maximum-size-to-diverge-from-core-size	Allows the configuration for <code>maximumSize</code> to take effect. That value can then be equal to, or higher, than <code>coreSize</code> .	false	Boolean
camel.hystrix.circuit-breaker-enabled	Whether to use a <code>HystrixCircuitBreaker</code> or not. If false no circuit-breaker logic will be used and all requests permitted. This is similar in effect to <code>circuitBreakerForceClosed()</code> except that continues tracking metrics and knowing whether it should be open/closed, this property results in not even instantiating a circuit-breaker.	true	Boolean
camel.hystrix.circuit-breaker-error-threshold-percentage	Error percentage threshold (as whole number such as 50) at which point the circuit breaker will trip open and reject requests. It will stay tripped for the duration defined in <code>circuitBreakerSleepWindowInMilliseconds</code> ; The error percentage this is compared against comes from <code>HystrixCommandMetrics.getHealthCounts()</code> .	50	Integer
camel.hystrix.circuit-breaker-force-closed	If true the <code>HystrixCircuitBreaker#allowRequest()</code> will always return true to allow requests regardless of the error percentage from <code>HystrixCommandMetrics.getHealthCounts()</code> . The <code>circuitBreakerForceOpen()</code> property takes precedence so if it set to true this property does nothing.	false	Boolean
camel.hystrix.circuit-breaker-force-open	If true the <code>HystrixCircuitBreaker.allowRequest()</code> will always return false, causing the circuit to be open (tripped) and reject all requests. This property takes precedence over <code>circuitBreakerForceClosed()</code> ;	false	Boolean
camel.hystrix.circuit-breaker-request-volume-threshold	Minimum number of requests in the <code>metricsRollingStatisticalWindowInMilliseconds()</code> that must exist before the <code>HystrixCircuitBreaker</code> will trip. If below this number the circuit will not trip regardless of error percentage.	20	Integer
camel.hystrix.circuit-breaker-sleep-window-in-milliseconds	The time in milliseconds after a <code>HystrixCircuitBreaker</code> trips open that it should wait before trying requests again.	5000	Integer
camel.hystrix.configurations	Define additional configuration definitions.		Map

Name	Description	Default	Type
camel.hystrix.core-pool-size	Core thread-pool size that gets passed to <code>java.util.concurrent.ThreadPoolExecutor#setCorePoolSize(int)</code> .	10	Integer
camel.hystrix.enabled	Enable the component.	true	Boolean
camel.hystrix.execution-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.run()</code> . Requests beyond the concurrent limit will be rejected. Applicable only when <code>executionIsolationStrategy == SEMAPHORE</code> .	20	Integer
camel.hystrix.execution-isolation-strategy	What isolation strategy <code>HystrixCommand.run()</code> will be executed with. If <code>THREAD</code> then it will be executed on a separate thread and concurrent requests limited by the number of threads in the thread-pool. If <code>SEMAPHORE</code> then it will be executed on the calling thread and concurrent requests limited by the semaphore count.	THREAD	String
camel.hystrix.execution-isolation-thread-interrupt-on-timeout	Whether the execution thread should attempt an interrupt (using <code>Future#cancel()</code>) when a thread times out. Applicable only when <code>executionIsolationStrategy() == THREAD</code> .	true	Boolean
camel.hystrix.execution-timeout-enabled	Whether the timeout mechanism is enabled for this command.	true	Boolean
camel.hystrix.execution-timeout-in-milliseconds	Time in milliseconds at which point the command will timeout and halt execution. If <code>executionIsolationThreadInterruptOnTimeout == true</code> and the command is thread-isolated, the executing thread will be interrupted. If the command is semaphore-isolated and a <code>HystrixObservableCommand</code> , that command will get unsubscribed.	1000	Integer
camel.hystrix.fallback-ack-enabled	Whether <code>HystrixCommand.getFallback()</code> should be attempted when failure occurs.	true	Boolean
camel.hystrix.fallback-isolation-semaphore-max-concurrent-requests	Number of concurrent requests permitted to <code>HystrixCommand.getFallback()</code> . Requests beyond the concurrent limit will fail-fast and not attempt retrieving a fallback.	10	Integer

Name	Description	Default	Type
camel.hystrix.group-key	Sets the group key to use. The default value is CamelHystrix.	Camel Hystrix	String
camel.hystrix.keep-alive-time	Keep-alive time in minutes that gets passed to <code>ThreadPoolExecutor#setKeepAliveTime(long, TimeUnit)</code> .	1	Integer
camel.hystrix.max-queue-size	Max queue size that gets passed to <code>BlockingQueue</code> in <code>HystrixConcurrencyStrategy.getBlockingQueue(int)</code> . This should only affect the instantiation of a threadpool - it is not eligible to change a queue size on the fly. For that, use <code>queueSizeRejectionThreshold()</code> .	-1	Integer
camel.hystrix.maximum-size	Maximum thread-pool size that gets passed to <code>ThreadPoolExecutor#setMaximumPoolSize(int)</code> . This is the maximum amount of concurrency that can be supported without starting to reject <code>HystrixCommands</code> . Please note that this setting only takes effect if you also set <code>allowMaximumSizeToDivergeFromCoreSize</code> .	10	Integer
camel.hystrix.metrics-health-snapshot-interval-in-milliseconds	Time in milliseconds to wait between allowing health snapshots to be taken that calculate success and error percentages and affect <code>HystrixCircuitBreaker.isOpen()</code> status. On high-volume circuits the continual calculation of error percentage can become CPU intensive thus this controls how often it is calculated.	500	Integer
camel.hystrix.metrics-rolling-percentile-bucket-size	Maximum number of values stored in each bucket of the rolling percentile. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	10	Integer
camel.hystrix.metrics-rolling-percentile-enabled	Whether percentile metrics should be captured using <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	true	Boolean
camel.hystrix.metrics-rolling-percentile-window-buckets	Number of buckets the rolling percentile window is broken into. This is passed into <code>HystrixRollingPercentile</code> inside <code>HystrixCommandMetrics</code> .	6	Integer

Name	Description	Default	Type
camel.hystrix.metrics-rolling-percentile-window-in-milliseconds	Duration of percentile rolling window in milliseconds. This is passed into HystrixRollingPercentile inside HystrixCommandMetrics.	10000	Integer
camel.hystrix.metrics-rolling-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside HystrixCommandMetrics.	10	Integer
camel.hystrix.metrics-rolling-statistical-window-in-milliseconds	This property sets the duration of the statistical rolling window, in milliseconds. This is how long metrics are kept for the thread pool. The window is divided into buckets and rolls by those increments.	10000	Integer
camel.hystrix.queue-size-rejection-threshold	Queue size rejection threshold is an artificial max size at which rejections will occur even if maxQueueSize has not been reached. This is done because the maxQueueSize of a BlockingQueue can not be dynamically changed and we want to support dynamically changing the queue size that affects rejections. This is used by HystrixCommand when queuing a thread for execution.	5	Integer
camel.hystrix.request-log-enabled	Whether HystrixCommand execution and events should be logged to HystrixRequestLog.	true	Boolean
camel.hystrix.thread-pool-key	Sets the thread pool key to use. Will by default use the same value as groupKey has been configured to use.	Camel Hystrix	String
camel.hystrix.thread-pool-rolling-number-statistical-window-buckets	Number of buckets the rolling statistical window is broken into. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10	Integer
camel.hystrix.thread-pool-rolling-number-statistical-window-in-milliseconds	Duration of statistical rolling window in milliseconds. This is passed into HystrixRollingNumber inside each HystrixThreadPoolMetrics instance.	10000	Integer
camel.language.constant.enabled	Whether to enable auto configuration of the constant language. This is enabled by default.		Boolean

Name	Description	Default	Type
<code>camel.language.constant.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.csimple.enabled</code>	Whether to enable auto configuration of the csimple language. This is enabled by default.		Boolean
<code>camel.language.csimple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.exchangeproperty.enabled</code>	Whether to enable auto configuration of the exchangeProperty language. This is enabled by default.		Boolean
<code>camel.language.exchangeproperty.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.file.enabled</code>	Whether to enable auto configuration of the file language. This is enabled by default.		Boolean
<code>camel.language.file.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.header.enabled</code>	Whether to enable auto configuration of the header language. This is enabled by default.		Boolean
<code>camel.language.header.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.ref.enabled</code>	Whether to enable auto configuration of the ref language. This is enabled by default.		Boolean
<code>camel.language.ref.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.simple.enabled</code>	Whether to enable auto configuration of the simple language. This is enabled by default.		Boolean
<code>camel.language.simple.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
<code>camel.language.tokenize.enabled</code>	Whether to enable auto configuration of the tokenize language. This is enabled by default.		Boolean

Name	Description	Default	Type
camel.language.tokenize.group-delimiter	Sets the delimiter to use when grouping. If this has not been set then token will be used as the delimiter.		String
camel.language.tokenize.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean
camel.resilience4j.automatic-transition-from-open-to-half-open-enabled	Enables automatic transition from OPEN to HALF_OPEN state once the waitDurationInOpenState has passed.	false	Boolean
camel.resilience4j.circuit-breaker-ref	Refers to an existing io.github.resilience4j.circuitbreaker.CircuitBreaker instance to lookup and use from the registry. When using this, then any other circuit breaker options are not in use.		String
camel.resilience4j.config-ref	Refers to an existing io.github.resilience4j.circuitbreaker.CircuitBreakerConfig instance to lookup and use from the registry.		String
camel.resilience4j.configurations	Define additional configuration definitions.		Map
camel.resilience4j.enabled	Enable the component.	true	Boolean
camel.resilience4j.failure-rate-threshold	Configures the failure rate threshold in percentage. If the failure rate is equal or greater than the threshold the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 50 percentage.		Float
camel.resilience4j.minimum-number-of-calls	Configures the minimum number of calls which are required (per sliding window period) before the CircuitBreaker can calculate the error rate. For example, if minimumNumberOfCalls is 10, then at least 10 calls must be recorded, before the failure rate can be calculated. If only 9 calls have been recorded the CircuitBreaker will not transition to open even if all 9 calls have failed. Default minimumNumberOfCalls is 100.	100	Integer

Name	Description	Default	Type
camel.resilience4j .permitted-number-of-calls-in-half-open-state	Configures the number of permitted calls when the CircuitBreaker is half open. The size must be greater than 0. Default size is 10.	10	Integer
camel.resilience4j .sliding-window-size	Configures the size of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. slidingWindowSize configures the size of the sliding window. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. The slidingWindowSize must be greater than 0. The minimumNumberOfCalls must be greater than 0. If the slidingWindowType is COUNT_BASED, the minimumNumberOfCalls cannot be greater than slidingWindowSize . If the slidingWindowType is TIME_BASED, you can pick whatever you want. Default slidingWindowSize is 100.	100	Integer
camel.resilience4j .sliding-window-type	Configures the type of the sliding window which is used to record the outcome of calls when the CircuitBreaker is closed. Sliding window can either be count-based or time-based. If slidingWindowType is COUNT_BASED, the last slidingWindowSize calls are recorded and aggregated. If slidingWindowType is TIME_BASED, the calls of the last slidingWindowSize seconds are recorded and aggregated. Default slidingWindowType is COUNT_BASED.	COUNT_BASED	String
camel.resilience4j .slow-call-duration-threshold	Configures the duration threshold (seconds) above which calls are considered as slow and increase the slow calls percentage. Default value is 60 seconds.	60	Integer
camel.resilience4j .slow-call-rate-threshold	Configures a threshold in percentage. The CircuitBreaker considers a call as slow when the call duration is greater than slowCallDurationThreshold Duration. When the percentage of slow calls is equal or greater the threshold, the CircuitBreaker transitions to open and starts short-circuiting calls. The threshold must be greater than 0 and not greater than 100. Default value is 100 percentage which means that all recorded calls must be slower than slowCallDurationThreshold.		Float

Name	Description	Default	Type
camel.resilience4j.wait-duration-in-open-state	Configures the wait duration (in seconds) which specifies how long the CircuitBreaker should stay open, before it switches to half open. Default value is 60 seconds.	60	Integer
camel.resilience4j.writable-stack-trace-enabled	Enables writable stack traces. When set to false, <code>Exception.getStackTrace</code> returns a zero length array. This may be used to reduce log spam when the circuit breaker is open as the cause of the exceptions is already known (the circuit breaker is short-circuiting calls).	true	Boolean
camel.rest.api-component	The name of the Camel component to use as the REST API (such as swagger) If no API Component has been explicit configured, then Camel will lookup if there is a Camel component responsible for servicing and generating the REST API documentation, or if a <code>org.apache.camel.spi.RestApiProcessorFactory</code> is registered in the registry. If either one is found, then that is being used.		String
camel.rest.api-context-path	Sets a leading API context-path the REST API services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path.		String
camel.rest.api-context-route-id	Sets the route id to use for the route that services the REST API. The route will by default use an auto assigned route id.		String
camel.rest.api-host	To use an specific hostname for the API documentation (eg swagger) This can be used to override the generated host with this configured hostname.		String
camel.rest.api-property	Allows to configure as many additional properties for the api documentation (swagger). For example set property <code>api.title</code> to my cool stuff.		Map
camel.rest.api-vendor-extension	Whether vendor extension is enabled in the Rest APIs. If enabled then Camel will include additional information as vendor extension (eg keys starting with x-) such as route ids, class names etc. Not all 3rd party API gateways and tools supports vendor-extensions when importing your API docs.	false	Boolean
camel.rest.binding-mode	Sets the binding mode to use. The default value is off.		RestBindingMode

Name	Description	Default	Type
camel.rest.client-request-validation	Whether to enable validation of the client request to check whether the Content-Type and Accept headers from the client is supported by the Rest-DSL configuration of its consumes/produces settings. This can be turned on, to enable this check. In case of validation error, then HTTP Status codes 415 or 406 is returned. The default value is false.	false	Boolean
camel.rest.component	The Camel Rest component to use for the REST transport (consumer), such as netty-http, jetty, servlet, undertow. If no component has been explicit configured, then Camel will lookup if there is a Camel component that integrates with the Rest DSL, or if a org.apache.camel.spi.RestConsumerFactory is registered in the registry. If either one is found, then that is being used.		String
camel.rest.component-property	Allows to configure as many additional properties for the rest component in use.		Map
camel.rest.consumer-property	Allows to configure as many additional properties for the rest consumer in use.		Map
camel.rest.context-path	Sets a leading context-path the REST services will be using. This can be used when using components such as camel-servlet where the deployed web application is deployed using a context-path. Or for components such as camel-jetty or camel-netty-http that includes a HTTP server.		String
camel.rest.cors-headers	Allows to configure custom CORS headers.		Map
camel.rest.data-format-property	Allows to configure as many additional properties for the data formats in use. For example set property prettyPrint to true to have json outputted in pretty mode. The properties can be prefixed to denote the option is only for either JSON or XML and for either the IN or the OUT. The prefixes are: json.in. json.out. xml.in. xml.out. For example a key with value xml.out.mustBeJAXBELEMENT is only for the XML data format for the outgoing. A key without a prefix is a common key for all situations.		Map
camel.rest.enable-cors	Whether to enable CORS headers in the HTTP response. The default value is false.	false	Boolean

Name	Description	Default	Type
camel.rest.endpoint-property	Allows to configure as many additional properties for the rest endpoint in use.		Map
camel.rest.host	The hostname to use for exposing the REST service.		String
camel.rest.hostname-resolver	If no hostname has been explicit configured, then this resolver is used to compute the hostname the REST service will be using.		RestHostNameResolver
camel.rest.json-data-format	Name of specific json data format to use. By default json-jackson will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
camel.rest.port	The port number to use for exposing the REST service. Notice if you use servlet component then the port number configured here does not apply, as the port number in use is the actual port number the servlet component is using. eg if using Apache Tomcat its the tomcat http port, if using Apache Karaf its the HTTP service in Karaf that uses port 8181 by default etc. Though in those situations setting the port number here, allows tooling and JMX to know the port number, so its recommended to set the port number to the number that the servlet engine uses.		String
camel.rest.producer-api-doc	Sets the location of the api document (swagger api) the REST producer will use to validate the REST uri and query parameters are valid accordingly to the api document. This requires adding camel-swagger-java to the classpath, and any miss configuration will let Camel fail on startup and report the error(s). The location of the api document is loaded from classpath by default, but you can use file: or http: to refer to resources to load from file or http url.		String
camel.rest.producer-component	Sets the name of the Camel component to use as the REST producer.		String
camel.rest.scheme	The scheme to use for exposing the REST service. Usually http or https is supported. The default value is http.		String
camel.rest.skip-binding-on-error-code	Whether to skip binding on output if there is a custom HTTP error code header. This allows to build custom error messages that do not bind to json / xml etc, as success messages otherwise will do.	false	Boolean

Name	Description	Default	Type
<code>camel.rest.use-x-forward-headers</code>	Whether to use X-Forward headers for Host and related setting. The default value is true.	true	Boolean
<code>camel.rest.xml-data-format</code>	Name of specific XML data format to use. By default jaxb will be used. Important: This option is only for setting a custom name of the data format, not to refer to an existing data format instance.		String
<code>camel.rest.api-context-id-pattern</code>	Deprecated Sets an CamelContext id pattern to only allow Rest APIs from rest services within CamelContext's which name matches the pattern. The pattern name refers to the CamelContext name, to match on the current CamelContext only. For any other value, the pattern uses the rules from PatternHelper#matchPattern(String,String).		String
<code>camel.rest.api-context-listing</code>	Deprecated Sets whether listing of all available CamelContext's with REST services in the JVM is enabled. If enabled it allows to discover these contexts, if false then only the current CamelContext is in use.	false	Boolean

CHAPTER 148. VALIDATOR

Only producer is supported

The Validation component performs XML validation of the message body using the JAXP Validation API and based on any of the supported XML schema languages, which defaults to [XML Schema](#)

Note that the component also supports the following useful schema languages:

- [RelaxNG Compact Syntax](#)
- [RelaxNG XML Syntax](#)

The [MSV](#) component also supports [RelaxNG XML Syntax](#).

148.1. DEPENDENCIES

When using **validator** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-validator-starter</artifactId>
</dependency>
```

148.2. URI FORMAT

```
validator:someLocalOrRemoteResource
```

Where **someLocalOrRemoteResource** is some URL to a local resource on the classpath or a full URL to a remote resource or resource on the file system which contains the XSD to validate against. For example:

- **msv:org/foo/bar.xsd**
- **msv:file:../foo/bar.xsd**
- **msv:http://acme.com/cheese.xsd**
- **validator:com/mypackage/myschema.xsd**

The Validation component is provided directly in the camel-core.

148.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

148.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

148.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

148.4. COMPONENT OPTIONS

The Validator component supports 3 options, which are listed below.

Name	Description	Default	Type
		t	

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
resourceResolverFactory (advanced)	To use a custom LSResourceResolver which depends on a dynamic endpoint resource URL.		ValidatorResourceResolverFactory

148.5. ENDPOINT OPTIONS

The Validator endpoint is configured using URI syntax:

```
validator:resourceUri
```

with the following path and query parameters:

148.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
resourceUri (producer)	Required URL to a local resource on the classpath, or a reference to lookup a bean in the Registry, or a full URL to a remote resource or resource on the file system which contains the XSD to validate against.		String

148.5.2. Query Parameters (10 parameters)

Name	Description	Default	Type
failOnNullBody (producer)	Whether to fail if no body exists.	true	boolean
failOnNullHeader (producer)	Whether to fail if no header exists when validating against a header.	true	boolean
headerName (producer)	To validate against a header instead of the message body.		String
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
errorHandler (advanced)	To use a custom org.apache.camel.processor.validation.ValidatorErrorHandler. The default error handler captures the errors and throws an exception.		ValidatorErrorHandler
resourceResolver (advanced)	To use a custom LSResourceResolver. Do not use together with resourceResolverFactory.		LSResourceResolver
resourceResolverFactory (advanced)	To use a custom LSResourceResolver which depends on a dynamic endpoint resource URI. The default resource resolver factory returns a resource resolver which can read files from the class path and file system. Do not use together with resourceResolver.		ValidatorResourceResolverFactory
schemaFactory (advanced)	To use a custom javax.xml.validation.SchemaFactory.		SchemaFactory
schemaLanguage (advanced)	Configures the W3C XML Schema Namespace URI.	http://www.w3.org/2001/XMLSchema	String
useSharedSchema (advanced)	Whether the Schema instance should be shared or not. This option is introduced to work around a JDK 1.6.x bug. Xerces should not have this issue.	true	boolean

148.6. EXAMPLE

The following [example](#) shows how to configure a route from endpoint **direct:start** which then goes to one of two endpoints, either **mock:valid** or **mock:invalid** based on whether or not the XML matches the given schema (which is supplied on the classpath).

148.7. ADVANCED: JMX METHOD CLEARCACHEDSCHEMA

You can force that the cached schema in the validator endpoint is cleared and reread with the next process call with the JMX operation **clearCachedSchema**. You can also use this method to programmatically clear the cache. This method is available on the **ValidatorEndpoint** class.

148.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 4 options, which are listed below.

Name	Description	Default	Type
camel.component.validator.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.validator.enabled	Whether to enable auto configuration of the validator component. This is enabled by default.		Boolean
camel.component.validator.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.validator.resource-resolver-factory	To use a custom LSResourceResolver which depends on a dynamic endpoint resource URI. The option is a <code>org.apache.camel.component.validator.ValidatorResourceResolverFactory</code> type.		ValidatorResourceResolverFactory

CHAPTER 149. VELOCITY

Since Camel 1.2

Only producer is supported

The Velocity component allows you to process a message using an [Apache Velocity](#) template. This can be ideal when using a template to generate responses for requests.

149.1. DEPENDENCIES

When using **velocity** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-velocity-starter</artifactId>
</dependency>
```

149.2. URI FORMAT

```
velocity:templateName[?options]
```

Where **templateName** is the classpath-local URI of the template to invoke; or the complete URL of the remote template (for example, [file://folder/myfile.vm](#)).

149.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

149.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

149.3.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

149.4. COMPONENT OPTIONS

The Velocity component supports 5 options, which are listed below.

Name	Description	Default	Type
allowContextMapAll (producer)	Sets whether the context map should allow access to all details. By default only the message body and headers can be accessed. This option can be enabled for full access to the current Exchange and CamelContext. Doing so impose a potential security risk as this opens access to the full power of CamelContext API.	false	boolean
allowTemplateFromHeader (producer)	Whether to allow to use resource template from header or not (default false). Enabling this allows to specify dynamic templates via message header. However this can be seen as a potential security vulnerability if the header is coming from a malicious user, so use this with care.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
velocityEngine (advanced)	To use the VelocityEngine otherwise a new engine is created.		VelocityEngine

149.5. ENDPOINT OPTIONS

The Velocity endpoint is configured using URI syntax:

```
velocity:resourceUri
```

with the following path and query parameters:

149.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
resourceUri (producer)	Required Path to the resource. You can prefix with: classpath, file, http, ref, or bean. classpath, file and http loads the resource using these protocols (classpath is default). ref will lookup the resource in the registry. bean will call a method on a bean to be used as the resource. For bean you can specify the method name after dot, eg bean:myBean.myMethod.		String

149.5.2. Query Parameters (7 parameters)

Name	Description	Default	Type
allowContextMap All (producer)	Sets whether the context map should allow access to all details. By default only the message body and headers can be accessed. This option can be enabled for full access to the current Exchange and CamelContext. Doing so impose a potential security risk as this opens access to the full power of CamelContext API.	false	boolean
allowTemplateFromHeader (producer)	Whether to allow to use resource template from header or not (default false). Enabling this allows to specify dynamic templates via message header. However this can be seen as a potential security vulnerability if the header is coming from a malicious user, so use this with care.	false	boolean
contentCache (producer)	Sets whether to use resource content cache or not.	false	boolean
encoding (producer)	Character encoding of the resource content.		String
loaderCache (producer)	Enables / disables the velocity resource loader cache which is enabled by default.	true	boolean

Name	Description	Default	Type
propertiesFile (producer)	The URI of the properties file which is used for VelocityEngine initialization.		String
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

149.6. MESSAGE HEADERS

The Velocity component supports 4 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelVelocityResourceUri (producer) Constant: VELOCITY_RESOURCE_URI	The name of the velocity template.		String
CamelVelocityTemplate (producer) Constant: VELOCITY_TEMPLATE	The content of the velocity template.		String
CamelVelocityContext (producer) Constant: VELOCITY_CONTEXT	The velocity context to use.		Context

Name	Description	Default	Type
CamelVelocitySupplementalContext (producer) Constant: VELOCITY_SUPPLEMENTAL_CONTEXT	To add additional information to the used VelocityContext. The value of this header should be a Map with key/values that will added (override any existing key with the same name). This can be used to pre setup some common key/values you want to reuse in your velocity endpoints.		Map

Headers set during the Velocity evaluation are returned to the message and added as headers. Then it is possible to return values from Velocity to the Message.

For example, to set the header value of **fruit** in the Velocity template **.tm**:

```
$in.setHeader("fruit", "Apple")
```

The **fruit** header is now accessible from the **message.out.headers**.

149.7. VELOCITY CONTEXT

Camel will provide exchange information in the Velocity context (just a **Map**). The **Exchange** is transfered as:

key	value
exchange	The Exchange itself.
exchange.properties	The Exchange properties.
headers	The headers of the In message.
camelContext	The Camel Context instance.
request	The In message.
in	The In message.

key	value
body	The In message body.
out	The Out message (only for InOut message exchange pattern).
response	The Out message (only for InOut message exchange pattern).

You can setup a custom Velocity Context yourself by setting property **allowTemplateFromHeader=true** and setting the message header **CamelVelocityContext** just like this

```
VelocityContext velocityContext = new VelocityContext(variableMap);
exchange.getIn().setHeader("CamelVelocityContext", velocityContext);
```

149.8. HOT RELOADING

The Velocity template resource is, by default, hot reloadable for both file and classpath resources (expanded jar). If you set **contentCache=true**, Camel will only load the resource once, and thus hot reloading is not possible. This scenario can be used in production, when the resource never changes.

149.9. DYNAMIC TEMPLATES

Since Camel 2.1

Camel provides two headers by which you can define a different resource location for a template or the template content itself. If any of these headers is set then Camel uses this over the endpoint configured resource. This allows you to provide a dynamic template at runtime.

Header	Type	Description
Camel VelocityResourceUri	String	A URI for the template resource to use instead of the endpoint configured.
Camel VelocityTemplate	String	The template to use instead of the endpoint configured.

149.10. SAMPLES

For example, you can use:

```
from("activemq:My.Queue").
to("velocity:com/acme/MyResponse.vm");
```

To use a Velocity template to formulate a response to a message for **InOut** message exchanges (where there is a **JMSReplyTo** header).

If you want to use **InOnly** and consume the message and send it to another destination, you could use the following route:

```
from("activemq:My.Queue").
  to("velocity:com/acme/MyResponse.vm").
  to("activemq:Another.Queue");
```

And to use the content cache, for example, for use in production, where the **.vm** template never changes:

```
from("activemq:My.Queue").
  to("velocity:com/acme/MyResponse.vm?contentCache=true").
  to("activemq:Another.Queue");
```

And a file based resource:

```
from("activemq:My.Queue").
  to("velocity:file://myfolder/MyResponse.vm?contentCache=true").
  to("activemq:Another.Queue");
```

It is possible to specify which template the component should use dynamically via a header, for example:

```
from("direct:in").
  setHeader("CamelVelocityResourceUri").constant("path/to/my/template.vm").
  to("velocity:dummy?allowTemplateFromHeader=true");
```

It is possible to specify a template directly as a header the component should use dynamically via a header, so for example:

```
from("direct:in").
  setHeader("CamelVelocityTemplate").constant("Hi this is a velocity template that can do templating  
${body}").
  to("velocity:dummy?allowTemplateFromHeader=true");
```

149.11. THE EMAIL SAMPLE

In this sample, to use the Velocity templating for an order confirmation email. The email template is laid out in Velocity as:

letter.vm

```
Dear ${headers.lastName}, ${headers.firstName}

Thanks for the order of ${headers.item}.

Regards Camel Riders Bookstore
${body}
```

And the java code (from an unit test):

■


```

private Exchange createLetter() {
    Exchange exchange = context.getEndpoint("direct:a").createExchange();
    Message msg = exchange.getIn();
    msg.setHeader("firstName", "Claus");
    msg.setHeader("lastName", "Ibsen");
    msg.setHeader("item", "Camel in Action");
    msg.setBody("PS: Next beer is on me, James");
    return exchange;
}

@Test
public void testVelocityLetter() throws Exception {
    MockEndpoint mock = getMockEndpoint("mock:result");
    mock.expectedMessageCount(1);
    mock.message(0).body(String.class).contains("Thanks for the order of Camel in Action");

    template.send("direct:a", createLetter());

    mock.assertIsSatisfied();
}

@Override
protected RouteBuilder createRouteBuilder() {
    return new RouteBuilder() {
        public void configure() {
            from("direct:a")
                .to("velocity:org/apache/camel/component/velocity/letter.vm")
                .to("mock:result");
        }
    };
}

```

149.12. SPRING BOOT AUTO-CONFIGURATION

The component supports 6 options, which are listed below.

Name	Description	Default	Type
camel.component-velocity.allow-context-map-all	Sets whether the context map should allow access to all details. By default only the message body and headers can be accessed. This option can be enabled for full access to the current Exchange and CamelContext. Doing so impose a potential security risk as this opens access to the full power of CamelContext API.	false	Boolean
camel.component-velocity.allow-template-from-header	Whether to allow to use resource template from header or not (default false). Enabling this allows to specify dynamic templates via message header. However this can be seen as a potential security vulnerability if the header is coming from a malicious user, so use this with care.	false	Boolean

Name	Description	Default	Type
camel.component.velocity.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.velocity.enabled	Whether to enable auto configuration of the velocity component. This is enabled by default.		Boolean
camel.component.velocity.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.velocity.velocity-engine	To use the VelocityEngine otherwise a new engine is created. The option is a org.apache.velocity.app.VelocityEngine type.		VelocityEngine

CHAPTER 150. VERT.X HTTP CLIENT

Since Camel 3.5

Only producer is supported

The [Vert.x](#) HTTP component provides the capability to produce messages to HTTP endpoints via the [Vert.x Web Client](#).

150.1. DEPENDENCIES

When using **vertx-http** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-vertx-http-starter</artifactId>
</dependency>
```

150.2. URI FORMAT

```
vertx-http:hostname[:port][/resourceUri][?options]
```

150.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

150.3.1. Configuring component options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

150.3.2. Configuring endpoint options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

150.4. COMPONENT OPTIONS

The Vert.x HTTP Client component supports 19 options, which are listed below.

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
responsePayloadAsByteArray (producer)	Whether the response body should be byte or as io.vertx.core.buffer.Buffer.	true	boolean
allowJavaSerializedObject (advanced)	Whether to allow java serialization when a request has the Content-Type application/x-java-serialized-object This is disabled by default. If you enable this, be aware that Java will deserialize the incoming data from the request. This can be a potential security risk.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
vertx (advanced)	To use an existing vertx instead of creating a new instance.		Vertx
vertxHttpBinding (advanced)	A custom VertxHttpBinding which can control how to bind between Vert.x and Camel.		VertxHttpBinding

Name	Description	Default	Type
vertxOptions (advanced)	To provide a custom set of vertx options for configuring vertx.		VertxOptions
webClientOptions (advanced)	To provide a custom set of options for configuring vertx web client.		WebClientOptions
headerFilterStrategy (filter)	To use a custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message.		HeaderFilterStrategy
proxyHost (proxy)	The proxy server host address.		String
proxyPassword (proxy)	The proxy server password if authentication is required.		String
proxyPort (proxy)	The proxy server port.		Integer
proxyType (proxy)	The proxy server type. Enum values: • HTTP • SOCKS4 • SOCKS5		ProxyType
proxyUsername (proxy)	The proxy server username if authentication is required.		String
basicAuthPassword (security)	The password to use for basic authentication.		String
basicAuthUsername (security)	The user name to use for basic authentication.		String
bearerToken (security)	The bearer token to use for bearer token authentication.		String
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean

150.5. ENDPOINT OPTIONS

The Vert.x HTTP Client endpoint is configured using URI syntax:

```
vertx-http:httpUri
```

with the following path and query parameters:

150.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
httpUri (producer)	Required The HTTP URI to connect to.		URI

150.5.2. Query Parameters (23 parameters)

Name	Description	Default	Type
connectTimeout (producer)	The amount of time in milliseconds until a connection is established. A timeout value of zero is interpreted as an infinite timeout.	60000	int
cookieStore (producer)	A custom CookieStore to use when session management is enabled. If this option is not set then an in-memory CookieStore is used.	InMemoryCookieStore	CookieStore
headerFilterStrategy (producer)	A custom org.apache.camel.spi.HeaderFilterStrategy to filter header to and from Camel message.	VertxHttpRequestHeaderFilterStrategy	HeaderFilterStrategy
httpMethod (producer)	The HTTP method to use. The HttpMethod header cannot override this option if set.		HttpMethod
okStatusCodeRange (producer)	The status codes which are considered a success response. The values are inclusive. Multiple ranges can be defined, separated by comma, e.g. 200-204,209,301-304. Each range must be a single number or from-to with the dash included.	200-299	String
responsePayloadAsByteArray (producer)	Whether the response body should be byte or as io.vertx.core.buffer.Buffer.	true	boolean

Name	Description	Default	Type
sessionManagement (producer)	Enables session management via WebClientSession. By default the client is configured to use an in-memory CookieStore. The cookieStore option can be used to override this.	false	boolean
throwExceptionOnFailure (producer)	Disable throwing HttpOperationFailedException in case of failed responses from the remote server.	true	boolean
timeout (producer)	The amount of time in milliseconds after which if the request does not return any data within the timeout period a TimeoutException fails the request. Setting zero or a negative value disables the timeout.	-1	long
transferException (producer)	If enabled and an Exchange failed processing on the consumer side, and if the caused Exception was sent back serialized in the response as a application/x-java-serialized-object content type. On the producer side the exception will be deserialized and thrown as is, instead of HttpOperationFailedException. The caused exception is required to be serialized. This is by default turned off. If you enable this then be aware that Camel will deserialize the incoming data from the request to a Java object, which can be a potential security risk.	false	boolean
useCompression (producer)	Set whether compression is enabled to handle compressed (E.g gzipped) responses.	false	boolean
vertxHttpBinding (producer)	A custom VertxHttpBinding which can control how to bind between Vert.x and Camel.		VertxHttpBinding
webClientOptions (producer)	Sets customized options for configuring the Vert.x WebClient.		WebClientOptions
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
proxyHost (proxy)	The proxy server host address.		String
proxyPassword (proxy)	The proxy server password if authentication is required.		String
proxyPort (proxy)	The proxy server port.		Integer
proxyType (proxy)	The proxy server type. Enum values: • HTTP • SOCKS4 • SOCKS5		ProxyType
proxyUsername (proxy)	The proxy server username if authentication is required.		String
basicAuthPassword (security)	The password to use for basic authentication.		String
basicAuthUsername (security)	The user name to use for basic authentication.		String
bearerToken (security)	The bearer token to use for bearer token authentication.		String
sslContextParameters (security)	To configure security using SSLContextParameters.		SSLContextParameters

150.6. MESSAGE HEADERS

The Vert.x HTTP Client component supports 8 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelHttpMethod (producer) Constant: HTTP_METHOD	The http method.		HttpMethod

Name	Description	Default	Type
CamelHttpResponseCode (producer) Constant: HTTP_RESPONSE_CODE	The HTTP response code from the external server.		Integer
CamelHttpResponseText (producer) Constant: HTTP_RESPONSE_TEXT	The HTTP response text from the external server.		String
Content-Type (producer) Constant: CONTENT_TYPE	The HTTP content type. Is set on both the IN and OUT message to provide a content type, such as text/html.		String
CamelHttpQuery (producer) Constant: HTTP_QUERY	URI parameters. Will override existing URI parameters set directly on the endpoint.		String
CamelHttpUri (producer) Constant: HTTP_URI	URI to call. Will override the existing URI set directly on the endpoint. This URI is the URI of the http server to call. Its not the same as the Camel endpoint URI, where you can configure endpoint options such as security etc. This header does not support that, its only the URI of the http server.		String
CamelHttpPath (producer) Constant: HTTP_PATH	Request URI's path, the header will be used to build the request URI with the HTTP_URI.		String
Content-Encoding (producer) Constant: CONTENT_ENCODING	The HTTP content encoding. Is set to provide a content encoding, such as gzip.		String

150.7. USAGE

The following example shows how to send a request to an HTTP endpoint.

You can override the URI configured on the **vertex-http** producer via headers **Exchange.HTTP_URI** and **Exchange.HTTP_PATH**.

```
from("direct:start")
.to("vertex-http:https://camel.apache.org");
```

150.8. URI PARAMETERS

The **vertex-http** producer supports URI parameters to be sent to the HTTP server. The URI parameters can either be set directly on the endpoint URI, or as a header with the key **Exchange.HTTP_QUERY** on the message.

150.9. RESPONSE CODE

Camel will handle according to the HTTP response code:

- Response code is in the range 100..299, Camel regards it as a success response.
- Response code is in the range 300..399, Camel regards it as a redirection response and will throw a **HttpOperationFailedException** with the information.
- Response code is 400+, Camel regards it as an external server failure and will throw a **HttpOperationFailedException** with the information.

150.10. THROWEXCEPTIONONFAILURE

The option, **throwExceptionOnFailure**, can be set to **false** to prevent the **HttpOperationFailedException** from being thrown for failed response codes. This allows you to get any response from the remote server.

150.11. EXCEPTIONS

HttpOperationFailedException exception contains the following information:

- The HTTP status code
- The HTTP status line (text of the status code)
- Redirect location, if server returned a redirect
- Response body as a **java.lang.String**, if server provided a body as response

150.12. HTTP METHOD

The following algorithm determines the HTTP method to be used:

1. Use method provided as endpoint configuration (**httpMethod**).
2. Use method provided in header (**Exchange.HTTP_METHOD**).
3. **GET** if query string is provided in header.

4. **GET** if endpoint is configured with a query string.
5. **POST** if there is data to send (body is not **null**).
6. **GET** otherwise.

150.13. HTTP FORM PARAMETERS

You can send HTTP form parameters in one of two ways.

1. Set the **Exchange.CONTENT_TYPE** header to the value **application/x-www-form-urlencoded** and ensure the message body is a **String** formatted as form variables. For example, **param1=value1¶m2=value2**.
2. Set the message body as a [MultiMap](#) which allows you to configure form parameter names and values.

150.14. MULTIPART FORM DATA

You can upload text or binary files by setting the message body as a [MultipartForm](#).

150.15. CUSTOMIZING VERT.X WEB CLIENT OPTIONS

When finer control of the Vert.x Web Client configuration is required, you can bind a custom [WebClientOptions](#) instance to the registry.

```
WebClientOptions options = new WebClientOptions().setMaxRedirects(5)
    .setIdleTimeout(10)
    .setConnectTimeout(3);

camelContext.getRegistry().bind("clientOptions", options);
```

Then reference the options on the **vertx-http** producer.

```
from("direct:start")
    .to("vertx-http:http://localhost:8080?webClientOptions=#clientOptions")
```

150.15.1. SSL

The Vert.x HTTP component supports SSL/TLS configuration through the [Camel JSSE Configuration Utility](#).

It is also possible to configure SSL options by providing a custom **WebClientOptions**.

150.16. SESSION MANAGEMENT

Session management can be enabled via the **sessionManagement** URI option. When enabled, an in-memory cookie store is used to track cookies. This can be overridden by providing a custom **CookieStore** via the **cookieStore** URI option.

150.17. SPRING BOOT AUTO-CONFIGURATION

The component supports 20 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.vertx-http.allow-java-serialized-object</code>	Whether to allow java serialization when a request has the Content-Type application/x-java-serialized-object This is disabled by default. If you enable this, be aware that Java will deserialize the incoming data from the request. This can be a potential security risk.	false	Boolean
<code>camel.component.vertx-http.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.vertx-http.basic-auth-password</code>	The password to use for basic authentication.		String
<code>camel.component.vertx-http.basic-auth-username</code>	The user name to use for basic authentication.		String
<code>camel.component.vertx-http.bearer-token</code>	The bearer token to use for bearer token authentication.		String
<code>camel.component.vertx-http.enabled</code>	Whether to enable auto configuration of the vertx-http component. This is enabled by default.		Boolean
<code>camel.component.vertx-http.header-filter-strategy</code>	To use a custom <code>org.apache.camel.spi.HeaderFilterStrategy</code> to filter header to and from Camel message. The option is a <code>org.apache.camel.spi.HeaderFilterStrategy</code> type.		HeaderFilterStrategy
<code>camel.component.vertx-http.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
<code>camel.component.vertx-http.proxy-host</code>	The proxy server host address.		String
<code>camel.component.vertx-http.proxy-password</code>	The proxy server password if authentication is required.		String
<code>camel.component.vertx-http.proxy-port</code>	The proxy server port.		Integer
<code>camel.component.vertx-http.proxy-type</code>	The proxy server type.		ProxyType
<code>camel.component.vertx-http.proxy-username</code>	The proxy server username if authentication is required.		String
<code>camel.component.vertx-http.response-payload-as-byte-array</code>	Whether the response body should be byte or as <code>io.vertx.core.buffer.Buffer</code> .	true	Boolean
<code>camel.component.vertx-http.ssl-context-parameters</code>	To configure security using <code>SSLContextParameters</code> . The option is a <code>org.apache.camel.support.jsse.SSLContextParameters</code> type.		<code>SSLContextParameters</code>
<code>camel.component.vertx-http.use-global-ssl-context-parameters</code>	Enable usage of global SSL context parameters.	false	Boolean
<code>camel.component.vertx-http.vertx</code>	To use an existing vertx instead of creating a new instance. The option is a <code>io.vertx.core.Vertx</code> type.		Vertx
<code>camel.component.vertx-http.vertx-binding</code>	A custom <code>VertxHttpBinding</code> which can control how to bind between Vert.x and Camel. The option is a <code>org.apache.camel.component.vertx.http.VertxHttpBinding</code> type.		<code>VertxHttpBinding</code>

Name	Description	Default	Type
camel.component.vertx-http.vertx-options	To provide a custom set of vertx options for configuring vertx. The option is a <code>io.vertx.core.VertxOptions</code> type.		VertxOptions
camel.component.vertx-http.web-client-options	To provide a custom set of options for configuring vertx web client. The option is a <code>io.vertx.ext.web.client.WebClientOptions</code> type.		WebClientOptions

CHAPTER 151. VERT.X WEBSOCKET

Since Camel 3.5

Both producer and consumer are supported

The <http://vertx.io/Vertx> WebSocket component provides WebSocket capabilities as a WebSocket server, or as a client to connect to an existing WebSocket.

151.1. DEPENDENCIES

When using **vertx-websocket** with Red Hat build of Camel Spring Boot, use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-vertx-websocket-starter</artifactId>
</dependency>
```

151.2. URI FORMAT

```
vertx-websocket://hostname[:port][/resourceUri][?options]
```

151.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

151.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

151.3.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

151.4. COMPONENT OPTIONS

The Vert.x WebSocket component supports 11 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
allowOriginHeader (advanced)	Whether the WebSocket client should add the Origin header to the WebSocket handshake request.	true	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
defaultHost (advanced)	Default value for host name that the WebSocket should bind to.	0.0.0.0	String
defaultPort (advanced)	Default value for the port that the WebSocket should bind to.	0	int

Name	Description	Default	Type
originHeaderUrl (advanced)	The value of the Origin header that the WebSocket client should use on the WebSocket handshake request. When not specified, the WebSocket client will automatically determine the value for the Origin from the request URL.		String
router (advanced)	To provide a custom vertx router to use on the WebSocket server.		Router
vertx (advanced)	To use an existing vertx instead of creating a new instance.		Vertx
vertxOptions (advanced)	To provide a custom set of vertx options for configuring vertx.		VertxOptions
useGlobalSslContextParameters (security)	Enable usage of global SSL context parameters.	false	boolean

151.5. ENDPOINT OPTIONS

The Vert.x WebSocket endpoint is configured using URI syntax:

```
vertx-websocket:host:port/path
```

with the following path and query parameters:

151.5.1. Path Parameters (3 parameters)

Name	Description	Default	Type
host (common)	Required WebSocket hostname, such as localhost or a remote host when in client mode.		String
port (common)	Required WebSocket port number to use.		int
path (common)	WebSocket path to use.		String

151.5.2. Query Parameters (18 parameters)

Name	Description	Default	Type
allowedOriginPattern (consumer)	Regex pattern to match the origin header sent by WebSocket clients.		String
allowOriginHeader (consumer)	Whether the WebSocket client should add the Origin header to the WebSocket handshake request.	true	boolean
consumeAsClient (consumer)	When set to true, the consumer acts as a WebSocket client, creating exchanges on each received WebSocket event.	false	boolean
fireWebSocketConnectionEvents (consumer)	Whether the server consumer will create a message exchange when a new WebSocket peer connects or disconnects.	false	boolean
maxReconnectAttempts (consumer)	When consumeAsClient is set to true this sets the maximum number of allowed reconnection attempts to a previously closed WebSocket. A value of 0 (the default) will attempt to reconnect indefinitely.	0	int
originHeaderUrl (consumer)	The value of the Origin header that the WebSocket client should use on the WebSocket handshake request. When not specified, the WebSocket client will automatically determine the value for the Origin from the request URL.		String
reconnectInitialDelay (consumer)	When consumeAsClient is set to true this sets the initial delay in milliseconds before attempting to reconnect to a previously closed WebSocket.	0	int
reconnectInterval (consumer)	When consumeAsClient is set to true this sets the interval in milliseconds at which reconnecting to a previously closed WebSocket occurs.	1000	int
router (consumer)	To use an existing vertx router for the HTTP server.		Router
serverOptions (consumer)	Sets customized options for configuring the HTTP server hosting the WebSocket for the consumer.		HttpServerOptions
bridgeErrorHandler (consumer (advanced))	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the org.apache.camel.spi.ExceptionHandler to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
exceptionHandler (consumer (advanced))	To let the consumer use a custom <code>ExceptionHandler</code> . Notice if the option <code>bridgeErrorHandler</code> is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at <code>WARN</code> or <code>ERROR</code> level and ignored.		<code>ExceptionHandler</code>
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: • <code>InOnly</code> • <code>InOut</code>		<code>ExchangePattern</code>
clientOptions (producer)	Sets customized options for configuring the <code>WebSocket</code> client used in the producer.		<code>HttpClientOptions</code>
clientSubProtocols (producer)	Comma separated list of <code>WebSocket</code> subprotocols that the client should use for the <code>Sec-WebSocket-Protocol</code> header.		<code>String</code>
sendToAll (producer)	To send to all websocket subscribers. Can be used to configure at the endpoint level, instead of providing the <code>VertxWebsocketConstants.SEND_TO_ALL</code> header on the message. Note that when using this option, the host name specified for the <code>vertx-websocket</code> producer URI must match one used for an existing <code>vertx-websocket</code> consumer. Note that this option only applies when producing messages to endpoints hosted by the <code>vertx-websocket</code> consumer and not to an externally hosted <code>WebSocket</code> .	<code>false</code>	<code>boolean</code>
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow <code>CamelContext</code> and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	<code>false</code>	<code>boolean</code>
sslContextParameters (security)	To configure security using <code>SSLContextParameters</code> .		<code>SSLContextParameters</code>

151.6. MESSAGE HEADERS

The Vert.x WebSocket component supports 4 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelVertxWebSocket.connectionKey (common) Constant: CONNECTION_KEY	Sends the message to the client with the given connection key. You can use a comma separated list of keys to send a message to multiple clients. Note that this option only applies when producing messages to endpoints hosted by the vertx-websocket consumer and not to an externally hosted WebSocket.		String
CamelVertxWebSocket.sendToAll (producer) Constant: SEND_TO_ALL	Sends the message to all clients which are currently connected. You can use the sendToAll option on the endpoint instead of using this header. Note that this option only applies when producing messages to endpoints hosted by the vertx-websocket consumer and not to an externally hosted WebSocket.		boolean
CamelVertxWebSocket.remoteAddress (consumer) Constant: REMOTE_ADDRESS	The remote address.		SocketAddress
CamelVertxWebSocket.event (consumer) Constant: EVENT	The WebSocket event that triggered the message exchange. Enum values: ● CLOSE ● ERROR ● MESSAGE ● OPEN		VertxWebSocketEvent

151.7. USAGE

The following example shows how to expose a WebSocket on <http://localhost:8080/echo> and returns an 'echo' response back to the same channel:

```
from("vertx-websocket:localhost:8080/echo")
  .transform().simple("Echo: ${body}")
  .to("vertx-websocket:localhost:8080/echo");
```

It is also possible to configure the consumer to connect as a WebSocket client on a remote address with the **consumeAsClient** option:

```
from("vertx-websocket:my.websocket.com:8080/chat?consumeAsClient=true")
  .log("Got WebSocket message ${body}");
```

151.8. PATH AND QUERY PARAMETERS

The WebSocket server consumer supports the configuration of parameterized paths. The path parameter value will be set as a Camel exchange header:

```
from("vertx-websocket:localhost:8080/chat/{user}")
  .log("New message from ${header.user} >>> ${body}");
```

You can also retrieve any query parameter values that were used by the WebSocket client to connect to the server endpoint:

```
from("direct:sendChatMessage")
  .to("vertx-websocket:localhost:8080/chat/camel?role=admin");

from("vertx-websocket:localhost:8080/chat/{user}")
  .log("New message from ${header.user} (${header.role}) >>> ${body}");
```

151.9. SENDING MESSAGES TO PEERS CONNECTED TO THE VERTX-WEBSOCKET SERVER CONSUMER



NOTE

This section only applies when producing messages to a WebSocket hosted by the camel-vertx-websocket consumer. It is not relevant when producing messages to an externally hosted WebSocket.

To send a message to all peers connected to a WebSocket hosted by the vertx-websocket server consumer, use the **sendToAll=true** endpoint option, or the **CamelVertxWebsocket.sendToAll** header.

```
from("vertx-websocket:localhost:8080/chat")
  .log("Got WebSocket message ${body}");

from("direct:broadcastMessage")
  .setBody().constant("This is a broadcast message!")
  .to("vertx-websocket:localhost:8080/chat?sendToAll=true");
```

Alternatively, you can send messages to specific peers by using the **CamelVertxWebsocket.connectionKey** header. Multiple peers can be specified as a comma separated list.

The value of the **connectionKey** can be determined whenever a peer triggers an event on the vertx-websocket consumer, where a unique key identifying the peer will be propagated via the **CamelVertxWebsocket.connectionKey** header.

```
from("vertx-websocket:localhost:8080/chat")
```

```

.log("Got WebSocket message ${body}");

from("direct:broadcastMessage")
  .setBody().constant("This is a broadcast message!")
  .setHeader(VertexWebsocketConstants.CONNECTION_KEY).constant("key-1,key-2,key-3")
  .to("vertx-websocket:localhost:8080/chat");

```

151.10. SSL

By default, the **ws://** protocol is used, but secure connections with **wss://** are supported by configuring the consumer or producer via the **sslContextParameters** URI parameter and the [Camel JSSE Configuration Utility](#).

151.11. SPRING BOOT AUTO-CONFIGURATION

The component supports 12 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.vertx-websocket.allow-origin-header</code>	Whether the WebSocket client should add the Origin header to the WebSocket handshake request.	true	Boolean
<code>camel.component.vertx-websocket.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.vertx-websocket.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.vertx-websocket.default-host</code>	Default value for host name that the WebSocket should bind to.	0.0.0.0	String
<code>camel.component.vertx-websocket.default-port</code>	Default value for the port that the WebSocket should bind to.	0	Integer

Name	Description	Default	Type
<code>camel.component.vertx-websocket.enabled</code>	Whether to enable auto configuration of the vertx-websocket component. This is enabled by default.		Boolean
<code>camel.component.vertx-websocket.lazy-start-producer</code>	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
<code>camel.component.vertx-websocket.origin-header-url</code>	The value of the Origin header that the WebSocket client should use on the WebSocket handshake request. When not specified, the WebSocket client will automatically determine the value for the Origin from the request URL.		String
<code>camel.component.vertx-websocket.router</code>	To provide a custom vertx router to use on the WebSocket server. The option is a <code>io.vertx.ext.web.Router</code> type.		Router
<code>camel.component.vertx-websocket.use-global-ssl-context-parameters</code>	Enable usage of global SSL context parameters.	false	Boolean
<code>camel.component.vertx-websocket.vertx</code>	To use an existing vertx instead of creating a new instance. The option is a <code>io.vertx.core.Vertx</code> type.		Vertx
<code>camel.component.vertx-websocket.vertx-options</code>	To provide a custom set of vertx options for configuring vertx. The option is a <code>io.vertx.core.VertxOptions</code> type.		VertxOptions

CHAPTER 152. WEBHOOK

Only consumer is supported

The Webhook meta component allows other Camel components to configure webhooks on a remote webhook provider and listening for them.

The following components currently provide webhook endpoints:

- **Telegram**

Typically, other components that support webhook will bring this dependency transitively.

152.1. DEPENDENCIES

When using **webhook** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-webhook-starter</artifactId>
</dependency>
```

152.2. URI FORMAT

```
webhook:endpoint[?options]
```

152.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

152.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

152.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

152.4. COMPONENT OPTIONS

The Webhook component supports 8 options, which are listed below.

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean
webhookAutoRegister (consumer)	Automatically register the webhook at startup and unregister it on shutdown.	true	boolean
webhookBasePath (consumer)	The first (base) path element where the webhook will be exposed. It's a good practice to set it to a random string, so that it cannot be guessed by unauthorized parties.		String
webhookComponentName (consumer)	The Camel Rest component to use for the REST transport, such as netty-http.		String
webhookExternalUrl (consumer)	The URL of the current service as seen by the webhook provider.		String

Name	Description	Default	Type
webhookPath (consumer)	The path where the webhook endpoint will be exposed (relative to basePath, if any).		String
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
configuration (advanced)	Set the default configuration for the webhook meta-component.		WebhookConfiguration

152.5. ENDPOINT OPTIONS

The Webhook endpoint is configured using URI syntax:

```
webhook:endpointUri
```

with the following path and query parameters:

152.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
endpointUri (consumer)	Required The delegate uri. Must belong to a component that supports webhooks.		String

152.5.2. Query Parameters (8 parameters)

Name	Description	Default	Type
bridgeErrorHandler (consumer)	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	boolean

Name	Description	Default	Type
webhookAutoRegister (consumer)	Automatically register the webhook at startup and unregister it on shutdown.	true	boolean
webhookBasePath (consumer)	The first (base) path element where the webhook will be exposed. It's a good practice to set it to a random string, so that it cannot be guessed by unauthorized parties.		String
webhookComponentName (consumer)	The Camel Rest component to use for the REST transport, such as netty-http.		String
webhookExternalUrl (consumer)	The URL of the current service as seen by the webhook provider.		String
webhookPath (consumer)	The path where the webhook endpoint will be exposed (relative to basePath, if any).		String
exceptionHandler (consumer (advanced))	To let the consumer use a custom ExceptionHandler. Notice if the option bridgeErrorHandler is enabled then this option is not in use. By default the consumer will deal with exceptions, that will be logged at WARN or ERROR level and ignored.		ExceptionHandler
exchangePattern (consumer (advanced))	Sets the exchange pattern when the consumer creates an exchange. Enum values: ● InOnly ● InOut ● InOptionalOut		ExchangePattern

152.6. EXAMPLES

Examples of webhook component are provided in the documentation of the delegate components that support it.

152.7. SPRING BOOT AUTO-CONFIGURATION

The component supports 9 options, which are listed below.

Name	Description	Default	Type
<code>camel.component.webhook.autowired-enabled</code>	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
<code>camel.component.webhook.bridge-error-handler</code>	Allows for bridging the consumer to the Camel routing Error Handler, which mean any exceptions occurred while the consumer is trying to pickup incoming messages, or the likes, will now be processed as a message and handled by the routing Error Handler. By default the consumer will use the <code>org.apache.camel.spi.ExceptionHandler</code> to deal with exceptions, that will be logged at WARN or ERROR level and ignored.	false	Boolean
<code>camel.component.webhook.configuration</code>	Set the default configuration for the webhook meta-component. The option is a <code>org.apache.camel.component.webhook.WebhookConfiguration</code> type.		WebhookConfiguration
<code>camel.component.webhook.enabled</code>	Whether to enable auto configuration of the webhook component. This is enabled by default.		Boolean
<code>camel.component.webhook.webhook-auto-register</code>	Automatically register the webhook at startup and unregister it on shutdown.	true	Boolean
<code>camel.component.webhook.webhook-base-path</code>	The first (base) path element where the webhook will be exposed. It's a good practice to set it to a random string, so that it cannot be guessed by unauthorized parties.		String
<code>camel.component.webhook.webhook-component-name</code>	The Camel Rest component to use for the REST transport, such as netty-http.		String
<code>camel.component.webhook.webhook-external-url</code>	The URL of the current service as seen by the webhook provider.		String
<code>camel.component.webhook.webhook-path</code>	The path where the webhook endpoint will be exposed (relative to basePath, if any).		String

CHAPTER 153. XJ

Since Camel 3.0

Only producer is supported

The XJ component allows you to convert XML and JSON documents directly forth and back without the need of intermediate java objects. You can even specify an XSLT stylesheet to convert directly to the target JSON / XML (domain) model.

153.1. DEPENDENCIES

When using **xj** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-xj-starter</artifactId>
</dependency>
```

153.2. URI FORMAT

```
xj:templateName?transformDirection=XML2JSON|JSON2XML[&options]
```



NOTE

The XJ component extends the XSLT component and therefore it supports all options provided by the XSLT component as well. At least look at the XSLT component documentation how to configure the xsl template.

The **transformDirection** option is mandatory and must be either XML2JSON or JSON2XML. The **templateName** parameter allows to use *identify transform* by specifying the name **identity**.

153.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

153.3.1. Configuring component options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

153.3.2. Configuring endpoint options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

153.4. COMPONENT OPTIONS

The XJ component supports 11 options, which are listed below.

Name	Description	Default	Type
contentCache (producer)	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the clearCachedStylesheet operation.	true	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean

Name	Description	Default	Type
saxonConfiguration (advanced)	To use a custom Saxon configuration.		Configuration
saxonConfigurationProperties (advanced)	To set custom Saxon configuration properties.		Map
saxonExtensionFunctions (advanced)	Allows you to use a custom <code>net.sf.saxon.lib.ExtensionFunctionDefinition</code> . You would need to add camel-saxon to the classpath. The function is looked up in the registry, where you can use commas to separate multiple values to lookup.		String
secureProcessing (advanced)	Feature for XML secure processing (see <code>javax.xml.XMLConstants</code>). This is enabled by default. However, when using Saxon Professional you may need to turn this off to allow Saxon to be able to use Java extension functions.	true	boolean
transformerFactoryClass (advanced)	To use a custom XSLT transformer factory, specified as a FQN class name.		String
transformerFactoryConfigurationStrategy (advanced)	A configuration strategy to apply on freshly created instances of <code>TransformerFactory</code> .		<code>TransformerFactoryConfigurationStrategy</code>
uriResolver (advanced)	To use a custom <code>UriResolver</code> . Should not be used together with the option 'uriResolverFactory'.		<code>UriResolver</code>
uriResolverFactory (advanced)	To use a custom <code>UriResolver</code> which depends on a dynamic endpoint resource URI. Should not be used together with the option 'uriResolver'.		<code>XsltUriResolverFactory</code>

153.5. ENDPOINT OPTIONS

The XJ endpoint is configured using URI syntax:

```
xj:resourceUri
```

with the following path and query parameters:

153.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
resourceUri (producer)	Required Path to the template. The following is supported by the default URIResolver. You can prefix with: classpath, file, http, ref, or bean. classpath, file and http loads the resource using these protocols (classpath is default). ref will lookup the resource in the registry. bean will call a method on a bean to be used as the resource. For bean you can specify the method name after dot, eg bean:myBean.myMethod.		String

153.5.2. Query Parameters (19 parameters)

Name	Description	Default	Type
allowStAX (producer)	Whether to allow using StAX as the javax.xml.transform.Source. You can enable this if the XSLT library supports StAX such as the Saxon library (camel-saxon). The Xalan library (default in JVM) does not support StAXSource.	true	boolean
contentCache (producer)	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the clearCachedStylesheet operation.	true	boolean
deleteOutputFile (producer)	If you have output=file then this option dictates whether or not the output file should be deleted when the Exchange is done processing. For example suppose the output file is a temporary file, then it can be a good idea to delete it after use.	false	boolean
failOnNullBody (producer)	Whether or not to throw an exception if the input body is null.	true	boolean

Name	Description	Default	Type
output (producer)	Option to specify which output type to use. Possible values are: string, bytes, DOM, file. The first three options are all in memory based, where as file is streamed directly to a java.io.File. For file you must specify the filename in the IN header with the key XsltConstants.XSLT_FILE_NAME which is also CamelXsltFileName. Also any paths leading to the filename must be created beforehand, otherwise an exception is thrown at runtime. Enum values: • string • bytes • DOM • file	string	XsltOutput
transformDirection (producer)	Required Transform direction. Either XML2JSON or JSON2XML. Enum values: • XML2JSON • JSON2XML		TransformDirection
transformerCacheSize (producer)	The number of javax.xml.transform.Transformer object that are cached for reuse to avoid calls to Template.newTransformer().	0	int
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
entityResolver (advanced)	To use a custom org.xml.sax.EntityResolver with javax.xml.transform.sax.SAXSource.		EntityResolver

Name	Description	Default	Type
errorListener (advanced)	Allows to configure to use a custom <code>javax.xml.transform.ErrorListener</code> . Beware when doing this then the default error listener which captures any errors or fatal errors and store information on the Exchange as properties is not in use. So only use this option for special use-cases.		ErrorListener
resultHandlerFactory (advanced)	Allows you to use a custom <code>org.apache.camel.builder.xml.ResultHandlerFactory</code> which is capable of using custom <code>org.apache.camel.builder.xml.ResultHandler</code> types.		ResultHandlerFactory
saxonConfiguration (advanced)	To use a custom Saxon configuration.		Configuration
saxonExtensionFunctions (advanced)	Allows you to use a custom <code>net.sf.saxon.lib.ExtensionFunctionDefinition</code> . You would need to add camel-saxon to the classpath. The function is looked up in the registry, where you can comma to separate multiple values to lookup.		String
secureProcessing (advanced)	Feature for XML secure processing (see <code>javax.xml.XMLConstants</code>). This is enabled by default. However, when using Saxon Professional you may need to turn this off to allow Saxon to be able to use Java extension functions.	true	boolean
transformerFactory (advanced)	To use a custom XSLT transformer factory.		TransformerFactory
transformerFactoryClass (advanced)	To use a custom XSLT transformer factory, specified as a FQN class name.		String
transformerFactoryConfigurationStrategy (advanced)	A configuration strategy to apply on freshly created instances of TransformerFactory.		TransformerFactoryConfigurationStrategy
uriResolver (advanced)	To use a custom <code>javax.xml.transform.URIResolver</code> .		URIResolver
xsltMessageLogger (advanced)	A consumer to messages generated during XSLT transformations.		XsltMessageLogger

153.6. MESSAGE HEADERS

The XJ component supports 1 message header(s), which is/are listed below:

Name	Description	Default	Type
CamelXsltFileName (producer) Constant: XSLT_FILE_NAME	The XSLT file name.		String

153.7. USING XJ ENDPOINTS

153.7.1. Converting JSON to XML

The following route does an "identity" transform of the message because no xslt stylesheet is given. In the context of xml to xml transformations, "Identity" transform means that the output document is just a copy of the input document. In case of XJ it means it transforms the json document to an equivalent xml representation.

```
from("direct:start").
  to("xj:identity?transformDirection=JSON2XML");
```

Sample:

The input:

```
{
  "firstname": "camel",
  "lastname": "apache",
  "personalnumber": 42,
  "active": true,
  "ranking": 3.1415926,
  "roles": [
    "a",
    {
      "x": null
    }
  ],
  "state": {
    "needsWater": true
  }
}
```

will output

```
<?xml version="1.0" encoding="UTF-8"?>
<object xmlns:xj="http://camel.apache.org/component/xj" xj:type="object">
  <object xj:name="firstname" xj:type="string">camel</object>
  <object xj:name="lastname" xj:type="string">apache</object>
  <object xj:name="personalnumber" xj:type="int">42</object>
  <object xj:name="active" xj:type="boolean">true</object>
```

```

<object xj:name="ranking" xj:type="float">3.1415926</object>
<object xj:name="roles" xj:type="array">
  <object xj:type="string">a</object>
  <object xj:type="object">
    <object xj:name="x" xj:type="null">null</object>
  </object>
</object>
<object xj:name="state" xj:type="object">
  <object xj:name="needsWater" xj:type="boolean">true</object>
</object>
</object>

```

As can be seen in the output above, XJ writes some metadata in the resulting xml that can be used in further processing:

- XJ metadata nodes are always in the **<http://camel.apache.org/component/xj>** namespace.
- JSON key names are placed in the xj:name attribute.
- The parsed JSON type can be found in the xj:type attribute. The above example already contains all possible types.
- Generated XML elements are always named "object".

Now we can apply a stylesheet, for example:

```

<?xml version="1.0" encoding="UTF-8" ?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:xj="http://camel.apache.org/component/xj"
  exclude-result-prefixes="xj">

  <xsl:output omit-xml-declaration="no" encoding="UTF-8" method="xml" indent="yes"/>

  <xsl:template match="/">
    <person>
      <xsl:apply-templates select="//object"/>
    </person>
  </xsl:template>

  <xsl:template match="object[@xj:type != 'object' and @xj:type != 'array' and string-
length(@xj:name) > 0]">
    <xsl:variable name="name" select="@xj:name"/>
    <xsl:element name="{ $name }">
      <xsl:value-of select="text()"/>
    </xsl:element>
  </xsl:template>

  <xsl:template match="@*|node()"/>
</xsl:stylesheet>

```

to the above sample by specifying the template on the endpoint:

```

from("direct:start").
  to("xj:com/example/json2xml.xml?transformDirection=JSON2XML");

```

and get the following output:

```
<?xml version="1.0" encoding="UTF-8"?>
<person>
  <firstname>camel</firstname>
  <lastname>apache</lastname>
  <personalnumber>42</personalnumber>
  <active>true</active>
  <ranking>3.1415926</ranking>
  <x>null</x>
  <needsWater>true</needsWater>
</person>
```

153.7.2. Converting XML to JSON

Based on the explanations above an "identity" transform will be performed when no stylesheet is given:

```
from("direct:start").
  to("xj:identity?transformDirection=XML2JSON");
```

Given the sample input

```
<?xml version="1.0" encoding="UTF-8"?>
<person>
  <firstname>camel</firstname>
  <lastname>apache</lastname>
  <personalnumber>42</personalnumber>
  <active>true</active>
  <ranking>3.1415926</ranking>
  <roles>
    <entry>a</entry>
    <entry>
      <x>null</x>
    </entry>
  </roles>
  <state>
    <needsWater>true</needsWater>
  </state>
</person>
```

will result in

```
{
  "firstname": "camel",
  "lastname": "apache",
  "personalnumber": "42",
  "active": "true",
  "ranking": "3.1415926",
  "roles": [
    "a",
    {
      "x": "null"
    }
  ],
}
```

```

"state": {
  "needsWater": "true"
}
}

```

You may have noted that the input xml and output json is very similar to the examples above when converting from json to xml although nothing special is done here. We only transformed an arbitrary XML document to json. XJ uses the following rules by default:

- The XML root element can be named somehow, it will always end in a json root object declaration '\{\}'
- The json key name is the name of the xml element
- If there is an name clash as in "<roles>" above where two "<entry>" elements exists a json array will be generated.
- XML elements with text-only-child-nodes will result in the usual key/string-value pair. Mixed content elements results in key/child-object pair as seen in "<state>" above.

Now we can apply again a stylesheet, for example:

```

<?xml version="1.0" encoding="UTF-8" ?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:xj="http://camel.apache.org/component/xj"
  exclude-result-prefixes="xj">

  <xsl:output omit-xml-declaration="no" encoding="UTF-8" method="xml" indent="yes"/>

  <xsl:template match="/">
    <xsl:apply-templates/>
  </xsl:template>

  <xsl:template match="personalnumber">
    <xsl:element name="{local-name()}">
      <xsl:attribute name="xj:type">
        <xsl:value-of select="int"/>
      </xsl:attribute>
      <xsl:apply-templates/>
    </xsl:element>
  </xsl:template>

  <xsl:template match="active|needsWater">
    <xsl:element name="{local-name()}">
      <xsl:attribute name="xj:type">
        <xsl:value-of select="boolean"/>
      </xsl:attribute>
      <xsl:apply-templates/>
    </xsl:element>
  </xsl:template>

  <xsl:template match="ranking">
    <xsl:element name="{local-name()}">
      <xsl:attribute name="xj:type">
        <xsl:value-of select="float"/>

```

```

        </xsl:attribute>
        <xsl:apply-templates/>
    </xsl:element>
</xsl:template>

<xsl:template match="roles">
    <xsl:element name="{local-name()}">
        <xsl:attribute name="xj:type">
            <xsl:value-of select="array"/>
        </xsl:attribute>
        <xsl:apply-templates/>
    </xsl:element>
</xsl:template>

<xsl:template match="*[normalize-space(text()) = 'null']">
    <xsl:element name="{local-name()}">
        <xsl:attribute name="xj:type">
            <xsl:value-of select="null"/>
        </xsl:attribute>
        <xsl:apply-templates/>
    </xsl:element>
</xsl:template>

<xsl:template match="@*|node()">
    <xsl:copy>
        <xsl:apply-templates select="@*|node()"/>
    </xsl:copy>
</xsl:template>
</xsl:stylesheet>

```

to the sample above by specifying the template on the endpoint:

```

from("direct:start").
to("xj:com/example/xml2json.xsl?transformDirection=XML2JSON");

```

and get the following output:

```

{
  "firstname": "camel",
  "lastname": "apache",
  "personalnumber": 42,
  "active": true,
  "ranking": 3.1415926,
  "roles": [
    "a",
    {
      "x": null
    }
  ],
  "state": {
    "needsWater": true
  }
}

```

Note, this transformation resulted in exactly the same json document as we used as input to the json2xml conversion. The following XML document is that what is passed to XJ after xsl transformation:

```
<?xml version="1.0" encoding="UTF-8"?>
<person>
  <firstname>camel</firstname>
  <lastname>apache</lastname>
  <personalnumber xmlns:xj="http://camel.apache.org/component/xj"
xj:type="int">42</personalnumber>
  <active xmlns:xj="http://camel.apache.org/component/xj" xj:type="boolean">true</active>
  <ranking xmlns:xj="http://camel.apache.org/component/xj" xj:type="float">3.1415926</ranking>
  <roles xmlns:xj="http://camel.apache.org/component/xj" xj:type="array">
    <entry>a</entry>
    <entry>
      <x xj:type="null">null</x>
    </entry>
  </roles>
  <state>
    <needsWater xmlns:xj="http://camel.apache.org/component/xj"
xj:type="boolean">true</needsWater>
  </state>
</person>
```

In the stylesheet we just provided the minimal required type hints to get the same result. The supported type hints are exactly the same as XJ writes to a XML document when converting from json to xml.

In the end that means that we can feed back in the result document from the json to xml transformation sample above:

```
<?xml version="1.0" encoding="UTF-8"?>
<object xmlns:xj="http://camel.apache.org/component/xj" xj:type="object">
  <object xj:name="firstname" xj:type="string">camel</object>
  <object xj:name="lastname" xj:type="string">apache</object>
  <object xj:name="personalnumber" xj:type="int">42</object>
  <object xj:name="active" xj:type="boolean">true</object>
  <object xj:name="ranking" xj:type="float">3.1415926</object>
  <object xj:name="roles" xj:type="array">
    <object xj:type="string">a</object>
    <object xj:type="object">
      <object xj:name="x" xj:type="null">null</object>
    </object>
  </object>
  <object xj:name="state" xj:type="object">
    <object xj:name="needsWater" xj:type="boolean">true</object>
  </object>
</object>
```

and get the same output again:

```
{
  "firstname": "camel",
  "lastname": "apache",
  "personalnumber": 42,
  "active": true,
  "ranking": 3.1415926,
```



```

    "roles": [
      "a",
      {
        "x": null
      }
    ],
    "state": {
      "needsWater": true
    }
  }
}

```

As seen in the example above: * `xj:type` lets you specify exactly the desired output type * `xj:name` lets you overrule the json key name. This is required when you want to generate key names which contains chars that aren't allowed in XML element names.

153.7.2.1. Available type hints

@xj:type=	Description
object	Generate a json object
array	Generate a json array
string	Generate a json string
int	Generate a json number without fractional part
float	Generate a json number with fractional part
boolean	Generate a json boolean
null	Generate an empty value, using the word null

153.8. SPRING BOOT AUTO-CONFIGURATION

The component supports 12 options, which are listed below.

Name	Description	Default	Type
camel.component.xj.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean

Name	Description	Default	Type
camel.component.xj.content-cache	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the <code>clearCachedStylesheet</code> operation.	true	Boolean
camel.component.xj.enabled	Whether to enable auto configuration of the xj component. This is enabled by default.		Boolean
camel.component.xj.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.xj.saxon-configuration	To use a custom Saxon configuration. The option is a <code>net.sf.saxon.Configuration</code> type.		Configuration
camel.component.xj.saxon-configuration-properties	To set custom Saxon configuration properties.		Map
camel.component.xj.saxon-extension-functions	Allows you to use a custom <code>net.sf.saxon.lib.ExtensionFunctionDefinition</code> . You would need to add camel-saxon to the classpath. The function is looked up in the registry, where you can use commas to separate multiple values to lookup.		String
camel.component.xj.secure-processing	Feature for XML secure processing (see <code>javax.xml.XMLConstants</code>). This is enabled by default. However, when using Saxon Professional you may need to turn this off to allow Saxon to be able to use Java extension functions.	true	Boolean
camel.component.xj.transformer-factory-class	To use a custom XSLT transformer factory, specified as a FQN class name.		String

Name	Description	Default	Type
camel.component.xj.transformer-factory-configuration-strategy	A configuration strategy to apply on freshly created instances of TransformerFactory. The option is a <code>org.apache.camel.component.xslt.TransformerFactoryConfigurationStrategy</code> type.		TransformerFactoryConfigurationStrategy
camel.component.xj.uri-resolver	To use a custom UriResolver. Should not be used together with the option 'uriResolverFactory'. The option is a <code>javax.xml.transform.URIResolver</code> type.		URIResolver
camel.component.xj.uri-resolver-factory	To use a custom UriResolver which depends on a dynamic endpoint resource URI. Should not be used together with the option 'uriResolver'. The option is a <code>org.apache.camel.component.xslt.XsltUriResolverFactory</code> type.		XsltUriResolverFactory

CHAPTER 154. XML TOKENIZE

The XML Tokenize language is a built-in language in **camel-xml-jaxp**, which is a truly XML-aware tokenizer that can be used with the Split EIP as the conventional [Tokenize](#) to efficiently and effectively tokenize XML documents..

XML Tokenize is capable of not only recognizing XML namespaces and hierarchical structures of the document but also more efficiently tokenizing XML documents than the conventional [Tokenize](#) language.

154.1. DEPENDENCIES

When using **xtokenize** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-xml-jaxp-starter</artifactId>
</dependency>
```

Additional dependency

In order to use this component, an additional dependency is required as follows:

```
<dependency>
  <groupId>org.codehaus.woodstox</groupId>
  <artifactId>woodstox-core-asl</artifactId>
  <version>4.4.1</version>
</dependency>
```

or

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-stax-starter</artifactId>
</dependency>
```

154.2. XML TOKENIZER OPTIONS

The XML Tokenize language supports 4 options, which are listed below.

Name	Default	Java Type	Description
headerName		String	Name of header to tokenize instead of using the message body.

Name	Default	Java Type	Description
mode		Enum	The extraction mode. The available extraction modes are: i - injecting the contextual namespace bindings into the extracted token (default) w - wrapping the extracted token in its ancestor context u - unwrapping the extracted token to its child content t - extracting the text content of the specified element. Enum values: • i • w • u • t
group		Integer	To group N parts together.
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

154.3. EXAMPLE

See [Split](#) EIP which has examples using the XML Tokenize language.

154.4. SPRING BOOT AUTO-CONFIGURATION

The component supports 3 options, which are listed below.

Name	Description	Default	Type
camel.language.xmltokenize.enabled	Whether to enable auto configuration of the xmltokenize language. This is enabled by default.		Boolean
camel.language.xmltokenize.mode	The extraction mode. The available extraction modes are: i - injecting the contextual namespace bindings into the extracted token (default) w - wrapping the extracted token in its ancestor context u - unwrapping the extracted token to its child content t - extracting the text content of the specified element.		String
camel.language.xmltokenize.trim	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

CHAPTER 155. XPATH

Camel supports [XPath](#) to allow an [Expression](#) or [Predicate](#) to be used in the [DSL](#).

For example, you could use XPath to create a predicate in a [Message Filter](#) or as an expression for a [Recipient List](#).

155.1. DEPENDENCIES

When using **xpath** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-xpath-starter</artifactId>
</dependency>
```

155.2. XPATH LANGUAGE OPTIONS

The XPath language supports 10 options, which are listed below.

Name	Default	Java Type	Description
documentType		String	Name of class for document type The default value is org.w3c.dom.Document.
resultType		Enum	Sets the class name of the result type (type from output) The default result type is NodeSet. Enum values: ● NUMBER ● STRING ● BOOLEAN ● NODESET ● NODE
saxon		Boolean	Whether to use Saxon.
factoryRef		String	References to a custom XPathFactory to lookup in the registry.
objectModel		String	The XPath object model to use.
logNamespaces		Boolean	Whether to log namespaces which can assist during troubleshooting.

Name	Default	Java Type	Description
headerName		String	Name of header to use as input, instead of the message body.
threadSafety		Boolean	Whether to enable thread-safety for the returned result of the xpath expression. This applies to when using NODESET as the result type, and the returned set has multiple elements. In this situation there can be thread-safety issues if you process the NODESET concurrently such as from a Camel Splitter EIP in parallel processing mode. This option prevents concurrency issues by doing defensive copies of the nodes. It is recommended to turn this option on if you are using camel-saxon or Saxon in your application. Saxon has thread-safety issues which can be prevented by turning this option on.
preCompile		Boolean	Whether to enable pre-compiling the xpath expression during initialization phase. pre-compile is enabled by default. This can be used to turn off, for example in cases the compilation phase is desired at the starting phase, such as if the application is ahead of time compiled (for example with camel-quarkus) which would then load the xpath factory of the built operating system, and not a JVM runtime.
trim		Boolean	Whether to trim the value to remove leading and trailing whitespaces and line breaks.

155.3. NAMESPACES

You can easily use namespaces with XPath expressions using the **Namespaces** helper class.

155.4. VARIABLES

Variables in XPath is defined in different namespaces. The default namespace is <http://camel.apache.org/schema/spring>.

Namespace URI	Local part	Type	Description
http://camel.apache.org/xml/in/	in	Message	the message
http://camel.apache.org/xml/out/	out	Message	deprecated the output message (do not use)
http://camel.apache.org/xml/function/	functions	Object	Additional functions

Namespace URI	Local part	Type	Description
http://camel.apache.org/xml/variables/environment-variables	env	Object	OS environment variables
http://camel.apache.org/xml/variables/system-properties	system	Object	Java System properties
http://camel.apache.org/xml/variables/exchange-property		Object	the exchange property

Camel will resolve variables according to either:

- namespace given
- no namespace given

155.4.1. Namespace given

If the namespace is given then Camel is instructed exactly what to return. However, when resolving Camel will try to resolve a header with the given local part first, and return it. If the local part has the value **body** then the body is returned instead.

155.4.2. No namespace given

If there is no namespace given then Camel resolves only based on the local part. Camel will try to resolve a variable in the following steps:

- from **variables** that has been set using the **variable(name, value)** fluent builder
- from **message.in.header** if there is a header with the given key
- from **exchange.properties** if there is a property with the given key

155.5. FUNCTIONS

Camel adds the following XPath functions that can be used to access the exchange:

Function	Argument	Type	Description
in:body	none	Object	Will return the message body.
in:header	the header name	Object	Will return the message header.
out:body	none	Object	deprecated Will return the out message body.

Function	Argument	Type	Description
out:header	the header name	Object	deprecated Will return the out message header.
function:properties	key for property	String	To use a .
function:simple	simple expression	Object	To evaluate a language.



NOTE

function:properties and **function:simple** is not supported when the return type is a **NodeSet**, such as when using with a [Split](#) EIP.

Here's an example showing some of these functions in use.

155.5.1. Functions example

If you prefer to configure your routes in your Spring XML file then you can use XPath expressions as follows

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://camel.apache.org/schema/spring http://camel.apache.org/schema/spring/camel-
    spring.xsd">

  <camelContext id="camel" xmlns="http://activemq.apache.org/camel/schema/spring"
    xmlns:foo="http://example.com/person">
    <route>
      <from uri="activemq:MyQueue"/>
      <filter>
        <xpath>/foo:person[@name='James']</xpath>
        <to uri="mqseries:SomeOtherQueue"/>
      </filter>
    </route>
  </camelContext>
</beans>
```

Notice how we can reuse the namespace prefixes, **foo** in this case, in the XPath expression for easier namespace based XPath expressions.

155.6. STREAM BASED MESSAGE BODIES

If the message body is stream based, which means the input it receives is submitted to Camel as a stream. That means you will only be able to read the content of the stream **once**. So often when you use [XPath](#) as Message Filter or Content Based Router then you need to access the data multiple times, and you should use Stream Caching or convert the message body to a **String** prior which is safe to be re-read multiple times.

```
from("queue:foo").
  filter().xpath("//foo").
  to("queue:bar")
```

```
from("queue:foo").
  choice().xpath("//foo").to("queue:bar").
  otherwise().to("queue:others");
```

155.7. SETTING RESULT TYPE

The XPath expression will return a result type using native XML objects such as **org.w3c.dom.NodeList**. However, many times you want a result type to be a **String**. To do this you have to instruct the XPath which result type to use.

In Java DSL:

```
xpath("/foo:person/@id", String.class)
```

In XML DSL you use the **resultType** attribute to provide the fully qualified classname.

```
<xpath resultType="java.lang.String">/foo:person/@id</xpath>
```



NOTE

Classes from **java.lang** can omit the FQN name, so you can use **resultType="String"**

Using **@XPath** annotation:

```
@XPath(value = "concat('foo-',//order/name)", resultType = String.class) String name)
```

Where we use the xpath function concat to prefix the order name with **foo-**. In this case we have to specify that we want a **String** as result type, so the concat function works.

155.8. USING XPATH ON HEADERS

Some users may have XML stored in a header. To apply an XPath to a header's value you can do this by defining the 'headerName' attribute.

```
<xpath headerName="invoiceDetails">/invoice/@orderType = 'premium'</xpath>
```

And in Java DSL you specify the headerName as the 2nd parameter as shown:

```
xpath("/invoice/@orderType = 'premium'", "invoiceDetails")
```

155.9. EXAMPLE

Here is a simple example using an XPath expression as a predicate in a [Message Filter](#):

```
from("direct:start")
  .filter().xpath("/person[@name='James']")
```

```
.to("mock:result");
```

And in XML

```
<route>
  <from uri="direct:start"/>
  <filter>
    <xpath>/person[@name='James']</xpath>
    <to uri="mock:result"/>
  </filter>
</route>
```

155.10. USING NAMESPACES

If you have a standard set of namespaces you wish to work with and wish to share them across many XPath expressions you can use the **org.apache.camel.support.builder.Namespaces** when using Java DSL as shown:

```
Namespaces ns = new Namespaces("c", "http://acme.com/cheese");

from("direct:start")
  .filter(xpath("/c:person[@name='James']", ns))
  .to("mock:result");
```

Notice how the namespaces are provided to **xpath** with the **ns** variable that are passed in as the 2nd parameter.

Each namespace is a key=value pair, where the prefix is the key. In the XPath expression then the namespace is used by its prefix, eg:

```
/c:person[@name='James']
```

The namespace builder supports adding multiple namespaces as shown:

```
Namespaces ns = new Namespaces("c", "http://acme.com/cheese")
  .add("w", "http://acme.com/wine")
  .add("b", "http://acme.com/beer");
```

When using namespaces in XML DSL then its different, as you setup the namespaces in the XML root tag (or one of the **camelContext**, **routes**, **route** tags).

In the XML example below we use Spring XML where the namespace is declared in the root tag **beans**, in the line with **xmlns:foo="http://example.com/person"**:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:foo="http://example.com/person"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://camel.apache.org/schema/spring http://camel.apache.org/schema/spring/camel-spring.xsd"
  >
```

```

<camelContext xmlns="http://camel.apache.org/schema/spring">
  <route>
    <from uri="direct:start"/>
    <filter>
      <xpath logNamespaces="true">/foo:person[@name='James']</xpath>
      <to uri="mock:result"/>
    </filter>
  </route>
</camelContext>

</beans>

```

This namespace uses **foo** as prefix, so the **<xpath>** expression uses **/foo:** to use this namespace.

155.11. USING @XPath ANNOTATION FOR BEAN INTEGRATION

You can use [Bean Integration](#) to invoke a method on a bean and use various languages such as **@XPath** to extract a value from the message and bind it to a method parameter.



NOTE

The default **@XPath** annotation has SOAP and XML namespaces available.

```

public class Foo {

    @Consume(uri = "activemq:my.queue")
    public void doSomething(@XPath("/person/@name") String name, String xml) {
        // process the inbound message here
    }
}

```

155.12. USING XPathBuilder WITHOUT AN EXCHANGE

You can now use the **org.apache.camel.language.xpath.XPathBuilder** without the need for an **Exchange**. This comes handy if you want to use it as a helper to do custom XPath evaluations.

It requires that you pass in a **CamelContext** since a lot of the moving parts inside the **XPathBuilder** requires access to the Camel [Type Converter](#) and hence why **CamelContext** is needed.

For example, you can do something like this:

```

boolean matches = XPathBuilder.xpath("/foo/bar/@xyz").matches(context, "<foo><bar xyz='cheese'>
</foo>");

```

This will match the given predicate.

You can also evaluate as shown in the following three examples:

```

String name = XPathBuilder.xpath("/foo/bar").evaluate(context, "<foo><bar>cheese</bar></foo>",
String.class);
Integer number = XPathBuilder.xpath("/foo/bar").evaluate(context, "<foo><bar>123</bar></foo>",
Integer.class);
Boolean bool = XPathBuilder.xpath("/foo/bar").evaluate(context, "<foo><bar>true</bar></foo>",

```

```
Boolean.class);
```

Evaluating with a **String** result is a common requirement and make this simpler:

```
String name = XPathBuilder.xpath("foo/bar").evaluate(context, "<foo><bar>cheese</bar></foo>");
```

155.13. USING SAXON WITH XPATHBUILDER

You need to add **camel-saxon** as dependency to your project.

It's now easier to use [Saxon](#) with the XPathBuilder which can be done in several ways as shown below

- Using a custom XPathFactory
- Using ObjectModel

155.13.1. Setting a custom XPathFactory using System Property

Camel now supports reading the [JVM system property](#) **javax.xml.xpath.XPathFactory** that can be used to set a custom XPathFactory to use.

This unit test shows how this can be done to use Saxon instead:

Camel will log at **INFO** level if it uses a non default XPathFactory such as:

```
XPathBuilder INFO Using system property
javax.xml.xpath.XPathFactory:http://saxon.sf.net/jaxp/xpath/om with value:
net.sf.saxon.xpath.XPathFactoryImpl when creating XPathFactory
```

To use Apache Xerces you can configure the system property

```
-Djavax.xml.xpath.XPathFactory=org.apache.xpath.jaxp.XPathFactoryImpl
```

155.13.2. Enabling Saxon from XML DSL

Similarly to Java DSL, to enable Saxon from XML DSL you have three options:

Referring to a custom factory:

```
<xpath factoryRef="saxonFactory" resultType="java.lang.String">current-dateTime()</xpath>
```

And declare a bean with the factory:

```
<bean id="saxonFactory" class="net.sf.saxon.xpath.XPathFactoryImpl"/>
```

Specifying the object model:

```
<xpath objectModel="http://saxon.sf.net/jaxp/xpath/om" resultType="java.lang.String">current-dateTime()</xpath>
```

And the recommended approach is to set **saxon=true** as shown:

```
<xpath saxon="true" resultType="java.lang.String">current-dateTime()</xpath>
```

155.14. NAMESPACE AUDITING TO AID DEBUGGING

Many XPath-related issues that users frequently face are linked to the usage of namespaces. You may have some misalignment between the namespaces present in your message, and those that your XPath expression is aware of or referencing. XPath predicates or expressions that are unable to locate the XML elements and attributes due to namespaces issues may simply look like *they are not working*, when in reality all there is to it is a lack of namespace definition.

Namespaces in XML are completely necessary, and while we would love to simplify their usage by implementing some magic or voodoo to wire namespaces automatically, truth is that any action down this path would disagree with the standards and would greatly hinder interoperability.

Therefore, the utmost we can do is assist you in debugging such issues by adding two new features to the XPath Expression Language and are thus accessible from both predicates and expressions.

155.14.1. Logging the Namespace Context of your XPath expression/predicate

Every time a new XPath expression is created in the internal pool, Camel will log the namespace context of the expression under the **org.apache.camel.language.xpath.XPathBuilder** logger. Since Camel represents Namespace Contexts in a hierarchical fashion (parent-child relationships), the entire tree is output in a recursive manner with the following format:

```
[me: {prefix -> namespace}, {prefix -> namespace}], [parent: [me: {prefix -> namespace}, {prefix -> namespace}], [parent: [me: {prefix -> namespace}]]]
```

Any of these options can be used to activate this logging:

- Enable TRACE logging on the **org.apache.camel.language.xpath.XPathBuilder** logger, or some parent logger such as **org.apache.camel** or the root logger
- Enable the **logNamespaces** option as indicated in the following section, in which case the logging will occur on the INFO level

155.14.2. Auditing namespaces

Camel is able to discover and dump all namespaces present on every incoming message before evaluating an XPath expression, providing all the richness of information you need to help you analyse and pinpoint possible namespace issues.

To achieve this, it in turn internally uses another specially tailored XPath expression to extract all namespace mappings that appear in the message, displaying the prefix and the full namespace URI(s) for each individual mapping.

Some points to take into account:

- The implicit XML namespace (**xmlns:xml="http://www.w3.org/XML/1998/namespace"**) is suppressed from the output because it adds no value
- Default namespaces are listed under the **DEFAULT** keyword in the output

- Keep in mind that namespaces can be remapped under different scopes. Think of a top-level 'a' prefix which in inner elements can be assigned a different namespace, or the default namespace changing in inner scopes. For each discovered prefix, all associated URIs are listed.

You can enable this option in Java DSL and XML DSL:

Java DSL:

```
XPathBuilder.xpath("/foo:person/@id", String.class).logNamespaces()
```

XML DSL:

```
<xpath logNamespaces="true" resultType="String">/foo:person/@id</xpath>
```

The result of the auditing will be appeared at the INFO level under the **org.apache.camel.language.xpath.XPathBuilder** logger and will look like the following:

```
2012-01-16 13:23:45,878 [stSaxonWithFlag] INFO XPathBuilder - Namespaces discovered in
message:
{xmlns:a=[http://apache.org/camel], DEFAULT=[http://apache.org/default],
xmlns:b=[http://apache.org/camelA, http://apache.org/camelB]}
```

155.15. LOADING SCRIPT FROM EXTERNAL RESOURCE

You can externalize the script and have Camel load it from a resource such as **"classpath:"**, **"file:"**, or **"http:"**. This is done using the following syntax: **"resource:scheme:location"**, eg to refer to a file on the classpath you can do:

```
.setHeader("myHeader").xpath("resource:classpath:myxpath.txt", String.class)
```

155.16. SPRING BOOT AUTO-CONFIGURATION

The component supports 9 options, which are listed below.

Name	Description	Default	Type
<code>camel.language.xpath.document-type</code>	Name of class for document type The default value is <code>org.w3c.dom.Document</code> .		String
<code>camel.language.xpath.enabled</code>	Whether to enable auto configuration of the xpath language. This is enabled by default.		Boolean
<code>camel.language.xpath.factory-ref</code>	References to a custom XPathFactory to lookup in the registry.		String
<code>camel.language.xpath.log-namespaces</code>	Whether to log namespaces which can assist during troubleshooting.	false	Boolean

Name	Description	Default	Type
<code>camel.language.xpath.object-model</code>	The XPath object model to use.		String
<code>camel.language.xpath.pre-compile</code>	Whether to enable pre-compiling the xpath expression during initialization phase. pre-compile is enabled by default. This can be used to turn off, for example in cases the compilation phase is desired at the starting phase, such as if the application is ahead of time compiled (for example with camel-quarkus) which would then load the xpath factory of the built operating system, and not a JVM runtime.	true	Boolean
<code>camel.language.xpath.saxon</code>	Whether to use Saxon.	false	Boolean
<code>camel.language.xpath.thread-safety</code>	Whether to enable thread-safety for the returned result of the xpath expression. This applies to when using NODESET as the result type, and the returned set has multiple elements. In this situation there can be thread-safety issues if you process the NODESET concurrently such as from a Camel Splitter EIP in parallel processing mode. This option prevents concurrency issues by doing defensive copies of the nodes. It is recommended to turn this option on if you are using camel-saxon or Saxon in your application. Saxon has thread-safety issues which can be prevented by turning this option on.	false	Boolean
<code>camel.language.xpath.trim</code>	Whether to trim the value to remove leading and trailing whitespaces and line breaks.	true	Boolean

CHAPTER 156. XSLT

Only producer is supported

The XSLT component allows you to process a message using an [XSLT](#) template. This can be ideal when using Templating to generate response for requests.

156.1. DEPENDENCIES

When using **xslt** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-xslt-starter</artifactId>
</dependency>
```

156.2. URI FORMAT

```
xslt:templateName[?options]
```

The URI format contains **templateName**, which can be one of the following:

- the classpath-local URI of the template to invoke
- the complete URL of the remote template.

You can append query options to the URI in the following format:

?option=value&option=value&...

Table 156.1. Table 1. Example URIs

URI	Description
xslt:com/acme/mytransform.xml	Refers to the file com/acme/mytransform.xml on the classpath
xslt:file:///foo/bar.xml	Refers to the file /foo/bar.xml
xslt:http://acme.com/cheese/foo.xml	Refers to the remote http resource

156.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

156.3.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

156.3.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

156.4. COMPONENT OPTIONS

The XSLT component supports 7 options, which are listed below.

Name	Description	Default	Type
contentCache (producer)	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the clearCachedStylesheet operation.	true	boolean

Name	Description	Default	Type
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
transformerFactoryClass (advanced)	To use a custom XSLT transformer factory, specified as a FQN class name.		String
transformerFactoryConfigurationStrategy (advanced)	A configuration strategy to apply on freshly created instances of TransformerFactory.		TransformerFactoryConfigurationStrategy
uriResolver (advanced)	To use a custom UriResolver. Should not be used together with the option 'uriResolverFactory'.		UriResolver
uriResolverFactory (advanced)	To use a custom UriResolver which depends on a dynamic endpoint resource URI. Should not be used together with the option 'uriResolver'.		XsltUriResolverFactory

156.5. ENDPOINT OPTIONS

The XSLT endpoint is configured using URI syntax:

```
xslt:resourceUri
```

with the following path and query parameters:

156.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
resourceUri (producer)	Required Path to the template. The following is supported by the default URIResolver. You can prefix with: classpath, file, http, ref, or bean. classpath, file and http loads the resource using these protocols (classpath is default). ref will lookup the resource in the registry. bean will call a method on a bean to be used as the resource. For bean you can specify the method name after dot, eg bean:myBean.myMethod.		String

156.5.2. Query Parameters (13 parameters)

Name	Description	Default	Type
contentCache (producer)	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the clearCachedStylesheet operation.	true	boolean
deleteOutputFile (producer)	If you have output=file then this option dictates whether or not the output file should be deleted when the Exchange is done processing. For example suppose the output file is a temporary file, then it can be a good idea to delete it after use.	false	boolean
failOnNullBody (producer)	Whether or not to throw an exception if the input body is null.	true	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
output (producer)	Option to specify which output type to use. Possible values are: string, bytes, DOM, file. The first three options are all in memory based, where as file is streamed directly to a java.io.File. For file you must specify the filename in the IN header with the key Exchange.XSLT_FILE_NAME which is also CamelXsltFileName. Also any paths leading to the filename must be created beforehand, otherwise an exception is thrown at runtime. Enum values: • string • bytes • DOM • file	string	XsltOutput
transformerCacheSize (producer)	The number of javax.xml.transform.Transformer object that are cached for reuse to avoid calls to Template.newTransformer().	0	int
entityResolver (advanced)	To use a custom org.xml.sax.EntityResolver with javax.xml.transform.sax.SAXSource.		EntityResolver
errorListener (advanced)	Allows to configure to use a custom javax.xml.transform.ErrorListener. Beware when doing this then the default error listener which captures any errors or fatal errors and store information on the Exchange as properties is not in use. So only use this option for special use-cases.		ErrorListener
resultHandlerFactory (advanced)	Allows you to use a custom org.apache.camel.builder.xml.ResultHandlerFactory which is capable of using custom org.apache.camel.builder.xml.ResultHandler types.		ResultHandlerFactory
transformerFactory (advanced)	To use a custom XSLT transformer factory.		TransformerFactory
transformerFactoryClass (advanced)	To use a custom XSLT transformer factory, specified as a FQN class name.		String

Name	Description	Default	Type
transformerFactoryConfigurationStrategy (advanced)	A configuration strategy to apply on freshly created instances of TransformerFactory.		TransformerFactoryConfigurationStrategy
uriResolver (advanced)	To use a custom javax.xml.transform.URIResolver.		URIResolver

156.6. USING XSLT ENDPOINTS

The following format is an example of using an XSLT template to formulate a response for a message for InOut message exchanges (where there is a **JMSReplyTo** header)

```
from("activemq:My.Queue").
  to("xslt:com/acme/mytransform.xml");
```

If you want to use InOnly and consume the message and send it to another destination you could use the following route:

```
from("activemq:My.Queue").
  to("xslt:com/acme/mytransform.xml").
  to("activemq:Another.Queue");
```

156.7. GETTING USEABLE PARAMETERS INTO THE XSLT

By default, all headers are added as parameters which are then available in the XSLT. To make the parameters useable, you will need to declare them.

```
<setHeader name="myParam"><constant>42</constant></setHeader>
<to uri="xslt:MyTransform.xml"/>
```

The parameter also needs to be declared in the top level of the XSLT for it to be available:

```
<xsl: ..... >
  <xsl:param name="myParam"/>
  <xsl:template ...>
```

156.8. SPRING XML VERSIONS

To use the above examples in Spring XML you would use something like the following code:

```
<camelContext xmlns="http://activemq.apache.org/camel/schema/spring">
  <route>
    <from uri="activemq:My.Queue"/>
    <to uri="xslt:org/apache/camel/spring/processor/example.xml"/>
```

```

    <to uri="activemq:Another.Queue"/>
  </route>
</camelContext>

```

156.9. USING XSL:INCLUDE

Camel provides its own implementation of **URIResolver**. This allows Camel to load included files from the classpath.

For example the include file in the following code will be located relative to the starting endpoint.

```
<xsl:include href="staff_template.xml"/>
```

This means that Camel will locate the file in the **classpath** as **org/apache/camel/component/xslt/staff_template.xml**

You can use **classpath:** or **file:** to instruct Camel to look either in the classpath or file system. If you omit the prefix then Camel uses the prefix from the endpoint configuration. If no prefix is specified in the endpoint configuration, the default is **classpath:**.

You can also refer backwards in the include paths. In the following example, the xml file will be resolved under **org/apache/camel/component**.

```
<xsl:include href="../staff_other_template.xml"/>
```

156.10. USING XSL:INCLUDE AND DEFAULT PREFIX

Camel will use the prefix from the endpoint configuration as the default prefix.

You can explicitly specify **file:** or **classpath:** loading. The two loading types can be mixed in a XSLT script, if necessary.

156.11. DYNAMIC STYLESHEETS

To provide a dynamic stylesheet at runtime you can define a dynamic URI. See [How to use a dynamic URI in to\(\)](#) for more information.

156.12. ACCESSING WARNINGS, ERRORS AND FATALERRORS FROM XSLT ERRORLISTENER

Any warning/error or fatalError is stored on the current Exchange as a property with the keys **Exchange.XSLT_ERROR**, **Exchange.XSLT_FATAL_ERROR**, or **Exchange.XSLT_WARNING** which allows end users to get hold of any errors happening during transformation.

For example in the stylesheet below, we want to terminate if a staff has an empty dob field. And to include a custom error message using xsl:message.

```

<xsl:template match="/">
  <html>
  <body>
    <xsl:for-each select="staff/programmer">
      <p>Name: <xsl:value-of select="name"/><br />

```

```

        <xsl:if test="dob="">
            <xsl:message terminate="yes">Error: DOB is an empty string!</xsl:message>
        </xsl:if>
    </p>
</xsl:for-each>
</body>
</html>
</xsl:template>

```

The exception is stored on the Exchange as a warning with the key **Exchange.XSLT_WARNING**.

156.13. SPRING BOOT AUTO-CONFIGURATION

The component supports 8 options, which are listed below.

Name	Description	Default	Type
camel.component.xslt.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.xslt.content-cache	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the clearCachedStylesheet operation.	true	Boolean
camel.component.xslt.enabled	Whether to enable auto configuration of the xslt component. This is enabled by default.		Boolean
camel.component.xslt.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean
camel.component.xslt.transformer-factory-class	To use a custom XSLT transformer factory, specified as a FQN class name.		String

Name	Description	Default	Type
camel.component.xslt.transformer-factory-configuration-strategy	A configuration strategy to apply on freshly created instances of TransformerFactory. The option is a <code>org.apache.camel.component.xslt.TransformerFactoryConfigurationStrategy</code> type.		TransformerFactoryConfigurationStrategy
camel.component.xslt.uri-resolver	To use a custom UriResolver. Should not be used together with the option 'uriResolverFactory'. The option is a <code>javax.xml.transform.URIResolver</code> type.		URIResolver
camel.component.xslt.uri-resolver-factory	To use a custom UriResolver which depends on a dynamic endpoint resource URI. Should not be used together with the option 'uriResolver'. The option is a <code>org.apache.camel.component.xslt.XsltUriResolverFactory</code> type.		XsltUriResolverFactory

CHAPTER 157. XSLT SAXON

Since Camel 3.0

Only producer is supported

The XSLT Saxon component allows you to process a message using an [XSLT](#) template using Saxon. This is ideal when using Templating to generate responses for requests.

157.1. DEPENDENCIES

When using **xslt-saxon** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
<groupId>org.apache.camel.springboot</groupId>
<artifactId>camel-xslt-saxon-starter</artifactId>
</dependency>
```

157.2. URI FORMAT

```
xslt-saxon:templateName[?options]
```

The URI format contains **templateName**, which can be one of the following:

- the classpath-local URI of the template to invoke
- the complete URL of the remote template.

You can append query options to the URI in the following format:

```
?option=value&option=value&...
```

Table 157.1. Example URIs

URI	Description
xslt-saxon:com/acme/mytransform.xml	Refers to the file com/acme/mytransform.xml on the classpath
xslt-saxon:file:///foo/bar.xml	Refers to the file /foo/bar.xml
xslt-saxon:http://acme.com/cheese/foo.xml	Refers to the remote http resource

157.3. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

157.3.1. Configuring Component Options

The component level is the highest level which holds general and common configurations that are inherited by the endpoints. For example a component may have security settings, credentials for authentication, urls for network connection and so forth.

Some components only have a few options, and others may have many. Because components typically have pre configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

Configuring components can be done with the [Component DSL](#), in a configuration file (application.properties|yaml), or directly with Java code.

157.3.2. Configuring Endpoint Options

Where you find yourself configuring the most is on endpoints, as endpoints often have many options, which allows you to configure what you need the endpoint to do. The options are also categorized into whether the endpoint is used as consumer (from) or as a producer (to), or used for both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) as a type safe way of configuring endpoints.

A good practice when configuring options is to use [Property Placeholders](#), which allows to not hardcode urls, port numbers, sensitive information, and other settings. In other words placeholders allows to externalize the configuration from your code, and gives more flexibility and reuse.

The following two sections lists all the options, firstly for the component followed by the endpoint.

157.4. COMPONENT OPTIONS

The XSLT Saxon component supports 11 options, which are listed below.

Name	Description	Default	Type
contentCache (producer)	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the clearCachedStylesheet operation.	true	boolean
lazyStartProducer (producer)	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean

Name	Description	Default	Type
autowiredEnabled (advanced)	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	boolean
saxonConfiguration (advanced)	To use a custom Saxon configuration.		Configuration
saxonConfigurationProperties (advanced)	To set custom Saxon configuration properties.		Map
saxonExtensionFunctions (advanced)	Allows you to use a custom <code>net.sf.saxon.lib.ExtensionFunctionDefinition</code> . You would need to add camel-saxon to the classpath. The function is looked up in the registry, where you can use commas to separate multiple values to lookup.		String
secureProcessing (advanced)	Feature for XML secure processing (see <code>javax.xml.XMLConstants</code>). This is enabled by default. However, when using Saxon Professional you may need to turn this off to allow Saxon to be able to use Java extension functions.	true	boolean
transformerFactoryClass (advanced)	To use a custom XSLT transformer factory, specified as a FQN class name.		String
transformerFactoryConfigurationStrategy (advanced)	A configuration strategy to apply on freshly created instances of <code>TransformerFactory</code> .		<code>TransformerFactoryConfigurationStrategy</code>
uriResolver (advanced)	To use a custom <code>UriResolver</code> . Should not be used together with the option 'uriResolverFactory'.		<code>UriResolver</code>
uriResolverFactory (advanced)	To use a custom <code>UriResolver</code> which depends on a dynamic endpoint resource URI. Should not be used together with the option 'uriResolver'.		<code>XsltUriResolverFactory</code>

157.5. ENDPOINT OPTIONS

The XSLT Saxon endpoint is configured using URI syntax:

```
| xslt-saxon:resourceUri
```

with the following path and query parameters:

157.5.1. Path Parameters (1 parameters)

Name	Description	Default	Type
resourceUri (producer)	Required Path to the template. The following is supported by the default URIResolver. You can prefix with: classpath, file, http, ref, or bean. classpath, file and http loads the resource using these protocols (classpath is default). ref will lookup the resource in the registry. bean will call a method on a bean to be used as the resource. For bean you can specify the method name after dot, eg bean:myBean.myMethod.		String

157.5.2. Query Parameters (18 parameters)

Name	Description	Default	Type
allowStAX (producer)	Whether to allow using StAX as the javax.xml.transform.Source. You can enable this if the XSLT library supports StAX such as the Saxon library (camel-saxon). The Xalan library (default in JVM) does not support StAXSource.	true	boolean
contentCache (producer)	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the clearCachedStylesheet operation.	true	boolean
deleteOutputFile (producer)	If you have output=file then this option dictates whether or not the output file should be deleted when the Exchange is done processing. For example suppose the output file is a temporary file, then it can be a good idea to delete it after use.	false	boolean
failOnNullBody (producer)	Whether or not to throw an exception if the input body is null.	true	boolean

Name	Description	Default	Type
output (producer)	Option to specify which output type to use. Possible values are: string, bytes, DOM, file. The first three options are all in memory based, where as file is streamed directly to a java.io.File. For file you must specify the filename in the IN header with the key XsltConstants.XSLT_FILE_NAME which is also CamelXsltFileName. Also any paths leading to the filename must be created beforehand, otherwise an exception is thrown at runtime. Enum values: • string • bytes • DOM • file	string	XsltOutput
transformerCacheSize (producer)	The number of javax.xml.transform.Transformer object that are cached for reuse to avoid calls to Template.newTransformer().	0	int
lazyStartProducer (producer (advanced))	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	boolean
entityResolver (advanced)	To use a custom org.xml.sax.EntityResolver with javax.xml.transform.sax.SAXSource.		EntityResolver
errorListener (advanced)	Allows to configure to use a custom javax.xml.transform.ErrorListener. Beware when doing this then the default error listener which captures any errors or fatal errors and store information on the Exchange as properties is not in use. So only use this option for special use-cases.		ErrorListener

Name	Description	Default	Type
resultHandlerFactory (advanced)	Allows you to use a custom <code>org.apache.camel.builder.xml.ResultHandlerFactory</code> which is capable of using custom <code>org.apache.camel.builder.xml.ResultHandler</code> types.		<code>ResultHandlerFactory</code>
saxonConfiguration (advanced)	To use a custom Saxon configuration.		Configuration
saxonExtensionFunctions (advanced)	Allows you to use a custom <code>net.sf.saxon.lib.ExtensionFunctionDefinition</code> . You would need to add <code>camel-saxon</code> to the classpath. The function is looked up in the registry, where you can comma to separate multiple values to lookup.		String
secureProcessing (advanced)	Feature for XML secure processing (see <code>javax.xml.XMLConstants</code>). This is enabled by default. However, when using Saxon Professional you may need to turn this off to allow Saxon to be able to use Java extension functions.	true	boolean
transformerFactory (advanced)	To use a custom XSLT transformer factory.		<code>TransformerFactory</code>
transformerFactoryClass (advanced)	To use a custom XSLT transformer factory, specified as a FQN class name.		String
transformerFactoryConfigurationStrategy (advanced)	A configuration strategy to apply on freshly created instances of <code>TransformerFactory</code> .		<code>TransformerFactoryConfigurationStrategy</code>
uriResolver (advanced)	To use a custom <code>javax.xml.transform.URIResolver</code> .		<code>URIResolver</code>
xsltMessageLogger (advanced)	A consumer to messages generated during XSLT transformations.		<code>XsltMessageLogger</code>

157.6. USING XSLT ENDPOINTS

The following format is an example of using an XSLT template to formulate a response for a message for **InOut** message exchanges (where there is a **JMSReplyTo** header).

```
from("activemq:My.Queue").
  to("xslt-saxon:com/acme/mytransform.xml");
```

If you want to use **InOnly** and consume the message and send it to another destination you could use the following route:

```
from("activemq:My.Queue").
  to("xslt-saxon:com/acme/mytransform.xml").
  to("activemq:Another.Queue");
```

157.7. GETTING USEABLE PARAMETERS INTO THE XSLT

By default, all headers are added as parameters which are then available in the XSLT. To make the parameters useable, you will need to declare them.

```
<setHeader name="myParam"><constant>42</constant></setHeader>
<to uri="xslt:MyTransform.xml"/>
```

The parameter also needs to be declared in the top level of the XSLT for it to be available:

```
<xsl: ..... >

  <xsl:param name="myParam"/>

  <xsl:template ...>
```

157.8. SPRING XML VERSIONS

To use the above examples in Spring XML, use something like the following code:

```
<camelContext xmlns="http://activemq.apache.org/camel/schema/spring">
  <route>
    <from uri="activemq:My.Queue"/>
    <to uri="xslt-saxon:org/apache/camel/spring/processor/example.xml"/>
    <to uri="activemq:Another.Queue"/>
  </route>
</camelContext>
```

157.9. USING XSL:INCLUDE

Camel provides its own implementation of **URIResolver**. This allows Camel to load included files from the classpath. For example the include file in the following code will be located relative to the starting endpoint.

```
<xsl:include href="staff_template.xml"/>
```

This means that Camel will locate the file in the **classpath** as **org/apache/camel/component/xslt/staff_template.xml**. You can use **classpath:** or **file:** to instruct Camel to look either in the classpath or file system. If you omit the prefix then Camel uses the prefix from the endpoint configuration. If no prefix is specified in the endpoint configuration, the default is **classpath:**.

You can also refer backwards in the include paths. In the following example, the xml file will be resolved under **org/apache/camel/component**.

```
<xsl:include href="../staff_other_template.xml"/>
```


157.10. USING XSL:INCLUDE AND DEFAULT PREFIX

Camel uses the prefix from the endpoint configuration as the default prefix. You can explicitly specify **file:** or **classpath:** loading. The two loading types can be mixed in a XSLT script, if necessary.

157.11. USING SAXON EXTENSION FUNCTIONS

Since Saxon 9.2, writing extension functions has been supplemented by a new mechanism, referred to as [integrated extension functions](#). You can now easily use camel as shown in the below example:

```
SimpleRegistry registry = new SimpleRegistry();
registry.put("function1", new MyExtensionFunction1());
registry.put("function2", new MyExtensionFunction2());

CamelContext context = new DefaultCamelContext(registry);
context.addRoutes(new RouteBuilder() {
    @Override
    public void configure() throws Exception {
        from("direct:start")
            .to("xslt-saxon:org/apache/camel/component/xslt/extensions/extensions.xslt?
saxonExtensionFunctions=#function1,#function2");
    }
});
```

With Spring XML:

```
<bean id="function1" class="org.apache.camel.component.xslt.extensions.MyExtensionFunction1"/>
<bean id="function2" class="org.apache.camel.component.xslt.extensions.MyExtensionFunction2"/>

<camelContext xmlns="http://camel.apache.org/schema/spring">
    <route>
        <from uri="direct:extensions"/>
        <to uri="xslt-saxon:org/apache/camel/component/xslt/extensions/extensions.xslt?
saxonExtensionFunctions=#function1,#function2"/>
    </route>
</camelContext>
```

157.12. DYNAMIC STYLESHEETS

To provide a dynamic stylesheet at runtime you can define a dynamic URI. See [How to use a dynamic URI in to\(\)](#) for more information.

157.13. ACCESSING WARNINGS, ERRORS AND FATAL ERRORS FROM XSLT ERRORLISTENER

Any warning/error or fatalError is stored on the current Exchange as a property with the keys **Exchange.XSLT_ERROR**, **Exchange.XSLT_FATAL_ERROR**, or **Exchange.XSLT_WARNING** which allows end users to get hold of any errors happening during transformation.

For example, in the stylesheet below, we want to determinate whether a staff has an empty dob field. And to include a custom error message using xsl:message.

```
<xsl:template match="/">
```

```

<html>
<body>
<xsl:for-each select="staff/programmer">
  <p>Name: <xsl:value-of select="name"/><br />
  <xsl:if test="dob="">
    <xsl:message terminate="yes">Error: DOB is an empty string!</xsl:message>
  </xsl:if>
</p>
</xsl:for-each>
</body>
</html>
</xsl:template>

```

The exception is stored on the Exchange as a warning with the key **Exchange.XSLT_WARNING**.

157.14. SPRING BOOT AUTO-CONFIGURATION

The component supports 12 options, which are listed below.

Name	Description	Default	Type
camel.component.xslt-saxon.autowired-enabled	Whether autowiring is enabled. This is used for automatic autowiring options (the option must be marked as autowired) by looking up in the registry to find if there is a single instance of matching type, which then gets configured on the component. This can be used for automatic configuring JDBC data sources, JMS connection factories, AWS Clients, etc.	true	Boolean
camel.component.xslt-saxon.content-cache	Cache for the resource content (the stylesheet file) when it is loaded. If set to false Camel will reload the stylesheet file on each message processing. This is good for development. A cached stylesheet can be forced to reload at runtime via JMX using the clearCachedStylesheet operation.	true	Boolean
camel.component.xslt-saxon.enabled	Whether to enable auto configuration of the xslt-saxon component. This is enabled by default.		Boolean
camel.component.xslt-saxon.lazy-start-producer	Whether the producer should be started lazy (on the first message). By starting lazy you can use this to allow CamelContext and routes to startup in situations where a producer may otherwise fail during starting and cause the route to fail being started. By deferring this startup to be lazy then the startup failure can be handled during routing messages via Camel's routing error handlers. Beware that when the first message is processed then creating and starting the producer may take a little time and prolong the total processing time of the processing.	false	Boolean

Name	Description	Default	Type
camel.component.xslt-saxon.saxon-configuration	To use a custom Saxon configuration. The option is a <code>net.sf.saxon.Configuration</code> type.		Configuration
camel.component.xslt-saxon.saxon-configuration-properties	To set custom Saxon configuration properties.		Map
camel.component.xslt-saxon.saxon-extension-functions	Allows you to use a custom <code>net.sf.saxon.lib.ExtensionFunctionDefinition</code> . You would need to add camel-saxon to the classpath. The function is looked up in the registry, where you can use commas to separate multiple values to lookup.		String
camel.component.xslt-saxon.secure-processing	Feature for XML secure processing (see <code>javax.xml.XMLConstants</code>). This is enabled by default. However, when using Saxon Professional you may need to turn this off to allow Saxon to be able to use Java extension functions.	true	Boolean
camel.component.xslt-saxon.transformer-factory-class	To use a custom XSLT transformer factory, specified as a FQN class name.		String
camel.component.xslt-saxon.transformer-factory-configuration-strategy	A configuration strategy to apply on freshly created instances of <code>TransformerFactory</code> . The option is a <code>org.apache.camel.component.xslt.TransformerFactoryConfigurationStrategy</code> type.		<code>TransformerFactoryConfigurationStrategy</code>
camel.component.xslt-saxon.uri-resolver	To use a custom <code>UriResolver</code> . Should not be used together with the option 'uriResolverFactory'. The option is a <code>javax.xml.transform.URIResolver</code> type.		<code>URIResolver</code>
camel.component.xslt-saxon.uri-resolver-factory	To use a custom <code>UriResolver</code> which depends on a dynamic endpoint resource URI. Should not be used together with the option 'uriResolver'. The option is a <code>org.apache.camel.component.xslt.XsltUriResolverFactory</code> type.		<code>XsltUriResolverFactory</code>

CHAPTER 158. YAML DSL

Since Camel 3.9

The YAML DSL provides the capability to define your Camel routes, route templates & REST DSL configuration in YAML.

158.1. DEFINING A ROUTE

A route is a collection of elements defined as follows:

```
- from: ❶
  uri: "direct:start"
  steps: ❷
    - filter:
      expression:
        simple: "${in.header.continue} == true"
      steps:
        - to:
          uri: "log:filtered"
    - to:
      uri: "log:original"
```

Where,

- ❶ Route entry point, by default **from** and **rest** are supported.
- ❷ Processing steps



NOTE

Each step represents a YAML map that has a single entry where the field name is the EIP name.

As a general rule, each step provides all the parameters the related definition declares, but there are some minor differences/enhancements:

- Output Aware Steps

Some steps, such as **filter** and **split**, have their own pipeline when an exchange matches the filter expression or for the items generated by the split expression. You can define these pipelines in the **steps** field:

```
filter:
  expression:
    simple: "${in.header.continue} == true"
  steps:
    - to:
      uri: "log:filtered"
```

- Expression Aware Steps

Some EIP, such as **filter** and **split**, supports the definition of an expression through the **expression** field:

Explicit Expression field

```
filter:
  expression:
    simple: "${in.header.continue} == true"
```

To make the DSL less verbose, you can omit the **expression** field.

Implicit Expression field

```
filter:
  simple: "${in.header.continue} == true"
```

In general, expressions can be defined inline, such as within the examples above but if you need provide more information, you can 'unroll' the expression definition and configure any single parameter the expression defines.

Full Expression definition

```
filter:
  tokenize:
    token: "<"
    end-token: ">"
```

- Data Format Aware Steps

The EIP **marshal** and **unmarshal** supports the definition of data formats:

```
marshal:
  json:
    library: Gson
```



NOTE

In case you want to use the data-format's default settings, you need to place an empty block as data format parameters, like **json: {}**

158.2. DEFINING ENDPOINTS

To define an endpoint with the YAML DSL you have two options:

- Using a classic Camel URI:

```
- from:
  uri: "timer:tick?period=1s"
  steps:
    - to:
      uri: "telegram:bots?authorizationToken=XXX"
```

- Using URI and parameters:

```
- from:
```

```
uri: "timer://tick"
parameters:
  period: "1s"
steps:
  - to:
      uri: "telegram:bots"
      parameters:
        authorizationToken: "XXX"
```

158.3. DEFINING BEANS

In addition to the general support for creating beans provided by [Camel Main](#), the YAML DSL provide a convenient syntax to define and configure them:

```
- beans:
  - name: beanFromMap 1
    type: com.acme.MyBean 2
    properties: 3
      foo: bar
```

Where,

- 1** The name of the bean which will bound the instance to the Camel Registry.
- 2** The full qualified class name of the bean
- 3** The properties of the bean to be set

The properties of the bean can be defined using either a map or properties style, as shown in the example below:

```
- beans:
  # map style
  - name: beanFromMap
    type: com.acme.MyBean
    properties:
      field1: 'f1'
      field2: 'f2'
      nested:
        field1: 'nf1'
        field2: 'nf2'
  # properties style
  - name: beanFromProps
    type: com.acme.MyBean
    properties:
      field1: 'f1_p'
      field2: 'f2_p'
      nested.field1: 'nf1_p'
      nested.field2: 'nf2_p'
```



NOTE

The **beans** elements is only used as root element.

158.4. CONFIGURING OPTIONS

Camel components are configured on two separate levels:

- component level
- endpoint level

158.4.1. Configuring Component Options

At the component level, you set general and shared configurations that are, then, inherited by the endpoints. It is the highest configuration level. For example, a component may have security settings, credentials for authentication, urls for network connection and so forth. Some components only have a few options, and others may have many. Because components typically have pre-configured defaults that are commonly used, then you may often only need to configure a few options on a component; or none at all.

You can configure components using:

- the [Component DSL](#).
- in a configuration file (application.properties, *.yaml files, etc).
- directly in the Java code.

158.4.2. Configuring Endpoint Options

You usually spend more time setting up endpoints because they have many options. These options help you customize what you want the endpoint to do. The options are also categorized into whether the endpoint is used as a consumer (from), as a producer (to), or both.

Configuring endpoints is most often done directly in the endpoint URI as path and query parameters. You can also use the [Endpoint DSL](#) and [DataFormat DSL](#) as a type safe way of configuring endpoints and data formats in Java.

A good practice when configuring options is to use [Property Placeholders](#).

Property placeholders provide a few benefits:

- They help prevent using hardcoded urls, port numbers, sensitive information, and other settings.
- They allow externalizing the configuration from the code.
- They help the code to become more flexible and reusable.

The following two sections list all the options, firstly for the component followed by the endpoint.

158.5. CONFIGURING OPTIONS ON LANGUAGES

Some [languages](#) have additional configurations that you may need to use.

For example, the [JSONPath](#) can be configured to ignore JSON parsing errors. This is intended when you use a [Content Based Router](#) and want to route the message to different endpoints. The JSON payload of the message can be in different forms, meaning that the JSonPath expressions in some cases would fail with an exception, and other times not. In this situation you must set **suppress-exception** to true, as shown below:

```
- from:
  uri: "direct:start"
  steps:
    - choice:
      when:
        - jsonpath:
            expression: "person middlename"
            suppress-exceptions: true
        steps:
          - to: "mock:middle"
        - jsonpath:
            expression: "person.lastname"
            suppress-exceptions: true
        steps:
          - to: "mock:last"
      otherwise:
        steps:
          - to: "mock:other"
```

In the route above, the following message would have failed the JXPath expression **person middlename** because the JSON payload does not have a **middlename** field. To remedy this, we have suppressed the exception.

```
{
  "person": {
    "firstname": "John",
    "lastname": "Doe"
  }
}
```

158.6. EXTERNAL EXAMPLES

You can find a set of examples using **main-yaml** in [Camel examples](#) that demonstrate how to create the Camel Routes with YAML. You can also refer to [Camel Kamelets](#) where each Kamelet is defined using YAML.

CHAPTER 159. ZIP FILE

The Zip File Data Format is a message compression and de-compression format. Messages can be marshalled (compressed) to Zip files containing a single entry, and Zip files containing a single entry can be unmarshalled (decompressed) to the original file contents. This data format supports ZIP64, as long as Java 7 or later is being used].

159.1. DEPENDENCIES

When using **zipfile** with Red Hat build of Camel Spring Boot make sure to use the following Maven dependency to have support for auto configuration:

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-zipfile-starter</artifactId>
</dependency>
```

159.2. ZIPFILE OPTIONS

The Zip File dataformat supports 4 options, which are listed below.

Name	Default	Java Type	Description
<code>usingIterator</code>		Boolean	If the zip file has more then one entry, the setting this option to true, allows to work with the splitter EIP, to split the data using an iterator in a streaming mode.
<code>allowEmptyDirectory</code>		Boolean	If the zip file has more then one entry, setting this option to true, allows to get the iterator even if the directory is empty.
<code>preservePathElements</code>		Boolean	If the file name contains path elements, setting this option to true, allows the path to be maintained in the zip file.
<code>maxDecompressedSize</code>		Integer	Set the maximum decompressed size of a zip file (in bytes). The default value if not specified corresponds to 1 gigabyte. An <code>IOException</code> will be thrown if the decompressed size exceeds this amount. Set to -1 to disable setting a maximum decompressed size.

159.3. MARSHAL

In this example we marshal a regular text/XML payload to a compressed payload using Zip file compression, and send it to an ActiveMQ queue called MY_QUEUE.

```
from("direct:start")
  .marshal().zipFile()
  .to("activemq:queue:MY_QUEUE");
```

The name of the Zip entry inside the created Zip file is based on the incoming **CamelFileName** message

header, which is the standard message header used by the file component. Additionally, the outgoing **CamelFileName** message header is automatically set to the value of the incoming **CamelFileName** message header, with the ".zip" suffix. So for example, if the following route finds a file named "test.txt" in the input directory, the output will be a Zip file named "test.txt.zip" containing a single Zip entry named "test.txt":

```
from("file:input/directory?antInclude=/*.txt")
    .marshal().zipFile()
    .to("file:output/directory");
```

If there is no incoming **CamelFileName** message header (for example, if the file component is not the consumer), then the message ID is used by default, and since the message ID is normally a unique generated ID, you will end up with filenames like **ID-MACHINENAME-2443-1211718892437-1-0.zip**. If you want to override this behavior, then you can set the value of the **CamelFileName** header explicitly in your route:

```
from("direct:start")
    .setHeader(Exchange.FILE_NAME, constant("report.txt"))
    .marshal().zipFile()
    .to("file:output/directory");
```

This route would result in a Zip file named "report.txt.zip" in the output directory, containing a single Zip entry named "report.txt".

159.4. UNMARSHAL

In this example we unmarshal a Zip file payload from an ActiveMQ queue called MY_QUEUE to its original format, and forward it for processing to the **UnZippedMessageProcessor**.

```
from("activemq:queue:MY_QUEUE")
    .unmarshal().zipFile()
    .process(new UnZippedMessageProcessor());
```

If the zip file has more than one entry, the usingIterator option of ZipFileDataFormat to be true, and you can use splitter to do the further work.

```
ZipFileDataFormat zipFile = new ZipFileDataFormat();
zipFile.setUsingIterator(true);

from("file:src/test/resources/org/apache/camel/dataformat/zipfile/?delay=1000&noop=true")
    .unmarshal(zipFile)
    .split(body(Iterator.class)).streaming()
    .process(new UnZippedMessageProcessor())
    .end();
```

Or you can use the ZipSplitter as an expression for splitter directly like this

```
from("file:src/test/resources/org/apache/camel/dataformat/zipfile?delay=1000&noop=true")
    .split(new ZipSplitter()).streaming()
    .process(new UnZippedMessageProcessor())
    .end();
```

159.4.1. Aggregate



NOTE

This aggregation strategy requires eager completion check to work properly.

In this example we aggregate all text files found in the input directory into a single Zip file that is stored in the output directory.

```
from("file:input/directory?antInclude=/*.txt")
    .aggregate(constant(true), new ZipAggregationStrategy())
    .completionFromBatchConsumer().eagerCheckCompletion()
    .to("file:output/directory");
```

The outgoing **CamelFileName** message header is created using `java.io.File.createTempFile`, with the ".zip" suffix. If you want to override this behavior, then you can set the value of the **CamelFileName** header explicitly in your route:

```
from("file:input/directory?antInclude=/*.txt")
    .aggregate(constant(true), new ZipAggregationStrategy())
    .completionFromBatchConsumer().eagerCheckCompletion()
    .setHeader(Exchange.FILE_NAME, constant("reports.zip"))
    .to("file:output/directory");
```

159.5. SPRING BOOT AUTO-CONFIGURATION

The component supports 5 options, which are listed below.

Name	Description	Default	Type
camel.dataformat.zipfile.allow-empty-directory	If the zip file has more than one entry, setting this option to true, allows to get the iterator even if the directory is empty.	false	Boolean
camel.dataformat.zipfile.enabled	Whether to enable auto configuration of the zipfile data format. This is enabled by default.		Boolean
camel.dataformat.zipfile.max-decompressed-size	Set the maximum decompressed size of a zip file (in bytes). The default value if not specified corresponds to 1 gigabyte. An <code>IOException</code> will be thrown if the decompressed size exceeds this amount. Set to -1 to disable setting a maximum decompressed size.	1073741824	Long
camel.dataformat.zipfile.preserve-path-elements	If the file name contains path elements, setting this option to true, allows the path to be maintained in the zip file.	false	Boolean

Name	Description	Default	Type
camel.dataformat.zipfile.using-iterator	If the zip file has more then one entry, the setting this option to true, allows to work with the splitter EIP, to split the data using an iterator in a streaming mode.	false	Boolean